

# AWS Certified Solutions Architect Associate (SAA-C03)

**Full course slides (v5.0)**

By Chetan Agrawal ([www.awswithchetan.com](http://www.awswithchetan.com))

Udemy Instructor | Ex. Senior Solutions Architect at AWS

Published Date: 05/07/2026

# Disclaimer and Copyrights

- These slides are copyrighted and intended strictly for personal use.
- This material is exclusively for learners enrolled in the “AWS Certified Solutions Architect – Associate (SAA-C03)” course by Chetan Agrawal
- Redistribution, sharing, or resale of this content is not permitted.
- These slides are provided solely for learning and exam preparation purposes.
- If you obtained this material from any source other than the official course platform, please report it to [awswithchetan@gmail.com](mailto:awswithchetan@gmail.com)

# Course Sections

- [Getting started with AWS](#)
- [AWS Identity and Access Management \(IAM\)](#)
- [Amazon EC2](#)
- [EC2 Advanced](#)
- [Elastic Load Balancer and Autoscaling Group](#)
- [VPC and Networking](#)
- [Amazon S3](#)
- [AWS Container Services](#)
- [AWS Databases](#)
- [Bigdata and Analytics](#)
- [Machine Learning and AI](#)
- [AWS Edge Networking](#)
- [Route 53 \(DNS\)](#)
- [AWS Serverless \(API gateway and Lambda\)](#)
- [AWS Integration Services](#)
- [AWS Data Security Services](#)
- [Infrastructure as Code \(AWS CloudFormation\)](#)
- [Application Deployment](#)
- [AWS Monitoring and Logging](#)
- [AWS Systems Manager and more](#)
- [AWS Security Services](#)
- [Storage and Data Migration](#)
- [Migration and Disaster Recovery](#)
- [AWS Multi-Account Management](#)
- [AWS Billing and Cost Management](#)

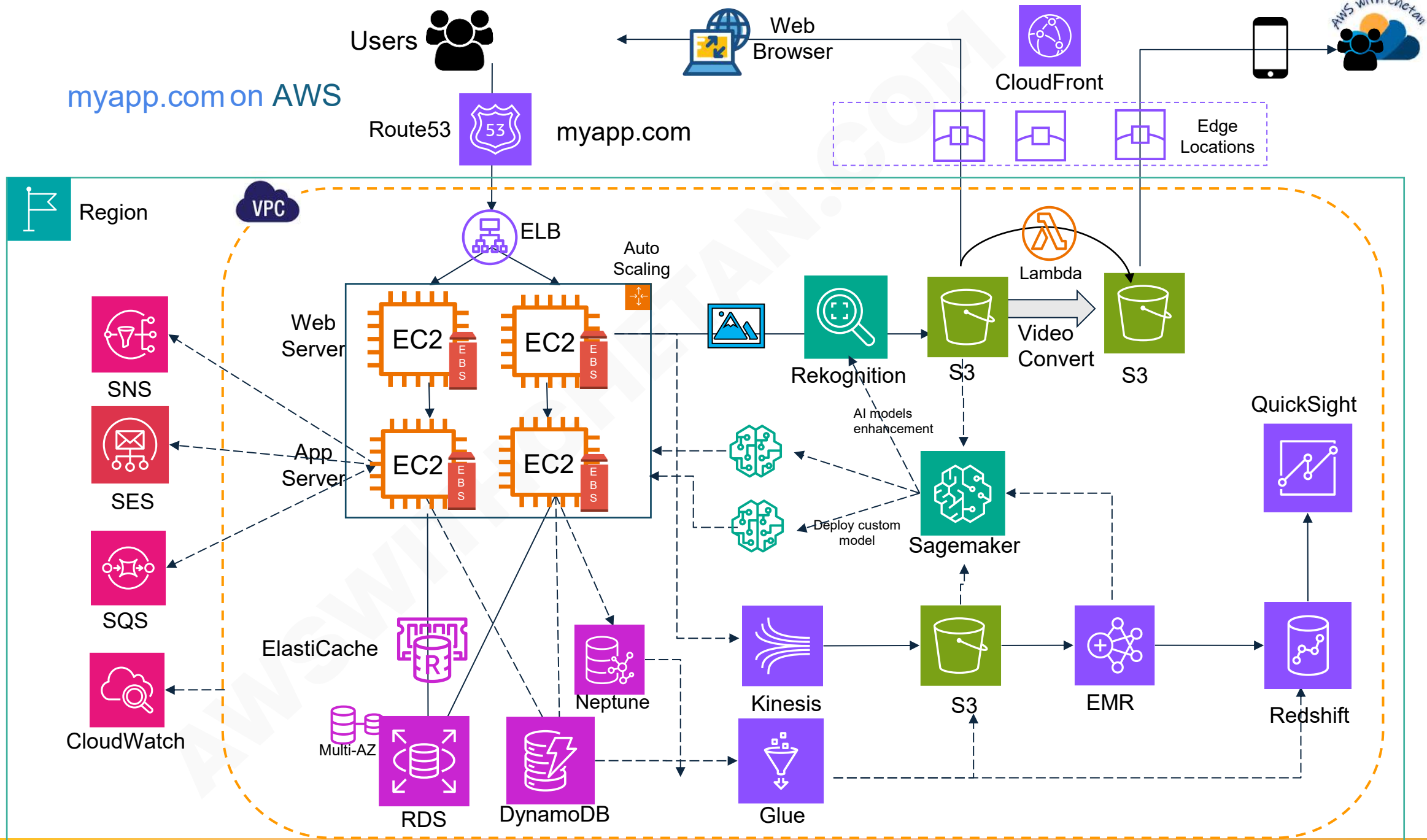
# Getting started with AWS..



# Getting started with AWS..

1. AWS Global Infrastructure
2. AWS Account
3. **Overview of AWS Services** (from Solutions Architect perspective)

myapp.com on AWS



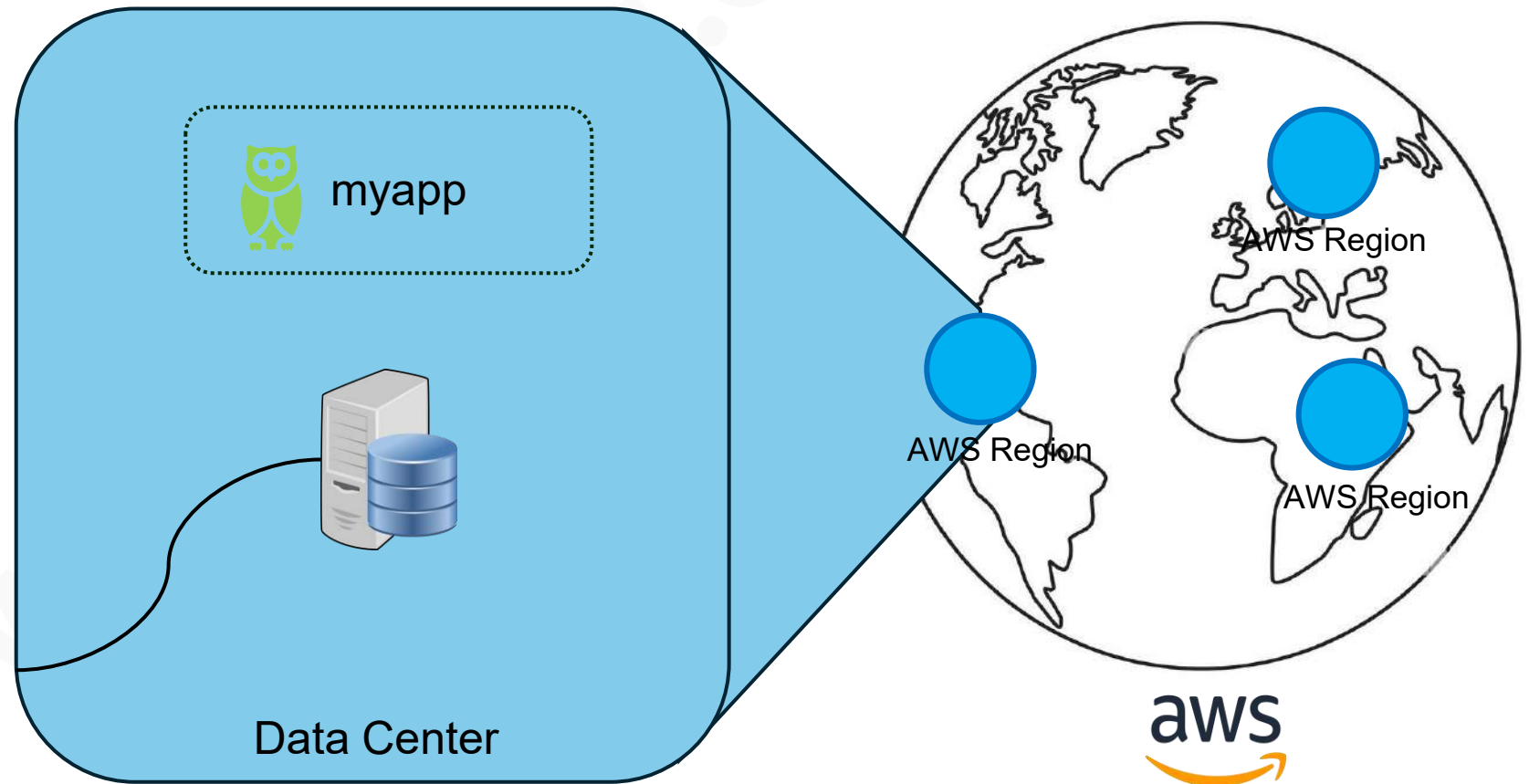
# AWS Global Infrastructure

1. **Regions and Availability Zones**
2. Edge Locations
3. Local Zones
4. Wavelength Zone
5. Outpost

# To deploy and run any application..

## We need:

- Compute
- Storage
- Network



# AWS Regions

A world map with a light gray background. Red dots indicate AWS regions, and blue dots indicate AWS Availability Zones. Red dots are located in North America (USA), South America (Brazil), Europe (UK, Germany, France, Ireland, Sweden, Poland, Spain, Italy, Greece, Turkey, India, Singapore, Japan, South Korea, Australia, and New Zealand). Blue dots are located in North America (USA), South America (Brazil), Europe (UK, Germany, France, Ireland, Sweden, Poland, Spain, Italy, Greece, Turkey, India, Singapore, Japan, South Korea, Australia, and New Zealand).

#1 – Low latency for end users

#2 – Data Residency & Compliance

#3 – Disaster Recovery

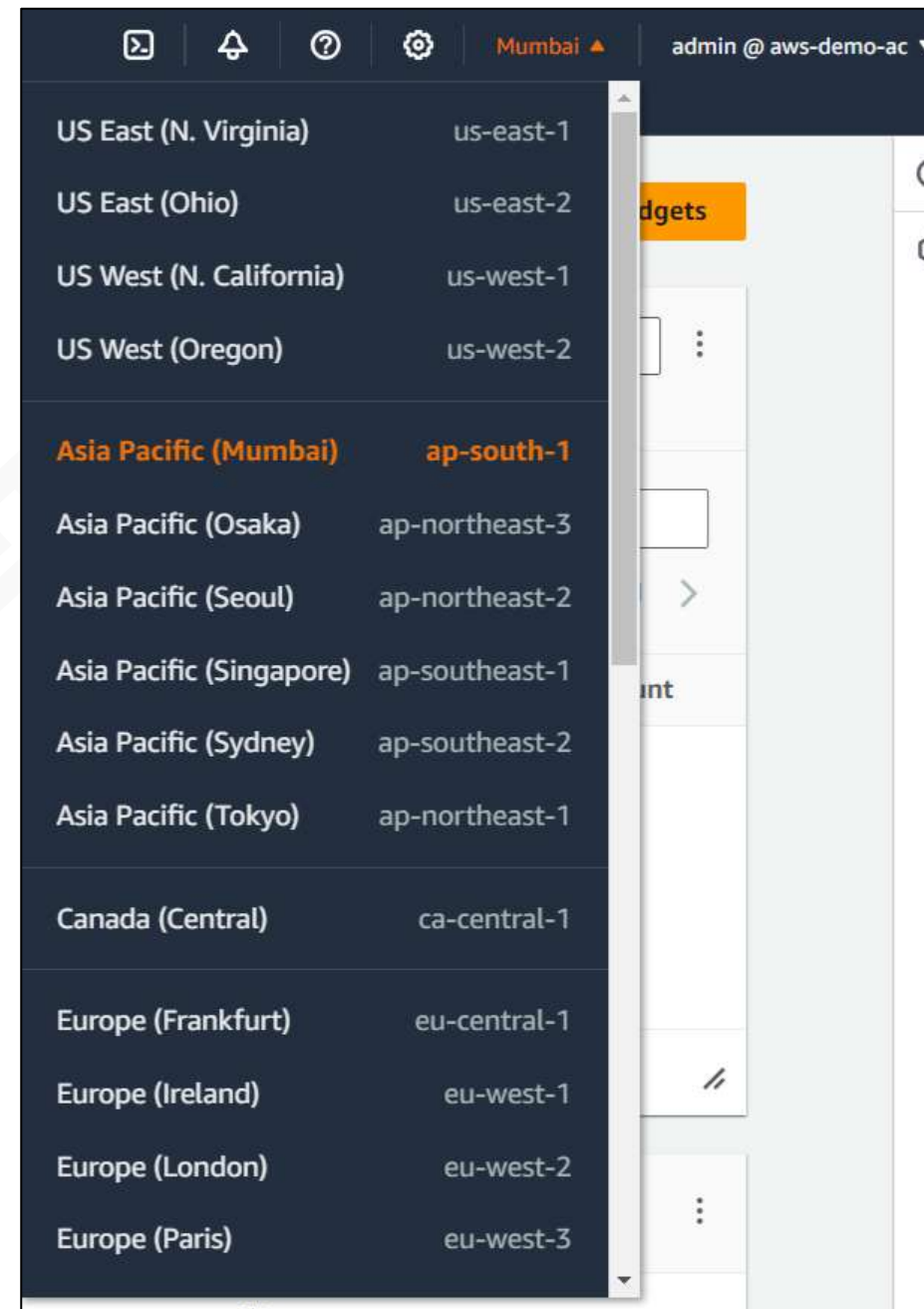
#4 – AWS services pricing difference

#5 – Availability of AWS services

Refer <https://aws.amazon.com/about-aws/global-infrastructure> for latest

# AWS Region naming

- Every AWS Region has a name and a code
  - N. Virginia = us-east-1
  - Mumbai = ap-south-1
  - London = eu-west-2



# 1 Region = Multiple Availability Zones



<https://aws.amazon.com/about-aws/global-infrastructure/>



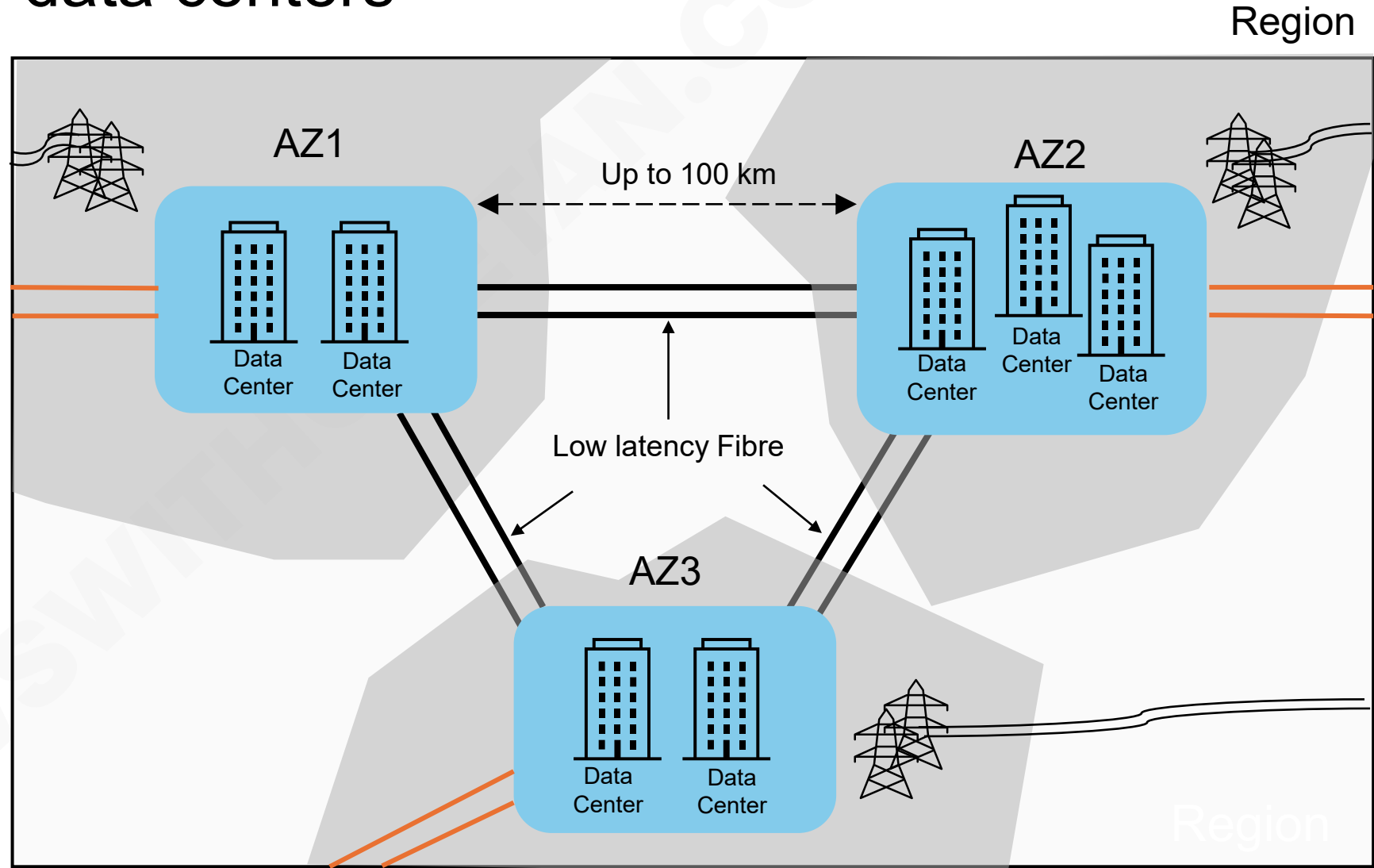
# 1 Region = Multiple AZs

## 1 AZ = Multiple data centers

Different floodplains  
(in most cases)

Redundant Power  
Supply

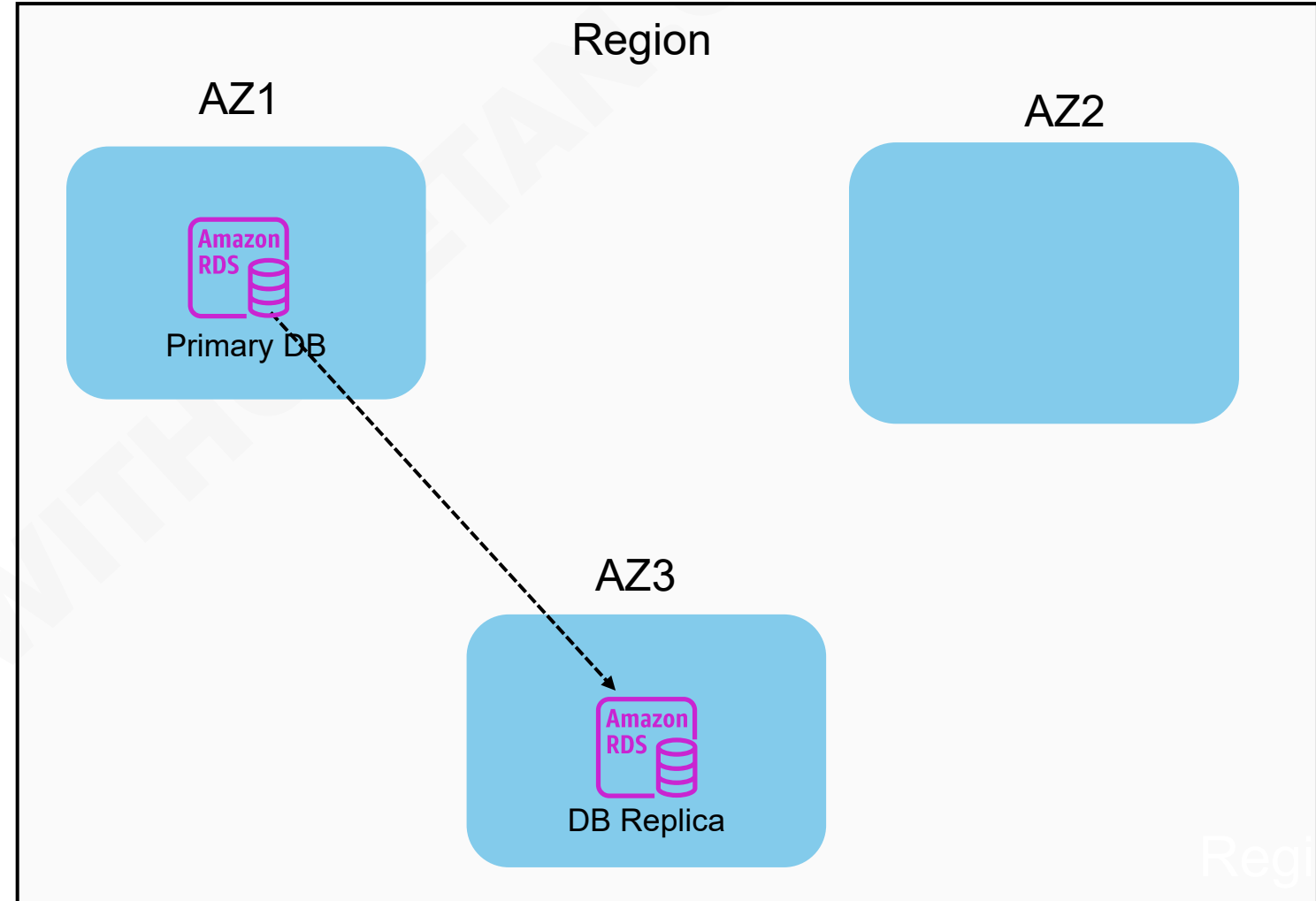
Redundant Network  
Connectivity





# Why 3 or more Availability Zones in a Region?

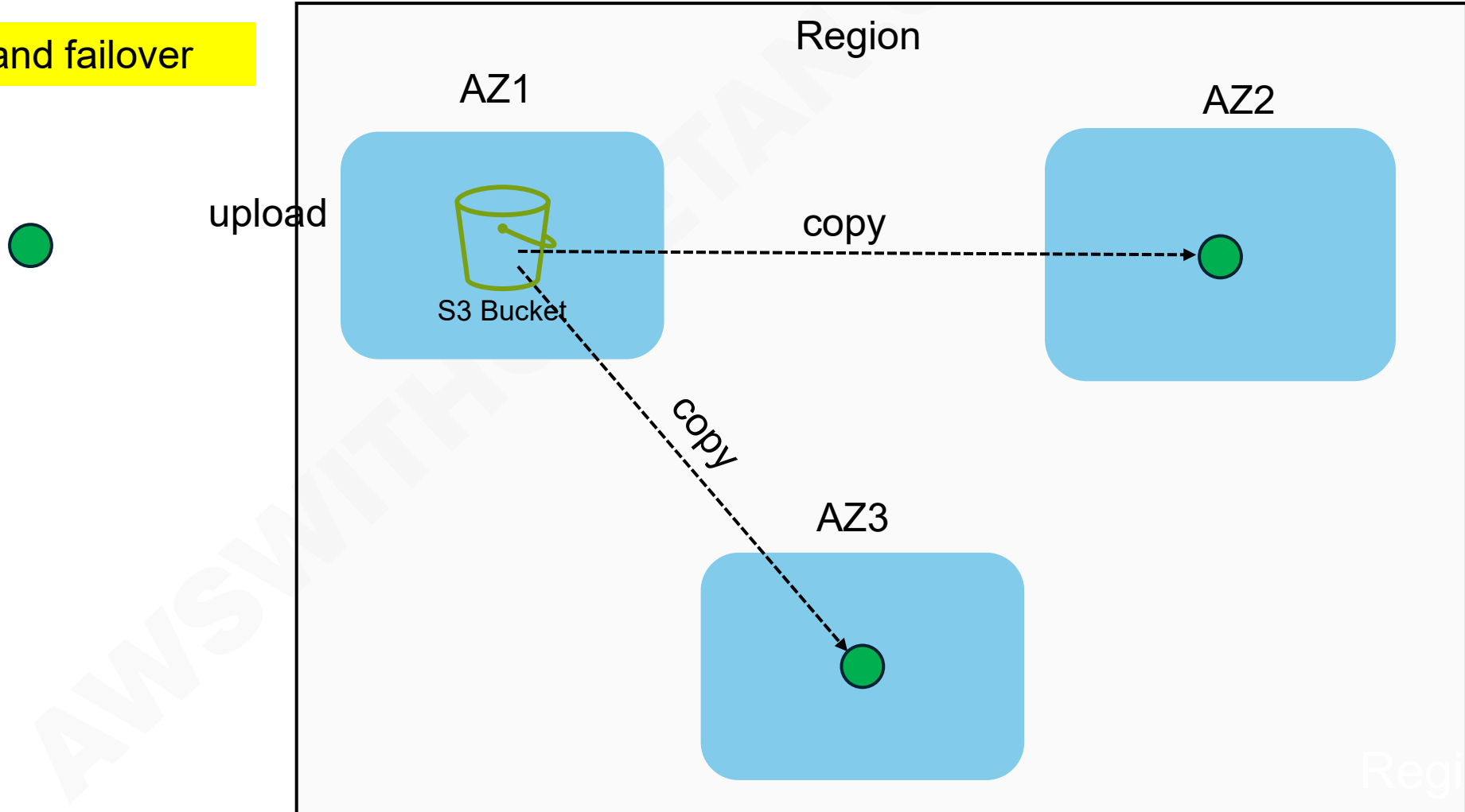
High Availability and failover



# Why 3 or more Availability Zones in a Region?

High Availability and failover

Durability

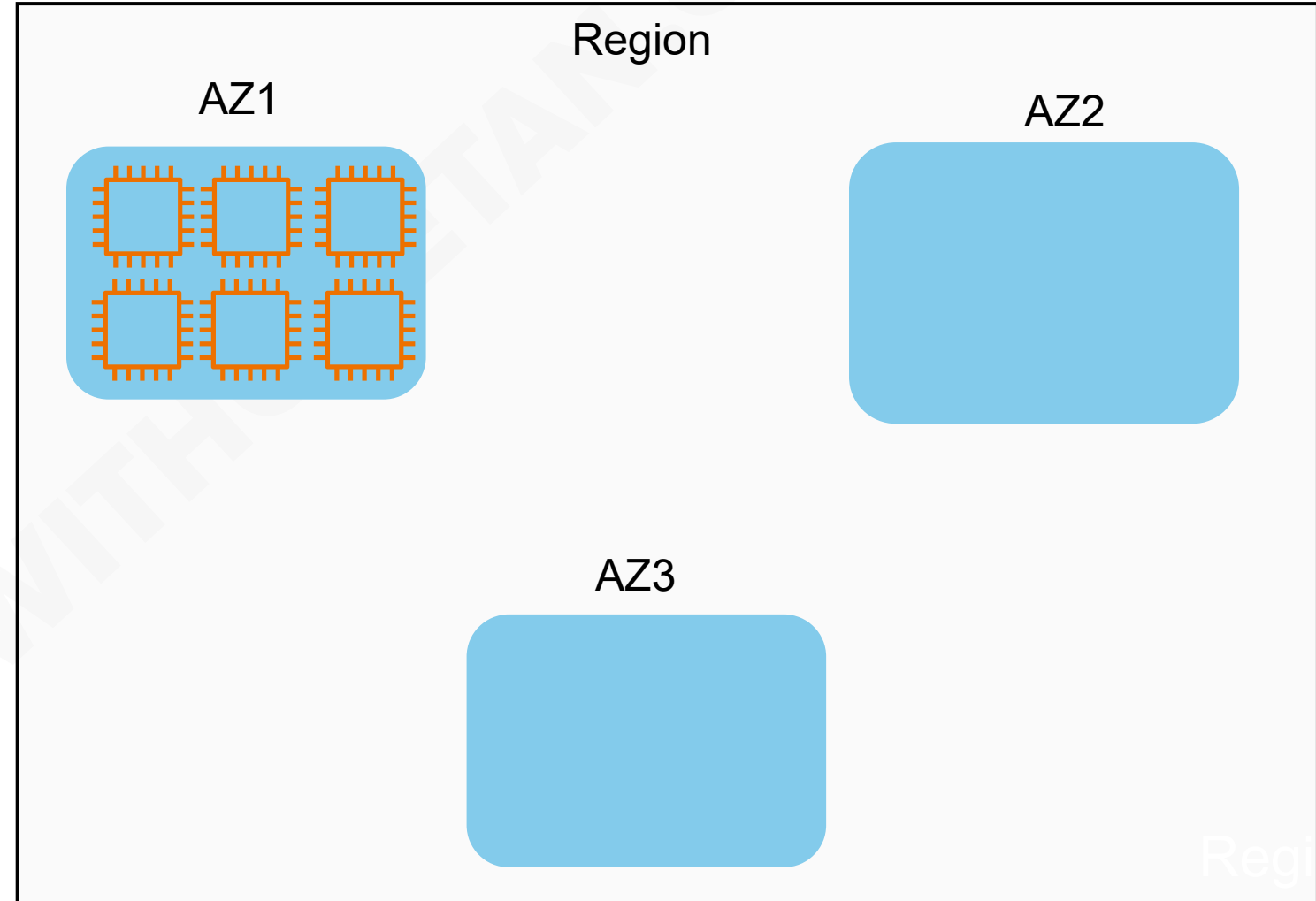


# Why 3 or more Availability Zones in a Region?

High Availability and failover

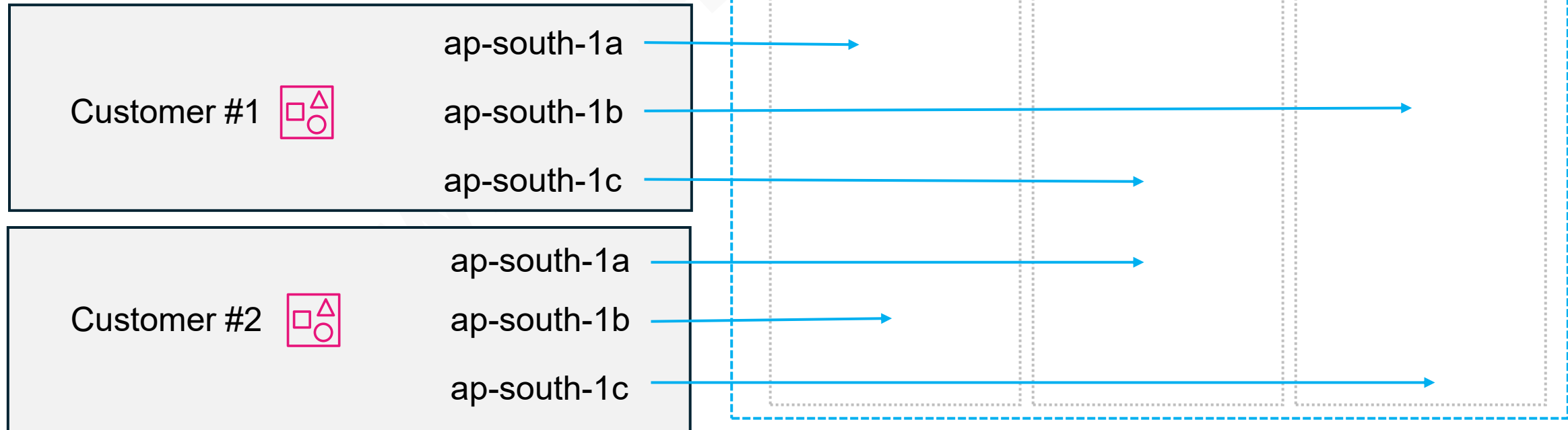
Durability

Scaling and load distribution



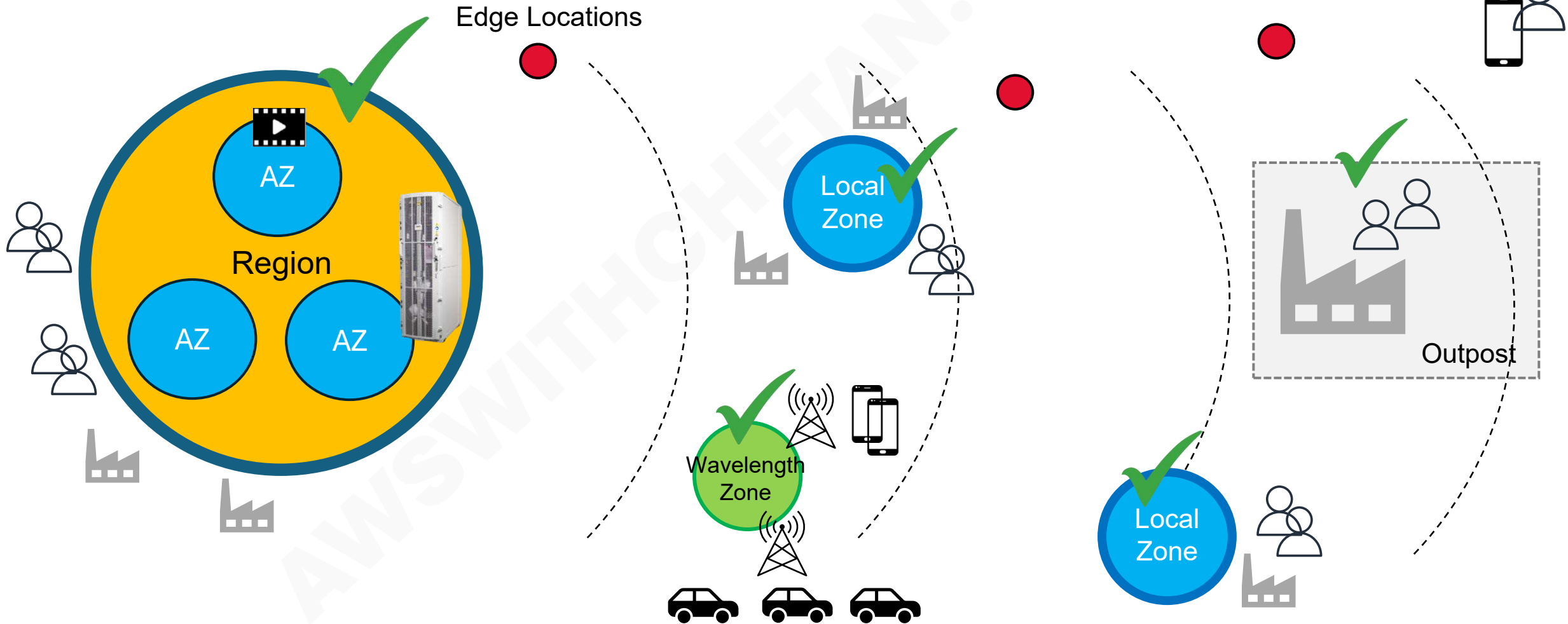
# Availability Zones

- There are minimum 3 AZs in each region
- Availability Zone has name (or code) and IDs
- Same AZ name may map to different AZ ID for different AWS account



# Beyond the AWS Region..

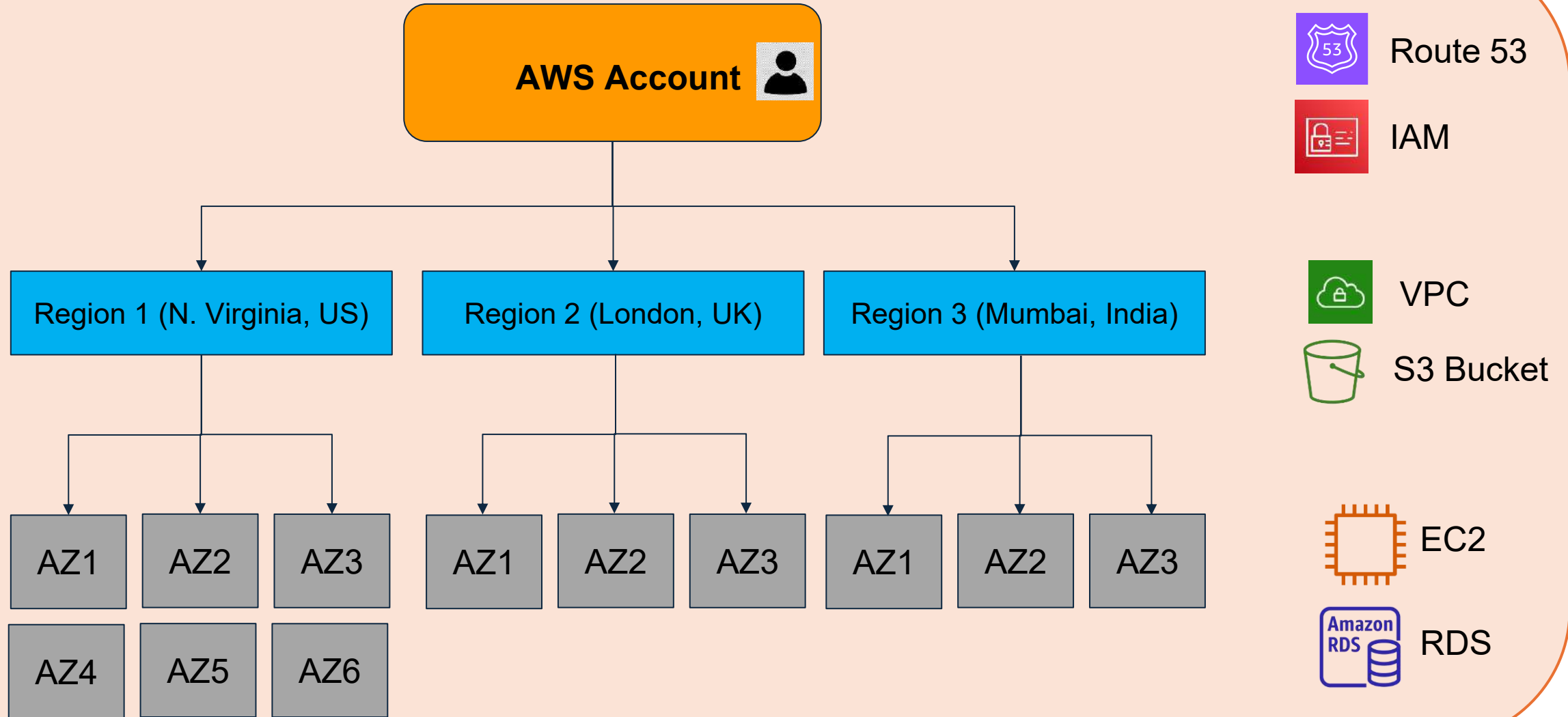
Access AWS Services...



# Getting started with AWS..

1. AWS Global Infrastructure
2. **AWS Account**
3. Overview of AWS Services (from Solutions Architect perspective)

# AWS Account



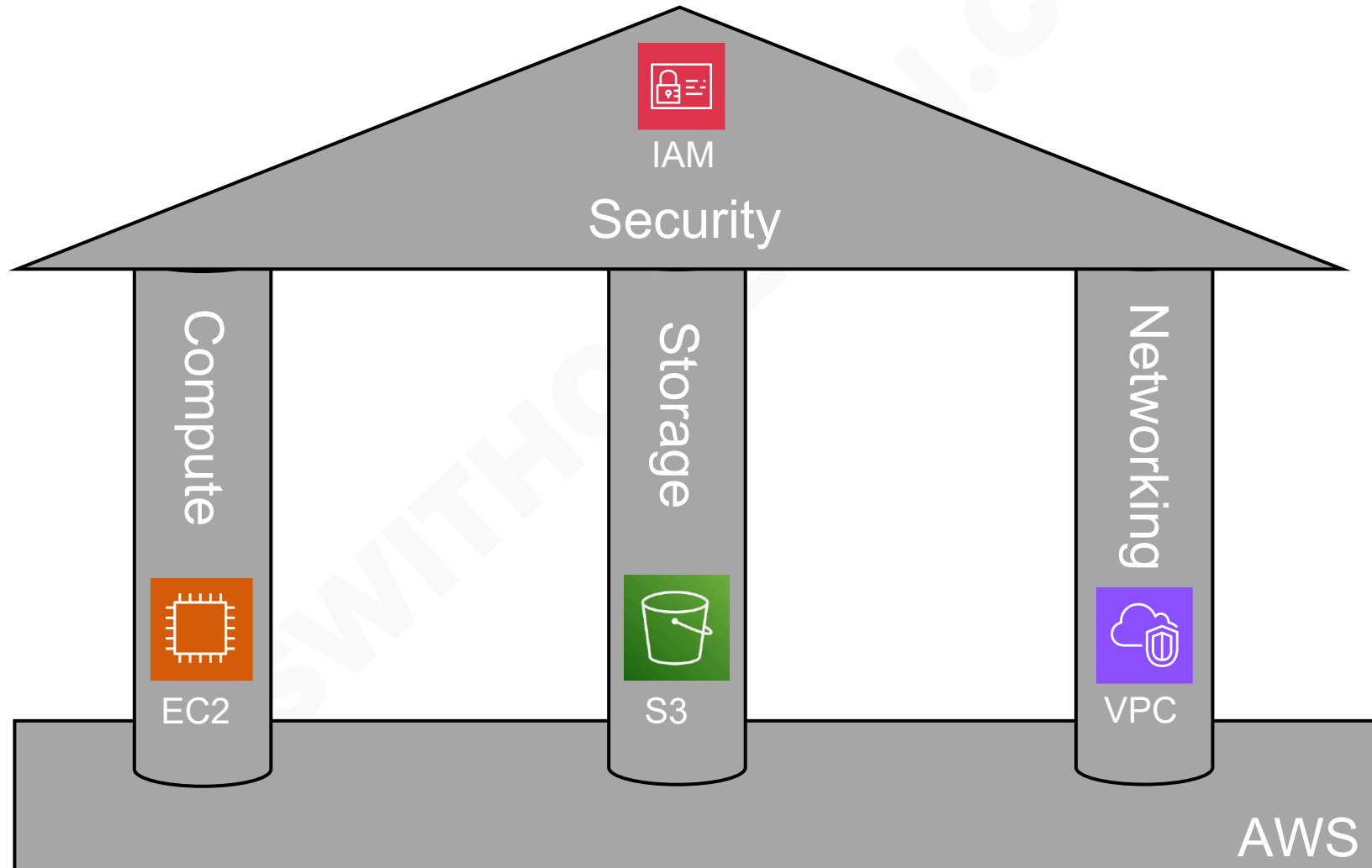


# AWS IAM

## Identity & Access Management

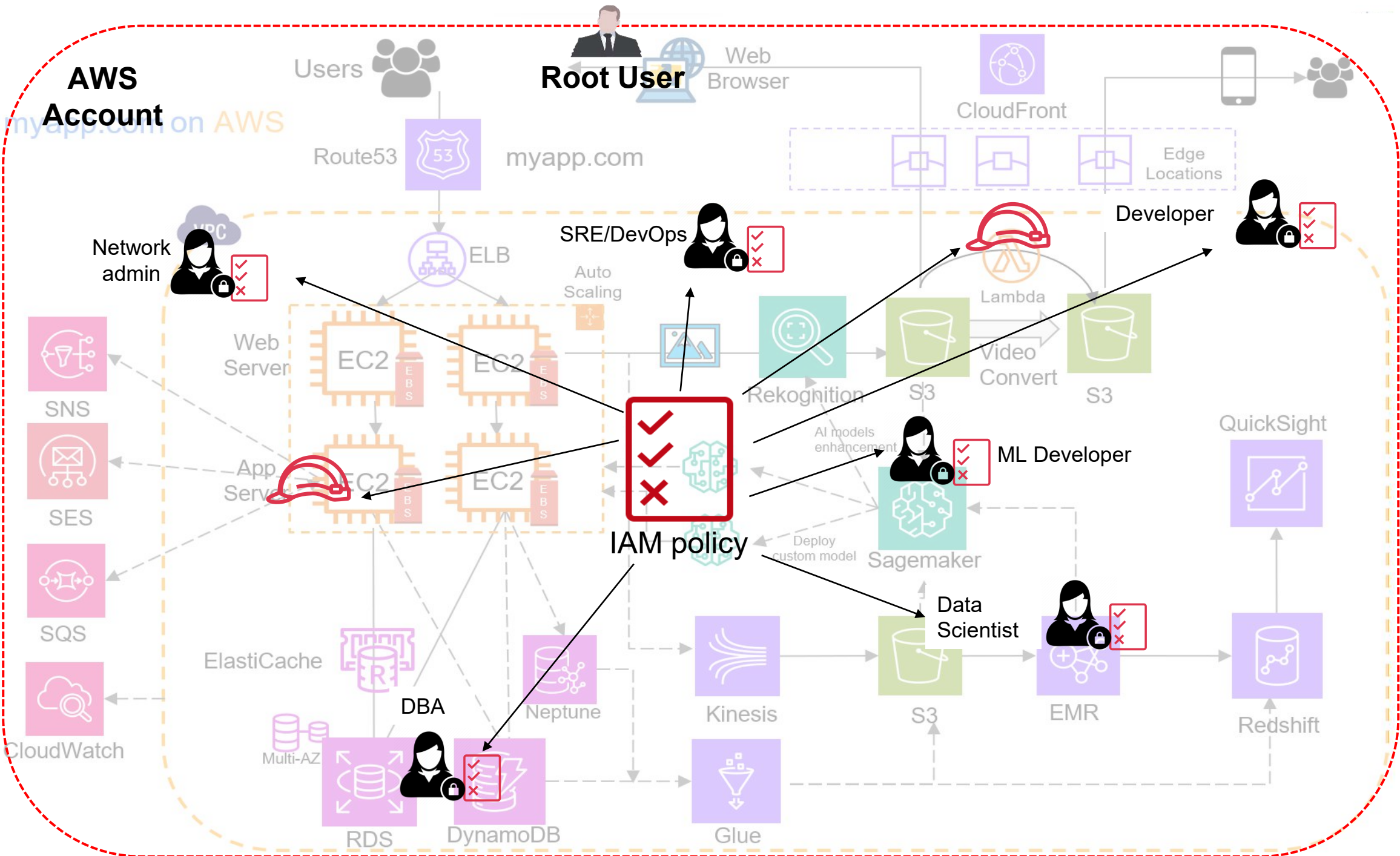


# AWS core services





IAM

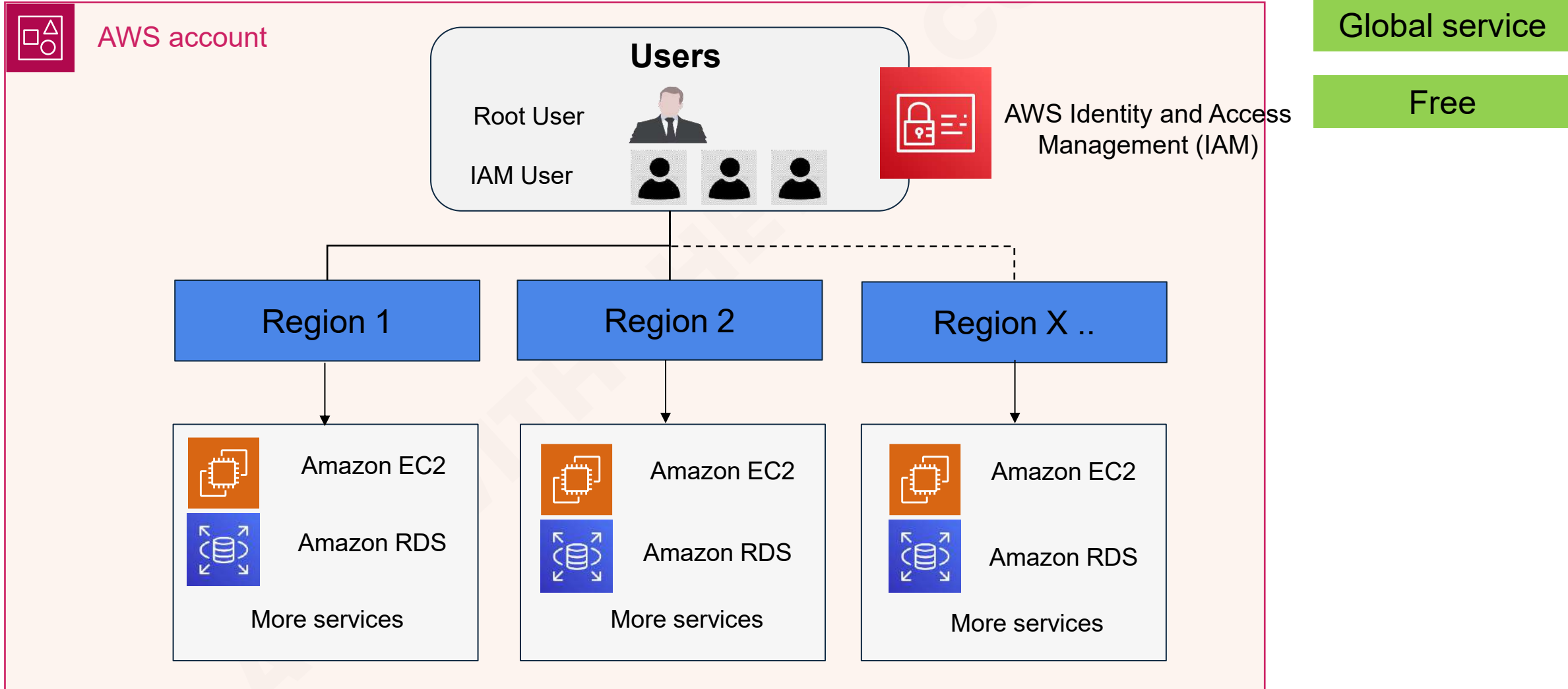


# In this section

- AWS account and Users
- Accessing AWS
- IAM Credentials – Username/password, Access Key, Temporary Credentials
- IAM Policy, types and features
- IAM Group
- IAM Role
- AWS CLI, SDK and AWS CloudShell
- IAM Permissions boundary
- IAM Tools – IAM Access Analyzer, IAM Policy Simulator & Policy generator
- IAM Audit and compliance
- IAM Best practices

# AWS Account and Users

# AWS Account and users



# AWS Account - User types



Root User

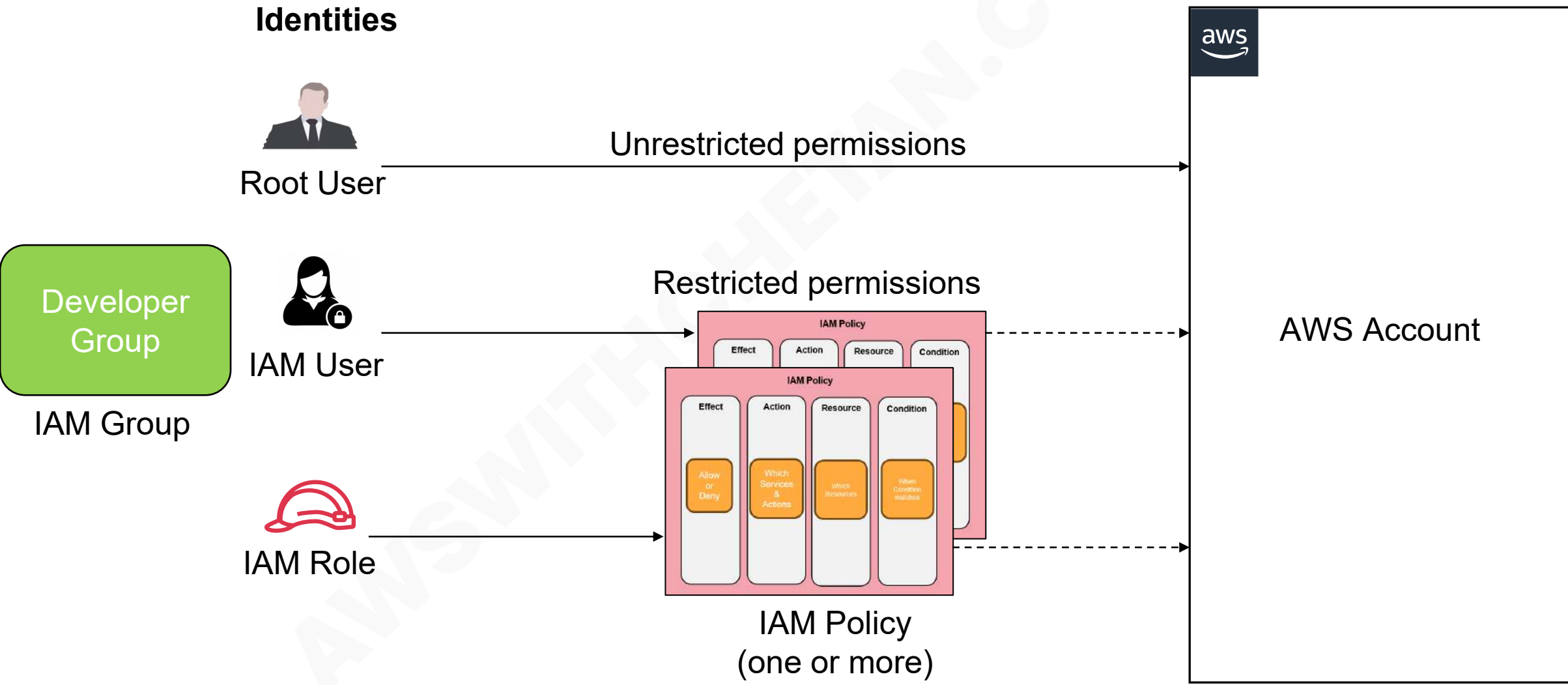


IAM User / Role



# Root user, IAM user & group and IAM Role

## Identities



# AWS account user types



## Root User

- Owner of the AWS account
- **IAM policy is not applicable and hence no control over the permissions**
- Use only when you need to perform special actions (account closure, billing access, payment etc.)
- **Not recommended for day-to-day operations**



## IAM User / Role

- Individual users or services or applications having specific permissions
- **IAM policy is applicable and can grant granular permissions**
- Should be used to by an individual person or application with only required access to AWS services and resources to perform the job.
- **Recommended for day-to-day operations**

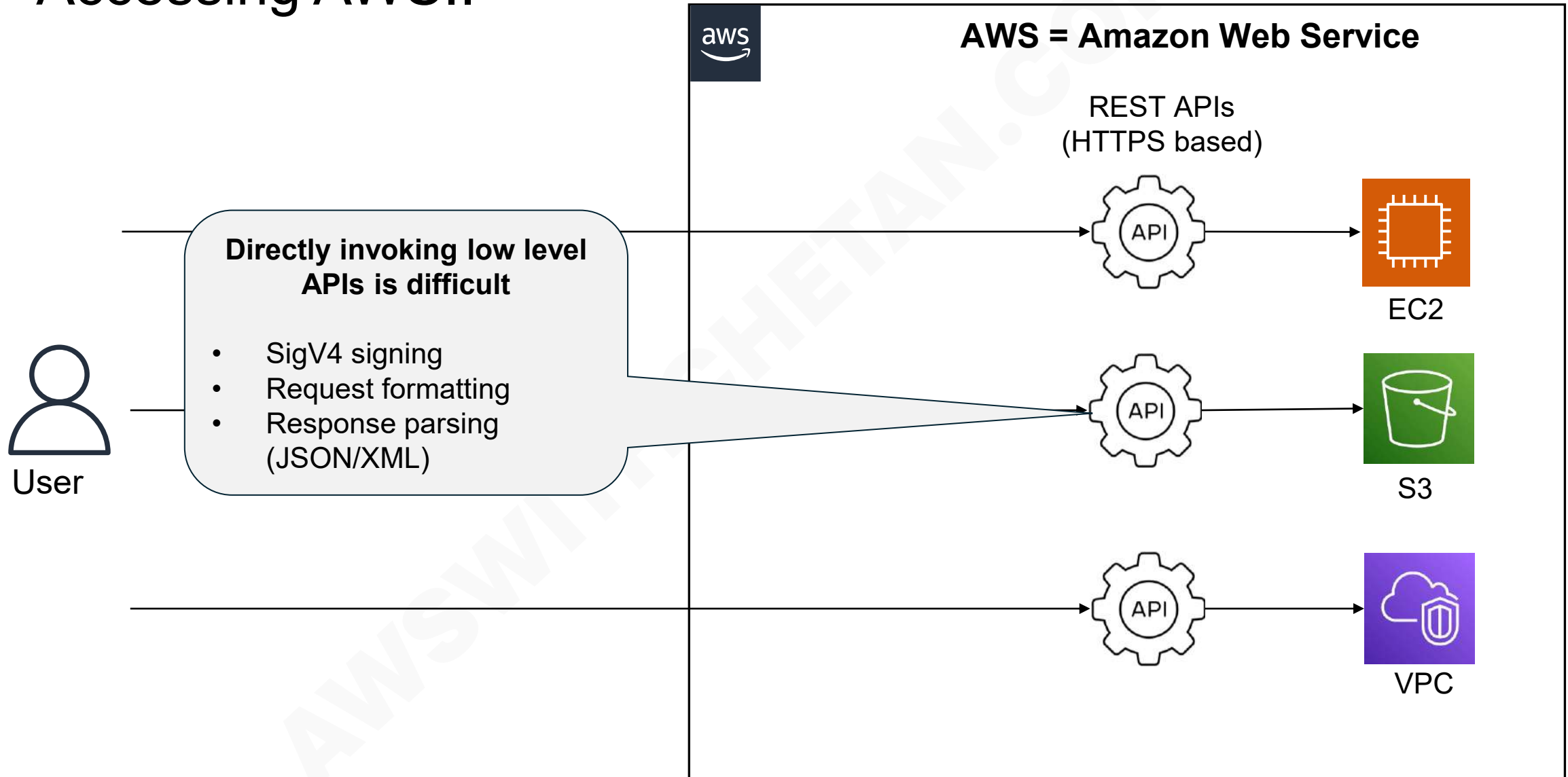


# Tasks that require Root User Access

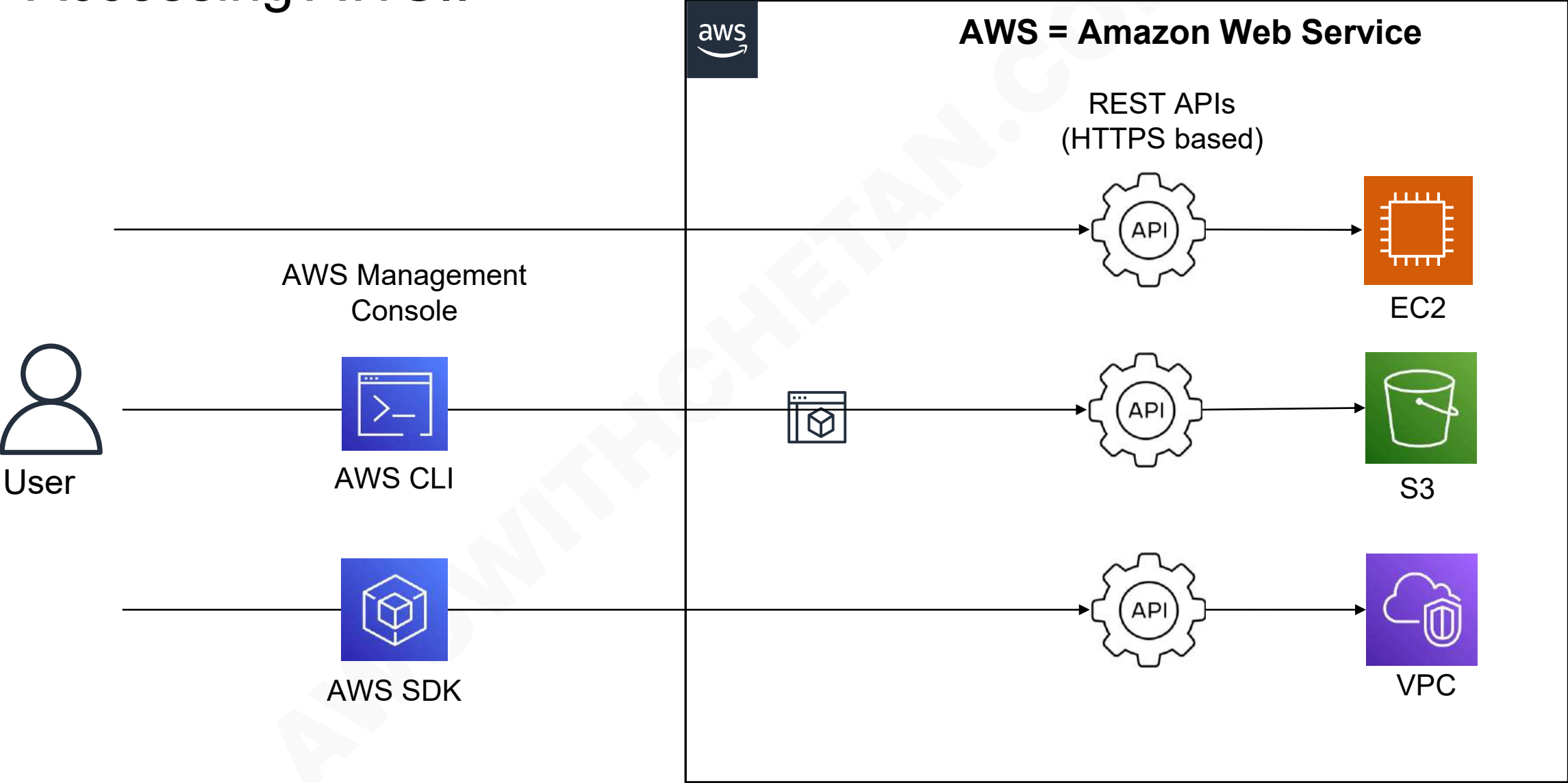
- ✓ Change your account Settings
- ✓ Restore IAM User Permission
- ✓ Close your AWS Account
- ✓ Activate IAM access to the Billing and Cost Management Console
- ✓ Configure an Amazon S3 bucket to enable MFA
- ✓ Change AWS support plan

# Accessing AWS

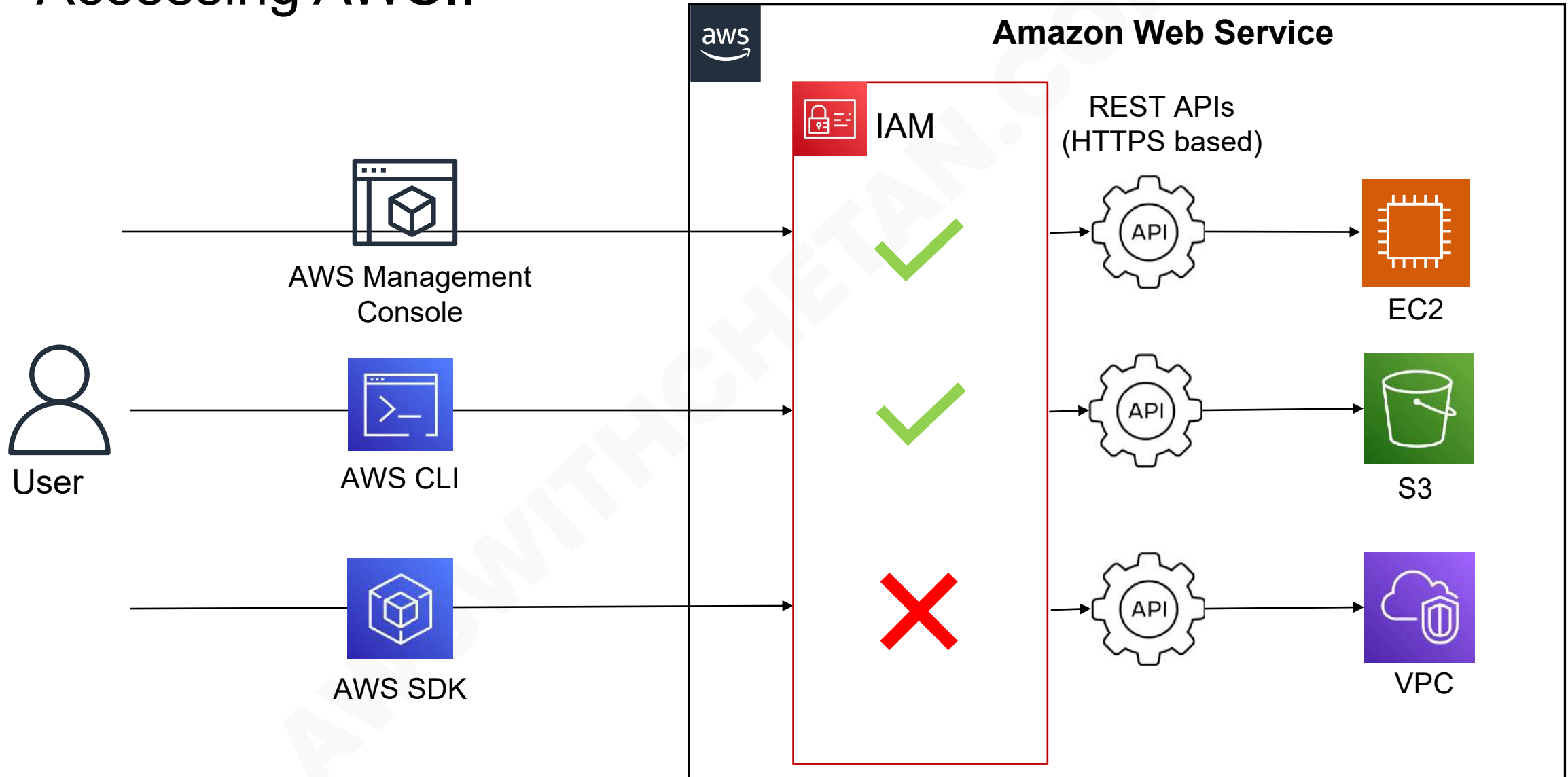
# Accessing AWS..



# Accessing AWS..

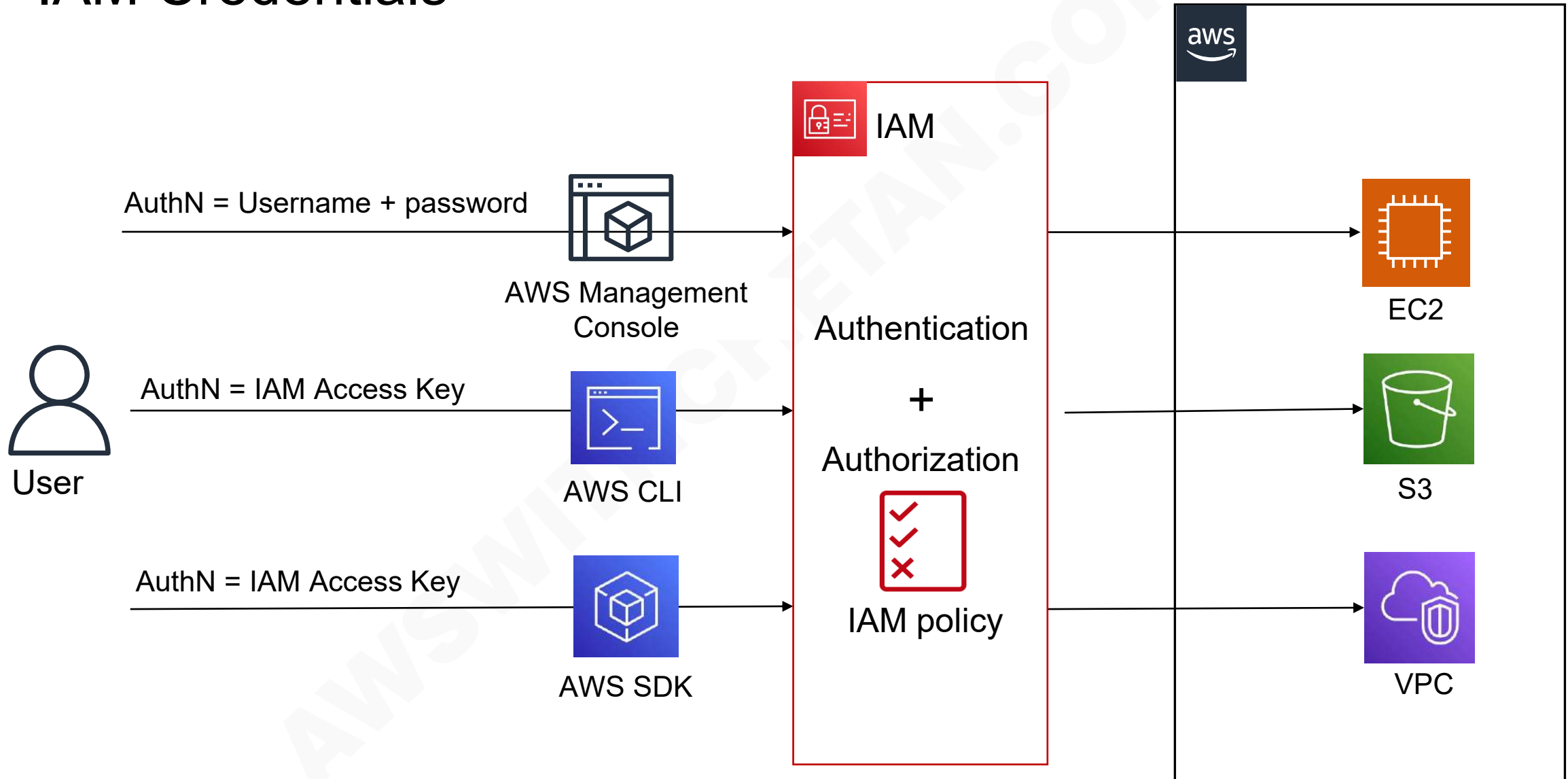


# Accessing AWS..



# IAM credentials

# IAM Credentials



# IAM Password policy

- There is a default Password policy which enforces certain restrictions on passwords such as – minimum length 8 chars, mix of char types etc.
- You can define custom password policy to define:
  - Password minimum length
  - Password strength (e.g. at least one uppercase, lowercase, special char etc.)
  - Turn on password expiration (1 to 1095 days)
  - Allow or deny users to change their own password
  - Prevent password reuse (1 to 24 previous passwords)



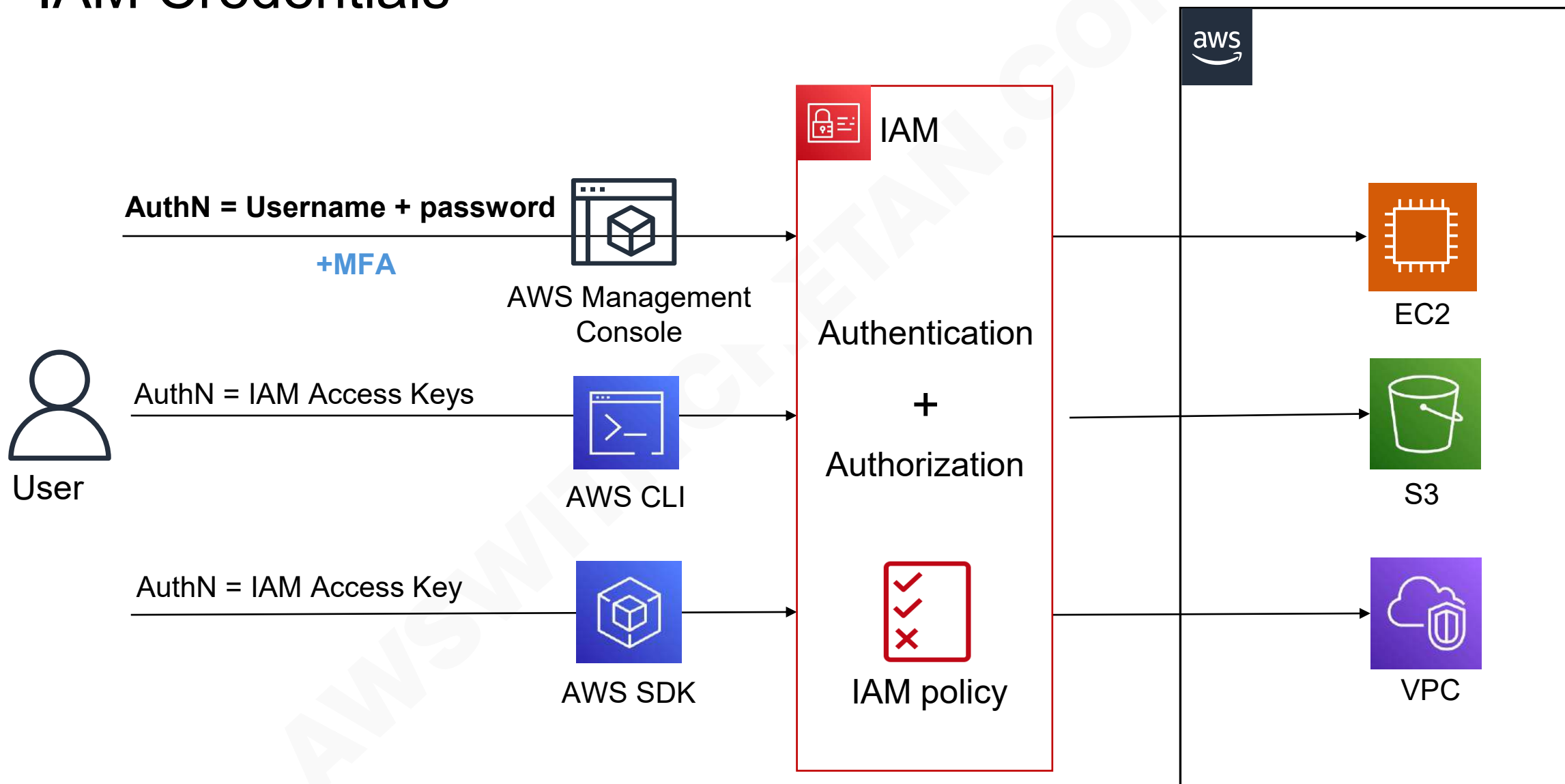
# Demo - IAM password policy

# Exercise : Create an IAM user

- 1 Login to AWS Management Console using Root user credentials (email/password) and navigate to IAM service
- 2 Click account id (Top right) and Turn on multi-session support
- 3 Go to Users -> Create user -> Provide username (say Ben) -> Provide user access to AWS Management console -> I want to create IAM user -> Custom password -> Enter the password -> Uncheck users must create a new password at next sign in -> Next
- 4 On the permissions screen -> Do not attach any policies at this moment
- 5 Go to IAM Dashboard page and copy the account sign-in URL under AWS account details
- 6 This time provide IAM username/password to login to AWS
- 7 Once logged in, you can check whether you can perform all AWS actions. Can you access EC2, S3 or IAM ?

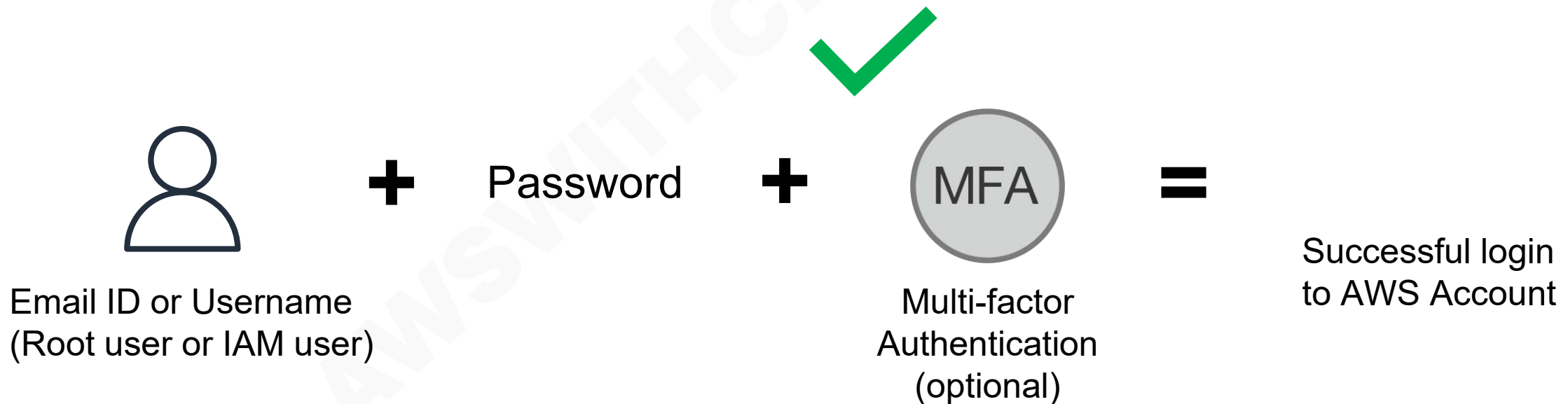
At this moment, your AWS account should have 3 users – Root User, *admin* IAM user (created as a part of pre-requisites) and *Ben* IAM user

# IAM Credentials



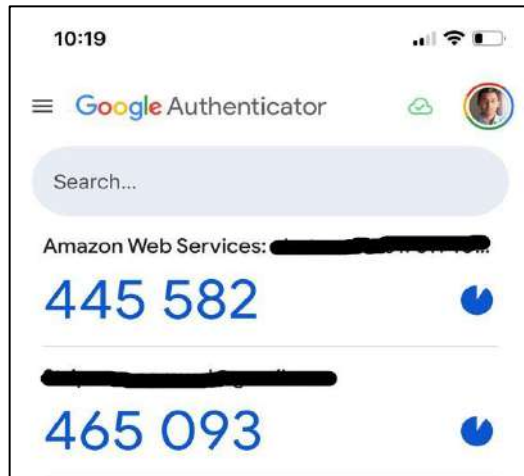
# Multi-factor authentication

- Multi-Factor Authentication adds a second layer of security
- **Once enabled**, user will need to provide not only a password to authenticate but also temporary digital token sent through a preset device like a smartphone running an Authenticator app.



# Multi-factor Authentication (MFA)

## Virtual Authentication Apps



Google Authenticator  
Microsoft Authenticator  
More apps..

## FIDO security keys



By third-party providers such as Yubico, acs, Gotrust etc.

## Hardware TOTP tokens



By Thales  
(3rd party provider)



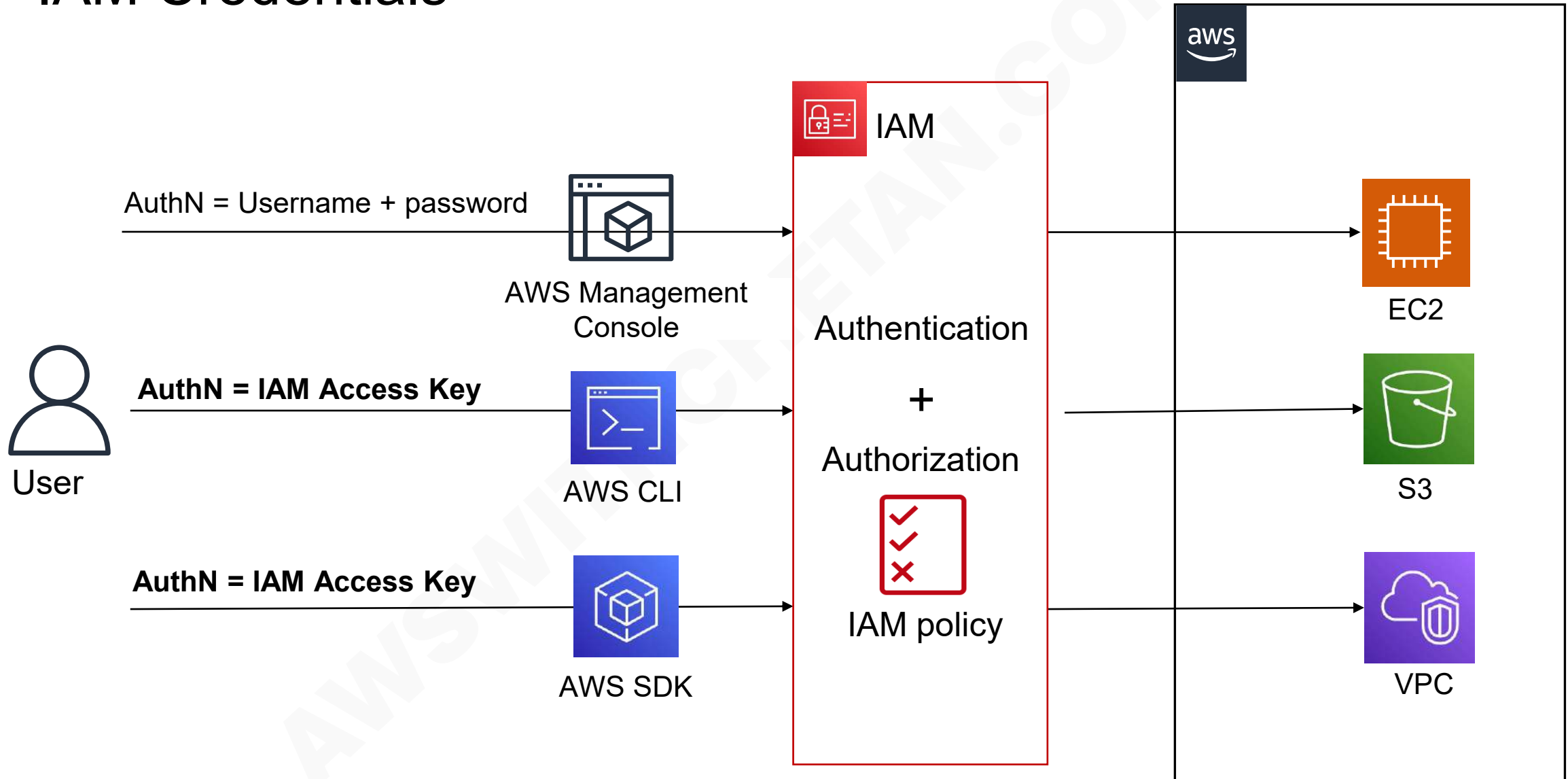
By Hypersecu  
(3rd party provider)

# Assignment : Set up MFA for IAM User

- 1 Download an AWS compatible Authenticator App e.g. Google Authenticator
- 2 Login to AWS Management Console using Root user and navigate to IAM.
- 3 Click on your IAM Username for which you want to setup MFA
  - Click Users on the left navigation panel and click on the user for which you want to set up MFA
  - Click on Security credentials tab
  - Click on Manage in Assigned MFA device row
  - Select Virtual MFA device and click Continue
  - You will see a dialog window with instruction to setup MFA and a Show QR code button
  - Click on Show QR in the above dialog so that you can scan it using your app
- 4 Open Authenticator App installed in Step 1 and scan QR code.
  - App detects your account.
  - Click on Add ACCOUNT in Authenticator app to add your AWS account in the authenticator app.
  - Enter 2 Consecutive MFA codes from your Authenticator App.
- 5 Verify MFA Setup
  - Log out of your account and try to login again.
  - You will be prompted for an MFA code after you enter your username/password, provide MFA from your authenticator App.

# IAM credentials

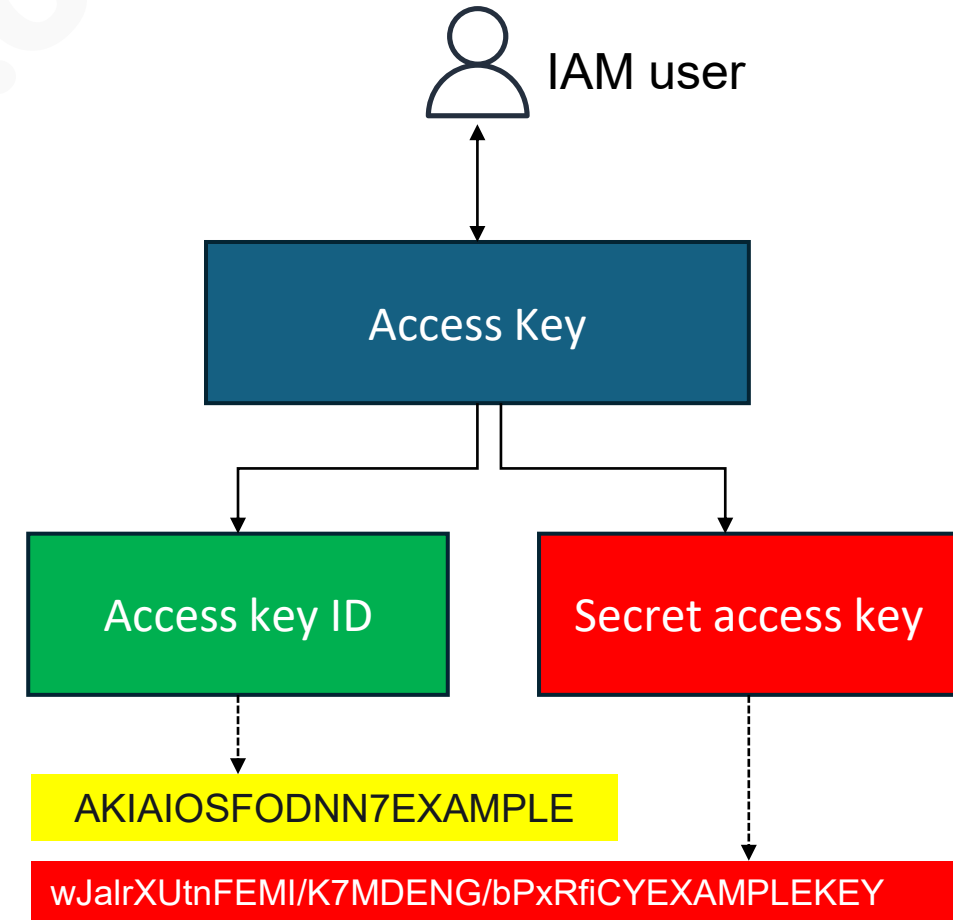
# IAM Credentials





# IAM credentials – Access key

- Access keys are long-term credentials for an IAM user or the AWS account root user.
- Used to sign AWS API programmatic requests directly or via AWS CLI or AWS SDK (SigV4)
- Access keys consist of two parts:
  - Access key ID =~ Identifies user
  - Secret access key =~ Like password
- A user can have maximum 2 active Access keys at any time
- It can not be re-generated.
- After generating Access key, store both Access key id and secret access key securely.



# Exercise : Create IAM user access key

- 1 Login to AWS account using Root user or IAM admin user
- 2 Go to IAM console -> Users -> Select the user for which IAM access key to be created
- 3 Go to Security Credentials -> Access Keys -> Create access key -> Save the keys locally

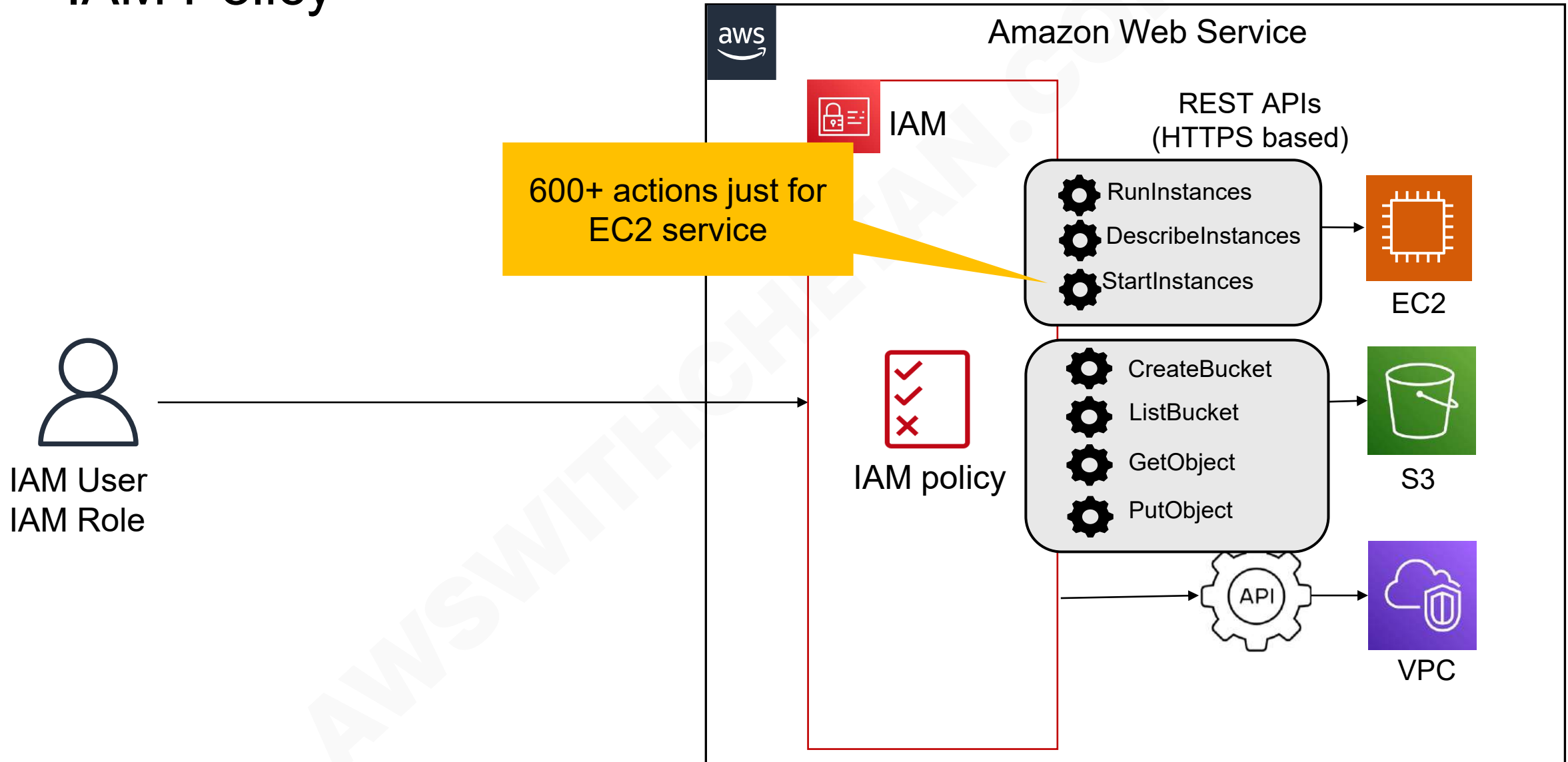
# IAM policy

# IAM Policy

- In AWS, a policy is a JSON document that defines permissions to access AWS services and resources.
- These policies are associated with AWS IAM users, groups, and roles to control their permissions

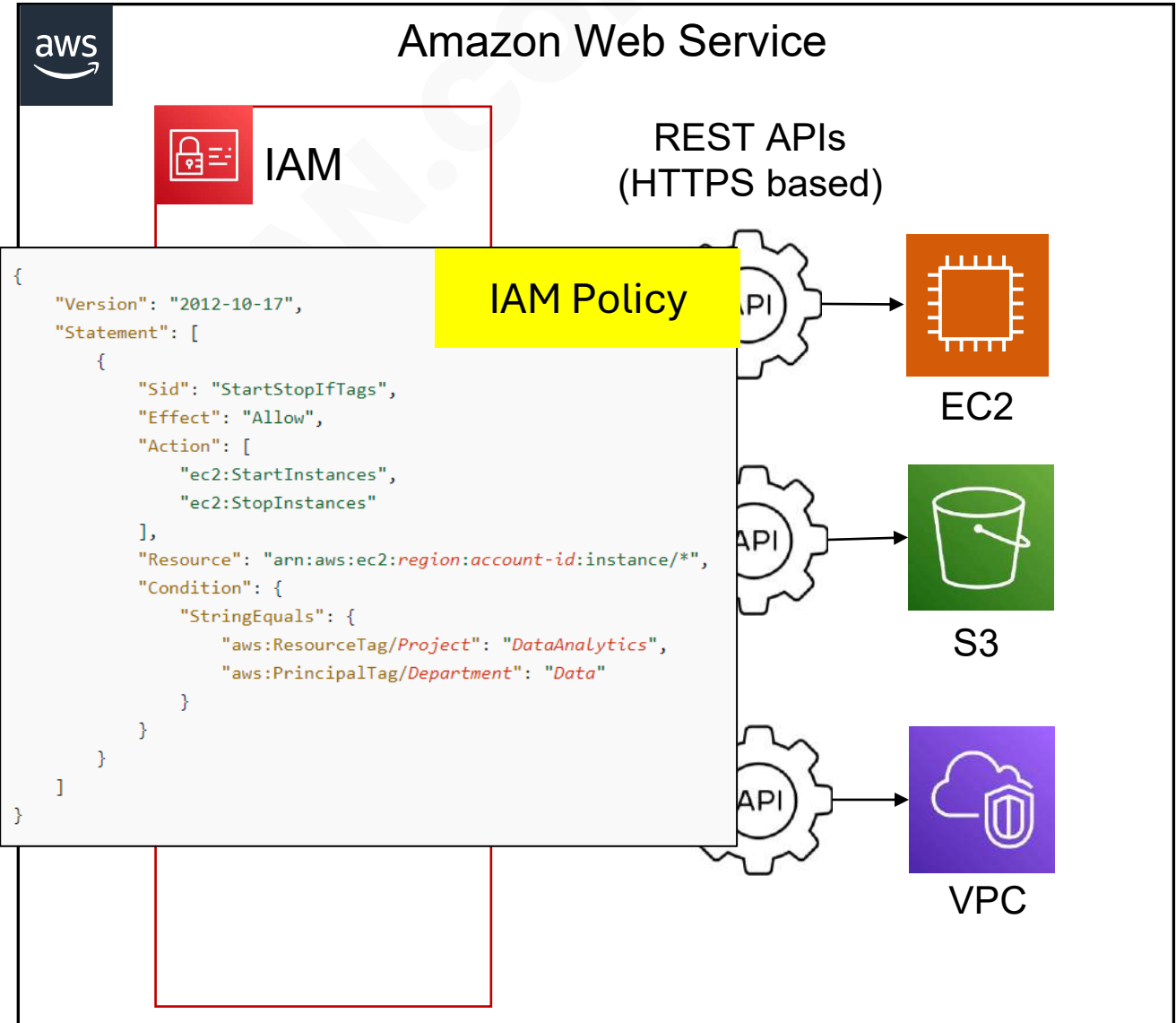
AWSWITHCHETAN.COM

# IAM Policy

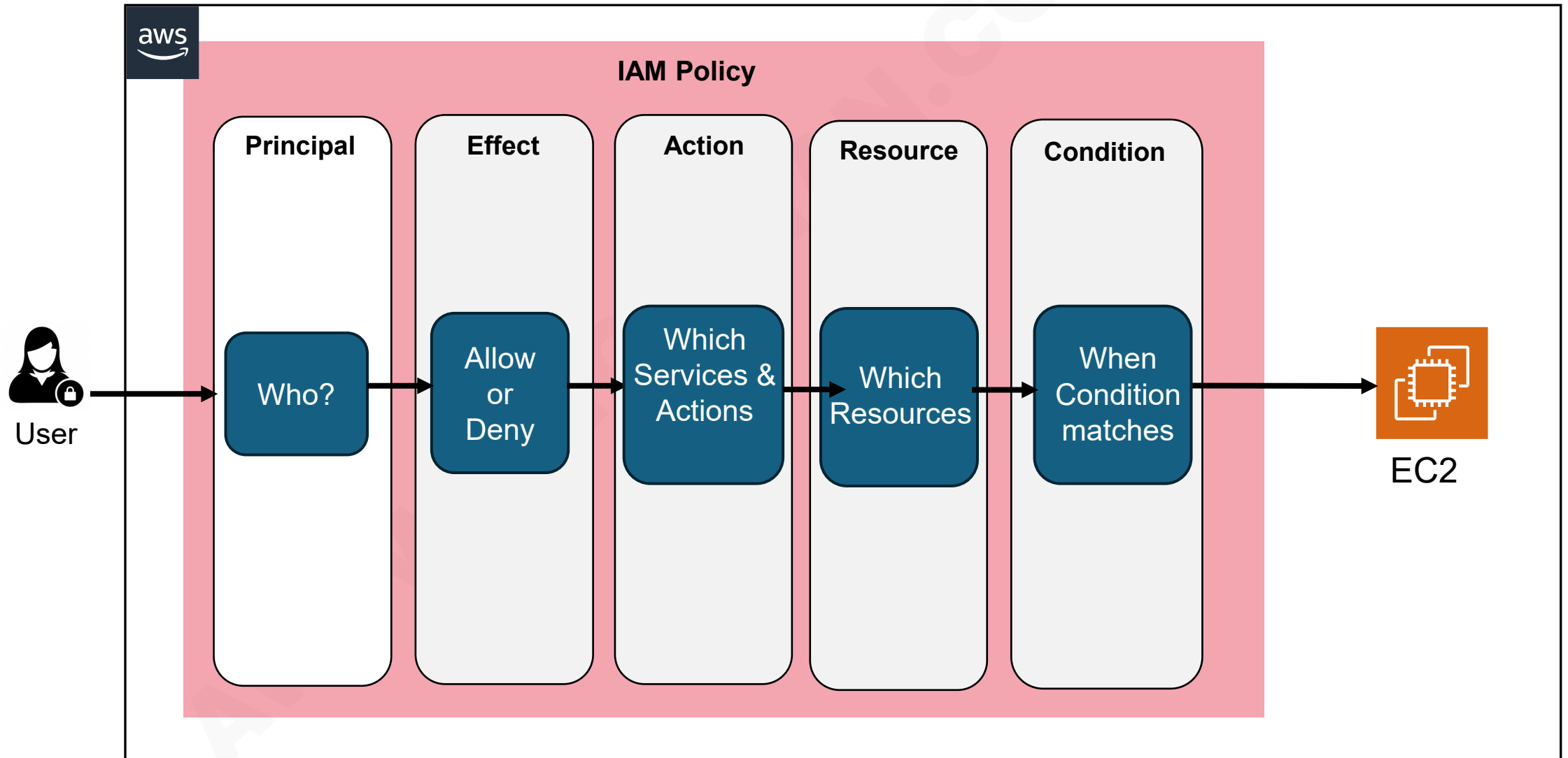


# IAM Policy

  
IAM User  
IAM Role



# IAM Policy components



# IAM Policy

- IAM Policy is a JSON document

What is JSON?

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "StartStopIfTags",
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "arn:aws:ec2:region:account-id:instance/*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Project": "DataAnalytics",
          "aws:PrincipalTag/Department": "Data"
        }
      }
    }
  ]
}
```

Example IAM Policy: Start or stop instances based on tags



# JSON

- JSON (JavaScript Object Notation) is a lightweight, text-based format used to store and exchange data.
- Uses Simple key-value pairs
  - "Action": "s3:ListBucket"
- Supports nesting (multi-level) structure
- Readable by both Humans and the Computers

```
{  
  "name": "John Doe",  
  "address": {  
    "city": "New York",  
    "country": "USA"  
  },  
  "email": "john.doe@example.com"  
}
```

# IAM Policy

- IAM Policy is a JSON document
- **Policy Version:** 2012-10-17
- **Id:** string-to-identify-policy (*optional*)
- **Statements:** one or more statements
- Each statement has:
  - **Sid:** Identifier for the statement (*optional*)
  - **Effect:** Allow or Deny access
  - **Principal:** AWS account or IAM user or IAM role
  - **Action:** List of actions (*use \* for all*)
  - **Resource:** Particular resources for which perm
  - **Condition:** Apply policy statement only when c

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "StartStopIfTags",
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "arn:aws:ec2:region:account-id:instance/*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Project": "DataAnalytics",
          "aws:PrincipalTag/Department": "Data"
        }
      }
    }
  ]
}
```

Example IAM Policy: Start or stop instances based on tags

# Sample IAM policies

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:DescribeInstances"
      ],
      "Resource": "arn:aws:ec2:ap-south-1:123456789012:"
    }
  ]
}
```

## Users not allowed to Terminate instances

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action":
        "ec2:TerminateInstances",
      "Resource": "*"
    }
  ]
}
```

# Sample IAM policies

## { **To launch EC2 instances only in Mumbai region**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:RunInstances",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "ap-south-1"
        }
      }
    }
  ]
}
```

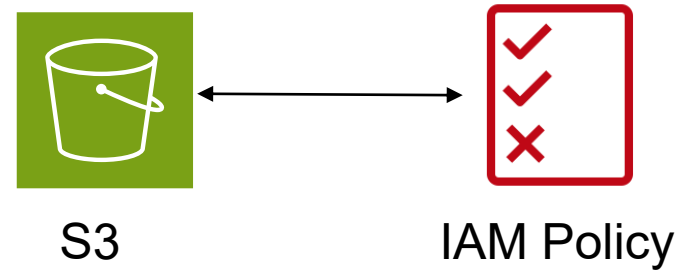
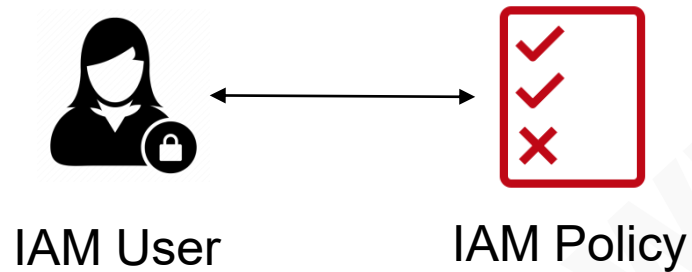
## **Lambda to upload/download S3 objects**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::my-app-bucket/*"
    }
  ]
}
```

# IAM policy types

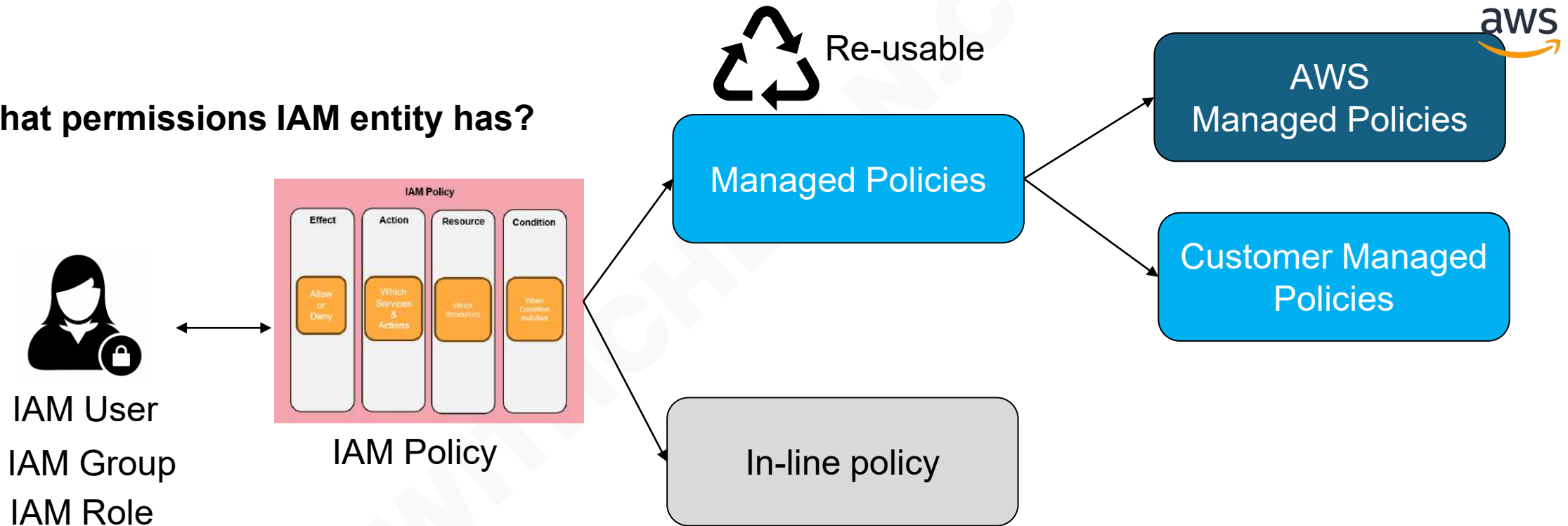
# IAM Policy types

- Identity based policy -> Attached to IAM user, group or role
- Resource based policy -> Attached to AWS resource e.g. S3 bucket



# Identity based policy

What permissions IAM entity has?



embedded directly inside a specific IAM user, group, or role. Can't be re-used.

# Identity based IAM policy - Example

## { **User can start/stop EC2 instances**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:DescribeInstances"
      ],
      "Resource": "arn:aws:ec2:ap-south-1:123456789012:"
    }
  ]
}
```

## **Users not allowed to Terminate instances**

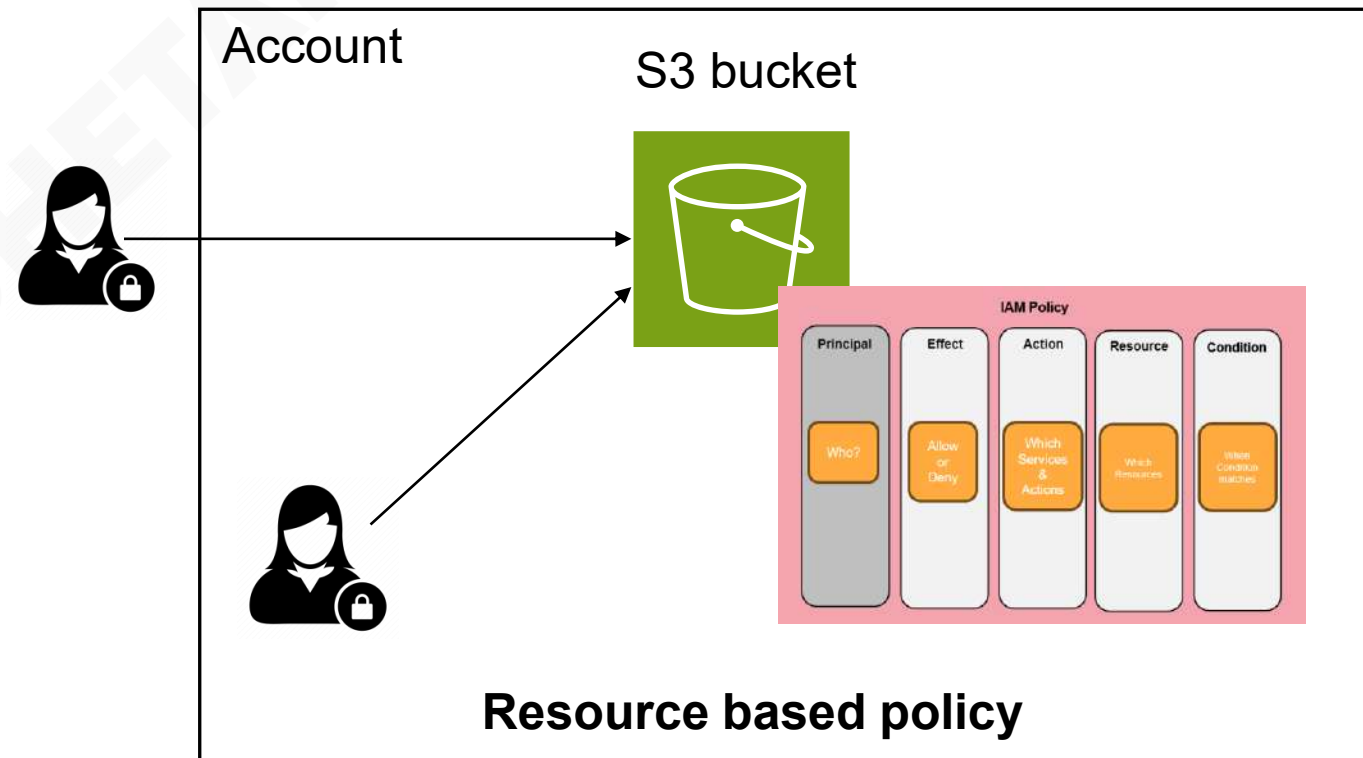
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action":
        "ec2:TerminateInstances",
      "Resource": "*"
    }
  ]
}
```



# Resource based policy

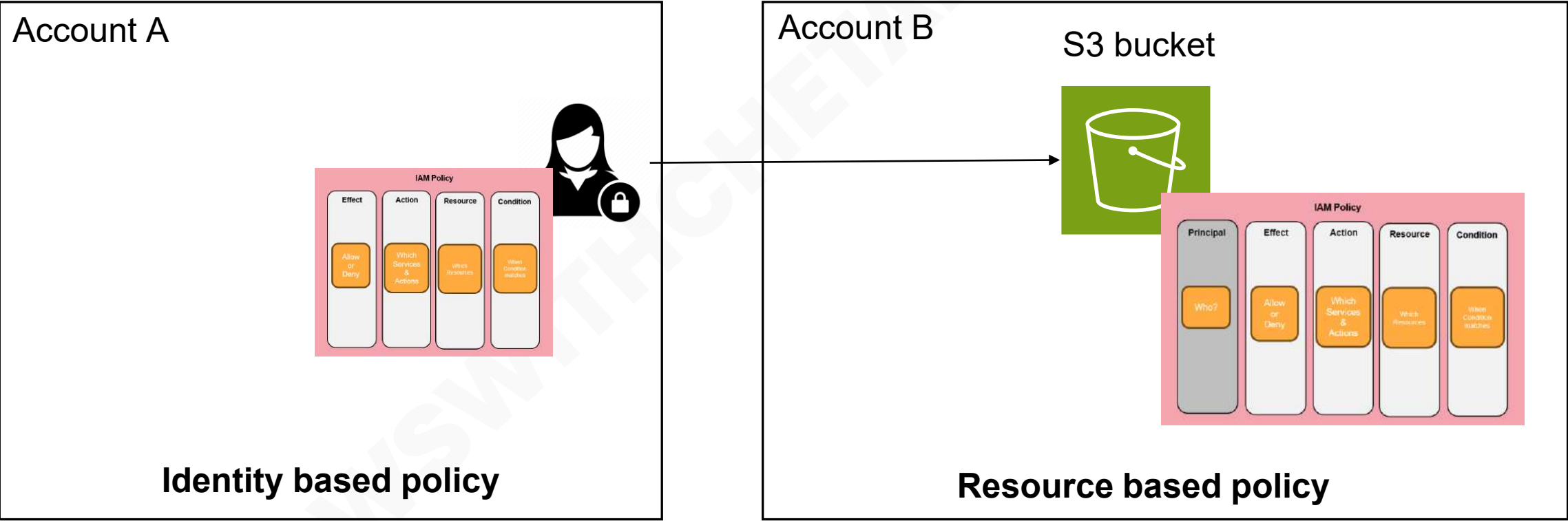
- A resource-based policy is attached directly to a resource instead of being attached to an IAM user, group, or role.
- It defines who (**principal**) can access the resource and under what conditions.
- It includes a "**Principal**" element — which identity-based policies do not
- Supported by S3, KMS, SQS, SNS and many other AWS services

Who has permissions to access this resource?



# Resource based policy – Use case

## Cross-account Access to S3 bucket



# Resource based policy - Examples

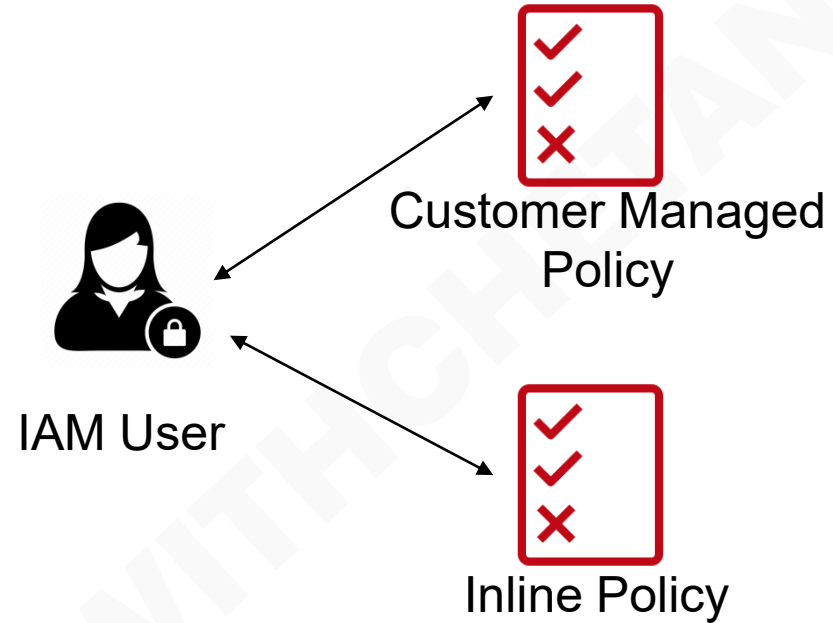
Make S3 bucket publicly accessible  
(e.g. For hosting static website)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicRead",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-public-
bucket/*"
    }
  ]
}
```

Allows an external AWS account  
Account A to send messages to your Queue in Account B.

```
{
  "Version": "2012-10-17",
  "Id": "CrossAccountSQSAccess",
  "Statement": [
    {
      "Sid": "AllowAccountAtoSend",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::111122223333:root"
      },
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:ap-south-
1:444455556666:MyQueue"
    }
  ]
}
```

# Exercise: Create IAM policies



# Exercise : Create IAM policies

- 1 Login to AWS account using admin IAM user and Create a new IAM policy with the name say "CustomPolicy-S3-Full-Access". Use following JSON to define the policy permissions.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Statement1",
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": "*"
    }
  ]
}
```

- 2 Attach this policy to IAM user that you created earlier (Ben in my case)
- 3 Also create and attach Inline policy to this user. (See Policy document in the next slide.)

# Exercise : Create IAM policies

- 4 Inline policy to grant EC2 read access to this user.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*"
      ],
      "Resource": "*"
    }
  ]
}
```

- 5 Go to S3 console and see if this user can create a new S3 bucket. Provide unique bucket name.
- 6 Go to EC2 console and see if this user can view EC2 details. Can the user launch an ec2 instance?

**Tip:** Generally adding AWS account id in the bucket name is a good practice to make sure that bucket names are unique

# IAM policy - Conditions

# IAM Policy conditions

- Conditions in IAM policies let you fine-tune access control b
- They specify "when", "where", or "how" a policy is effective.
- Condition Operators:
  - StringEquals or StringNotEquals
  - Bool
  - IpAddress / NotIpAddress
  - DateGreaterThan / DateLessThan
- Condition keys:
  - aws:RequestedRegion
  - aws:SourceIp
  - aws:MultiFactorAuthPresent
  - aws:username
  - aws:CurrentTime

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:RunInstances",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:RequestedRegion": "ap-south-1"
        }
      }
    }
  ]
}
```



# Common policies with conditions

- Allow launching EC2 only in North Virginia region using condition key `aws:RequestedRegion`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:RunInstances",
      "Resource": "*",
      "Condition": {
        "StringEquals": { "aws:RequestedRegion": "us-east-1" }
      }
    }
  ]
}
```

# Common policies with conditions

- Enforce MFA for the sensitive actions using condition key `aws:MultiFactorAuthPresent`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:DeleteObject",
        "s3:PutBucketPolicy"
      ],
      "Resource": "arn:aws:s3:::my-sensitive-bucket/*",
      "Condition": {
        "Bool": { "aws:MultiFactorAuthPresent": "true" }
      }
    }
  ]
}
```

# Common policies with conditions

- Deny all action if not logged in from the corporate network using condition key `aws:SourceIp`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyAllOutsideCorporateNetwork",
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "NotIpAddress": {
          "aws:SourceIp": [ "203.55.22.0/24" ]
        }
      }
    }
  ]
}
```

# Common policies with conditions

- Allow each user to access only their own folder/path in S3 using condition key `aws:username`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::my-company-data/${aws:username}",
        "arn:aws:s3:::my-company-data/${aws:username}/*"
      ]
    }
  ]
}
```

# Common policies with conditions

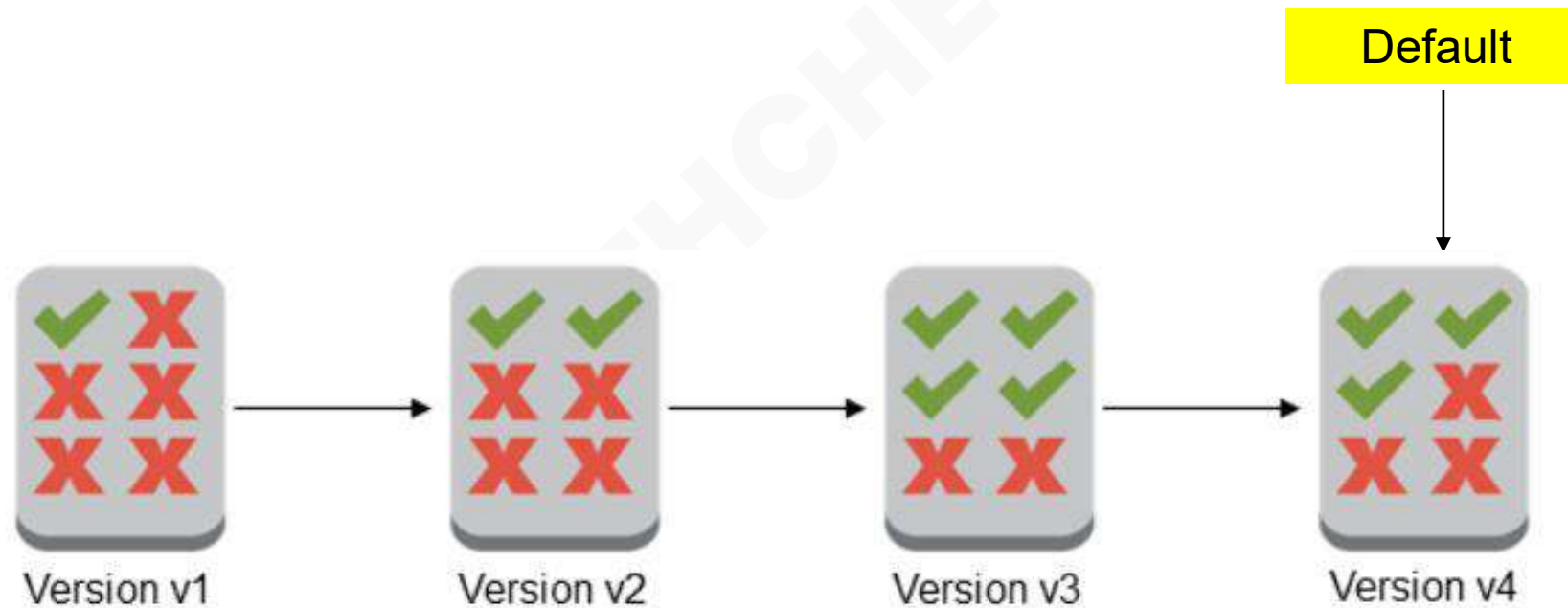
- Allow EC2 actions only during office hours (9 AM – 6 PM UTC) using condition key `aws:CurrentTime`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [ "ec2:StartInstances", "ec2:StopInstances" ],
      "Resource": "*",
      "Condition": {
        "DateGreaterThan": { "aws:CurrentTime": "2025-10-26T09:00:00Z" },
        "DateLessThan": { "aws:CurrentTime": "2025-10-26T18:00:00Z" }
      }
    }
  ]
}
```

# IAM Policy – Good to know

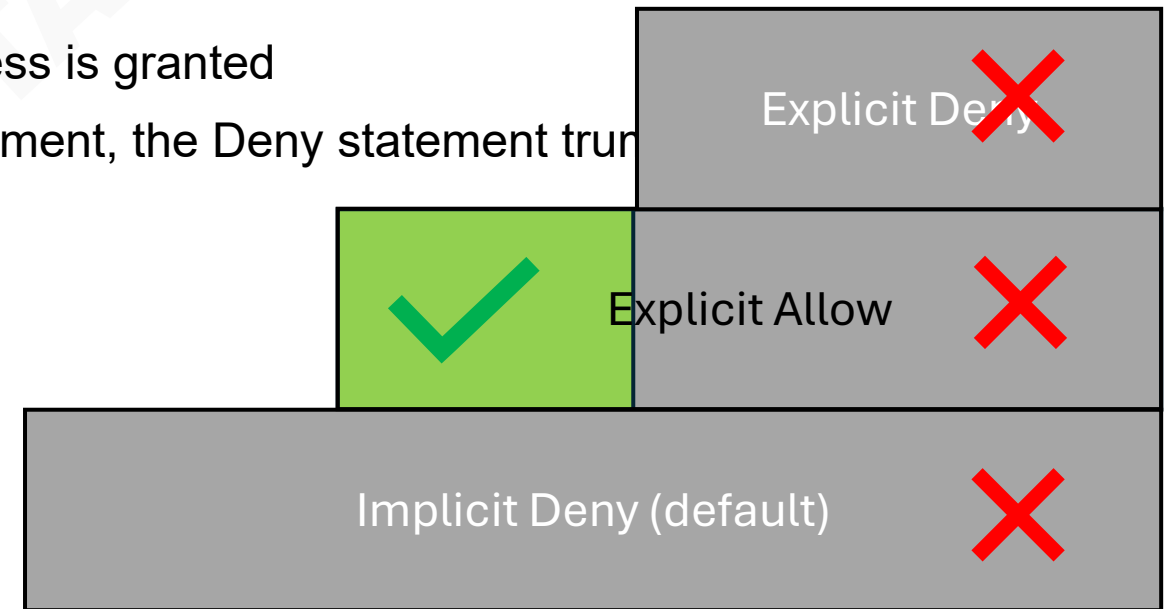
# Versioning IAM policies

- When a change is made to existing IAM policy, a new version of the policy is created
- This ensures that you can rollback the policy in case there is any side effect observed
- You can have up to 5 versions



# IAM policy effect – Deny vs Allow

- IAM principal is denied access by default. This is called **Implicit Deny**.
- An implicit denial occurs when there is no applicable Deny statement but also no applicable Allow statement
- If policies only include Allow Statements, then the access is granted
- If policies include an Allow statement and a Deny statement, the Deny statement trumps the Allow statement. This is called **Explicit Deny**.



“Explicit Deny overrides Allow.”



Policy 1



Policy 2

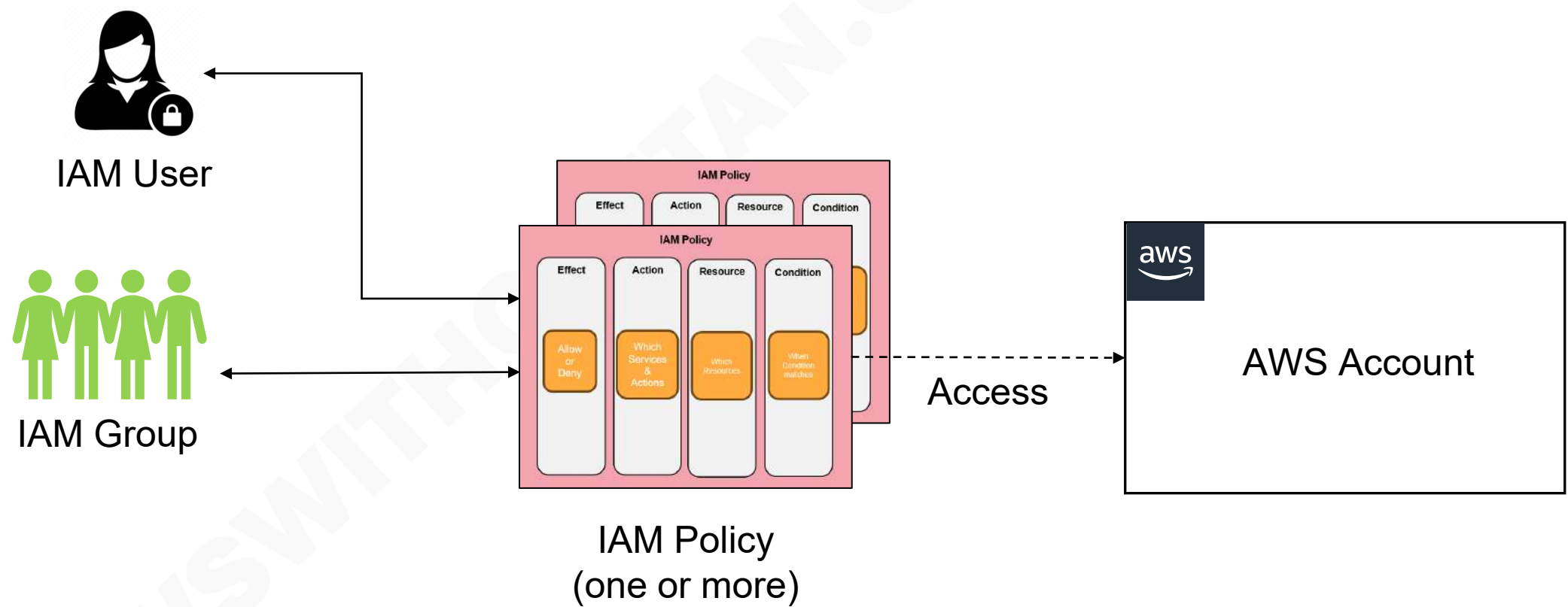
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowStartInstances",
      "Effect": "Allow",
      "Action": "ec2:StartInstances",
      "Resource": "*"
    }
  ]
}
```



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyEC2",
      "Effect": "Deny",
      "Action": "ec2:*",
      "Resource": "*"
    }
  ]
}
```

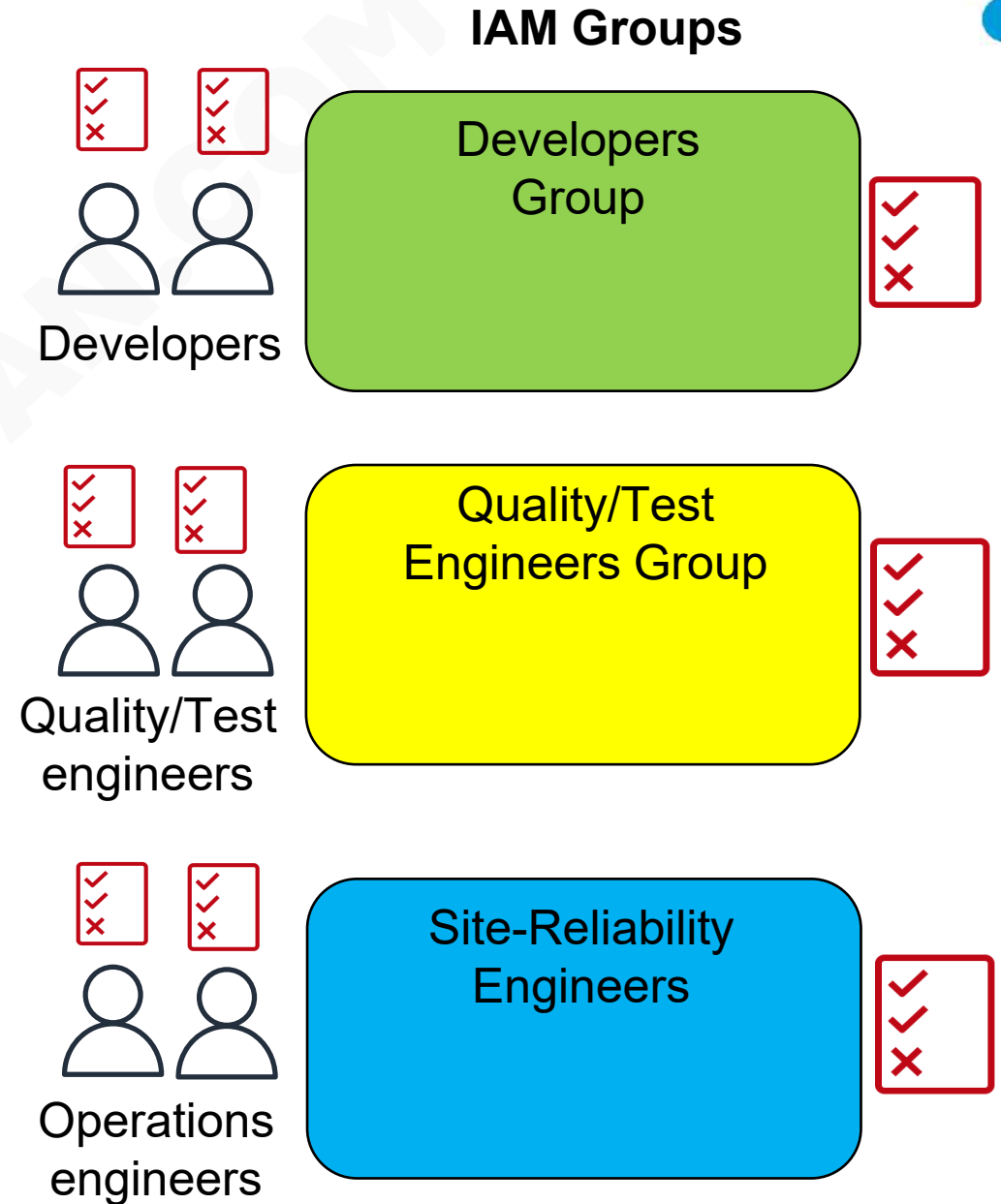
# IAM Group

# IAM Group



# IAM Groups

- Create IAM group and add IAM users to the group
- Instead of associating IAM policies to individual users, associate IAM policies to the group
- A group can contain multiple users, and user can belong to multiple groups.



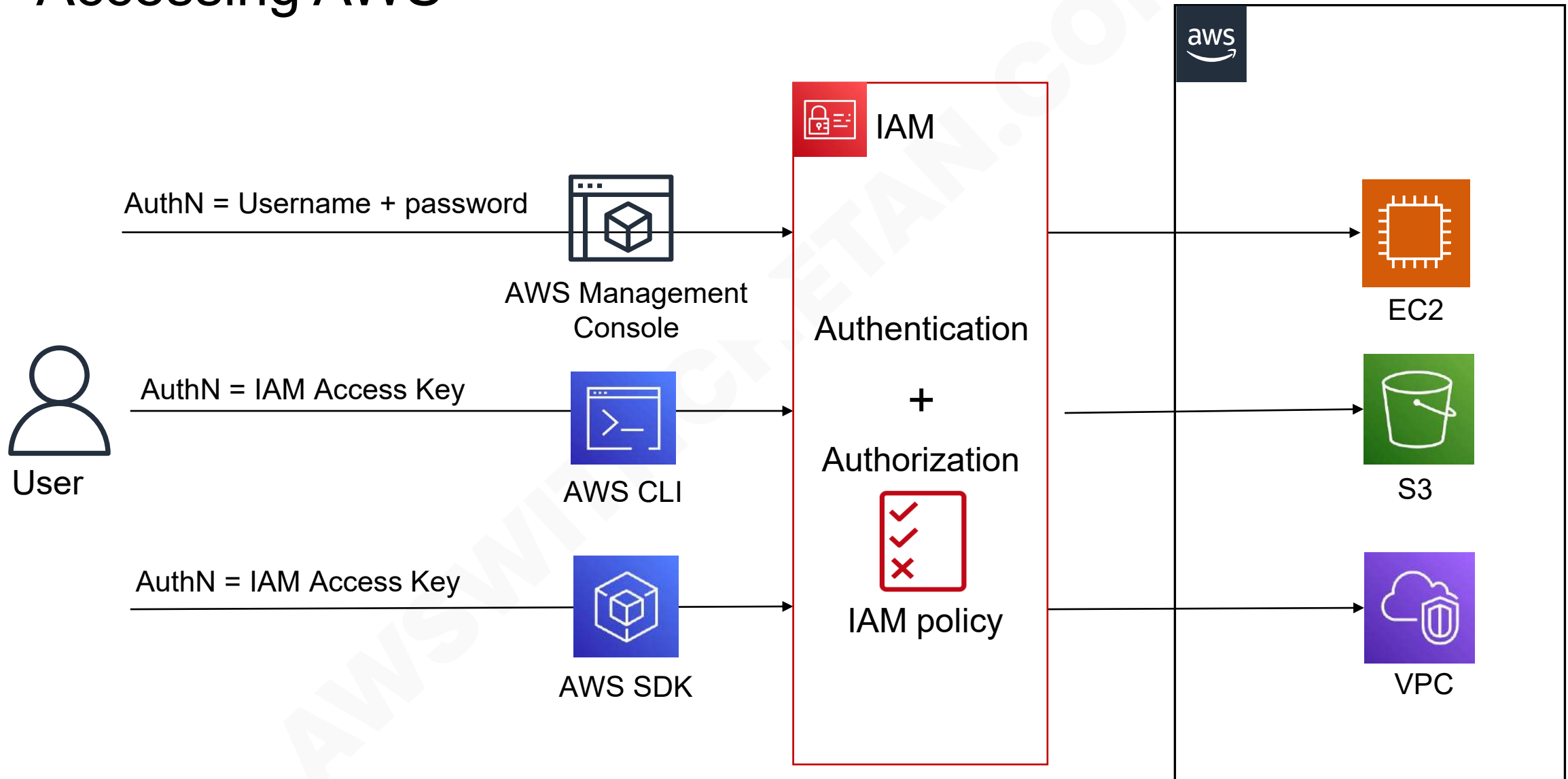
# Assignment: IAM group

- 1 Login as **admin** IAM user and create a new IAM group say 'DevOps' and attach following AWS managed policies to this group
  - AmazonCloudWatchFullAccess
  - AmazonSSMFullAccess
  - AWSEC2FullAccess
  - AWSCloudFormationFullAccess
  - AWSLambda\_FullAccess
- 2 Login to AWS account as Ben (*you can have multiple sessions in AWS console*) and try creating a new Lambda function -> Access denied error
- 3 Now Add Ben user to this group
- 4 Try creating a new Lambda function
  - Use a Blueprint -> Select Hello World function (Python)
  - Function Name: MyFirstLambdaFunction
  - Create function -> Should be successful

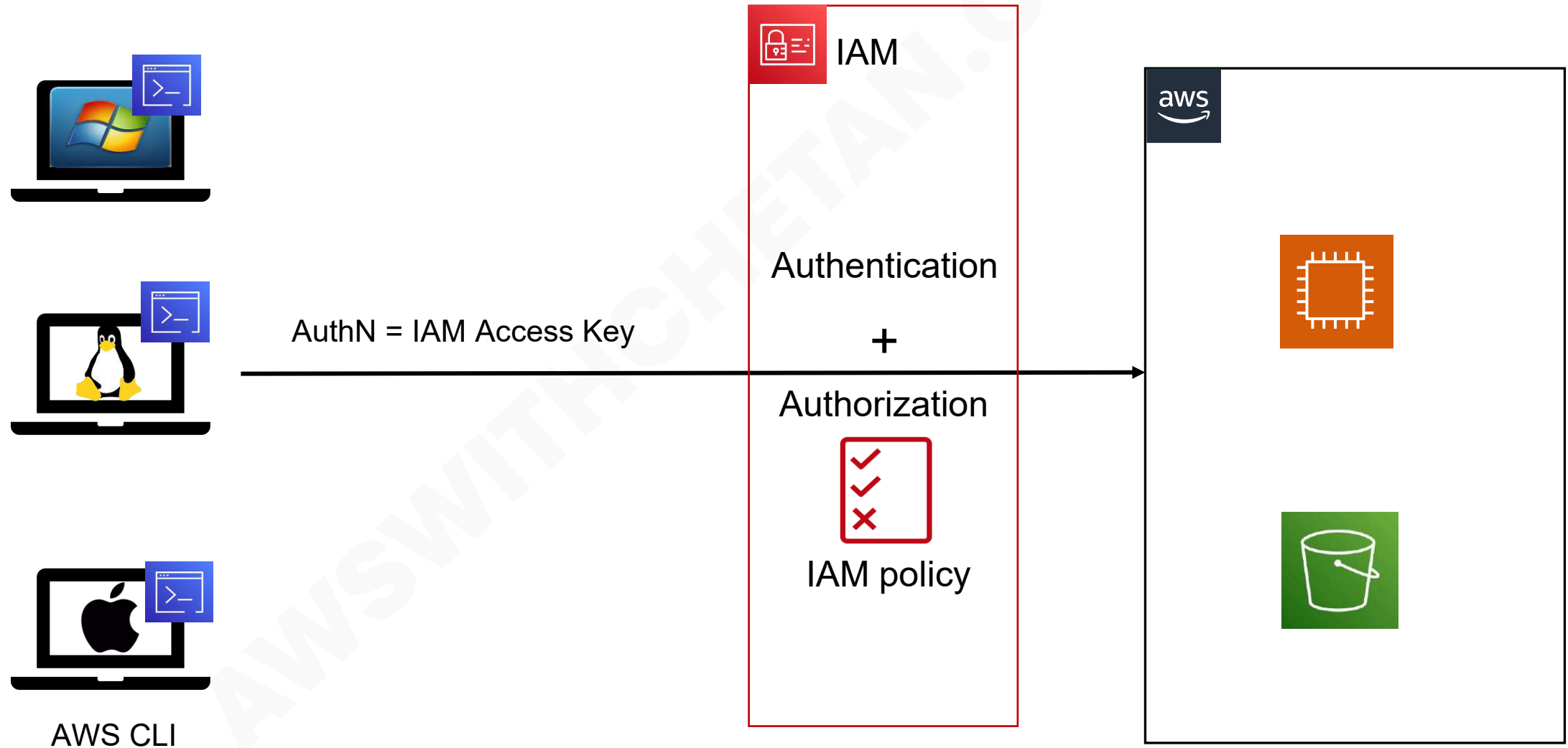
# AWS CLI

## AWS Command Line Interface

# Accessing AWS



# Accessing AWS using AWS CLI





# AWS CLI

- A Command Line Interface (CLI) to access AWS
- Install CLI into your workstation (as per the operating system)
- Configure AWS CLI by providing:
  - AWS region -> default region for CLI. Provide region code.
  - Access Key ID -> your access key id
  - Secret access key -> your secret access key
  - Output format -> text or json
- Access AWS using CLI commands:

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

```
[cloudshell-user@ip-10-134-40-230 ~]$ aws ec2 describe-regions
{
  "Regions": [
    {
      "Endpoint": "ec2.ap-south-1.amazonaws.com",
      "RegionName": "ap-south-1",
      "OptInStatus": "opt-in-not-required"
    },
    {
      "Endpoint": "ec2.eu-north-1.amazonaws.com",
      "RegionName": "eu-north-1",
      "OptInStatus": "opt-in-not-required"
    },
    {

```

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

# Exercise : Using AWS CLI

- 1 Install AWS CLI into your workstation (follow the steps as per your workstation operating system)  
<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
- 2 Configure AWS CLI with Ben user Access key credentials (you had created access key previously)
  - AWS region -> Provide default region code.
  - Access Key ID -> your access key id
  - Secret access key -> your secret access key
  - Output format -> text or json

- 3 Run the AWS CLI commands:

```
aws ec2 describe-instances //should be successful with empty list
aws s3 ls                  //should show buckets in your account
```

# AWS SDK

## AWS Software Development Kit

# AWS SDK

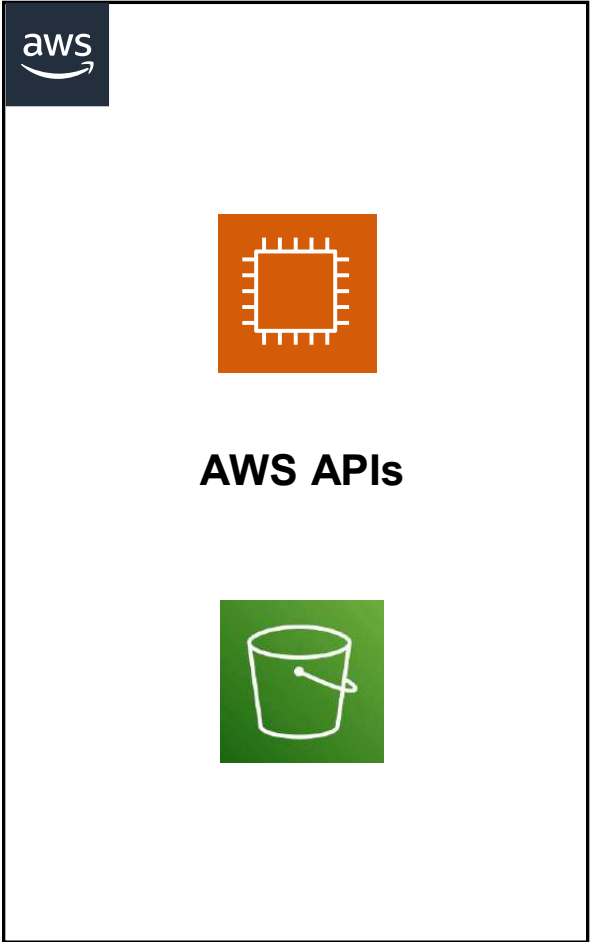
- AWS web services are exposed through REST & HTTP APIs
- SDK wraps these APIs to handle low level operations e.g. SigV4 request signing, authentication, headers management
- SDKs are language specific
- Cloud based Applications are built using corresponding language SDK

|         |                     |
|---------|---------------------|
| Java    | AWS SDK for Java v2 |
| Node.JS | AWS SDK for Java v2 |
| Python  | Boto3               |
| C++     | AWS SDK for C++     |
| C#      | AWS SDK for .NET    |
| Go      | AWS SDK for Go v2   |
| Ruby    | AWS SDK for Ruby v3 |
| Kotlin  | AWS SDK for Kotlin  |
| ...     |                     |

SDK

Which SDK does AWS CLI use?

Python Boto3



# Assignment : Use SDK to Launch & terminate ec2 instance

Assuming you are using Ben user's access key configured in your terminal for AWS CLI:

1

In your terminal, install python3 and boto3

<https://boto3.amazonaws.com/v1/documentation/api/latest/guide/quickstart.html>

2

Download the following scripts into your workstation:

[https://raw.githubusercontent.com/awswithchetan/aws-solutions-architect-associate/refs/heads/main/launch\\_ec2.py](https://raw.githubusercontent.com/awswithchetan/aws-solutions-architect-associate/refs/heads/main/launch_ec2.py)

[https://raw.githubusercontent.com/awswithchetan/aws-solutions-architect-associate/refs/heads/main/terminate\\_ec2.py](https://raw.githubusercontent.com/awswithchetan/aws-solutions-architect-associate/refs/heads/main/terminate_ec2.py)

4

Launch EC2 instance by running the script (provide region code parameter):

> *python3 launch\_ec2.py <region code>*

5

Go to AWS console for your region and check if you see EC2 instance running

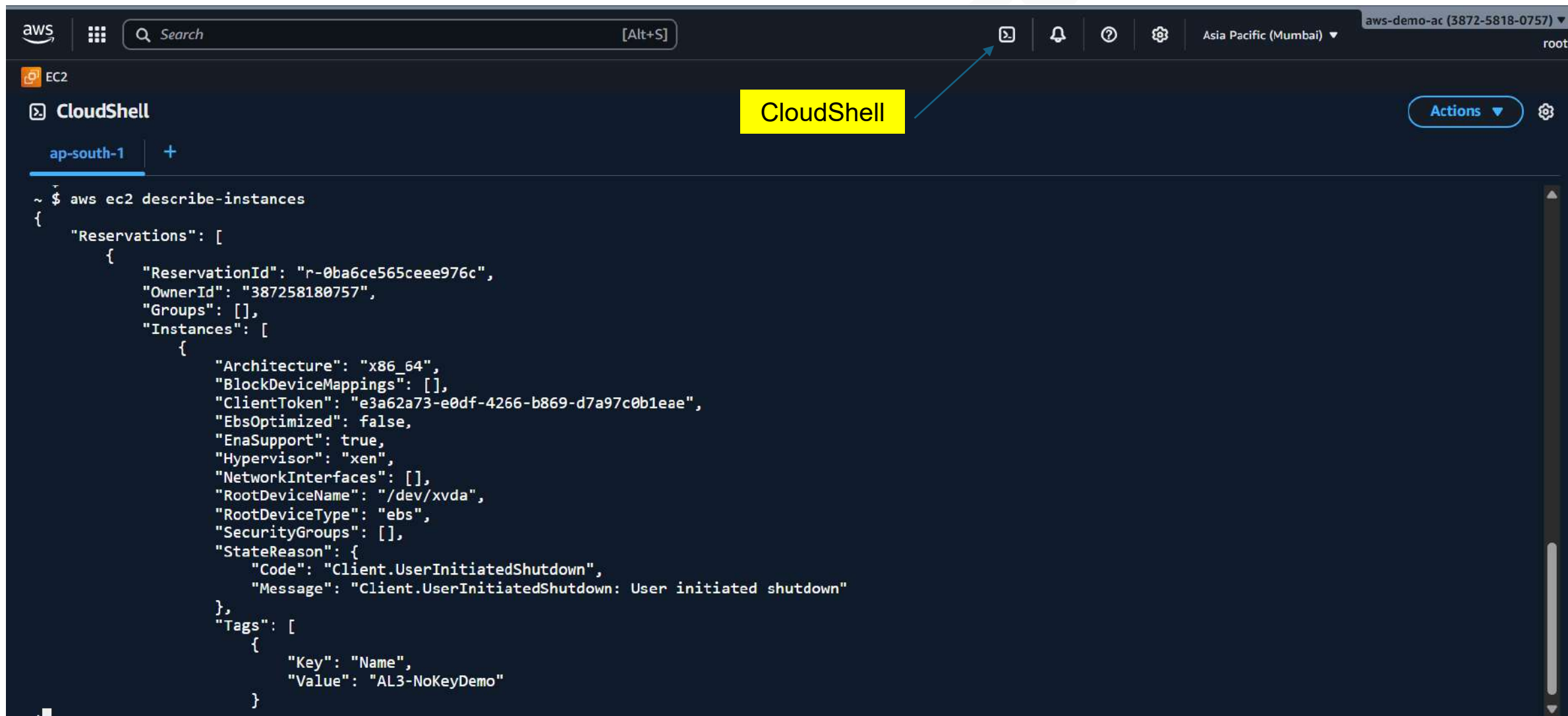
4

Terminate EC2 instance by running the script (provide region code and instance id parameters):

> *python3 terminate\_ec2.py <region code> <instance\_id>*

# AWS CloudShell

# AWS CloudShell



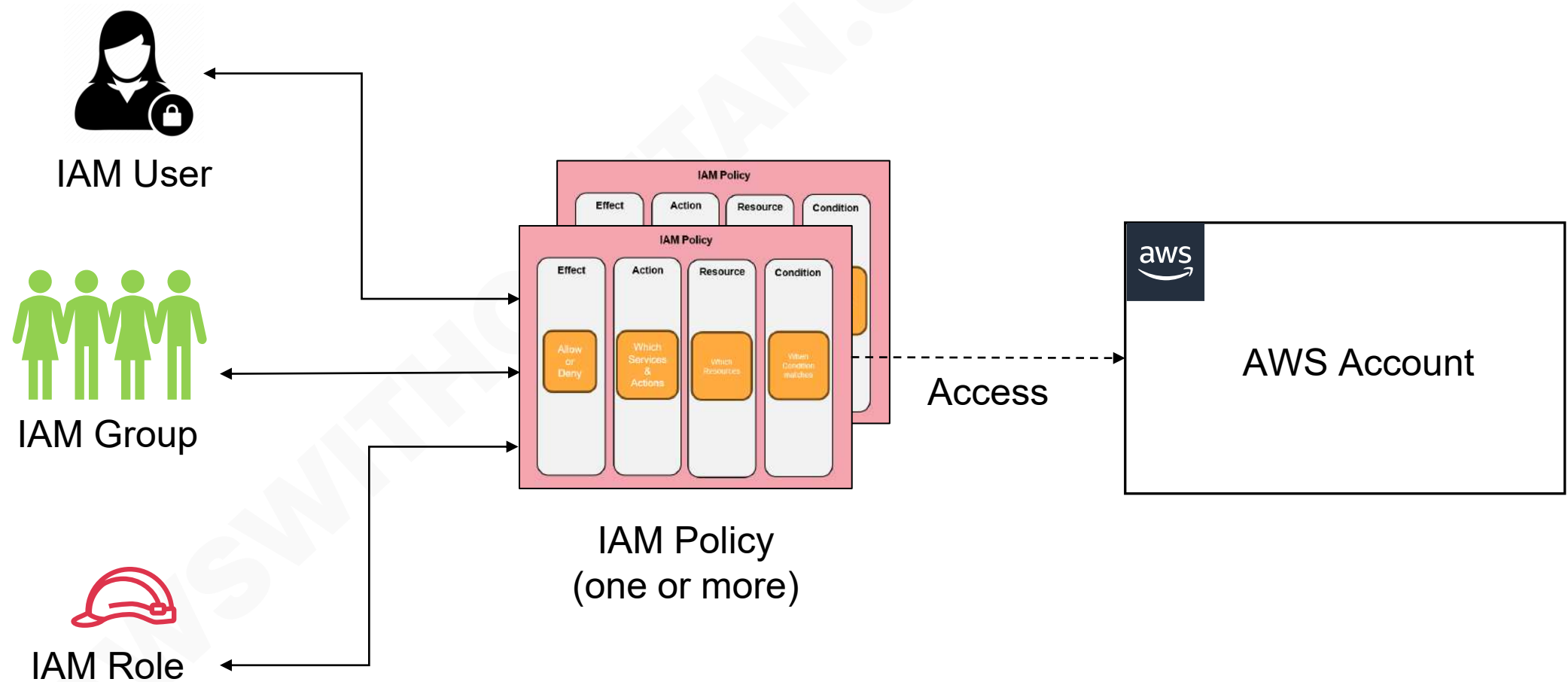
The screenshot displays the AWS CloudShell interface. At the top, there's a navigation bar with the AWS logo, a search bar, and several icons. A yellow box labeled "CloudShell" points to the CloudShell icon in the top bar. Below the navigation bar, the "CloudShell" header is visible, followed by the region "ap-south-1" and a "+" button. The main area shows a terminal window with the command `aws ec2 describe-instances` and its output. The output is a JSON object representing the details of an EC2 instance.

```
~$ aws ec2 describe-instances
{
  "Reservations": [
    {
      "ReservationId": "r-0ba6ce565ceee976c",
      "OwnerId": "387258180757",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [],
          "ClientToken": "e3a62a73-e0df-4266-b869-d7a97c0b1eae",
          "EbsOptimized": false,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "NetworkInterfaces": [],
          "RootDeviceName": "/dev/xvda",
          "RootDeviceType": "ebs",
          "SecurityGroups": [],
          "StateReason": {
            "Code": "Client.UserInitiatedShutdown",
            "Message": "Client.UserInitiatedShutdown: User initiated shutdown"
          },
          "Tags": [
            {
              "Key": "Name",
              "Value": "AL3-NoKeyDemo"
            }
          ]
        }
      ]
    }
  ]
}
```

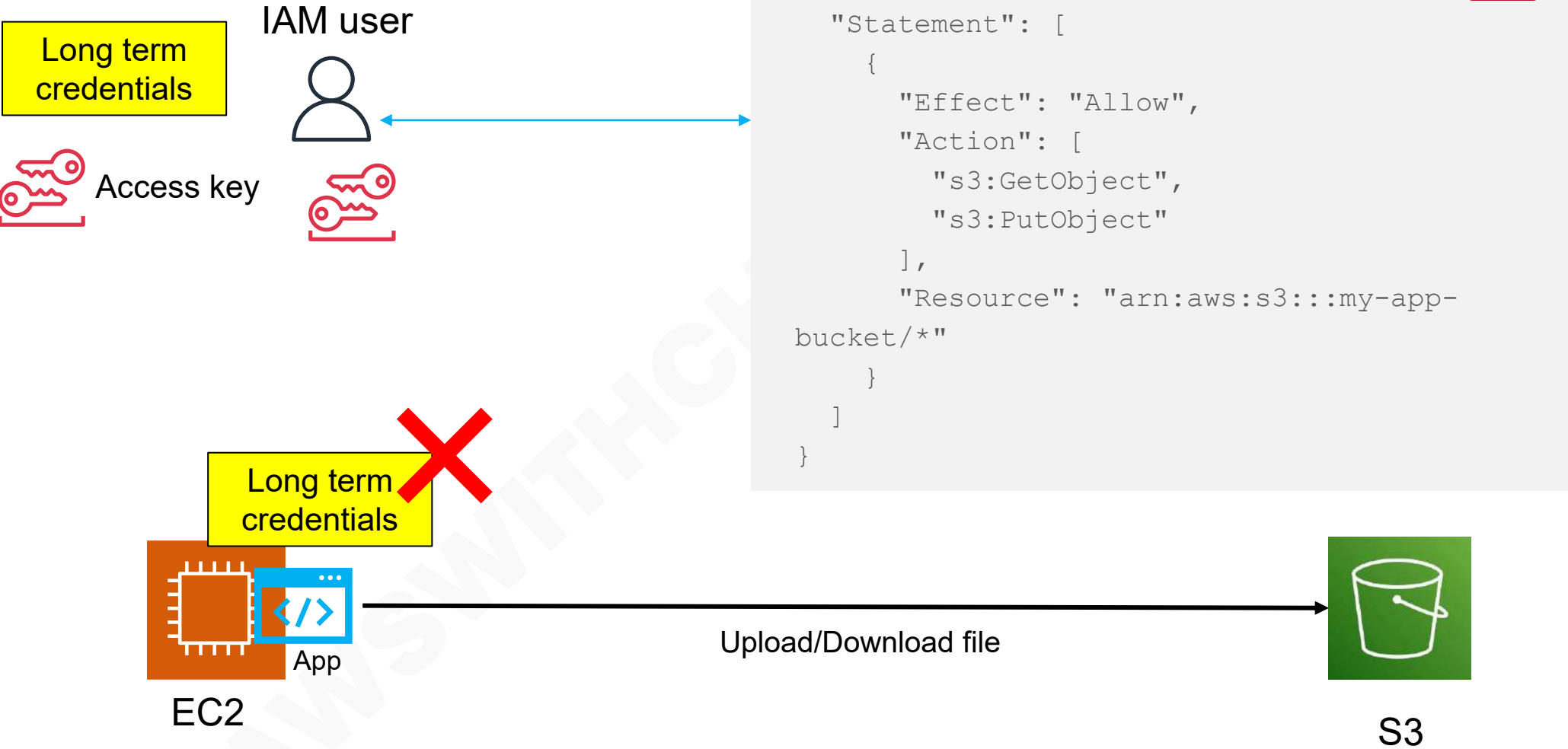
# IAM Role



# IAM User, Group and Role



# Why IAM Role?

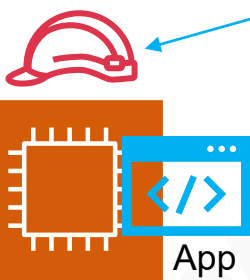


# Why IAM Role?



- Valid for short duration
- Rotated every 1 hr by default

IAM Role  
For EC2



EC2

Upload/Download file



S3

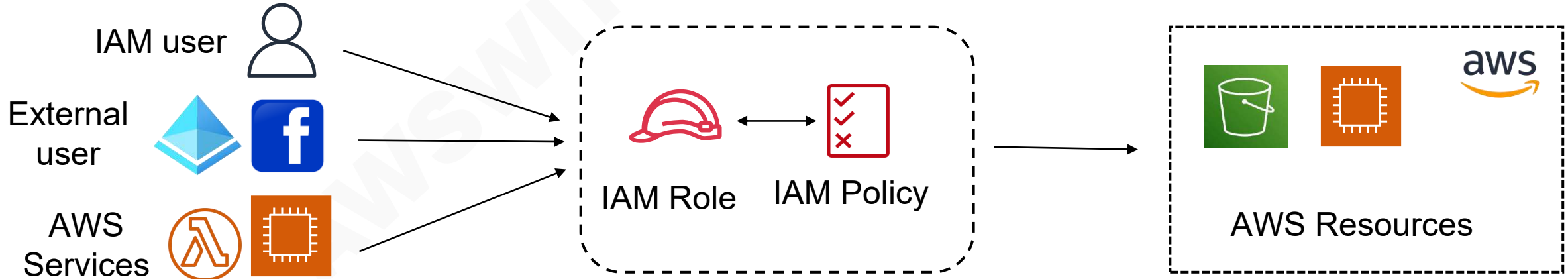
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::my-app-
bucket/*"
    }
  ]
}
```

IAM Policy

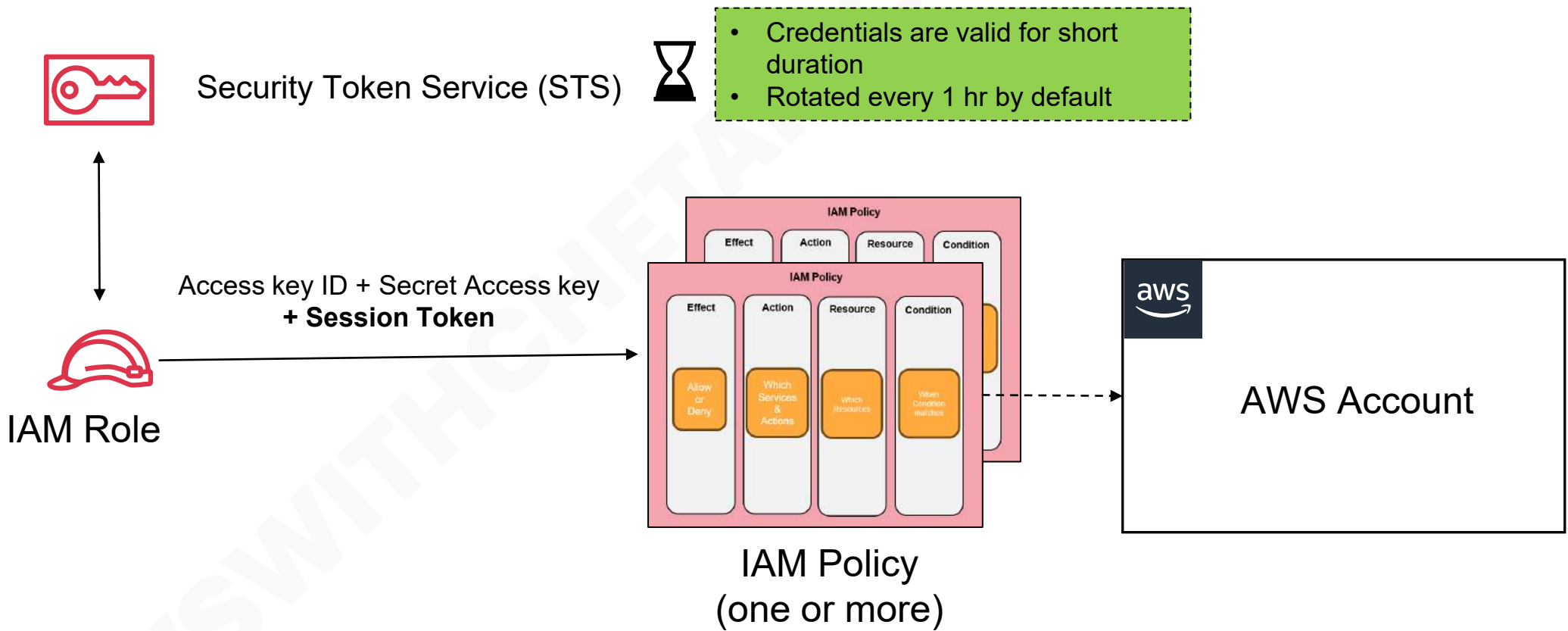


# IAM Role

- An IAM role is an AWS identity that can be assumed temporarily by users, applications, or services to securely access AWS resources without using long-term credentials.
- **IAM Role is used by:**
  - AWS service such as Amazon EC2, Lambda to access other AWS service or resource
  - An IAM user in the same or a different AWS account (e.g. for cross-account access)
  - An external user authenticated by an external identity provider (IdP) service such as Google, Facebook to get access to AWS account

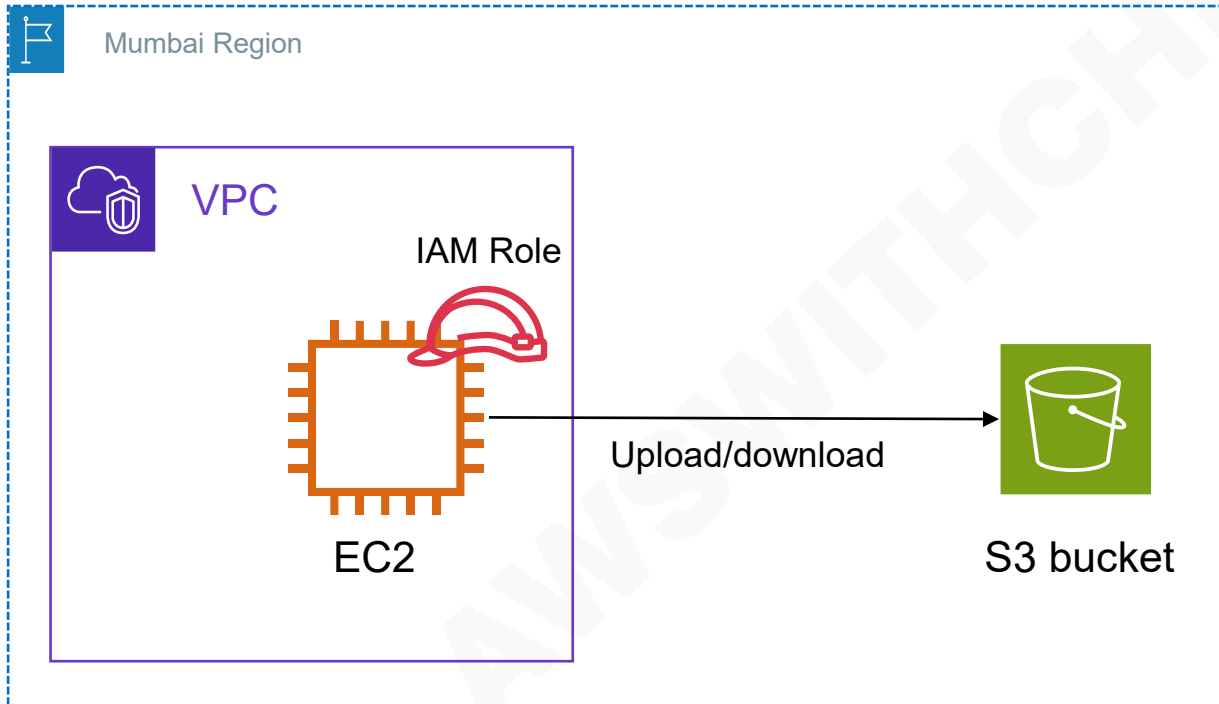


# IAM Role temporary credentials



# Exercise: Create IAM Role for EC2

We will revisit this exercise in EC2 section



## Exercise steps

- 1 Launch EC2 Linux instance in a default VPC and connect to it over SSH.
- 2 Create S3 bucket in the same region and upload some sample text or image file.
- 3 From EC2 terminal, try to download file from S3 using AWS CLI command – Access denied.
- 4 Go to AWS IAM and create IAM role for EC2. Attach S3 full permissions policy to the role.
- 5 Associate this new role to EC2 instance
- 6 Try again to download file from S3 – should be successful.
- 7 Terminate EC2 instance. Optionally delete S3 bucket.

# IAM Advanced Topics

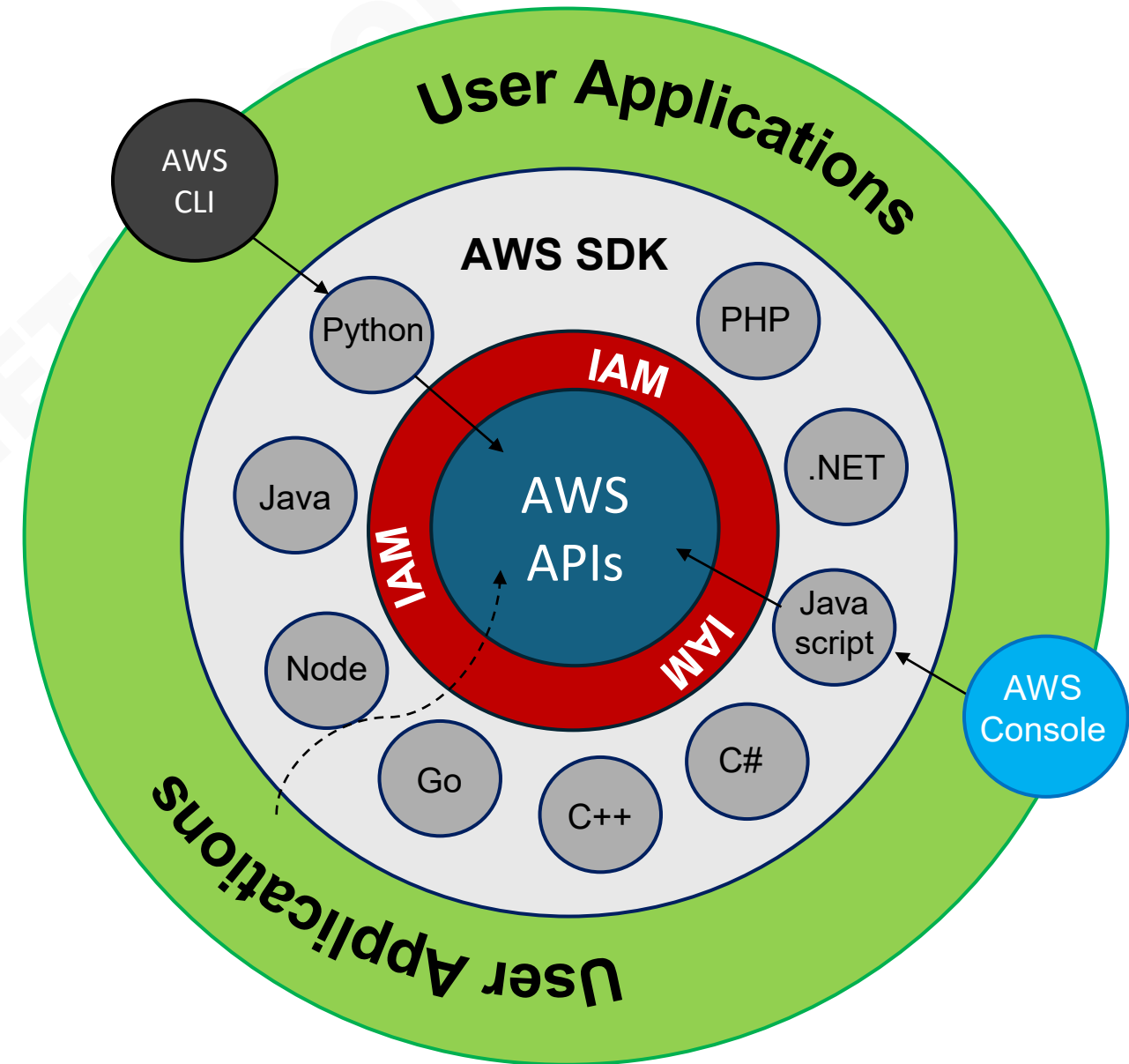
- IAM Role types – Service Account, Service-linked, Cross-account, Federated role and AWS STS service
- Hands-on exercise for cross-account IAM role
- AWS Organization and more
- AWS IAM Identity Center (successor to AWS Single Sign-On)

# AWS APIs, CLI, SDK - Putting it together



# Another way to look at it..

- AWS services are exposed through REST (HTTP) APIs
- AWS SDK wraps these AWS APIs to handle low level operations e.g. request signing, authentication, headers management. SDKs are language specific.
- Applications are typically built using corresponding language SDK
- AWS APIs are protected using IAM. User and application needs IAM permissions to invoke AWS APIs



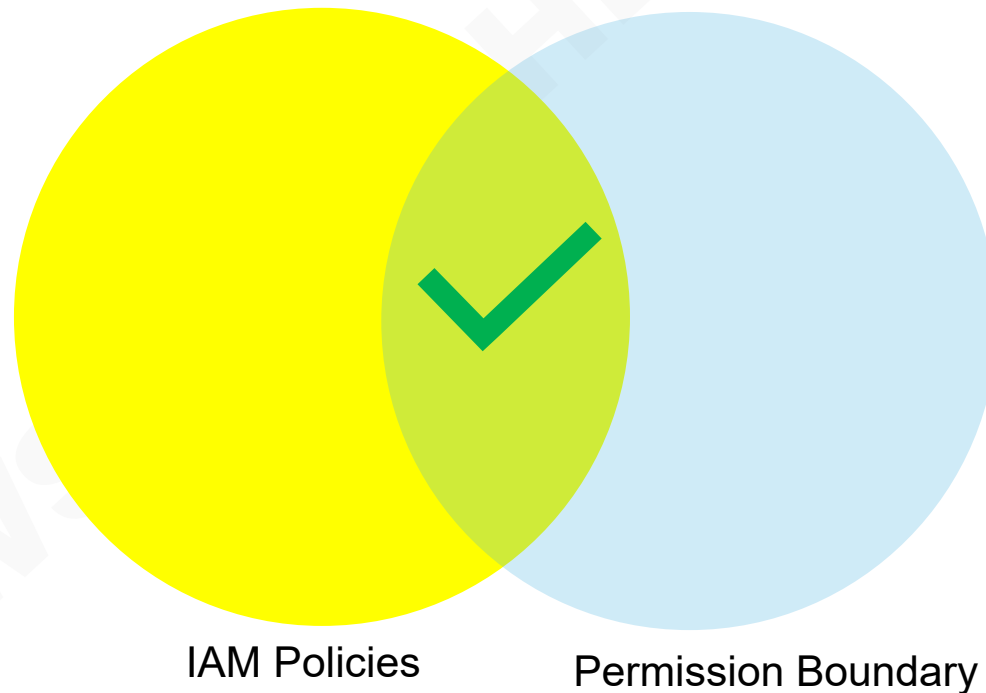
## Example:

- AWS CLI uses python (boto3) SDK
- AWS Console uses Java script SDK

# IAM Permissions Boundary

# Permissions boundary

- A permission boundary is an advanced IAM feature that sets the maximum permissions an IAM user or role can have — even if their policies allow more.
- Think of it as a “guardrail” that defines the upper limit of access.
- **It does not grant the additional permissions**



# IAM Access Advisor

## IAM Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:*",
        "cloudwatch:*",
        "ec2:*"
      ],
      "Resource": "*"
    }
  ]
}
```

## Permissions Boundary

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:*",
      ],
      "Resource": "*"
    }
  ]
}
```

# IAM Tools

# IAM Tools

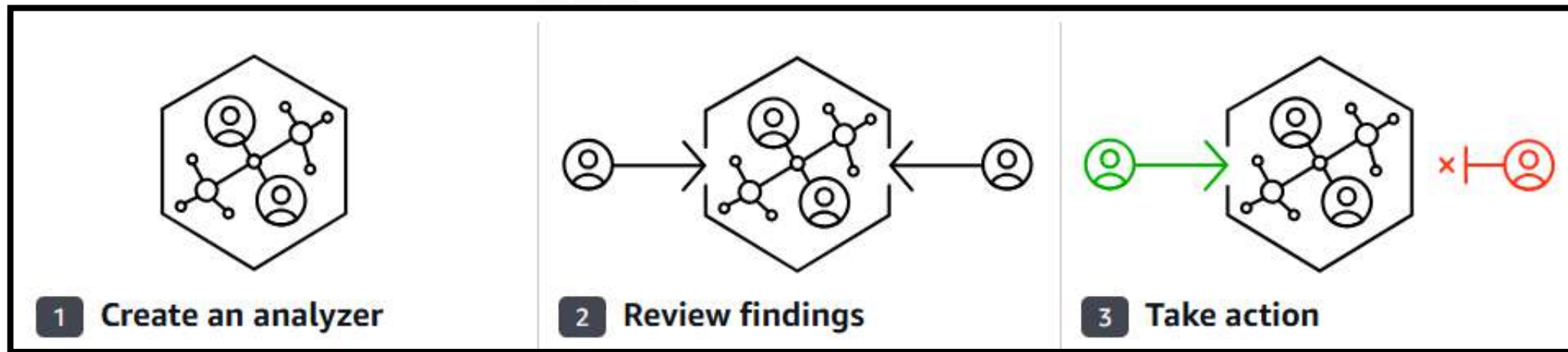
- IAM Access Analyzer
- IAM Policy Simulator
- IAM Policy Generator

AWSWITHCHETAN.COM

# IAM Access Analyzer

# IAM Access Analyzer

- A tool that continuously monitors and analyzes IAM policies (including resource-based policies like S3 buckets, KMS keys, and IAM user & roles policies)
- It tells you “who can access your resources”
- It helps detect unused access, unintended public or cross-account access.
- Helps maintain **least privilege** and compliance by detecting risky permissions.





# IAM Access Analyzer - Findings

- Resource Analysis – External Access
  - Scans resources which can be accessed by external identities (e.g. Public S3 buckets)
- Resource Analysis – Internal Access
  - Which all roles or users can access specific resource (e.g. KMS key)
- Unused Access
  - Unused roles – Roles with no access activity within the specified usage window.
  - Unused IAM user access keys and passwords – Credentials belonging to IAM users that have not been used to access your AWS account in the specified usage window.
  - Unused permissions - Permissions not used within specified usage window

For unused permission findings, IAM Access Analyzer can recommend policies to remove from an IAM user or role and provide new policies to replace existing permissions policies.

# IAM Policy Simulator

# IAM Policy simulator

- Used to test IAM policies before applying and troubleshoot *AccessDenied* issues
- Simulates if a specific action e.g. check if s3:PutObject is Allowed or Denied.
- Can test inline, managed, and resource-based policies.
- Available via AWS Console (or CLI): <https://policysim.aws.amazon.com/>

AWSWITHCHETAN.COM

# IAM Policy Generator

# IAM Policy Generator

- There are two ways to automatically generate IAM policy
  1. Create a new policy using graphical tool
  2. Create a policy based on AWS account activities by the user or a role

AWSWITHCHETAN.COM

# New policy generator

- Use this graphical tool: <https://awspolicygen.s3.amazonaws.com/policygen.html>
- Provides a web-based wizard to select Service → Actions → Resources.
- Generates valid JSON policies for users, groups, or roles.
- Supports multiple policy types – Identity based policies (IAM) and Resource based policies (S3, SNS, SQS)

AWSWITHCHETAN.COM

# Generate IAM policy based on CloudTrail activity

- IAM Access Analyzer analyzes your CloudTrail events to identify actions and services that have been used by an IAM entity (user or role)
- How it works?
  - Go to IAM User or Role -> Permissions -> Generate Policy based on CloudTrail events
  - Set up a time range up to 90 days for IAM Access Analyzer to analyze historical CloudTrail events
  - Generate Policy
  - Review and customize policy
  - Attach policy to the user or a role

AWSWITHCHETAN.COM

# IAM Audit & Reports



# AWS IAM Reports

- IAM Credentials Report**
  - Generate and download a credential report that lists all users in your account and the status of their various credentials, including passwords, access keys, and MFA devices.
  - Used for Auditing and compliance

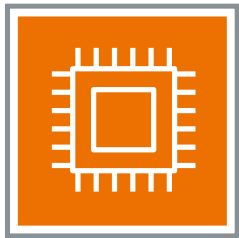
| A              | B                     | C                    | D                | E                    | F                     | G            | H          | I                   |
|----------------|-----------------------|----------------------|------------------|----------------------|-----------------------|--------------|------------|---------------------|
| user           | arn                   | user_creation_time   | password_enabled | password_last_used   | password_last_changed | password_ne  | mfa_active | access_key_1_active |
| <root_account> | arn:aws:iam::38725818 | 2017-08-09T04:59:09Z | TRUE             | 2025-10-26T11:56:22Z | 2023-03-29T13:43:27Z  | not_supporte | TRUE       | FALSE               |
| admin          | arn:aws:iam::38725818 | 2021-03-13T12:32:48Z | TRUE             | 2025-08-16T13:49:23Z | 2025-09-23T12:42:23Z  | N/A          | FALSE      | TRUE                |
| chetan         | arn:aws:iam::38725818 | 2025-07-05T16:55:47Z | TRUE             | 2025-10-27T05:13:34Z | 2025-10-26T11:58:14Z  | N/A          | FALSE      | TRUE                |
|                |                       |                      |                  |                      |                       |              |            |                     |

There are other IAM reports such as Organization activity, Service Control Policies (SCP) and Resource control policies (RCP) access reports which we will talk about in AWS Organization section

# IAM Best practices

# AWS IAM Best Practices

- Do not use Root user for day-to-day activities, instead use IAM user
- Apply strong password policy for console access
- Enable multi-factor authentication (MFA) for root user and IAM users
- Use temporary credentials with IAM roles to access AWS
- Rotate access keys regularly for use cases that require long-term credentials and keep them safe.
- Always apply **Least-privilege** permissions when creating or assigning IAM policies
- Use permissions boundaries to restrict and delegate permissions management within an account
- Use conditions in IAM policies to further restrict access e.g. Allow access when: "aws:ResourceTag/environment": "development" or "aws:RequestedRegion": "ap-south-1" etc.
- Detect public and cross-account access to resources with IAM Access Analyzer
- Use IAM Access Analyzer to validate your IAM policies to ensure secure and functional permissions
- Use IAM Access Analyzer to generate least-privilege policies based on CloudTrail access activities
- Regularly review and remove unused users, roles, permissions, policies, and credentials

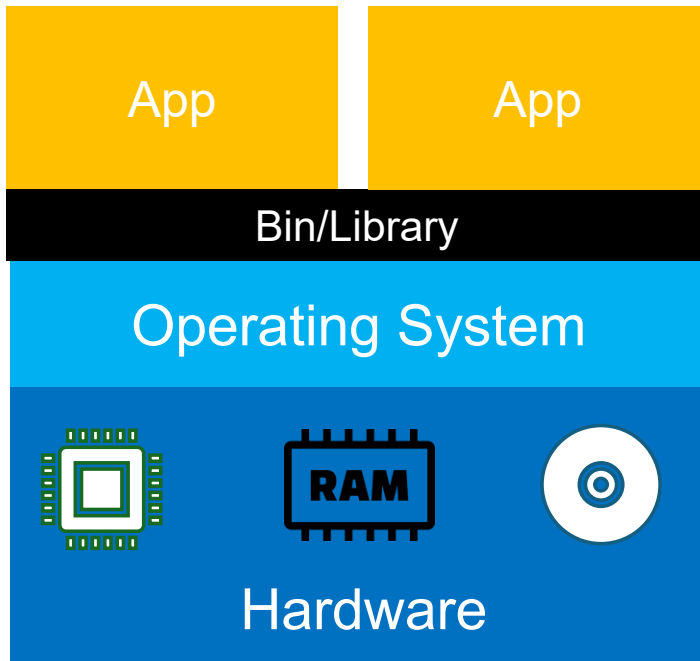


# AWS Compute services

EC2, ECS, EKS and Lambda

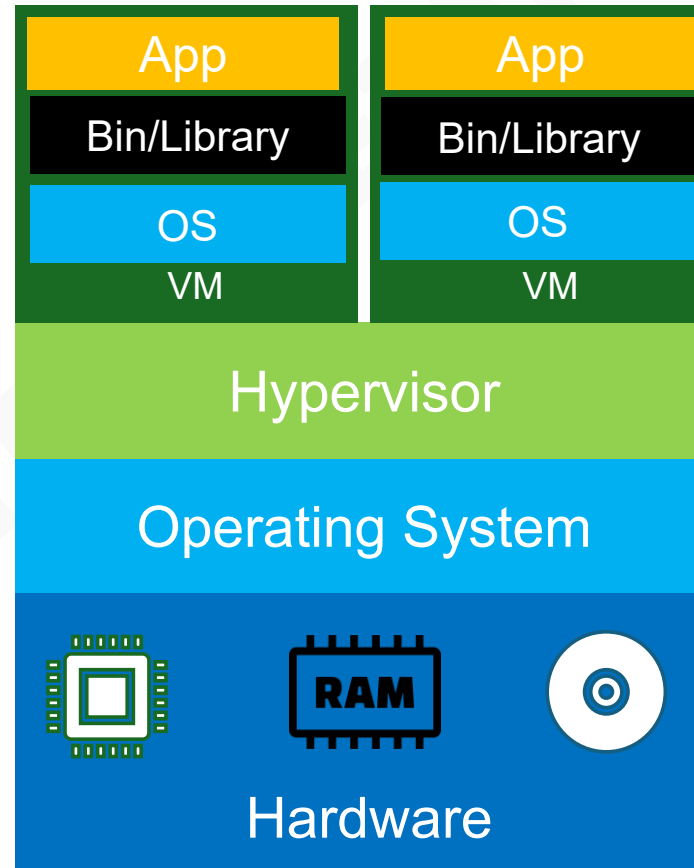
# Physical server vs Virtual Machines vs Containers

- ✓ High Performance
- ✓ Physical isolation / License compliance
- ✗ Resource contention for Apps
- ✗ Noisy neighbor problem



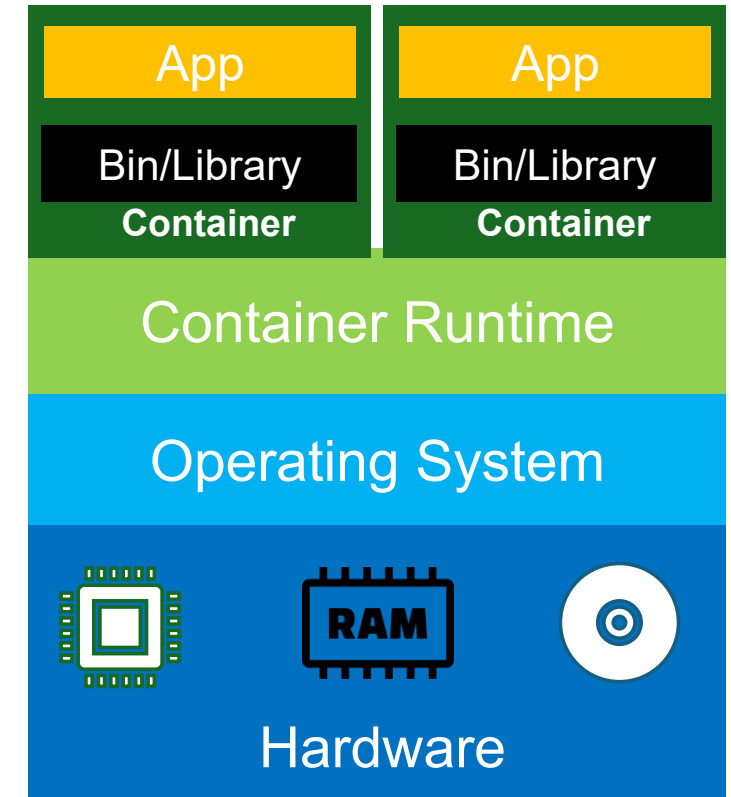
Physical Servers

- ✓ OS/Libraries flexibility
- ✓ VM level resource utilization flexibility



Virtual Machines

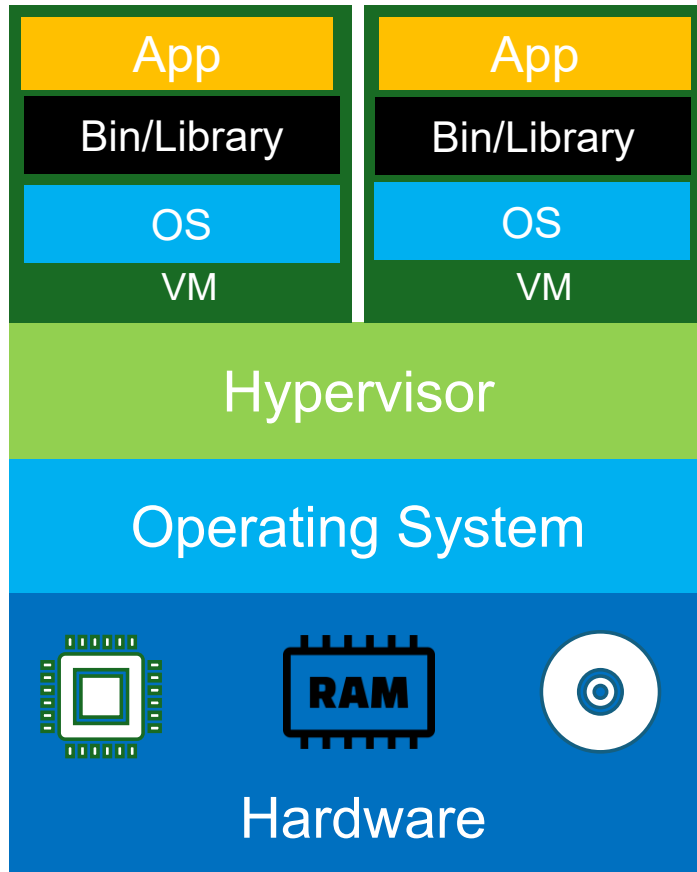
- ✓ Lightweight – shares underlying OS
- ✓ Faster to deploy
- ✓ Portability



Containers

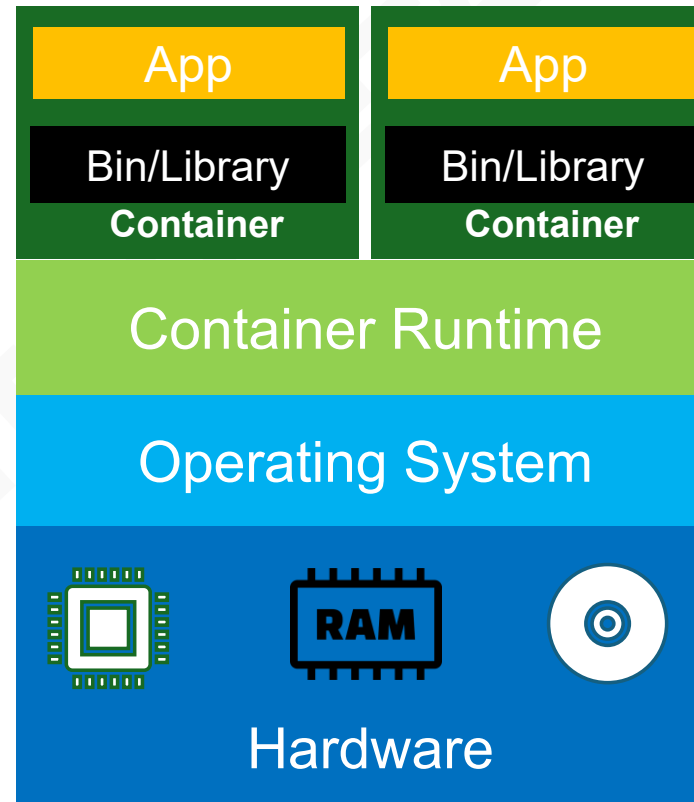
# Physical server vs Virtual Machines vs Containers

- ✓ OS/Libraries flexibility
- ✓ VM level resource utilization flexibility

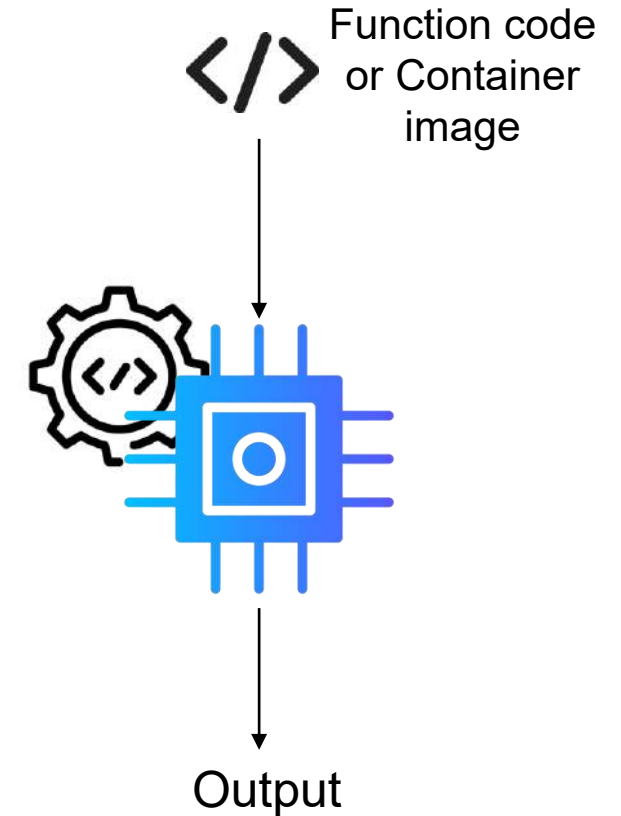


Virtual Machines

- ✓ Lightweight – shares underlying OS
- ✓ Faster to deploy
- ✓ Portability



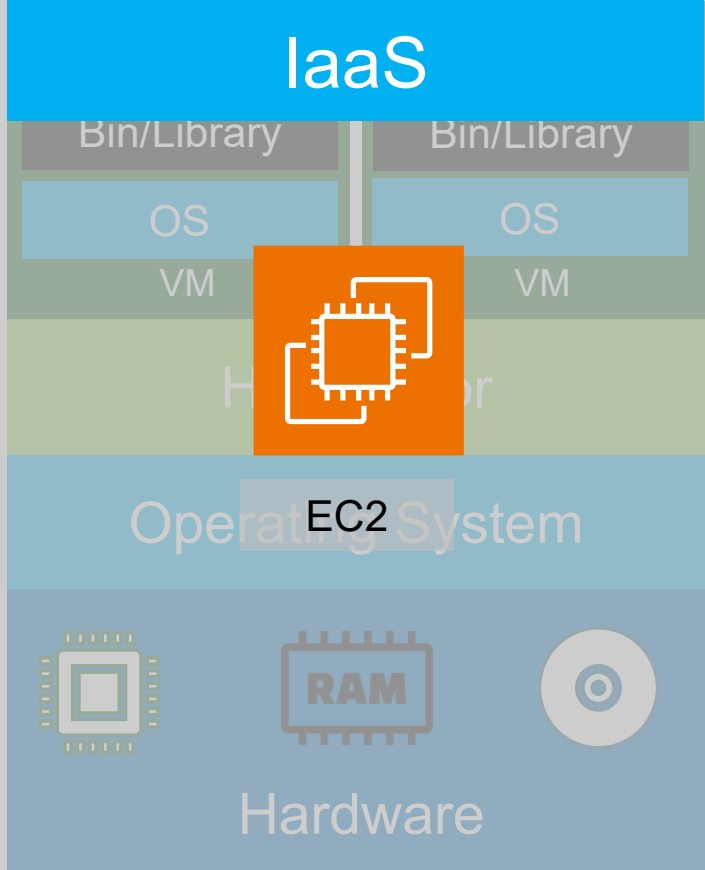
Containers



Serverless Compute

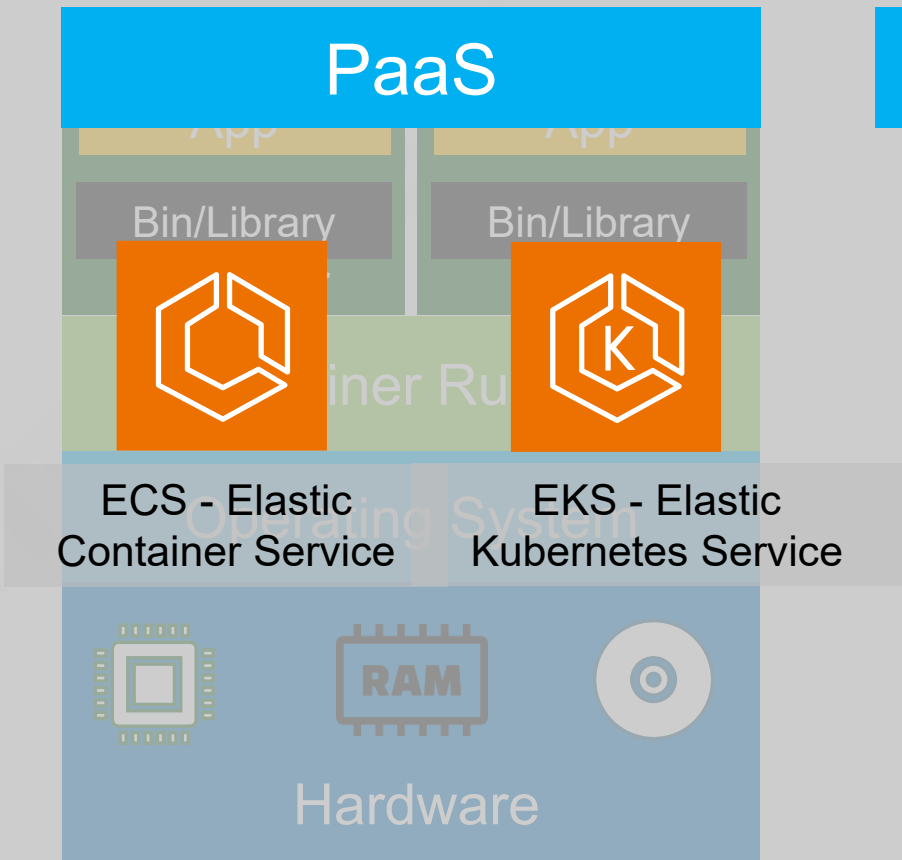
# Virtual Machines vs Containers vs Serverless

- ✓ OS/Libraries flexibility
- ✓ VM level resource utilization flexibility



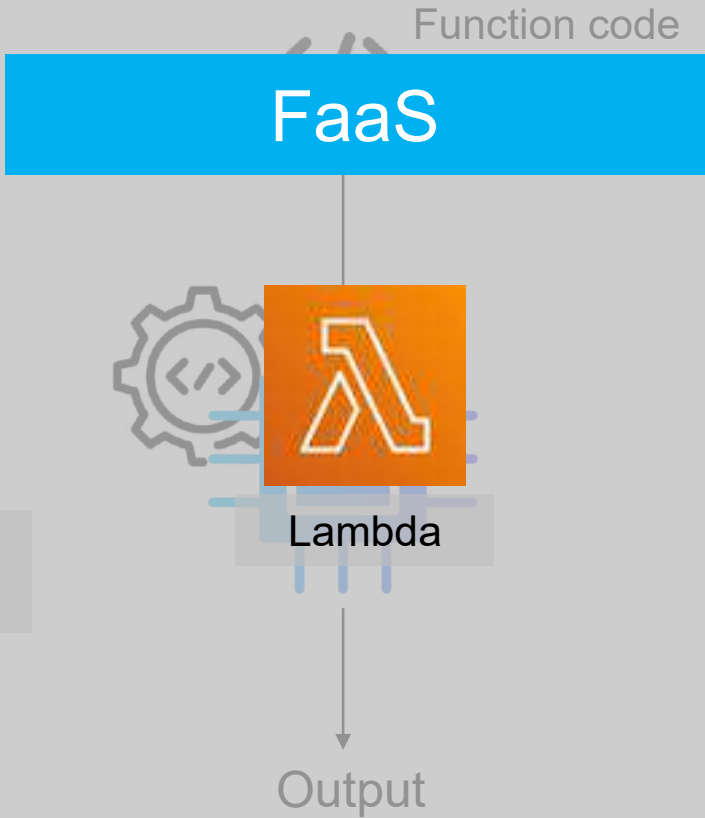
Virtual Machines

- ✓ Lightweight – shares underlying OS
- ✓ Faster to deploy
- ✓ Portability



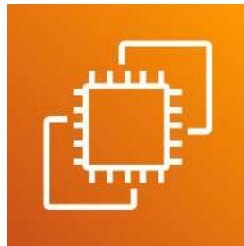
PaaS

Containers



FaaS

Serverless Compute

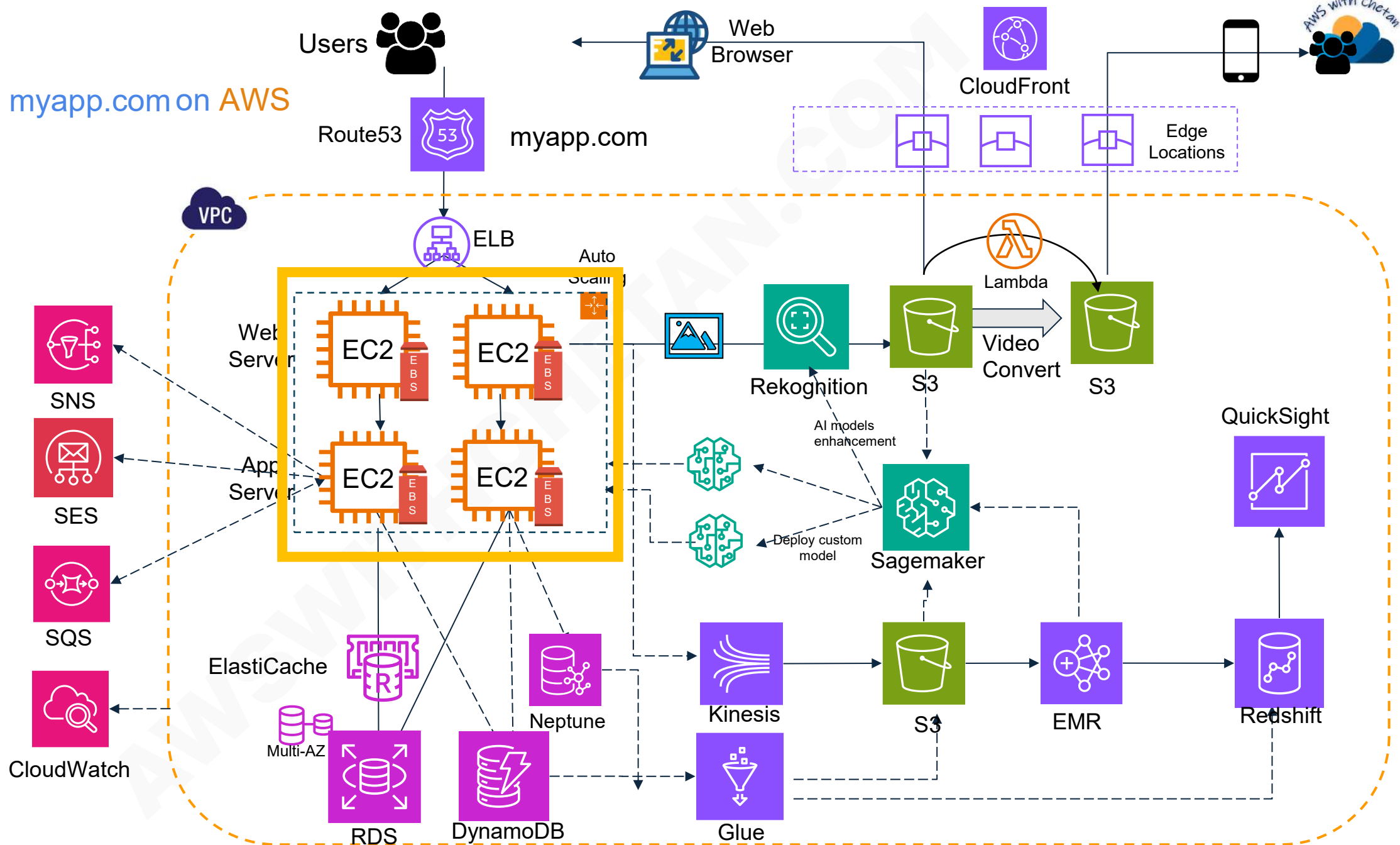


# Amazon EC2

Elastic Compute Cloud

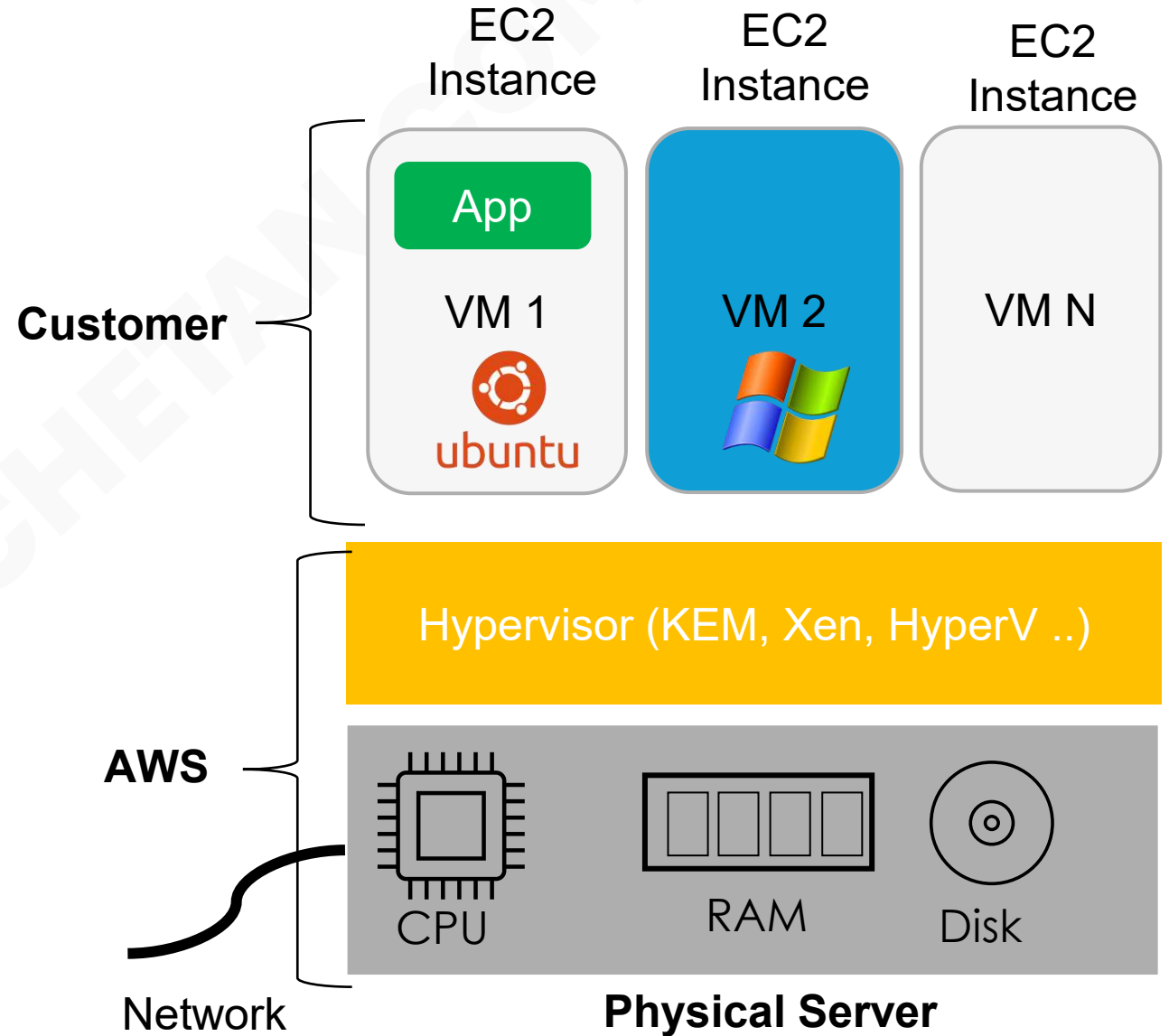


myapp.com on AWS



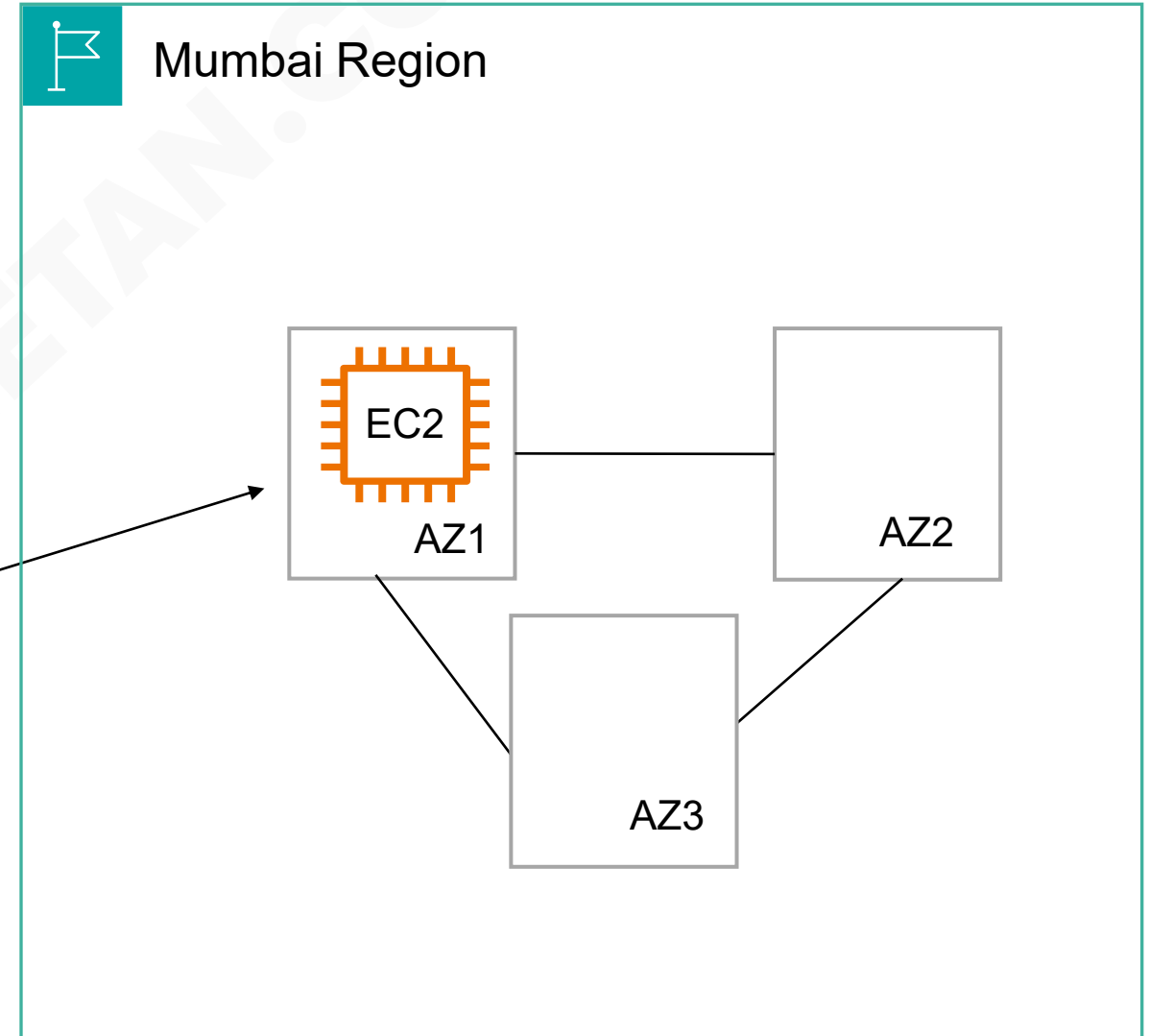
# What is EC2?

- **Elastic Compute Cloud**
- Virtual Machine in AWS Cloud
- AWS customers get access to EC2 instance
- Customer do not have access to underlying physical hardware



# Where EC2 is hosted?

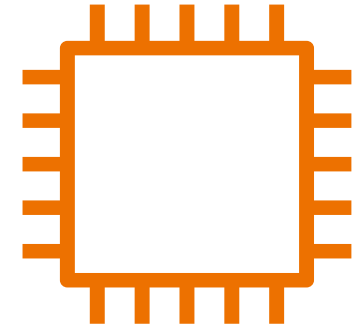
AWS Region -> Availability Zone -> EC2



# Configuration options for EC2

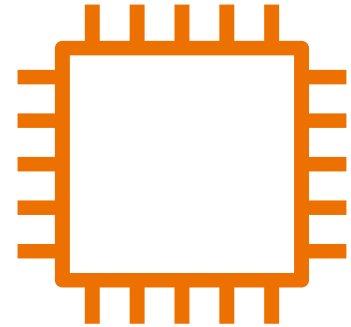
# Configuration options for EC2

- Name of the instance (Optional) - **Tag (key-value)**
- Operating System - **Amazon Machine Image (AMI)**
- CPU, Processor family, Memory - **Instance Type**
- Login credentials - **SSH key pair**
- Network, IP address (Public / Private / Elastic), AZ - **VPC**
- Storage Disk - **Elastic Block Storage (EBS)**
- Firewall - **Security Group**
- Purchasing Option - **OnDemand (default), Spot, Savings Plan, Reserved Instances**
- Hardware Tenancy - **Shared (default), Dedicated**

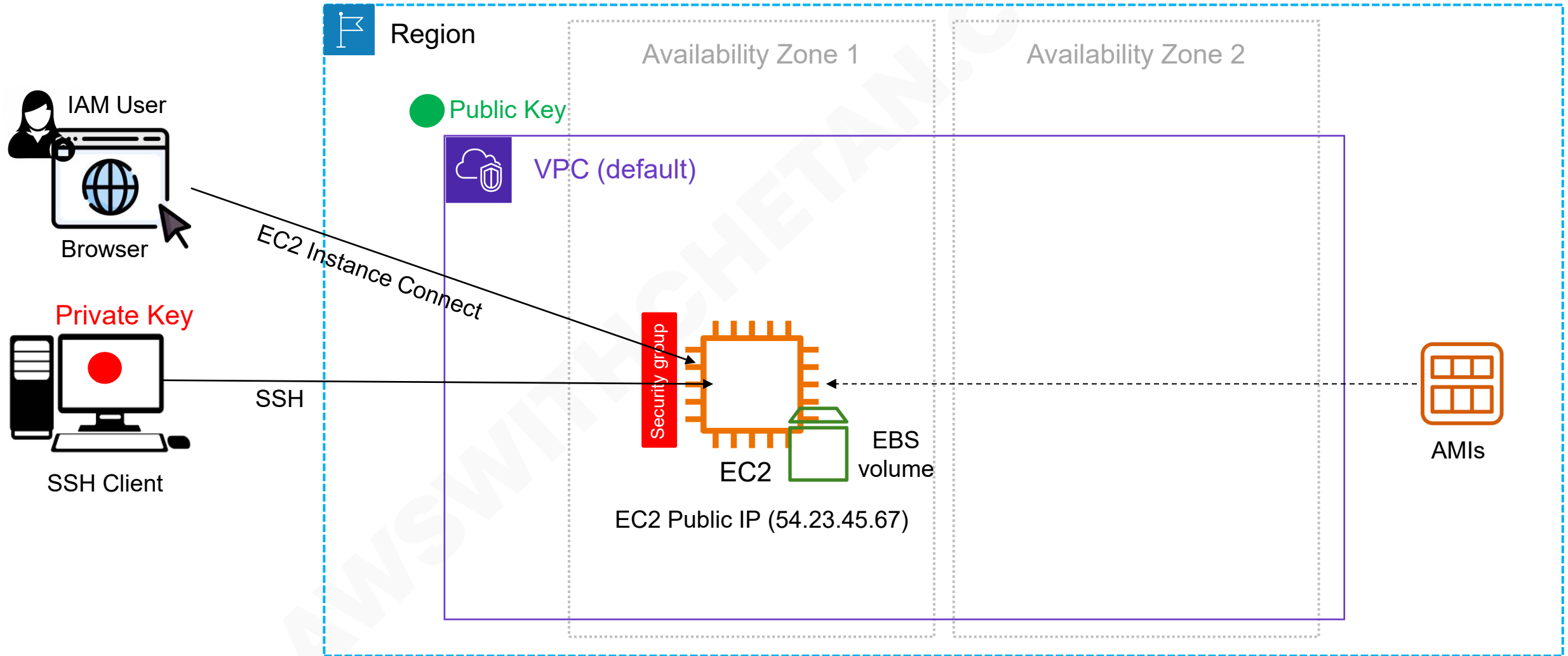


# Other configuration options

- Initialization Scripts - **User Data**
- IAM Permissions - **IAM Role**



# Process to launch & connect to EC2 instance



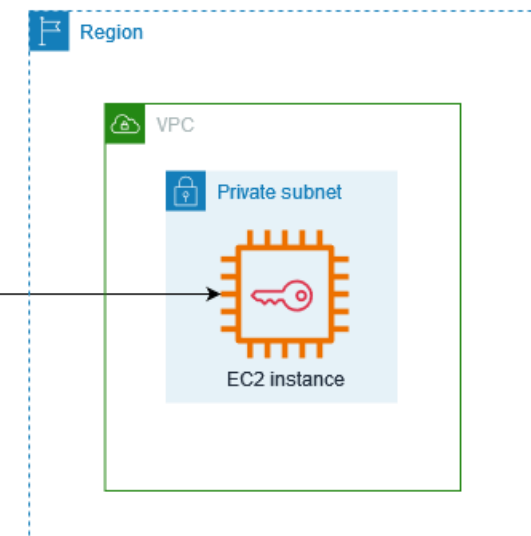
# Ways to connect to EC2 instance

1. Connect over SSH using SSH Client (e.g. Terminal or PuTTY) – For Linux EC2
2. Connect over Remote Desktop using RDP client - For Windows EC2
3. EC2 Instance Connect over the browser – For Linux EC2
4. AWS Session Manager (SSM) over the browser – For Linux and Windows EC2
5. EC2 Instance connect endpoint



# SSH Key-pair

- A key pair consists of a public key and a private key
- For Linux instances, the private key allows you to securely
- For Windows instances, the private key is required to decrypt
- **SSH Key pairs are regional**
- We can use same key-pair to launch more than one instance security credentials.



# EC2 Instance Connect

- Amazon EC2 Instance Connect provides a secure and convenient way to connect to your Linux instances over SSH
- It uses your IAM permissions to generate and use temporary SSH keys for login

## Requirements

- The instance must have the EC2 Instance Connect package installed (Amazon Linux 2, 2023 and Ubuntu 20.04+ AMIs has it by default, for other OS install the instance connect package)
- SSH Port 22 must be open in the Security Group.
- The IAM user or role must have IAM permission: `ec2-instance-connect:SendSSHPublicKey`

# SSH vs EC2 Instance Connect

## SSH

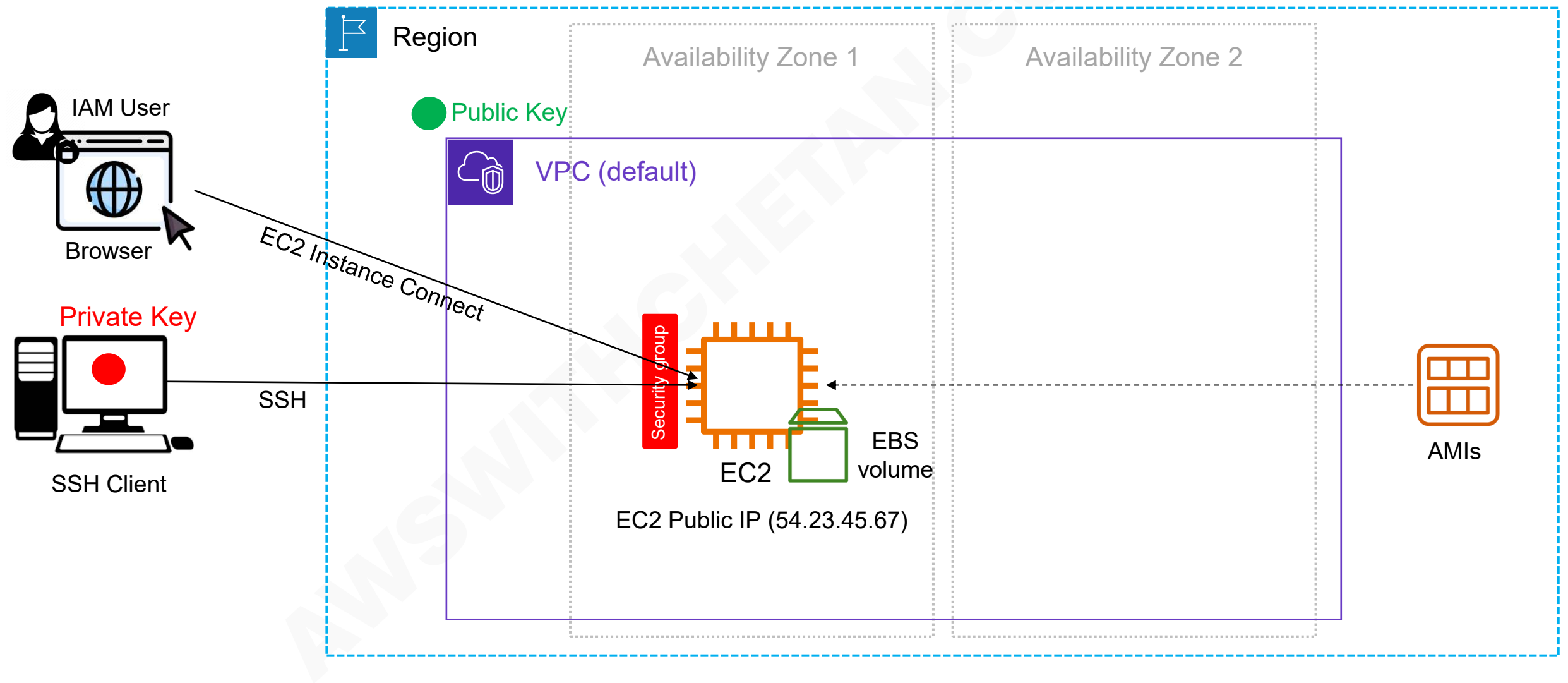
- Connect using SSH Client
- Uses permanent / Long term key pairs
- Do not need IAM permissions
- Needs manual key management

## EC2 Instance Connect

- Connect over the browser session
- Generates temporary ssh keys and uses IAM permissions to push ssh keys to EC2 instance
- Needs IAM permissions
- More secure as no key management
- Logs access in AWS CloudTrail

You can launch EC2 instance without SSH key pair. In that case, which of the above method will you use to connect to the instance?

# Exercise - Launch EC2 instance (Linux)



# Exercise : Launch EC2 instance (Linux)

1

## Launch EC2 (Linux) instance in Mumbai Region

- a) Go to EC2 Service -> EC2 Dashboard -> Launch Instances
- b) Name: MyEC2Linux
- c) Select Application and OS Images (Amazon Machine Image): Amazon Linux 2023 (default)
- d) Select instance type: t2.micro (default)
- e) Select key pair : *Your key-pair that you had created earlier in pre-requisites*
- f) Network settings -> Default VPC
- g) Make sure Auto-Assign Public IP is enabled
- h) Firewall -> Create security group
  - a) Allow SSH traffic from -> Select My IP [Type-> SSH, port Range-> 22, source type -> My IP]
  - b) Allow HTTP traffic from -> Everywhere-IPv4
- a) Configure Storage -> 8GiB, gp3 (default)
- b) Launch Instance

2

## Connect over SSH using SSH client

- a) If using windows workstation – Open Putty session, Load key file and use EC2 public IP or DNS for connection
- b) If using Mac or Linux workstation – Open terminal and use ssh command: *ssh -i <keyfile> <ec2 public IP>*
- c) Use login user as **ec2-user**

# Exercise : Launch EC2 instance (Linux)

3

## **Connect to EC2 instance using EC2 instance instance**

- a) Open the Amazon EC2 console
- b) In the navigation pane, choose Instances.
- c) Select the instance and choose Connect.
- d) Choose the EC2 Instance Connect tab.
- e) Choose Connect using a Public IP.
- f) Choose Connect to establish a connection. An in-browser terminal window opens.

4

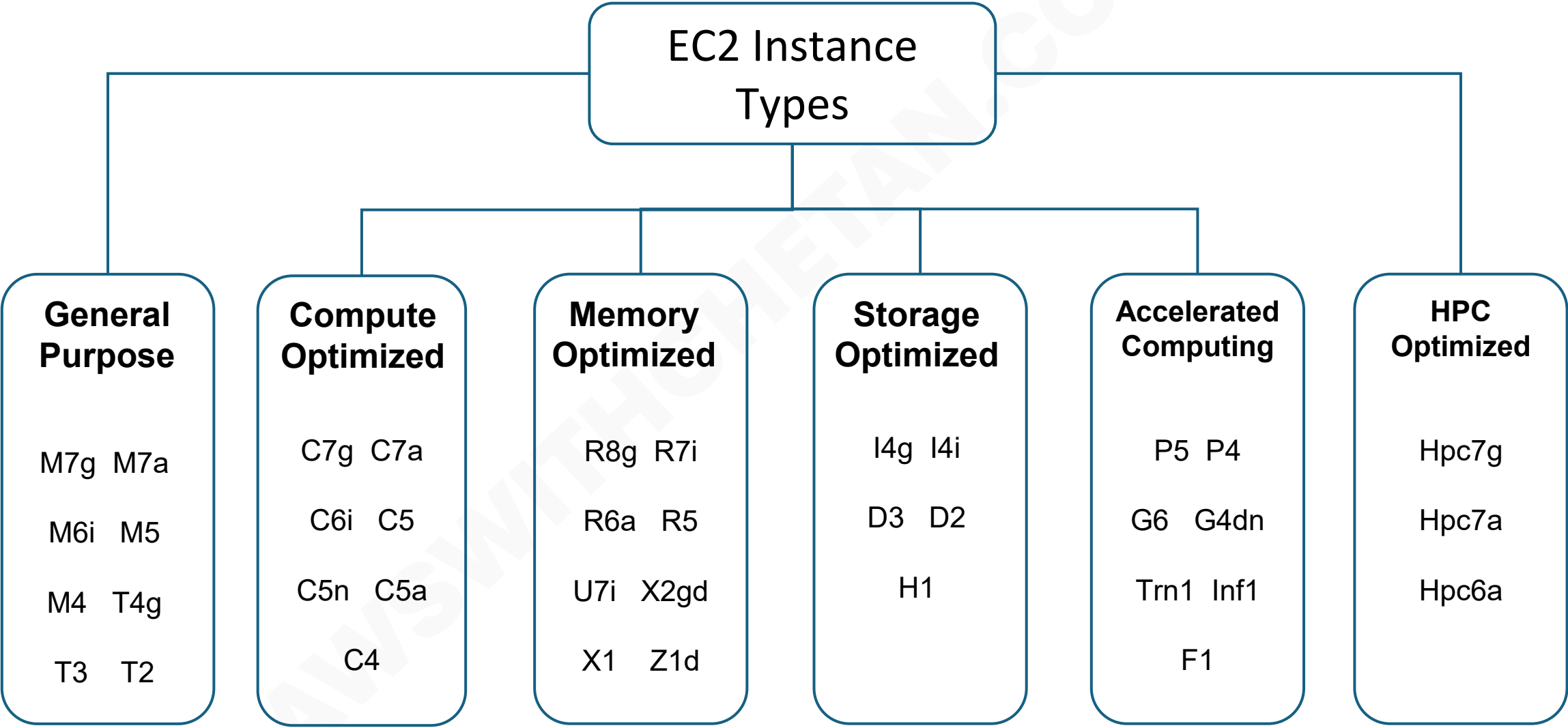
## **Terminate the instance**

- a) EC2 console -> Select your instance -> Instance State -> Terminate instance

# Assignment : Launch EC2 instance (Windows)

- 1 **Launch EC2 (Linux) instance in Mumbai Region**
  - a) Go to EC2 Service -> EC2 Dashboard -> Launch Instances
  - b) Name: MyEC2Windows
  - c) Select Application and OS Images (Windows): Microsoft Windows Server 20XX Base
  - d) Select instance type: t2.micro (default)
  - e) Select key pair : *Your key-pair that you had created earlier in pre-requisites*
  - f) Network settings -> Default VPC
  - g) Make sure Auto-Assign Public IP is enabled
  - h) Firewall -> Create security group
  - i) Allow RDP traffic from -> Select My IP [Type-> RDP, port Range-> 3389, source type -> My IP]
  - j) Configure Storage -> 8GiB, gp3 (default)
  - k) Launch Instance
- 2 **Retrieve Windows Administrator password**
  - a) Go to EC2 console -> Select your instance -> Actions -> Get Windows Password
  - b) Paste your .pem key file content (or browse and load .pem file). AWS will provide you the decrypted Windows password
- 3 **Connect over RDP**
  - a) Open RDP session using Remote Desktop Client to Public IP/DNS of EC2 Windows instance
  - b) Provide Username= Administrator, Password= <decrypted password>
- 4 **Terminate the instance**
  - a) EC2 console -> Select your instance -> Instance State -> Terminate instance

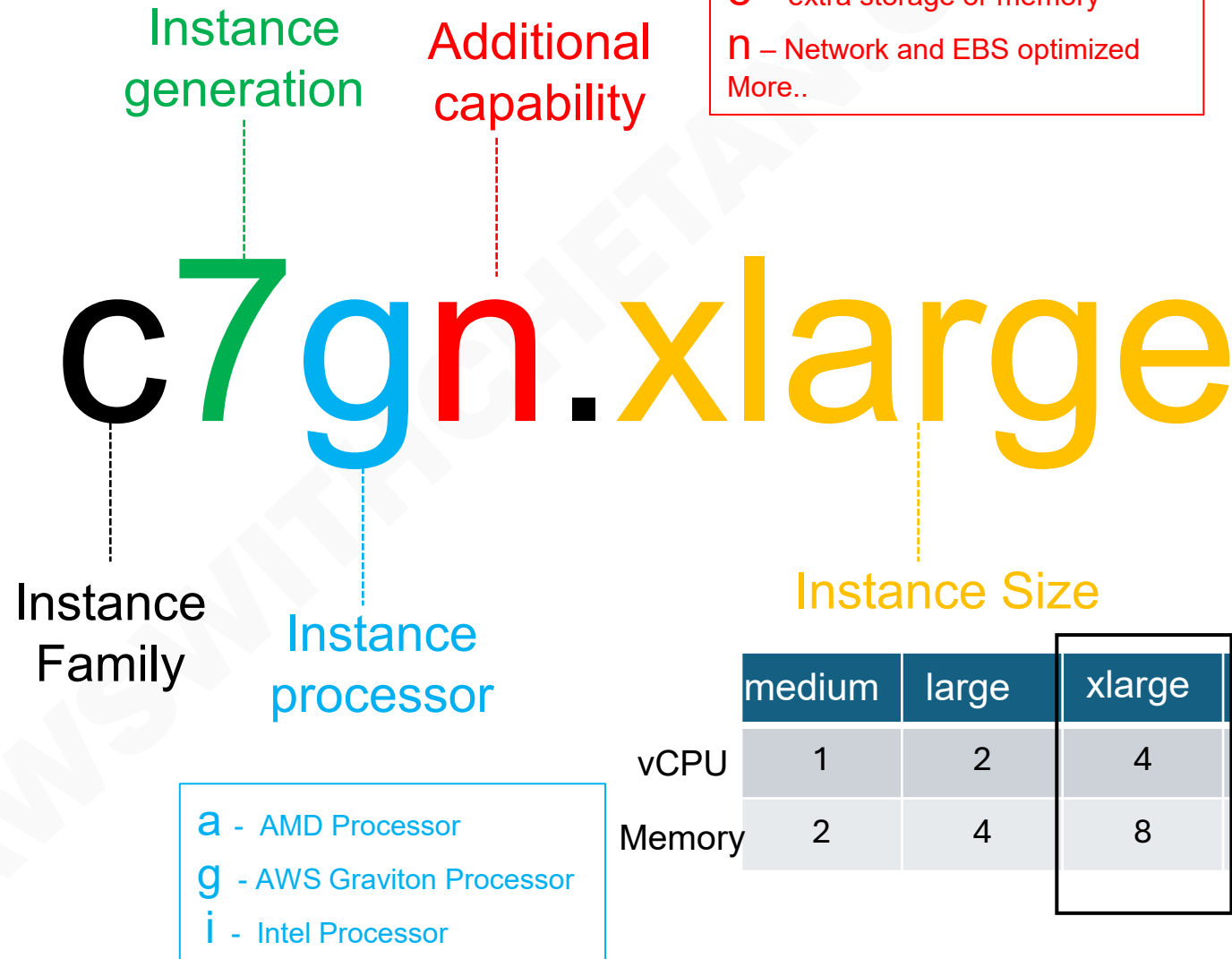
# EC2 Instance Types





# EC2 Instance naming

- C** - Compute Optimized
- D** - Dense storage
- F** - FPGA
- G** - Graphics Intensive
- I** - Storage Optimized
- Hpc** - HPC optimized
- P** - GPU accelerated
- M** - General Purpose
- T** - Burstable Performance
- U** - High Memory
- VT** - Video Transcoding
- X** - Memory intensive
- More..

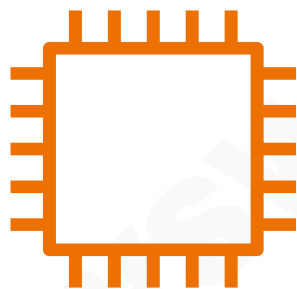


- b** – block storage optimization
- d** – instance store volume
- e** – extra storage or memory
- n** – Network and EBS optimized
- More..

|        | medium | large | xlarge | 2xlarge | 4xlarge | 16xlarge |
|--------|--------|-------|--------|---------|---------|----------|
| vCPU   | 1      | 2     | 4      | 8       | 16      | 64       |
| Memory | 2      | 4     | 8      | 16      | 32      | 128      |

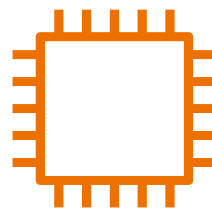
# Choosing right EC2 instance type

- **Identify workload type** - Understand whether it's general purpose, compute-heavy, memory-heavy, or storage-intensive.
- **Resource requirements** - Estimate vCPUs, memory, storage size, and network throughput your application needs.
- **Pricing** – Use [AWS Pricing calculator](#) to check and compare instance pricing
- Start small and scale as required and think of horizontal scaling

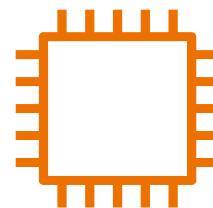


m6.2xlarge

=



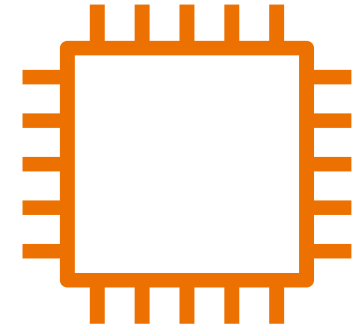
m6.xlarge



m6.xlarge

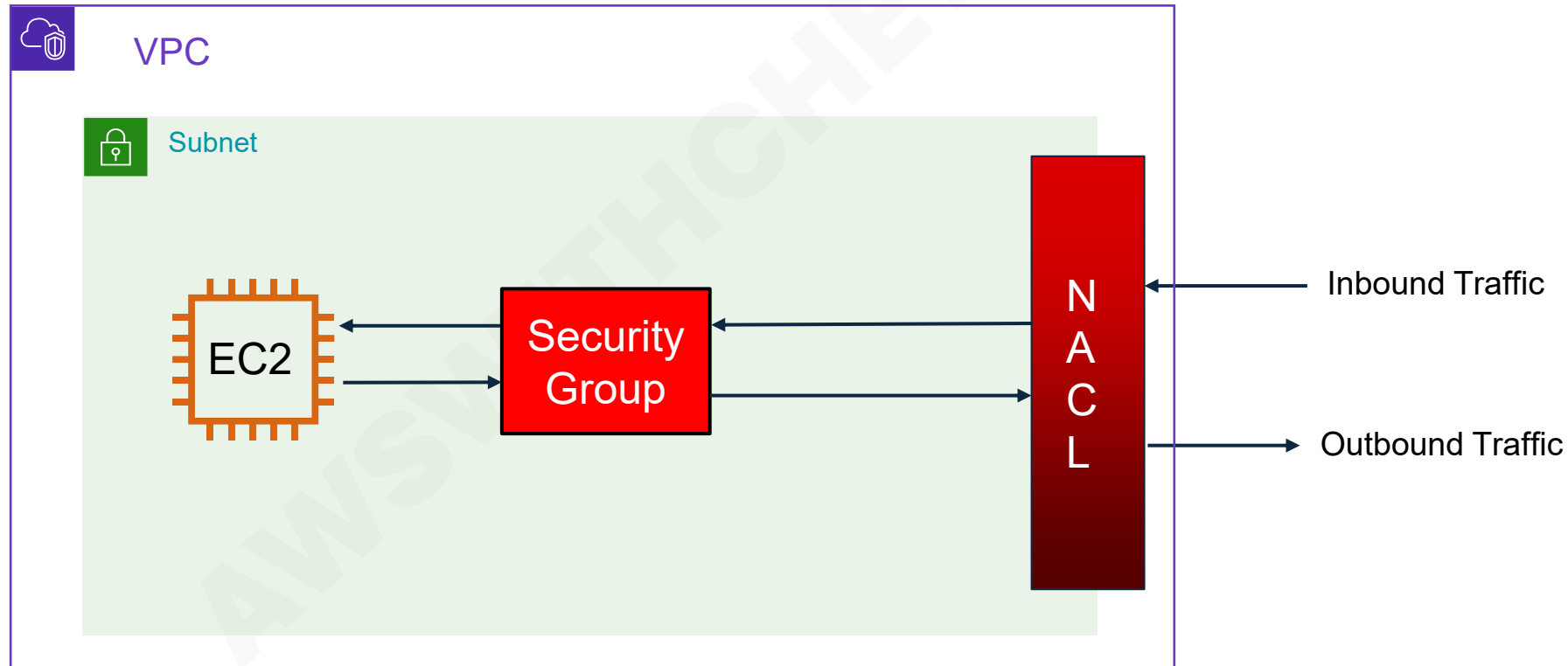
# Configuration options for EC2

- CPU, Processor family, Memory - **Instance Type**
- Firewall - **Security Group**
- Storage Disk - **Elastic Block Storage (EBS)**
- Operating System - **Amazon Machine Image (AMI)**
- Login credentials - **SSH key pair**
- Network, IP address (Public / Private / Elastic), AZ - **VPC**
- Initialization Scripts - **User Data**
- IAM Permissions - **IAM Role**
- Purchasing Option - **OnDemand (default), Spot, Savings Plan, Reserved Instances**
- Hardware Tenancy - **Shared (default), Dedicated**
- Name of the instance (Optional) - **Tag (key-value)**



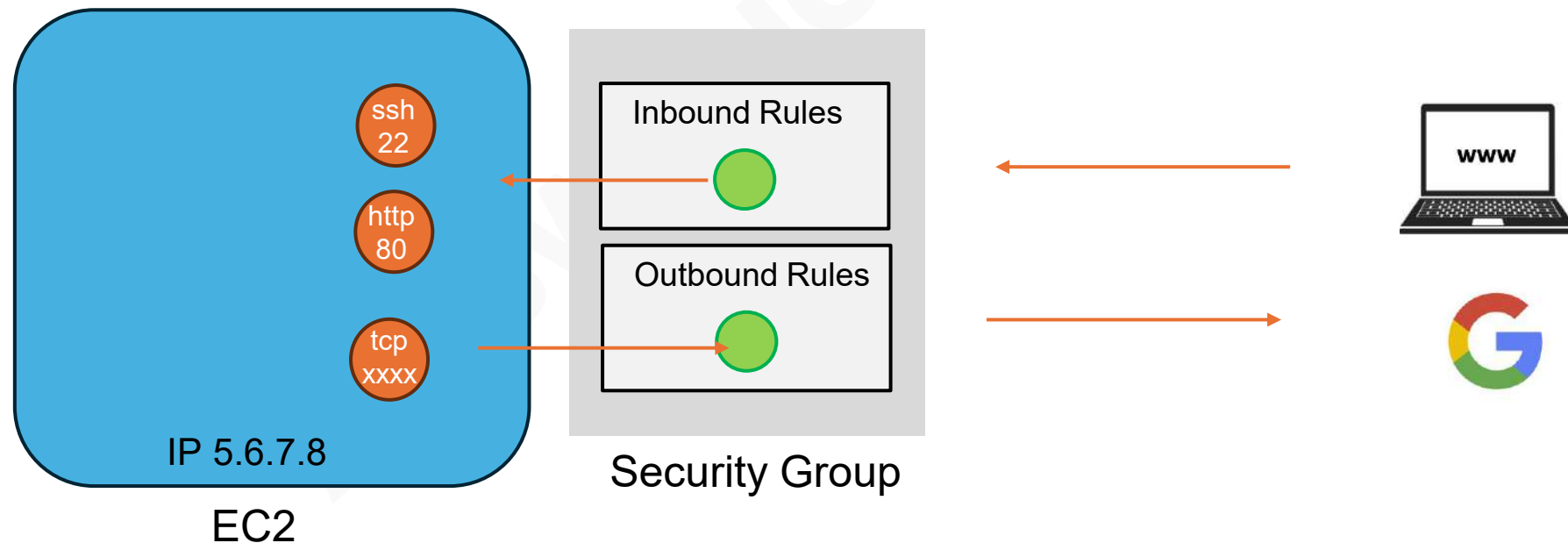
# Security Group - Firewall for EC2 instance

- Security Groups
- Network Access Control List (NACL)



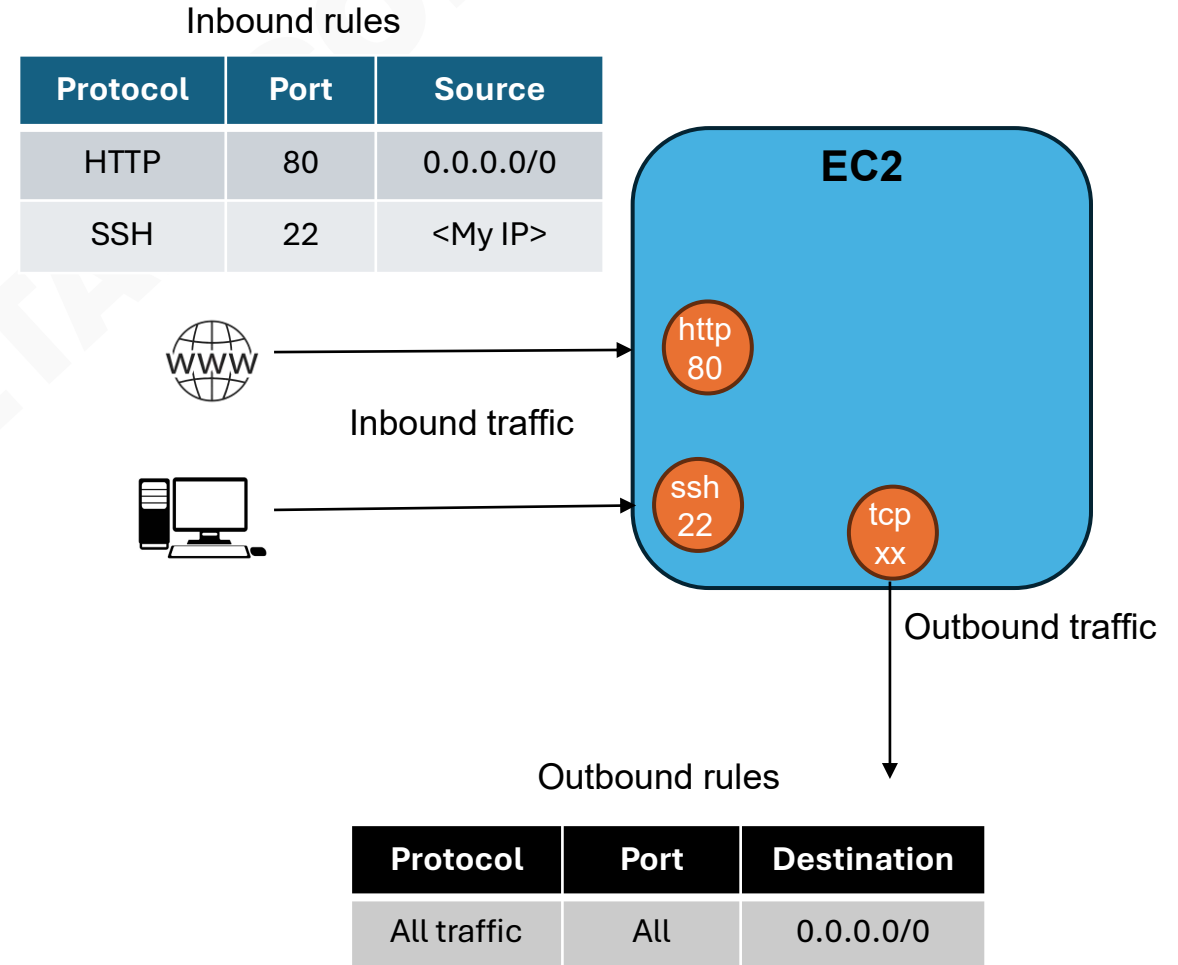
# Security Group

- Security Groups are most basic, native and important firewall for EC2 instances
- Security group has Inbound and Outbound rules
- Security group has only ALLOW rules. Does not support DENY/Block rules.
- Default Security group in each VPC
- Authorises traffic for both IPv4 and IPv6 traffic
- Security groups are stateful – return traffic is automatically allowed



# Typical ports – good to know

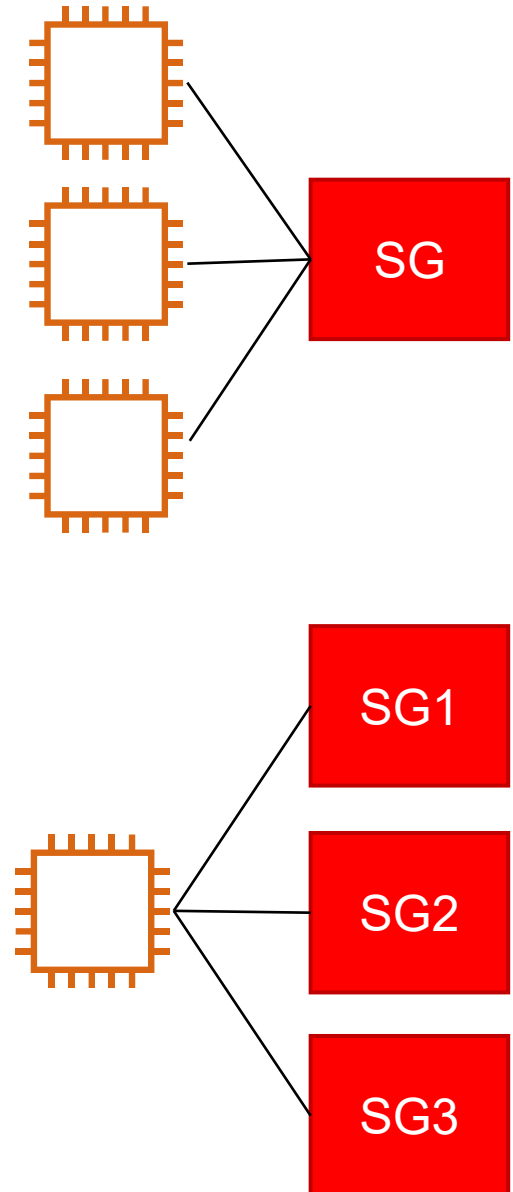
- **22** = SSH (Secure Shell) for logging into a Linux server instance
- **3389** = RDP (Remote Desktop Protocol) for logging into a Windows server instance
- **21** = FTP (File Transfer Protocol) for uploading files to a file share
- **22** = SFTP (Secure File Transfer Protocol) for uploading files securely (over SSH)
- **80** = HTTP for accessing websites or web applications
- **443** = HTTPS for TLS secured websites



# Security Groups - Summary

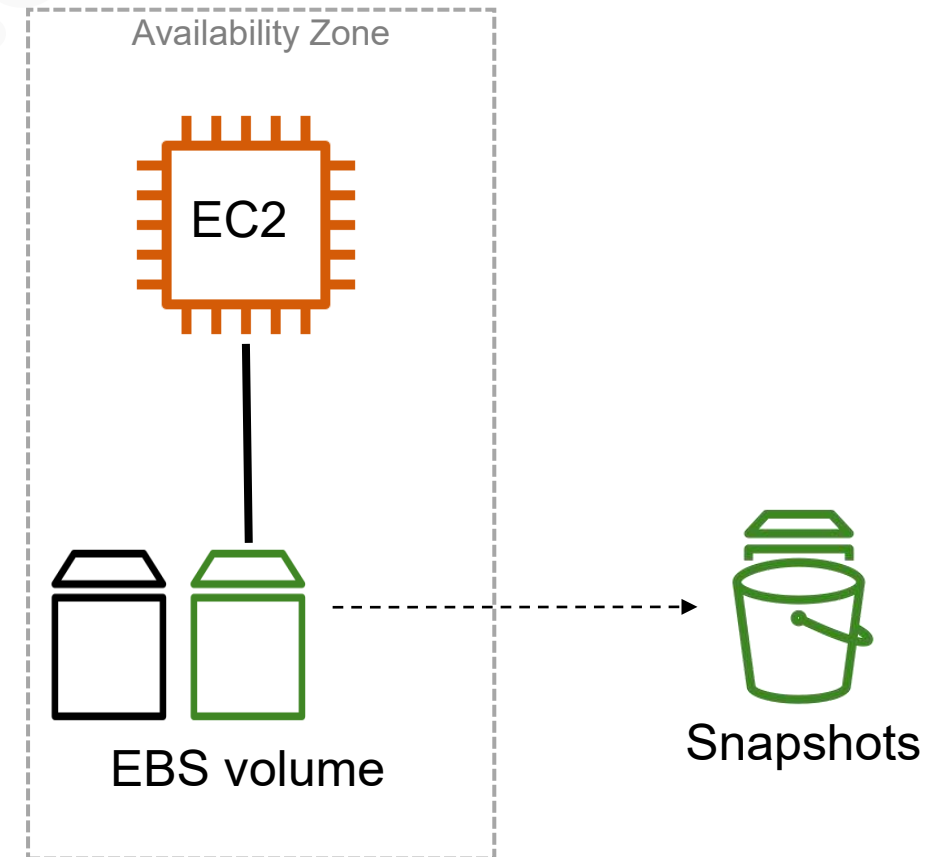
- Security group rules enable you to filter traffic based on protocols and port numbers.
- Single Security Group can be attached to multiple instances
- Single instance can have up to 16 Security groups with maximum 1000 rules\*
- All inbound traffic is blocked by default
- By default, security groups contain outbound rules that allow all outbound traffic.
- Authorises traffic for both IPv4 and IPv6 traffic
- Security groups are stateful - if you send a request from your instance, the response traffic for that request is allowed to flow in regardless of inbound security group rules.
- You can add and remove rules at any time. Your changes are automatically applied to the instances that are associated with the security group.

*\*limits may change in the future*



# Storage for EC2 = EBS

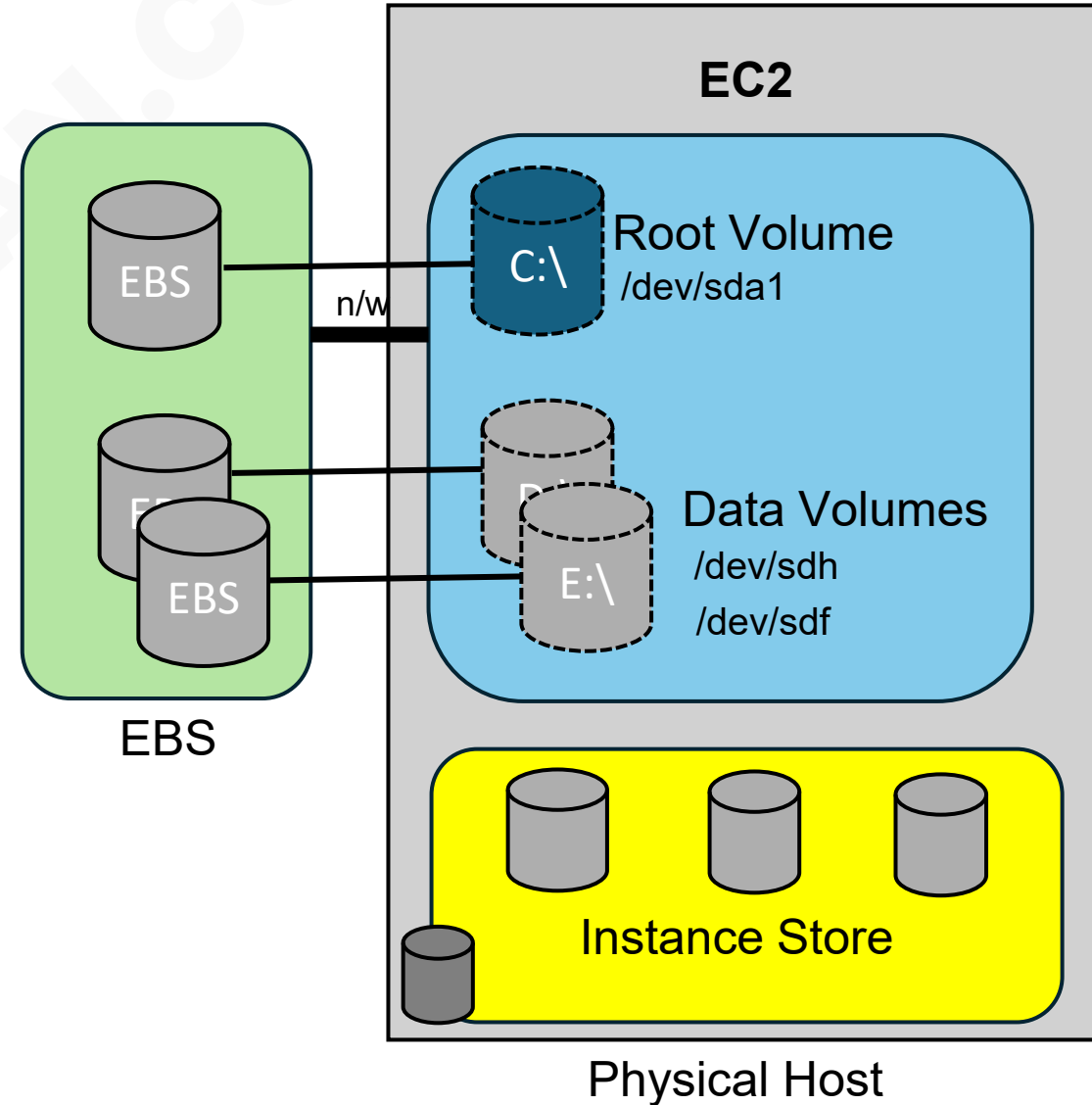
- EBS is **E**lastic **B**lock **S**torage service which provides persistent block storage for EC2 instance
- EBS is a Storage Attached Network (SAN) and hence volumes can be persisted irrespective of EC2 lifecycle
- We can create one or more EBS volumes (disk) and attach to EC2 instance
- EC2 has a root volumes (contains operating system) and optionally can have one or more data volumes
- EBS volumes are elastic – we can increase the size of EBS volumes as required
- EBS volume data can be backed up using Snapshots



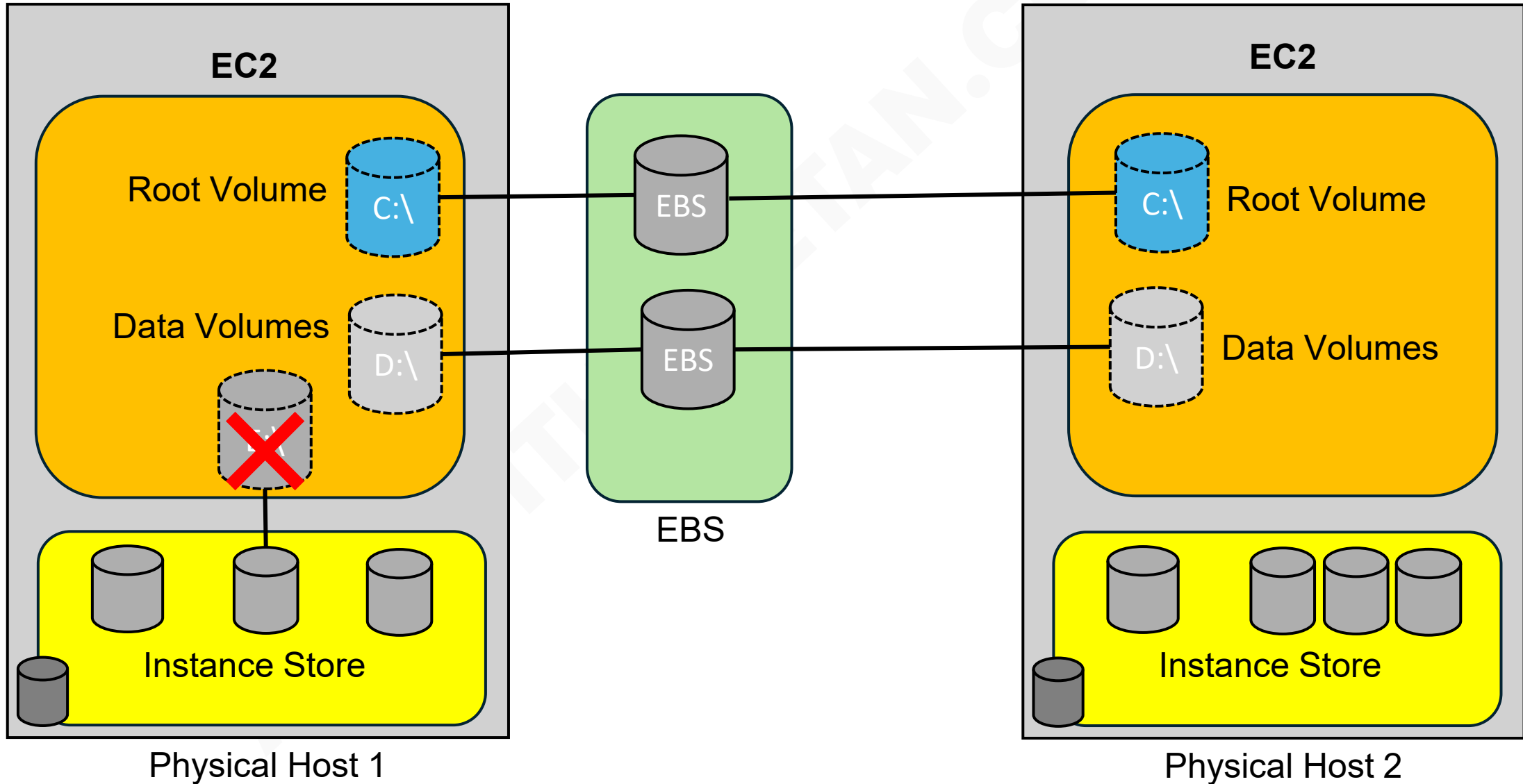


# Instance store

- EC2 disks can be of 2 types:
  - Elastic Block Storage (external to EC2 host)
  - Instance Store (optional) – Local to EC2 host
- EBS volume data is persisted until volume is deleted
- Instance Store volume data is wiped out if EC2 is stopped and started.

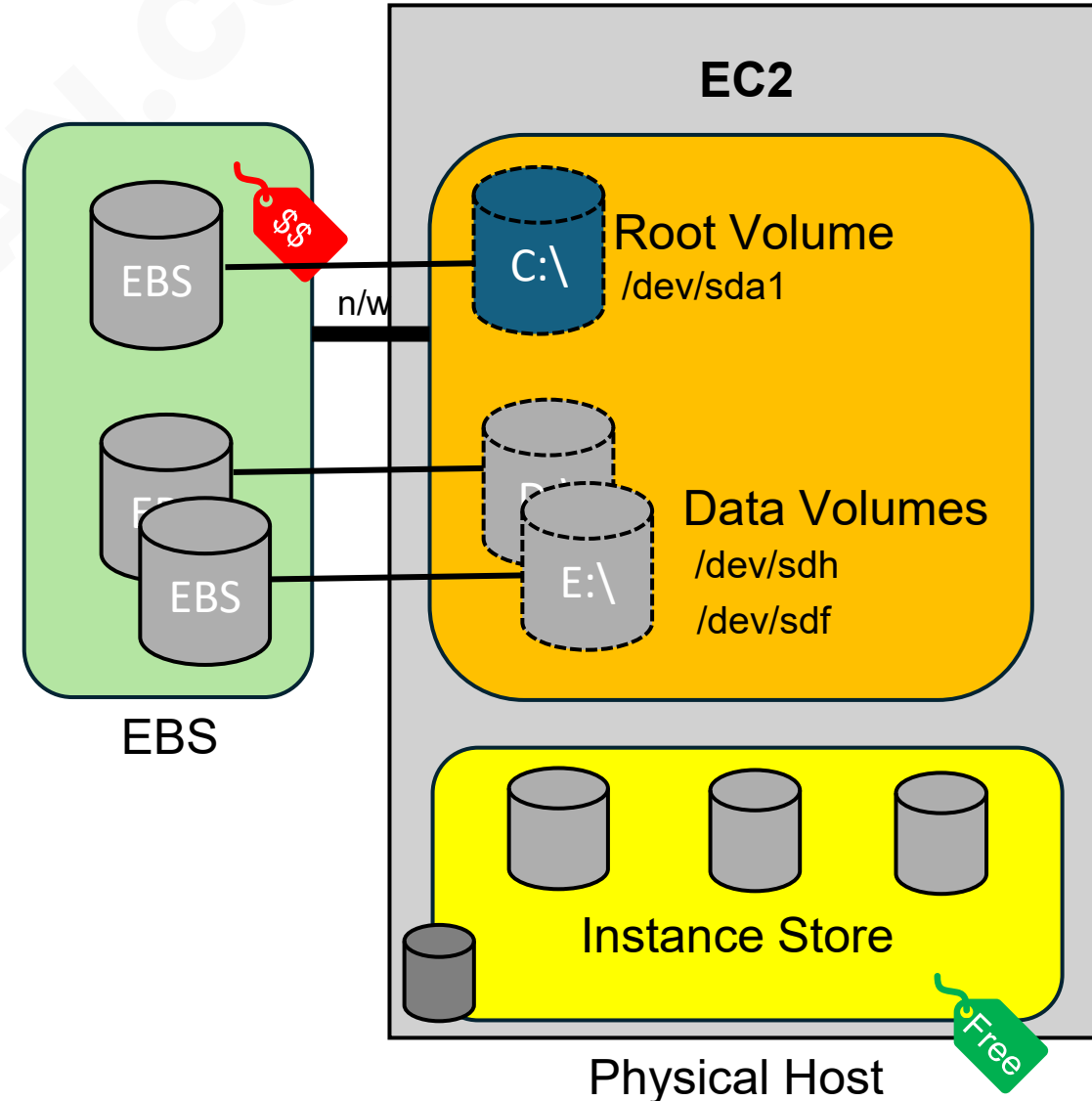


# EBS vs Instance store



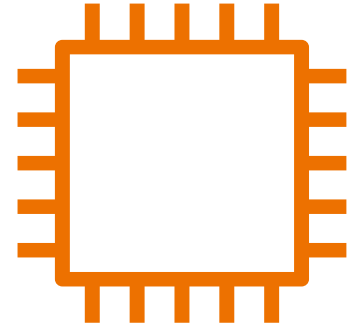
# Instance store

- EC2 disks can be of 2 types:
  - Elastic Block Storage (external to EC2 host)
  - Instance Store (optional) – Local to EC2 host
- EBS volume data is persisted until volume is deleted
- Instance Store volume data is wiped out if EC2 is stopped and started.
- EBS volumes can be created and attached to EC2 instance. Instance store volume has size limit and comes with specific EC2 instance types only.
- EBS has per GB cost. Instance volumes are free.
- EBS is used for almost all purposes whereas Instance store can be used for **temp directory, buffer or cache**



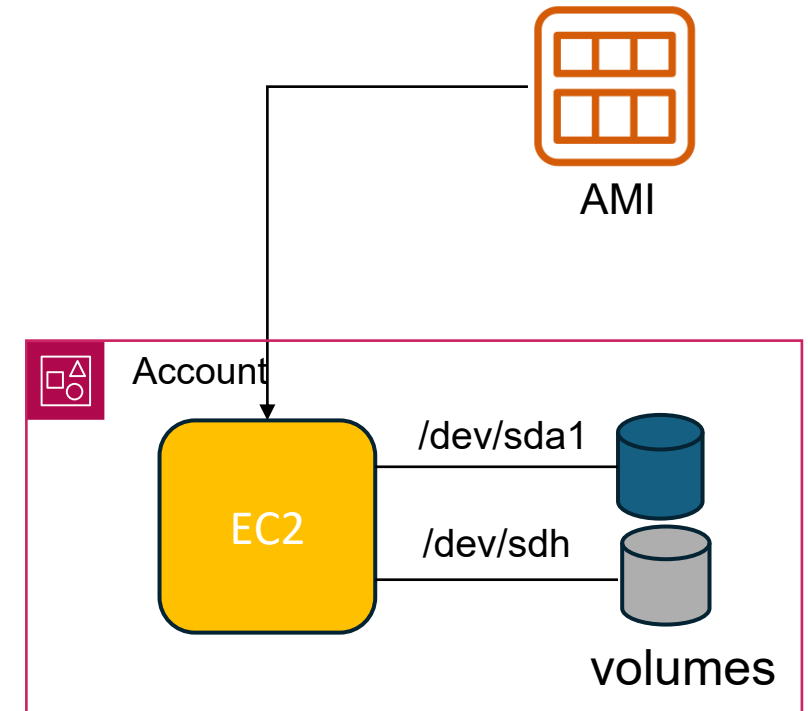
# Configuration options for EC2

- CPU, Processor family, Memory - **Instance Type**
- Firewall - Security Group
- Storage Disk - Elastic Block Storage (EBS)
- Operating System - **Amazon Machine Image (AMI)**
- Login credentials - **SSH key pair**
- Network, IP address (Public / Private / Elastic), AZ - **VPC**
- Initialization Scripts - **User Data**
- IAM Permissions - **IAM Role**
- Purchasing Option - **OnDemand (default), Spot, Savings Plan, Reserved Instances**
- Hardware Tenancy - **Shared (default), Dedicated**
- Name of the instance (Optional) - **Tag (key-value)**



# Amazon Machine Image (AMI)

- An AMI provides information required to launch an instance.
- You can launch multiple instances from a single AMI when you require multiple instances with the same configuration.
- An AMI includes the below:
  - A template for the root volume for the instance
  - One or more Amazon EBS snapshots
- Launch permissions that control which AWS accounts can use AMI to launch instances
- You can use AMIs from AWS Community or AWS Marketplace AMIs or you can create your own AMIs.
- AMIs are region specific



# Amazon Machine Image (AMI)

Quick Start AMIs (45)  
Commonly used AMIs

My AMIs (2)  
Created by me

AWS Marketplace AMIs (10621)  
AWS & trusted third-party AMIs

Community AMIs (500)  
Published by anyone

Refine results

Clear all filters

☐ Free tier only [Info](#)

▼ OS category

☐ All Linux/Unix

All products (45 filtered, 45 unfiltered)



Amazon Linux

Free tier eligible

Verified provider

Amazon Linux 2023 AMI

ami-0e53db6fd757e38c7 (64-bit (x86), uefi-prefer

Amazon Linux 2023 is a modern, general purpose L

support. It is optimized for AWS and designed to p

environment to develop and run your cloud applica

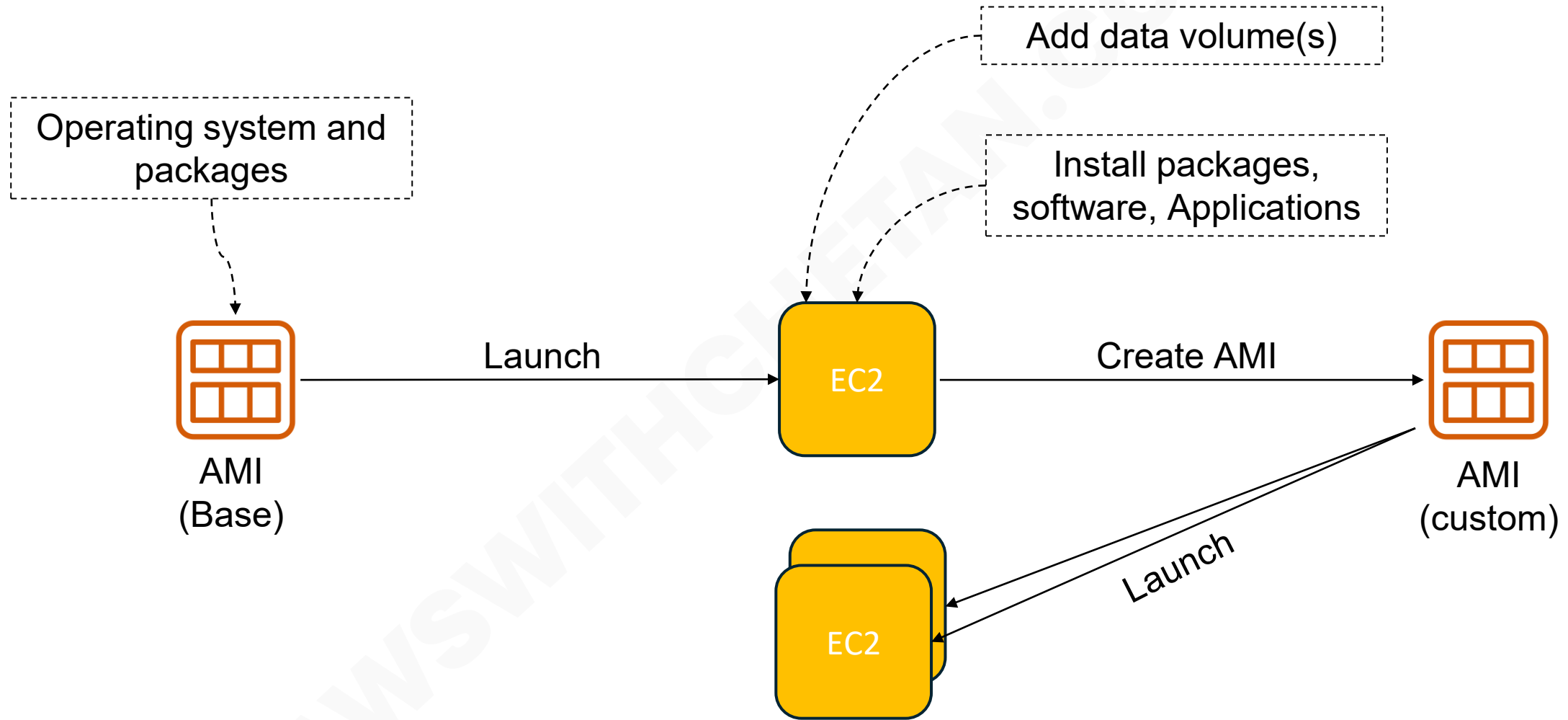
Platform: amazon

Root device type: ebs

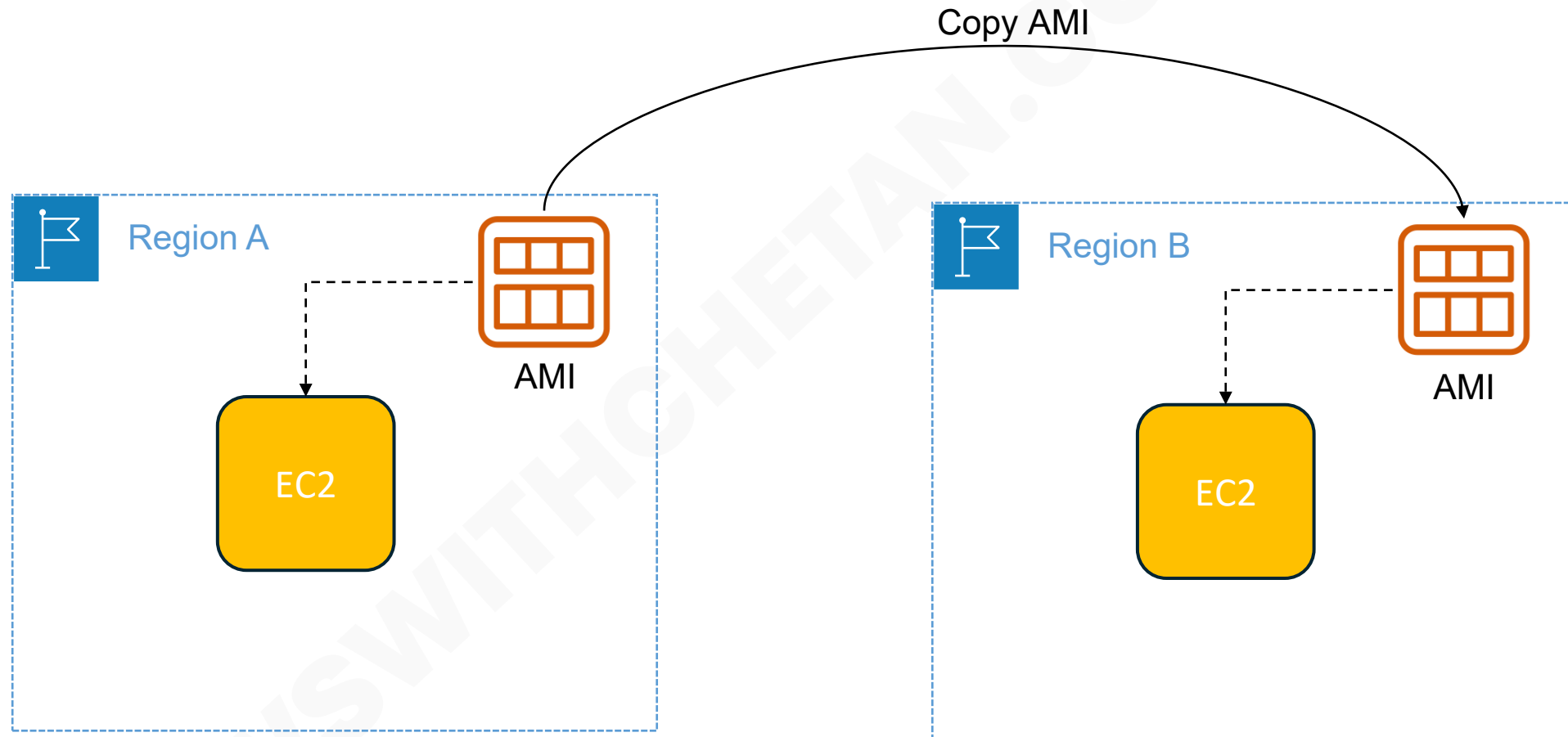
All rights reserved © www.awswithchetan.com

Chetan Agrawal

# Creating custom AMI



# AMIs are regional

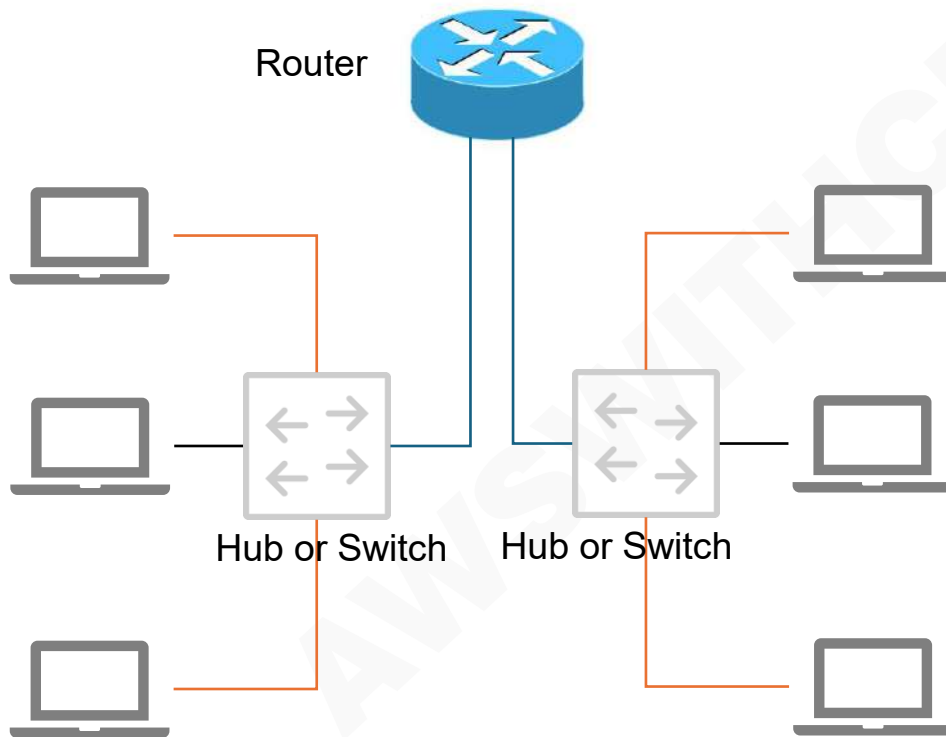




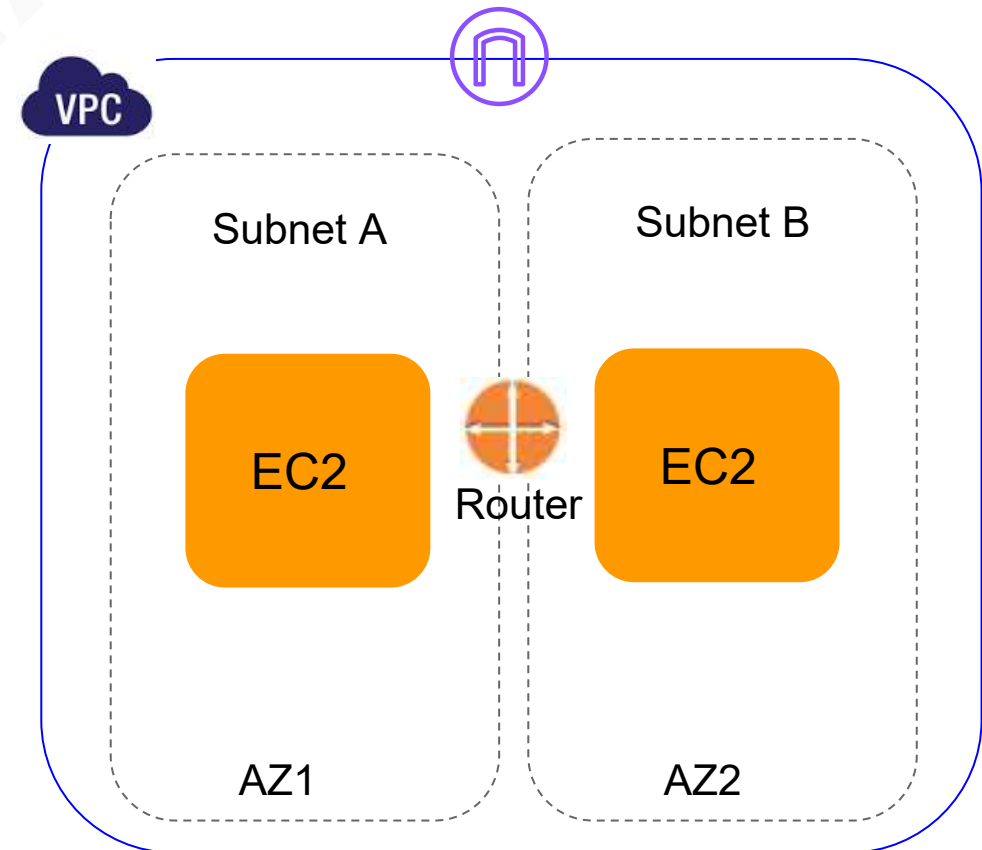
# Virtual Private Cloud (VPC)

*A virtual private cloud (VPC) is a virtual network dedicated to your AWS account. It is logically isolated from other virtual networks in the AWS Cloud.*

## Physical Network

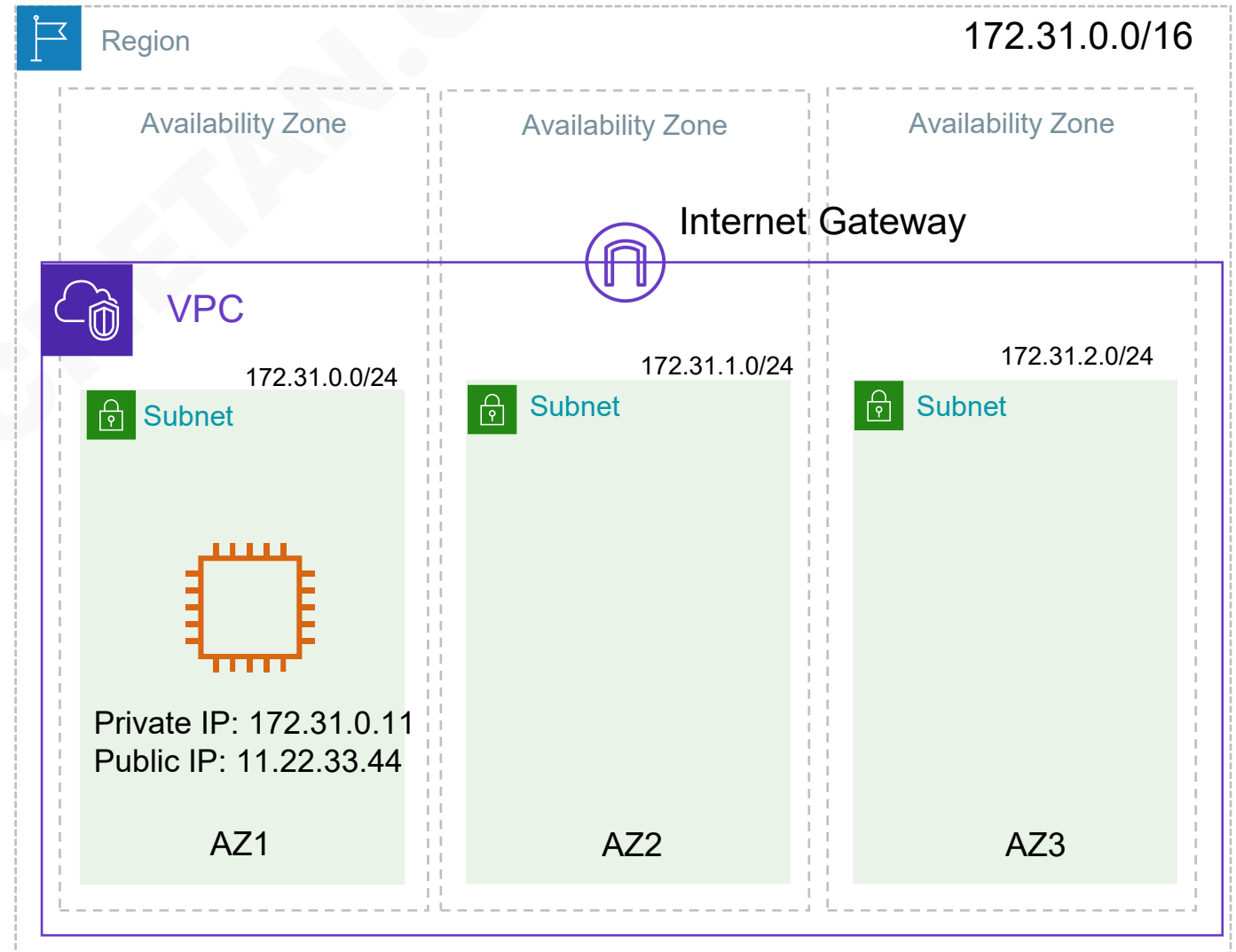


## AWS VPC



# Default VPC

- AWS Creates Default VPC in each AWS region
- VPC with CIDR 172.31.0.0/16
- If you don't select any VPC then this default VPC is used to launch an EC2 instance
- Create one subnet in each AZ
- Has components like Internet Gateway, route table to allow communication over the internet
- EC2 instance receives the Private IP from the subnet CIDR range and Public IP from the Amazon's pool of Public IPs



# EC2 Public IP vs Private IP vs Elastic IP

Whenever we launch an EC2 instance, a public and a private IP get allocated to it

- **Private IP:** Allocated from the subnet CIDR range. Enables communication over Private Ips within the VPC.

Note: Private IP designated to an EC2 remains associated with EC2 until terminated



- **Public IP:** Allocated from a Amazon's pool of available Ips. Public IP is required let you connect EC2 instance over the internet.

Note: One you Stop and Start you will lose Public IP

- **Elastic IP:** - It is static Public IP that can be allocated to EC2 instance. Once assigned, existing public Ip gets released and replaced with the newly assigned Elastic IP.
- They are allocated to the AWS account so that we can release it from specific EC2 and re-assign it to any other EC2 instances (if needed).

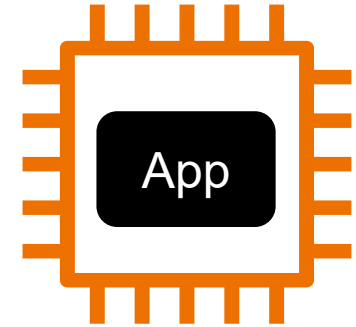


Note: Elastic IP allocated to an EC2 remains associated with EC2 until terminated or released manually.



# Different ways to install & configure applications in EC2

- **Manual** – Connect over SSH/RDP and manually install the required software/applications
- **Use AMI** – Use the pre-created AMIs to launch the instance
- **Use EC2 User data** – Run the required installation scripts during the EC2 boot time



# EC2 User data

- It is a **bootstrap** script to automatically configure the instance at the time of first launch
- EC2 User data script will run only **once** when instance first starts.
- It is used to automate boot tasks such as :
  - Install OS and package updates
  - Install software or agents
  - Download common files from the Internet
- EC2 User data scripts run with a root user.

## Sample User data Script

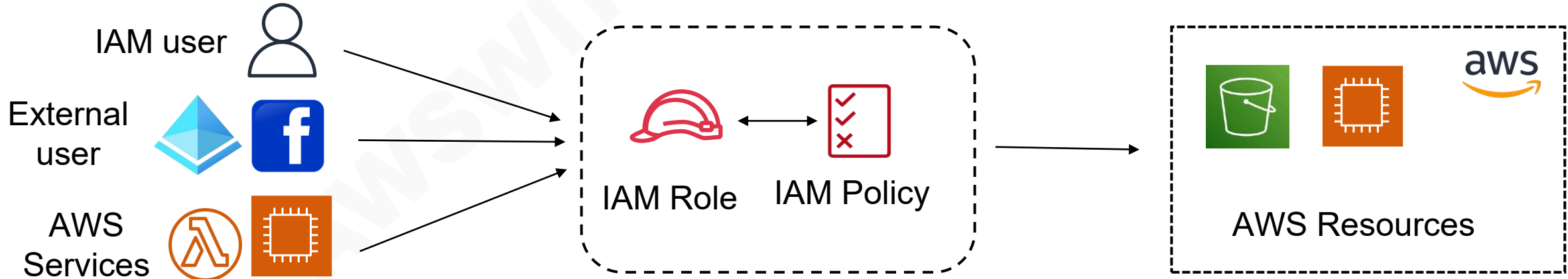
```
#!/bin/bash
# Update the instance and install Apache
yum update -y
yum install -y httpd

# Start Apache and enable it on boot
systemctl start httpd
systemctl enable httpd

# Create a sample web page
echo "<h1>Welcome to my EC2 instance via
User Data!</h1>" >
/var/www/html/index.html
```

# IAM Role

- An IAM role is an AWS identity that can be assumed temporarily by users, applications, or services to securely access AWS resources without using long-term credentials.
- IAM Role is used by:
  - AWS service such as Amazon EC2, Lambda to access other AWS service or resource
  - An IAM user in the same or a different AWS account (e.g. for cross-account access)
  - An external user authenticated by an external identity provider (IdP) service such as Google, Facebook to get access to AWS account



# Exercise : Launch EC2 instance with User data

1

Launch EC2 (Linux) using same configurations as previous exercise. Just add the User data script below in the EC2 launch page -> Advanced details -> User data section

```
#!/bin/bash
# Install httpd (Amazon Linux 2023 version)

dnf update -y
dnf install -y httpd

# Start and enable the httpd service
systemctl start httpd
systemctl enable httpd

# Create a simple index.html file
echo "<h1>Hello I am loving this AWS course :)</h1>" > /var/www/html/index.html
```

[https://github.com/awswithchetan/aws-solutions-architect-associate/blob/main/ec2\\_userdata.txt](https://github.com/awswithchetan/aws-solutions-architect-associate/blob/main/ec2_userdata.txt)

2

After instance is in “Running” state, open the browser and access website using URL `http://<public IP>`

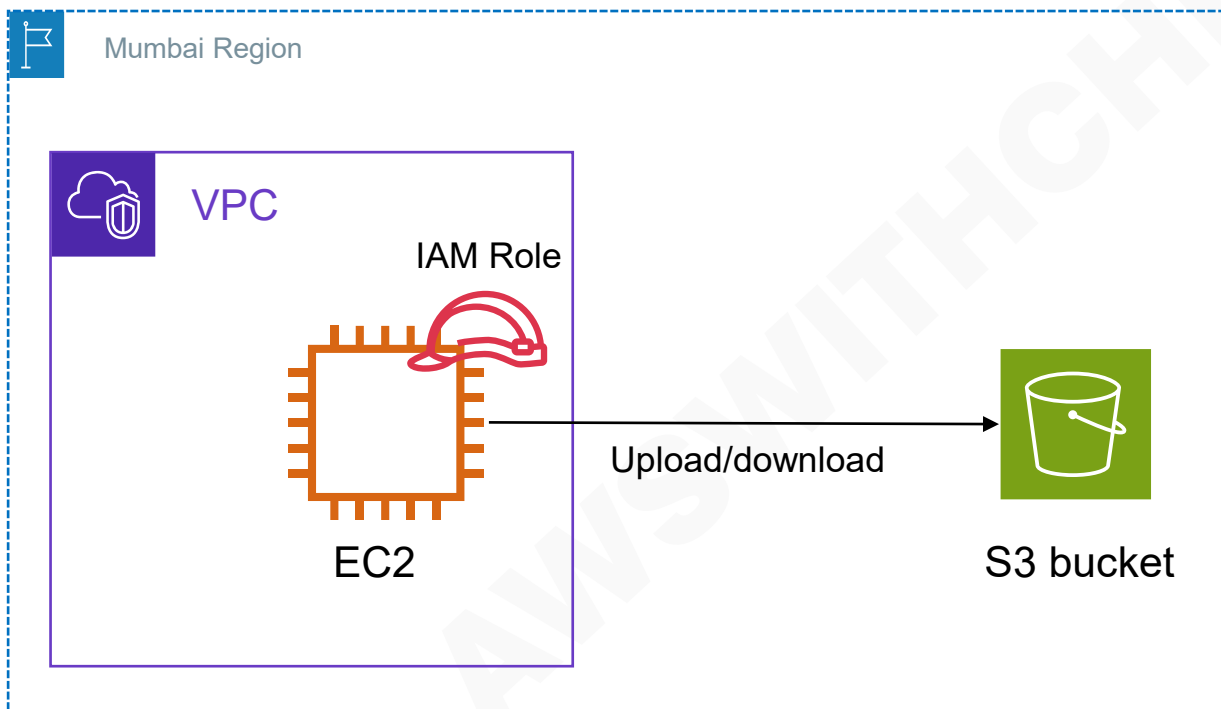
Do not terminate the instance yet. We will re-use this instance for the next exercise

# Exercise: EC2 IAM Role

We will revisit this exercise in EC2 section

## Exercise steps

- 1 Launch EC2 Linux instance in a default VPC and connect to it over SSH.
- 2 Create S3 bucket in the same region and upload some sample text or image file.
- 3 From EC2 terminal, try to download file from S3 using AWS CLI command – Access denied.
- 4 Go to AWS IAM and create IAM role for EC2. Attach S3 full permissions policy to the role.
- 5 Associate this new role to EC2 instance
- 6 Try again to download file from S3 – should be successful.
- 7 Optionally copy the downloaded file into webserver /var/www/html/ directory and check if accessible over the browser





# Steps

## Pre-requisite:

1. S3 bucket and some file in the bucket
2. IAM role for EC2 instance having S3 Read or S3 Full access permissions.

- 1 Connect to EC2 instance over Instance Connect and try downloading file from S3 using following command

```
aws s3 cp s3://bucket_name/file_name .
```

- 2 Go to Instance -> Actions -> Security -> Modify IAM Role -> Select the IAM role that had created previously for EC2 instance

- 3 Try to download the same file again

```
aws s3 cp s3://bucket_name/file_name .
```

- 4 Also copy this file into webserver directory to check if you can access it over the internet

```
sudo cp file_name /var/www/html/
```

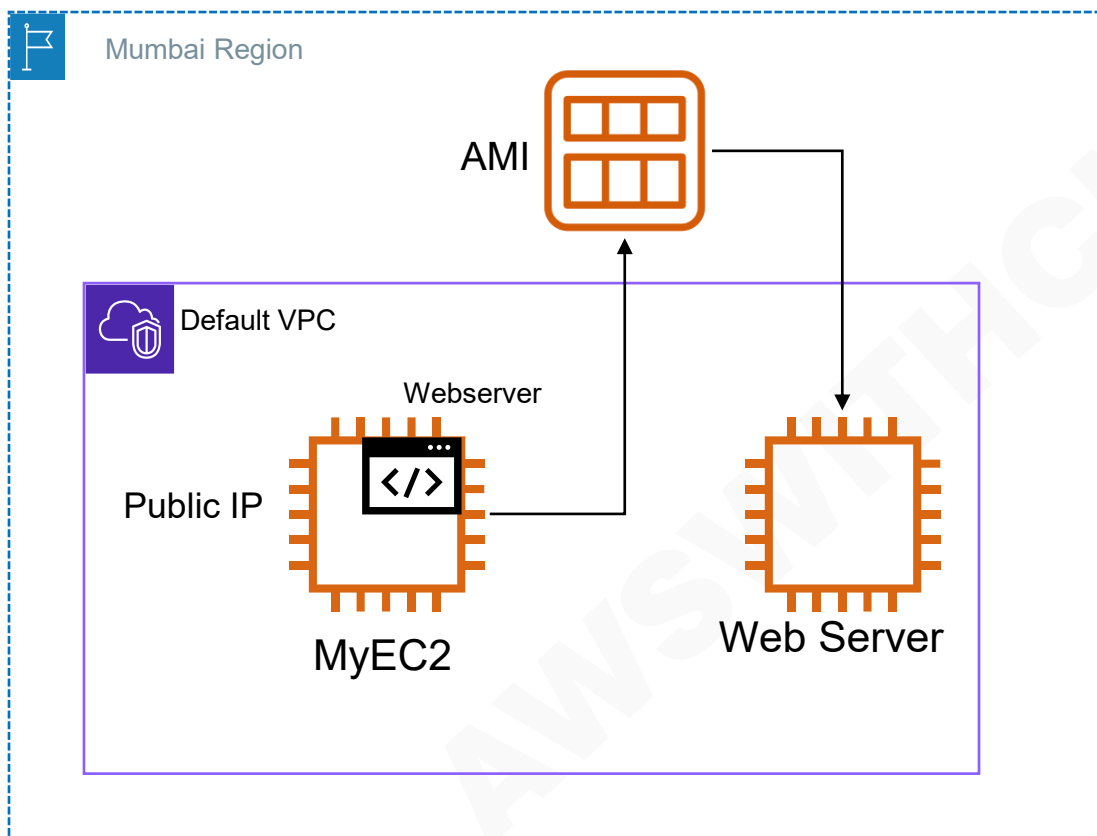
- 5 Access over the browser: [http://EC2\\_PUBLIC\\_IP/file\\_name](http://EC2_PUBLIC_IP/file_name)

# Exercise – EC2 AMI

## High level steps

### Pre-requisite:

- Running EC2 instance with WebServer installed and running



- 1 Select instance -> Actions -> Image and templates -> Create Image
- 2 Provide the name and description and Create Image
- 3 Wait for Image to be available
- 4 Launch a new EC2 instance from the AMI you created (Select your IAM while launching EC2 instance)
- 5 After instance is up and running, check if Website is accessible with new instance IP
- 6 Terminate both the instances

# Steps

1

## Create an AMI

- EC2 console -> Select your instance -> Actions -> Image and templates -> Create image
- Provide Image name and description -> Create image
- Wait for image to be created. Check in EC2 console -> Left menu -> AMIs -> Owned by me

2

## Launch a new EC2 instance using your AMI

- Launch new EC2 instance but this time select your own AMI instead of Amazon Linux 2023 AMI.  
Application and OS images -> My AMIs -> Select your AMI
- Select existing key pair, default VPC, existing security group (allow SSH and HTTP) and launch the instance
- Wait for instance to be Running

3

## Verify and access the website on new EC2 instance

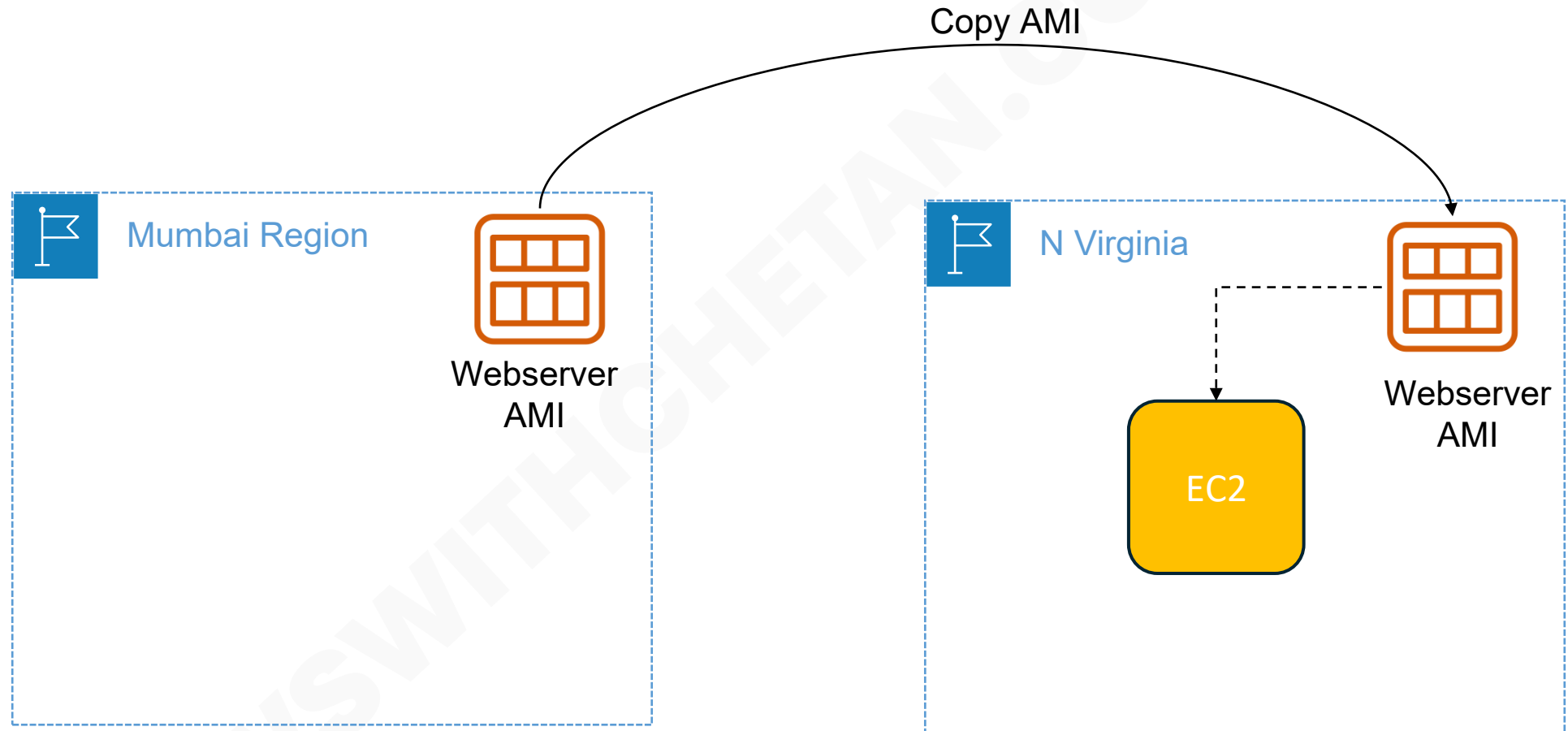
- Get the Public IP of the new EC2 instance
- Open the browser URL `http://EC2_PUBLIC_IP`

4

## Terminate both the EC2 instances

- Select both the instances -> Instance State -> Terminate

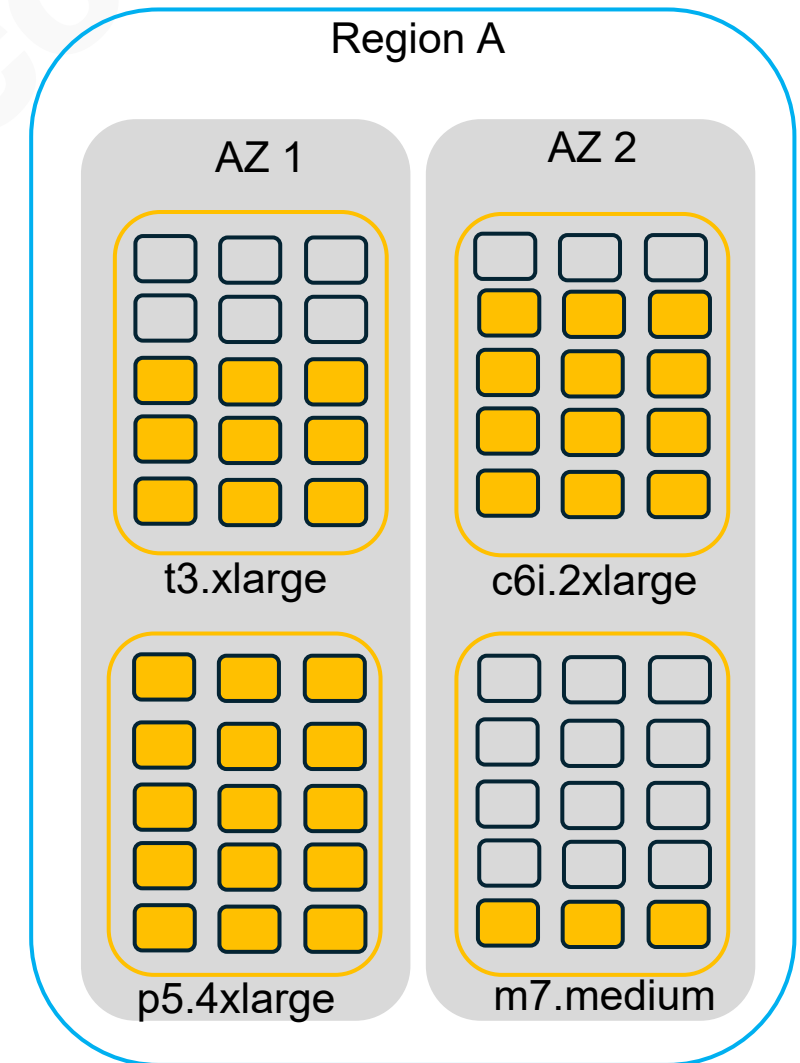
# Assignment – Copy AMI across region



# EC2 Pricing options

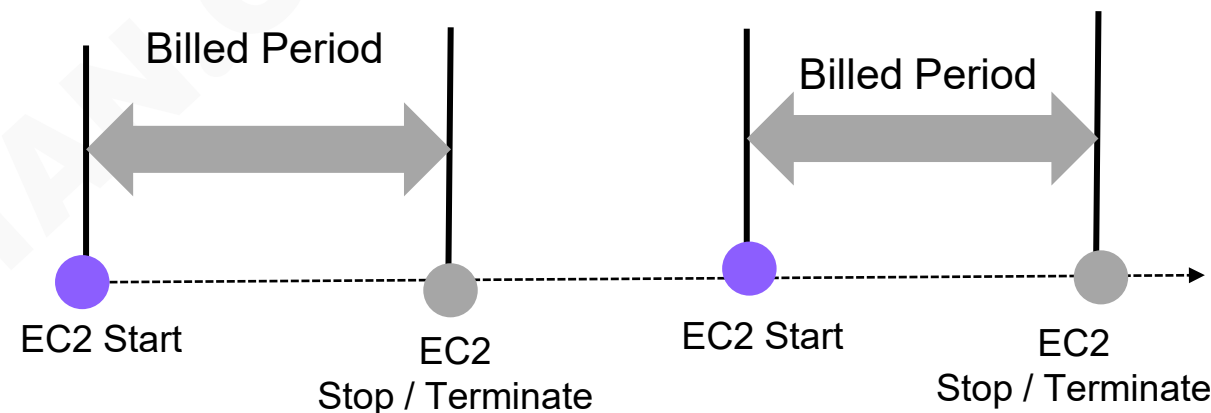
1. On-Demand
2. Spot
3. Savings Plan
4. Reserved Instances

For EC2, you pay by per second for Amazon Linux, Windows, RHEL, Ubuntu/Pro Instances and by an hour for other operating systems



# On-Demand

- Default pricing option
- No long-term commitment, no discounts
- Flexible – Any instance type, Launch / Start / Stop / Terminate at any time
- Best for unpredictable, spiky, stateful workloads



## Use cases:

- For development, Test or staging environments where uptime is unpredictable
- For POCs (short-term), experimentation and benchmarking where different instance types can be tried before deciding on Reserved instances or Savings plans

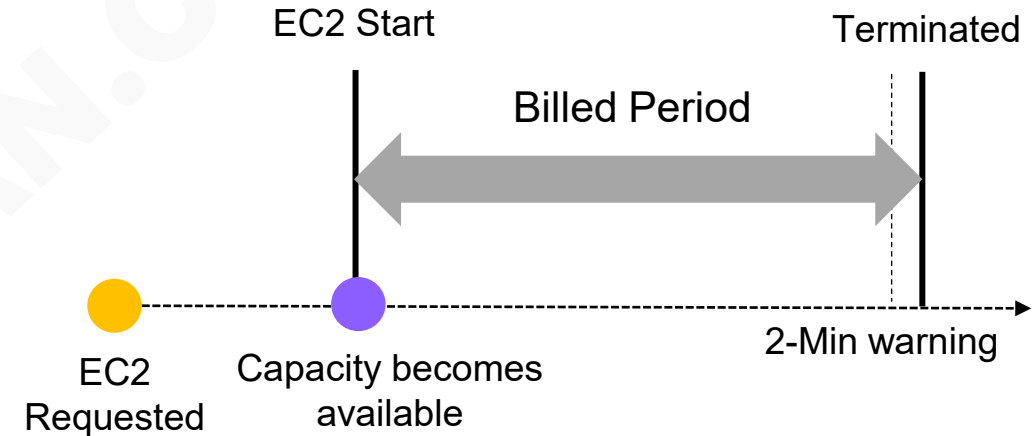
## Anti-pattern:

- Long term steady-state workloads (RI or SP more suitable)
- Interruptible / stateless workloads (Spot is more suitable)



# Spot

- Available at up to **90%** discount compared to On-demand pricing
- Provided from the spare capacity available
- Can be interrupted by AWS with 2 minutes of warning
- Can be used with Autoscaling group and Spot fleets
- Best for fault tolerant, non time sensitive and stateless workloads



## Strategies for using Spot instances:

- Be flexible with the time to run the workload
- Try changing instance type and have flexibility to choose across instance families
- Try changing Availability Zone and even the region

## Use cases:

- Big data, Graphics rendering, CI/CD, Machine learning training jobs

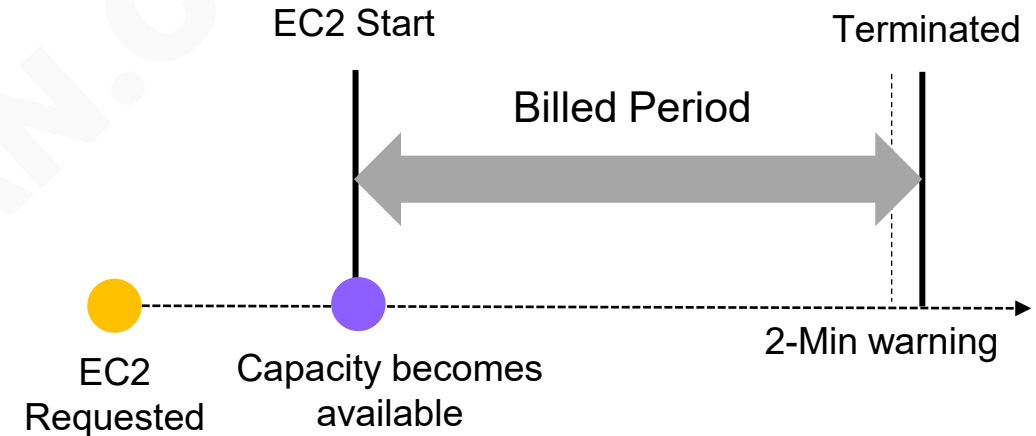
## Anti-pattern:

- Critical or stateful workloads that can't handle interruptions



# Demo – Spot instances

- Available at up to **90%** discount compared to On-demand pricing
- Provided from the spare capacity available
- Can be interrupted by AWS with 2 minutes of warning
- Can be used with Autoscaling group and Spot fleets
- Best for fault tolerant, non time sensitive and stateless workloads



## Strategies for using Spot instances:

- Be flexible with the time to run the workload
- Try changing instance type and have flexibility to choose across instance families
- Try changing Availability Zone and even the region

## Use cases:

- Big data, Graphics rendering, CI/CD, Machine learning training jobs

## Anti-pattern:

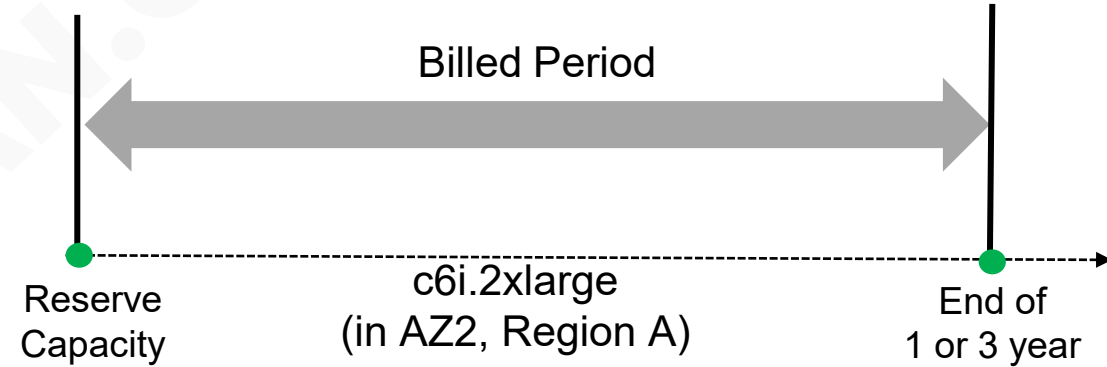
- Critical or stateful workloads that can't handle interruptions





# Savings Plan (SP)

- Compute capacity (\$/hour) is committed for 1 or 3 year
- Discounts up to 72% compared to On-demand pricing
- Pay whether you use the capacity or not
- **EC2 Savings Plan** – Usage for EC2 instance family in selected Region (max discount)
- **Compute Savings Plan** – Usage across any EC2, Lambda and Fargate in any region
- Can pay Full upfront (max discount), Partial upfront or No upfront
- Best for steady state and predictable workloads



## Use cases:

- Databases, Steady-state workloads
- Having mix of compute workloads including EC2, Lambda and Containers

## Anti-pattern

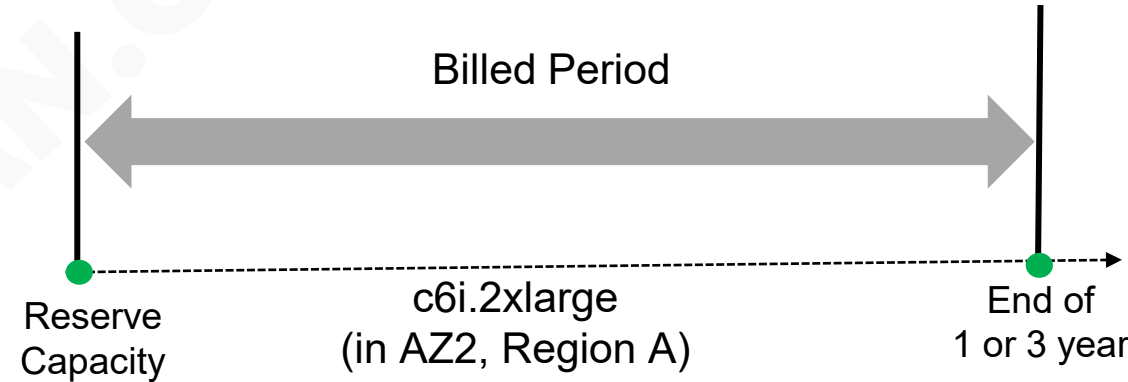
- Short-term workloads like Development/Test environment, Proof of Concept, Pilot workloads, Weekly or monthly batch jobs



Max discount = EC2 Savings Plan + 3 Year Term + Full Upfront payment

# Reserved Instances (RI)

- Long-term commitment for given Instance type/size
- Discounts up to 72% compared to On-demand pricing
- EC2 capacity is reserved for 1 or 3 year term
- Pay whether you use the capacity or not
- **Standard RI** – Fixed instance type, AZ (max discount)
- **Convertible RI** – Can change instance type, AZ or Region
- Can pay Full upfront, Partial upfront or No upfront
- Best for steady state and predictable workloads



## Use cases:

- Databases, Steady-state applications

## Anti-pattern:

- Short-term workloads like Development/Test environment, Proof of Concept, Pilot workloads, Weekly or monthly batch jobs



# How to compare pricing?

The screenshot displays the AWS Pricing Calculator interface. The browser address bar shows the URL `calculator.aws/#/estimate`. The page header includes the AWS logo, 'pricing calculator', and links for 'Feedback', 'Language: English', 'Contact Sales', and 'Create an AWS Account'.

The main content area is titled 'My Estimate' and includes an 'Export' button and a 'Share' button. The 'Estimate summary' section shows the following costs:

| Category                                     | Cost         |
|----------------------------------------------|--------------|
| Upfront cost                                 | 0.00 USD     |
| Monthly cost                                 | 361.98 USD   |
| Total 12 months cost (Includes upfront cost) | 4,343.76 USD |

The 'Getting Started with AWS' section contains buttons for 'Get started for free' and 'Contact Sales'.

A yellow box with the text 'AWS Pricing Calculator' is overlaid on the page.

The 'My Estimate' section includes a search bar and a table of services:

| Service Name           | Status | Upfront cost | Monthly cost | Description | Region         |
|------------------------|--------|--------------|--------------|-------------|----------------|
| Amazon EC2             | -      | 0.00 USD     | 163.18 USD   | -           | US East (Ohio) |
| Amazon Simple Stora... | -      | 0.00 USD     | 32.54 USD    | -           | US East (Ohio) |
| Amazon RDS for Post... | -      | 0.00 USD     | 166.26 USD   | -           | US East (Ohio) |

# EC2 Pricing options comparison

| On-Demand                                                                                                                                                                                                                                                                                                        | Spot                                                                                                                                                                                                                                                                                       | Savings Plan                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Reserved Instances                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• <b>No commitment</b></li> <li>• Pay by second</li> <li>• Start or Stop/Terminate at any time</li> <li>• Billed only when instance is in Running state</li> <li>• <b>Highest cost</b></li> <li>• Suitable for <b>unpredictable</b>, spiky, stateful workloads</li> </ul> | <ul style="list-style-type: none"> <li>• No commitment</li> <li>• <b>Up to 90% discount</b></li> <li>• Instance can be terminated with 2-minute warning</li> <li>• <b>Lowest cost</b></li> <li>• Suitable for fault tolerant, non-time sensitive and <b>Stateless</b> workloads</li> </ul> | <ul style="list-style-type: none"> <li>• <b>1-year or 3-year commitment for \$/hour</b></li> <li>• Up to <b>72%</b> discount</li> <li>• 3 payment options: <ul style="list-style-type: none"> <li>Full upfront</li> <li>Partial Upfront</li> <li>No upfront</li> </ul> </li> <li>• <b>Pay even if not using the capacity</b></li> <li>• EC2 Savings Plan</li> <li>• Compute Savings plan</li> <li>• Suitable for <b>steady state</b> and predictable workloads</li> </ul> | <ul style="list-style-type: none"> <li>• <b>1-year or 3-year commitment for EC2 instance type/hour</b></li> <li>• Up to <b>72%</b> discount</li> <li>• 3 payment options: <ul style="list-style-type: none"> <li>Full upfront</li> <li>Partial Upfront</li> <li>No upfront</li> </ul> </li> <li>• <b>Pay even if not using the capacity</b></li> <li>• Standard RI</li> <li>• Convertible RI</li> <li>• Suitable for <b>steady state</b> and predictable workloads</li> </ul> |

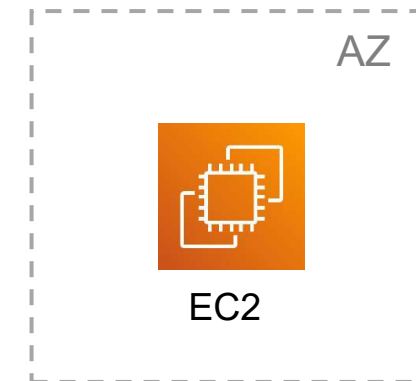
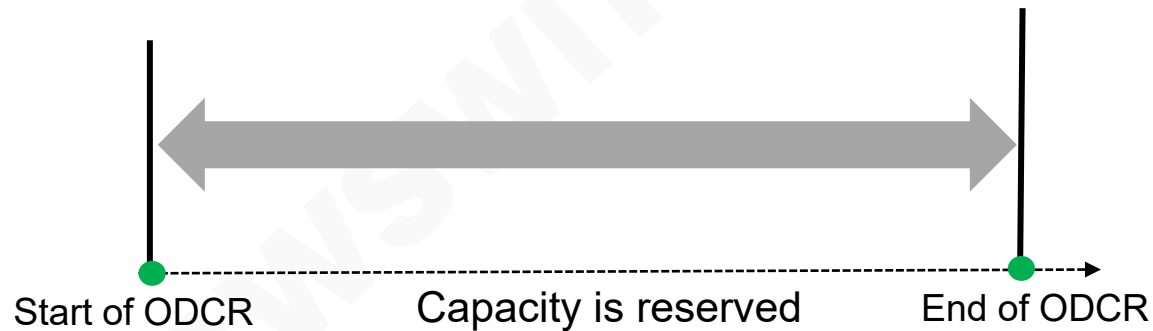
# Pricing example with different EC2 pricing models

m5.large (2vcpu, 8GiB RAM) in N. Virginia region

| Pricing model                                    | Price (Per Hour)                                                    |
|--------------------------------------------------|---------------------------------------------------------------------|
| On-Demand                                        | <b>\$0.096</b>                                                      |
| Spot Instance (Spot Price)                       | \$0.038 - \$0.04 (up to 60% off as per current trend)               |
| Reserved Instance (1 year)                       | \$0.060 (No Upfront) - \$0.057 (All Upfront)                        |
| Reserved Instance (3 years)                      | \$0.041 (No Upfront) - \$0.036 (All Upfront)                        |
| Reserved Convertible Instance (1 year)           | \$0.071 (No Upfront) - \$0.066 (All Upfront)                        |
| EC2 Savings Plan (1 year)                        | \$0.060 (No Upfront) - \$0.056 (All Upfront)                        |
| EC2 Savings Plan (3 year)                        | \$0.041 (No Upfront) - \$0.036 (All Upfront)                        |
| Compute Savings Plan (1 year)                    | \$0.071 (No Upfront) - \$0.066 (All Upfront)                        |
| Compute Savings Plan (3 year)                    | \$0.049 (No Upfront) - \$0.044 (All Upfront)                        |
| Dedicated Instance (Instance/hr + \$2/hr/region) | $\$0.102 + \$2/\text{hr}/\text{region} = \$2.102$ (On-Demand Price) |
| Dedicated Host (m5 host) – (96 vCPU/48 core)     | \$5.069 (On-Demand Price)                                           |
| On-Demand Capacity Reservations (ODCR)           | \$0.096 (On-Demand Price)                                           |

# EC2 On-demand Capacity Reservation (ODCR)

- Reserve EC2 capacity in a specific Availability Zone for any duration.
- Create ODCR anytime **without** entering a 1-year or 3-year term commitment.
- Cancel the ODCR anytime to release the capacity and stop incurring charges.
- For ODCR – specify AZ, number of instances with instance attributes – type, size, platform, tenancy.
- By default, charged at EC2 On-Demand rate. RI and SP discounts applies to matching capacity.
- Suitable for business-critical workloads that require a capacity assurance.



# EC2 Tenancy

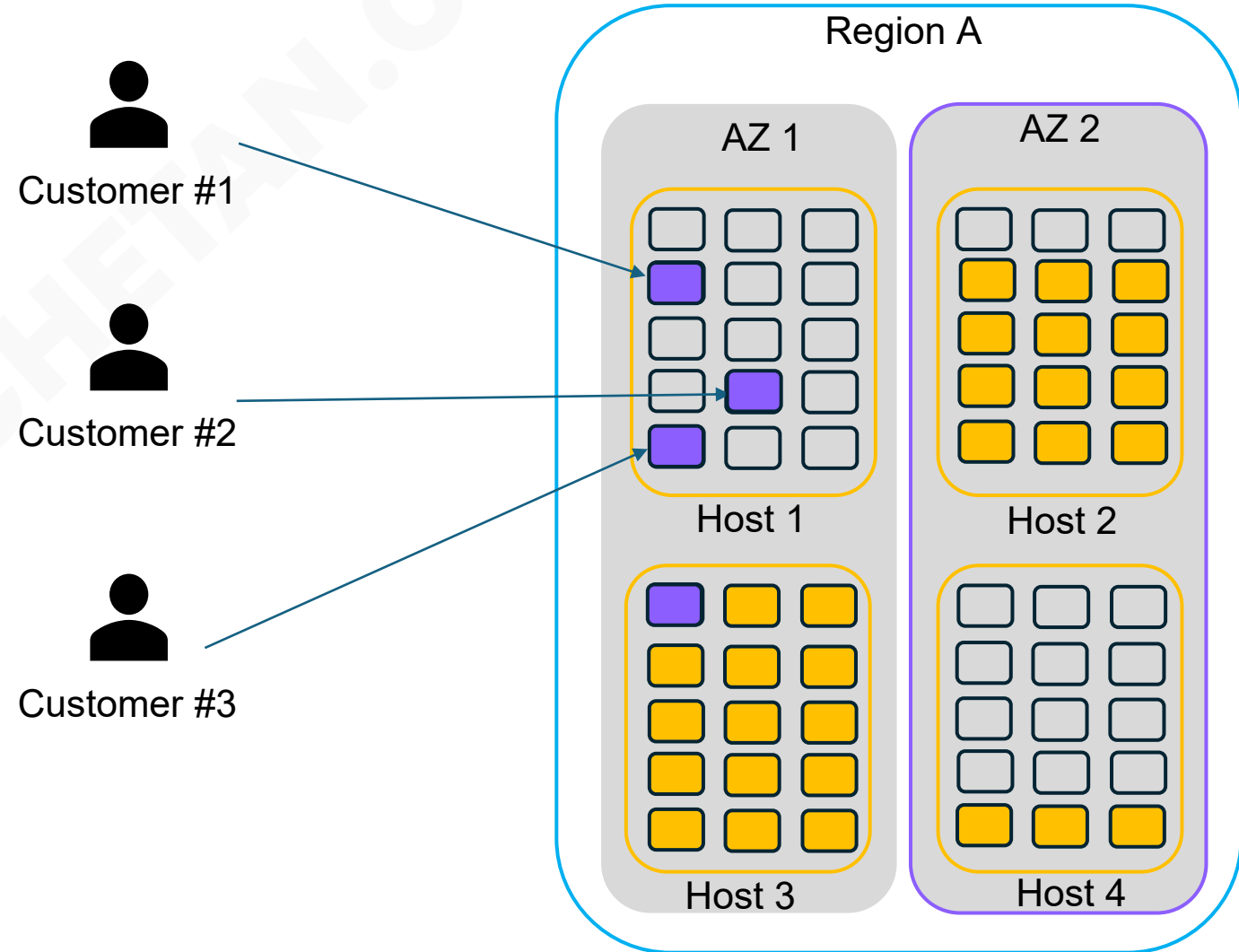
- Tenancy determines how your EC2 instances are hosted on the underlying physical hardware.
- It defines whether your instances share hardware with other AWS customers or are isolated on dedicated servers.

AWSWITHCHETAN.COM

# EC2 Tenancy – Shared vs Dedicated

## Shared Tenancy

- Underlying physical host is shared across different AWS customers (AWS account)
- This is a default tenancy for VPC and EC2

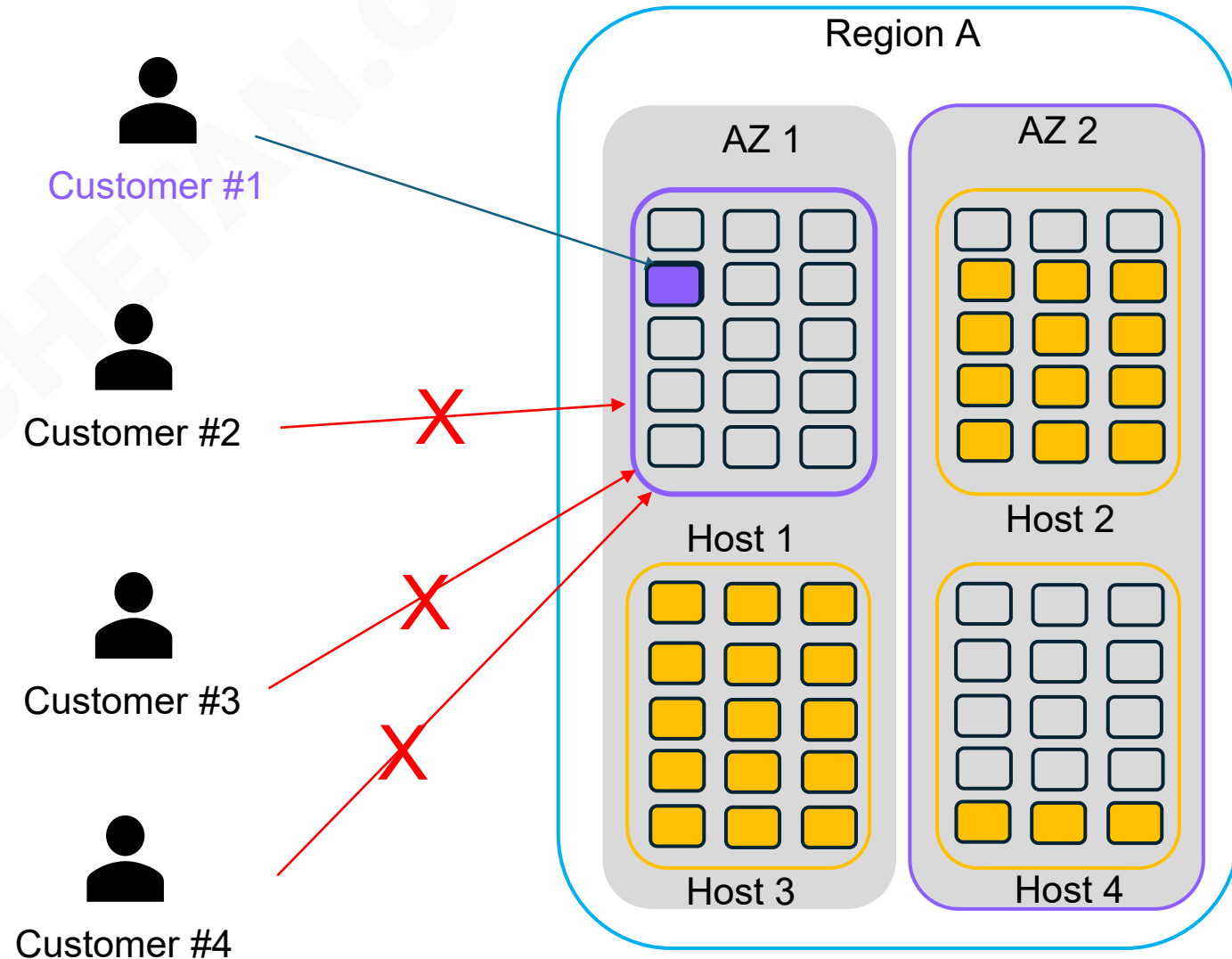




# EC2 Tenancy – Shared vs Dedicated

## Dedicated Tenancy

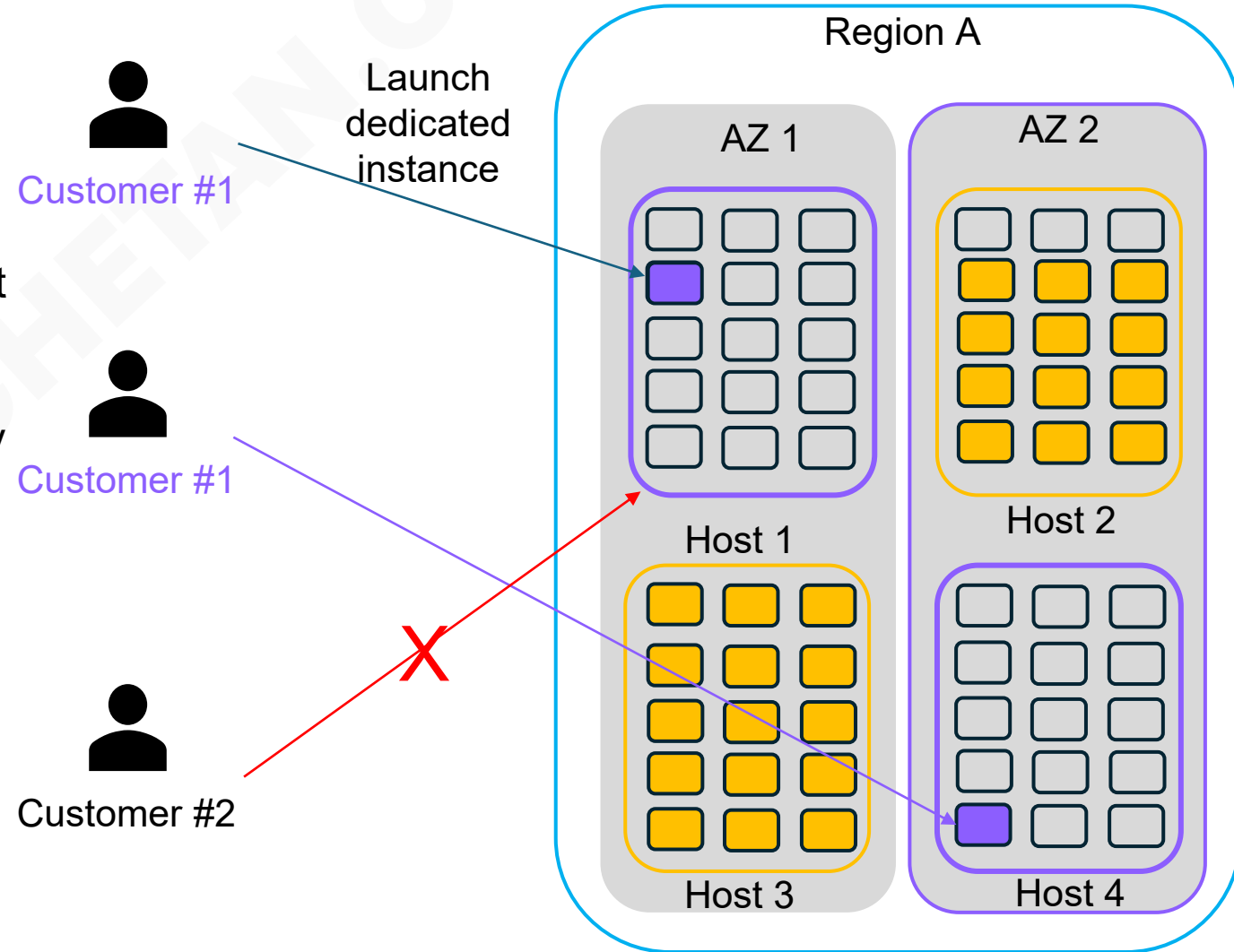
- Underlying physical host is dedicated for a single customer (AWS account)
- Used for Regulatory and Compliance requirements
- On-demand, Spot and Reserved pricing is available
- There are 2 options for Dedicated Tenancy:
  - **Dedicated Instances**
  - **Dedicated Hosts**



# EC2 Dedicated Instances

## Dedicated Instances

- Instance is launched on a host which is dedicated for the customer, but customer do not decide on which host to launch the instance
- Do not get access, placement choice or visibility of the underlying host
- There are 2 pricing elements:
  - EC2 per hour running charge
  - Dedicated host per hour charge (\$2/hour)
  - On-demand, Spot, Reserved



# EC2 Dedicated Host

## Dedicated Host

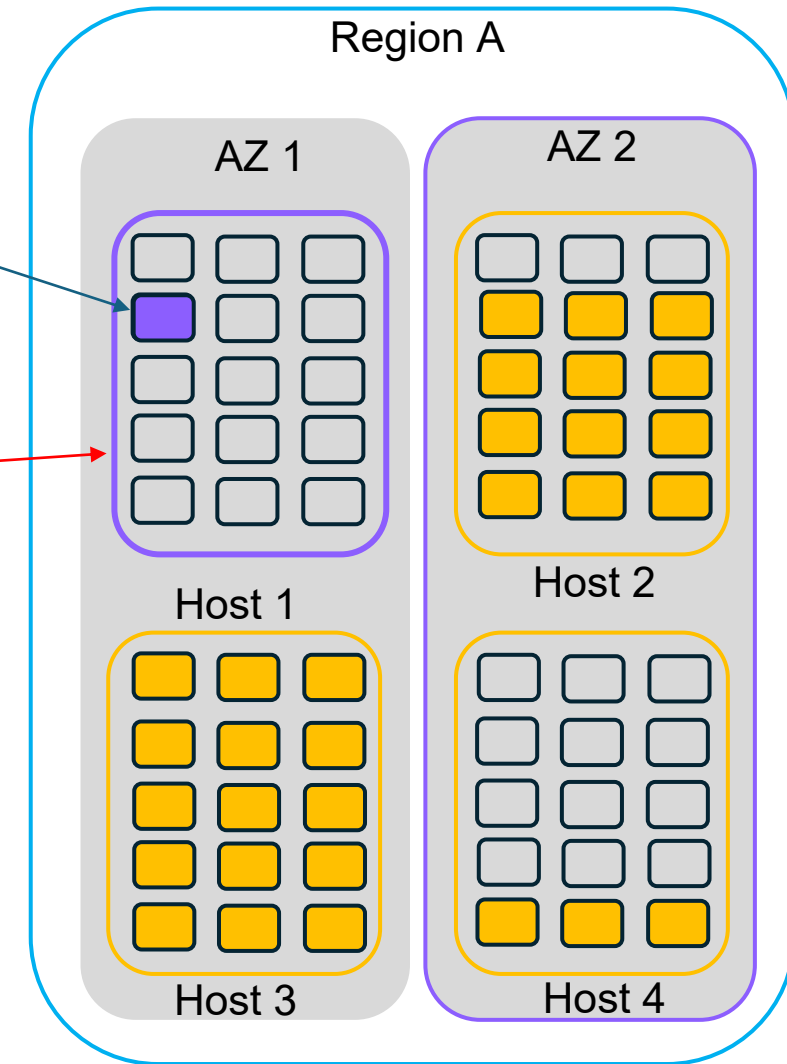
- Allow to use existing per-socket, per-core, or software licenses tied to the physical host
- Host affinity where you can choose this host to launch EC2 instances
- Pricing
  - Per host instead of per instance billing
  - On-demand, Reserved, Savings Plan

Customer #1

Customer #2

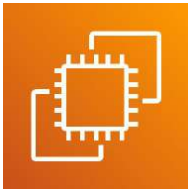
Customer #3

Customer #4

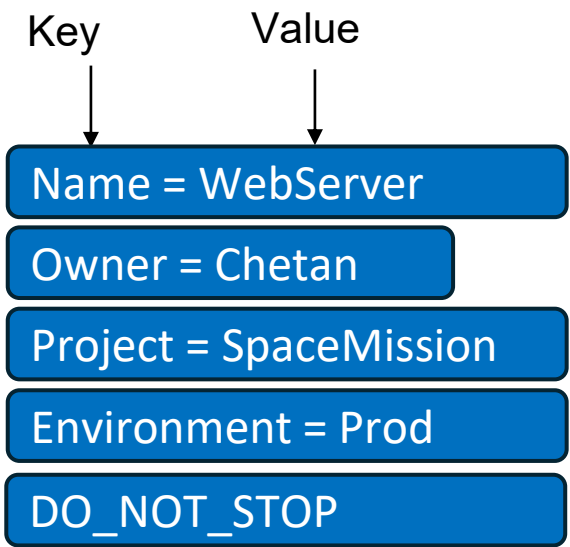


# EC2 Instance Tags

- Tags are in the form of **Key Value** Pairs.
- Can be useful to filter resources while queries AWS resources.
- Are useful when you use AWS deployment services like Code deploy, Code pipeline.
- Can use tags in IAM policies to restrict access to users for a particular instance(s).
- Very useful for cost allocation e.g. per project or environments etc.



EC2



Tags

<

1

>

⚙

Manage tags

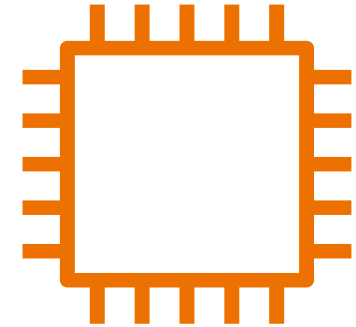
| Key         | Value     |
|-------------|-----------|
| Envrionment | demo      |
| Name        | Webserver |
| Owner       | Chetan    |

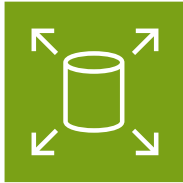
# EC2 Advanced

For Solutions Architect

# EC2 Advanced section – For Solutions Architect

- EBS Deep dive
- AMIs vs EBS Snapshot vs EC2 User data
- Elastic Network Interface (ENI)
- EC2 Hibernation
- EC2 Placement groups
- EC2 Instance Metadata Service (IMDSv2)





# Elastic Block Storage

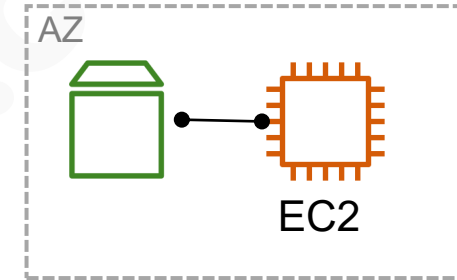
# EBS – Key terminologies

- Volume Size - The storage capacity of an EBS volume, measured in GiB (Gibibytes).
- IOPS (Input/Output Operations Per Second)
  - Measures how many read and write operations per second a volume can handle
  - Indicates the speed and responsiveness of storage. Important for transaction-heavy workloads (e.g. databases).
- Baseline IOPS
  - The default guaranteed performance for your volume type (before bursting or provisioning).
- Provisioned IOPS (PIOPS)
  - A feature where you explicitly set desired IOPS (up to 256,000).
- Burst Performance / Burst Credits
  - Temporary performance boost above baseline IOPS when needed.
  - Volumes accumulate burst credits when idle and spend them under load.
  - Helps small volumes handle occasional heavy I/O spikes.
- Throughput (MB/s)
  - Measures the amount of data transferred per second, typically in MB/s.
  - IOPS = “how many”; Throughput = “how fast”.

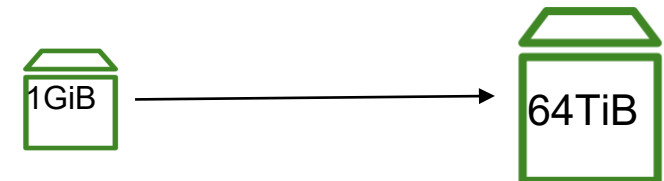


# EBS fundamentals

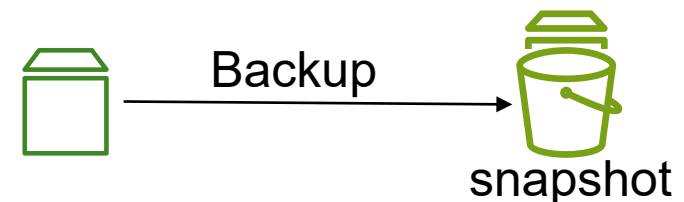
- An Amazon EBS is a block-level storage device that you can attach to your EC2 instance in a given AZ.
- EBS provides high availability, reliability and durability for the data stored in EBS volumes
- Dynamically increase size, modify the provisioned IOPS capacity, and change volume type for existing volumes.
- Allows encryption using encryption keys from AWS Key Management System (KMS).
- EBS volumes data can be backed up using point-in-time snapshots. Snapshots are stored in S3.



 99.9% Availability (within AZ)  
99.999% Durability

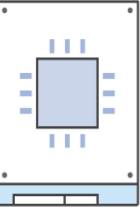


 Data at rest encryption

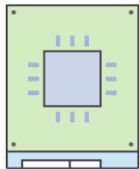


# EBS volume types

## Performance

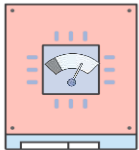


SSD



**gp3, gp2**

General Purpose SSD

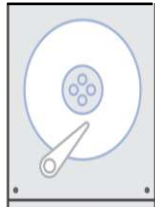


**io1, io2 Block Express**

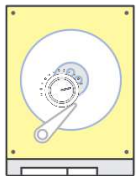
Provisioned IOPS  
SSD

gp3 - 3000 baseline IOPS, scales up to 80k IOPS  
gp 2 - 3 IOPS x per GB, scales up to 16k IOPS

64k to 256k IOPS

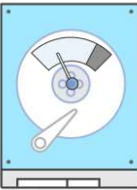


HDD



**st1**

Throughput Optimized  
HDD



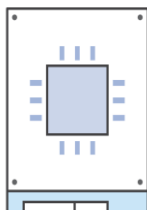
**sc1**

Cold HDD

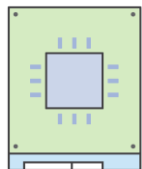
250 to 500 IOPS

# EBS volume types

## Use cases

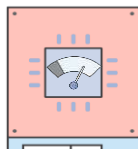


SSD



**gp3, gp2**

General Purpose SSD

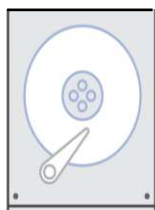


**io1, io2 Block Express**

Provisioned IOPS  
SSD

- Operating system
- Relational databases: MySQL, SQL Server, PostgreSQL, SAP, Oracle

- NoSQL Databases
- Cassandra, MongoDB, CouchDB

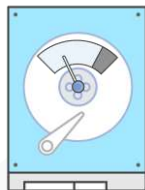


HDD



**st1**

Throughput Optimized  
HDD



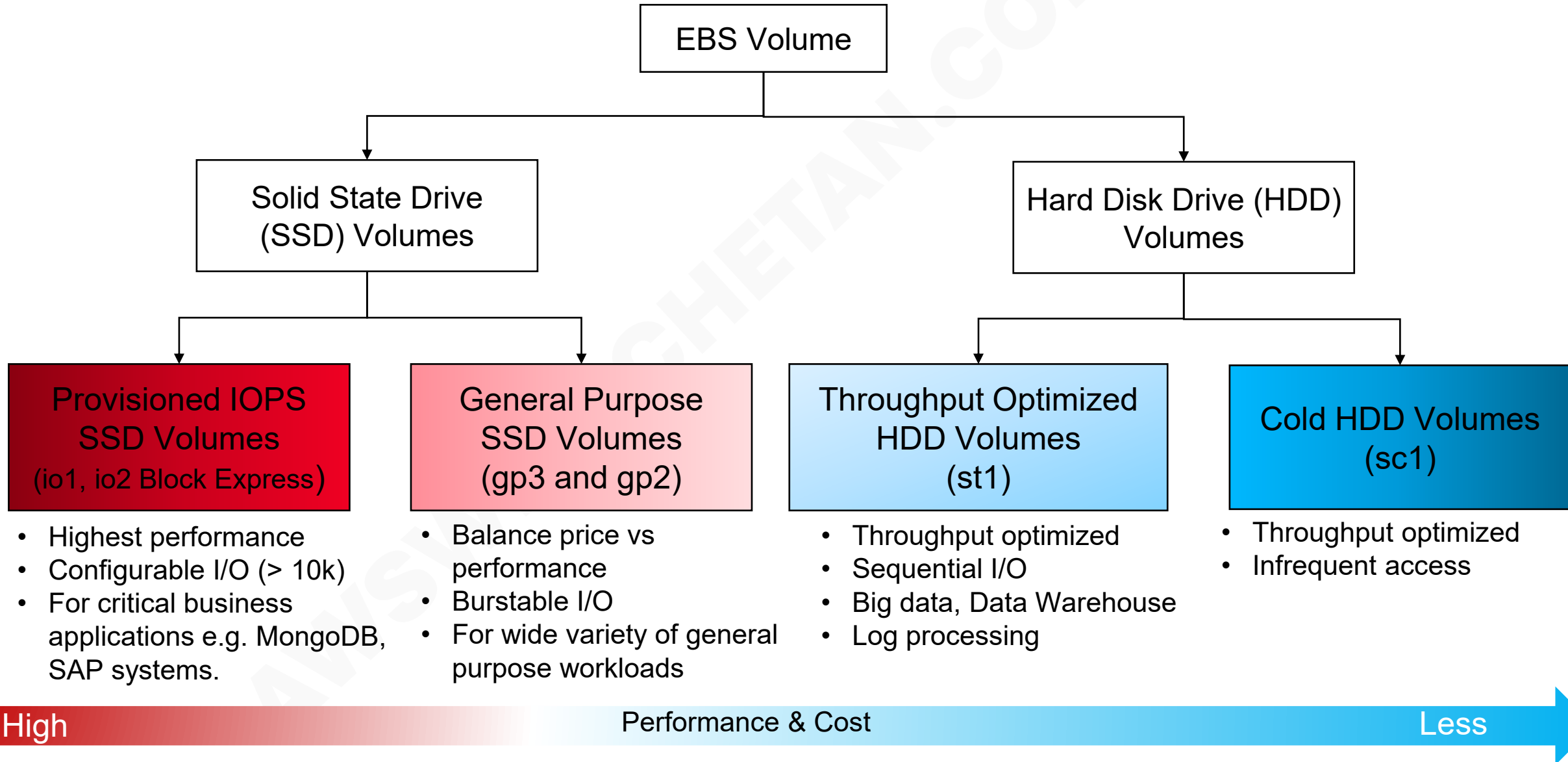
**sc1**

Cold HDD

- Big Data , Analytics
- Kafka, Splunk, Hadoop, Data Warehousing

- File / Media CIFS/NFS
- Transcoding, Encoding, Rendering

# EBS volume types



# EBS Performance – good to know

## CloudWatch Monitoring

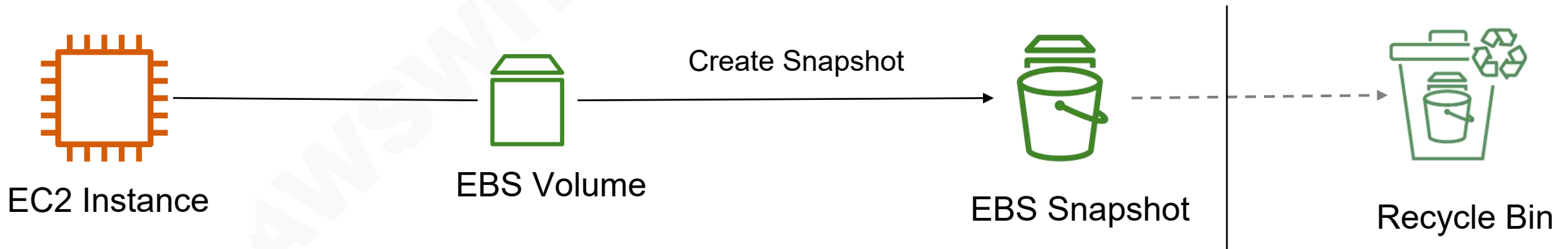
- EBS volume performance like IOPS, throughput, latency etc. can be monitored using Amazon CloudWatch.
- Typically, CW metrics like VolumeReadOps / VolumeWriteOps / VolumeReadBytes / VolumeWriteBytes / VolumeQueueLength etc. are used for troubleshooting performance bottlenecks

## EBS-Optimized Instances:

- Amazon EBS-Optimized instances use an optimized configuration stack and provide additional, dedicated bandwidth for Amazon EBS I/O.
- This optimization provides the best performance for your EBS volumes by minimizing contention between Amazon EBS I/O and other traffic from your instance.

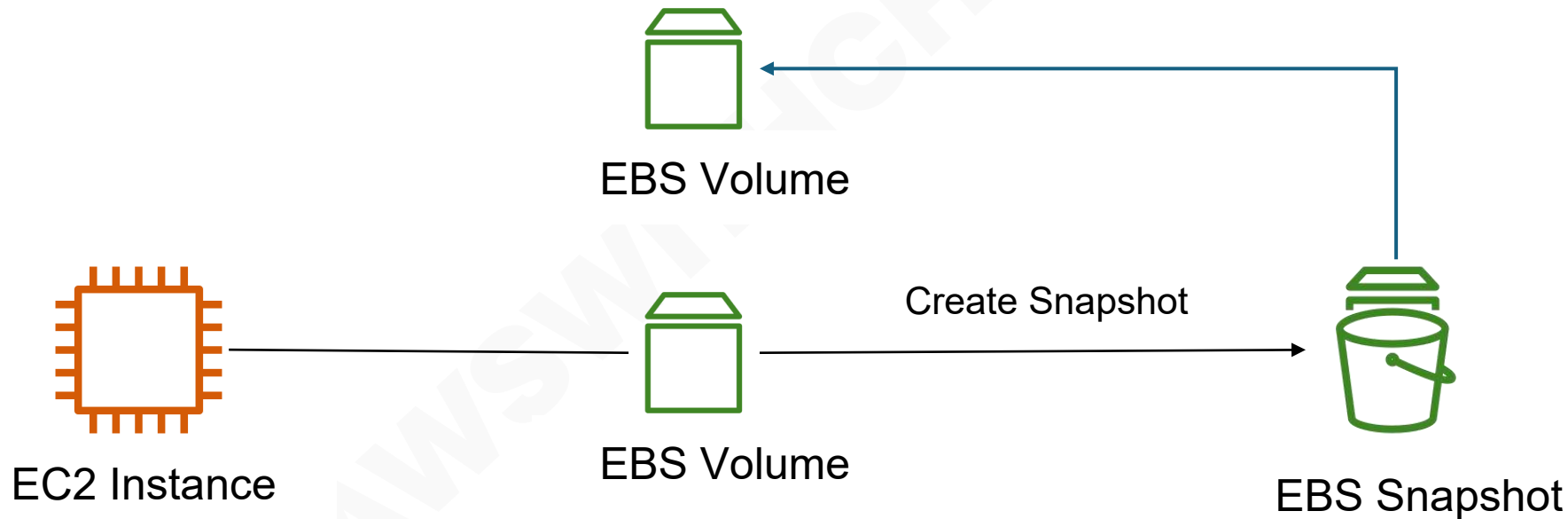
# EBS Snapshots

- Snapshots are point in time backup of EBS Volume.
- Snapshot are stored in S3 and are highly durable within a region.
- **Snapshots are incremental – One full snapshot + changed data only**
- Can be shared with other AWS Accounts and can be copied to other AWS Regions
- You can delete snapshots and retain it in Recycle Bin for a desired retention period
- **Amazon Data Lifecycle Manager (DLM)** automates EBS snapshot creation and retention.



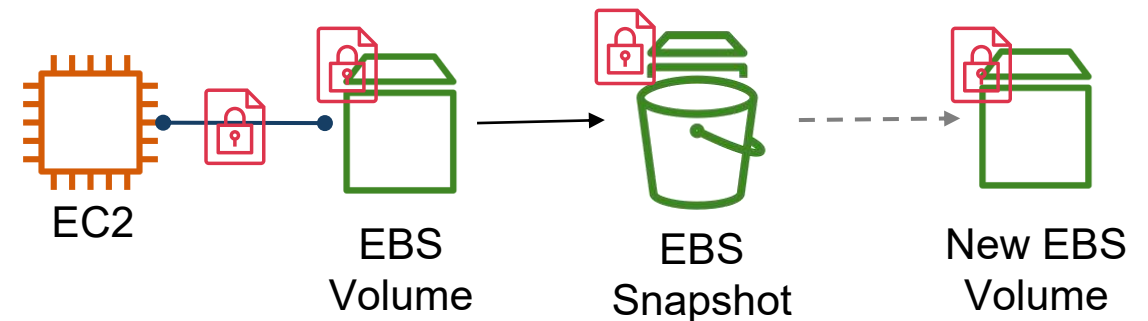
# EBS Fast Snapshot Restore

- Amazon EBS fast snapshot restore (FSR) enables you to create a volume from a snapshot that is fully initialized at creation.
- This eliminates the latency of I/O operations on a block when it is accessed for the first time.
- Volumes that are created using fast snapshot restore instantly deliver all of their provisioned performance.



# EBS Encryption

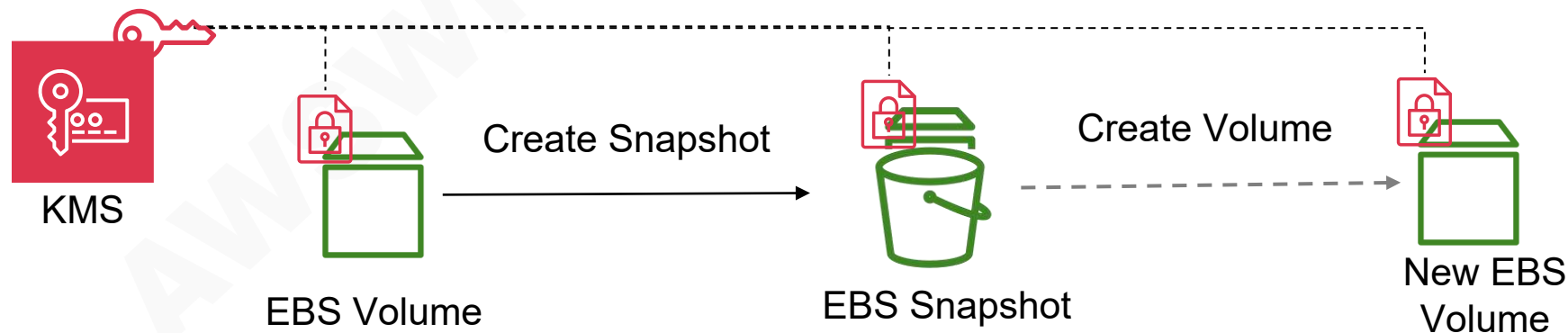
- Amazon EBS supports encryption using AWS KMS (Key Management System).
- You encrypt EBS volumes by enabling -
  1. Encryption by default. This is a region level setting. In every region a default KMS key is created with alias **aws/ebs**
  2. Manually when creating the volume. Uses customer managed Encryption (CMK) keys from KMS.
- When you create an encrypted EBS volume and attach it to a supported instance type, the following types of data are encrypted:
  - Data at rest inside the volume
  - All data moving between the volume and the instance
  - All snapshots created from the volume
  - All volumes created from those snapshots
- No performance impact for encrypted volumes.





# EBS Encryption - scenarios

- The snapshot of encrypted volumes are also encrypted and to create a new volume from the snapshot the access to encryption key is required.
- You can't directly encrypt existing unencrypted volumes or snapshots. To encrypt an unencrypted volume, create a snapshot of that volume, and then use the snapshot to create a new encrypted volume.
- If a Customer-Managed Key (CMK) is disabled, the encrypted EBS volume becomes temporarily inaccessible. If the CMK is deleted, it results in permanent data loss, as the data can no longer be decrypted.

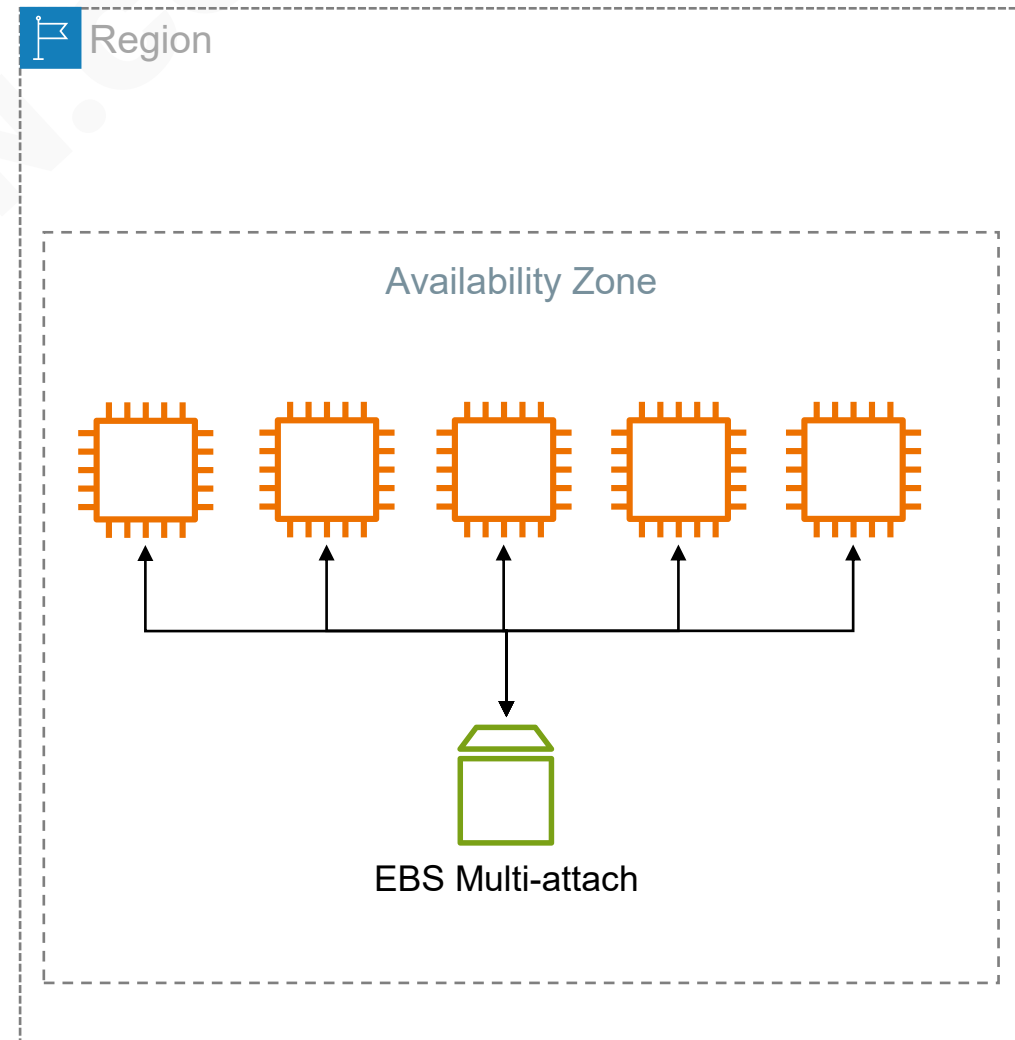


# EBS Multi Attach

- EBS supports a feature called Multi-Attach for io1 and io2 Block Express volumes (Provisioned IOPS)
- Multi-Attach allows a single EBS volume to be shared by up to 16 EC2 instances (Linux) within the **same Availability Zone**
- Each EC2 instance has full read/write access.
- Use case:
  - Highly available clustered applications (Oracle RAC, SAP)
  - Zero downtime applications
  - Analytics or rendering applications where nodes process shared input/output files in parallel.

## Key considerations:

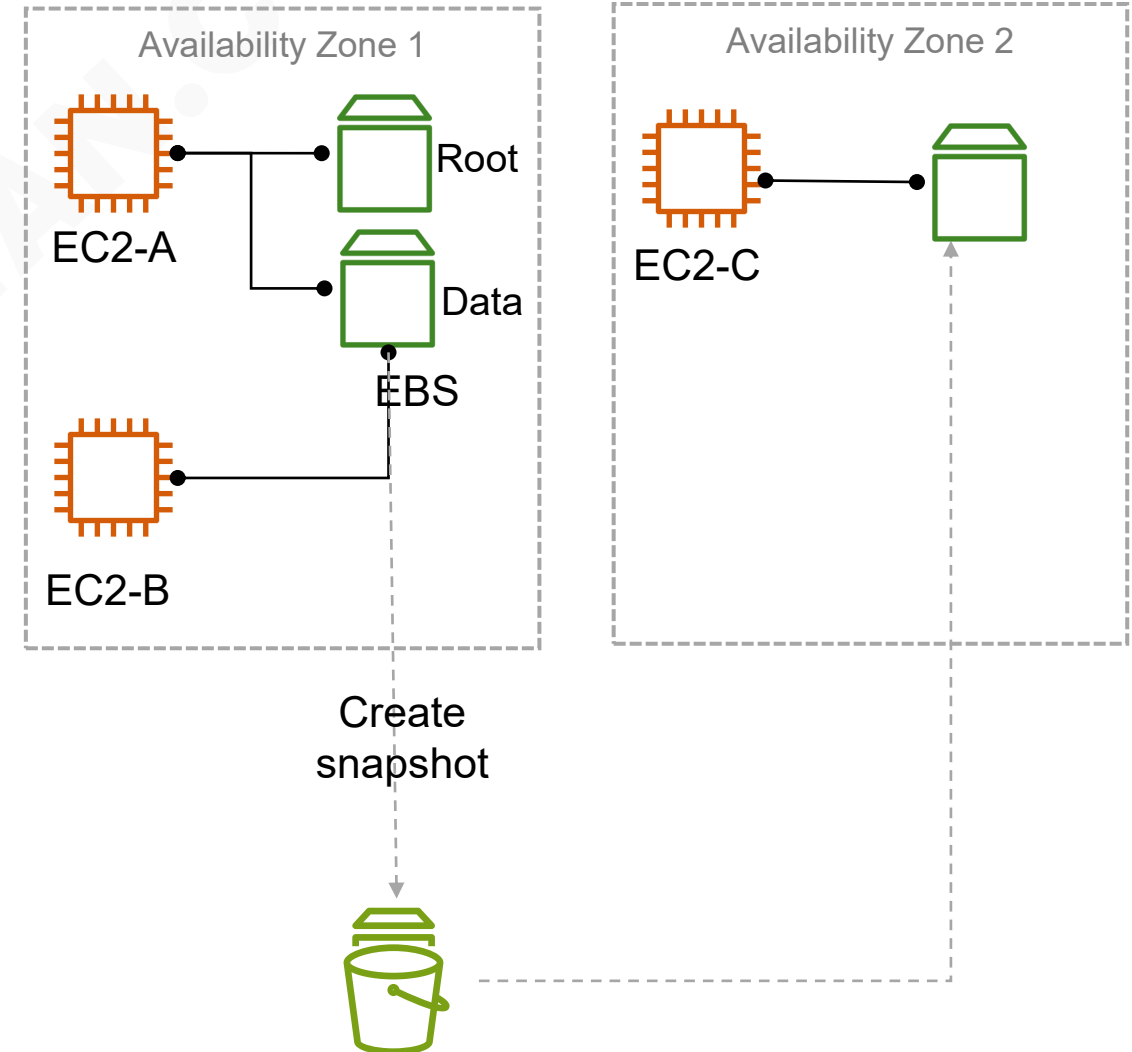
- Cannot be used as a EC2 boot volume.
- Applications must handle concurrent writes safely to maintain data consistency.



# EBS – Good to know

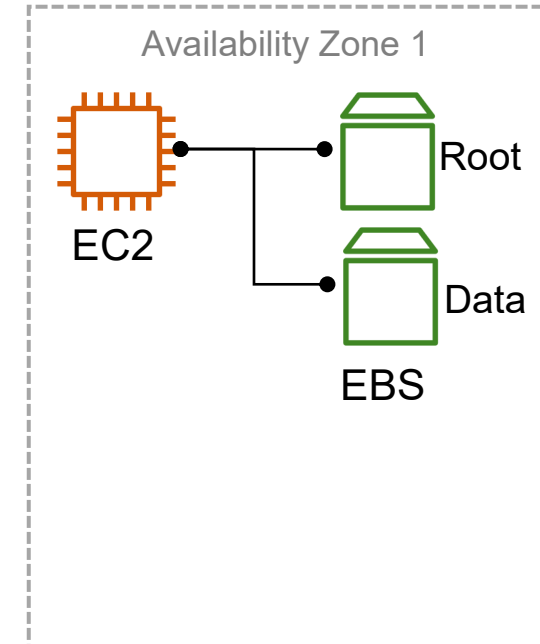
# EBS volume AZ considerations

- EBS volumes reside in given AZ
- EBS volumes are automatically replicated within an AZ for high availability.
- EBS volume can be detached and attached to another instance in the same AZ
- For moving EBS volume across the AZs, create a snapshot of volume and then create a new volume from the snapshot in another AZ



# Delete on Termination Attribute

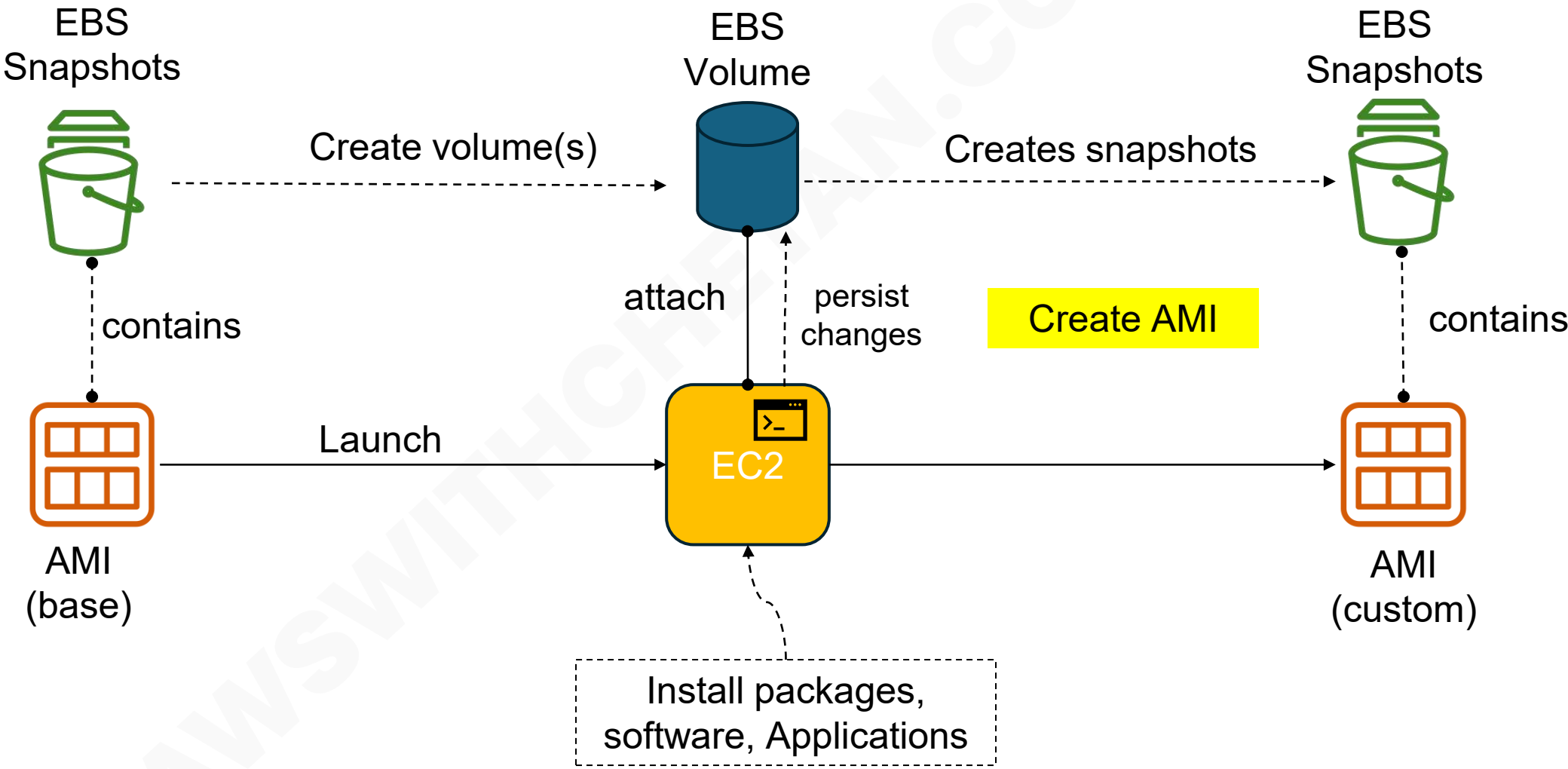
- EBS volume attribute **DeleteOnTermination** defines whether volume is deleted or retained when EC2 instance terminates
- By default, the root EBS volume is deleted (DeleteOnTermination = true)
- By default, any other (non-root) EBS volume is not deleted (DeleteOnTermination = false)
- This attribute value can be controlled by the user (AWS console / CLI)



| <input type="checkbox"/>            | Volume ID      | Device name | Volume size (GiB) | Attachment status | Attachment time           | Encrypted | KMS key ID | Delete on termination |
|-------------------------------------|----------------|-------------|-------------------|-------------------|---------------------------|-----------|------------|-----------------------|
| <input checked="" type="checkbox"/> | vol-██████████ | /dev/xvda   | 8                 | ✓ Attached        | 2024/07/27 21:58 GMT+5:30 | No        | –          | Yes                   |
| <input type="checkbox"/>            | vol-██████████ | /dev/sdb    | 20                | ✓ Attached        | 2024/07/27 21:58 GMT+5:30 | No        | –          | No                    |

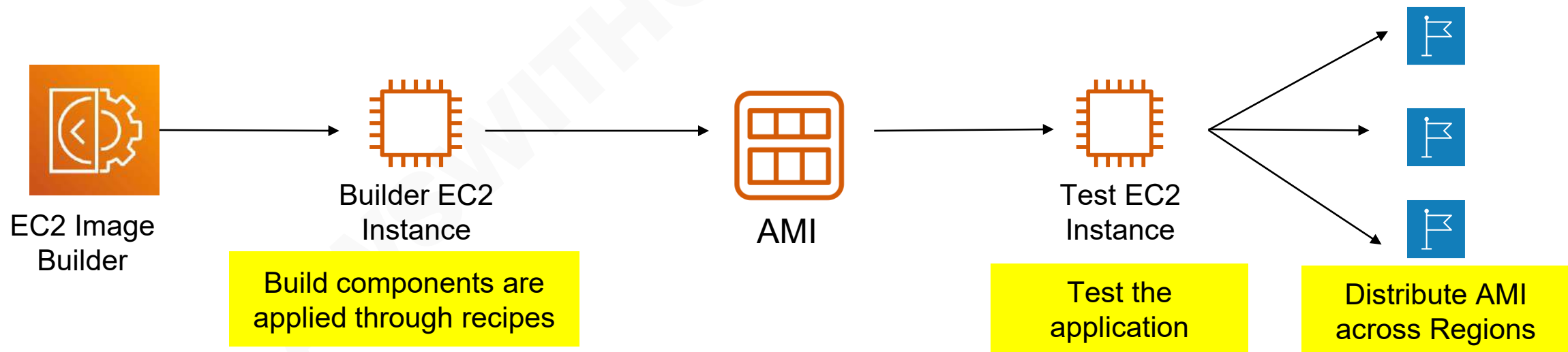
# EC2 Image Builder

# Process to create a custom AMI



# EC2 Image Builder

- Automate the creation, management, and deployment of AMIs
- Configure pipelines to automate updates and system patching for the images.
- Can be run on a schedule basis (weekly, whenever packages are updated).
- Works across AWS regions and AWS Accounts
- Free service



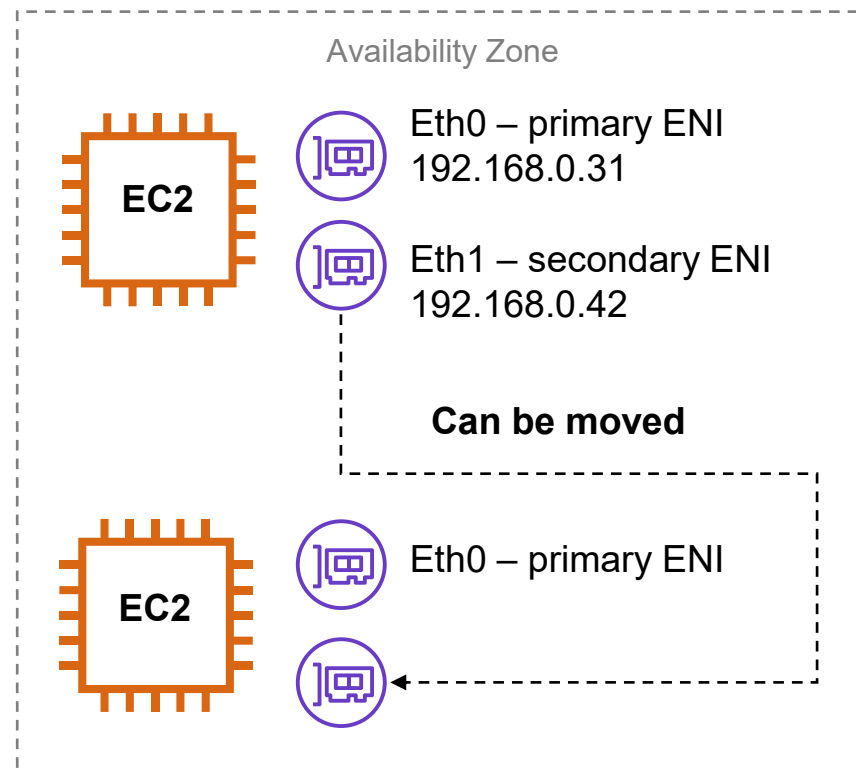




# Elastic Network Interface (ENI)

# Elastic Network Interfaces (ENI)

- Logical component in a VPC that represents a **virtual network card**
- EC2 instance has one primary ENI and can have multiple secondary ENIs depending on instance type
- The ENI can have the following attributes:
  - Primary private IPv4, one or more secondary IPv4
  - One Elastic IP (IPv4) per private IPv4
  - One Public IPv4
  - One or more security groups
  - A MAC address
- You can create ENI independently and attach or move them across EC2 instances
- ENIs are AZ-scoped. Can't move across AZs.
- Each ENI can have its own security group, IP, and routing.



# ENI Use cases

## High Availability / Failover within an AZ

- Move an ENI (and its private IP) from one instance to another to achieve quick failover.

## Static IP retention

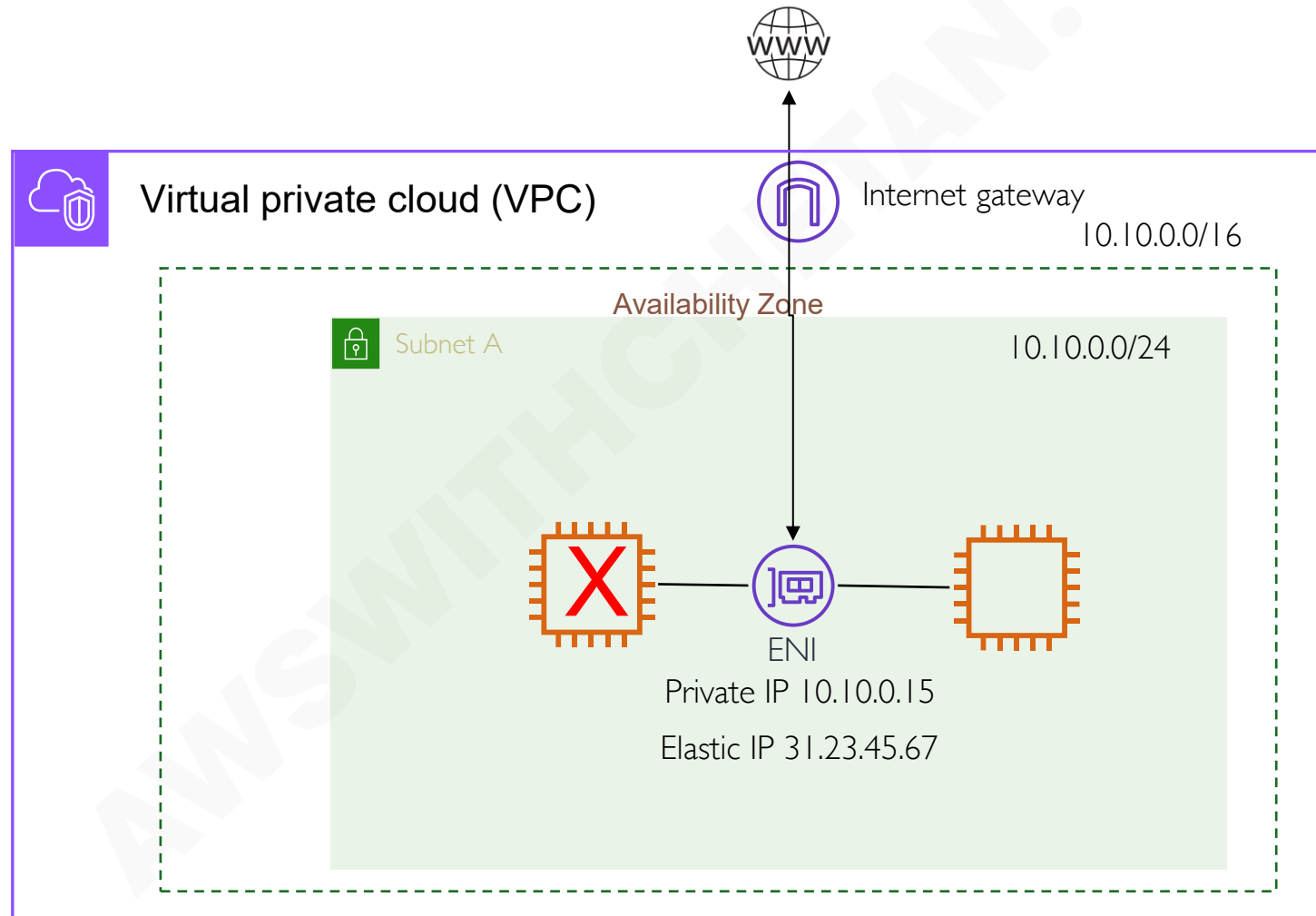
- Detach the ENI from a terminated or failed instance and attach it to a new one to retain the same private IP and MAC.
- Helps avoid DNS updates or reconfiguration.

## Multi-Network (Multi-Homed) Instances

- Attach multiple ENIs to a single EC2 instance, each connected to a different subnet or security group.
- Enables segregation of traffic (e.g., app network vs. database network).
- Assign a secondary ENI specifically for administration or monitoring traffic.

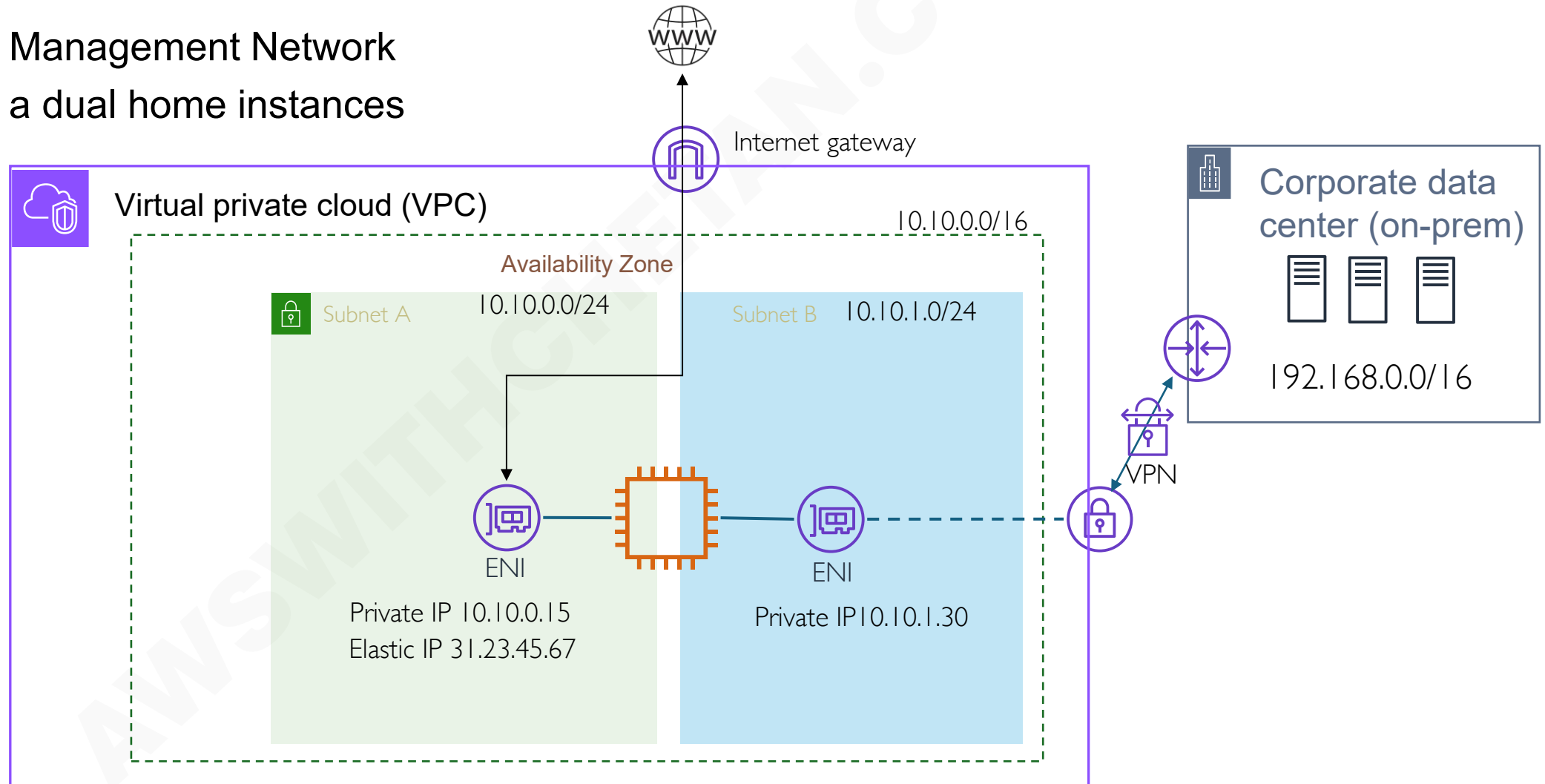
# ENI Use cases

- High Availability solution by attaching ENI to hot standby instance in case of failure



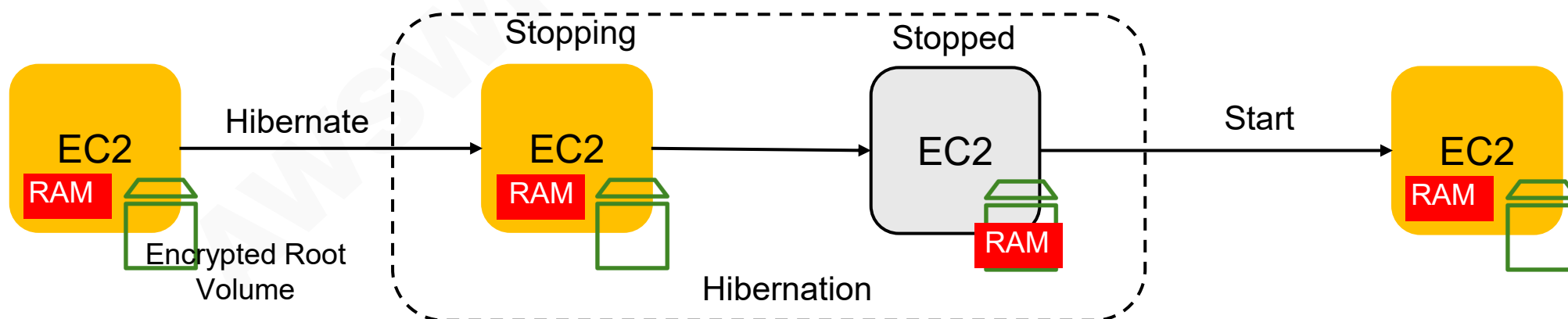
# ENI Use cases

- Creating Management Network
- Creating a dual home instances



# EC2 Hibernation

- EC2 supports hibernation (suspend-to-disk) for supported OS and AMI types
- Hibernation saves the contents from the instance memory (RAM) to EBS root volume.
- When the instance is started again:
  - The EBS root volume is restored to its previous state
  - The RAM contents are reloaded
  - The processes that were previously running on the instance are resumed
  - Previously attached data volumes are re-attached and the instance retains its instance ID
- **Use case:** Saves cost by not paying for compute while preserving RAM state for long-running applications e.g. analytics workload, VDI, development workstations etc.



# EC2 Instance metadata service (IMDSv2)

- IMDSv2 allows applications and users on an EC2 instance to retrieve metadata about the instance like instance ID, AMI ID, IPs, security groups, IAM role credentials.
- Used by applications or scripts for automation, configuration, and identity awareness.
- Service runs on 169.254.169.254 IPv4 address and [fd00:ec2::254] IPv6 address

```
> export TOKEN=$(curl -X PUT -H "X-aws-ec2-metadata-token-ttl-seconds: 300"  
http://169.254.169.254/latest/api/token)  
  
> curl http://169.254.169.254/latest/meta-data/ -H "X-aws-ec2-metadata-token: $TOKEN"
```

Remember this IP address for IMDS service: **169.254.169.254**

# EC2 Placements groups

- A placement group is a logical grouping of EC2 instances within an Availability Zone
- It lets you control how instances are placed on AWS infrastructure to meet performance, latency, compliance needs and avoiding system wide failures at hardware level
- AWS offers three placement strategies:
  - Cluster placement group
  - Spread placement group
  - Partition placement group





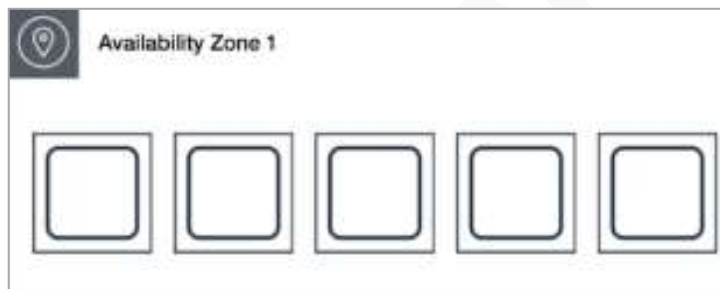
# EC2 Placement groups

## Cluster Placement group



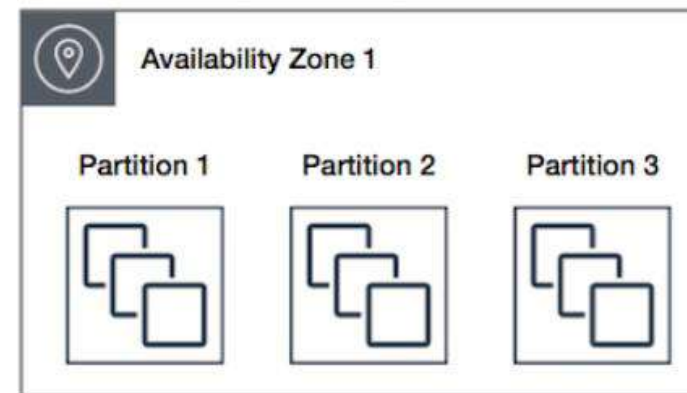
- Packs instances close together inside an AZ.
- Enables workloads to achieve the low-latency network performance for tightly-coupled node-to-node communication
- Ideal for high-performance computing (HPC) applications.

## Spread Placement group

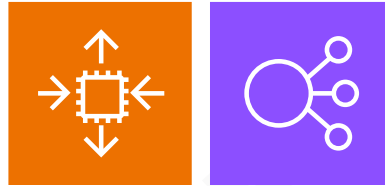


- Strictly places a small group of instances across distinct underlying hardware
- Reduces correlated failures.
- Ideal for applications that have a small number of critical instances to reduce the risk of simultaneous failures in the underlying hardware

## Partition Placement group

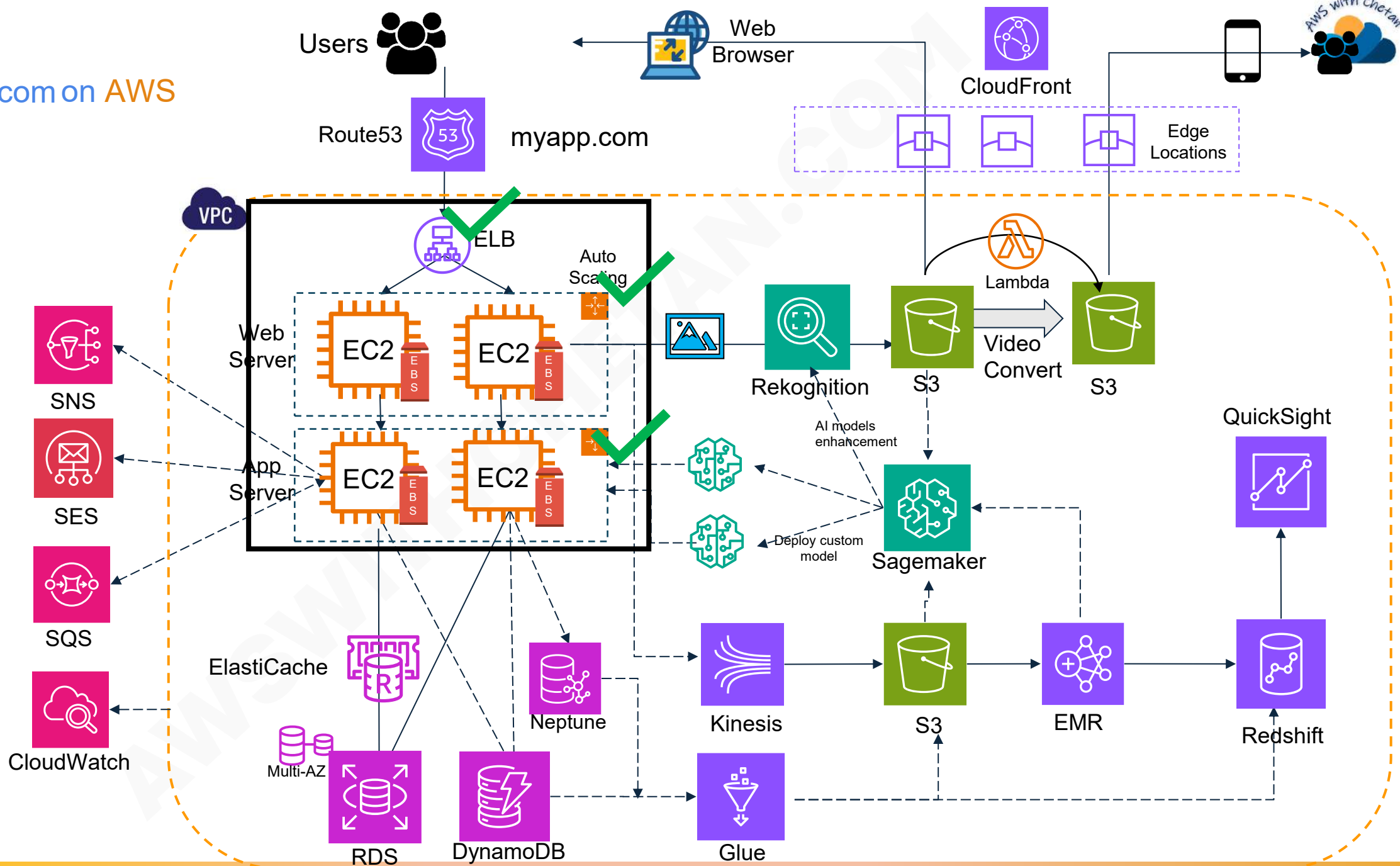


- Spreads instances across logical partitions
- Groups of instances in one partition do not share the underlying hardware with groups of instances in other partitions.
- Ideal for large distributed and replicated workloads, such as Hadoop, Cassandra, and Kafka.



# Load Balancing and Autoscaling

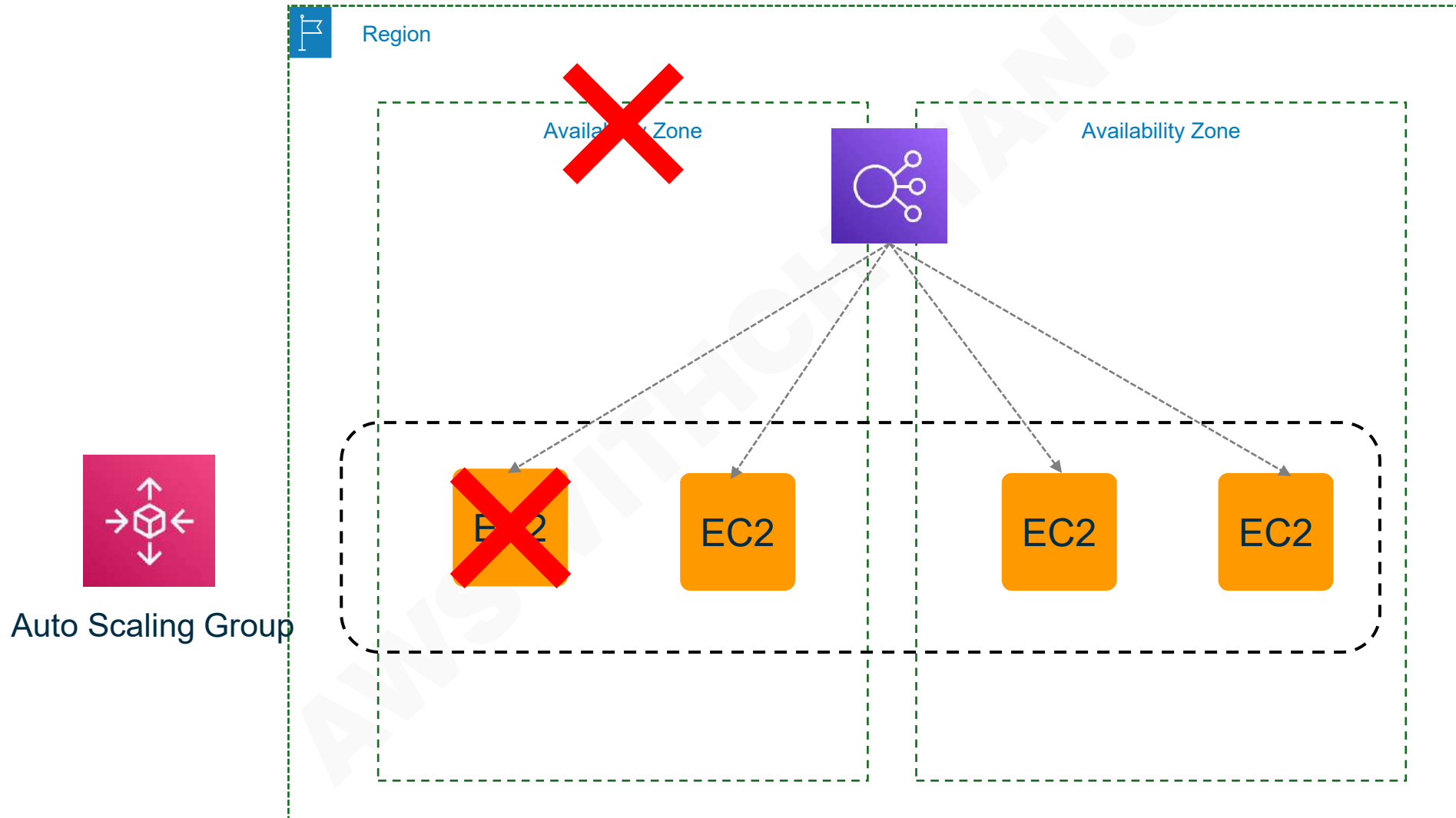
myapp.com on AWS



# High Availability and Scaling

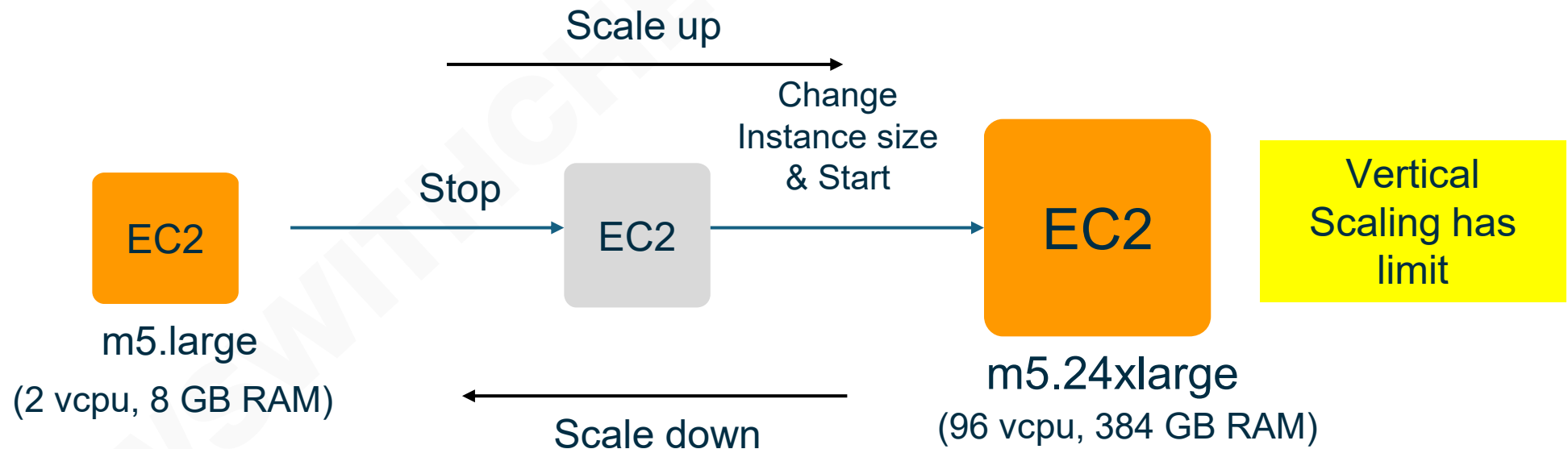
- **High Availability** means your application runs despite failure in some of the components and during increased traffic
- In AWS world High availability generally refers to using multiple AZs
- **Scalability** means ability to scale the capacity up or down as per demand
  - Horizontal scaling is used in distributed systems - Web applications, Big data processing
  - Vertical scaling is used in non-distributed systems - Relational databases
- High Availability and Scalability go hand-in-hand
- Use Elastic Load Balancer (ELB), Auto Scaling group (ASG) for achieving High Availability and Scalability

# High Availability



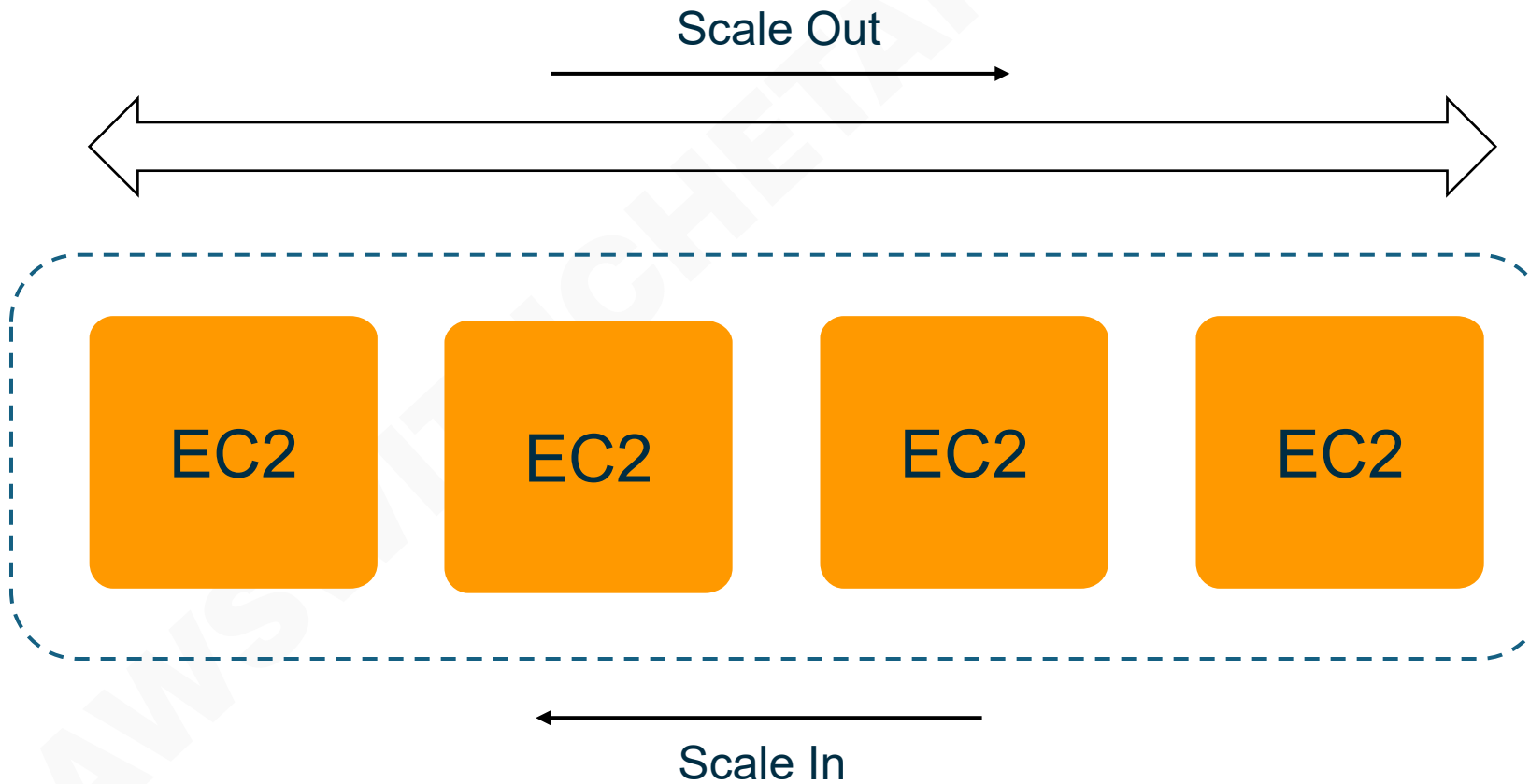
# Scaling

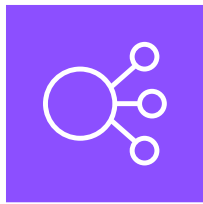
## Option 1: Vertical Scaling



# Scaling

## Option 2: Horizontal Scaling



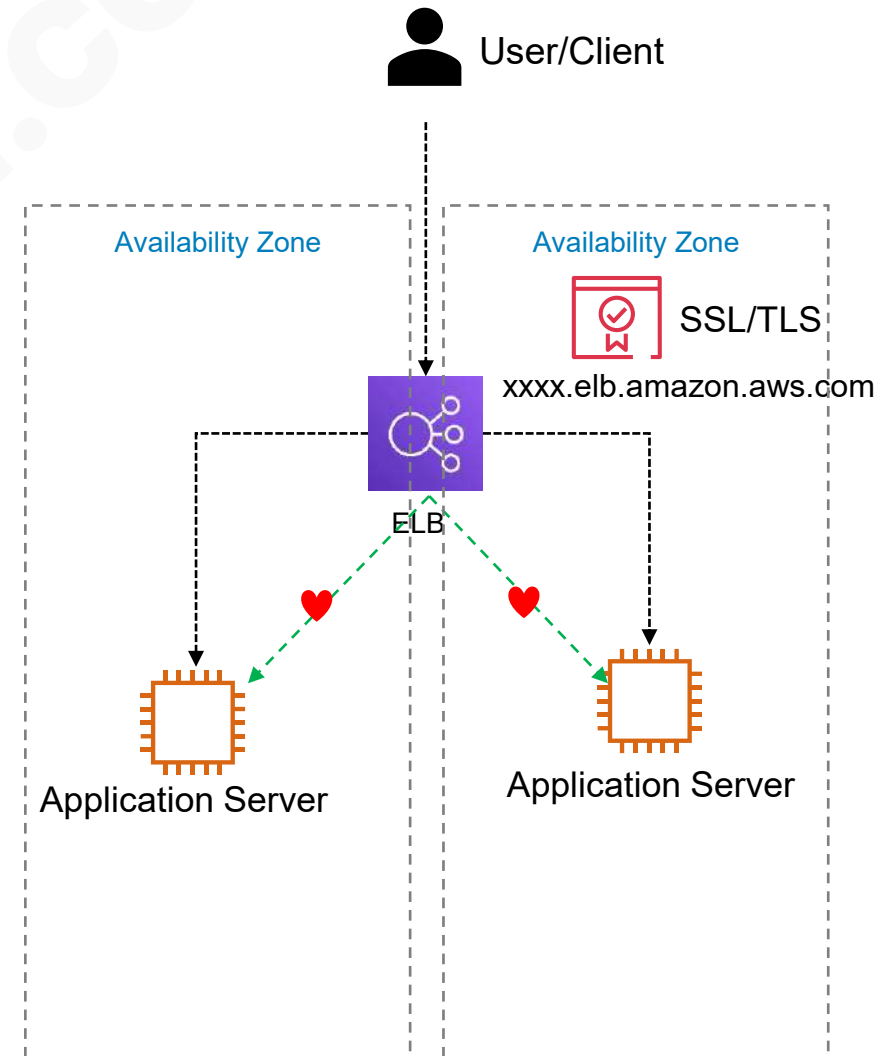


# Elastic Load Balancer



# AWS Elastic Load Balancer

- Load Balancer distributes incoming traffic to multiple downstream servers:
  - EC2 instances
  - Containers
  - IP addresses
  - Lambda functions
- It's a regional service. High availability across Availability zones (AZs)
- Supports protocols like HTTP, HTTPS, HTTP/2, TCP, UDP, gRPC
- Expose a single point of access (DNS) to your application
- Supports Dual-stack mode (IPv4 and IPv6 traffic)
- Seamlessly handle failures of downstream instances by performing health checks to the instances
- Provide SSL/TLS termination for your websites and web applications



# HTTPS and SSL/TLS

Certificate  
Authority (CA)



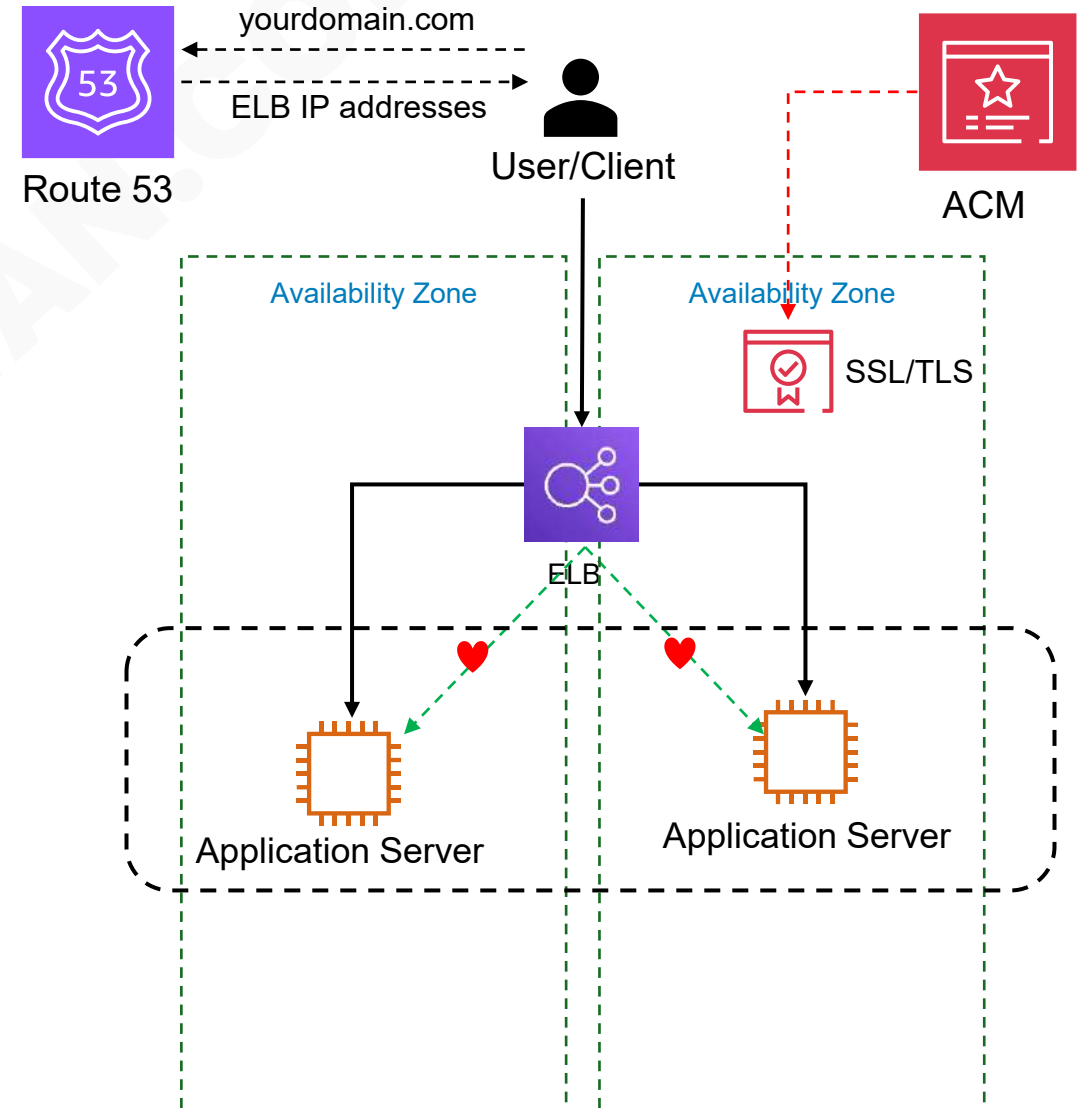
HTTP**S** Traffic



<https://youtu.be/cLYv4uSFJA8>

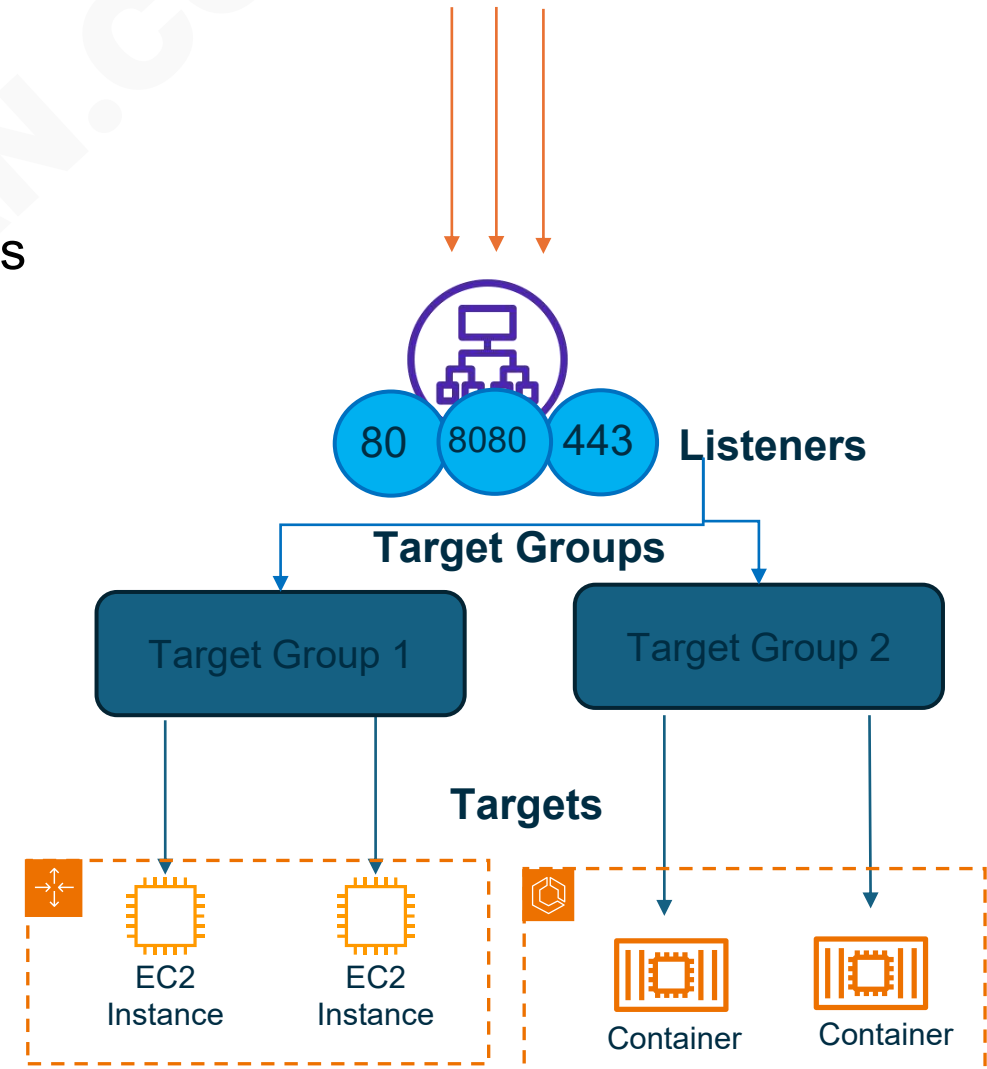
# Elastic Load Balancer

- ELB integrates with many other AWS services
  - Auto Scaling Groups
  - Amazon Route 53
  - AWS Certificate Manager (ACM)
  - Amazon ECS
  - Amazon CloudWatch
  - AWS WAF
  - AWS Global Accelerator



# Elastic Load Balancer components

- **Listener:** Supports multiple listeners on different ports
- **Target Group:** group of targets
- **Target:** EC2 instance, Container, IP , Lambda functions



# Security Groups



Load Balancer Security Group (sg-0bc494c71e6cf1896):

| Type ▾ | Protocol ▾ | Port range ▾ | Source ▾  | Description |
|--------|------------|--------------|-----------|-------------|
| HTTP   | TCP        | 80           | 0.0.0.0/0 | Allow HTTP  |
| HTTPS  | TCP        | 443          | 0.0.0.0/0 | Allow HTTPS |

Application Security Group: Allow traffic only from Load Balancer

| Type ▾ | Protocol ▾ | Port range ▾ | Source ▾                      | Description            |
|--------|------------|--------------|-------------------------------|------------------------|
| HTTP   | TCP        | 80           | sg-0bc494c71e6cf1896 / ALB-SG | Allow traffic from ALB |

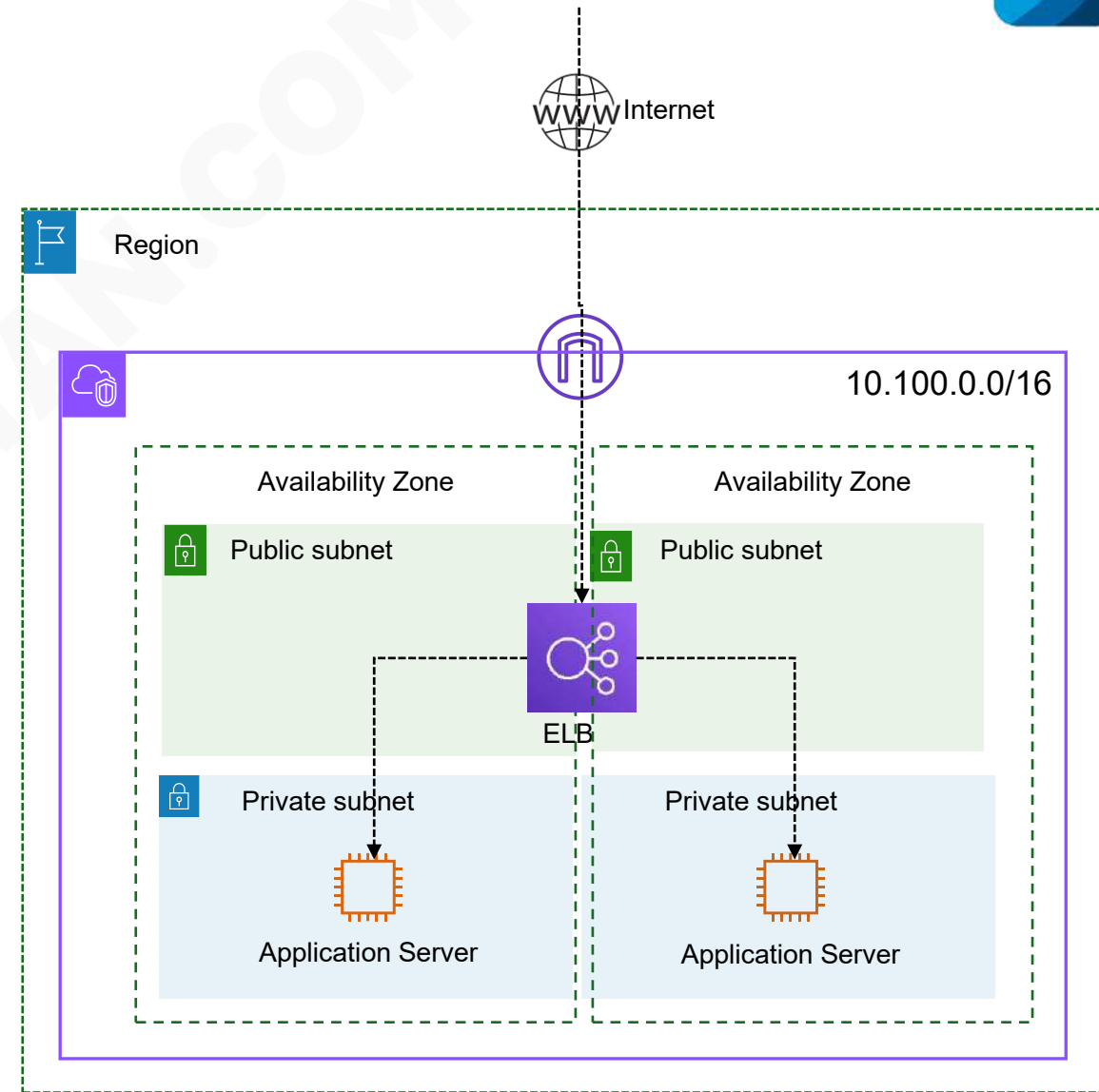
# Load Balancer types

# Types of Elastic Load Balancers

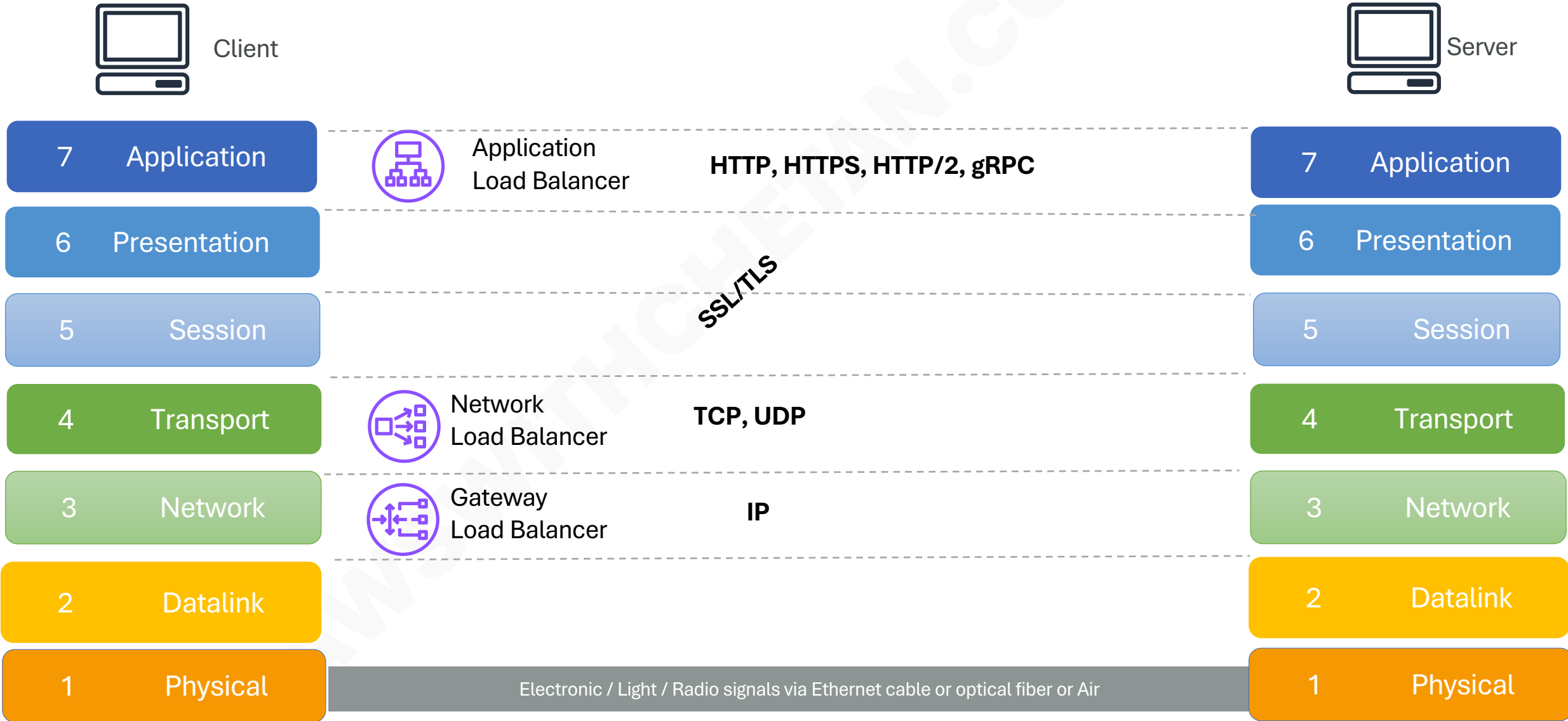
AWS offers following types of Load Balancers

1. Application Load Balancer (ALB) – Layer 7  
HTTP, HTTPS, WebSocket, HTTP/2, gRPC
2. Network Load Balancer (NLB) – Layer 4  
TCP, TLS (secure TCP), UDP
3. Gateway Load Balancer (GWLB) – Layer 3  
GENEVE Protocol on IP
4. Classic Load Balancer (CLB) – Layer 4 & 7  
HTTP, HTTPS, TCP, SSL/TLS (secure TCP)

Legacy



# OSI Network Layers



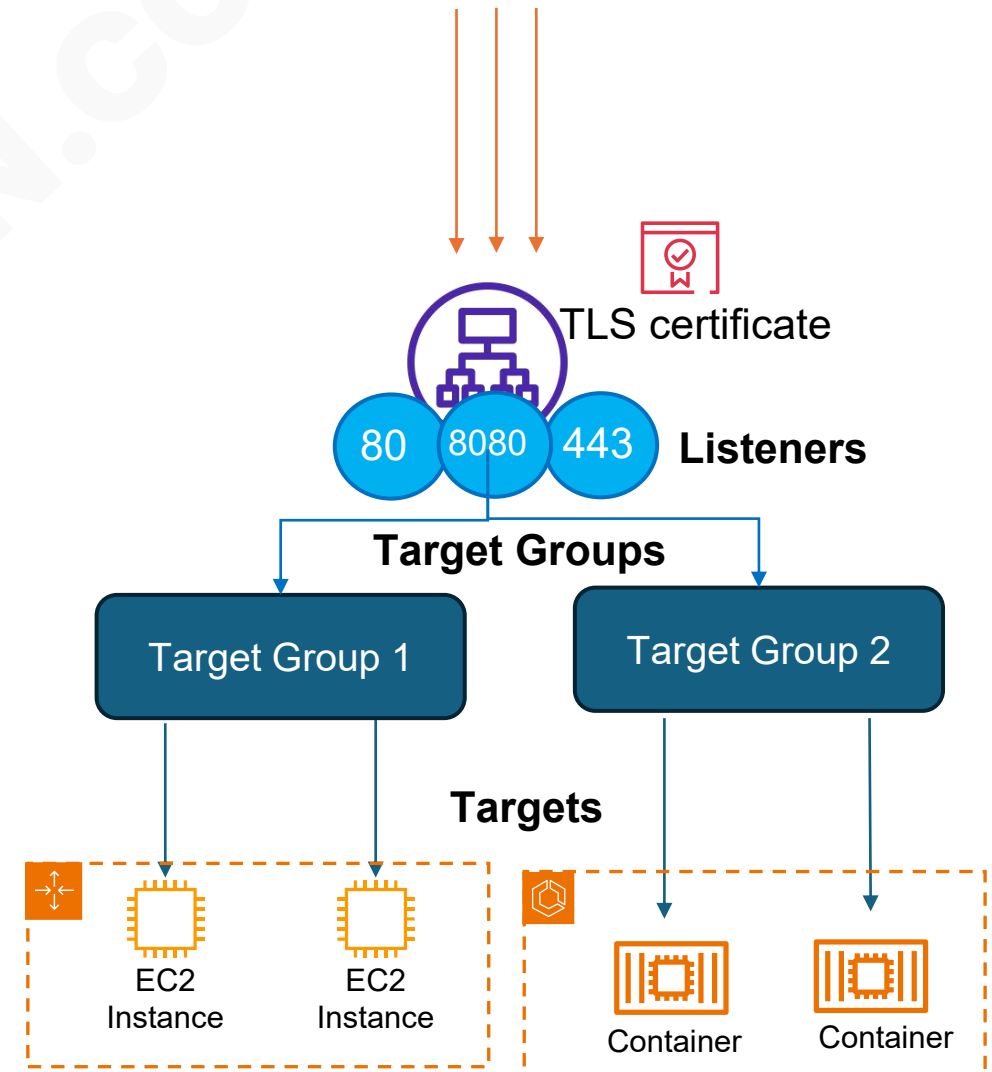




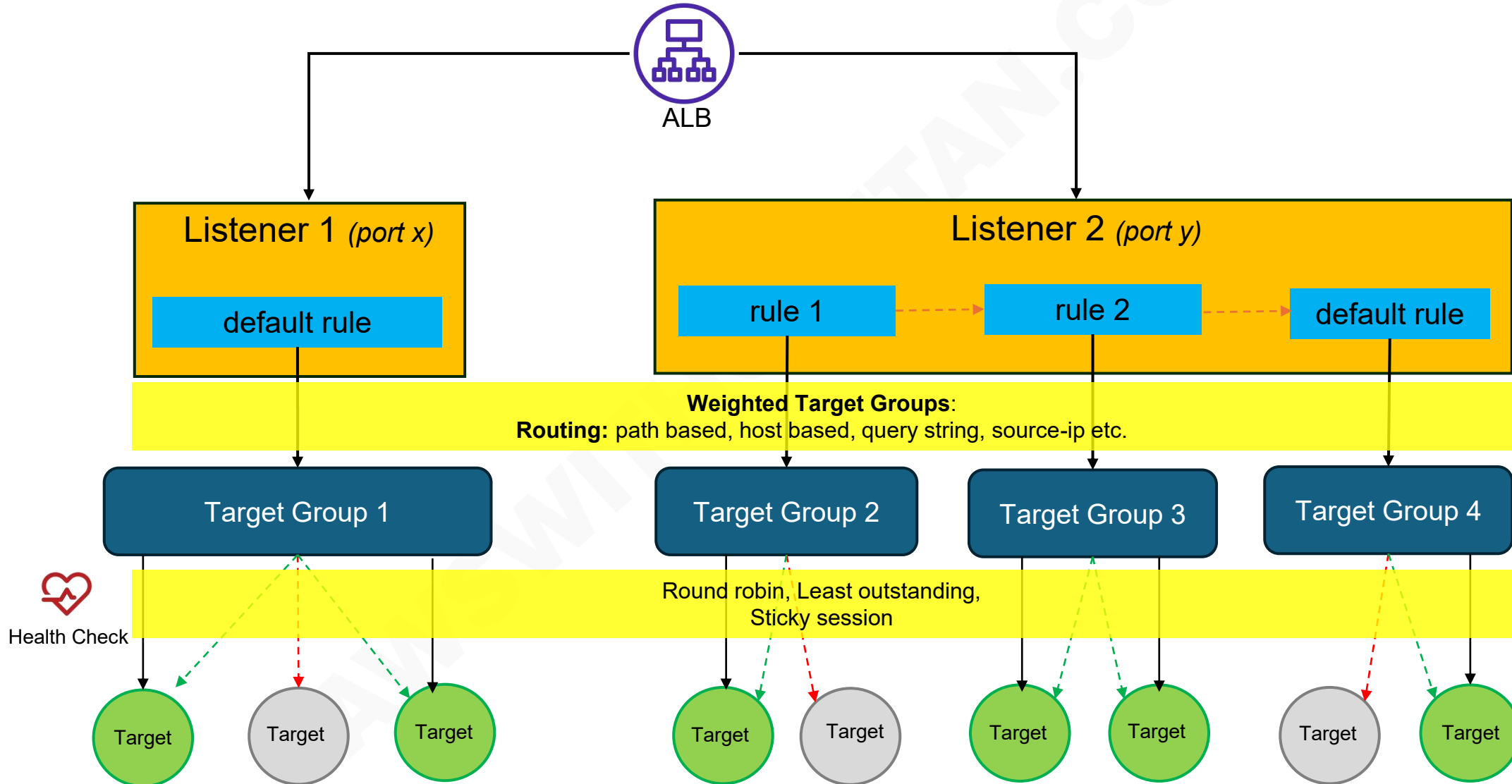
# Application Load Balancer

# Application Load Balancer (ALB)

- Operates at Layer 7
- Supported protocols HTTP, HTTPS, WebSocket, HTTP/2 and gRPC
- Supports multiple listeners e.g. http, https, custom port
- Load balancing to multiple Target groups which contains targets across EC2 instances, container tasks, IP addresses & Lambda functions
- Supports SSL/TLS termination
- Supports Target Group Weighting (Blue/green, A/B testing)
- Supports authentication using Amazon Cognito and federation (AD, OIDC, SAML etc.)
- Supports request routing - Host based, URL/path based, source-ip, http-header, query-string

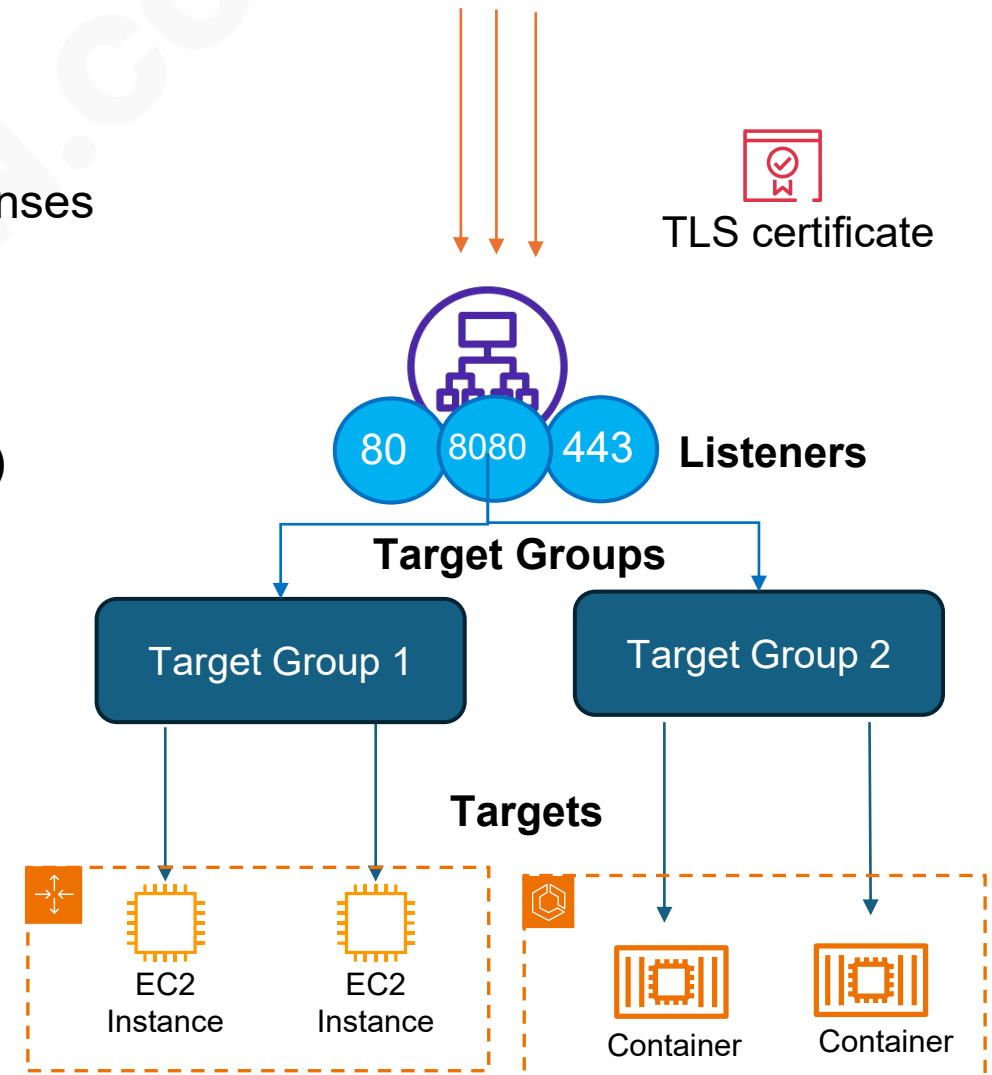


# ALB -> Listener -> Rules -> Target group -> Targets



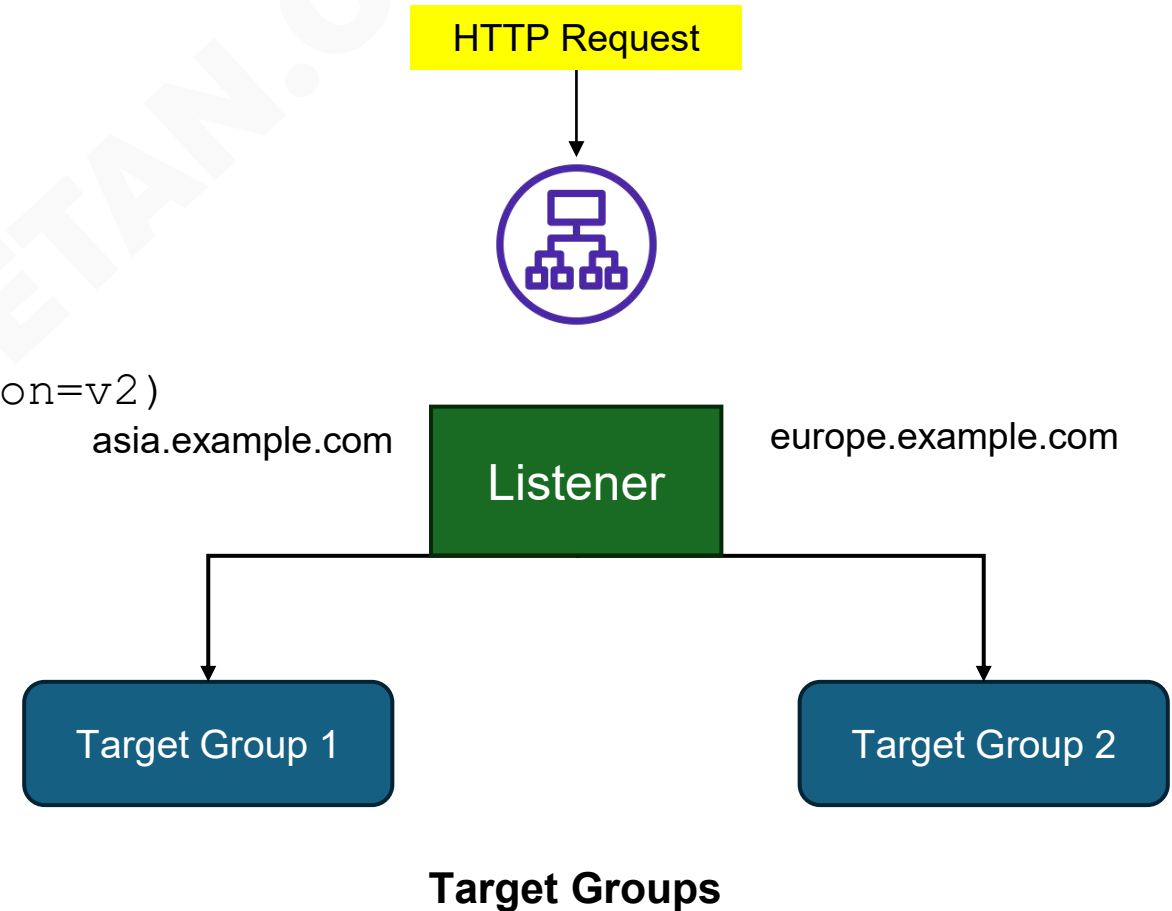
# Application Load Balancer – Advanced Features

- HTTP to HTTPS redirection
- Support for returning custom HTTP responses or fixed responses
- Sticky sessions
  - Duration based cookies (Cookie name: AWSALB)
  - Application based cookies (Cookie name: AWSALBAPP)
- Connection Draining / deregistration delay (default: 300 sec)
- Dual Stack DNS – Supports both IPv4 and IPv6 traffic
- Dynamic port mapping for Container targets
- Mutual TLS Authentication (X.509 certificates)
- Preserves Client IP: X-Forwarded-For
- Request Tracing: X-Amzn-Trace-Id



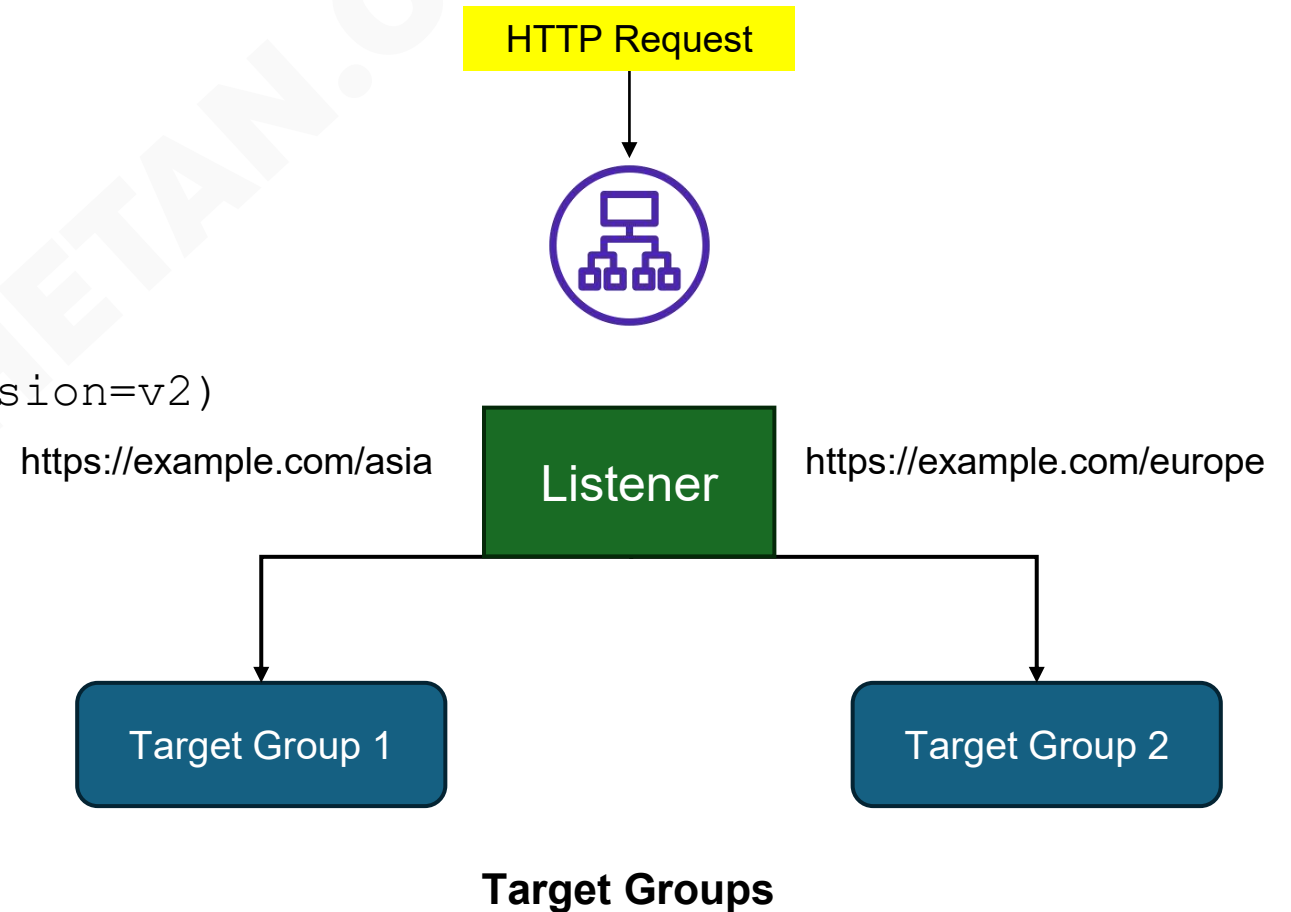
# Application Load Balancer – Advanced Features

- Content Based Routing
  - Host-based routing (`api.example.com`)
  - Path-based routing (`/images/*`)
  - HTTP method based (`GET`, `POST`)
  - Query string parameter-based (`/app?version=v2`)
  - Source IP CIDR-based



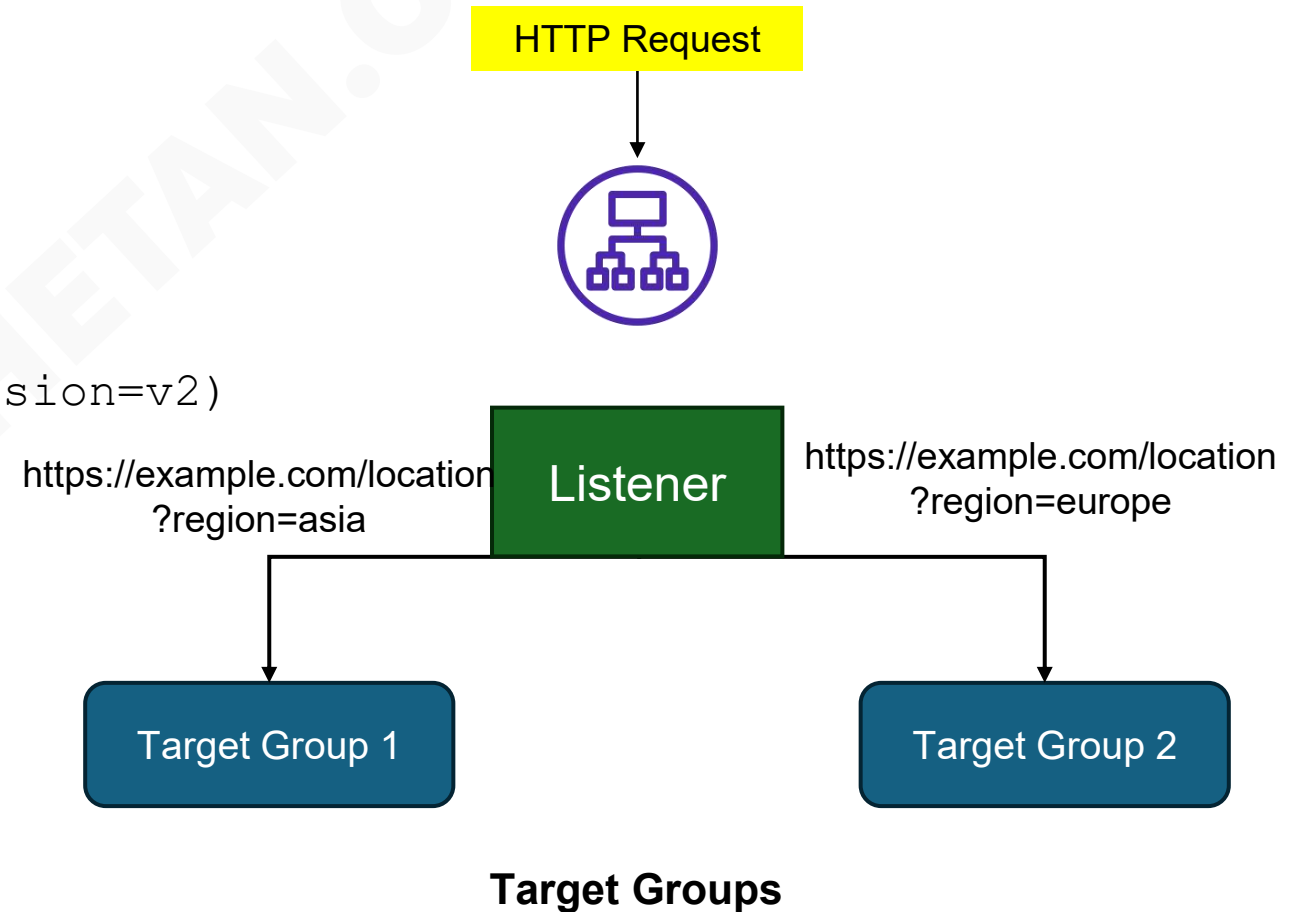
# Application Load Balancer – Advanced Features

- Content Based Routing
  - Host-based routing (`api.example.com`)
  - Path-based routing (`/images/*`)
  - HTTP method based (`GET`, `POST`)
  - Query string parameter-based (`/app?version=v2`)
  - Source IP CIDR-based



# Application Load Balancer – Advanced Features

- Content Based Routing
  - Host-based routing (`api.example.com`)
  - Path-based routing (`/images/*`)
  - HTTP method based (`GET`, `POST`)
  - Query string parameter-based (`/app?version=v2`)
  - Source IP CIDR-based



# Application Load Balancer – Advanced Features

- Operates at Layer 7 (Application Layer)
- Supported protocols HTTP, HTTPS, WebSocket, HTTP/2 and gRPC
- HTTP to HTTPS redirection
- Support for returning custom HTTP responses or fixed responses
- Sticky sessions - Duration based (Cookie: AWSALB) , Application based (Cookie: AWSALBAPP)
- Connection Draining / deregistration delay (default: 300 sec)
- Dual Stack DNS – Supports both IPv4 and IPv6 traffic
- Dynamic port mapping for Container targets
- Mutual TLS Authentication (X.509 certificates)
- Preserves Client IP: X-Forwarded-For
- Request Tracing: X-Amzn-Trace-Id





# ALB Exercises

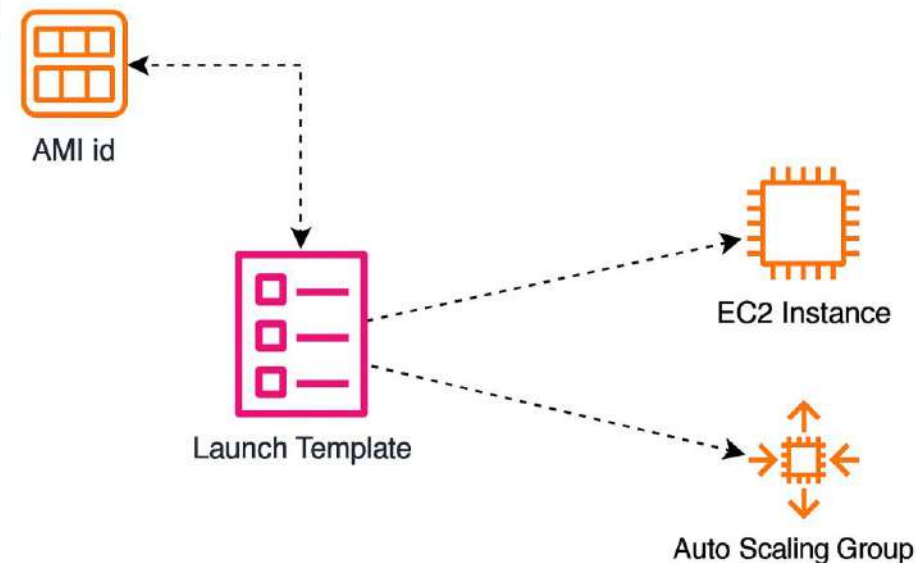
# Pre-requisites

- Public Domain name and Route 53 – Public Hosted Zone
- EC2 Launch Template

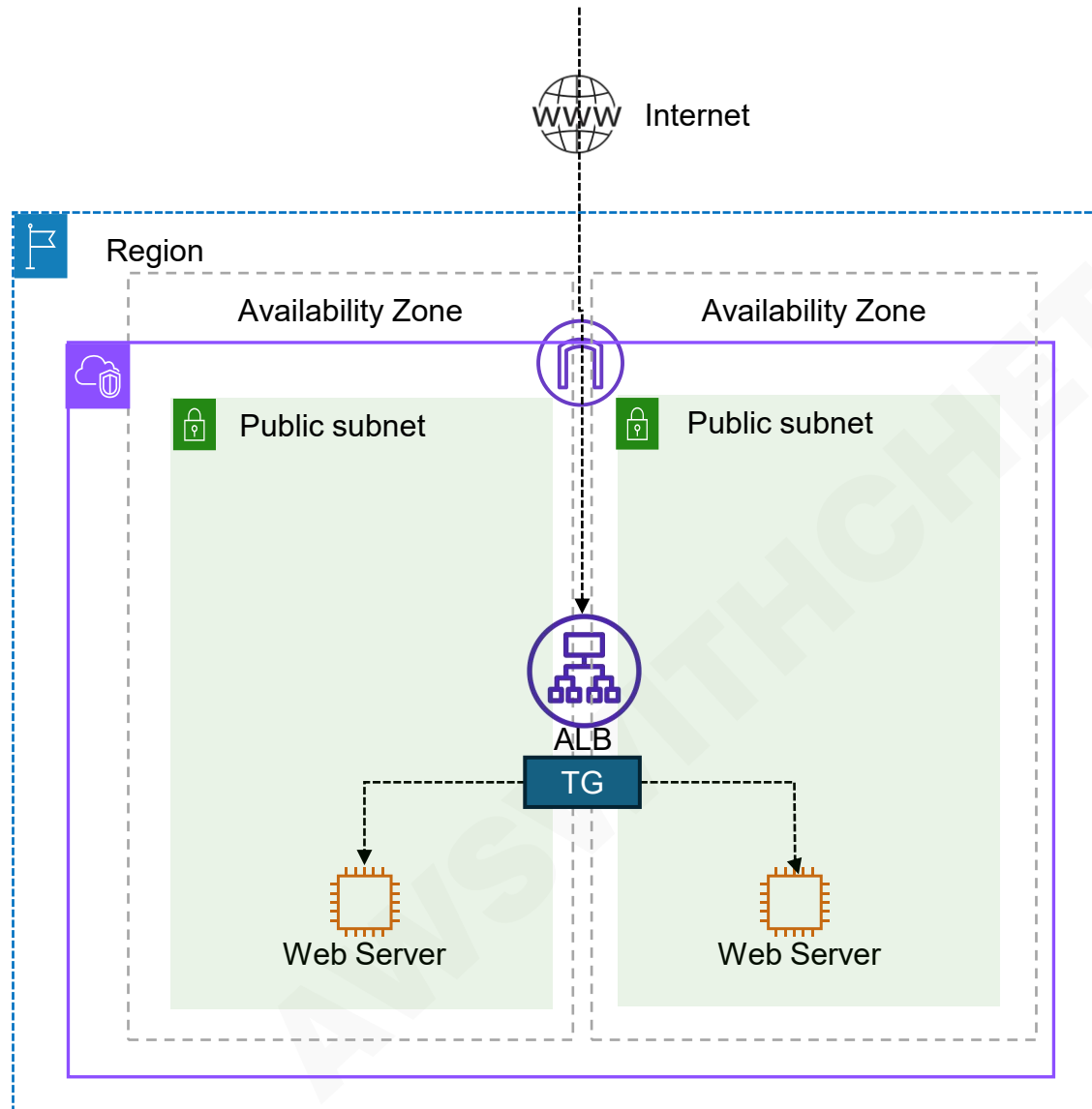
AWSWITHCHETAN.COM

# EC2 Launch Templates

- An easy way to define the settings required to launch EC2 instances
- We can define EC2 launch parameters like
  - Amazon Machine Image (AMI) ID,
  - Instance type
  - Key pair
  - Security groups
  - User data
  - And other configuration parameters.
- Supports versioning



# Launching EC2 instances



- Default VPC
- Different Subnets (for choosing different AZs)
- Same Security Group
- Same SSH Key-pair
- Different website content (User data)

# Exercise – Create EC2 launch templates

- Let's first create a Security group so that we can reuse it (Optional) and note down default VPC ID.
- Now let's create a Launch Templates for Webservers
  - We want different index.html for both the web servers. Hence, we will create 2 separate launch templates using following User data scripts
  - Make sure to use different AZ (subnets)

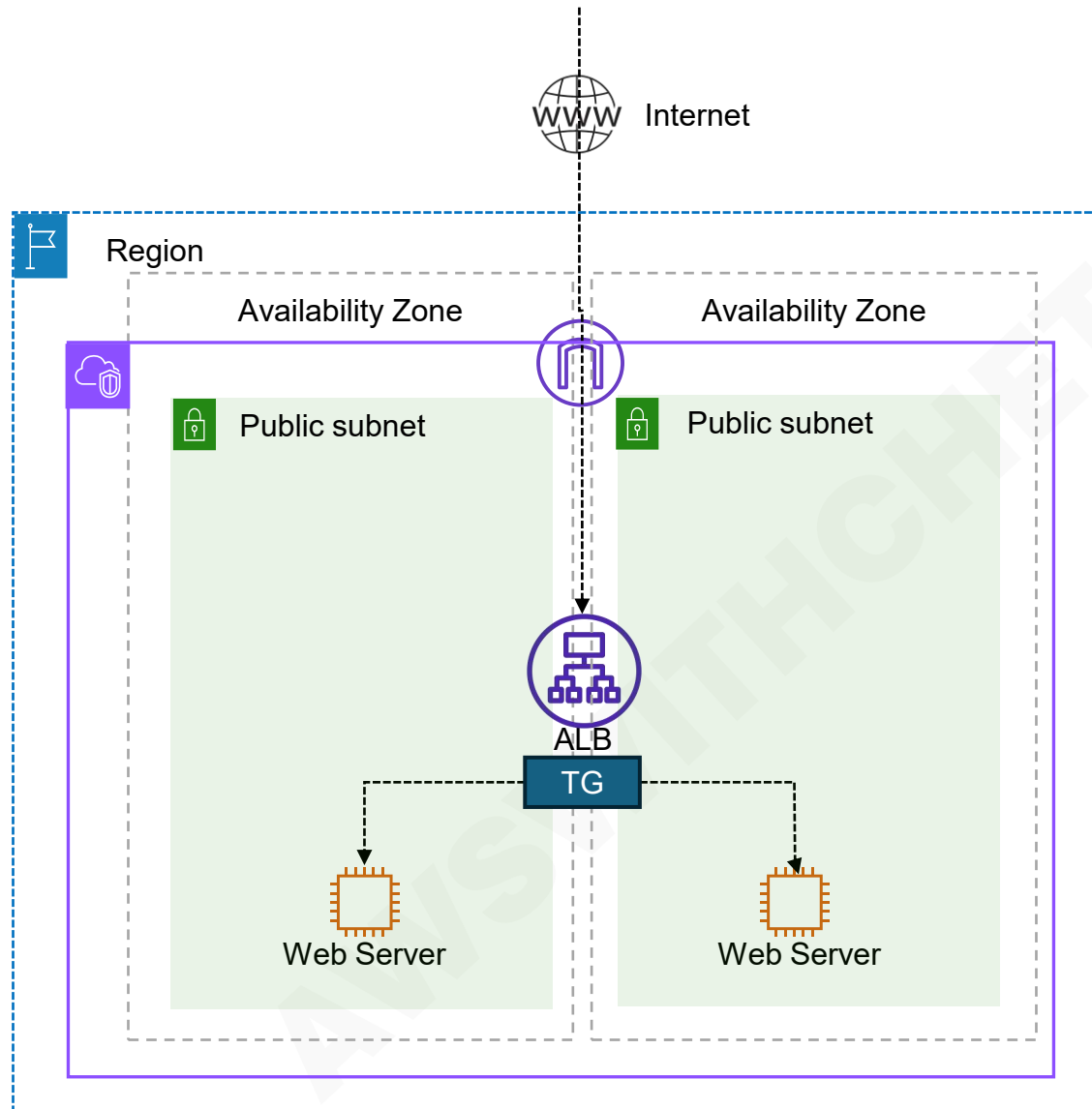
```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>You have reached a website for Asia</h1>" > /var/www/html/index.html
```

Launch Template 1 – User data

```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>You have reached a website for Europe</h1>" > /var/www/html/index.html
```

Launch Template 2 – User data

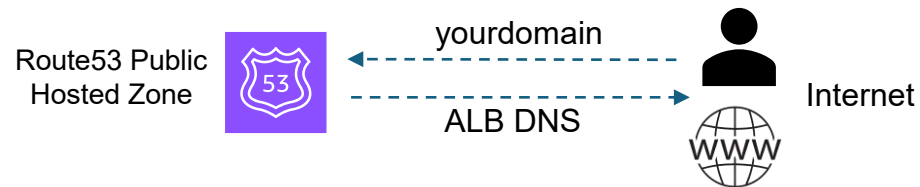
# Exercise – ALB with two backend EC2 instances



## High level steps

- 1 Launch Web server 1 and Web server 2 using the corresponding Launch Templates.
- 2 Create ALB Target group and add both EC2 instances. Configure health check on port 80 with /index.html
- 3 Create ALB with HTTP listener and forward the traffic to Target group. Open ALB Security group for HTTP traffic.
- 4 Update EC2 security group to allow HTTP traffic from ALB security group
- 5 Access ALB over the browser using AWS provided ALB DNS. Hit multiple times.

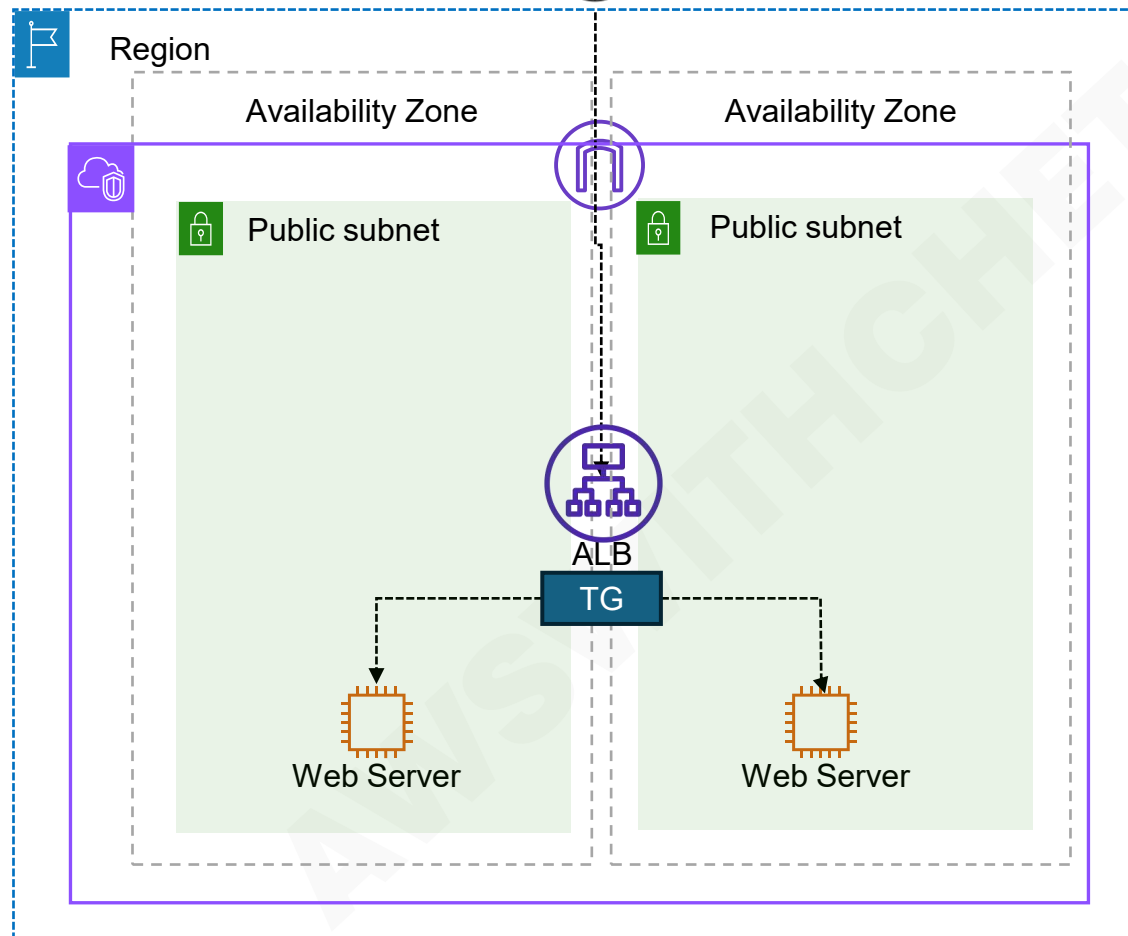
# Exercise – ALB with custom domain name



## Continuing with earlier setup

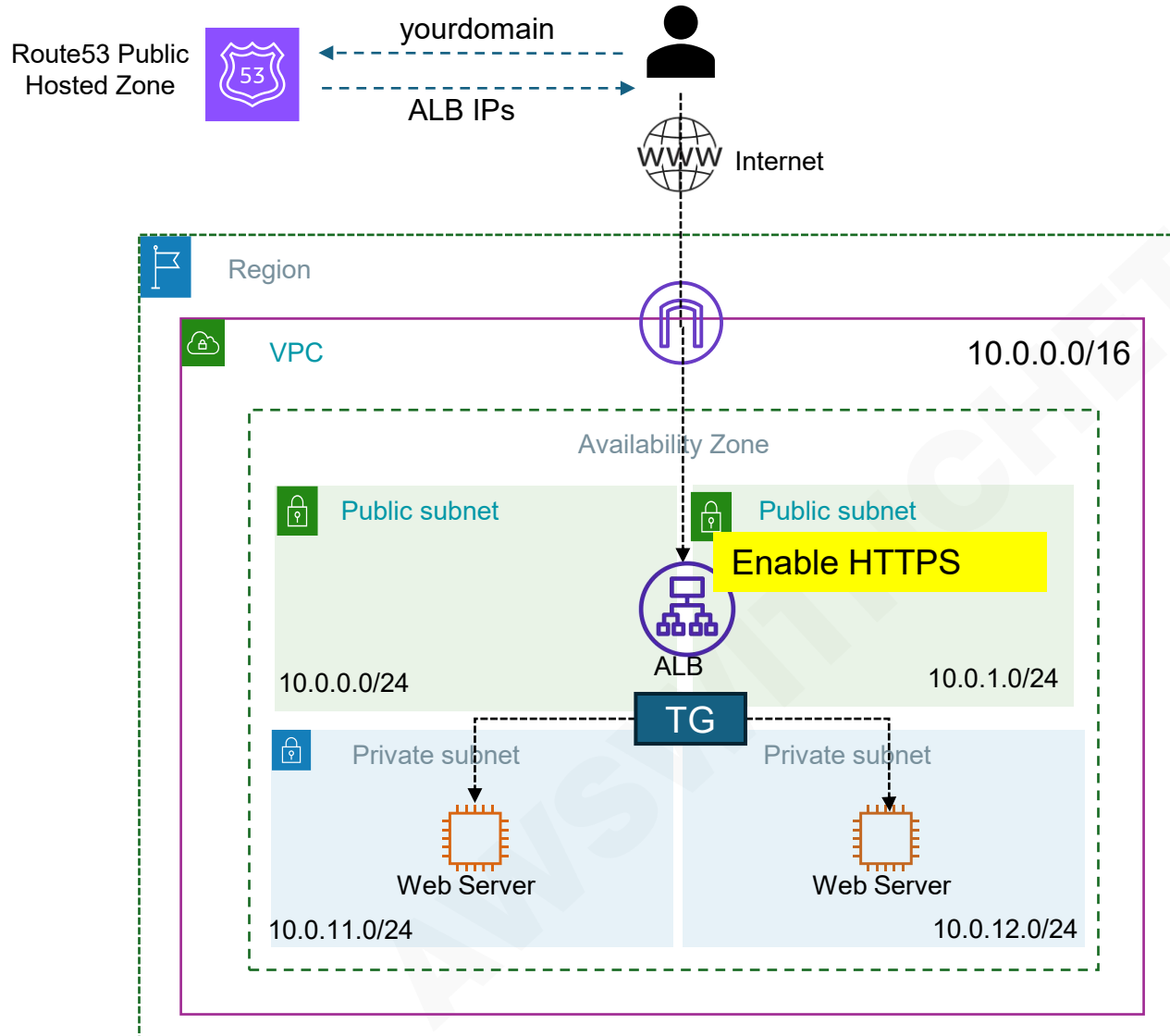
### Pre-requisite for this exercise:

- You should have your public domain-name and DNS should be pointing to Route53 public hosted zone
- Refer Labs prerequisites section if you haven't done this earlier.



- 1 In Route53 Public hosted zone create an A (Alias) record and point it to ALB DNS
- 2 Wait for some time and access application using your custom domain name

# Exercise – Enable HTTPS (TLS termination)

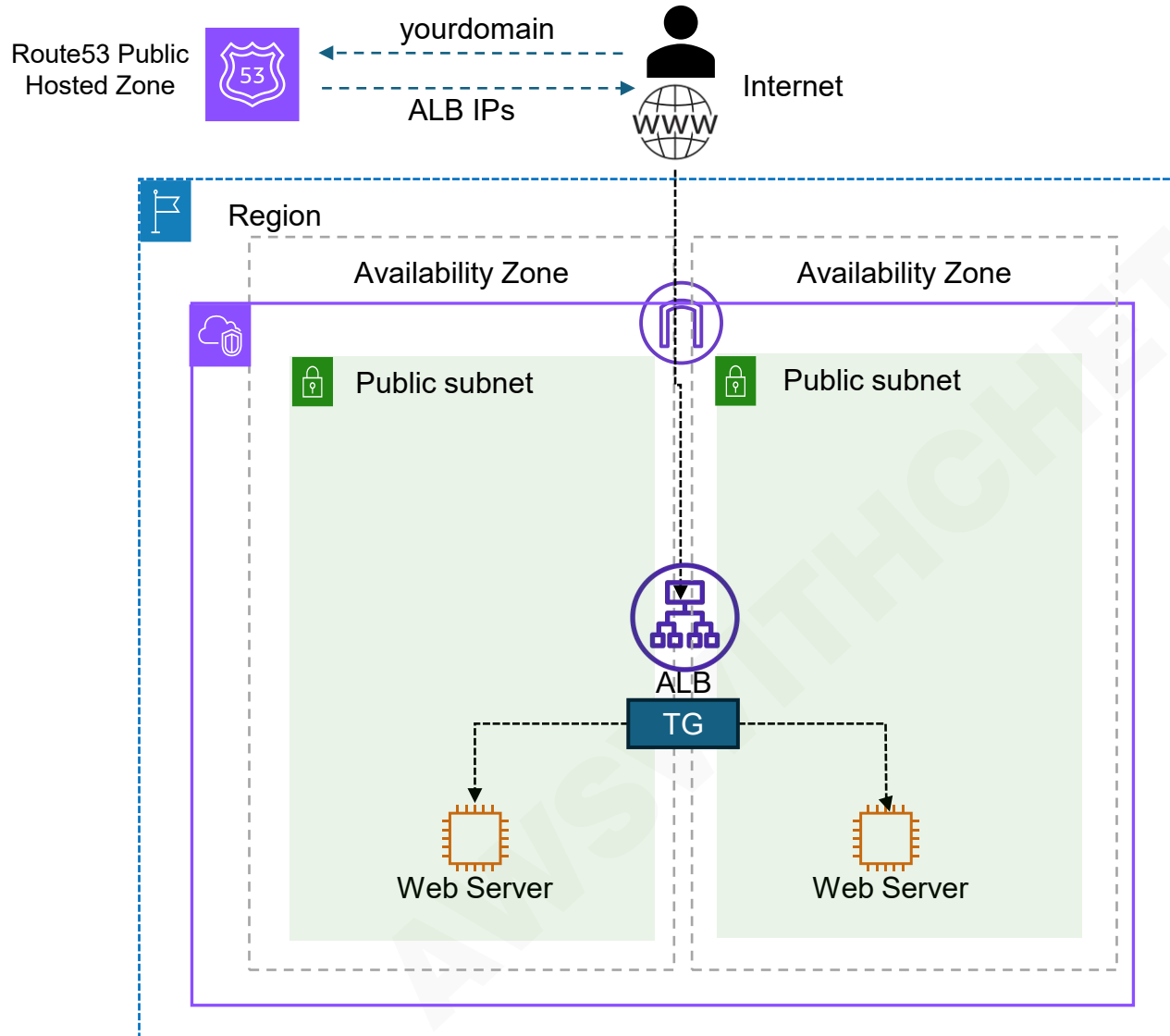


## Continuing with earlier setup

- 1 Get TLS certificate from Amazon ACM for your domain name. Validate the domain ownership through ACM portal.
- 2 Modify ALB listener to HTTPS (443) and associate TLS Certificate. Forward traffic to same target group as earlier.
- 3 Update ALB security group to allow HTTPS (443) traffic
- 4 Access website using `https://yourdomain`



# Exercise – ALB Sticky session



## Continuing with earlier setup

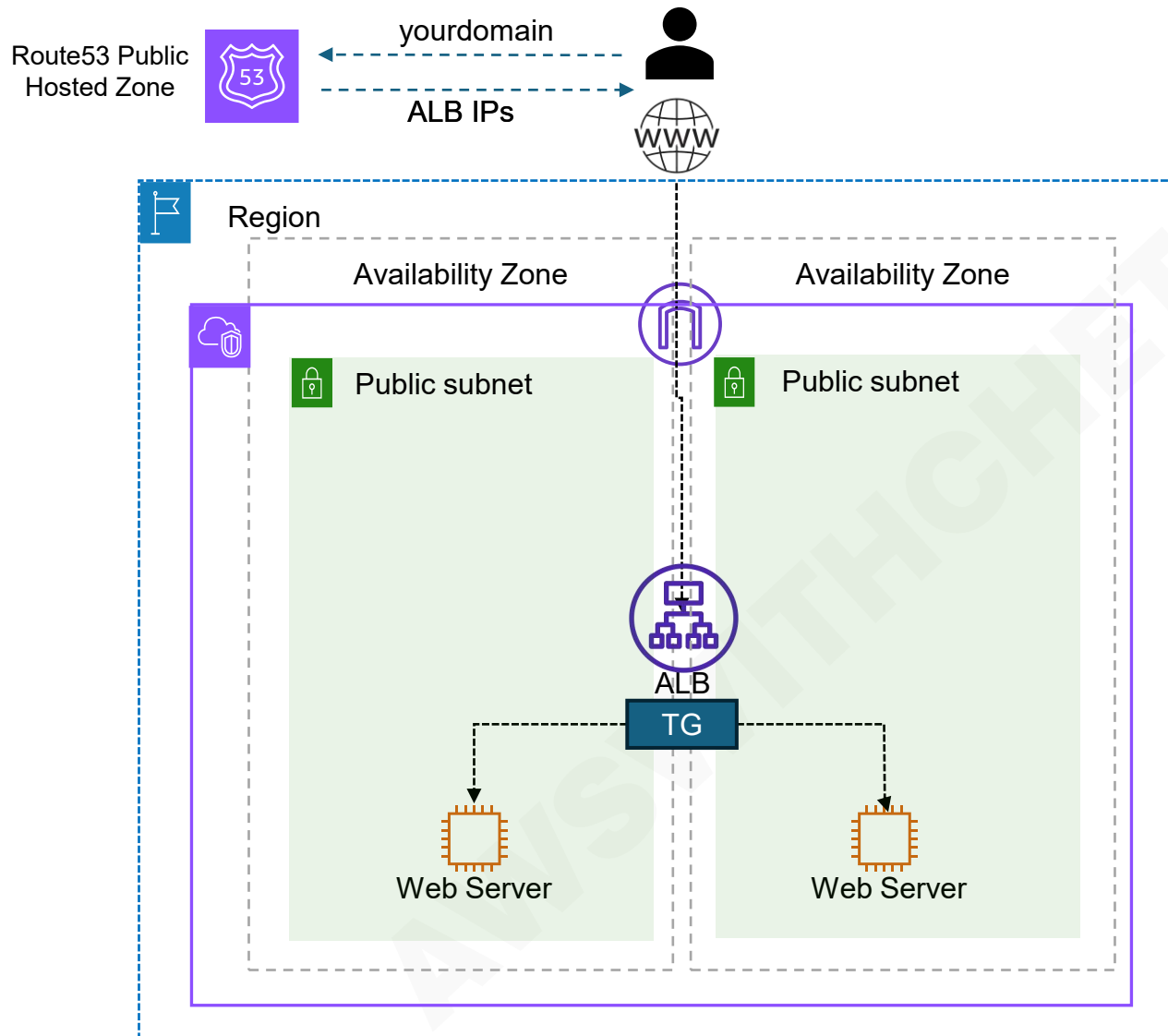
- 1 Update target group attribute to enable Session stickiness for 30 seconds duration
- 2 Try accessing website with your custom domain name, refresh couple of times, you should see that you land onto the same EC2 instance. Wait for 30 seconds and try again.

# Cleanup – **Very important**

1. Terminate both the EC2 instances
2. Delete Load Balancer Target groups
3. Delete Load Balancer
4. Delete Route53 records

AWSWITHCHETAN.COM

# Assignment – ALB path-based routing



- 1 Create two new EC2 launch templates using the User data scripts provided in the next slide. (We need /asia and /Europe directories this time)
- 2 Launch Web server 1 and Web server 2 in respective subnets using Launch Templates created above for Asia website and Europe website. Security group to allow HTTP traffic.
- 2 Create 2 target groups and register one EC2 instance in each target group as per the intended traffic routing (Webserver 1 for /asia and Webserver 2 for /europe).
- 3 Create Load Balancer listener rules to route traffic to respective Target groups with matching paths i.e. /asia to TG1 and /europe to TG2
- 4 Access web application with domain name and append path /asia. Traffic should be routed to web server for Asia. Do the same for /europe path and traffic should be routed to the webserver for Europe.

# User data for Launch Templates

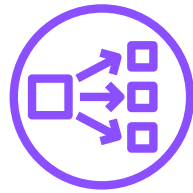
```
#!/bin/bash
dnf install httpd -y
systemctl start httpd.service
systemctl enable httpd.service
echo "<h1>This is a Webserver for Asia</h1>" > /var/www/html/index.html
mkdir /var/www/html/asia
echo "<h1>This is a Webserver for Asia</h1>" >
/var/www/html/asia/index.html
```

```
#!/bin/bash
dnf install httpd -y
systemctl start httpd.service
systemctl enable httpd.service
echo "<h1>Hello to the world ! </h1>" > /var/www/html/index.html
mkdir /var/www/html/europe
echo "<h1>This is a Webserver for Europe</h1>" >
/var/www/html/europe/index.html
echo "Configured successfully"
```

# Cleanup – **Very important**

1. Terminate both the EC2 instances
2. Delete Load Balancer Target groups
3. Delete Load Balancer

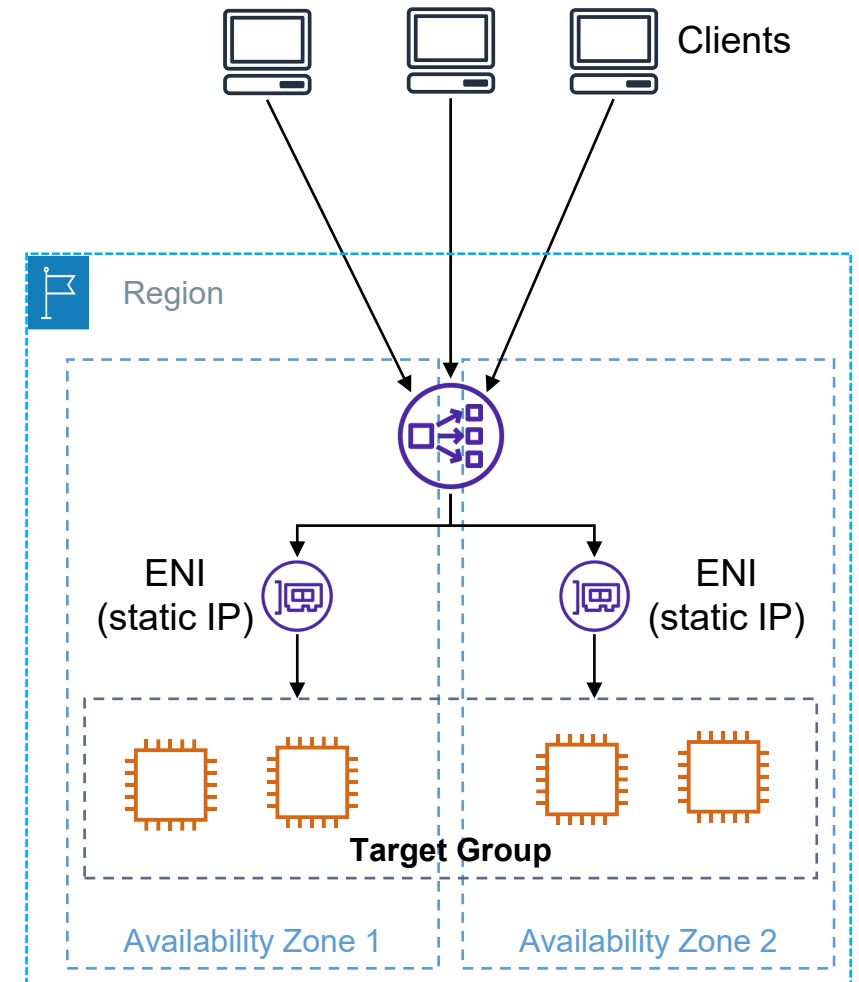
AWSWITHCHETAN.COM



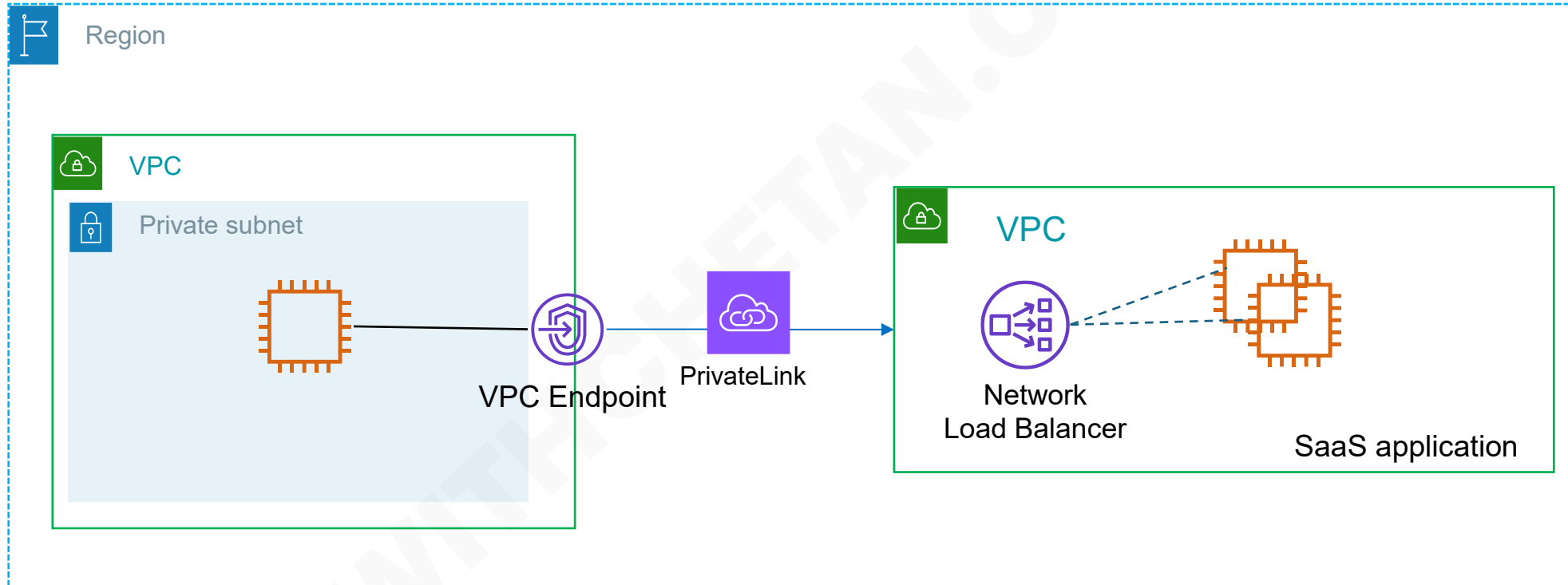
# Network Load Balancer

# Network Load Balancer

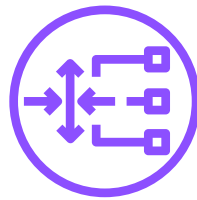
- Operates at Layer 4
- Supported protocols TCP, UDP, and TLS
- Targets: Instance, IP, Containers and Application Load Balancer (Lambda not supported)
- Handle millions of requests per second.
- NLB has one static IP per AZ, and supports assigning Elastic IPs
  - Useful for IP whitelisting on the client side
- Sticky Sessions:
  - Supported using the client's source IP address.
  - Uses Flow Hash algorithm (Source IP, Source Port, Protocol, Dest IP, Dest Port)
- Connection preservation: Maintains the same TCP connection between client and target (long sessions for gaming or IoT).
- IP preservation: The original client IP address is preserved and visible to the backend
- Used with AWS VPC PrivateLink to expose services privately



# VPC PrivateLink



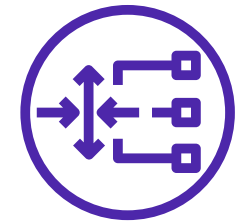




# Gateway Load Balancer

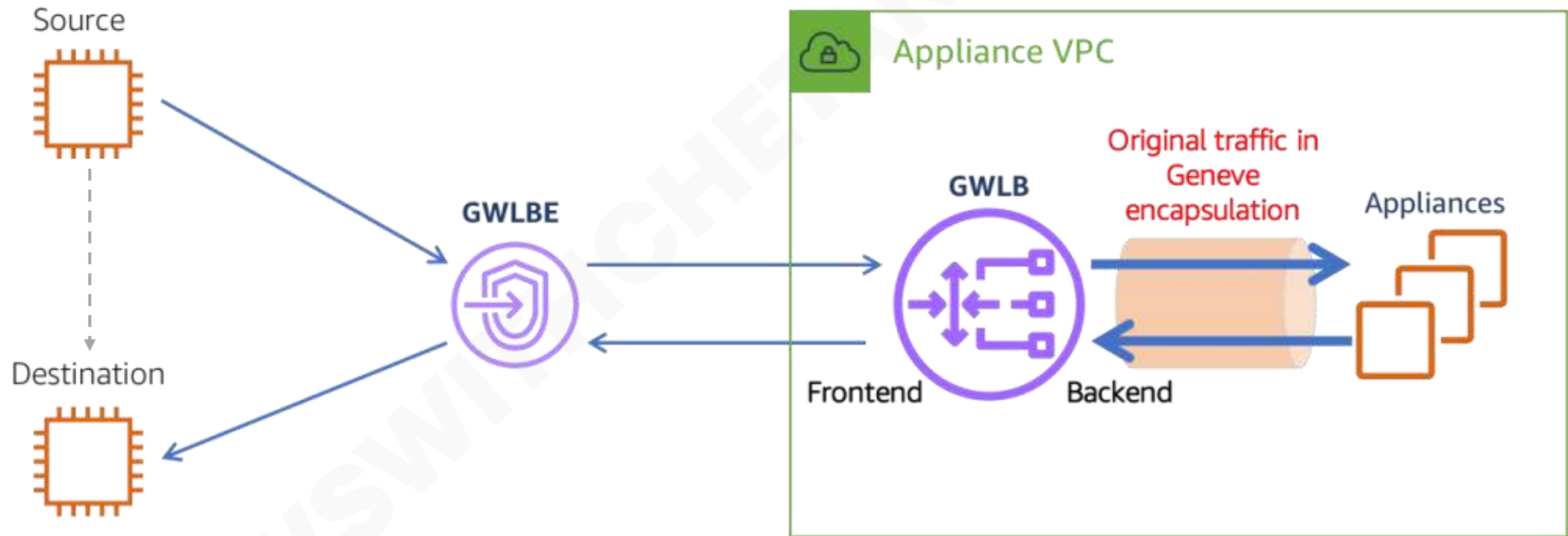
# Gateway Load Balancer (GWLB)

- It is designed to deploy, scale, and manage third-party virtual appliances like firewalls, intrusion detection/prevention systems (IDS/IPS), and deep packet inspection tools.
- Gateway Load Balancer (GWLB) operates at Layer 3 (IP layer).
- Uses the **GENEVE** (Generic Network Virtualization Encapsulation) protocol on port 6081 to forward traffic between clients and appliances.
- Transparent to traffic - doesn't modify IP addresses, so appliances see original source and destination IPs.
- Traffic is sent to GWLB endpoints (GWLBes) created in each VPC using AWS PrivateLink.
- GWLB integrates with industry leading partners Aviatrix, Cisco Systems, Fortinet, Palo Alto Networks etc.

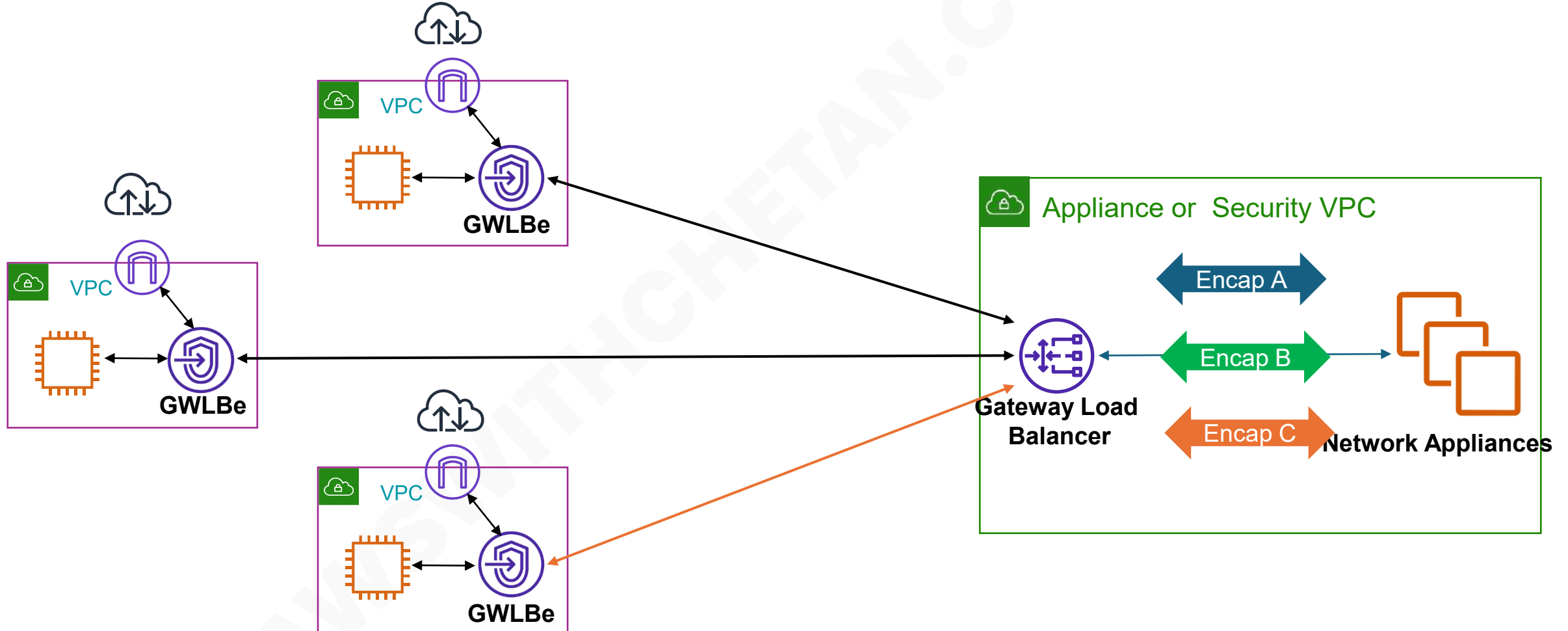


Gateway Load Balancer

# How Gateway Load Balancer work?



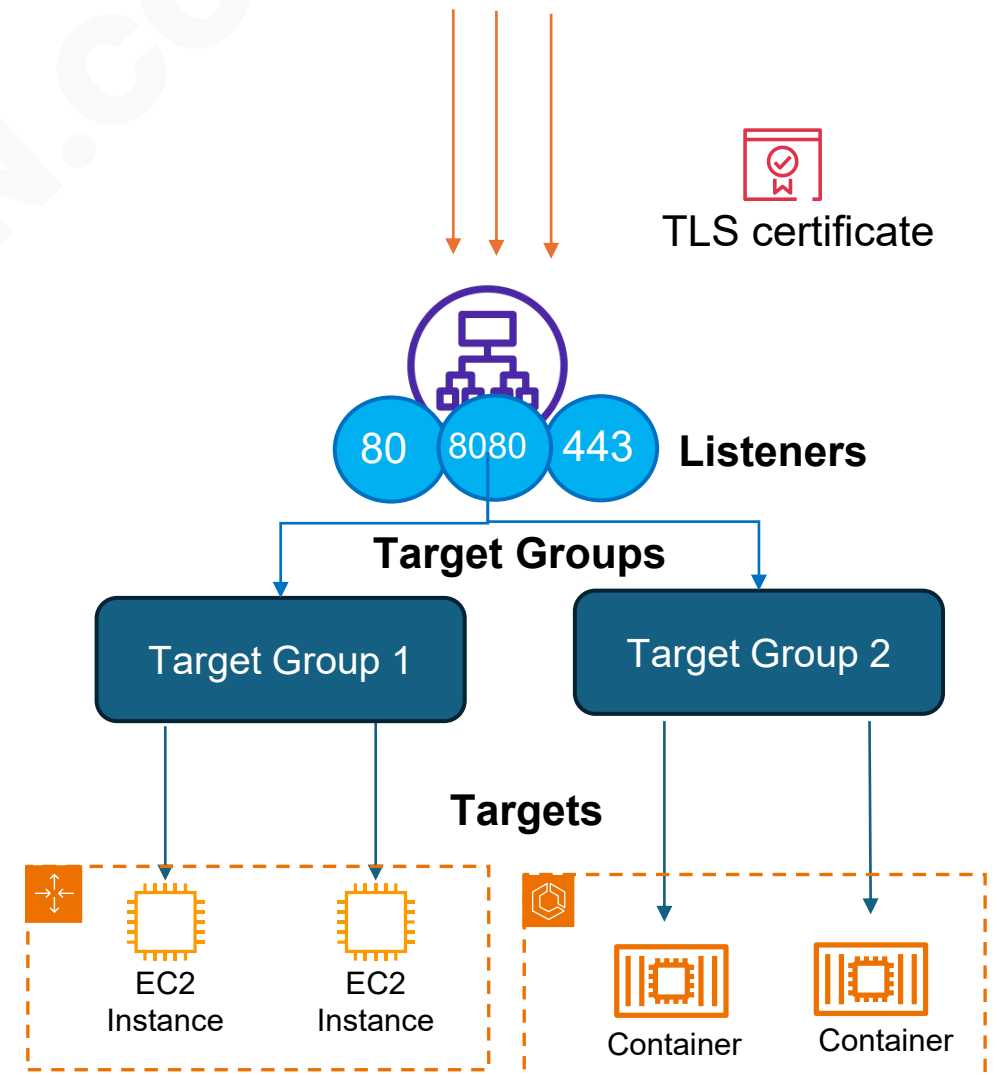
# Use case: Centralized Inspection using GWLB



Important to know for your exam

# Important to know for your exam

- External and Internal Load balancers
- Cross-zone load balancing
- Server Name Indication (SNI)
- Client IP preservation
- Proxy protocol



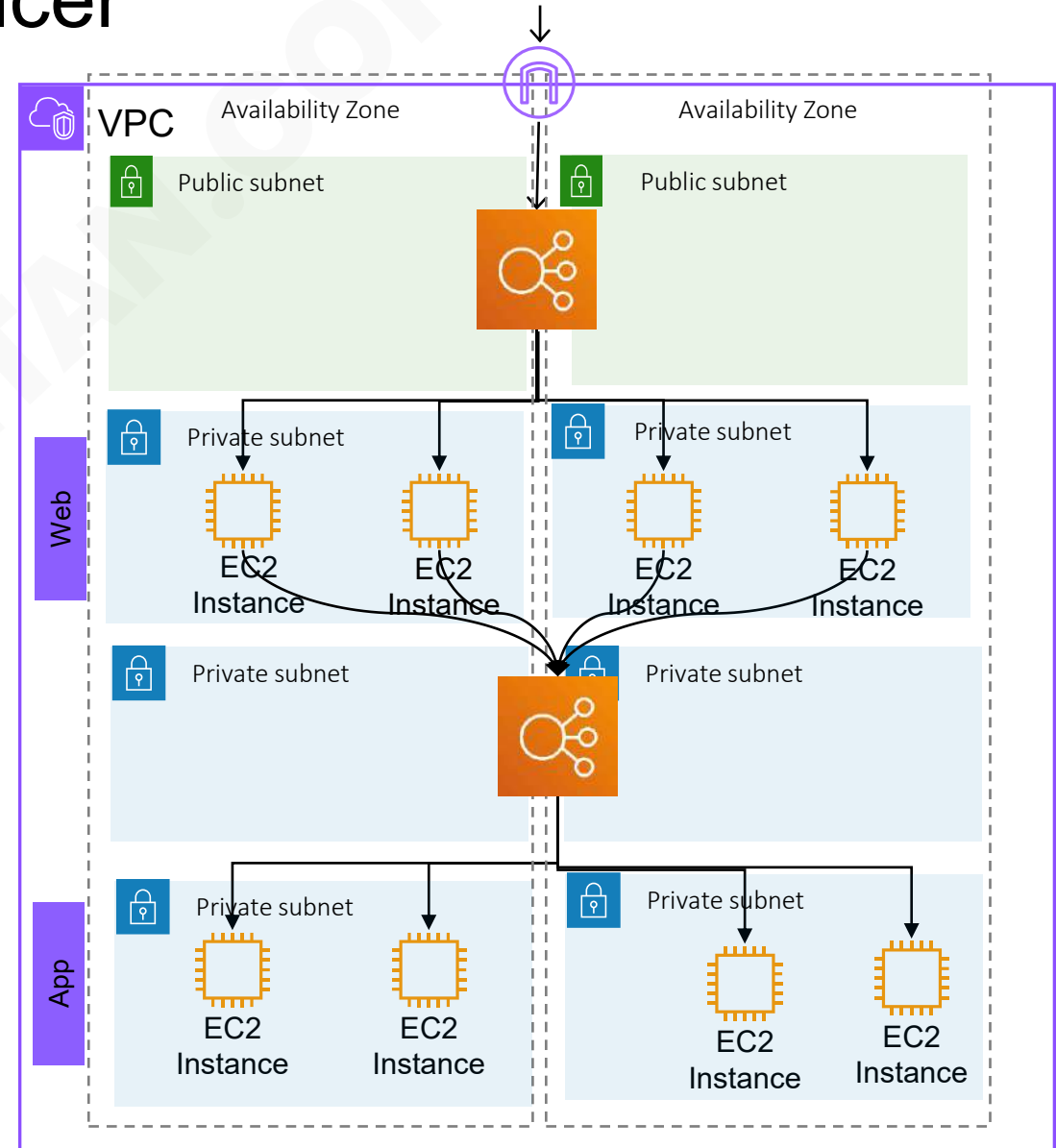
# External and Internal Load Balancer

## External Load Balancer (Public)

- To be launched in Public Subnet
- Receives traffic from the internet

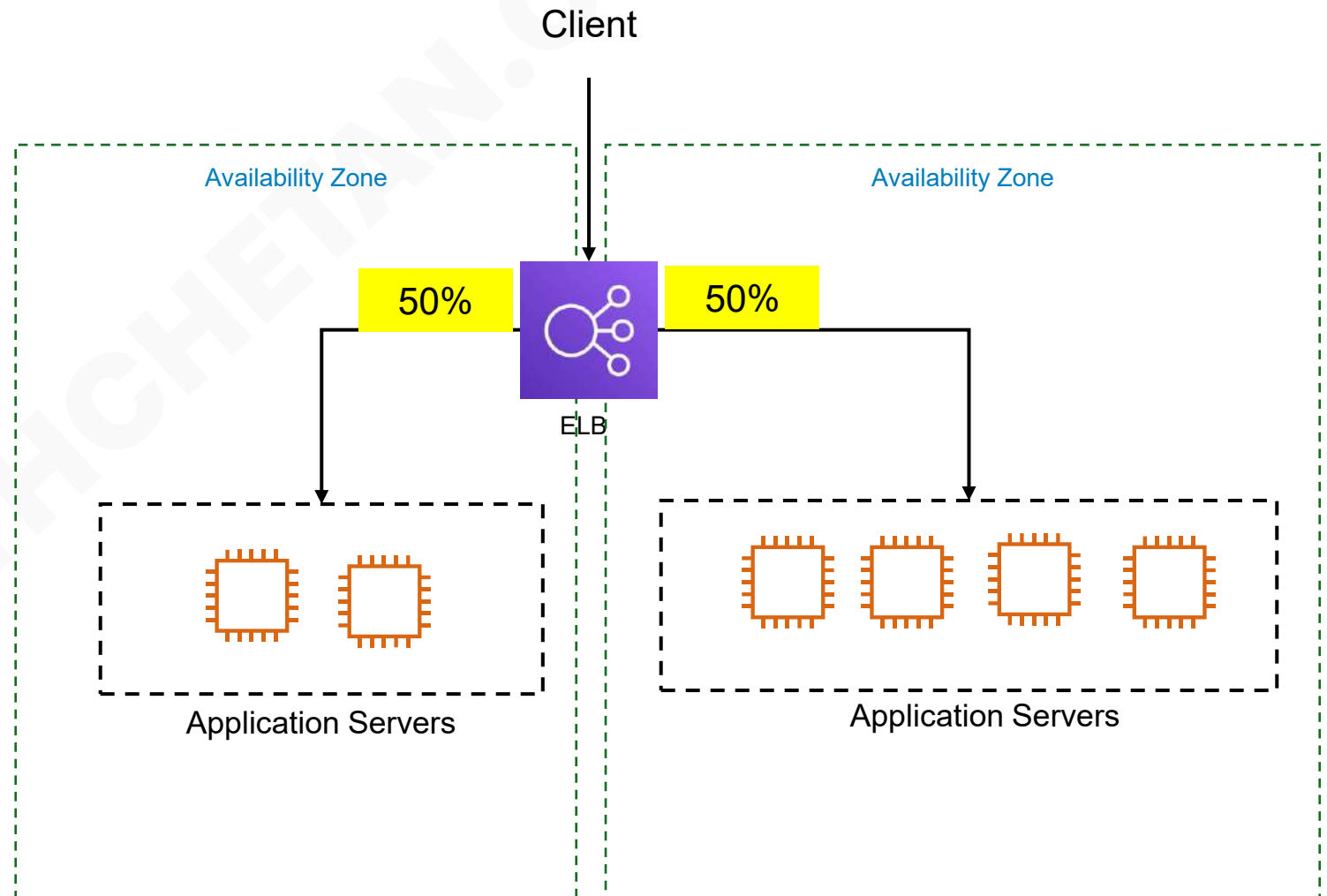
## Internal Load Balancer (Private)

- To be launched inside the Private subnet
- Receives traffic from within the VPC



# Cross-zone load balancing

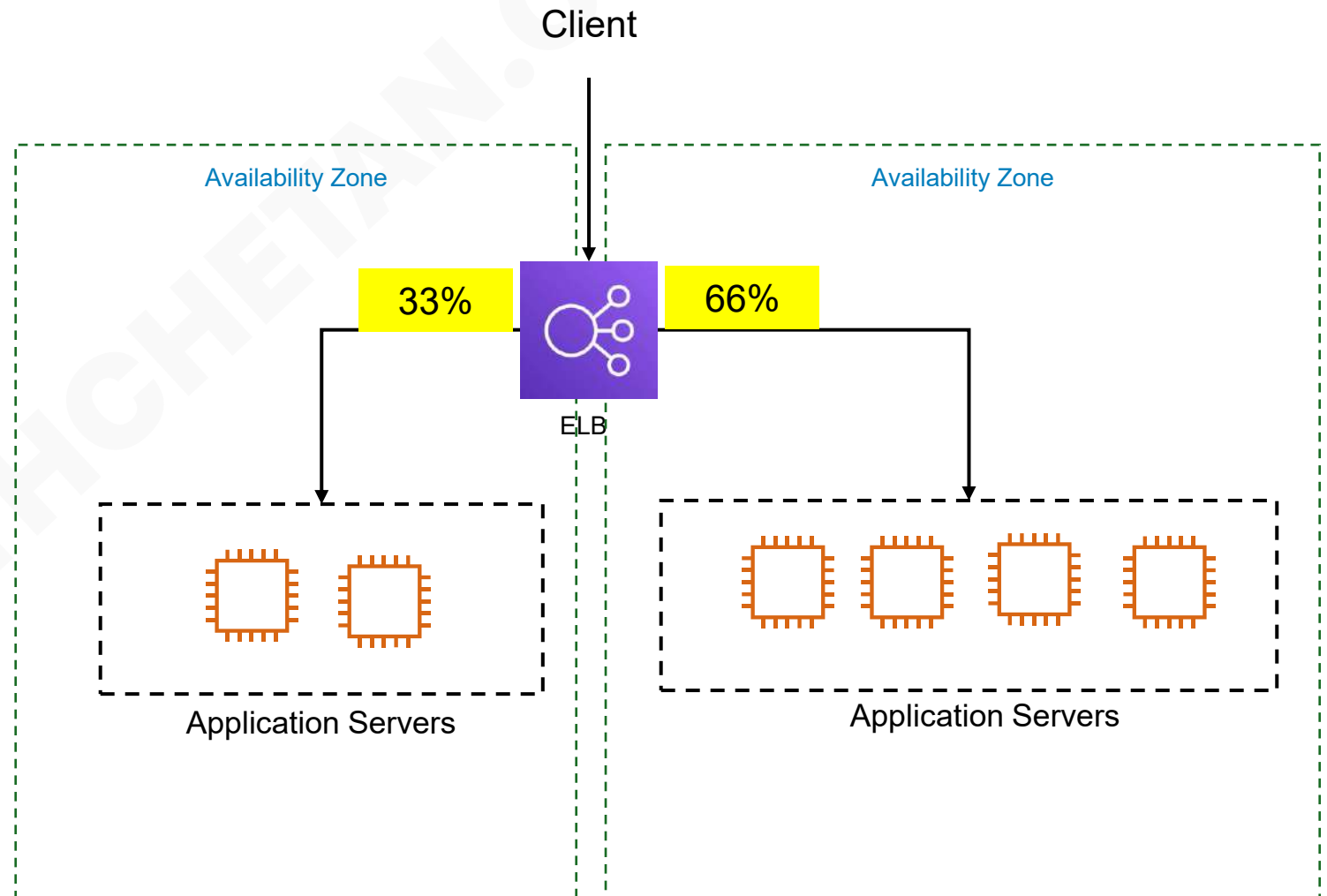
Without Cross-Zone load balancing





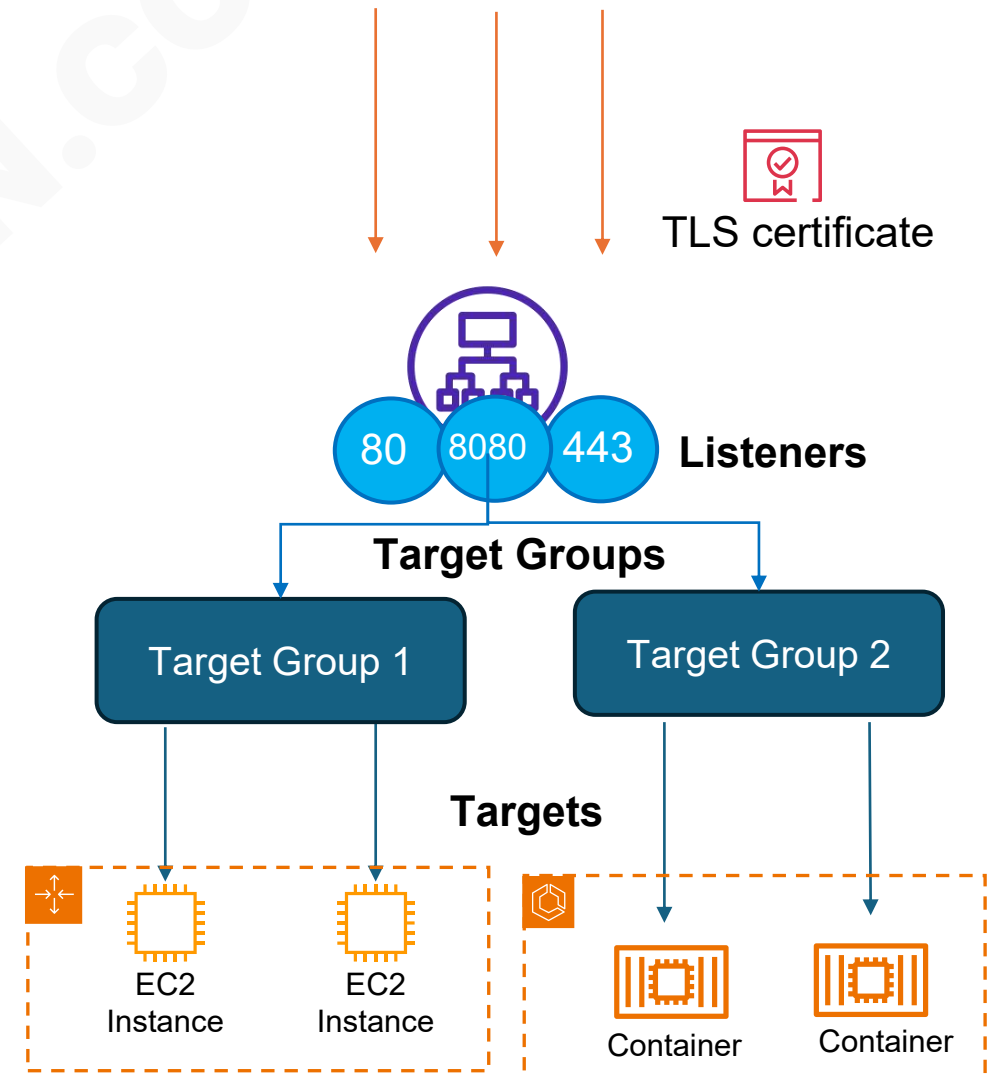
# Cross-zone load balancing

With Cross-Zone load balancing



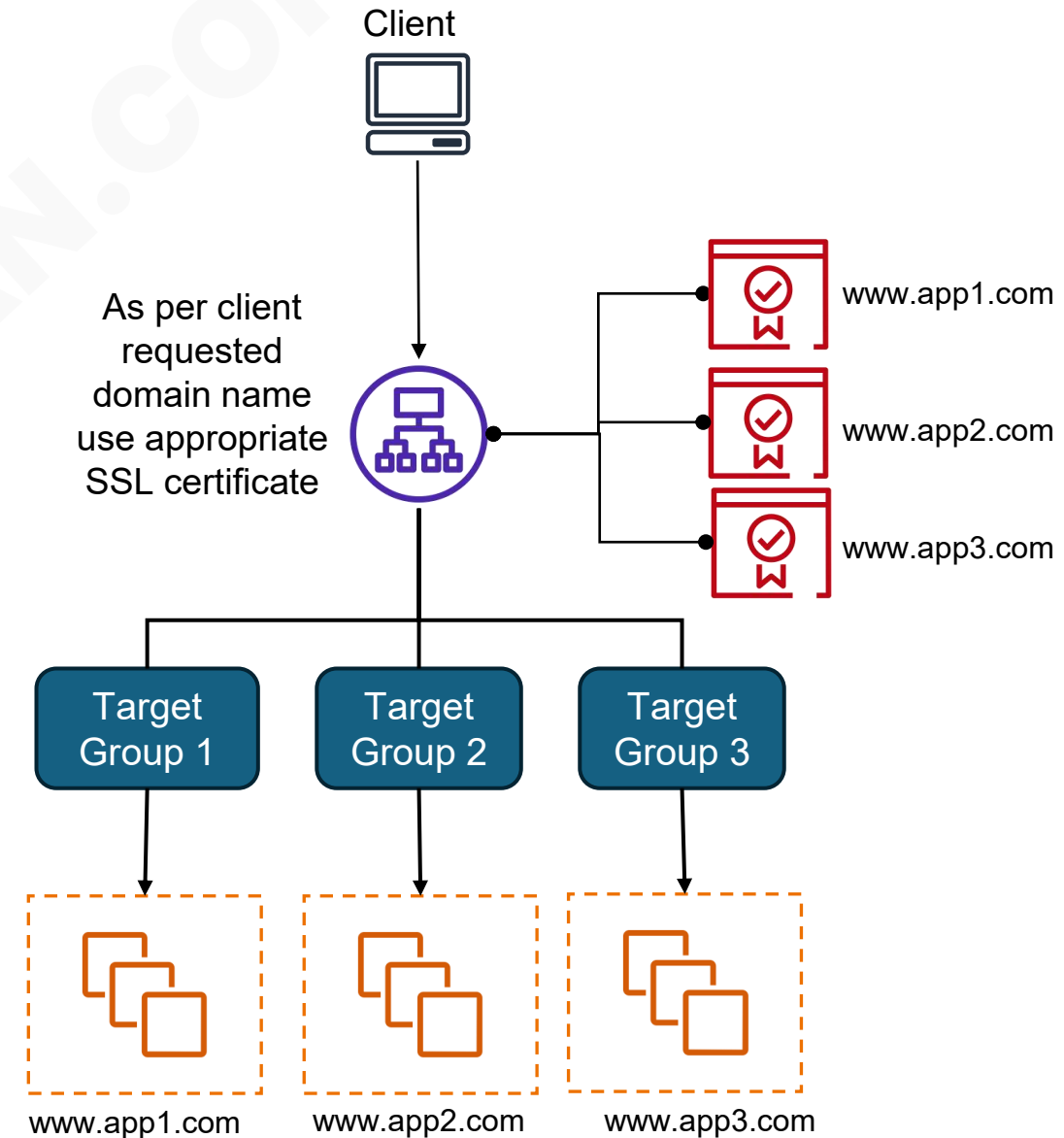
# Cross-zone load balancing

- Enabled by default for Application Load Balancer
- Disabled by default for Network Load Balancer
- Disabled by default for Gateway Load Balancer



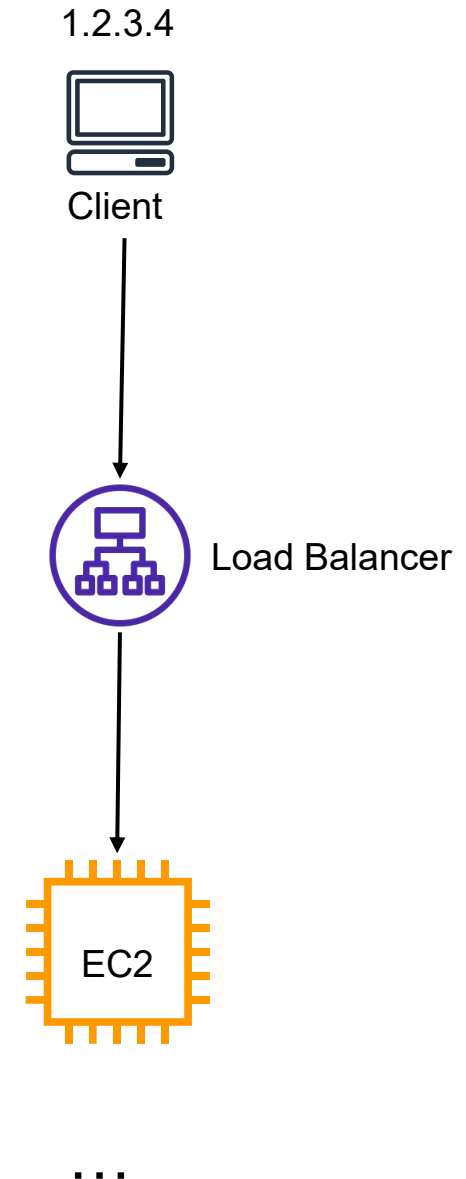
# Server Name Indication (SNI)

- SNI allows using multiple SSL certificates with the load balancer
- Load balancer can host multiple different websites or web applications
- Client to initiate SSL handshake request using desired hostname
- The load balancer will identify the correct certificate according to the hostname and return the correct certificate
- Only works for ALB & NLB



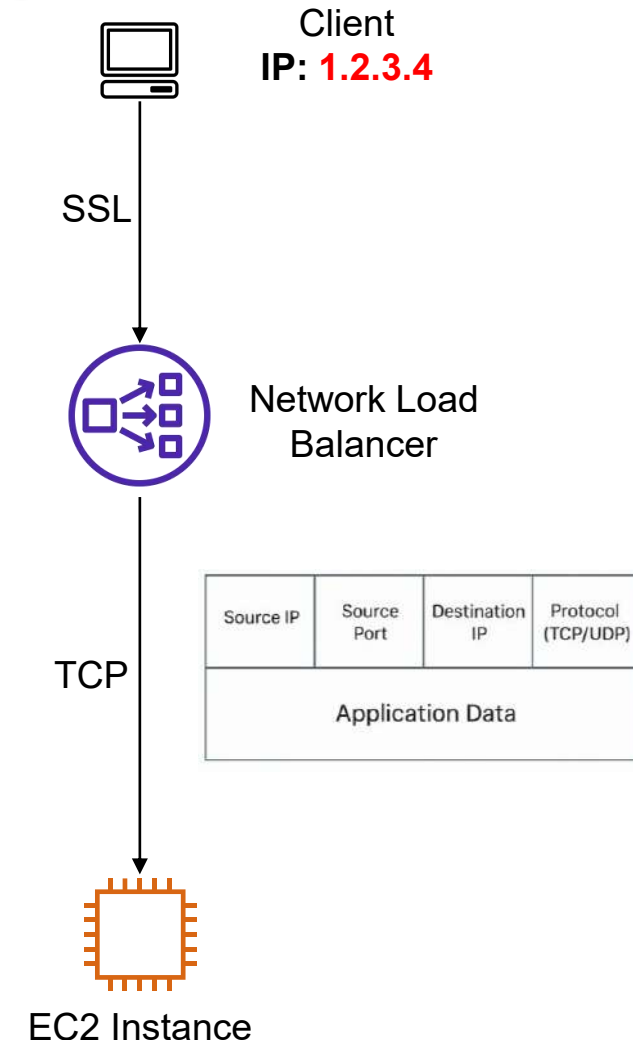
# Client IP preservation

- In case of ALB, the target (EC2) sees the ALB's private IP as the source.
- To get the real client IP, use the `X-Forwarded-For` header in the HTTP request.
- NLB is a pass-through load balancer, so the target sees the client's original IP directly.
- If you use a TLS listener on NLB (with termination at NLB), the target won't see the client IP, and in that case, you can enable Proxy Protocol v2 to forward that info.



# NLB Proxy Protocol

- An Internet protocol used to carry information from the source (requesting the connection) to the destination (where connection was requested)
- If the Load Balancer terminates the connection (e.g. TLS termination), the source IP address of the client is not preserved.
- Enable Proxy protocol v2 for passing the connection info to the backend.
- When Proxy protocol v2 is enabled, NLB adds small binary header in the TCP packet which contains
  - Original source IP and port (client)
  - Destination IP and port (load balancer)
  - Protocol type (TCP/UDP/TLS)
  - Optional fields for SSL/TLS info





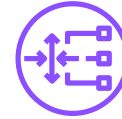
## Application Load Balancer

- Operates at Layer 7
- Targets: Instance, IP, Lambda, Containers
- Supported protocols HTTP, HTTPS, WebSocket, HTTP/2 and gRPC
- Use header X-Forwarded-For for Client Source IP
- Content based routing – Host, path, Query string, SourceIP
- Cross-zone load balancing enabled by default
- Use: Web Applications, API hosting



## Network Load Balancer

- Operates at Layer 4
- Targets: Instance, IP, ALB
- Supported protocols TCP, UDP, TLS
- Provides Static IP so that client can whitelist
- Flow-based routing (5-tuple hash) – Long TCP connections
- Proxy Protocol v2 for Source IP when using TLS
- Use: Ultra low latency requirement. Example: Real-time systems, IoT, Gaming, Streaming



## Gateway Load Balancer

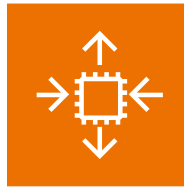
- Operates at Layer 3
- Targets: Instance, IP
- Supported protocols GENEVE
- Use: Traffic inspection where traffic is routed to backed Firewall or network Appliance instances



## Classic Load Balancer

- Operates at Layer 4 & 7
- Targets: Instance, IP, Lambda, Containers
- Supported protocols HTTP, HTTPS, TCP, SSL
- Use: Web applications

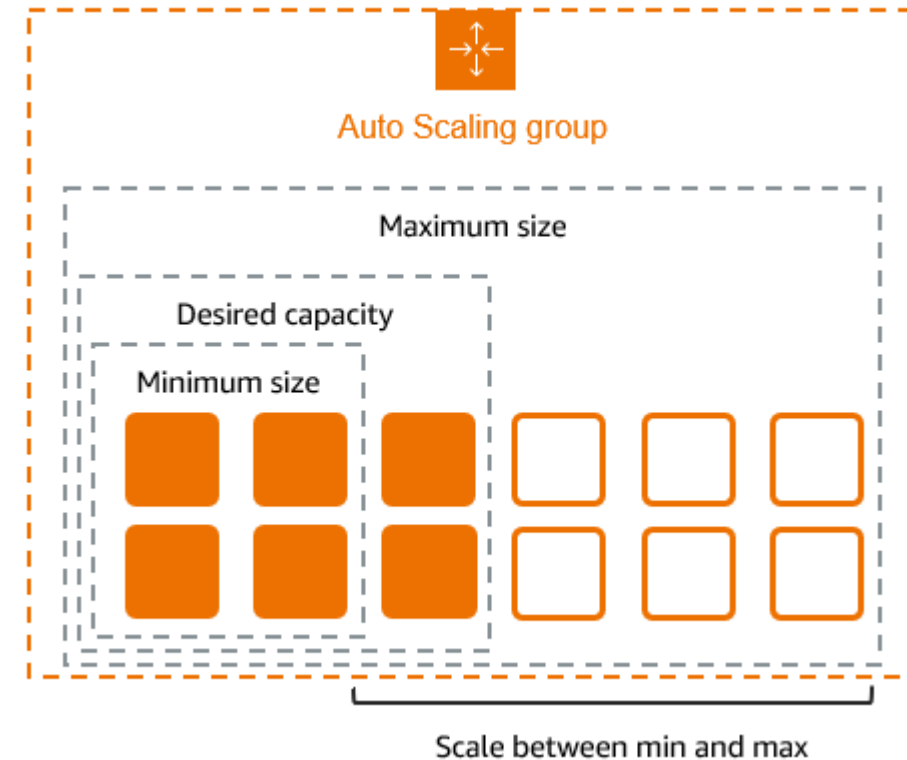




# Autoscaling group

# Auto Scaling group (ASG)

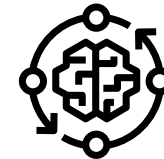
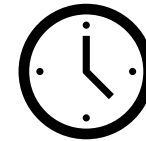
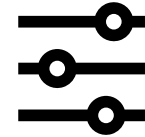
- Auto Scaling groups automate the scaling of EC2 instances
- You specify Minimum, Maximum, and Desired instance counts
- ASG ensures the desired number is always running.
- Requires a **Launch Template** to define instance details such as AMI, type, security groups, and key pair.
- Integrates tightly with Elastic Load Balancers (ALB/NLB) and automatically registers/deregisters instances.
- Performs Health Checks (EC2 status and optionally ELB health checks) to replaces unhealthy instances automatically.
- Supports different Scaling policies
- Uses CloudWatch Alarms to trigger scaling actions based on metrics.





# Auto Scaling group scaling policies

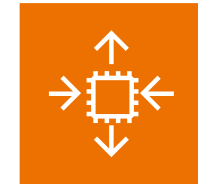
- **Manual Scaling**
  - Set the required number of EC2 instances in the ASG configuration
- **Dynamic Scaling**
  - Simple & Step scaling
    - When avg. CPU > 50% add 1 EC2 instance
    - When avg. CPU > 75% add 2 EC2 instances
    - When avg. CPU < 40% remove 1 EC2 instance
    - When SQS queue has high number of messages
  - Target Tracking
    - Maintain the avg. CPU utilization to ~40%
- **Scheduled Scaling**
  - Schedule the scaling for given period – Saturday evening 6-11pm
- **Predictive Scaling**
  - User Machine Learning to scale based on the historic utilization



# Demo

# Auto Scaling Group - Advanced features

- **Termination Policy** - Decides which instance to remove first (default: oldest)
- **Cooldown Period** - Prevents new scaling actions from happening too quickly after a previous one (default: 300 seconds).
- **Instance Refresh** - Rolling updates of instances to apply new configurations safely.
- **Lifecycle Hooks** - Allows custom actions (e.g. run scripts) when launching or terminating instances.
- **Warm Pools** - Let ASG keep pre-initialized instances ready to start faster during scale-out.



Autoscaling  
Group



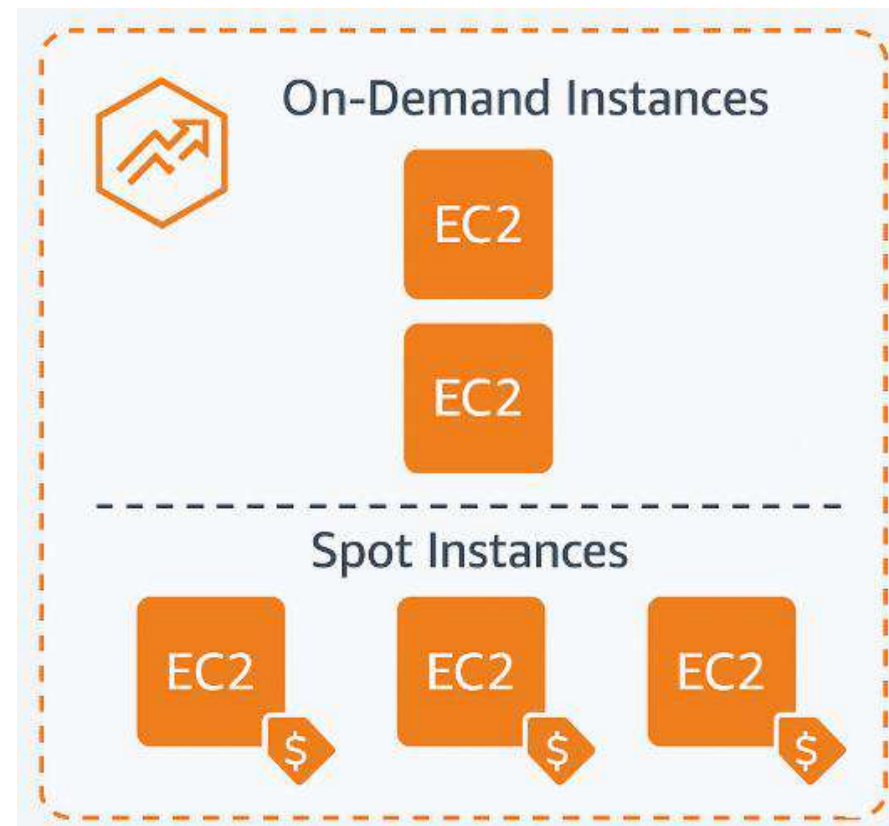
# ASG with On-demand and spot instances

## In ASG, define:

- Base Capacity - Minimum number of On-Demand instances.
- On-Demand Percentage Above Base - Percentage of additional instances that should be On-Demand.
- Spot Pools - Groups of instance types that can be used as Spot instances (always diversify).

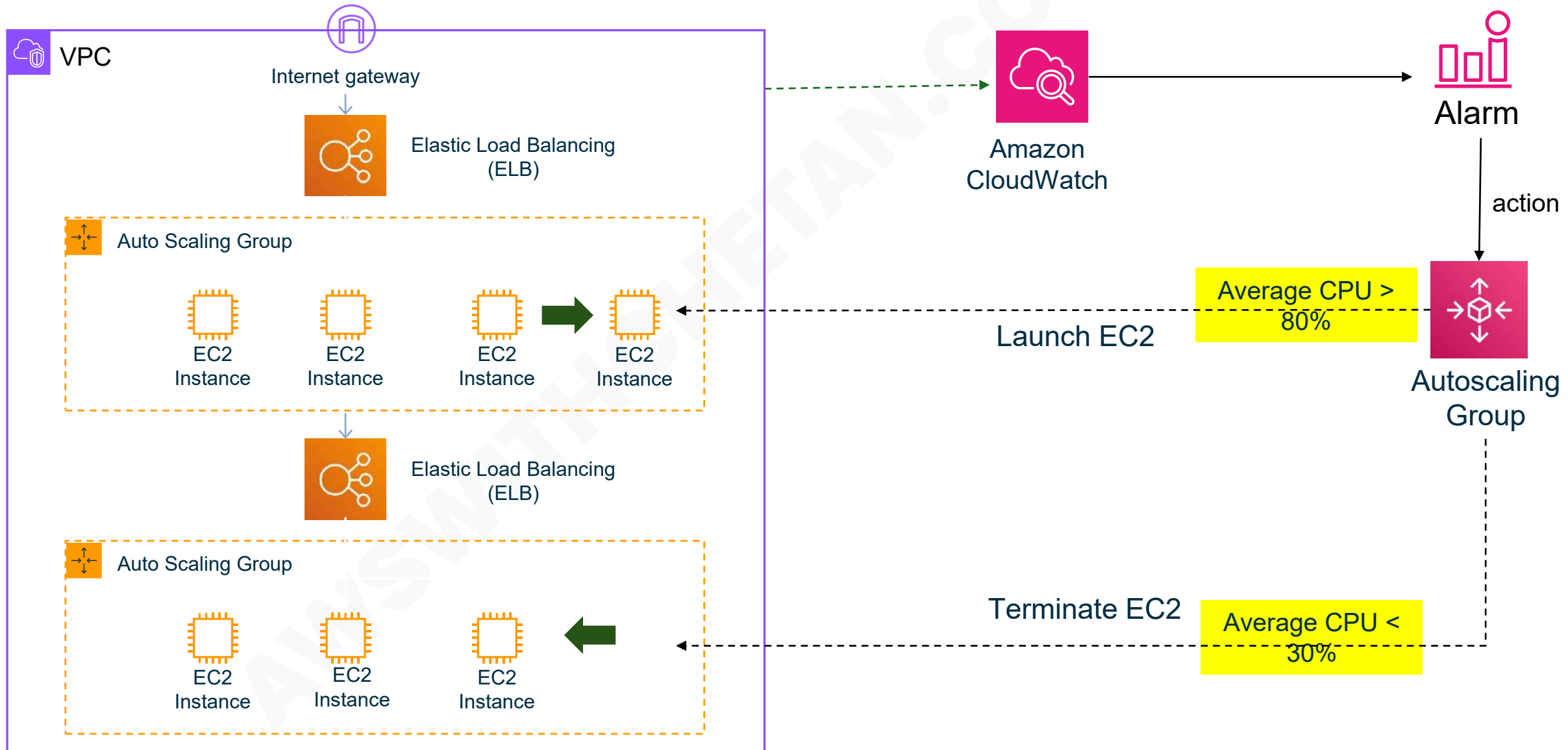
## The ASG will then automatically:

- Launch On-Demand instances first for the base.
- Add Spot instances from multiple pools to reduce interruption risk and replace interrupted Spot instances automatically.
- Use Lifecycle Hooks to handle graceful shutdowns when Spot interruptions happen.

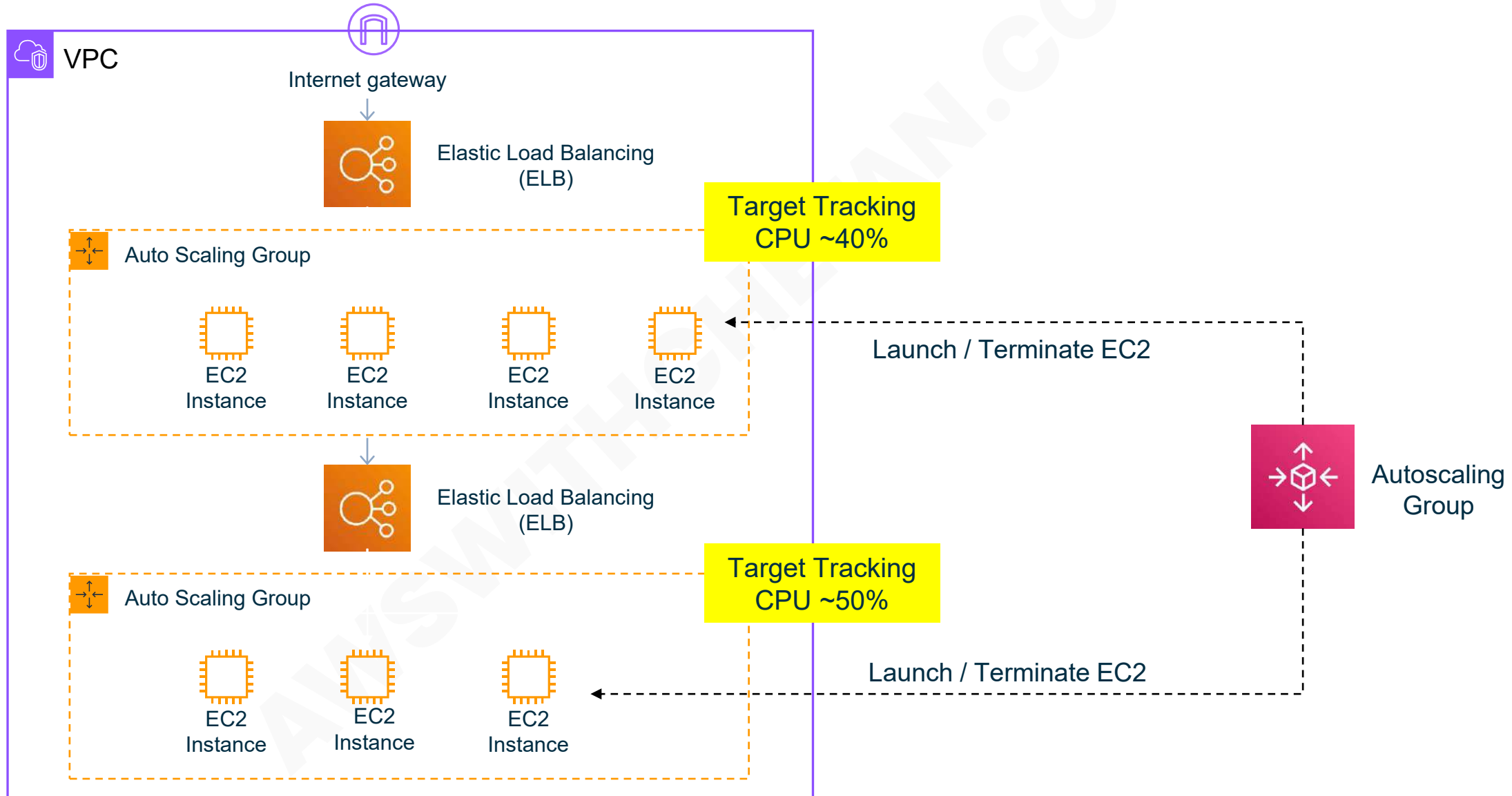


Let's talk architecture

# ELB + Autoscaling Group with Simple scaling

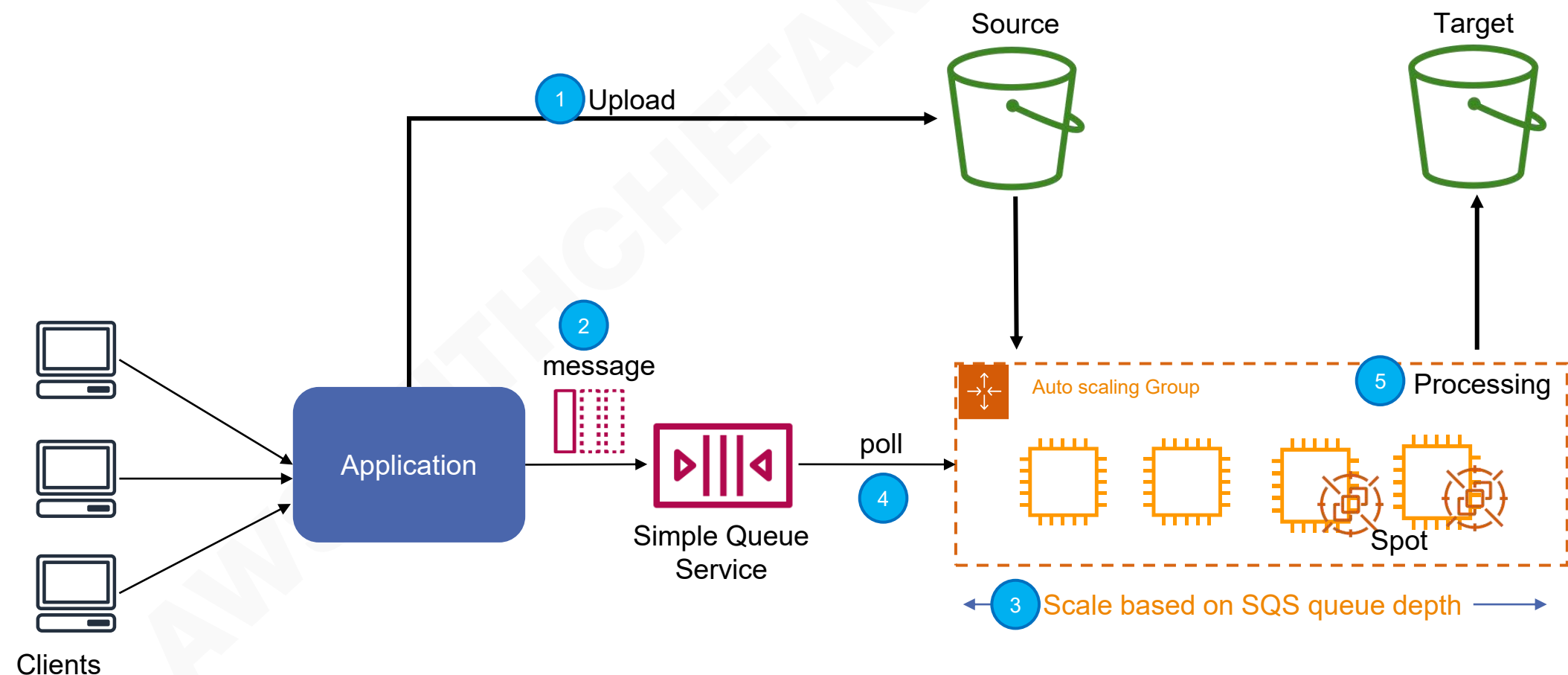


# ELB + Autoscaling Group with Target tracking



# Job processing

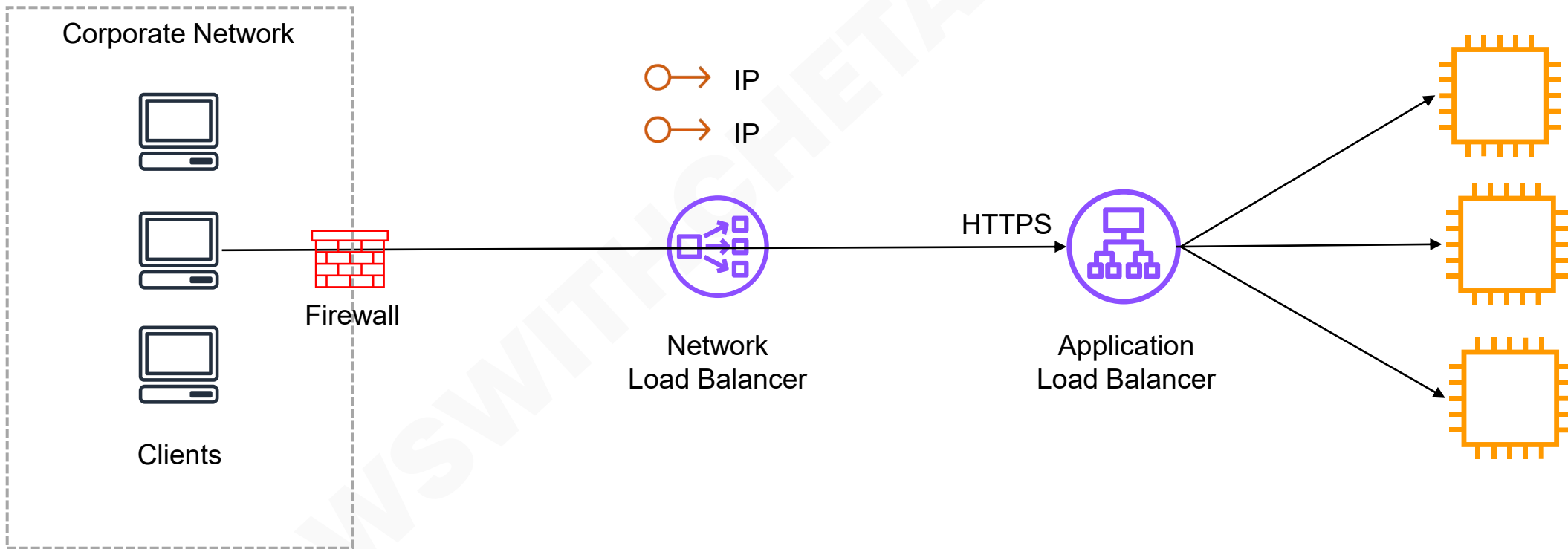
**Use case:** Dynamically adjust the EC2 capacity depending on the incoming Job requests for data processing





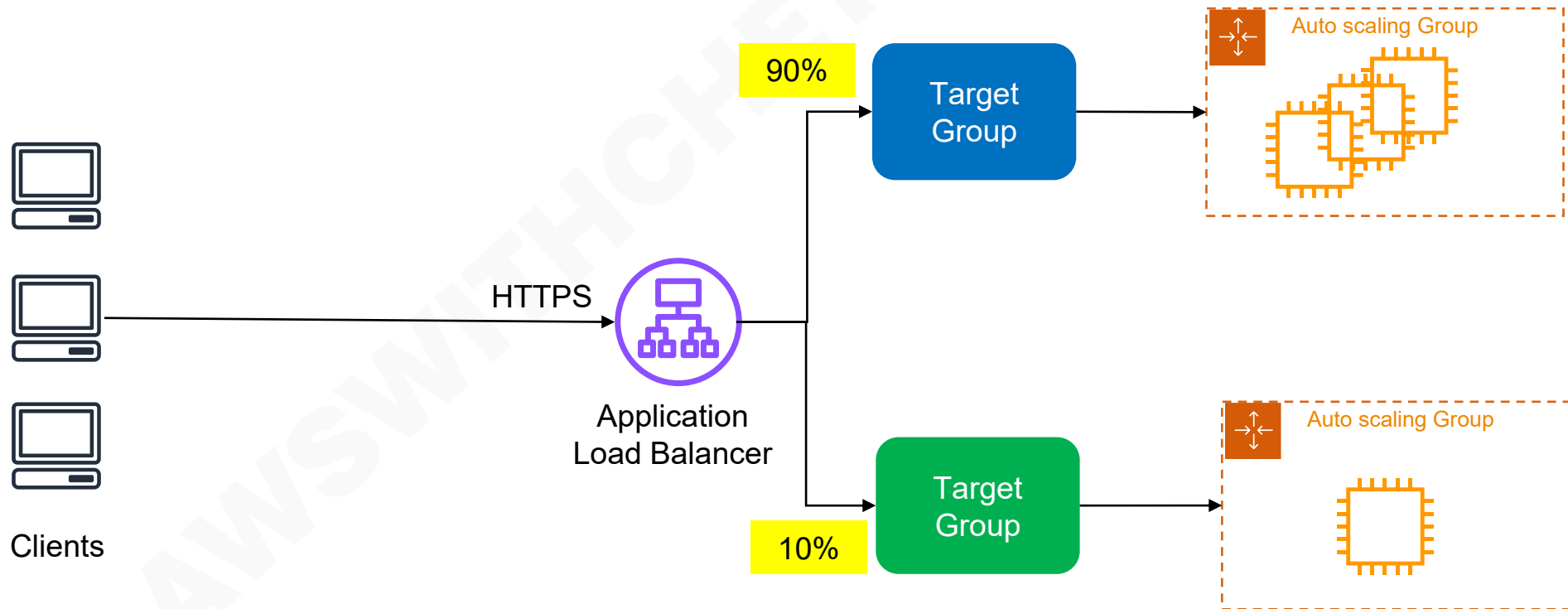
# ALB as a target of NLB

**Use case:** Client needs Static IPs to be whitelisted in the corporate network firewall for the web application hosted in AWS



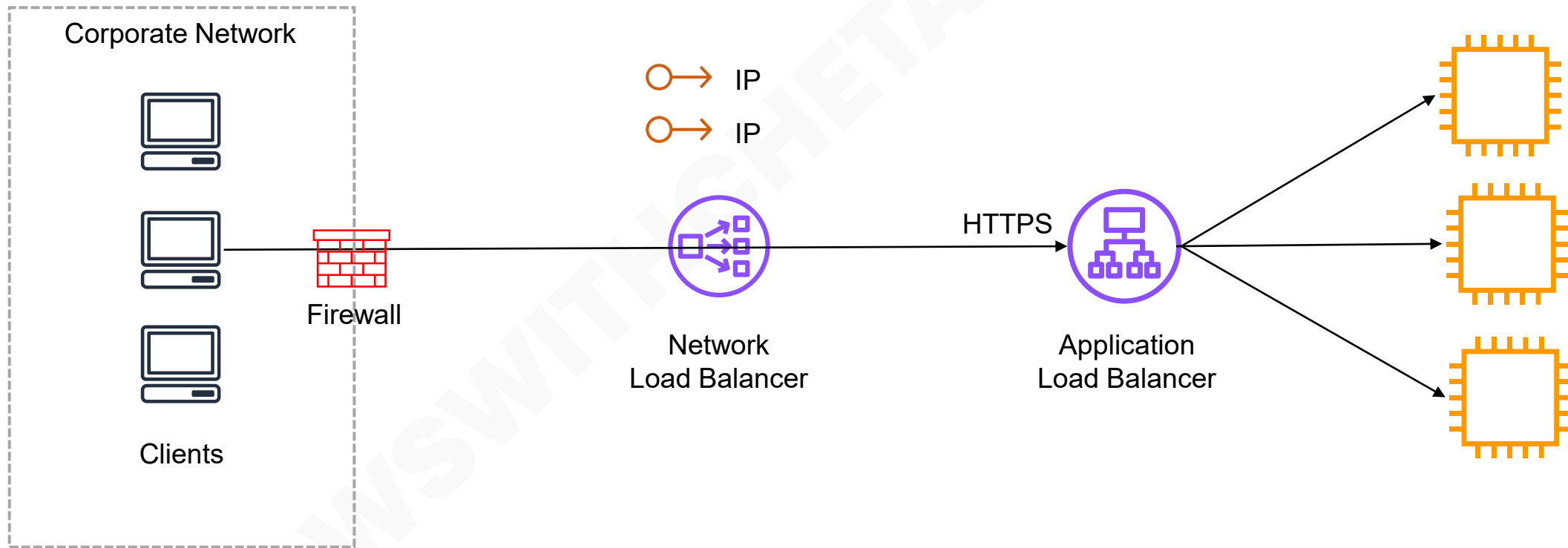
# Blue/Green deployment – ASG + ALB

**Use case:** Blue/green deployment where traffic is gradually shifted to green deployment. Use Weighted target groups to shift the traffic gradually. Use separate Autoscaling groups for both target groups and manage the capacity manually as traffic shifts.



## 2. ALB as a target of NLB

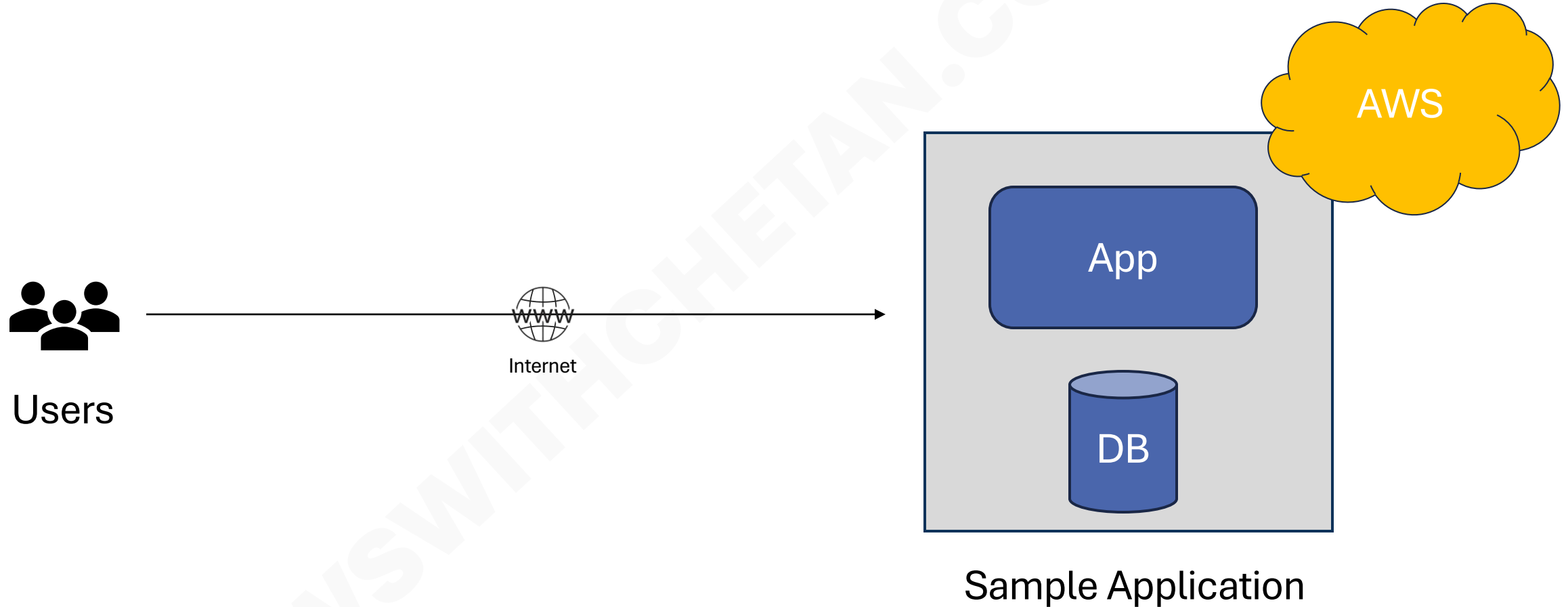
Use case: Client needs Static IPs to be whitelisted in the corporate network firewall for the web application hosted in AWS



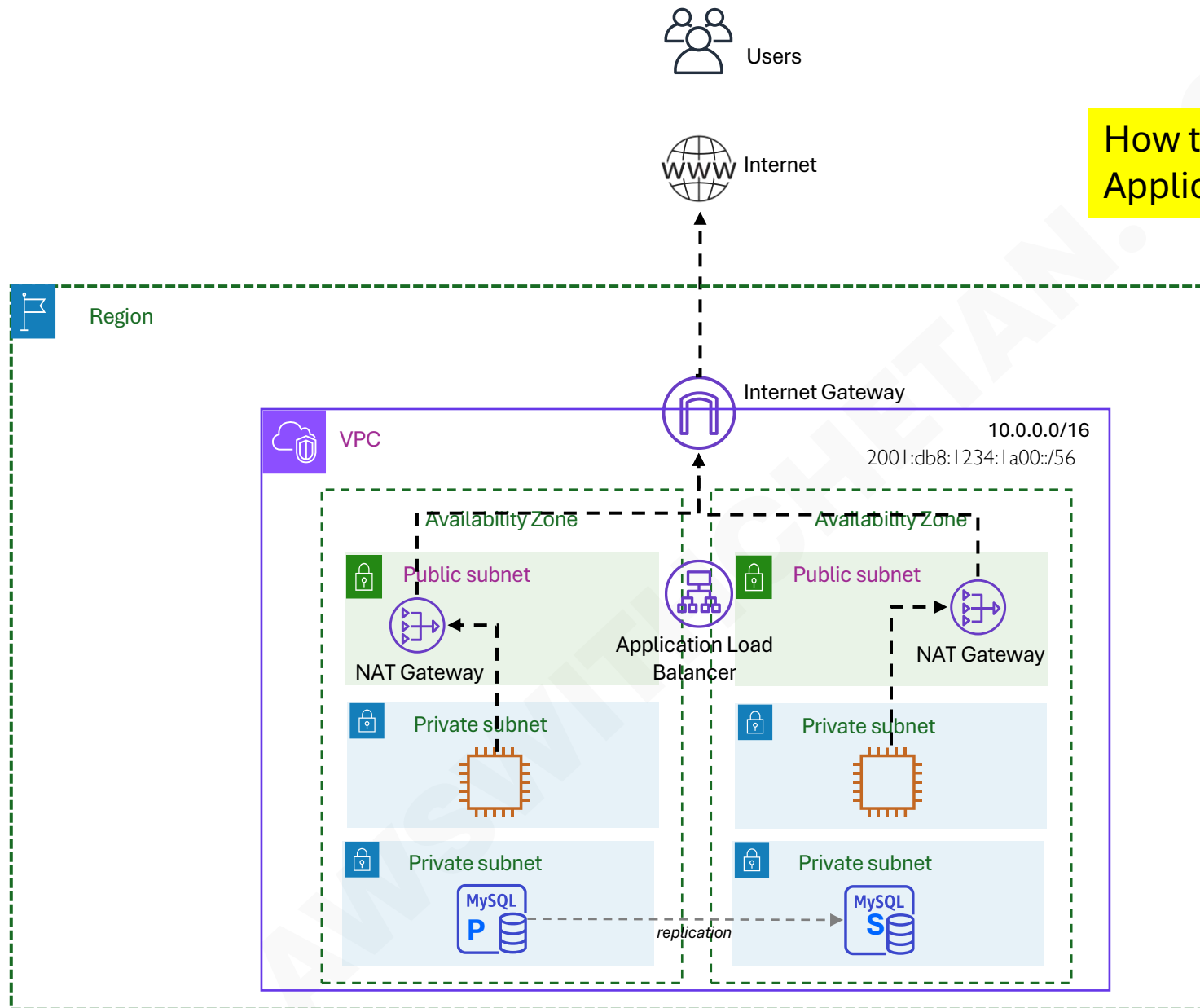


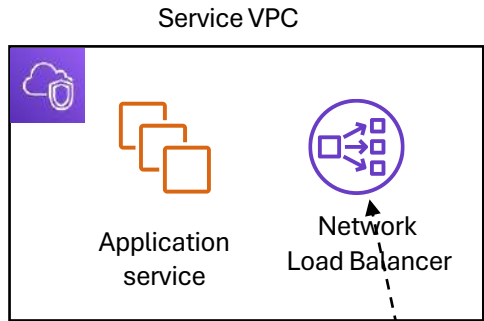
# VPC and Networking

# We want to a web application in AWS

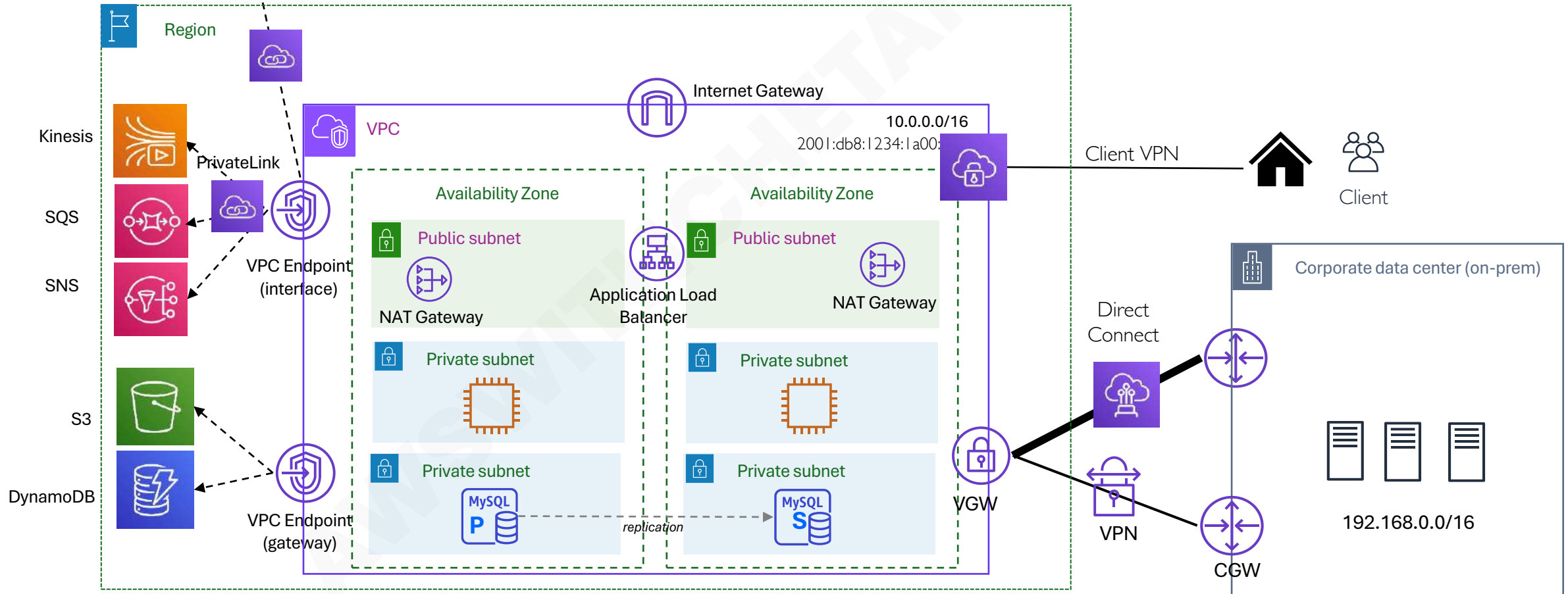


How to access outbound internet from Application servers?





How to access VPC resources privately from employee workstation?



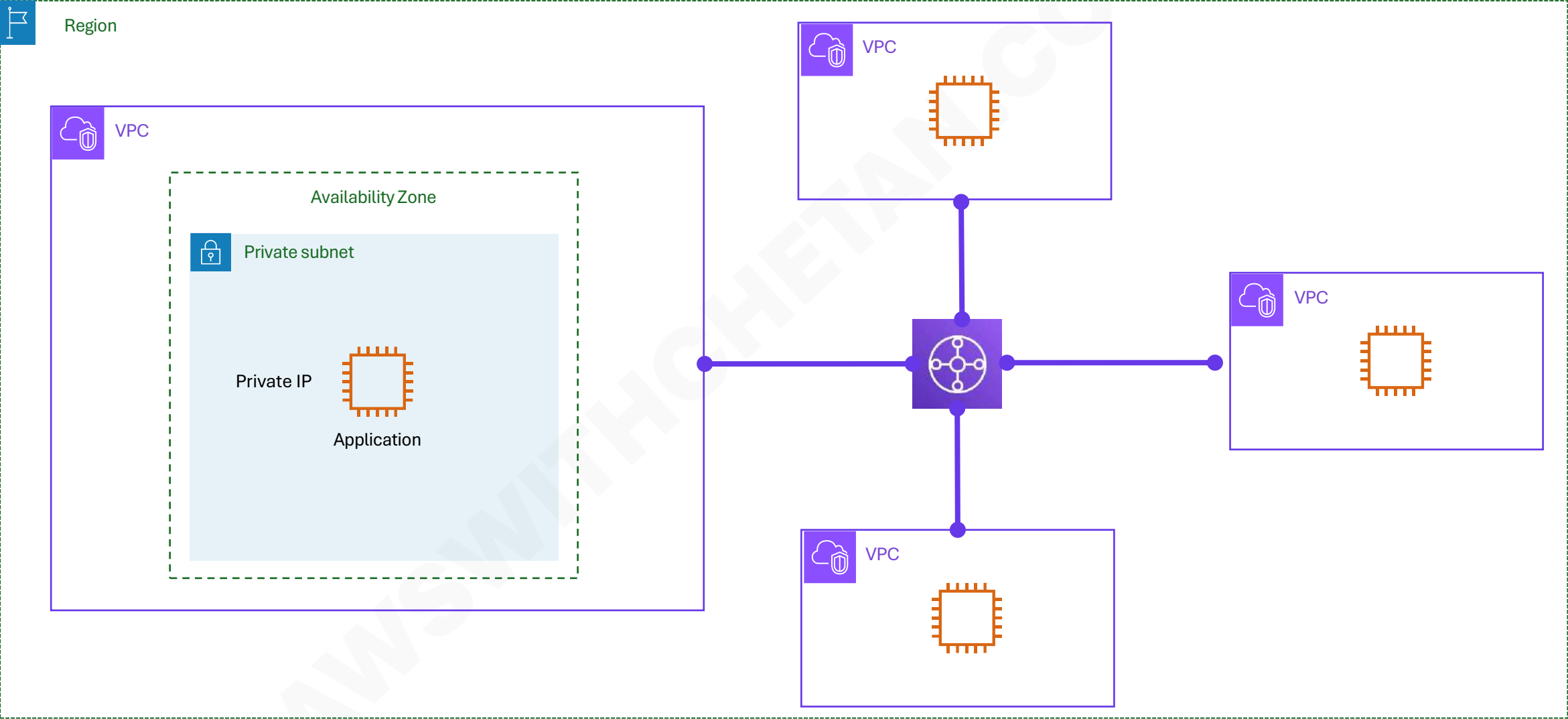
# VPC Private connectivity options

- VPC Peering connection
- AWS Transit Gateway
- AWS Cloud WAN
- Amazon VPC Lattice

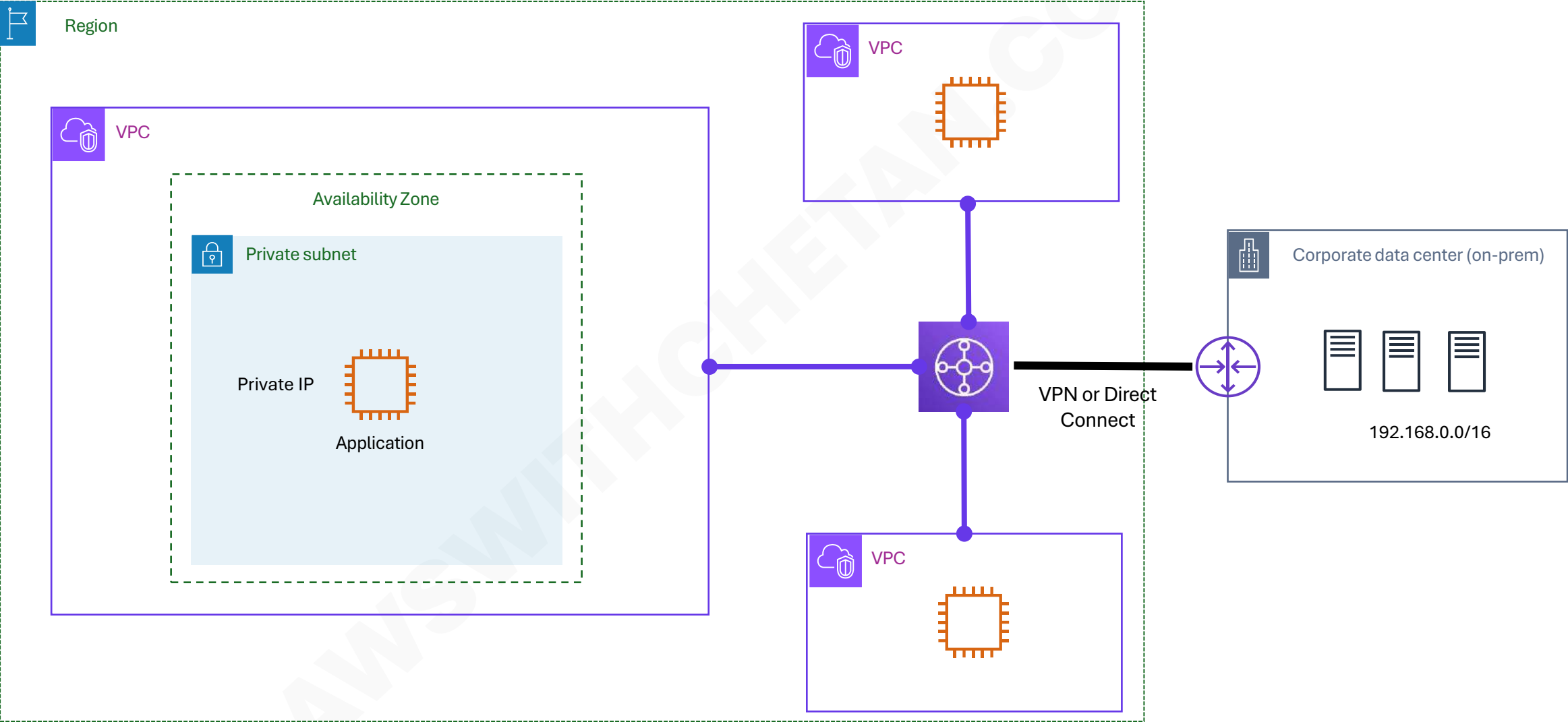
AWSWITHCHETAN.COM



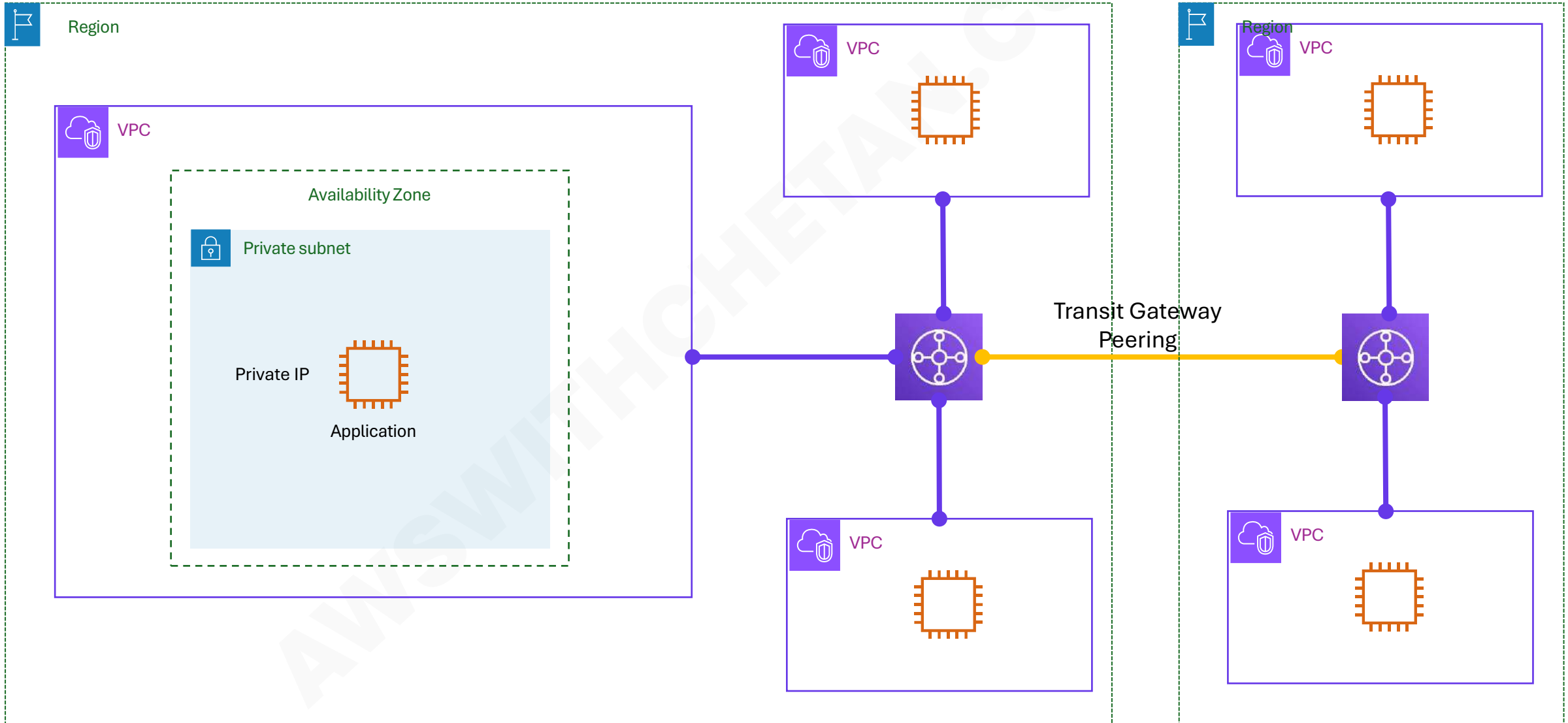
# Transit Gateway

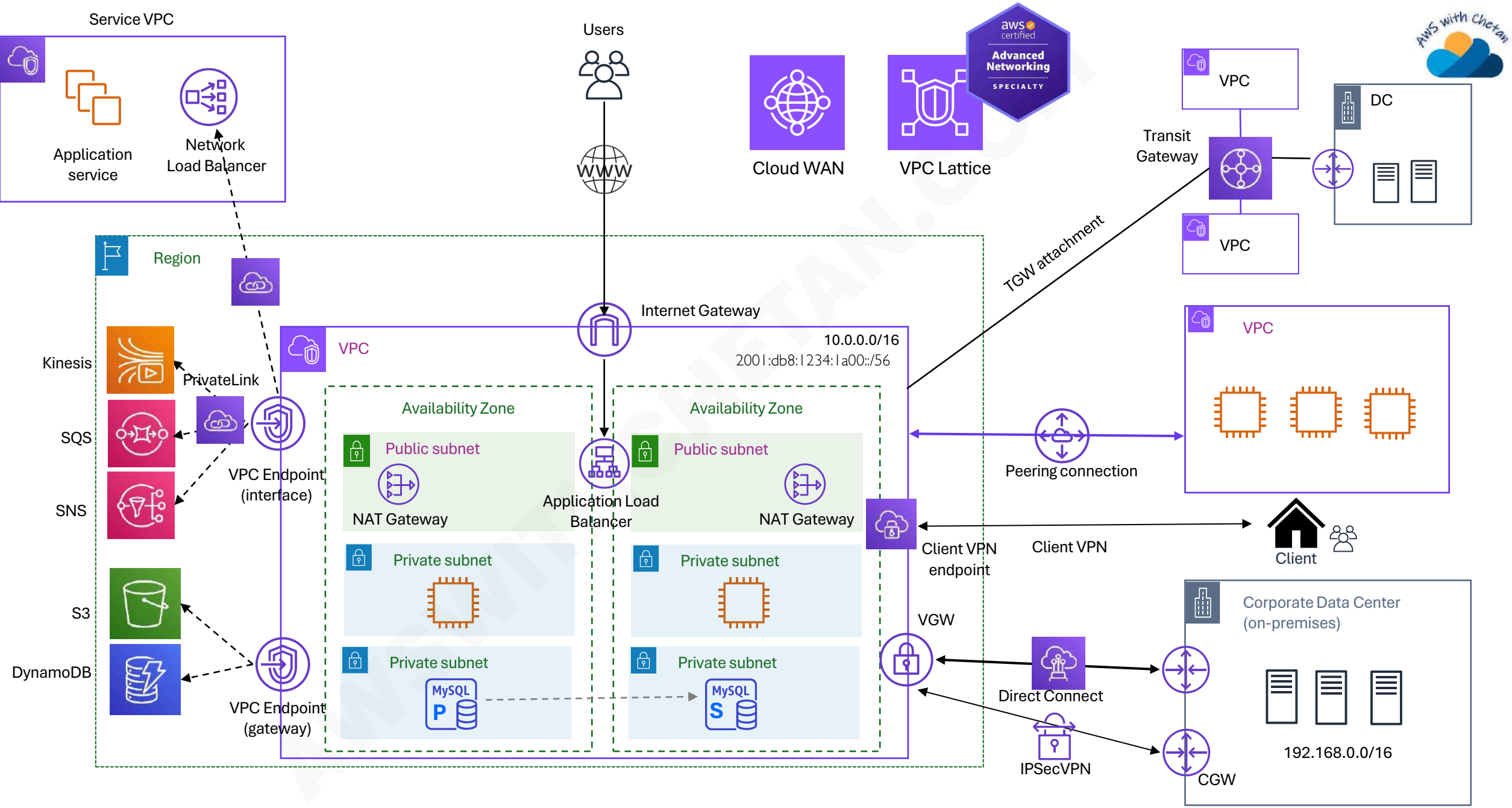


# Transit Gateway



# Transit Gateway





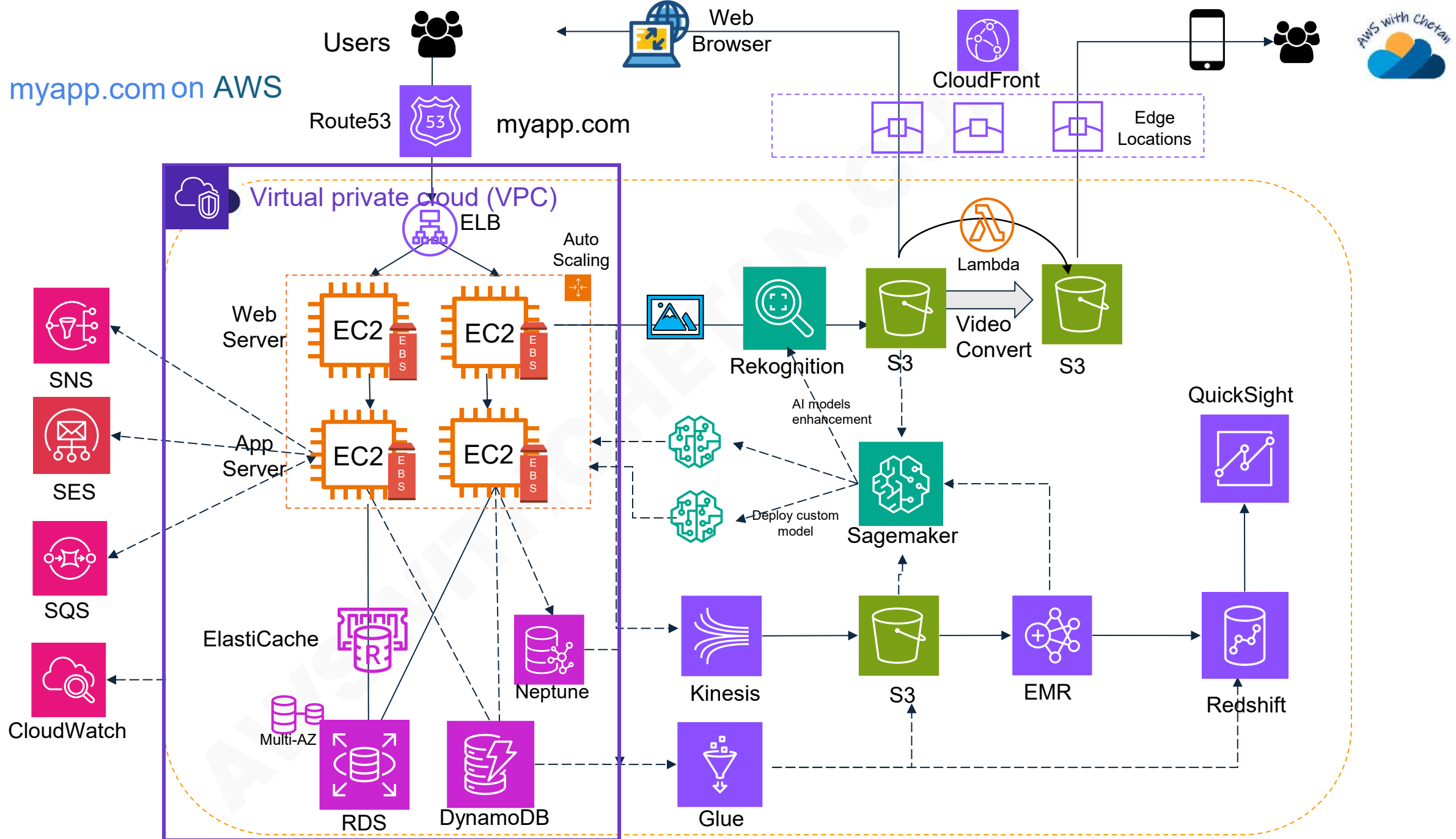
# In this section..

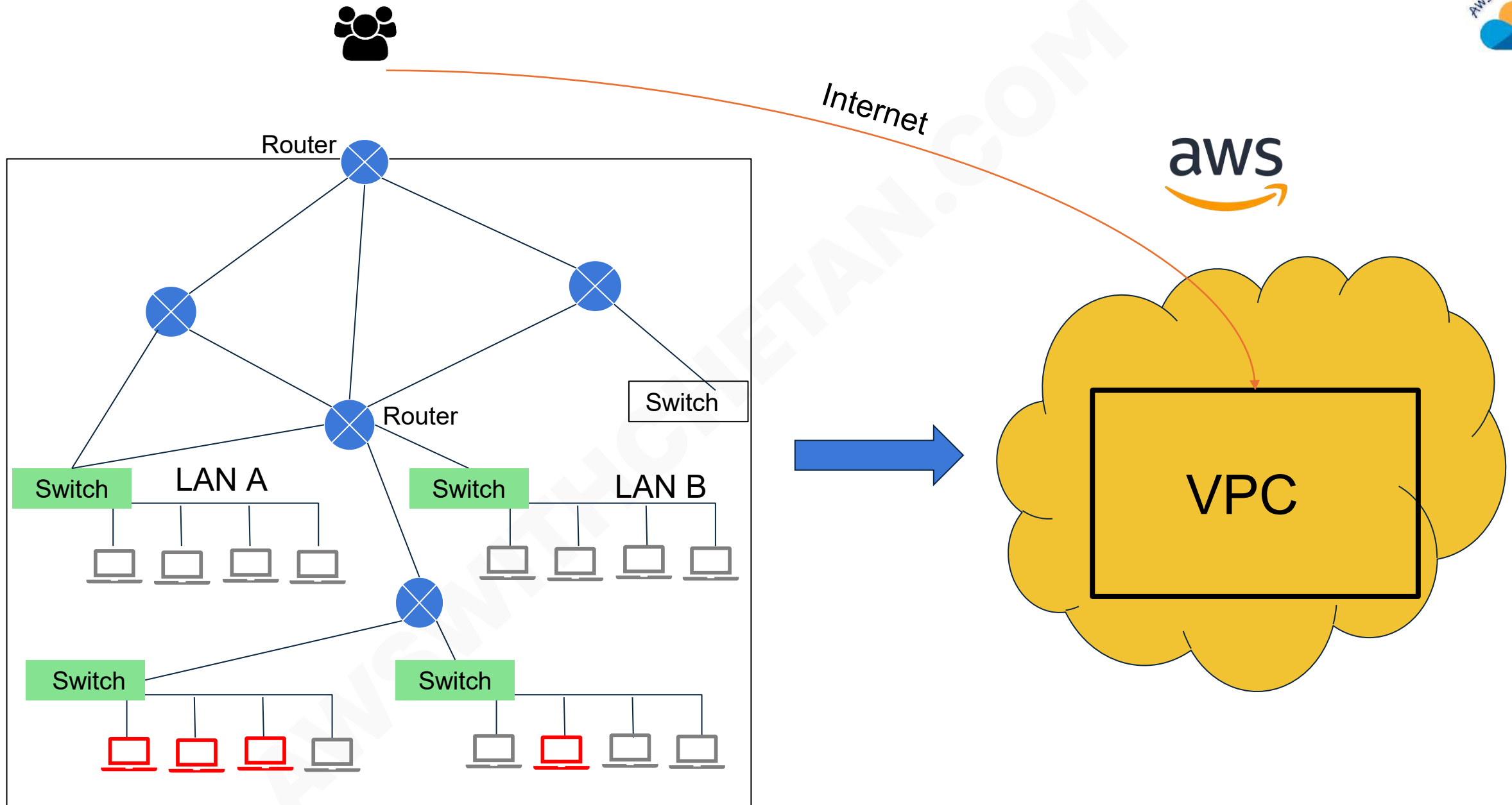
- VPC and it's components
  - Subnets, Route tables, Internet gateway, NAT gateways
  - Security group and Network ACL
  - VPC endpoints and PrivateLink
- VPC Private Connectivity Options
  - VPC Peering
  - Transit Gateway
- Hybrid Networking
  - Site-to-Site VPN
  - Direct Connect
- Network Monitoring
  - VPC Flow Logs
  - VPC Traffic Mirroring
- IPv6
  - IPv6 in VPC and Dual-stack
  - Egress-only Internet Gateway



# Amazon VPC

# myapp.com on AWS

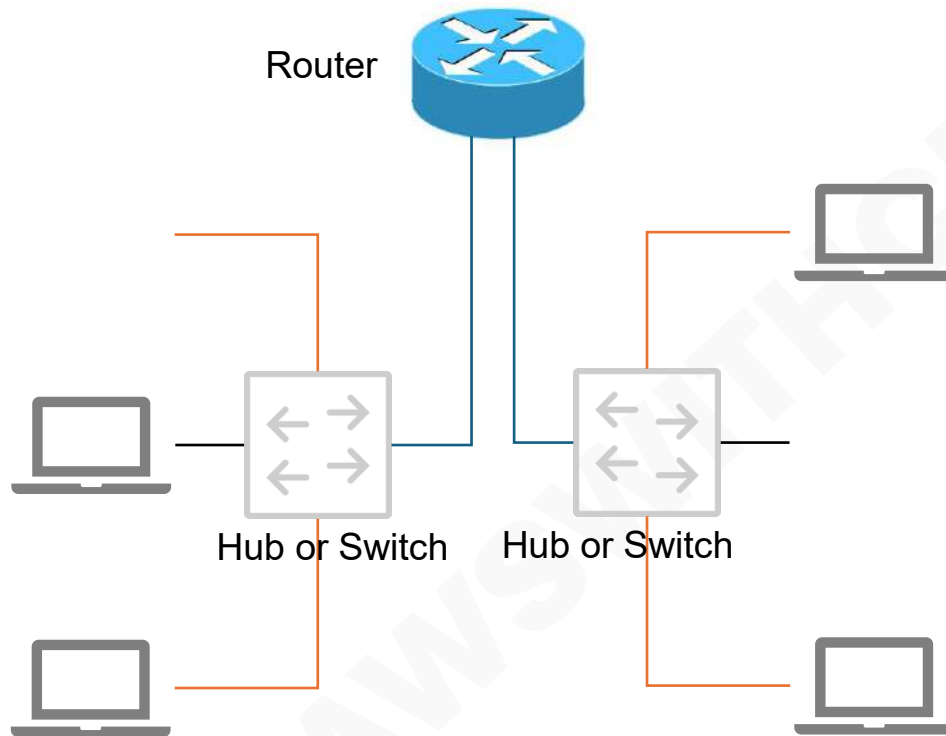




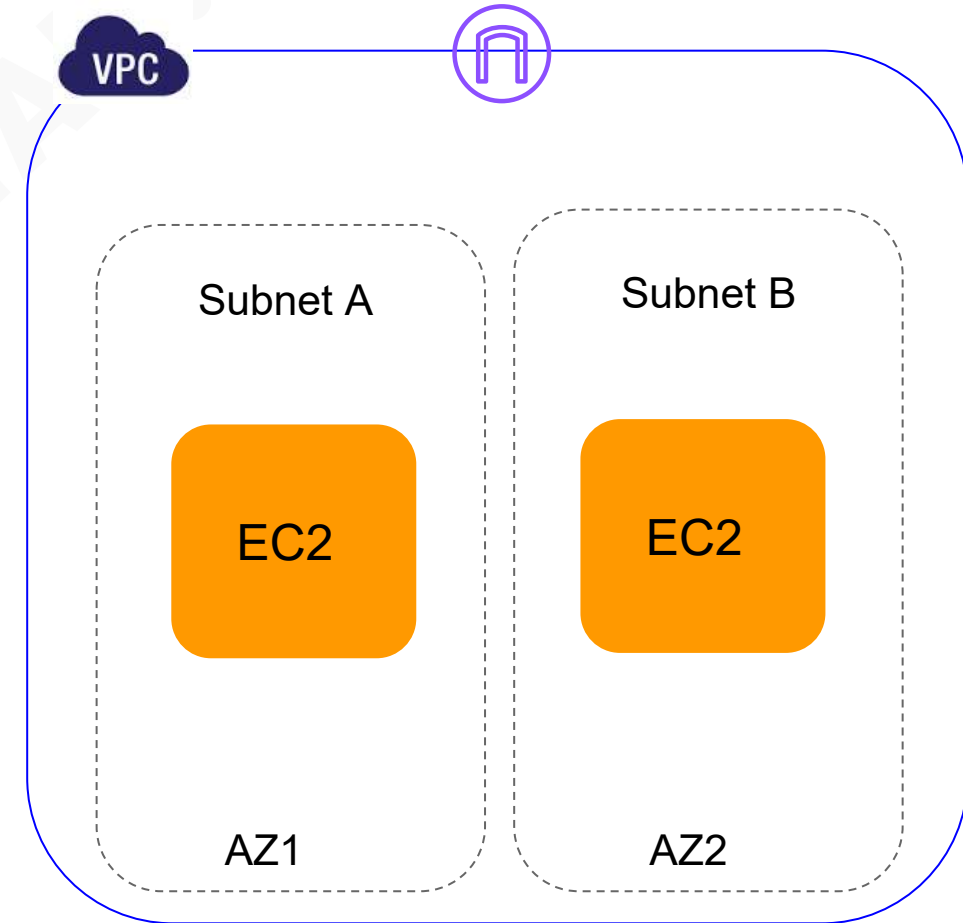


# Traditional IT network vs VPC

## Physical Network



## AWS VPC

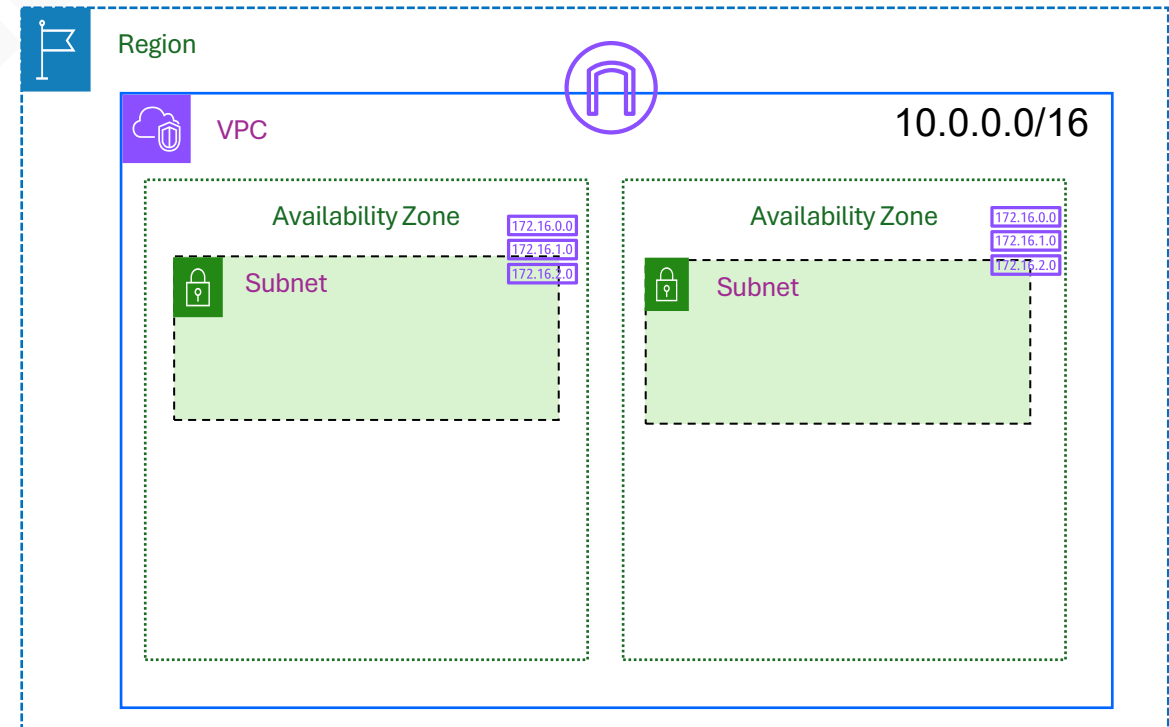


# Amazon Virtual Private Cloud (VPC)

- A logically isolated virtual network in the cloud which closely resembles the traditional IT network
- VPC is assigned a Private IP Address range (called CIDR) e.g. 10.0.0.0/16 (IPv4)
- VPC can have both IPv4 (32 bit) or IPv6 (128 bit) IP addresses

## VPC building blocks:

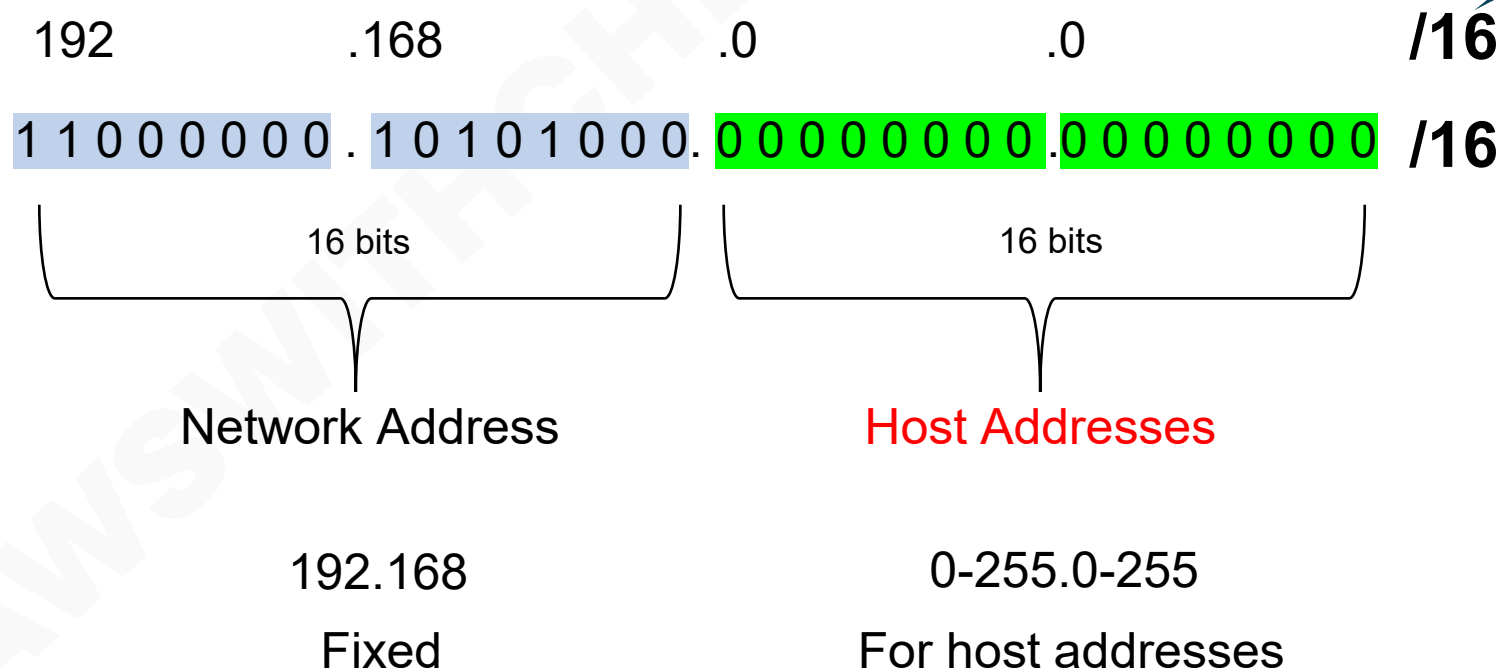
- VPC CIDR
- Subnets and Route table
- IP Addresses – IPv4 and IPv6
- Internet Gateway
- NAT Gateway
- VPC Firewalls - Security Group and Network ACL



# VPC Addressing - CIDR

# CIDR – Classless Inter Domain Routing

- IP addressing scheme that replaces old address style of Class A, B, C
- Represented as an IP address and prefix
  - Example: IPv4 CIDR 192.168.0.0/16



# CIDR – Classless Inter Domain Routing

Formula to calculate number of available IP addresses inside the VPC or Subnets

- Total addresses =  $2^{\text{(total bits – prefix)}}$

Examples:

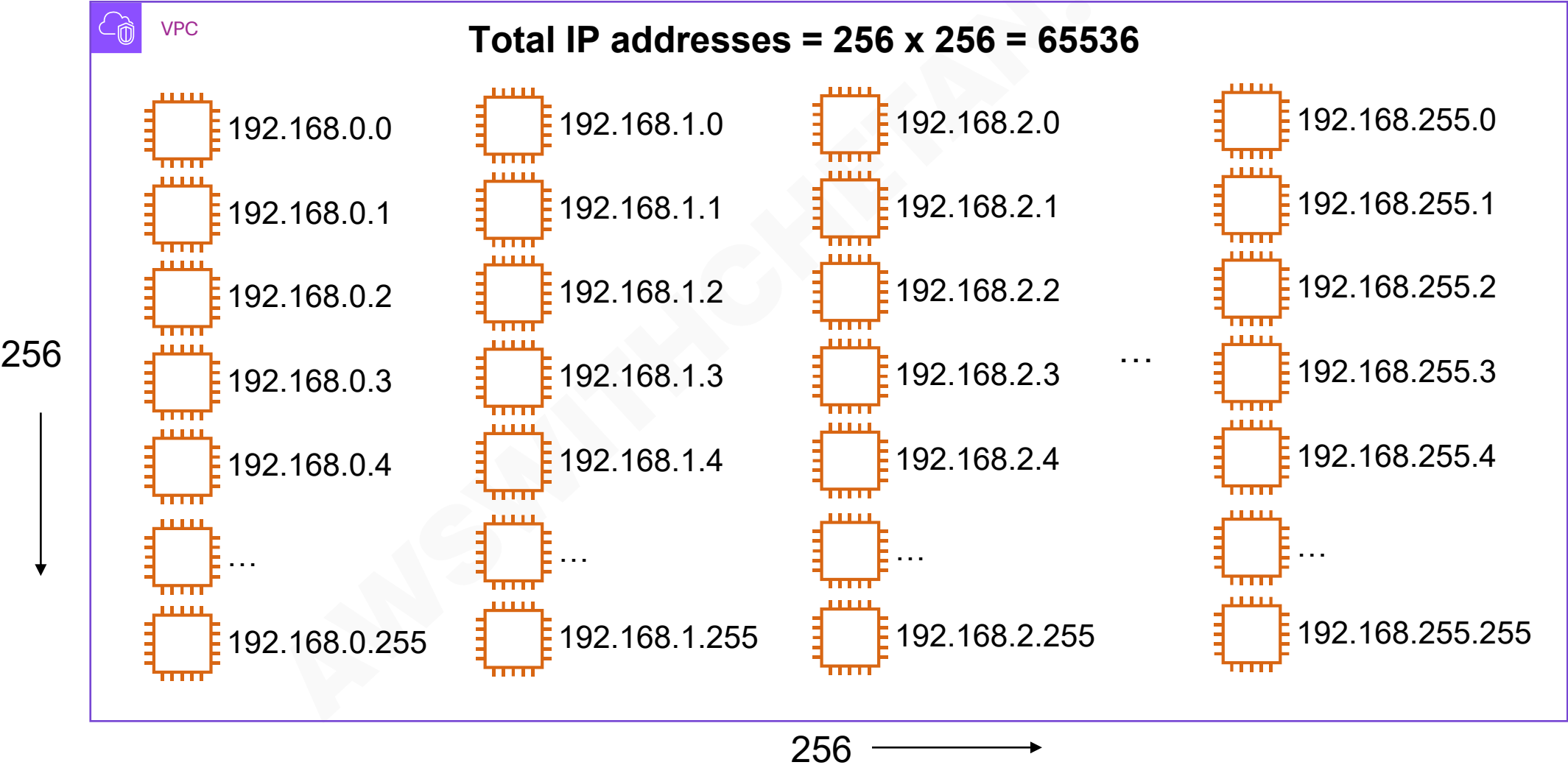
- $192.168.0.0/16 \Rightarrow 2^{(32 - 16)} = 2^{16} = 65536$
- $192.168.0.0/24 \Rightarrow 2^{(32 - 24)} = 2^8 = 256$

192                      .168                      .0                      .0  
 1 1 0 0 0 0 0 0 . 1 0 1 0 1 0 0 0 . 0 0 0 0 0 0 0 0 . 0 0 0 0 0 0 0 0 /16

192                      .168                      .0                      .0  
 1 1 0 0 0 0 0 0 . 1 0 1 0 1 0 0 0 . 0 0 0 0 0 0 0 0 . 0 0 0 0 0 0 0 0 /24

# CIDR – Classless Inter Domain Routing

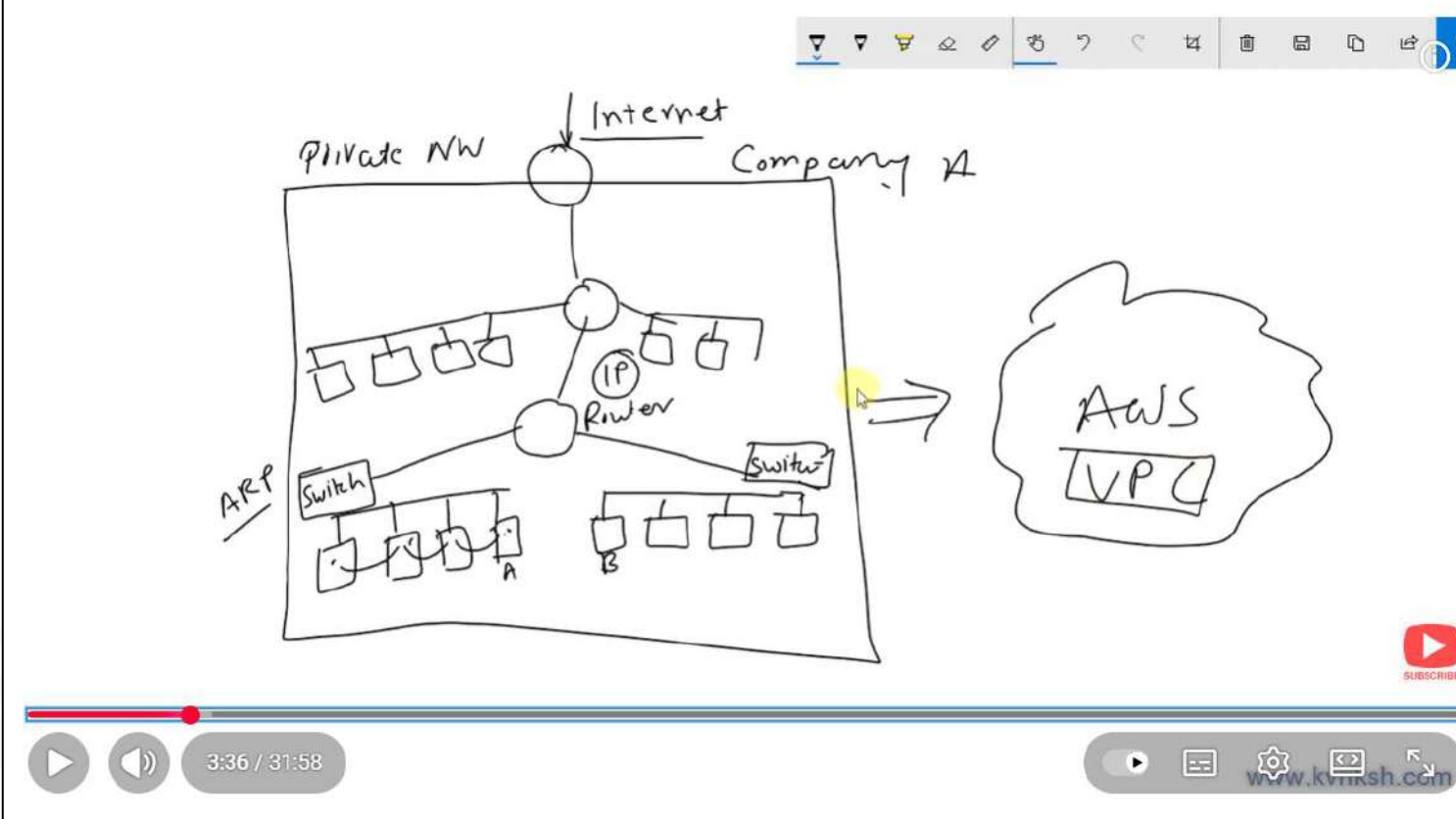
192.168.0.0/16



# VPC Addressing

- **AWS VPC CIDR (IPv4)**
  - VPC prefix between /16 (65536 IPs) and /28 (16 IPs)
  - RFC 1918 IP ranges for Private network and corresponding AWS recommended ranges
    - 10.0.0.0/8 => 10.0.0.0 – 10.255.255.255    => **AWS CIDR 10.X.0.0/16**
    - 172.16.0.0/12 => 172.16.0.0 - 172.31.255.255    => **AWS CIDR 172.16.0.0/16 to 172.31.0.0/16**
    - 192.168.0.0/16 => 192.168.0.0 - 192.168.255.255    => **AWS CIDR 192.168.0.0/16**
- **AWS VPC CIDR (IPv6)**
  - VPC CIDR with prefix /56 ( $2^{72}$  IPs)
  - IPv6 CIDR is allocated by AWS
  - IPv6 IP addresses are globally unique and publicly routable

# More about CIDR.. Watch this youtube video



The diagram illustrates a Private Network (Company A) connected to the Internet. The Private Network contains a Router, two Switches (labeled 'Switch' and 'Switch'), and several Servers (labeled 'A' and 'B'). The Router is connected to the Internet. The diagram shows the mapping of the Private Network to an AWS VPC (Virtual Private Cloud) cloud.

Video player controls: 3:36 / 31:58

Video title: AWS VPC Basics - Understanding what is VPC and Calculating CIDR for VPC and Subnets

<https://youtu.be/O3fgul-fJCK>



# Exercise – Create a VPC



- Create a VPC with CIDR 10.10.0.0/16

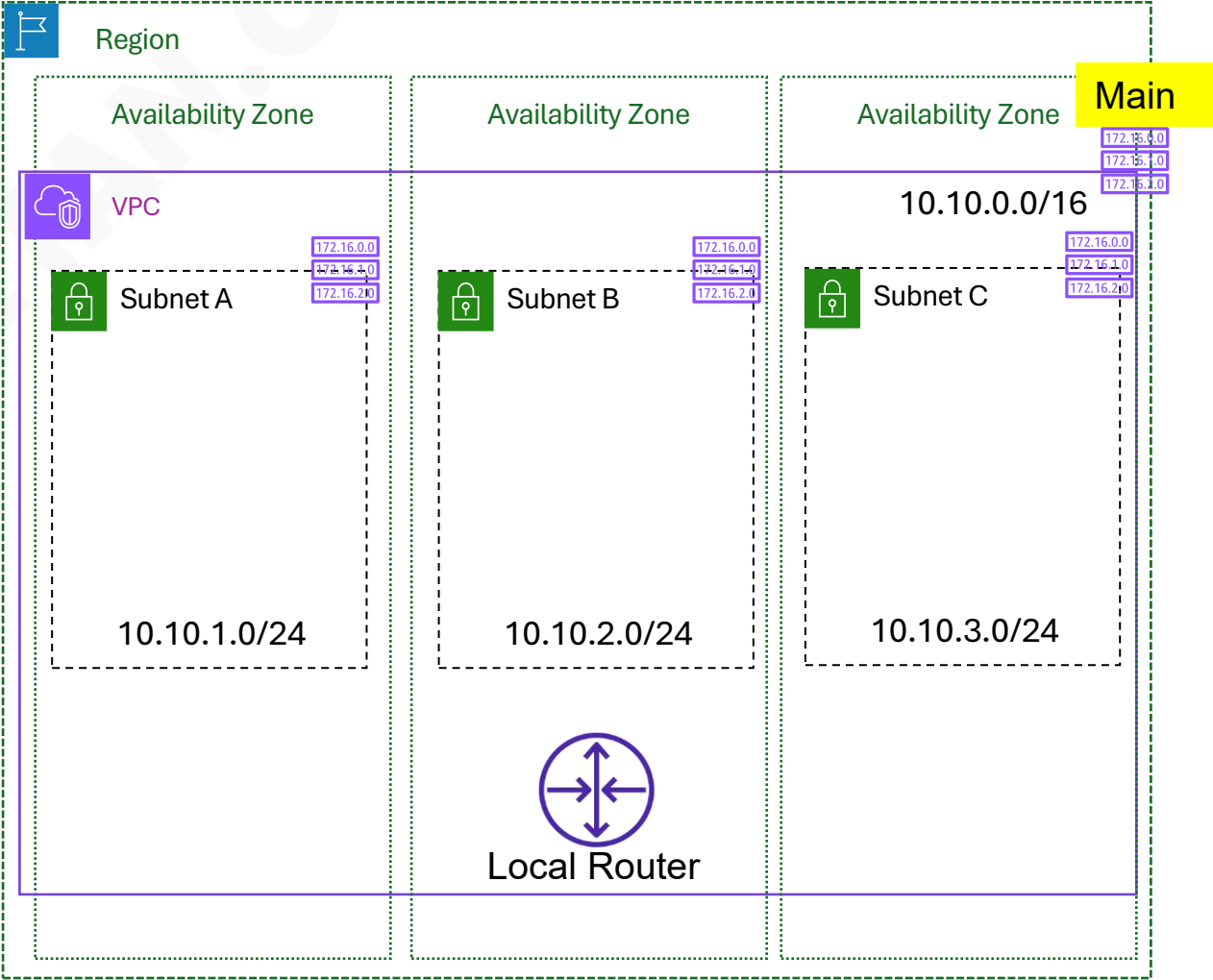
# Subnets, Route tables & Internet gateway

# VPC Subnets

- VPC network is partitioned into smaller networks called Subnets
- Subnets are created inside a specific Availability Zone
- VPC has a local router which routes traffic within the VPC
- Route tables defines the routing logic for the subnets
- VPC has a main (default) route table. All subnets follow main route table by default.

| Destination  | Target |
|--------------|--------|
| 10.10.0.0/16 | Local  |

- We can create subnet specific route tables.



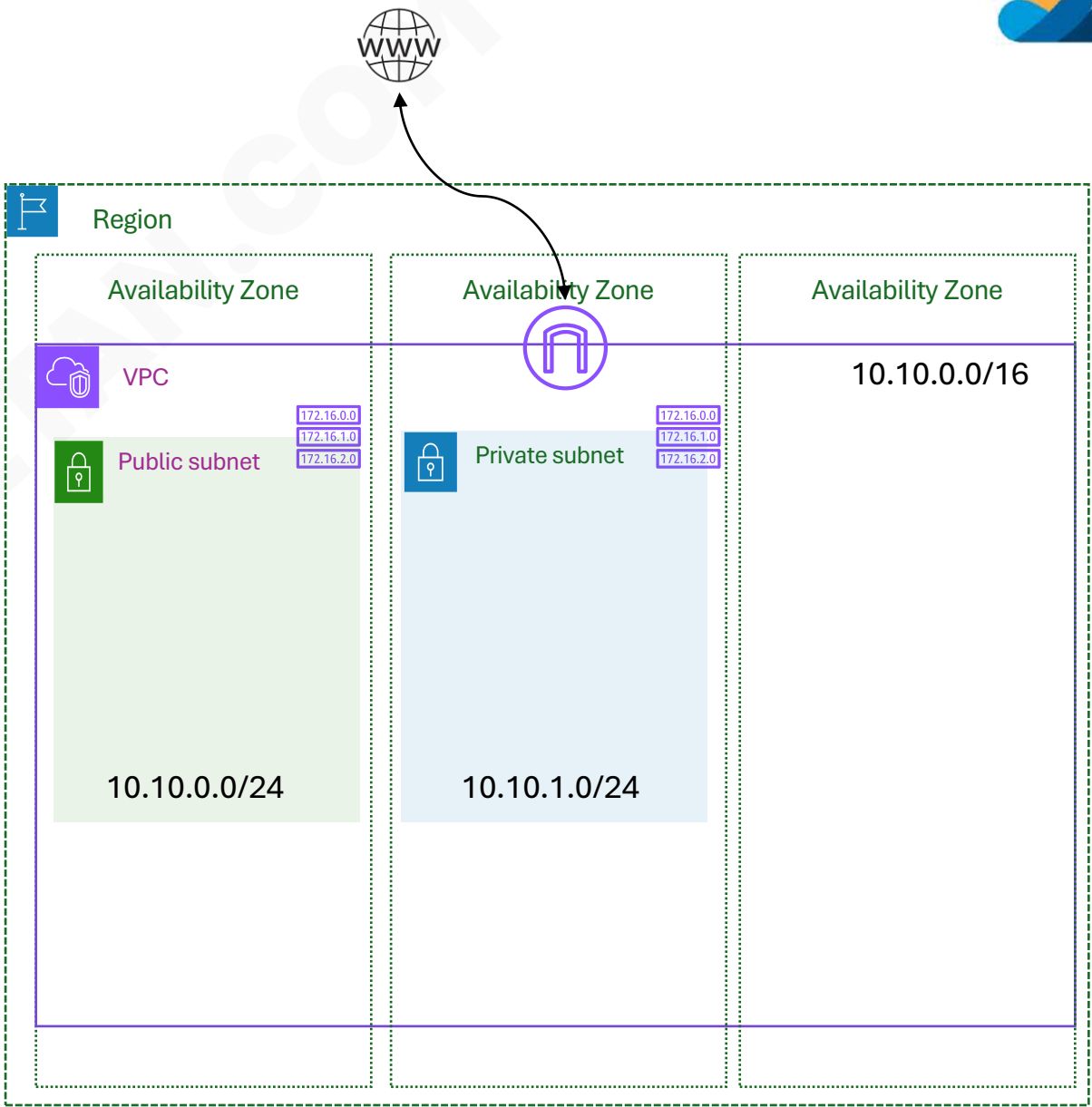
# Internet Gateway

- Internet gateway connects VPC to the internet
- If subnet route table has route to internet via the internet gateway, it's called Public subnet

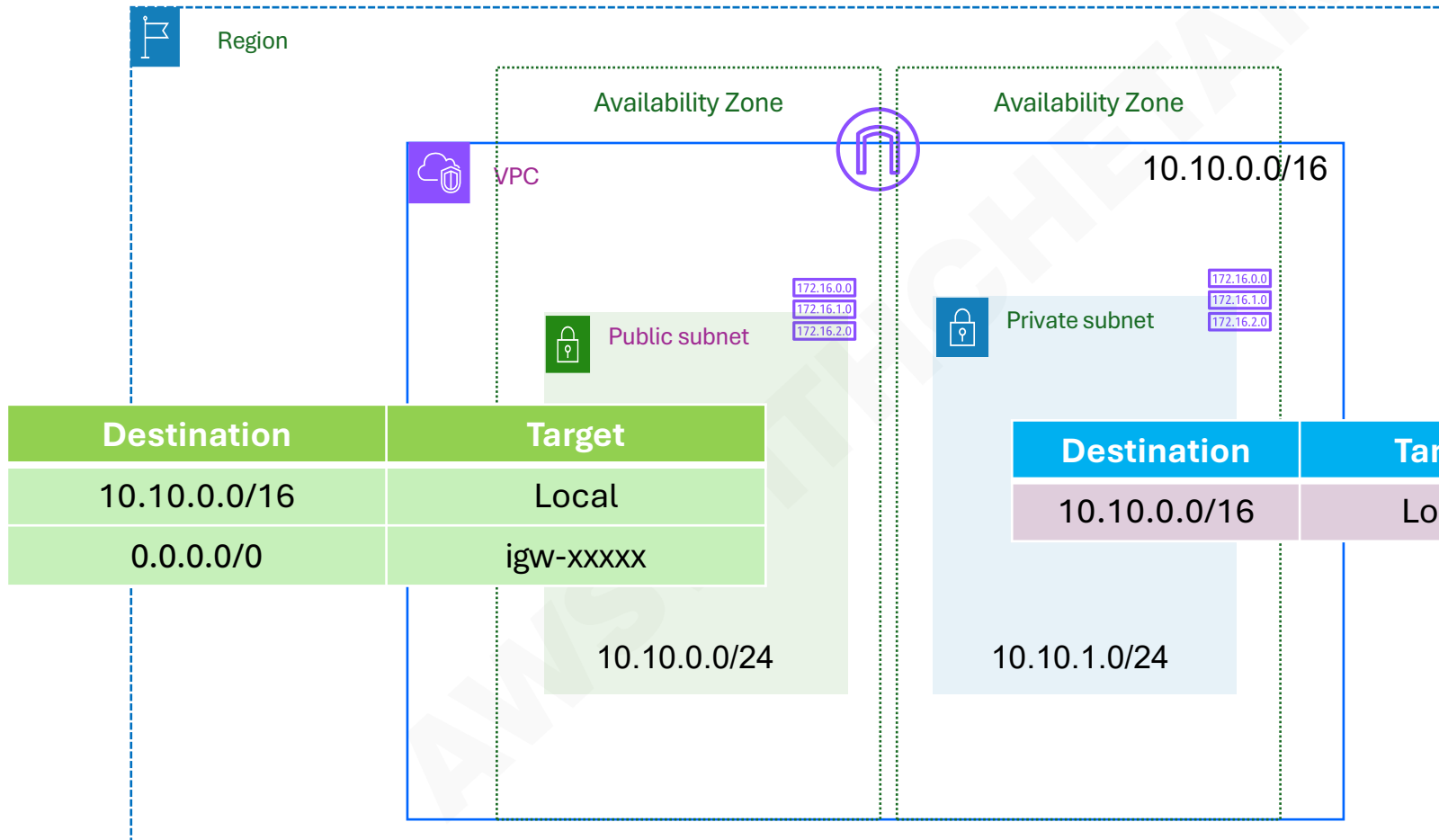
| Destination  | Target                   |
|--------------|--------------------------|
| 10.10.0.0/16 | Local                    |
| 0.0.0.0/0    | Internet Gateway (igw-x) |

- If subnet route table does not have route to internet, it's called a Private subnet

| Destination  | Target |
|--------------|--------|
| 10.10.0.0/16 | Local  |

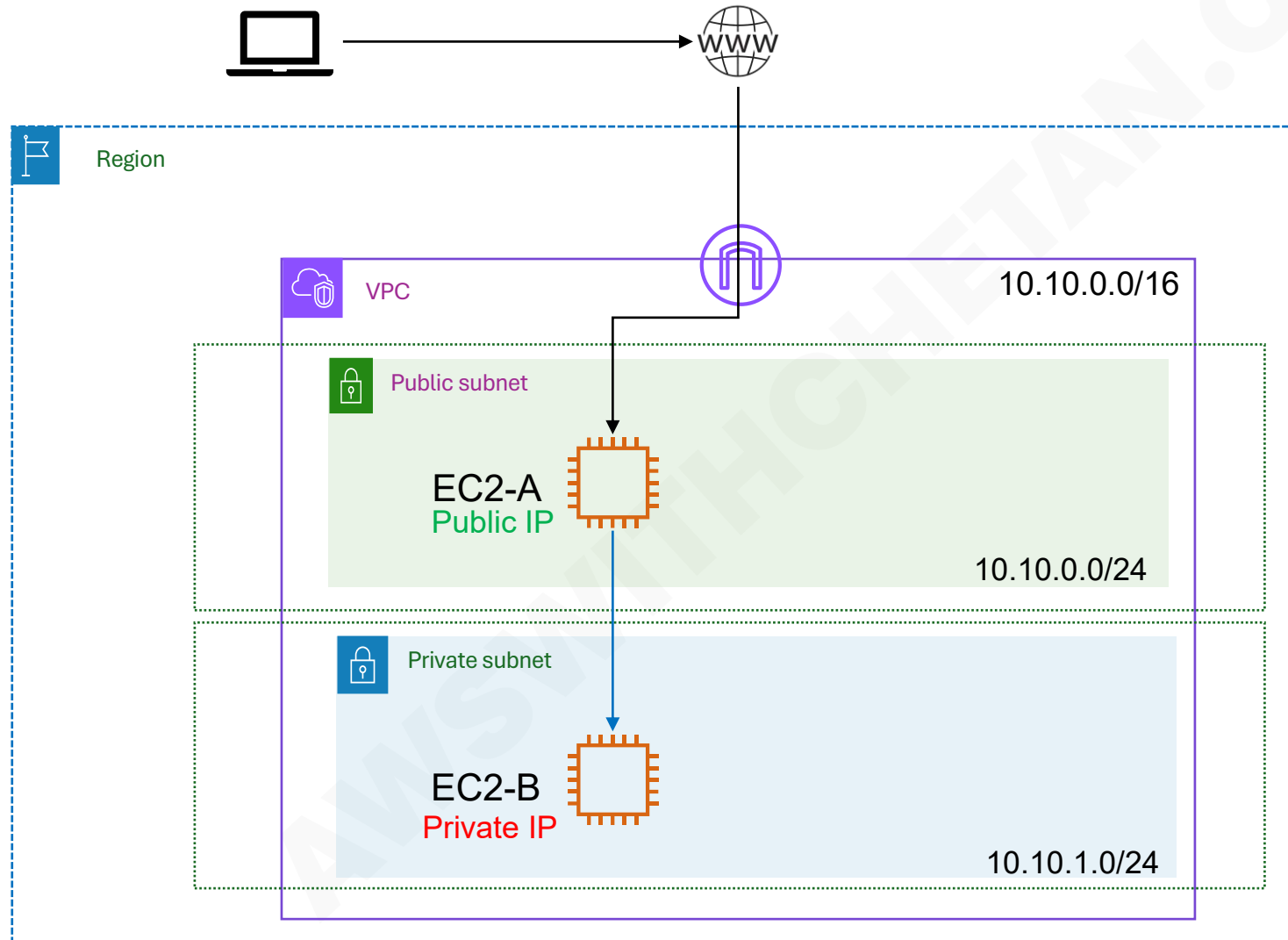


# Exercise – Add Internet gateway, Public/Private subnets



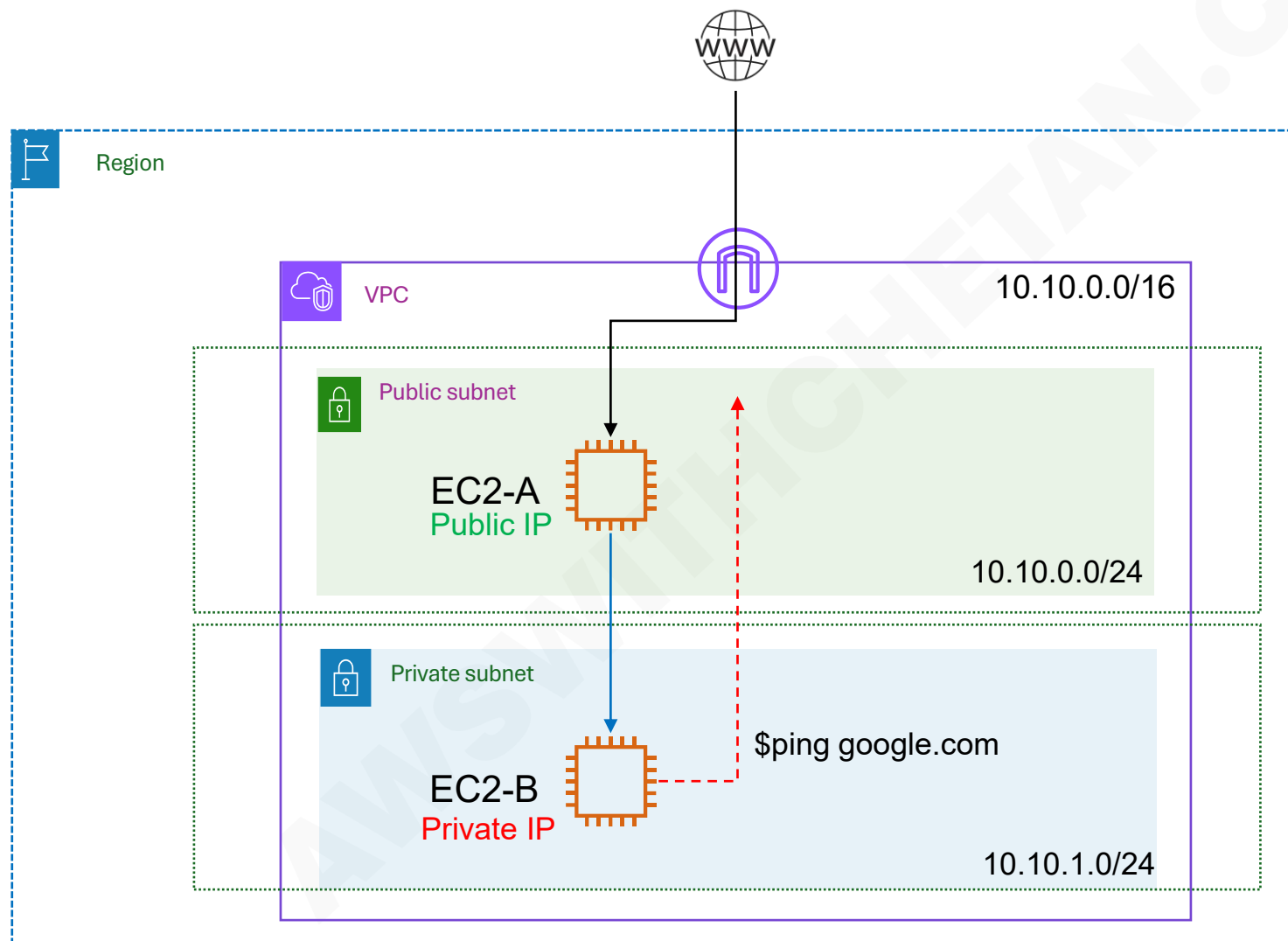
- Create a VPC with CIDR 10.10.0.0/16
- Create an Internet Gateway and associate with the VPC
- Create 2 subnets in 2 different availability zones
- Create a route table, add route entry for internet and associate with first subnet (Public subnet)
- Create another route table and associate with another subnet (Private subnet)

# Exercise – Launch EC2 instances



- Launch EC2-A in Public subnet (assign Public IP)
- Launch EC2-B in the Private subnet (no Public IP)
- Connect to EC2-A over SSH from your workstation.
- From EC2-A, SSH into EC2-B (for this you will need ssh private key on EC2-A instance)

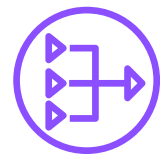
# Exercise – Launch EC2 instances



- Launch EC2-A in Public subnet (assign Public IP)
- Launch EC2-B in the Private subnet (no Public IP)
- Connect to EC2-A over SSH from your workstation.
- From EC2-A, SSH into EC2-B (for this you will need ssh private key on EC2-A instance)
- After logged into EC2-B, try to ping to google.com

## Does it work?

- EC2-B is not in a Public subnet
- EC2-B does not have a Public IP

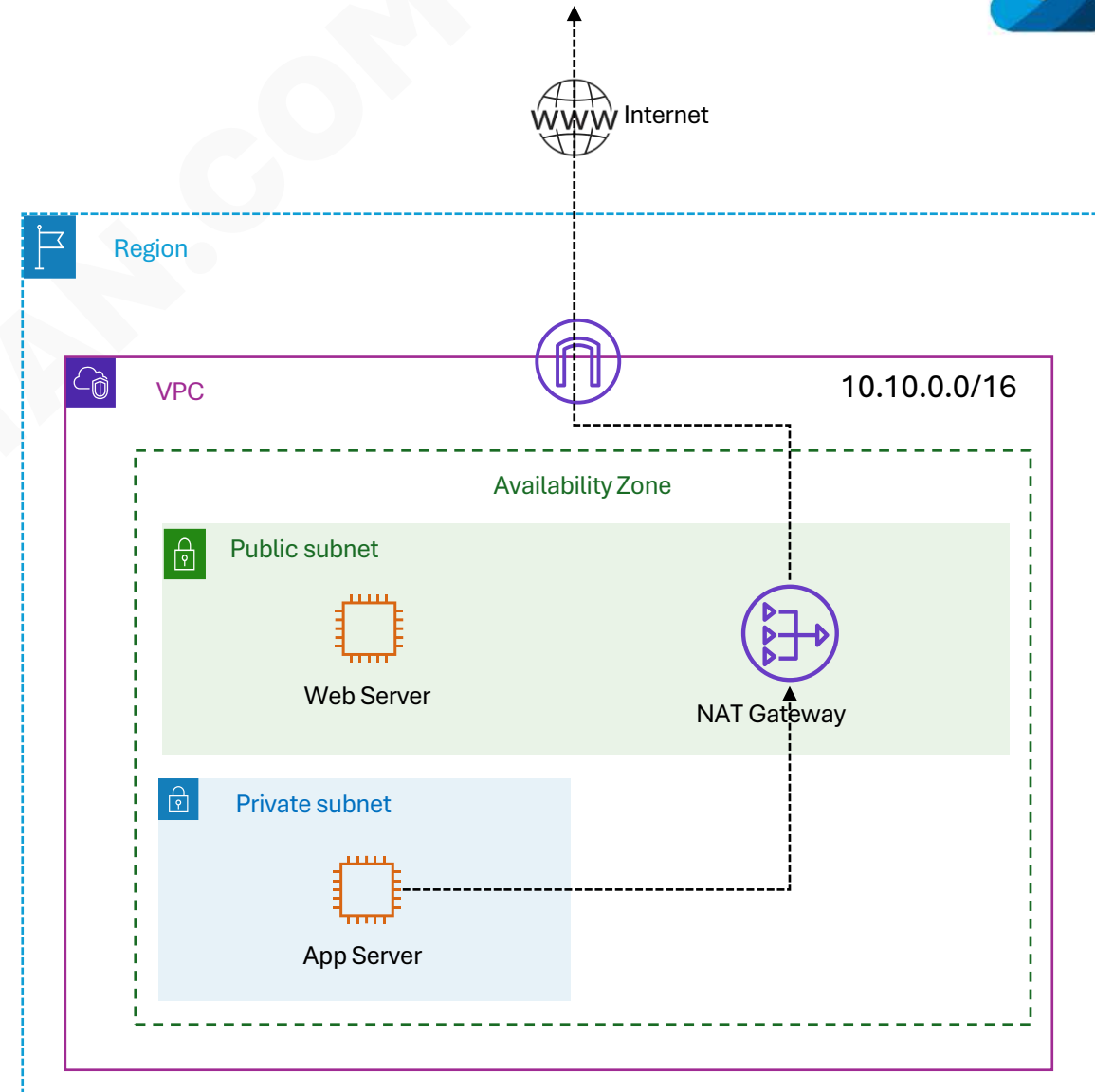


# NAT Gateway



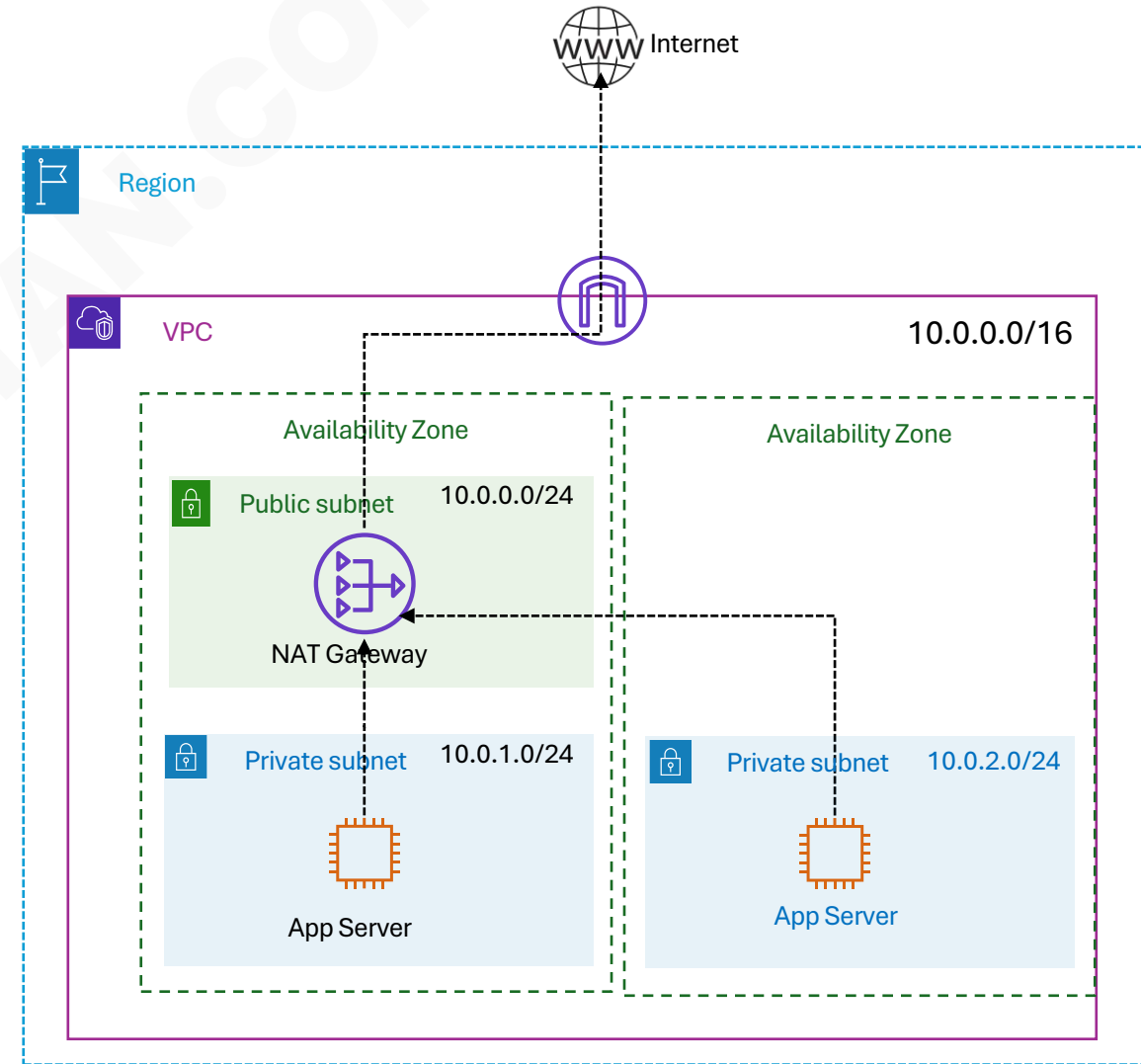
# NAT Gateway

- NAT Gateway allows instances in a private subnet to connect to services outside of the VPC, but external services cannot initiate a connection with those instances.
- It's AWS managed providing higher bandwidth, better availability, no administration
- Pay by the hour for usage and bandwidth
- 5 Gbps of bandwidth with automatic scaling up to 100 Gbps
- No security groups
- Supported protocols: TCP, UDP, and ICMP
- For outbound internet access, NAT Gateway should be created in Public Subnet and should be allocated an Elastic IP



# NAT Gateway High availability

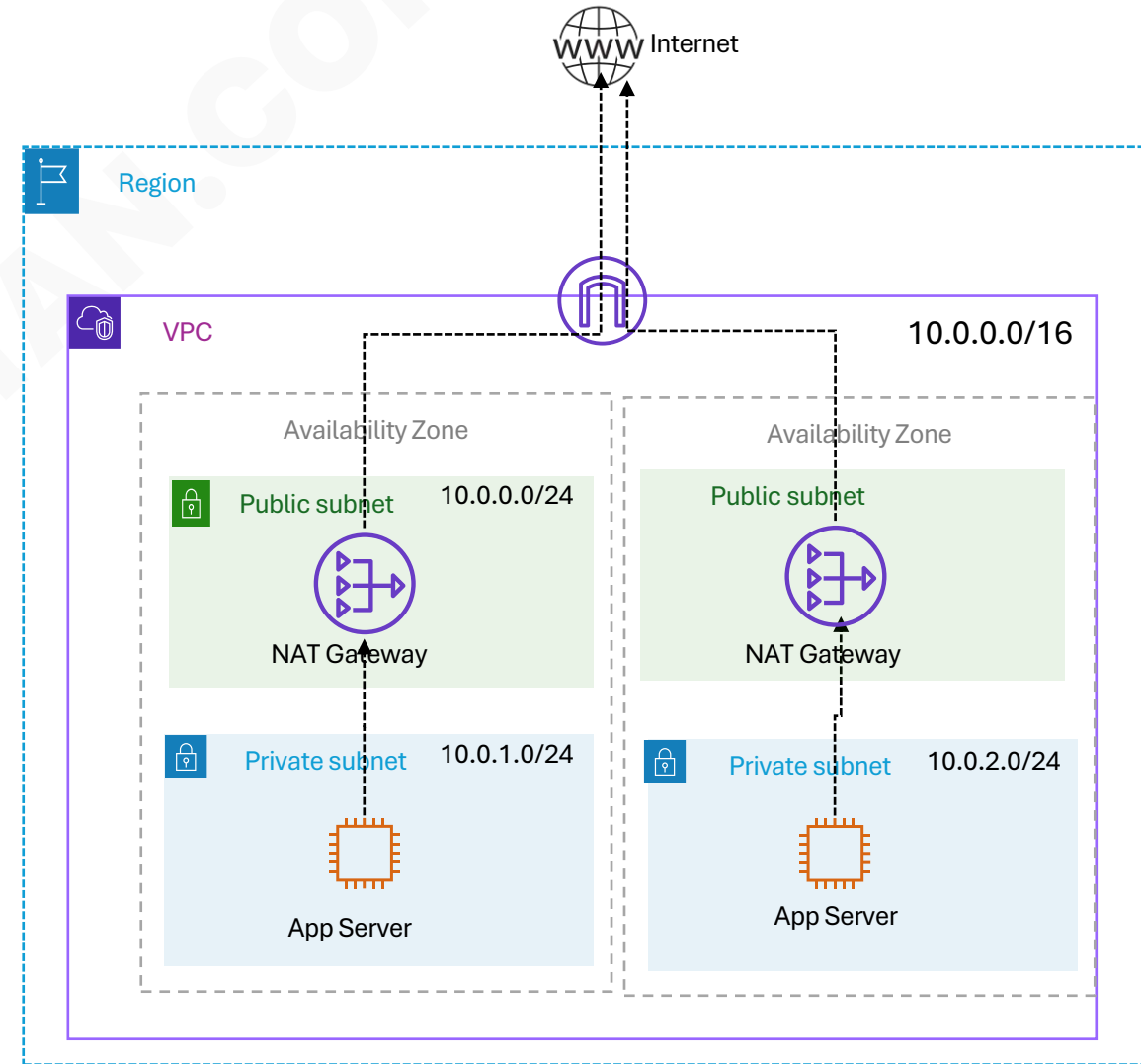
- NAT Gateways are highly available within a single AZ
- For HA across multiple AZs, multiple NAT gateways can be launched



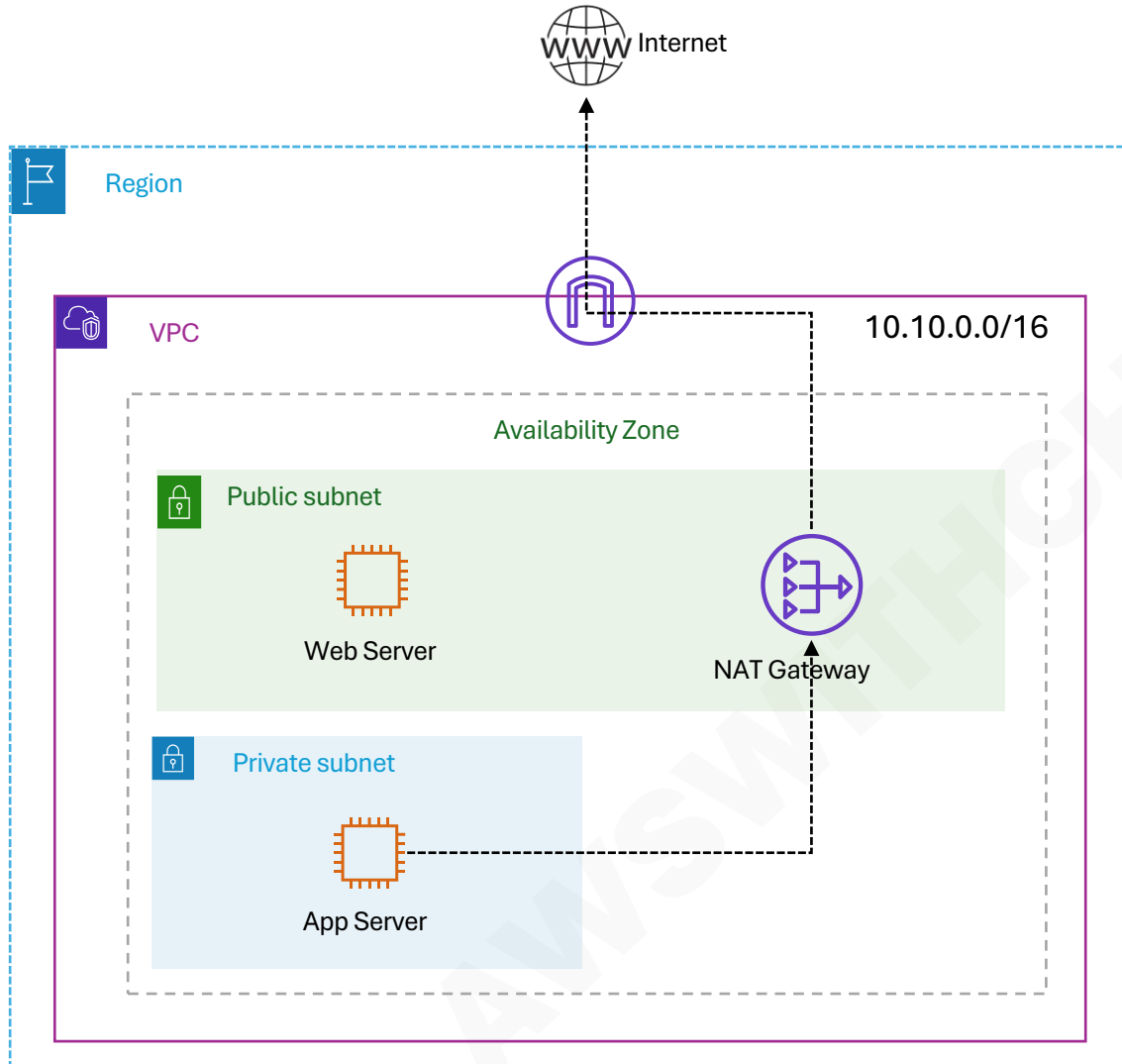
# NAT Gateway High availability

- NAT Gateways are highly available within a single AZ
- For HA across multiple AZs, multiple NAT gateways can be launched

AWS has launched Regional NAT gateway during Re:Invent 2025



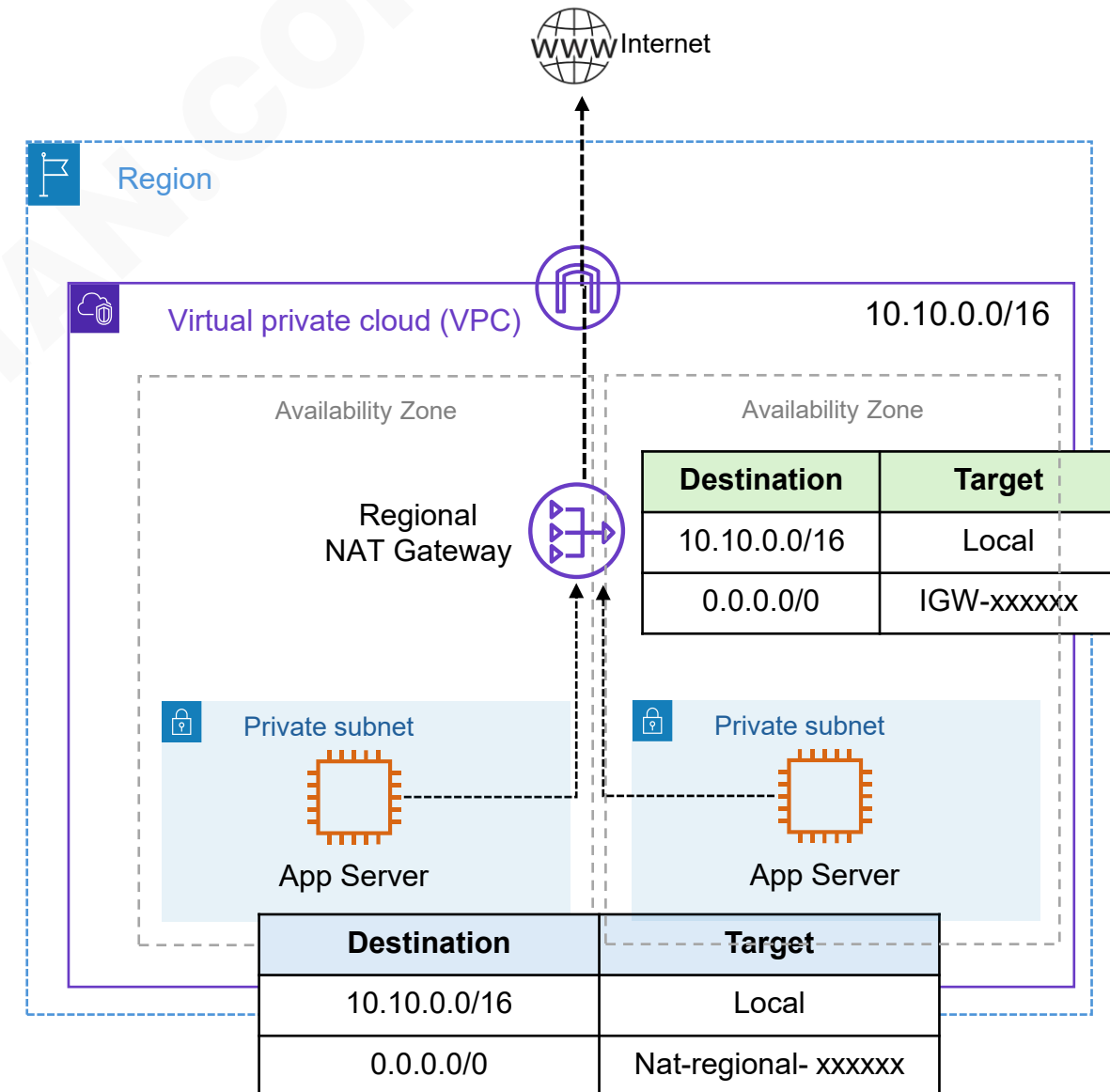
# Exercise - NAT Gateway



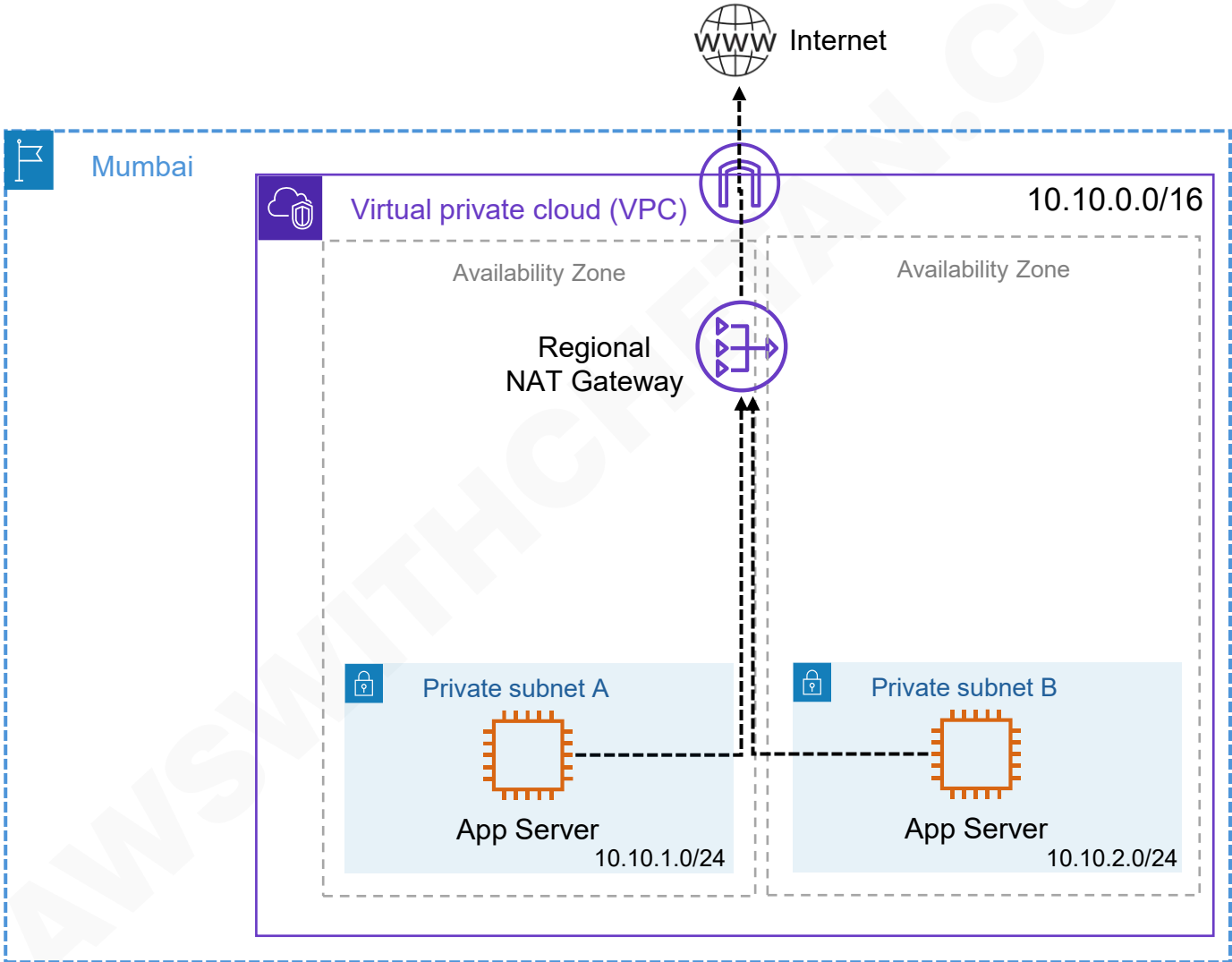
- Launch EC2-A in Public subnet (assign Public IP)
- Launch EC2-B in the Private subnet (no Public IP)
- Connect to EC2-A over SSH from your workstation.
- From EC2-A
- Create another Public subnet C
- Create NAT gateway in this new subnet.
- Update Private subnet route table and add entry for the destination 0.0.0.0/0 with target as NAT gateway.
- Check the outbound internet connectivity from EC2-B

# Regional NAT Gateway (New)

- Regional NAT gateway (RNAT) works at VPC level and spans across multiple Availability Zones
- Automatically expands into newer AZs as you deploy your workloads
- Optionally can choose a manual mode to configure AZs for the RNAT
- Maintains Zonal affinity (Saves inter-AZ Data transfer charge)
- No need to have Public subnets for the regional NAT gateway
- Comes with its own Route Table



# Regional NAT Gateway



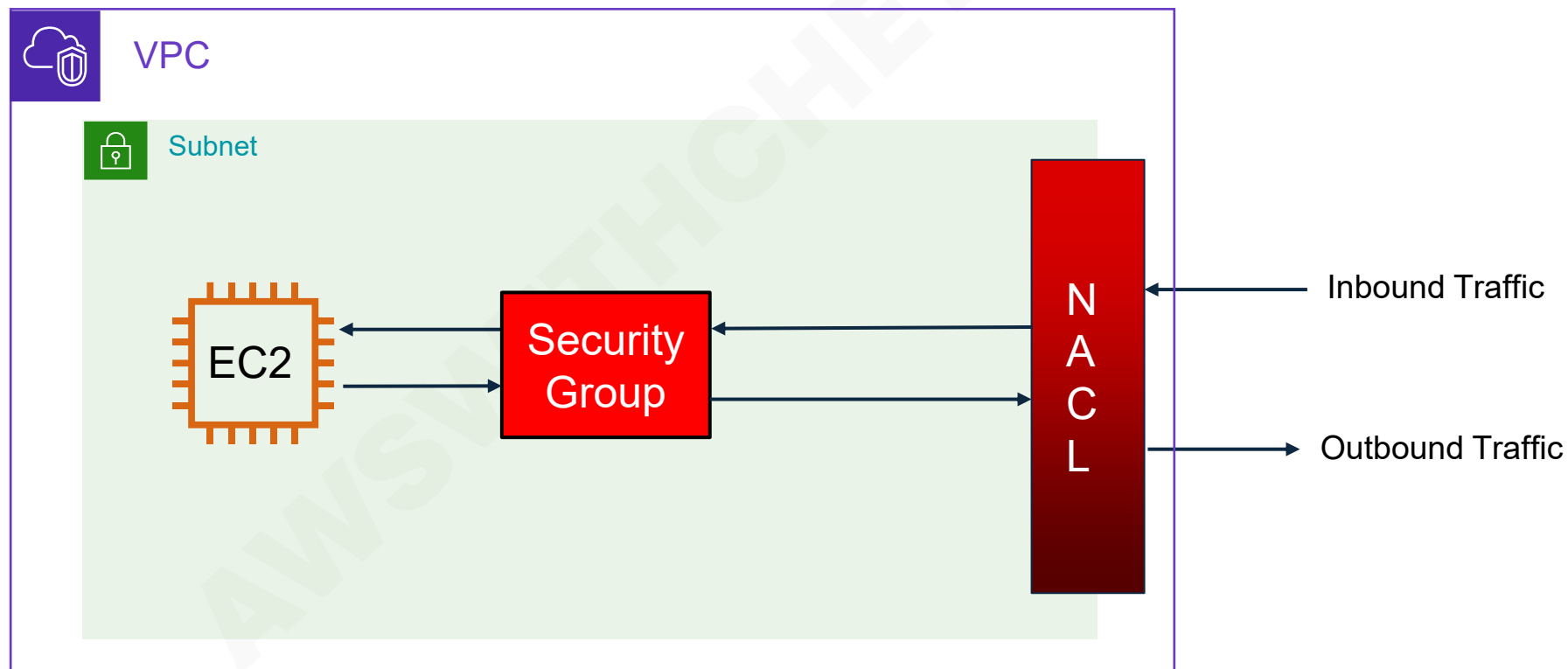
| Destination  | Target     |
|--------------|------------|
| 10.10.0.0/16 | Local      |
| 0.0.0.0/0    | IGW-xxxxxx |

| Destination  | Target               |
|--------------|----------------------|
| 10.10.0.0/16 | Local                |
| 0.0.0.0/0    | Nat-regional- xxxxxx |

# Security group and Network ACL

# Firewalls in VPC

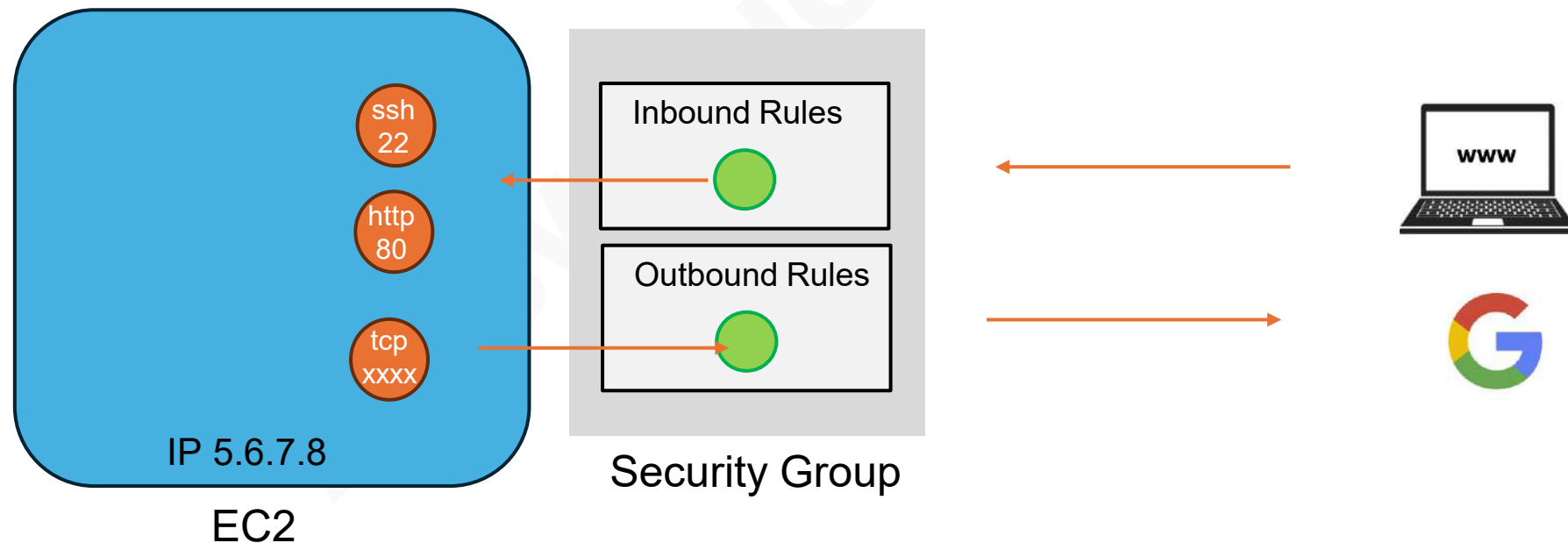
- Security Groups
- Network Access Control List (NACL)





# Security Group - Recap

- Security Groups are most basic, native and important firewall for EC2 instances
- Security group has Inbound and Outbound rules
- Security group has only ALLOW rules. Does not support DENY/Block rules.
- Default Security group in each VPC
- Authorises traffic for both IPv4 and IPv6 traffic
- **Security groups are stateful** – return traffic is automatically allowed



# Default Security Group

| Inbound              |          |            |                                                                                                                                  |
|----------------------|----------|------------|----------------------------------------------------------------------------------------------------------------------------------|
| Source               | Protocol | Port range | Description                                                                                                                      |
| sg-1234567890abcdef0 | All      | All        | Allows inbound traffic from all resources that are assigned to this security group. The source is the ID of this security group. |

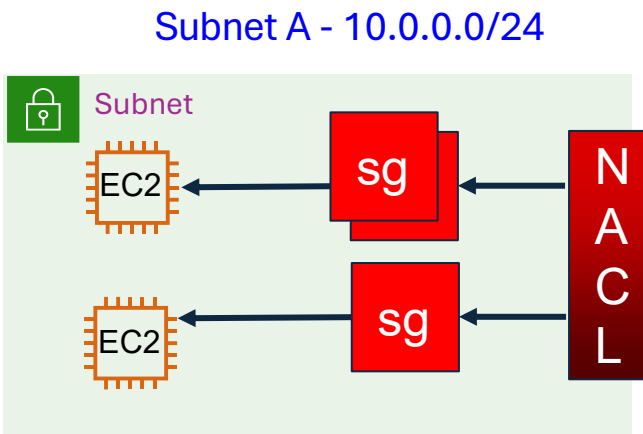
| Outbound    |          |            |                                                                                                          |
|-------------|----------|------------|----------------------------------------------------------------------------------------------------------|
| Destination | Protocol | Port range | Description                                                                                              |
| 0.0.0.0/0   | All      | All        | Allows all outbound IPv4 traffic.                                                                        |
| ::/0        | All      | All        | Allows all outbound IPv6 traffic. This rule is added only if your VPC has an associated IPv6 CIDR block. |



# Network ACLs

# Network Access Control List (NACL)

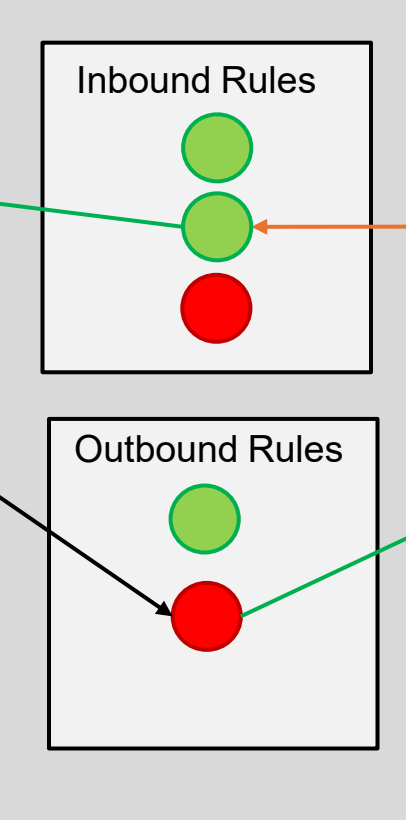
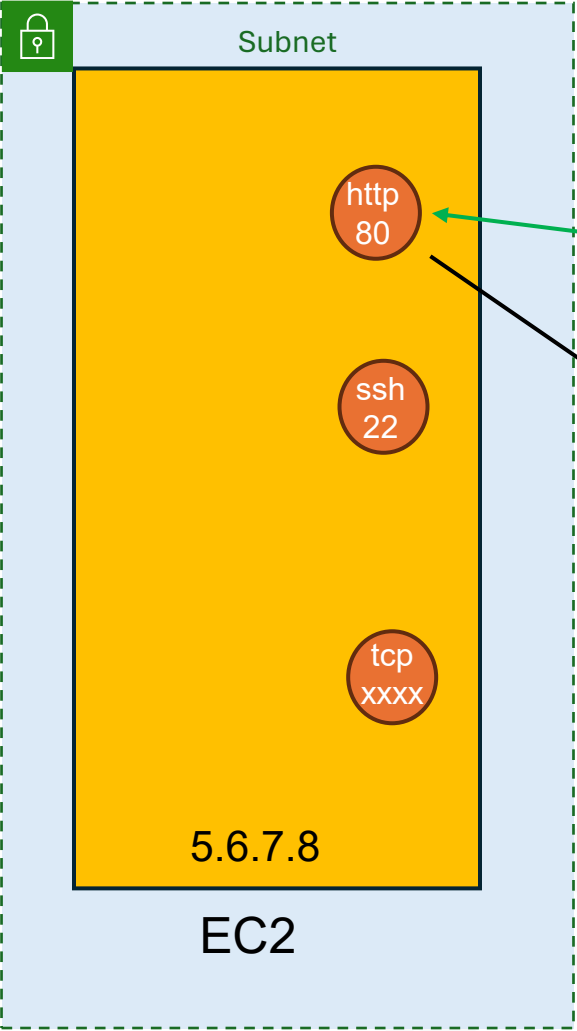
- Works at Subnet level – Hence automatically applied to all instances
- Contains both Allow and Deny rules. Rules are numbered.
- Rules are evaluated in the order of rule number (1 to 32766)
- Stateless – We need to explicitly open ports for return traffic
- Default NACL allows all inbound and outbound traffic
- **NACL are a great way of blocking a specific IP at the subnet level**



Network ACL inbound rules

| #Rule | Type             | Protocol | Port | Source            | Allow/Deny |
|-------|------------------|----------|------|-------------------|------------|
| 100   | All IPv4 traffic | All      | All  | 180.151.138.43/32 | DENY       |
| 101   | HTTPS            | TCP      | 443  | 0.0.0.0/0         | ALLOW      |
| *     | All IPv4 traffic | All      | All  | 0.0.0.0/0         | DENY       |

# Network ACL – Allow return traffic



Network ACL

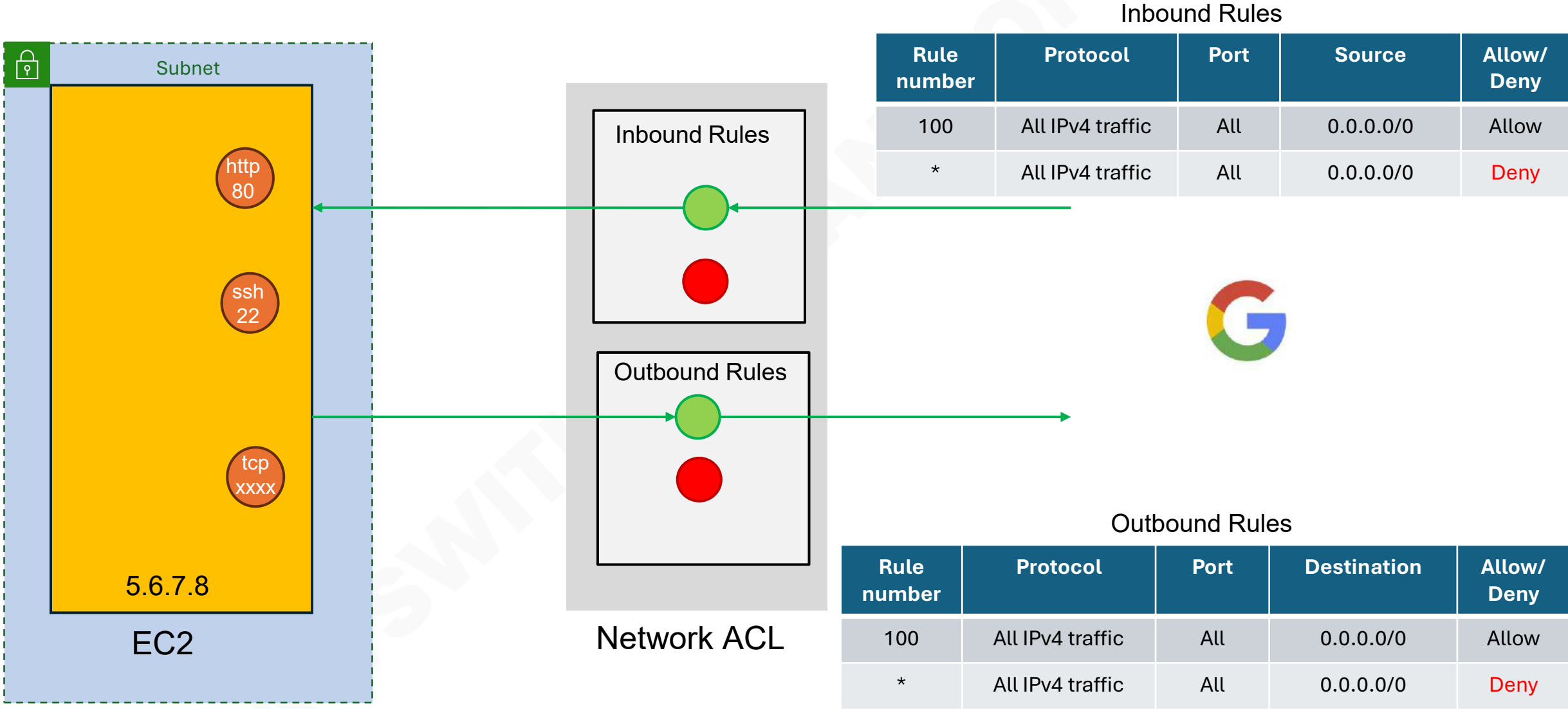
| Inbound Rules |                  |      |               |            |
|---------------|------------------|------|---------------|------------|
| Rule number   | Protocol         | Port | Source        | Allow/Deny |
| 100           | TCP              | 80   | 1.2.3.4/32    | Allow      |
| 200           | TCP              | 80   | 9.10.11.12/32 | Allow      |
| *             | All IPv4 traffic | All  | 0.0.0.0/0     | Deny       |



9.10.11.12  
(port: XXXX)

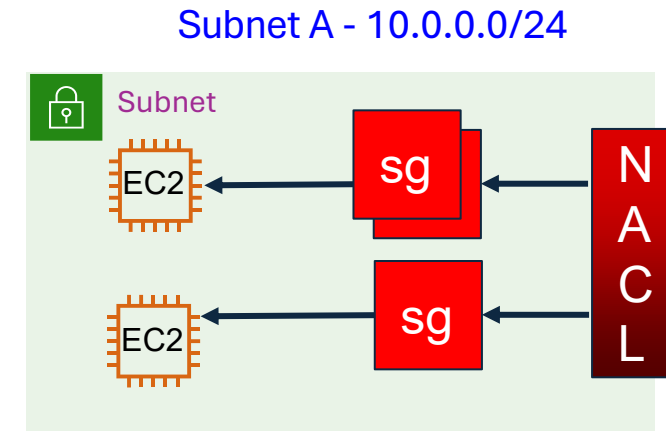
| Outbound Rules |                  |      |               |            |
|----------------|------------------|------|---------------|------------|
| Rule number    | Protocol         | Port | Destination   | Allow/Deny |
| 100            | TCP              | XXXX | 1.2.3.4/32    | Allow      |
| 200            | TCP              | XXXX | 9.10.11.12/32 | Allow      |
| *              | All IPv4 traffic | All  | 0.0.0.0/0     | Deny       |

# Default Network ACL



# Network Access Control List (NACL)

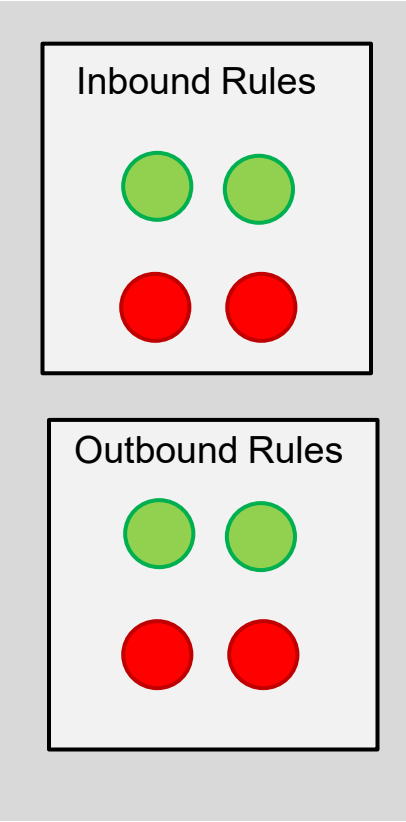
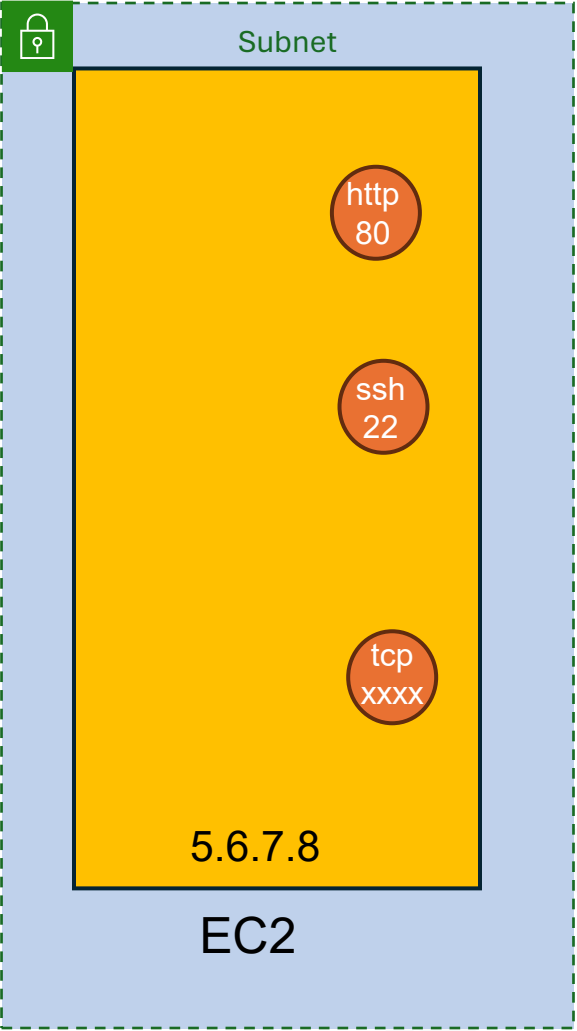
- Works at Subnet level – Hence automatically applied to all instances
- Contains both Allow and Deny rules. Rules are numbered.
- Rules are evaluated in the order of rule number (1 to 32766)
- Stateless – We need to explicitly open ports for return traffic
- Default NACL allows all inbound and outbound traffic
- **NACL are a great way of blocking a specific IP at the subnet level**



Network ACL inbound rules

| #Rule | Type             | Protocol | Port | Source            | Allow/Deny |
|-------|------------------|----------|------|-------------------|------------|
| 100   | All IPv4 traffic | All      | All  | 180.151.138.43/32 | DENY       |
| 101   | HTTPS            | TCP      | 443  | 0.0.0.0/0         | ALLOW      |
| *     | All IPv4 traffic | All      | All  | 0.0.0.0/0         | DENY       |

# Default Network ACL (IPv4 & IPv6)



Network ACL

Inbound Rules

| Rule number | Protocol         | Port | Source    | Allow/Deny |
|-------------|------------------|------|-----------|------------|
| 100         | All IPv4 traffic | All  | 0.0.0.0/0 | Allow      |
| 101         | All IPv6 traffic | All  | ::/0      | Allow      |
| *           | All IPv4 traffic | All  | 0.0.0.0/0 | Deny       |
| *           | All IPv6 traffic | All  | ::/0      | Deny       |



Outbound Rules

| Rule number | Protocol         | Port | Destination | Allow/Deny |
|-------------|------------------|------|-------------|------------|
| 100         | All IPv4 traffic | All  | 0.0.0.0/0   | Allow      |
| 101         | All IPv6 traffic | All  | ::/0        | Allow      |
| *           | All IPv4 traffic | All  | 0.0.0.0/0   | Deny       |
| *           | All IPv6 traffic | All  | ::/0        | Deny       |



# Security Groups vs Network ACL

## Security Group

Operates at EC2 instance

Supports only Allow rules

Stateful – Return traffic is allowed

All rules are evaluated before making a decision

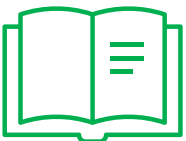
## Network ACL

Operates at Subnet level

Supports both Allow and Deny rules

Stateless – Return traffic needs to be authorized in Outbound rules

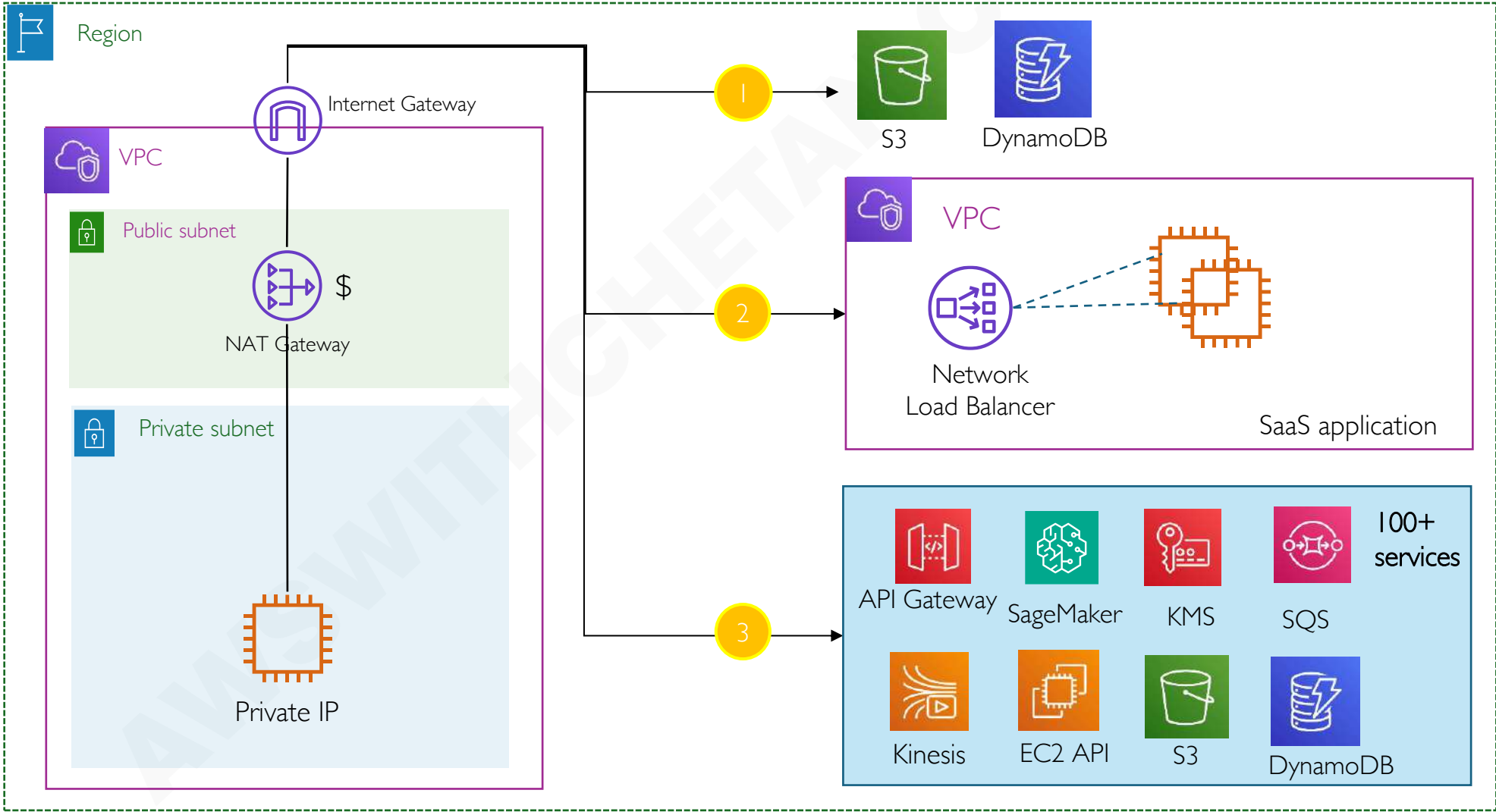
Rules are evaluated in the order (lower to higher) and first matching rule is applied



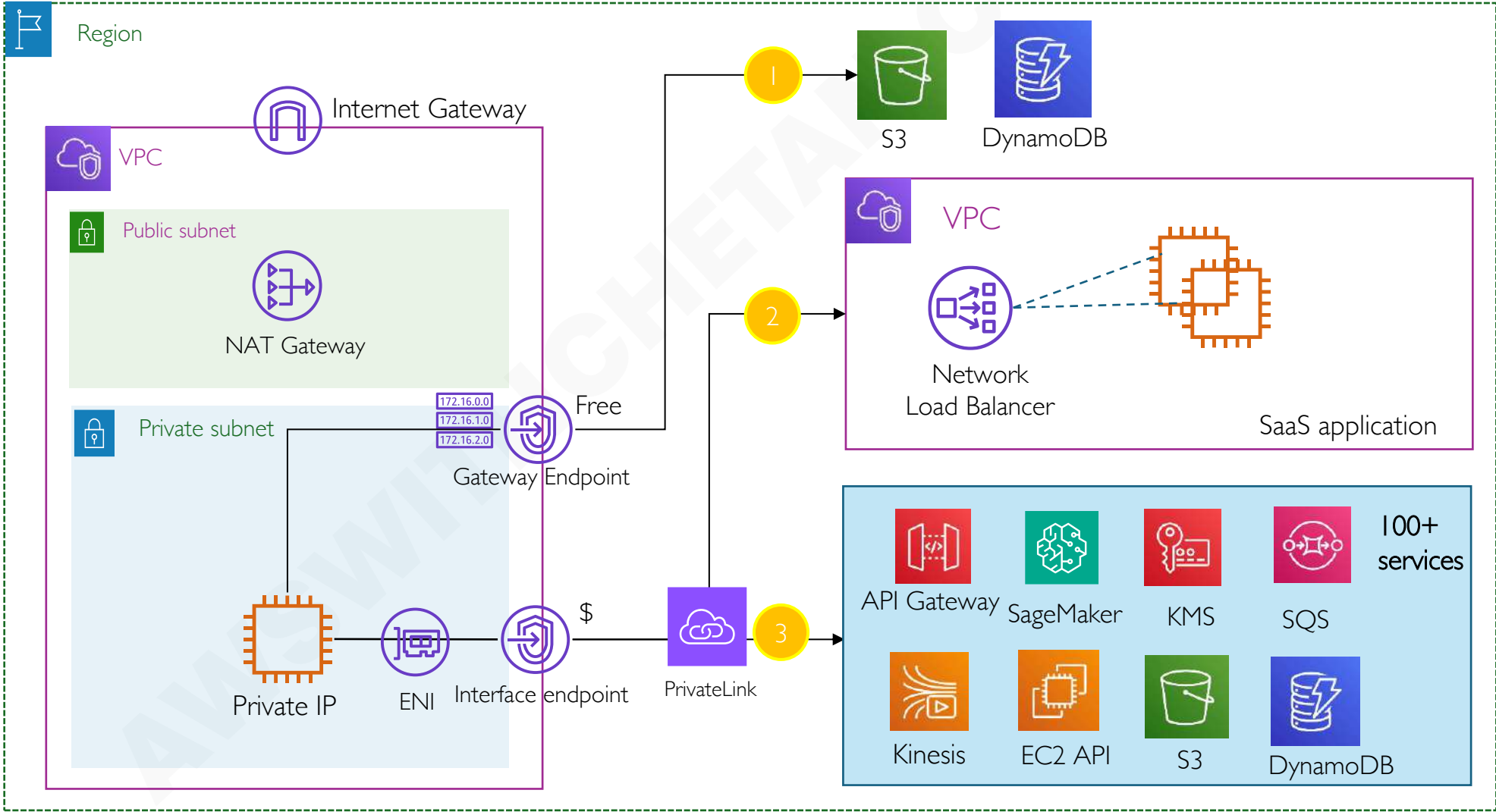


# VPC Endpoints and AWS PrivateLink

# Without VPC endpoints and PrivateLink



# With VPC endpoints and PrivateLink



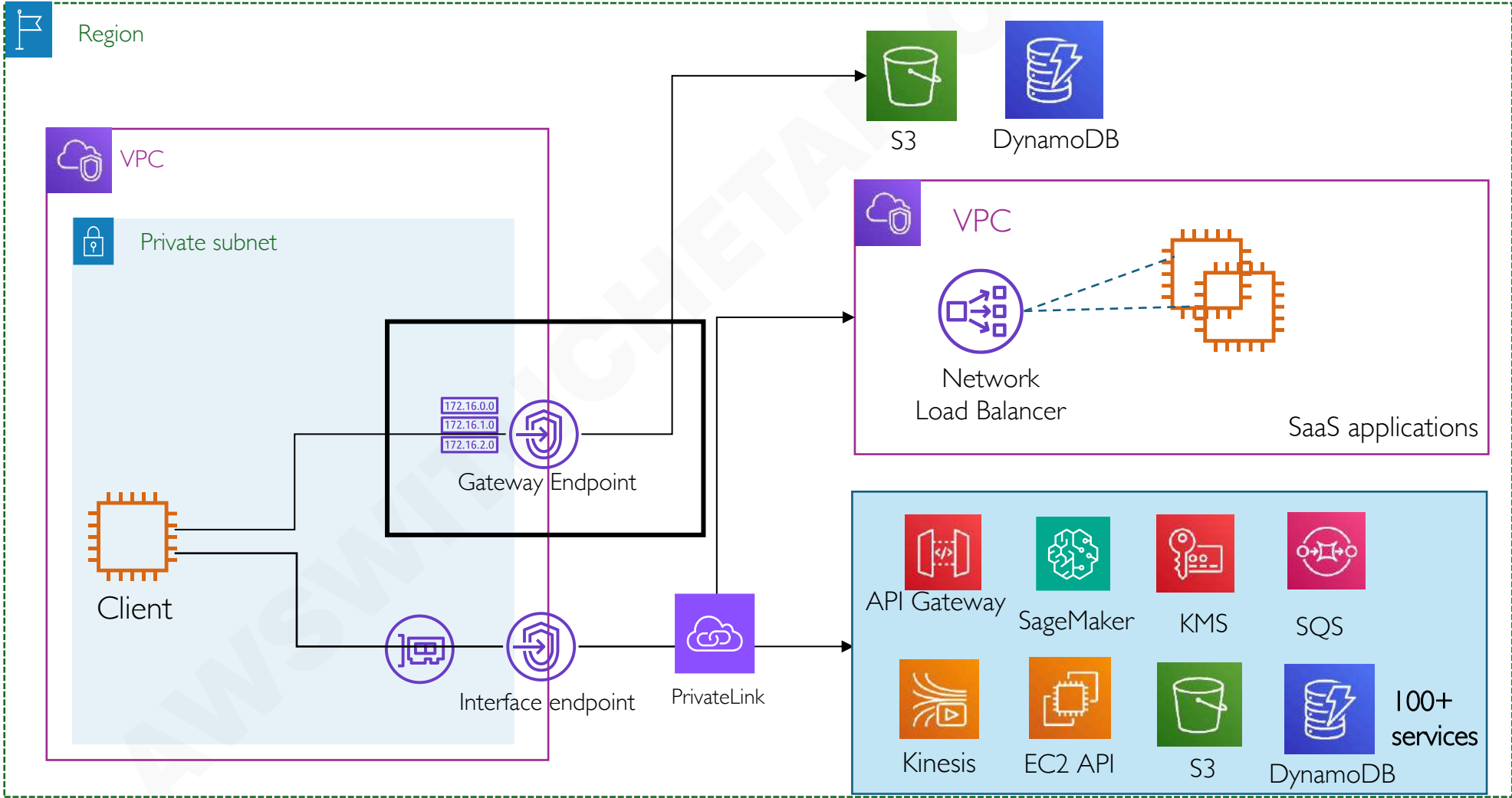
# VPC Endpoints and AWS PrivateLink

- VPC Endpoints allow you to connect to resources in another VPCs and AWS Services using a private network instead of the public network
- They remove the need of IGW, NAT GW to access AWS Services
- Endpoint devices are horizontally scaled, redundant and highly available without any bandwidth constraint on your network traffic
- **Gateway Endpoint:** To access Amazon S3 and DynamoDB only
- **Interface Endpoint:** To access broad set of other AWS services (e.g. SQS, SNS, Kinesis, S3 etc.) and customer services deployed in other VPCs and across AWS accounts
- **Other types of Endpoints:** Gateway Load Balancer endpoint, Resource endpoint, VPC lattice Service-network endpoint



# VPC Gateway Endpoint

# VPC Gateway Endpoint



# VPC Gateway Endpoint

- Enables private connection between VPC and S3/DynamoDB
- Need to modify the route tables and add an entry to route the traffic to S3 or DynamoDB through the gateway VPC endpoint
- When we create Gateway endpoint, a *prefix list* is created in VPC
- The prefix list is the collection of IP addresses for AWS services such as Amazon S3 or DynamoDB.
- The Prefix list is formatted as pl-xxxxxxx and becomes an available option in both subnet routing tables and security groups

| Destination                                                                                           | Target                 | Status |
|-------------------------------------------------------------------------------------------------------|------------------------|--------|
| 10.0.0.0/16                                                                                           | local                  | active |
| pl-78a54011 (com.amazonaws.ap-south-1.s3, 52.219.62.0/23, 3.5.212.0/23, 3.5.208.0/22, 52.219.64.0/22) | vpce-0e89398e630ab0bcf | active |
| 0.0.0.0/0                                                                                             | nat-085190f27ccf2ca3e  | active |



# VPC Gateway Endpoint

- Prefix list should be added in Security group Outbound rule (if Security group outbound rules do not have default “Allow All” rule)

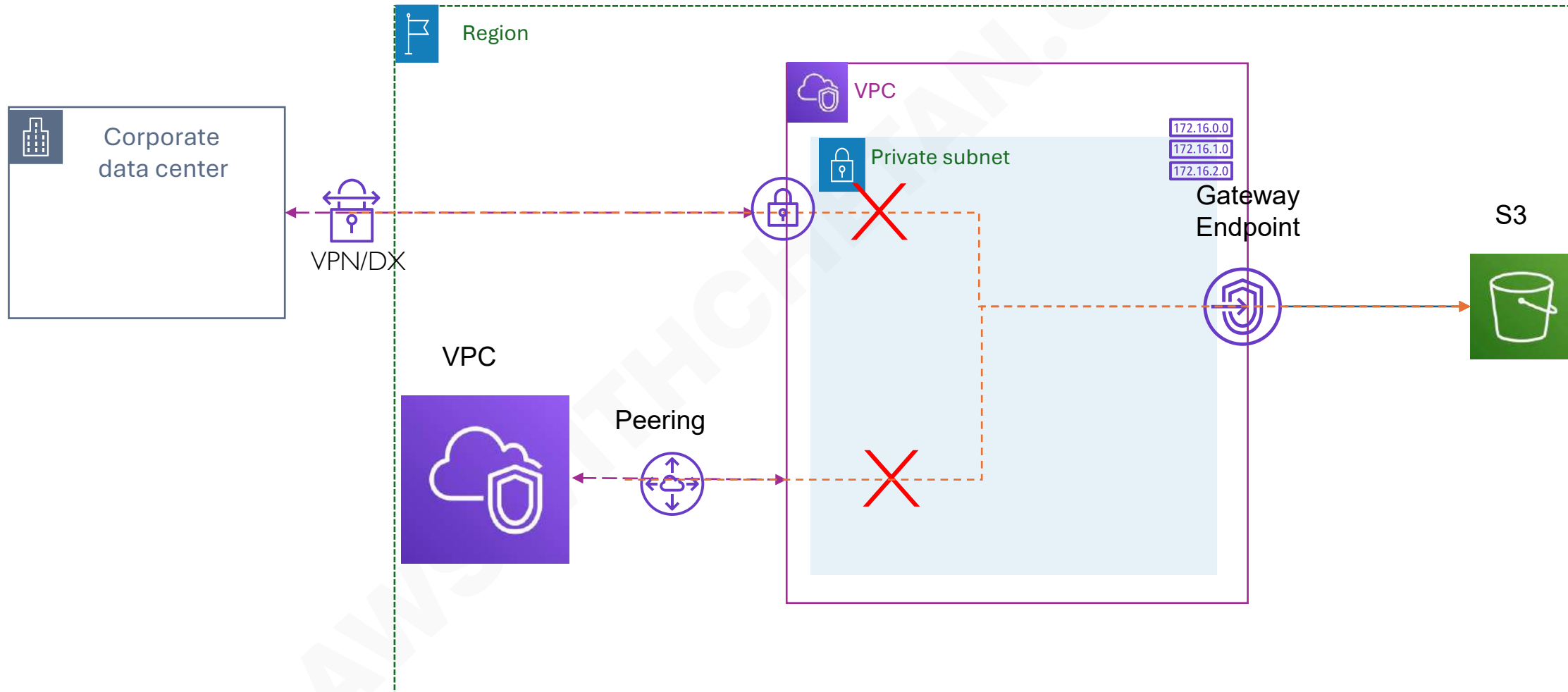


The screenshot shows the AWS Management Console interface for a Security Group named 'sg-17147973'. The 'Outbound' tab is selected, displaying a table of outbound rules. There are two rules: one for HTTPS on port 443 and one for HTTP on port 80, both allowing traffic to the prefix list 'pl-68a54001'.

| Type  | Protocol | Port Range | Destination |
|-------|----------|------------|-------------|
| HTTPS | TCP      | 443        | pl-68a54001 |
| HTTP  | TCP      | 80         | pl-68a54001 |

- VPC Gateway endpoint can only be accessed from within the VPC in which it's created in the same AWS region

# Can be accessed only from within the VPC



# VPC Gateway Endpoint

- Prefix list should be added in Security group Outbound rule (if Security group outbound rules do not have default “Allow All” rule)



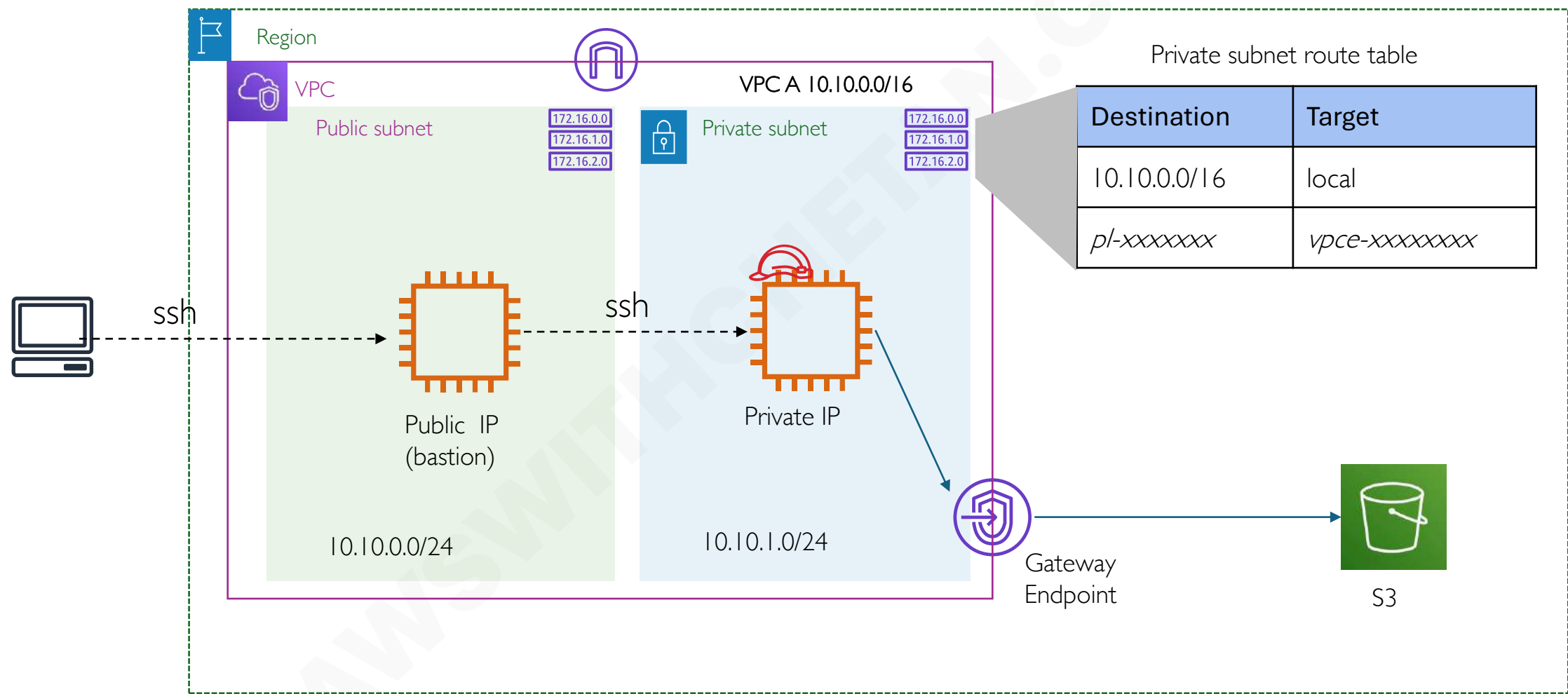
The screenshot shows the AWS Management Console interface for a Security Group named 'sg-17147973'. The 'Outbound' tab is selected, displaying a table of outbound rules. There are two rules: one for HTTPS (TCP, port 443) and one for HTTP (TCP, port 80), both with a destination prefix list 'pl-68a54001'. An 'Edit' button is visible above the table.

| Type ⓘ | Protocol ⓘ | Port Range ⓘ | Destination ⓘ |
|--------|------------|--------------|---------------|
| HTTPS  | TCP        | 443          | pl-68a54001   |
| HTTP   | TCP        | 80           | pl-68a54001   |

- VPC Gateway endpoint can only be accessed from within the VPC in which it's created in the same AWS region
- It's free to use and hence always recommended to use VPC Gateway endpoint when you want to access Amazon S3 or DynamoDB from applications deployed in the VPC

# VPC Gateway Endpoint - Demo

# VPC Endpoint for S3 - Demo



# Demo steps

1. Create VPC with two subnets – A public and a private subnet
2. Launch an EC2 instance (EC2-A) (bastion host) in the Public subnet (allow SSH for 0.0.0.0/0)
3. Launch an EC2 instance (EC2-B) in the Private subnet (allow SSH from SG of bastion host)
4. Create IAM role for EC2 instance to allow S3Full permissions. Attach it to EC2-B.
5. Login to EC2-A over SSH and from there SSH into EC2-B (you will need .pem key file on EC2-A)
6. Try accessing S3 bucket using
7. Create VPC Gateway endpoint for S3 - Select Private subnet
8. Modify Private subnet route table to route traffic to S3 via VPC gateway endpoint
9. Test the connectivity to S3 by uploading/downloading some files from any of your S3 bucket

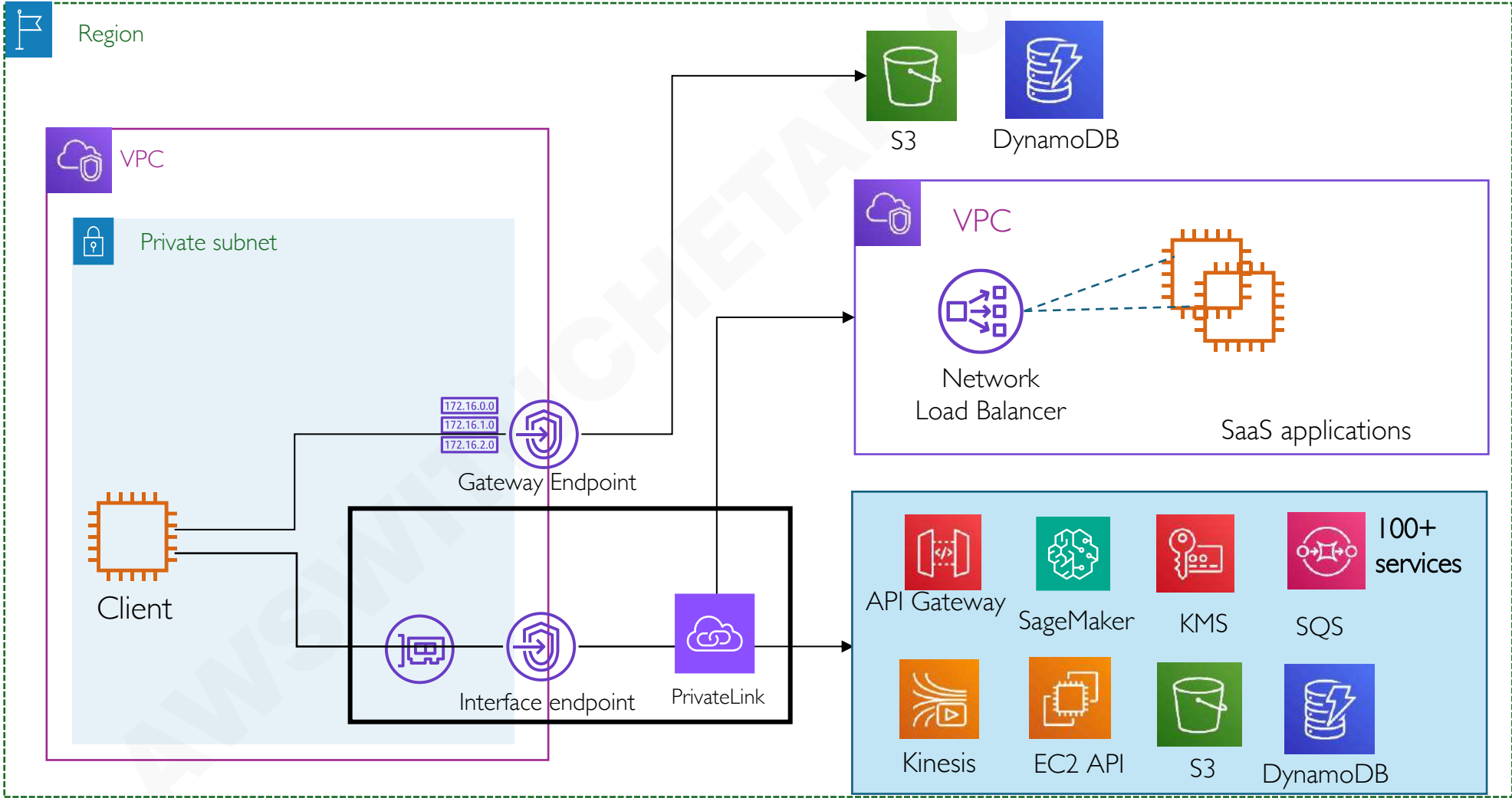
```
aws s3 ls
```

```
aws s3 cp s3://bucket_name/file_name .
```

# VPC Interface endpoint

Powered by AWS PrivateLink

# VPC Interface Endpoint

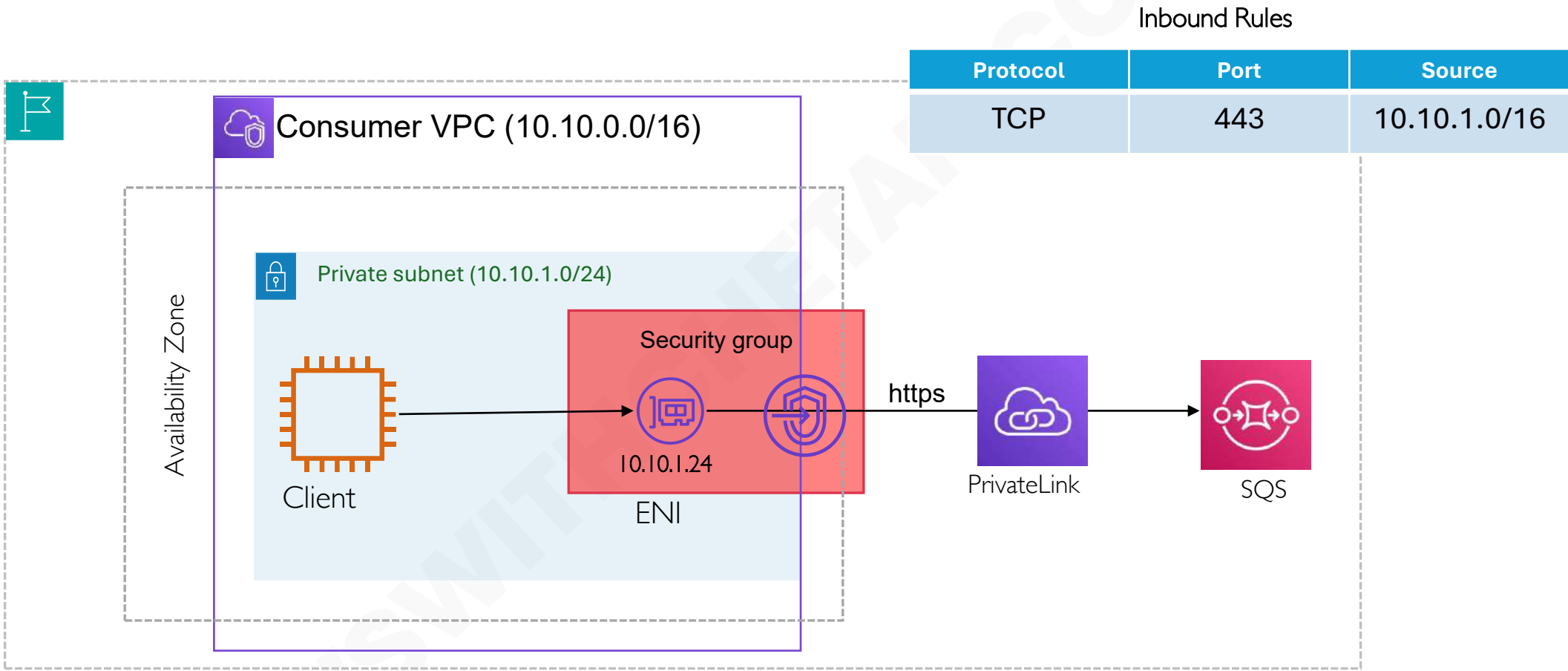




# VPC Interface endpoint

- Interface endpoints create local IP addresses (using ENI) in your VPC.
- Has Security Group – inbound rules to be configured
- For High Availability create VPC endpoints across multiple Availability zones
- Supports IPv4 and IPv6 traffic.
- Supports traffic over TCP and UDP protocols
- Interface endpoint gets a regional and zonal DNS
  - **Regional:** vpce-0b7d2995e9dfe5418-mwrthsyu.sqs.us-east-1.vpce.amazonaws.com
  - **Zonal:** vpce-0b7d2995e9dfe5418-mwrthsyu-us-east-1a.sqs.us-east-1.vpce.amazonaws.com

# VPC endpoint - Security group

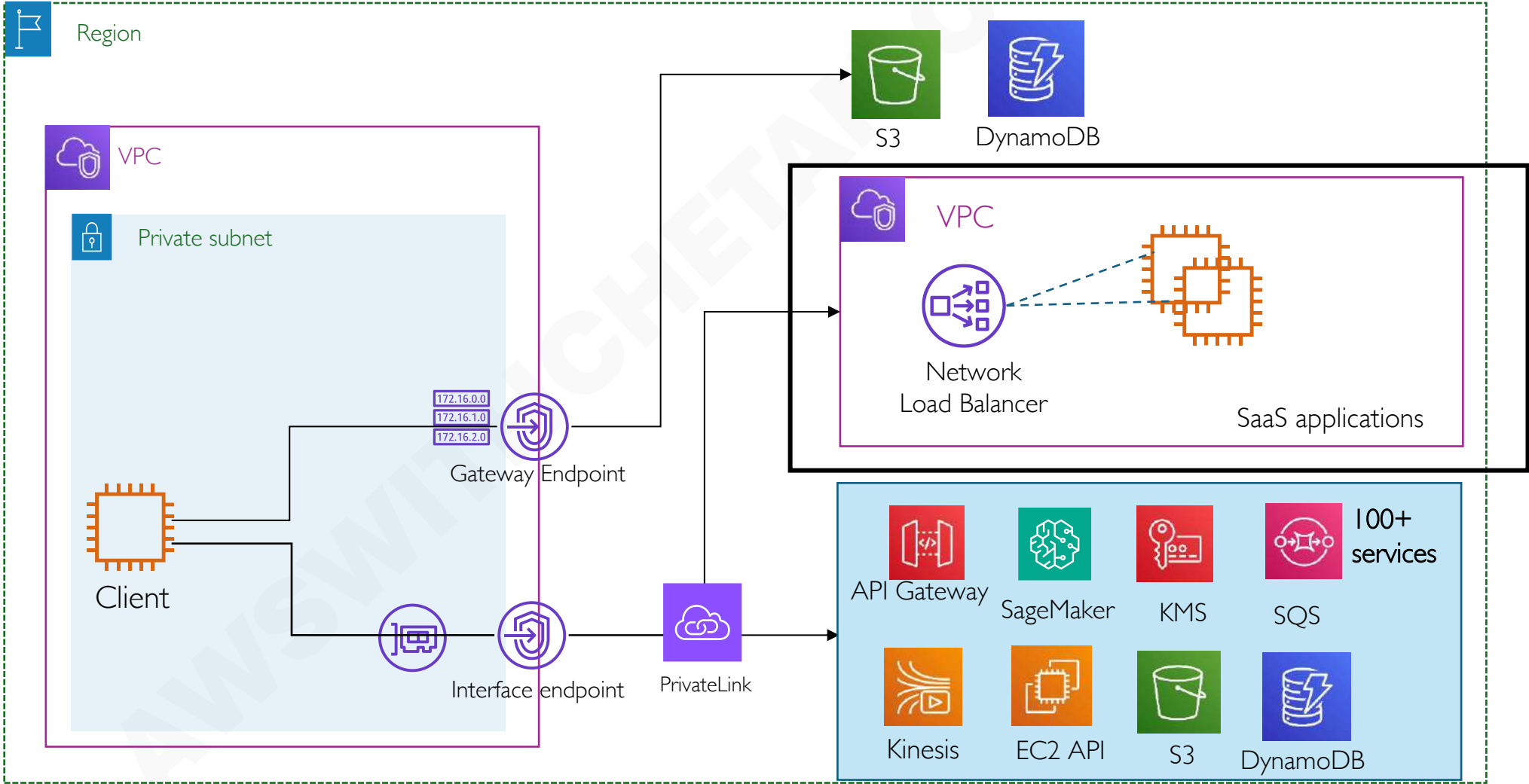


# VPC Interface endpoint – Important to know

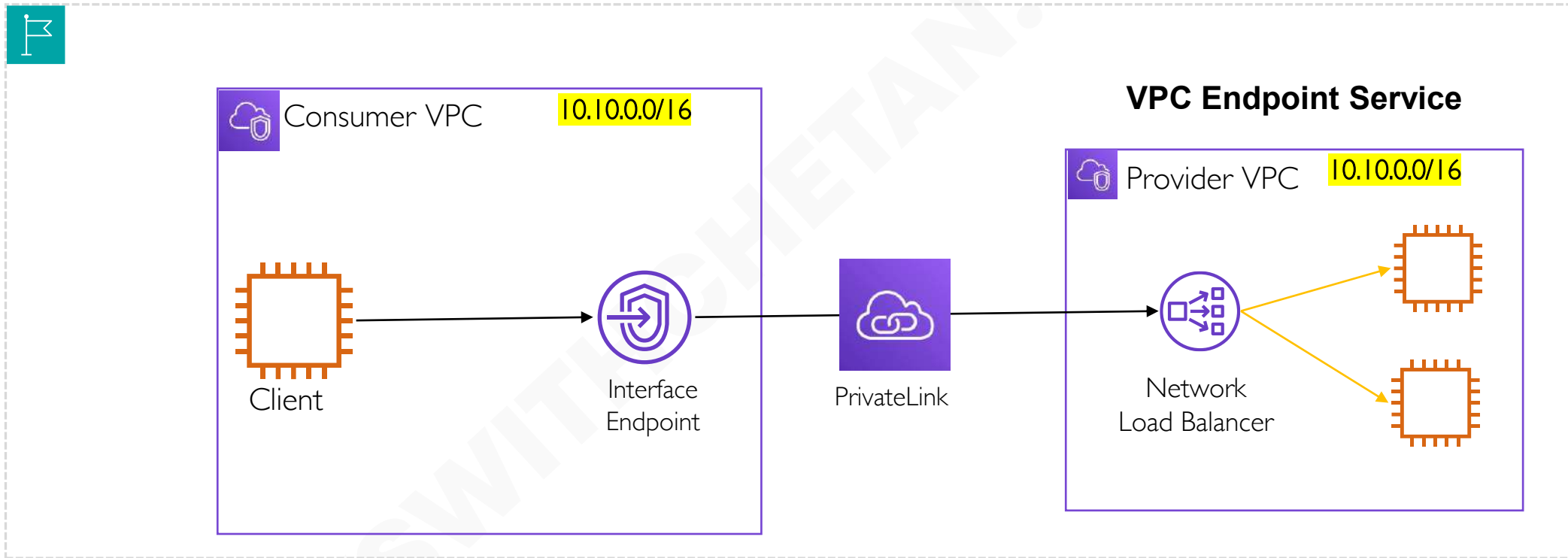
- You can expose your own services using VPC endpoint service
- The Service provider VPC and consumer VPC can have overlapping CIDR blocks
- Interface endpoint can be accessed from other networks e.g. Peered VPCs, Transit gateway, VPN or Direct Connect

AWSWITHCHETAN.COM

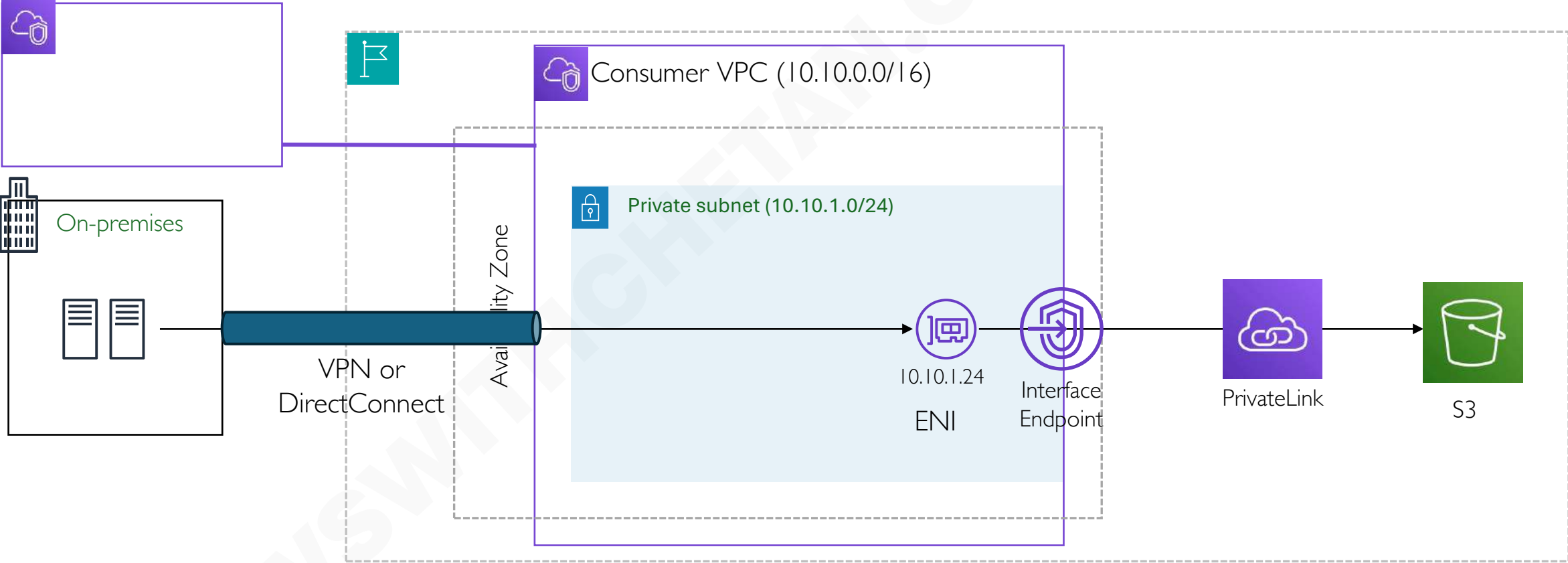
# VPC Endpoint Service



# VPC endpoint service



# Accessing Interface endpoint from remote network



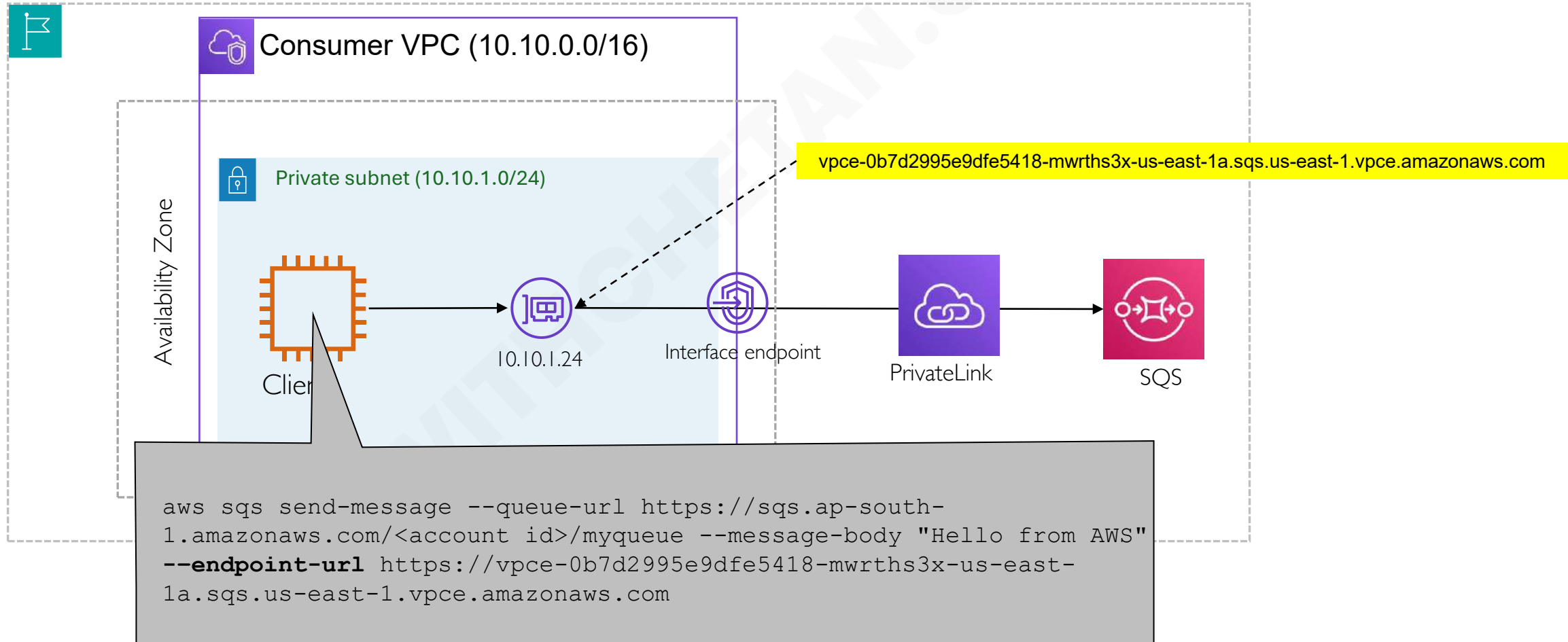
# VPC Interface endpoint - DNS

# Interface endpoint DNS

- Interface endpoint gets a regional and zonal DNS
  - Regional: `vpce-0b7d2995e9dfe5418-mwrthsyu.sqs.us-east-1.vpce.amazonaws.com`
  - Zonal: `vpce-0b7d2995e9dfe5418-mwrthsyu-us-east-1a.sqs.us-east-1.vpce.amazonaws.com`
- Private DNS settings for VPC interface endpoint
  - VPC Setting: “**Enable DNS hostnames**” and “**Enable DNS Support**” must be 'true'
  - The public hostname of an AWS service will resolve to the private Interface endpoint hostname
- With Private DNS enabled, the consumer VPC can access the endpoint services using Service's default DNS e.g **`sqs.us-east-1.amazonaws.com`** instead of using endpoint specific DNS e.g `vpce-12345-ab.ec2.us-east-1.vpce.amazonaws.com`

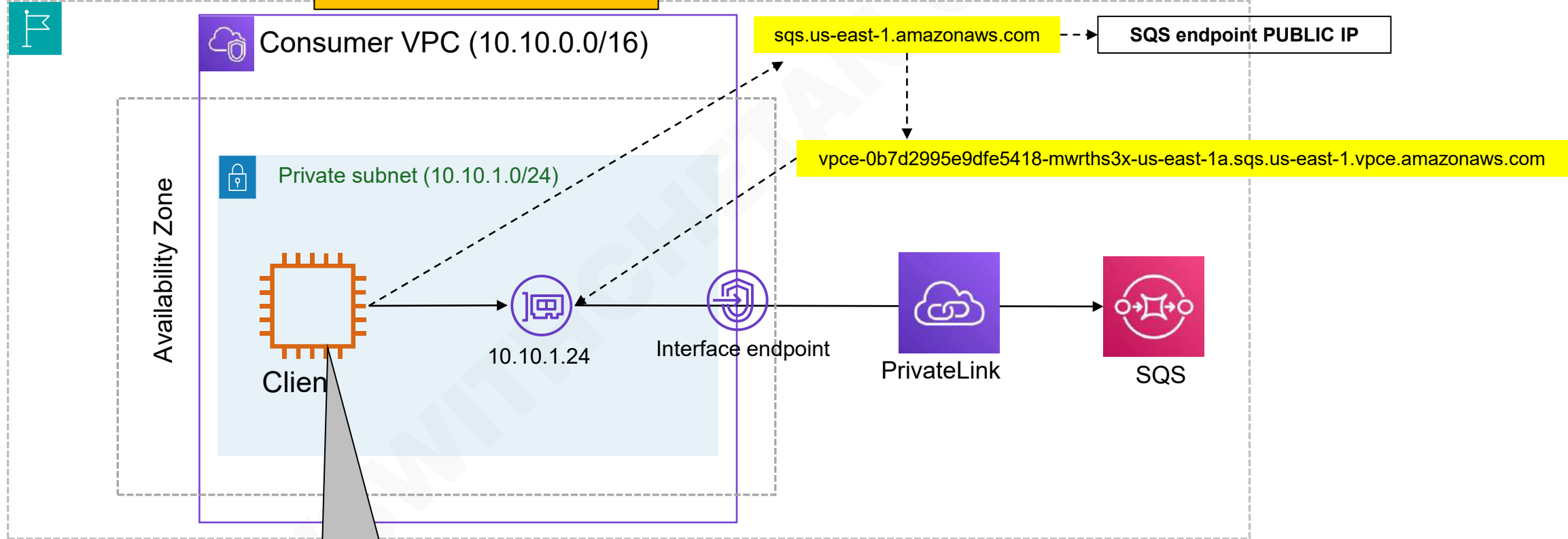


# VPC interface endpoint DNS



# VPC interface endpoint DNS

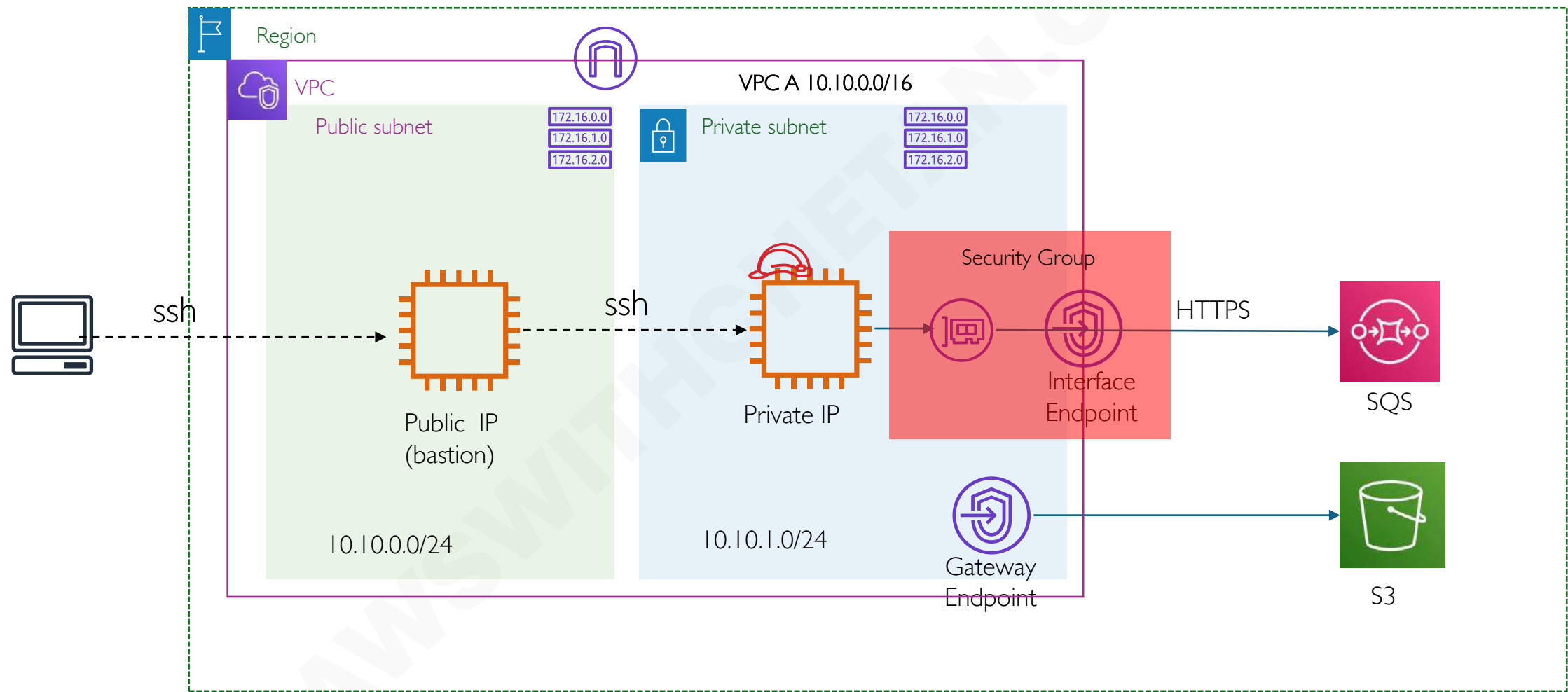
Enable DNS Support  
Enable DNS hostnames



```
aws sqs send-message --queue-url https://sqs.ap-south-1.amazonaws.com/<account id>/myqueue --message-body "Hello from AWS"
```

# VPC Interface Endpoint - Demo

# VPC Interface Endpoint - Demo



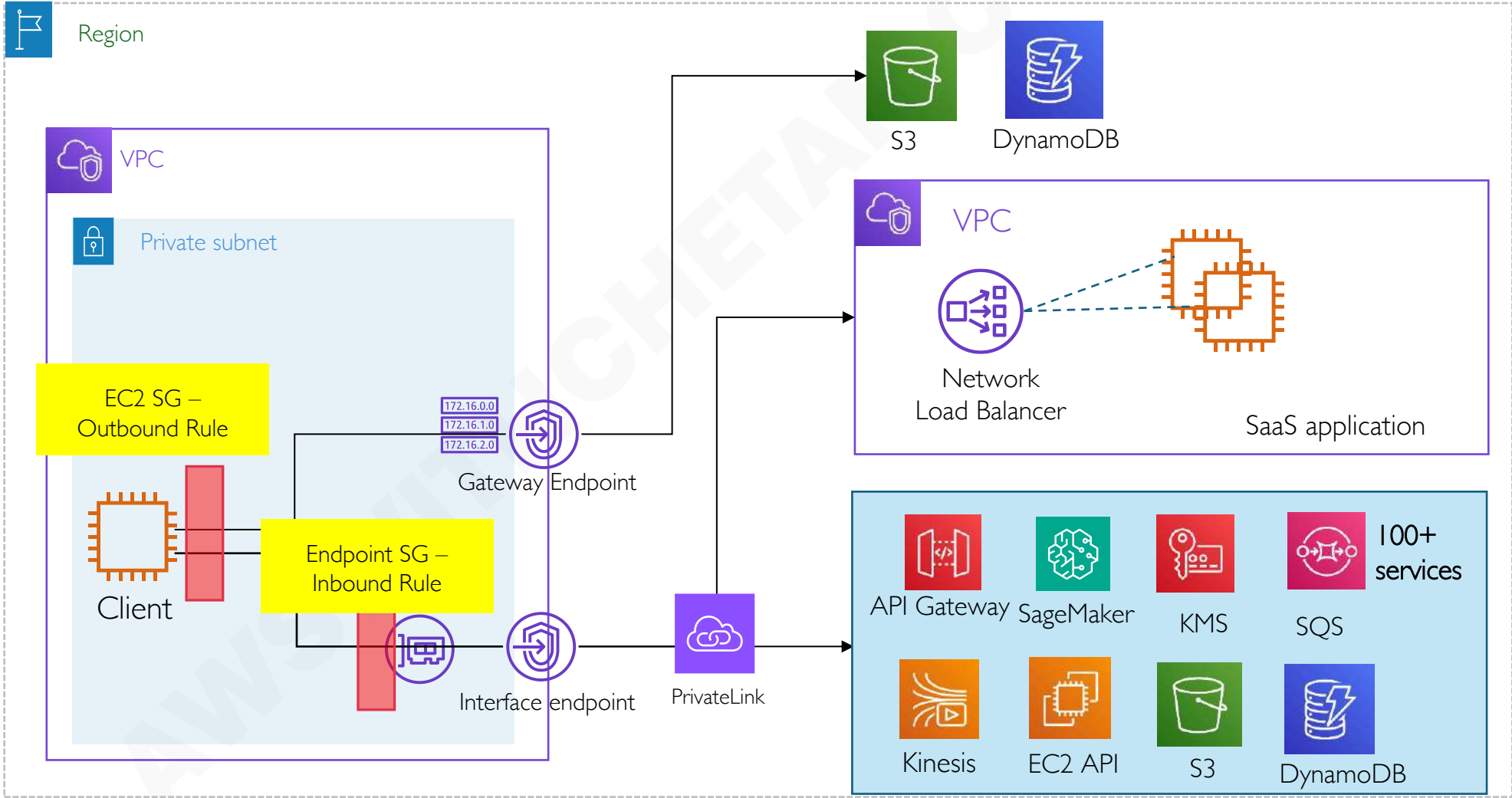
# Demo steps (using existing setup used for gateway endpoint demo)

1. Create SQS queue in the same AWS region (say myqueue)
2. Enable “Enable DNS hostnames” and “Enable DNS Support” for the VPC
3. Create Security group for VPC interface endpoint ENI – Inbound HTTPS (443) from EC2-B instance SG
4. Create or modify IAM role for EC2 to allow Amazon SQS permissions and attach it to EC2-B
5. Login (SSH) into EC2-A and from there SSH into EC2-B and try accessing SQS service using CLI (command below. No connectivity)
6. Create VPC interface endpoint for SQS - Select Private Subnet and Enable Private DNS
7. From EC2-B, now test the connectivity to SQS by sending some messages

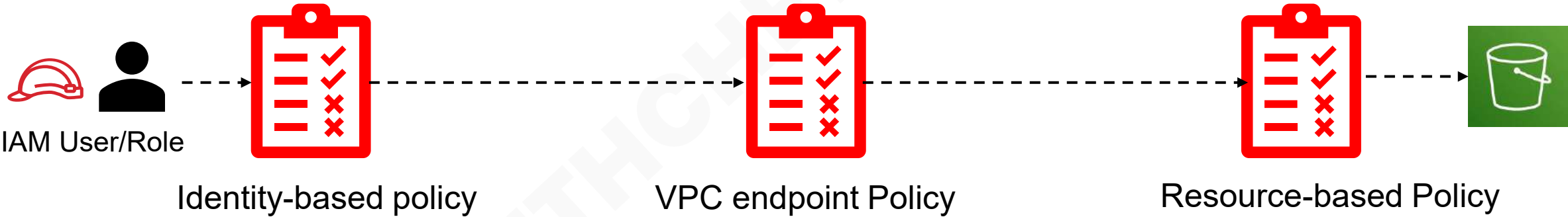
```
aws sqs send-message --queue-url https://sqs.ap-south-1.amazonaws.com/<account id>/myqueue --message-body "Hello from AWS nerd"
```

# VPC endpoints security

# VPC endpoint security – Network layer

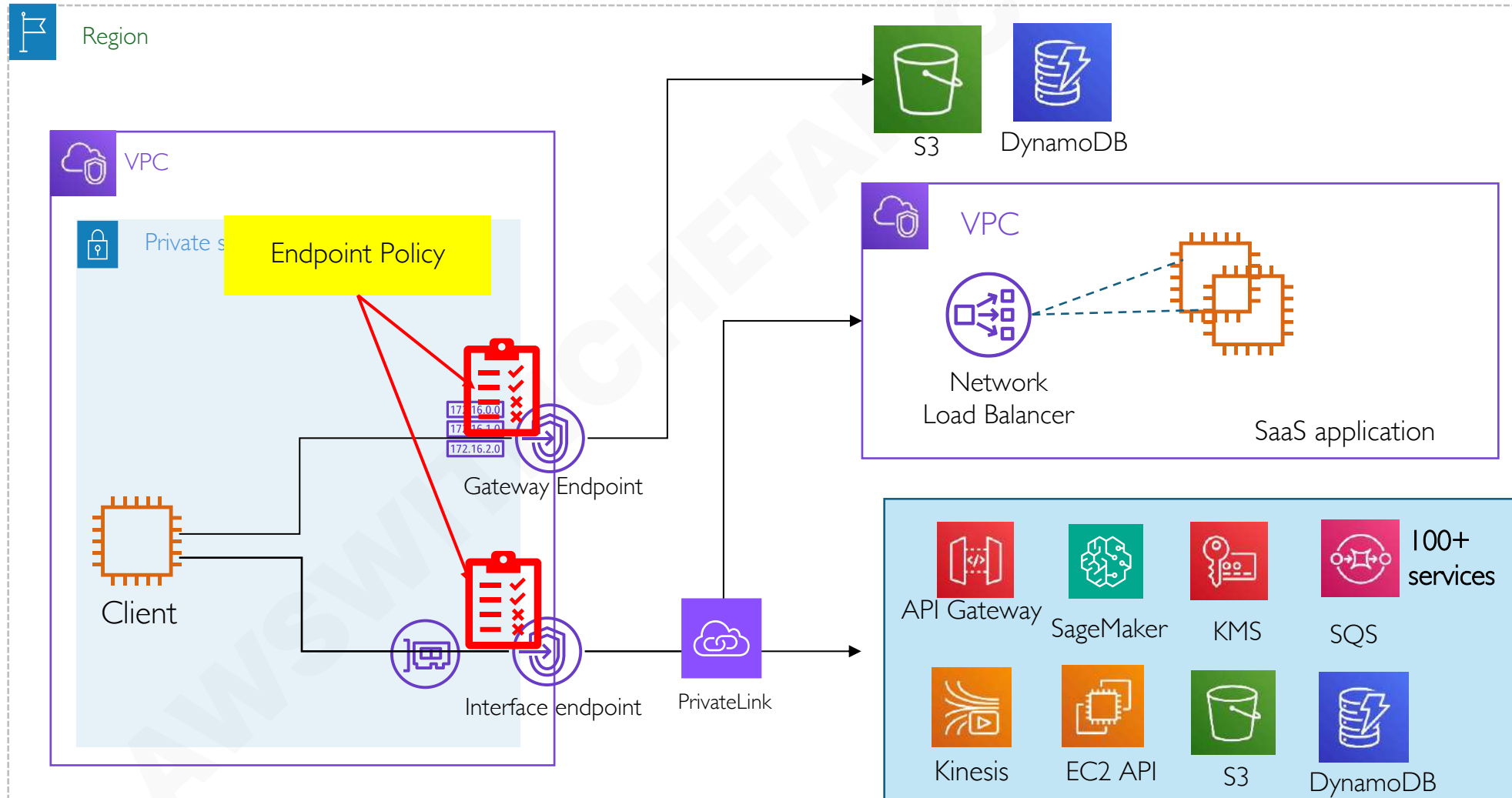


# IAM policies





# VPC endpoint security – Endpoint policies



# Default VPC Endpoint Policy

- Default endpoint policy grants full access to the endpoint

```
{  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Principal": "*",  
      "Action": "*",  
      "Resource": "*"   
    }  
  ]  
}
```

# VPC Endpoint Policy

- VPC Endpoint Policy to restrict access to a specific AWS service resource

```
{
  "Statement": [
    {
      "Sid": "Access-to-specific-bucket-only",
      "Principal": "*",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Effect": "Allow",
      "Resource": ["arn:aws:s3:::my_secure_bucket",
        "arn:aws:s3:::my_secure_bucket/*"]
    }
  ]
}
```

Restrict access to  
specific S3 Bucket

```
{
  "Statement": [
    {
      "Sid": "AccessToSpecificTable",
      "Principal": "*",
      "Action": [
        "dynamodb:Batch*",
        "dynamodb:Delete*",
        "dynamodb:DescribeTable",
        "dynamodb:GetItem",
        "dynamodb:PutItem",
        "dynamodb:Update*"
      ],
      "Effect": "Allow",
      "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/StockTable"
    }
  ]
}
```

Restrict access to  
DynamoDB table

# VPC Endpoint Policy

- VPC Endpoint Policy to restrict access for a specific IAM account/user/role

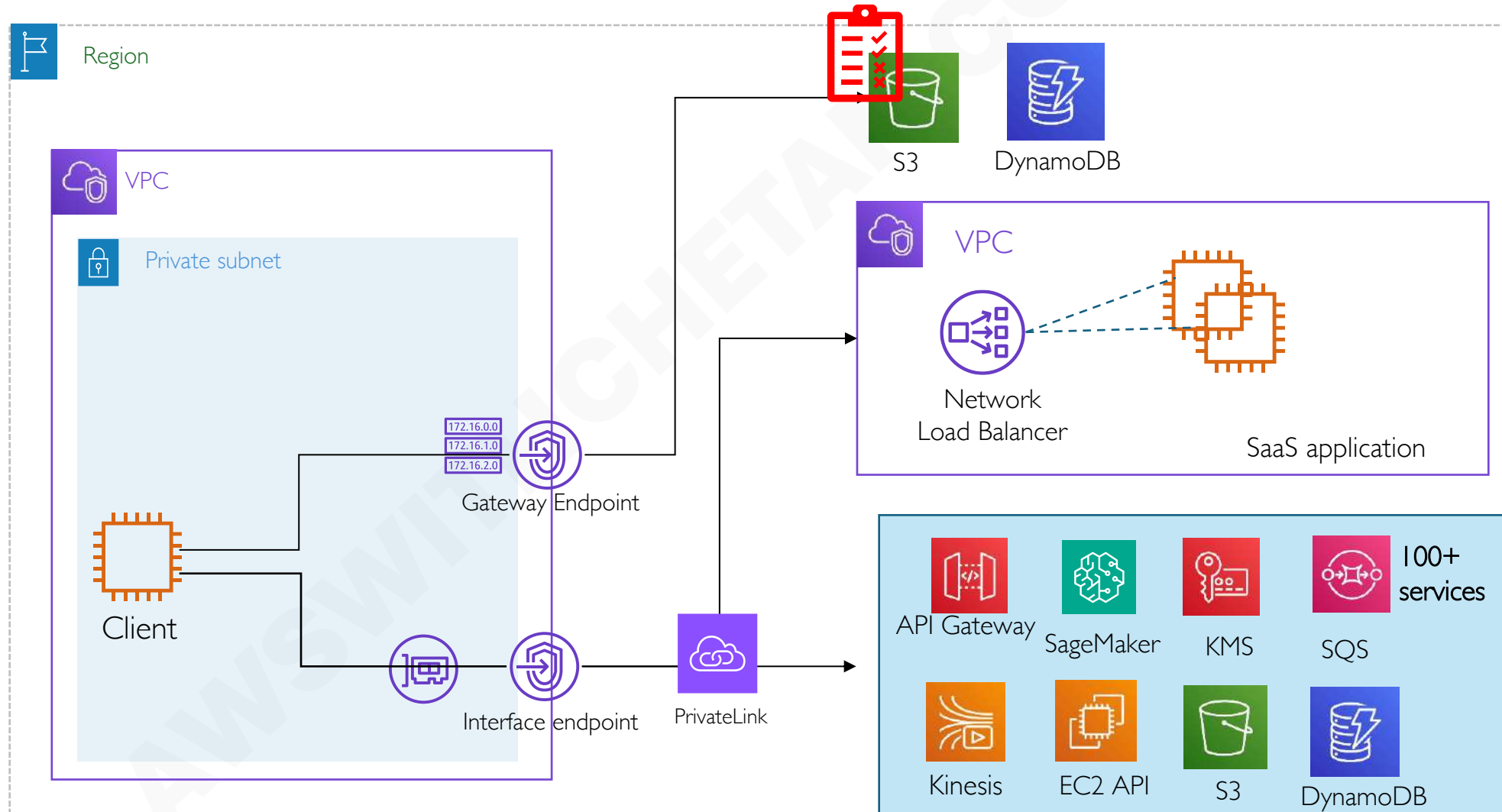
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Allow-callers-from-specific-account",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "aws:PrincipalAccount": "111122223333"
        }
      }
    }
  ]
}
```

Restrict access for  
specific AWS account

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Allow-access-to-specific-IAM-role",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "ArnEquals": {
          "aws:PrincipalArn": "arn:aws:iam::111122223333:role/role_name"
        }
      }
    }
  ]
}
```

Restrict access for  
specific IAM Role

# Using IAM Resource based policies



# AWS Resource based policy - S3 bucket policy

- S3 bucket policy may have
  - Condition: "`aws:sourceVpce`": "vpce-1a2b3c4d" -> Allow or Deny from a specific VPC endpoint
  - Condition: "`aws:sourceVpc`": "vpc-111bbb22" -> Allow or Deny access from a specific VPC

AWSWITHCHETAN.COM

# VPC Private Connectivity options

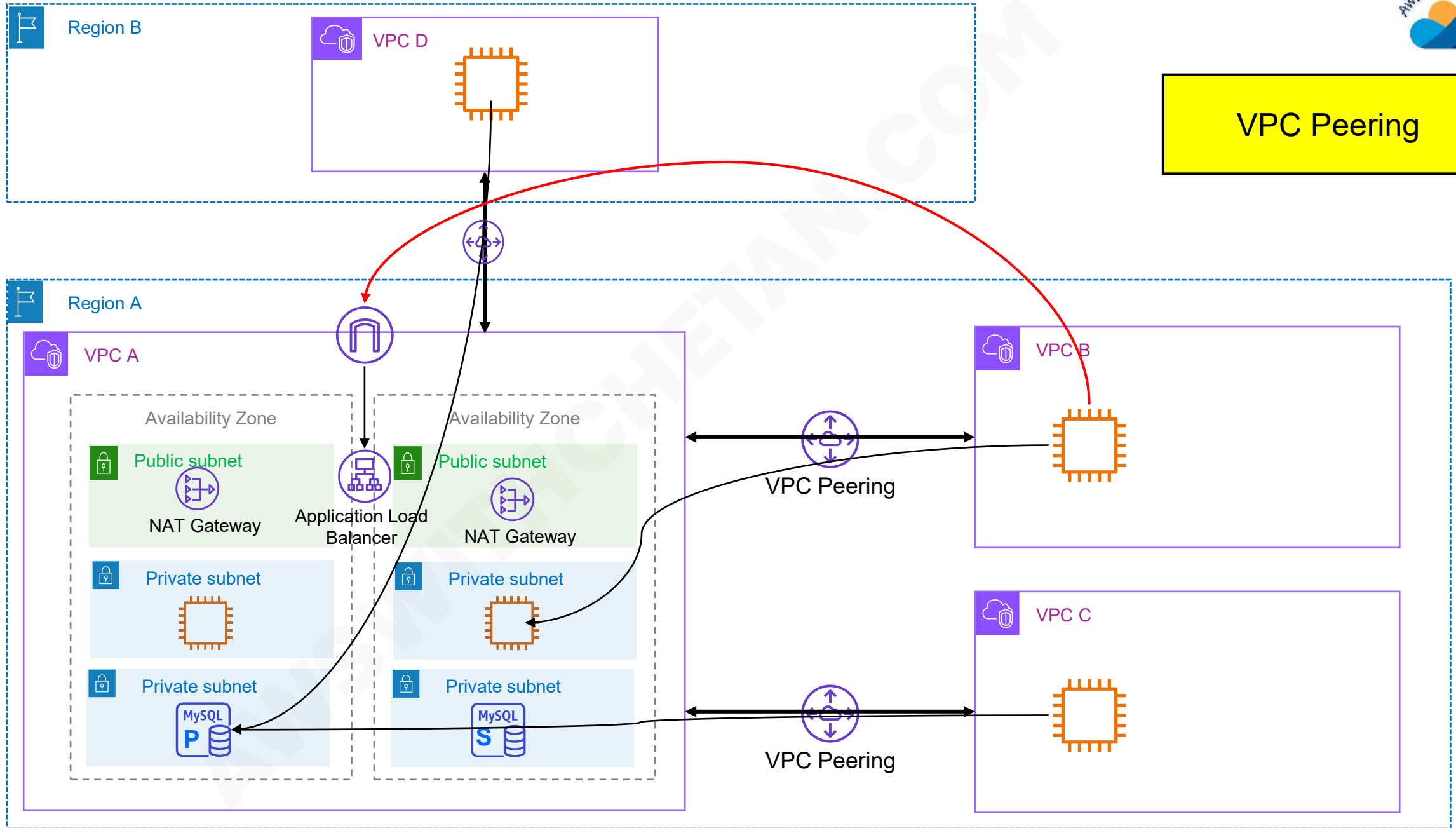
# VPC connectivity options

- VPC Peering connection
- AWS Transit Gateway
- AWS Site-2-Site VPN
- AWS Client VPN
- AWS Direct Connect

AWSWITHCHETAN.COM

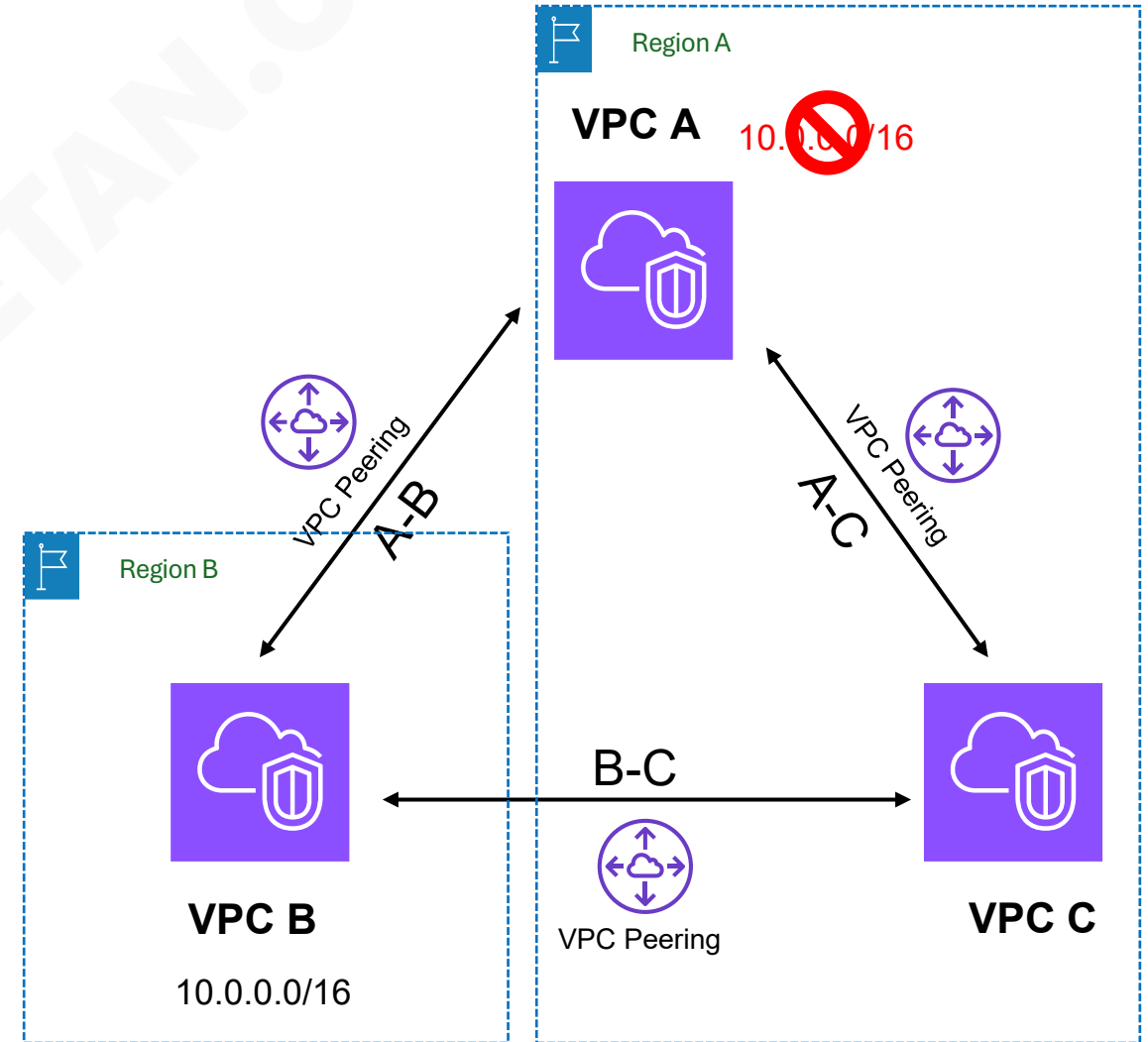


# VPC Peering

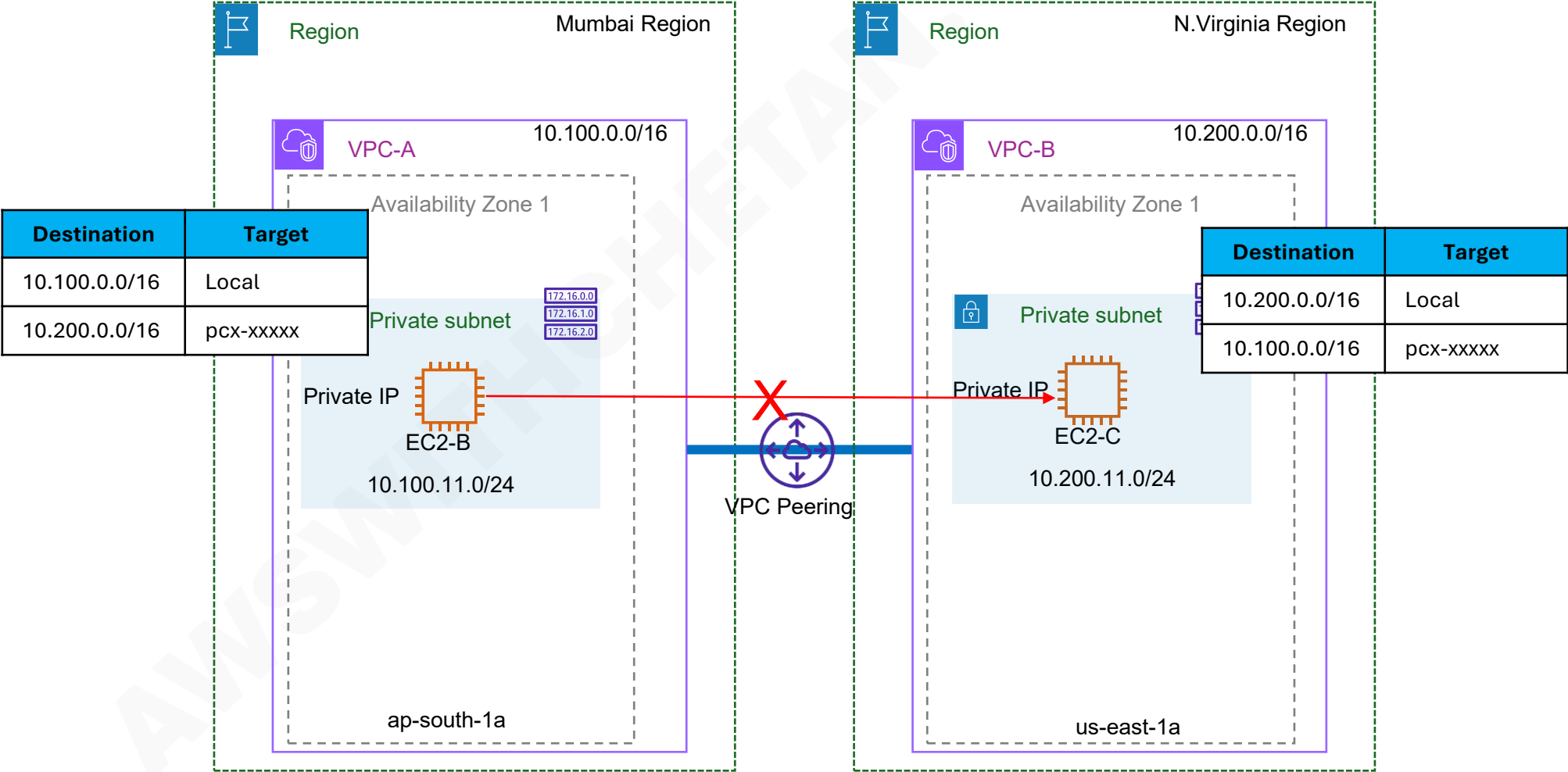


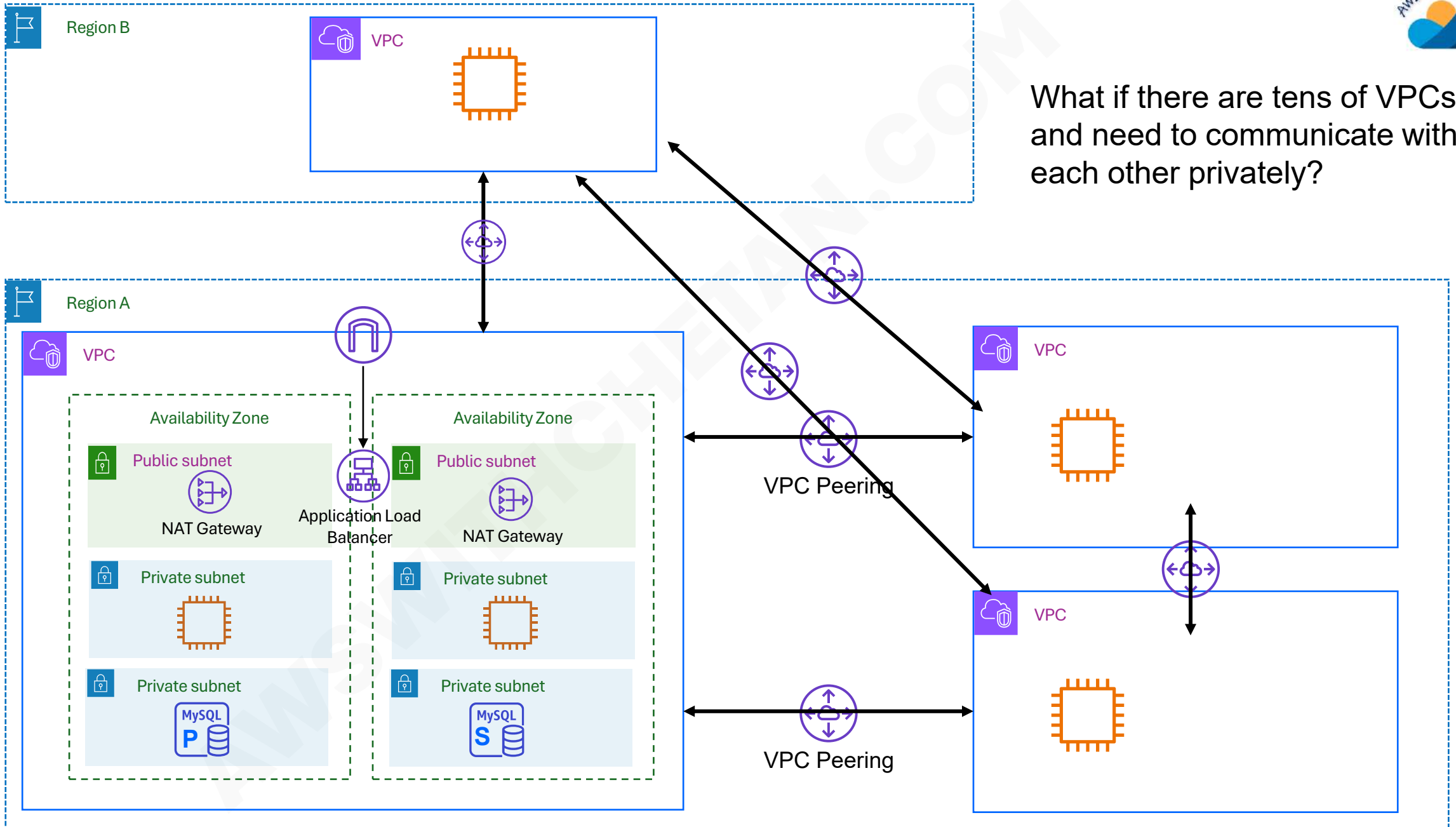
# VPC Peering

- Simplest way to connect two VPCs privately
- It's One-to-One connection
- No need to have internet gateways
- VPCs should not have overlapping CIDRs
- Support intra region and inter region peering
- Need to add route for routing the traffic through the peering connection

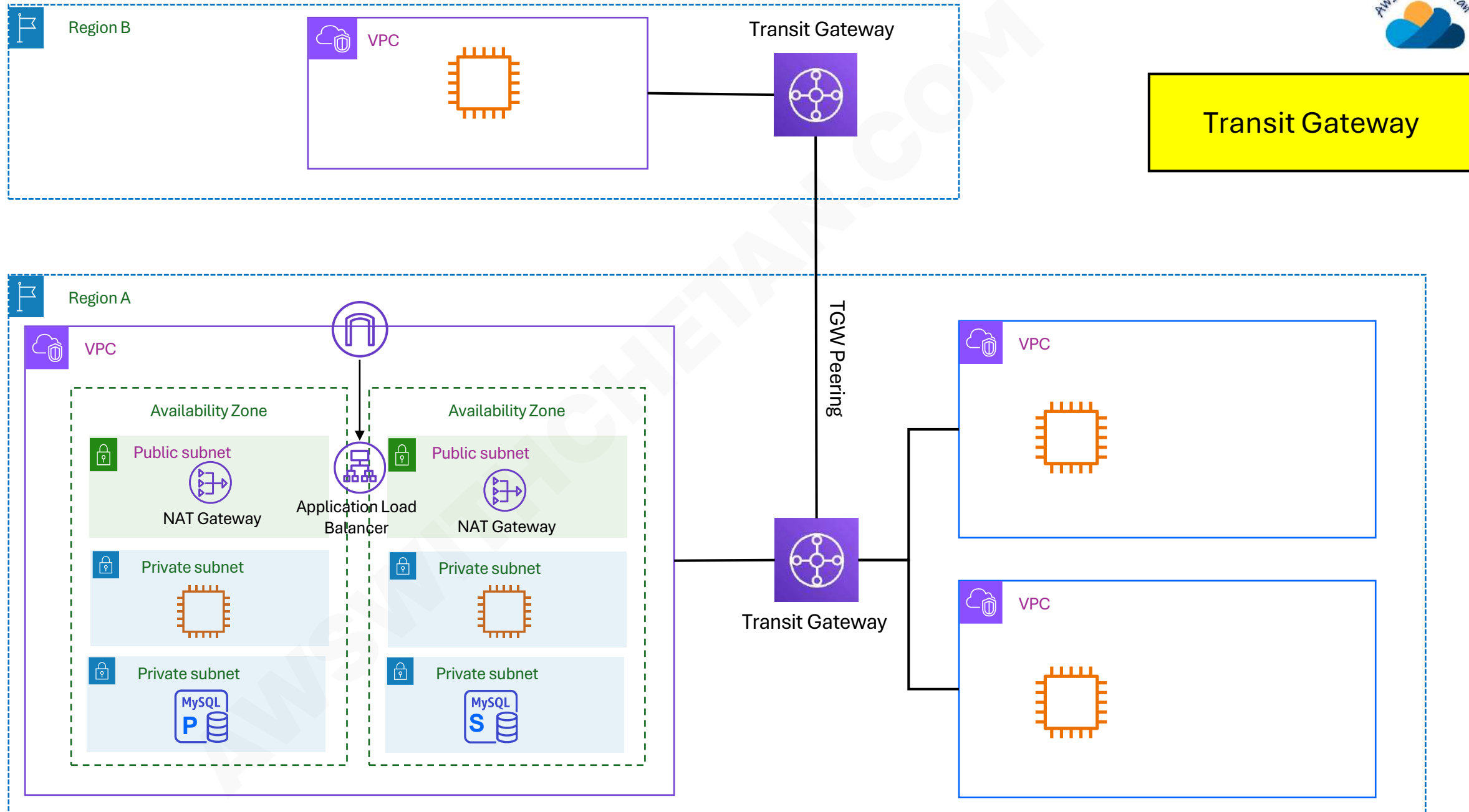


# VPC Peering



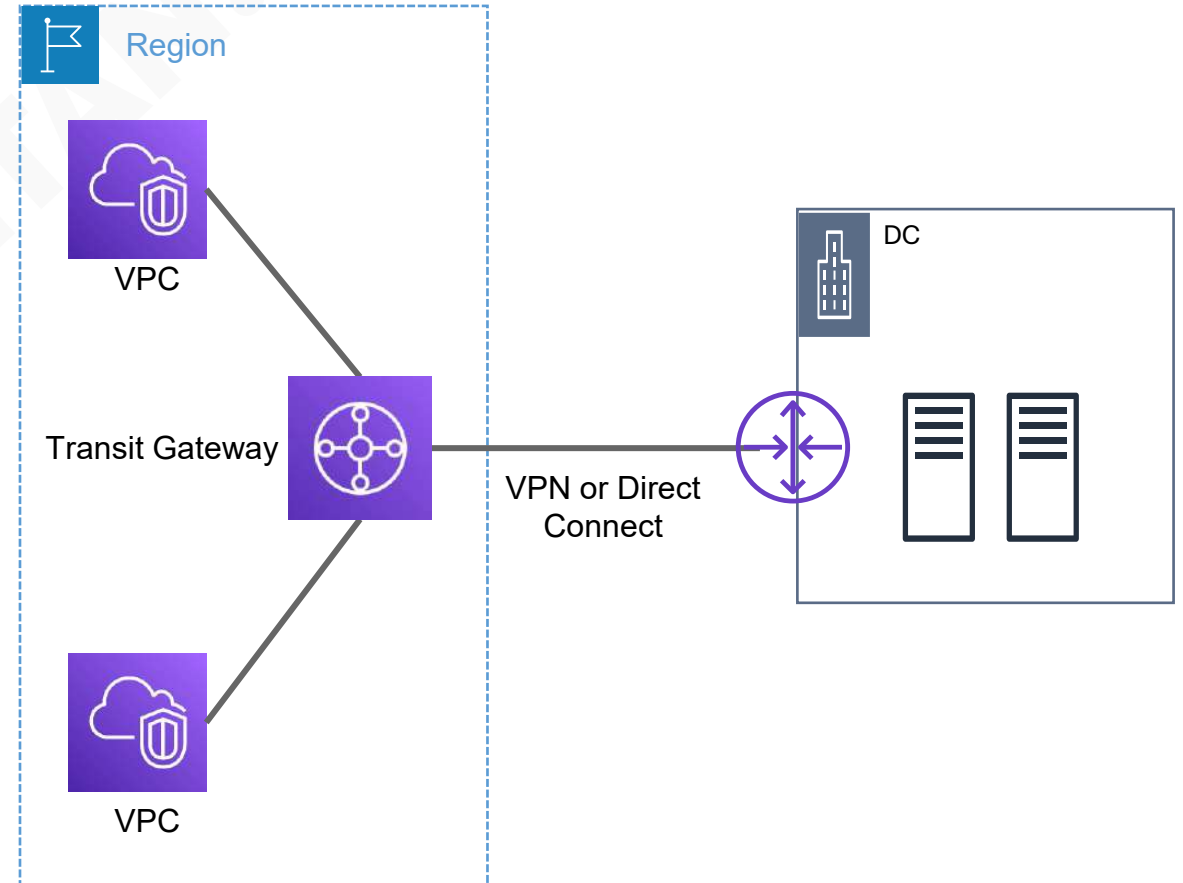


What if there are tens of VPCs and need to communicate with each other privately?



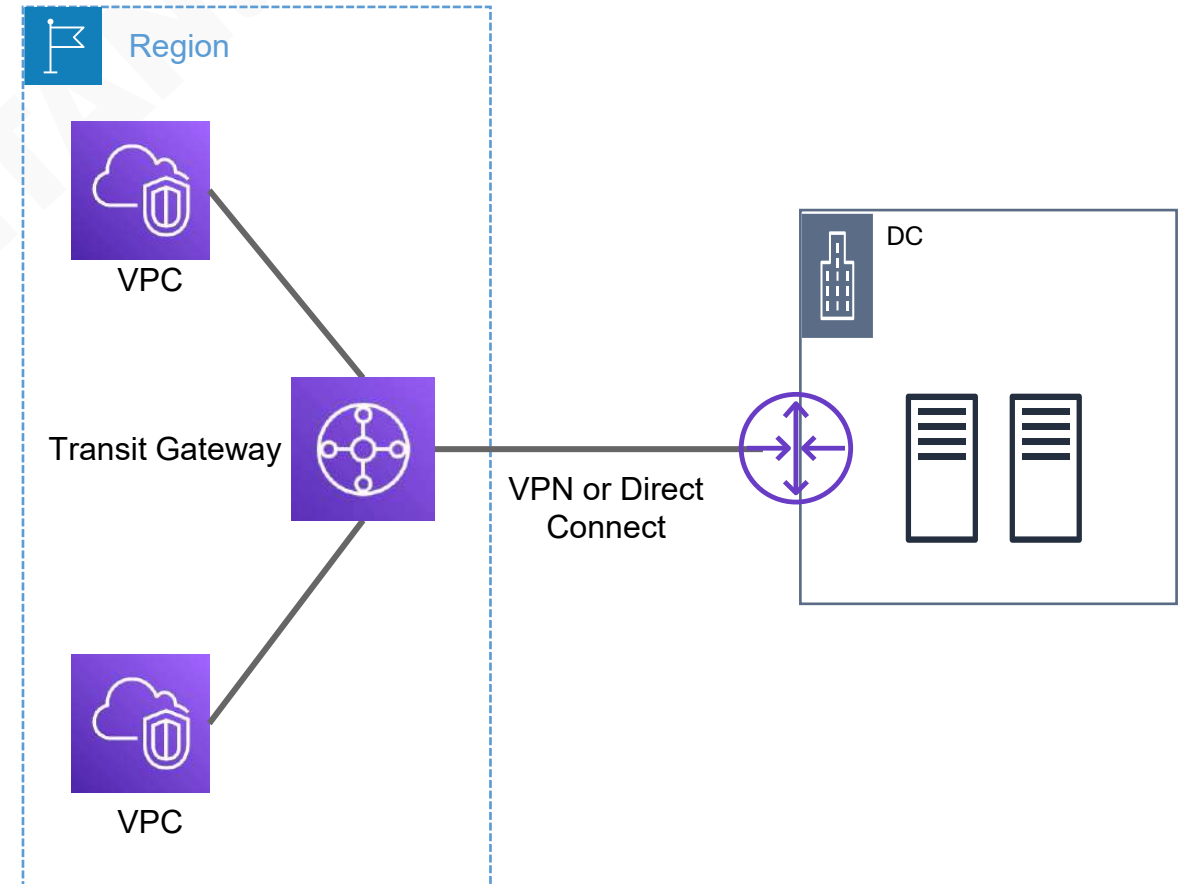
# Transit Gateway

- Allows customers to interconnect **thousands** of **VPCs** and **on-premises** networks.
- It's a regional router
- Hub and spoke architecture where we can connect
  - VPCs
  - VPN connection
  - Direct Connect Gateway
  - A Connect SD-WAN/third-party network appliance
  - Another Transit Gateway (TGW peering)
- Ideal for centralized traffic inspection and Hybrid network setup etc.



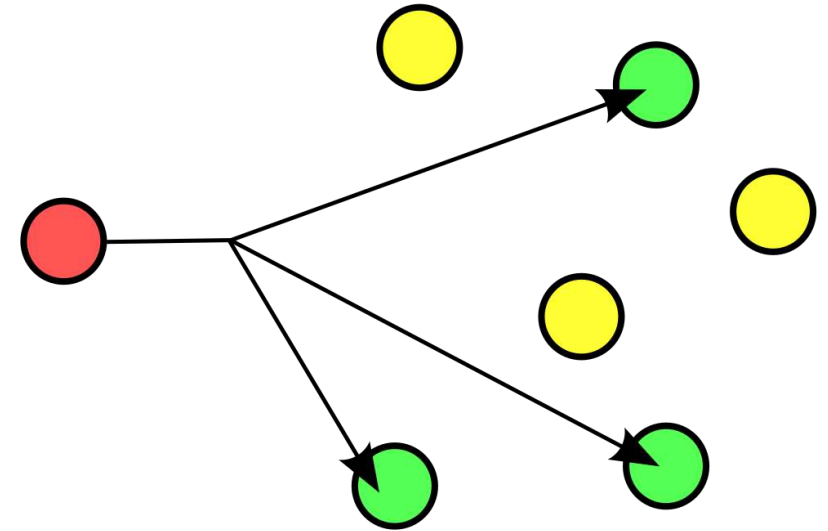
# Transit Gateway – Advanced features

- Supports IP Multi-cast
- AZ Affinity - tries to keep the traffic in the same AZ where the traffic originated. Use **Appliance mode** to influence the AZ selection.
- Supports sharing Transit Gateway across AWS accounts using Resource Access Manager (RAM)
- Architectures:
  - Centralized VPC endpoint
  - Centralized Egress VPC
  - Centralized Traffic Inspection using Gateway Load Balancer
  - Centralized Traffic Inspection using AWS Network Firewall



# IP Multicast

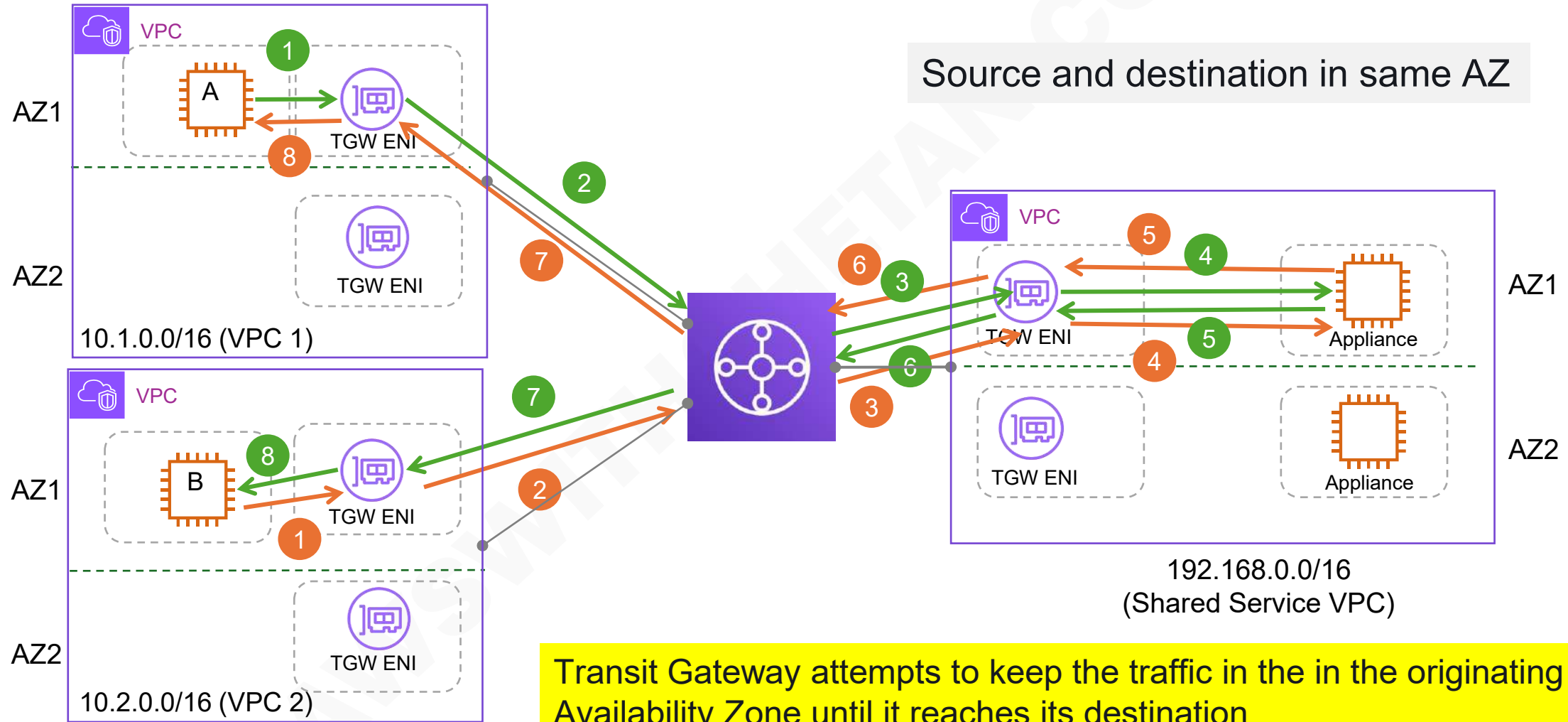
- Multicast is a communication protocol used for delivering a single stream of data to multiple receiving computers simultaneously.
- Single/multiple sources and destinations
- Destination is a multicast group address:
  - Class D - 224.0. 0.0 to 239.255. 255.255
- Connectionless UDP based transport
- One way communication
- Examples: Sending email to the email-list, Conference call / Group chat, OTT platforms / TV Media, Stock exchange transaction updates
- Enable Transit Gateway for Multicast services while creating the transit gateway



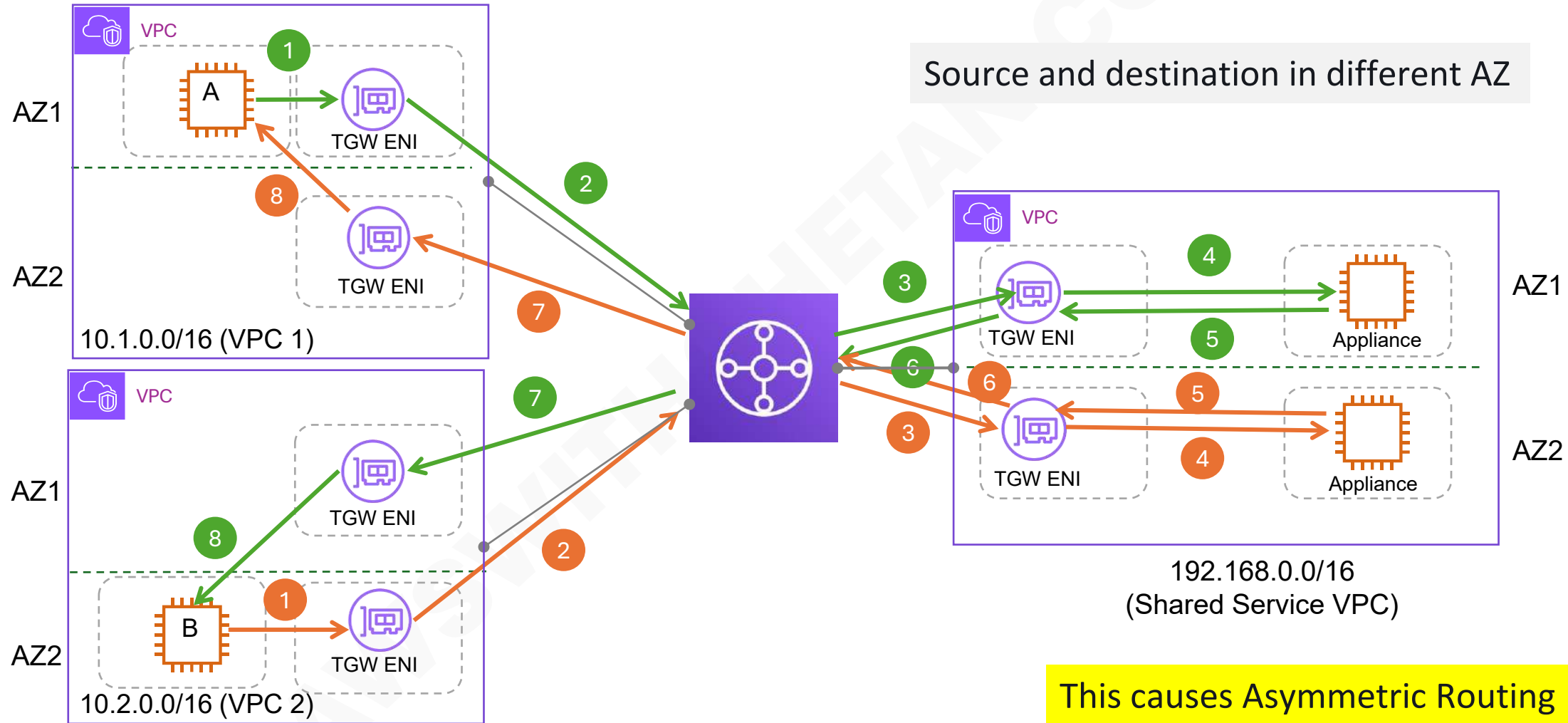
<https://en.wikipedia.org/wiki/Multicast>



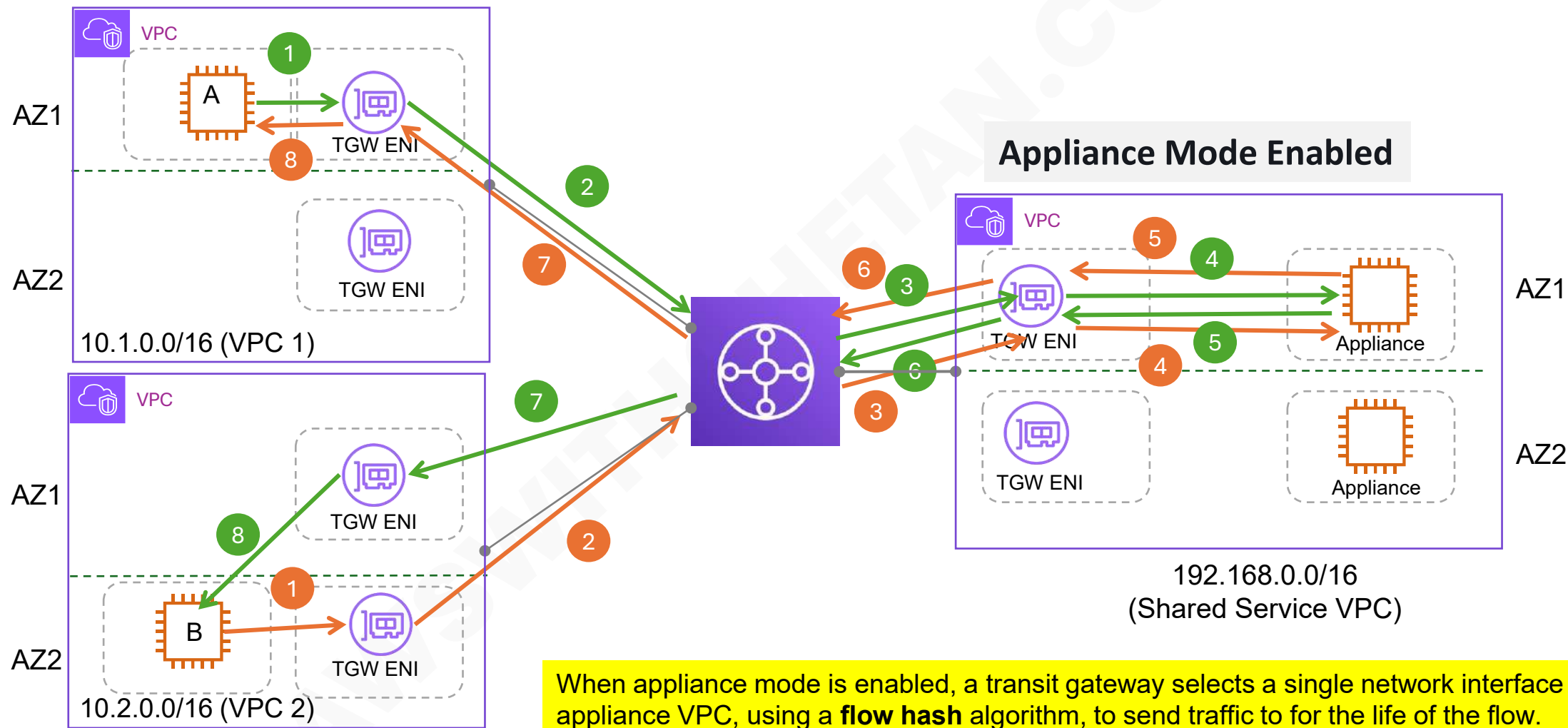
# Transit Gateway – AZ Affinity



# Transit Gateway – AZ Affinity



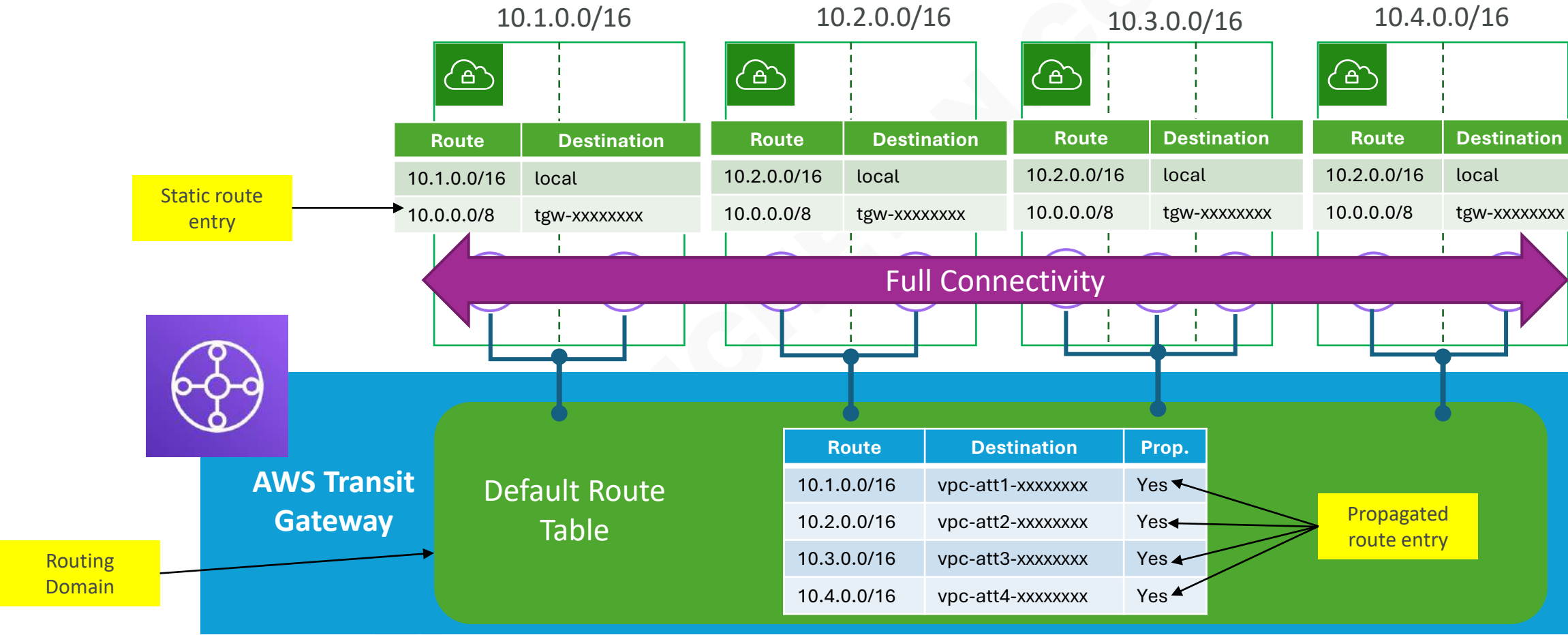
# Transit Gateway – AZ Affinity



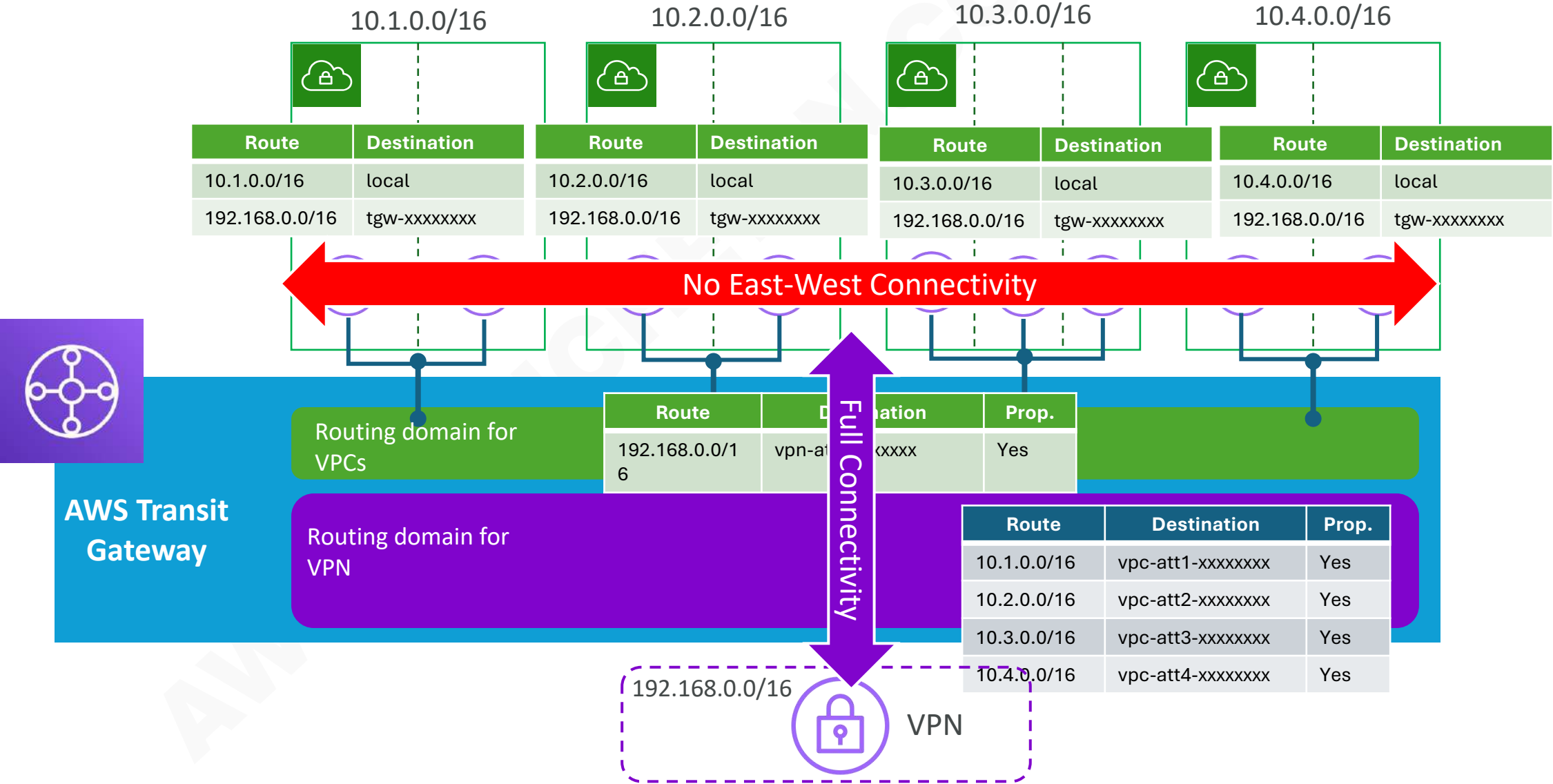
When appliance mode is enabled, a transit gateway selects a single network interface in the appliance VPC, using a **flow hash** algorithm, to send traffic to for the life of the flow.

# Transit Gateway – Route tables

# Flat network

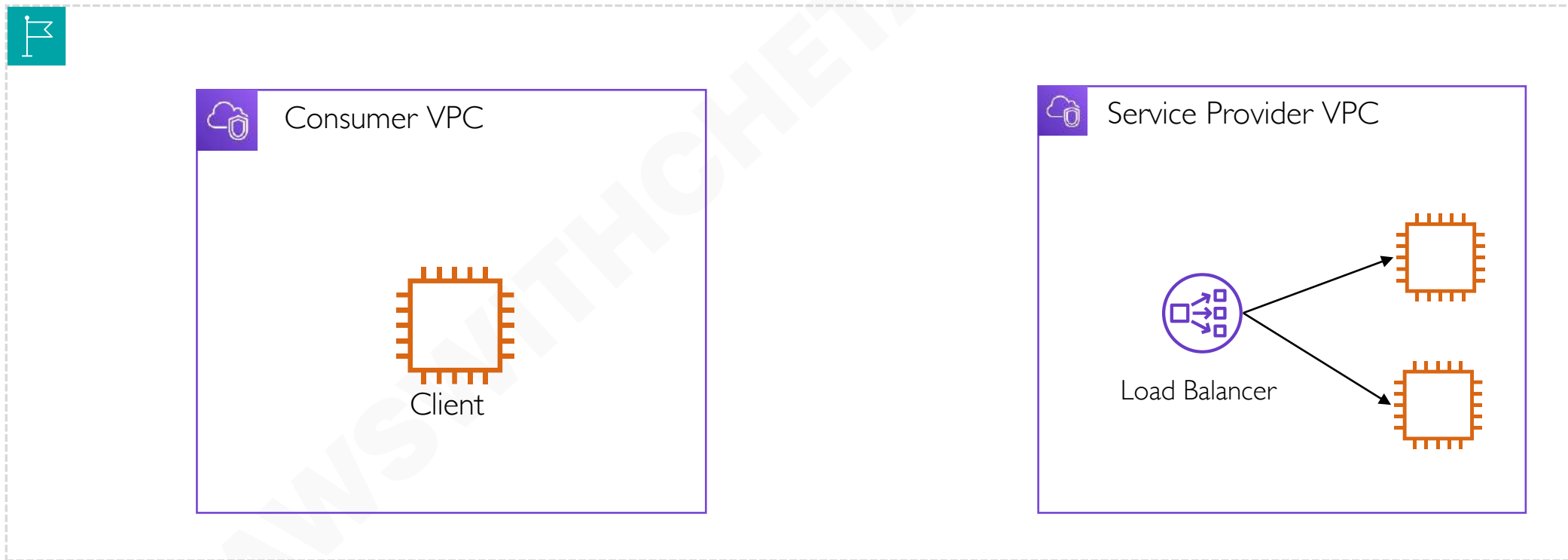


# Segmented Network



# VPC Peering vs VPC endpoint (PrivateLink)

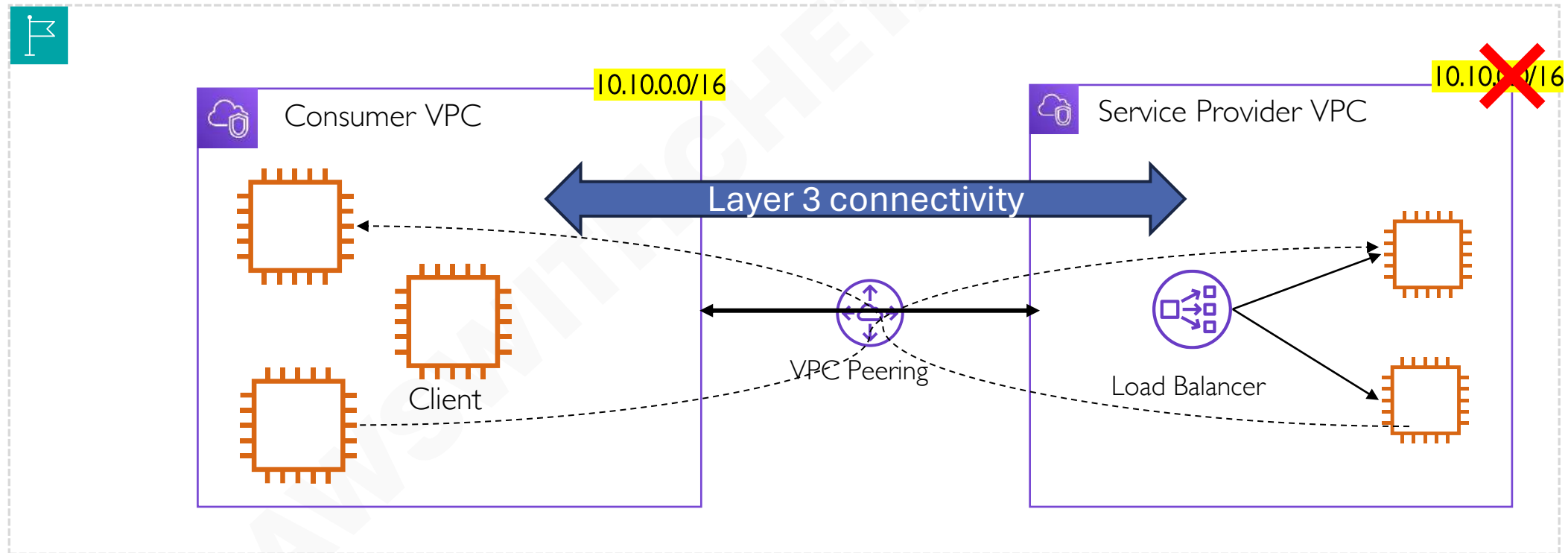
# Accessing Application privately across VPCs





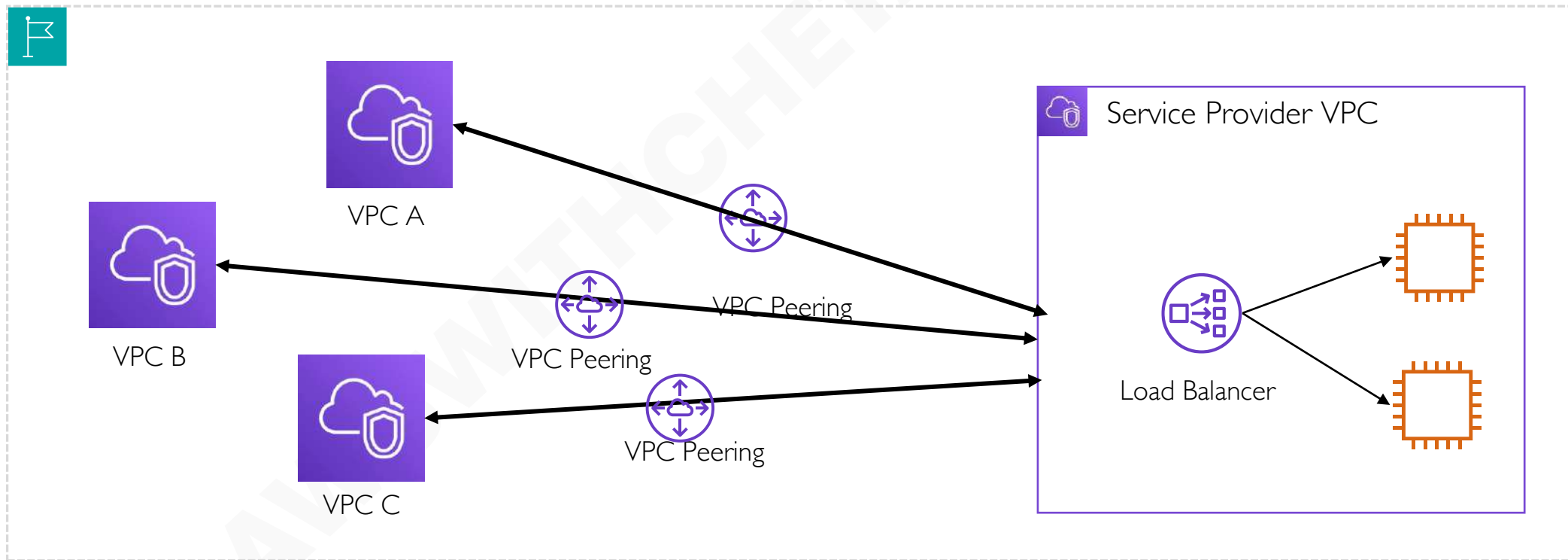
# Using VPC Peering

- VPC peering enables full layer 3 connectivity (and not application-level connectivity)
- Two-way communication
- Can not have VPC peering between VPCs having overlapping CIDRs



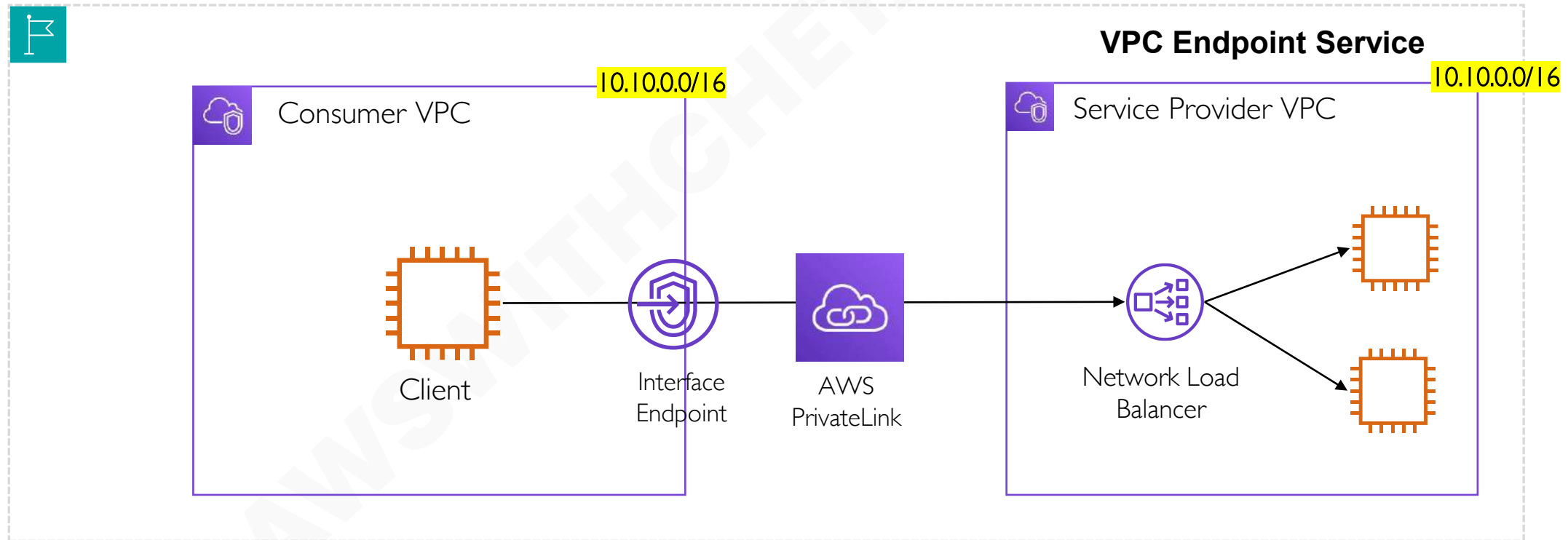
# Using VPC Peering

- There is a limit on number of VPC peering connections (as of now 125 connections)



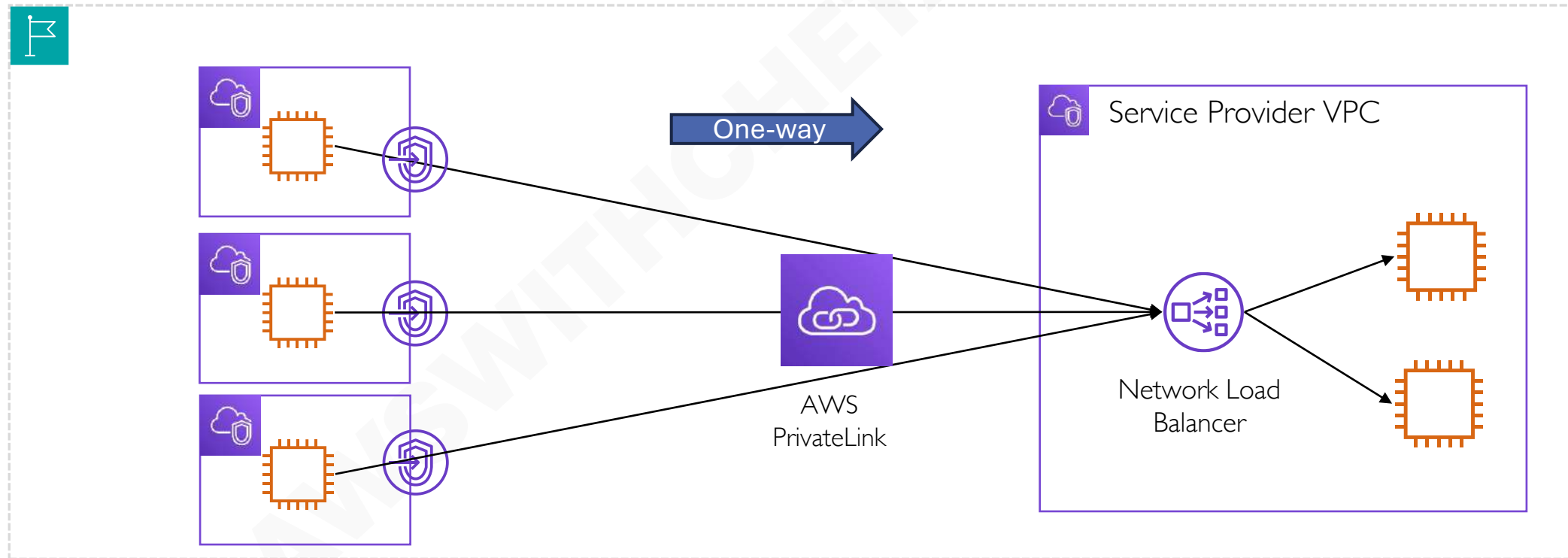
# Using VPC endpoint (PrivateLink)

- VPC endpoint + AWS PrivateLink = VPC Interface endpoint + VPC endpoint service
- Allows connecting to only Application Services privately
- No problem if Service provide and consume VPCs have overlapping CIDRs



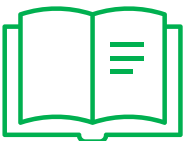
# Using VPC endpoint (PrivateLink)

- Can connect Clients across thousands of consumer VPCs
- One-way communication – From client to the VPC endpoint service



# VPC Peering vs VPC endpoint (PrivateLink)

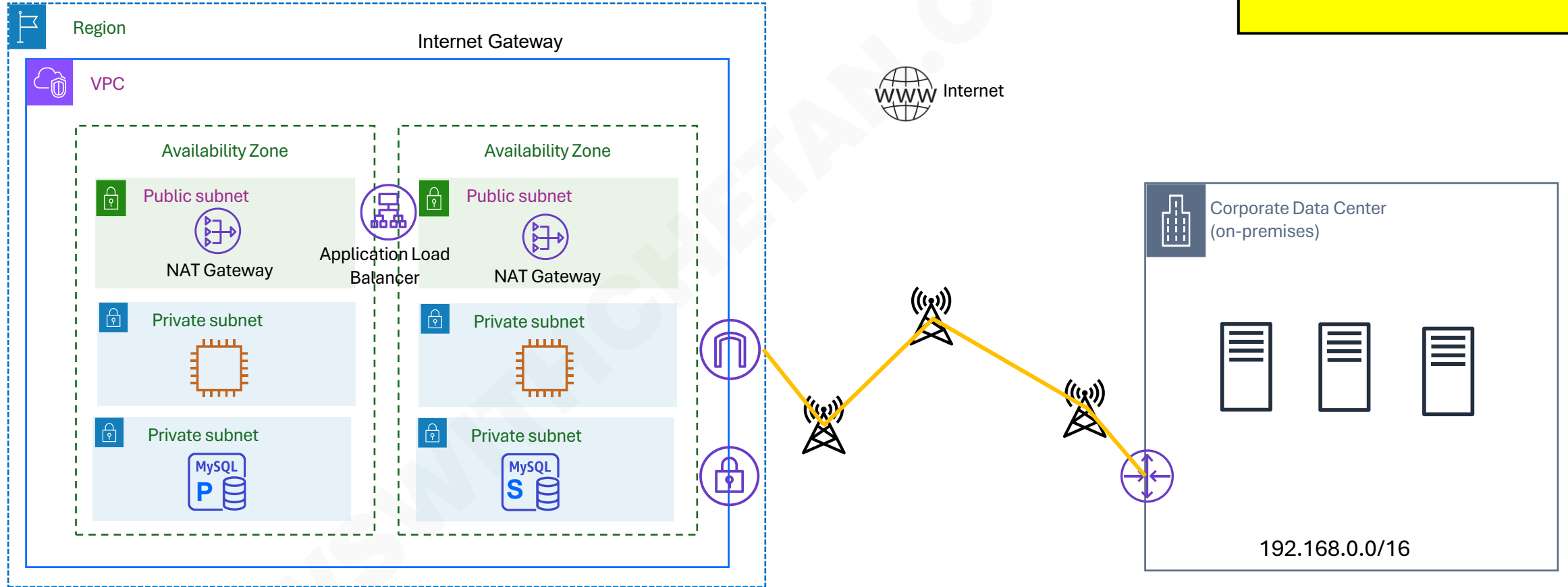
- VPC peering is useful when there are many resources that should communicate between peered VPCs
- VPC endpoint (AWS PrivateLink) should be used when you want to allow access to only single application hosted in your VPC to the clients in other VPCs (without peering the VPCs)
- When there is overlapping CIDRs, VPC peering connection cannot be created. However, AWS PrivateLink does support overlapping CIDR.
- We can create a maximum of 125 peering connections. There is no limit on AWS PrivateLink connections.
- VPC peering enables bidirectional traffic origin. AWS PrivateLink allows only consumer to originate the traffic.



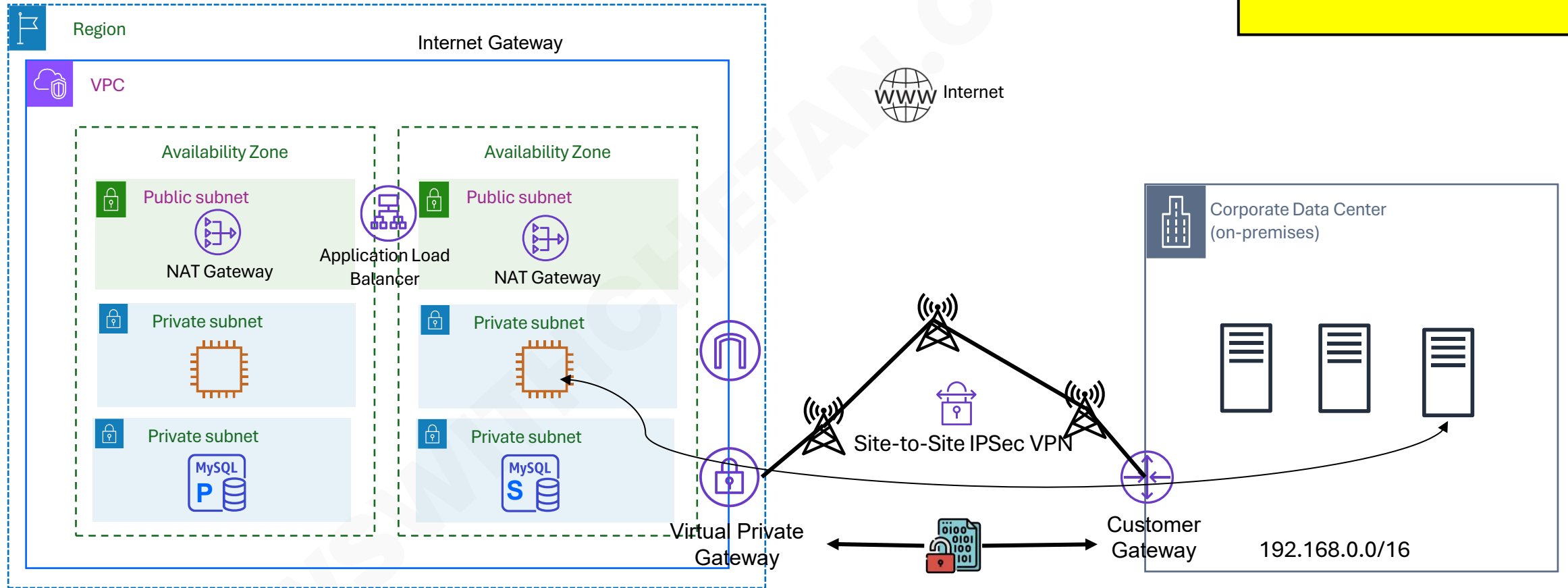
# VPC private connectivity options

- VPC Peering connection
  - AWS Transit Gateway
  - AWS Site-2-Site VPN
  - AWS Client VPN
  - AWS Direct Connect
- } Hybrid Networking

## Site-to-Site VPN



# Site-to-Site VPN





# Let's see how IPSec VPN works



## IKE Phase 1 (IKE SA)

- Decide IKEv1 or IKEv2
- Exchange Credentials (Pre-shared key or Certs)
- Decide on NAT-T, Dead Peer Detection, Keep-Alive
- Auth type – SHA-2, SHA-1, MD5
- Encryption – DES, 3DES, AES
- SA Life – Phase 1 SA expiration (Typically 8 hrs)
- DH Group - 2, 14-24

Play

© Stephane Maerek, Chetan Agrawal

## How IPSec VPN works?

Unlisted



AWS with Chetan

99K subscribers

Analytics

Edit video



Share

Promote

Download

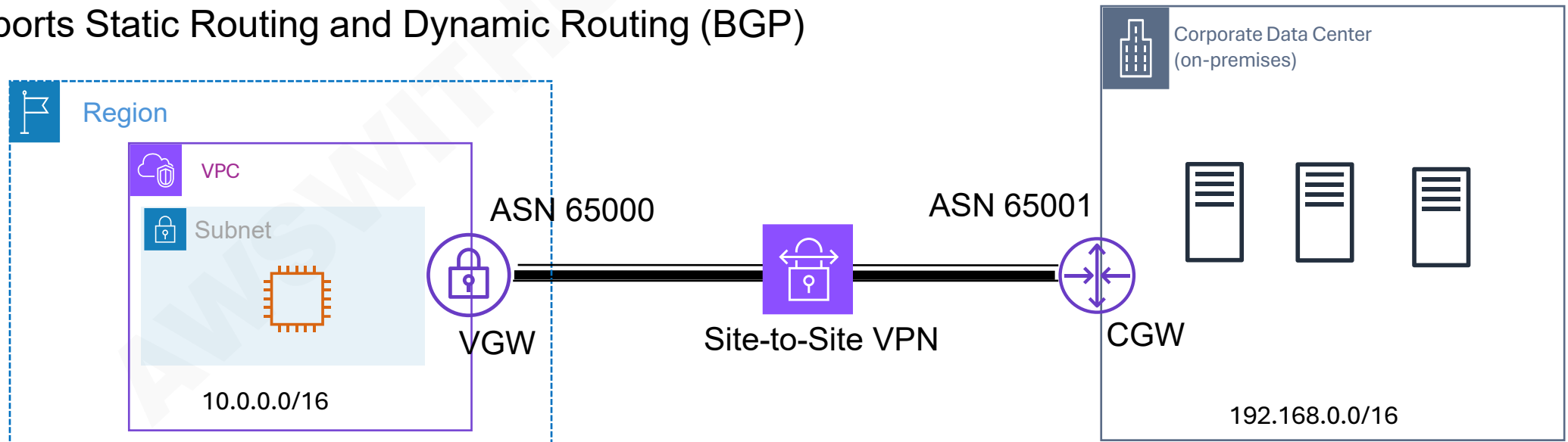


<https://youtu.be/e-kTDqcuVFQ>



# AWS Site-to-Site VPN

- A Managed IPsec VPN connection between on-premises router and AWS VPC
- Customer Gateway (CGW) on Customer side and Virtual Private Gateway (VGW) on AWS side
- Traffic flows over the internet but **encrypted at Layer 3**
- 2 VPN Tunnels for High Availability.
- Single connection provides bandwidth of ~1.25 Gbps
- Supports Static Routing and Dynamic Routing (BGP)



# Dynamic Routing using Border Gateway Protocol (BGP)

© Stephane Maarek, Chetan Agrawal



## AWS Networking - BGP Basics

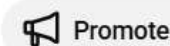
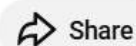
Unlisted



**AWS with Chetan**  
99.1K subscribers

Analytics

Edit video

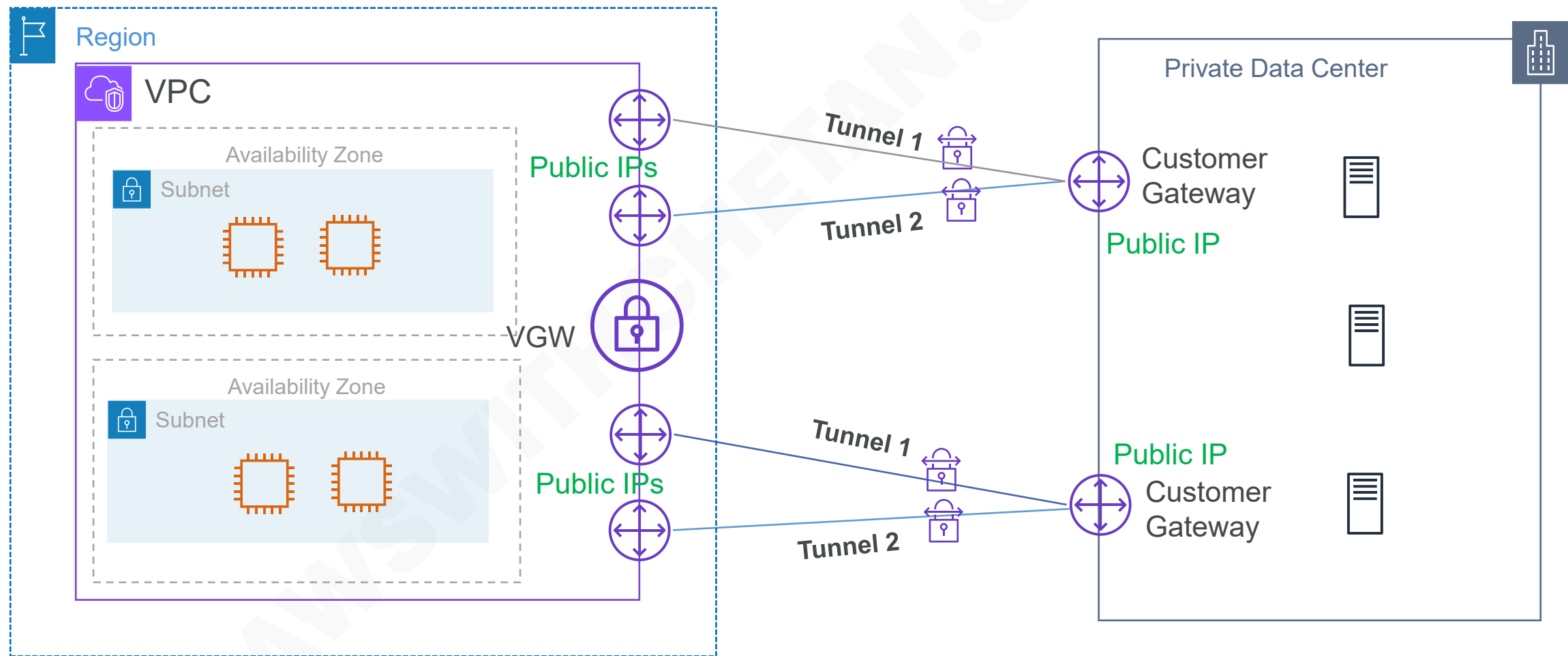


[https://youtu.be/LP\\_n396Rhy4](https://youtu.be/LP_n396Rhy4)



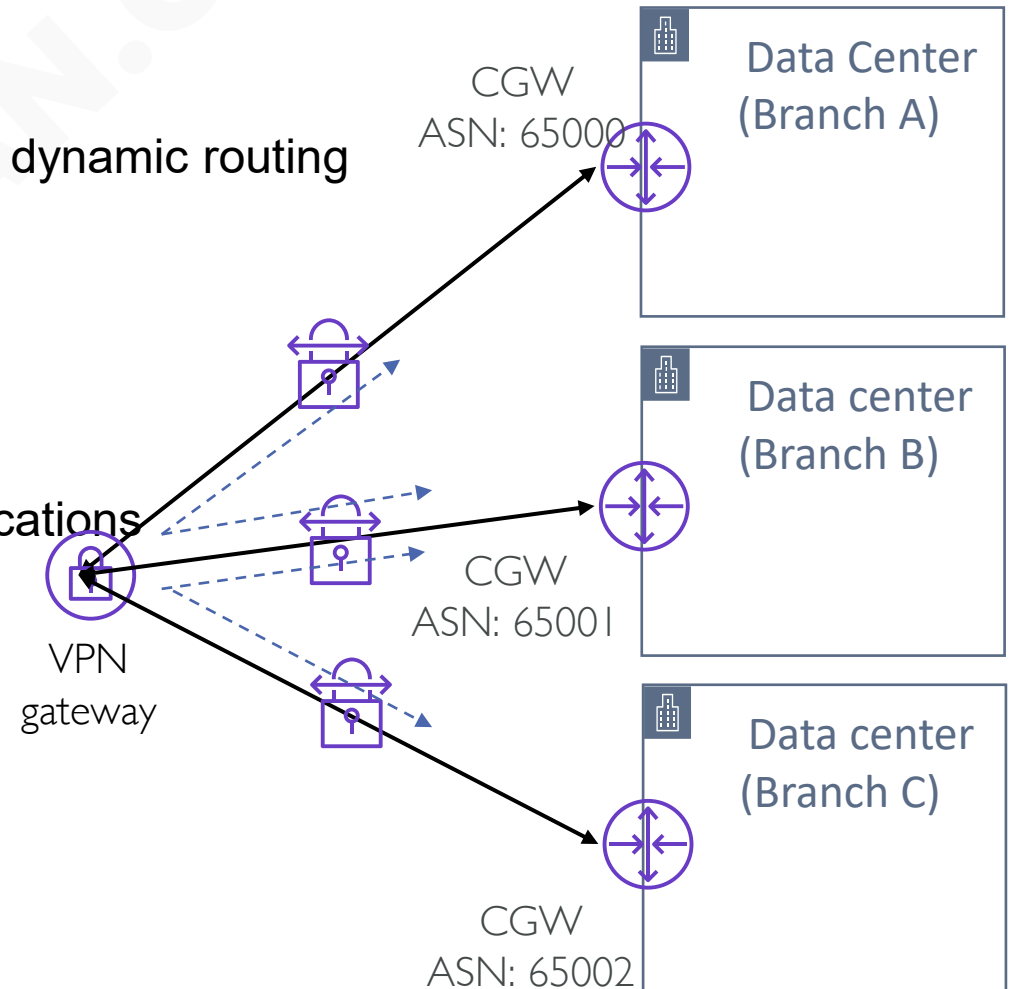
# VPN Architectures

# Primary and Failover VPN connections



# VPN CloudHub – Routing between multiple customer sites

- Using VPN Gateway in detached mode
- Each customer gateway must have unique BGP ASN with dynamic routing
- Sites must not have overlapping IP ranges
- Connect upto 10 Customer Gateways
- Can serve as failover connection between on-premises locations



# VPC private connectivity options

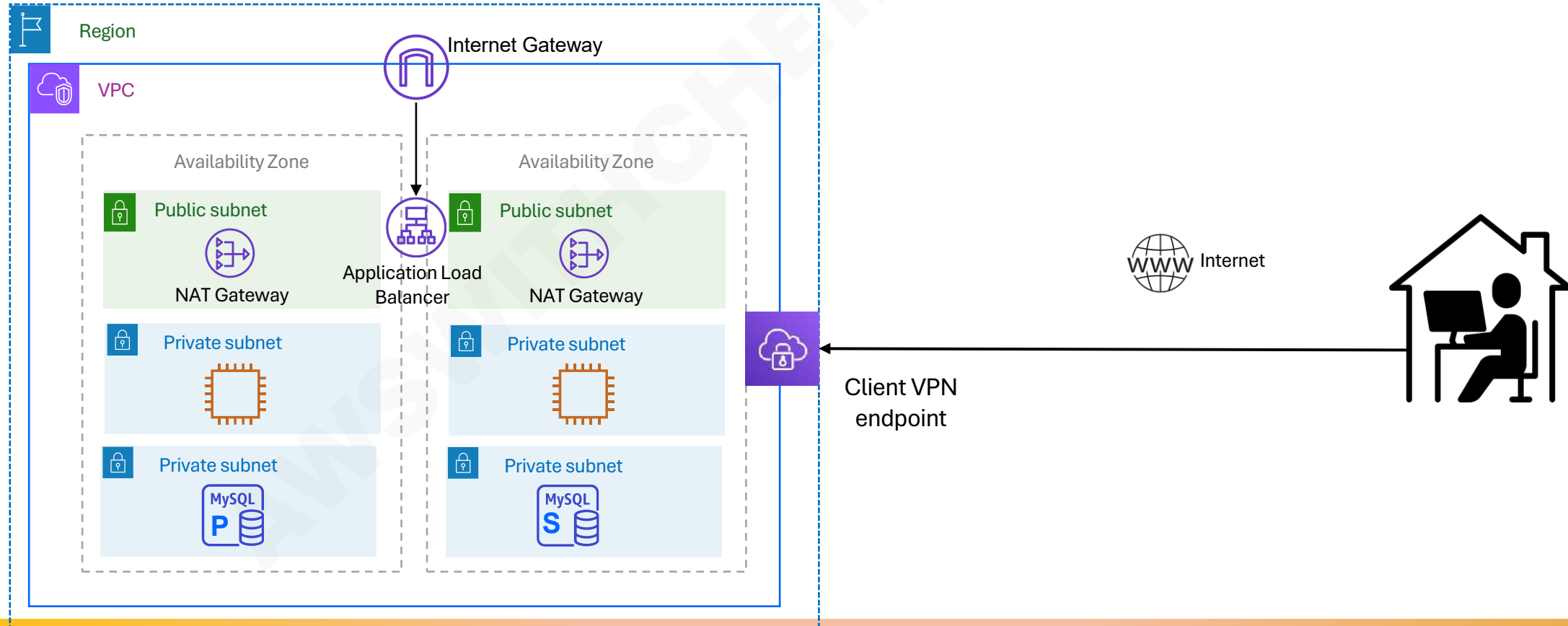
- VPC Peering connection
- AWS Transit Gateway
- AWS Site-2-Site VPN
- **AWS Client VPN**
- AWS Direct Connect

AWSWITHCHETAN.COM

# AWS Client VPN

## Client VPN

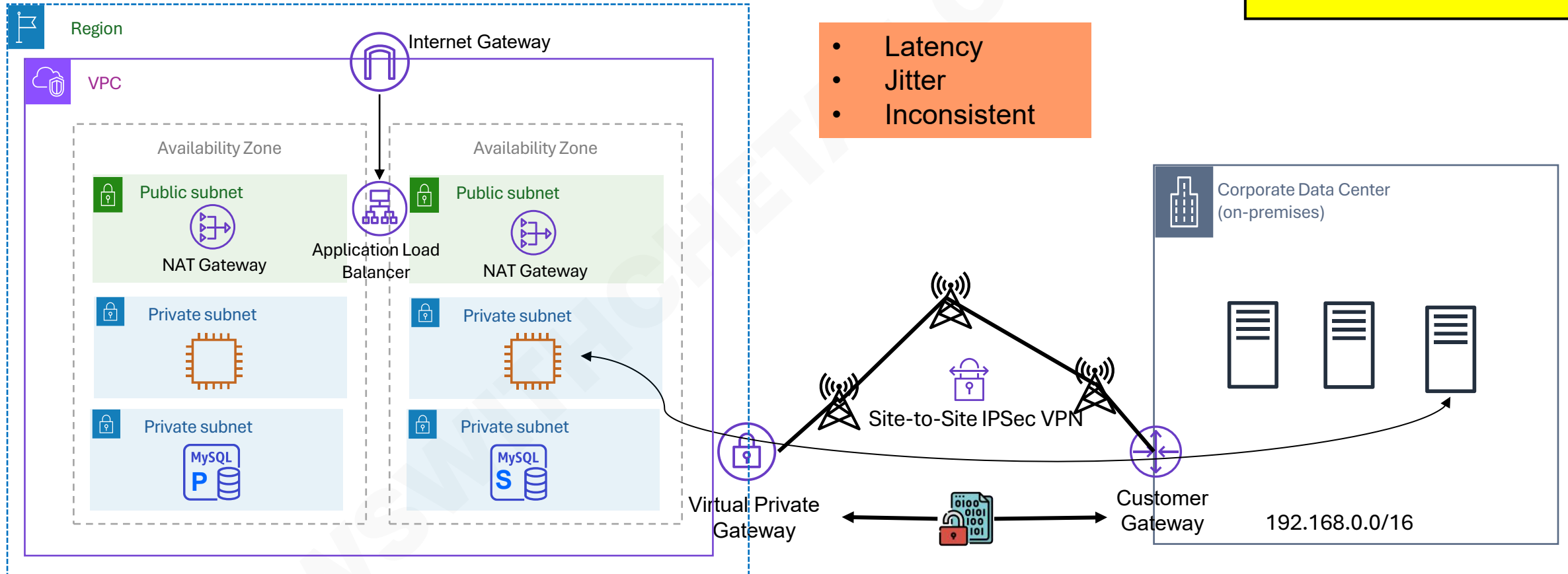
- Connect from your workstation privately to VPC using OpenVPN
- Encrypted traffic goes over the internet
- Access VPC resources over the Private IPs as if you are part of the same network



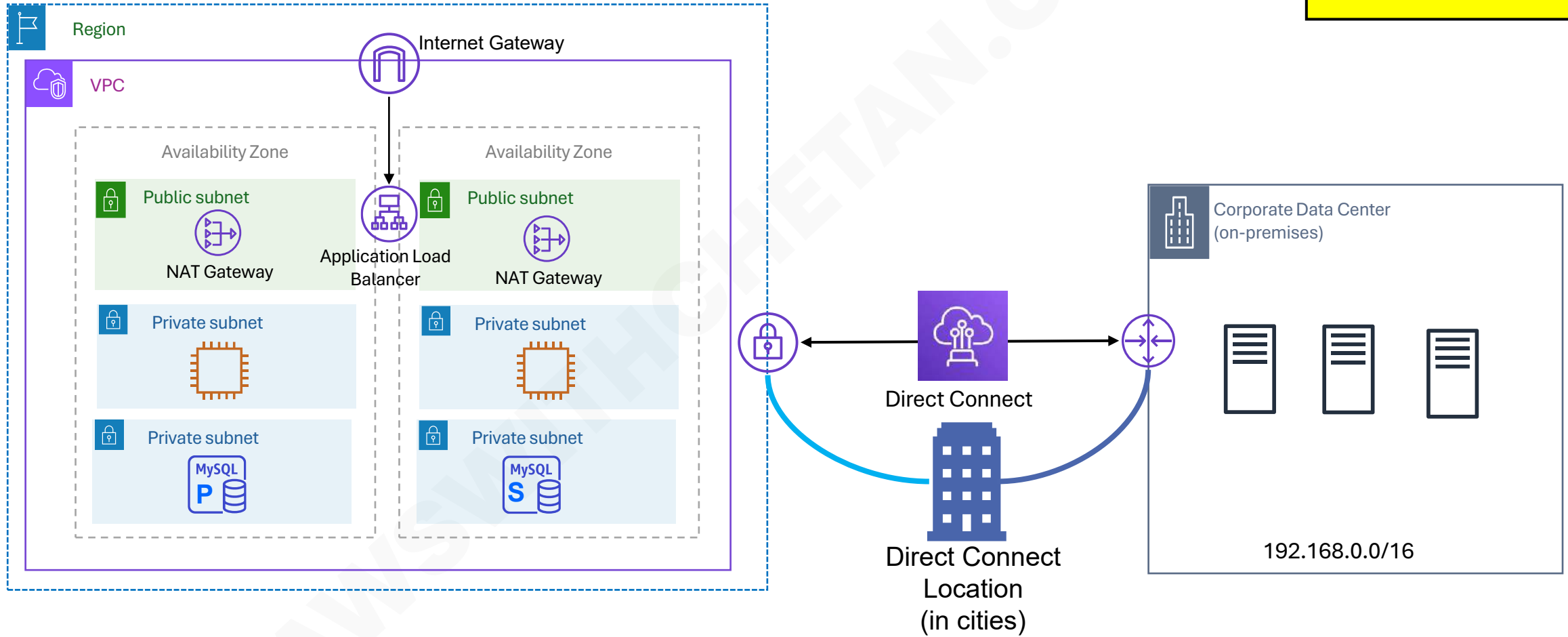


## Site-to-Site VPN

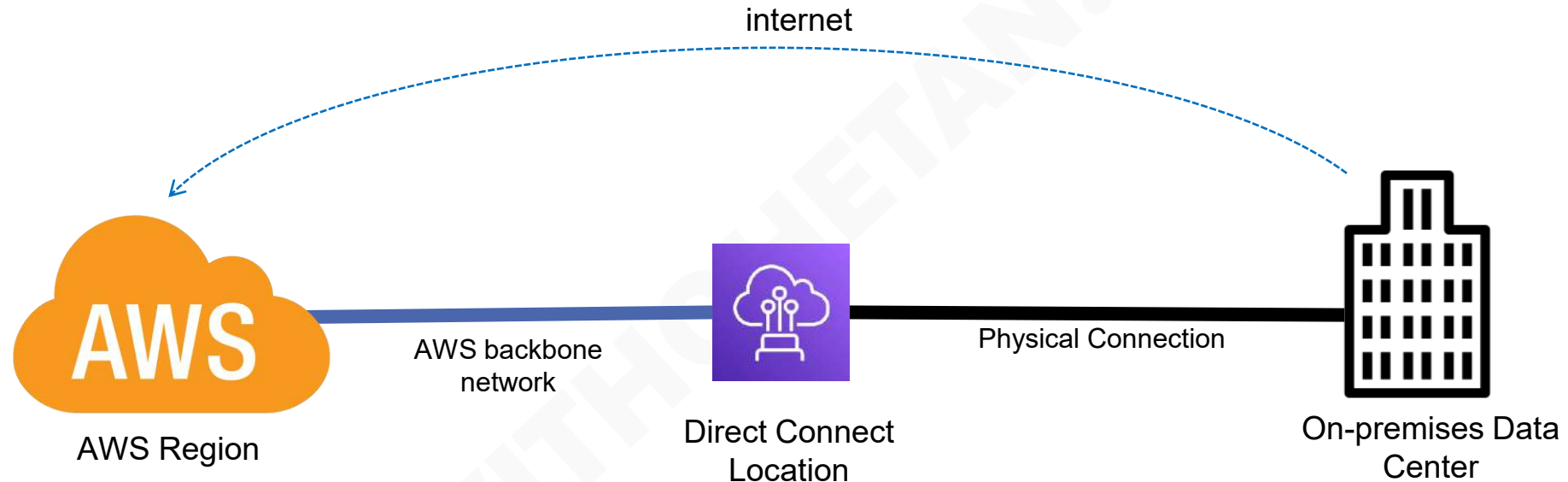
- Latency
- Jitter
- Inconsistent



# Direct Connect



# Direct Connect (DX)



- Provides dedicated and consistent network
- Provides network bandwidth from 50 Mbps up to 100 Gbps over a single connection
- Low data transfer cost
- May take up to 1-3 months to establish end-to-end connectivity.

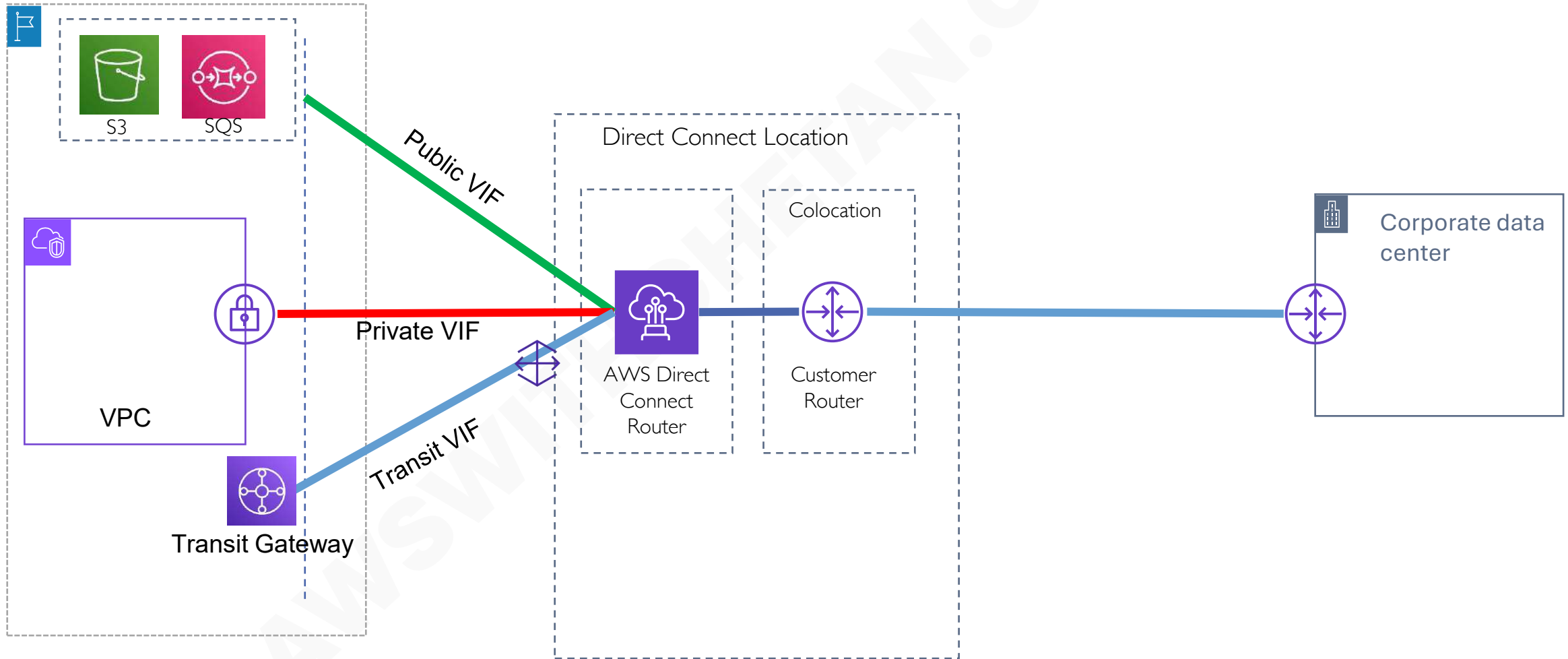
# AWS Direct Connect (DX)

- AWS Direct Connect provides private, dedicated network connectivity between on-premises and AWS, bypassing the public internet
- Offers bandwidth options: 1 Gbps, 10 Gbps, 100 Gbps on dedicated connection, and 50 Mbps to multi-Gbps on hosted connection
- Fixed port-hour charges based on the port bandwidth
- Helps reduce data transfer costs, especially outbound traffic.
- Use **Private VIF** to access a VPC, **Public VIF** to privately access AWS public services like S3 or DynamoDB and **Transit VIF** to connect to Transit Gateway (via Direct Connect Gateway)
- **Use cases:** Large data migrations, hybrid workloads, latency-sensitive applications

# Direct Connect – Connection Types

- **Dedicated Connections:** 1Gbps, 10 Gbps and 100 Gbps capacity
  - Physical ethernet port **dedicated to a customer**
  - Can be either setup by your Network Provider or AWS Direct Connect Partner
- **Hosted Connections:**
  - 50, 100, 200, 300, 400, 500 Mbps and 1 Gbps, 2 Gbps, 5 Gbps, 10 Gbps
  - Connection requests are made via **AWS Direct Connect Partners**
  - AWS uses traffic policing on hosted connections – excess traffic is dropped

# AWS Direct Connect - Virtual Interfaces (VIFs)

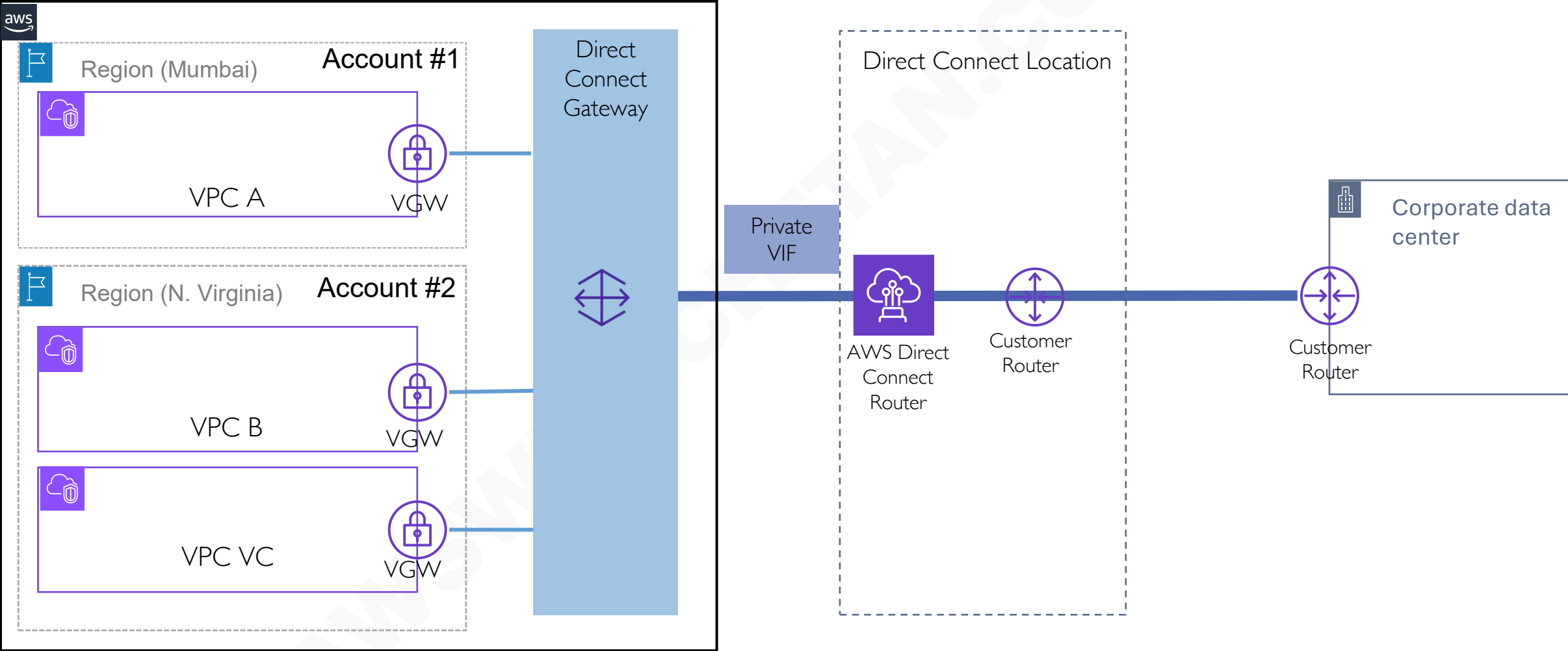


# Direct Connect Gateway

- **Global network device – Accessible in all regions**
- Direct Connect integrates via a private VIF or a transit VIF

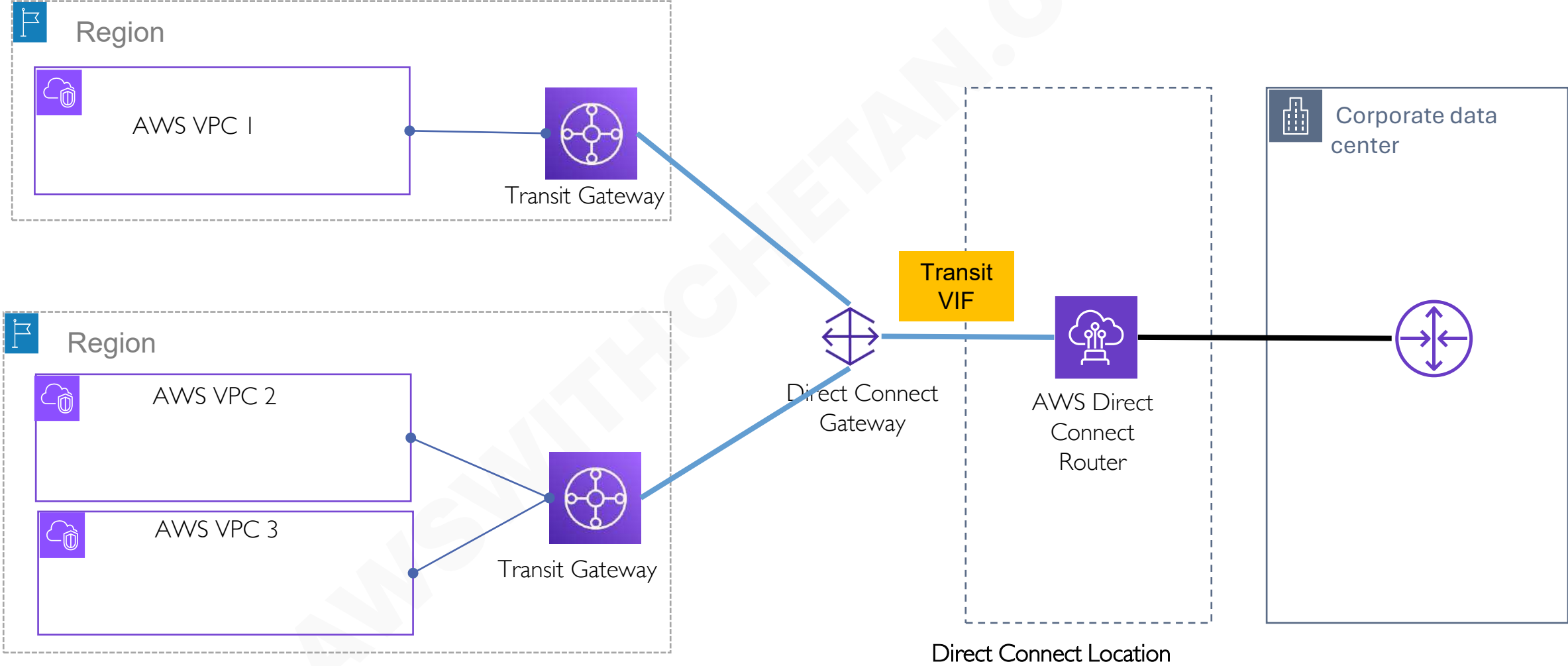
AWSWITHCHETAN.COM

# DX Gateway with Private VIF





# DX Gateway with Transit VIF



# AWS Direct Connect – Important to know

- AWS Direct Connect provides private, dedicated network connectivity between on-premises and AWS.
- Uses BGP for Dynamic routing
- Offers high bandwidth options: 1 Gbps, 10 Gbps, 100 Gbps on dedicated ports, and hosted DX from 50 Mbps to multi-Gbps.
- Traffic is not encrypted by default; for encryption use VPN over DX or MACsec (where supported).
- It takes few weeks to month to establish Direct Connect connectivity (hence need advance planning)
- Use Private VIF to access a VPC, Public VIF to privately access AWS public services like S3 or DynamoDB and Transit VIF to connect to Transit Gateway (via DX Gateway)
- Use DX Gateway to connect a single DX to multiple VPCs across multiple Regions.
- Provides reduce data transfer costs (e.g. \$0.02/GB as compared to internet charge of \$0.09/GB)
- Architecture: For high availability, deploy multiple DX connections in different DX locations;
- Architecture: You can use DX+VPN for resilient connectivity where VPN acts as a failover network.



# VPC Traffic Monitoring

VPC Flow logs and Traffic mirroring

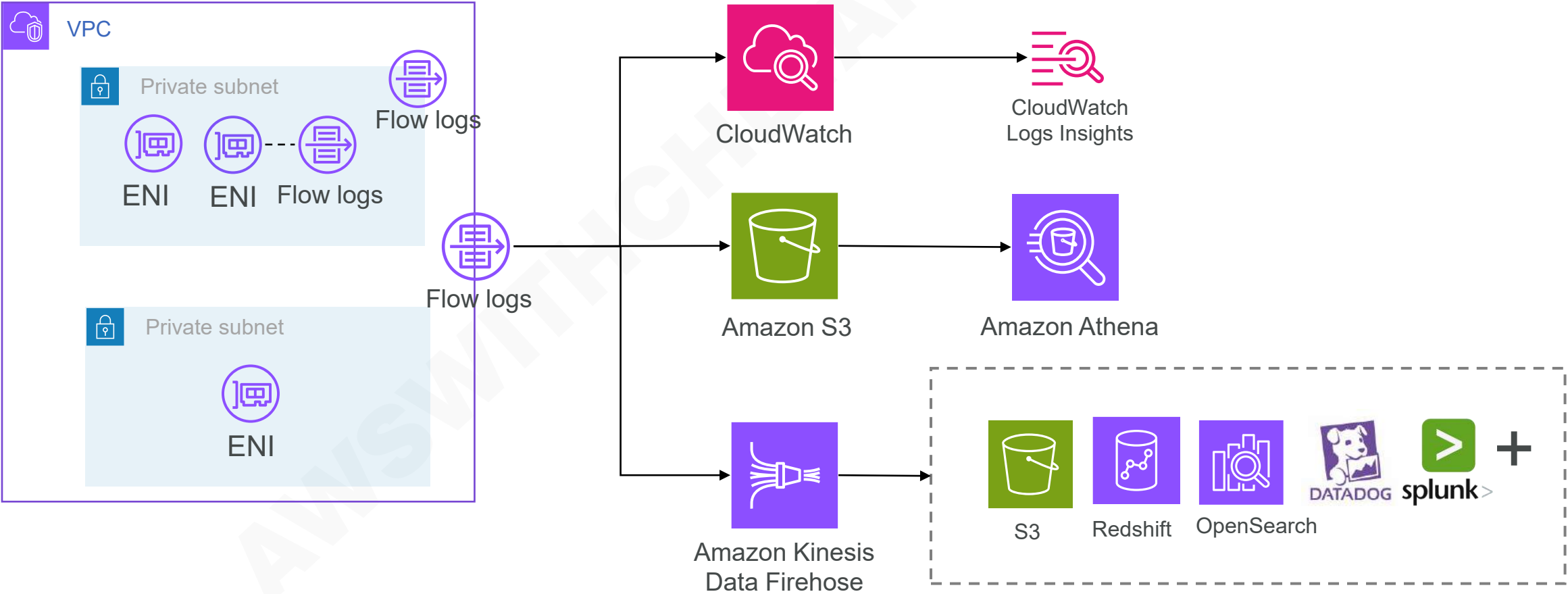
# VPC Flow Logs

- Capture information about IP traffic going in/out of your elastic network interfaces (ENI):
  - Elastic Network Interface Flow Logs
  - Subnet Flow Logs
  - VPC Flow Logs
- Helps to monitor & troubleshoot connectivity issues
  - Troubleshooting connectivity issues (e.g. "Why can't my EC2 talk to RDS?").
  - Identifying blocked traffic by Security Groups or NACLs
  - Detecting anomalous or suspicious traffic (e.g. unusually high egress).
  - Validating firewall rules and network configurations across subnets/VPCs.
  - Cost analysis by checking high bandwidth usage.
- Also captures network information from AWS managed interfaces: ELB, RDS, ElastiCache, Redshift, Amazon WorkSpaces etc.
- There is no impact to network performance for enabling VPC flow logs.
- Flow logs data can be sent to S3 / CloudWatch Logs / Kinesis Data Firehose for storage and analysis

# VPC Flow logs – default format

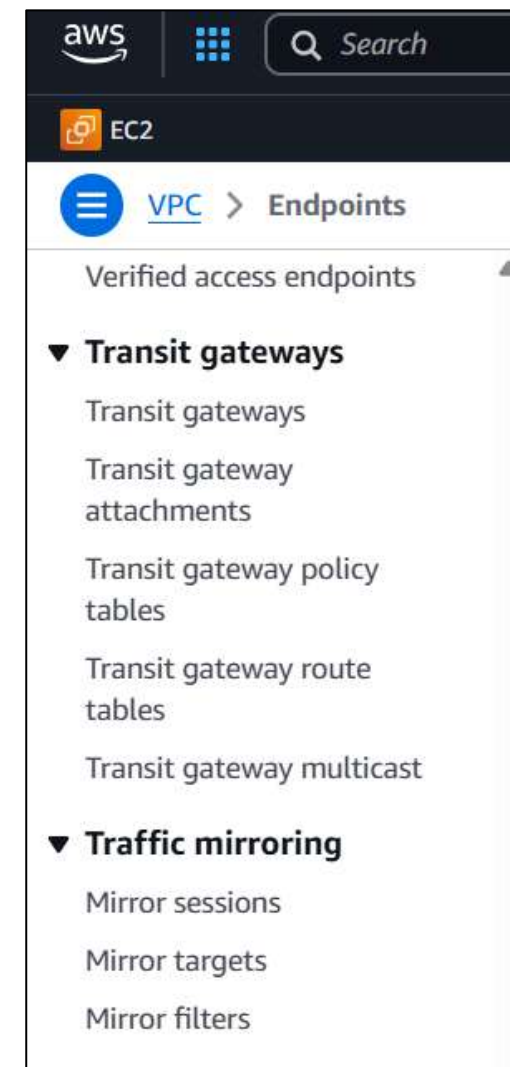
| version                 | account-id | interface-id | srcaddr               | dstaddr       | srcport       | dstport | protocol | packets | bytes | start | end        | action     | log-status |    |
|-------------------------|------------|--------------|-----------------------|---------------|---------------|---------|----------|---------|-------|-------|------------|------------|------------|----|
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28472   | 80       | 6       | 2     | 92    | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 10.100.0.224  | 27.57.246.36  | 80      | 28472    | 6       | 1     | 52    | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28595   | 80       | 6       | 4     | 173   | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 10.100.0.224  | 27.57.246.36  | 80      | 28595    | 6       | 3     | 144   | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 45.129.33.8   | 10.100.0.224  | 58523   | 30018    | 6       | 1     | 40    | 1608988983 | 1608989040 | REJECT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28478   | 443      | 6       | 1     | 52    | 1608988983 | 1608989040 | REJECT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28090   | 443      | 6       | 2     | 104   | 1608988983 | 1608989040 | REJECT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28103   | 443      | 6       | 1     | 52    | 1608988983 | 1608989040 | REJECT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28472   | 443      | 6       | 1     | 52    | 1608988983 | 1608989040 | REJECT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28605   | 22       | 6       | 6     | 528   | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 10.100.0.224  | 27.57.246.36  | 22      | 28605    | 6       | 6     | 240   | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 27.57.246.36  | 10.100.0.224  | 28108   | 443      | 6       | 2     | 104   | 1608988983 | 1608989040 | REJECT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 46.227.200.78 | 10.100.0.224  | 123     | 58455    | 17      | 1     | 76    | 1608988983 | 1608989040 | ACCEPT     | OK |
| ▶ 2020-12-26T18:53:0... | 2          | 523265502623 | eni-0e6a89b99a1bac86d | 10.100.0.224  | 46.227.200.78 | 58455   | 123      | 17      | 1     | 76    | 1608988983 | 1608989040 | ACCEPT     | OK |

# Publishing VPC flow logs

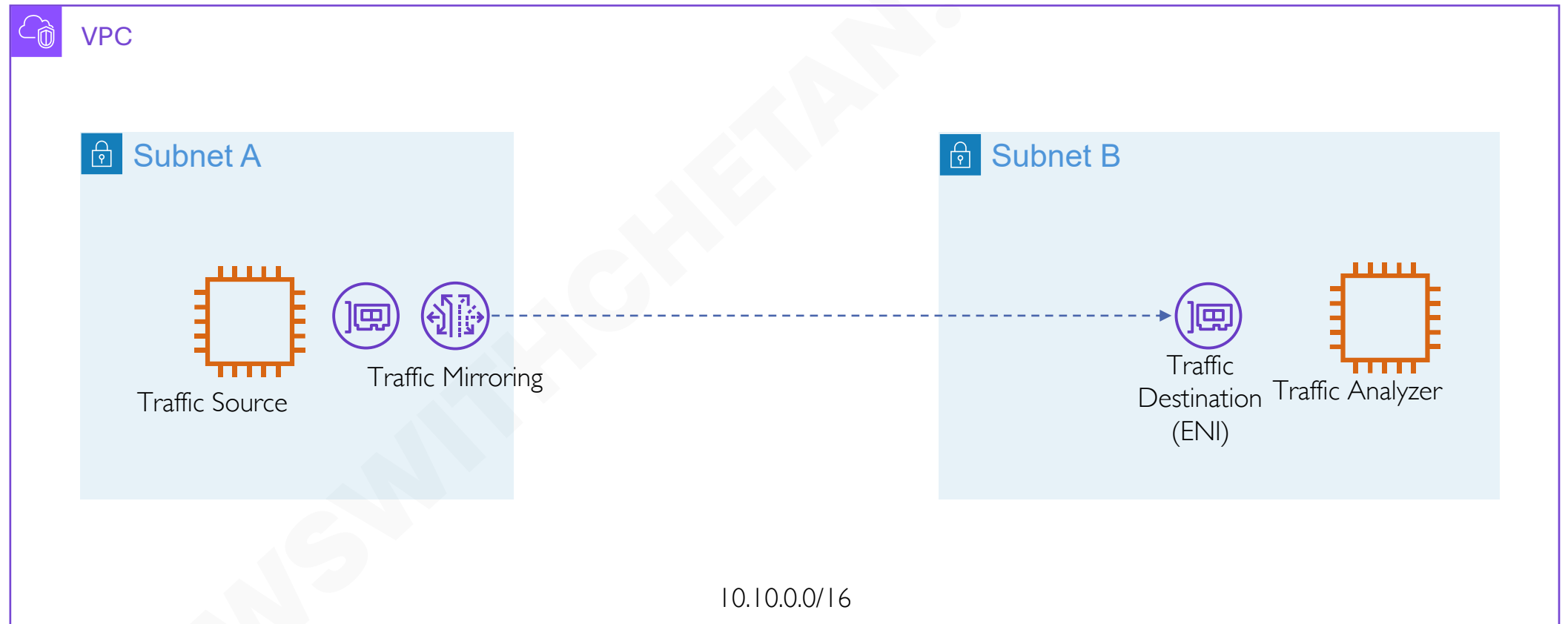


# VPC Traffic Mirroring

- Copies network **traffic** from an elastic network interface of Amazon EC2 instances
- Use cases:
  - Advanced troubleshooting that requires full payload visibility.
  - Deep packet inspection (DPI) for security appliances.
  - Routing mirrored packets to third-party security tools (e.g. Palo Alto, Check Point).
- How to set up Traffic Mirroring (via AWS VPC console)
  - Create the Mirror Target -> Define the Traffic Filter -> Create Mirror Session
- Mirror source is ENI and target could be another ENI or Network Load Balancer (UDP - 4789)
- Filter the traffic to be mirrored by protocol, source/dest port, CIDR
- Source and destination can be in the same VPC or across VPCs which are connected via VPC peering or Transit gateway
- Source and destination VPCs can be in different AWS accounts

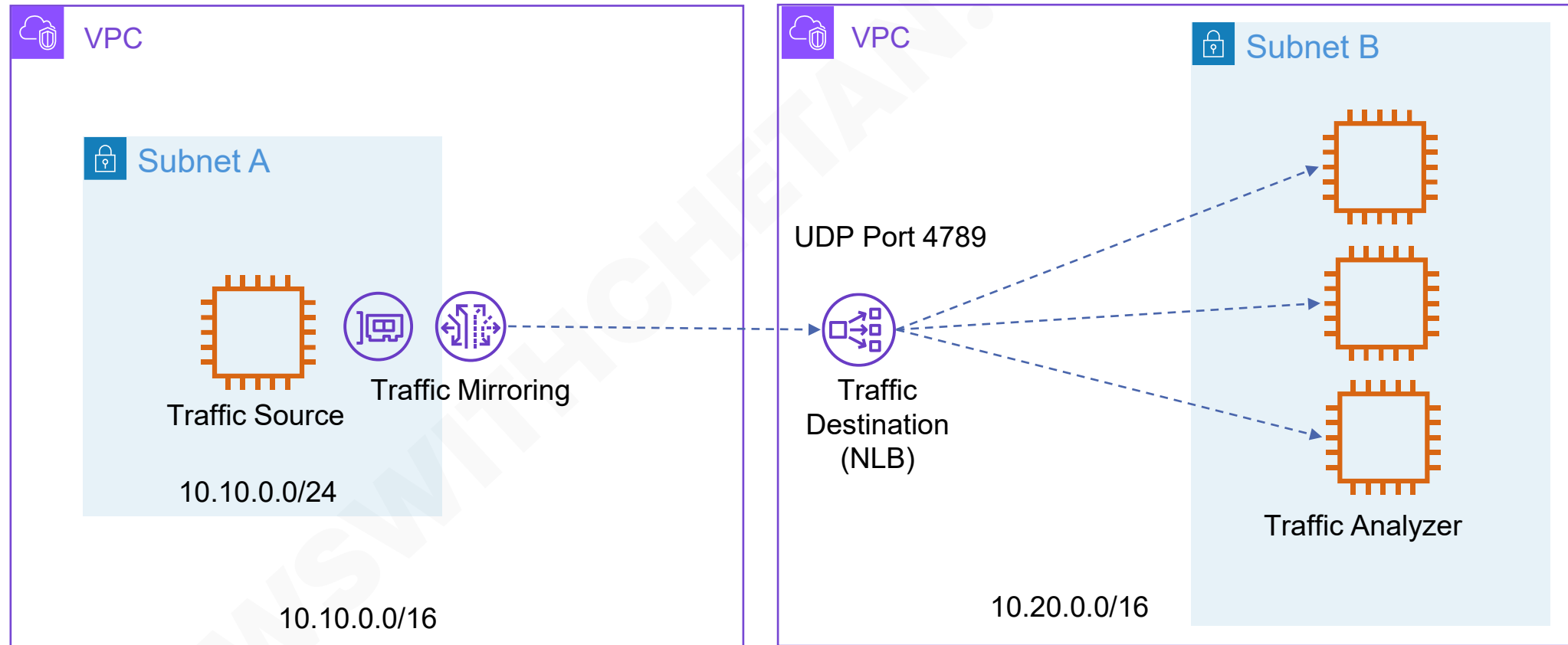


# VPC Traffic Mirroring – ENI as Target





# VPC Traffic Mirroring – NLB as Target



# VPC Flow logs vs Traffic mirroring

## VPC Flow Logs

- Captures only traffic metadata (source/destination, ports, allow/deny)
- Used for troubleshooting network connectivity issues
- Logs can be stored in Amazon CloudWatch, Amazon S3 or can be streamed using Kinesis Data Firehose for further analysis.
- Lightweight
- Low cost

## VPC Traffic mirroring

- Captures actual packet payloads
- Used for deep packet inspection, intrusion detection and threat investigations
- Sends mirrored traffic to 3<sup>rd</sup> party Network security appliances for real-time analysis
- Heavy payload
- High cost



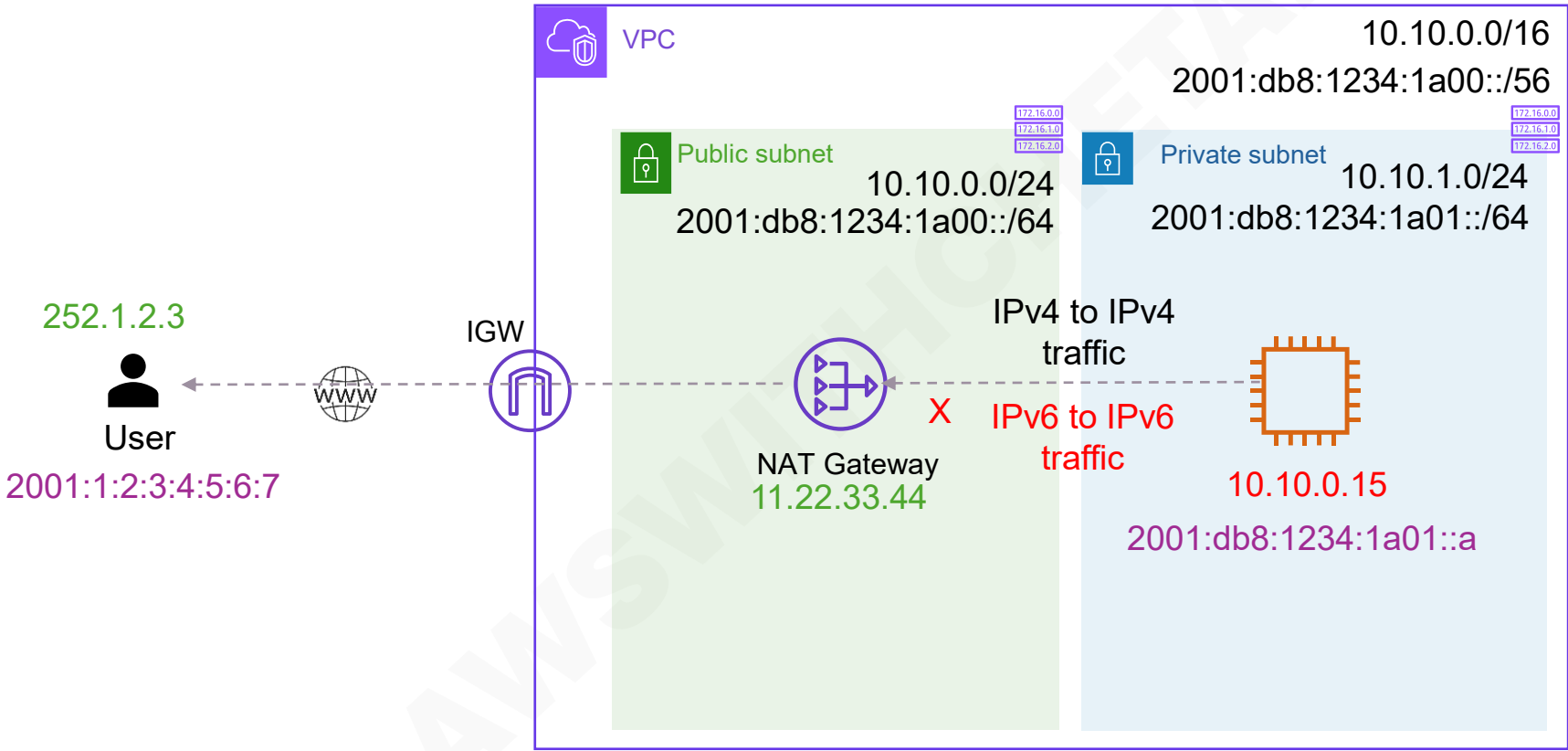
# IPv6 in VPC

# IPv6 in VPC

- VPC supports dual-stack mode
- Resources inside VPC can communicate
- IPv4 support can not be disabled for the
- VPC supports IPv6-only subnets
- IPv6 addresses are Public by default

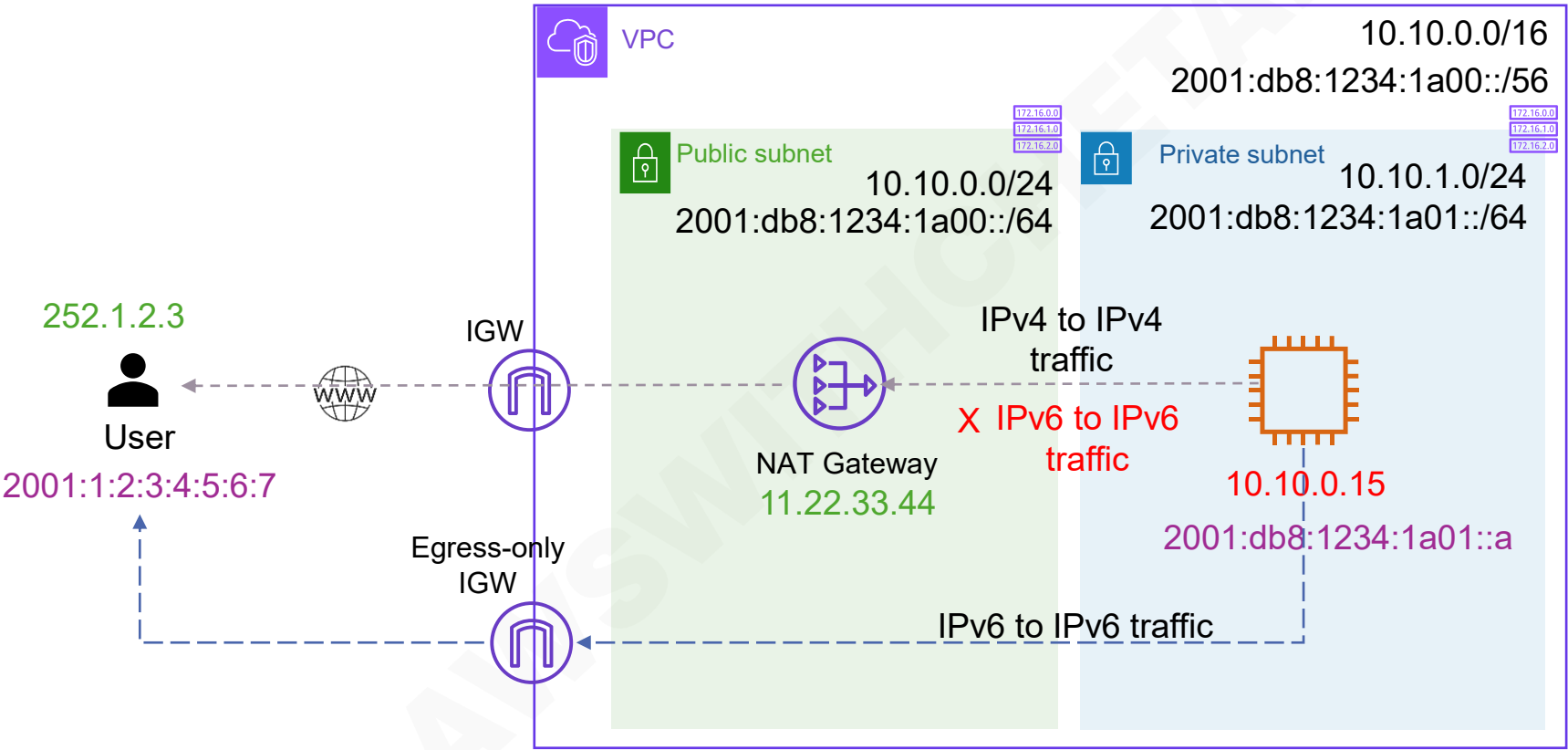


# Outbound Internet access for instance in a Private subnet (IPv6)



| Destination             | Target |
|-------------------------|--------|
| 10.0.0.0/16             | Local  |
| 2001:db8:1234:1a00::/56 | Local  |
| 0.0.0.0/0               | NAT-GW |

# Egress-only internet gateway



| Destination             | Target     |
|-------------------------|------------|
| 10.0.0.0/16             | Local      |
| 2001:db8:1234:1a00::/56 | Local      |
| 0.0.0.0/0               | NAT-GW     |
| ::/0                    | Egress-IGW |

# IPv6 in VPC – Important to know

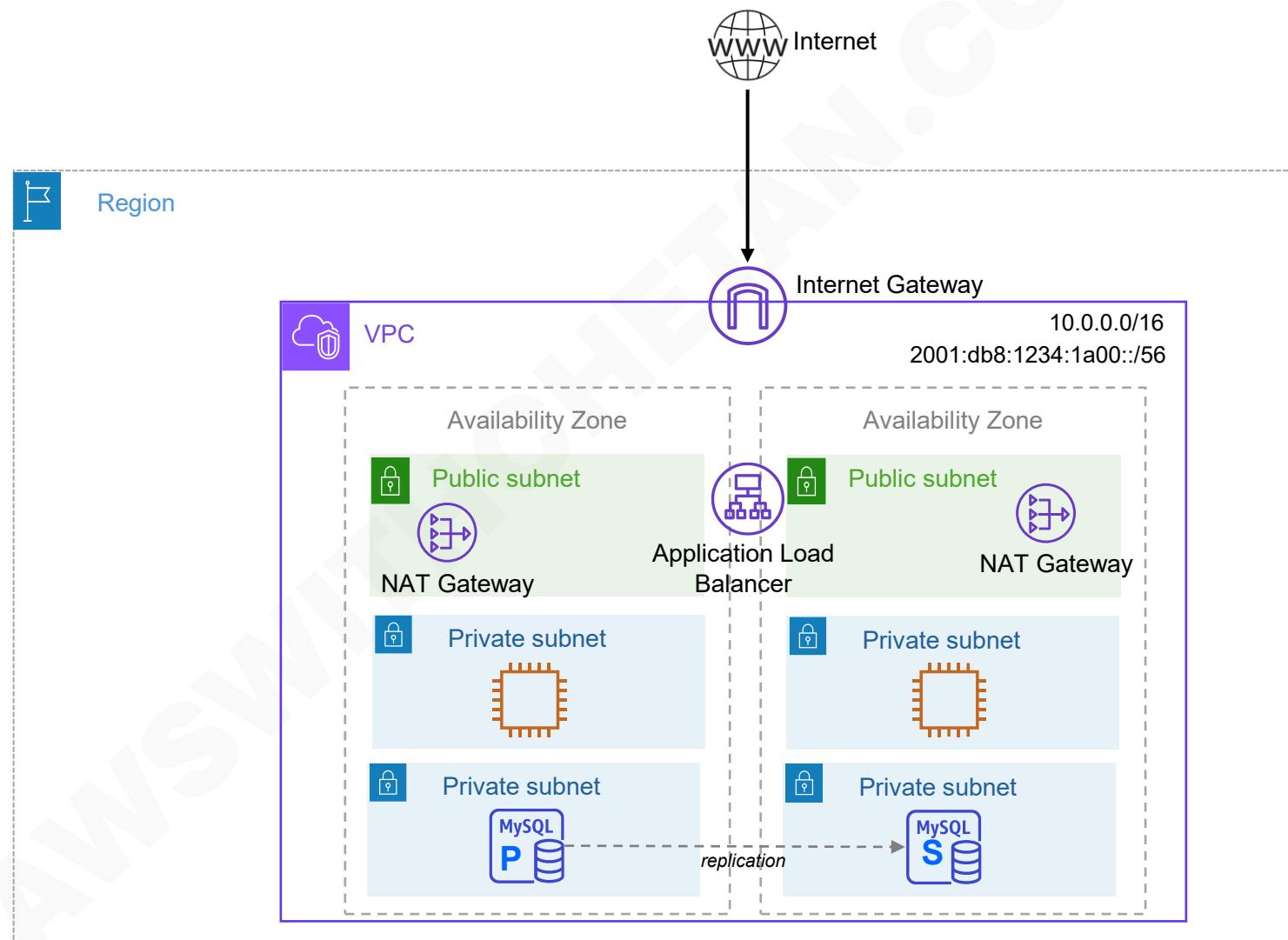
- IPv6 provides globally unique, publicly routable 128-bit addresses
- IPv6 addressing in VPC uses /56 at VPC level and /64 per subnet, which is the minimum required size for IPv6 routing.
- NAT Gateway does not work with IPv6; instead, IPv6 uses an **Egress-Only Internet Gateway**, which allows outbound-only IPv6 traffic from instances while blocking incoming IPv6 connections from the internet.
- Security Groups and NACLs support separate IPv4 and IPv6 rules (:::/0), so you must explicitly Allow or Deny IPv6 traffic.
- Dual-stack mode lets you run IPv4 and IPv6 together in the same VPC and subnets, improving compatibility during migration.
- VPC Endpoints (Interface/Gateway) support IPv6 in many regions, allowing private IPv6 access to AWS services.
- Route tables include separate IPv6 routes, commonly pointing to Egress-Only IGW or Internet Gateway depending on inbound/outbound needs.
- Useful for IoT, global-scale workloads, and scenarios needing large address space or avoiding NAT bottlenecks.



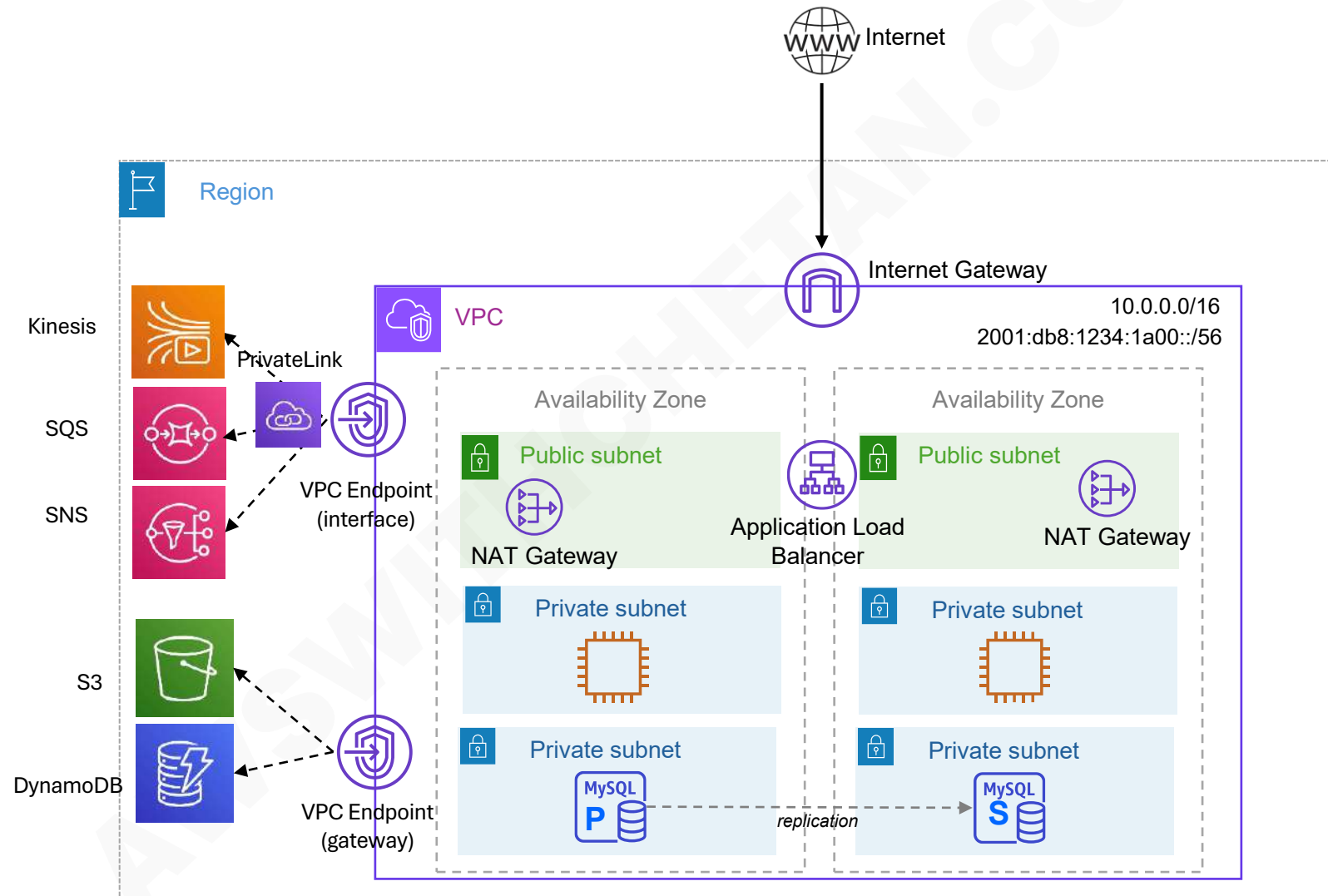
# Let's talk Architecture



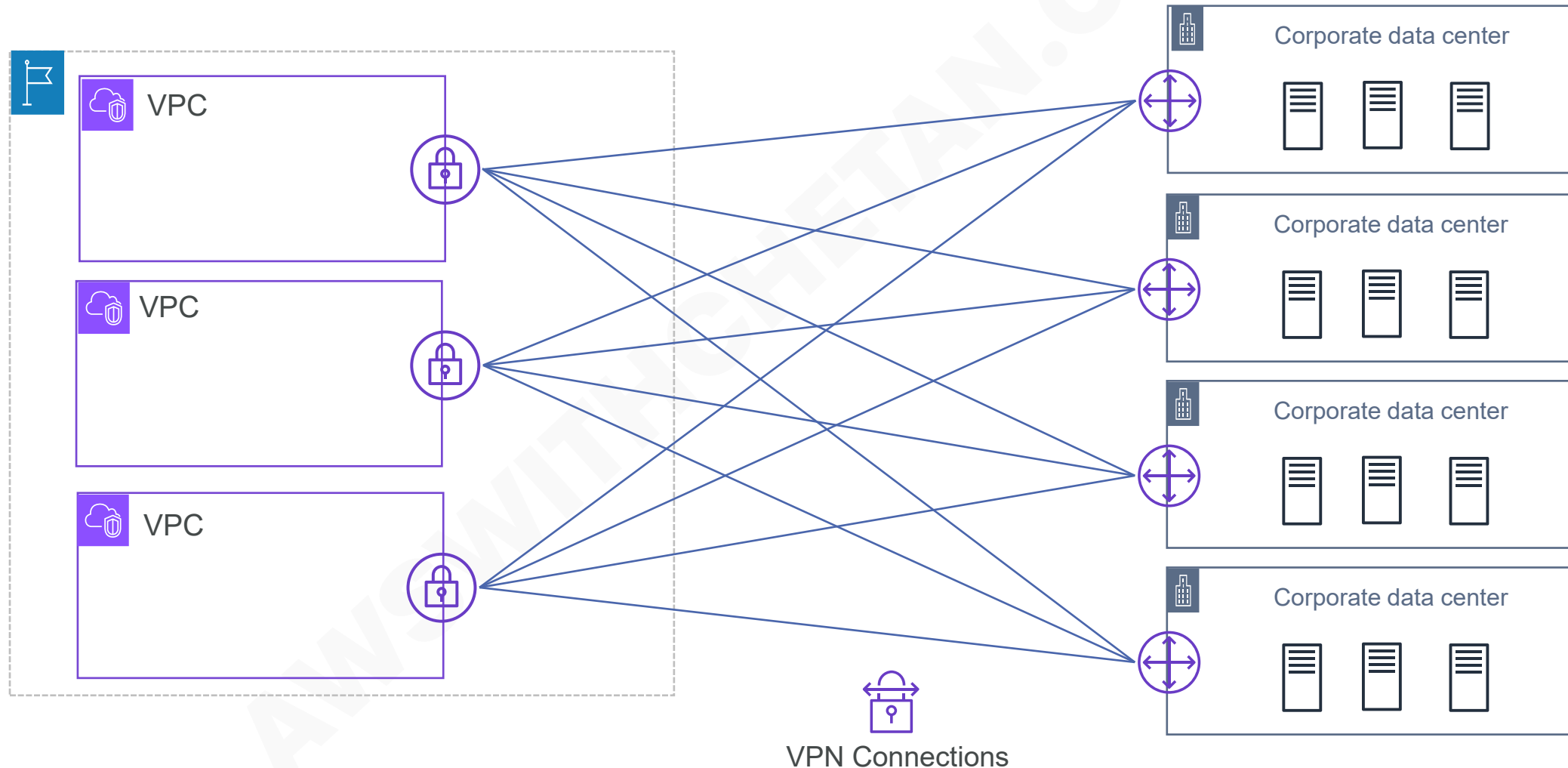
# VPC architecture for hosting simple Web Application



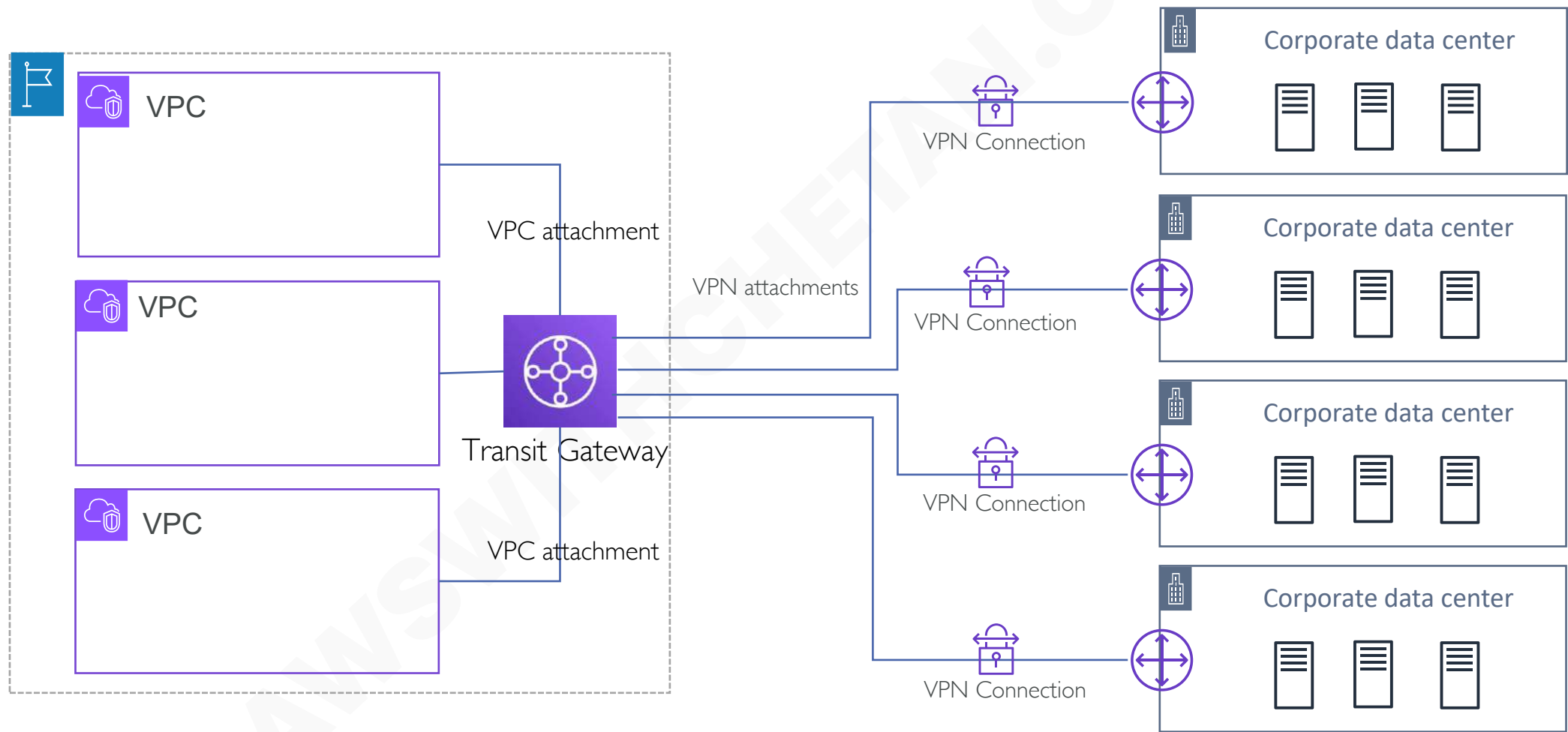
# VPC Gateway endpoint and Interface endpoints



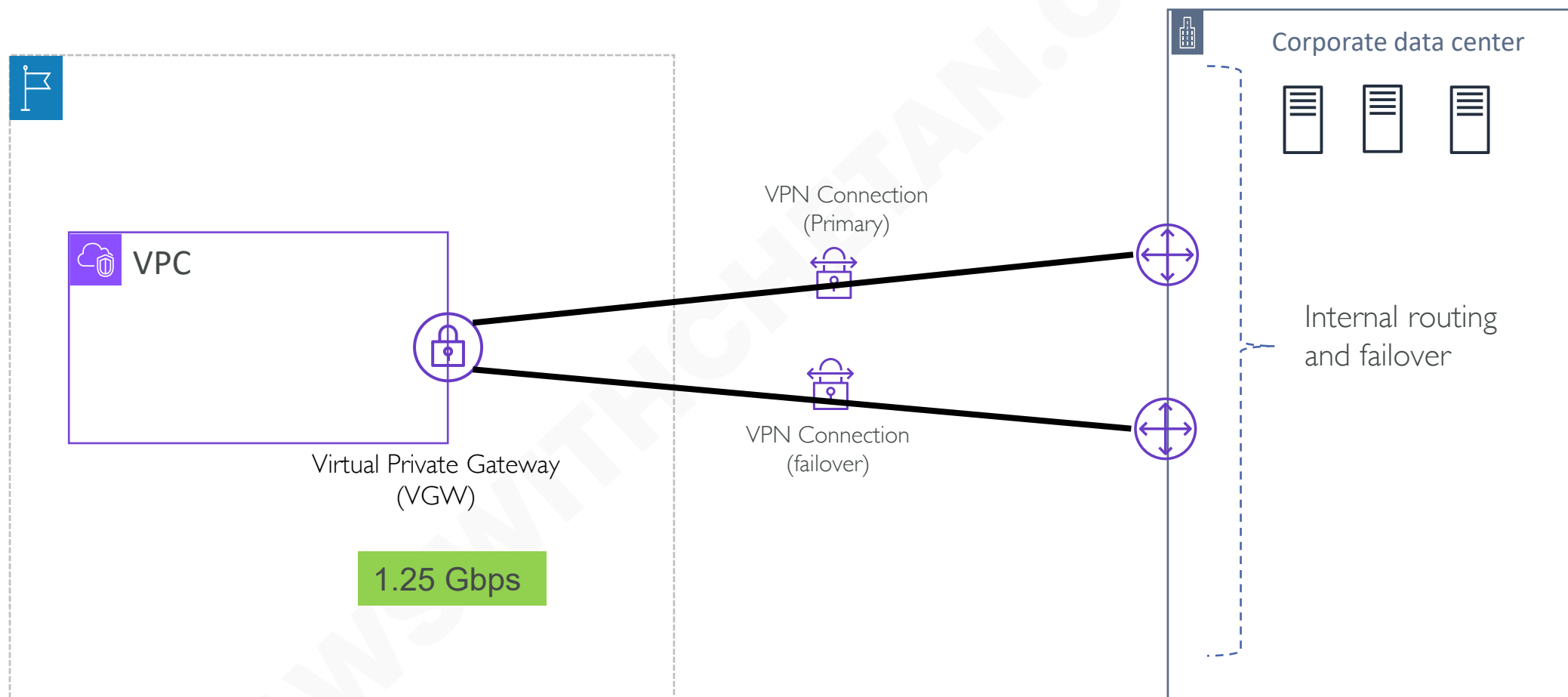
# Connecting multiple branch offices over Site-to-Site VPN



# Simplify Site-to-Site VPN network with Transit Gateway

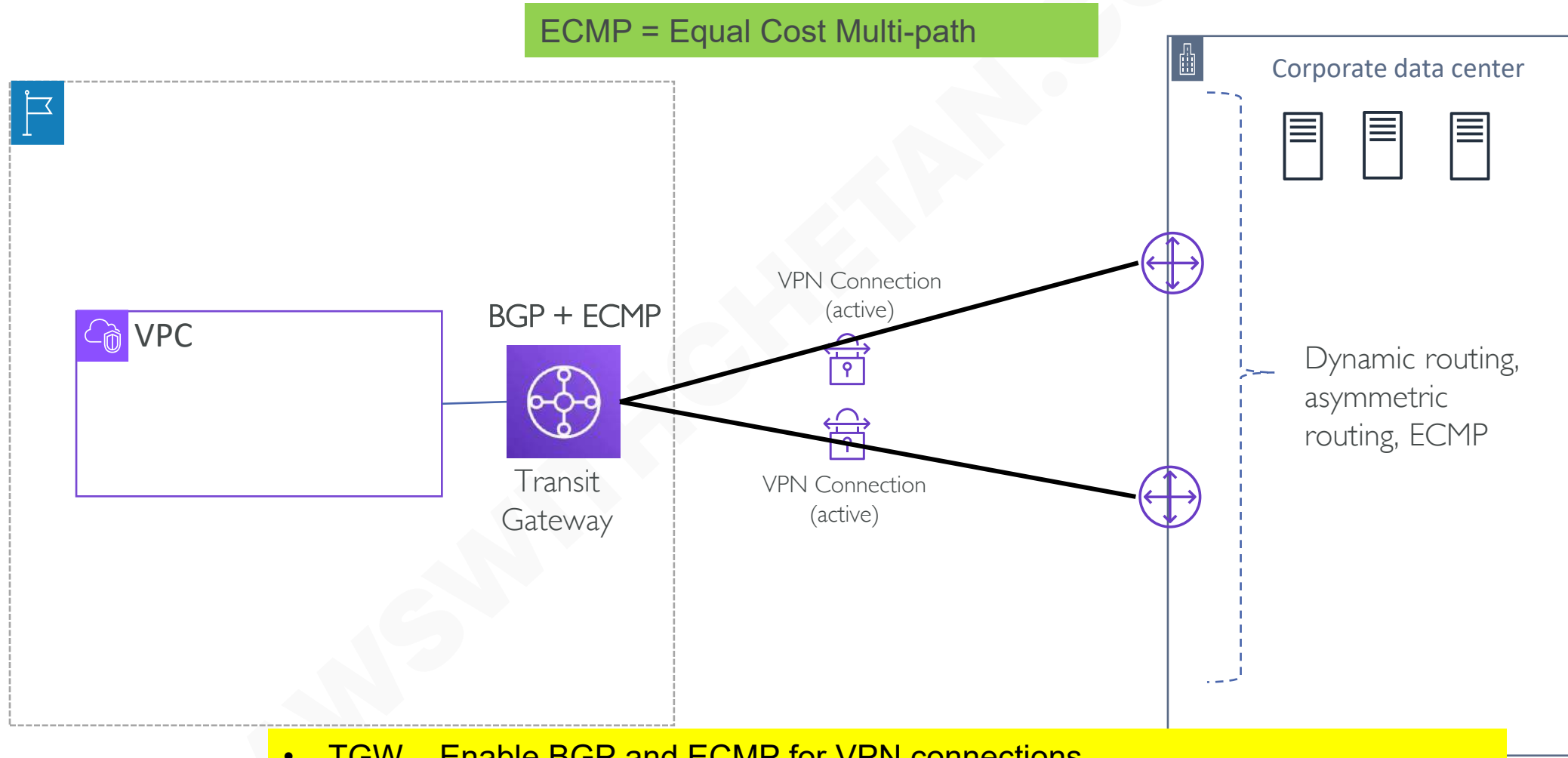


# Multiple VPN Connections over VGW – Limited bandwidth



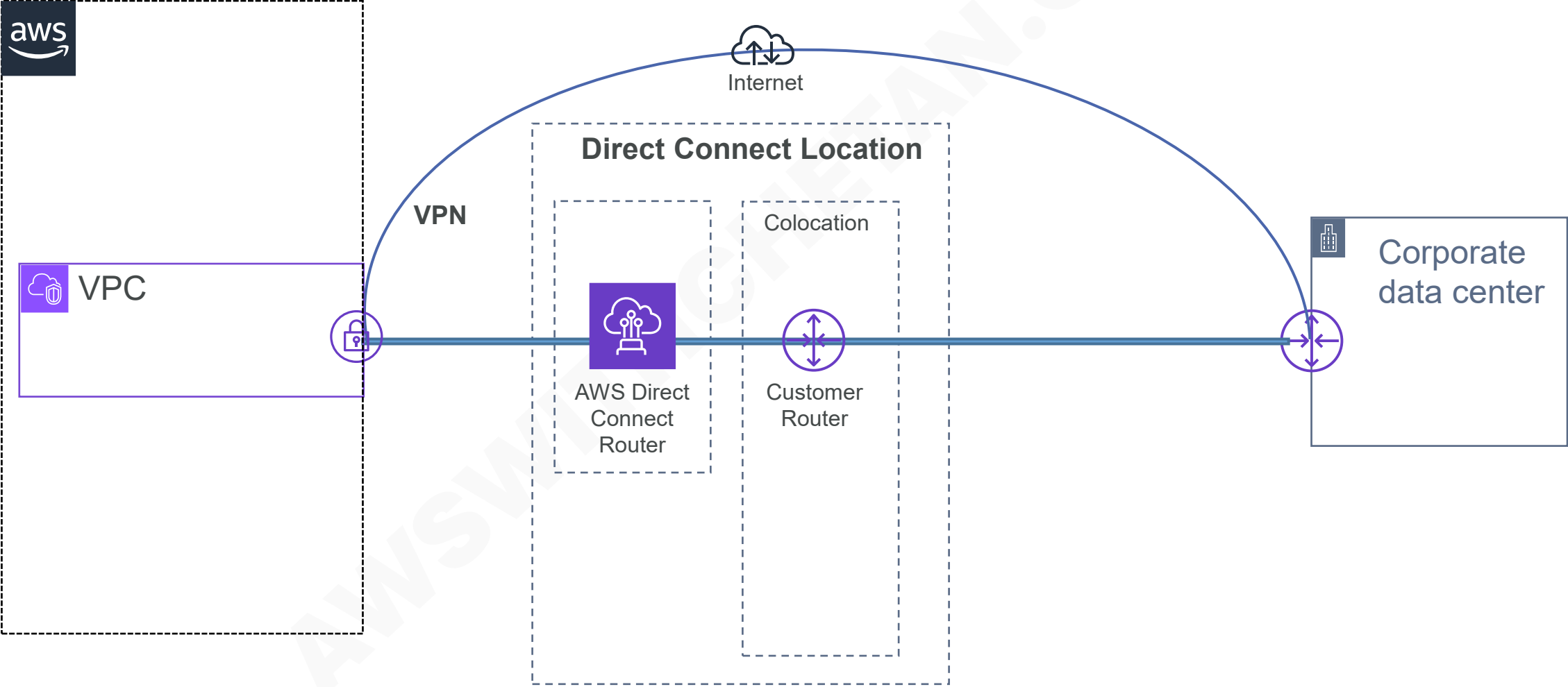
- VPN connection throughput 1.25 Gbps
- Virtual Private Gateway does not support ECMP.

# Transit Gateway & VPN - Higher aggregate bandwidth

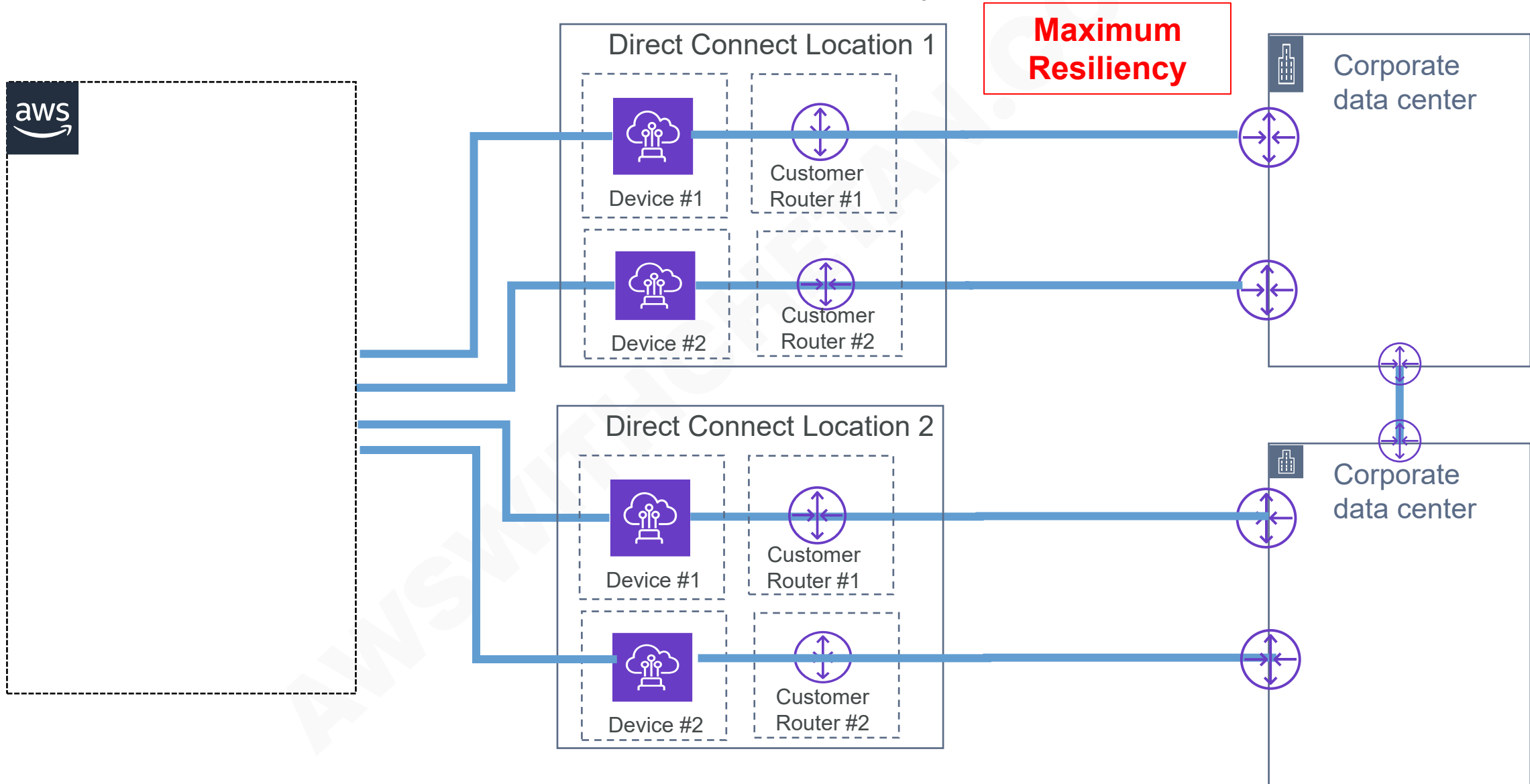


- TGW – Enable BGP and ECMP for VPN connections.
- 2 VPN connection x 2 Tunnels per connection x 1.25 Gbps/tunnel = ~5 Gbps

# Single DX Connection: VPN as a backup



# Direct Connect maximum resiliency







# Amazon S3

# AWS Storage services

## Block



Amazon EBS

EC2 instance Storage

## File



Amazon EFS

Amazon FSx for NetApp ONTAP

Amazon FSx for Windows File Server

Amazon FSx for Lustre

Amazon FSx for OpenZFS

Amazon File Cache

## Object



Amazon S3



## Hybrid Storage and Data Transfer

### Hybrid Storage



AWS Storage Gateway

### Data Transfer

Online



AWS DataSync



AWS Transfer Family

Offline



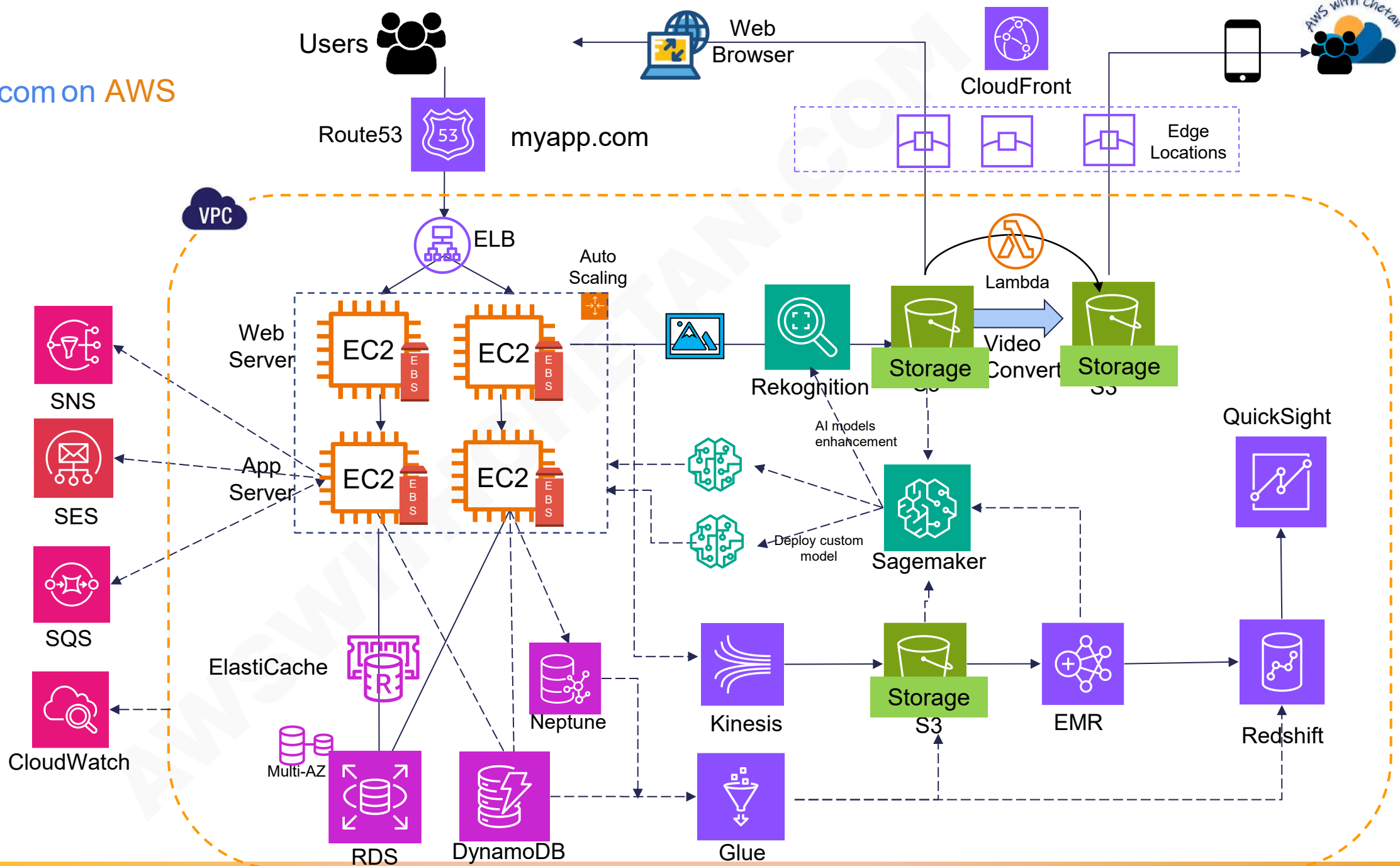
AWS Snowball

## Backup



AWS Backup

myapp.com on AWS



# Amazon S3 features

| Storage Classes                                                                                                                                                                                                                                                                                 | Security                                                                                                                                                                                              | Storage Management                                                                                                        | Event based automation                                                                                                              | Logging, Monitoring and reporting                                                                                                                           | Performance Optimization                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Different storage types based on durability, availability, access pattern, latency and cost requirements                                                                                                                                                                                        | Auditing and managing access to your buckets and objects                                                                                                                                              | To manage costs, reduce latency and meet regulatory requirements like data residency, retention etc.                      | To transform data and trigger workflows to automate a variety of data processing activities at scale                                | To help gain visibility into storage usage to better understand, analyze, and optimize storage at scale                                                     | To improve the performance of S3 storage, access, latency and more                                                                                                                                 |
| <ul style="list-style-type: none"> <li>• S3 Standard</li> <li>• S3 Standard-IA</li> <li>• S3 One Zone-IA</li> <li>• S3 Intelligent Tiering</li> <li>• Glacier Instant Retrieval</li> <li>• Glacier Flexible Retrieval</li> <li>• Glacier Deep Archive</li> <li>• S3 Express One Zone</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Block Public Access</li> <li>• S3 Bucket Policy</li> <li>• S3 Encryption</li> <li>• S3 Versioning</li> <li>• Object Lock</li> <li>• MFA Delete</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Lifecycle</li> <li>• S3 Replication</li> <li>• S3 Batch Operations</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Event Notification</li> <li>• Amazon EventBridge</li> <li>• S3 Object Lambda</li> </ul> | <ul style="list-style-type: none"> <li>• Server Access Logging</li> <li>• S3 Inventory</li> <li>• IAM Access Analyzer</li> <li>• S3 Storage Lens</li> </ul> | <ul style="list-style-type: none"> <li>• Prefix partition</li> <li>• Multi-part uploads</li> <li>• S3 Byte range</li> <li>• S3 Transfer Acceleration</li> <li>• Caching with CloudFront</li> </ul> |

# Amazon S3 – Additional features

- Static Website Hosting
- Requester Pays
- S3 Pre-signed URL
- Cross Origin Resource Sharing (CORS)
- S3 Access Points

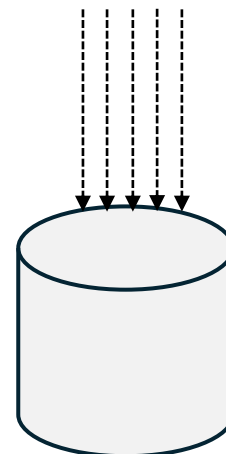
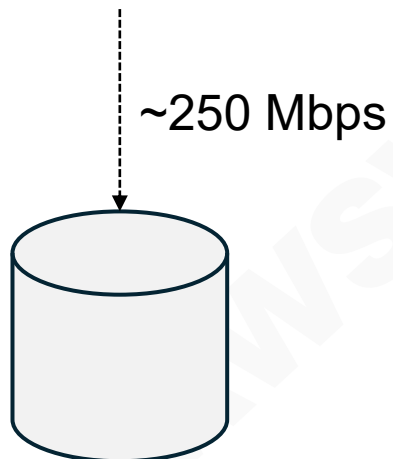
AWSWITHCHETAN.COM

# Block storage / File storage / Object Storage

# Quick look at storage performance parameters

## Storage Performance parameters – IOPS, Throughput, Latency

- What is IOPS? – Input Output operations per second e.g. 3000 IOPS
- What is Throughput? – How fast storage can read/write data e.g. 10 MB/s
- What is Latency? – Time delay between request of data and response of data



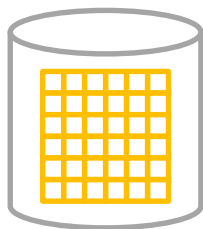
Aggregate throughput  
~10 Gbps

# AWS Storage services



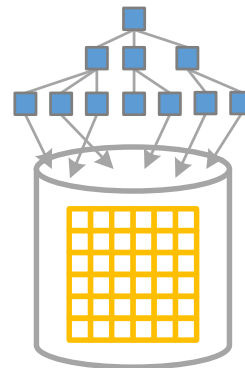
EBS

- **Block Storage** – Data is stored into unique blocks
- Host File System places data on Disk using protocols like iSCSI
- Must be attached to EC2
- High performance, high IOPS, low latency



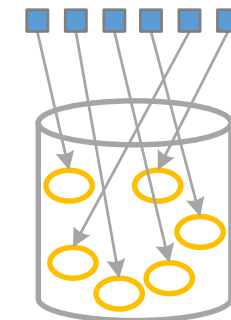
EFS/FSx

- **Shared file system** – Hierarchical structure
- There is a serving File system
- Should be mounted on EC2 or on-premises servers
- High throughput, moderate performance



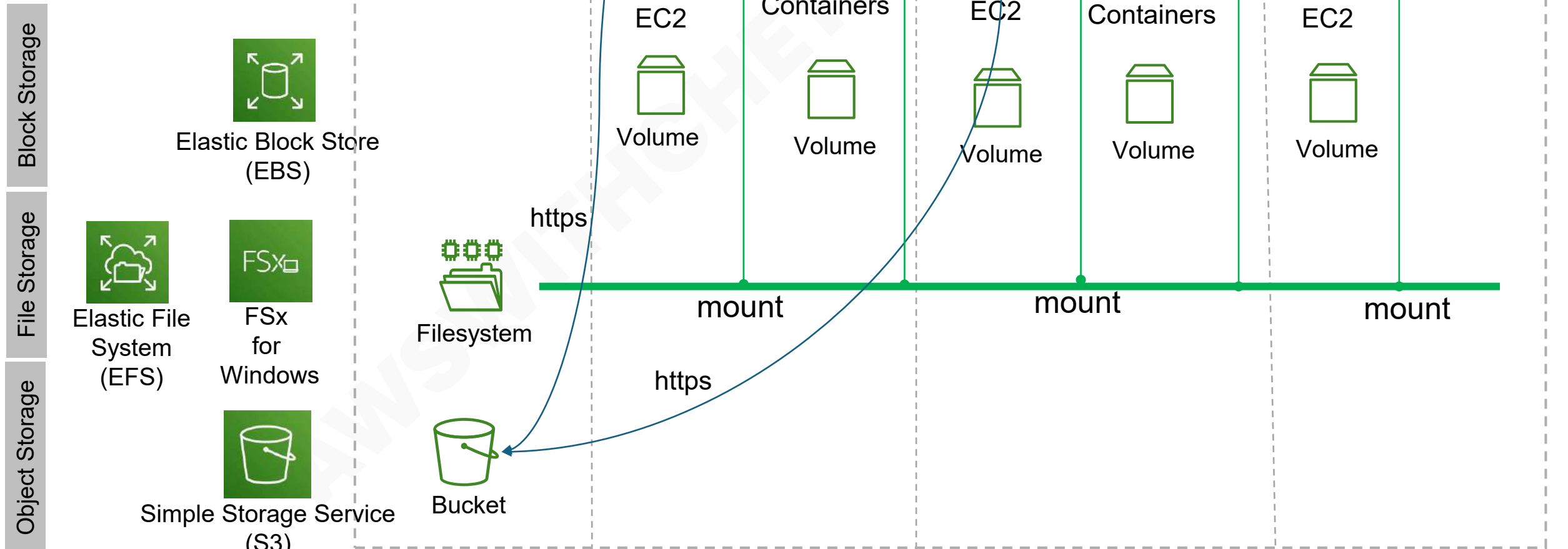
S3

- **Object storage** – Flat structure
- **API access to data (HTTPS)**
- Metadata driven (Attributes, Policy)
- High Throughput, unlimited storage, moderate performance





# How to access?



# Amazon S3

- One of the early and most powerful service launched by AWS (in 2006)
- Amazon S3 is a web-based object storage service which provides virtually unlimited space
- Designed for 99.999999999% durability.
- Customers of all sizes and industries use S3 for range of use cases.
- Ideal for wide variety of use cases including:
  - Enterprise Applications
  - Data Lake
  - Media store
  - Backup and Archive
  - Big data Analytics
  - Static websites





# S3 Bucket and Objects

# Bucket

- For storing data in Amazon S3, we need to create S3 Bucket.
- Buckets are regional - all data is stored in the AWS Region where the bucket is created.
- Bucket names must be globally unique across all AWS customers.
- By default, all buckets and objects are private - no public access unless explicitly allowed.
- Amazon S3 supports creating different types of the buckets:
  - **General Purpose bucket - Most common and widely used**
  - Directory Bucket - Designed for high performance workloads
  - Table Bucket - For tabular datasets, Optimized for Apache Iceberg table format
  - Vector Bucket - For storing vector embeddings for ML and LLM models



Bucket Name: [awswithchetan](#)

# Object

- **Key** – Uniquely identifies the object in the bucket
- **Metadata** - Set of name-value pairs that is stored along with the object.
  - Amazon S3 also assigns system-metadata to these objects, which it uses for managing objects such as creation time, size, storage class etc.
  - You can assign user-defined metadata to your objects e.g. format=json
- **Version ID** – If versioning is enabled for the bucket, Version ID uniquely identifies the specific version of an object.
- **Access control information** - Object ACL for granular access
- **Tags** - User defined key-value pairs that can be used for access control in IAM policies, cost allocation etc.
- **ETAG** – System generated hash (mostly MD5) but not always



**Bucket:** [awswithchetan](#)

**Object Key:** [section5/s3-overview.pdf](#)

**Object URI:** s3://awswithchetan/section5/s3-overview.pdf

**Object URL:** <https://awswithchetan.s3.ap-south-1.amazonaws.com/section5/section5/s3-overview.pdf>

# Object User-defined Metadata vs Tags

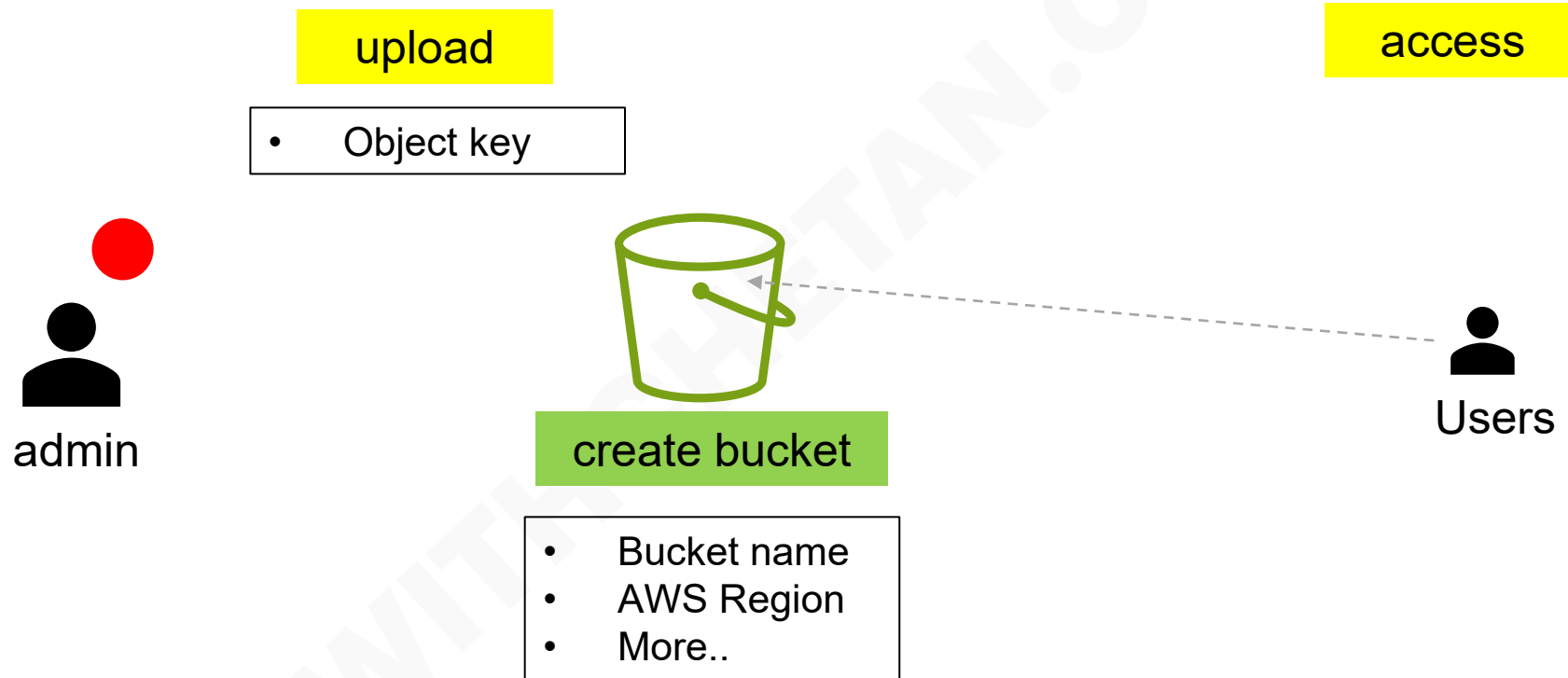
## User-defined Metadata

- Key-value pairs stored inside the object's metadata.
- Added only at the time of upload (PUT or POST).
- Cannot be modified later without re-uploading the object.
- Keys are stored as HTTP headers and sent back on every GET request.
- Used mostly by applications to store custom information.
- Examples:
  - x-amz-meta-source: camera-1
  - x-amz-meta-resolution: 1080p

## Tags

- Key-value pairs stored separately from object metadata.
- Can be added, updated, or removed anytime
- Not returned with the object payload (separate API)
- Used by AWS services for automation and governance:
  - Lifecycle transitions
  - Expiration
  - Replication
  - Object Lock retention rules
  - Access controls (IAM conditions)
  - S3 Inventory filtering
- Examples:
  - env=development
  - tier=premium

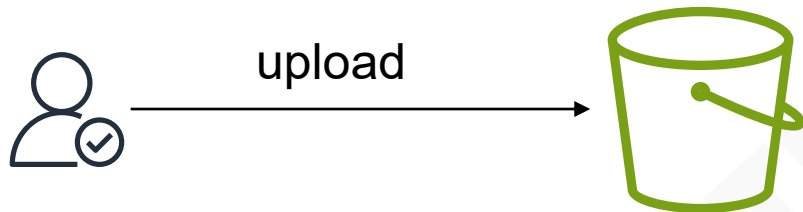
# How to use S3?



- *Admin must have permissions to create bucket and upload object*
- *Users must have permissions to read objects from this bucket*

# Exercise - Create bucket and upload file

- 1 Create a S3 bucket in region of your choice
- 2 Upload an object (image/video/pdf anything)





# Amazon S3 Features

# Amazon S3 features

| Storage Classes                                                                                                                                                                                                                                                                                 | Security                                                                                                                                                                                              | Storage Management                                                                                                        | Event based automation                                                                                                              | Logging, Monitoring and reporting                                                                                                                           | Performance Optimization                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Different storage types based on durability, availability, access pattern, latency and cost requirements                                                                                                                                                                                        | Auditing and managing access to your buckets and objects                                                                                                                                              | To manage costs, reduce latency and meet regulatory requirements like data residency, retention etc.                      | To transform data and trigger workflows to automate a variety of data processing activities at scale                                | To help gain visibility into storage usage to better understand, analyze, and optimize storage at scale                                                     | To improve the performance of S3 storage, access, latency and more                                                                                                                                 |
| <ul style="list-style-type: none"> <li>• S3 Standard</li> <li>• S3 Standard-IA</li> <li>• S3 One Zone-IA</li> <li>• S3 Intelligent Tiering</li> <li>• Glacier Instant Retrieval</li> <li>• Glacier Flexible Retrieval</li> <li>• Glacier Deep Archive</li> <li>• S3 Express One Zone</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Block Public Access</li> <li>• S3 Bucket Policy</li> <li>• S3 Versioning</li> <li>• S3 Encryption</li> <li>• Object Lock</li> <li>• MFA Delete</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Lifecycle</li> <li>• S3 Replication</li> <li>• S3 Batch Operations</li> </ul> | <ul style="list-style-type: none"> <li>• S3 Event Notification</li> <li>• Amazon EventBridge</li> <li>• S3 Object Lambda</li> </ul> | <ul style="list-style-type: none"> <li>• Server Access Logging</li> <li>• S3 Inventory</li> <li>• IAM Access Analyzer</li> <li>• S3 Storage Lens</li> </ul> | <ul style="list-style-type: none"> <li>• Prefix partition</li> <li>• Multi-part uploads</li> <li>• S3 Byte range</li> <li>• S3 Transfer Acceleration</li> <li>• Caching with CloudFront</li> </ul> |

# S3 Storage Classes

# S3 Storage Classes

## Frequently Accessed Objects

- S3 Standard (General Purpose)
- 11 9's (99.999999999%) of durability

## Infrequently Accessed Objects

- Standard-IA
- S3 One Zone-IA

## Automatically move Data across storage classes

- S3 Intelligent-Tiering

## Archiving Objects – S3 Glacier

- S3 Glacier Instant Retrieval
- S3 Glacier Flexible Retrieval
- S3 Glacier Deep Archive



S3 Standard



S3 Standard-IA



S3 One Zone-IA



S3 Intelligent-Tiering



S3 Glacier Instant Retrieval



S3 Glacier Flexible Retrieval



S3 Glacier Deep Archive

# S3 Storage Classes



S3 Standard



S3 Intelligent-Tiering



S3 Standard-IA



S3 One Zone-IA



S3 Glacier  
Instant Retrieval



S3 Glacier  
Flexible Retrieval



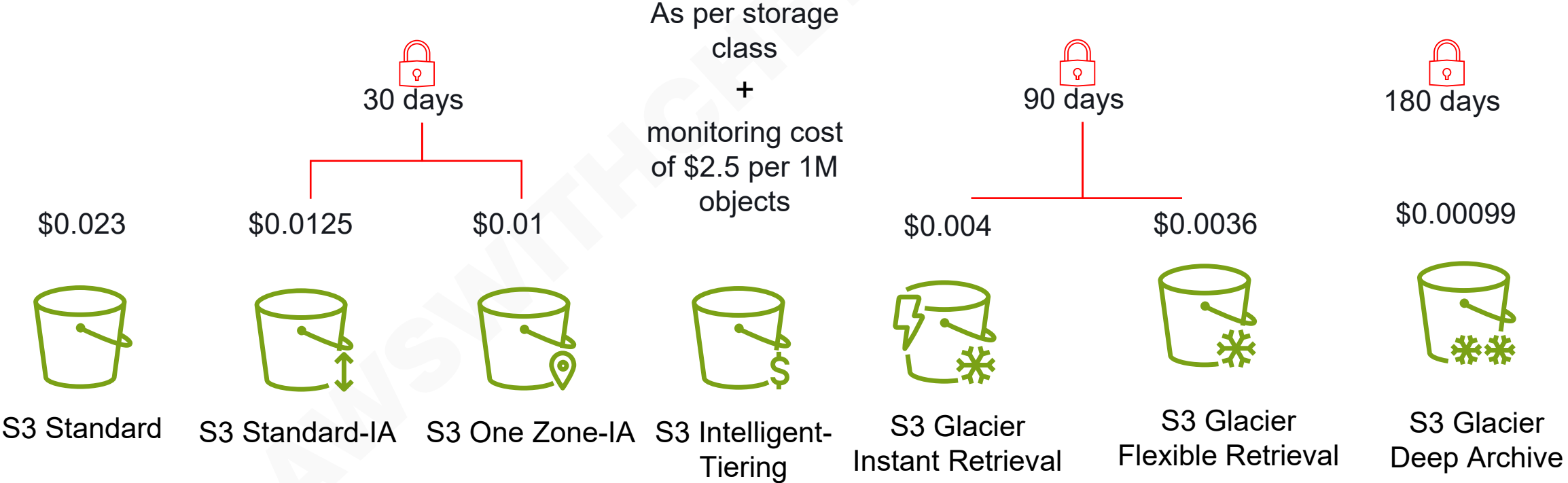
S3 Glacier  
Deep Archive

|                         |      |                            |  |                         |                    |               |      |
|-------------------------|------|----------------------------|--|-------------------------|--------------------|---------------|------|
| Access Latency          | Low  | < single-digit millisecond |  |                         | > minutes to hours | High          |      |
| Access Frequency        | High | once a quarter             |  | once or twice in a year |                    | < once a year | Low  |
| Storage cost            | High |                            |  |                         |                    |               | Low  |
| Data Retrieval cost     | Low  |                            |  |                         |                    |               | High |
| Number of requests cost | Low  |                            |  |                         |                    |               | High |

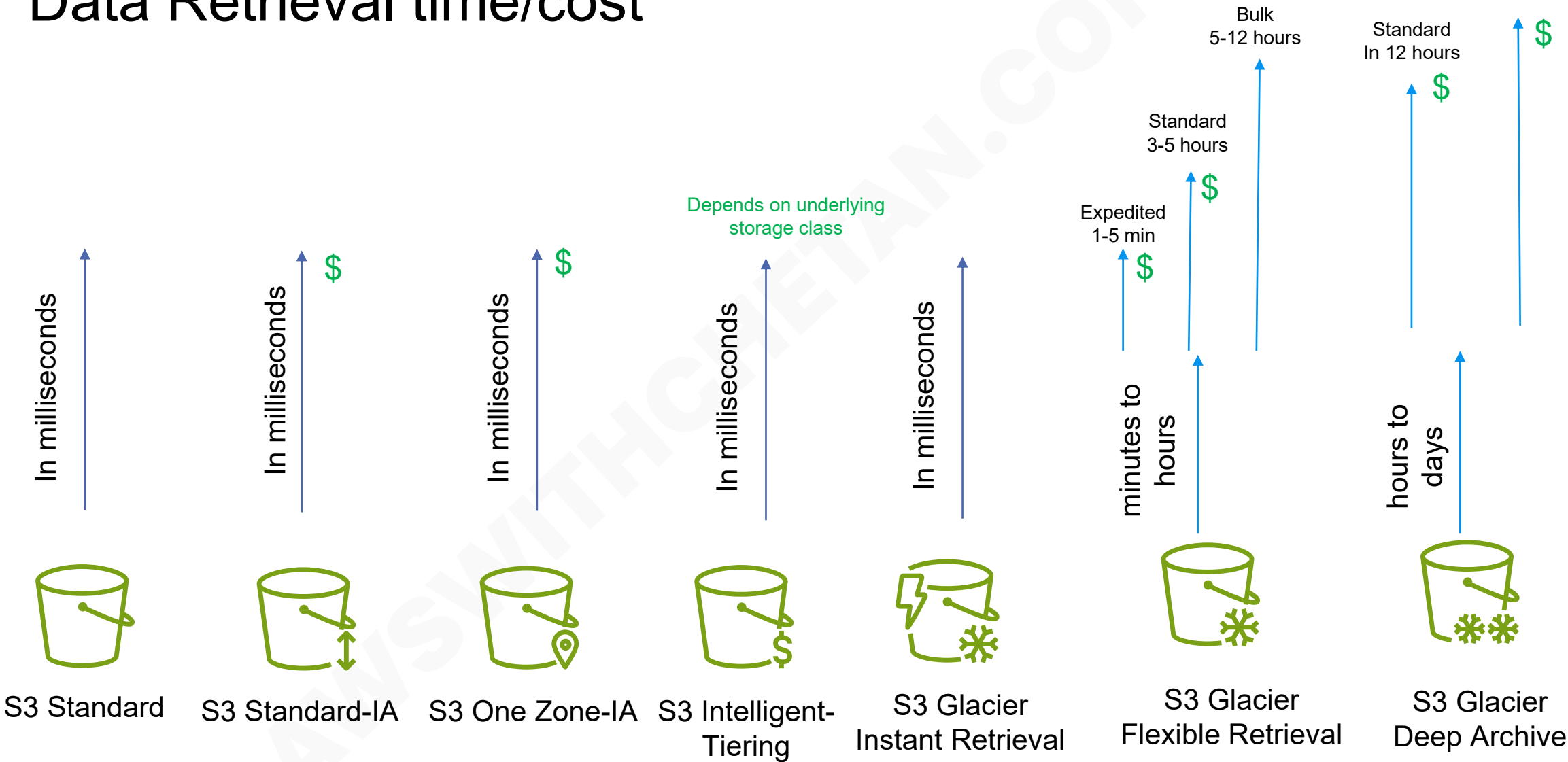
# Data storage cost

\*N. Virginia region, initial tier

Cost per GB per month



# Data Retrieval time/cost



# S3 Storage Classes



S3 Standard



S3 Intelligent-Tiering



S3 Standard-IA



S3 One Zone-IA



S3 Glacier  
Instant Retrieval



S3 Glacier  
Flexible Retrieval



S3 Glacier  
Deep Archive

|                         |      |                            |     |                         |                    |               |      |
|-------------------------|------|----------------------------|-----|-------------------------|--------------------|---------------|------|
| Access Latency          | Low  | < single-digit millisecond |     |                         | > minutes to hours | High          |      |
| Access Frequency        | High | once a quarter             |     | once or twice in a year |                    | < once a year | Low  |
| Storage cost            | High |                            |     |                         |                    |               | Low  |
| Data Retrieval cost     |      |                            | Low |                         |                    |               | High |
| Number of requests cost | Low  |                            |     |                         |                    |               | High |



# S3 Storage Classes – Use cases



## S3 Standard

- Common Applications
- Mobile and Gaming applications
- Media Content & distribution



## S3 Intelligent-Tiering

- For unknown, changing or unpredictable data access pattern
- Data lakes, data analytics



## S3 Standard-IA

- Hot backups
- Data for disaster recovery
- Older media contents, logs



## S3 One Zone-IA

- Storing secondary backup copies of on-premise data
- Storing data that you can recreate



## S3 Glacier

- Backup and Archive
- Long term storage for regulatory and compliance

# S3 Storage classes - Summary

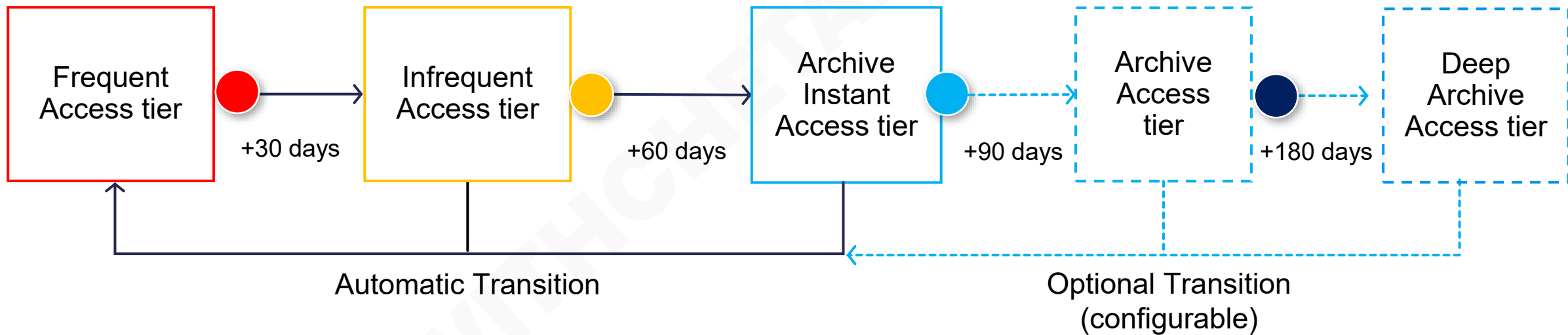
|                                 | S3 Standard                                                 | S3 Intelligent-Tiering                                                         | S3 Standard-IA                                              | S3 One Zone-IA                                                     | S3 Glacier Instant Retrieval                                                  | S3 Glacier Flexible Retrieval                                                                                     | S3 Glacier Deep Archive                                     |
|---------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Use cases                       | General purpose storage for frequently accessed data        | Automatic cost savings for data with unknown or changing access patterns       | Infrequently accessed data that needs millisecond access    | Re-creatable infrequently accessed data                            | Long-lived data that is accessed a few times per year with instant retrievals | Backup and archive data that is rarely accessed and low cost with expedited, standard, and bulk retrieval options | Archive data that is very rarely accessed and very low cost |
| First byte latency              | milliseconds                                                | milliseconds                                                                   | milliseconds                                                | milliseconds                                                       | milliseconds                                                                  | minutes or hours                                                                                                  | hours                                                       |
| Durability                      | Designed for 99.99999999% (11 nines) durability in a region | Durability depends on the underlying storage class where the objects are moved | Designed for 99.99999999% (11 nines) durability in a region | Designed for 99.99999999% (11 nines) durability within a Single AZ | Designed for 99.99999999% (11 nines) durability in a region                   | Designed for 99.99999999% (11 nines) durability in a region                                                       | Designed for 99.99999999% (11 nines) durability in a region |
| Availability                    | 99.99%                                                      | 99.90%                                                                         | 99.90%                                                      | 99.50%                                                             | 99.90%                                                                        | 99.99%                                                                                                            | 99.99%                                                      |
| Availability SLA                | 99.90%                                                      | 99%                                                                            | 99%                                                         | 99%                                                                | 99%                                                                           | 99.90%                                                                                                            | 99.90%                                                      |
| Availability Zones              | ≥3                                                          | ≥3                                                                             | ≥3                                                          | 1                                                                  | ≥3                                                                            | ≥3                                                                                                                | ≥3                                                          |
| Minimum storage duration charge | N/A                                                         | N/A                                                                            | 30 days                                                     | 30 days                                                            | 90 days                                                                       | 90 days                                                                                                           | 180 days                                                    |
| Retrieval charge                | N/A                                                         | N/A                                                                            | per GB retrieved                                            | per GB retrieved                                                   | per GB retrieved                                                              | per GB retrieved                                                                                                  | per GB retrieved                                            |

<https://aws.amazon.com/s3/storage-classes/>

# S3 Intelligent Tiering

# Amazon S3 Intelligent-Tiering

For data with unknown or changing access patterns



Moves objects between three access tiers for a small monthly monitoring and automation fee

# S3 Express One Zone

- Provides High-performance with single-digit millisecond data access at scale
- Single-Availability Zone storage class where you can select the specific AZ in which bucket should be created.
- Designed specifically to work with S3 **Directory Buckets** (a new bucket type optimized for low-latency, high-request-rate operations).
- Delivers consistent data access with over 2 million requests per second
- Data access speed up to 10x faster and request costs up to 80% lower than S3 Standard
- You can choose to co-locate your storage and compute resources (EC2, EKS etc.) in the same Availability Zone
- Ideal for low latency storage access requirement applications like ML and AI model training



# Amazon S3 Security

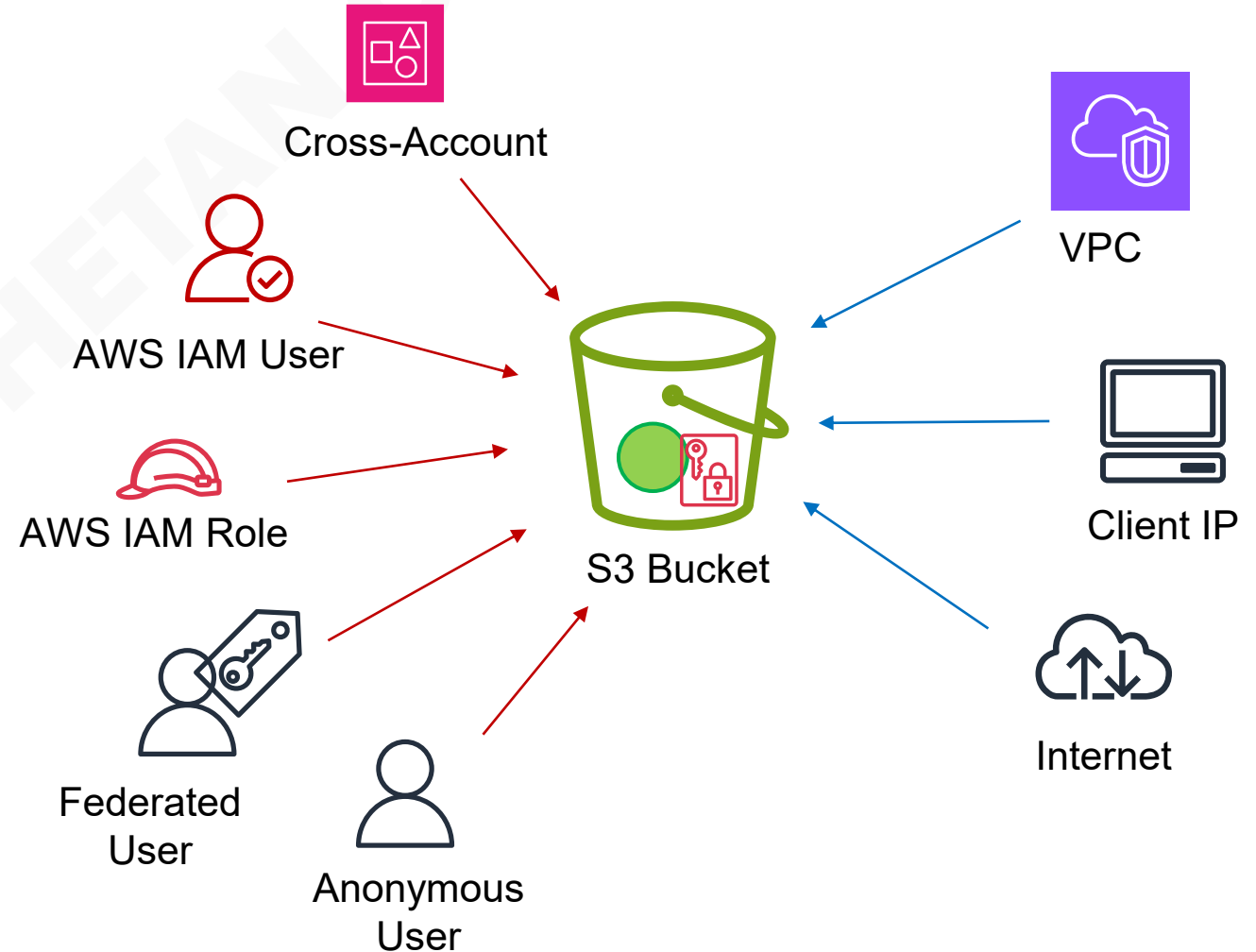
# S3 Security

## Access or Permission layer Security

- Block Public Access
- Access Control List (ACL)
- Bucket Policy
  - Restrict access to IAM user/role
  - Cross-account access
  - Restrict access from VPC or IP address

## Data layer Security

- Encryption
- Object Lock
- Data deletion protection (MFA)
- S3 Versioning



# S3 Block Public Access

- To prevent company data leaks
- Set at the account level or bucket level
- Best practice: Leave these ON unless bucket needs to be publicly accessible

## ☒ Block *all* public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

### ☒ Block public access to buckets and objects granted through *new* access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for permissions that allow public access to S3 resources using ACLs.

### ☒ Block public access to buckets and objects granted through *any* access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

### ☒ Block public access to buckets and objects granted through *new* public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existir

### ☒ Block public and cross-account access to buckets and objects through *any* public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.



# Access Control List (ACL)

- Each bucket and object has an ACL attached to it
- ACL controls which AWS accounts or groups are granted access and the type of access
- Bucket owner can enable/disable ACLs
- ACLs are disabled by default



Bucket Level ACL



Object Level ACL

### Access control list (ACL)

Grant basic read/write permissions to other AWS accounts. [Learn more](#)

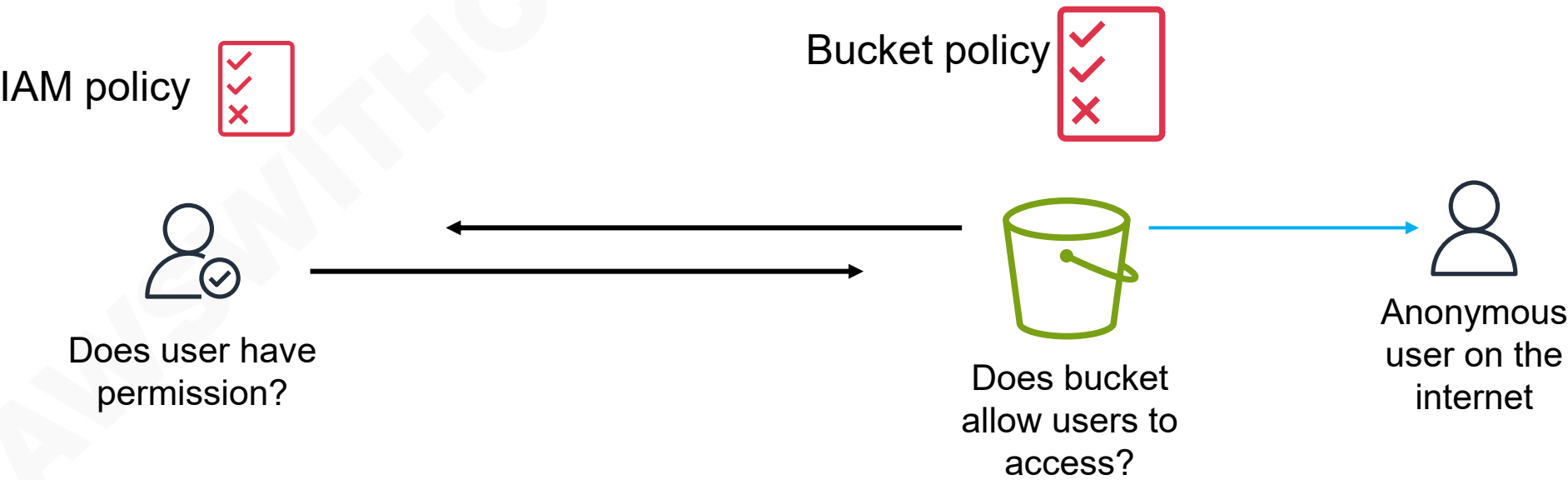
| Grantee                                                                                                                                                                                                      | Objects                                                                               | Bucket ACL                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Bucket owner (your AWS account)</b><br>Canonical ID:<br><div style="background-color: black; width: 100px; height: 1.2em;"></div>                                                                         | <input checked="" type="checkbox"/> List<br><input checked="" type="checkbox"/> Write | <input checked="" type="checkbox"/> Read<br><input checked="" type="checkbox"/> Write |
| <b>Everyone (public access)</b><br>Group:<br><a href="http://acs.amazonaws.com/groups/global/AllUsers">http://acs.amazonaws.com/groups/global/AllUsers</a>                                                   | <input type="checkbox"/> List<br><input type="checkbox"/> Write                       | <input type="checkbox"/> Read<br><input type="checkbox"/> Write                       |
| <b>Authenticated users group (anyone with an AWS account)</b><br>Group:<br><a href="http://acs.amazonaws.com/groups/global/AuthenticatedUsers">http://acs.amazonaws.com/groups/global/AuthenticatedUsers</a> | <input type="checkbox"/> List<br><input type="checkbox"/> Write                       | <input type="checkbox"/> Read<br><input type="checkbox"/> Write                       |
| <b>S3 log delivery group</b><br>Group:<br><a href="http://acs.amazonaws.com/groups/s3/LogDelivery">http://acs.amazonaws.com/groups/s3/LogDelivery</a>                                                        | <input type="checkbox"/> List<br><input type="checkbox"/> Write                       | <input type="checkbox"/> Read<br><input type="checkbox"/> Write                       |

A majority of modern use cases in Amazon S3 **no longer** require the use of ACLs.

# Bucket Policy

## IAM User or Role policy

## S3 Bucket Policy



# Bucket Policy

It's a JSON-based policy

- **Resources:** buckets and objects
- **Effect:** Allow / Deny
- **Actions:** Set of API to Allow or Deny
- **Principal:** The account or user to apply the policy to
- **Condition:** Apply policy when condition is true

Examples of bucket policy:

- Grant public read access to the bucket e.g. website
- Objects must be encrypted at upload
- Cross Account access to bucket and objects

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Principal": {
      "AWS": [
        "arn:aws:iam::111122223333:user/JohnDoe"
      ]
    },
    "Effect": "Allow",
    "Action": [
      "s3:PutObject"
    ],
    "Resource": [
      "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*"
    ],
    "Condition": {
      "StringEquals": {
        "s3:RequestObjectTag/Department": "Finance"
      }
    }
  }]
}
```

# Example: How to make S3 bucket public?

1. Block Public Access should be disabled

## ☐ Block *all* public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

### ☐ Block public access to buckets and objects granted through *new* access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for permissions that allow public access to S3 resources using ACLs.

### ☐ Block public access to buckets and objects granted through *any* access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

### ☐ Block public access to buckets and objects granted through *new* public bucket or access point policies

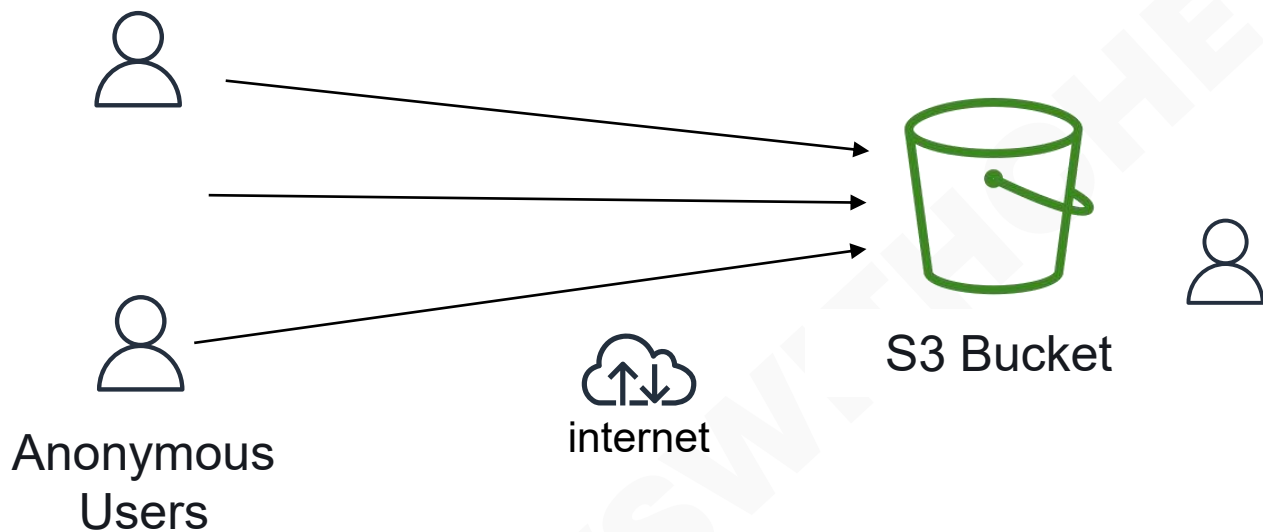
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies.

### ☐ Block public and cross-account access to buckets and objects through *any* public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

# Exercise: Public access to bucket

1. Block Public Access should be disabled
2. Bucket policy to allow access

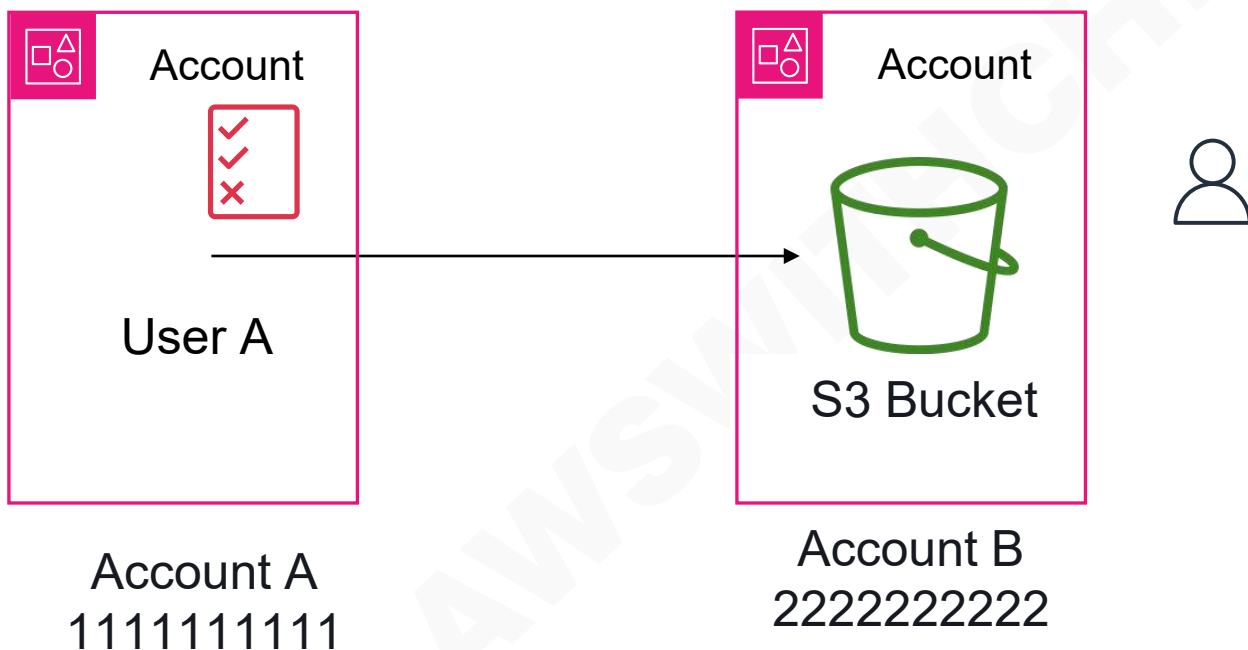


## Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::EXAMPLE-BUCKET/*"
    }
  ]
}
```

# Exercise: S3 bucket cross-account access

1. Bucket policy in Account B to allow access to Account A
2. IAM user/role in Account A to allow UserA to access Bucket in Account B

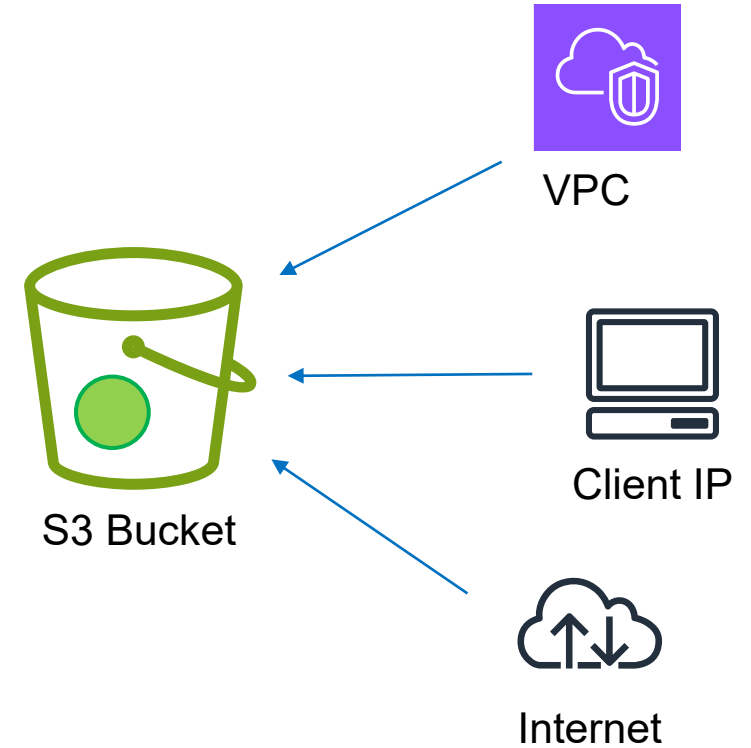


## Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "
{ \"AWS\": \"arn:aws:iam::1111111111:root\" }",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::EXAMPLE-
BUCKET/*"
    }
  ]
}
```

# Bucket Policy

- Restrict access from specific VPC
- Restrict access from specific VPC endpoint
- Restrict access from VPC source IP ranges
- Restrict access from specific external IP



# Bucket Policy

- **Restrict access from specific VPC**
- Restrict access from specific VPC endpoint
- Restrict access from VPC source IP ranges
- Restrict access from specific external IP

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PutObjectIfNotVPCID",
      "Effect": "Deny",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::amzn-s3-demo-bucket3/*",
      "Condition": {
        "StringNotEqualsIfExists": {
          "aws:SourceVpc": "vpc-1234567890abcdef0"
        },
        "Bool": {
          "aws:ViaAWSService": "false"
        }
      }
    }
  ]
}
```



# Bucket Policy

- Restrict access from specific VPC
- **Restrict access from specific VPC endpoint**
- Restrict access from VPC source IP ranges
- Restrict access from specific external IP

```
{
  "Id": "VPCE",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VPCE",
      "Action": "s3:*",
      "Effect": "Deny",
      "Resource": [
        "arn:aws:s3:::DOC-EXAMPLE-BUCKET",
        "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*"
      ],
      "Condition": {
        "StringNotEquals": {
          "aws:SourceVpce": [
            "vpce-1111111",
            "vpce-2222222"
          ]
        }
      }
    }
  ],
  "Principal": "*"
}
```

# Bucket Policy

- Restrict access from specific VPC
- Restrict access from specific VPC endpoint
- **Restrict access from VPC source IP ranges**
- Restrict access from specific external IP

```
{
  "Id": "VpcSourceIp",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VpcSourceIp",
      "Action": "s3:*",
      "Effect": "Deny",
      "Resource": [
        "arn:aws:s3:::DOC-EXAMPLE-BUCKET",
        "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*"
      ],
      "Condition": {
        "NotIpAddress": {
          "aws:VpcSourceIp": [
            "10.1.1.1/32",
            "172.1.1.1/32"
          ]
        }
      }
    }
  ],
  "Principal": "*"
}
```

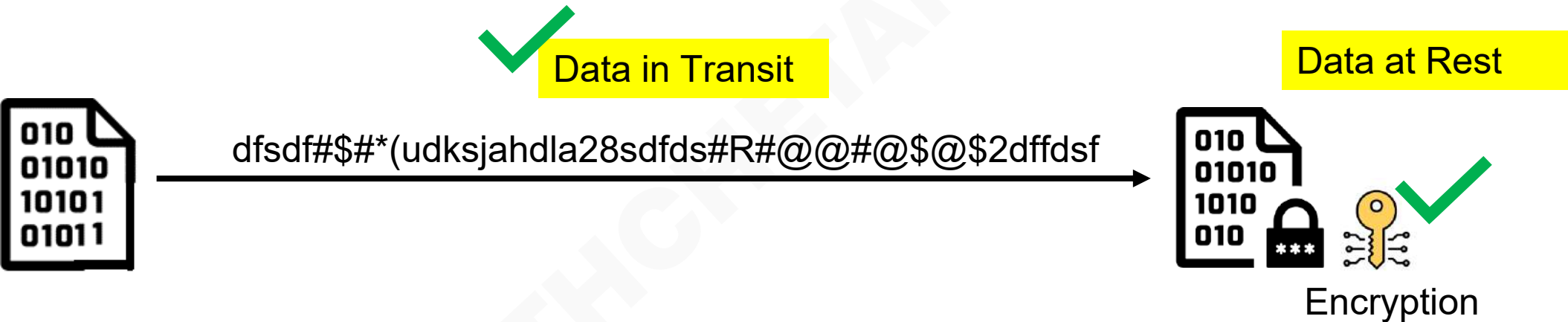
# Bucket Policy

- Restrict access from specific VPC
- Restrict access from specific VPC endpoint
- Restrict access from VPC source IP ranges
- **Restrict access from specific external IP**

```
{
  "Id": "SourceIP",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SourceIP",
      "Action": "s3:*",
      "Effect": "Deny",
      "Resource": [
        "arn:aws:s3:::DOC-EXAMPLE-BUCKET",
        "arn:aws:s3:::DOC-EXAMPLE-BUCKET/*"
      ],
      "Condition": {
        "NotIpAddress": {
          "aws:SourceIp": [
            "11.11.11.11/32",
            "22.22.22.22/32"
          ]
        }
      },
      "Principal": "*"
    }
  ]
}
```

# S3 Data Security

# Data Security – Encryption in transit and rest



# HTTPS and SSL/TLS

Certificate  
Authority (CA)



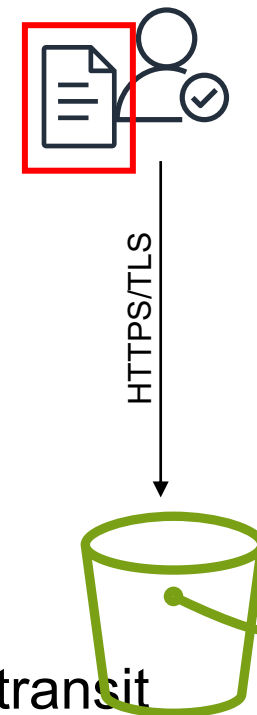
HTTP**S** Traffic



<https://youtu.be/cLYv4uSFJA8>

# Encrypting data in transit – HTTPS (SSL/TLS)

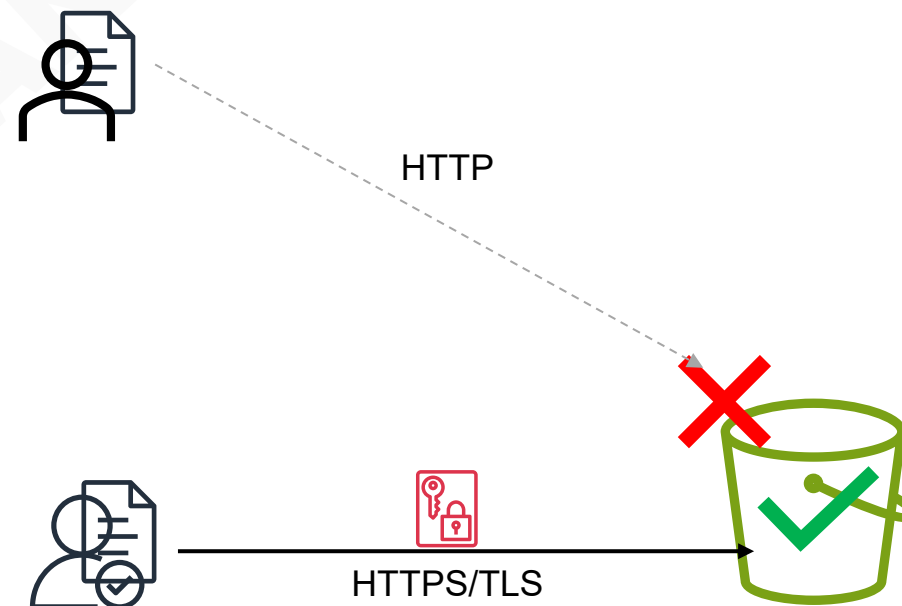
- Amazon S3 provides endpoints
  - IPv4: s3.us-east-1.amazonaws.com
  - IPv4 and IPv6: s3.dualstack.us-east-1.amazonaws.com
  - FIPS IPv4: s3-fips.us-east-1.amazonaws.com
  - FIPS IPv4 and IPv6: s3-fips.dualstack.us-east-1.amazonaws.com
- It supports both HTTP and HTTPS transport layer protocols
- Recommended to use HTTPS as it uses TLS encryption for the data in transit



# Enforce HTTPS (SSL/TLS)

## Bucket Policy

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "RestrictToTLSRequestsOnly",  
    "Action": "s3:*",  
    "Effect": "Deny",  
    "Resource": [  
      "arn:aws:s3:::amzn-s3-demo-bucket",  
      "arn:aws:s3:::amzn-s3-demo-bucket/*"  
    ],  
    "Condition": {  
      "Bool": {  
        "aws:SecureTransport": "false"  
      }  
    },  
    "Principal": "*"   
  }]  
}
```





# Securing data at Rest - Encryption

## Server-side encryption

- S3 service encrypts object while saving it on disk and decrypts it when you access the object
- Supports different Encryption keys SSE-S3, SSE-KMS, SSE-C

## Client-side encryption

- Data is encrypted at the client side and then uploaded to S3 in the encrypted format.
- S3 doesn't know about the encryption and can not decrypt objects



HTTPS/TLS



Server-side encryption



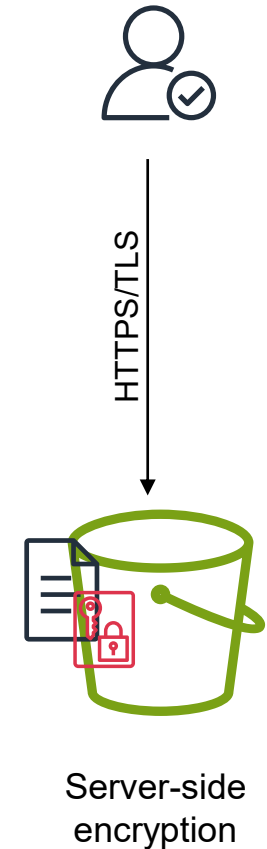
HTTPS/TLS



Client-side encryption

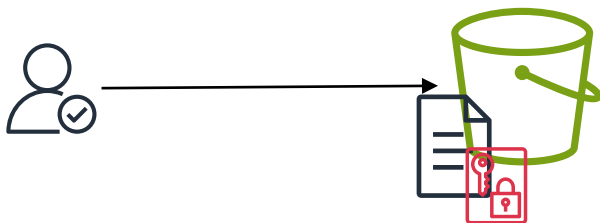
# Server-side encryption

- **SSE-S3** (Server-Side Encryption with Amazon S3-Managed Keys)
  - Enabled by default
  - Encryption keys are owned and managed by AWS
- **SSE-KMS** (Server-Side Encryption with KMS Keys)
  - AWS Key Management Store (KMS) to store the keys
- **SSE-C** (Server-Side Encryption with Customer-Provided Keys)
  - Customer owns and manage the encryption keys



# Server-side encryption

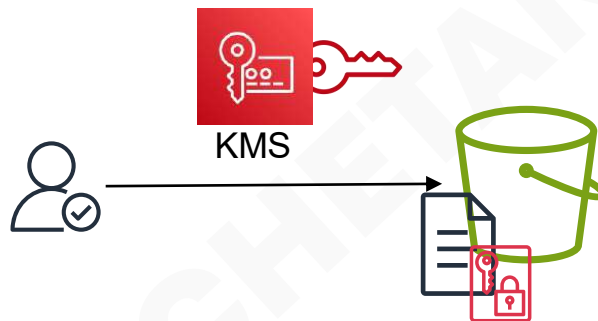
## SSE-S3



- Encryption Keys managed by AWS S3 service
- Enabled by default when you create new bucket or objects
- Should have request header:

"x-amz-server-side-encryption": "AES256"

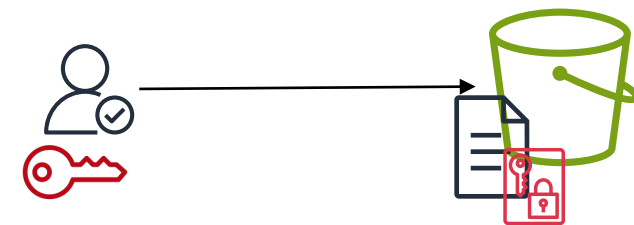
## SSE-KMS



- Encryption Keys managed by AWS KMS
- Useful if encryption key usage needs to be audited by using CloudTrail logs
- Should have request header:

"x-amz-server-side-encryption": "aws:kms"

## SSE-C



- Encryption Keys managed by the Customer (outside of AWS)
- HTTPS must be enabled
- Encryption key should be sent in the HTTP header:

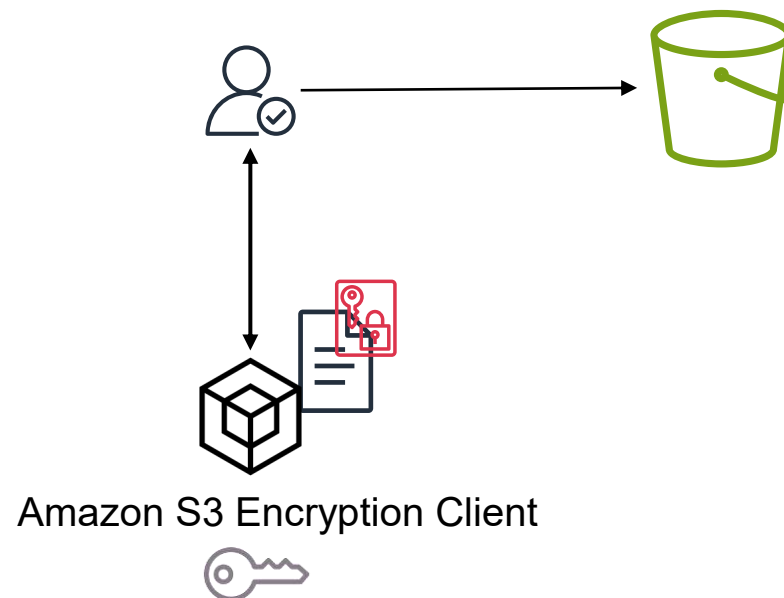
"x-amz-server-side-encryption-customer-algorithm": "AES256"

"x-amz-server-side-encryption-customer-key":  
:Base64,256bit encryption key

# Client-side encryption

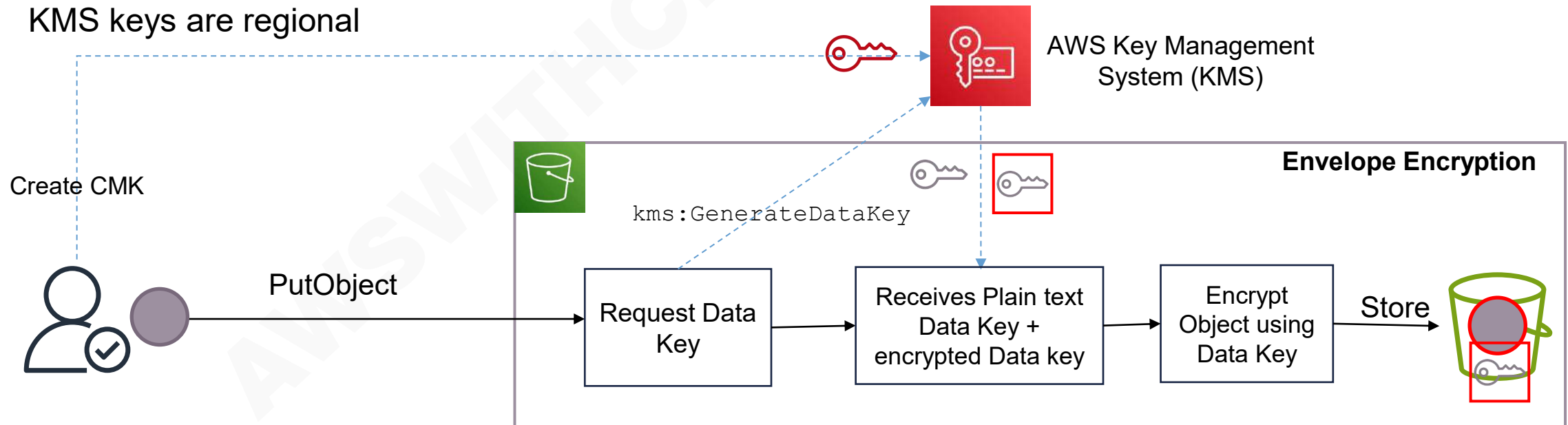
- Client must encrypt the objects locally before sending it to S3
- Client must also decrypt the objects locally after retrieving it from S3
- Amazon S3 has no role in encryption or decryption
- Use **Amazon S3 Encryption Client** to handle the client-side encryption
- You can additionally enable S3 Server-Side Encryption

## Client-side Encryption



# SSE-KMS – important to know

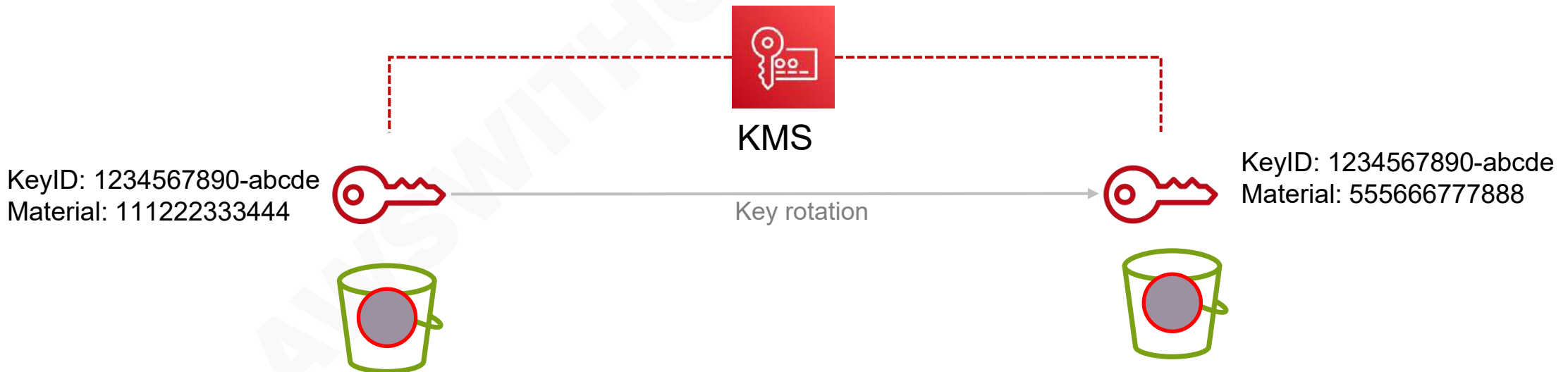
- S3 Uses Envelope encryption for SSE-S3, SSE-KMS and SSE-C
- When using SSE-KMS, user must have IAM permissions
  - For PutObject: kms:GenerateDataKey
  - For GetObject: kms:Decrypt, kms:DescribeKey
- If you get "Access Denied (KMS)" when reading/writing S3 objects, the cause is usually missing `kms:Decrypt` or `kms:GenerateDataKey` in the KMS key policy.
- KMS keys are regional



# SSE-KMS – important to know

## KMS key rotation:

- It's a good practice to enable the automatic KMS key rotation (default 1 year if ON)
- KMS key rotation creates new key material but Key ID, ARN etc. remains same. It also keeps older key material (version), so existing S3 objects remain decryptable.
- New uploads use the latest key version automatically, ensuring seamless rotation with no impact on applications.



# Enforcing encryption using Bucket policy

Must use SSE-S3 encryption

## Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyWhenNoSSES3Encryption",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    }
  ]
}
```

# Enforcing encryption using Bucket policy

Must use SSE-KMS encryption

## Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyWhenNoSSEKMS",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "aws:kms"
        }
      }
    }
  ]
}
```



# Enforcing encryption using Bucket policy

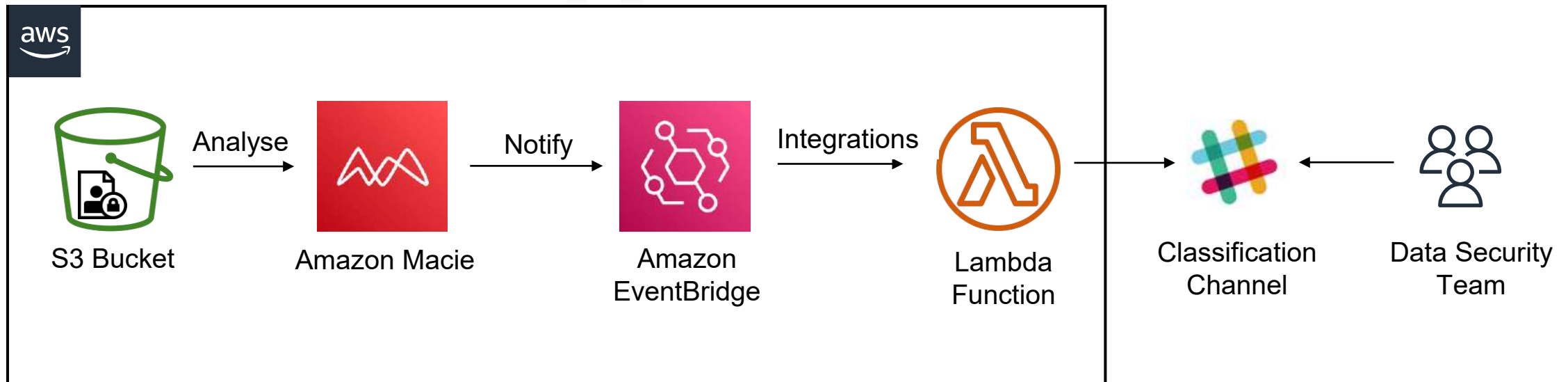
Must use specific SSE-KMS Key

Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyWhenNoSSEKMSKey",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*",
      "Condition": {
        "ArnNotEqualsIfExists": {
          "s3:x-amz-server-side-encryption-aws-kms-key-id": "arn:aws:kms:us-east-1:111122223333:key/01234567-89ab-cdef-0123-45abc"
        }
      }
    }
  ]
}
```

# Amazon Macie

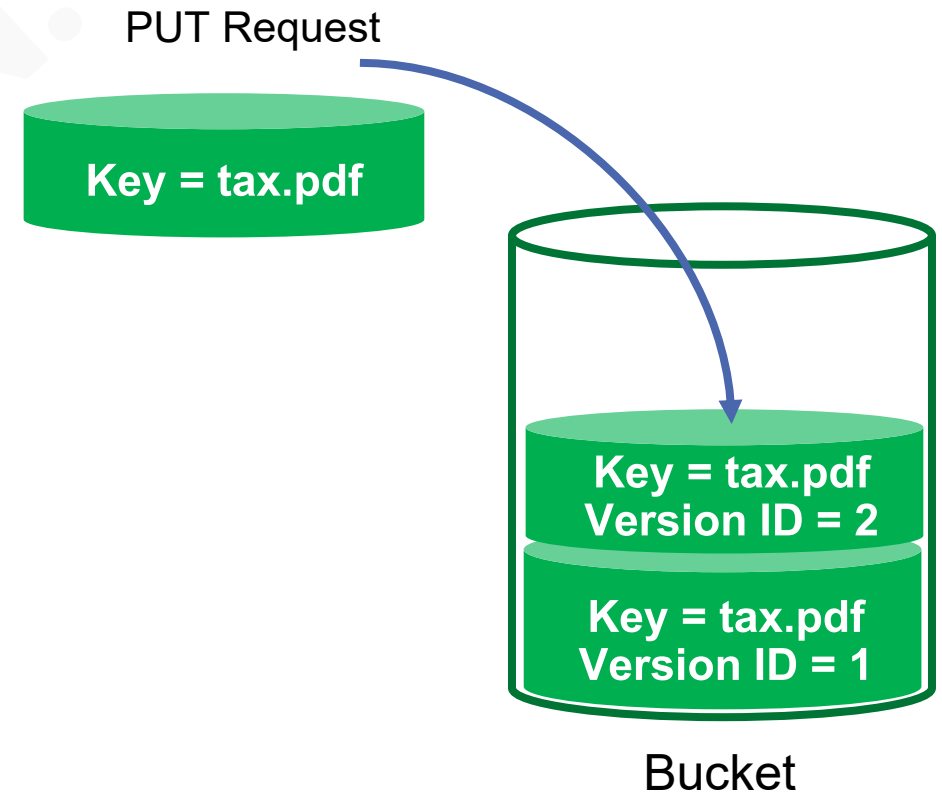
- Fully managed S3 data security service
- Can discover **P**ersonally **I**dentifiable **I**nformation (**PII**) data
- Uses machine learning and pattern matching to discover, monitor, and protect your data in S3
- Triggers alert when sensitive data is identified



# S3 Versioning

# Amazon S3 bucket versioning

- Creates a new version with every upload of the object.
- Previous versions are **not overwritten**.
- Should specify version ID for deleting the object version.
- Delete requests without a version ID removes access to objects (error 404) but keeps the data. There is a **delete marker** added as a new version.
- You can transition or expire current and non-current versions with S3 lifecycle rules
- There will be cost for storing all the versions.
- Must enable bucket versioning for using Bucket replication, Object lock and MFA delete features



Use versioning to protect your data from accidental deletion or for recovering / rollback to the older versions of the objects

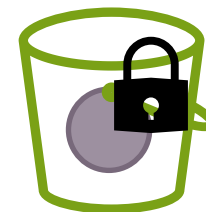
# Exercise: Enable S3 Versioning

- 1 Create S3 Bucket.
- 2 Go to Bucket -> Properties -> Versioning -> Enable
- 3 Upload sample text file to S3 Bucket.
- 4 Upload modified text file with same name multiple times by doing small changes every time.
- 5 Check Versions. On S3 Console -> Bucket -> Versions -> Show. How many versions do you see?
- 6 Try deleting a particular Version of the uploaded file.
- 7 Click Hide Version. S3 Console -> Bucket -> Versions -> Hide Versions (You should see only one file)
- 8 Delete a File. Deleted?
- 9 Click Show Version. S3 console -> Versions -> Show versions. Restore file using desired version.

# Object Lock

- S3 Object Lock helps prevent S3 objects from being deleted or overwritten for a fixed amount of time or indefinitely
- Useful to meet WORM (Write Once Read Many) storage regulatory compliances required in Finance, Insurance and healthcare etc. industries.
- S3 bucket **Versioning** must be enabled for using Object lock feature
- Object Retention – How long and under which conditions object should be retained
  - Retention Mode
    - Compliance Mode – Even root users cannot delete/alter objects until retention expires.
    - Governance Mode – Protected unless special permissions allow override  
`s3:BypassGovernanceRetention`
  - Retention Period
    - Fixed period during which Object remains locked
    - Can set default retention period at the S3 bucket level or unique retention period at individual object level
  - Legal Hold
    - No expiration date
    - Remains in place until explicitly removed at object version using permissions  
`s3:PutObjectLegalHold`

## Versioning



S3 Bucket

# MFA Delete

- Helps prevent accidental or malicious deletions by adding second layer of protection using Multi-Factor Authentication (MFA)
- Requires MFA code for:
  - Deleting object versions
  - Changing state of S3 bucket's versioning
- **S3 Bucket versioning must be enabled**
- **Must use AWS CLI, SDK or APIs to enable MFA delete**
- **Can be enabled/disabled only by the Root user.**
- Protection applies at the bucket level, not per-object.
- Works independently from S3 Object Lock (can be used together).



# To enable MFA Delete

- Generate an access key and secret key for the root user.
- Activate an MFA device for the root user.
- Configure the AWS CLI with the root user credentials.
- Configure MFA delete by running CLI command:

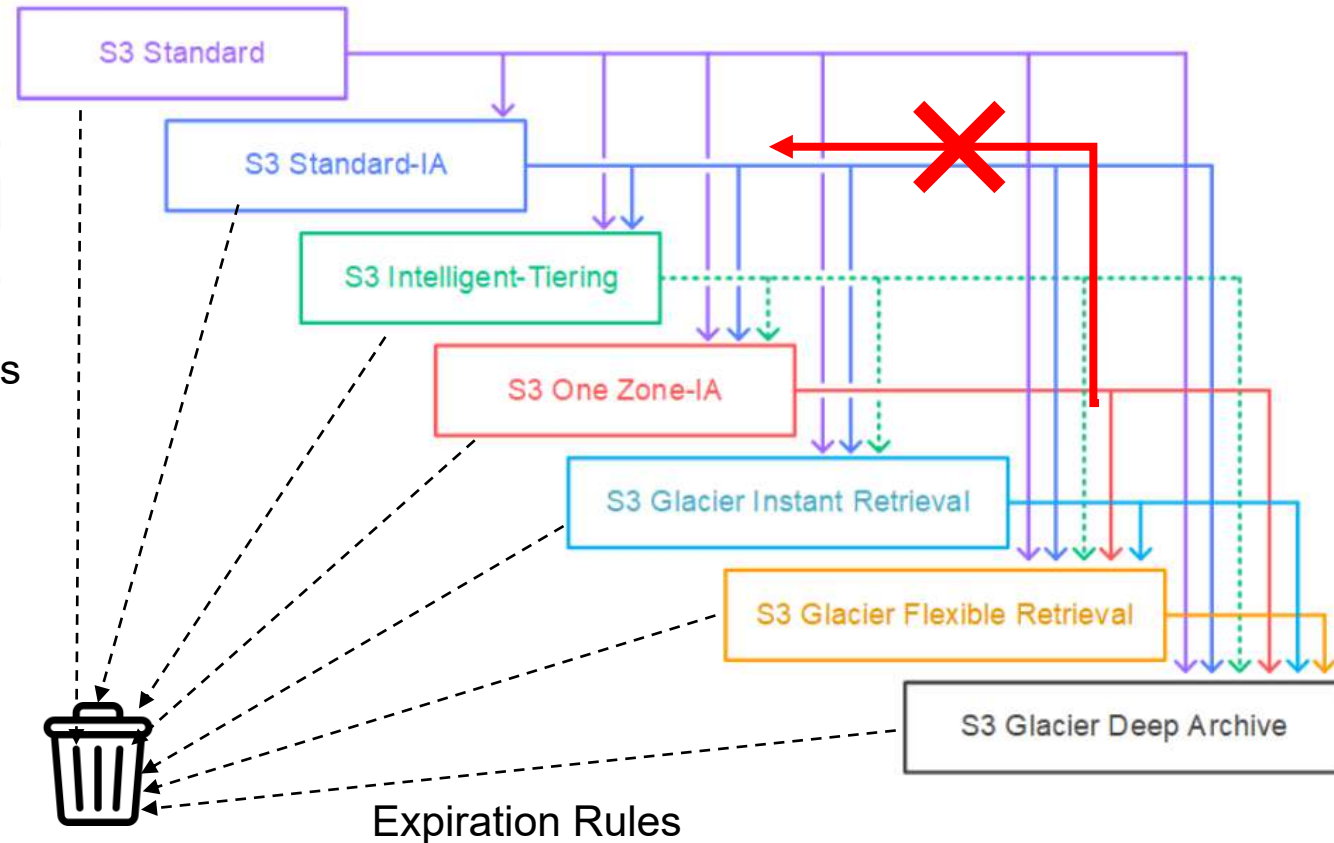
```
aws s3api put-bucket-versioning --bucket amzn-s3-demo-bucket-1  
--versioning-configuration Status=Enabled,MFADelete=Enabled  
--mfa "SerialNumber 123456"
```

Confirm that MFA delete is working and delete your root access keys.



# S3 Lifecycle

- Helps store objects cost effectively throughout their lifecycle by transitioning them to lower-cost storage classes, or deleting objects
- S3 follows waterfall model for lifecycle
- Transition Action
  - Transitions objects to different storage class
  - Example:** Move objects from S3 Standard class to S3 Standard-IA class after 30 days
- Expiration Action
  - Delete objects / versions after specified period
  - Delete incomplete multi-part uploads



# S3 Lifecycle Rules

- Transition minimum age - The minimum number of days an object must remain in its current storage class before a lifecycle rule can transition it to a colder storage class
  - Objects must be stored for at least 30 days before transitioning to S3 Standard-IA or S3 One Zone-IA
  - Similarly, S3 Glacier Instant Retrieval has a minimum storage duration of 90 days, and Glacier Deep Archive has a minimum storage duration of 180 days
- Lifecycle rules can be applied to specific prefixes, object tags or object size
  - Prefixes: logs/2025/november/\*
  - Tag: environment=development
  - Size: Define minimum and maximum e.g. between 1000MB and 5000MB
- Objects smaller than 128 KB will not transition by default to any storage class
- Lifecycle processing is asynchronous (does not do it immediately)
- Can be applied within a bucket. Can not be used for cross-bucket transfers.

# S3 Lifecycle Rules

|                                 | S3 Standard                                                 | S3 Intelligent-Tiering                                                         | S3 Standard-IA                                              | S3 One Zone-IA                                                     | S3 Glacier Instant Retrieval                                                  | S3 Glacier Flexible Retrieval                                                                                     | S3 Glacier Deep Archive                                     |
|---------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Use cases                       | General purpose storage for frequently accessed data        | Automatic cost savings for data with unknown or changing access patterns       | Infrequently accessed data that needs millisecond access    | Re-creatable infrequently accessed data.                           | Long-lived data that is accessed a few times per year with instant retrievals | Backup and archive data that is rarely accessed and low cost with expedited, standard, and bulk retrieval options | Archive data that is very rarely accessed and very low cost |
| First byte latency              | milliseconds                                                | milliseconds                                                                   | milliseconds                                                | milliseconds                                                       | milliseconds                                                                  | minutes or hours                                                                                                  | hours                                                       |
| Durability                      | Designed for 99.99999999% (11 nines) durability in a region | Durability depends on the underlying storage class where the objects are moved | Designed for 99.99999999% (11 nines) durability in a region | Designed for 99.99999999% (11 nines) durability within a Single AZ | Designed for 99.99999999% (11 nines) durability in a region                   | Designed for 99.99999999% (11 nines) durability in a region                                                       | Designed for 99.99999999% (11 nines) durability in a region |
| Availability                    | 99.99%                                                      | 99.90%                                                                         | 99.90%                                                      | 99.50%                                                             | 99.90%                                                                        | 99.99%                                                                                                            | 99.99%                                                      |
| Availability SLA                | 99.90%                                                      | 99%                                                                            | 99%                                                         | 99%                                                                | 99%                                                                           | 99.90%                                                                                                            | 99.90%                                                      |
| Availability Zones              | ≥3                                                          | ≥3                                                                             | ≥3                                                          | 1                                                                  | ≥3                                                                            | ≥3                                                                                                                | ≥3                                                          |
| Minimum storage duration charge | N/A                                                         | N/A                                                                            | 30 days                                                     | 30 days                                                            | 90 days                                                                       | 90 days                                                                                                           | 180 days                                                    |
| Retrieval charge                | N/A                                                         | N/A                                                                            | per GB retrieved                                            | per GB retrieved                                                   | per GB retrieved                                                              | per GB retrieved                                                                                                  | per GB retrieved                                            |

# S3 Lifecycle vs S3 Intelligent Tiering

## S3 Lifecycle

- User-defined rules move objects between storage classes
- Have to specify the duration (days) for the transition
- Follows waterfall model, one-directional
- Supports Expiration Rules
- Use case: Known access pattern
- Examples: Logs, Backups, Archives

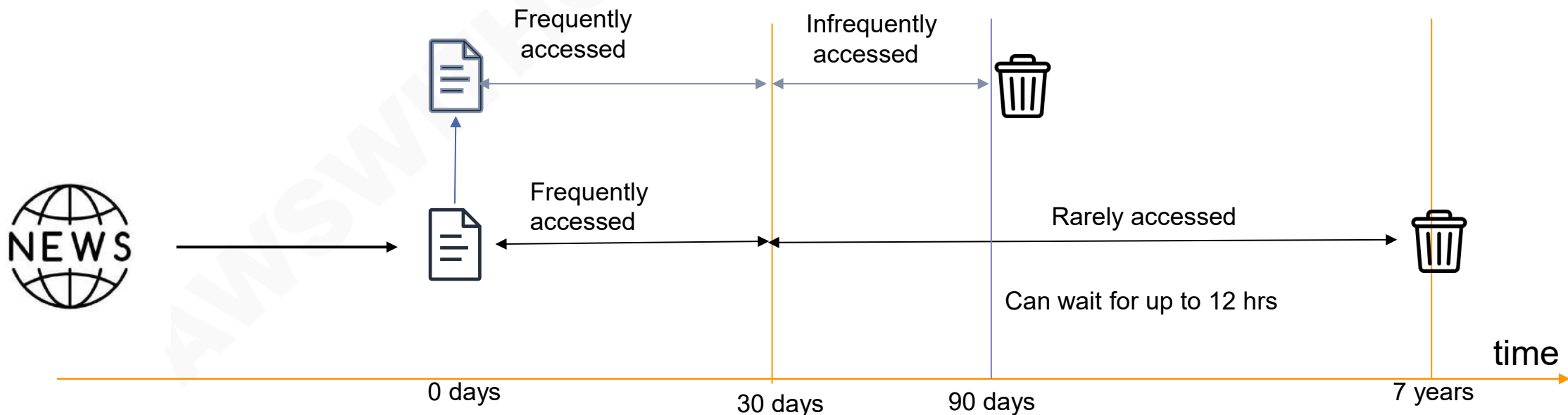
## S3 Intelligent Tiering

- Automatic tier changes based on object access patterns
- Can not specify duration. Transition is based on access pattern.
- Can move object both ways
- Does not support Expiration/Deletion rules
- Use case: Unknown access pattern
- Example: Data Lake, Analytics, Media or Application data

# Scenario

A News media company stores full news articles uploaded by editors into Amazon S3. These articles are frequently accessed for the first 30 days while they remain trending. After 30 days, they are rarely viewed but must be retained for archival/legal compliance for 7 years. If readers access older articles, they are fine waiting up to 12 hours for retrieval. A short summary snippet is also generated for each article for homepage display, and these snippets will be accessed primarily for first 30 days after which they can be accessed rarely for up to maximum 90 days.

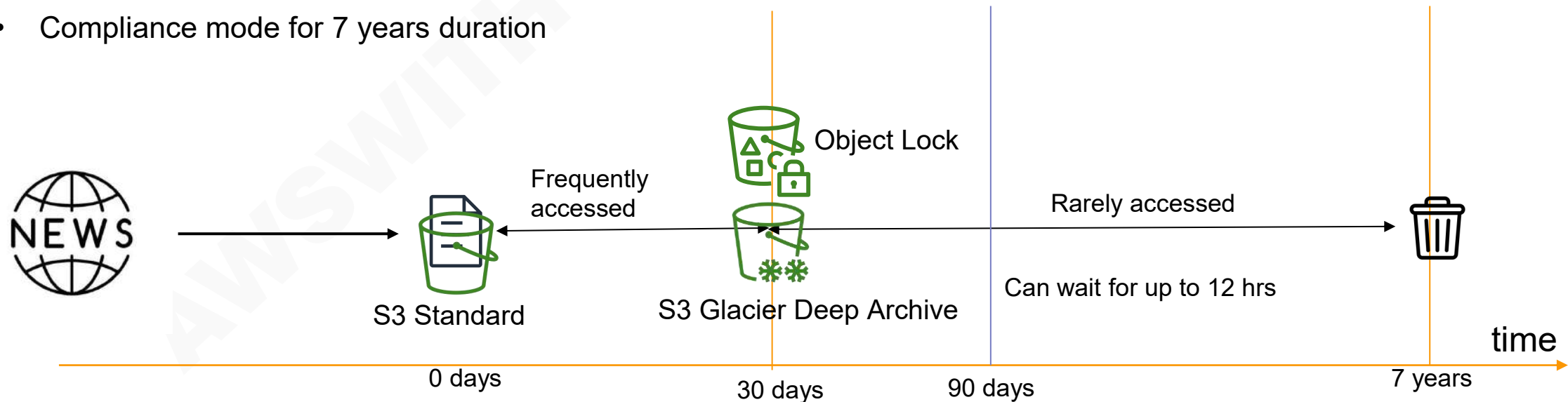
How would you design this?



# Scenario

## For Original News Articles

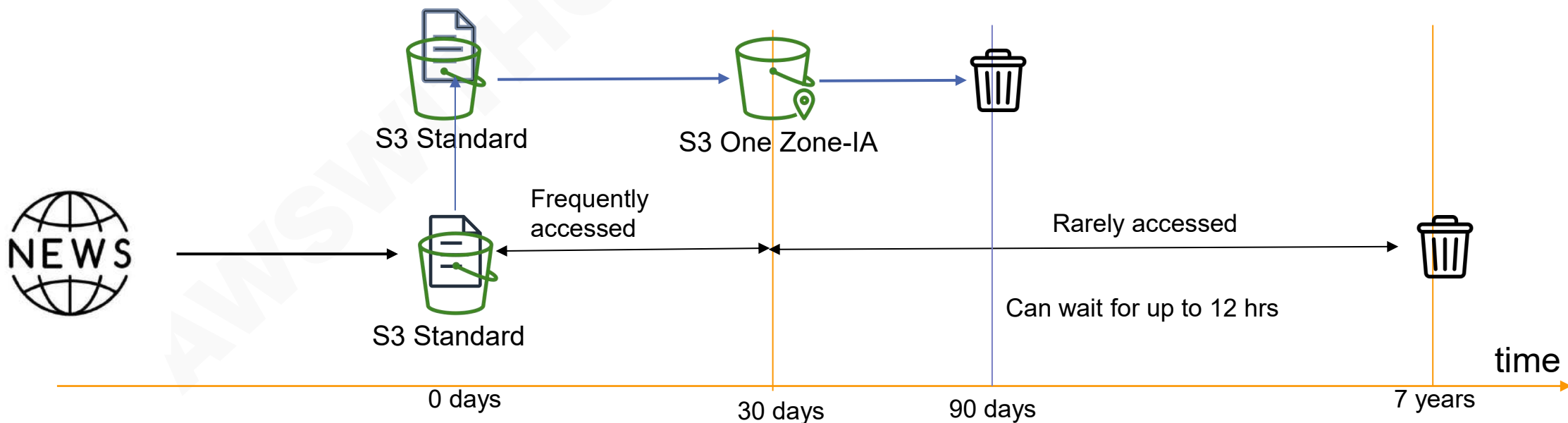
- Store new articles in S3 Standard for fast access during the first 30 days.
- Add an S3 lifecycle transition rule:
  - Transition articles to S3 Glacier Deep Archive after 30 days (cheapest tier, 12-hour retrieval acceptable).
- Add an S3 lifecycle Expiration rule:
  - Expire/delete articles from S3 Glacier Deep Archive after 7 years
- (Optionally) Add Object lock
  - Compliance mode for 7 years duration



# Scenario

## For Summary files

- Store Summary files in S3 Standard for first 30 days.
- Add an S3 Lifecycle rule to move summary files to S3 One Zone-IA after 30 days. (Note that summary files can be re-created and hence do not need to be stored in S3 standard-IA tier)
- Add an S3 lifecycle Expiration rule to delete summary files after 90 days



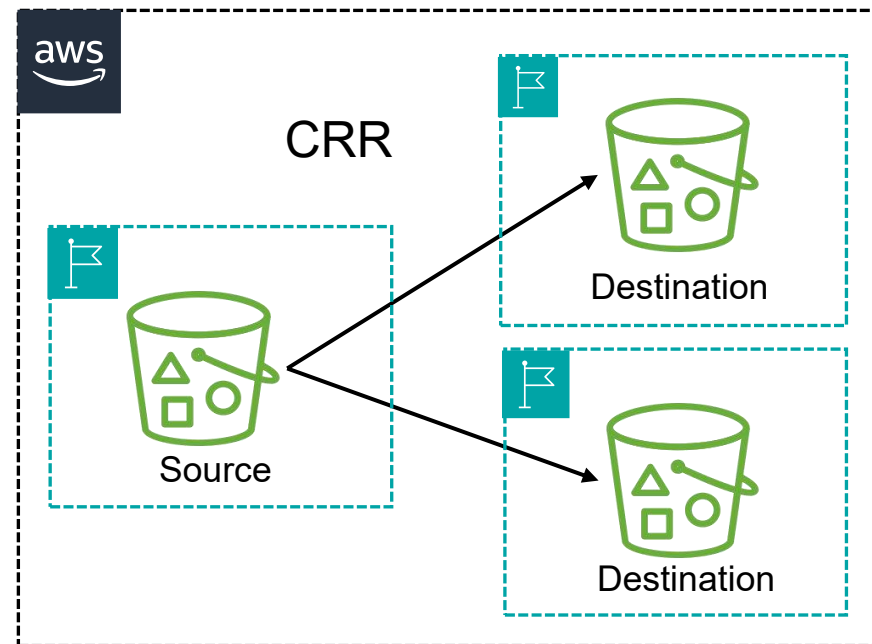
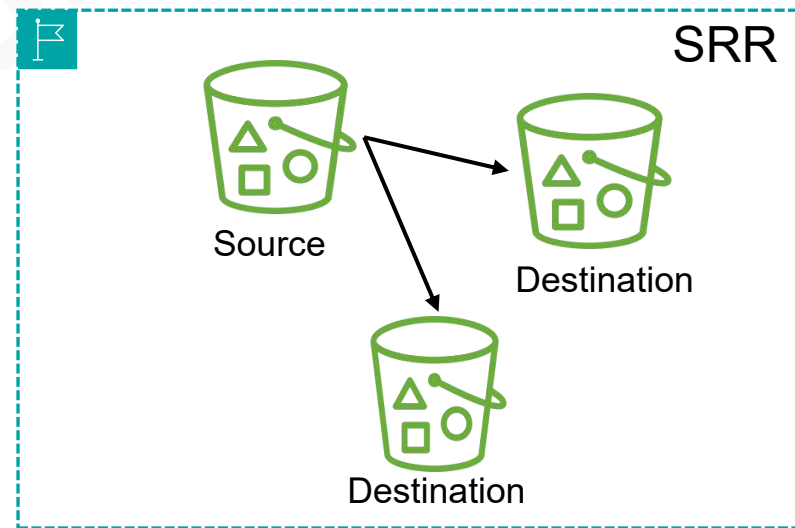


# S3 Replication

- Automatic and asynchronous replication of objects between S3 buckets
- **Bucket versioning must be enabled on both source and destination buckets**
- Supports - Same Region Replication (SRR) & Cross Region Replication (CRR)
- Supports Live Replication for new objects and On-demand S3 Batch Replication for existing objects
- (Optional) S3 Replication Time Control (RTC) for 15min SLA

## S3 Replication use cases:

- Minimize latency by bringing data closer to the end user (CRR)
- Aggregate logs into centralized bucket (SRR)
- Live replication from Production to Test environment (SRR)
- Data Sovereignty – Copies of data in different accounts
- Compliance – store copy of data at distance (CRR)





# S3 Replication – Important to know

- Cross-Region Replication incurs data transfer cost
- For encrypted objects (e.g. SSE-KMS) the appropriate IAM permissions for the KMS are required depending on SRR and CRR.
- You can use filters (object prefix, tags) to narrow down the scope for objects to be replicated
- Replication chaining is not supported (A -> B -> C). You can set up a separate rule for A -> C replication or use Batch replication for B -> C.
- Replication Time Control (RTC) offers 15-minute SLA replication ensuring 99.99% of new objects are replicated to the destination bucket within 15 minutes.
- Replication status can be monitored using CloudWatch Metrics:
  - Metrics: Bytes pending replication, Replication latency, Operations Pending/Failed Replication
  - You can receive the failed replication status using S3 Event Notification

# How delete operations affect replication?

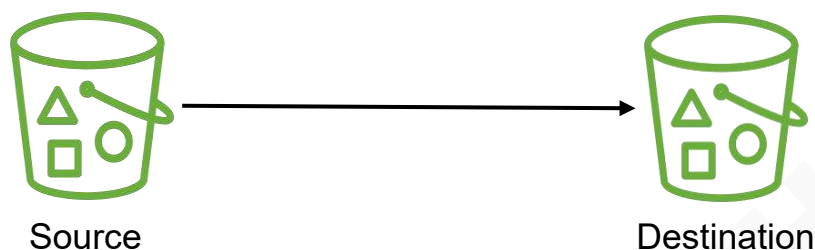
## When deleting Objects without using Version ID

- When Object is deleted in the source bucket, DELETE marker is created in the source bucket
- This DELETE marker is not replicated to the target bucket
- Optionally, we can enable DELETE marker replication at the bucket level or object level
- After enabling, the DELETE marker is also replicated, and object is considered to be deleted in the target bucket

## When deleting Objects using Version ID

- Object is deleted in the source bucket but its not deleted in the destination bucket
- This protects data from malicious deletions

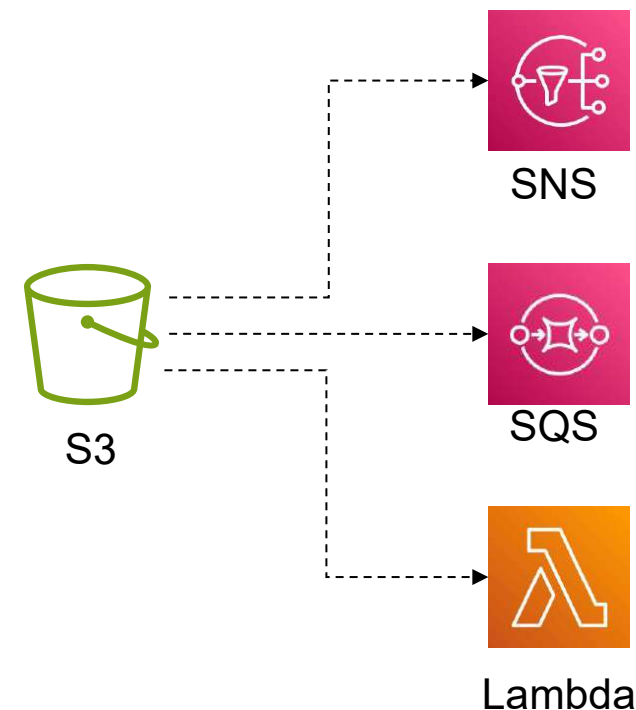
# Exercise - S3 Replication



- 1 Create source and destination buckets
- 2 Enable versioning for both the buckets
- 3 In source bucket, go to Management and create Replication rule
- 4 Upload an object into source bucket and wait for few minutes
- 5 Verify in the target bucket if you see the same object with same object version

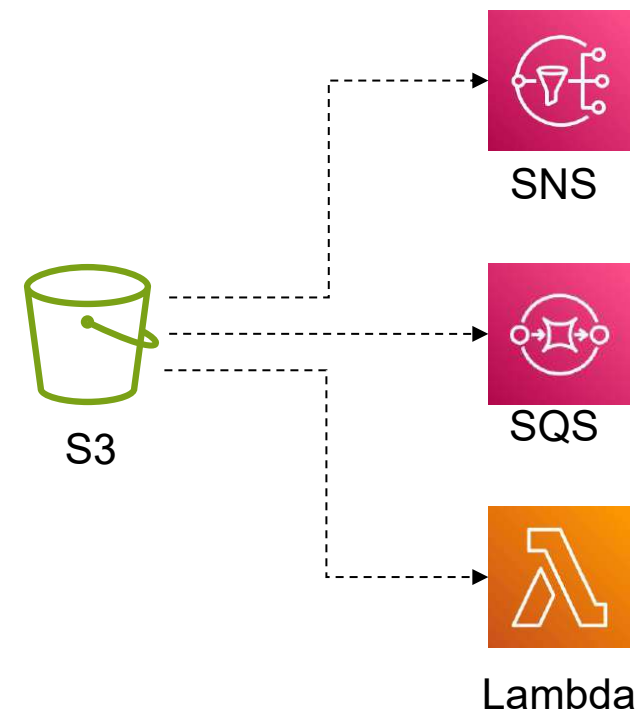
# S3 Event Notification

- Notifies the target service when certain S3 event occurs
- Supported notification destinations:
  - Amazon SNS (`SNS:Publish`)
  - Amazon SQS (`SQS:SendMessage`)
  - AWS Lambda (`lambda:InvokeFunction`)
- S3 Events:
  - `s3:ObjectCreated` -> New object added
  - `s3:ObjectRemoved` -> Object deleted
  - `s3:ObjectRestore:*` -> Glacier restore initiated or completed
  - `s3:Replication:*` -> Replication success/failure status events
  - `s3:Lifecycle*` -> Lifecycle transition and expiration events
  - `s3:IntelligentTiering` -> Intelligent Tiering archive events



# S3 Event Notification

- You can filter the notifications by Object prefix (/images/\*) or suffix (.jpg)
- If versioning is enabled, notifications are sent for each version
- Delivery – at-least-once (ordering is not guaranteed)
- Notification contains object metadata, not the object data.

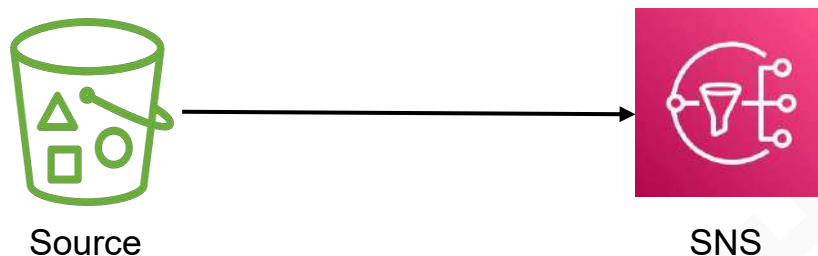


# Resource based policy to allow S3 send event notification

## Lambda Function Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Invoke",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:<REGION>:<ACCOUNT-ID>:function:<FUNCTION-NAME>",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:s3:::<BUCKET-NAME>"
        }
      }
    }
  ]
}
```

# Exercise - S3 Event Notification



- 1 Create SNS topic and create email subscription
- 2 SNS Topic Access Policy to allow S3 to send notification
- 3 Check your email and confirm the subscription
- 4 Create S3 event notification with SNS as target
- 5 Upload an object into source bucket and wait for few seconds
- 6 Verify if you have received an email

# SNS Topic - Sample Access policy

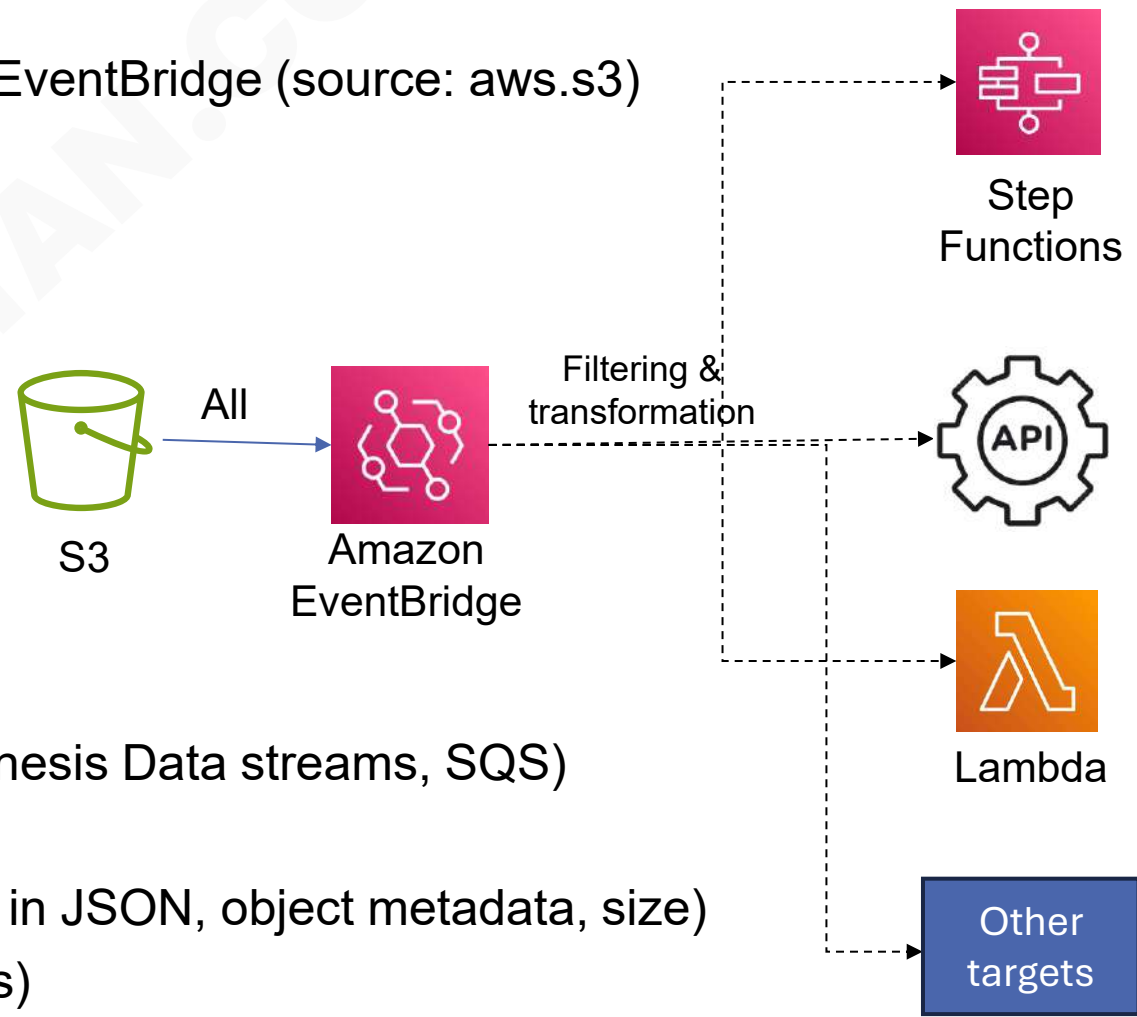
## SNS Access Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3ToPublish",
      "Effect": "Allow",
      "Principal": { "Service": "s3.amazonaws.com" },
      "Action": "SNS:Publish",
      "Resource": "arn:aws:sns:<region>:<account-id>:<topic-name>",
      "Condition": {
        "ArnLike": {
          "aws:SourceArn": "arn:aws:s3:::<bucket-name>"
        }
      }
    }
  ]
}
```



# S3 to Amazon EventBridge

- All S3 API events are automatically sent to Amazon EventBridge (source: aws.s3)
- EventBridge can trigger 25+ targets including
  - Step Functions
  - ECS tasks
  - SQS
  - Lambda functions
  - API destinations
  - Custom applications
- Use EventBridge when:
  - You need multiple downstream targets (e.g. Kinesis Data streams, SQS)
  - Cross-account event routing
  - Advanced filtering rules (based on event fields in JSON, object metadata, size)
  - Need to trigger a workflow (e.g. Step Functions)



# S3 events - Exam Scenarios

**1. Scenario:** Generate an image thumbnail immediately whenever a user uploads a profile picture into the images/ folder.

=> Use S3 Event Notification to Lambda where lambda can process the image and create thumbnails

**2. Scenario:** A compliance team wants to audit every S3 API call, including PUT, DELETE, tagging changes, access point usage, and internal S3 events, and route them to a central audit account.

=> Use Amazon EventBridge to route all the S3 API calls to centralized Audit account

**3. Scenario:** A video-processing pipeline needs to queue uploaded videos for background processing using a consumer fleet that scales automatically.

=> Use Event Notification to Simple Queue Service (SQS) where video (object) metadata e.g. path, date, size etc. is sent to SQS queue. These messages in the queue are processed asynchronously by the downstream applications like Lambda or ECS tasks.

# S3 events - Exam Scenarios

**4. Scenario:** A news app needs to fan-out notifications (email, SMS, mobile push) to multiple subscribers as soon as a new article is uploaded.

=> Use S3 Event Notification to Simple Notification Service (SNS) topic. Topic should be subscribed by the end users / mobile

**5. Scenario:** A finance team wants to trigger monthly billing workflows across multiple AWS accounts whenever a .csv report lands in a reports/ folder

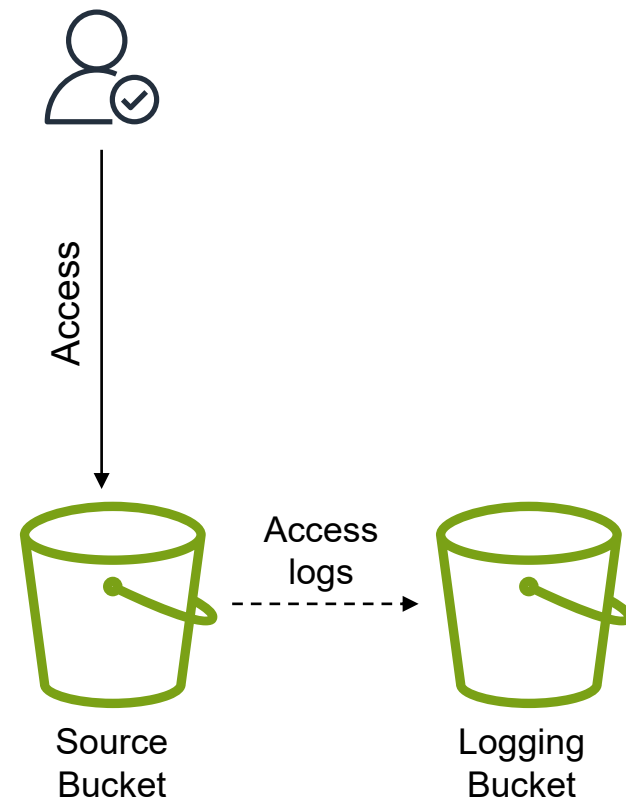
=> Use Amazon EventBridge to trigger AWS Step function workflow across multiple AWS accounts.

**6. Scenario:** A media archive system wants to process archived files automatically using Lambda when a Glacier restore completes

=> Use S3 Event Notification to Lambda when Glacier restore event is generated

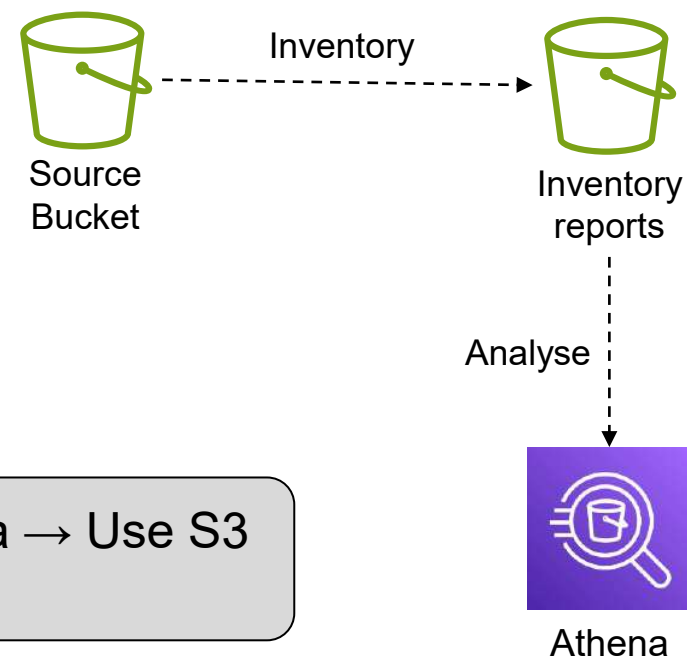
# S3 Server Access Logs

- S3 Server access logging provides detailed records for the requests made to a bucket
- Log record includes details like requester, bucket name, object key, operation, response status, error code, bytes transferred etc.
- Logs are delivered to another S3 bucket (can not be the same bucket as source)
- Not real-time, may take minutes to hours
- No additional charge for enabling Access logs but pay for logs storage.
- May generate huge logs, recommended to use S3 Lifecycle to move logs to cheaper storage and purge after some time
- **Use cases:**
  - Security / Audit
  - Access pattern analysis
  - Cost optimization by finding highly accessed objects
- **Never use the same bucket as source and target for Access logging**



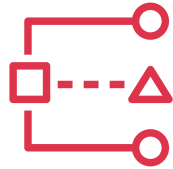
# S3 Inventory

- Provides a scheduled report (daily or weekly) listing all objects and metadata in a bucket.
- Includes details like object size, storage class, encryption status, replication status, object version, ETag, last modified date, and Object Lock info etc.
- Often used with Amazon Athena for large-scale analysis or auditing for encryption and replication compliance.
- Cost and performance efficient than using S3 List API operations which may otherwise scan millions of the objects for large size buckets
- Inventory scope may be configured by object prefix, current version or all versions

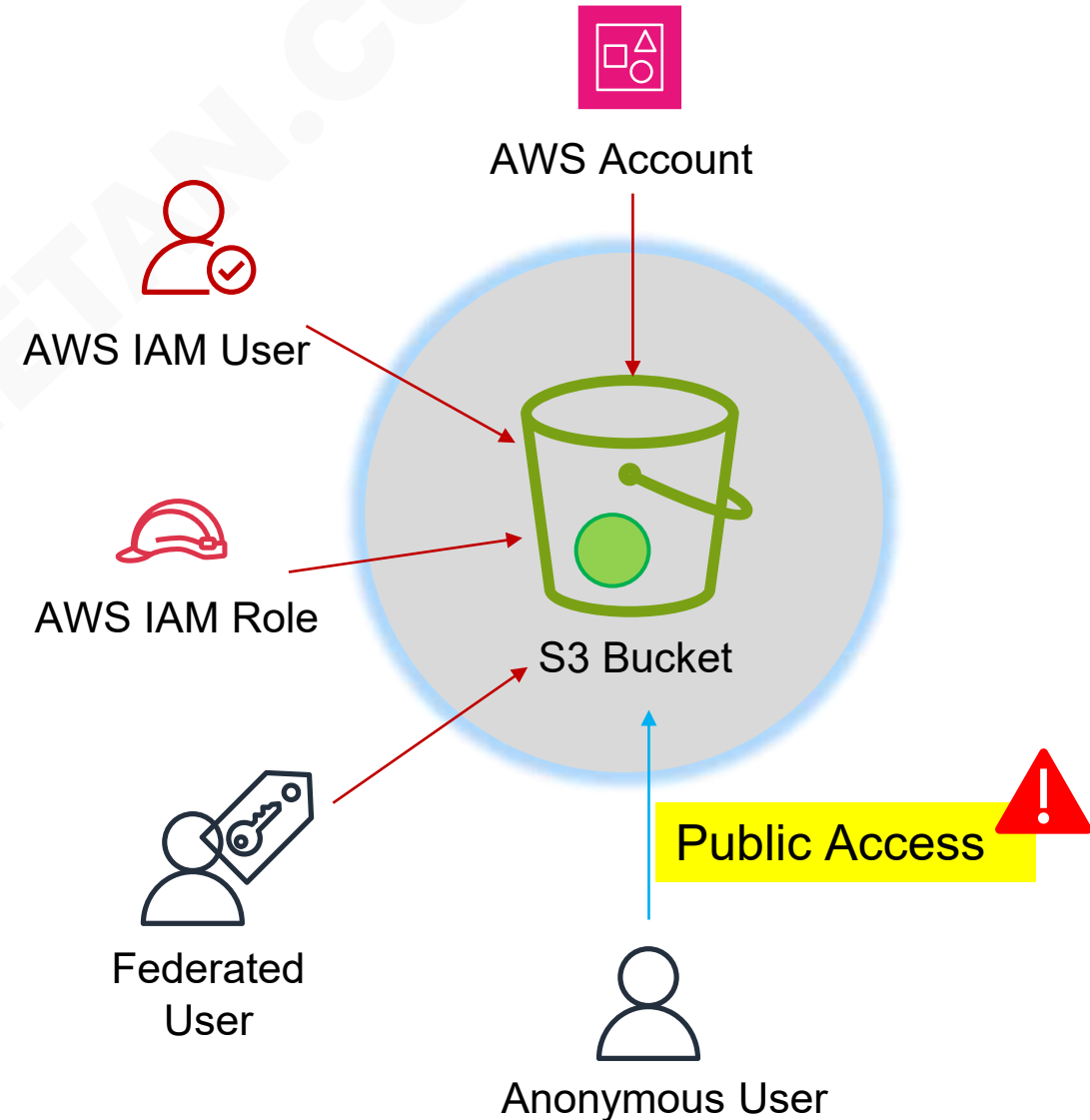


**Tip:** Need a scalable, periodic report of all objects and their metadata → Use S3 Inventory (not ListObjects API).

# IAM Access Analyzer for S3



- Analyses the S3 ACL and Bucket policies to ensure that only required entities have access to S3 buckets
- Identifies Publicly accessible S3 buckets
- Identifies buckets which are shared with other AWS accounts



# S3 Storage Lens

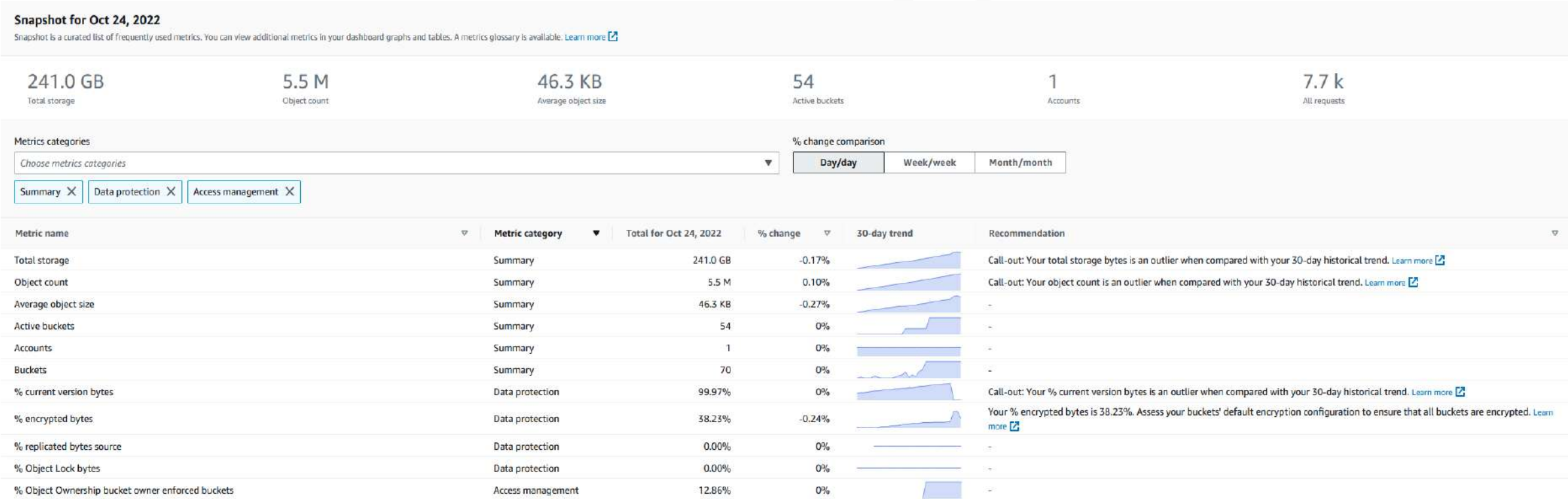
- It's a visualization tool which provides organization-wide visibility into S3 storage usage and activity across all buckets, Regions, and accounts (can include AWS Organizations).
- Offers dashboard-based analytics with over 100+ metrics, including storage usage, object counts, versions, encryption status, activity trends, and replication metrics.
- Helps identify cost optimization opportunities such as unused buckets, non-IA-eligible data, incomplete multipart uploads, and objects without encryption.
- Supports advanced metrics & recommendations (paid tier) including per-prefix insights, activity metrics (PUT/GET/delete rates), and security findings.



S3 Storage  
Lens

**Tip:** Need AWS organization-wide S3 usage & activity analytics → Use S3 Storage Lens

# S3 Storage Lens



Source: [https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage\\_lens.html](https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage_lens.html)



# S3 Monitoring, Logging and Reporting - Summary

| Tool                  | What it does                                                                           | Scope                             | Frequency            | Best for                                                                                     |
|-----------------------|----------------------------------------------------------------------------------------|-----------------------------------|----------------------|----------------------------------------------------------------------------------------------|
| S3 Server Access Logs | Captures detailed records of requests made to an S3 bucket                             | Bucket level                      | Delayed, best-effort | Understanding access patterns, troubleshooting access issues, analyzing who accessed objects |
| IAM Access Analyzer   | Detects buckets that allow public access or external account access                    | Policy level                      | Real-time            | Identifying unintended public or cross-account exposure                                      |
| S3 Inventory          | Generates object-level reports with metadata such as size, encryption, and replication | Bucket or prefix level            | Daily or weekly      | Compliance checks, auditing object properties, validating replication and encryption         |
| S3 Storage Lens       | Provides dashboards and metrics for usage and activity across S3 storage               | Account-wide or organization-wide | Daily                | Storage optimization, usage trends, activity insights across multiple buckets and accounts   |

Source: ChatGPT

# Amazon S3 Performance optimization

- Prefix partitioning - GET/PUT transaction optimization
- Multi-part Upload and byte-range fetches - Data upload and retrieval optimization
- S3 Transfer Acceleration - Long distance network latency optimization
- Amazon CloudFront - Frequently accessed data optimization using Cache

AWSWITHCHETAN.COM

# Amazon S3 – Prefix partitioning

- S3 automatically scales to high request rates by partitioning data based on prefixes.
- You can achieve at least:
  - 3,500 PUT/COPY/POST/DELETE requests/sec per prefix
  - 5,500 GET/HEAD requests/sec per prefix
- To increase throughput, use multiple prefixes so S3 can create more partitions.
- Avoid creating millions of objects under a single “hot” prefix (e.g., logs/ only).
- S3 scales horizontally, so the more parallel requests you make, the higher the throughput. Use parallelize operations especially for analytics workloads
- Prefixing also helps with efficient batch operations, list patterns, partitioned queries, and lifecycle management.

/logs/2024\_01\_05\_app.log



/logs/2024/01/05/app.log

**Example 1**

images/hash-01/img123.jpg  
images/hash-02/img124.jpg  
images/hash-03/img125.jpg

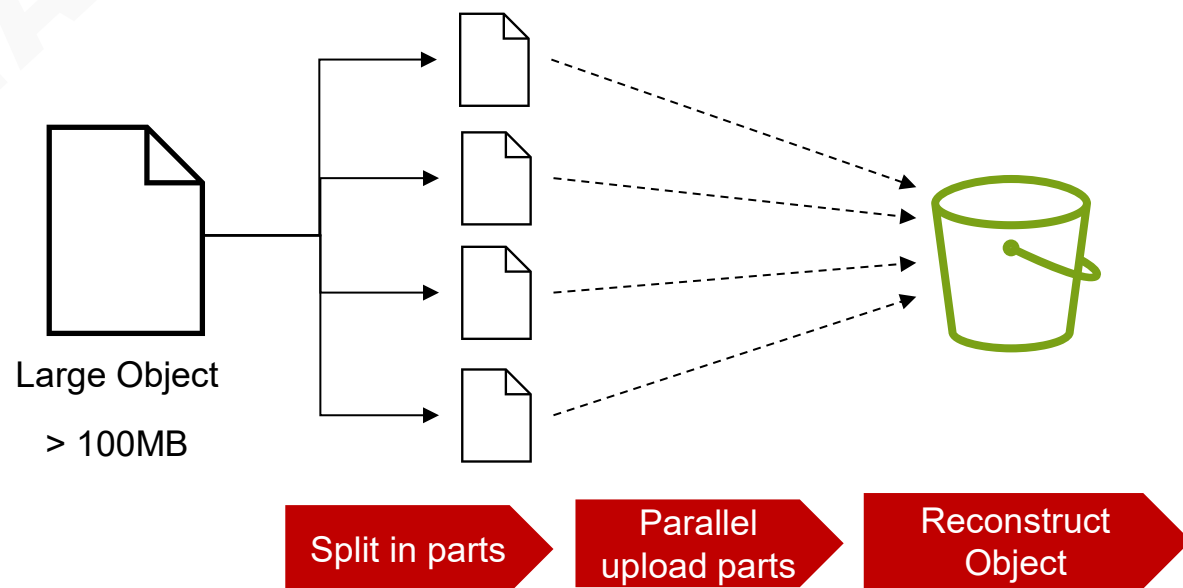
**Example 2**

iot/device-01/2024/01/06/data.json  
iot/device-02/2024/01/06/data.json  
iot/device-03/2024/01/06/data.json

**Example 3**

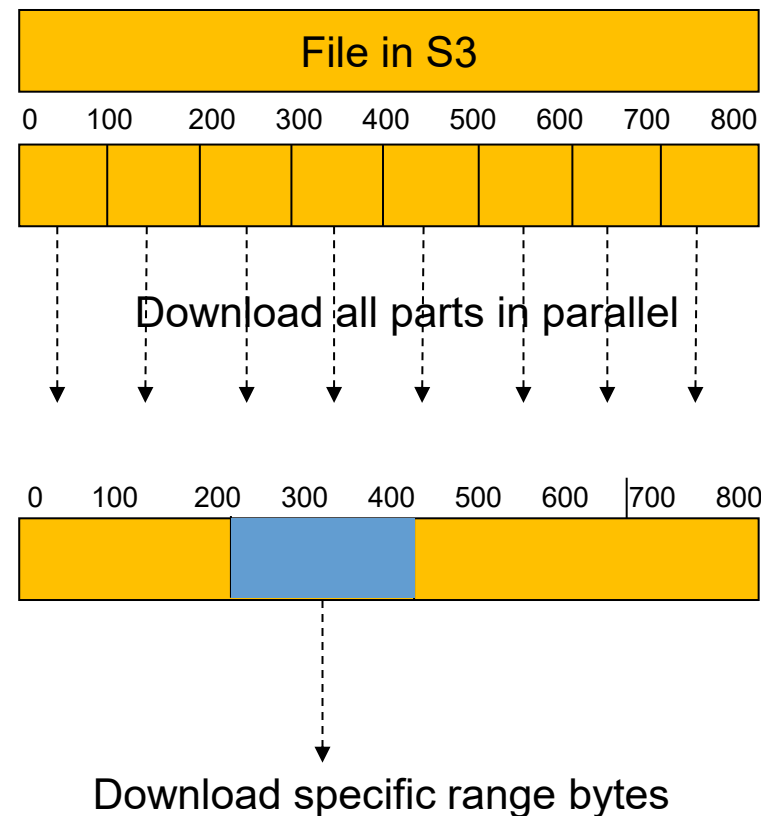
# Multi-part upload

- Recommended for objects >100 MB;
- Must for objects > 5 GB
- If a part fails, only that part is retried - faster recovery and better resilience.
- Use Multipart Upload + Parallel Threads to utilize maximum network bandwidth for the upload.



# Byte-range fetch

- Allows downloading different byte ranges of the same object in parallel.
- S3 supports extremely high read throughput when doing multiple range GETs.
- Great for:
  - Media streaming
  - Large file downloads
  - ETL workloads needing partial reads
  - Resumable downloads



## Example (using AWS CLI):

```
$aws s3api get-object --bucket amzn-s3-demo-bucket1 --key folder/my_data  
--range bytes=0-500 my_data_range.output
```

# S3 Transfer Acceleration

# AWS Edge locations



\*Diagram just for illustration, not actual



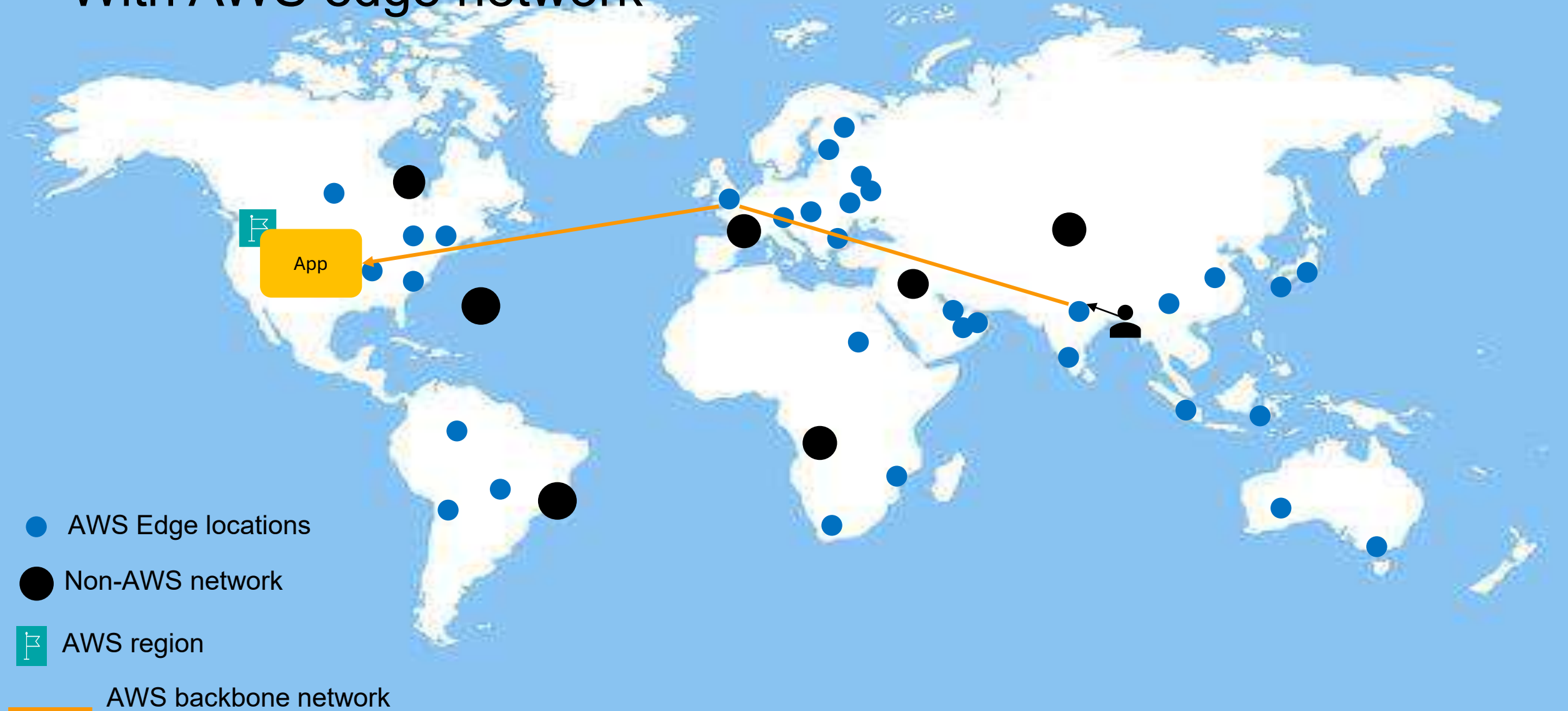
# Without AWS edge network



\*Diagram just for illustration, not actual



# With AWS edge network



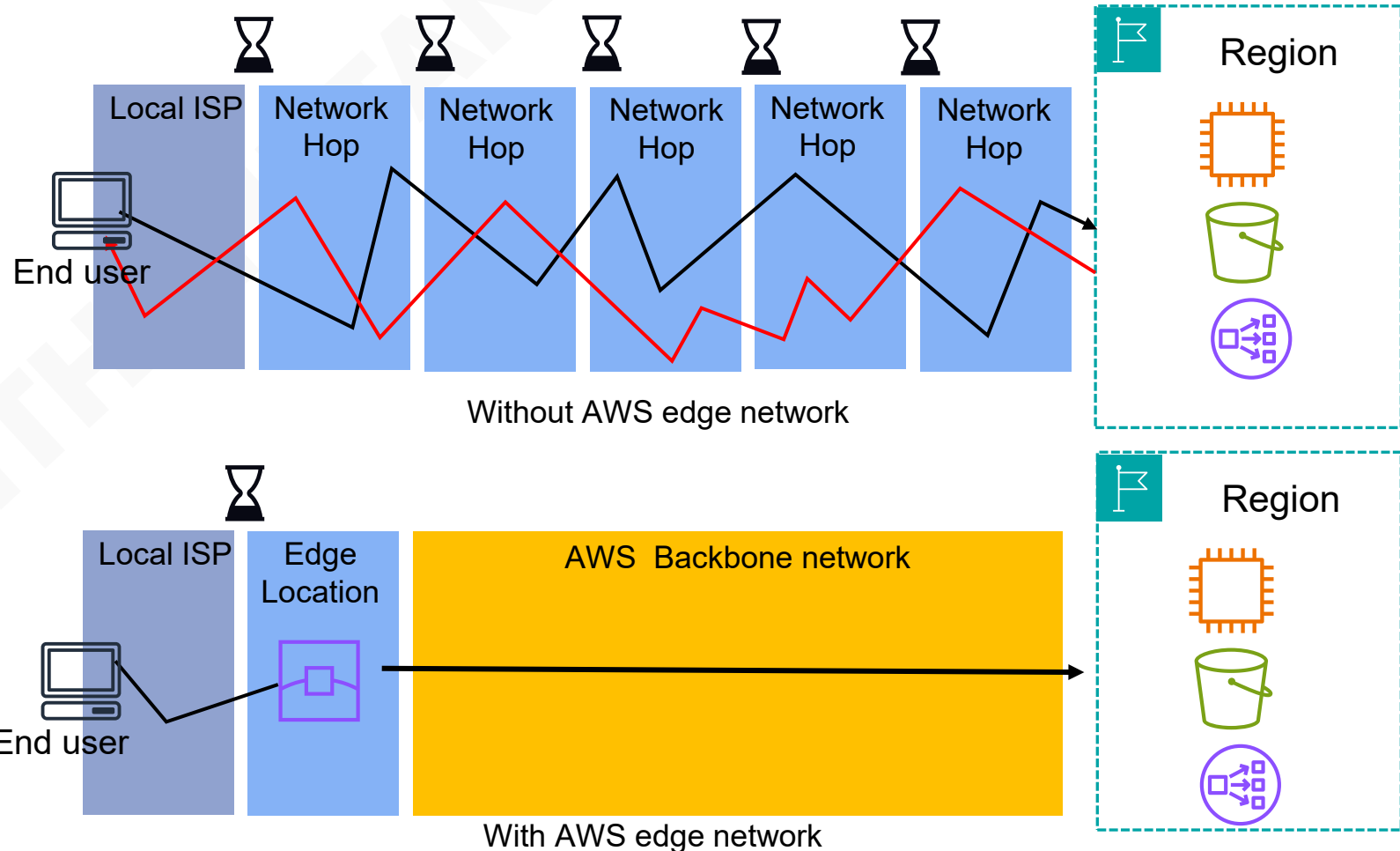
\*Diagram just for illustration, not actual  
Chetan Agrawal

# AWS edge network and services

AWS Edge locations are used by different AWS services to lower the network latency for end users

- Use AWS edge network by using:

- Amazon CloudFront
- AWS Global Accelerator
- AWS S3 Transfer Acceleration



# S3 Transfer Acceleration (S3TA)

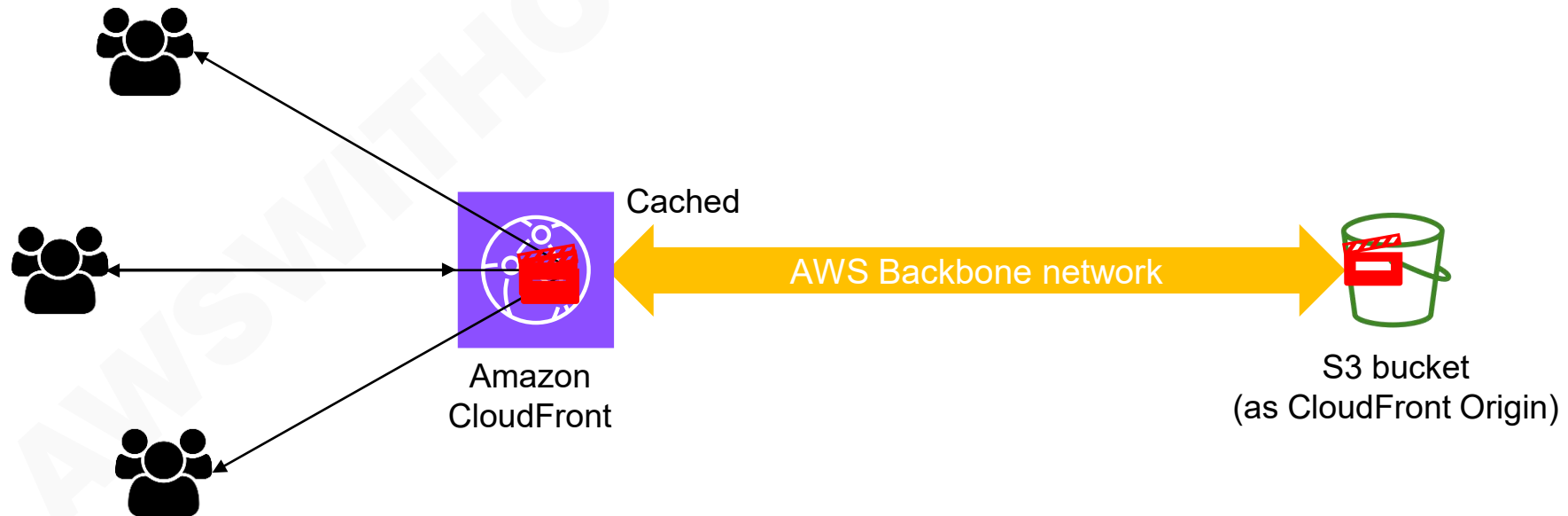
- Speeds up content transfer to and from S3 bucket by 50-500% for long-distance transfer of larger objects.
- Routes traffic through globally distributed Edge Locations and over AWS backbone networks
- Additionally, uses network protocol optimizations
- We can enable/disable S3 Acceleration for S3 bucket
- Provides new s3 bucket endpoint when accelerator is enabled e.g. `bucket-name.s3-accelerate.amazonaws.com`



[AWS Tool: https://s3-accelerate-speedtest.s3-accelerate.amazonaws.com/en/accelerate-speed-comparision.html](https://s3-accelerate-speedtest.s3-accelerate.amazonaws.com/en/accelerate-speed-comparision.html)

# Content caching using Amazon CloudFront

- Amazon CloudFront is a **Content Delivery Network** (CDN) service.
- It supports many different origins including S3 Bucket
- CloudFront caches the static contents e.g. images, videos etc. at the edge locations
- Reduced Data transfer out rate as compared to internet (+1TB free data transfer out per month)



# Block direct access to S3

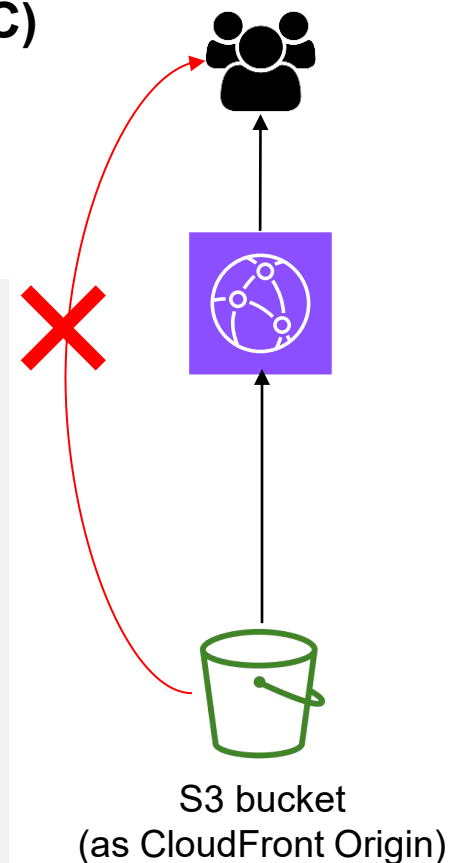
## CloudFront Origin Access Identity (OAI) CloudFront Origin Access Control (OAC)

- Legacy mechanism to access a private S3 bucket.
- Creates IAM-like identity

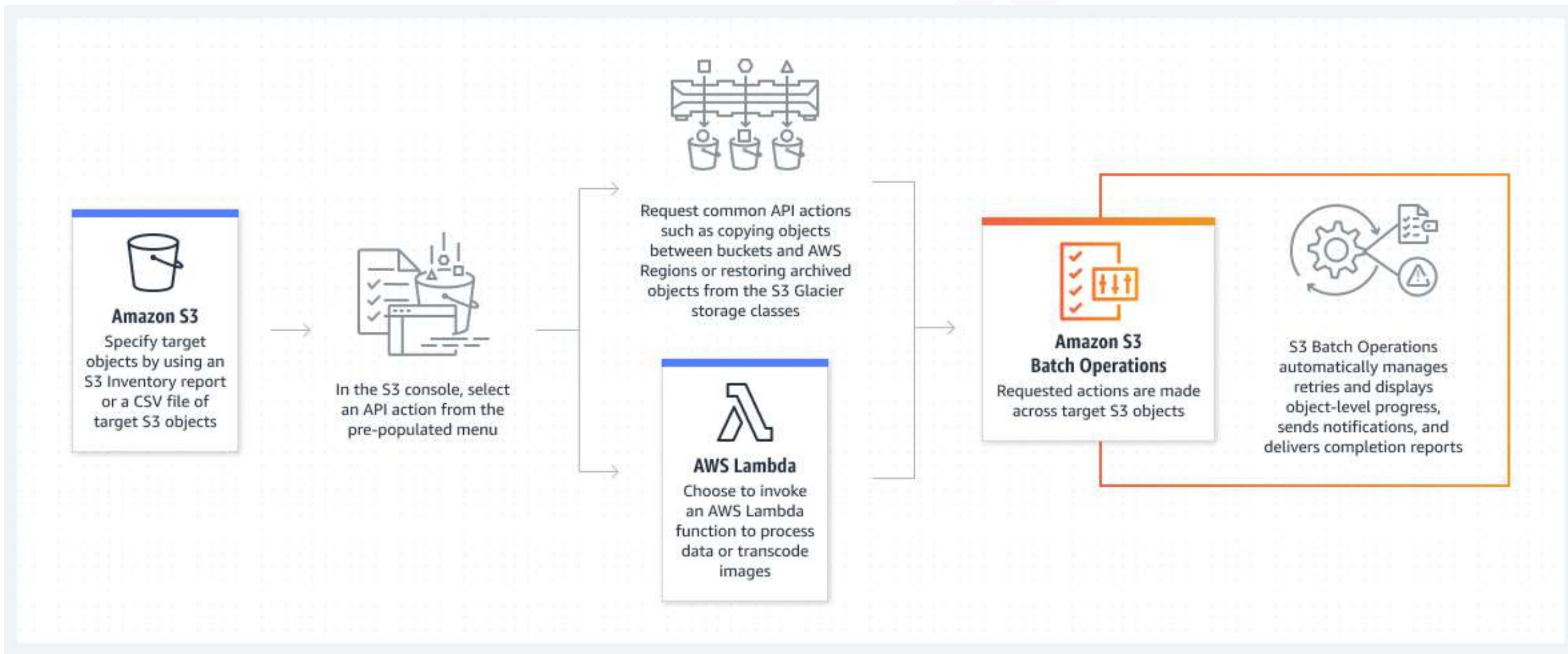
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS":
"arn:aws:iam::cloudfront:user/CloudFront Origin
Access Identity <OAI-ID>"
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::<bucket-name>/*"
    }
  ]
}
```

- New mechanism to access a private S3 bucket.
- Uses SigV4 signing

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOAC",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::<bucket-name>/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn":
"arn:aws:cloudfront::<account-
id>:distribution/<distribution-id>"
        }
      }
    }
  ]
}
```

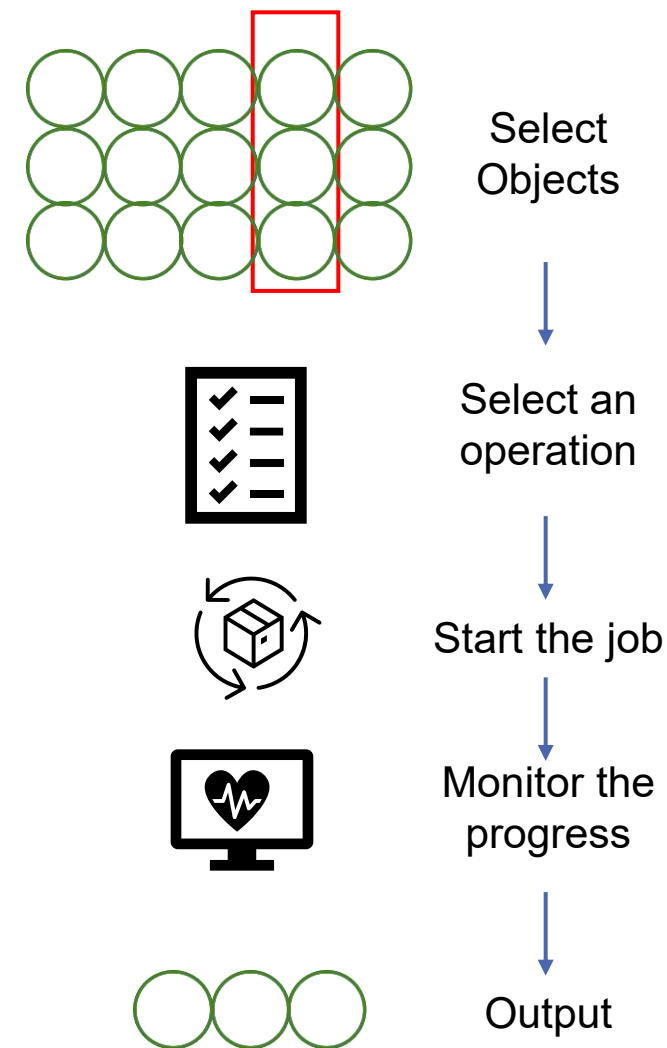


# S3 Batch Operations



# S3 Batch Operations

- Allows to perform bulk operations on millions or billions of S3 objects in a single managed job.
- Operates on a **manifest** file, which is a list of object keys generated by:
  - S3 Inventory file or custom CSV file
  - Generate objects list by specifying source and filter (New option)
- Supports multiple actions:
  - Copy objects across buckets or within same bucket (can also encrypt during copy)
  - Replace or Delete object tags
  - Modify ACLs
  - Restore Glacier objects
  - Initiate S3 object lock retention changes
  - Execute AWS Lambda functions on each object
- Entire job is fully managed by S3 - includes retry logic, progress tracking, and detailed status reporting.
- **Use cases:** Bulk updates or processing, Large migrations, Metadata cleanup, Object lock changes, Restoring objects from Archive





# Demo – Replace Object tags

- 1 Create a csv manifest file with list of objects and upload to your S3 bucket
- 2 Create IAM role with permissions and trust policy as defined here  
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/batch-ops-iam-role-policies.html>
- 3 Create S3 Batch Operations Jobs – Provide location for the manifest file and select Action as Replace Tags. Provide the Tags to be added. Optionally provide the details to save the job execution results.
- 4 Run the job and wait to finish
- 5 Verify whether tags are added for the objects listed in the manifest files



# S3 Batch Operations - Demo

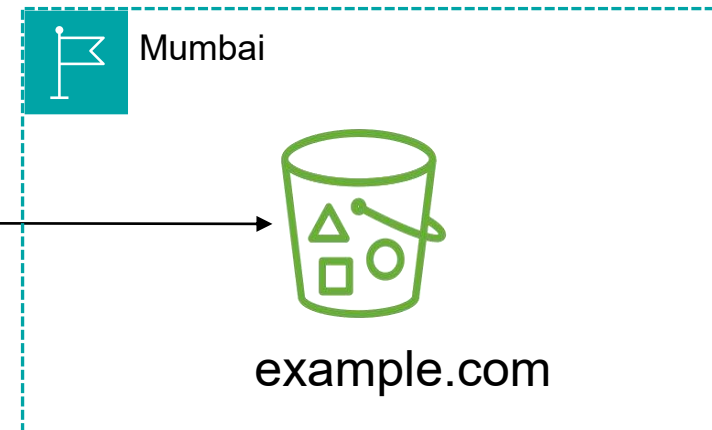
- Allows to perform bulk operations on millions or billions of S3 objects in a single managed job.
- Operates on a **manifest** file, which is a list of object keys generated by:
  - S3 Inventory file or custom CSV file
  - Generate objects list by specifying source and filter (New option)
- Supports multiple actions:
  - Copy objects across buckets
  - Replace or Delete object tags
  - Modify ACLs
  - Restore Glacier objects
  - Initiate S3 object lock retention changes
  - Execute AWS Lambda functions on each object
- Entire job is fully managed by S3 - includes retry logic, progress tracking, and detailed status reporting.
- **Use cases:** Bulk updates or processing, Large migrations, Metadata cleanup, Object lock changes, Restoring objects from Archive

# Static Website Hosting

- Because S3 can be accessed over the Web, we can host Static (**http**) website on S3
- For Setting up Static Website on S3:
  1. Upload static HTML files to S3 bucket
  2. Make S3 bucket Public (Disable Block Public Access & set Bucket Policy)
  3. Enable Static website hosting for the bucket
- Depending on your Region, your Amazon S3 website endpoint follows one of these two formats.
  - s3-website dash (-) Region - `http://bucket-name.s3-website-Region.amazonaws.com`
  - s3-website dot (.) Region - `http://bucket-name.s3-website-Region.amazonaws.com`

<http://example.com.s3-website.ap-south-1.amazonaws.com>

  
User  
(internet)



# Exercise: Static Website Hosting

- 1 Create S3 Bucket in the region of your choice.
- 2 Disable “Block Public Access” setting for the bucket
- 3 Download sample static website template and extract locally on your machine.
- 4 Upload static website content to S3 bucket:
  - Drag and drop all files and folders directly in the bucket.
  - Make sure index.html file is directly inside bucket and not inside folder.
- 5 Enable Static Website for the bucket:
  - Select Bucket -> Properties -> Static Website Hosting -> Use this bucket to host a website.
  - Provide index document as “index.html” -> Save
- 6 Try to access website using HTTP endpoint. You should get Permissions denied error.  
**<bucket-name>.s3-website.<AWS-region>.amazonaws.com**

# Exercise: Static Website Hosting

- 7 Go to bucket permissions and add following Bucket Policy. This allows public READ access to all objects in your bucket:

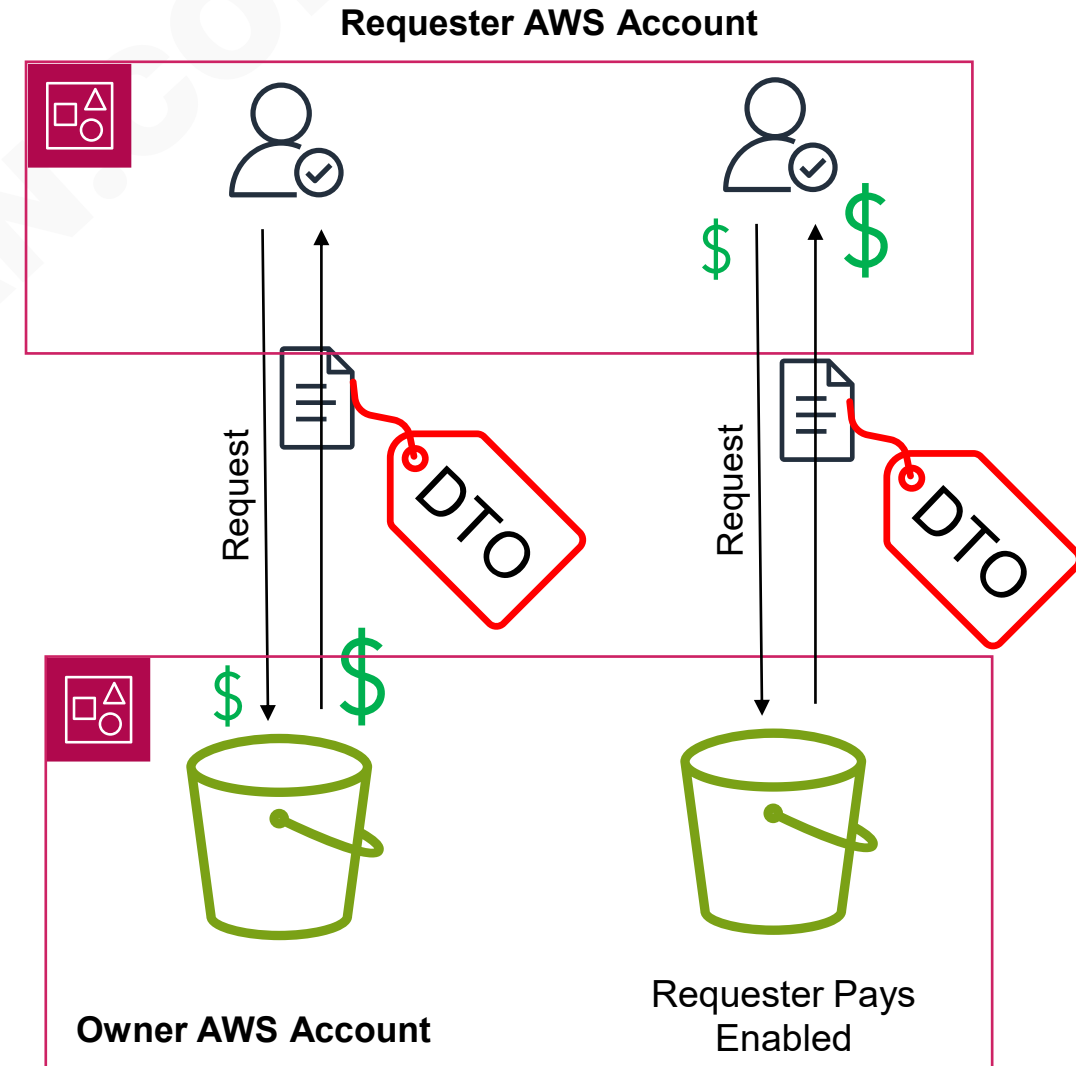
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::<your bucket name>/*"
    }
  ]
}
```

- 8 Now access website using following URL. You should be able to access it.

**<bucket-name>.s3-website.<AWS-region>.amazonaws.com**

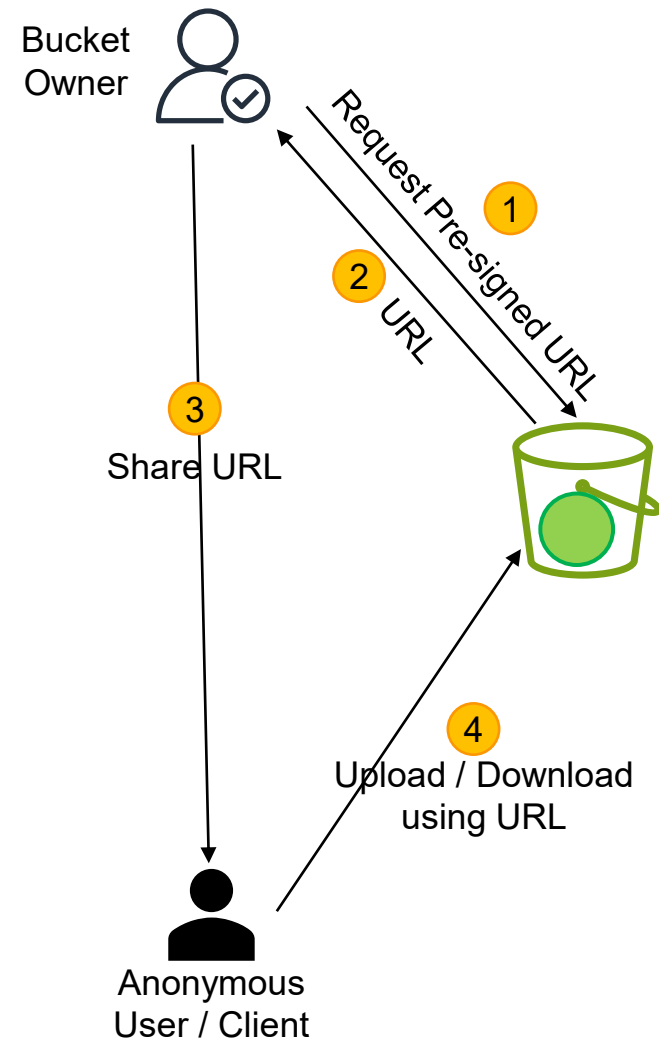
# Requester Pays

- A Feature that can be enabled for S3 Buckets
- Moves data access costs (GET, HEAD, LIST requests) and data transfer cost (DTO) from the bucket owner to the requester.
- Bucket owner still pays for S3 storage cost
- Use cases:
  - When you host public datasets or large shared files where many external users download content.
  - Often used by organizations sharing research data, logs, public archives, or community datasets.
  - Helps prevent unwanted or extremely high egress charges for open-access datasets.
- Requesters must use:
  - Header: `x-amz-request-payer: requester` (when using APIs)
  - Parameter: `--request-payer requester` (when using CLI)
- Works only when requesters have AWS accounts.



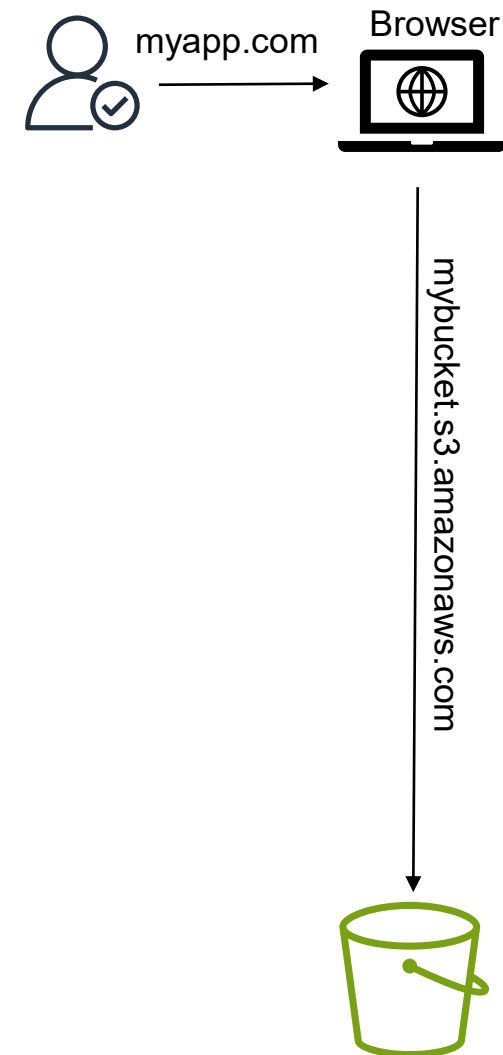
# S3 Pre-signed URLs

- Allows temporary access to an S3 object without giving users AWS credentials.
- A pre-signed URL is generated using creators' IAM credentials. If the creator cannot access/upload object, the URL will not work..
- Supports GET (download) and PUT (upload) operations
- Can be generated using AWS console or AWS CLI
- Use cases:
  - User wants to download an invoice PDF from a private S3 bucket.
  - A mobile app uploads profile pictures directly to S3.
  - A course instructor shares a file with students using a link that expires in 1 hour.
- **Important considerations:**
  - Anyone with the URL can access the object until the URL expires
  - If the IAM principal loses permissions after generating the URL, URL continues to work until expiry.

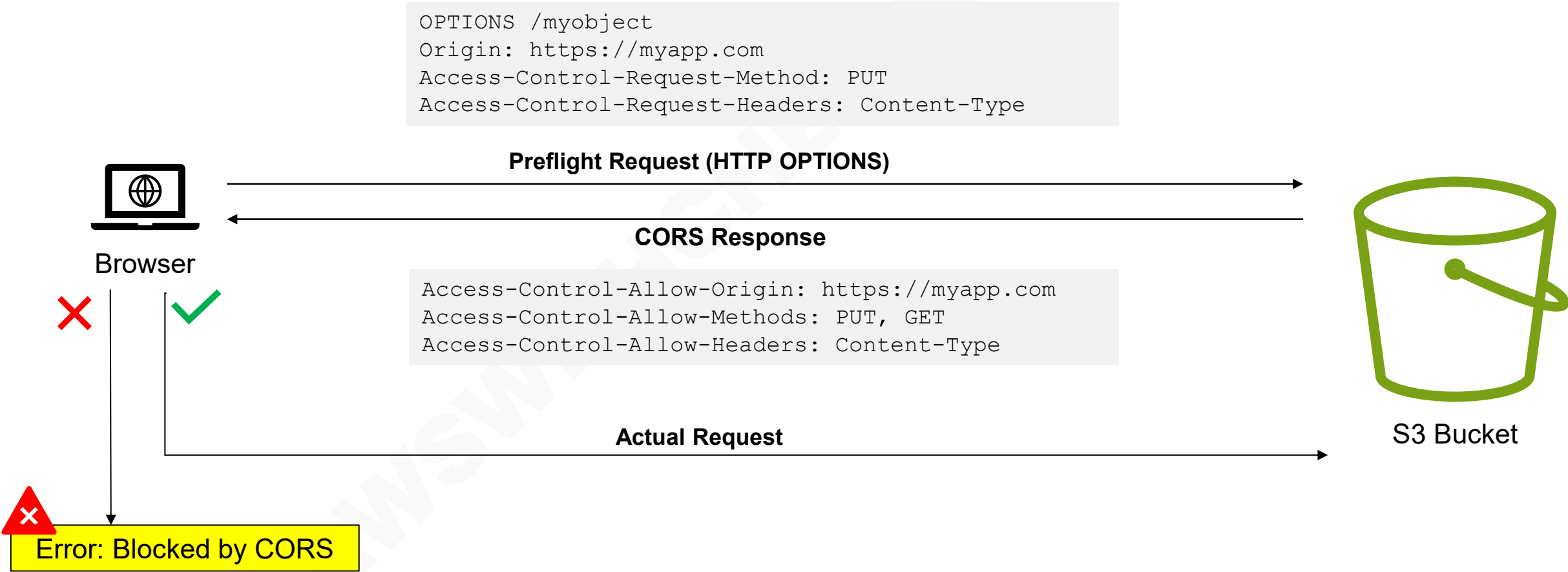


# CORS (Cross-Origin Resource sharing)

- CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls which websites are allowed to make requests to your domain.
- Browsers enforce a rule called Same-Origin Policy.
- A "cross-origin" request means a web page from one domain tries to access resources from another domain.
  - Example: Website <https://myapp.com> calling <https://mybucket.s3.amazonaws.com>
- Flow of the events:
  1. If browser see cross-origin request, it first sends Pre-flight (HTTP OPTIONS) request to check whether it is allowed to make cross-origin request to S3 bucket
  2. S3 checks CORS rules in the bucket to check if Origin and HTTP Method is allowed
  3. S3 returns CORS response
  4. If allowed, browser then sends actual request (GET/PUT/DELETE etc.)



# How it works?





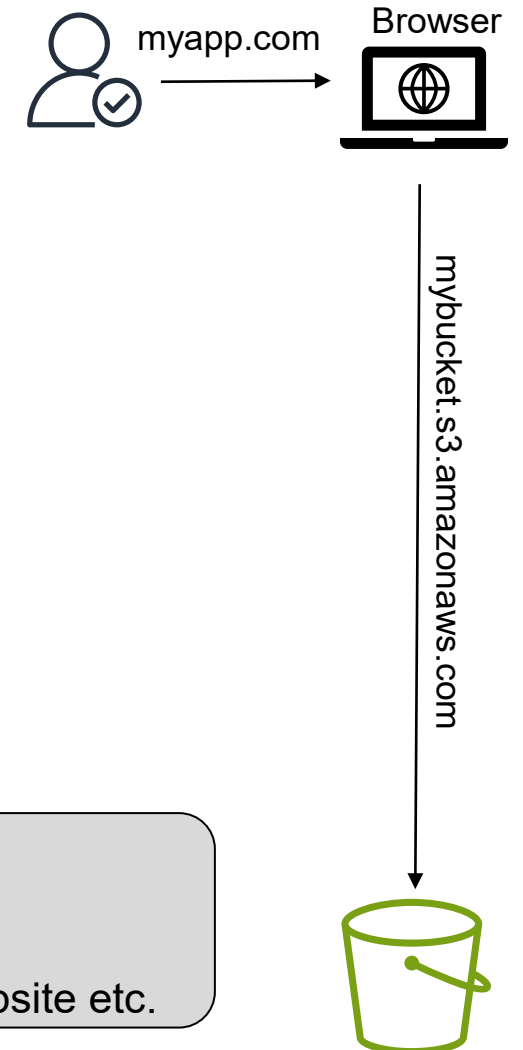
# CORS with S3

- S3 Bucket should allow correct CORS including Origin name, methods and headers
- We can use \* as a domain name to allow all origins
- Sample S3 CORS configuration

```
<CORSConfiguration>
  <CORSRule>
    <AllowedOrigin>https://myapp.com</AllowedOrigin>
    <AllowedMethod>GET</AllowedMethod>
    <AllowedMethod>PUT</AllowedMethod>
    <AllowedHeader>*</AllowedHeader>
  </CORSRule>
</CORSConfiguration>
```

**Note:**

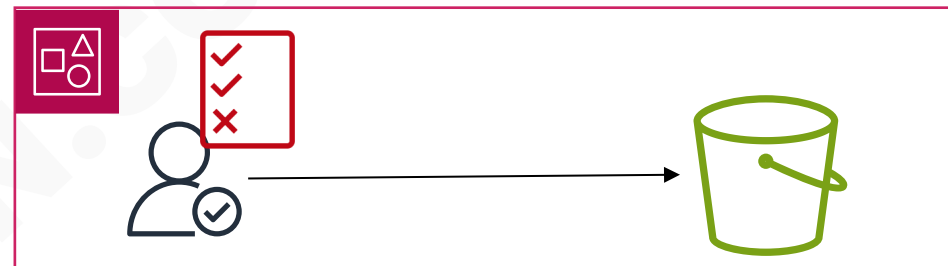
- CORS is browser only feature and hence not applicable when accessing S3 using cURL, postman, CLI, SDK, Lambda, APIs etc.
- Applicable when using Java Script, Presigned URL from browser, Accessing S3 static website etc.



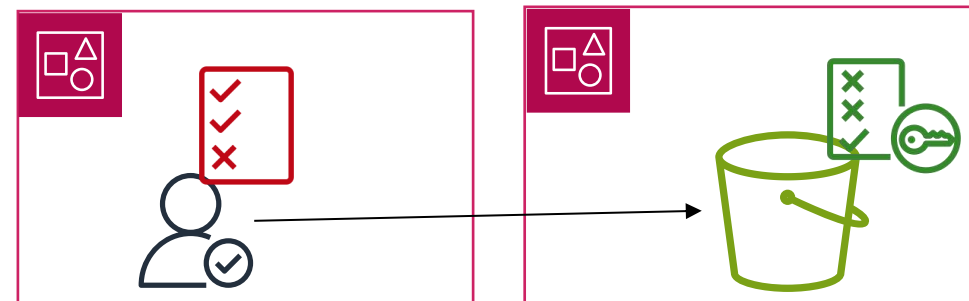
# S3 Access Points

- S3 Access point makes it simple to manage access at scale for applications using shared datasets
- Using Bucket policy for managing access to thousands of application with different level of access is complex
- Create multiple Access points per S3 bucket
  - Customized path into a bucket (prefix)
  - Unique hostname per Access Point
  - Access policy per Access point
  - Network controls (Internet or VPC)

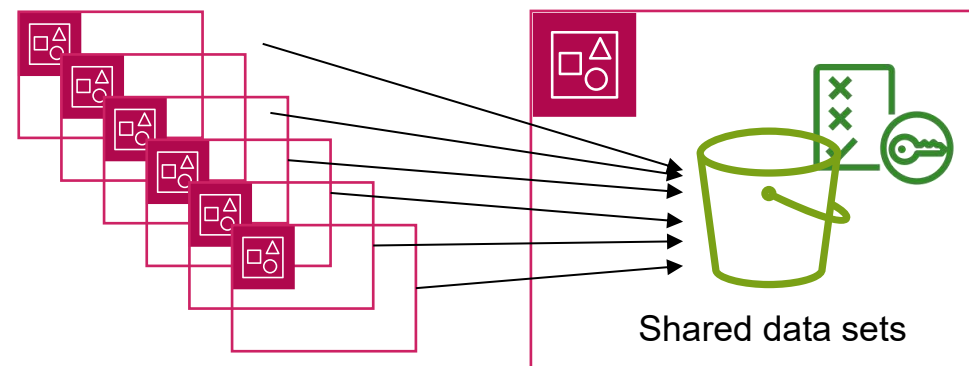
Same Account



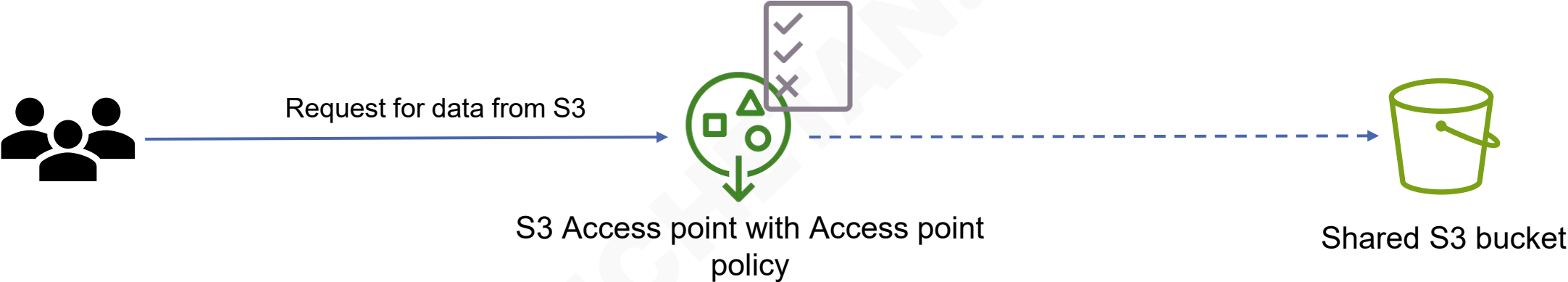
Cross-account



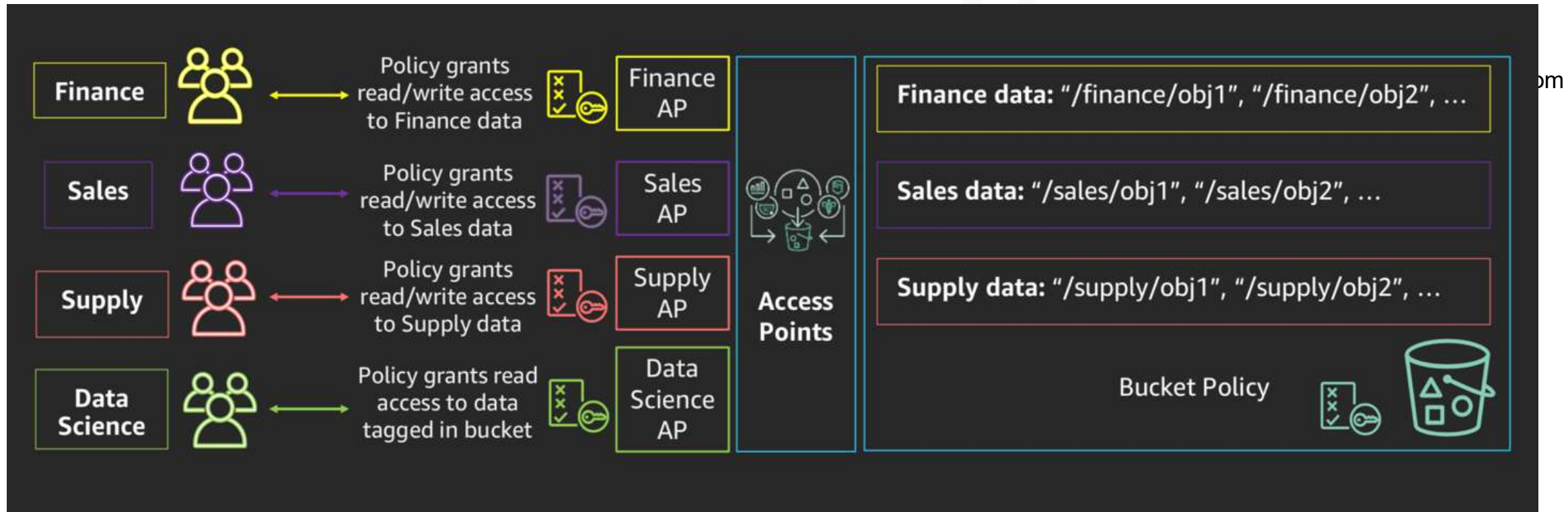
Multiple apps requiring access



# S3 Access Points



# S3 Access Points - Example



## Example Access point ARN

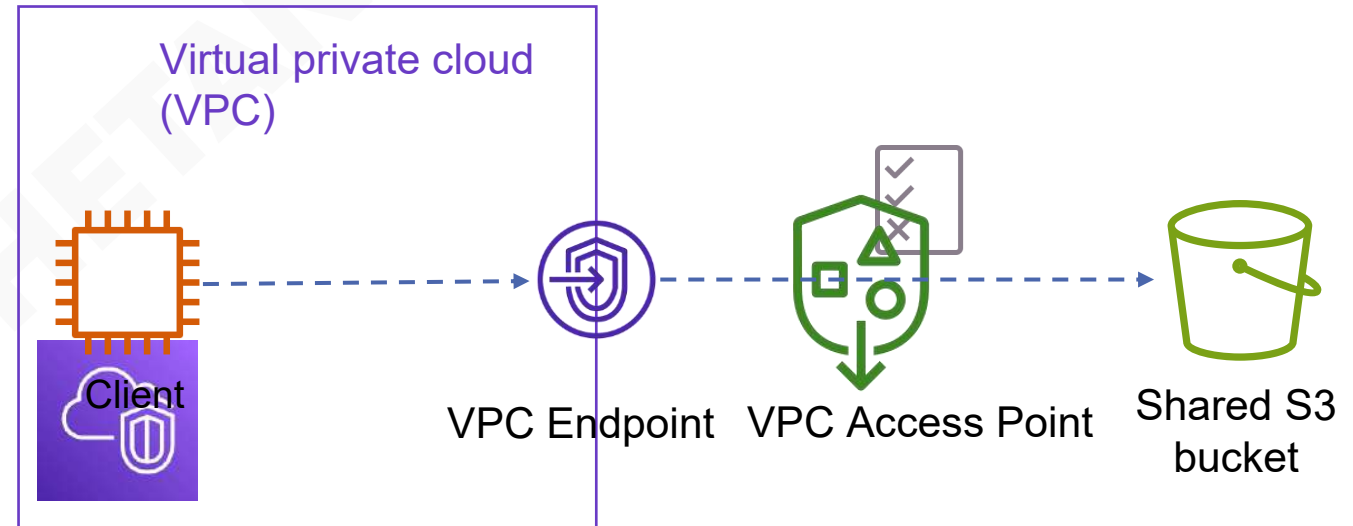
```
arn:aws:s3:us-east-1:123456789012:accesspoint/finance
arn:aws:s3:us-east-1:123456789012:accesspoint/sales
arn:aws:s3:us-east-1:123456789012:accesspoint/datascience
```

# S3 Access Points – Example policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowFinanceTeamRead",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/FinanceTeamRole"
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:us-east-1:123456789012:accesspoint/finance/object/*"
    }
  ]
}
```

# VPC only Access point

- You can configure an S3 Access Point so it is reachable only from a specific VPC.
- A VPC Endpoint (Gateway or Interface) is required for applications in that VPC to access the Access Point.
- The VPC Endpoint Policy must explicitly allow access to both the S3 bucket and the Access Point ARN.



# VPC only Access point – VPC endpoint policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAccessToBucketAndAccessPoint",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<account-id>:role/<iam-role-or-user>"
      },
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::<bucket-name>",
        "arn:aws:s3:::<bucket-name>/*",
        "arn:aws:s3:<region>:<account-id>:accesspoint/<access-point-name>",
        "arn:aws:s3:<region>:<account-id>:accesspoint/<access-point-name>/object/*"
      ]
    }
  ]
}
```



# AWS Container

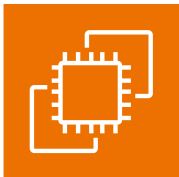
Amazon ECS, Amazon EKS and AWS Fargate

# services



# AWS compute choices

## Virtual Machines



EC2

## Containers

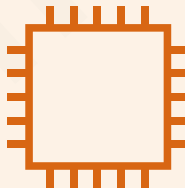


Elastic Container Service (ECS)

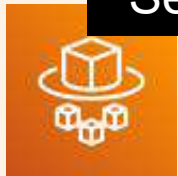


Elastic Kubernetes Service (EKS)

Data plane



EC2



Fargate

Serverless

## Functions

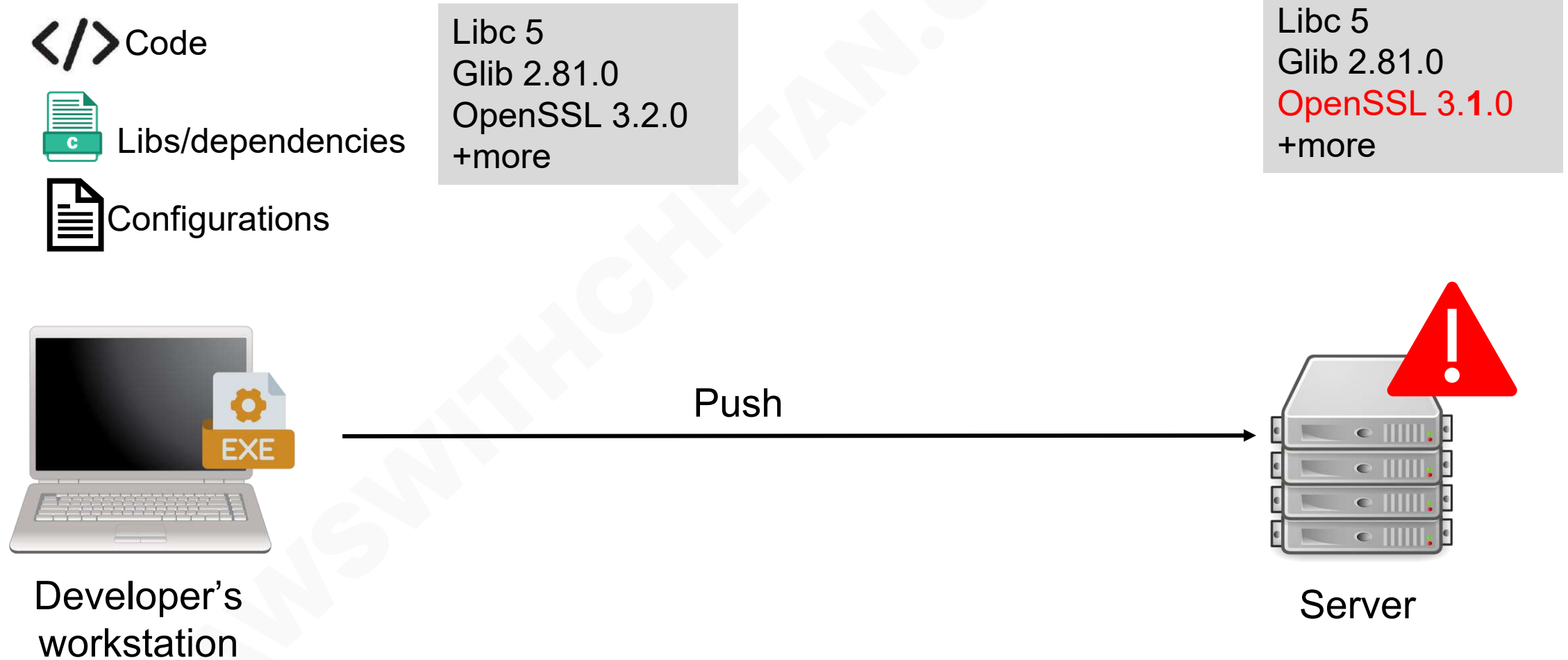


Lambda

Serverless

# Containers

# Why containers?



# Why containers?

 Code

 Libs/dependencies

 Configurations



Developer's  
workstation



Push



Server



# Containers

*Lightweight virtualization that allows to run applications and their dependencies in resource-isolated processes*

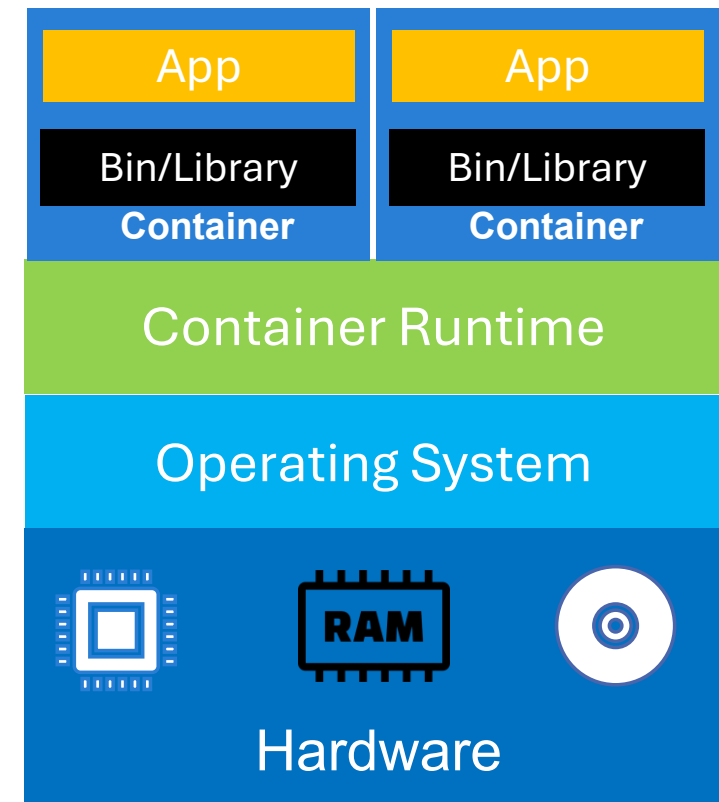
## Key benefits

**Isolated** - Isolated filesystem, process space, network, environment

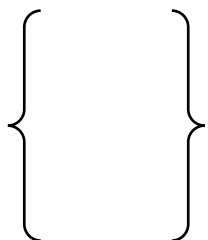
**Lightweight** - No full OS overhead, minimal kernel, fast

**Consistency** – Contains all dependencies, same everywhere

**Portability** – Can be easily packaged, shipped and run on any platform (local machine, server, cloud)

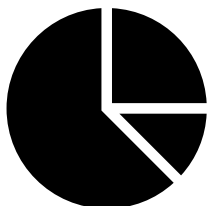


# What makes containers work?



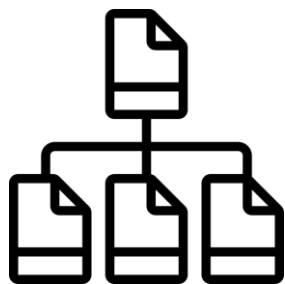
Namespaces

Provide isolation for running applications by limiting what they can see and access.



Control Groups  
(cgroups)

Limit and monitor the resource usage (CPU, memory, I/O) of containers.



Union Filesystem

Enables the layering of filesystems, allowing containers to share common files and efficiently manage disk space.

## Container Engines



docker



cri-o



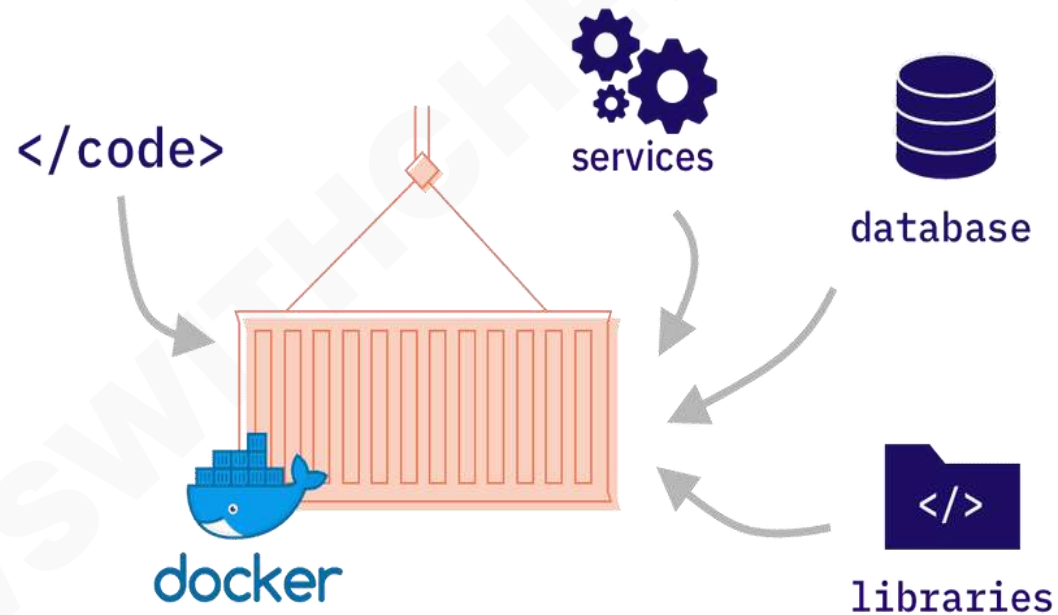
rkt



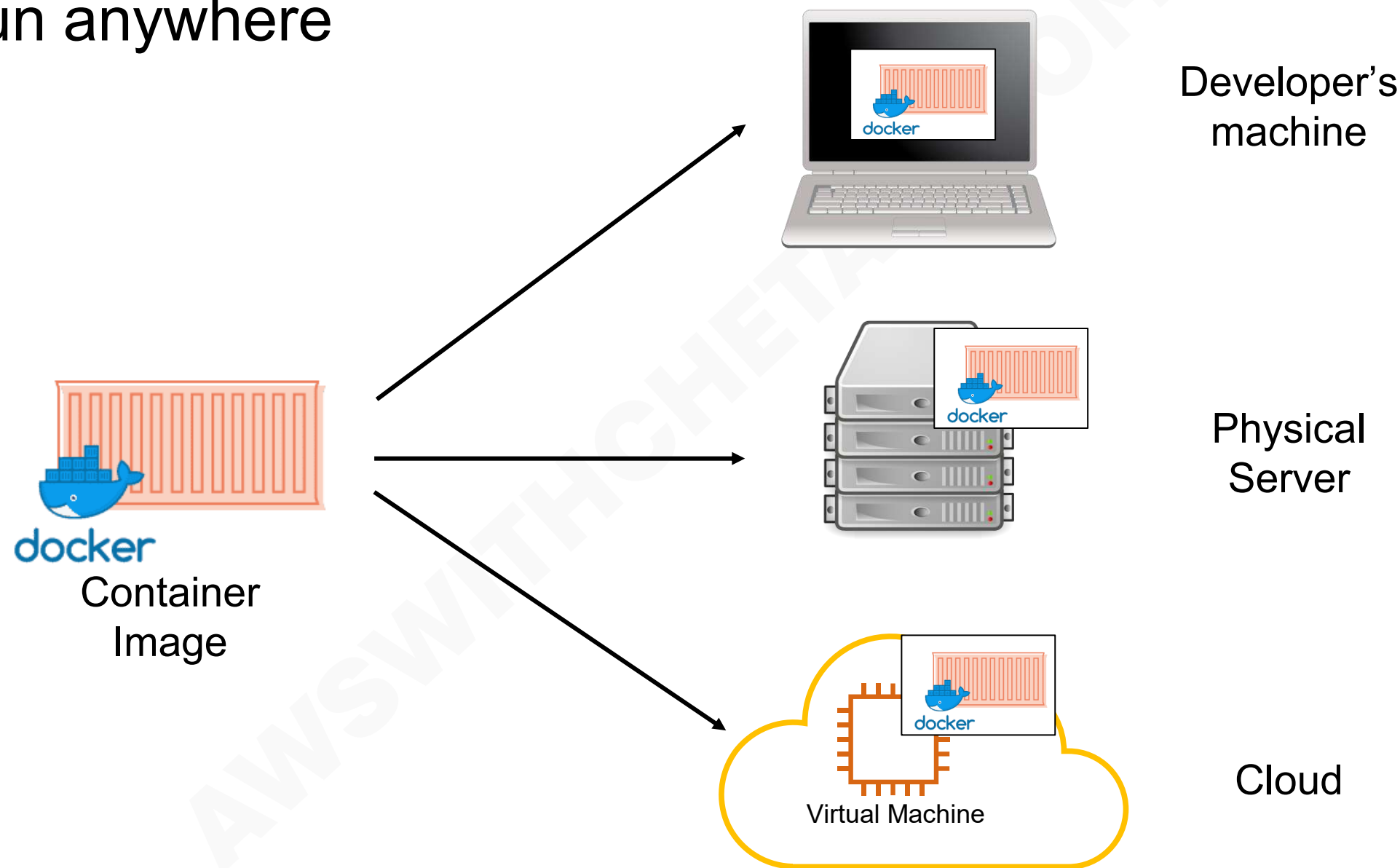
podman

# What is docker?

Docker is an open platform for developing, shipping, and running applications as containers



# Run anywhere



Developer's machine

Physical Server

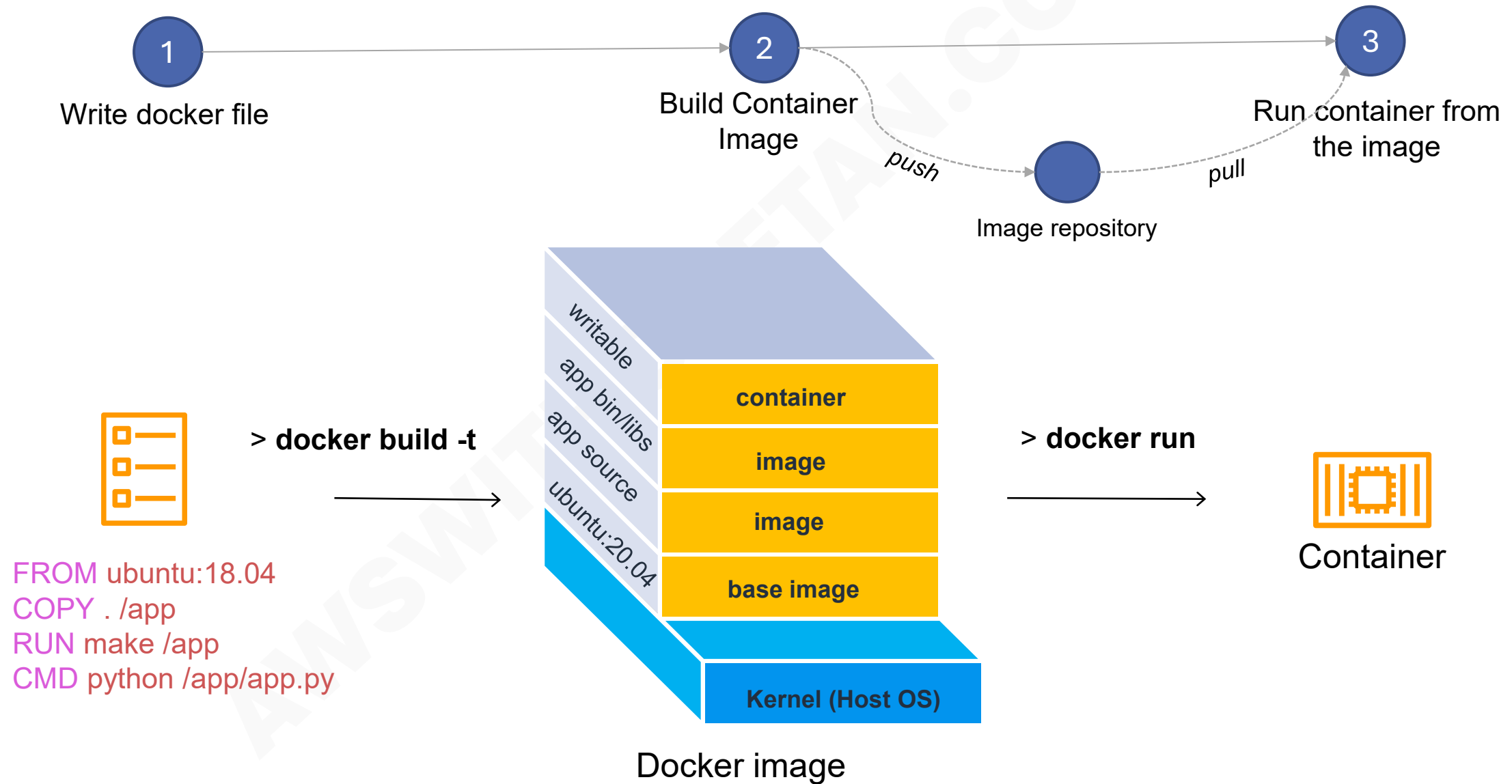
Cloud

Virtual Machine

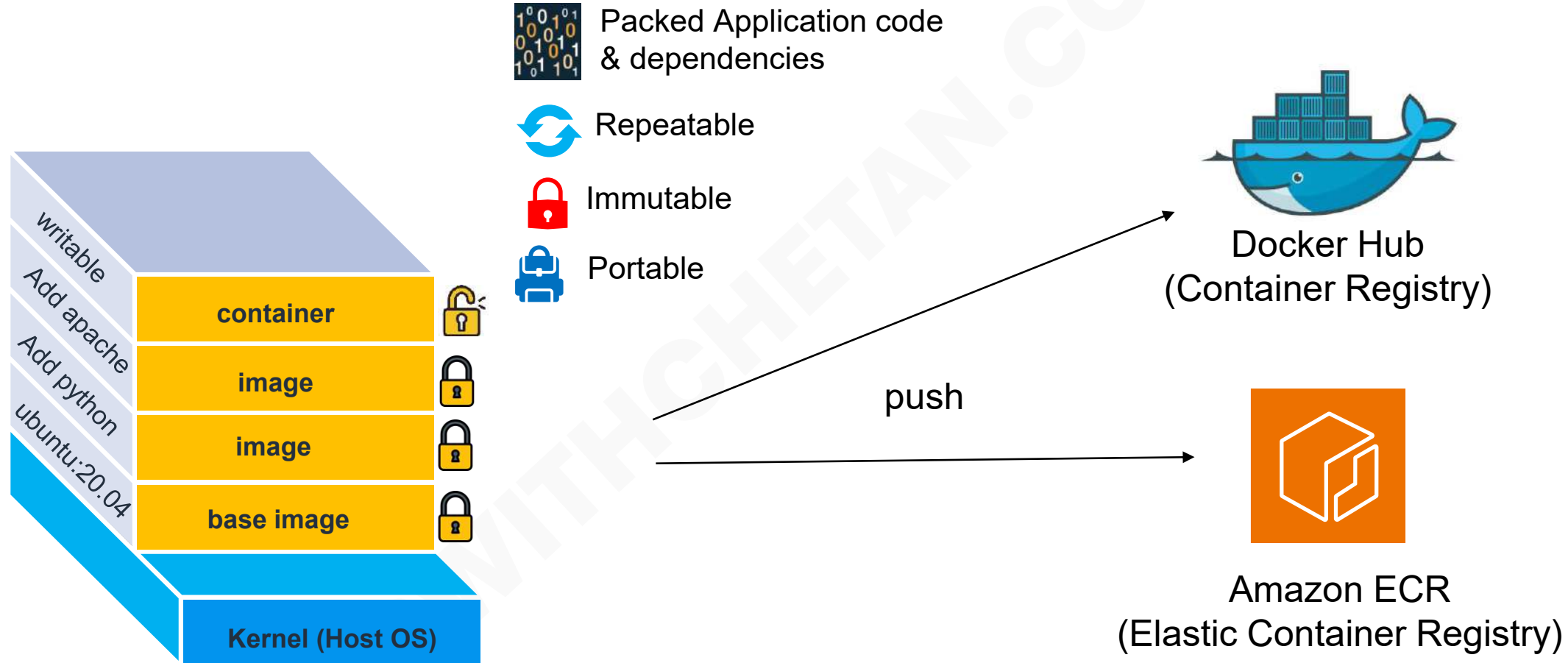
Container Image



# How to use docker?



# Docker Image



# Exercise – Run docker container on EC2 instance

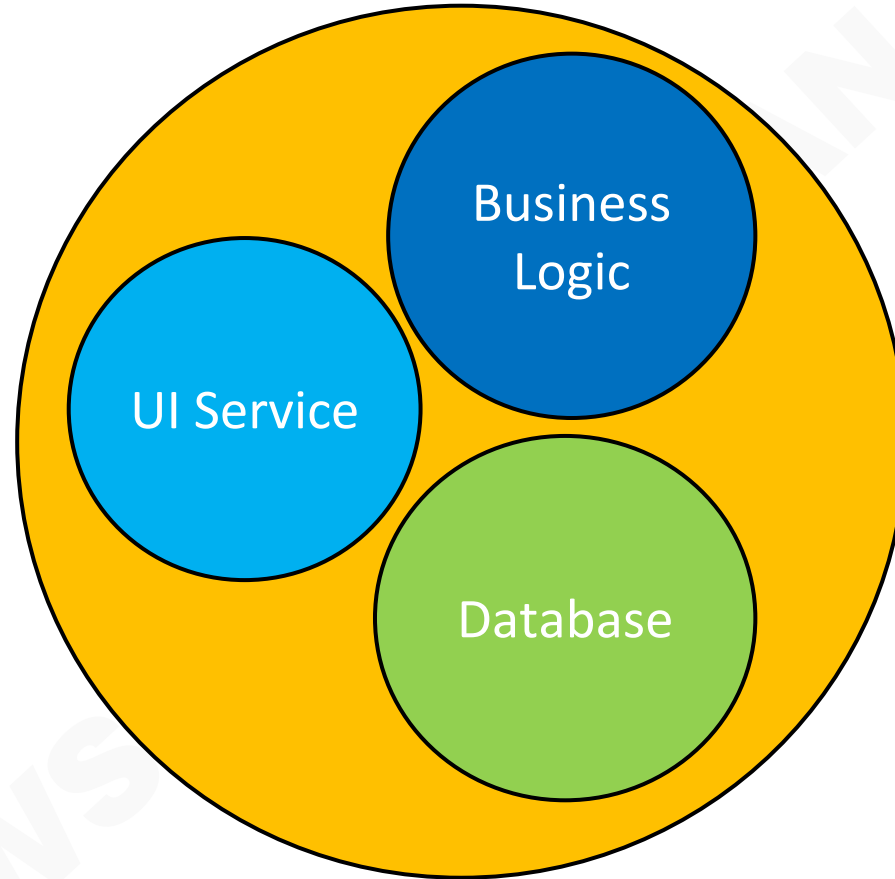
Run a docker container for a simple web server on an EC2 Linux instance

1. **Install Docker engine:** `sudo yum install docker -y`
2. **Start Docker daemon** `sudo service docker start`
3. **Create sample index.html file locally**
4. **Create Dockerfile:**

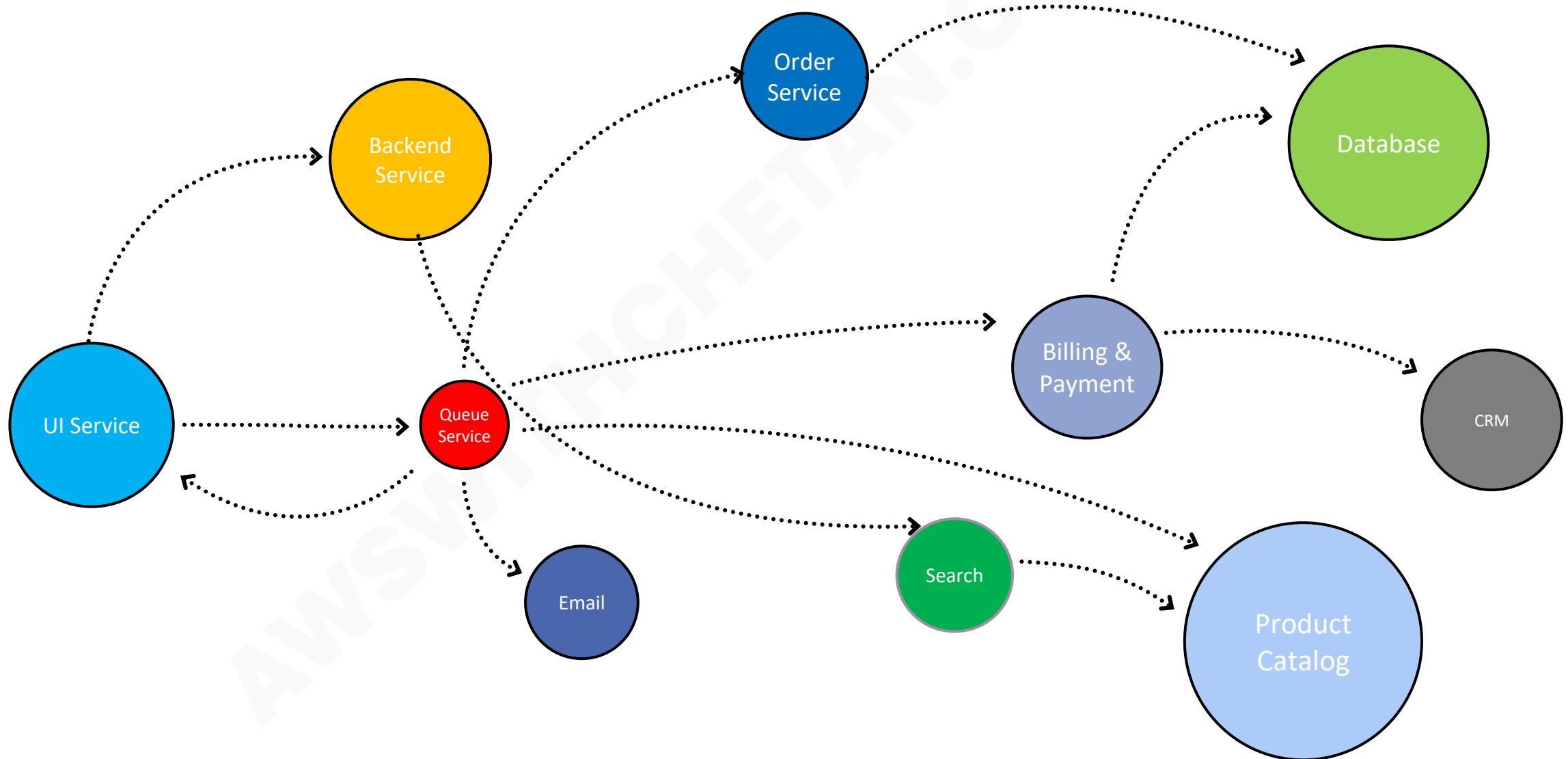
```
FROM httpd
COPY index.html /usr/local/apache2/htdocs/
```
5. **Create image:** `sudo docker build -t mywebserver .`
6. **Run container:** `sudo docker run --name my-web-server -p 80:80 -d mywebserver`
7. **Access webserver over EC2 Public IP**

# Containers Management and Orchestration

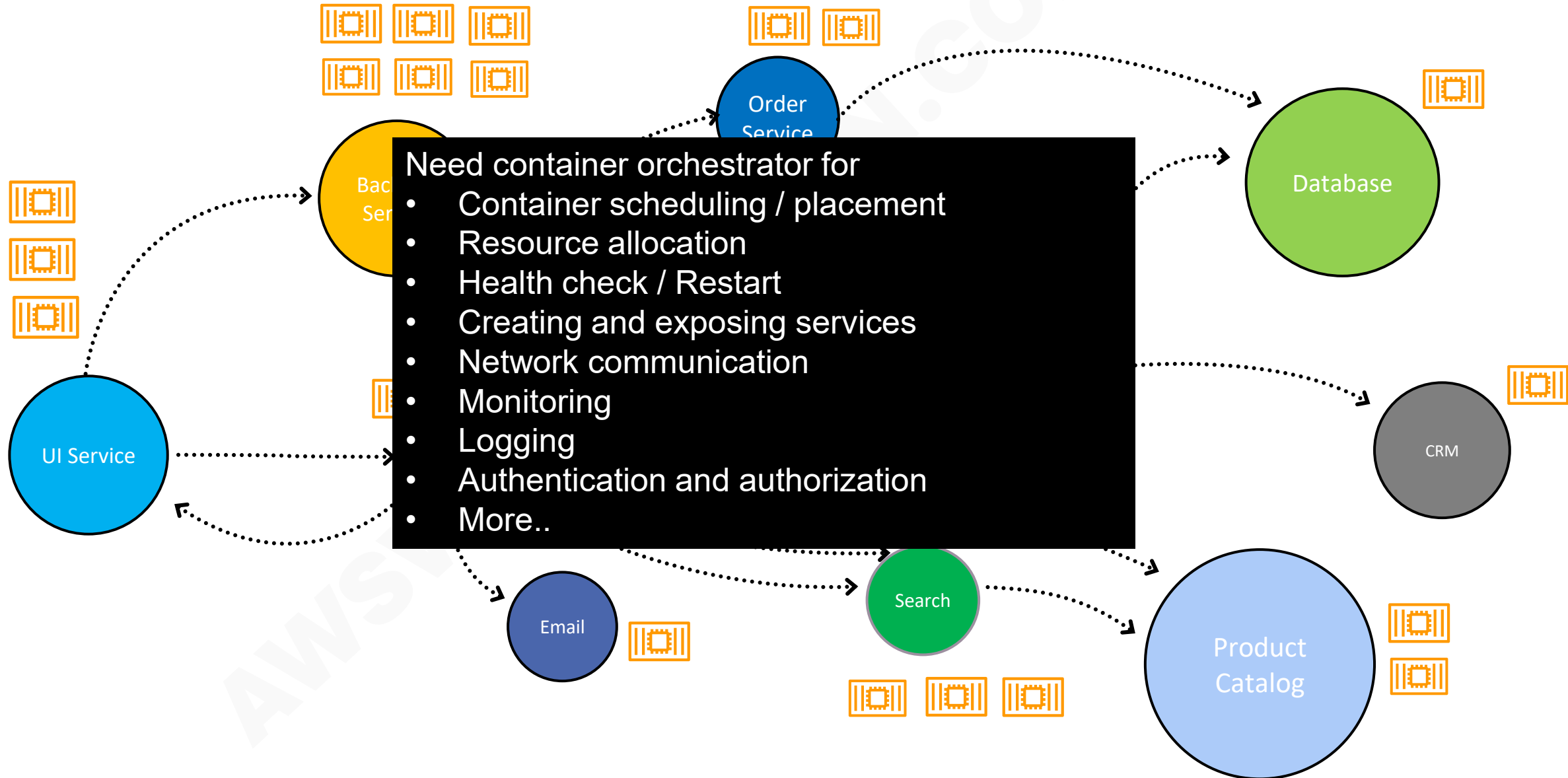
# Monolith application architecture



# Monolith application architecture



# Monolith application architecture



# What is orchestration?



Control Plane

Data Plane





# Opensource Container Orchestration Tools



Kubernetes



Docker Swarm

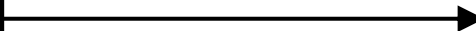
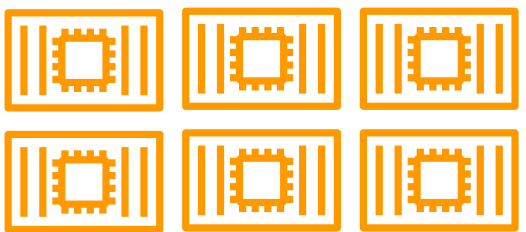


Apache Mesos



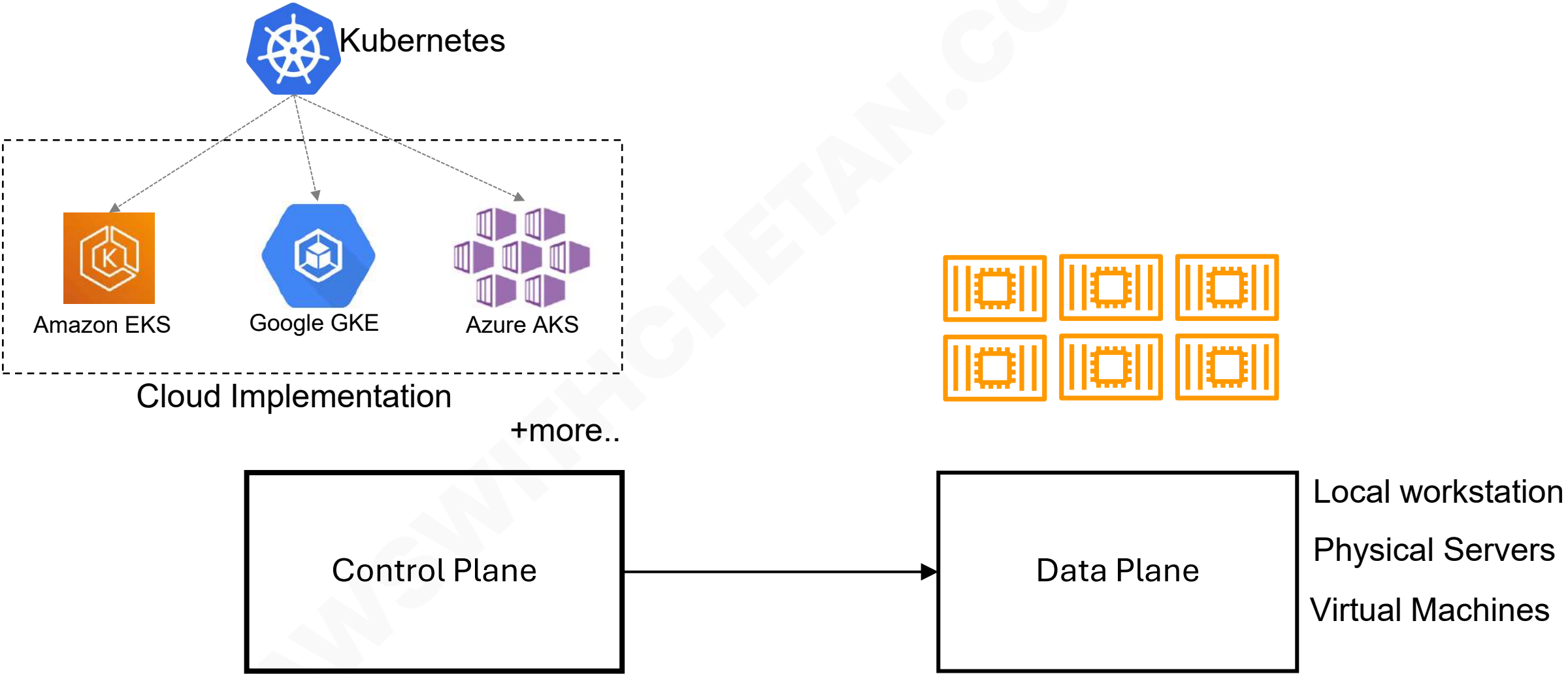
OpenShift

+more..

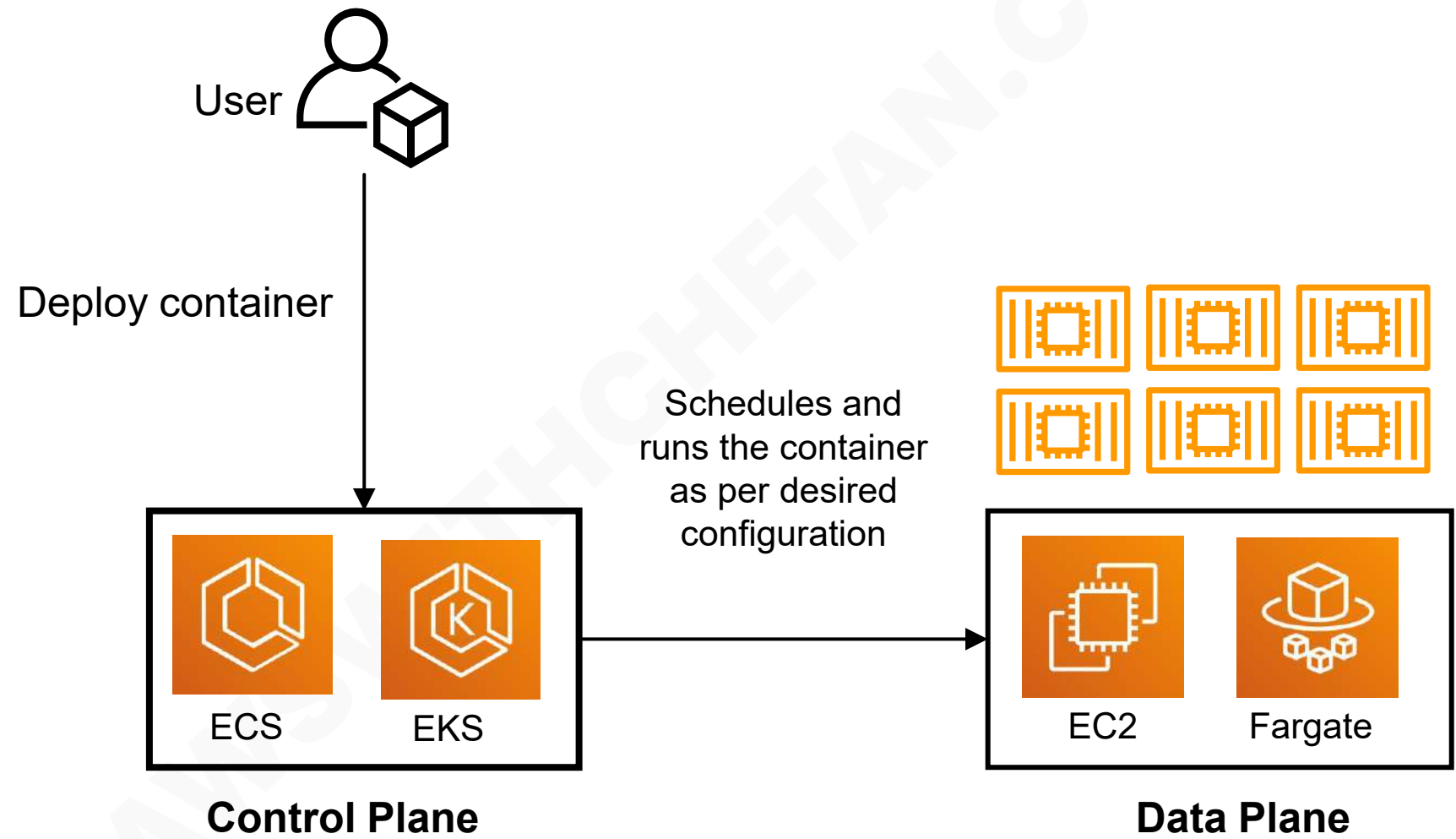


Local workstation  
Physical Servers  
Virtual Machines

# Opensource Container Orchestration Tools



# Container Orchestration services in AWS



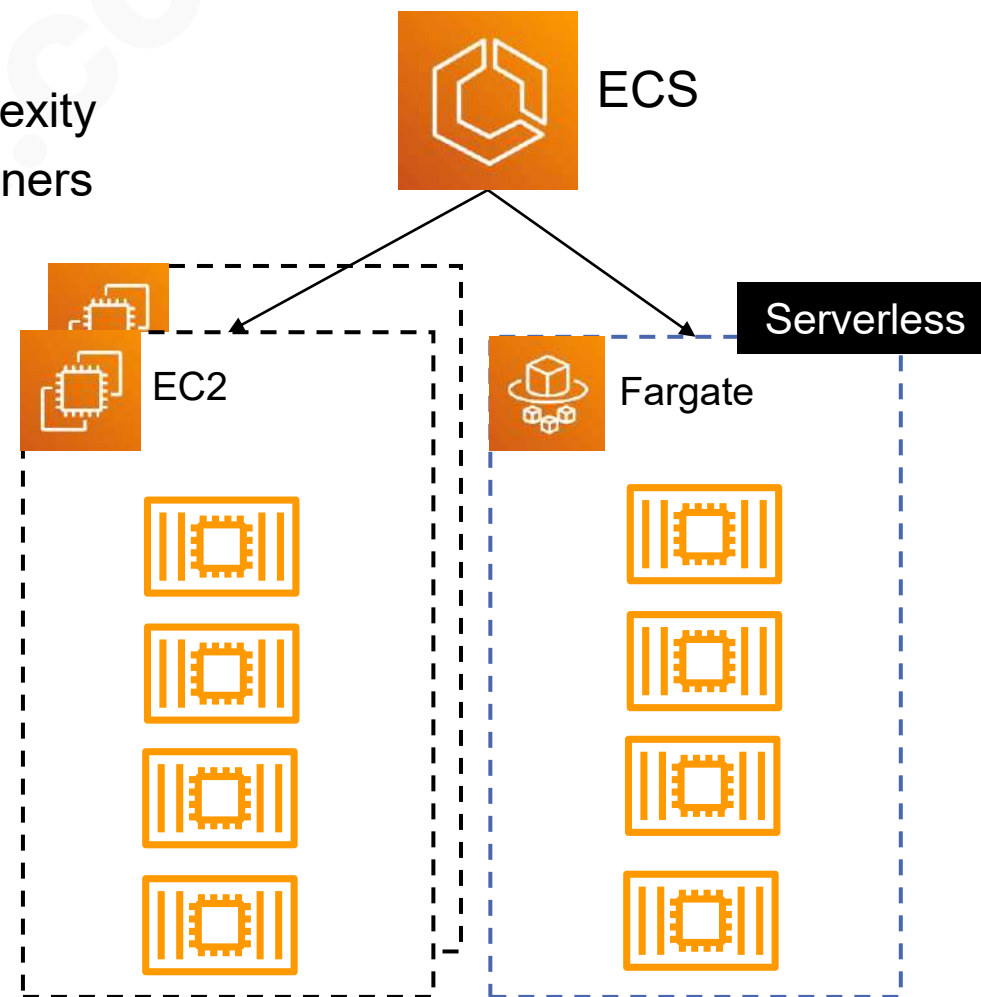


# Amazon ECS

# Amazon ECS

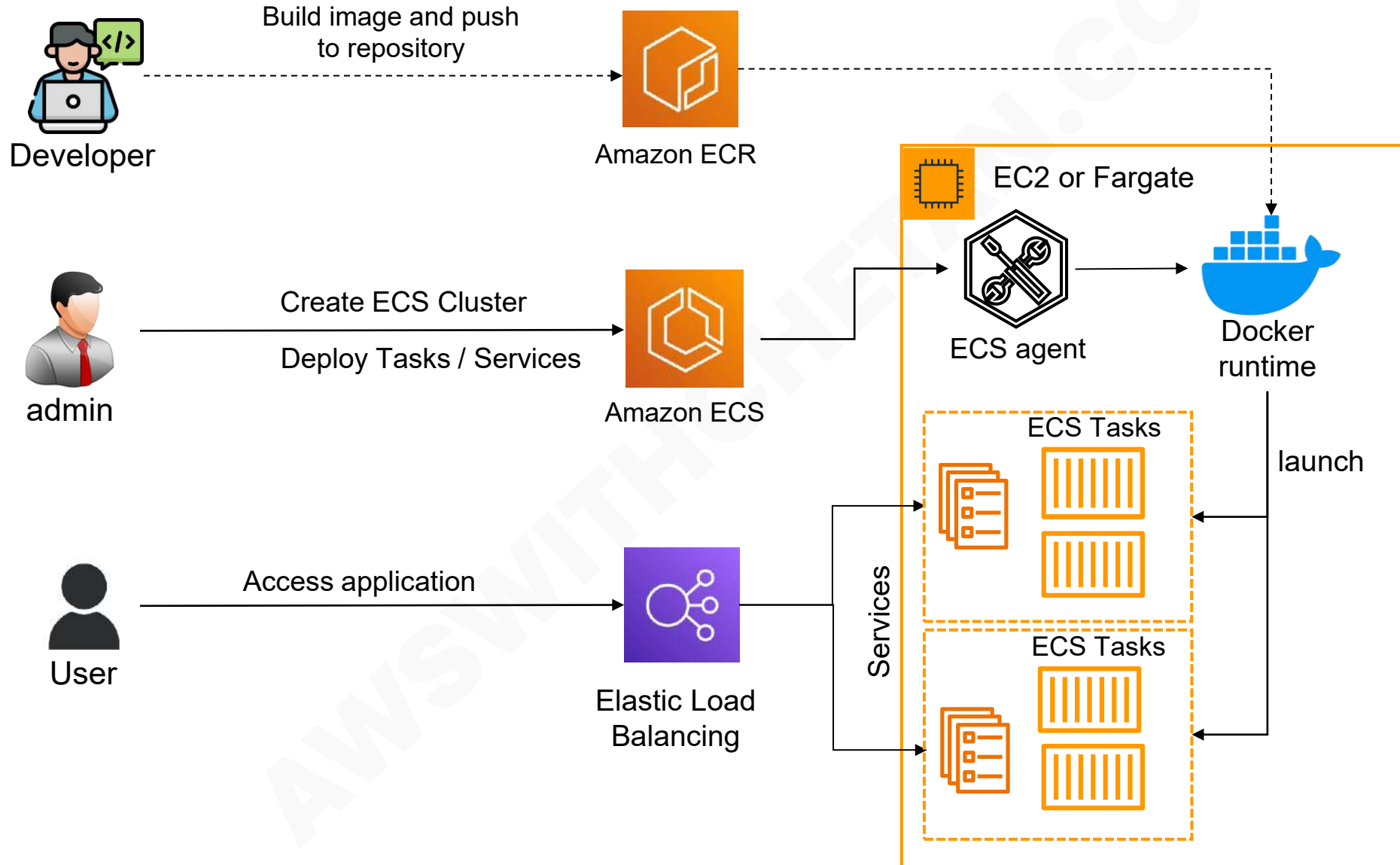
## Elastic Container Service

- Run and manage docker containers at scale without complexity
- Supports 2 different compute options for running the containers
  - EC2 – Customer to manage (Supports Spot instances)
  - Fargate – AWS manages the infrastructure (Serverless)
- ECS integrates with many AWS services
  - ALB and NLB (as application backend service)
  - Amazon API Gateway (as API backend service)
  - Amazon EventBridge (for event-based execution)
  - Amazon CloudWatch (for metrics and logs)
  - Application Auto scaling and EC2 Autoscaling group



ECS is a good choice if you want a simpler, AWS-native container orchestration service with minimal operational overhead.

# How it works?



# ECS Tasks and Services

- **ECS Task:**

- An ECS task is a single running instance of a task definition.
- Uses task definition to define container images, CPU, memory, ports, environment variables and IAM roles to be used.

- **ECS Service:**

- An ECS service runs and maintains a specified number of tasks continuously.
- It automatically restarts failed tasks to maintain the desired count.
- Services integrate with Application or Network Load Balancers.

```
{
  "family": "sample-task",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "taskRoleArn": "arn:aws:iam::112233445566:role/S3ReadTaskRole",
  "containerDefinitions": [
    {
      "name": "app",
      "image": "nginx:latest",
      "essential": true,
      "portMappings": [
        {
          "containerPort": 80
        }
      ]
    }
  ]
}
```

**Task definition**

# ECS Features – Important to know

- IAM Roles for ECS
- Data Store for ECS
- ECS Autoscaling

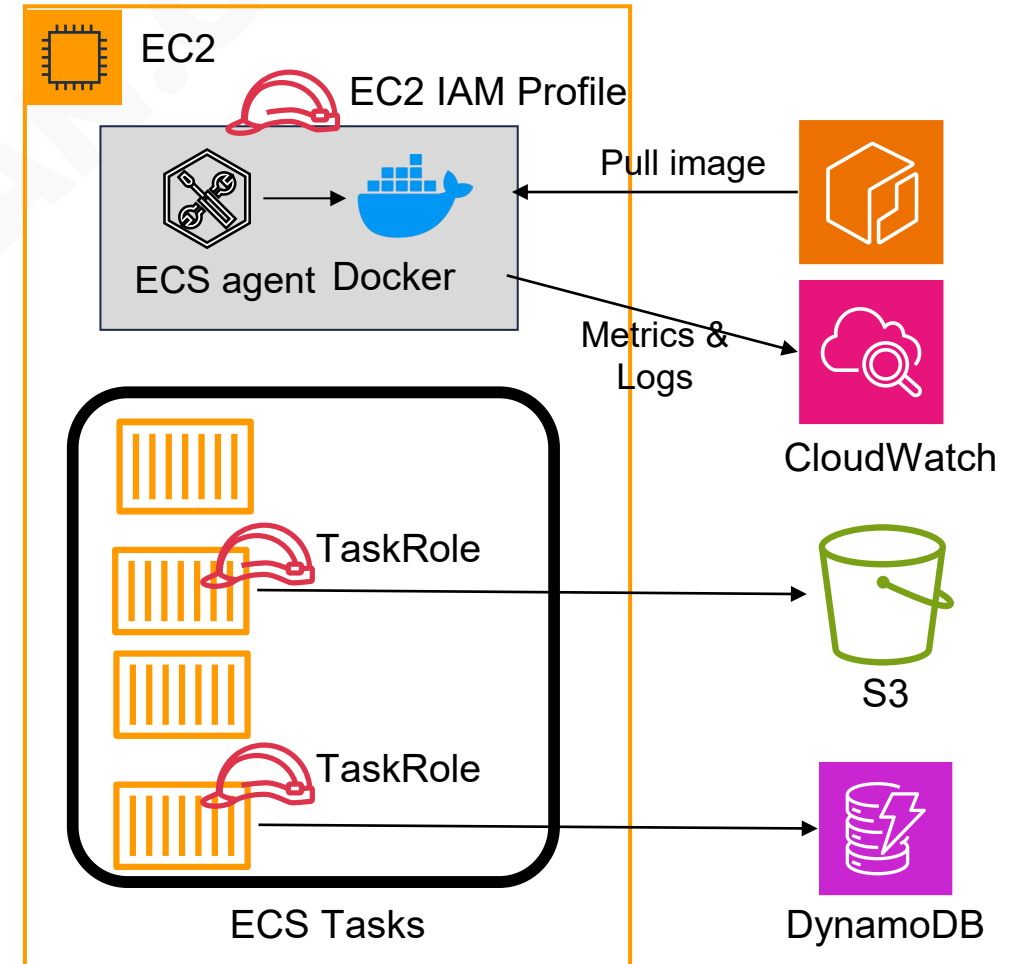
AWSWITHCHETAN.COM



# IAM Roles for ECS

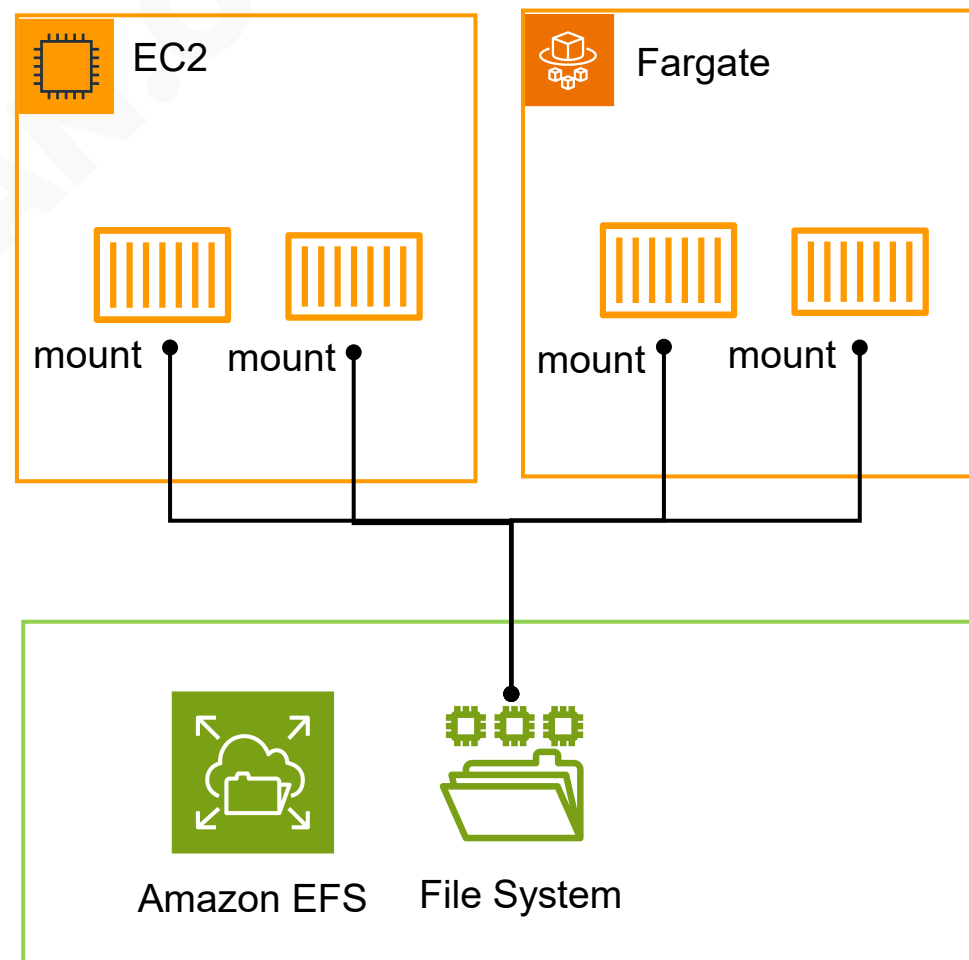
## ECS uses 2 types of IAM roles:

- EC2 Instance Profile (EC2 Launch Type):
  - Used by the ECS agent
  - Send container logs to CloudWatch Logs
  - Pull Docker image from ECR
  - Access secrets from Secrets Manager or parameters from SSM parameter store
- ECS Task Roles:
  - Use different IAM roles for the individual ECS tasks
  - Task Role is defined in the task definition as `taskRoleArn`



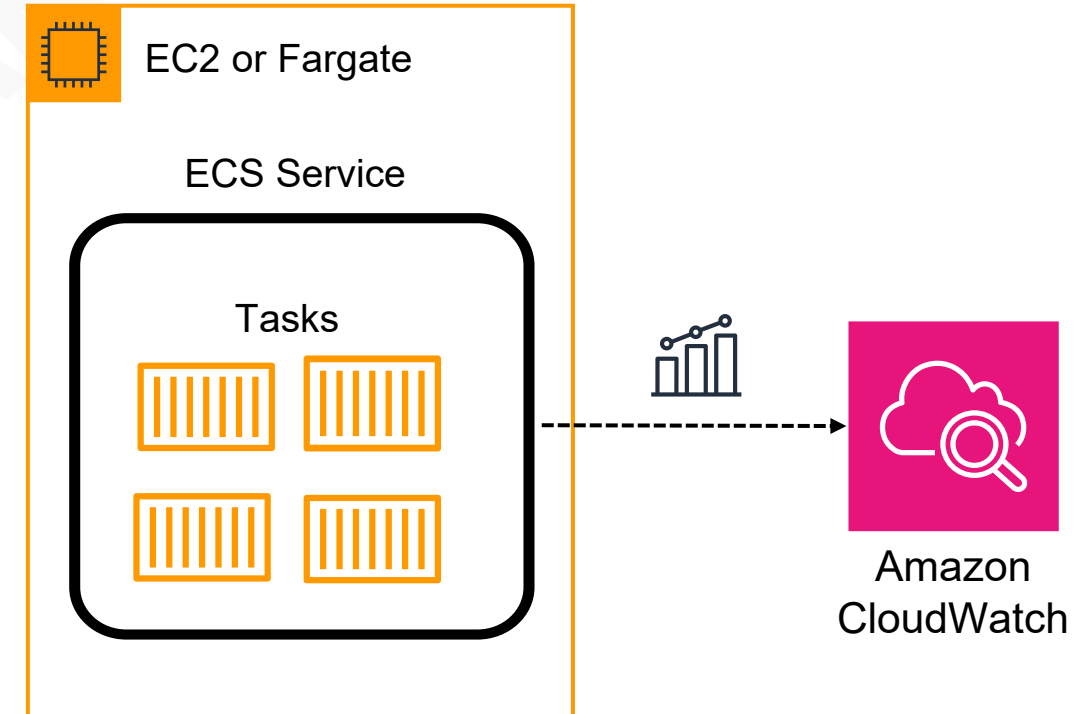
# Persistent data store for ECS tasks

- For persistent shared storage in ECS, it's a good choice to use Amazon EFS.
- Amazon EFS provides a managed linux file system that can be mounted by multiple tasks and is supported on both ECS on EC2 and ECS on Fargate.
- EFS volumes are mounted through the ECS task definition
- EFS supports multi-AZ access, making it suitable for highly available ECS services.
- **Use cases**
  - Applications allowing users to upload files
  - Shared configuration files or application assets
  - Stateful container workloads running on Fargate



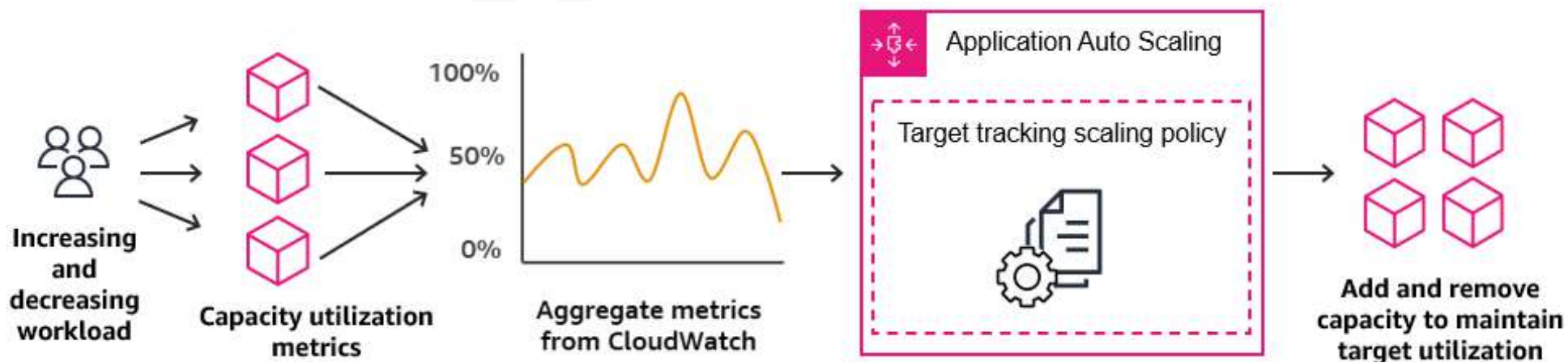
# ECS Service Auto Scaling

- ECS Service Auto scaling uses AWS Application Autoscaling service
- Amazon ECS publishes CloudWatch metrics with your service's average CPU and memory usage
- It scales out by adding more tasks and scales-in by removing running tasks to achieve desired tasks count
- For Fargate launch type, underlying compute is managed by AWS for scaling activities.
- For EC2 launch type, use EC2 Capacity provider which works with EC2 Autoscaling group (ASG) and scales the number of EC2 instances automatically.
- ECS Service Auto scaling supports following types of the scaling policies:
  - Target Tracking, Step scaling, Scheduled scaling and Predictive scaling



# ECS Service Auto Scaling

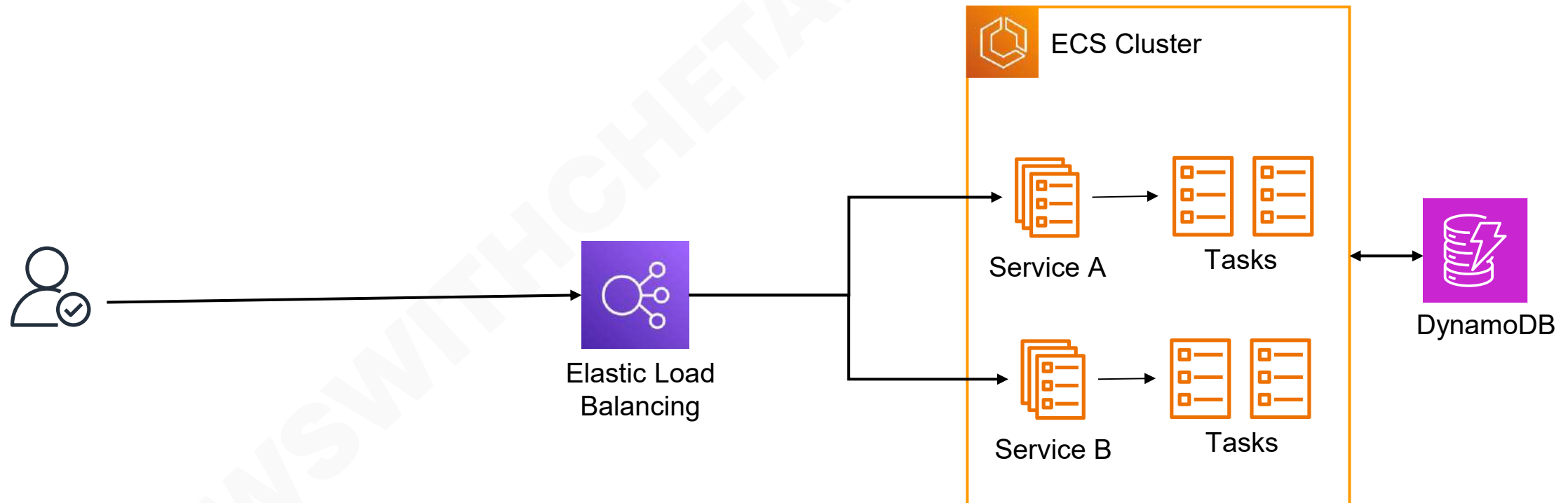
- **Target Tracking:** Automatically adjusts the number of ECS tasks to keep a metric (like CPU at 50%) steady, like how a thermostat maintains room temperature.
- **Step scaling:** Scales ECS tasks in predefined steps based on CloudWatch alarm thresholds, for example adding 1 task if CPU crosses 40% and adding 2 tasks if CPU crosses 60% likewise
- **Scheduled scaling:** Scales ECS tasks at specific times based on a schedule, such as increasing capacity every morning and reducing it at night.
- **Predictive scaling:** Uses historical metrics and ML forecasting to scale ECS tasks in advance of expected traffic spikes. Suitable for cyclic and recurring workloads.



# Let's Talk Architecture - ECS

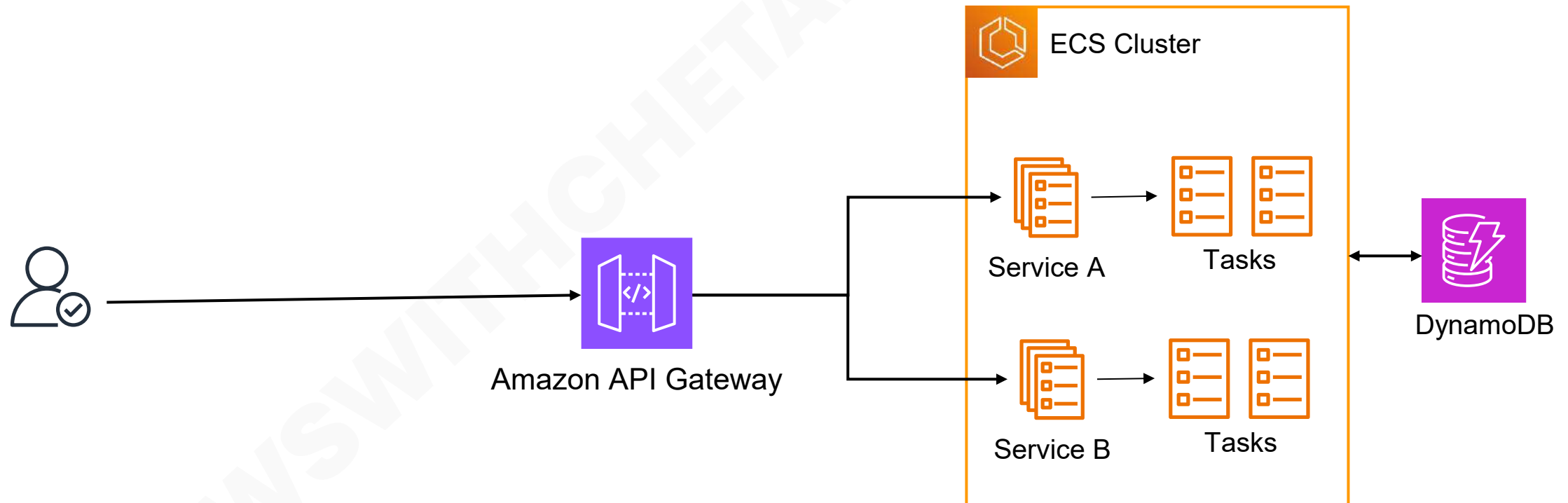
# ECS common architecture patterns

## Hosting Application Backend services



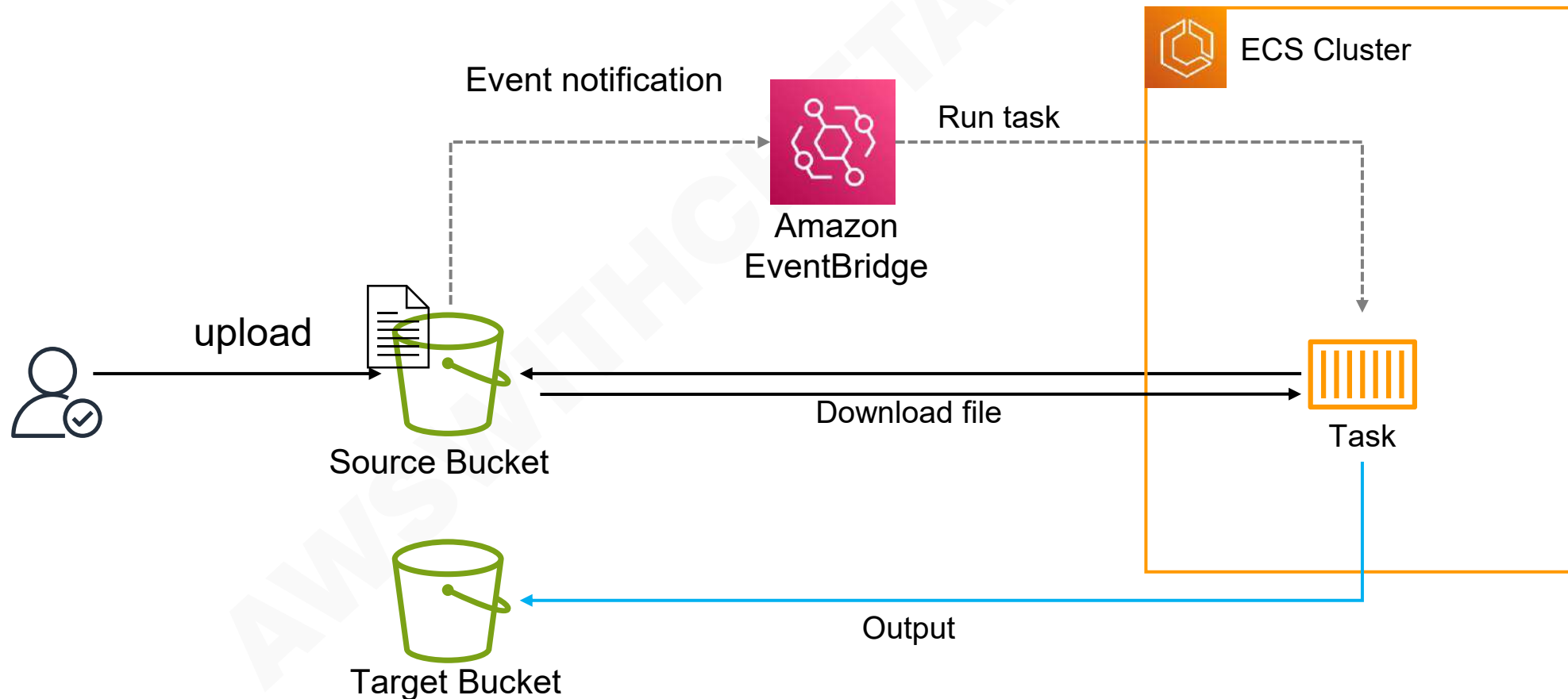
# ECS common architecture patterns

## Hosting API Backend services



# ECS common architecture patterns

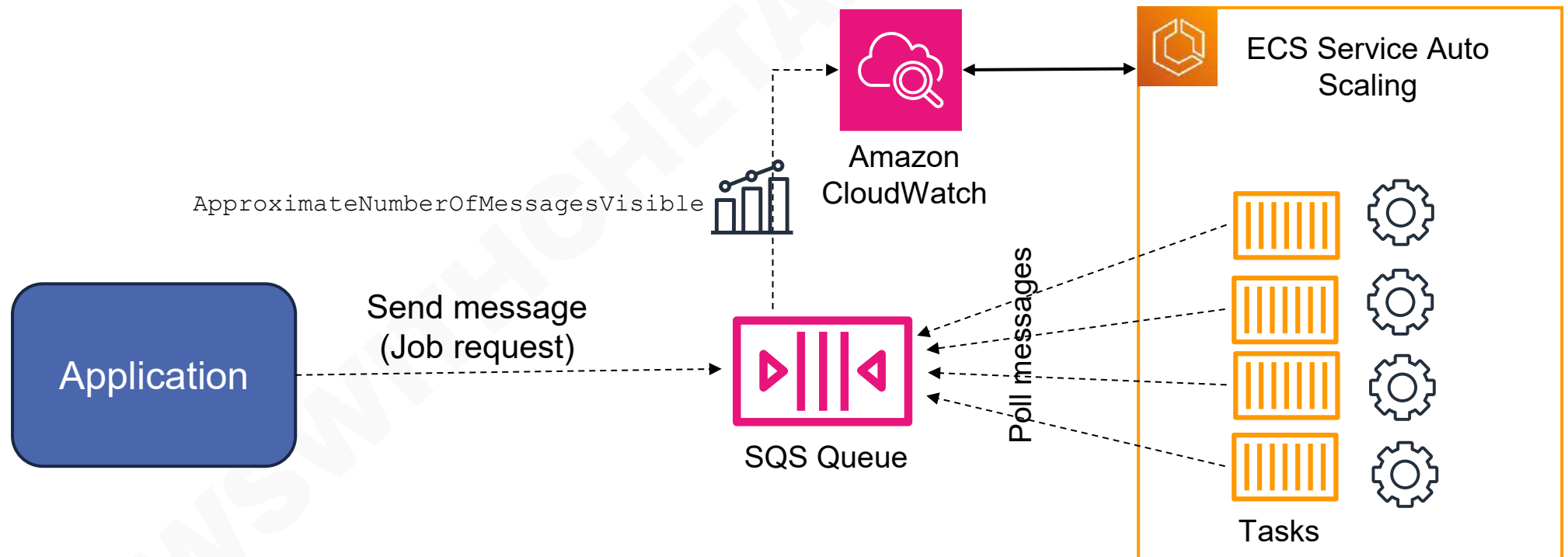
## Event driven architecture for data processing





# ECS common architecture patterns

## Batch job processing and scaling based on SQS message depth



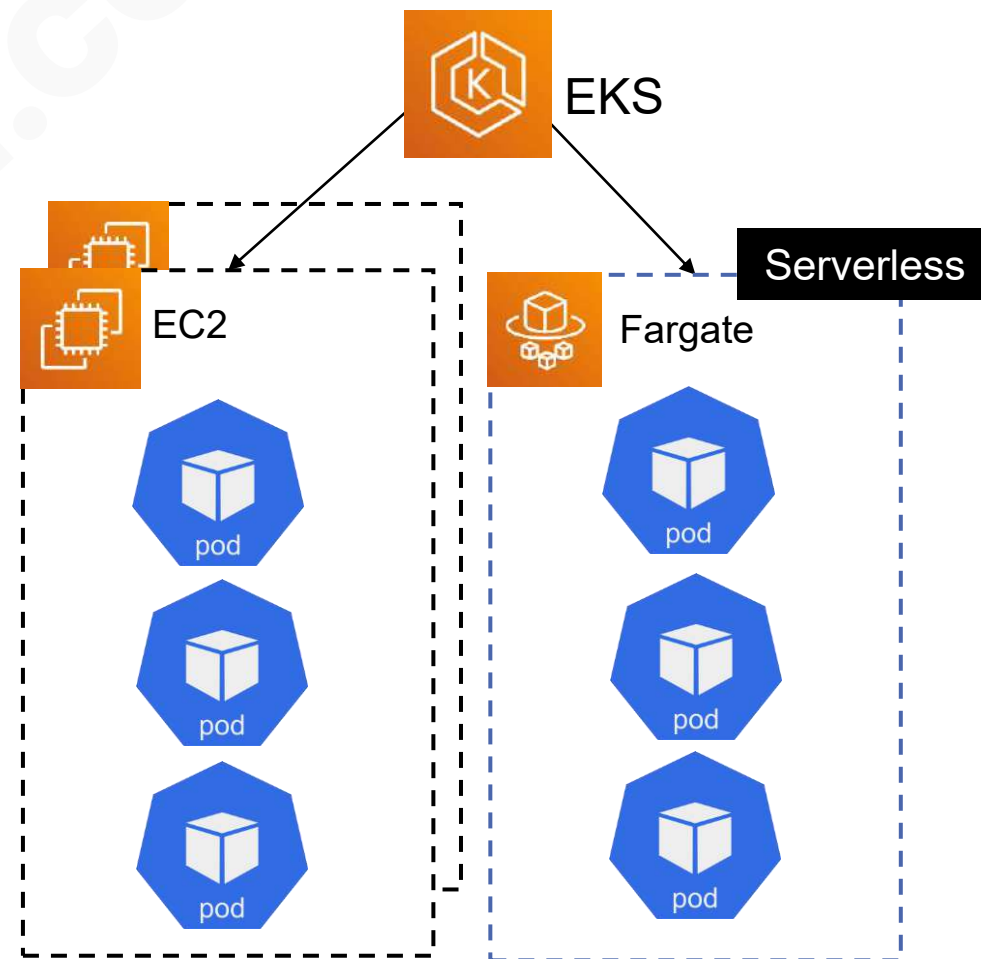


# Amazon EKS

# Amazon EKS

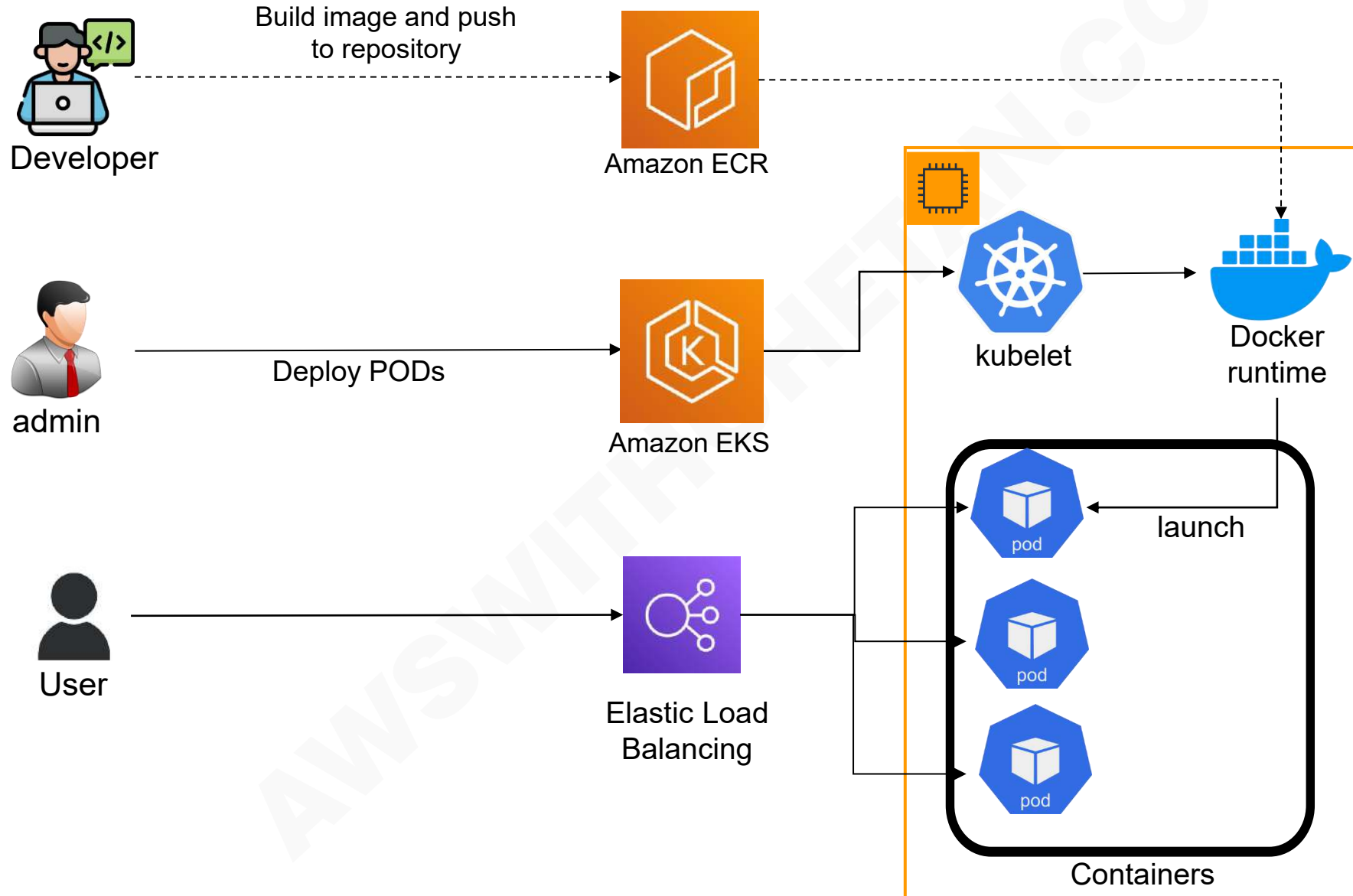
## Elastic Kubernetes Service

- Kubernetes is most popular opensource and broadly used container orchestration platform
- Amazon EKS runs vanilla Kubernetes which is upstream certified conformant version of Kubernetes
- Amazon EKS supports 4 versions of Kubernetes
- Amazon EKS supports 2 different compute options for hosting the PODs (one or more containers)
  - EC2 – Managed node group by EKS
  - Fargate – AWS managed infrastructure
- EKS supports Spot instances as worker nodes in managed or self-managed node groups.



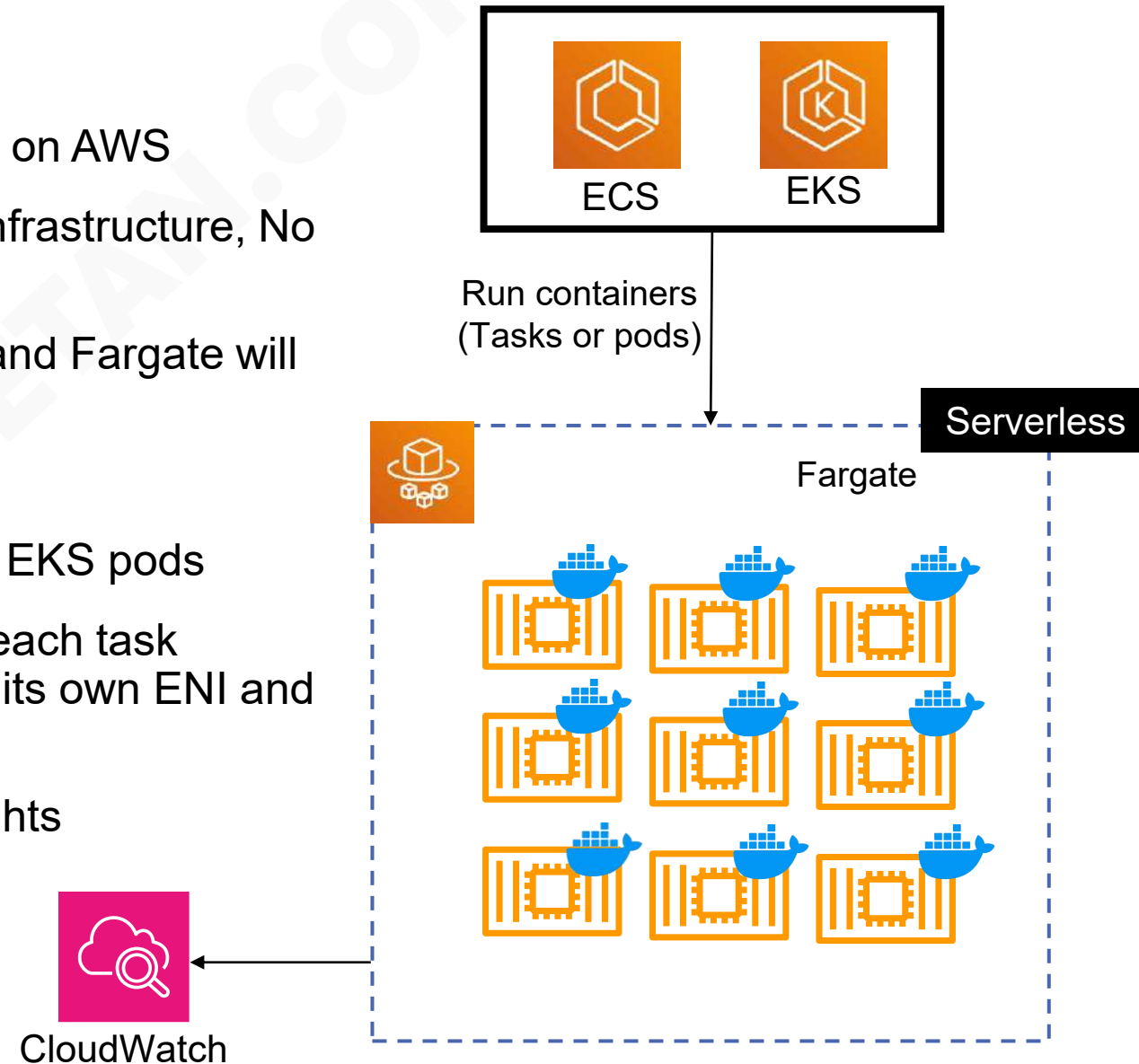
EKS is a good choice if you already have good experience managing and running Kubernetes cluster at scale in production environment

# Amazon EKS architecture

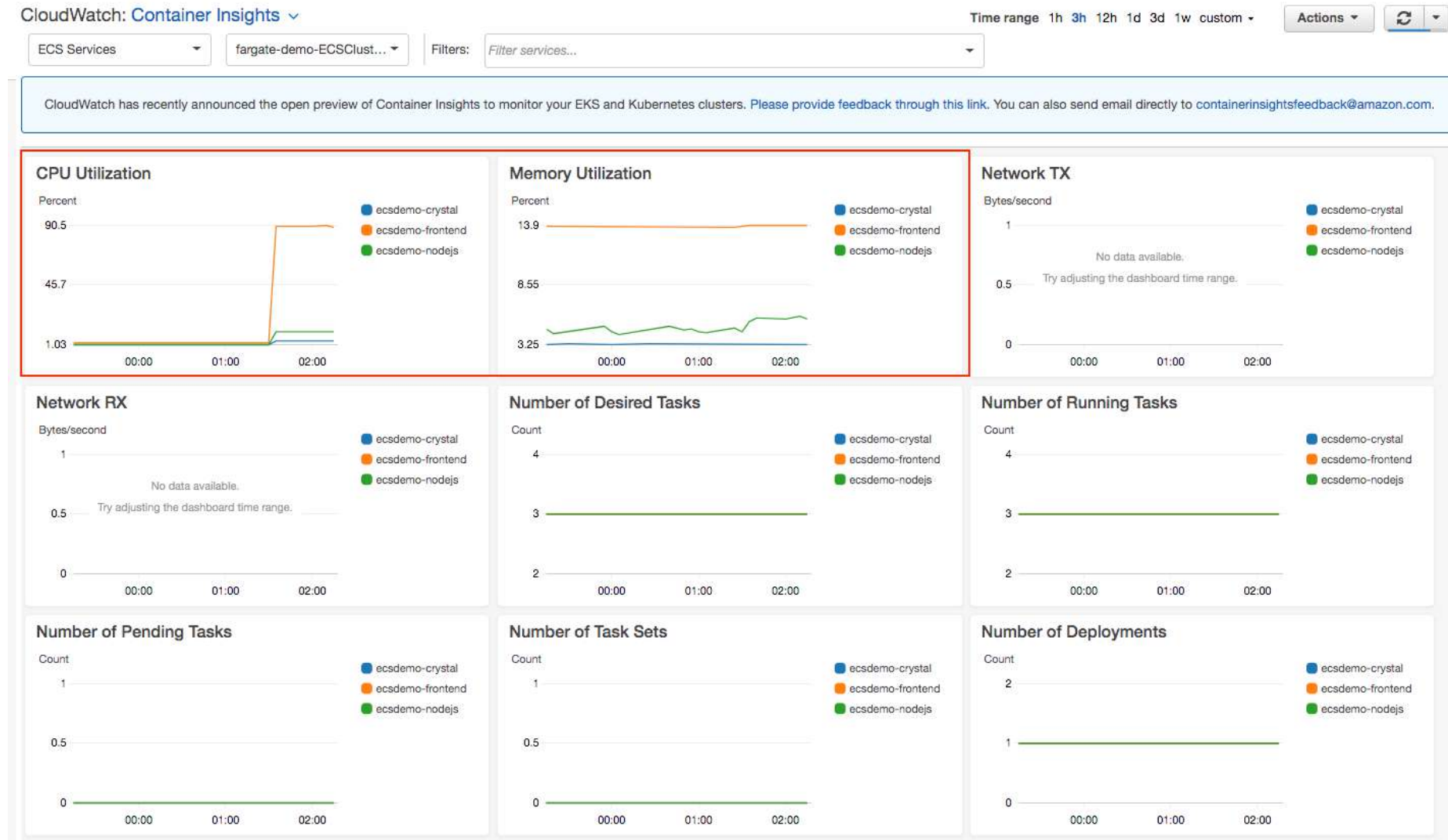


# AWS Fargate

- A Serverless compute for running containers on AWS
- Runs docker containers on AWS managed infrastructure, No EC2 – no management !
- Just define container image, CPU, memory and Fargate will run the containers.
- Pay-as-you-go pricing.
- Works with Amazon ECS tasks and Amazon EKS pods
- Fargate always uses **awsvpc** mode, where each task behaves like a first-class VPC resource with its own ENI and security groups.
- Monitoring using CloudWatch container insights



# CloudWatch container insights



<https://aws.amazon.com/blogs/mt/introducing-container-insights-for-amazon-ecs/>

# Amazon ECR

- Amazon ECR is a fully managed container image registry used to store container images.
- Images are stored in private repositories by default, with support for public repositories.
- Integrates natively with Amazon ECS and EKS.
- ECR supports image versioning using tags and digests.
- Authentication and access control are managed using IAM policies and roles.
- Images are encrypted at rest and in transit automatically.
- ECR can perform vulnerability scanning of images using **Amazon Inspector**.
- Provides lifecycle policies to automatically delete old or unused images.



Amazon ECR



Registry



Version 1



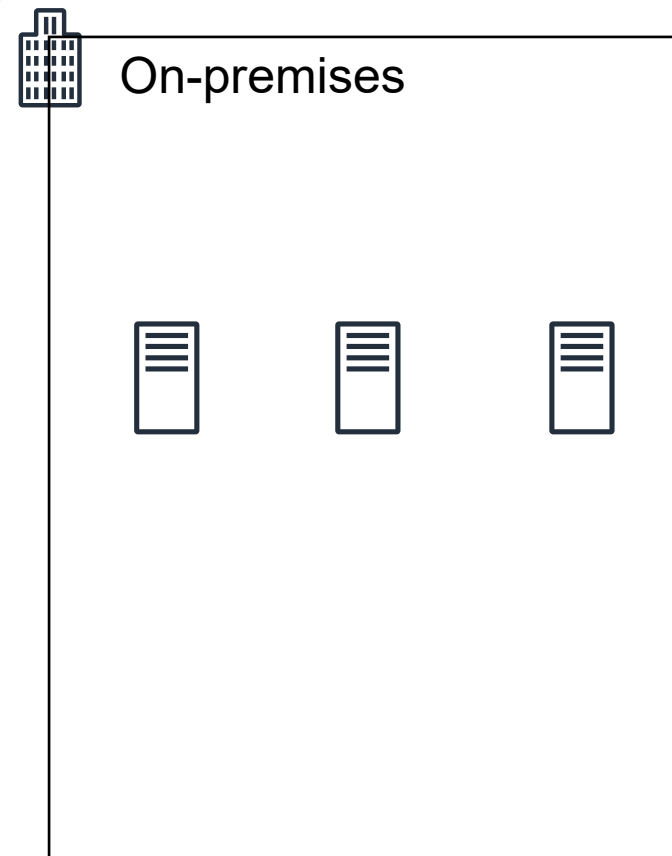
Version 2



Latest

# Running containers at on-premises location

- Amazon ECS Anywhere
- Amazon EKS Anywhere



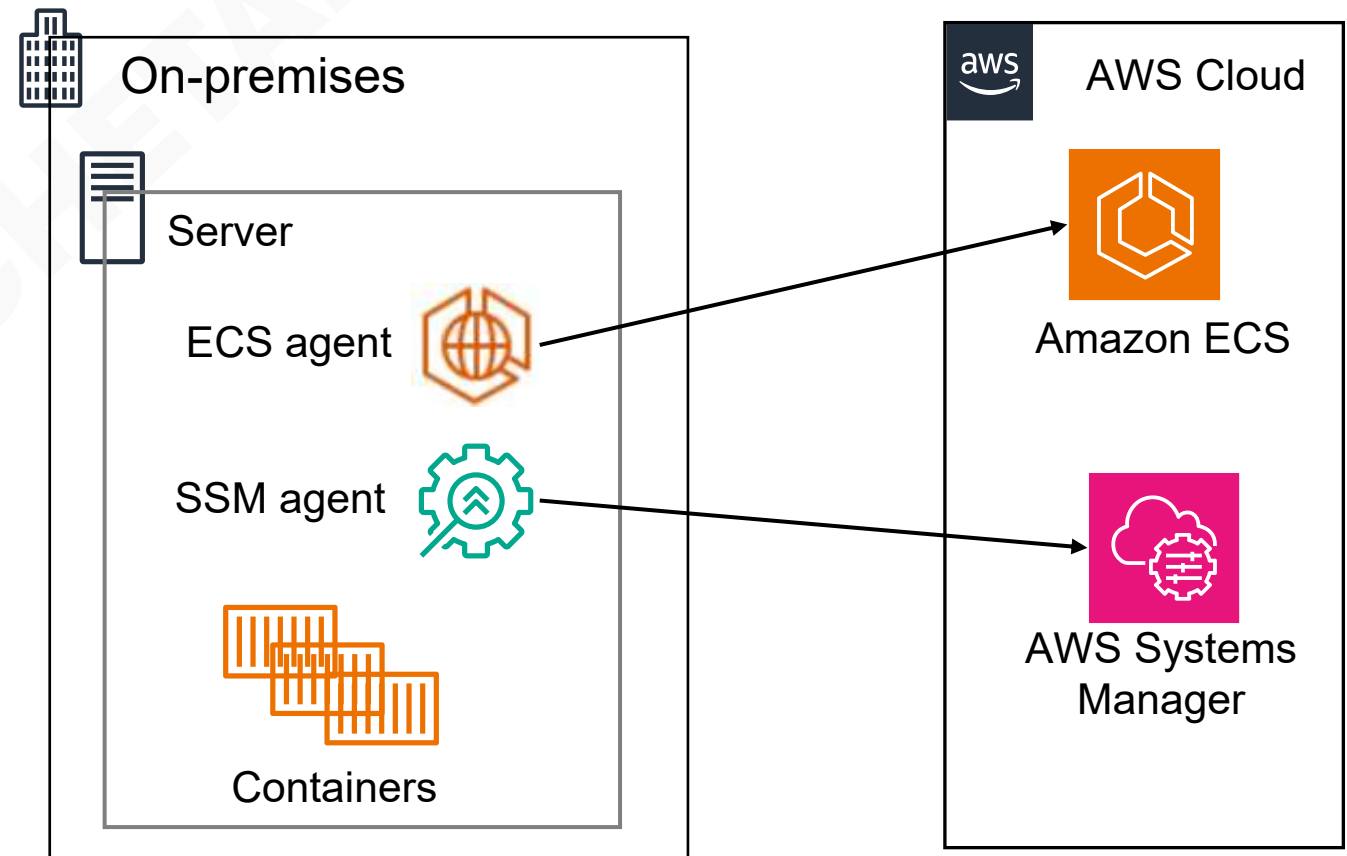


# Amazon ECS Anywhere



Amazon ECS  
Anywhere

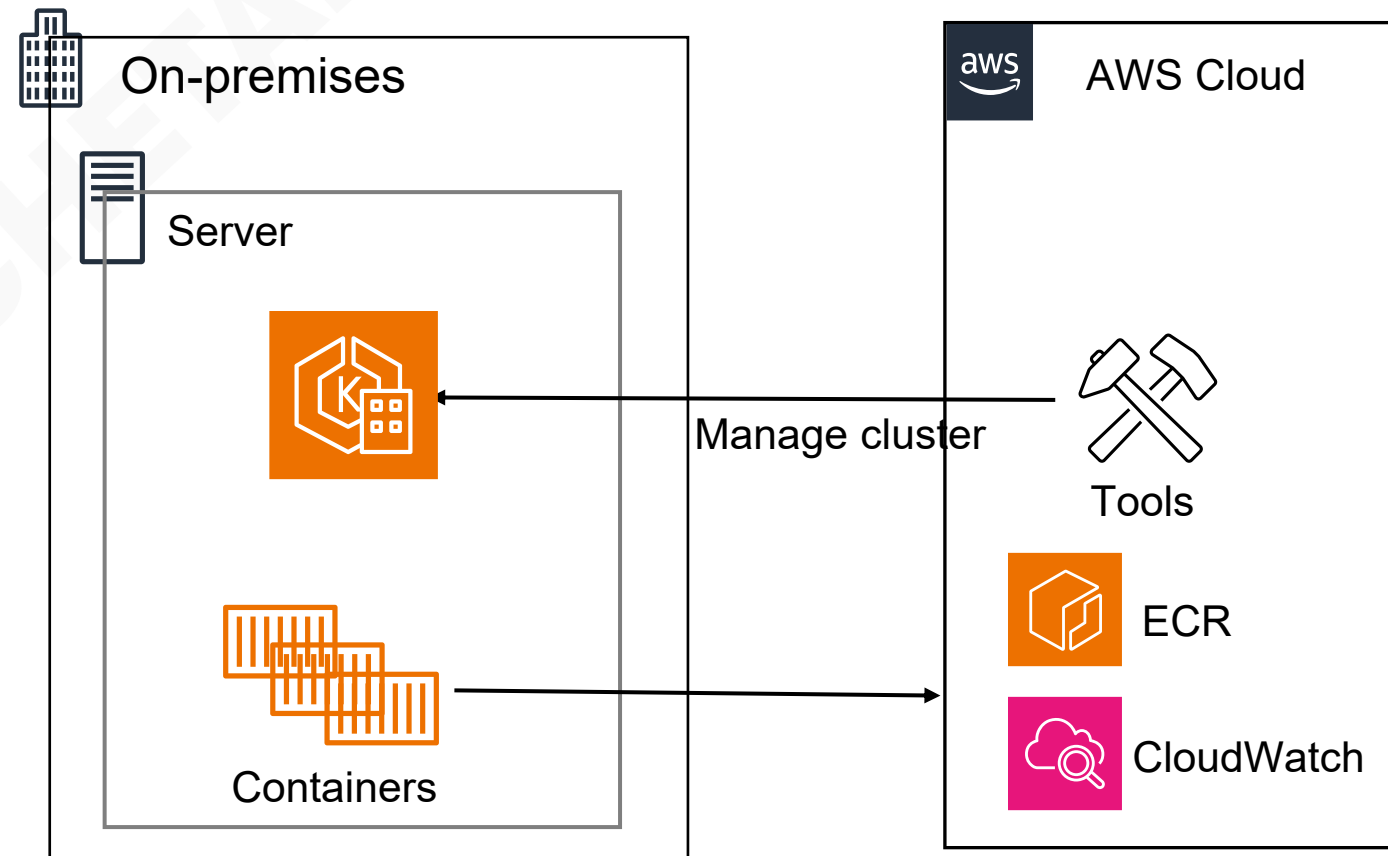
- Amazon ECS Anywhere provides support for registering an external instance such as an on-premises server or virtual machine (VM), to your Amazon ECS cluster.
- **ECS control plane is in AWS** with an agent running on your servers.



# Amazon EKS Anywhere



- Kubernetes clusters run on your own infrastructure and uses open-source Kubernetes, packaged and validated by AWS.
- AWS provides EKS Anywhere tooling to create, manage, and upgrade the Kubernetes clusters.
- Allows clusters to connect to AWS services such as Amazon ECR for images and CloudWatch for logs and metrics.





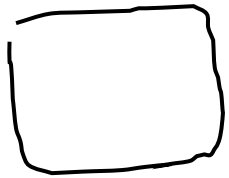
# AWS Databases



# Evolution of applications and databases



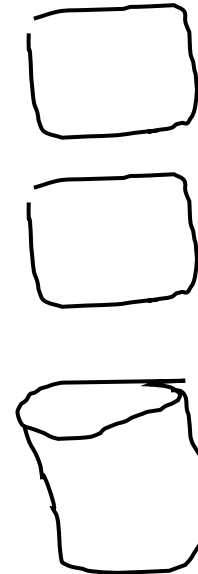
Mainframes



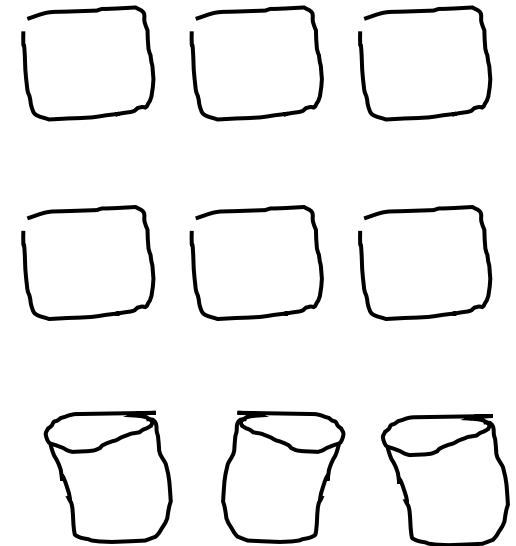
Client-Server



3-Tier



Microservices



# Today's need

What we used traditionally?



A Relational Database



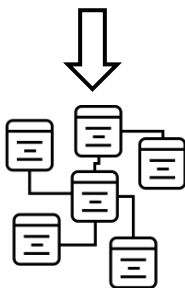
What is needed today?



Purpose built databases

# AWS Database services

Banking,  
Finance,  
Bookings,  
ERP, CRM



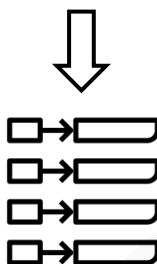
Relational  
Database

Leaderboards,  
real-time  
analytics,  
caching



In-memory  
Database

Shopping cart,  
Product catalog,  
Customer  
attributes



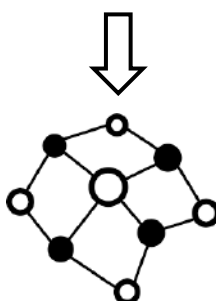
Key-value or  
NoSQL  
Database

Content  
management,  
personalization,  
mobile



Document  
Database

Fraud detection,  
social  
networking,  
recommendation  
engine



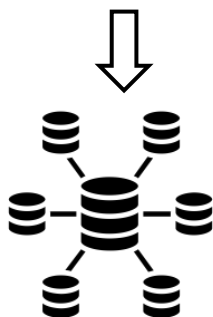
Graph  
Database

IoT  
applications,  
event tracking



Timeseries  
Database

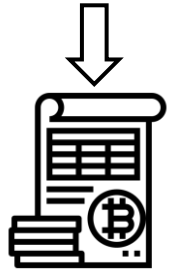
Analytics,  
Data Marts



Data  
Warehouse

# AWS Database services

Systems of record,  
Supply chain,  
health care,  
financial



Ledger  
Database

## Generative AI



Documents



images



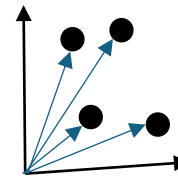
Audio

0.1, 2.9, 0.1, 0.9, 1.0,...

2.3, 9.2, 2.5, 9.4., 4.1,...

7.1, 1.0, 0.3, 3.4, 1.1,...

Vector representation  
of data for similarity  
search e.g. Storing  
data embedding for  
Generative AI RAG

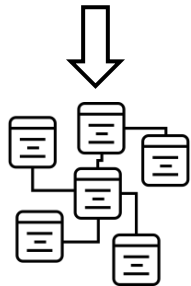


Vector (search)  
Database



# AWS Database services

Banking,  
Finance,  
Bookings,  
ERP, CRM



Relational  
Database



Amazon RDS    Amazon Aurora

Leaderboards,  
real-time  
analytics,  
caching

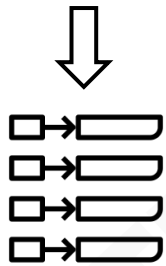


In-memory  
Database



Amazon ElastiCache

Shopping cart,  
Product catalog,  
Customer attributes



Key-value or  
NoSQL  
Database



Amazon DynamoDB

Content management,  
personalization,  
mobile

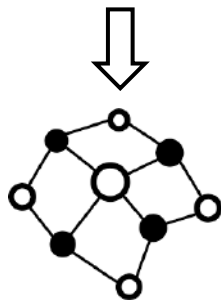


Document  
Database



Amazon DocumentDB

Fraud detection,  
social networking,  
recommendation engine



Graph  
Database



Amazon Neptune

IoT applications,  
event tracking

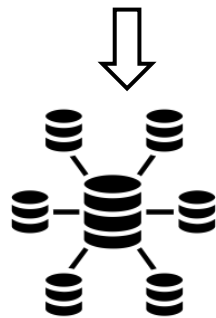


Timeseries  
Database



Amazon Timestream

Analytics,  
Data Marts



Data  
Warehouse



Amazon Redshift

# AWS Database services

Systems of record,  
Supply chain,  
health care,  
financial



Ledger  
Database



Amazon Quantum  
Ledger Database  
(Amazon QLDB)

## Generative AI



Documents



images

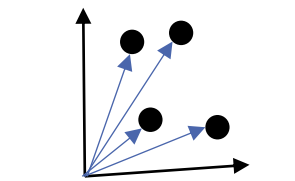


Audio



- 0.1, 2.9, 0.1, 0.9, 1.0,...
- 2.3, 9.2, 2.5, 9.4., 4.1,...
- 7.1, 1.0, 0.3, 3.4, 1.1,...

Vector representation  
of data for similarity  
search e.g. Storing  
data embedding for  
Generative AI RAG



Vector (search)  
Database



Amazon  
OpenSearch



Amazon Aurora



Amazon RDS

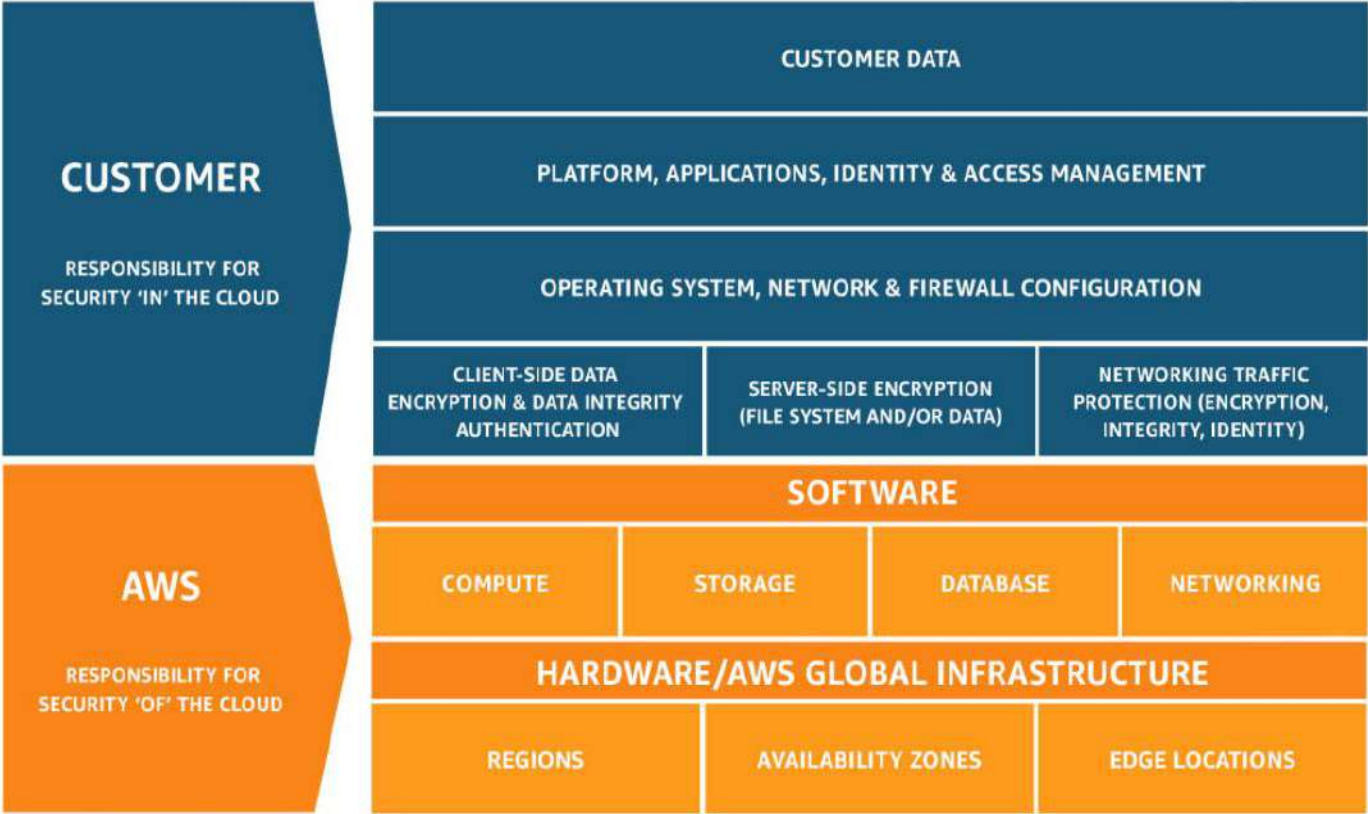


Amazon Neptune



Amazon  
DocumentDB

# AWS Shared Responsibility model



## “Security of the Cloud” - AWS responsibility

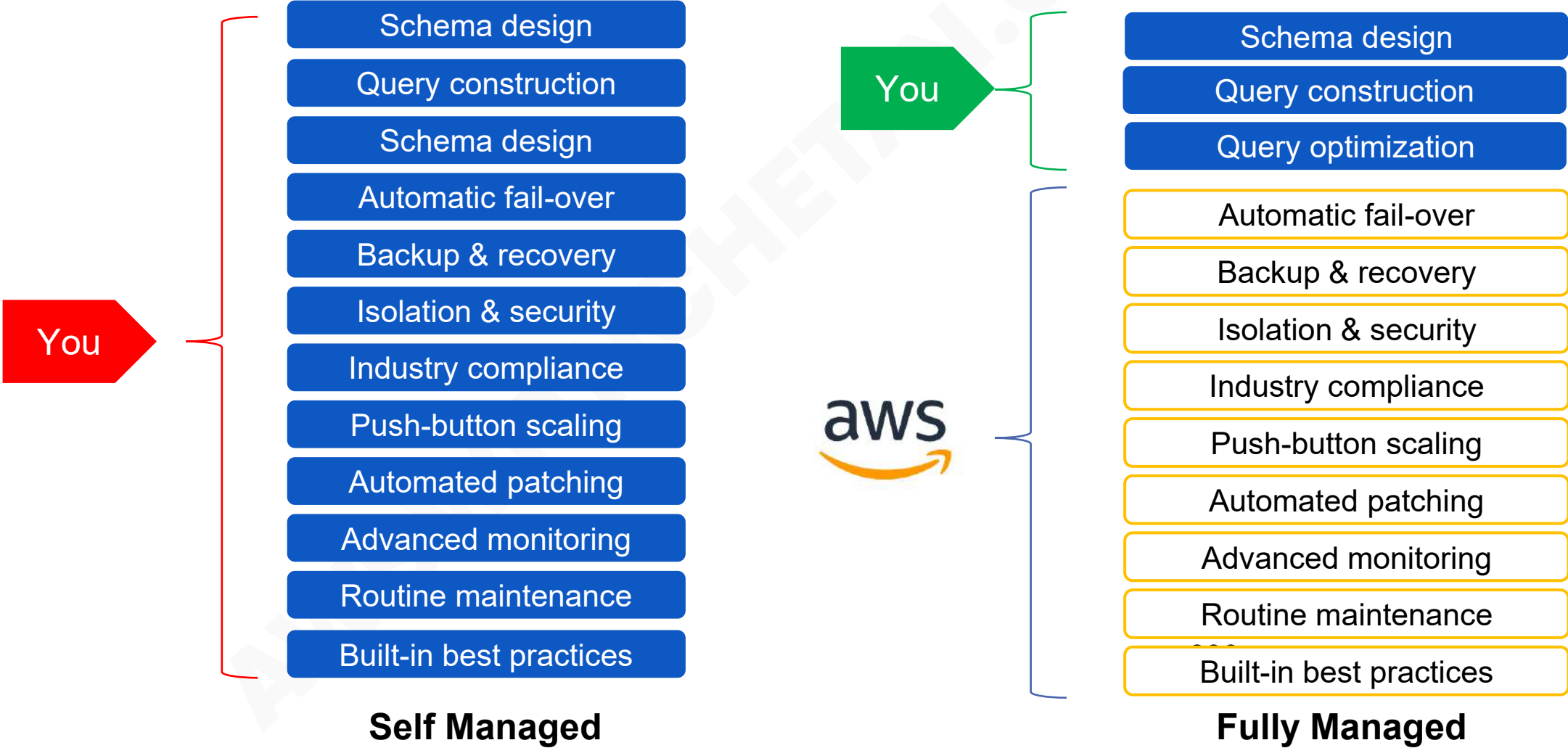
- Protecting the infrastructure that runs all of the services offered in the AWS Cloud.
- Physical security of the data center facilities
- Hardware, software, networking for fully managed services like DynamoDB, S3



## “Security in the Cloud” - Customer responsibility

- Depends on AWS service
- Typically protecting data, access, firewall configurations is customer’s responsibility

# Why managed database service?



# AWS Shared Responsibility model for databases



- Provision and manage the underlying EC2 instance hosting RDS database
- Automated backups (feature of RDS)
- Underlying OS and DB patching
- Database maintenance
- Public or Private network access to the database
- Security group inbound rules for database instance (e.g. 3306 port to be opened for MySQL DB)
- Enabling encryption for data at rest
- Database user creation and permissions
- Manual backups (on-demand)



# Amazon RDS

# Amazon RDS

- Amazon **R**elational **D**atabase **S**ervice
- A managed SQL database service where AWS handles provisioning, patching, upgrades, backup, recovery, repair, monitoring etc.
- RDS supports following database engines:

## AWS native DB



Amazon Aurora

## Open-source DBs



## Commercial DB



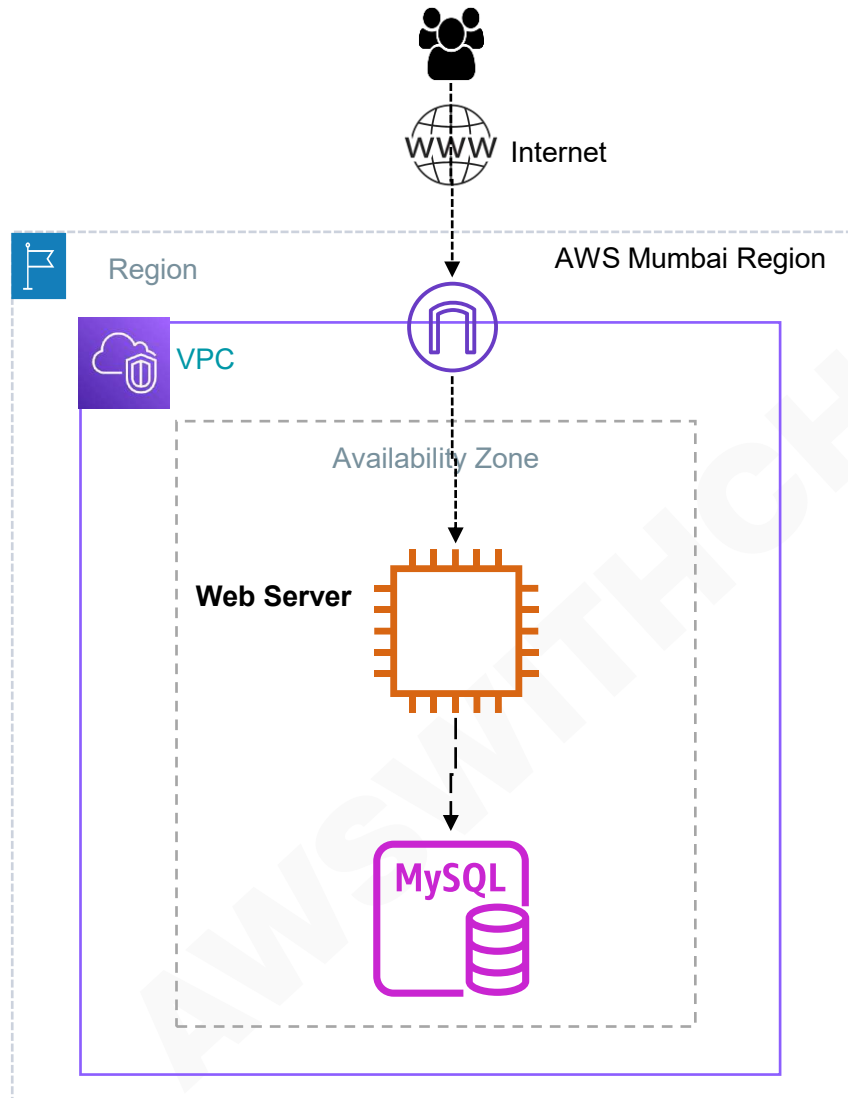
# RDS Features

- RDS Multi-AZ deployment – Deploy database across multiple AZs
- RDS Read Replica – For scaling read queries
- RDS Storage Auto Scaling – Increase storage automatically
- RDS Custom – Get access to underlying host
- RDS Backup – Automated and manual backup
- RDS Security - Authentication / Encryption / Network





# Demo: Launch RDS DB and connect



## High level steps

- 1 Launch an EC2 instance in the default VPC and connect over SSH
- 2 Create MySQL RDS database in the default VPC (No public access). Security group to allow traffic from EC2.
- 3 Install mysql client on EC2
- 4 Connect to RDS DB using database endpoint

# How to connect to Mysql RDS?

## 1. Install mysql client in the ec2 instance

```
# install pip (Amazon Linux 2023 does not have one by default)
$dnf install -y pip

# install dependencies
$dnf install -y mariadb105-devel gcc python3-devel

# install mysqlclient
$pip install mariadb105
```

## 2. Connect to Database and query the data

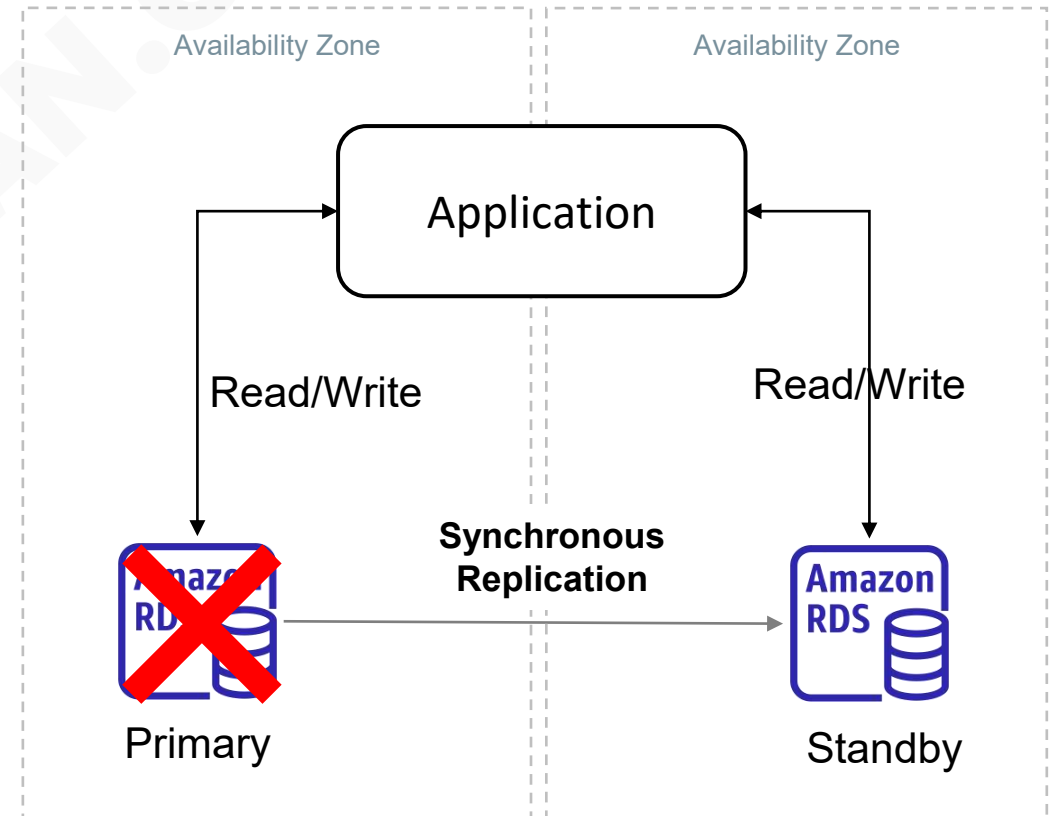
```
$mysql -h <database endpoint> -u admin -p
$MySQL [(none)]> create database awswithchetan;
$MySQL [(none)]> use awswithchetan;
```

## 3. Create table and add data.

```
MySQL [corp]> create table students (emp_id int, name varchar(64), department varchar(32), location varchar(64));
MySQL [corp]> insert into students values (1001, 'Chetan Agrawal', 'IT', 'Pune');
Query OK, 1 row affected (0.003 sec)
```

# RDS Multi-AZ deployment

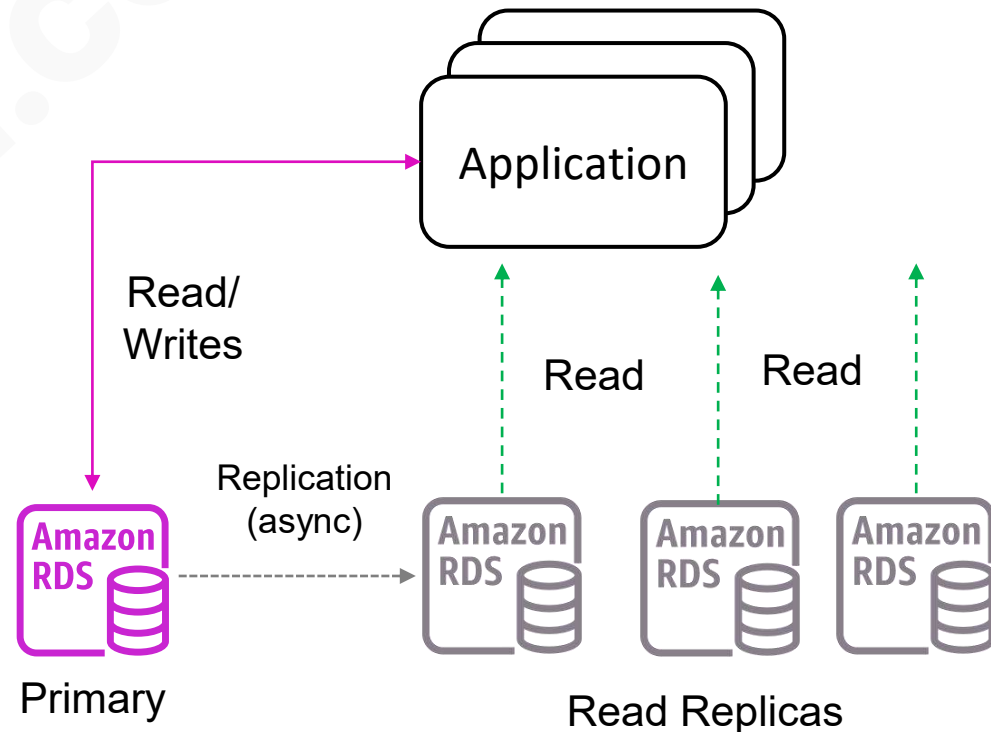
- Primary DB instance in one AZ and Standby DB instance(s) in another AZ
- Application reads from and writes to only Primary DB.
- Data is synchronously replicated from Primary DB to Standby DB.
- In case of Primary DB failure or AZ failure, Standby DB is made Primary, and the DNS endpoint points to Standby.
- Applications automatically gets redirected to standby for read/write queries. No change required at application end.



Multi-AZ is a High Availability solution, not a scaling solution

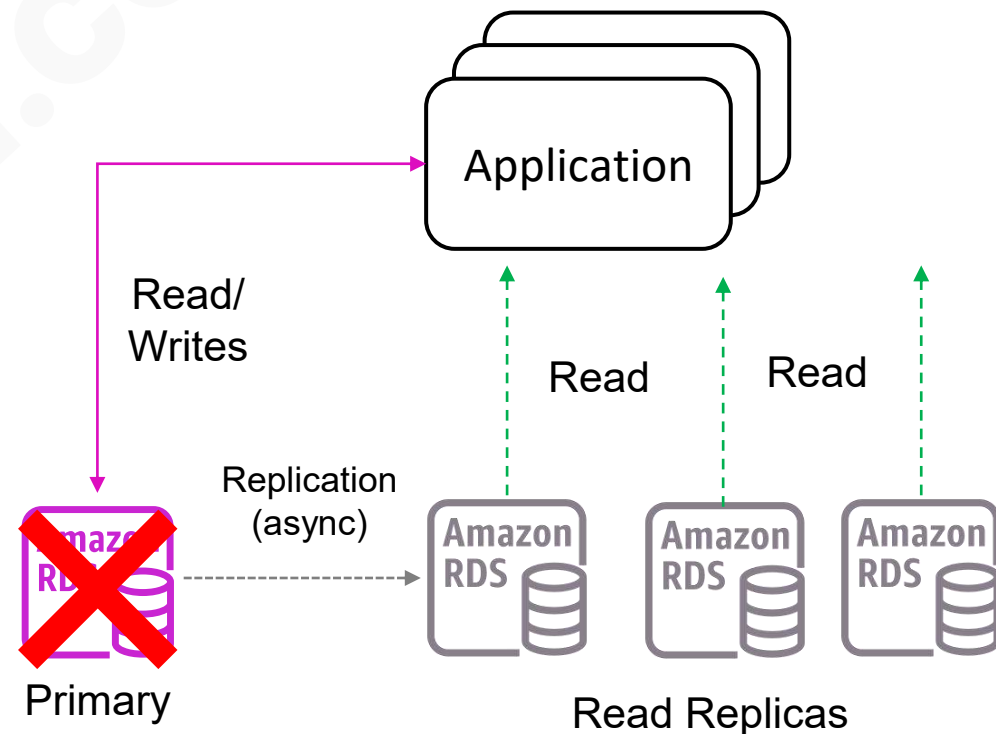
# RDS Read replicas

- Create Read Replicas to scale the Read queries thereby relieving pressure on Primary DB
- Data is only written to the Primary DB and can be read from any of the read replicas
- Create up to 15 Read Replicas in the **same or different AWS region** (for cross-region DTO charges apply)
- Data is replicated **asynchronously**
- Promote Read replica to standalone DB in case of failure of Primary DB instance (Disaster Recovery).



# RDS Read replicas

- Create Read Replicas to scale the Read queries thereby relieving pressure on Primary DB
- Data is only written to the Primary DB and can be read from any of the read replicas
- Create up to 15 Read Replicas in the **same or different AWS region** (for cross-region DTO charges apply)
- Data is replicated **asynchronously**
- Promote Read replica to standalone DB in case of failure of Primary DB instance (Disaster Recovery).



## Use cases:

1. Business reporting and data warehousing where queries can run against a read replica, instead of production DB instance.
2. Implementing disaster recovery.
3. Read data with low latency from the local region even if Primary DB is in another region.

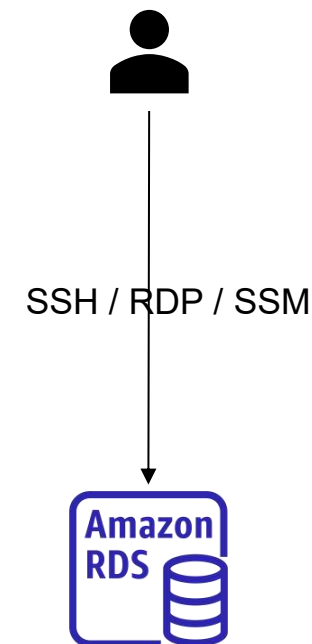
# RDS – Storage autoscaling

- Automatically increases storage capacity for an RDS DB instance when needed.
- Storage autoscaling happens only when all conditions are met:
  - Free storage falls below 10% of allocated storage.
  - Low-storage condition persists for at least 5 minutes.
  - At least 6 hours have passed since the last storage increase.
- You must configure a Maximum Storage Threshold, which defines the upper limit.
- Ideal for applications with unpredictable or spiky workloads.
- Supported for all RDS database engines (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server).
- Supported on gp2, gp3 and Provisioned IO volume types (not available for Magnetic volumes).



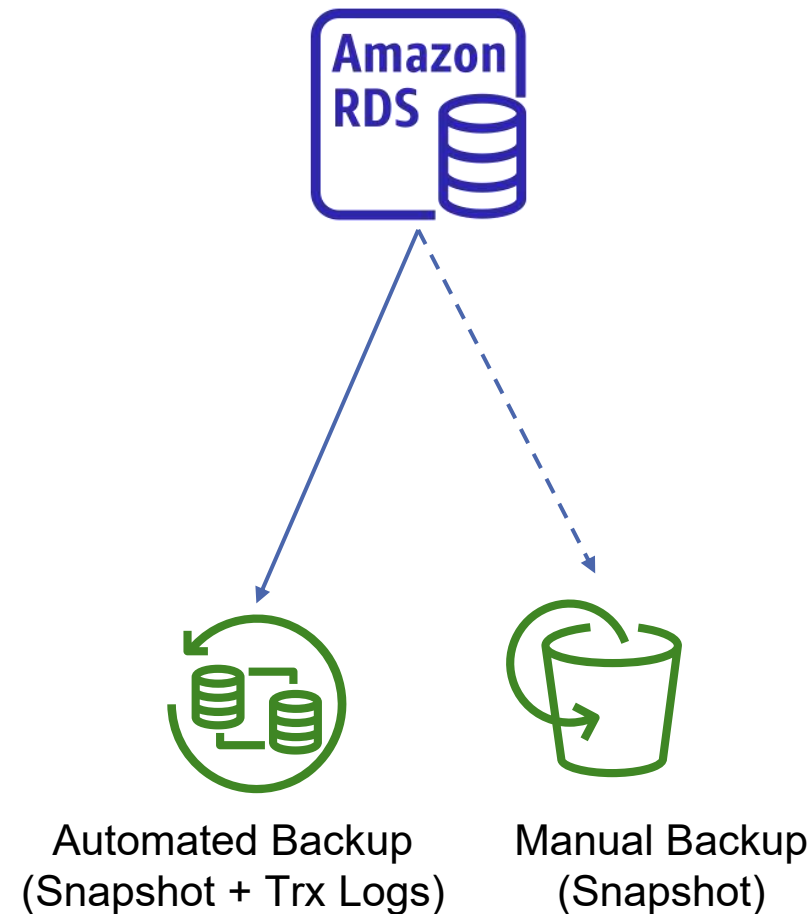
# RDS Custom

- Managed database service with OS/DB-level access - gives you SSH/RDP access to the underlying host, unlike standard RDS.
- Designed for customized, legacy, or packaged applications that require OS-level changes or special database configurations.
- Supports Oracle and SQL Server.
- You can install custom patches, drivers, agents, and third-party software on the database host.
- RDS still automates many tasks (backups, monitoring), but you control more of the environment.
- Use when you need features not available in standard RDS, such as OS-level file changes, Custom database extensions, or Specific security tools
- Note: De-activate **Automation Mode** when you have to apply customizations. Also it's recommended to take a DB snapshot before making the changes.



# RDS Backup

- RDS supports Automatics continuous backups and on-demand or scheduled backups in the form of snapshots.
- With continuous backups we can do **Point in time restore** (specific time with 1 sec granularity up to maximum 35 days retention)
- RDS manual snapshot are stored in S3 and can retain as long as you want.
- Snapshots can be copied across AWS accounts and regions to create a new database.
- RDS performs automated backups only within a daily backup window, which you can choose or otherwise chosen by AWS.
- Backups can introduce slight I/O impact, so choose a low-traffic time.





# RDS Security

## Access/Authentication:

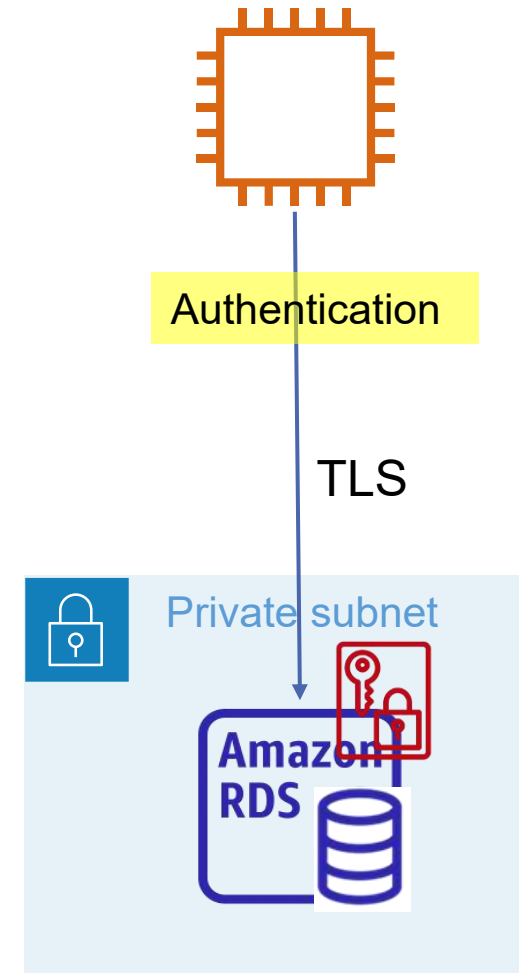
- Supports Username/password authentication for all databases. Recommended to use AWS Secrets Manager or SSM Parameter to store database credentials.
- Supports IAM authentication for MySQL & PostgreSQL DB engines.

## Encryption:

- You can enable encryption for RDS DB at the creation time. Uses AWS KMS.
- Supports TLS/SSL encryption in transit for secure client connections.
- Automatic backups and snapshots stored in S3 are encrypted if encryption is enabled for the database
- If you need encryption later, you must take a snapshot -> copy it with encryption -> restore a new encrypted DB.

## Network:

- Full VPC network isolation using subnets, security groups, and NACLs.
- No public internet access unless explicitly configured.

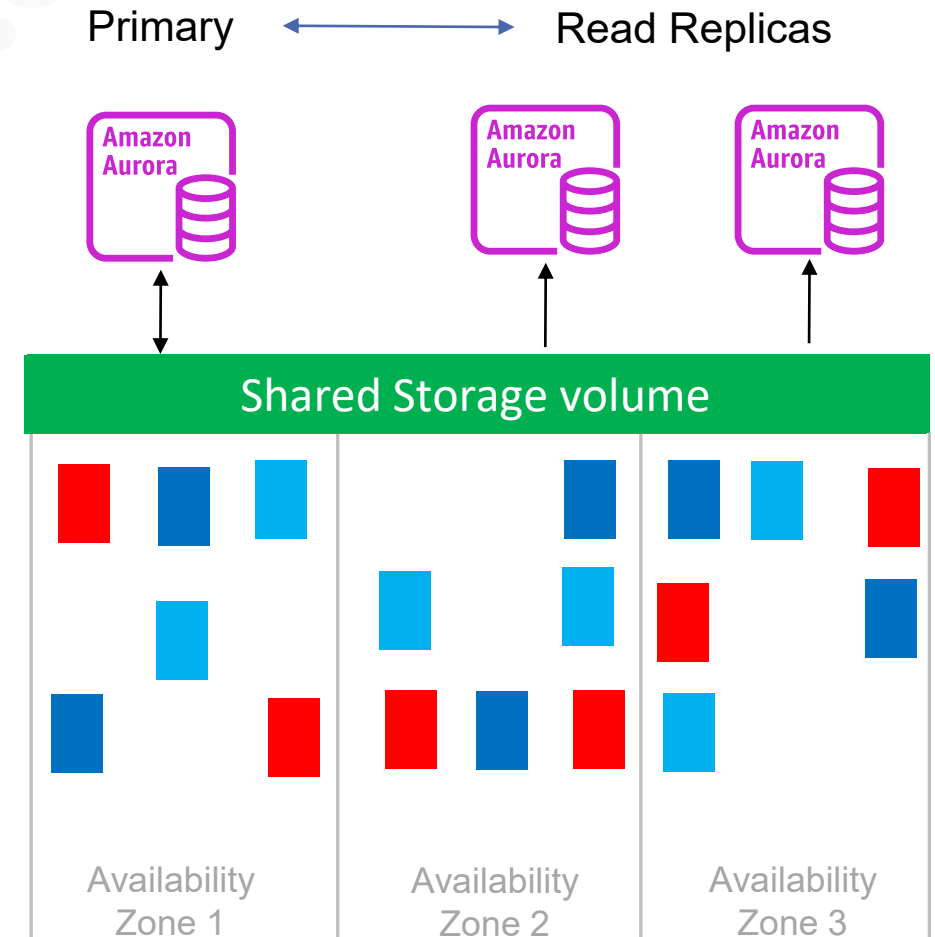




# Amazon Aurora

# Amazon Aurora

- **Aurora** is **PostgreSQL** and **MySQL** compatible relational database engines built by AWS. Aurora also supports **DSQL** (new).
- Aurora provide 5x the throughput of MySQL and 3x of PostgreSQL DB and 1/10<sup>th</sup> cost of commercial databases.
- Aurora DB cluster consist of Primary (writer) DB instance and Replicas (reader)
- Aurora maintains 6 copies of data across 3 Availability Zones with shared storage
- Aurora supports massive read scaling with up to 15 read replicas.



Aurora's shared distributed storage

# Amazon Aurora

- Aurora Global Database
- Aurora Serverless
- Aurora Cloning
- Aurora Machine Learning
- Aurora + RDS -> RDS Proxy
- Aurora + RDS -> RDS Reserved Instances

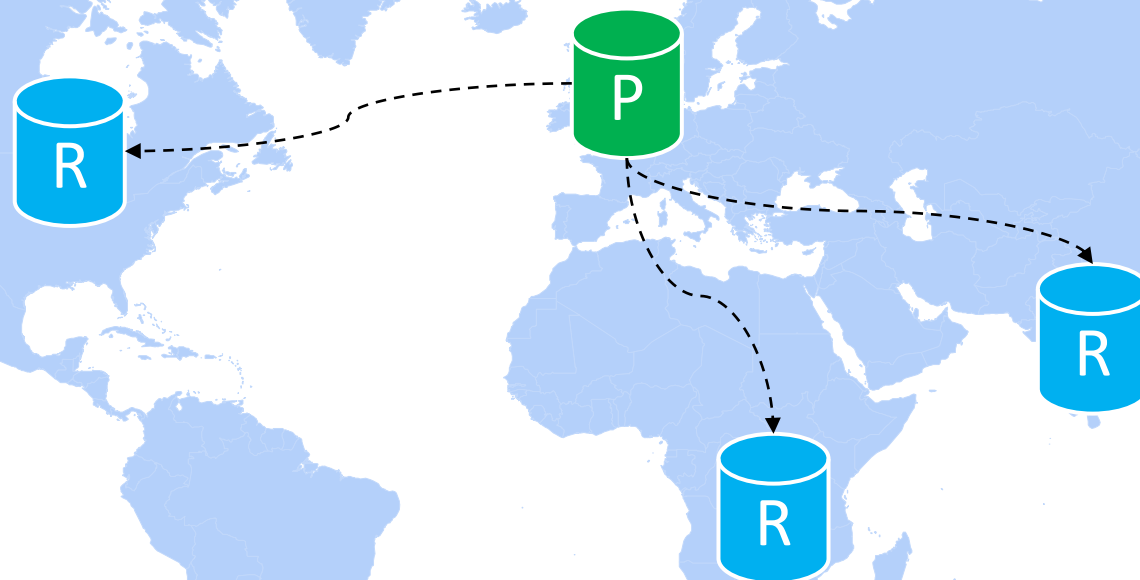


# Aurora Global Database

- Aurora global database spans across multiple regions
- Data is written to the primary DB in the primary region and is replicated to other regions (within 1 sec)

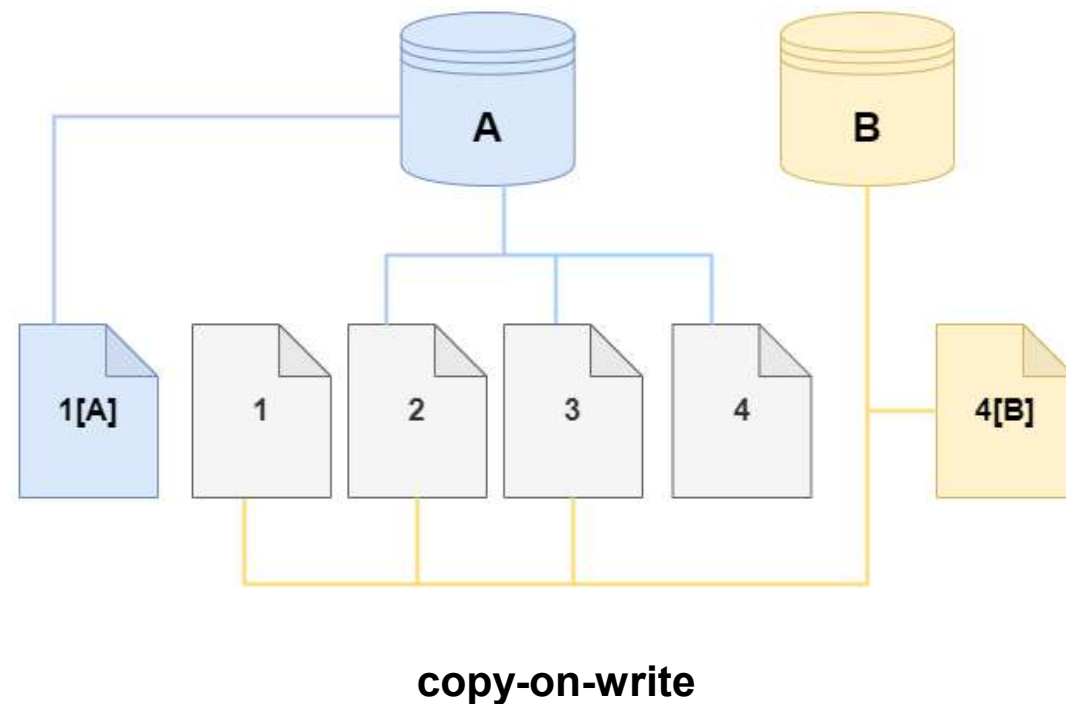
## Use cases:

- Bring data close to your customer's applications in **different regions**
- Promote read-replica to primary for faster recovery **in the event of disaster**



# Aurora Cloning

- Aurora lets you create a clone of a database in minutes without copying all the data.
- Uses **copy-on-write** technology - only changed data blocks are duplicated.
- Extremely fast, cost-efficient, and ideal for creating Test, Development databases from production DB
- Faster than snapshot and restore
- You can create multiple clones (up to 15) from the same source cluster.

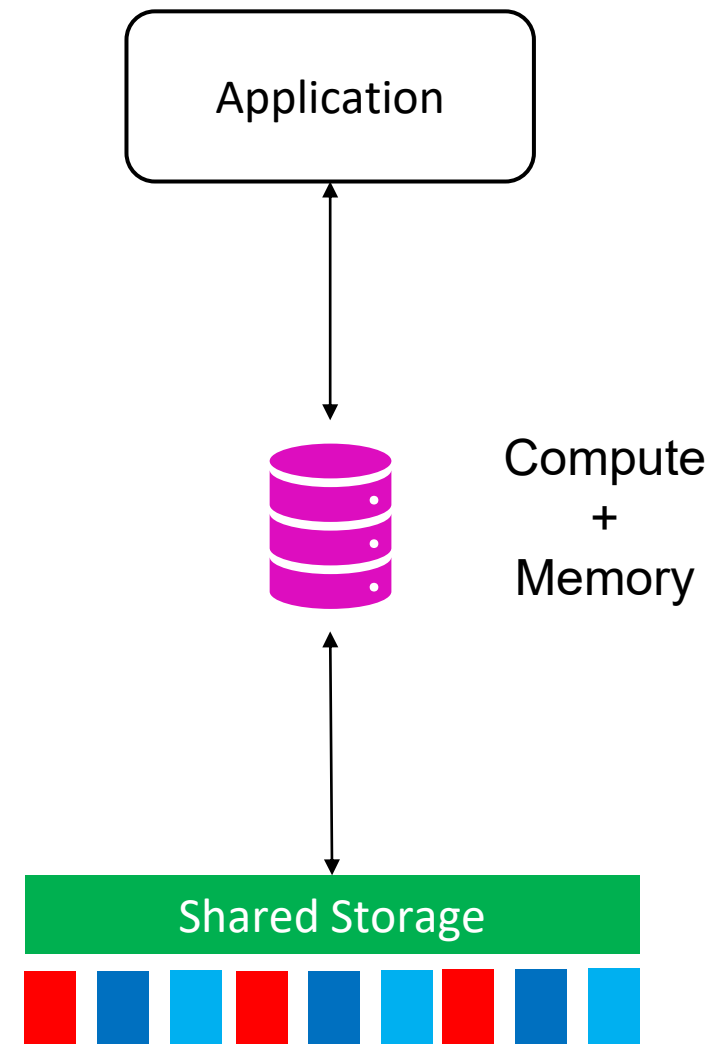


# Aurora Serverless

- It's hard to imagine having Relational Database as Serverless.
- No need to provision database for peak loads.
- With Aurora Serverless, you create a database, specify the desired database capacity range, and connect your applications.
- It automatically starts up, shuts down, and scales capacity up or down based on your application's needs.
- Pay on a per-second basis for the database capacity that you use.
- Supports full Aurora features, including global database, Multi-AZ deployments, and read replicas.

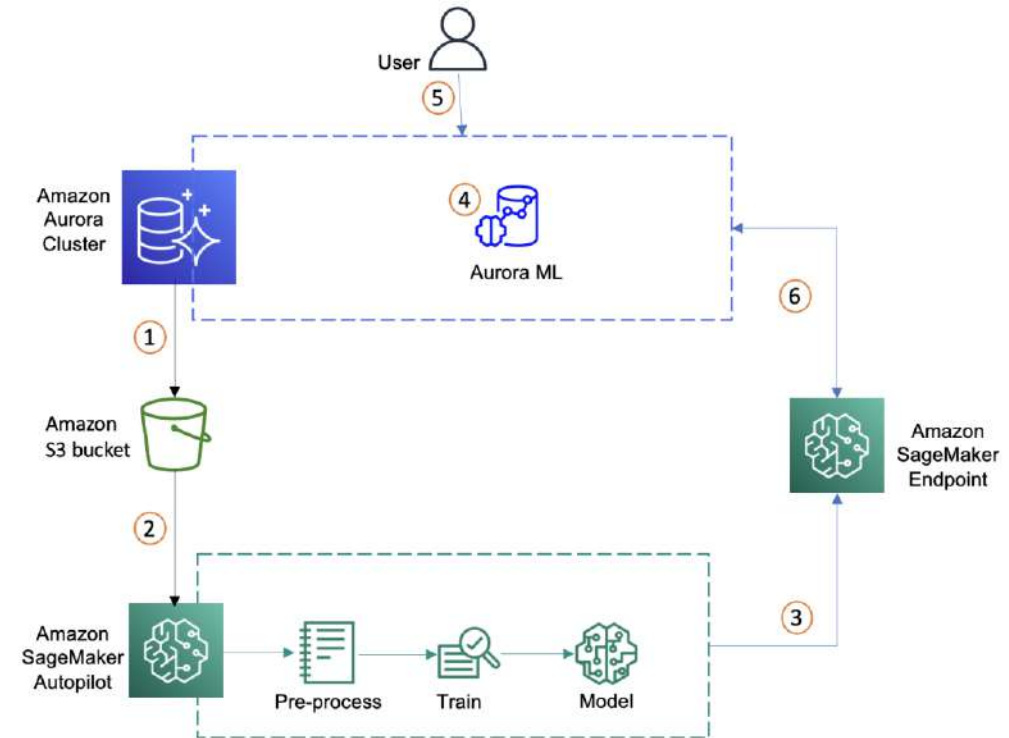
## Use cases:

- All types of database workloads including development and test environments, websites, applications having infrequent, intermittent or unpredictable workloads to business-critical applications requiring high scale



# Aurora Machine Learning

- Aurora Machine Learning allows your Applications to call machine-learning services directly from SQL queries, without writing application-level ML code.
- Aurora integrates with:
  - Amazon SageMaker AI – AI/ML platform
  - Amazon Comprehend – NLP, Sentiment Analysis
  - Amazon Bedrock – GenAI platform
- Aurora does NOT train or host models - inference happens in AWS ML services.
- Data stays in Aurora; only the necessary fields are sent securely to ML services.
- Useful for **real-time intelligence**: fraud detection, sentiment analysis, customer churn prediction, recommendations, summarization, and classification.



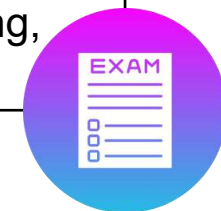
## Example: Using SageMaker AI

AWS Blog: <https://aws.amazon.com/blogs/database/part-1-adding-real-time-machine-learning-predictions-to-amazon-aurora/>



# Comparison between RDS and Aurora

|                    | Amazon RDS                                          | + Amazon Aurora                                                 |
|--------------------|-----------------------------------------------------|-----------------------------------------------------------------|
| DB Engine          | MySQL, PostgreSQL, MariaDB, SQL Server, Oracle, DB2 | MySQL, PostgreSQL, DSQL                                         |
| Architecture       | Compute + Storage coupled.                          | Decoupled compute and Storage                                   |
| Read Scalability   | 15 Read replicas, 5 for Oracle DB                   | 15 Read replicas                                                |
| Storage            | Up to 64TB (option to auto scale storage)           | Up to 128TB (auto scaled in 10GB increment)                     |
| HA                 | Multi-AZ with 60-120 sec failover                   | Automatic failover using Replica (< 30 sec)                     |
| Cost               | Low as compared to Aurora                           | Higher as compared to RDS                                       |
| Performance        | Low as compared to Aurora                           | Higher as compared to RDS                                       |
| Important Features | Automated backup, Multi-AZ, Read Replicas           | + Global database, Serverless, Cloning, Aurora Machine Learning |



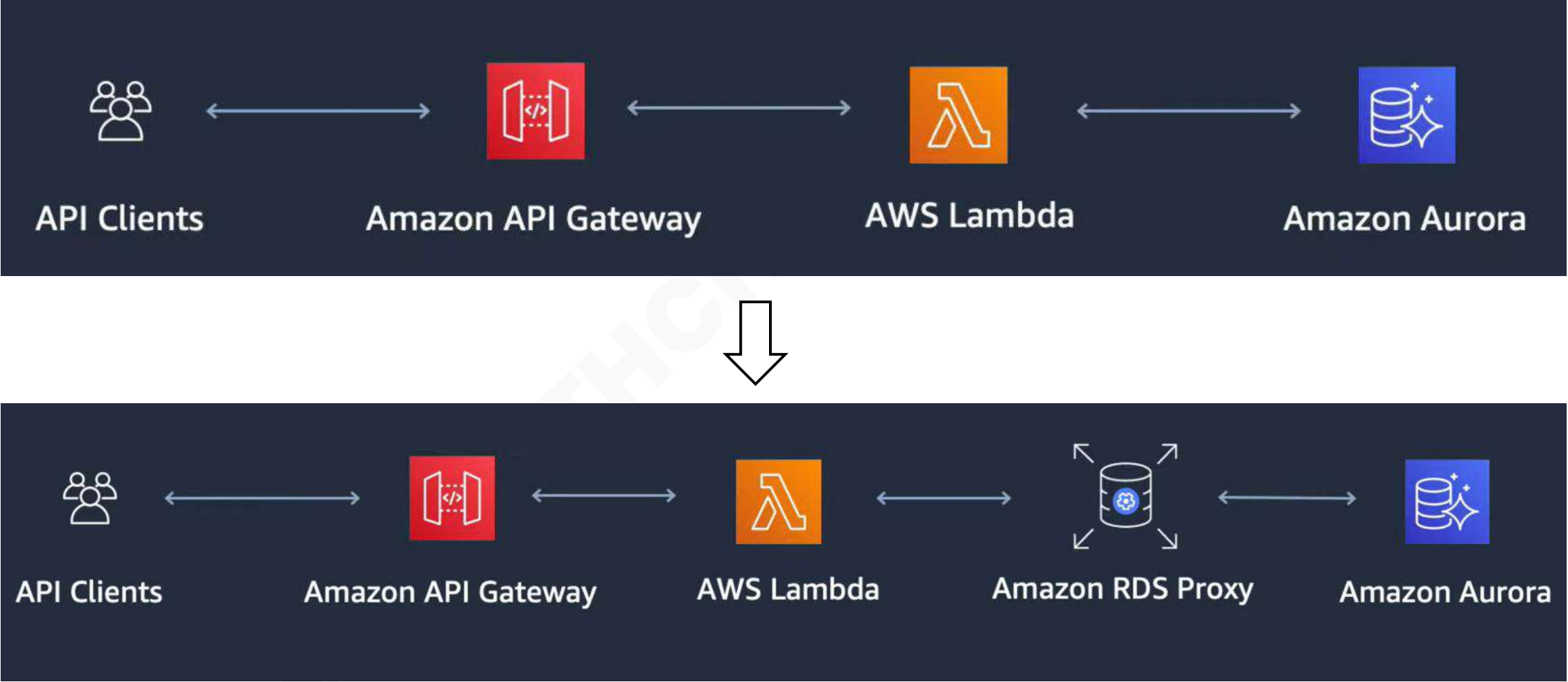
# RDS Proxy

## Why database proxy?

- Serverless and modern apps often create many short-lived DB connections, which can exhaust DB memory/CPU.
- Proxy allows sharing database connections, reducing overhead on the DB.



# RDS Proxy



# RDS Proxy

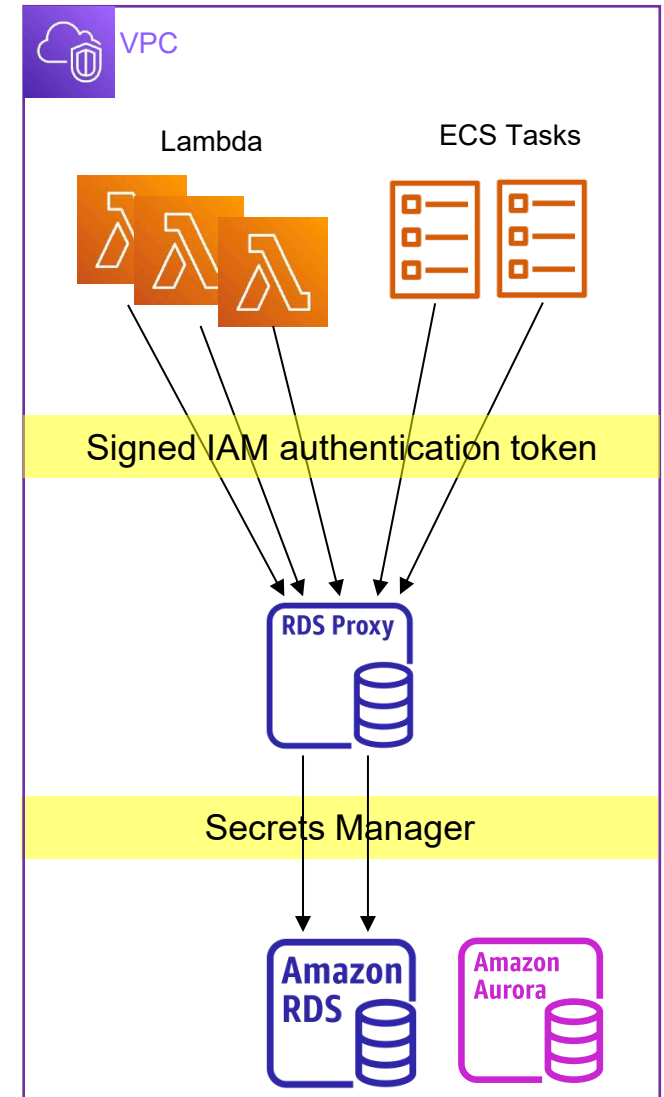
- Fully managed database proxy for RDS and Aurora.
- Sits between your app and DB to manage and pool connections efficiently.
- Reduces failover time by up to 66% for RDS and Aurora databases.
- RDS Proxy can be enabled for most applications with no code changes

## Security

- Enhances security by storing credentials in AWS Secrets Manager. Apps can optionally use IAM token authentication for proxy
- RDS Proxy is not publicly accessible; it always lives inside a VPC

## Use case:

Ideal for serverless apps with unpredictable or high-concurrency workloads.



# RDS Reserved Instances

- Reserved Instances are a great option for the steady state use case and can save up to 69% cost over On-Demand for RDS and Aurora
- RDS Reserved Instances provide three payment options: All Upfront, Partial Upfront, and No Upfront.
- Reserved Instances offers instance size flexibility for Amazon Aurora, MySQL, MariaDB, PostgreSQL, and DB2 database engines as well as the “Bring your own license” (BYOL) edition of the Oracle database engine.
- All Upfront and Partial Upfront Reserved Instances can be purchased for one or three-year terms and No Upfront Reserved Instances are only available for one year term.



Reserved  
Instances

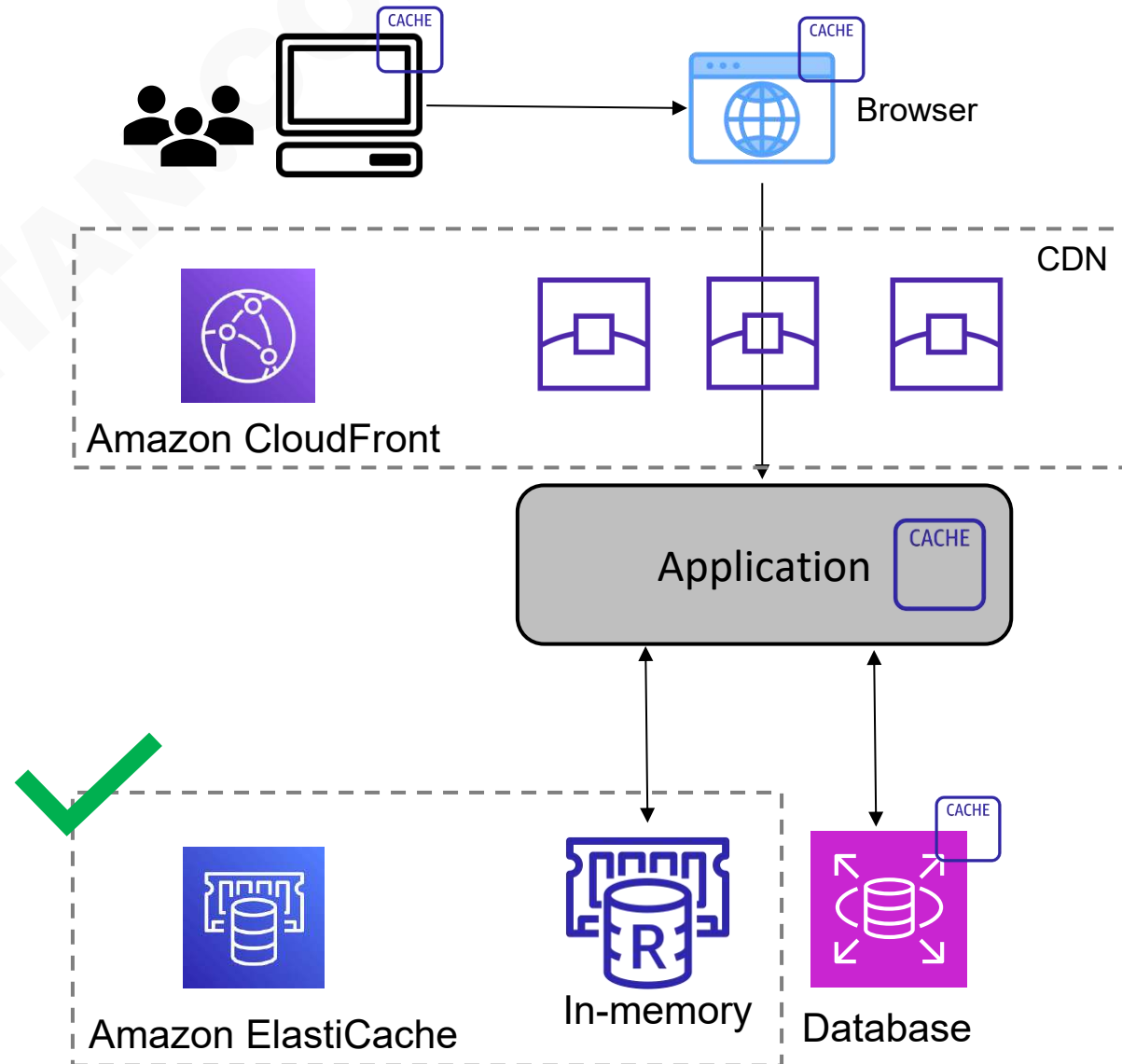




# Amazon ElastiCache

# Caching

- Caching is not limited to a single component of an application
- Each layer serves a different purpose
- Modern applications (e.g., eCommerce) commonly use a multi-layer caching strategy for optimal speed, scale, and cost.
- Caching can be done at
  - Client-side caching
  - Content Delivery Network
  - Application Layer
  - Distributed Cache
  - Database Cache (buffer)



# Amazon ElastiCache

- Database Caches are in-memory databases with high performance and extremely low latency to read/write data
- Helps reduce load off database I/O for read intensive workloads.
- Different caching strategies: Lazy load, Write-through, TTL
- Serverless, Multi-AZ, Cross region replication and Global Datastore
- Amazon ElastiCache supports following DB cache engines:



- **Redis OSS** - Rich data structures, pub/sub, geospatial, streams etc.

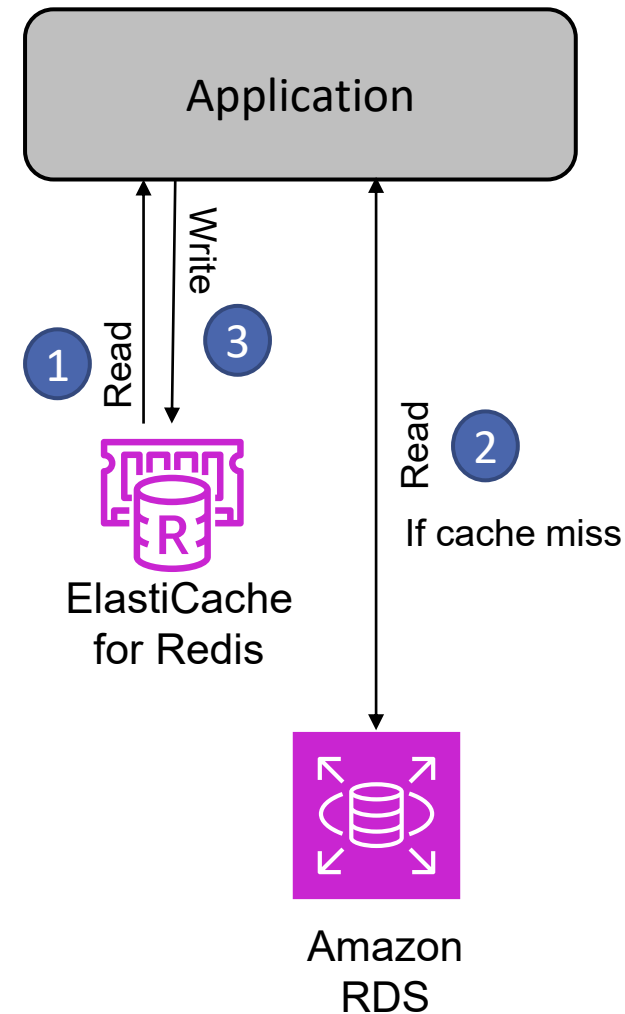


- **Memcached** - Simple, multi-threaded, horizontal scaling



- **Valkey** - Open-source Redis-compatible engine (community-driven), designed to avoid Redis licensing restrictions.

**Use cases:** Session stores, Gaming leader boards, Online shopping cart





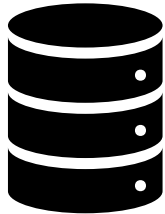
# Comparison between Redis and Memcached

|                    | Redis  | Memcached  |
|--------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Authentication     | Supports AUTH (data-plane). IAM for API actions                                           | No built-in AUTH; supports SASL authentication (advanced).                                    |
| Encryption         | TLS/SSL in-transit + KMS at rest encryption.                                              | No encryption at rest; limited in-transit support.                                            |
| High Availability  | Supports Multi-AZ with auto-failover.                                                     | No Multi-AZ failover.                                                                         |
| Replication        | Supports replication groups + Read replicas.                                              | No replication                                                                                |
| Backup and Restore | Supports automatic backups and snapshots.                                                 | <b>No persistence</b> , no backups.                                                           |
| Features           | Supports Global Datastore (global reads + DR)                                             | Not supported                                                                                 |
| Use cases          | Session store, leaderboards, caching complex objects, pub/sub                             | Simple key-value caching, multi-threaded very high throughput workloads                       |
| Network Features   | Runs inside a VPC; Security Groups, NACL and Subnet Route table rules apply               |                                                                                               |

# SQL vs NoSQL

- **Need to run ad-hoc queries?**
- Predictable traffic?
- Schema is mostly fixed?

Example: Flight booking system, Banking



SQL Database

Oracle, MSSQL, Amazon Aurora,  
PostgreSQL, MySQL

- **90-95% of queries are pre-defined?**
- Need consistent performance at any scale?
- Need flexible schema?

Example: eCommerce product catalog, multiplayer games



NoSQL Database

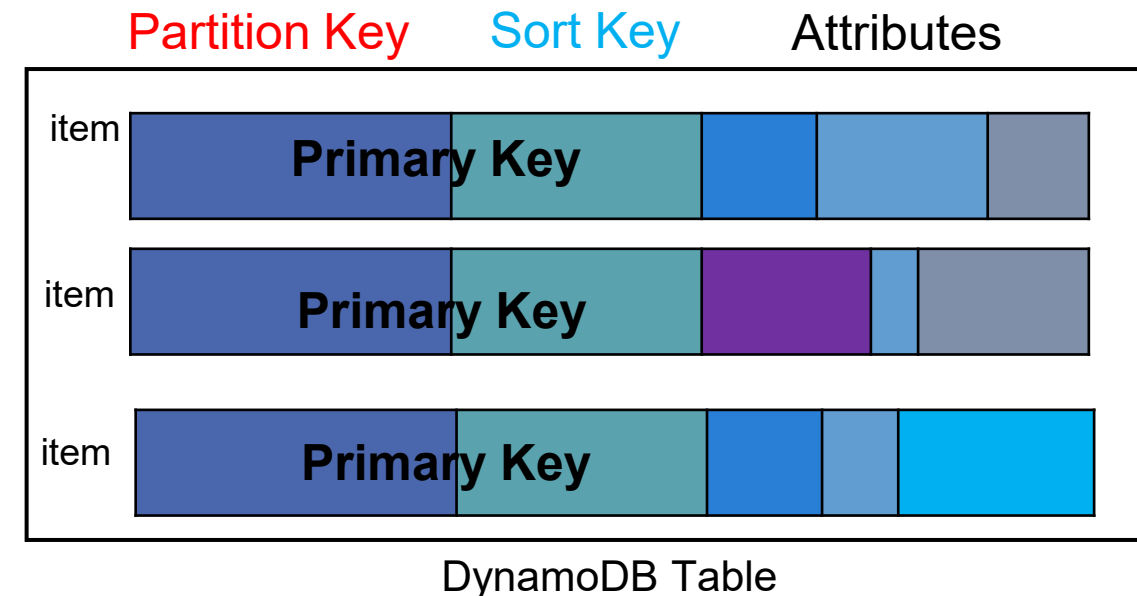
MongoDB, Cassandra, Amazon DynamoDB,  
Apache CouchDB, Azure CosmosDB

# Amazon DynamoDB

- Amazon DynamoDB is a fully managed, serverless **NoSQL** (key-value) database service
- Consistent, single-digit millisecond read and write performance at any scale – millions of requests/seconds, trillions of row, 100s of TB of storage
- Serverless: No provisioning or capacity management, Encryption for data at rest, 99.999% availability
- Support different data types i.e. Scalar (Number, String etc.), Document (JSON) and Set types

## DynamoDB has Table as basic entity to store data

- Table contains items (like rows)
- Items contains Partition Key, Sort Key (optional), and attributes (key-value pairs).
- Partition Key + Sort Key = Primary Key



# Amazon DynamoDB – Query and Scan

**Query** - fetches specific items using Partition Key

- Retrieves items by Partition Key (and optional Sort Key conditions).
- Can use conditions like <, >, <=, >= etc. with Sort Key
- Fast and efficient — looks only in matching partitions.
- Can use filters, but filtering happens after reading the data.

PartitionKey      SortKey

| Userld | Orderld | Product | OrderDate  |
|--------|---------|---------|------------|
| 101    | A1      | Widget  | 2024-04-20 |
| 101    | B2      | Gadget  | 2024-04-21 |
| 101    | C3      | Gizmo   | 2024-04-22 |
| 102    | D4      | Widget  | 2024-04-20 |
| 102    | E5      | Widget  | 2024-04-23 |
| 103    | F6      | Gizmo   | 2024-04-21 |
| 103    | G7      | Gadget  | 2024-04-22 |
| 104    | G7      | Gizmo   | 2024-04-22 |

```
aws dynamodb query \
--table-name Orders \
--key-condition-expression "UserId = :userid" \
--expression-attribute-values '":{"userid":{"N":"103"}}'
```

**Orders Table**

# Amazon DynamoDB – Query and Scan

**Scan** – fetches every item of the table

- Reads every item in the table or index.
- Slow and expensive for large tables.
- Can use filters to narrow down the results but still scans all items first.

```
aws dynamodb scan \
--table-name Orders \
--filter-expression "Product = :name" \
--expression-attribute-values '{":name":{"S":"Widget"}}'
```

PartitionKey      SortKey

| UserId | OrderId | Product | OrderDate  |
|--------|---------|---------|------------|
| 101    | A1      | Widget  | 2024-04-20 |
| 101    | B2      | Gadget  | 2024-04-21 |
| 101    | C3      | Gizmo   | 2024-04-22 |
| 102    | D4      | Widget  | 2024-04-20 |
| 102    | E5      | Widget  | 2024-04-23 |
| 103    | F6      | Gizmo   | 2024-04-21 |
| 103    | G7      | Gadget  | 2024-04-22 |
| 104    | G7      | Gizmo   | 2024-04-22 |

Orders Table

# DynamoDB - Global Secondary Index

- A GSI is an index with its own Partition Key and optional Sort Key, different from the base table's keys.
- It lets you query the same table in a completely different way (new access pattern).
- They maintain a copy of selected attributes from the base table (based on Projection).
- GSIs have their own read/write capacity (in provisioned mode).
- Updates to the base table are automatically propagated to the GSI.
- Useful when your application needs multiple query patterns that the primary key alone cannot support.

Most recent order?

| PartitionKey | SortKey |         |            |
|--------------|---------|---------|------------|
| Userld       | Orderld | Product | erDate     |
| 101          | A1      | Widget  | 2024-04-20 |
| 101          | B2      | Gadget  | 2024-04-21 |
| 101          | C3      | Gizmo   | 2024-04-22 |
| 102          | D4      | Widget  | 2024-04-20 |
| 102          | E5      | Widget  | 2024-04-23 |
| 103          | F6      | Gizmo   | 2024-04-21 |
| 103          | G7      | Gadget  | 2024-04-22 |
| 104          | G7      | Gizmo   | 2024-04-22 |

Orders Table



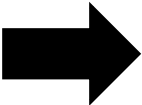
# Example – Global Secondary Index

What is the top score ever recorded for the game Meteor Blasters?

PartitionKey SortKey

| UserId | GameTitle         | TopScore | TopScoreDateTime      | Wins | Losses |
|--------|-------------------|----------|-----------------------|------|--------|
| "101"  | "Galaxy Invaders" | 5842     | "2015-09-15:17:24:31" | 21   | 72     |
| "101"  | "Meteor Blasters" | 1000     | "2015-10-22:23:18:01" | 12   | 3      |
| "101"  | "Starship X"      | 24       | "2015-08-31:13:14:21" | 4    | 9      |
| "102"  | "Alien Adventure" | 192      | "2015-07-12:11:07:56" | 32   | 192    |
| "102"  | "Galaxy Invaders" | 0        | "2015-09-18:07:33:42" | 0    | 5      |
| "103"  | "Attack Ships"    | 3        | "2015-10-19:01:13:24" | 1    | 8      |
| "103"  | "Galaxy Invaders" | 2317     | "2015-09-11:06:53:00" | 40   | 3      |
| "103"  | "Meteor Blasters" | 723      | "2015-10-19:01:13:24" | 22   | 12     |
| "103"  | "Starship X"      | 42       | "2015-07-11:06:53:00" | 4    | 19     |

GameScores Table



| GameTitle         | TopScore | UserId |
|-------------------|----------|--------|
| "Alien Adventure" | 192      | "102"  |
| "Attack Ships"    | 3        | "103"  |
| "Galaxy Invaders" | 0        | "102"  |
| "Galaxy Invaders" | 2317     | "103"  |
| "Galaxy Invaders" | 5842     | "101"  |
| "Meteor Blasters" | 723      | "103"  |
| "Meteor Blasters" | 1000     | "101"  |
| "Starship X"      | 24       | "101"  |
| "Starship X"      | 42       | "103"  |

GameTopScores Index

# Amazon DynamoDB features

- DynamoDB Table class
- DynamoDB Read/Write Capacity (RCU/WCU)
- DynamoDB Global Tables
- DynamoDB Accelerator (DAX)
- DynamoDB Streams
- DynamoDB TTL
- DynamoDB Backup
- DynamoDB export / import





# DynamoDB Table Class

DynamoDB supports Standard Table class and Infrequent Access (IA) Table class

## **Standard (default)**

- Best for frequently accessed data.
- Higher storage cost, lower read/write cost.
- Ideal for hot or regularly used items.

## **Standard-IA (Infrequent Access)**

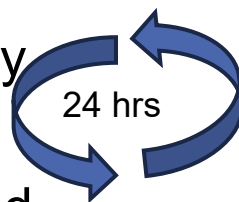
- Lower storage cost, higher read/write cost.
- Designed for rarely accessed or cold data.
- Useful for archival, history tables, old session data, etc.
- We can change the table class from Standard to IA

# DynamoDB Read and Write capacity

- We can define the DynamoDB Read and Write capacity as per the application need.
- There are two modes to set the capacity:

## Provisioned Capacity Mode (Default)

- You pre-define RCU/WCU for the table.
- Best when you have predictable or steady traffic.
- Cheaper than on-demand if your workload is consistent.
- Risk of throttling if traffic suddenly spikes above provisioned limits.



## On-Demand Capacity Mode

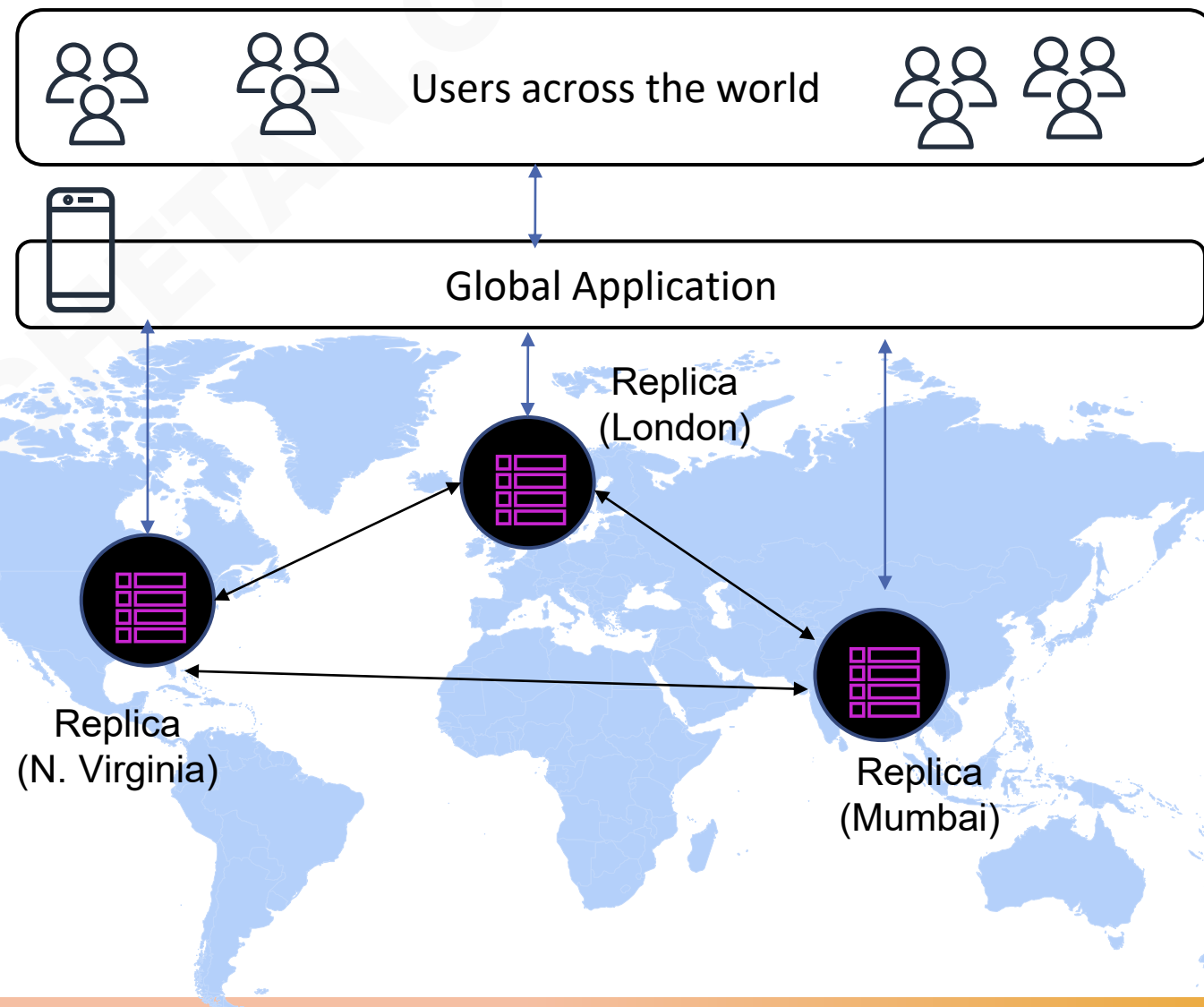
- No need to set RCU/WCU - DynamoDB scales automatically.
- Ideal for unpredictable or spiky workloads.
- You pay per request, which may be costlier for steady high throughput.
- Virtually no throttling unless you hit account-level limits.

# DynamoDB Global table

- Multi-region, multi-active, serverless tables across regions
- 99.999% availability
- Can read/write to any replica
- DynamoDB replicates data across all replicas

## Use cases:

- Global application requiring low latency access for users
- Can handle region level failure (DR)



# DynamoDB Accelerator - DAX

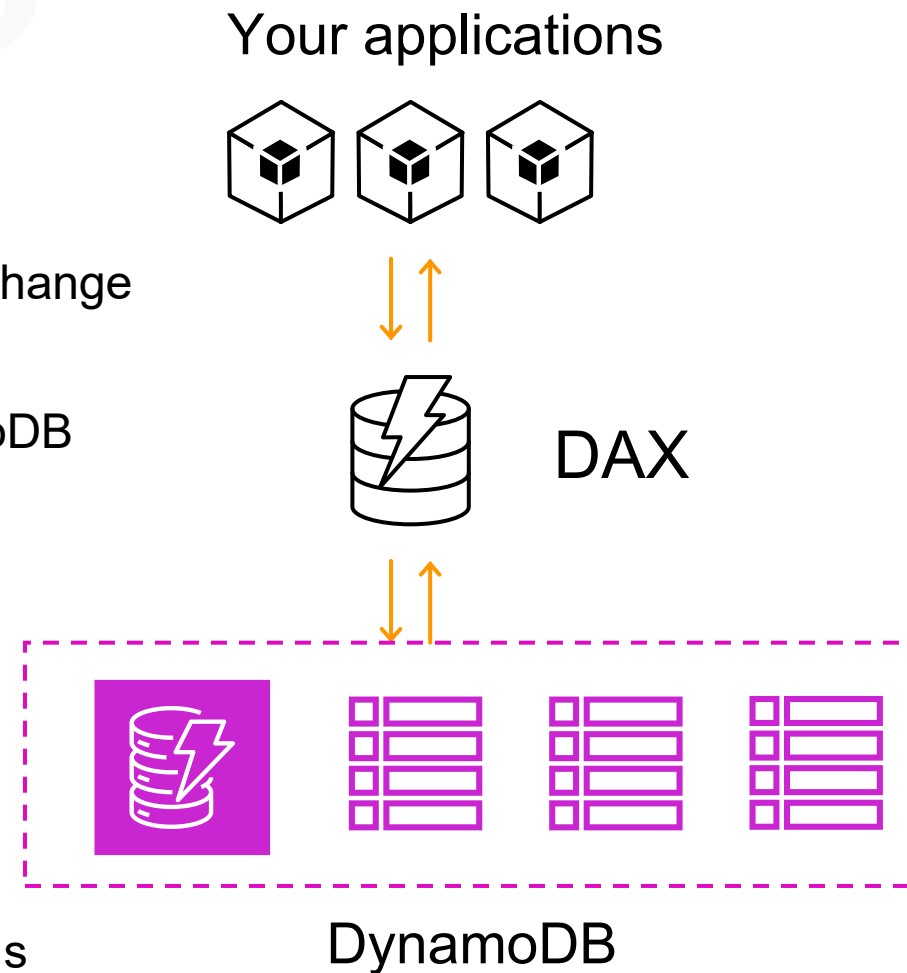
- Fully managed highly available **in-memory cache for DynamoDB**
- 10x performance improvement with single digit millisecond to microsecond level latency
- API-compatible with DynamoDB. Only client & endpoint needs to change to use with an existing application.
- DAX provides access to **eventually consistent** data from DynamoDB tables

## Use cases:

- Real-time bidding, social gaming, and trading applications

## Anti-pattern:

- Strongly consistent read, Write intensive, not having repeated reads

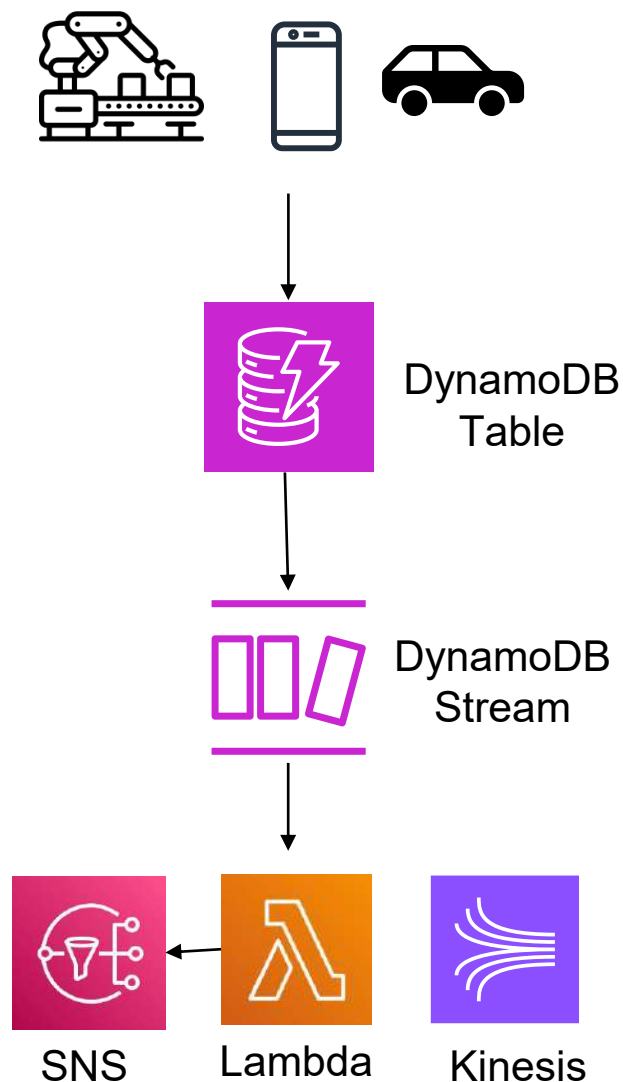


# DynamoDB Streams

- Captures the item level modifications in time-ordered sequence
- Stores the changes in the logs exactly once and strictly ordered for 24 hours
- You can enable/disable a stream on a new or existing table
- DynamoDB Streams operates asynchronously, so there is no performance impact on a table if you enable a stream.

## Use cases:

- Real-time monitoring e.g. connected vehicles, sensor data, Notifying everyone on an activity e.g. friend creates a post on social media
- Backup/Change data capture of DynamoDB table



# DynamoDB TTL (Time To Live)

- TTL lets you automatically delete expired items based on a timestamp attribute.
- You specify a numeric attribute (e.g. expiresAt) containing a Unix epoch time (in seconds).
- DynamoDB checks items in the background and removes expired ones asynchronously
- No extra cost for TTL deletion; only standard read/write charges apply when you insert/update items.
- **Helps reduce storage cost and keeps tables clean.**
- **Use cases:**
  - User sessions/tokens - Automatically remove expired login sessions or JWT tokens.
  - Rate-limiting counters - Keep count of API hits per minute and auto-reset them via TTL.
  - OTP / verification codes - Remove expired OTP codes, email verification tokens, etc.



Current time  $\geq$  1733151000

| userId | sessionId | expiresAt  |
|--------|-----------|------------|
|        |           | 0          |
| 22222  | b0b1c2d4  | 1733261076 |
| 33333  | c0b1c2d5  | 1735859237 |

User session table

# DynamoDB Backup

DynamoDB supports Point In Time Restore and On-demand manual backups

## Point In Time Restore (PITR)

- PITR continuously backs up the DynamoDB table
- PITR allows restore the DynamoDB table to any second in the last 35 days.
- The recovery creates a new table

## On-demand Backup

- On-demand backup creates a full manual snapshot that you can keep indefinitely.
- Can use AWS Backup service to create the backups including cross-region copy
- Restoring a backup always creates a new table.

# DynamoDB Export / Import

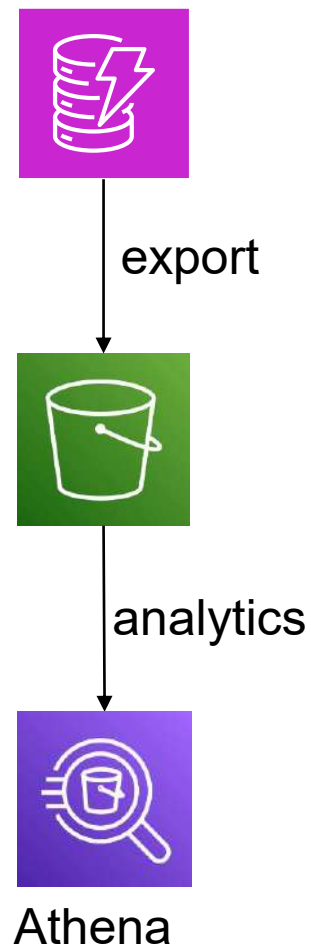
DynamoDB supports exporting data to S3 and importing data from S3

## Export to S3

- Uses PITR snapshots, so you can export data from a specific point in time.
- Export does not consume DynamoDB Read/Write capacity
- Useful for analytics, data pipelines, and long-term data storage.

## Import from S3

- Allows you to import data directly from Amazon S3 into a new DynamoDB table.
- Supports CSV, DynamoDB JSON, and ION formats.
- No write capacity consumed during import (doesn't use WCUs).
- Import creates a new table — it cannot overwrite an existing one.
- Useful for bulk loading, migrations, and rebuilding tables from exported data.





# Amazon DynamoDB - Summary

- Amazon DynamoDB is a fully managed, serverless cloud-native **NoSQL** database service that provides millisecond latency at any scale
- Has Partition key, Sort key (optional) and attributes. Partition key + Sort key = Primary Key
- Query and Scan operations – Query fetches specific item, Scan fetches all items in the table. Optionally use GSI.
- Supports Standard Table class and Infrequent Access (IA) Table class
- Supports Provisioned Capacity mode (default) and On-demand capacity mode (RCU/WCU)
- Supports Global tables (Active-Active) for global applications.
- Support DynamoDB accelerator (DAX) providing microsecond latency
- Supports Streams to act on table level modifications. Integrate with other AWS services like Lambda, SQS, SNS. Kinesis for stream processing.
- Supports Time-To-Live (TTL) to automatically expire and delete the items from the table
- Supports PITR and On-demand backups. PITR allows restoring a new table (up to 35 days)
- Supports Exporting data to S3 and importing data from S3

# What is document?

- A JSON-like schema with nested key-value structure

## Use cases:

- User profile information (like on LinkedIn)
- Product details, reviews (eCommerce)

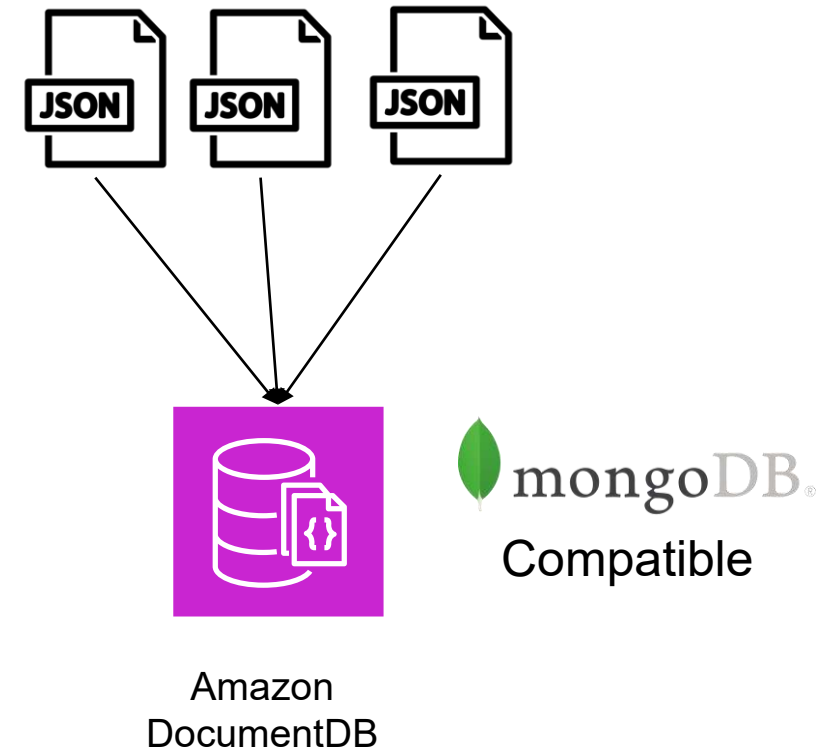
```
{
  "ProductID": "12345",
  "Category": "Mobile Phones",
  "Brand": "ExampleBrand",
  "Model": "ExampleModel X1",
  "Specifications": {
    "ScreenSize": "6.5 inches",
    "Resolution": "1080x2400 pixels",
    "Processor": "Octa-core 2.3 GHz",
    "RAM": "8 GB",
    "Storage": "128 GB",
    "Battery": "4000 mAh",
    "OS": "ExampleOS 12",
    "Camera": {
      "Rear": "48 MP + 8 MP + 5 MP",
      "Front": "16 MP"
    },
    "Dimensions": {
      "Height": "160 mm",
      "Width": "74 mm",
      "Depth": "8 mm",
      "Weight": "180 g"
    },
    "Connectivity": ["Wi-Fi", "Bluetooth 5.0", "NFC", "4G LTE"],
    "Ports": ["USB Type-C"]
  },
  "Price": 699.99
}
```

# Amazon DocumentDB

- Amazon DocumentDB (with **MongoDB** compatibility) is a fast, reliable, and fully managed database.
- Used to store, query, and index JSON-like documents
- DocumentDB storage automatically grows in increments of 10GB, up to 64 TB.
- Automatically scales to workloads with millions of requests per seconds.

## Use cases:

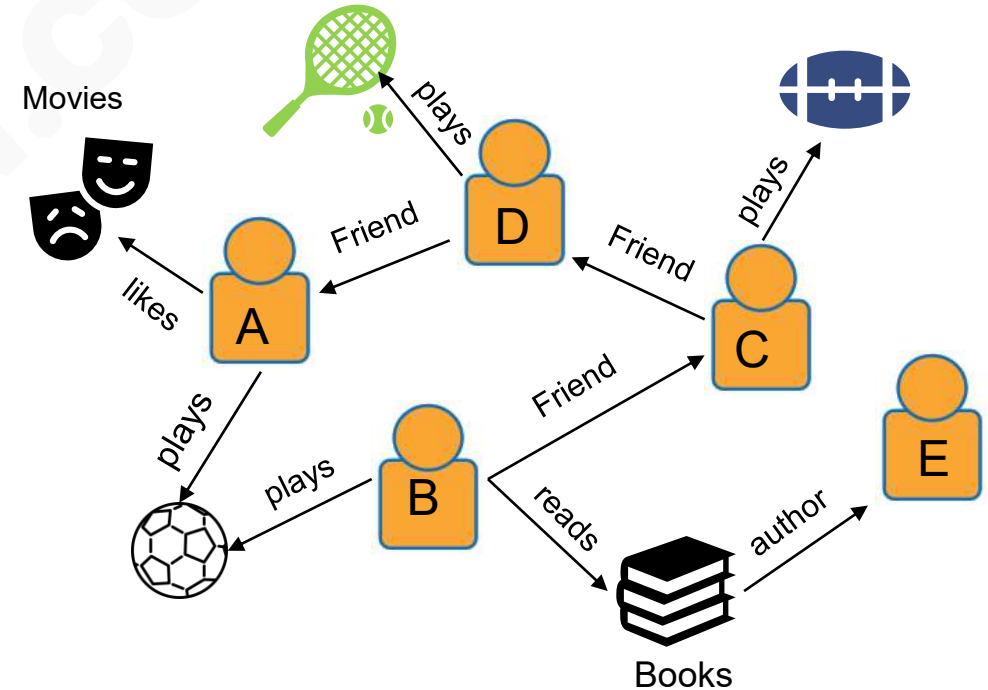
- User profile
- Content management like blogs, video metadata
- E-commerce product catalog (similar to DynamoDB)



# Amazon Neptune

- Graph databases store data as a network of entities and relationships.
- Graph consists of Node and Edges where Node represents the object (e.g. person, game) and edge represents relationship between the nodes (e.g. is friend of, plays).
- **Amazon Neptune is Serverless Graph database** (with all the features such as HA, replication etc.)
- Can analyse graph datasets with tens of billions of relationships within seconds using built-in algorithms
- Can perform similarity searches on vectors stored along with your graph for gen AI apps.

**Use cases:** Social networking, Fraud detection, Recommendation, engines, Route optimization, Knowledge graphs (e.g. Wikipedia)



Amazon Neptune

# Amazon Timestream, Amazon QLDB



## Amazon Timestream

- Time series Database
- Up to 1000 times faster and 1/10th of cost of Relational databases
- InfluxDB compatible

Use cases:

- Live analytics
- Real time IoT data ingestion and analytics
- Real time Web traffic, Operational metrics

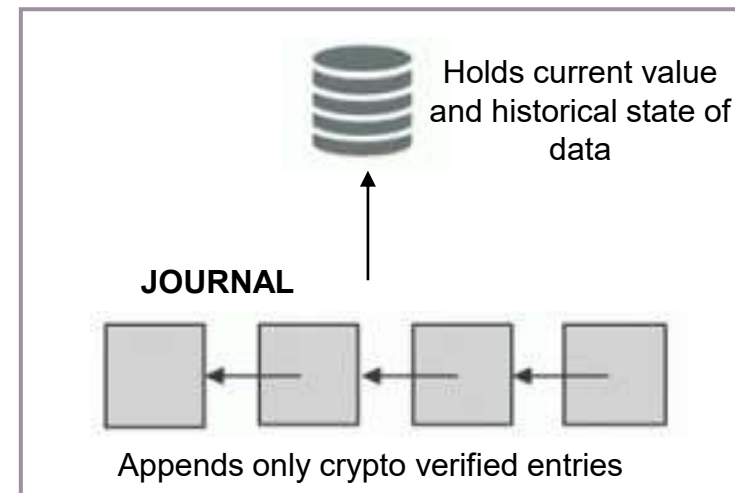


## Amazon QLDB

- Quantum Ledger Database
- Immutable chain of records
- Cryptographically verifiable log of data changes

Use cases:

- Financial records
- Supply chain system
- Claim history
- Trace and Tracking systems e.g. spare parts/inventory movement



# Exam Scenarios

# AWS Databases - Exam Scenarios

**1. Scenario:** A global e-commerce company wants an active-active database across multiple regions so customers can place orders from anywhere with low latency.

**Key-words:** Active-Active, multiple regions, place orders, from anywhere, low latency

**Answer: DynamoDB Global Tables**

Why not Amazon Aurora Global Database -> Place orders is a write operation and Aurora supports Read-replicas across the regions

# AWS Databases - Exam Scenarios

**2. Scenario:** A ride-sharing app needs to cache frequently accessed data like driver locations to reduce database load and achieve microsecond latency.

**Key-words:** Cache

**Answer: ElastiCache or DynamoDB Accelerator (DAX)**

It could be either of ElastiCache and DAX and if answer just contains one of them then it's an easy pick. However, if both the options are there, then look for additional information e.g. source database is DynamoDB then it will be DAX or if the source database is RDS/Aurora or it mentions about advance features then it will be ElastiCache



# AWS Databases - Exam Scenarios

**3. Scenario:** A streaming platform wants to store large volumes of user activity logs but only needs them for occasional analytics queries.

**Key-words:** occasional, analytics queries

**Answer:** DynamoDB IA Table class or DynamoDB export to S3

Depending on additional context, if its about saving the storage cost then answer will be DynamoDB Infrequent Access Table class or if its about analyzing the data using Athena or other tools then answer will be exporting the data to S3.

# AWS Databases - Exam Scenarios

**4. Scenario:** A ride-sharing platform wants to quickly determine the shortest path between drivers and riders based on a complex graph of roads and connections.

**Key-words:** shortest path, graph, connections

**Answer:** Amazon Neptune

# AWS Databases - Exam Scenarios

**5. Scenario:** A team needs to quickly create a full copy of their production database to run performance tests and simulate heavy workloads without affecting the live system. The copy should be created within minutes.

**Key-words:** full copy of production, within minutes

**Answer: Aurora Cloning**

You may think of Read replicas but the scenario doesn't mention about only reading the data. Further, Aurora's decoupled and shared storage makes the cloning really fast (copy-on-write) and hence the time to create database clone is faster than other possible options.

# DynamoDB DAX vs ElastiCache

## DynamoDB DAX

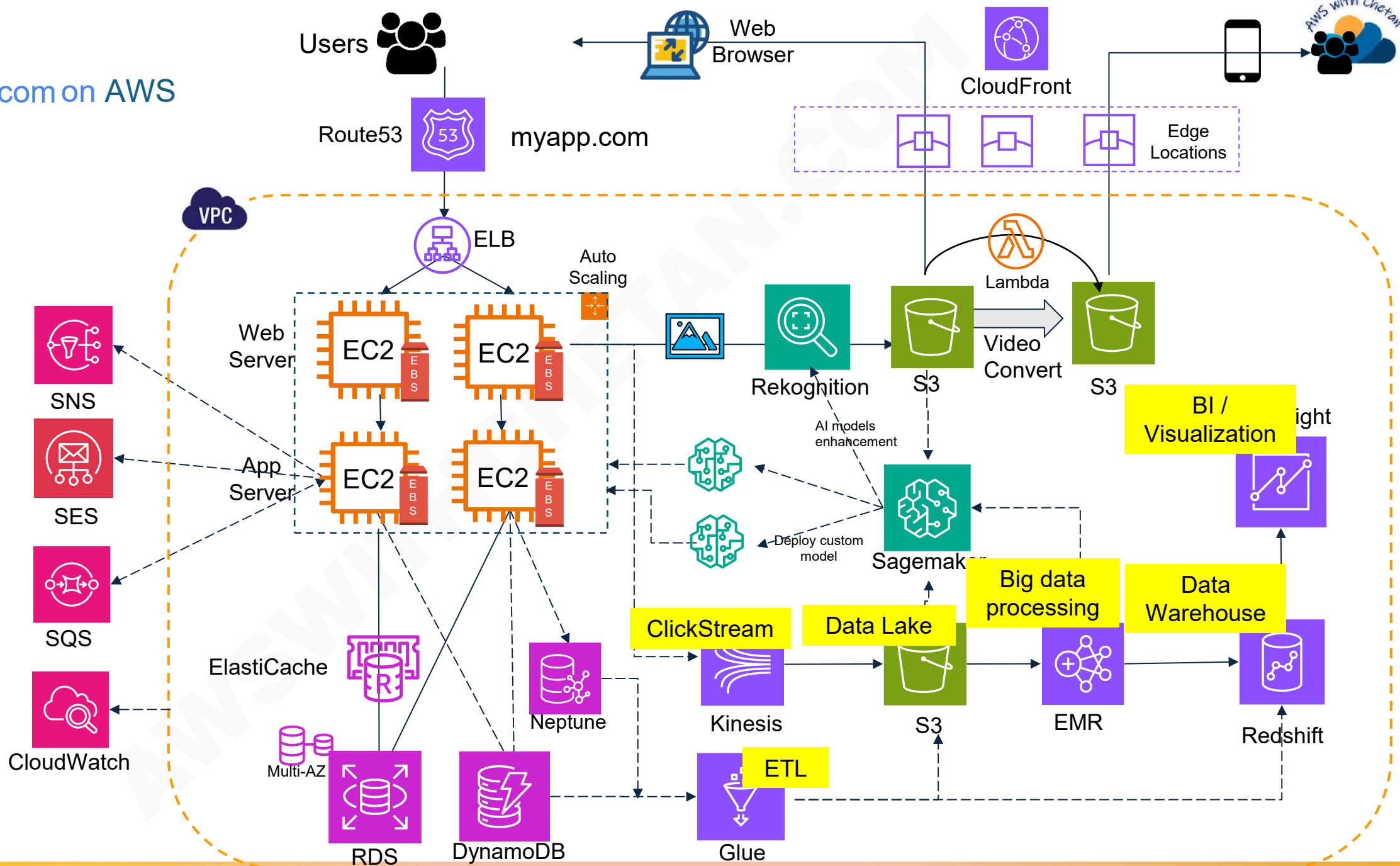
- Purpose-built cache only for DynamoDB and fully API-compatible with DynamoDB.
- Best for repeated read queries on DynamoDB data (hot keys, strongly read-heavy apps).
- No need to manage invalidation — DAX auto-syncs with DynamoDB.
- **When to use?**
  - ✓ You want to speed up DynamoDB reads without changing code.

## ElastiCache

- General-purpose, in-memory cache with advanced features like pub/sub, sorted sets, streams, Lua scripts, etc.
- Can cache any type of data - SQL queries, API responses, session data, ML results, etc.
- Requires your application to implement cache logic + invalidation.
- **When to use?**
  - ✓ You need a cache for multiple systems
  - ✓ You want advanced capabilities (counters, leaderboards, TTL per key, real-time messaging).
  - ✓ You want to cache complex data models or large aggregated results.

# Big data and Analytics

# myapp.com on AWS



# Before we move..

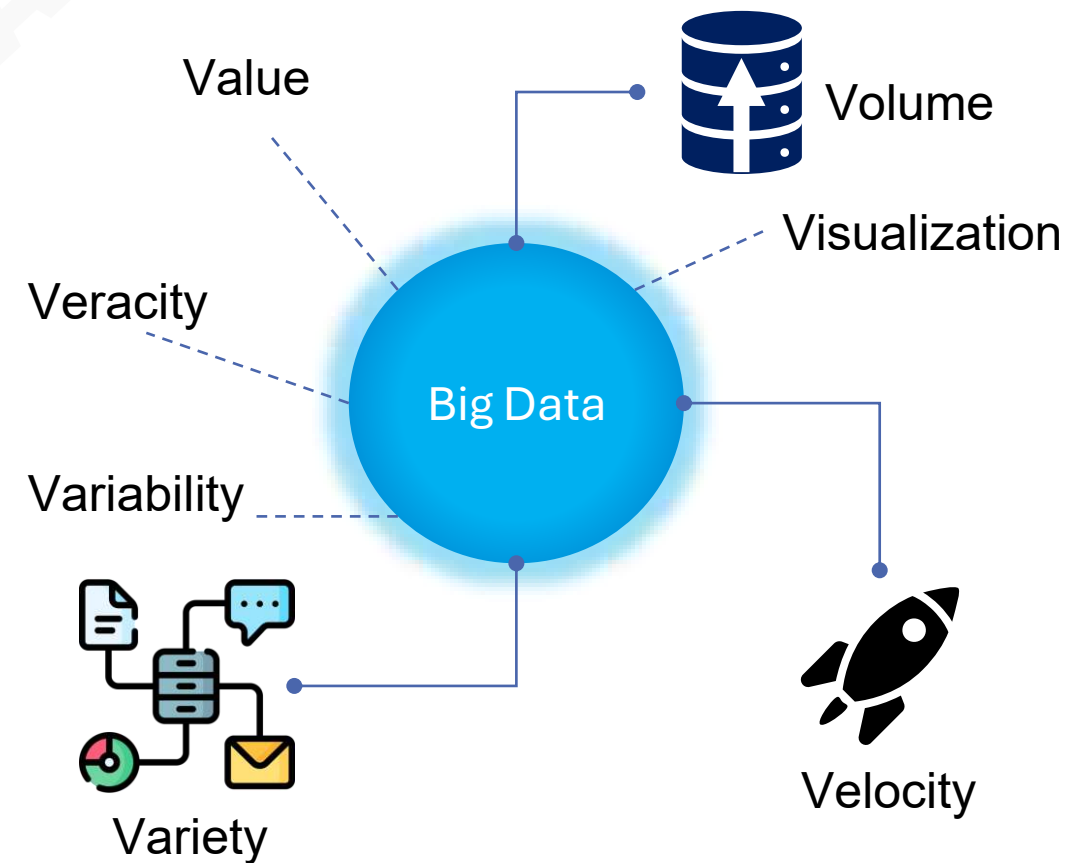
- Bigdata
- OLTP & OLAP
- Data warehouse
- Data Lake
- ETL
- Data formats – CSV / Parquet / Iceberg

# What is Big Data?

**Big Data** refers to extremely large datasets that cannot be easily managed, processed, or analysed using traditional data processing techniques.

## Big data is characterized by Three V's:

- **Volume:** The amount of data generated every second from various sources like social media, sensors, transactions, etc.
- **Velocity:** The speed at which new data is generated and needs to be processed. This includes real-time data feeds and streaming data.
- **Variety:** The different types of data, such as structured data (databases), semi-structured data (XML, JSON), and unstructured data (text, images, videos).





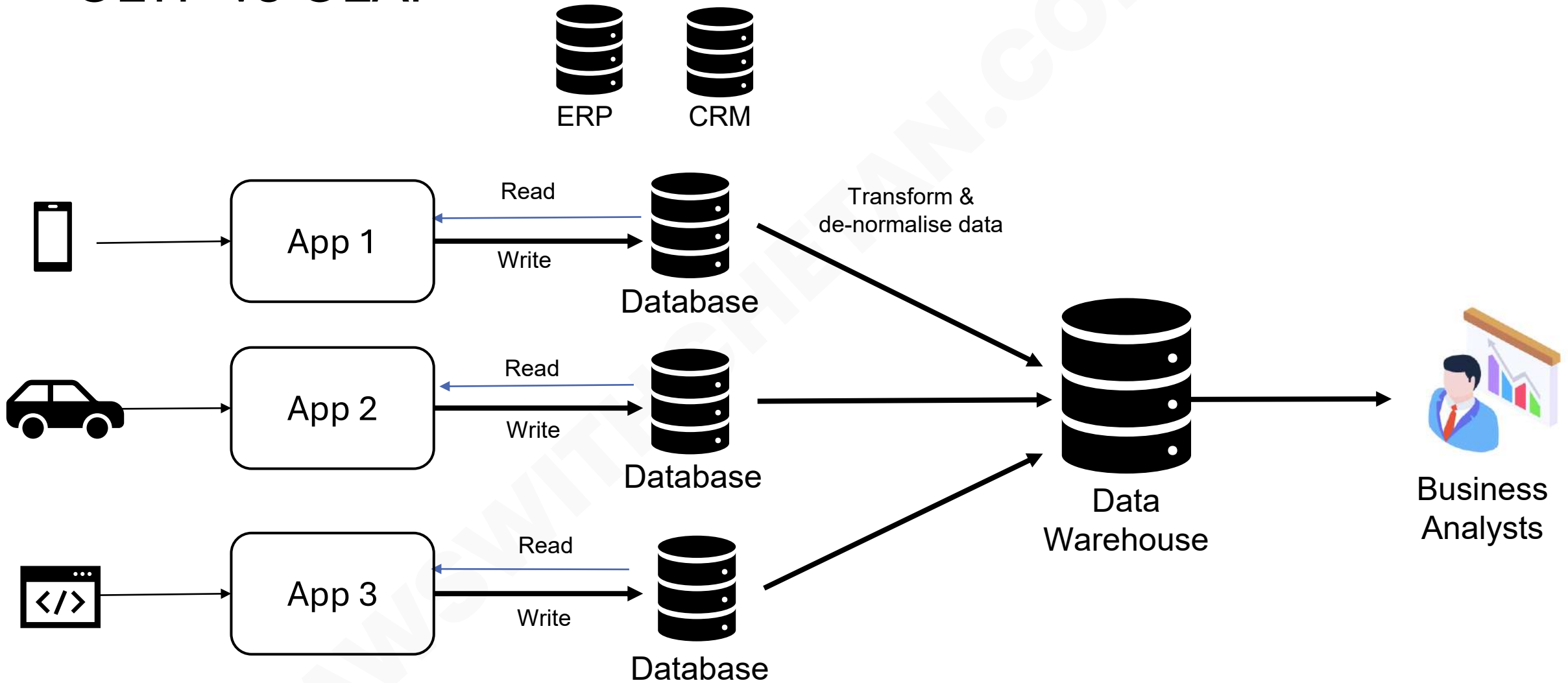
# Big data frameworks..

Big data frameworks are instruments that simplify the processing of big data.

## Popular Big Data frameworks

- **Hadoop:** Open-source batch-processing framework for the distributed storage and processing of big data sets. Hadoop operates by splitting files into large blocks of data and then distributing those datasets across the nodes in a cluster. Uses HDFS, MapReduce, YARN.
- **Apache Hive:** For data summarization, query, and analysis. Queries data from HDFS.
- **Apache Hbase:** A distributed big data store that supports structured data storage for large tables
- **Apache Spark:** In-memory real-time processing, batch processing
- **Presto:** Distributed SQL query engine optimized for running interactive analytic queries against data sources of all sizes ranging from gigabytes to petabytes.
- **Apache Pig, Apache Sqoop, Apache Flume, Apache Mahout** and more..

# OLTP vs OLAP

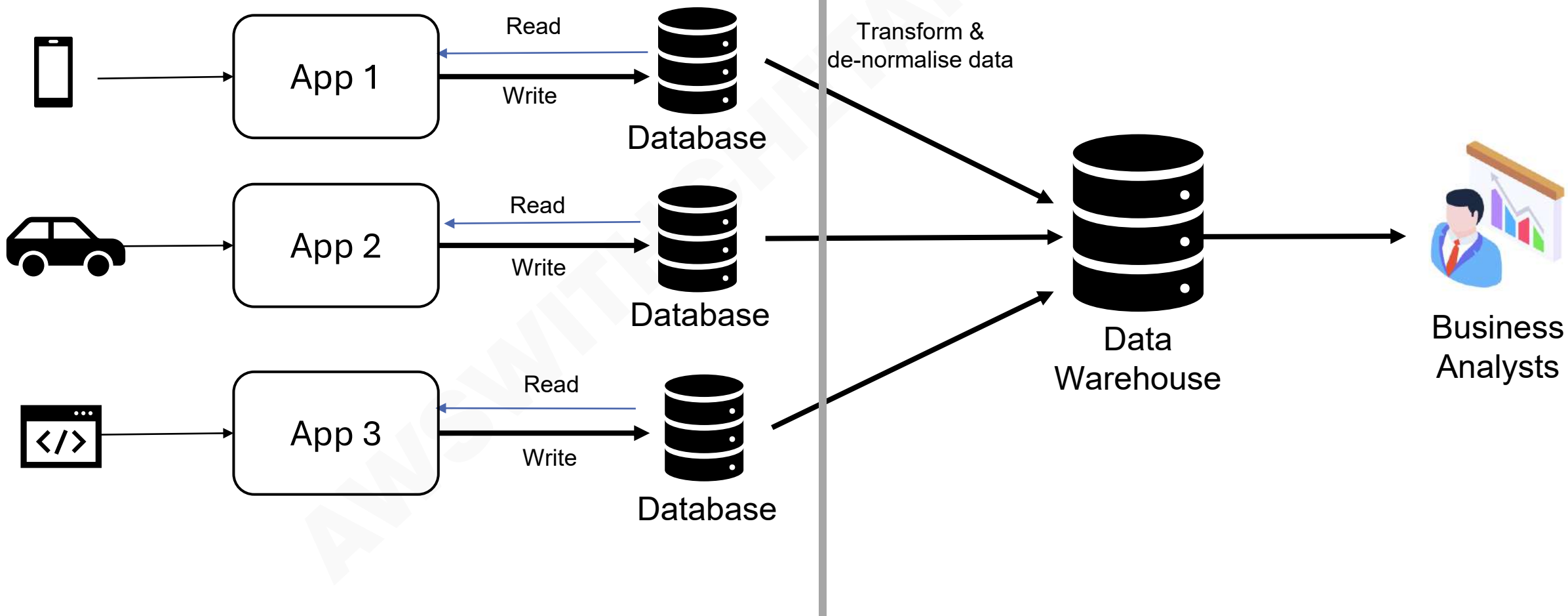


# OLTP

Online transaction Processing

# OLAP

Online Analytical Processing

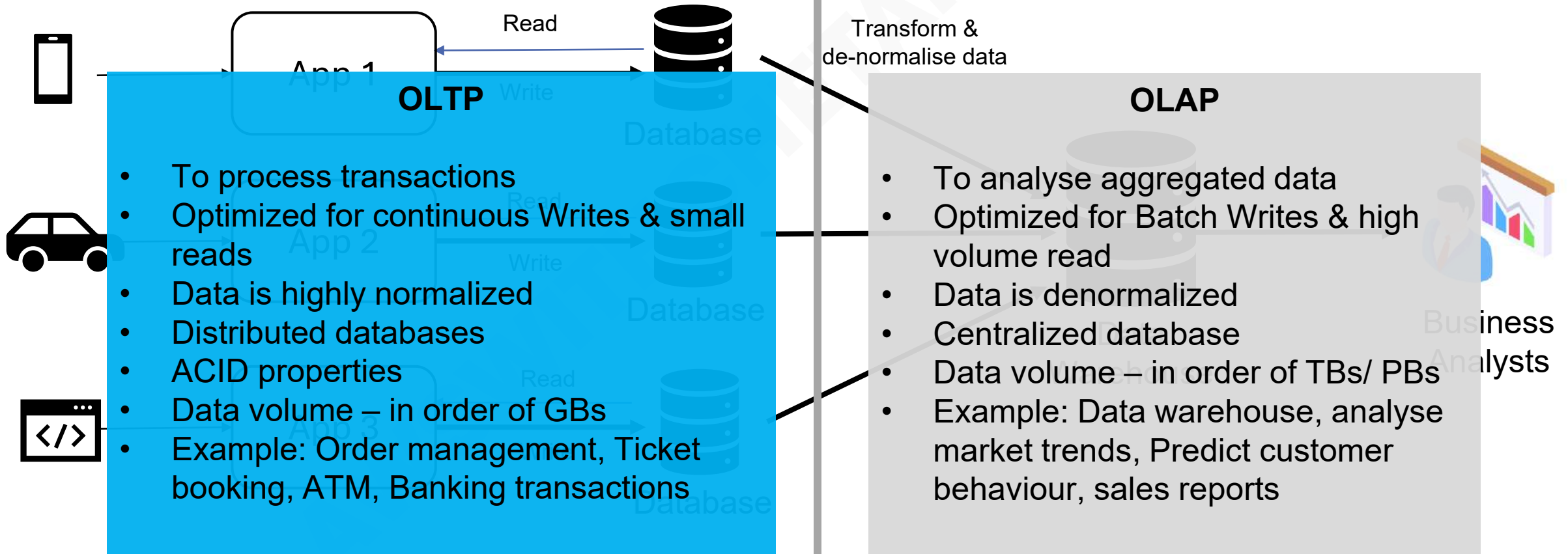


# OLTP

Online transaction Processing

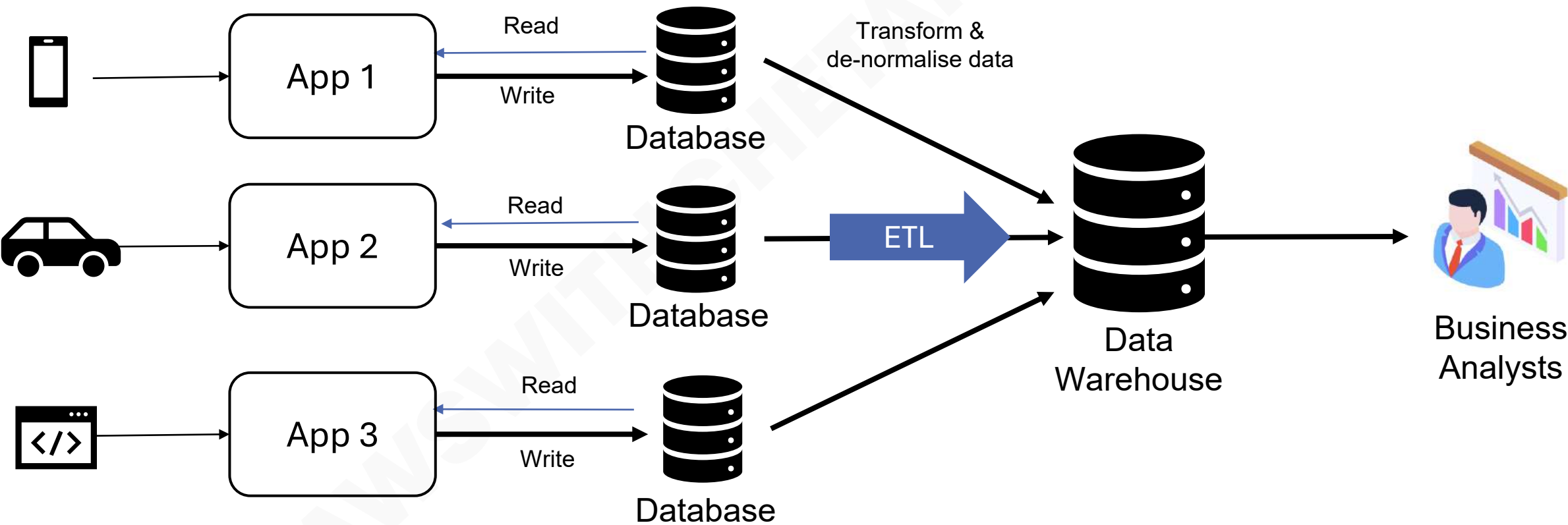
# OLAP

Online Analytical Processing



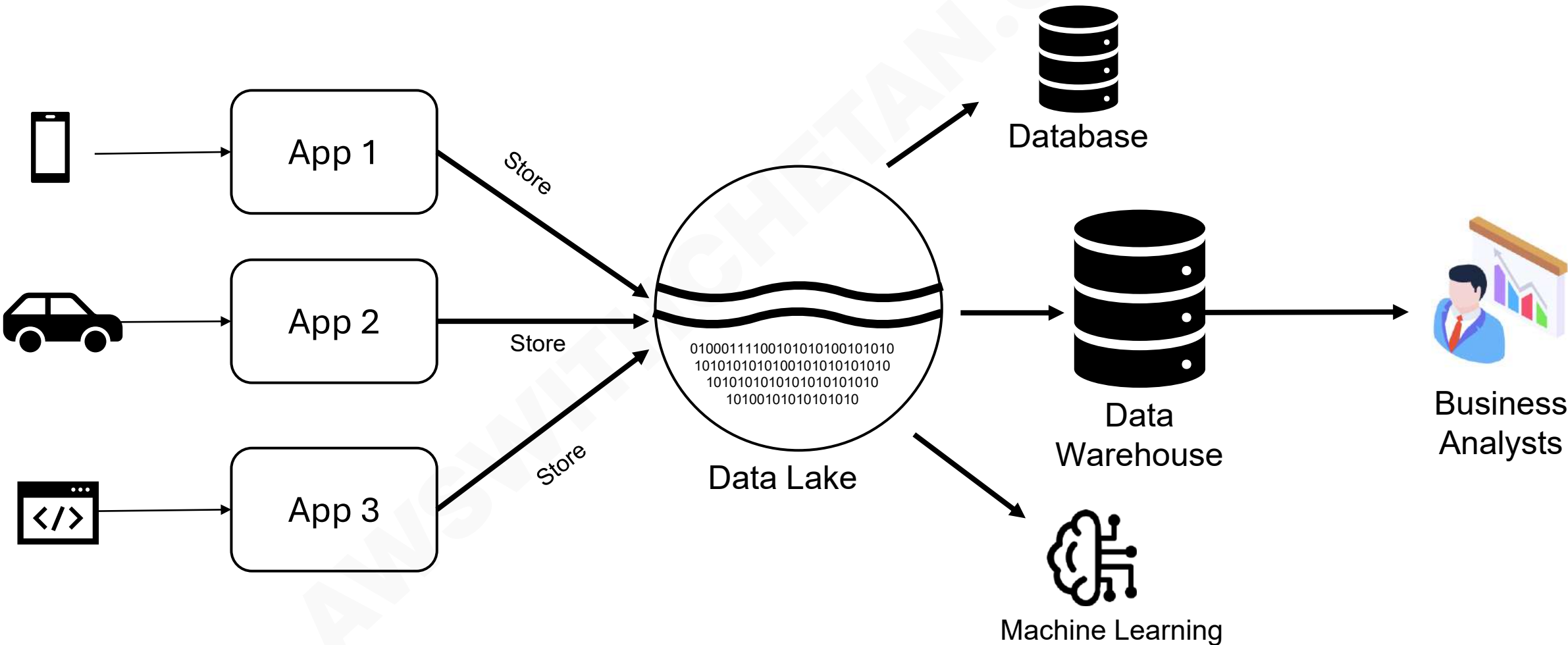
# Data Warehouse

A data warehouse is a system that stores and organizes data from multiple sources for reporting and analysis



# Data Lake

A data lake is a centralized repository for storing, processing, and securing large amounts of data



# Common Data Storage Formats in Data Lakes

## Row-based format

- Stores data by rows
- Simple and Human-readable
- Poor performance for analytics
- No support for schema evolution, versioning and transactions

Ex: CSV, JSON, TSV



## Columnar-based format

- Stores data by columns
- High performance for analytics
- Supports compression
- Designed for Data lakes
- Optimized for engines like Spark, Athena, Redshift spectrum

Ex: Apache Parquet



## Table-based format

- Stores data by columns (generally in parquet / ORC format)
- High performance for transactional data lakes
- Supports ACID
- Supports schema evolution
- Supports Updates/deletes
- Supports Time travel

Ex: Apache Iceberg



## Row based storage

| Name | Age | Country |
|------|-----|---------|
| John | 25  | USA     |
| Anna | 30  | Canada  |
| Mike | 28  | UK      |



|      |    |     |      |    |        |      |    |    |
|------|----|-----|------|----|--------|------|----|----|
| John | 25 | USA | Anna | 30 | Canada | Mike | 28 | UK |
|------|----|-----|------|----|--------|------|----|----|

## Column based storage

| Name | Age | Country |
|------|-----|---------|
| John | 25  | USA     |
| Anna | 30  | Canada  |
| Mike | 28  | UK      |

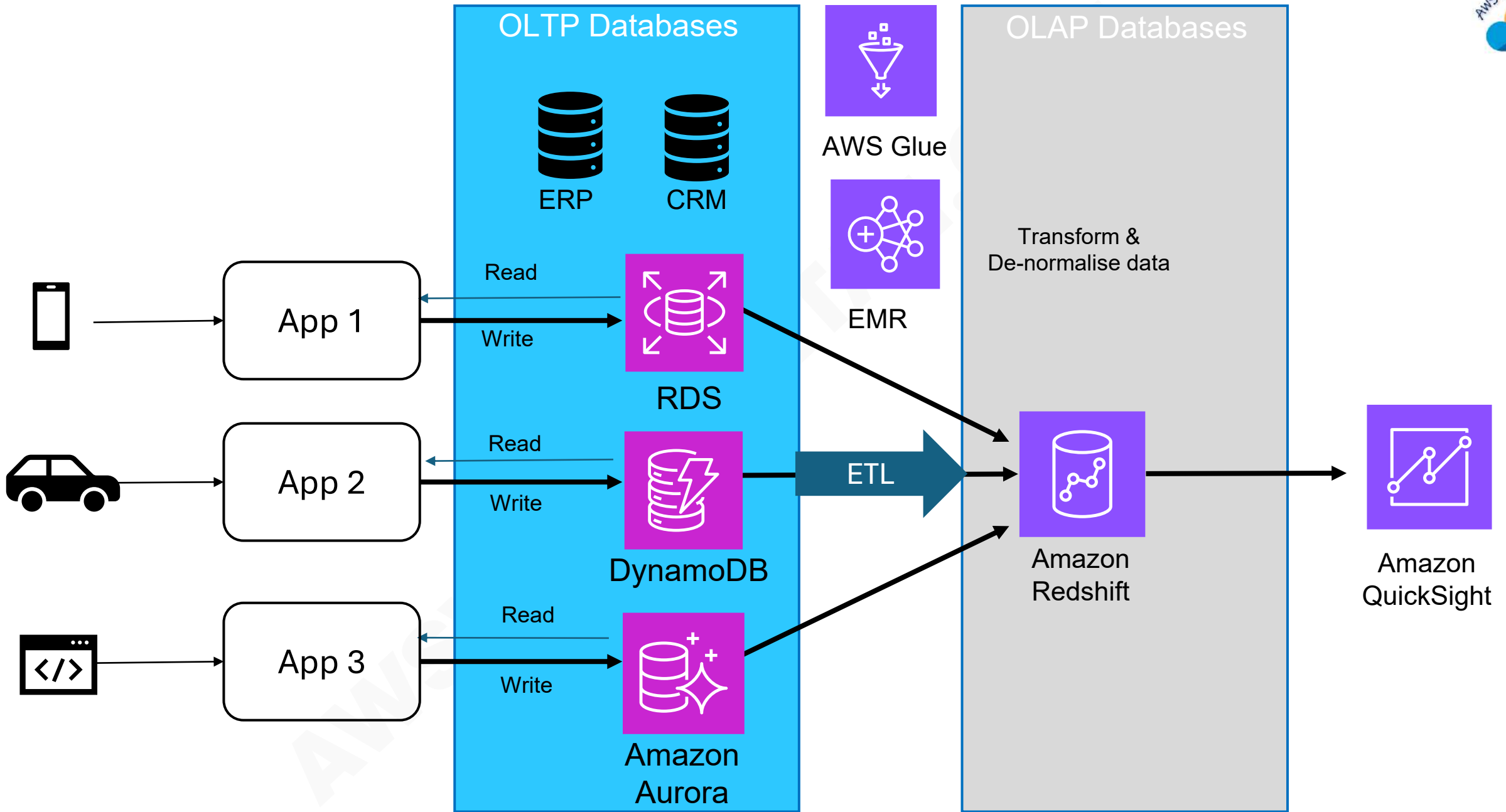


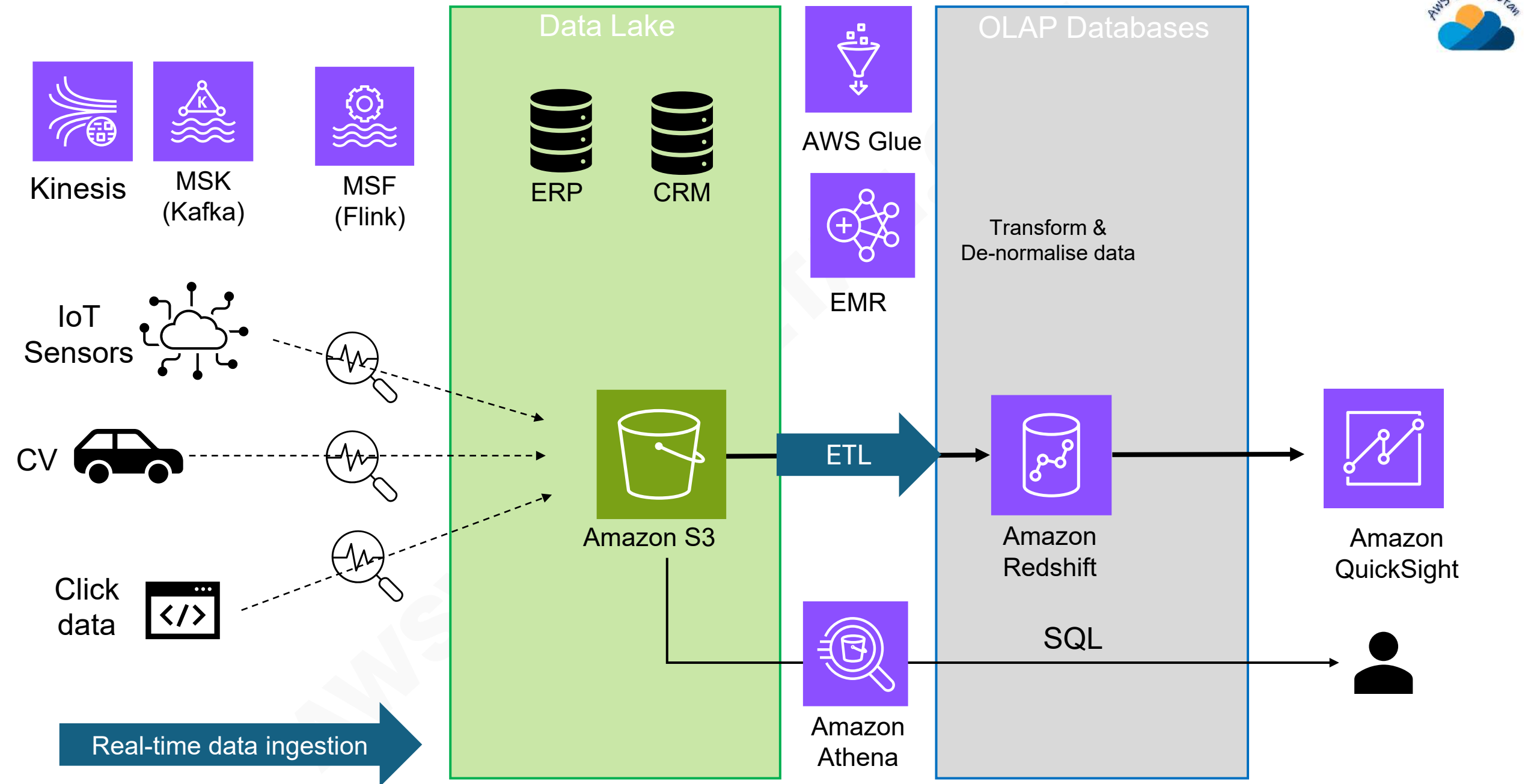
|      |      |      |    |    |    |     |        |    |
|------|------|------|----|----|----|-----|--------|----|
| John | Anna | Mike | 25 | 30 | 28 | USA | Canada | UK |
|------|------|------|----|----|----|-----|--------|----|





# AWS Data Analytics services

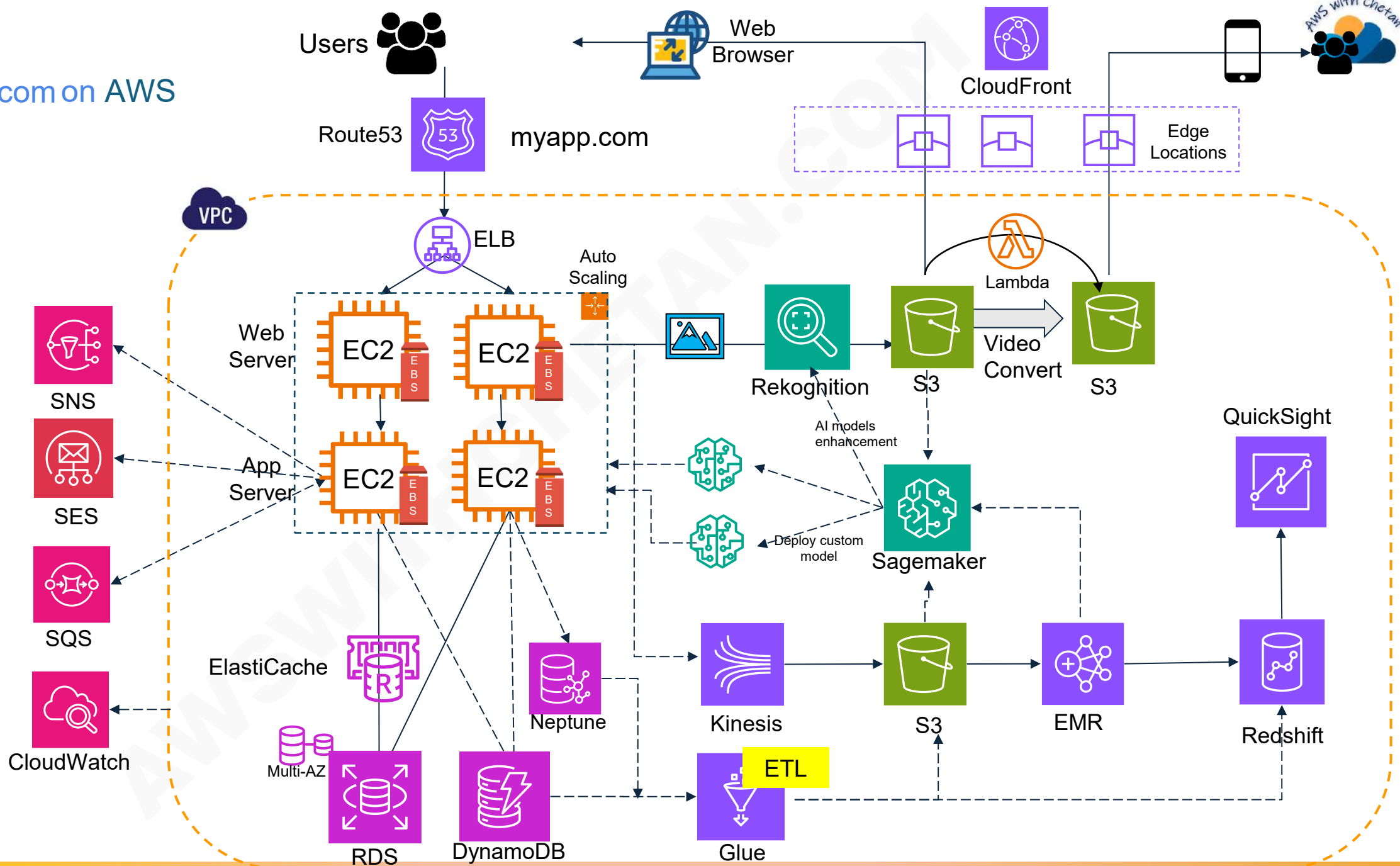






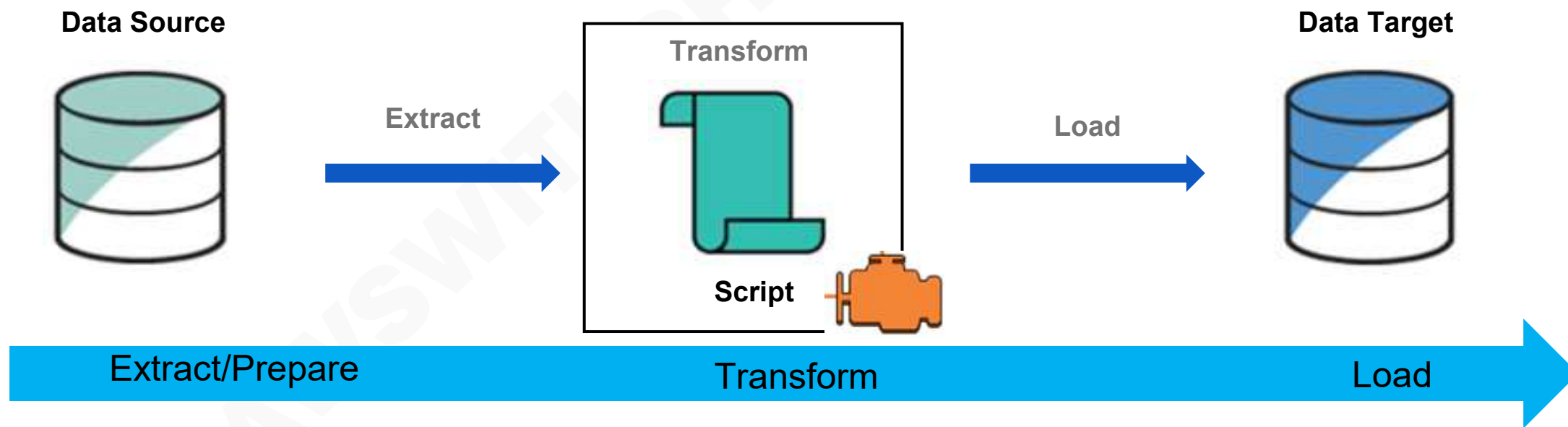
# AWS Glue

# myapp.com on AWS



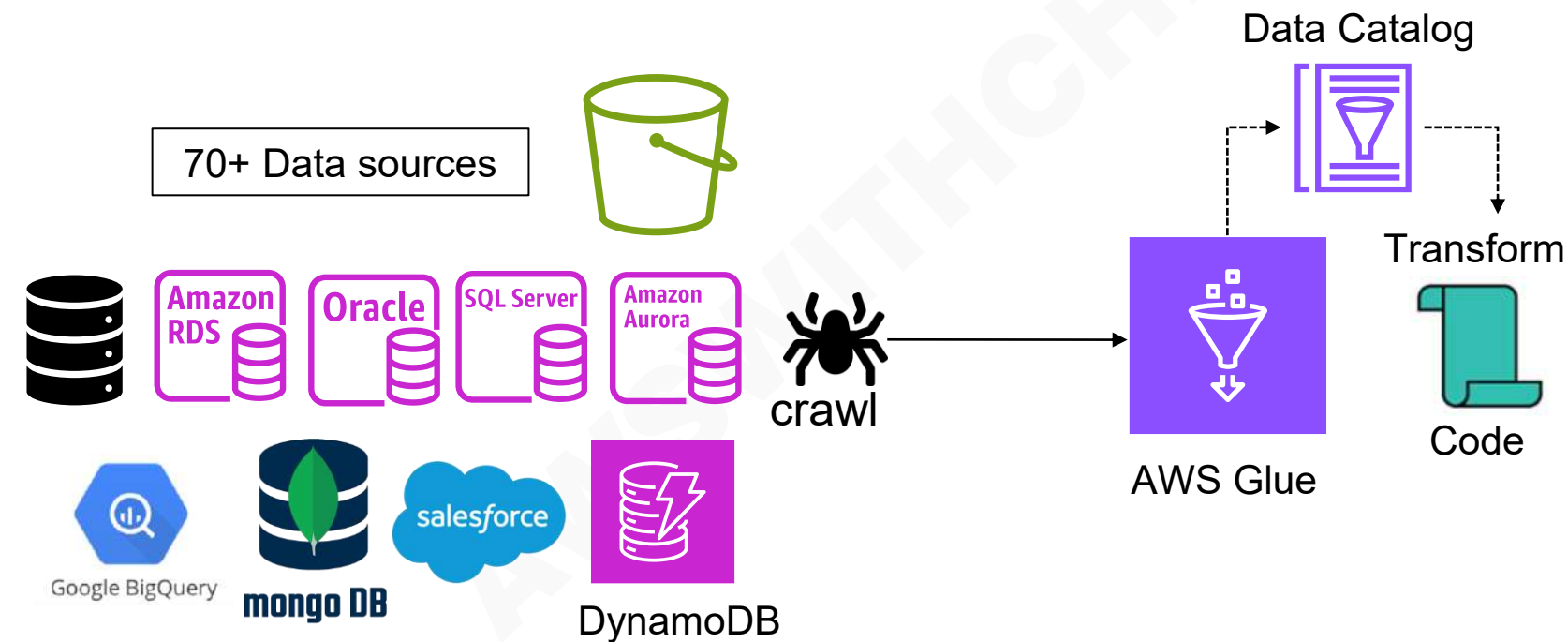
# AWS Glue

- Serverless Data integration and ETL (**E**xtract, **T**ransform, and **L**oad) service.



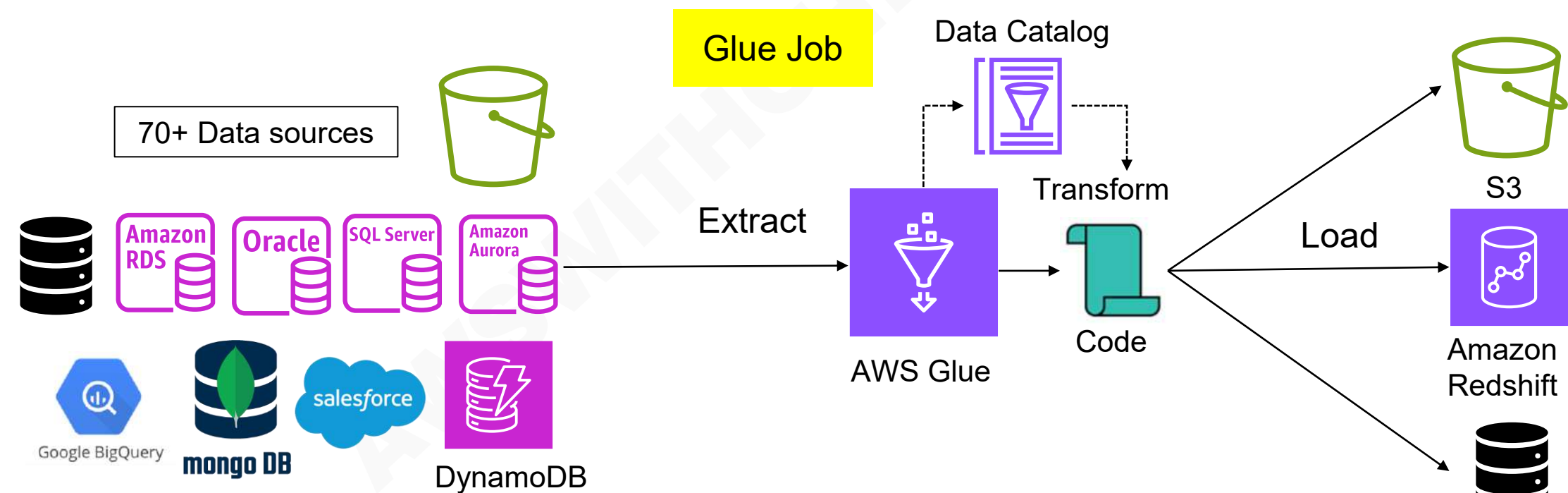
# AWS Glue

- Serverless Data integration and ETL (**E**xtract, **T**ransform, and **L**oad) service.
- Automatic Schema discovery and Data cataloging with Glue Crawler.
- Supports Data transformation (using Apache Spark / Python shell / Ray engines)



# AWS Glue

- Serverless Data integration and ETL (**E**xtract, **T**ransform, and **L**oad) service.
- Automatic Schema discovery and Data cataloging with Glue Crawler.
- Supports Data transformation (using Apache Spark / Python shell / Ray engines)





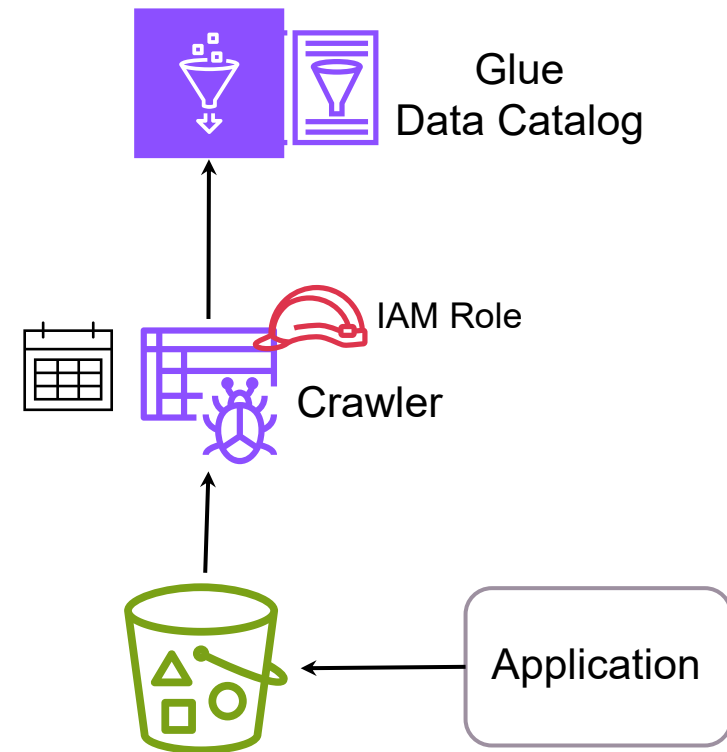
# AWS Glue Features – Good to know

- Glue Data Catalog and Crawlers
- Glue Jobs and Bookmarks
- Glue Studio
- Glue Streaming ETL
- Glue Data encryption

AWSWITHCHETAN.COM

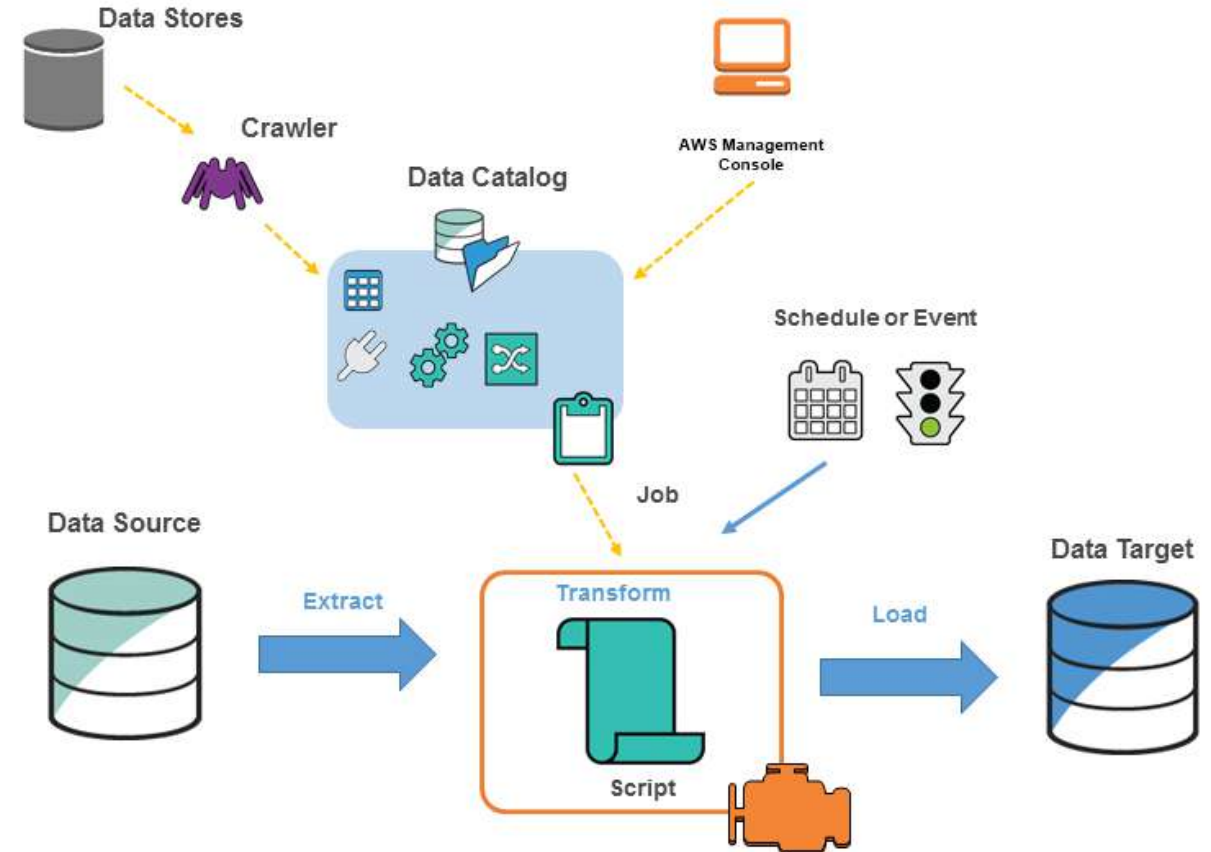
# Glue Data Catalog and crawlers

- **Glue Data Catalog** is a central metadata repository which acts as an index to the location and schema of the data sources
- Glue Data Catalog can be used by other AWS analytics services:
  - Amazon Athena
  - AWS Lake Formation
  - Amazon EMR
  - More..
- Data Catalog is automatically created and updated using **Glue Crawlers**
- You can run crawler on demand or on a schedule
- If the data source is an AWS service (for example, S3), the crawler IAM role must have permissions to read the S3 bucket or prefix.



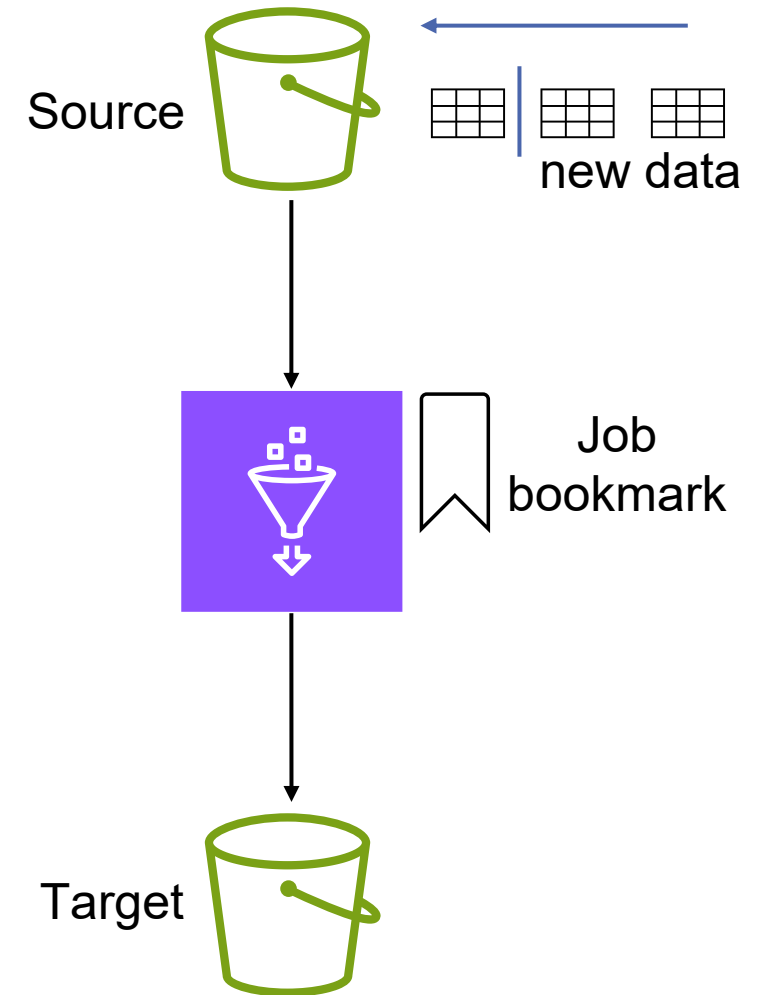
# Glue Jobs

- Define data source and target
- Define ETL jobs with transformation scripts to move and process data.
- Run jobs on-demand or based on triggers.
- Monitor job performance using dashboards.



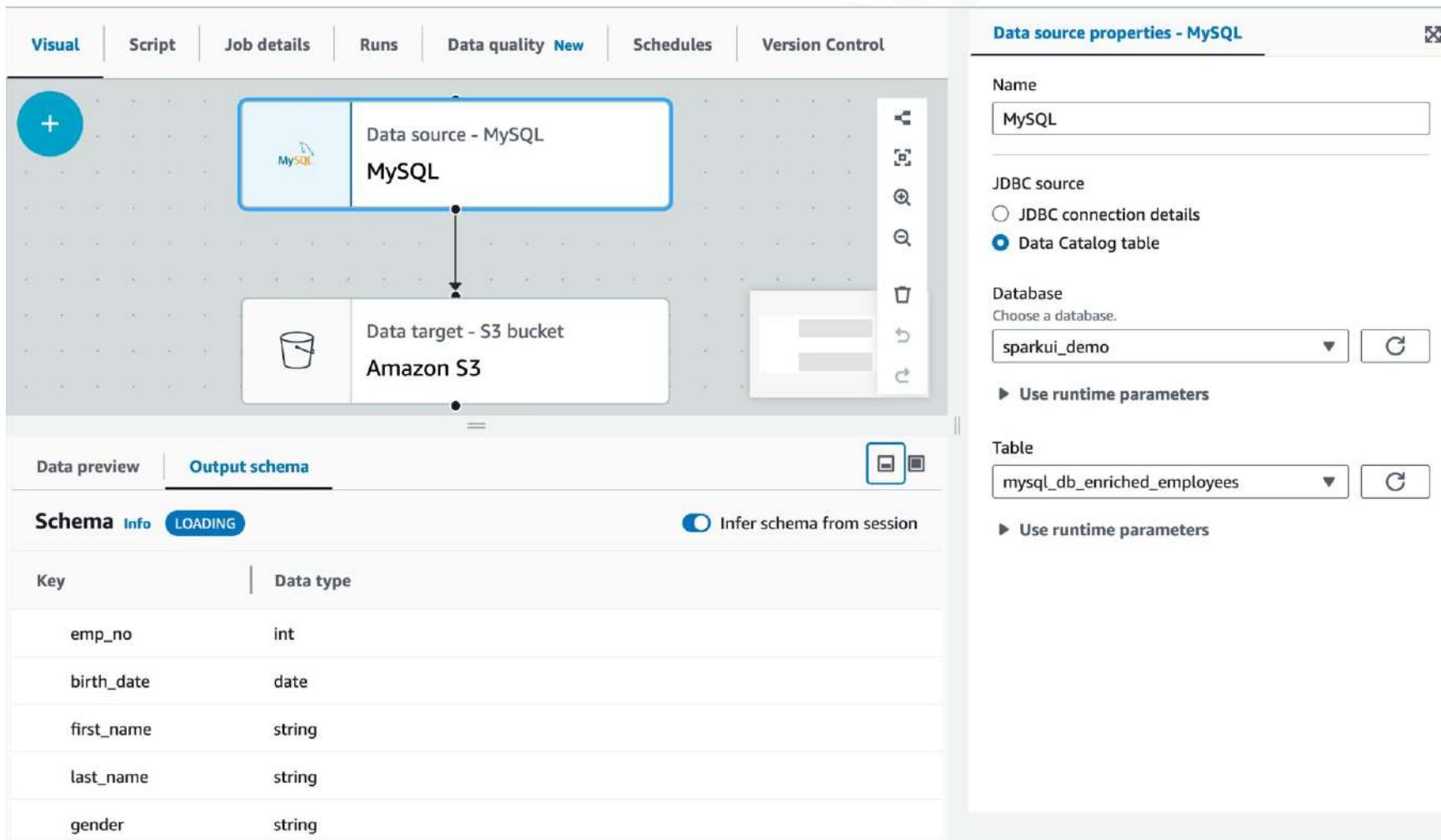
# Glue Job bookmarks

- Glue Job Bookmark remembers what data has already been processed by a Glue ETL job, so the next run processes only new or changed data.
- It saves the job run details like processed file paths, partition values, timestamps and job run state
- You can enable, disable or pause Job bookmark feature
- **Without bookmarks:**
  - Every job run reads all data again
  - Higher cost, time, and duplicate processing
- **With bookmarks:**
  - Glue processes only new files/records
  - Faster and cheaper ETL



# AWS Glue Studio (Visual ETL)

- AWS Glue Studio provides a visual interface for creating, running, and monitoring ETL jobs in AWS Glue.



The screenshot displays the AWS Glue Studio Visual ETL interface. The main workspace shows a workflow with two components: a 'Data source - MySQL' box and a 'Data target - S3 bucket Amazon S3' box, connected by a downward arrow. The top navigation bar includes tabs for Visual, Script, Job details, Runs, Data quality New, Schedules, and Version Control. The bottom left panel shows the 'Output schema' tab with a 'Schema' section and a 'LOADING' button. Below this is a table with columns 'Key' and 'Data type'.

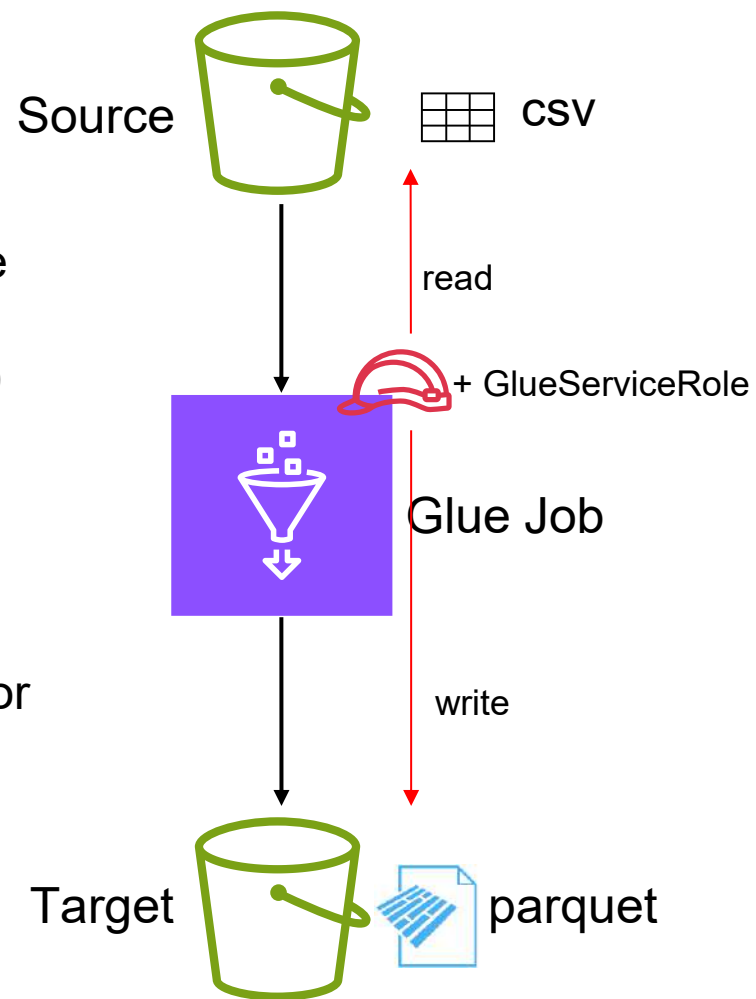
| Key        | Data type |
|------------|-----------|
| emp_no     | int       |
| birth_date | date      |
| first_name | string    |
| last_name  | string    |
| gender     | string    |

The right panel, titled 'Data source properties - MySQL', contains configuration options for the MySQL data source. It includes fields for Name (MySQL), JDBC source (Data Catalog table selected), Database (sparkui\_demo), and Table (mysql\_db\_enriched\_employees). There are also buttons for 'Use runtime parameters' and a refresh icon.

# Exercise: csv to parquet file conversion with Glue

1. Create source and target S3 bucket and load csv data in source bucket
2. Create an IAM role for Glue to allow S3 read/write and Glue job related permissions
  1. Include AWSGlueServiceRole IAM policy
  2. Create and add policy to include permissions for S3 read for source bucket and S3 write for target bucket.

*(Sample policy provided in the lecture resources. Replace your bucket names.)*
3. Create Glue job using Glue studio UI
4. Configure Source, Transformation and Target steps
  1. Source => S3 bucket / prefix (Data Format => CSV)
  2. Transformation => Change Schema (Choose columns to include or drop)
  3. Target => S3 bucket (Data Format => Parquet)
5. Save and run the job



# Sample IAM policy for S3 permissions

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadFromSourceBucket",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::chetan-glue-demo-source-bucket",
        "arn:aws:s3:::chetan-glue-demo-source-bucket/*"
      ]
    },
    {
      "Sid": "WriteToTargetBucket",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::chetan-glue-demo-target-bucket",
        "arn:aws:s3:::chetan-glue-demo-target-bucket/*"
      ]
    }
  ]
}
```

← Replace bucket names

← Replace bucket names

# Apache parquet

- Apache Parquet is a columnar storage file format designed for analytics.
- Stores data column-wise (not row-wise like CSV)
- Much smaller file size due to compression
- Faster queries with Athena / Redshift Spectrum
- Ideal format for data lakes on S3





# Row-based vs. Column-based File Formats

Example query: What is the average product price?

## Row-based (CSV)



| CSV File |       |              |
|----------|-------|--------------|
| Product  | Price | Availability |
| Apples   | \$5   | High         |
| Oranges  | \$3   | High         |
| Pears    | \$11  | Low          |

### Row-based Storage

- ✓ Stores data row by row
- ✓ Reads entire rows for queries
- ✓ Slower for analytics; full table scans

## Column-based (Parquet)



| Products | Price | Availability |
|----------|-------|--------------|
| Apples   | \$5   | High         |
| Oranges  | \$3   | High         |
| Pears    | \$11  | Low          |

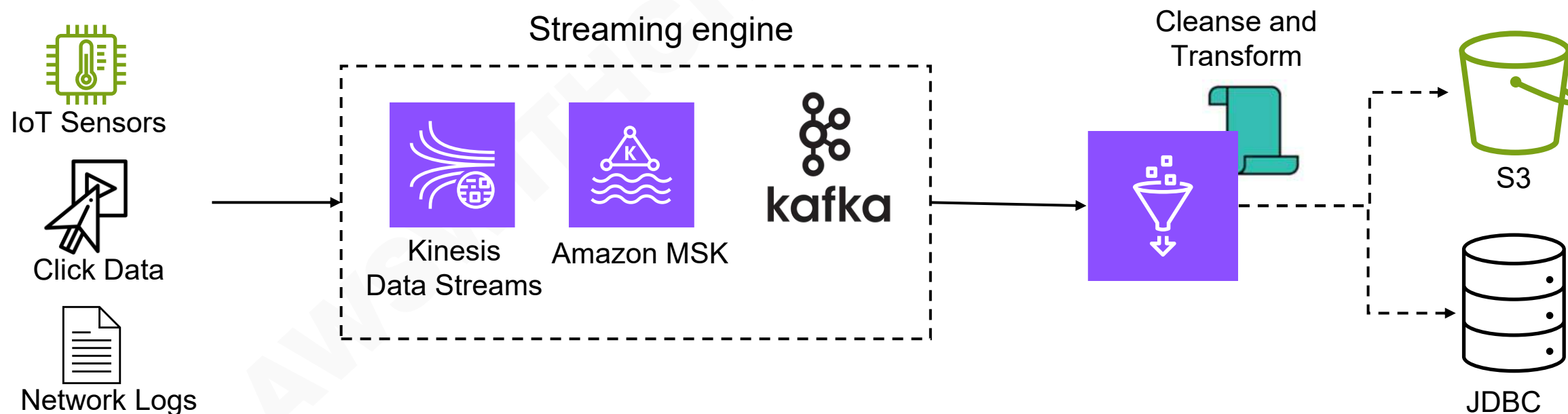
### Columnar Storage

- ✓ Stores data by columns
- ✓ Reads only relevant columns
- ✓ Faster for analytics; less data to scan

\*Image generated by Chatgpt

# Glue streaming ETL

- Allows ETL for real-time streaming data
- Supports Kinesis data stream and Kafka source and S3/JDBC as target
- Use Streaming ETL in AWS Glue to process event data like IoT streams, clickstreams, and network logs.



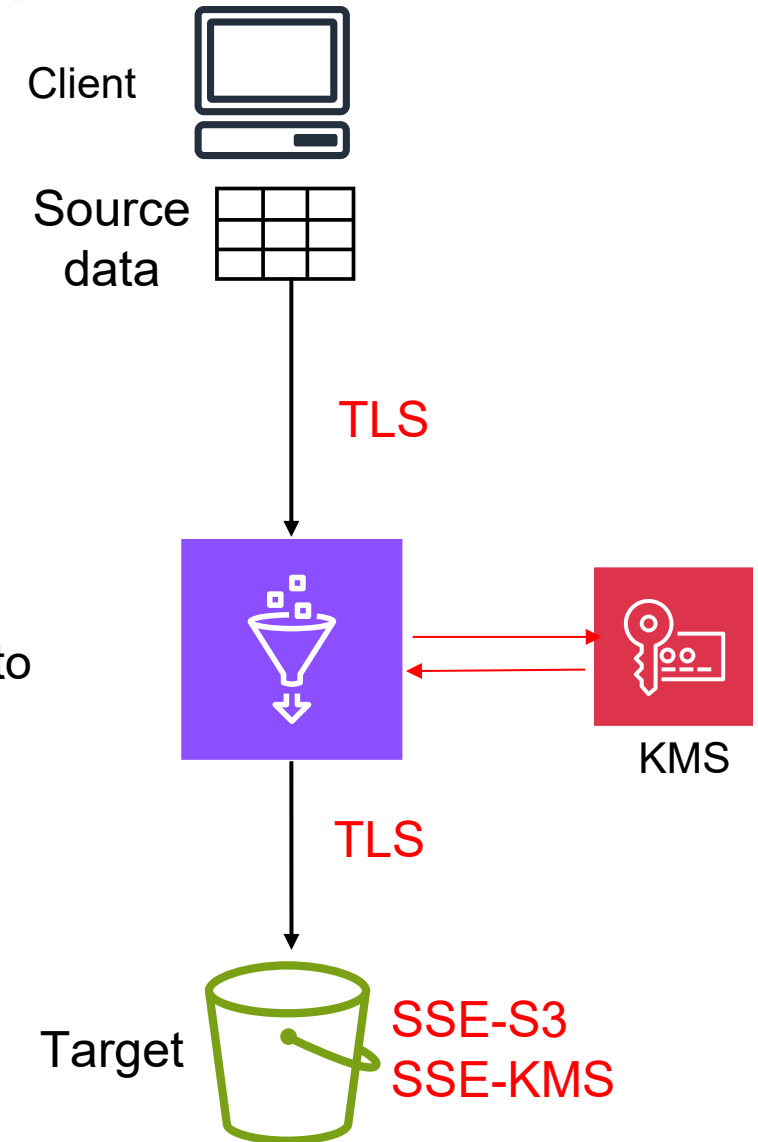
# Glue Data Encryption

## Data in Transit:

- Supports encryption for data in transit using TLS endpoints and TLS connections for source and the targets

## Data at Rest:

- Supports AWS KMS to enable encryption for Glue data catalog and Cloudwatch Logs generated by Glue crawler
- For S3 target, Use Security Configuration and attach it to Glue ETL jobs to enabled SSE-S3 or SSE-KMS encryption



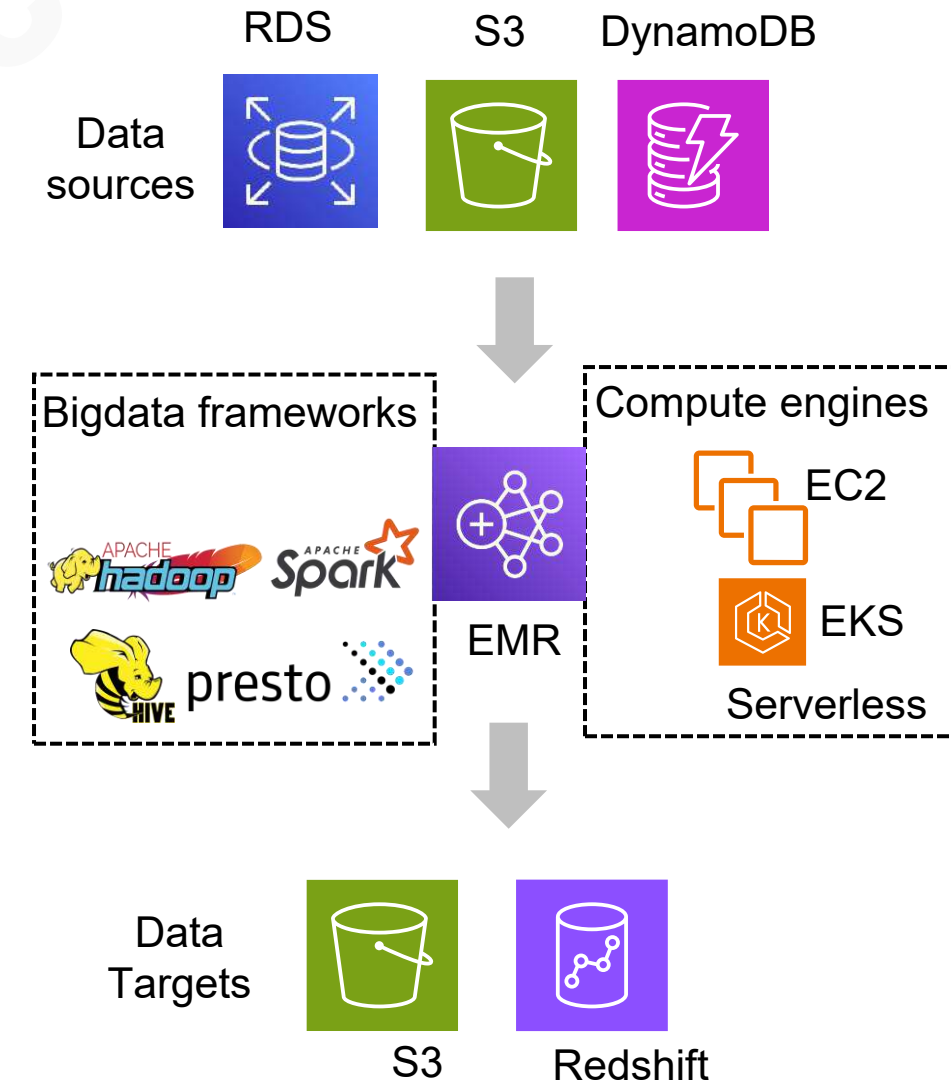


# Amazon EMR

Elastic Map Reduce

# Amazon EMR

- Amazon EMR (**Elastic MapReduce**) is used for general big data processing and analysis.
- Supports Hadoop Map Reduce and **Apache Spark**, Hive, Presto and many other frameworks
- Distribute your data and processing across resizable clusters of Amazon EC2, EKS. Also has serverless option.
- Auto-scaling and integrated with EC2 Spot instances.
- Uses HDFS (EBS/Instance store) and EMRFS (S3) for storage.
- **Use Cases:** Log analysis, Web indexing, Processing data for machine learning training, financial data analysis, market trends, customer preferences etc.



# AWS Glue vs Amazon EMR



## AWS Glue

- Primarily for running serverless ETL jobs, schema discovery and data catalogs
- **Simplicity** – Low code/no code (supports Spark, python and Ray engine)
- Suitable for ad-hoc & small batch jobs
- Fully managed and Serverless
- When to use AWS Glue?
  - **Visual and low code ETL development tools**
  - Need built-in capabilities like data source/target connectors, transformations, incremental load, job monitoring, orchestration.
  - Need centralized data catalogs and governance



## Amazon EMR

- Primarily for Big data processing and ETL
- **Flexibility** to run any big data framework i.e. Spark, Hadoop, Hive, HBase, Flink, Presto etc.
- Suitable for large scale distributed data processing with consistent usage
- Some level of infrastructure management
- When to use Amazon EMR?
  - **Hadoop Migration from on-prem**
  - Have expertise beyond just Spark, for ex. Hive, Presto
  - Customer is skilled in loading their own data source connector libraries for their jobs.





# Amazon Athena

# Amazon Athena

- Provides the easiest way to run ad hoc **interactive SQL queries** for data in Amazon S3 and other data sources without the need to setup or manage any servers.
- Athena is serverless, built on open-source Trino and Presto engines
- Pay only for the queries you run. Pricing: \$5.00 per TB of data scanned.
- Supports CSV, JSON, or columnar data formats such as Apache Parquet and Apache ORC.
- Athena integrates with Amazon **QuickSight** for easy data visualization.
- **Use cases:** Ad-hoc SQL queries for Business intelligence and analytics, query AWS logs e.g. VPC Flow Logs, ELB Logs, CloudTrail logs





## Data scanned when NOT partitioned

/reviews/products/product-X-reviews-2025-11  
 /reviews/products/product-X-reviews-2025-12  
 /reviews/products/product-X-reviews-2026-01  
 /reviews/products/product-Y-reviews-2025-11  
 /reviews/products/product-Y-reviews-2025-12  
 /reviews/products/product-Y-reviews-2026-01

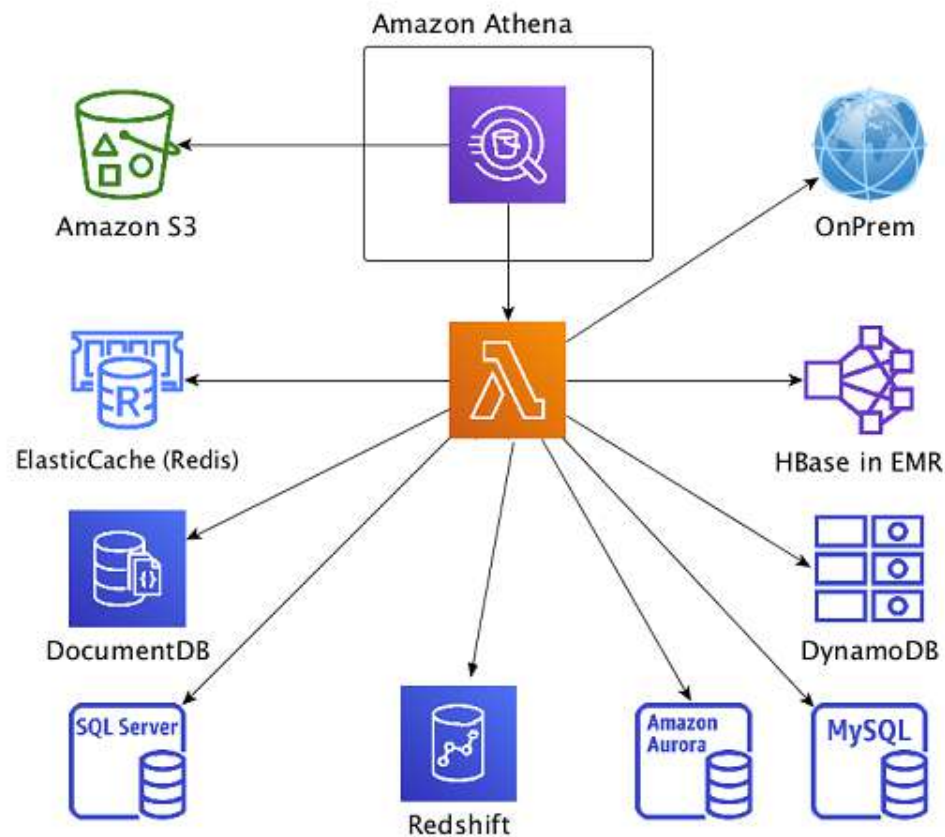
→ 6 GB

## Data scanned when partitioned

/reviews/products/X/2025/11/ → 1GB  
 /reviews/products/X/2025/12/ → 1GB  
 /reviews/products/X/2026/01/ → 1 GB  
 /reviews/products/Y/2025/11/ → 1GB  
 /reviews/products/Y/2025/12/ → 1GB  
 /reviews/products/Y/2026/01/ → 1GB

# Athena Federated queries

- If you have data in sources other than Amazon S3, you can use Athena Federated Query to query the data
- Supports data sources such as DynamoDB, RDS, CloudWatch Logs, and custom data sources including on-premises databases.
- Uses Data Source Connectors that run on AWS Lambda
- Query results are written to Amazon S3

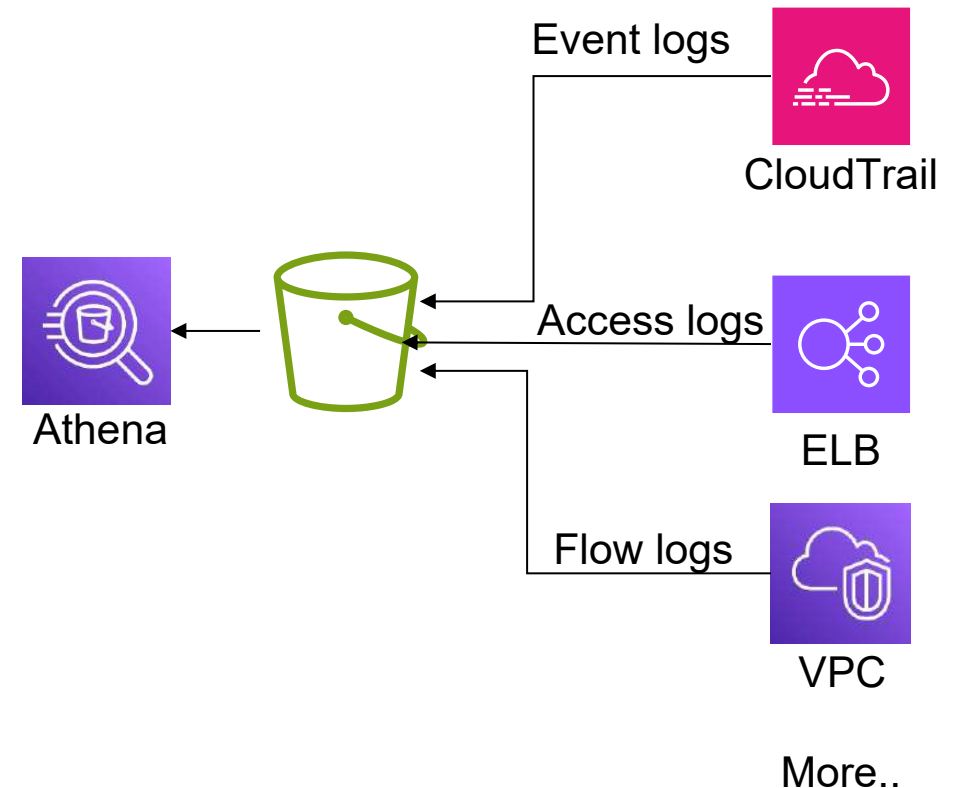


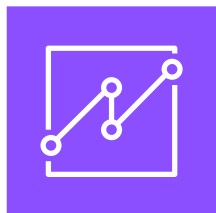
# Athena for querying AWS services logs

- We can use Athena to query logs generated by various AWS services such as AWS CloudTrail, Application load balancer access logs, S3 access logs, Network firewall logs, VPC flow logs and many more..

```
SELECT useridentity.username, sourceipaddress,
eventtime, additionaleventdata
  FROM your_athena_tablename
 WHERE eventname = 'ConsoleLogin'
        AND eventtime >= '2026-01-15T00:00:00Z'
        AND eventtime < '2026-01-17T00:00:00Z';
```

**Example:** CloudTrail event logs to query AWS console login events



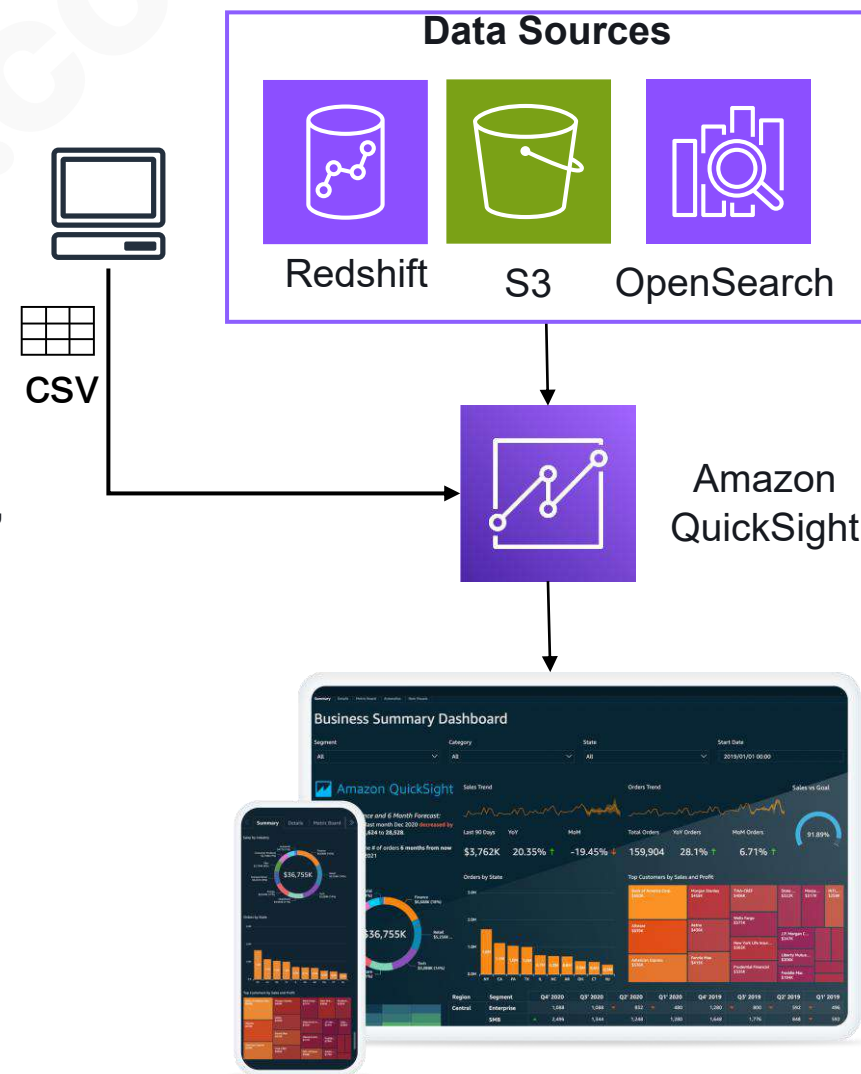


# Amazon QuickSight

# Amazon QuickSight

- Serverless machine learning-powered **Business Intelligence** service to create interactive dashboards.
- Fast, automatically scalable, embeddable, with per-session pricing.
- Integrates with Amazon RDS, Aurora, Athena, S3, OpenSearch, Redshift as data source
- Allows uploading files (CSV, XLS, JSON etc.) using file data source

**Use Cases:** Build Visualizations, Perform ad-hoc analysis



Smart Visualizations and Dashboards

# Amazon QuickSight - Important to know

## Amazon QuickSight SPICE

- SPICE is super fast, parallel and in-memory calculation engine designed to help accelerate QuickSight dashboard performance.
- No need to provision or manage infrastructure for SPICE.
- The data stored in a SPICE dataset is a snapshot of your source data.
- To fetch the latest data from the source and refresh a SPICE dataset, you can configure a schedule or use an event-driven approach.

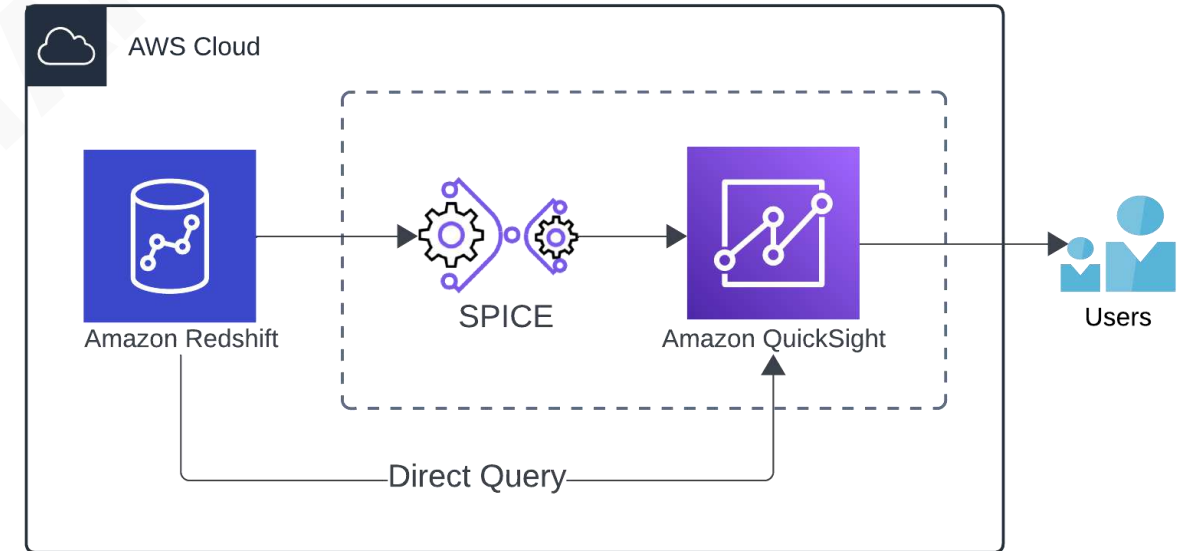


Image source: <https://aws.amazon.com/blogs/business-intelligence/best-practices-for-amazon-quicksight-spice-and-direct-query-mode/>

# Amazon QuickSight – Important to know

## Users & Groups

- Amazon QuickSight has its own users and groups
- These users are different than AWS IAM users
- You can create User Groups (Enterprise edition)

## Sharing analysis and dashboard

- You can share analyses or dashboards with individual users or groups.
- Users who receive a shared dashboard can view and interact with the results but cannot edit the analysis.

## Data Access and Security

- Users can view visualized data in dashboards, but do not have direct access to the raw underlying data sources.
- Supports Row-Level Security (RLS) in Enterprise edition.

Data set:

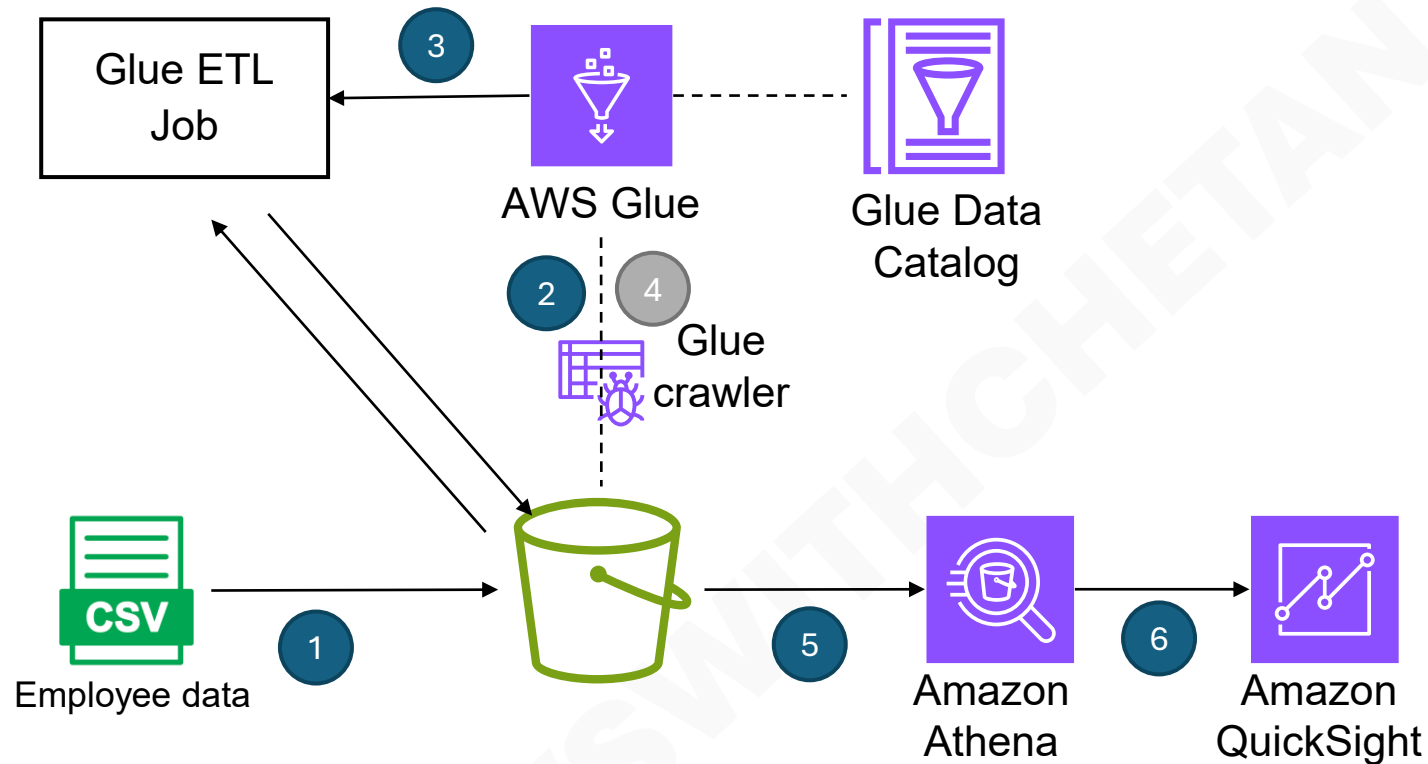
| Region | Revenue |
|--------|---------|
| APAC   | 500     |
| EMEA   | 300     |
| NA     | 450     |
| APAC   | 700     |

RLS Rule:

| User  | Region     |
|-------|------------|
| Alice | EMEA       |
| Bob   | APAC       |
| Carol | NA         |
| Dirk  | EMEA, APAC |



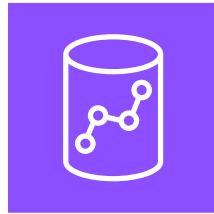
# Demo – Data pipeline using S3, Glue, Athena



## High level steps

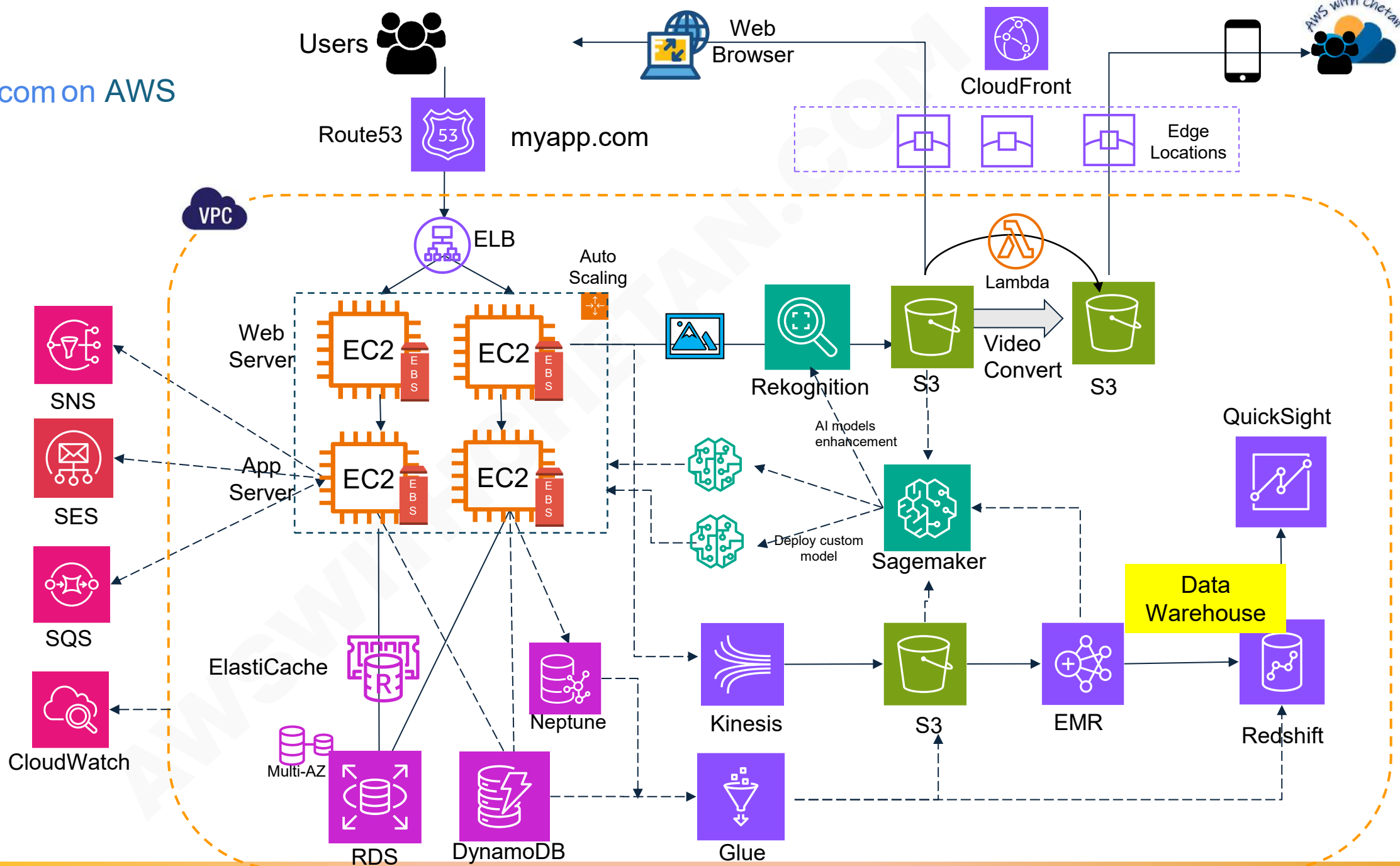
- 1 Upload sample CSV file to S3
- 2 Create AWS Glue crawler and run it. This should create a data catalog with Glue database and table
- 3 Create Glue ETL (python) job to modify one of the column in the CSV and create a new CSV file in S3
- 4 Rerun the Glue crawler again. This should create another table in the database
- 5 Using Amazon Athena query the Glue database and table created above
- 6 (Optionally) Create analysis and dashboard in Amazon QuickSight using Athena as a data source





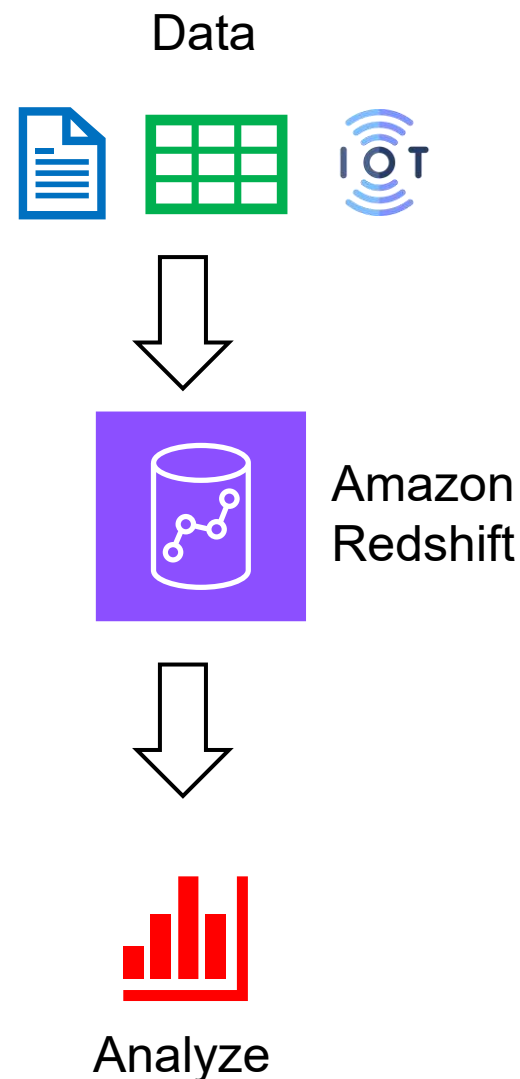
# Amazon Redshift

# myapp.com on AWS



# Amazon Redshift

- Amazon Redshift is a fully managed, **petabyte-scale data warehouse** service
- Amazon Redshift uses massively Parallel Processing (MPP) architecture and has a SQL interface for performing the queries.
- Can ingest data across data lakes, databases, streaming data - with no code/low code **zero-ETL** approach
- BI tools such as Amazon QuickSight, Tableau work seamlessly with Redshift.
- Allows sharing data across AWS regions, teams, and third-party data warehouses without data movement or data copying.
- Redshift Cluster options:
  - Redshift Serverless
  - Provisioned Cluster



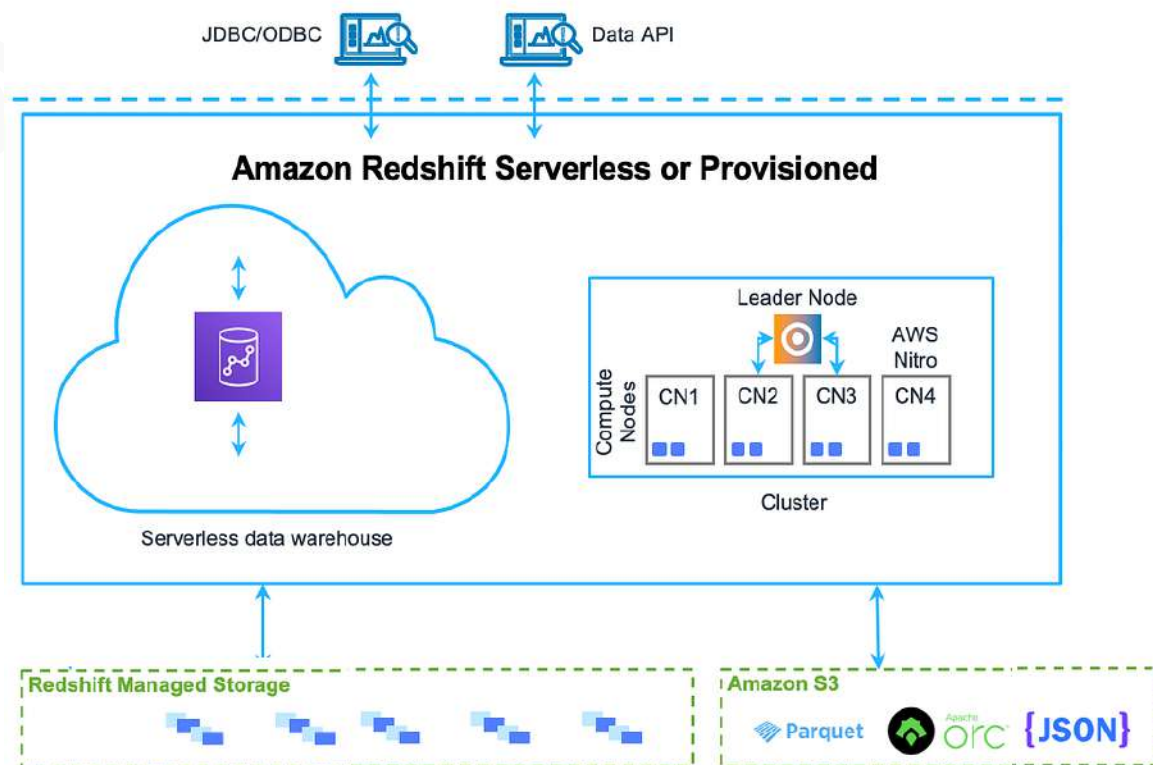
# Redshift Serverless vs Provisioned Cluster

## Redshift Serverless

- No cluster to manage. Fast setup, minimal admin effort
- Uses RPU (Redshift Processing Units) to scale up/down the CPU, memory and network
- Best for sporadic, unpredictable workloads

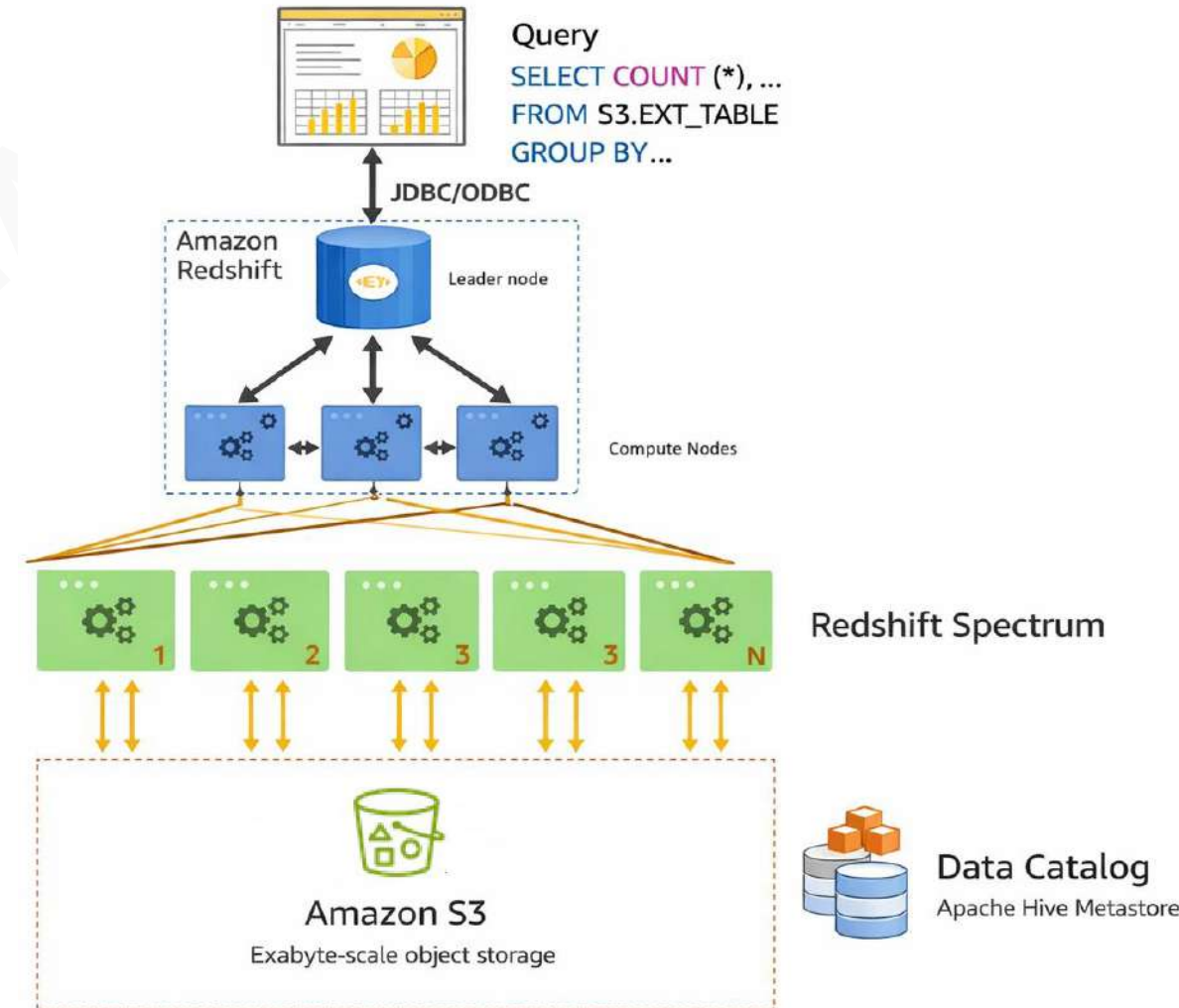
## Redshift Cluster

- There is a leader node and one or more compute nodes
- Leader node distribute the query across compute nodes
- You can choose node type and node count
- Manual and scheduled scaling
- Best for steady, predictable workloads



# Redshift Spectrum

- Allows Amazon Redshift to query data directly in Amazon S3 without loading it
- Automatically scales compute independently of the Redshift cluster
- Best for querying large, infrequently accessed data (data lake)
- Enables separation of storage and compute



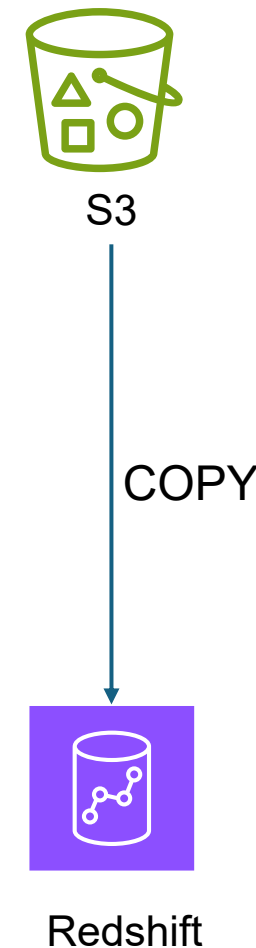
# Loading data from S3 into Redshift

- COPY command

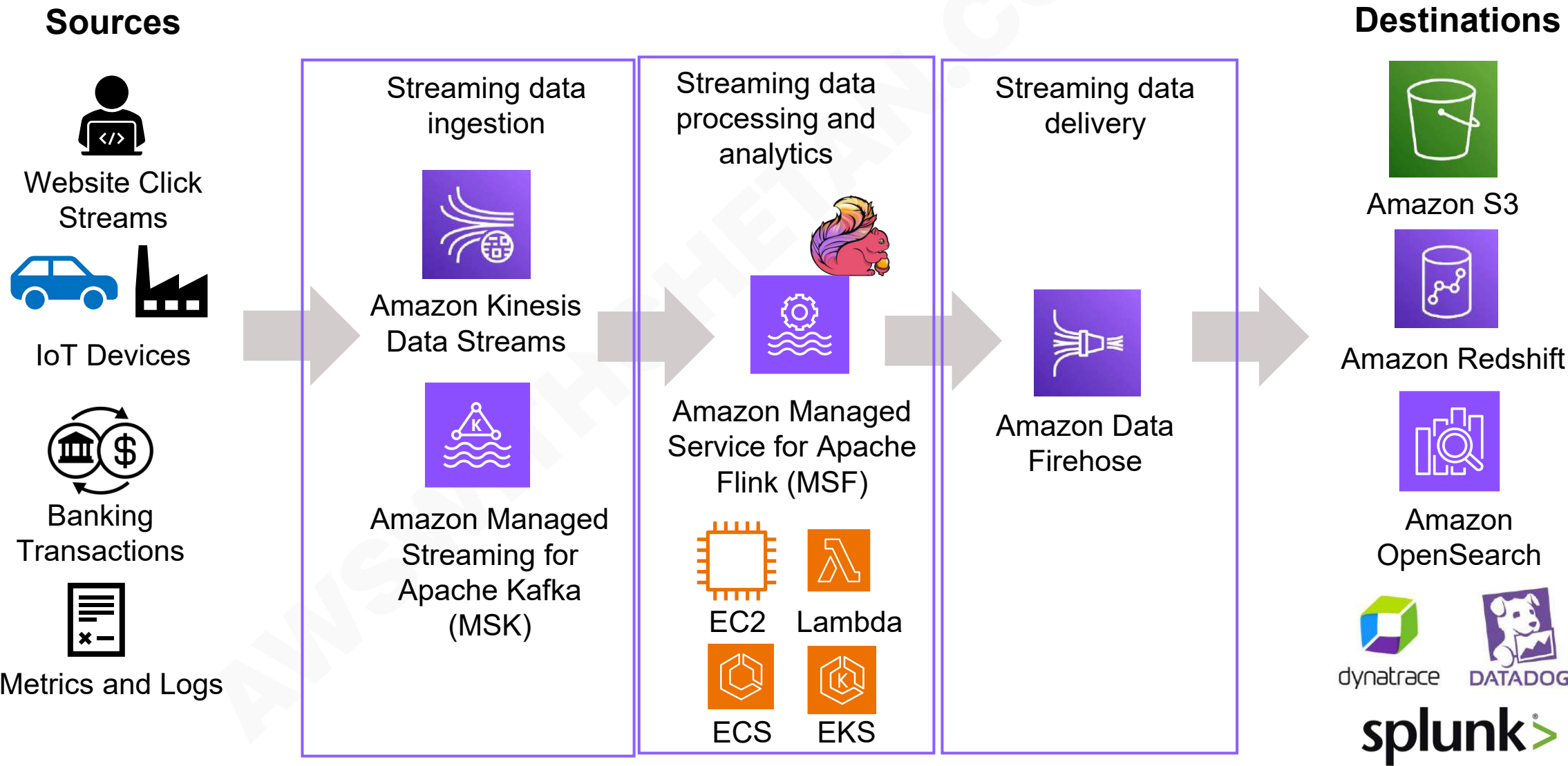
```
COPY customer FROM 's3://amzn-s3-demo-bucket/mydata'  
IAM_ROLE 'arn:aws:iam::0123456789012:role/MyRedshiftRole';
```

- Auto copy Jobs using S3 Event integration

```
COPY public.target_table FROM 's3://amzn-s3-demo-  
bucket/staging-folder' IAM_ROLE  
'arn:aws:iam::123456789012:role/MyLoadRoleName' JOB  
CREATE my_copy_job_name AUTO ON;
```



# Amazon Streaming data services

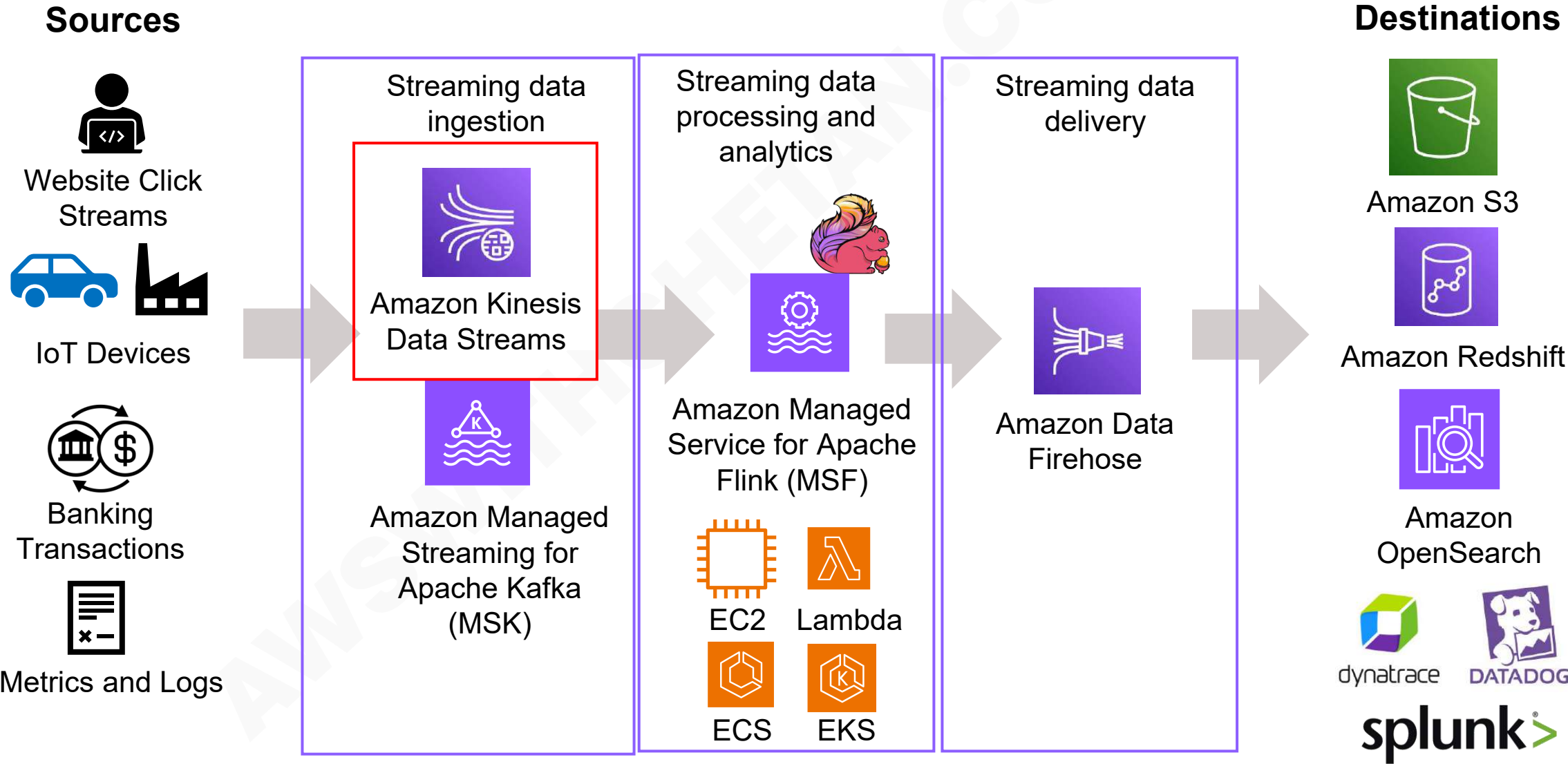




# Amazon Kinesis



# Amazon Streaming data services

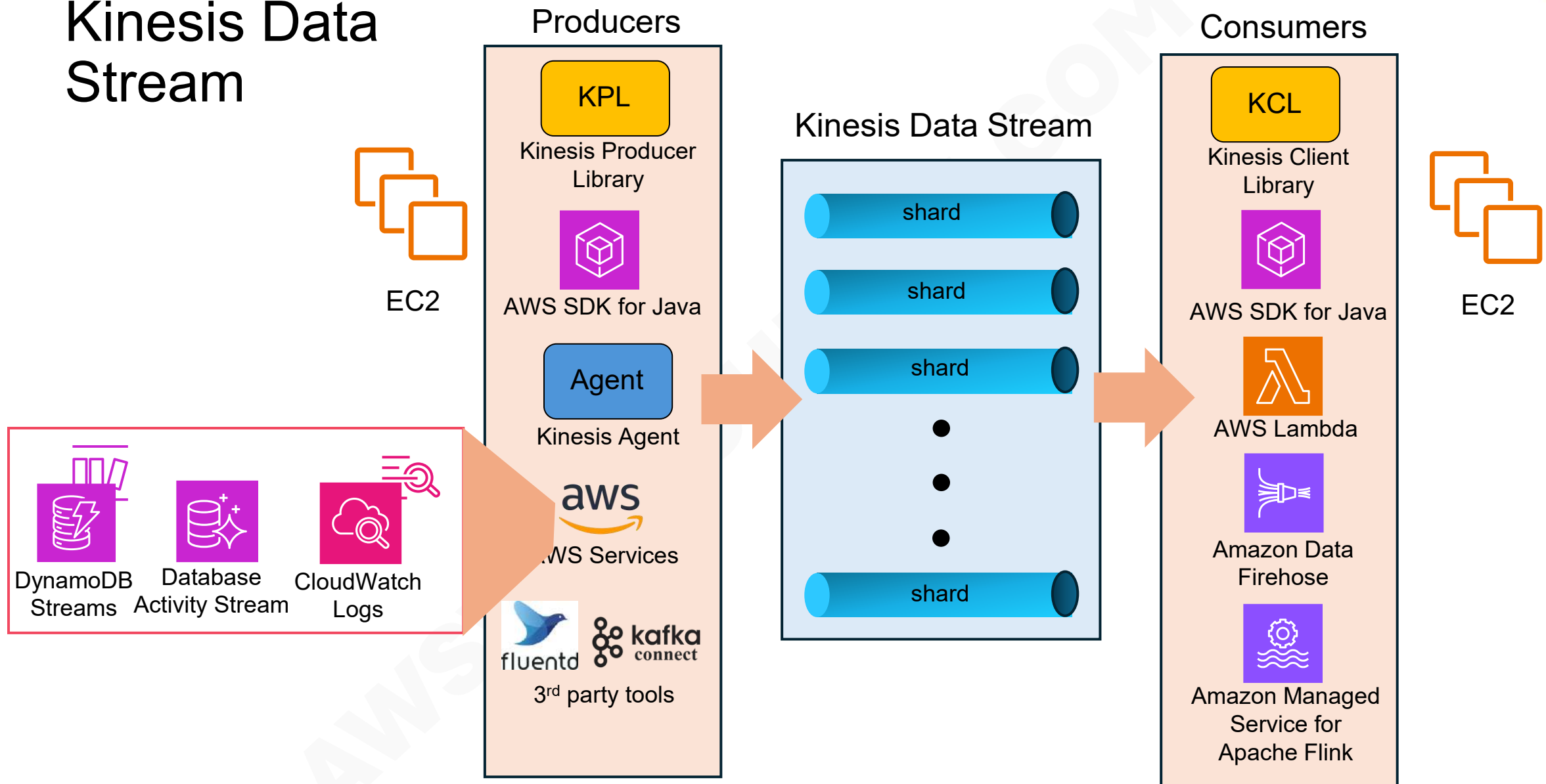


# Amazon Kinesis

- **Amazon Kinesis Data Streams:** A managed service to ingest real-time streaming data such as logs, metrics, and events.
- **Amazon Kinesis Video Streams:** A managed service to securely ingest, store, and process real-time video and media streams from devices and cameras.

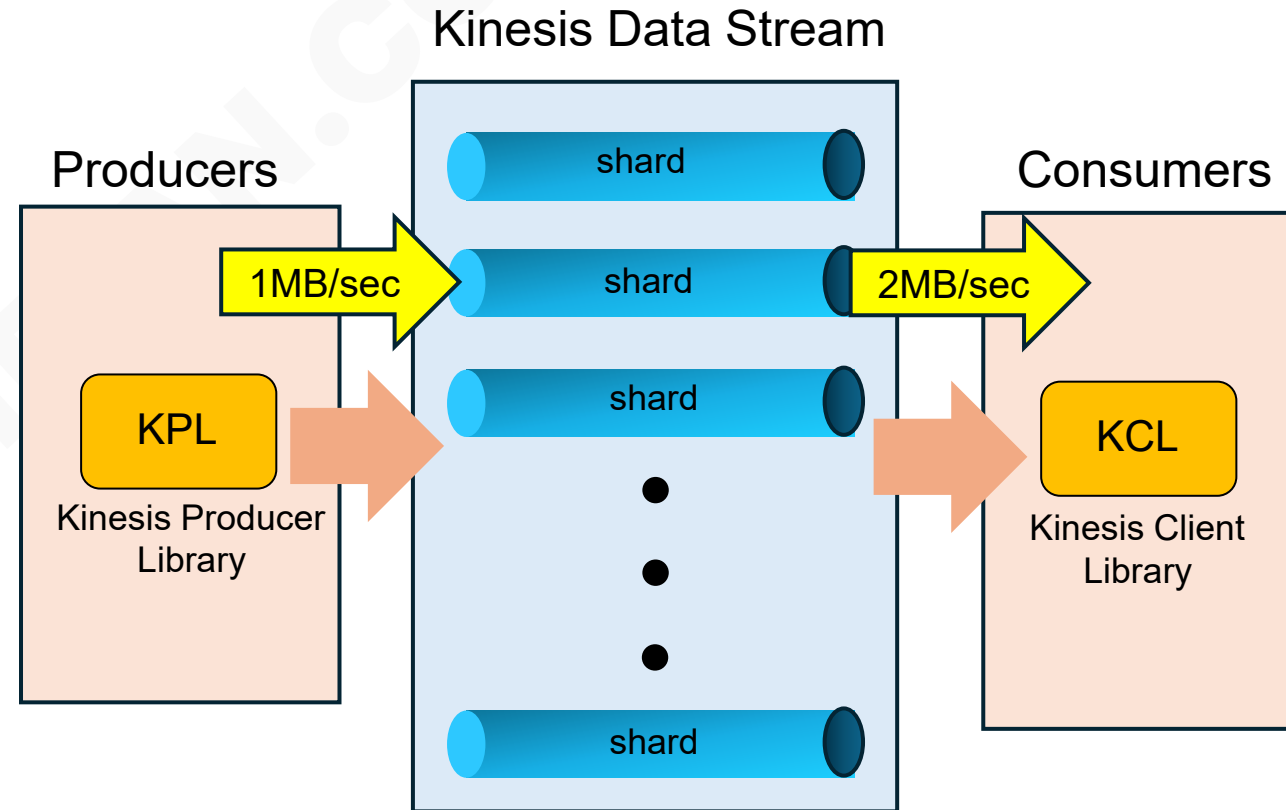
AWSWITHCHETAN.COM

# Kinesis Data Stream



# Kinesis Data Stream - Shards

- Kinesis Data Streams comprise of **Shards**
- Each shard provides maximum read data rate of 2MB/sec and write data rate of 1MB/sec (or 1000 records/sec)
- A partition key is used to group the records by shard within a stream ensuring ordering of records within a shard
- Number of shards decides the capacity of the stream.
- Data in stream can be retained for up to 365 days (24 hrs is default retention period)
- Data can't be deleted (until expires)
- Data can be replayed and reprocessed by the consumers (if required)



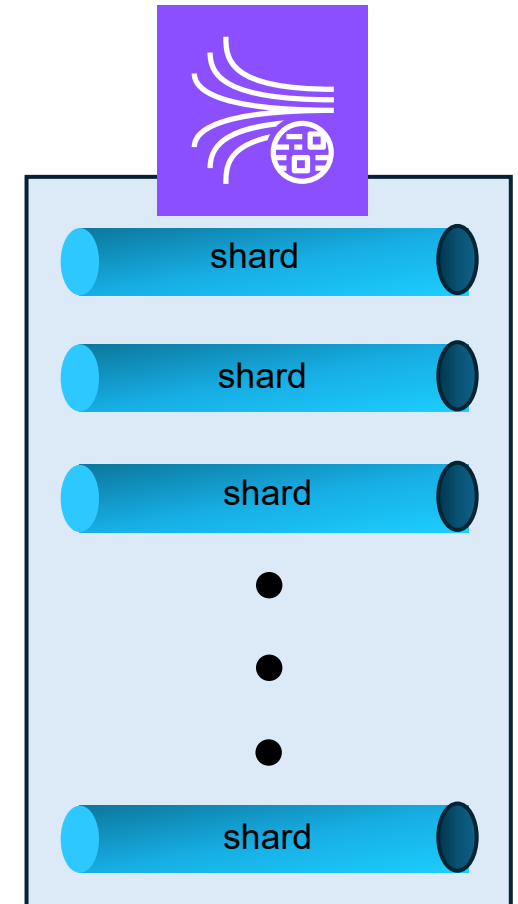
# Kinesis Data Stream – Capacity modes

## Provisioned Mode

- You explicitly define the number of shards in the stream
- Each shard provides 1 MB/sec write (1,000 records/sec) and 2 MB/sec read throughput
- Scaling requires manual shard split or merge operations
- Best suited for predictable and steady streaming workloads

## On-demand Mode

- Automatically scales read and write throughput based on observed traffic over the last 30 days
- No shard capacity planning or shard management required
- Ideal for unpredictable or highly variable workloads



Amazon Kinesis  
Data Stream

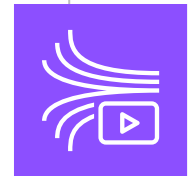
# Amazon Kinesis – Use cases

- Fraud detection in Financial services
- Analyzing customer behavior in real-time – personalized recommendations, offers, discount codes
- Logs monitoring and alerting
- Real time Ad targeting
- IoT Data processing – To detect malfunctions or anomalies
- Social Media – Trending topics, user engagement, content recommendations



Amazon Kinesis  
Data Streams

- Home security and Smart cameras – Motion detection, face detection
- Connected vehicles – Dizzy driver, Drive Behavior
- Industrial IoT and remote monitoring – Equipment health, Oil rigs in powerplant, security breaches

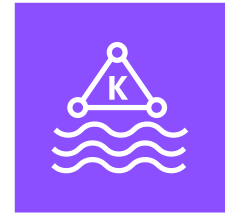


Amazon Kinesis  
Video Streams



# Amazon MSK

- Fully managed Apache Kafka service on AWS (alternative to Amazon Kinesis Data Stream)
- Used for real-time streaming ingestion and messaging
- AWS manages brokers, Zookeeper/KRaft, patching, and scaling
- Compatible with open-source Kafka APIs and tools
- Data is stored across Kafka topics and partitions
- Provides high availability using multi-AZ clusters, offers Serverless option.
- Integrates with IAM, TLS, and encryption at rest
- Commonly used with Apache Flink, Lambda, Spark, EC2 consumers



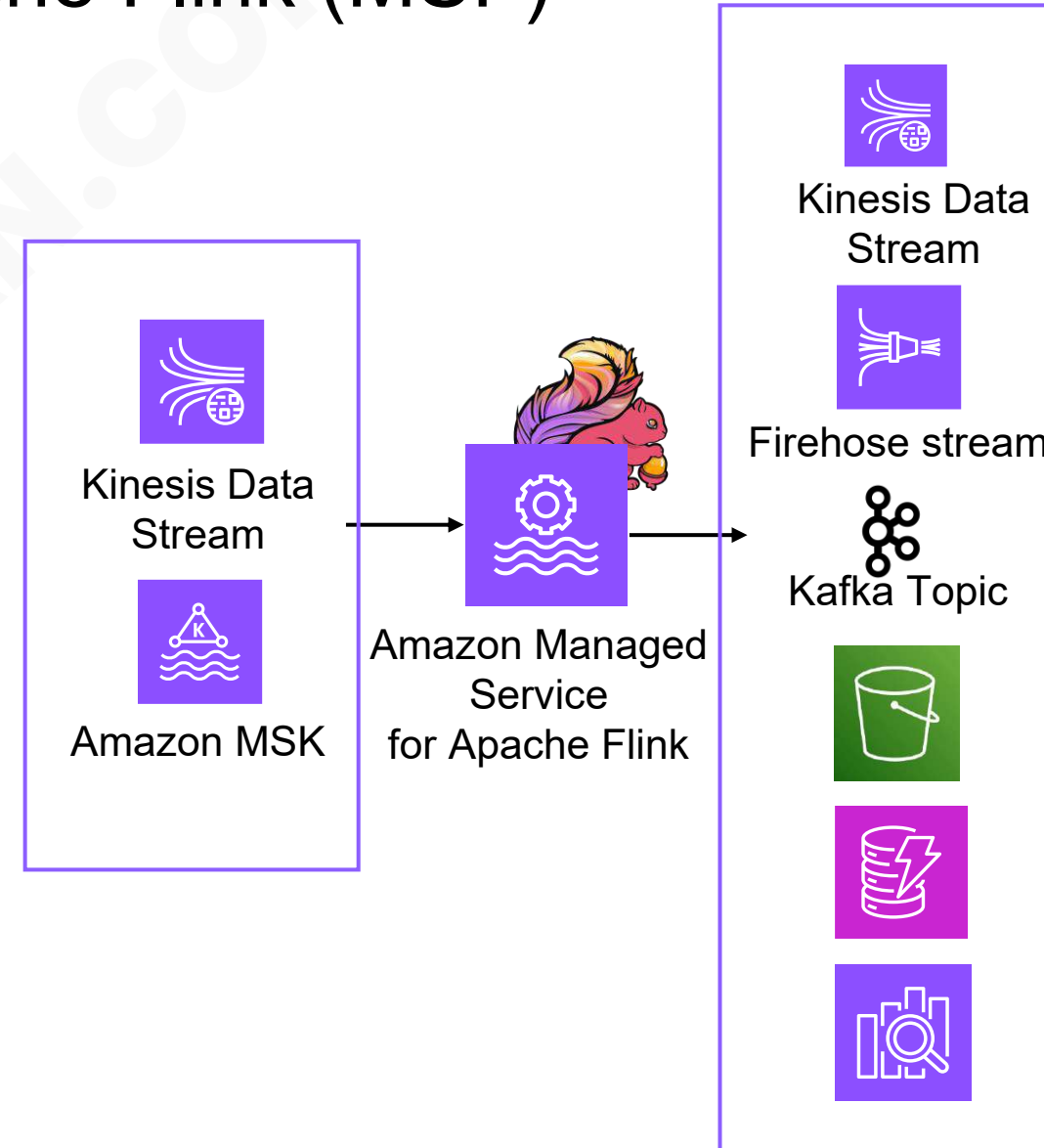
Amazon Managed Streaming  
for Apache Kafka  
(Amazon MSK)

# Amazon Managed Service for Apache Flink (MSF)

- Fully managed service to run Apache Flink for real-time stream processing
- Common sources include Kinesis Data Streams and Amazon MSK
- You define input sources and output sinks

## Use cases

- Real-time aggregations - Rolling counts, sums, averages over time windows
- Stream data enrichment - Join live events with reference data from databases
- Anomaly detection & alerts - Detect patterns like fraud, detect anomalies when thresholds are breached



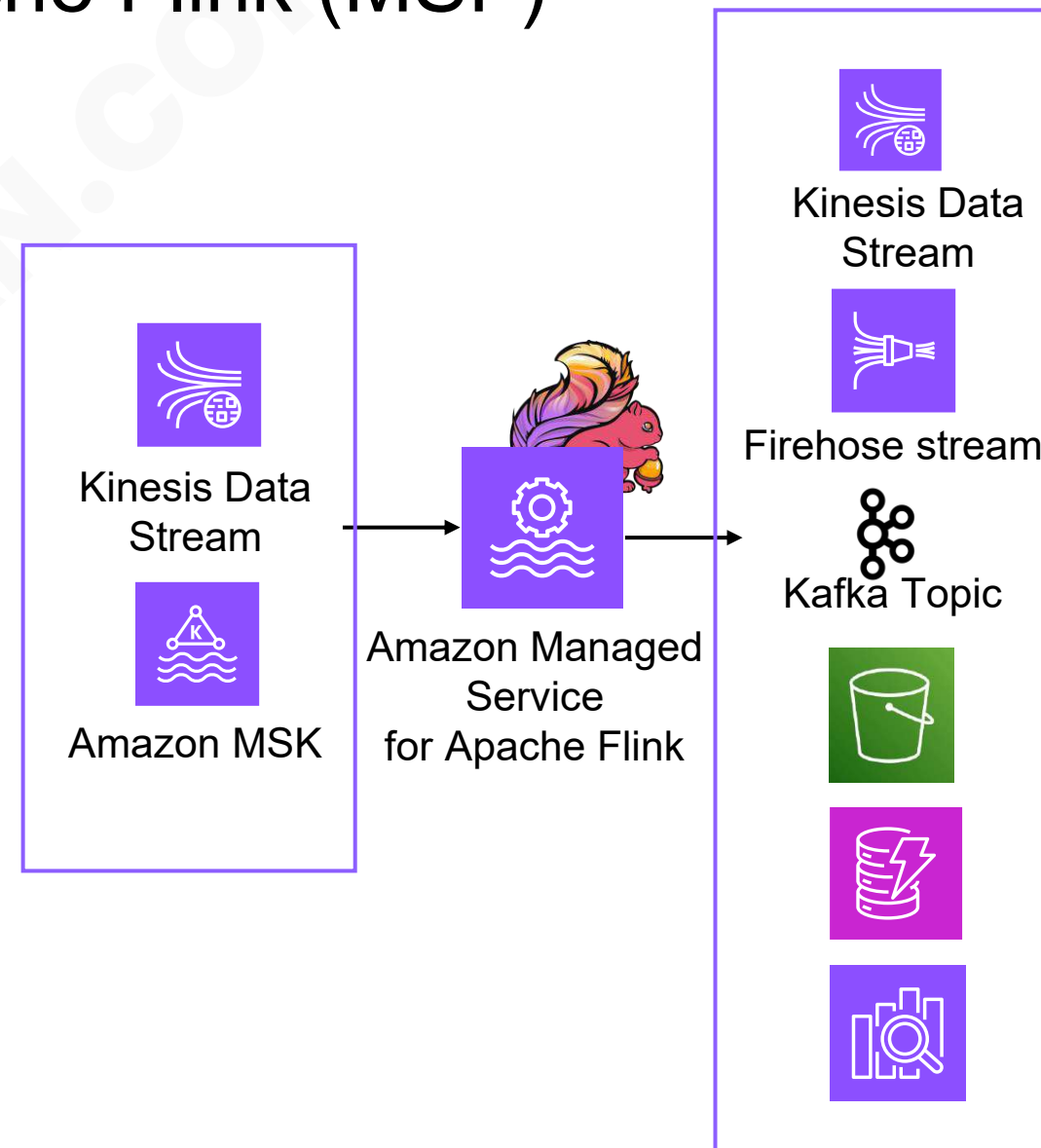


# Amazon Managed Service for Apache Flink (MSF)

- Fully managed service to run Apache Flink for real-time stream processing
- Common sources include Kinesis Data Streams and Amazon MSK
- You define input sources and output sinks

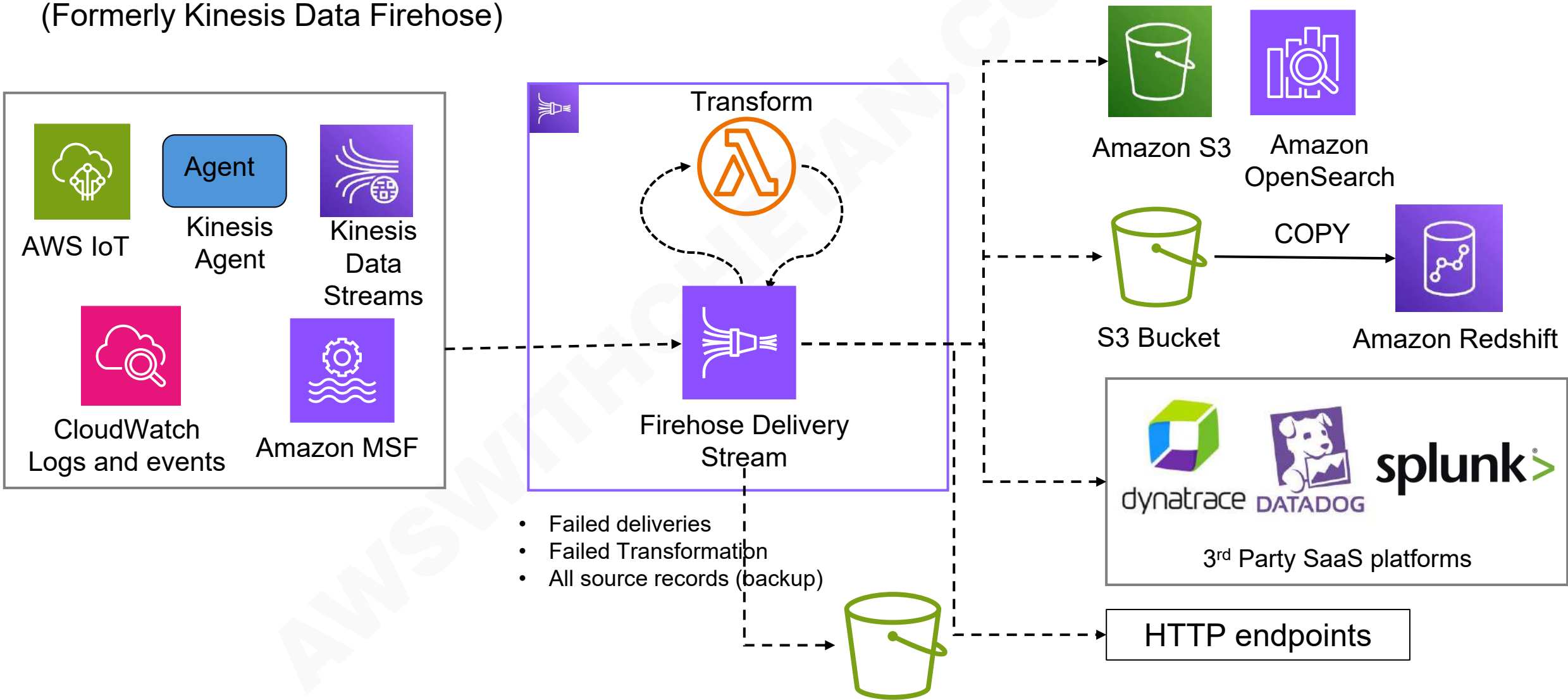
## Use cases

- Real-time aggregations - Rolling counts, sums, averages over time windows
- Stream data enrichment - Join live events with reference data from databases
- Anomaly detection & alerts - Detect patterns like fraud, detect anomalies when thresholds are breached



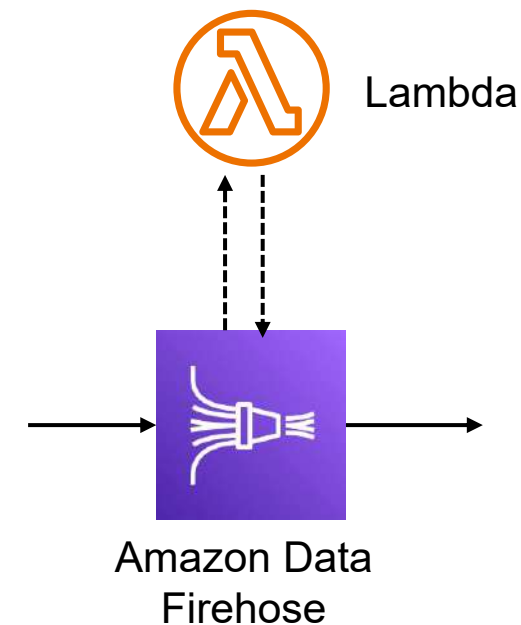
# Amazon Data Firehose

(Formerly Kinesis Data Firehose)



# Amazon Data Firehose

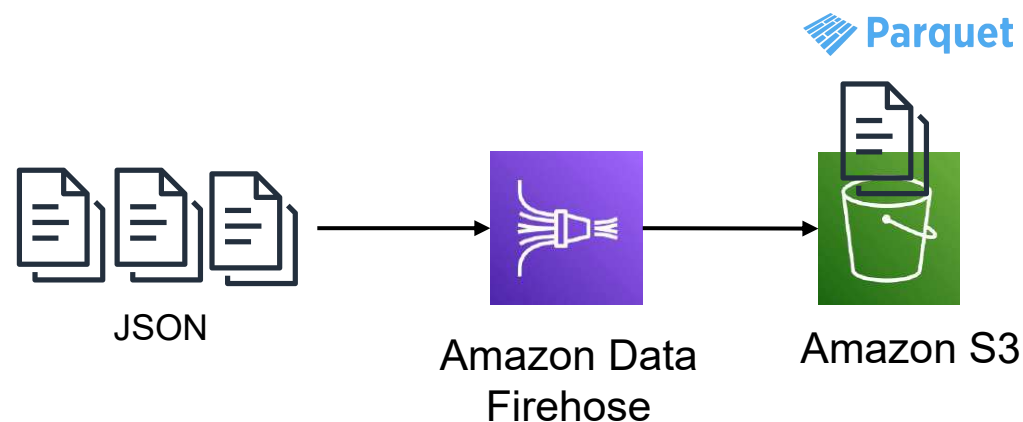
- Delivers streaming data to destinations such as Amazon S3, Amazon Redshift, Amazon OpenSearch Service, and HTTP endpoints
- Integrates with third-party destinations such as Splunk and Datadog
- Supports configurations like buffer size and buffer time to control speed and size of records
- Supports formats like JSON, CSV, Parquet, ORC, raw text, and binary
- Can convert data formats (e.g., JSON → Parquet / ORC) before delivery (built-in feature)
- Supports compression (GZIP, Snappy, ZIP) to reduce storage cost
- Allows inline data transformation using **AWS Lambda** before delivery



# Amazon Data Firehose

## Use cases:

- Ingest application logs and deliver them to Amazon S3 for long-term storage and analytics
- Send Clickstream or event data directly into Amazon Redshift for near-real-time reporting
- Send operational logs to Amazon OpenSearch Service for search and visualization
- Deliver metrics and traces to third-party tools like Splunk or Datadog
- Convert and compress streaming data (e.g., JSON → Parquet + GZIP) before landing in S3





# AWS Lake Formation

# Access S3 data directly

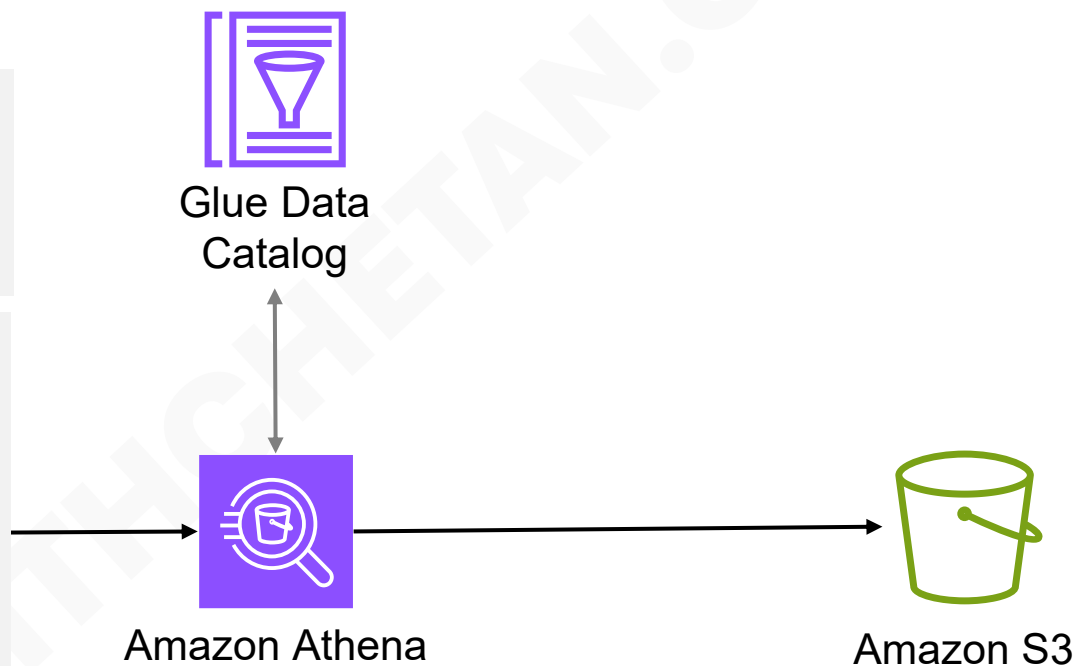
```
{  
  "Sid": "ReadObjects",  
  "Effect": "Allow",  
  "Action": "s3:GetObject",  
  "Resource": "arn:aws:s3:::example-bucket/*"  
},
```



# Access S3 data using Athena

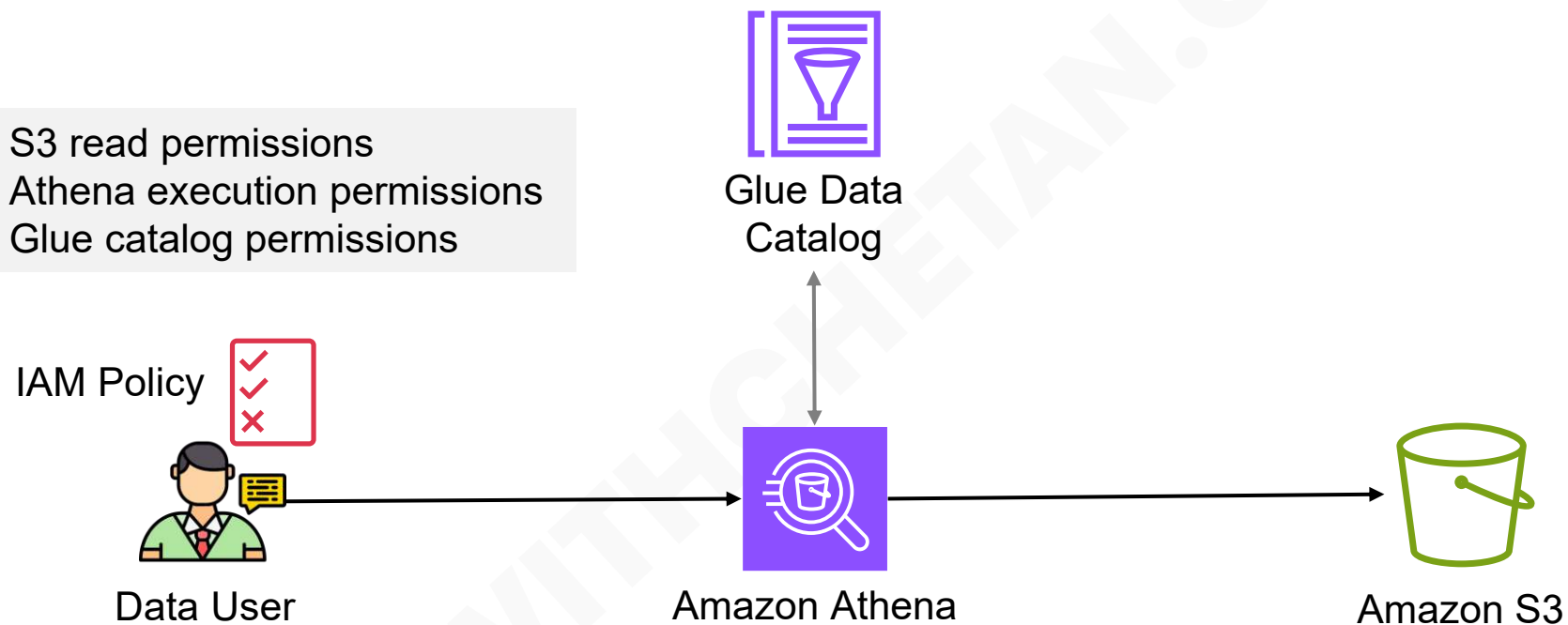
```
{
  "Sid": "ReadObjects",
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::example-bucket/*"
},
```

```
{
  "Sid": "AthenaQueryAccess",
  "Effect": "Allow",
  "Action": [
    "athena:StartQueryExecution",
    "athena:GetQueryExecution",
    "athena:GetQueryResults"
  ],
  "Resource": "*"
},
{
  "Sid": "GlueCatalogRead",
  "Effect": "Allow",
  "Action": [
    "glue:GetDatabase",
    "glue:GetTable",
    "glue:GetPartitions"
  ],
  "Resource": "*"
}
```



# Access S3 data using Athena

- S3 read permissions
- Athena execution permissions
- Glue catalog permissions

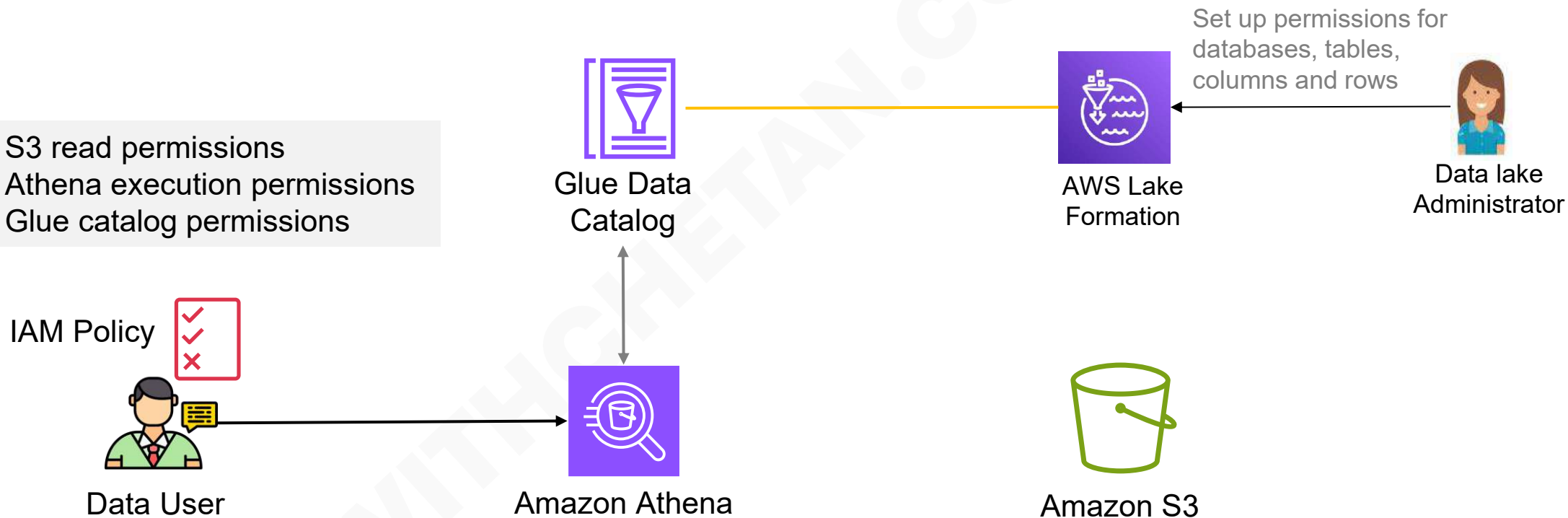


Permissions are S3 Object level



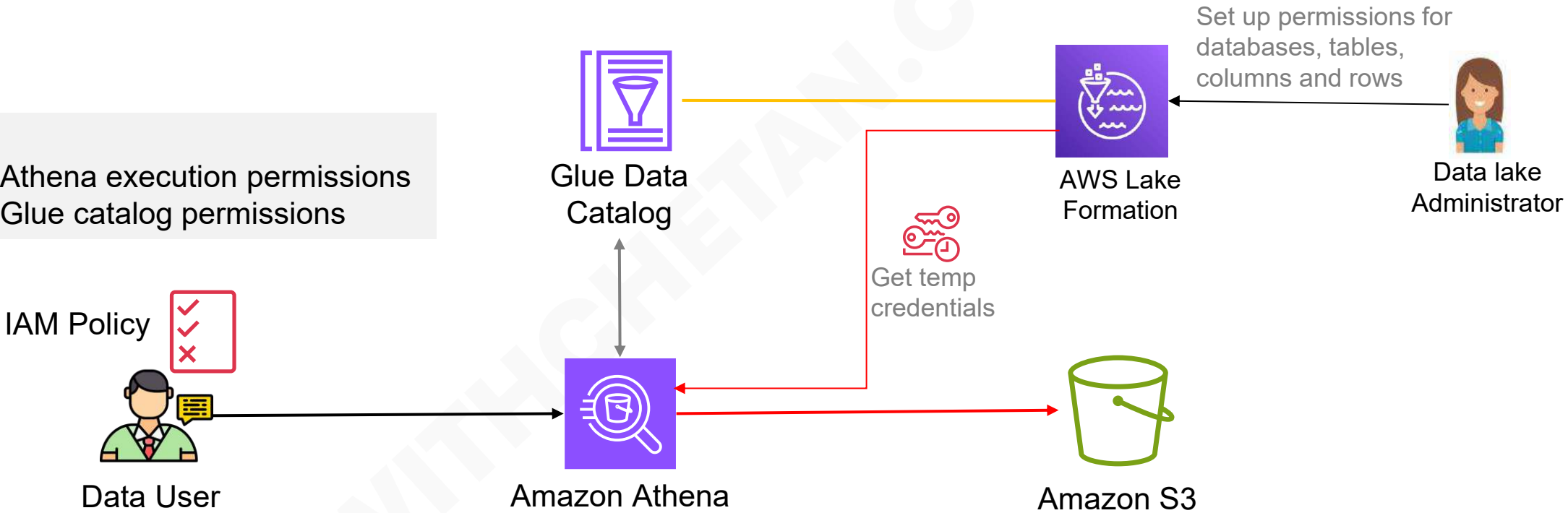
# Access S3 data using Lake formation permissions

- S3 read permissions
- Athena execution permissions
- Glue catalog permissions



# Access S3 data

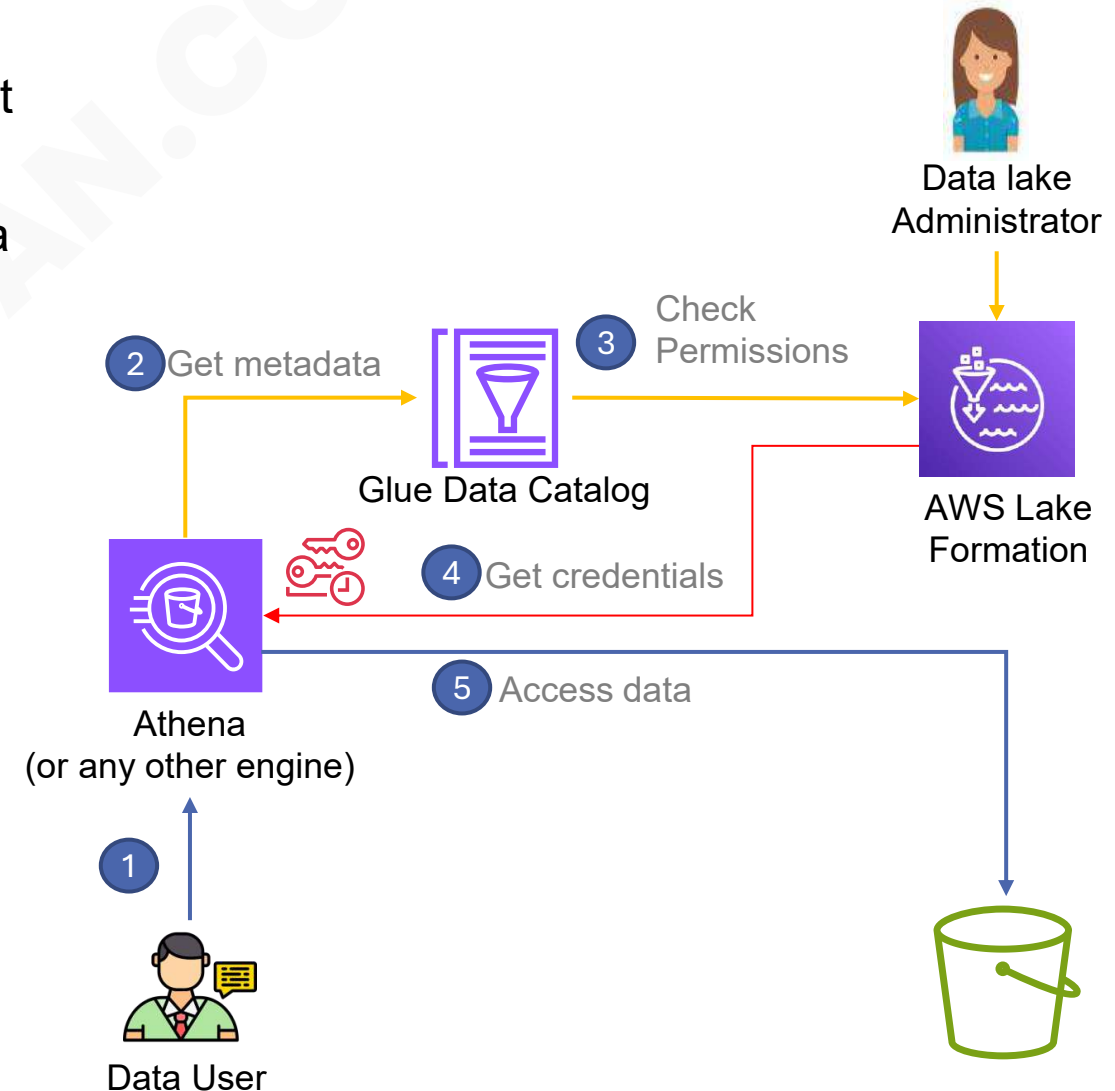
- Athena execution permissions
- Glue catalog permissions



Permissions are database, table, row or column level

# AWS Lake Formation

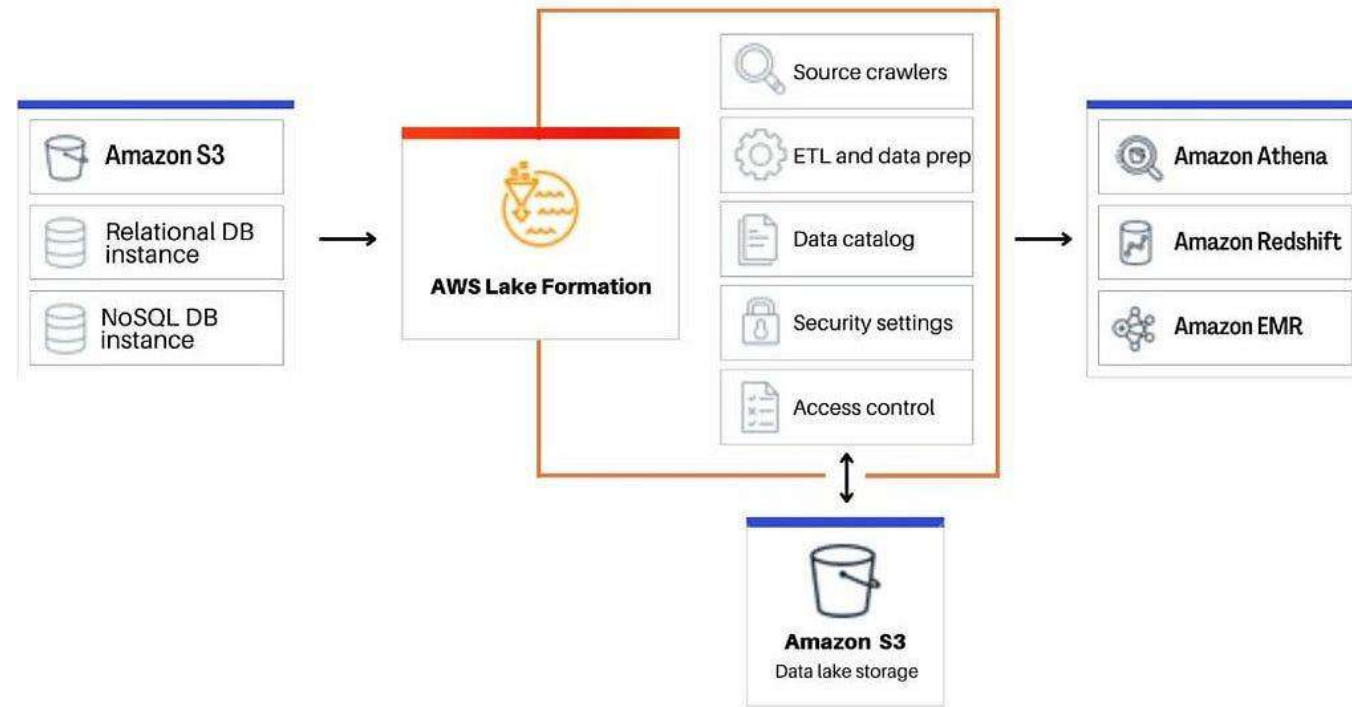
- Provides centralized, fine-grained data access control at table, column, and row level
- Builds on the AWS Glue Data Catalog to store metadata and enforce permissions
- Supports resource-based permissions (e.g. grant SELECT on a specific database or table such as company.finance)
- Supports attribute-based access control (ABAC) using LF-tags (key–value labels like department=finance)
- Without Lake Formation:
  - Users require IAM permissions to directly access data in Amazon S3
- With Lake Formation:
  - Users require Lake Formation permissions, and direct S3 access is not needed for querying data



# AWS Lake Formation for Data Lake setup

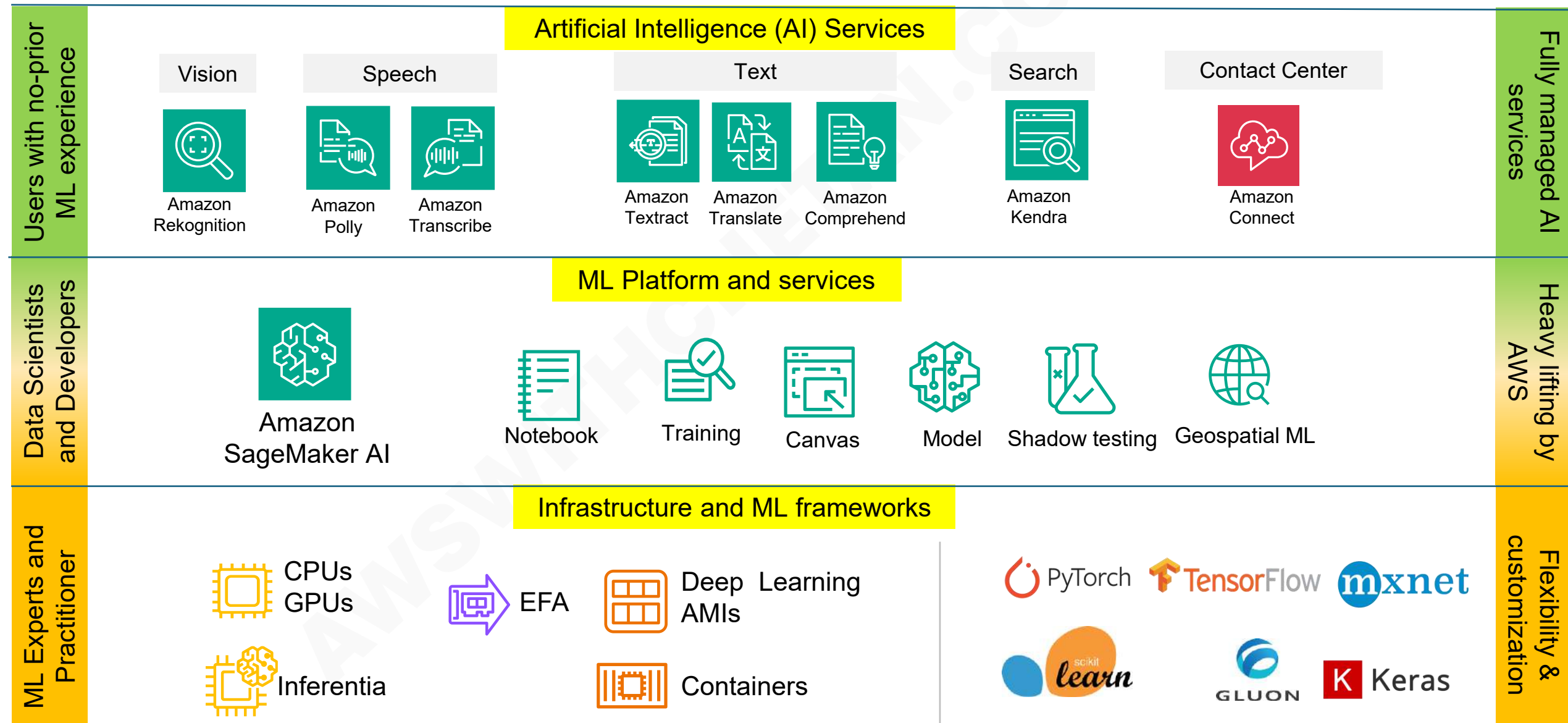
## Automates Data import using Lake Formation Workflows and Blueprints

- A workflow uses a blueprint to define and automate the steps required to ingest data into AWS Lake Formation
- This includes creating the data lake structure, running crawlers, and registering data locations.
- Blueprints take the data source, data target, and schedule as input to configure the workflow.
- AWS provides blueprints for sources like Relational databases, CloudTrail logs, NoSQL databases, S3 etc.



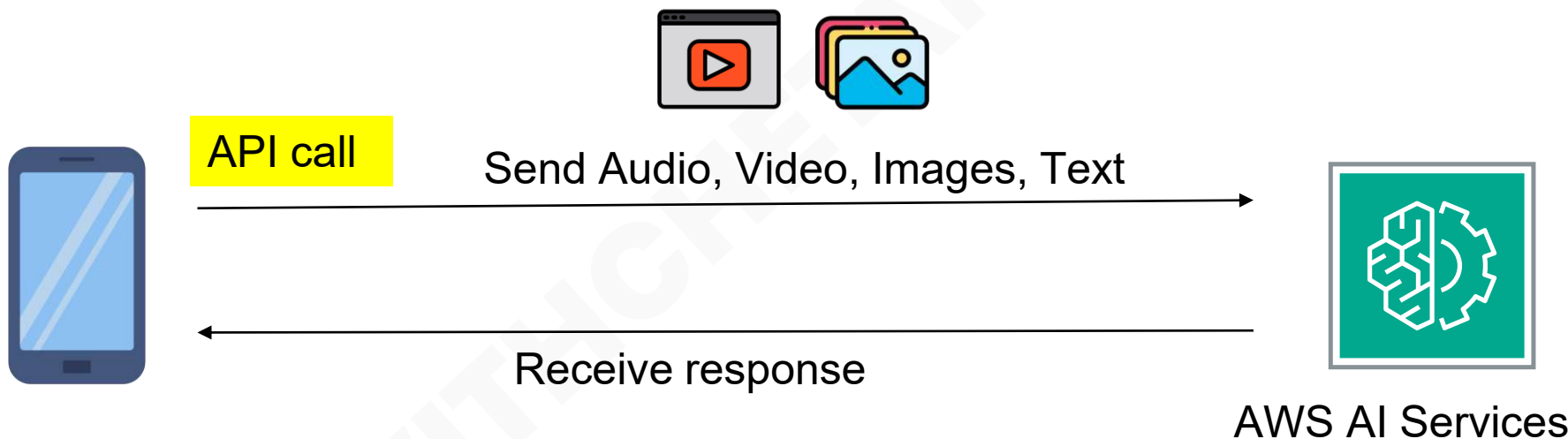
# AWS Machine Learning & AI

# AWS' three-layered approach to AI/ML



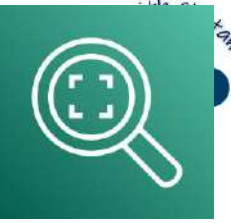
# AWS AI services

# Using AWS AI services

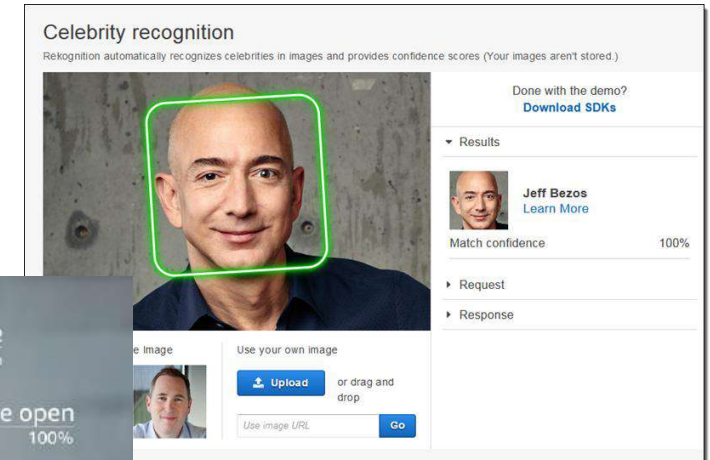
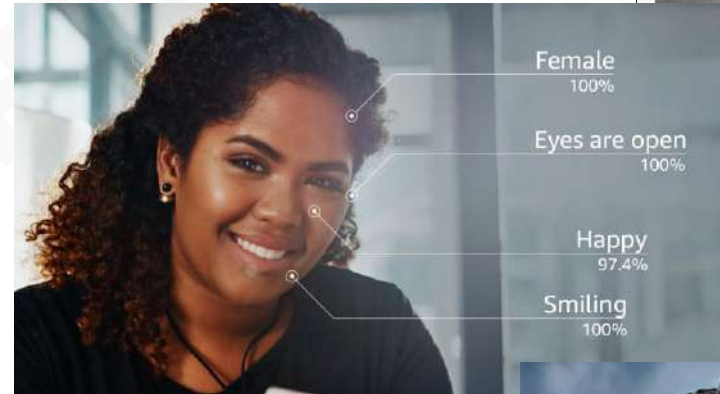




# Amazon Rekognition

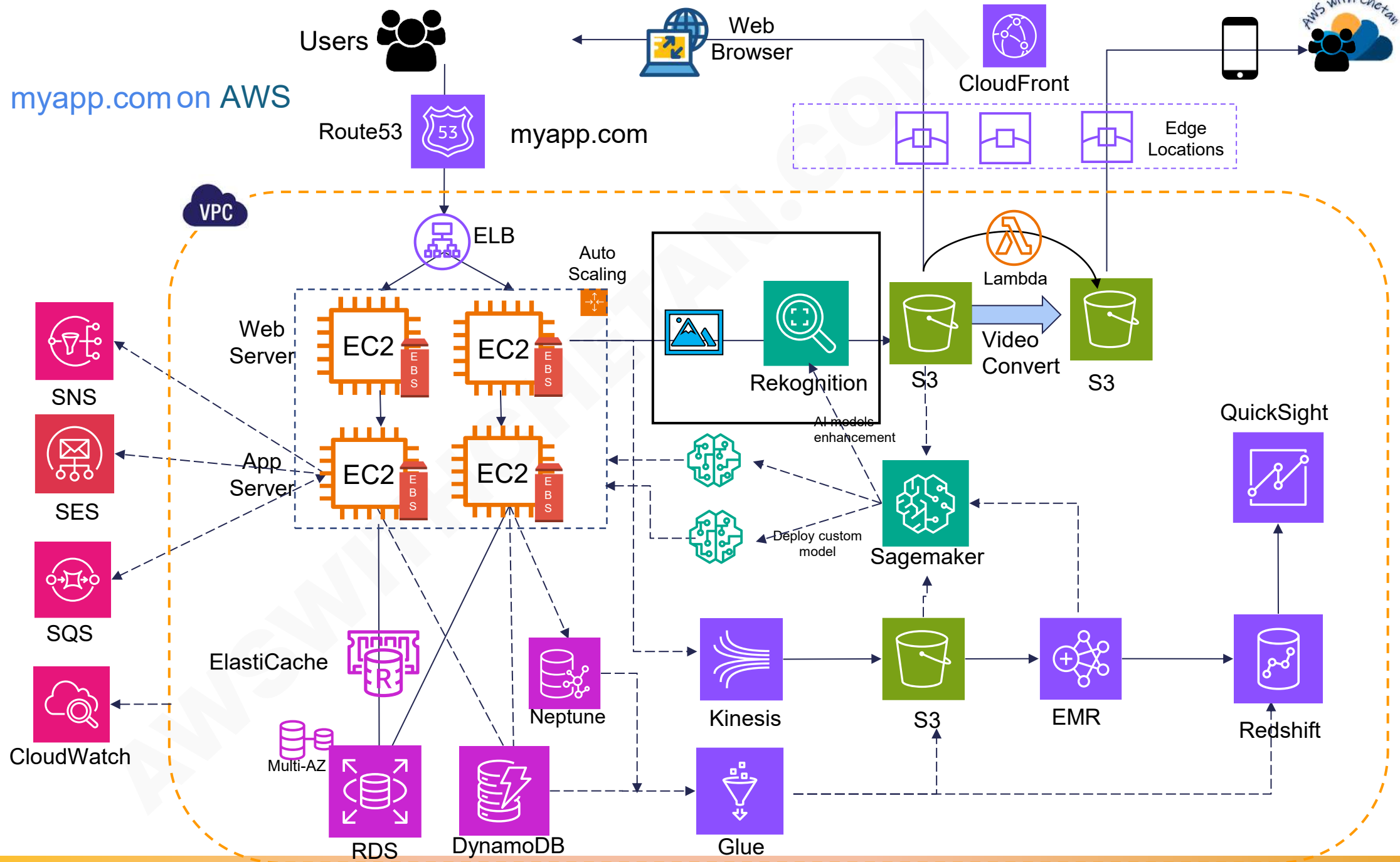


- Computer Vision based AI service to find people, texts, objects, scene in images and videos.
- Common use cases:
  - Celebrity recognition
  - Face compare and search
  - Face detection and analysis
  - Content moderation
  - Detect objects, Brand logos, Texts
  - Video segment detection (blank frames etc.)
  - Easy filtering of video for explicit and suggestive content



<https://aws.amazon.com/rekognition/>

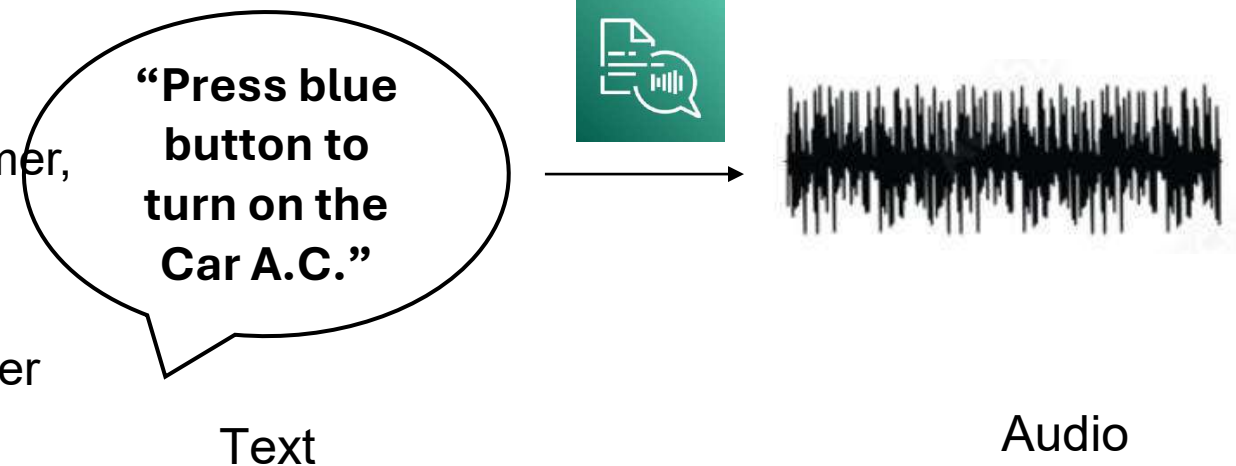
myapp.com on AWS



# Amazon Polly



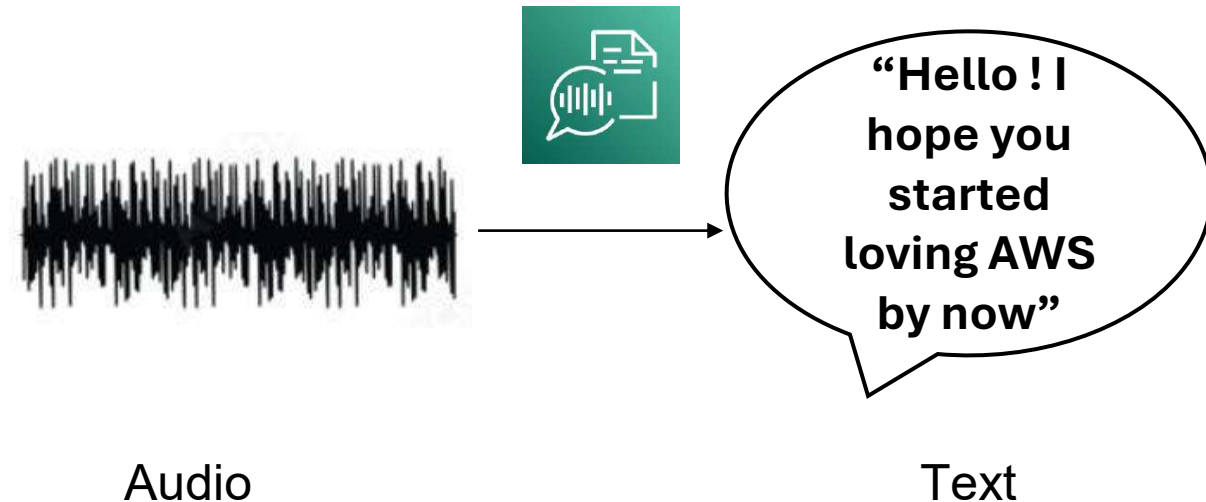
- Converts text to speech
- Create applications that talk to increase engagement and accessibility
- Supports Speech synthesis - phrases, punctuations, pauses
- **Common use cases:**
  - Add speech for global audience RSS feeds, websites, blogpost
  - Automated voice response system (“Dear customer, your account balance is zero 😊”)
  - Supports variety of lifelike voices that you can choose from to better serve your customers as per their location





# Amazon Transcribe

- Automatic speech recognition (ASR) service to convert **audio** to **text**.
- Supports Automatic Language Identification
- Supports over 100+ languages (English, Chinese, Arabic, French, Korean, Hindi, German etc.)
- Automatically removes Personally Identifiable Information (PII) using redaction.
- Multiple Speakers and channels identification
- Custom Vocabulary
- Common use cases:
  - ✓ Transcribe customer service calls
  - ✓ Automate closed captioning and subtitling
  - ✓ Generate metadata for media assets to create a fully searchable archive
  - ✓ Detect toxic content in the audio (social media)

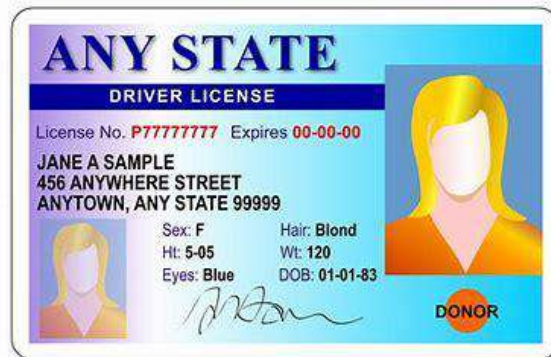


Amazon Transcribe for medical to convert clinical conversation into health records

# Amazon Textract



- Automatically extract printed text, handwriting, layout elements, and data from scanned documents.
- Much more powerful than simple OCR (optical character recognition) where it understands the document layout, forms, tables, images and extracts relevant and related data
- Use Cases:
  - Financial Services (e.g., Invoices, Financial reports)
  - Healthcare (e.g., Medical records, Insurance claims)
  - Public Sector (e.g., Tax forms, ID documents, Passports)



Analyse



Amazon Textract

Result

```
{
  "Document ID": "P777777777777",
  "Name": "JANE A SAMPLE",
  "SEX": "F",
  "DOB": "01-01-83",
  ...
}
```



# Amazon Translate

- Fluent and accurate language translation.
- Supports translating text between 75 languages (& growing)
- Use cases: Translate user manuals, books, documents, websites etc.

### Translation

Text | Document

Source language

English (en) ▼

Target language

Hindi (hi) ▼

Enter text

Congratulation on completing your AWS certification

Translated text

आपके AWS प्रमाणन को पूरा करने पर बधाई

51 characters, 51 of 10000 bytes used. [Info](#)

► Additional settings



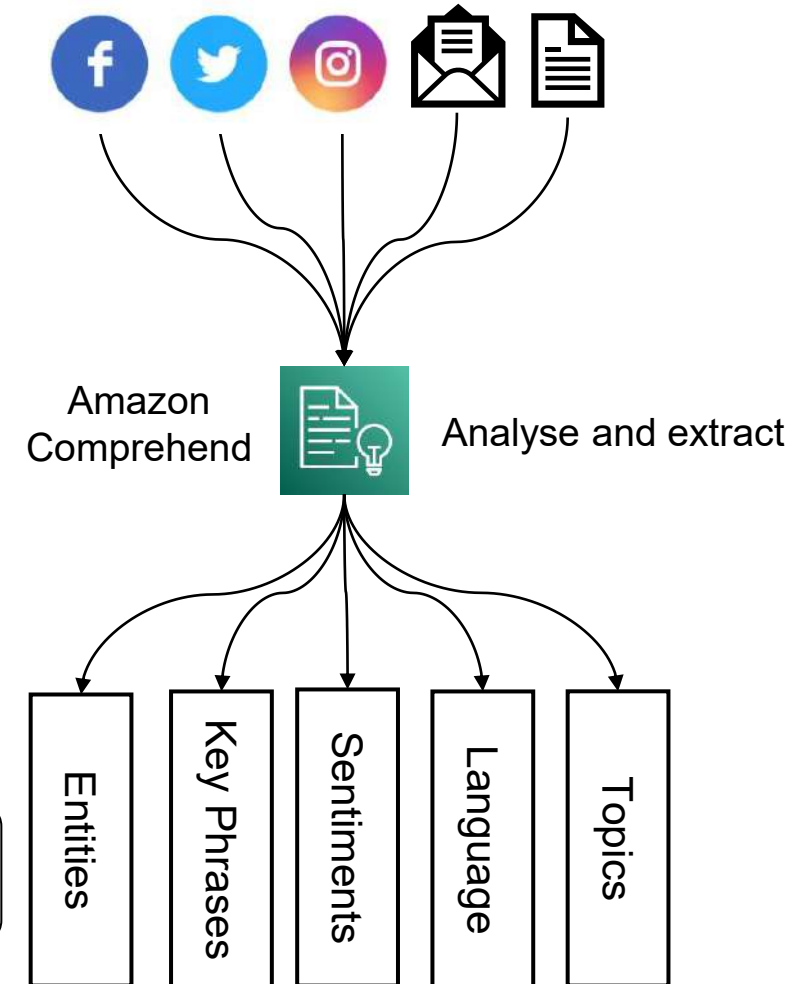
# Amazon Comprehend

- Uses **Natural Language Processing (NLP)** to extract insights about the content of documents.
- Finds insights and relationships in text such as:
  - Language of the text
  - Extracts key phrases, places, people, brands, or events
  - Understands how positive or negative sentiments are
  - Automatically organizes a collection of text files by topic

## Use Cases:

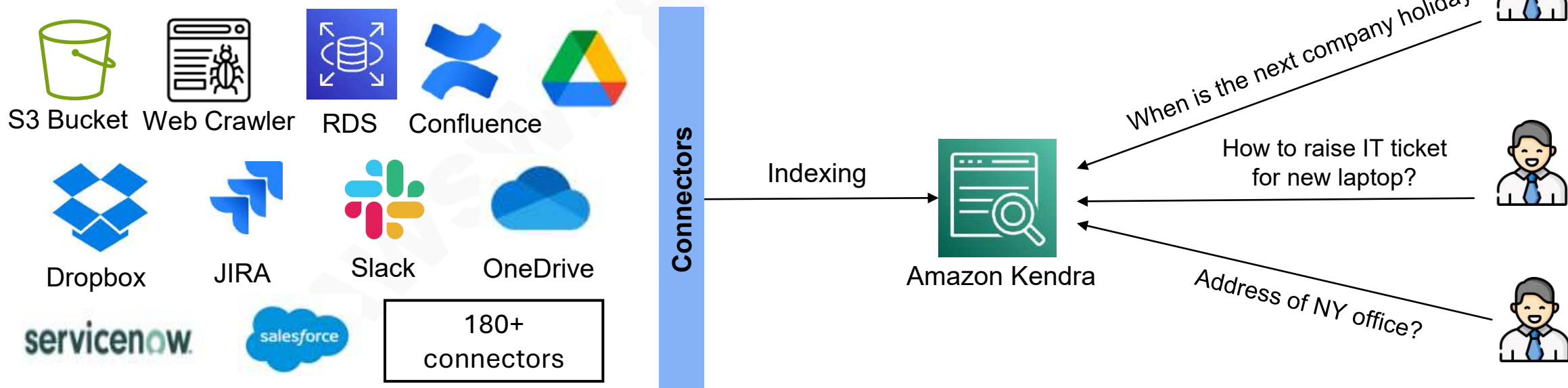
- Analyze customer interactions to find what leads to a positive or negative experience
- Create and groups articles by topics

**Amazon Comprehend Medical** - Extract medical information from medical text like doctors' notes, clinical trial reports, or radiology reports



# Amazon Kendra

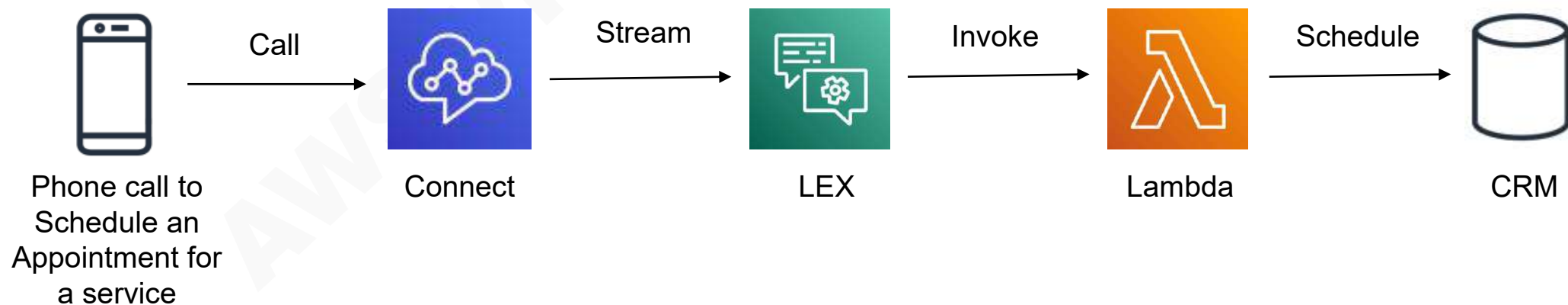
- An intelligent ML powered enterprise search service
- Employees and customers can find the content they're looking for from multiple locations and content repositories within your organization.
- Indexes variety of documents such as Websites/HTML, Share point, PPT, MS Word and provides accurate answers based on Natural language search capabilities.
- Incremental learning from user feedback





# Amazon Connect

- AI-powered cloud contact center
- Automatically detects customer issues and provides agents with contextual customer information and suggested responses and actions for faster resolution of issues.
- Easy to create flows and integration with other CRM systems or AWS

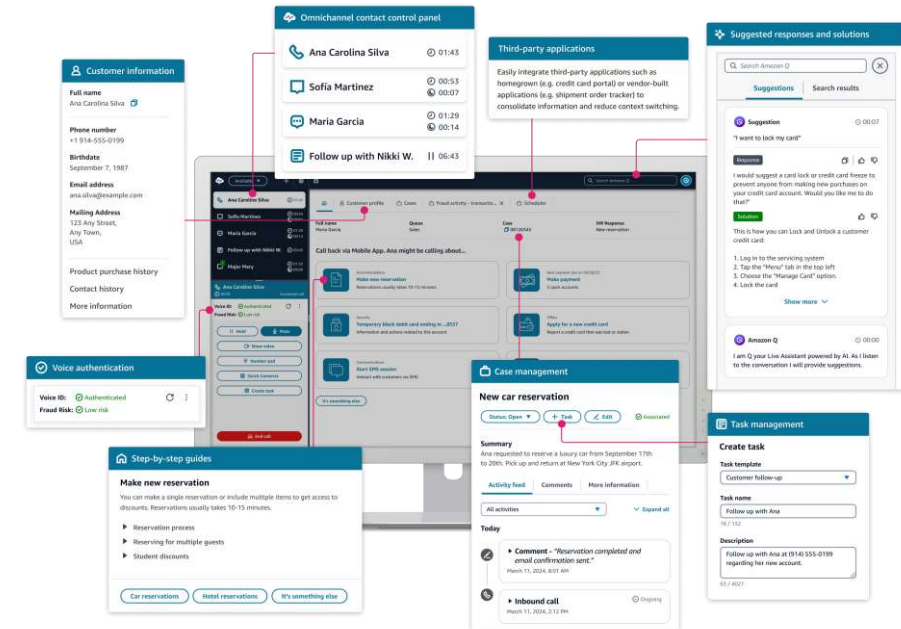


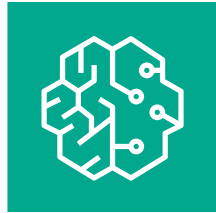
# Amazon Connect

- AI-powered cloud contact center
- Automatically detects customer issues and provides agents with contextual customer information and suggested responses and actions for faster resolution of issues.
- Easy to create flows and integration with other CRM systems or AWS

## Agent Workspace:

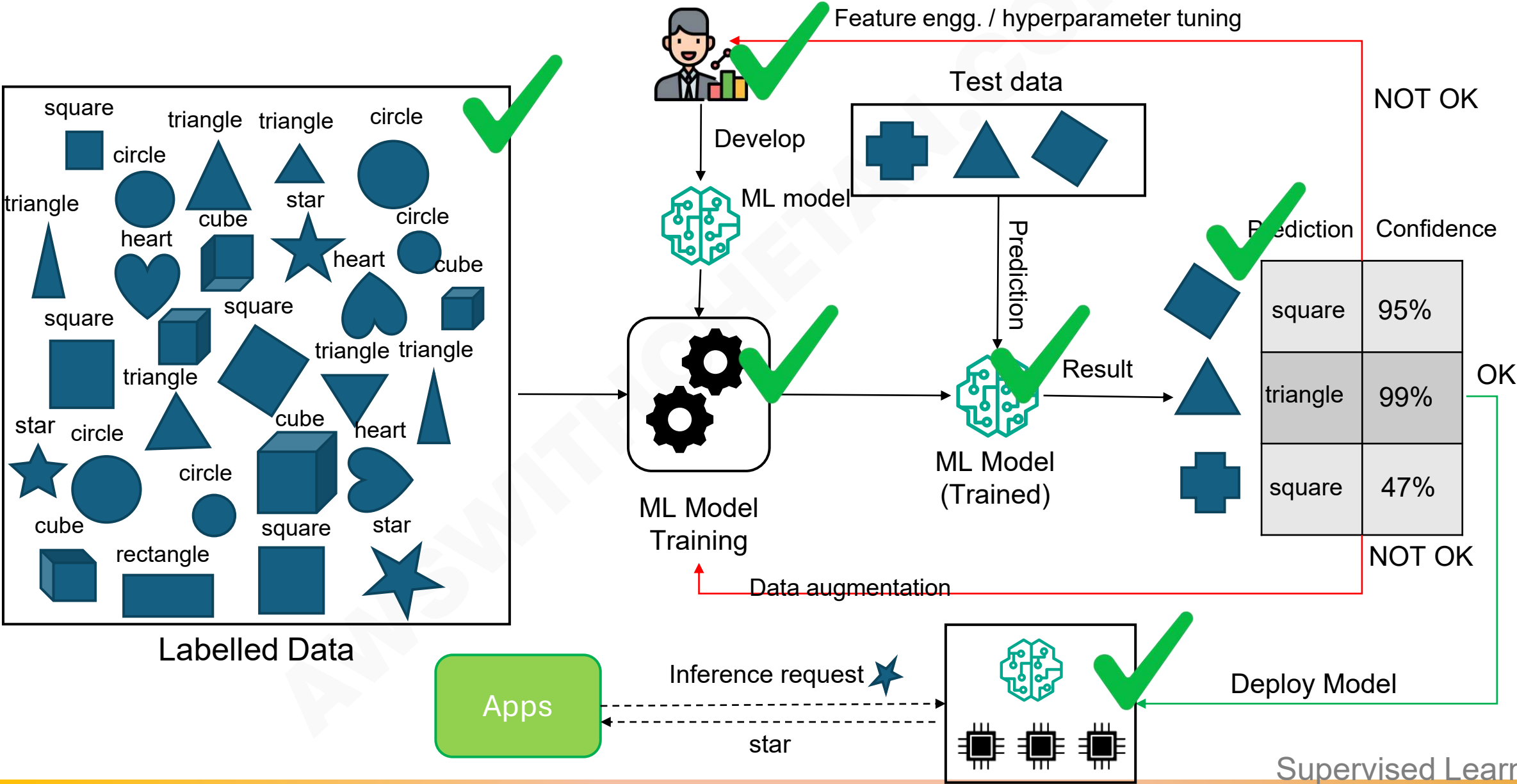
- When an agent accepts a call, chat, or task, they receive necessary information about the case and customer and real-time recommendations.
- 80% cheaper than traditional contact center solutions.





# Amazon SageMaker AI

# Machine Learning with Amazon SageMaker AI



# Amazon SageMaker AI

- Fully managed service for developers / data scientists to build, train and deploy ML models at scale
- Amazon SageMaker Features:
  - **SageMaker Studio** – A web-based IDE interface for building, training and deploying models.
  - **AutoML** (SageMaker Autopilot) - Automatically explores and creates the best ML models
  - **Built-in Algorithms** – A wide range of built-in machine learning algorithms optimized for performance and scalability
  - **Notebook Instances** - Managed Jupyter notebooks with pre-installed libraries, scalable compute, and data storage, making it easier to explore data and develop models.
  - **Training & Tuning** – Manages ML training infrastructure, Distributed training across multiple GPUs.
  - **Model deployment** – Deploy models to production in One click by creating endpoints, multi-model endpoints
  - **SageMaker GroundTruth** – A data labelling service (Text, images, video labelling)
  - **SageMaker Model Monitor** - Continuously monitors the quality of your models in production, detecting drift in model performance.
  - **SageMaker Pipelines**: Facilitates the automation and orchestration of machine learning workflows, supporting the implementation of MLOps best practices.

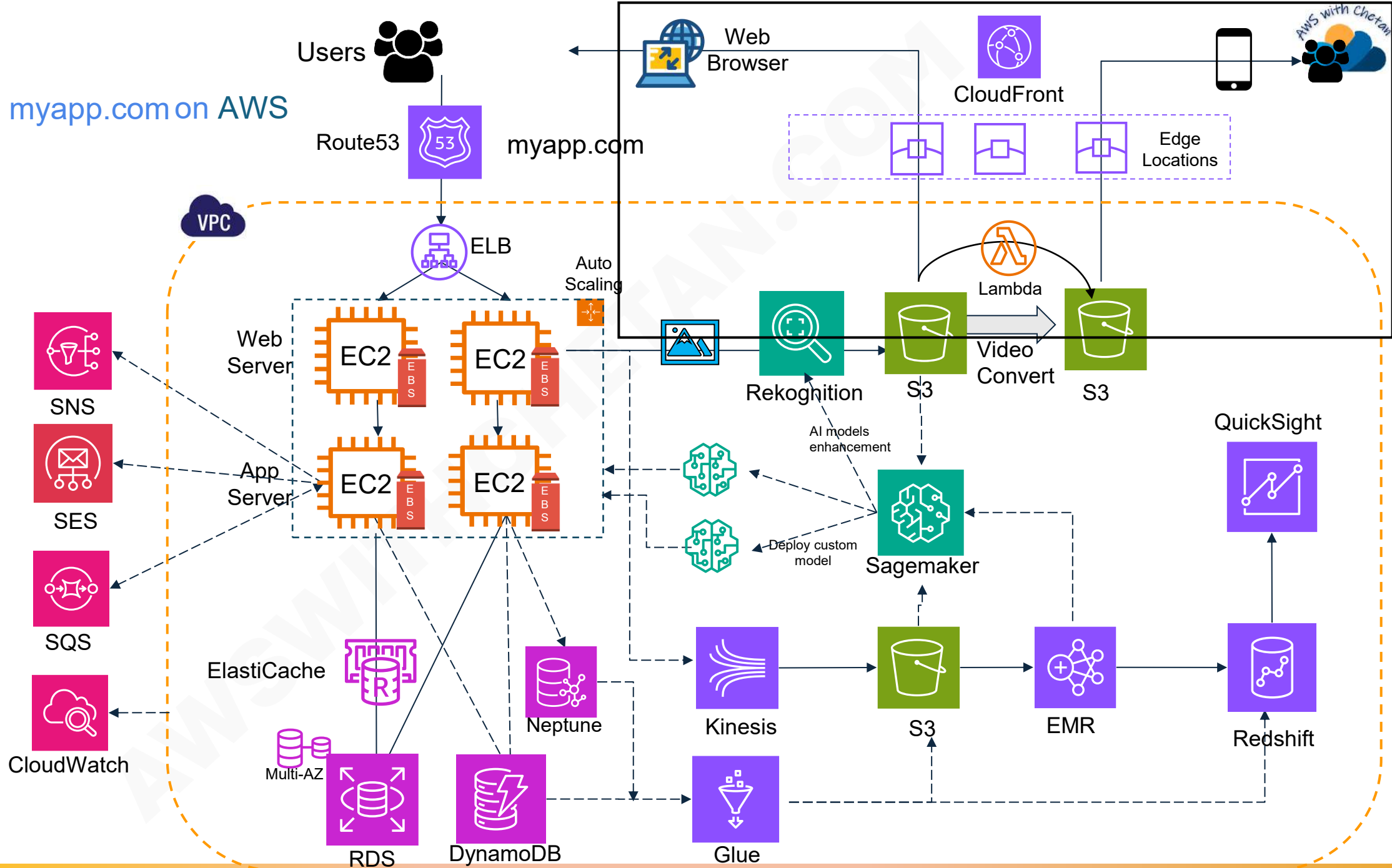
# AWS Edge Networking

Access applications with lowest latency across the globe



# Amazon CloudFront

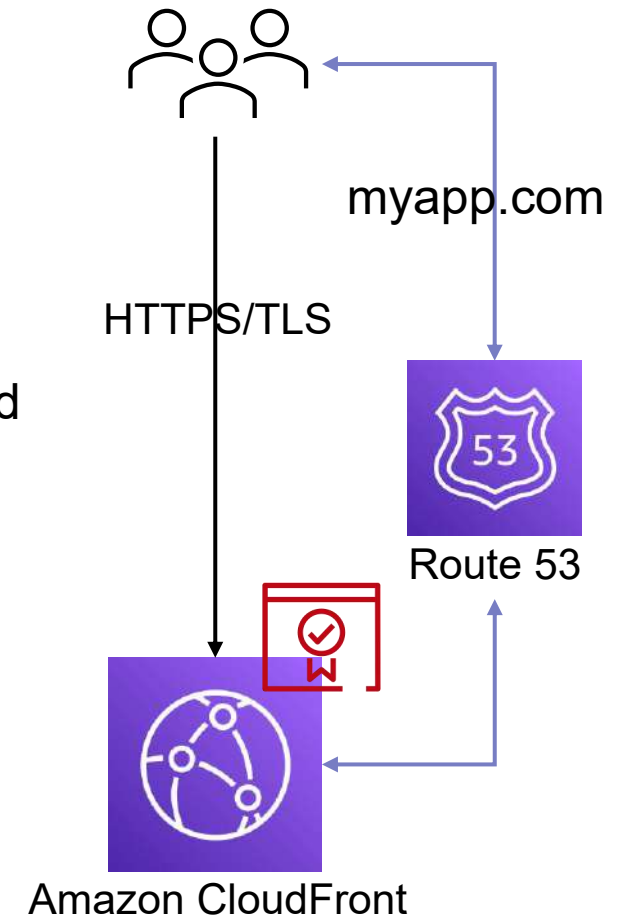
myapp.com on AWS

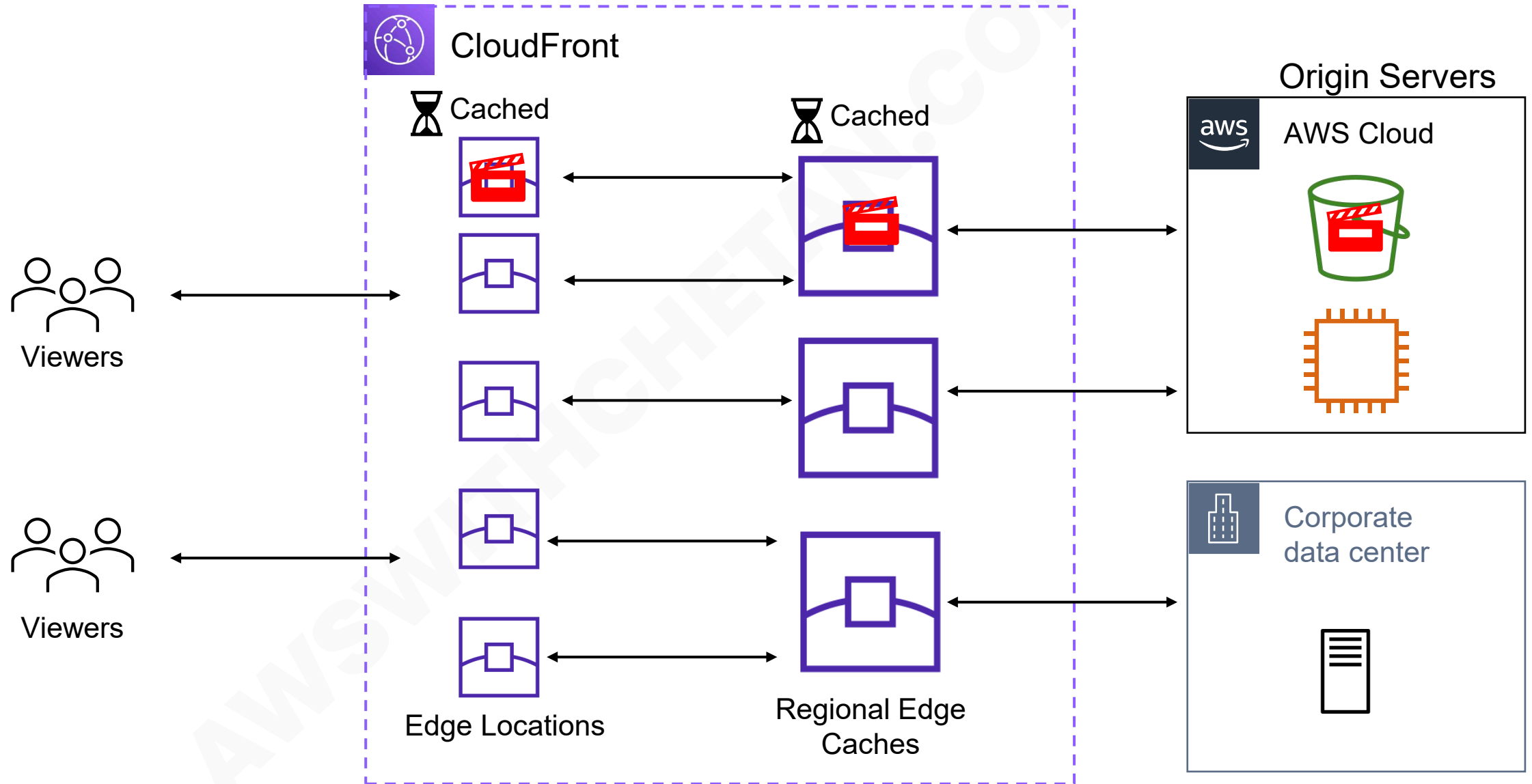




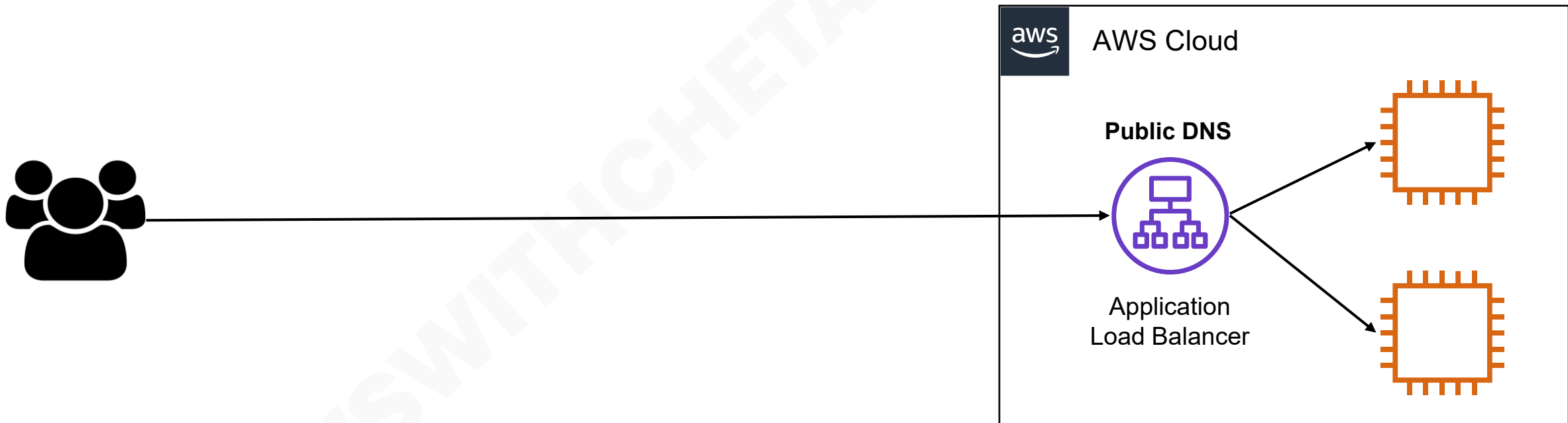
# Amazon CloudFront

- Amazon CloudFront is a global Content Delivery Network (CDN) service that delivers content with low latency through AWS edge locations.
- CloudFront operates with 100+ Points of Presence and 10+ Regional Edge Caches (RECs) globally.
- **Regional Edge** Caches act as a mid-tier caching layer between the origin and edge locations. They store objects longer than edge locations.
- Built-in DDoS protection with Amazon Shield (Layer 3 and 4) and Web attack protection with AWS WAF (Layer 7).
- Integrates with Amazon Certificate Manager (ACM) to use SSL/TLS certificates for secure HTTPS connections.
- Integrates with Amazon Route 53 (DNS) which allows you to use your own custom domain name with the CloudFront distribution.

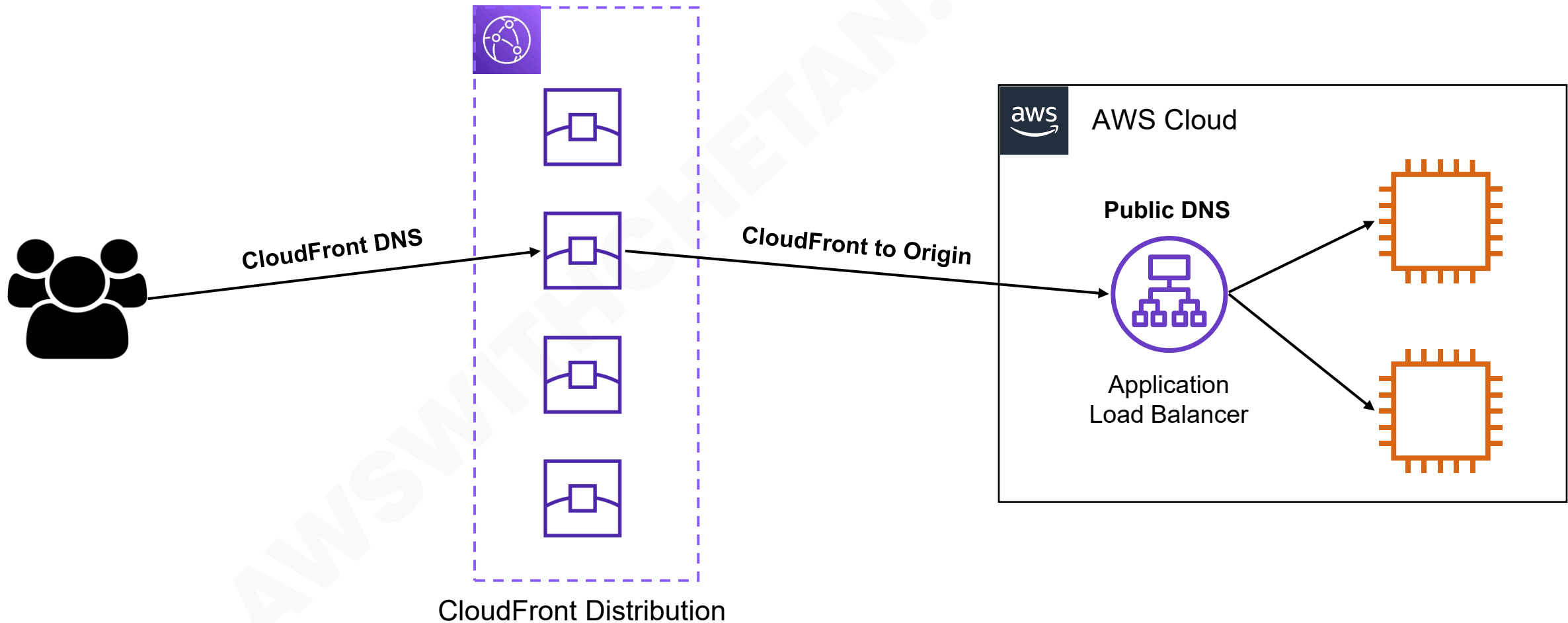




# How it works?

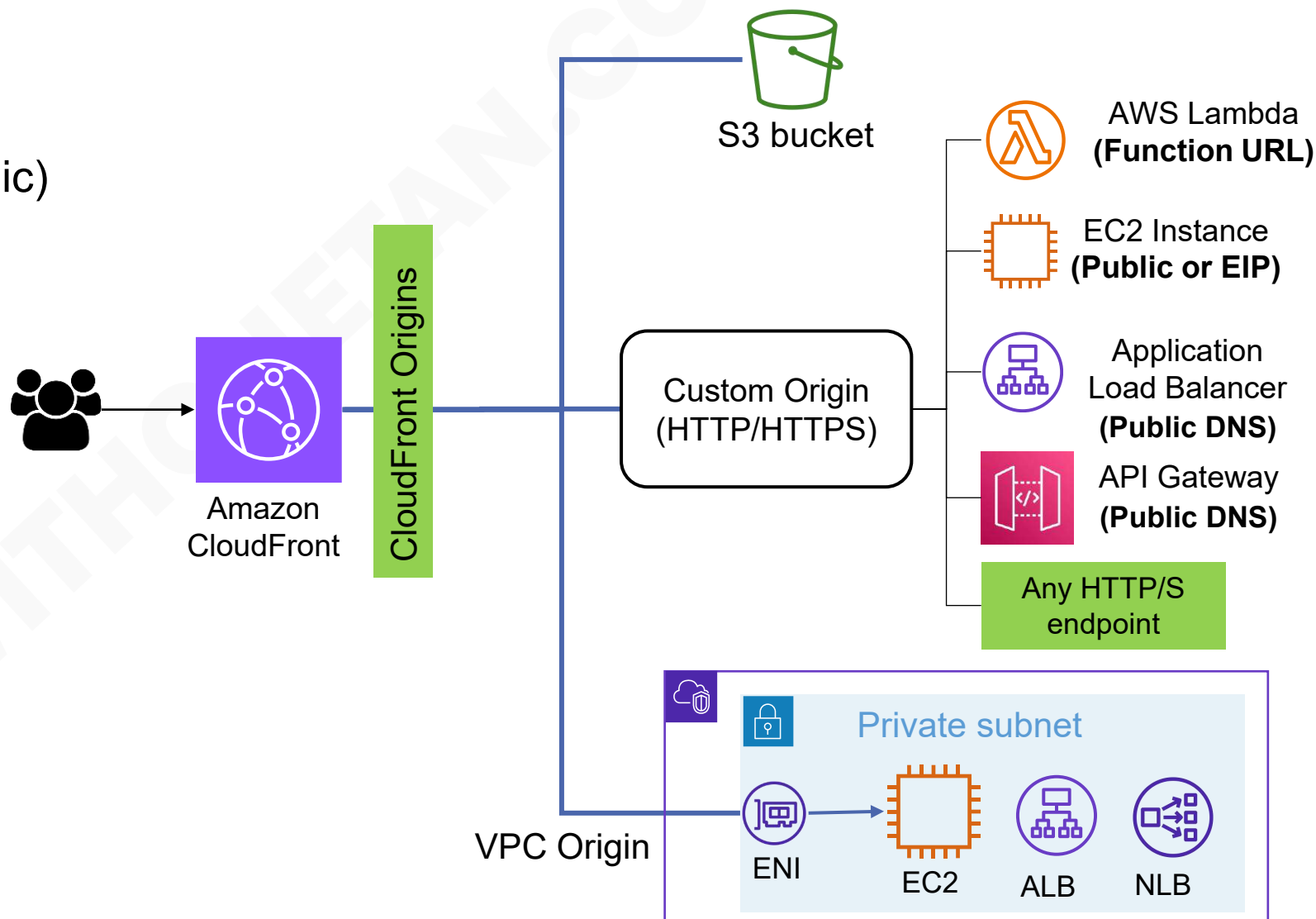


# How it works?



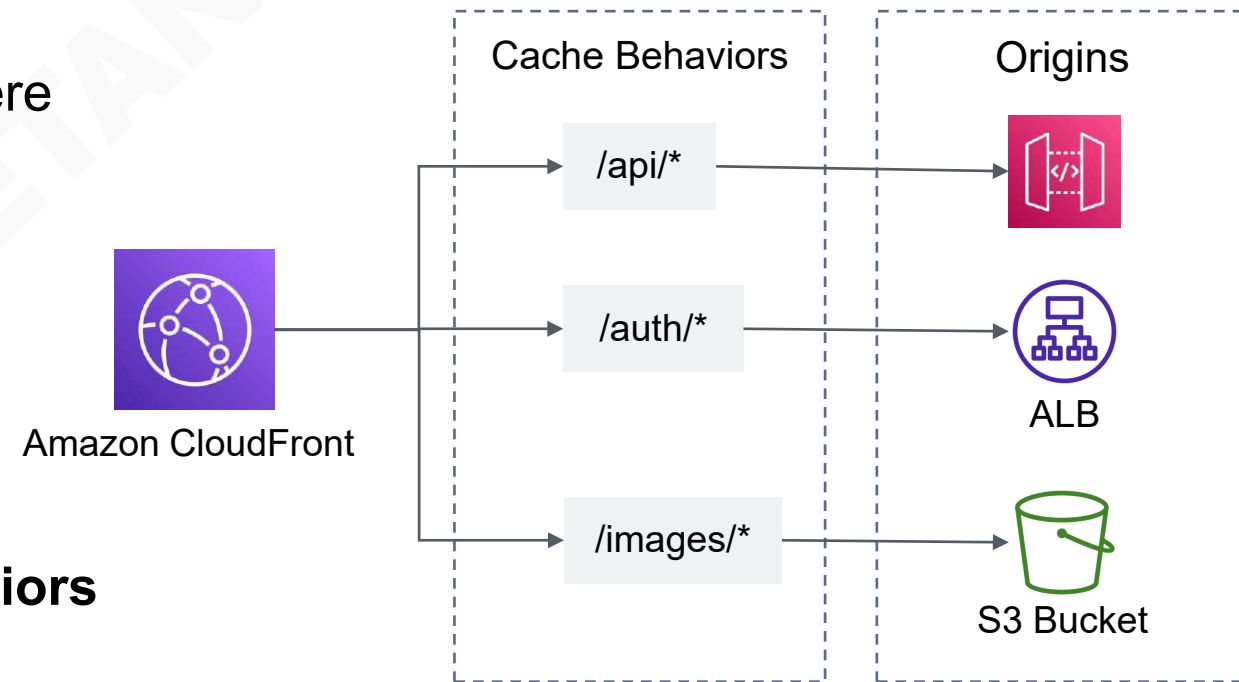
# Amazon CloudFront Origins

- S3 Bucket
- Custom HTTP Origins (Public)
  - EC2
  - ALB
  - API gateway
  - Lambda Function URL
  - Any HTTP endpoint
- VPC Origin (Private)
  - EC2
  - ALB
  - NLB



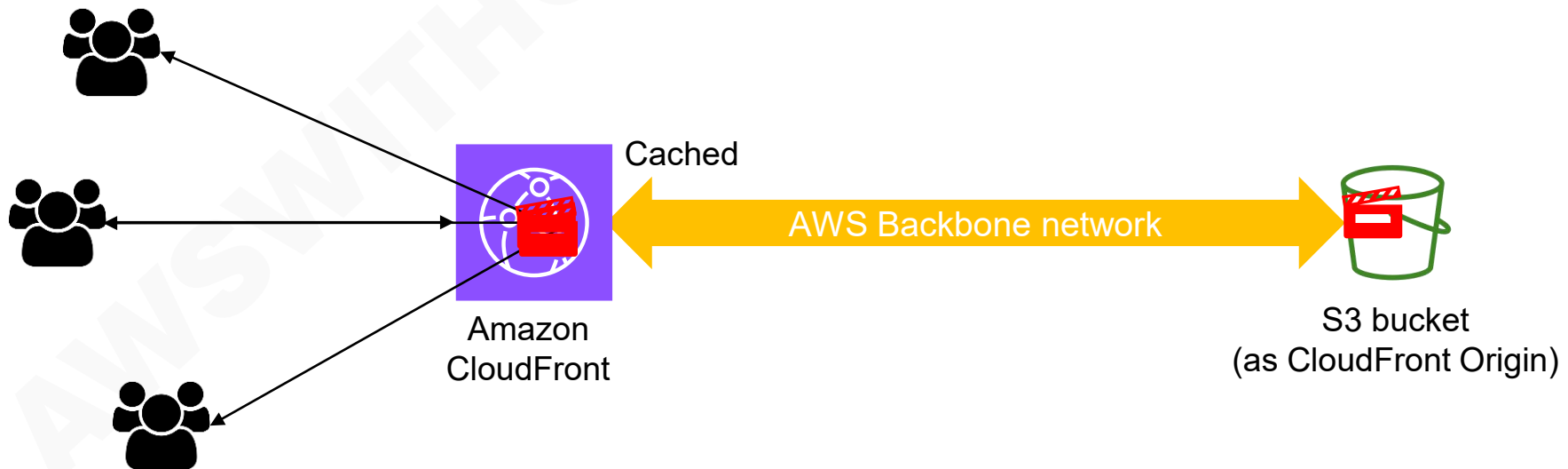
# CloudFront – Multiple Origins

- CloudFront can route viewer requests to different origins based on request attributes.
- A common method is path-based routing, where the URL path determines which origin should handle the request.
- Example:
  - `/api/*` → API Gateway origin
  - `/auth/*` → Application Load Balancer origin
  - `/images/*` → S3 origin
- CloudFront achieves this using **Cache Behaviors** for different origins



# S3 Origin

- S3 is a popular CloudFront origin because large applications store terabytes of static media (e.g. images, videos) in S3, and CloudFront delivers it faster through edge network and caching it closer to the end users.
- Reduced Data transfer out rate as compared to internet (+1TB free data transfer out per month)
- You can block direct access to S3 using CloudFront Origin Access Identity (OAI) or Origin Access Control (OAC)



# Block direct access to S3

## CloudFront Origin Access Identity (OAI)

- Legacy mechanism to access a private S3 bucket.
- Creates IAM-like identity

### S3 Bucket Policy for OAI

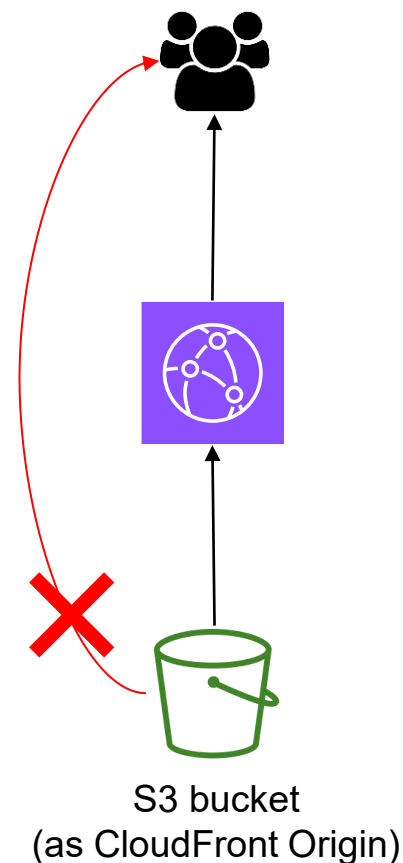
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS":
"arn:aws:iam::cloudfront:user/CloudFront
Origin Access Identity <OAI-ID>"
      },
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::<bucket-
name>/*"
    }
  ]
}
```

## CloudFront Origin Access Control (OAC)

- New mechanism to access a private S3 bucket.
- Uses SigV4 signing

### S3 Bucket Policy for OAC

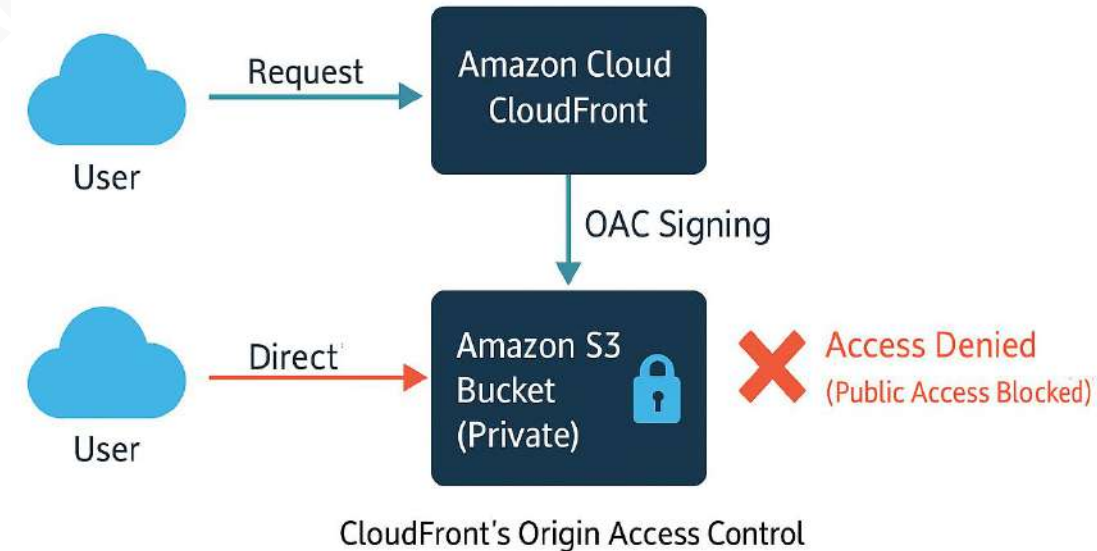
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOAC",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::<bucket-name>/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn":
"arn:aws:cloudfront::<account-
id>:distribution/<distribution-id>"
        }
      }
    }
  ]
}
```



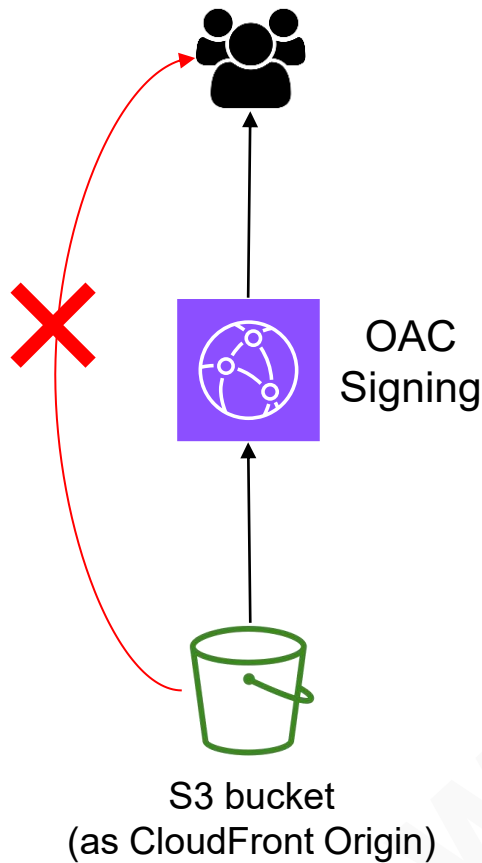


# Origin Access Control (OAC)

- Restricts direct access to your S3 bucket, ensuring content is only accessible through CloudFront.
- OAC is the modern, more secure replacement for Origin Access Identity (OAI is legacy).
- OAC comes with Enhanced security with AWS Signature Version 4 (SigV4). OAI uses Legacy (SigV2) for Authentication
- OAC supports SSE-KMS encryption (OAI does not)
- **Note:** You cannot use OAC (or OAI) with an S3 bucket configured as a Static website endpoint. For website endpoints, you must treat the S3 bucket URL as a Custom Origin.



# Exercise: CloudFront with S3 Origin



1. Create an S3 bucket and upload a sample image (keep bucket private).
2. Try accessing image with the S3 URL -> Access Denied
3. Create a CloudFront distribution and choose the S3 bucket as the origin.
4. Configure Settings – *Allow private S3 bucket access to CloudFront - Recommended*
5. Let CloudFront update the S3 bucket policy automatically to allow only CloudFront (via OAC) to access the bucket.
6. Deploy the CloudFront distribution and wait for propagation.
7. Access the image with CloudFront distribution DNS, it should work

# Amazon CloudFront – Security features

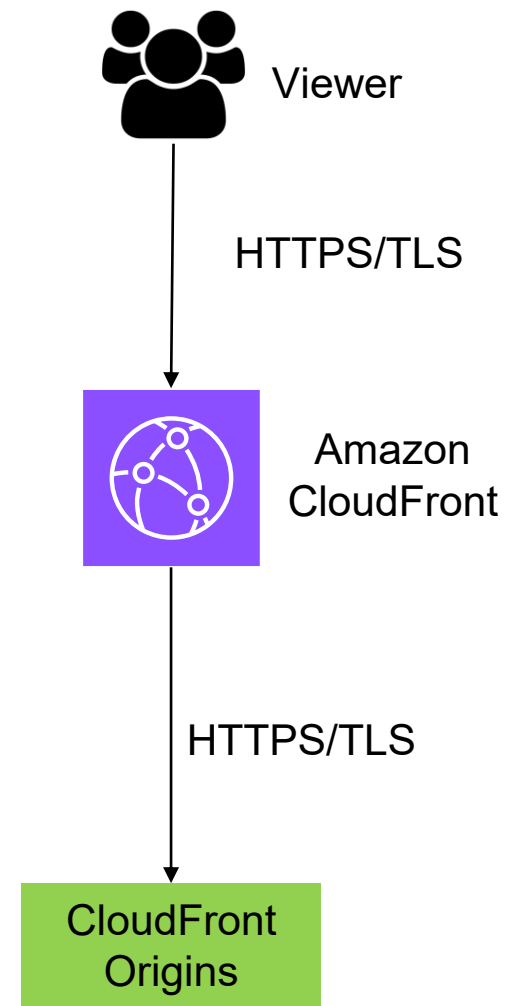
- HTTPS
- Mutual TLS (mTLS)
- CloudFront Field-level encryption
- Signed Cookies and Signed URL
- Restricting VPC based Origins access from CloudFront using Security Groups
- AWS WAF and AWS Shield for CloudFront



Amazon CloudFront

# HTTPS / TLS

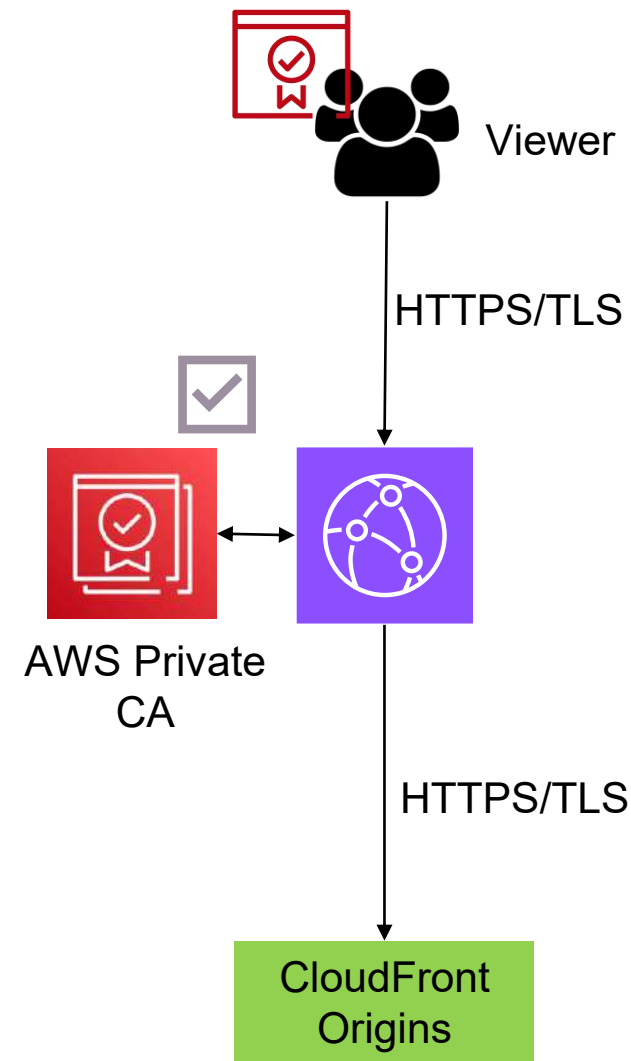
- CloudFront supports HTTPS between Viewer and CloudFront
- Viewer Protocol Policy:
  - Allow HTTP & HTTPS
  - Redirect HTTP to HTTPS
  - HTTPS only
- CloudFront supports HTTPS between CloudFront and Origin
- Origin Protocol Policy:
  - HTTPS only
  - Match viewer
  - HTTP only
- You can enforce minimum TLS versions (e.g. TLS1.2) using Security Policies.
- For using custom domain name (e.g. example.com) you need to have SSL/TLS Certificate in ACM in us-east-1 region



# Mutual TLS (mTLS)

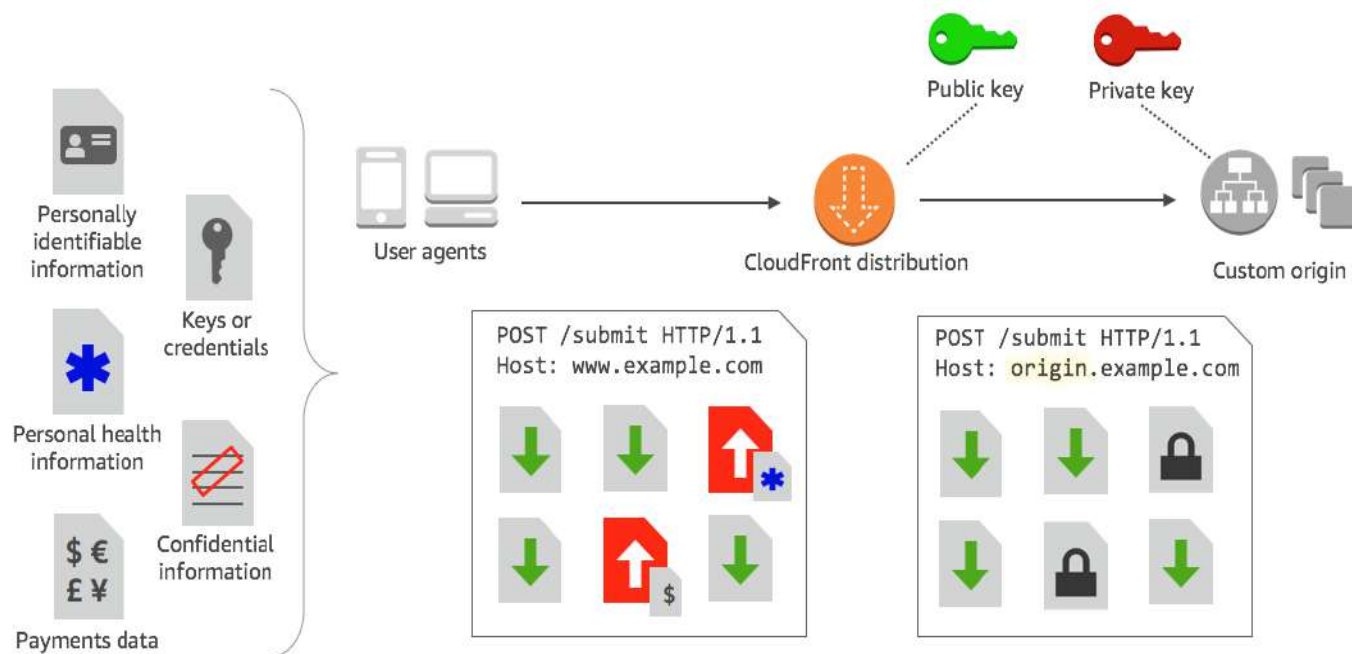
## mTLS = Client certificate authentication at CloudFront

- Viewer presents a client certificate during TLS handshake.
- CloudFront validates that certificate against a trusted CA stored in ACM.
- CloudFront verifies:
  - Certificate chain → must chain to the trusted CA in ACM
  - Certificate validity period → not expired, not before valid
  - Revocation supported only when using AWS Private CA (via CRL/OCSP behind the scenes)
- Works only with custom domains, not with the default \*.cloudfront.net domain.
- mTLS is enforced before CloudFront routes the request to the origin. If client cert fails, CloudFront returns 403 Forbidden.



# CloudFront's field-level encryption (FLE)

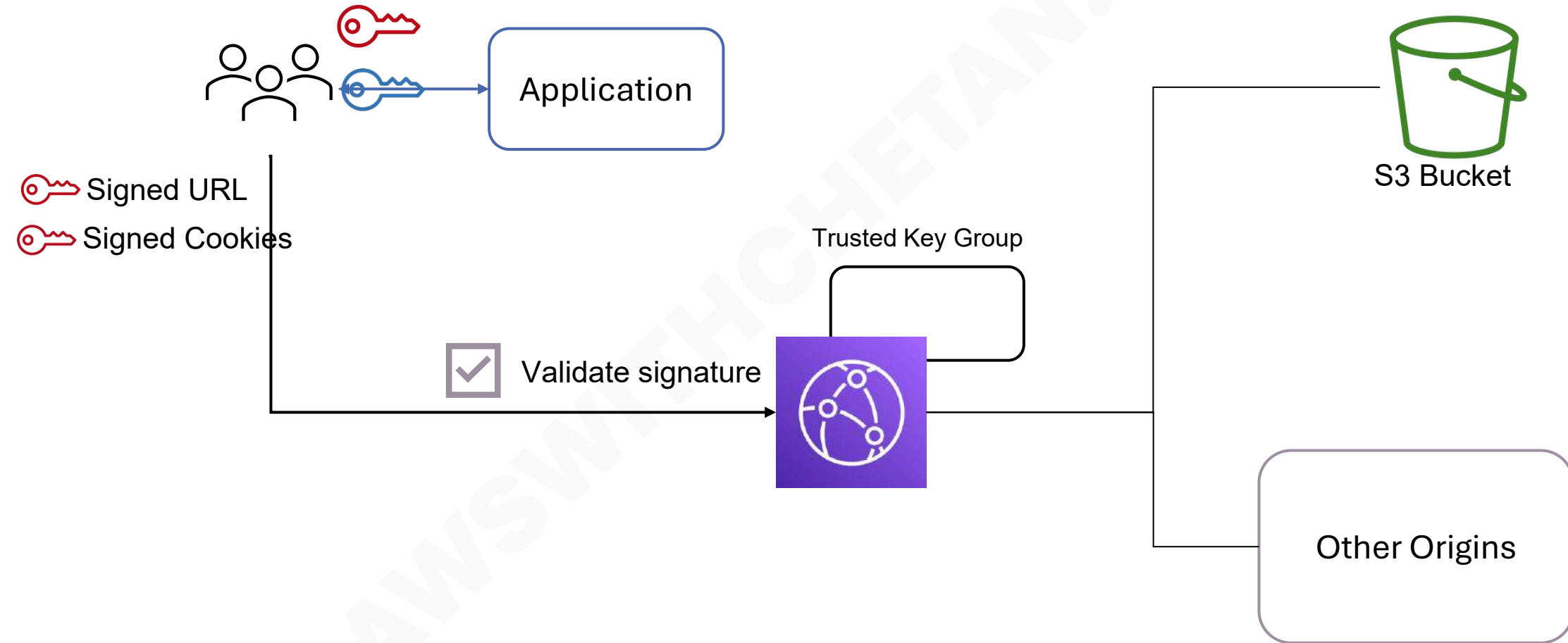
- Adds an extra security layer on top of HTTPS by encrypting specific sensitive fields (e.g., credit card numbers, PII).
- Encryption happens at CloudFront edge, so protected fields stay encrypted through proxies, logs, and internal systems.
- Uses asymmetric encryption: CloudFront uses public key, origin uses private key to decrypt. *(can use openssl to create keys)*
- Ensures only the origin application can read sensitive values; CloudFront and intermediaries cannot.



<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/field-level-encryption.html>

# Serving Private content through CloudFront

Allow access to only authenticated Viewers



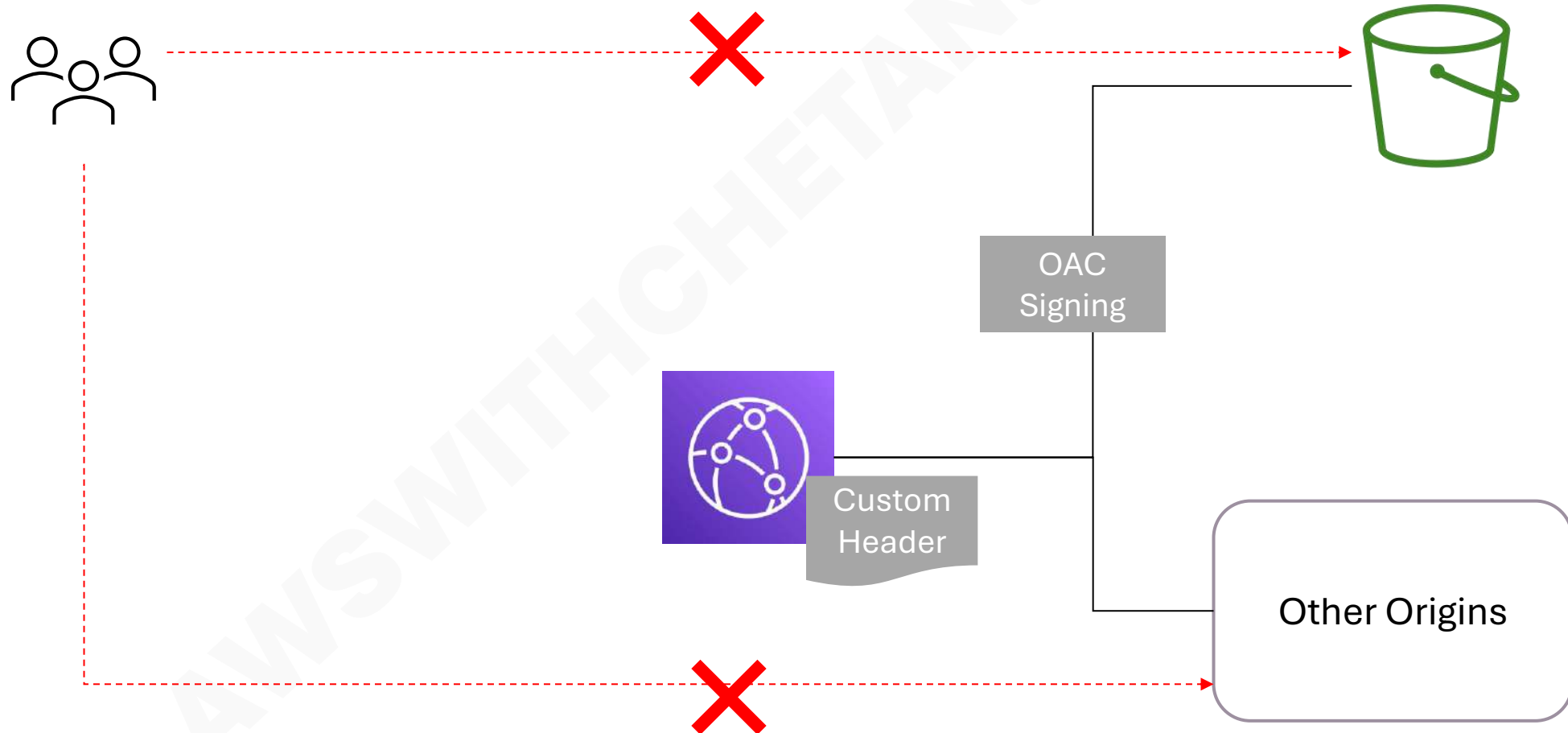
# Signed Cookies and Signed URL

- CloudFront Signed URLs / Signed Cookies are used to serve private content securely - content that should be accessible only to authorized users (e.g. paid users, subscribers, restricted documents/media).
- Signed URLs and Signed Cookies are designed for short-lived, temporary access - you specify an expiration time, so access automatically ends after a defined period.
- Signer has a public/private key pair. The private key is used to sign the URL or cookie; CloudFront uses the public key to verify validity.
- When a user requests a resource with a valid signature (URL or cookie), CloudFront verifies the signature + policy (e.g. expiration, allowed resource) and if valid, serves the content (from cache or origin) otherwise returns 403 Forbidden / Access Denied response.
- **Use Signed-URLs to restrict access to a single file (e.g. download, shareable link).**
- **Use Signed-Cookies to restrict access to many files / streaming media / entire section.**



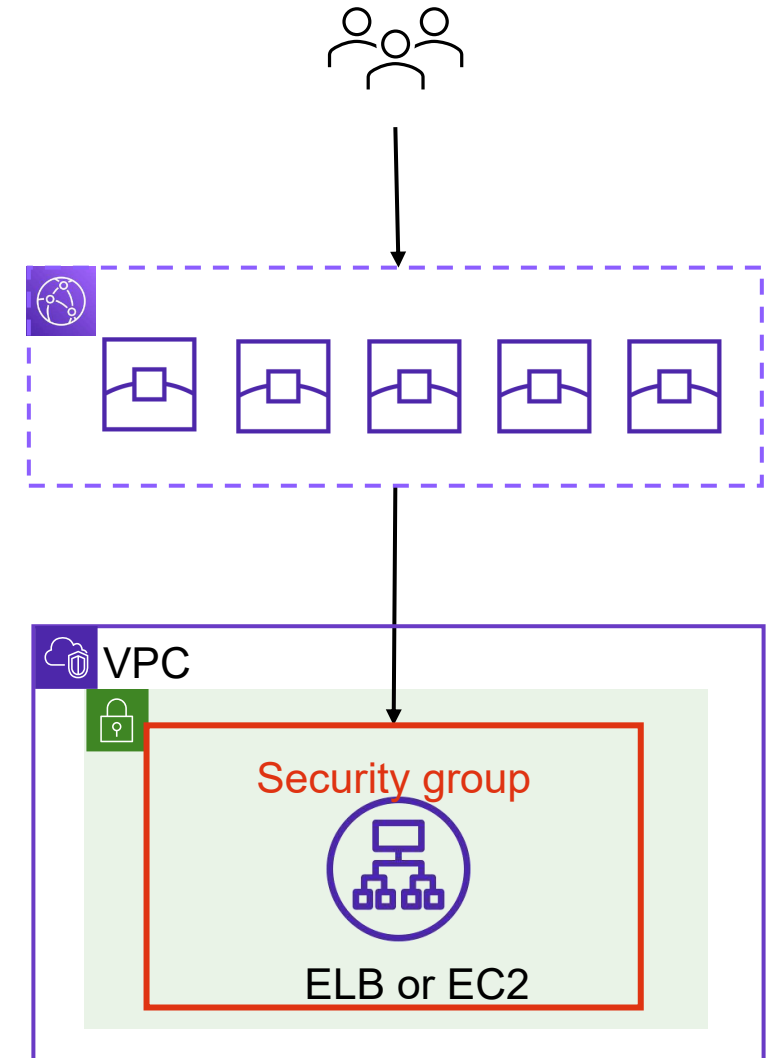
# Restricting direct access to Origins

## 1. Restricting direct access from Viewers to Origin



# Restricting origin access only from CloudFront

- Access to the origin can be restricted at network layer such that only CloudFront edge locations can reach it.
- This restriction can be applied only to origins that support Security Groups to whitelist (allow) CloudFront IP ranges.
  - ALB / NLB (inbound rules)
  - EC2 instances
  - VPC-hosted custom origins (ENIs)
- Two ways to get CloudFront's origin-facing IP ranges:
  - AWS ip-ranges.json (needs automation)
  - AWS Managed Prefix List (recommended)



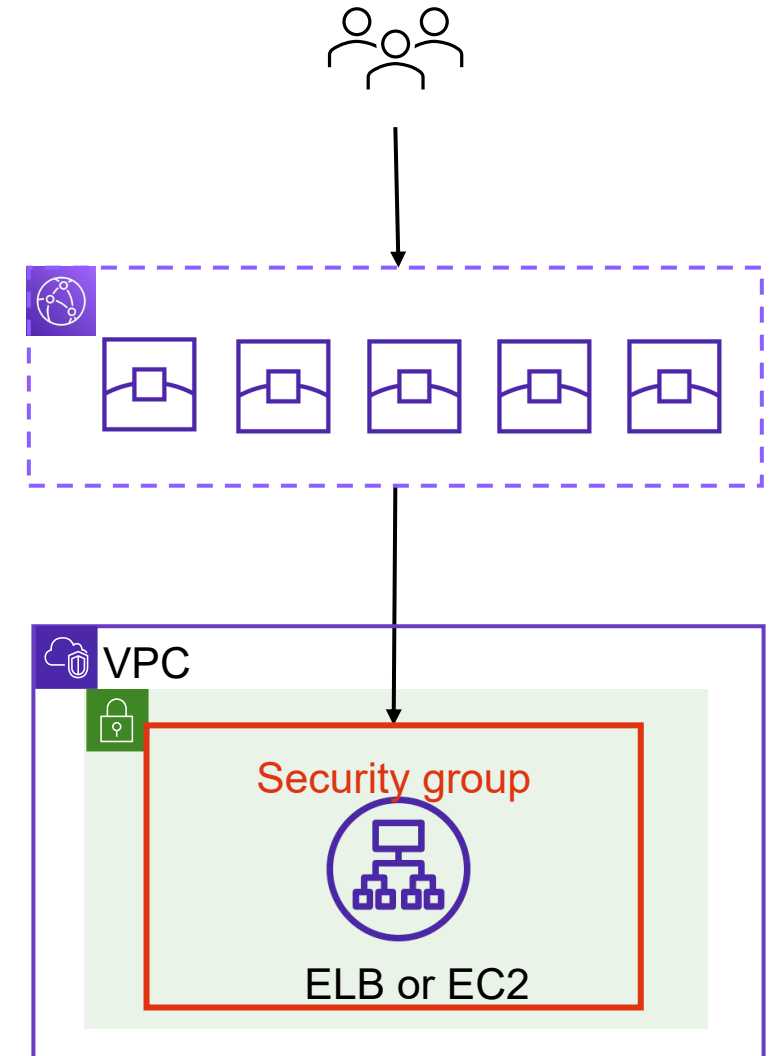
# Restricting origin access only from CloudFront

## 1. AWS ip-ranges.json

- The JSON file at <https://ip-ranges.amazonaws.com/ip-ranges.json> publishes all AWS public IP ranges.
- To find CloudFront origins, filter entries where:
  - "service": "CLOUDFRONT" and "region": "GLOBAL"
- These CIDRs can then be added as inbound Security Group rules.
- Requires automation (Lambda + EventBridge) because CloudFront IP ranges can change at any time hence
- Higher operational overhead.

## 2. AWS Managed Prefix List (recommended)

- AWS provides a managed prefix list:  
[com.amazonaws.global.cloudfront.origin-facing](https://com.amazonaws.global.cloudfront.origin-facing)
- Automatically maintained by AWS — no manual updates required.
- Can be referenced directly in Security Group rules and route tables.
- Simplifies configuration and reduces the risk of missing updates.



# Using CloudFront Prefix list

## Security group inbound rule:

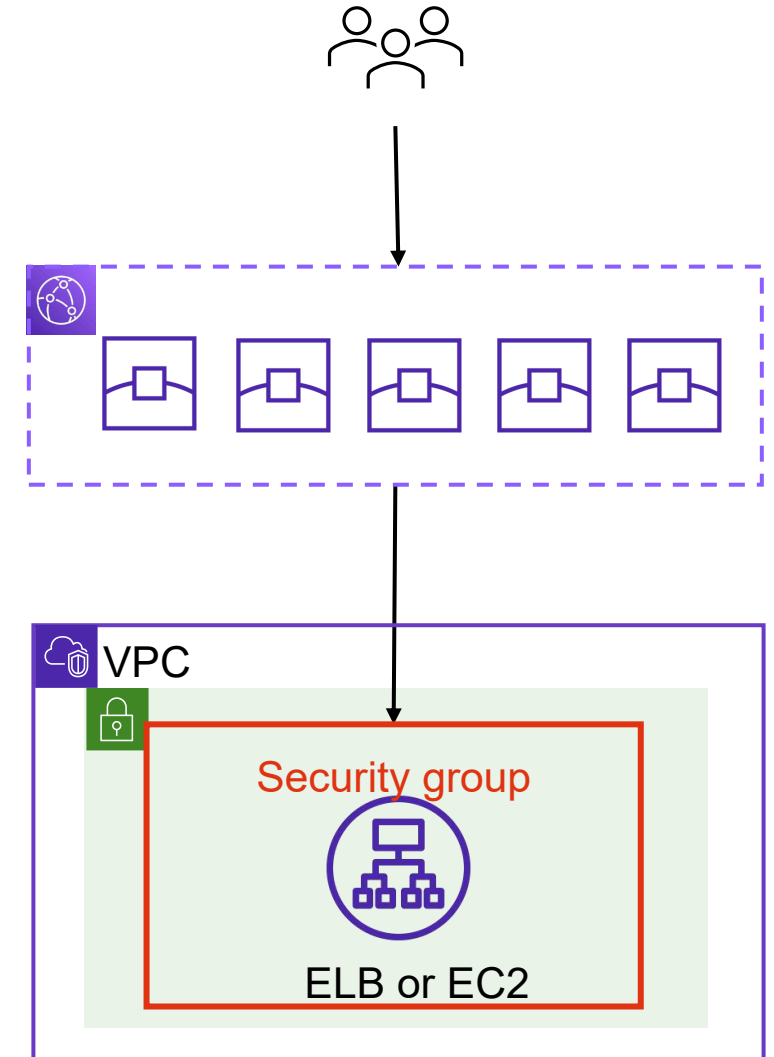
**Inbound rules** [Info](#)

| Type <a href="#">Info</a> | Protocol <a href="#">Info</a> | Port range <a href="#">Info</a> | Source <a href="#">Info</a>                    |
|---------------------------|-------------------------------|---------------------------------|------------------------------------------------|
| HTTP ▼                    | TCP                           | 80                              | Custom ▼ <input type="text" value="Q"/>        |
| HTTPS ▼                   | TCP                           | 443                             | Custom ▼ <input type="text" value="Q global"/> |

[Add rule](#)

Prefix lists

com.amazonaws.globa  
l... | pl-9aa247f3



# AWS WAF and AWS Shield for CloudFront

- CloudFront integrates directly with AWS WAF to protect applications from common Layer 7 web attacks such as SQL injection, XSS, bad bots, and request floods.
- AWS WAF filtering happens before traffic reaches your origin, reducing load and preventing malicious requests from propagating.
- CloudFront includes AWS Shield Standard automatically, providing always-on protection against common Layer 3/4 DDoS attacks at no extra cost.
- For mission-critical workloads, AWS Shield Advanced gives enhanced DDoS protection, larger capacity, near real-time attack visibility, and 24/7 AWS DDoS Response Team (DRT) support.
- Using CloudFront + WAF + Shield together provides a multi-layered, global defense against both network-layer and application-layer threats.



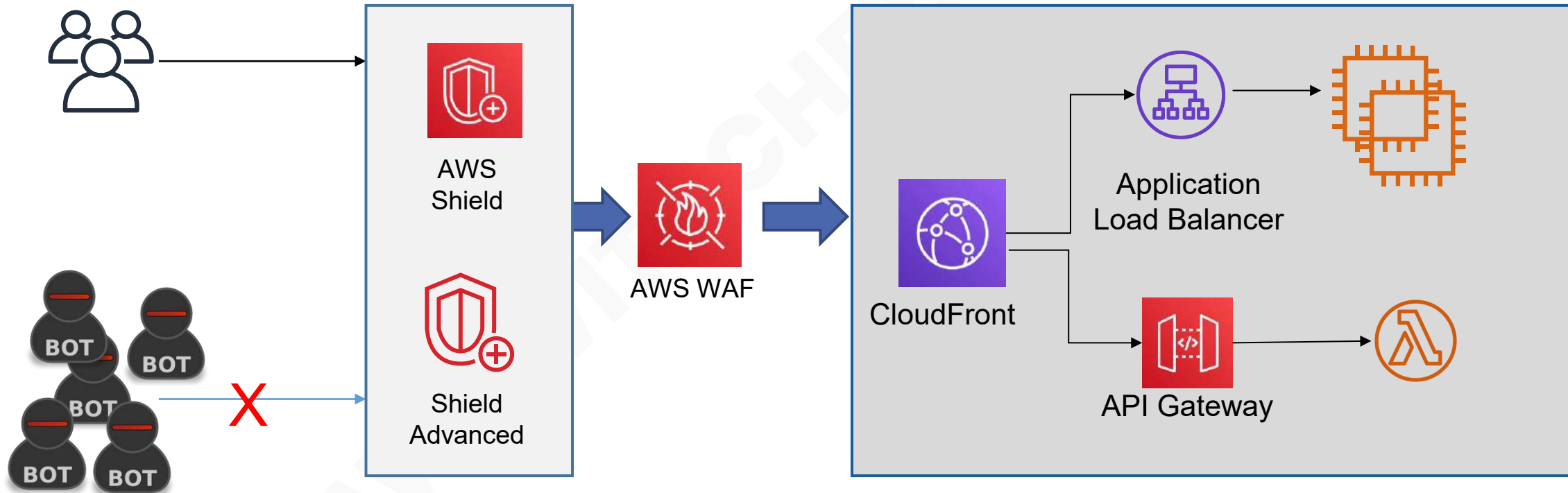
AWS Shield



AWS WAF

# AWS WAF and AWS Shield for CloudFront

## Sample Application architecture



# CloudFront Features – Important to know

- CloudFront Geo Restriction
- CloudFront Cache Behavior - TTL and Cache Invalidation
- Serverless edge computing:
  - CloudFront Functions
  - Lambda@Edge
- CloudFront with Custom Domain Name (Route 53)



Amazon CloudFront

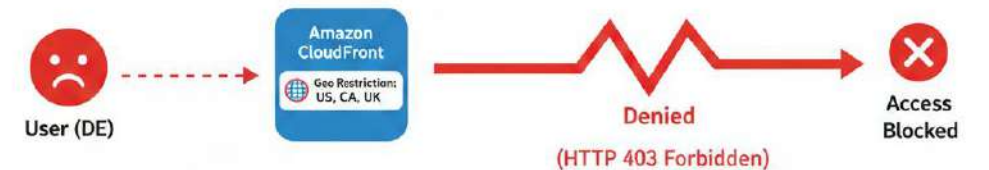
# CloudFront Geo Restriction

- CloudFront lets you control which countries can access your content.
- CloudFront identifies the viewer's country using a 3<sup>rd</sup> party trusted Geo-IP database.
- **Allowlist** (Whitelist):  
Only users from the approved countries can access your distribution.
- **Blocklist** (Blacklist):  
Users from the blocked countries are denied access.
- **Common Use Case:** Enforcing copyright rules, regional streaming restrictions, or limiting access to certain geographic regions.

## ALLOWED REQUESTS (Viewer Country = US)



## BLOCKED REQUESTS (Viewer Country = DE)

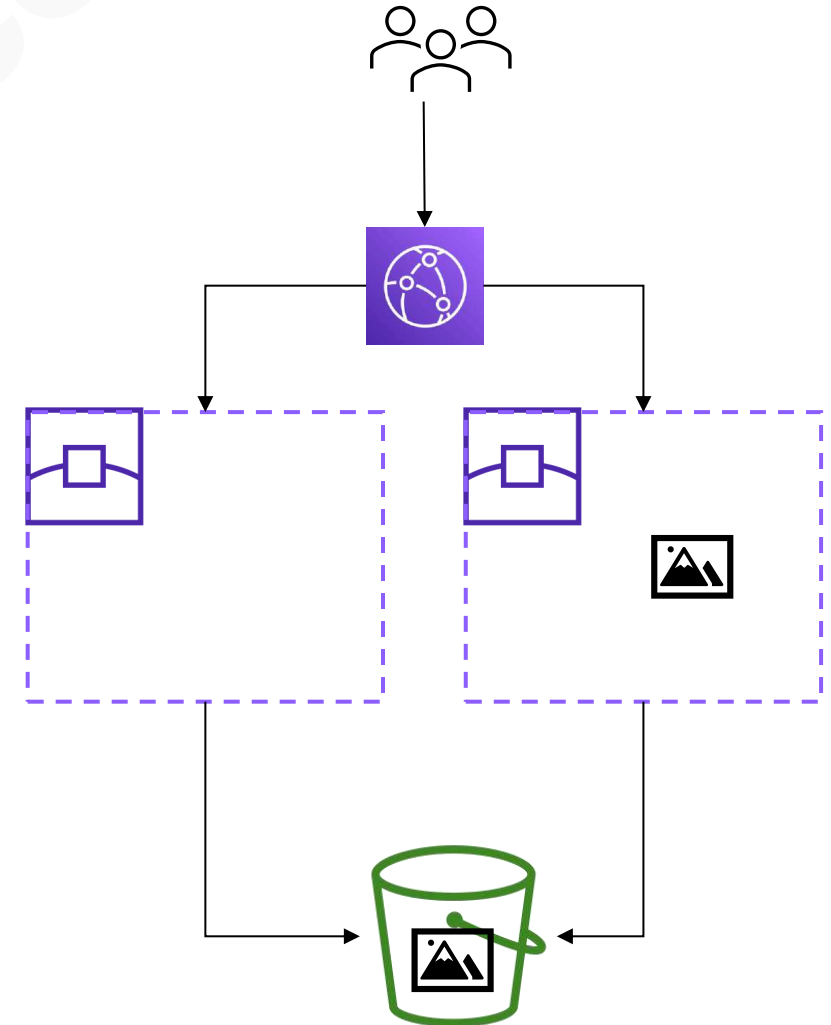


- Green Path: Allowed Request
- Red: Blocked Request



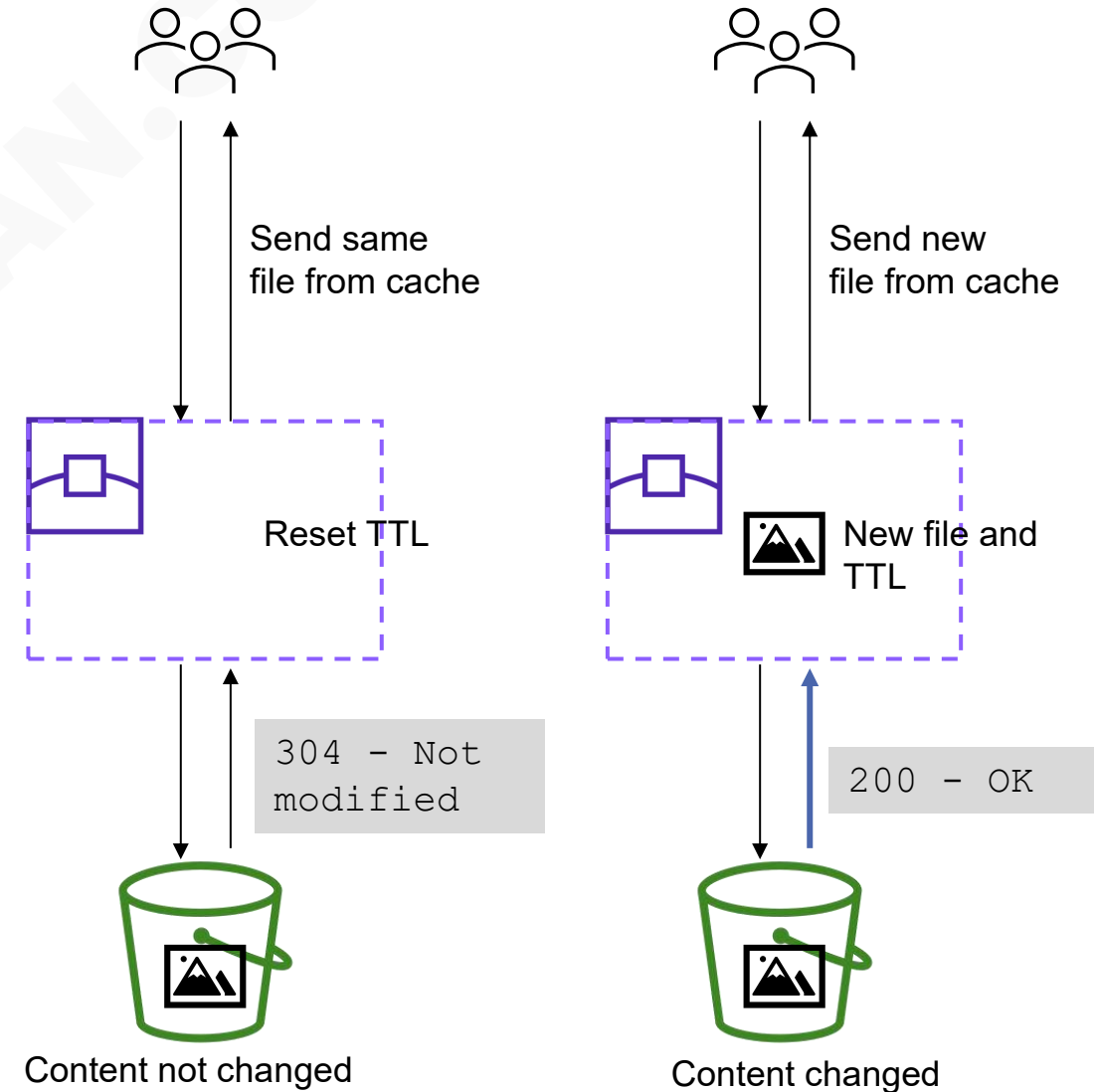
# Cache Behaviour - TTL (Time To Live)

- TTL defines how long an object should be kept in the cache before fetching it from the origin if requested.
- Default TTL is 24 hrs. Can set Minimum and Maximum TTL.
- Lower TTL helps serve latest content but causes higher origin load
- Higher TTL provides better performance and lower cost but slower updates.
- Dynamic APIs usually set low/no caching and static assets set high TTL.



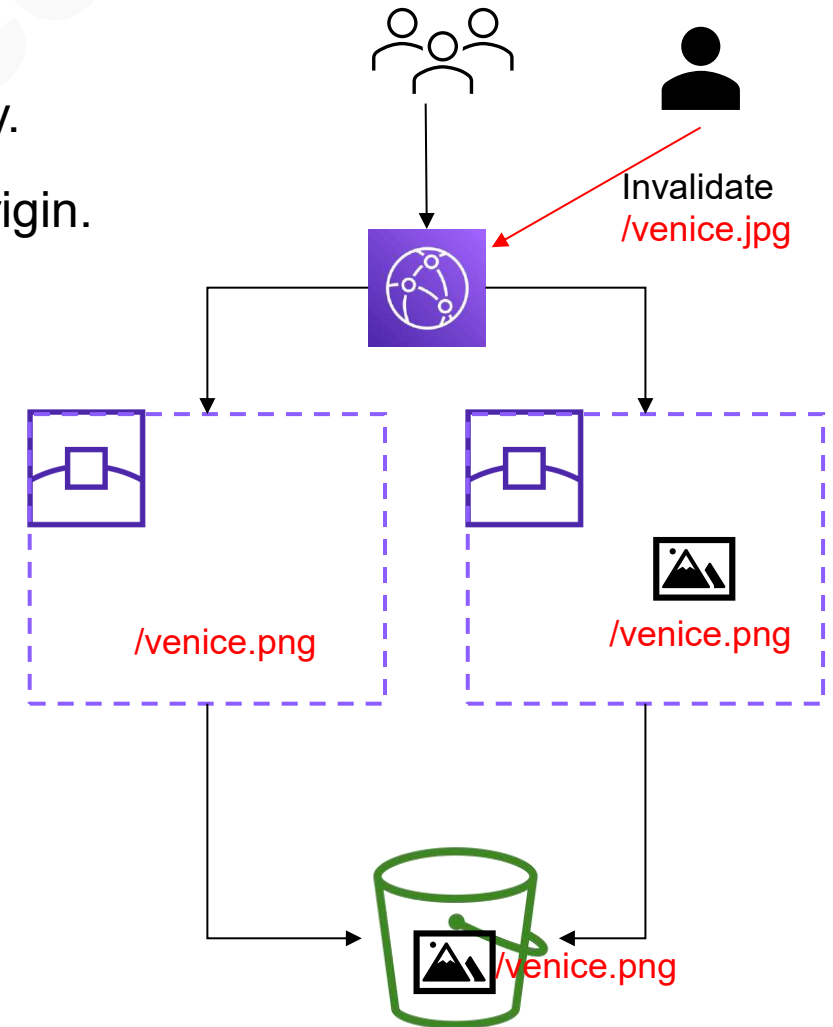
# TTL – Behind the scene

1. When the TTL for a file expires, and a new request for that file arrives, CloudFront contacts the origin to check for a newer version
2. If the file on the origin has not changed, the origin returns an HTTP status code 304 Not Modified.
3. CloudFront then resets the TTL and serves the existing cached file.
4. If the file on the origin has changed, the origin returns an HTTP status code 200 OK and sends the new version of the file. CloudFront replaces the old cached file with the new one, resets the TTL, and serves the new file.



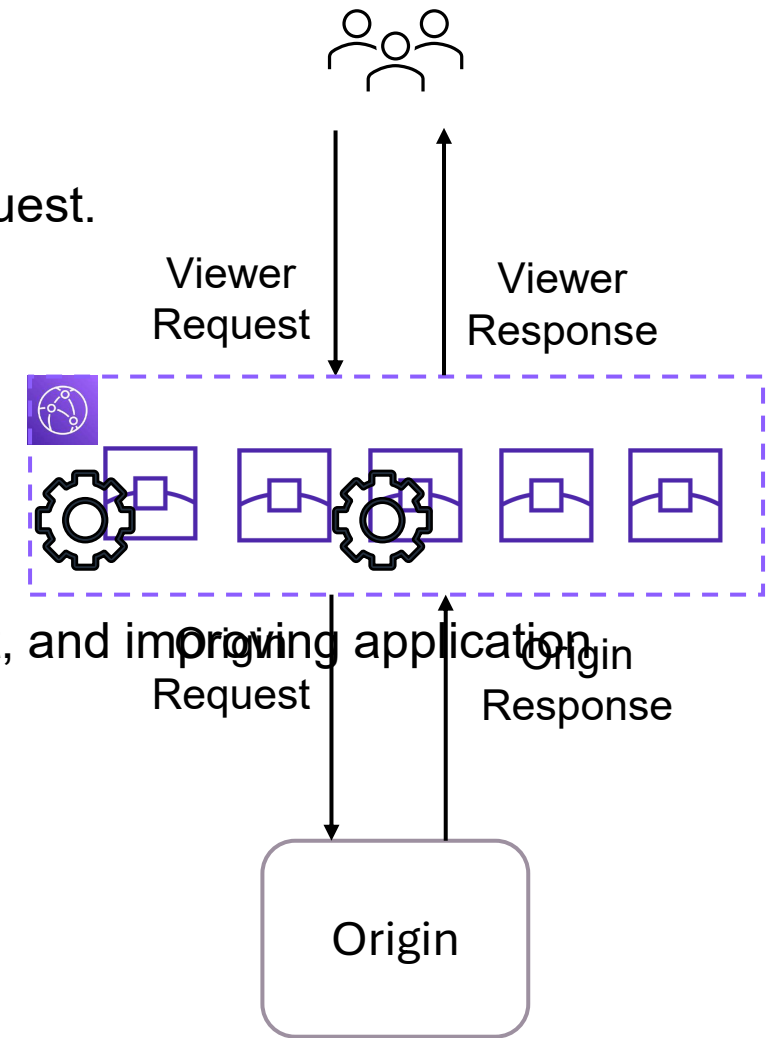
# Cache Behaviour – Cache Invalidation

- Used to remove objects from CloudFront caches before TTL expiry.
- After invalidation, CloudFront fetches the latest version from the origin.
- Supports:
  - Specific object: `/venice.png`
  - Wildcard patterns: `/images/*`
  - All objects: `*`
- First 1,000 invalidation paths per month are free 
- Invalidation affects all edge locations globally.
- Common use cases:
  - Removing sensitive / incorrect files immediately
  - Deploying new versions of static assets



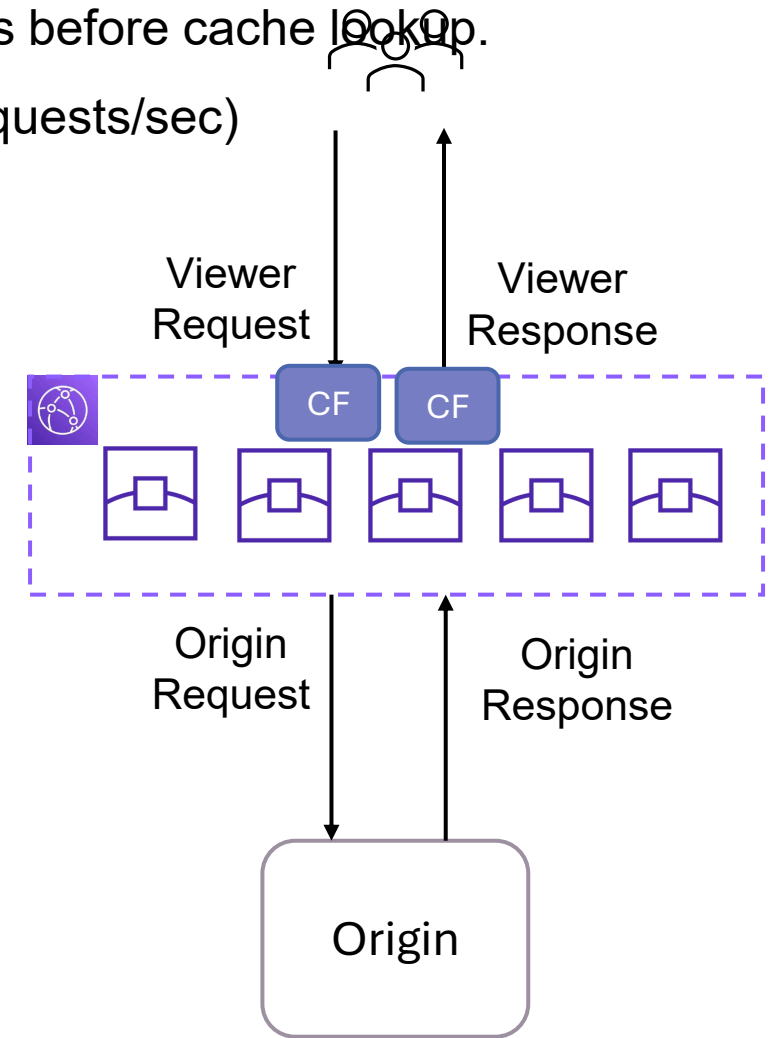
# CloudFront - Serverless edge compute

- Brings compute closer to users by running code edge locations
- Serverless - automatically handles millions of requests per second.
- Ideal for lightweight, fast-running logic that needs to run for every request.
- Can be applied at:
  - Viewer Request
  - Origin Request
  - Origin Response
  - Viewer Response
- Offloads work from origins, reducing backend load, infrastructure cost, and improving application performance.
- CloudFront offers: **CloudFront Functions** and **Lambda@Edge**.



# CloudFront Functions

- Lightweight JavaScript functions that run at CloudFront edge locations before cache look up.
- Designed for ultra-low latency, massively scalable (tens of millions requests/sec)
- CloudFront Functions support two event triggers:
  - Viewer Request – before CloudFront checks the cache.
  - Viewer Response – before CloudFront returns the response to the viewer.
- Use cases – URL rewrites/redirect, Header manipulation
- Limitations
  - Supports code only in Java Script
  - **No external network calls / APIs**
  - No file system access, no environment variables
  - **Execution time limit < 1ms**
  - Can not modify response body (Only headers/URI/status).



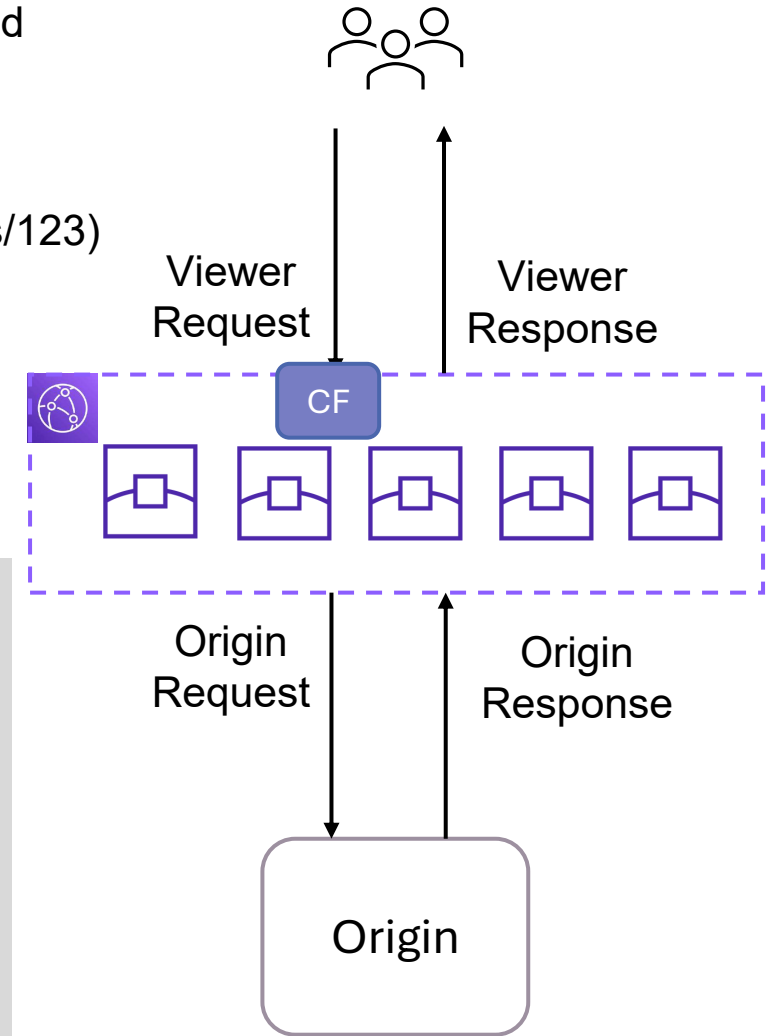
# Example - CloudFront Functions

**Use case:** Single-page apps (SPA) written in React/Vue etc. where every path should serve index.html

## How it works?

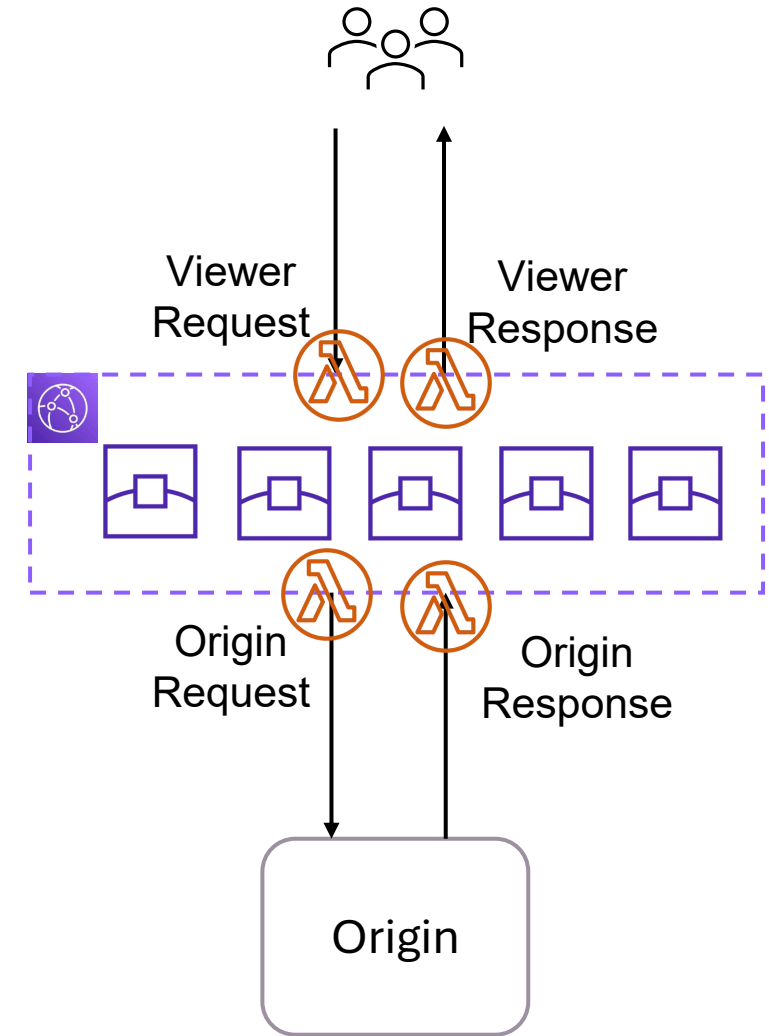
- On Viewer Request, rewrite requests that look like file-less routes (e.g., /products/123) to /index.html so the SPA can handle client-side routing.
- Attached to Viewer Request.
- Sample Java Script CloudFront Function:

```
function handler(event) {  
  var request = event.request;  
  var uri = request.uri;  
  // if URI looks like a file (has a dot) let it pass  
  if (uri.indexOf('.') !== -1) return request;  
  // if root or a route, rewrite to index.html  
  request.uri = '/index.html';  
  return request;  
}
```



# CloudFront - Lambda@Edge

- Run Lambda functions at AWS CloudFront edge locations globally.
- Allows you to adjust data between the Viewer & Origin.
- Lambda@Edge supports all four event triggers.
- Currently supports Node.js and Python.
- Function needs to be authored in N. Virginia (us-east-1) region
- **Use cases:**
  - Authentication and authorization checks by invoking external APIs
  - A/B testing, device-based or location-based routing.
  - Generate custom responses without calling the origin e.g. Site under maintenance
  - Generating thumbnails or specific resolution of the image

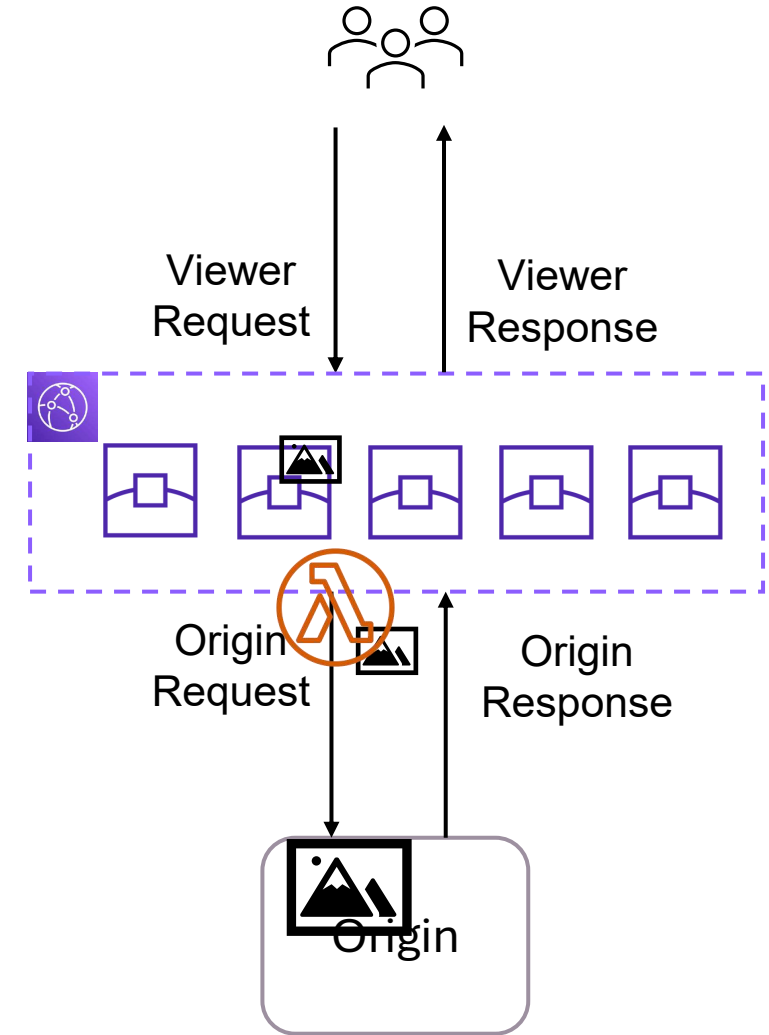


# Example – Lambda@Edge

**Use case:** Image resizing

**How it works?**

- User requests `/images/photo.jpg?w=300&h=200`
- CloudFront checks cache and if a cache miss, Lambda@Edge at Origin Request runs:
  - It fetches the original image from the origin
  - Run the code to generates the resized image
  - CloudFront caches the resized object.
- Future requests for the same size are served directly from edge cache.





# CloudFront Functions vs Lambda@Edge

|                       | CloudFront Functions                                                                                 | Lambda@Edge                                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Trigger Events        | Viewer Request, Viewer Response                                                                      | Viewer Request, Viewer Response, Origin Request, Origin Response                                                               |
| Execution Time        | < 1ms                                                                                                | Typically, 10-50ms but can be more                                                                                             |
| Memory / Package size | Memory max2MB<br>Package size10 KB                                                                   | Memory - 128MB to 10GB<br>Package size – Up to 50MB                                                                            |
| Runtime               | Java script only                                                                                     | Node.js and Python                                                                                                             |
| Complexity            | Very small logic; no external libraries; no network calls, no file system, no environment variables  | Supports larger logic; can use libraries; can call external systems, ephemeral storage (/tmp) 512MB, environment variables     |
| Performance           | Ultra-low latency                                                                                    | Moderate latency                                                                                                               |
| Cost                  | Very low                                                                                             | Higher as compared to CF                                                                                                       |
| Use cases             | Lightweight request/response modification, URL rewrites, redirects, header edits, simple auth checks | More complex logic, dynamic responses, origin routing, authentication, calling external APIs, custom responses, HTML rewriting |



# CloudFront - Alternate Domain Names

- You can use your own domain name(s) instead of the domain assigned by CloudFront to your distribution
- Example:
  - <https://www.example.com/sample.jpg>
  - <https://api.example.com/profile/customer/101>
  - <https://myapp.com/>
- Wildcards can also be used in the Domain Names (e.g., \*.example.com)
- Must have a valid SSL/TLS Certificate for
  - Your domain name
  - All Alternate Domain Names you added to your CloudFront distribution
- Certificate must be available in **US-EAST-1 (N. Virginia)** region

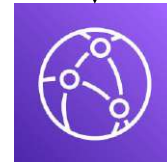
www.example.com



DNS



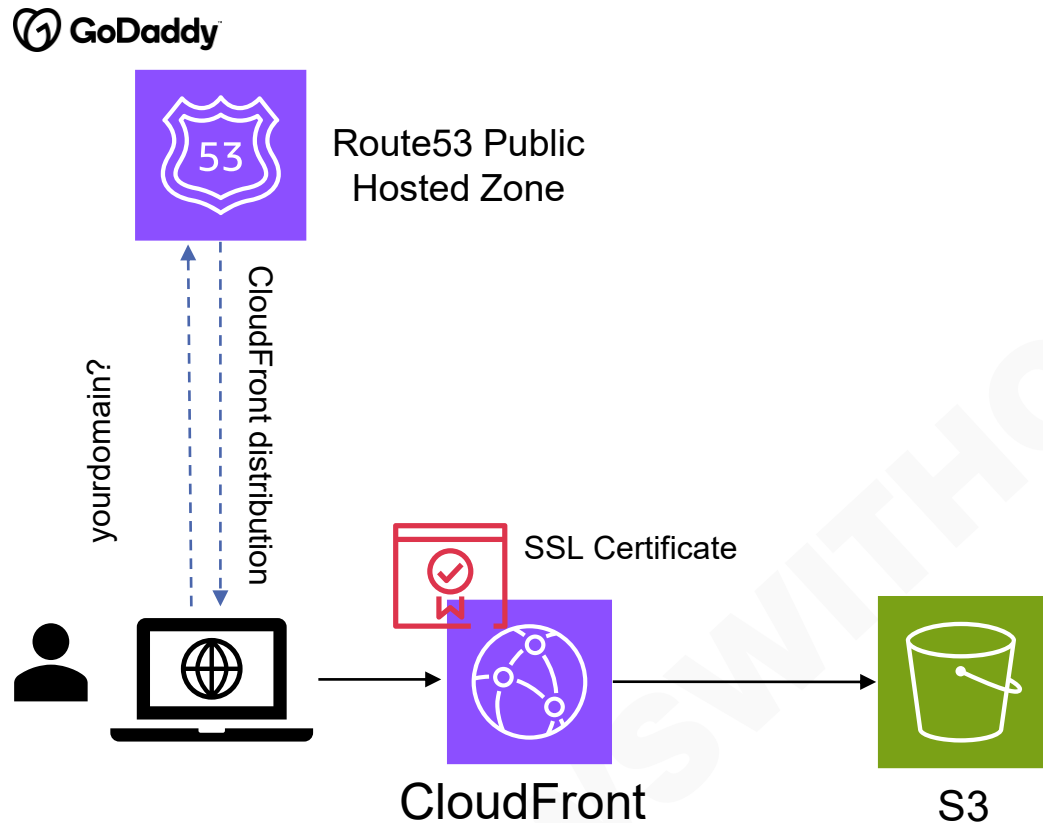
SSL/TLS  
Certificate



CloudFront

\*.example.com  
www.example.com

# Exercise - HTTPS static website using S3 and CloudFront



## Pre-requisites:

1. Should have your own domain name
2. Should have S3 bucket configured as a static website
- 1 Create TLS certificate for your domain name and www subdomain using Amazon Certificate Manager (ACM) in **N. Virginia region**
- 2 Create CloudFront distribution with S3 as an origin. S3 policy to allow OAC. Set viewer protocol policy to redirect HTTP to HTTPS. Set the alternate domain names (CNAME) for your www subdomain.
- 3 Update Route53 record for www to point to CloudFront distribution
- 4 Access website using your domain name. It should redirect to <https://www.domainname> and website should be accessible

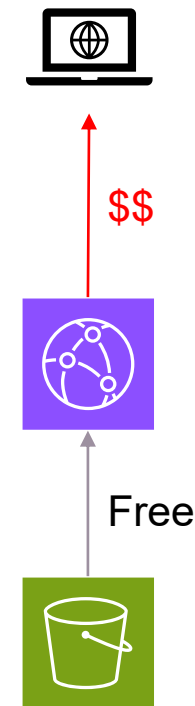
# Exercise resources

- Download the sample website content from the resource section
- Use this bucket policy to make the bucket public. Replace bucket name.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ForStaticWebsite",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::bucket-name/*"
      ]
    }
  ]
}
```

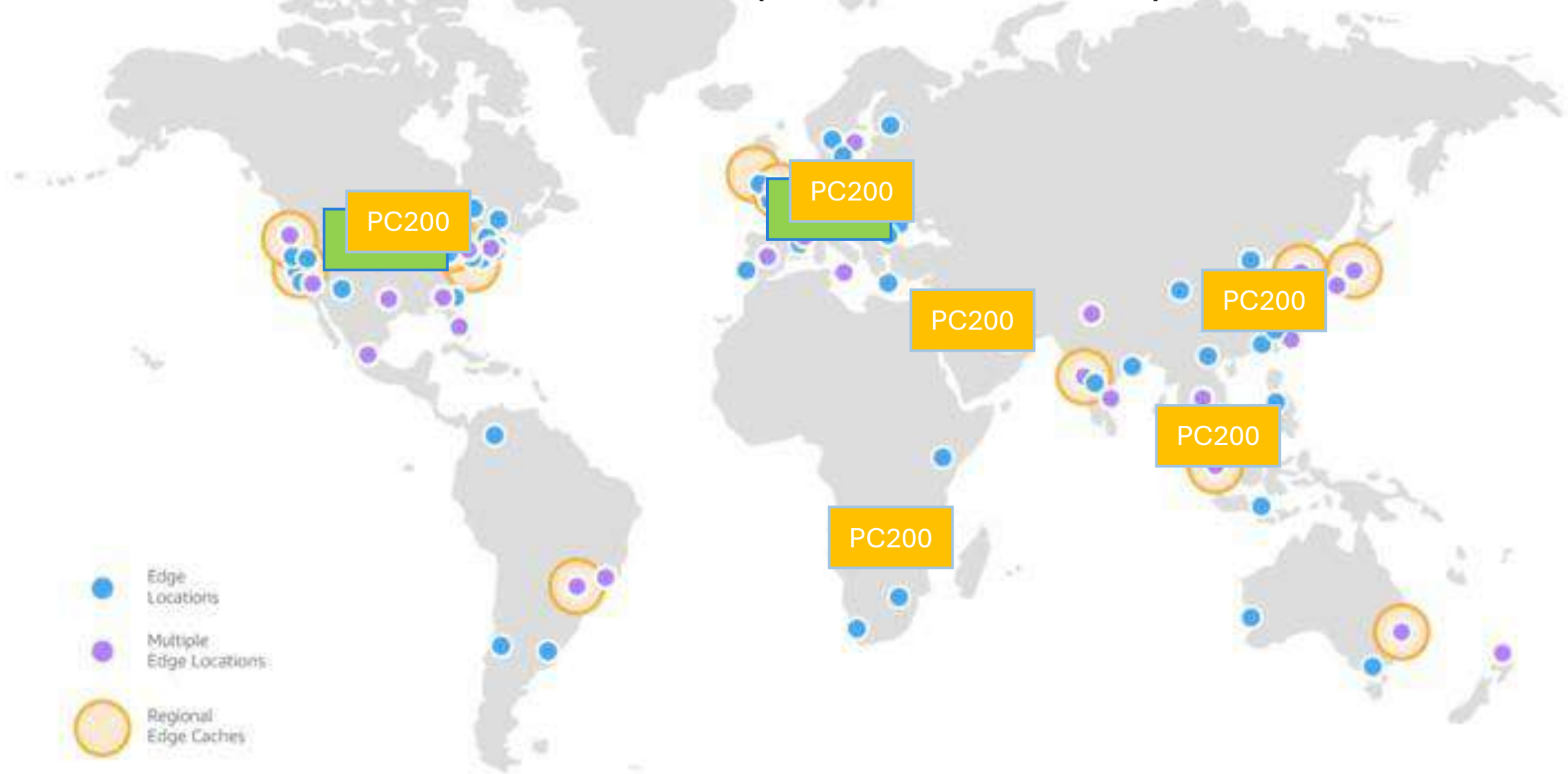
# CloudFront Pricing (Until Nov 2025)

- CloudFront pricing is mainly based on Data Transfer Out (DTO) + Request count.
- Data transfer from AWS origins to CloudFront is free.
- Additional optional charges: Cache invalidations, Lambda@Edge, Logs etc.
- Price Classes let you reduce cost by limiting which edge locations CloudFront uses:
  - Price Class 100 = Cheapest
  - Price Class 200 = Balanced
  - Price Class All = Best performance.
- Performance may reduce if edge locations are restricted.
- Default is Price Class **All** unless you change it.



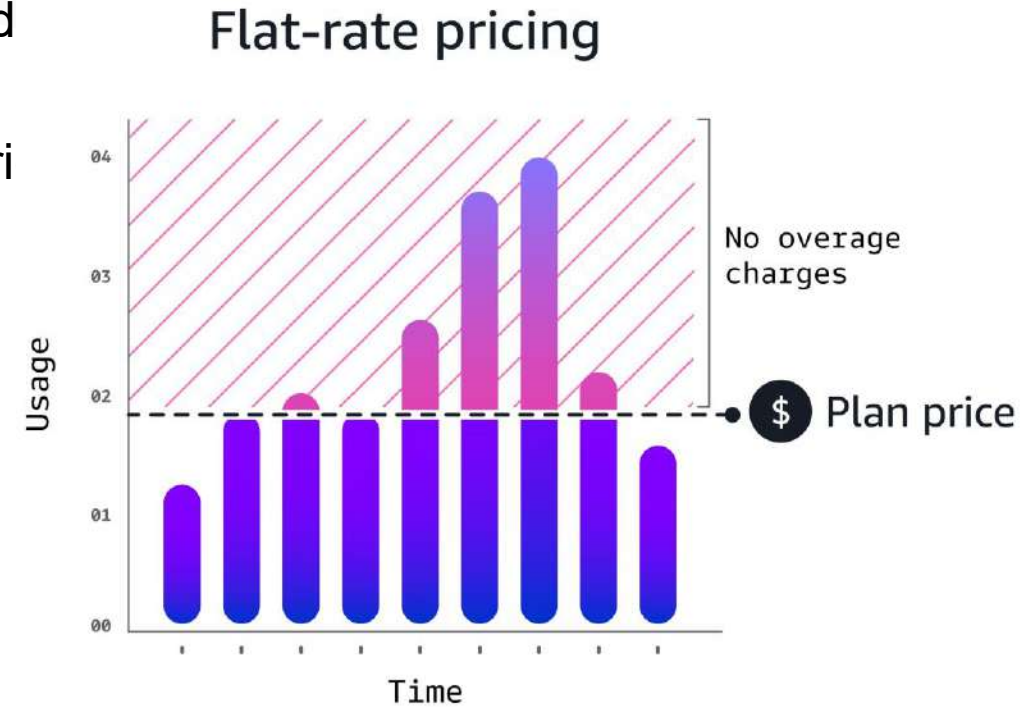
| Price Class            | What it Means                                   | Usage                                 |
|------------------------|-------------------------------------------------|---------------------------------------|
| <b>Price Class All</b> | Uses all edge locations worldwide               | Best performance, highest cost        |
| <b>Price Class 200</b> | Uses most regions, excludes costliest           | Balanced performance & cost           |
| <b>Price Class 100</b> | Uses only cheapest regions (US, Canada, Europe) | Lowest cost, lower global performance |

# CloudFront Price Classes (Until Nov 2025)



# CloudFront Pricing (From Nov 2025)

- **New Pricing:** Flat-rate pricing (fixed monthly CloudFront cost with no overages.)
- CloudFront flat-rate pricing bundles CDN usage and related with no overage charges.
- Flat-rate pricing plans include the following for a monthly price
  - CloudFront CDN
  - AWS WAF and DDoS protection
  - Bot management and analytics
  - Amazon Route 53 DNS
  - Amazon CloudWatch Logs ingestion
  - TLS certificate
  - Serverless edge compute
  - Amazon S3 storage credits each month
- Start with the \$0/month Free plan and upgrade to access more capabilities and larger usage allowances.



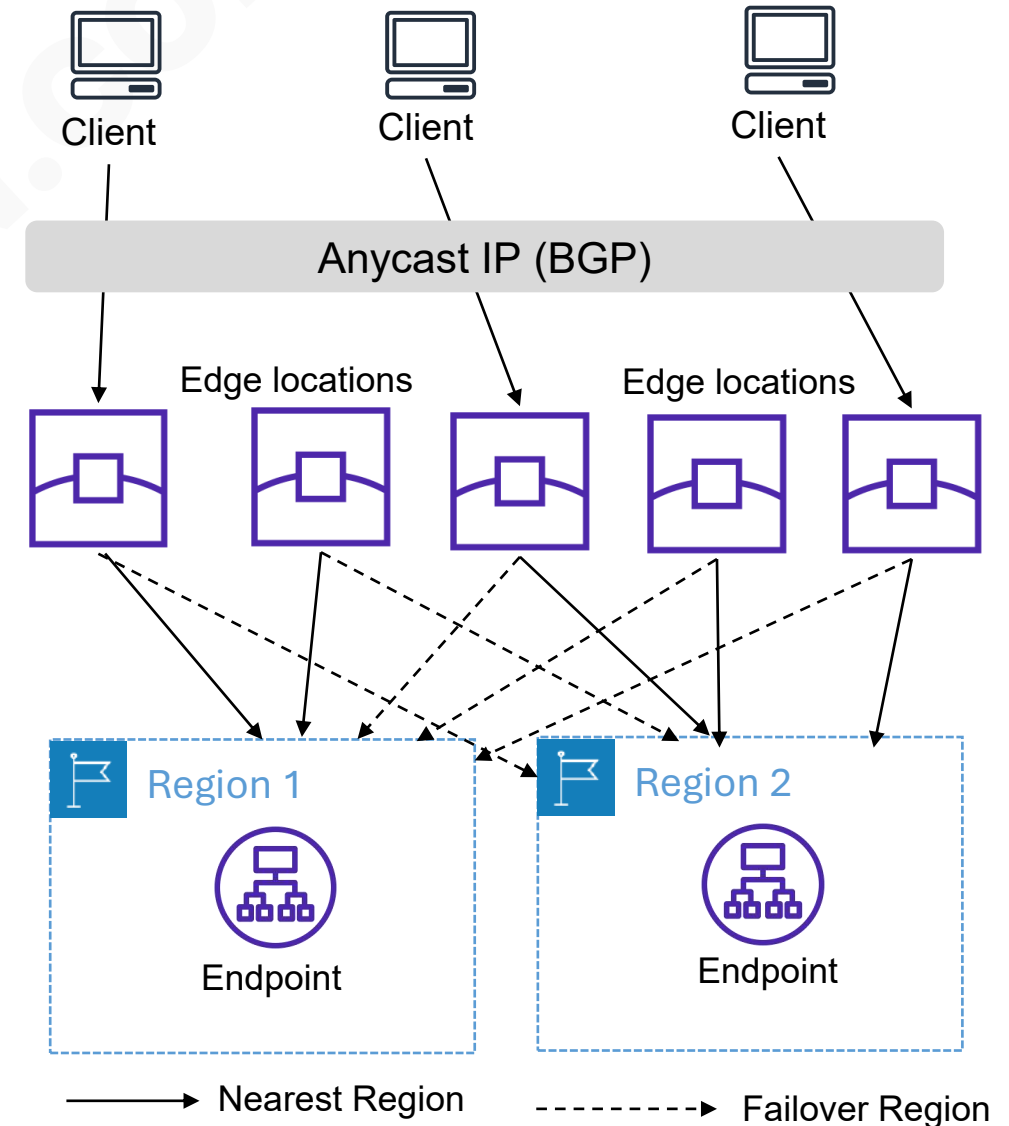


# AWS Global Accelerator



# AWS Global Accelerator

- Similar to Amazon CloudFront, AWS Global Accelerator also uses AWS edge locations
- Supports endpoints as Application Load Balancer, Network Load Balancer, EC2 instances and Elastic IPs
- **Allocates 2 Anycast IPs**
- Anycast IPs send traffic directly to Edge Locations
- It's possible to whitelist these IP addresses on client side
- Supports **TCP/UDP** protocols not just HTTP/S thereby suitable for VoIP, Gaming, Streaming, IoT applications
- Performs **health checks** and automatic failover across Regions.
- Improves availability across multi-Region architectures.
- Built-in DDoS protection with AWS Shield



# CloudFront vs Global Accelerator

|                  | CloudFront                                                                                                                 | Global Accelerator                                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Functionality    | Content Delivery which optimizes HTTP/HTTPS content delivery for both static and dynamic using Edge network and Edge Cache | Global Accelerator optimizes network routing for any TCP/UDP application, improving performance and availability by routing traffic to nearest and health endpoints |
| How to access    | DNS                                                                                                                        | 2 Static (Anycast) IPs                                                                                                                                              |
| Caching          | Yes (One of the primary feature)                                                                                           | No                                                                                                                                                                  |
| TCP/UDP          | No                                                                                                                         | Yes                                                                                                                                                                 |
| Failover Feature | Limited                                                                                                                    | Yes, performs health check                                                                                                                                          |
| Use cases        | Website, Media streaming, APIs                                                                                             | VoIP, Gaming, IoT (MQTT)                                                                                                                                            |
| DDoS Protection  | Yes, works with AWS Shield                                                                                                 | Yes, works with AWS Shield                                                                                                                                          |





# Amazon Route 53

A domain registration and DNS service

# In this section..

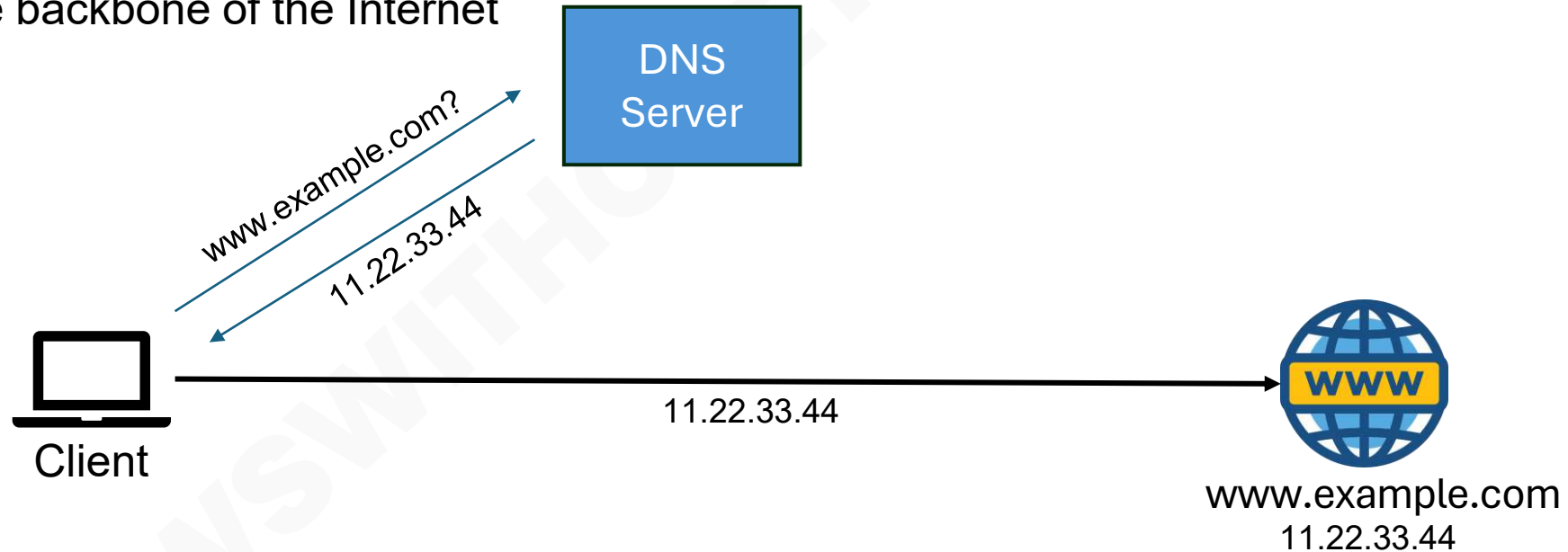
- DNS Basics
- Amazon Route 53
- Route 53 Hosted Zones
- Route 53 DNS Records types
- Route 53 Health Checks
- Route 53 Records TTL
- Route 53 Routing policies
- Route 53 Resolver endpoints and Hybrid DNS

+ *Hands-on exercises and demos*

# DNS Basics

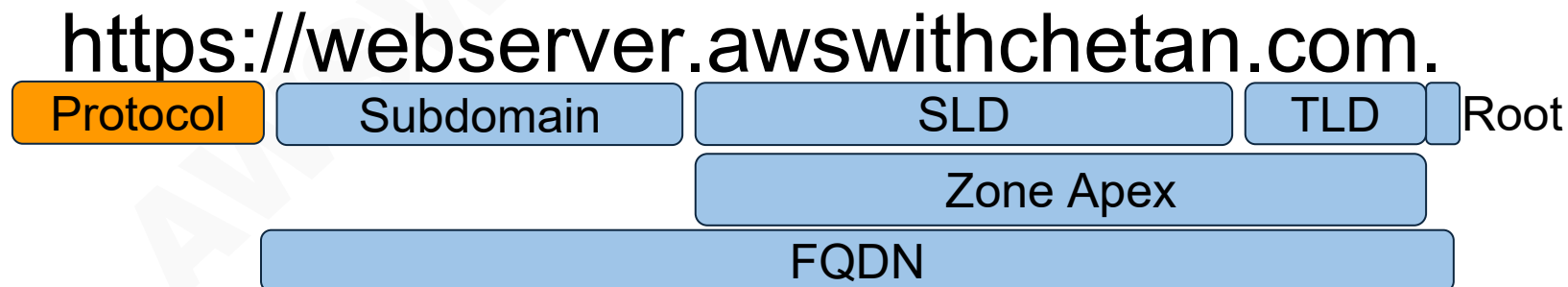
# What is DNS?

- Domain Name System which translates the human friendly hostnames into the machine IP addresses such as `www.google.com` => `142.250.205.46`
- DNS is the backbone of the Internet



# DNS Components and terminologies

- **Domain Registrar** - A company authorized to register and manage domain names on the internet. Ex. Amazon Route 53, Namecheap, GoDaddy, Domain.com etc.
- **Top Level Domains (TLD)** - .com, .us, .gov
- **Zone Apex** – Main domain e.g. amazon.com, example.com, awswithchetan.com
- **Subdomain or Second Level Domains (SLD)** – api.amazon.com, www.awswithchetan.com
- **Fully Qualified Domain Names (FQDN)** - FQDN uniquely identifies one specific host on the internet
- **Name Servers** – Answers DNS queries by looking up records stored in the zone file



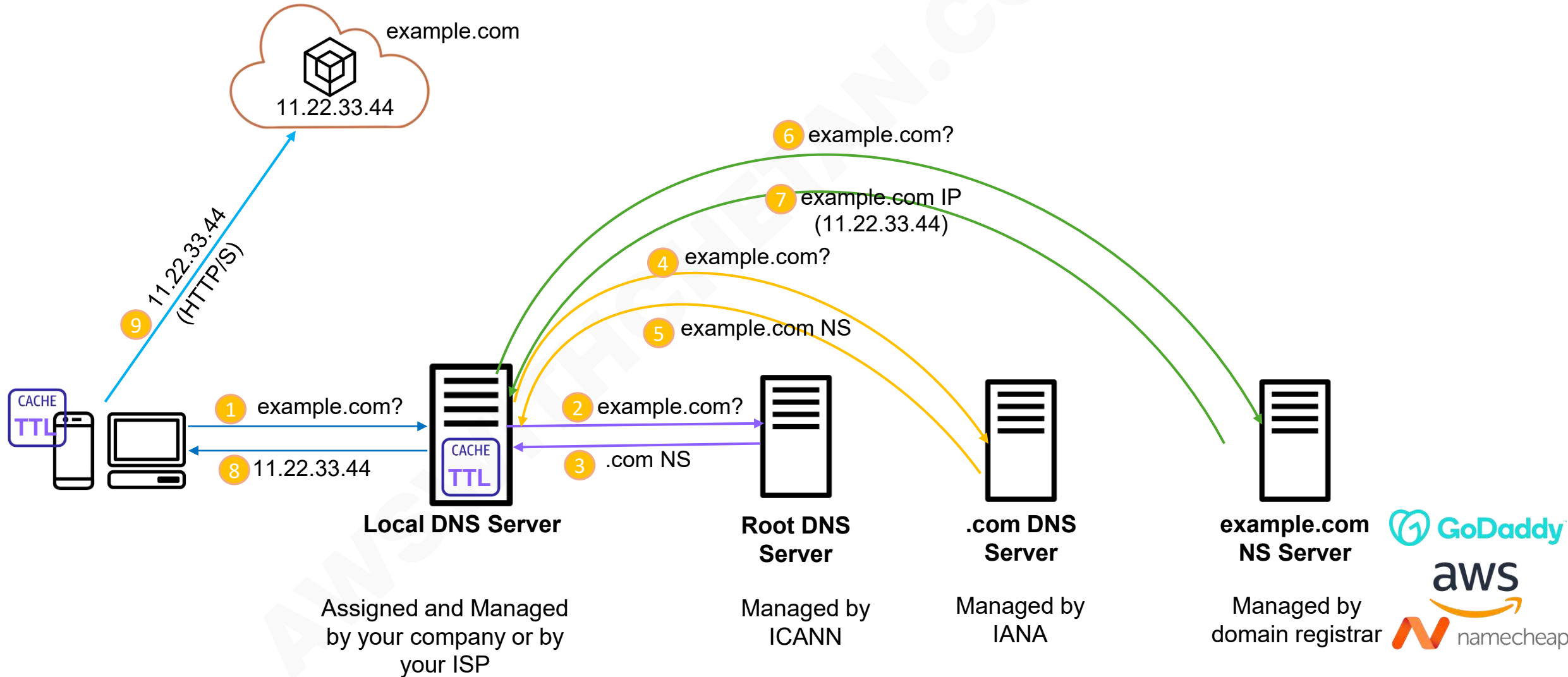
# DNS Components and terminologies

- **Domain Registrar** - A company authorized to register and manage domain names on the internet. Ex. Amazon Route 53, Namecheap, GoDaddy, Domain.com etc.
- **Top Level Domains (TLD)** - .com, .us, .gov
- **Zone Apex** – Main domain e.g. amazon.com, example.com, awswithchetan.com
- **Subdomain or Second Level Domains (SLD)** – api.amazon.com, www.awswithchetan.com
- **Fully Qualified Domain Names (FQDN)** - FQDN uniquely identifies one specific host on the internet
- **Name Servers** – Answers DNS queries by looking up records stored in the zone file
- **Zone Files** – Text file containing DNS records (A, AAAA, CNAME etc.)

```
example.com.    IN  SOA ns1.example.com. admin.example.com. ...
example.com.    IN  NS  ns1.example.com.
example.com.    IN  NS  ns2.example.com.
www             IN  A   192.0.2.10
mail           IN  MX  10 mail.example.com.
```



# How DNS resolution works?



# DNS Record types

- A – Maps a domain to an IPv4 address
- AAAA – Maps a domain to an IPv6 address
- CNAME – Maps a domain to another domain name
- MX – Mail exchange servers for a domain
- TXT – Stores text data (SPF, DKIM, verification, etc.)
- NS – Delegates a DNS zone to name servers
- SOA – Start of authority for the hosted zone
- SRV – Defines service location (protocol, port)
- PTR – Reverse DNS (IP → domain)
- CAA – Defines which CAs can issue certificates
- and more..

example.com → 11.22.33.44

example.com → 2001:db8:abcd:1234::1

www.example.com → example.com

example.com → mail.example.com (priority 10)

example.com → "v=spf1 include:\_spf.google.com ~all"

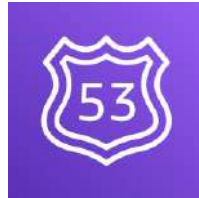
example.com → ns1.awsdns-12.com

example.com → Primary NS: ns1.awsdns-12.com,  
Admin: admin.example.com

\_chat.\_tcp.example.com → target  
chatserver.example.com, port 5222

44.33.22.11.in-addr.arpa → example.com

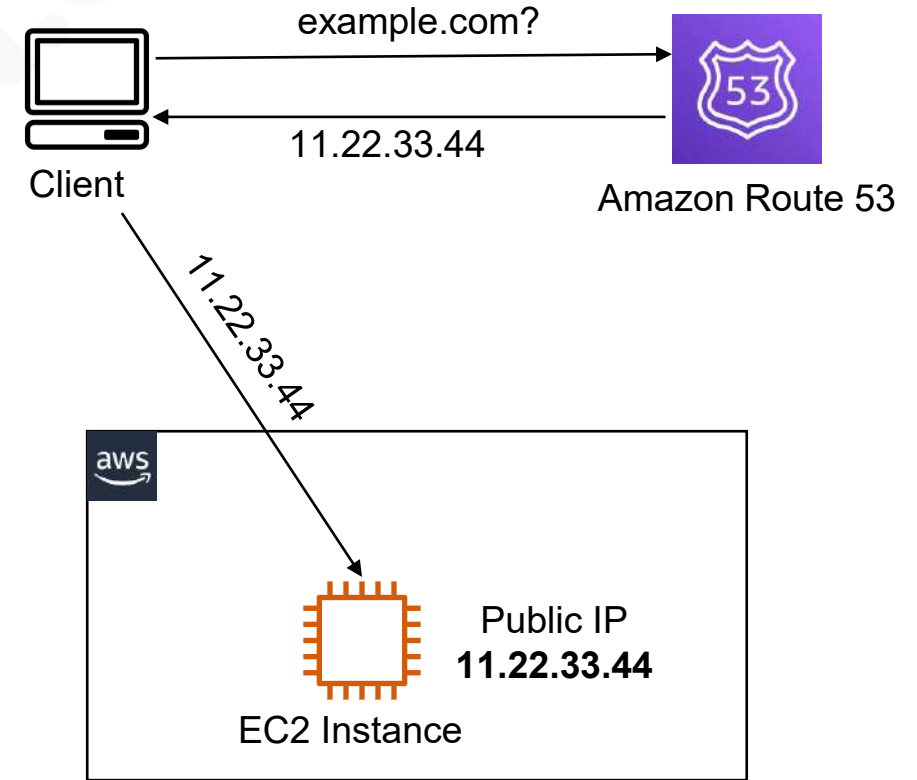
example.com → issue "letsencrypt.org"



# Amazon Route 53

# Amazon Route 53

- A fully managed global DNS service by AWS
- Route 53 is also a Domain Registrar
- The only AWS service which provides 100% availability SLA



Why 53 in Route 53?

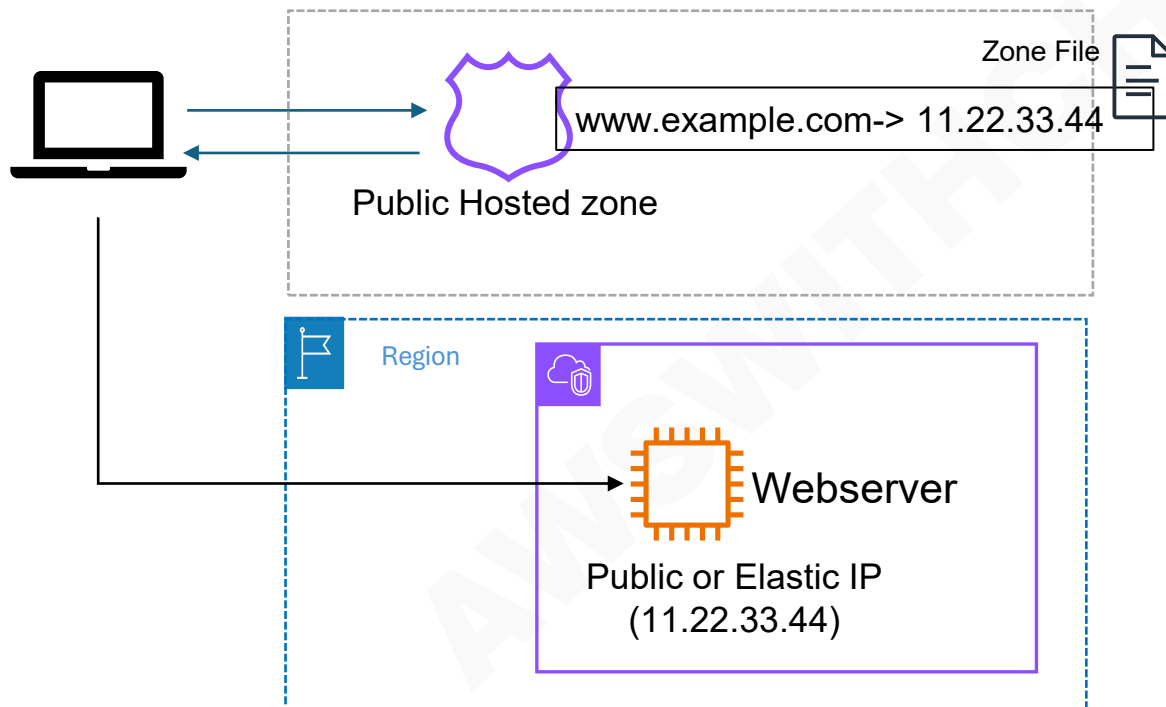
53 is a traditional DNS port



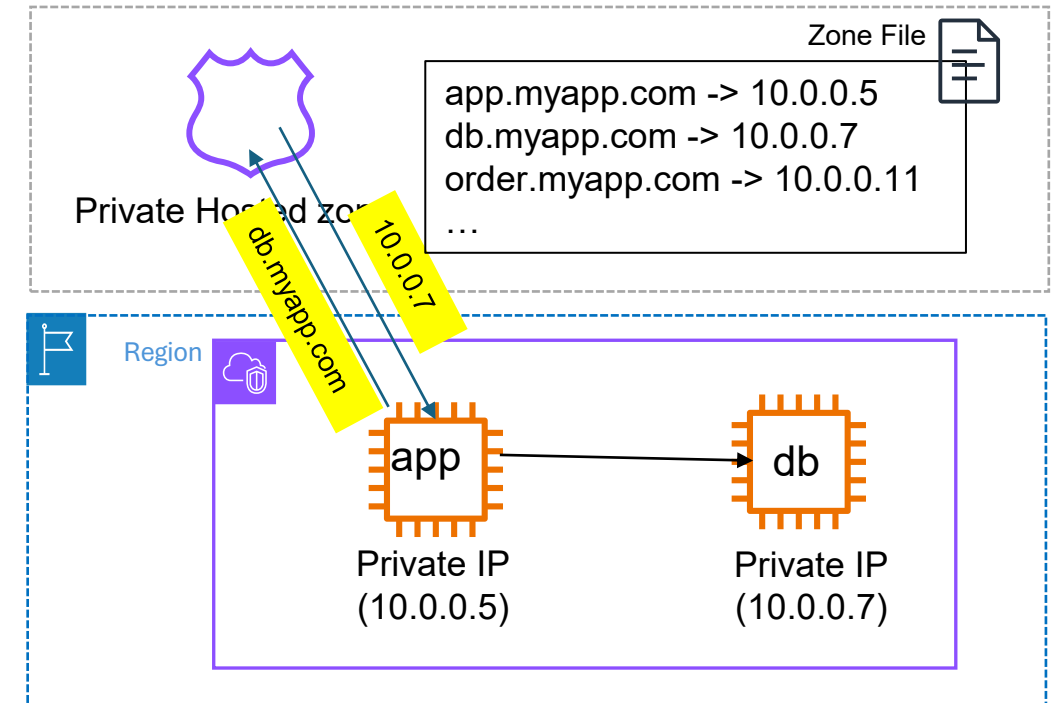
# Route 53 Hosted Zones

- Public Hosted Zone – For public domain names. DNS resolution over the internet.
- Private Hosted Zone – For private domain names. DNS resolution within the VPC.
- Zone file contains records to point domain name to target IP address or to another domain name.

## Public Hosted Zone

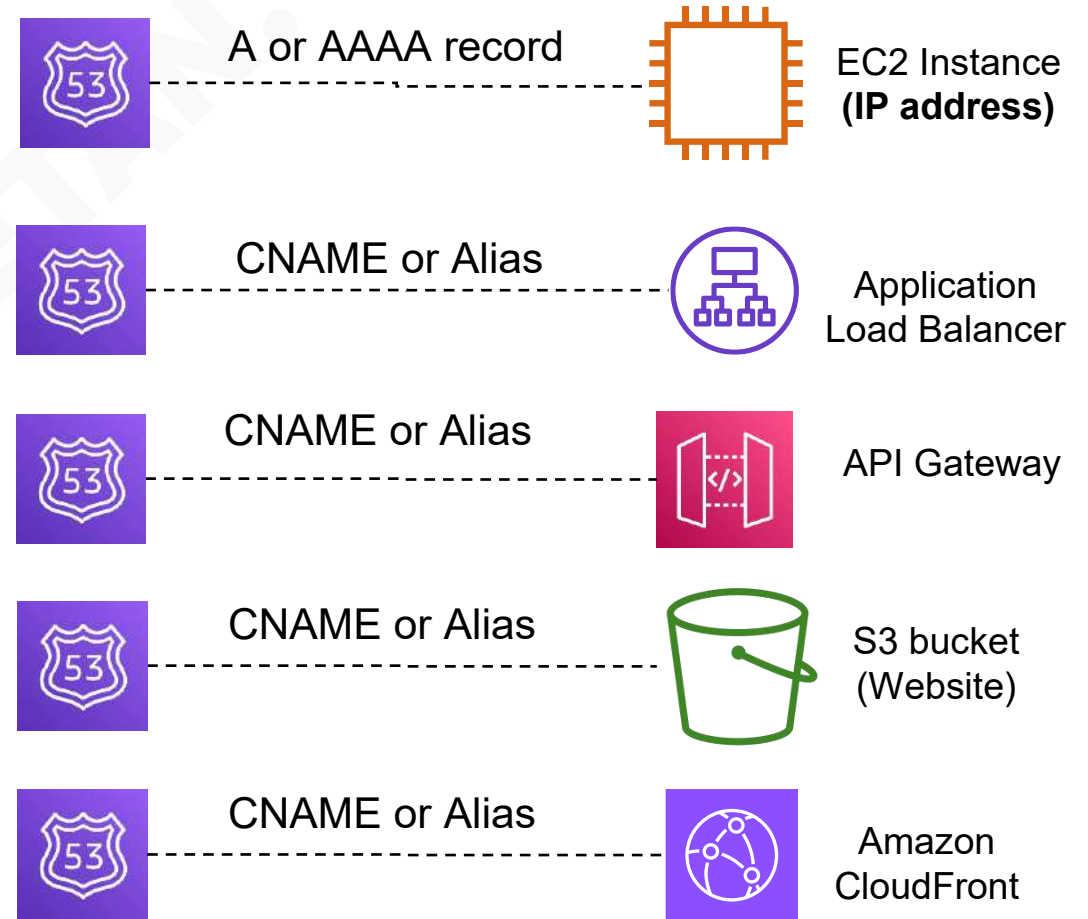


## Private Hosted Zone



# Route53 DNS Records

- Use A record to point domain name to IPv4 address
- Use AAAA record to point domain name to IPv6 address
- Use CNAME record to point a sub-domain to another domain name
- Use Alias record to point the apex domain or sub-domain to AWS provided DNS for AWS services e.g. ALB, CloudFront distribution, S3 website, API gateway etc.

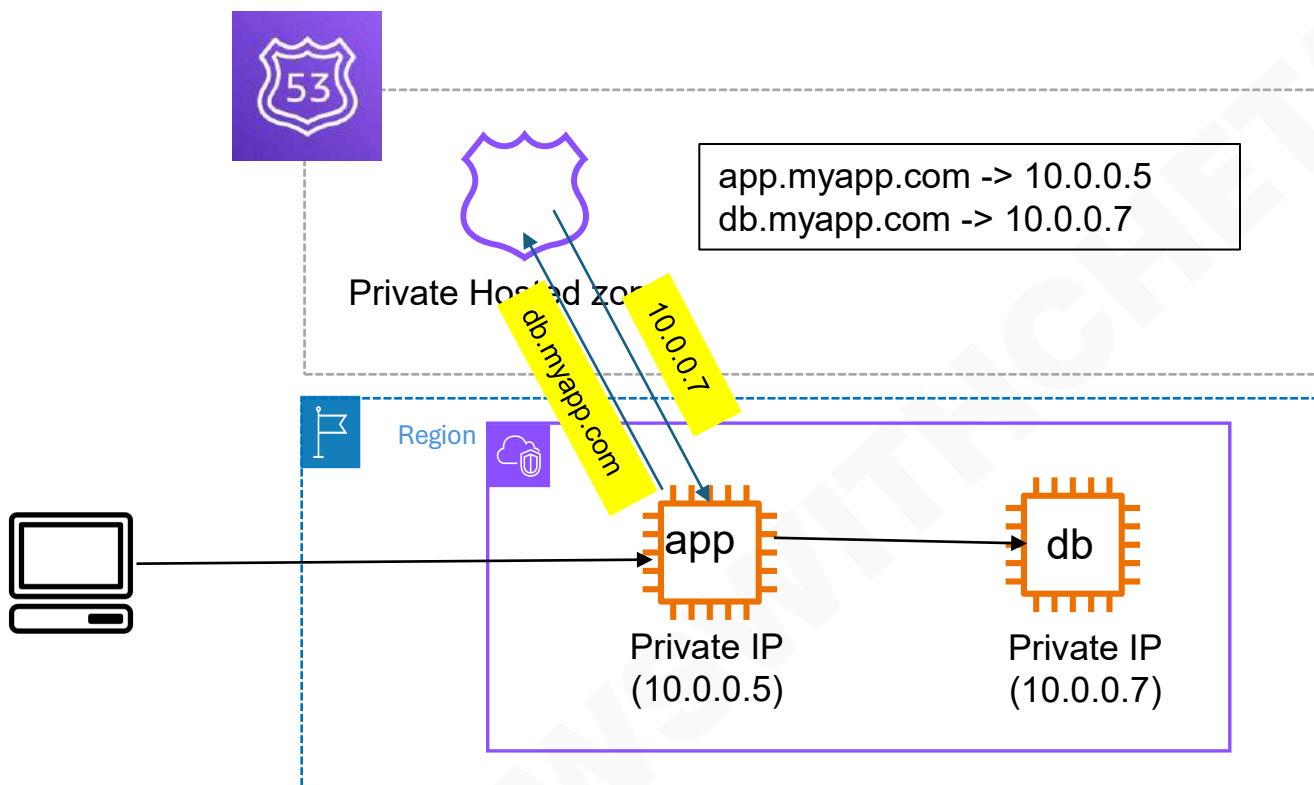


# CNAME vs ALIAS record

|                                      | CNAME                                                                             | ALIAS                                                                                                                                                                              |
|--------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is it?                          | Points hostname to another hostname                                               | Points hostname to AWS resource                                                                                                                                                    |
| Standard DNS record                  | Yes                                                                               | No – AWS Route 53 specific                                                                                                                                                         |
| Supported at zone apex (example.com) | No                                                                                | Yes                                                                                                                                                                                |
| DNS Query charges (Cost)             | Yes                                                                               | No                                                                                                                                                                                 |
| Auto detect IP change                | No – follows normal TTL                                                           | Yes – Do not wait for TTL expiry                                                                                                                                                   |
| Use when                             | <ul style="list-style-type: none"> <li>Target is outside AWS (any DNS)</li> </ul> | <ul style="list-style-type: none"> <li>Target is an AWS resource (CloudFront, ALB/NLB, S3 website, API gateway etc.)</li> <li>You need to create a record for zone-apex</li> </ul> |



# Exercise – Route 53 Private Hosted Zone

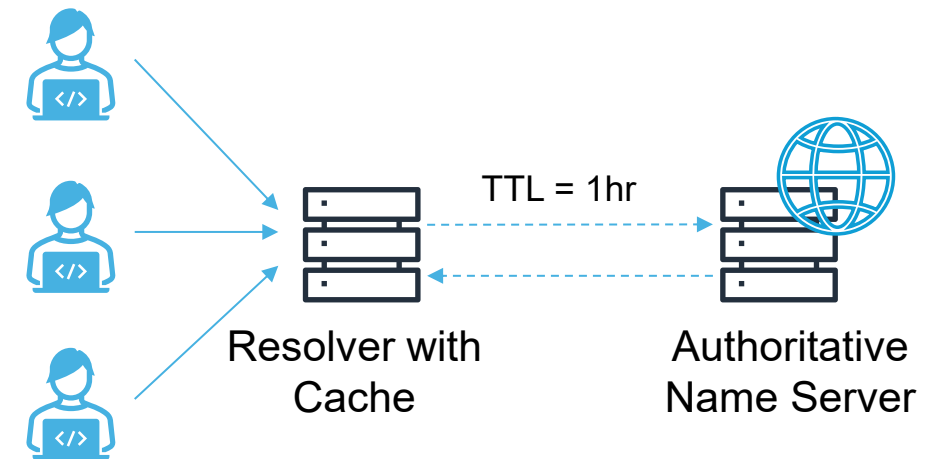


- 1 Launch 2 EC2 instances in default VPC. Allow SSH and Ping (ICMP) from anywhere in the security group
- 2 Create Route 53 Private Hosted Zone with some name say myapp.com
- 3 Create A records in hosted zone for app and db and point to EC2 instances private IPs.
- 4 Login to first (app) EC2 instance using EC2 instance connect and ping to db.myapp.com
- 5 DNS should resolve to db instance private IP address and ping should be successful.
- 6 Cleanup - Terminate both EC2 instances and delete Route 53 Private hosted zone.



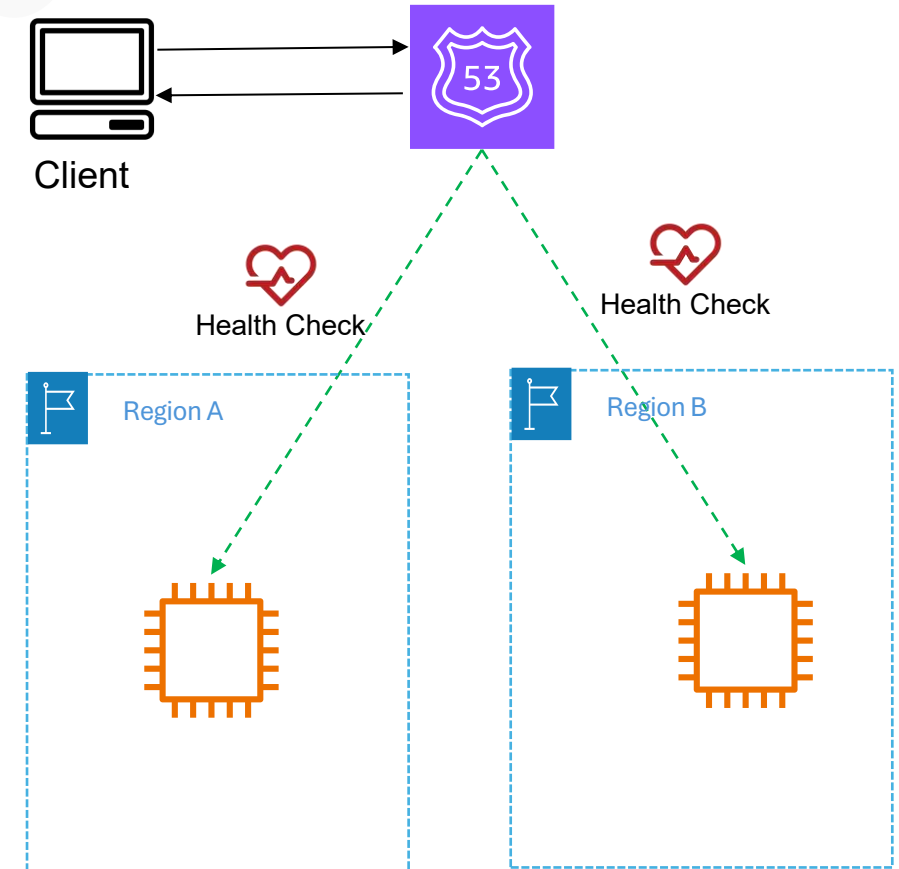
# Route 53 Record TTL (time to live)

- TTL is the number of seconds a DNS resolver (or client machine) will cache a DNS record before requesting a fresh copy from authoritative DNS/Name server
- Helps reduce repeated DNS lookups, improving performance and lowering Route 53 query costs.
- High TTL (e.g., 24 hours / 86400 sec):
  - Reduces DNS traffic and DNS query cost.
  - Ideal for static or rarely changing resources (stable IPs, static websites).
  - **Disadvantage:** Changes to DNS records propagate slowly because cached values remain active until TTL expires.
- Low TTL (e.g., 300 seconds):
  - Faster DNS updates and propagation.
  - Recommended when performing migrations, testing, or switching resources.
  - **Disadvantage:** More DNS queries = slightly higher latency and cost.
- Best Practice is to use low TTL during planned changes, then increase TTL after validation to optimize cost and performance.



# Route 53 - Health Checks

- Route 53 Health Checks monitor the availability and health of endpoints such as EC2 instances, ALBs, NLBs, CloudFront, or public IPs.
- Enables automated DNS failover using Route 53 routing policies by ensuring that traffic is routed only to healthy resources.
- Health checks can be performed using **HTTP, HTTPS, or TCP** protocols.
- Commonly used with Failover routing policy to enable DNS-based failover but also supported with other routing policies.
- Default interval: 30 seconds (Fast check: 10 seconds with extra cost).
- Default threshold: 3 consecutive failures/successes to mark unhealthy/healthy.
- Health check works for Publicly reachable endpoints; Private resources cannot be directly health-checked by Route 53.
- For Private resource health check, use CloudWatch alarm-based health checks instead.

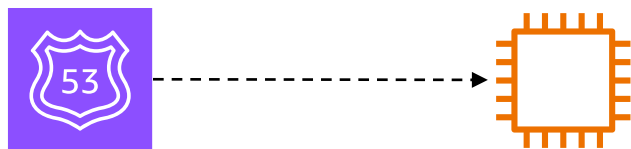


# Route 53 - Routing policies

- Defines how Route 53 should resolve the DNS queries, specially when there are multiple DNS records of the same hostname with different target values
- Supported Routing policies:
  - **Simple Routing** – Sends all traffic to a single resource.
  - **Failover Routing** – Switches traffic to a standby resource if the primary fails.
  - **Weighted Routing** – Splits traffic across resources based on assigned weights.
  - **Latency Routing** – Routes users to the region with the lowest network latency.
  - **Geolocation Routing** – Routes users based on their geographic location.
  - **Geoproximity Routing** – Routes based on resource location and allows traffic shifting between regions.
  - **IP-Based Routing** – Routes traffic using the client's source IP address.
  - **Multi-value Answer Routing** – Returns up to 8 healthy records randomly for DNS queries.

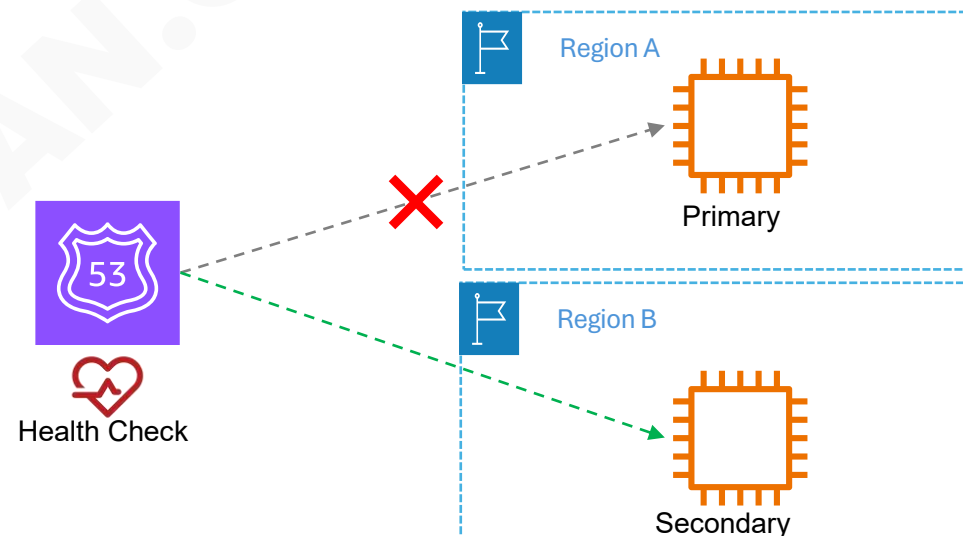
# Route 53 - Routing policies

## Simple Routing Policy



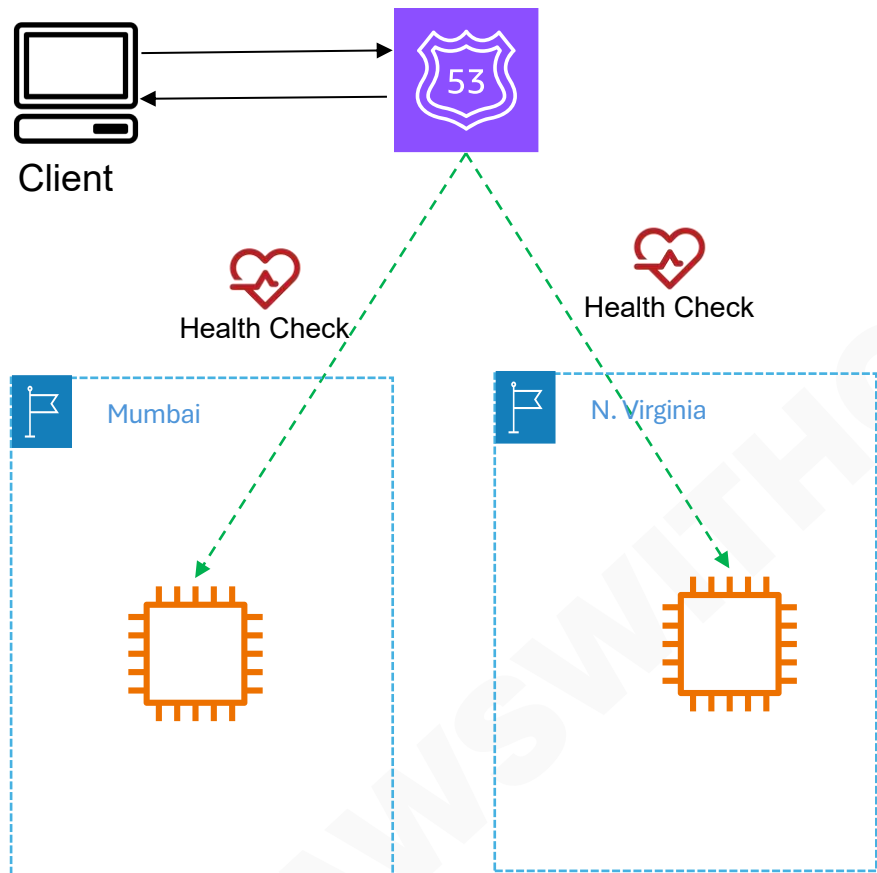
- Typically to route traffic to a single resource
- Optionally can specify multiple IPs and Route53 returns all the IPs in the random order
- Client to decide which IP to connect to.
- Health checks **not** supported, so all values are returned regardless of health.

## Failover Routing policy



- To achieve Active-passive failover (generally across AWS regions)
- Uses Health-checks
- You define Primary and Secondary (standby) records.
- Primary is returned only if its health check is healthy.
- If the Primary becomes unhealthy, returns the Secondary.
- Secondary should point to a backup system (DR site, another region, static maintenance page, etc.).

# Demo – Failover Routing policy

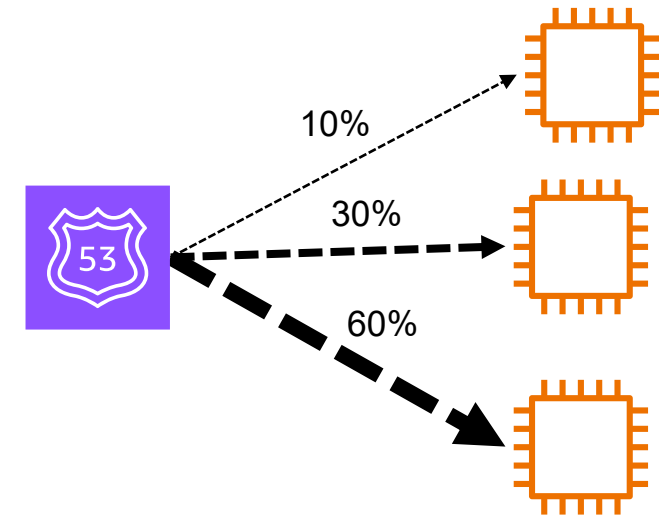


- 1 Launch 2 EC2 instances (Primary and Secondary) in different regions and configure them as a webserver. Allow HTTP from anywhere in SG.
- 2 Create Route 53 Public Hosted Zone for your domain name. (You should own the domain)
- 3 Create health check for the Primary webserver in Route 53
- 4 Create A record with Failover Routing policy pointing to Primary Public IP. Create another A record pointing to Secondary Public IP with Failover Routing policy (Secondary)
- 5 Access using you domain name. Always points to Primary.
- 6 Stop Primary EC2 instance to simulate failure. Wait for health-check to fail.
- 7 Access domain name, should point to Secondary.

# Route 53 - Routing policies

## Weighted Routing Policy

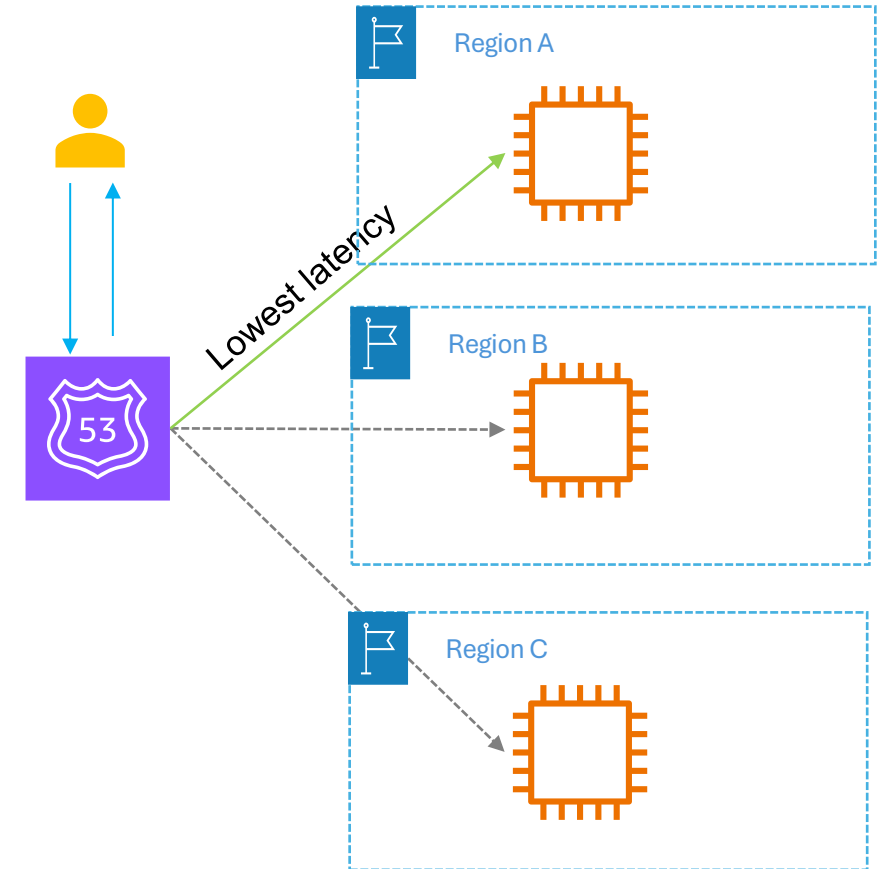
- You create multiple records with the same name and type with each record assigned a weight.
- Route 53 routes traffic proportionally based on those weights (e.g., 70/30, 50/50, 90/10).
- **Useful for load balancing, testing new versions, and gradual traffic shifting.**
- Supports health checks - unhealthy records are removed from responses.
- You can set a weight of 0 to exclude that target from routing



# Route 53 - Routing policies

## Latency Routing Policy

- Routes users to the AWS region that provides the lowest network latency for them.
- You create records with the same name but in different AWS regions.
- Uses latency measurements between AWS regions and the user's DNS resolver.
- Supports health checks - Route 53 avoids unhealthy endpoints.
- Ideal for latency sensitive global applications.



# Route 53 - Routing policies

## Geolocation Routing Policy

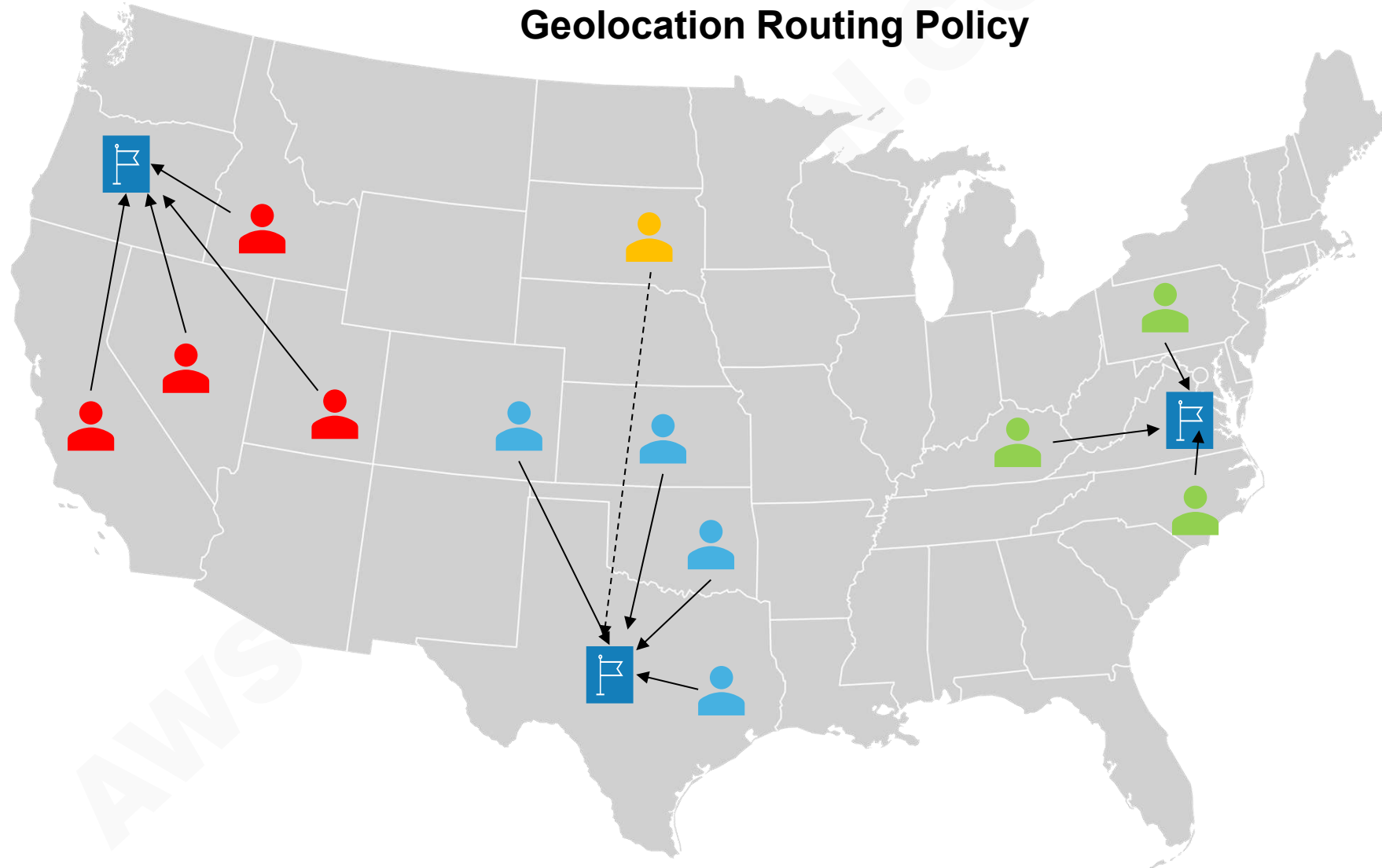
- Routes users based on their physical location (country, continent, or specific state).

## Geoproximity Routing policy

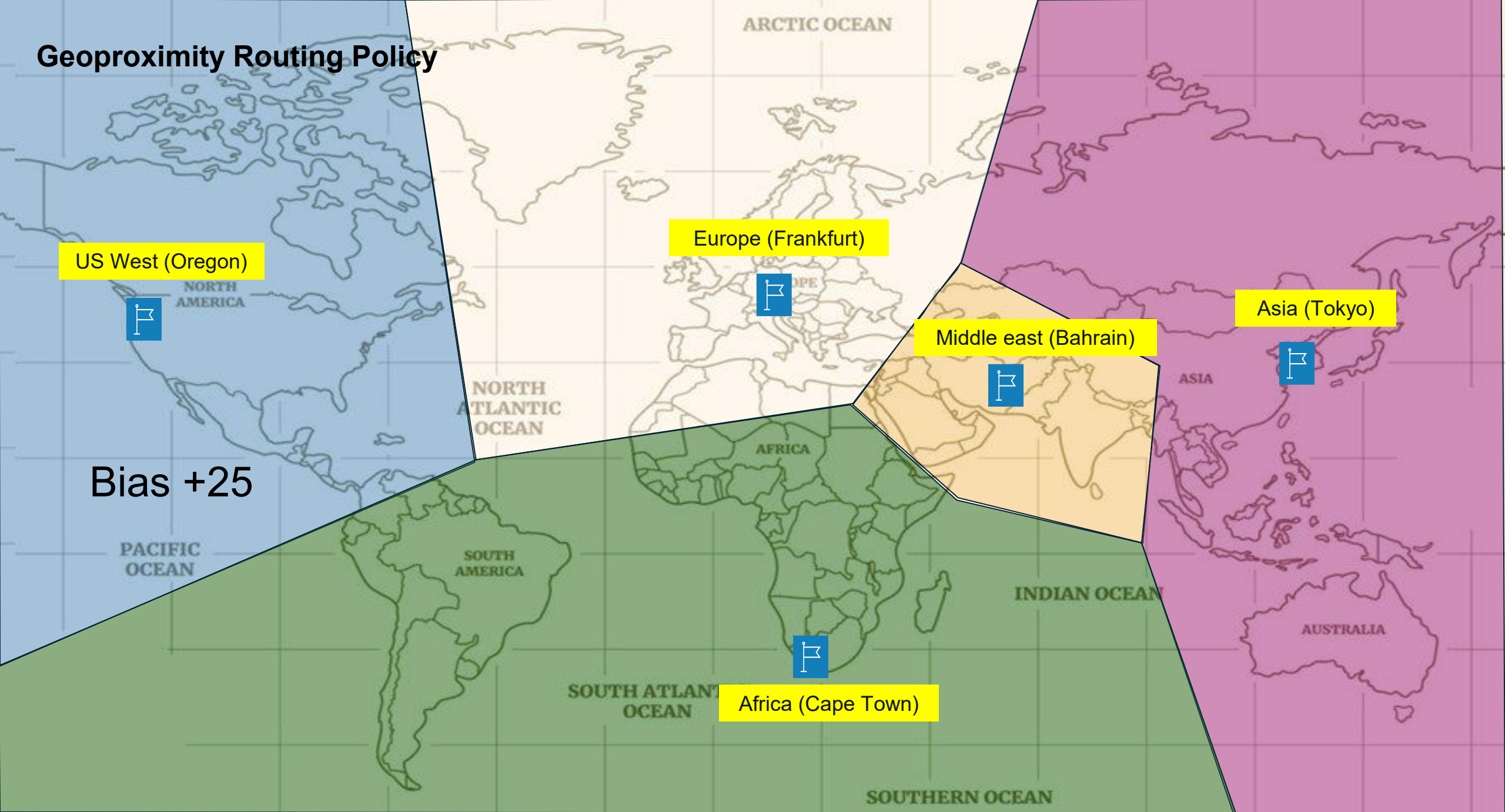
- Routes users based on geographic distance between the user and your AWS or on-prem endpoints.



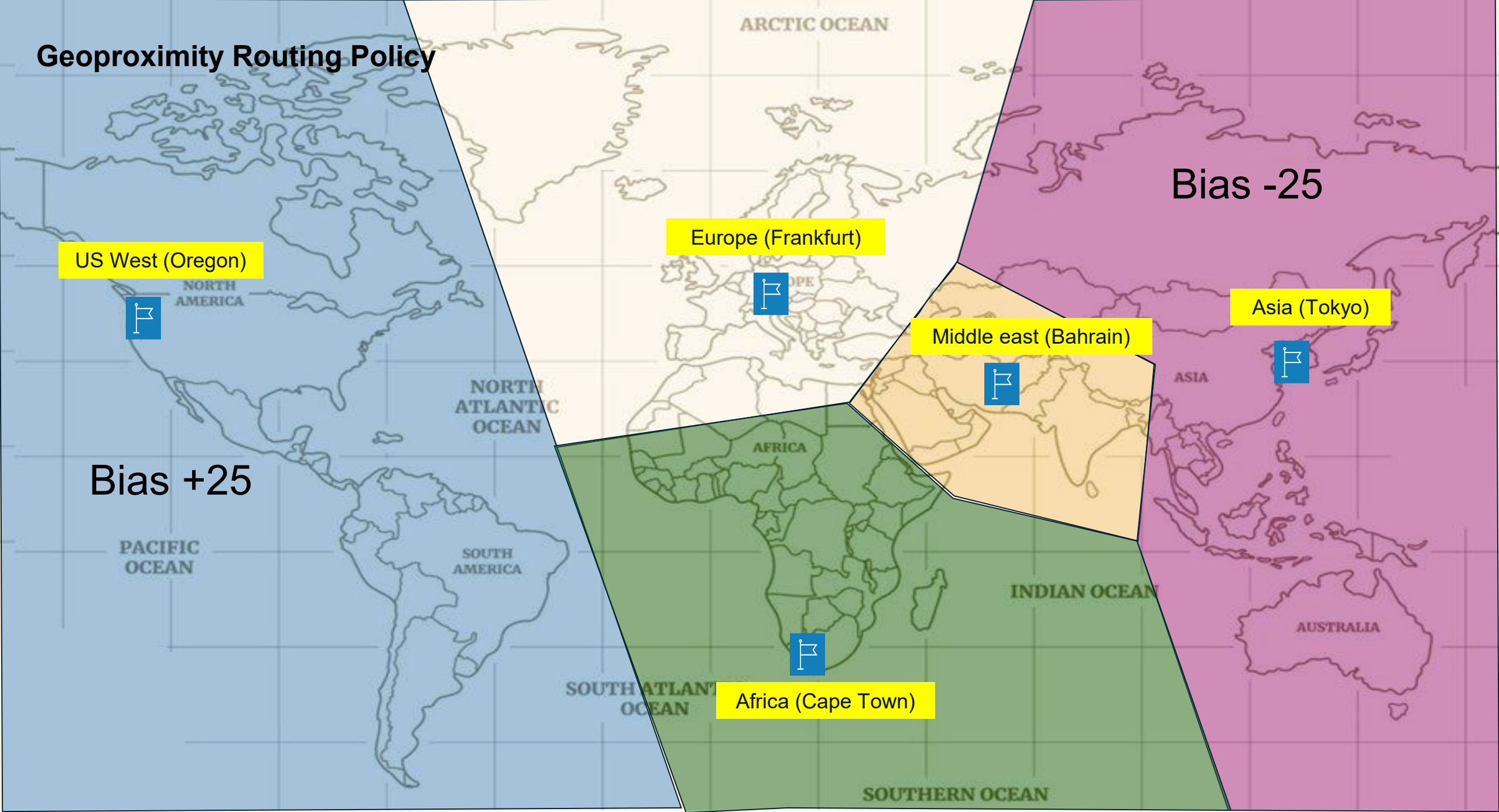
# Route 53 – Routing policies



# Geoproximity Routing Policy

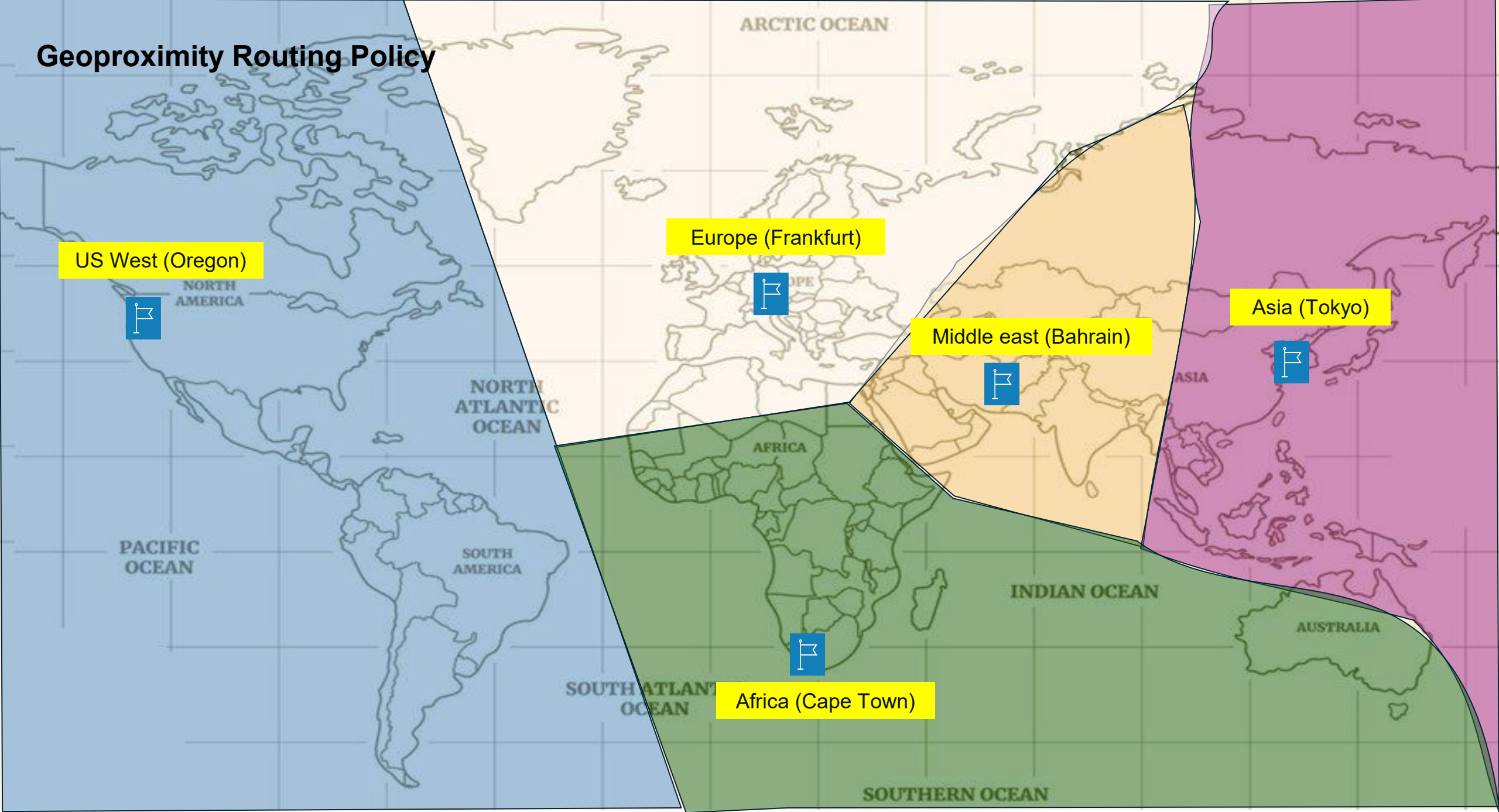


# Geoproximity Routing Policy





# Geoproximity Routing Policy



# Route 53 - Routing policies

## Geolocation Routing Policy

- Routes users based on their physical location (country, continent, or specific state).
- You explicitly assign which location goes to which endpoint.
- Strict mapping - if a user is from India, they will always go to the endpoint configured for India.
- Useful for compliance, content localization, or region-restricted applications.
- Must have a default record for unmatched locations.

## Geoproximity Routing policy

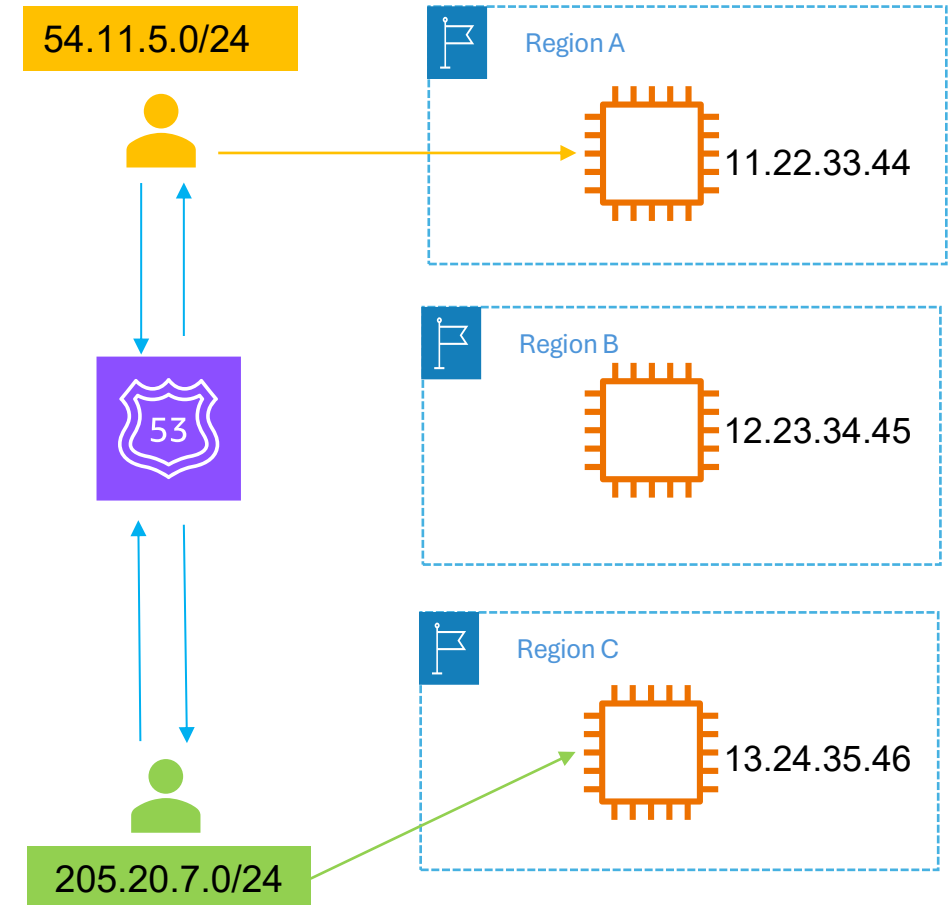
- Routes users based on geographic distance between the user and your AWS or on-prem endpoints.
- You can add a **bias** to shift more or less traffic toward a specific endpoint.
- Set bias 1 to 99 to expand the size of the geographic region
- Set bias -1 to -99 to shrink the size of the geographic region
- More flexible — traffic can shift dynamically as distances or biases change.
- Requires Route 53 Traffic Flow (policy-based routing).

# Route 53 - Routing policies

## IP-based Routing policy

- Routes traffic based on the client's IP address range (CIDR blocks).
- You create records mapped to specific IP ranges
- Gives fine-grained control, more precise than Geolocation (country-level).
- Useful when you want certain networks, ISPs, or corporate ranges to reach specific endpoints.

| Record Name | Value       | IP CIDR       |
|-------------|-------------|---------------|
| example.com | 11.22.33.44 | 54.11.5.0/24  |
| example.com | 12.23.34.45 | 70.20.9.0/24  |
| example.com | 13.24.35.46 | 205.20.7.0/24 |

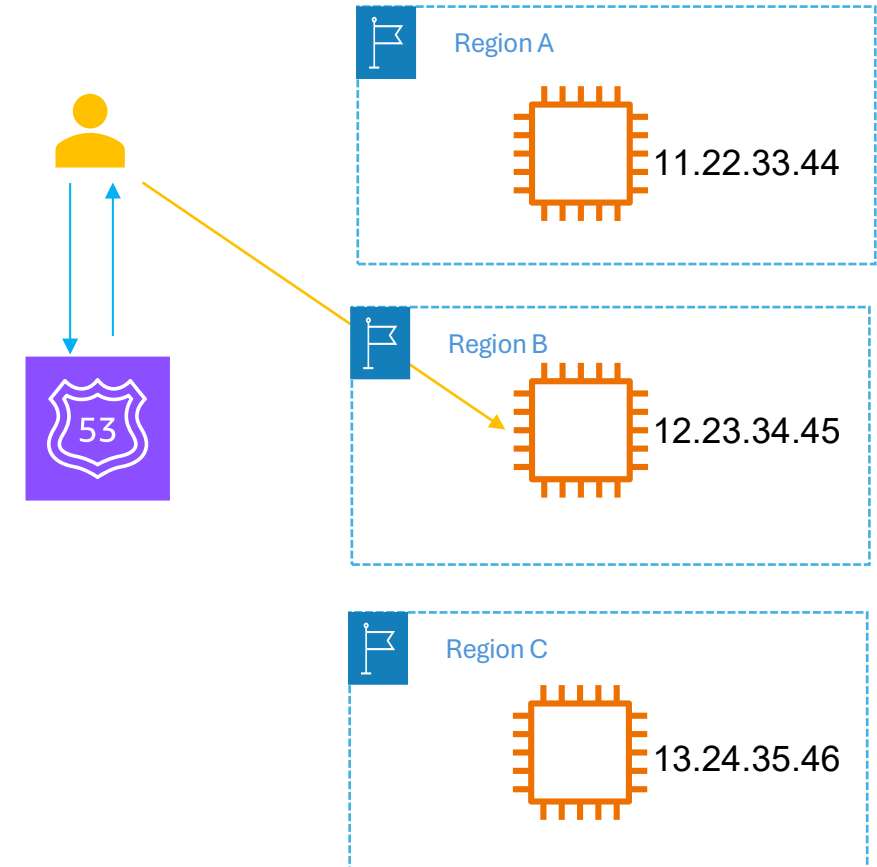


# Route 53 - Routing policies

## Multi-value Routing policy

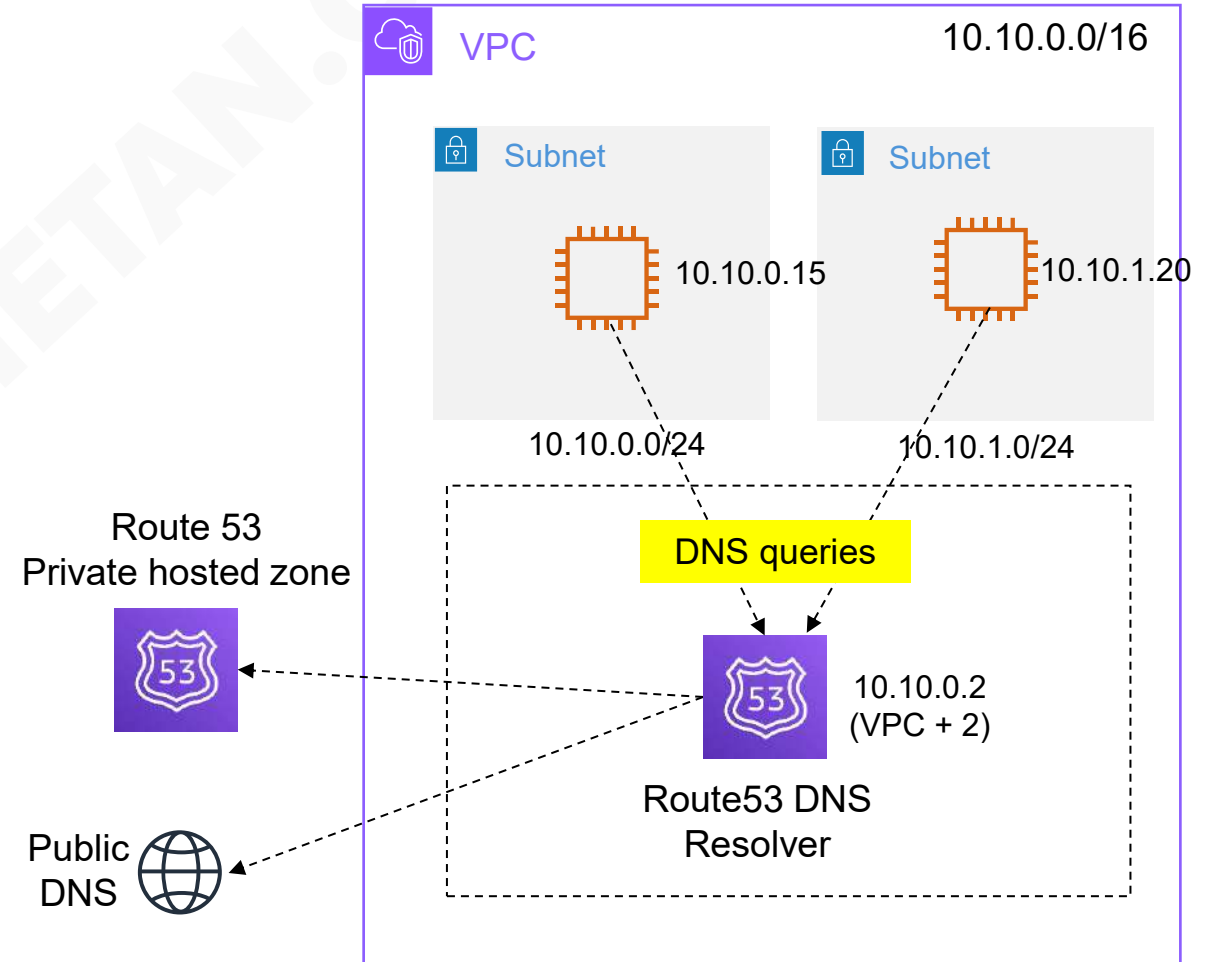
- Allows multiple records with the same name and type
- Route 53 returns up to 8 healthy IPs in each DNS response.
- The client chooses one of those IPs to connect to.
- Supports health checks - unhealthy endpoints are automatically removed.
- Useful for distributing traffic across multiple servers or endpoints.
- Provides basic **DNS-level load balancing** (not as advanced as ELB).

| Record Name | A           |
|-------------|-------------|
| example.com | 11.22.33.44 |
| example.com | 12.23.34.45 |
| example.com | 13.24.35.46 |



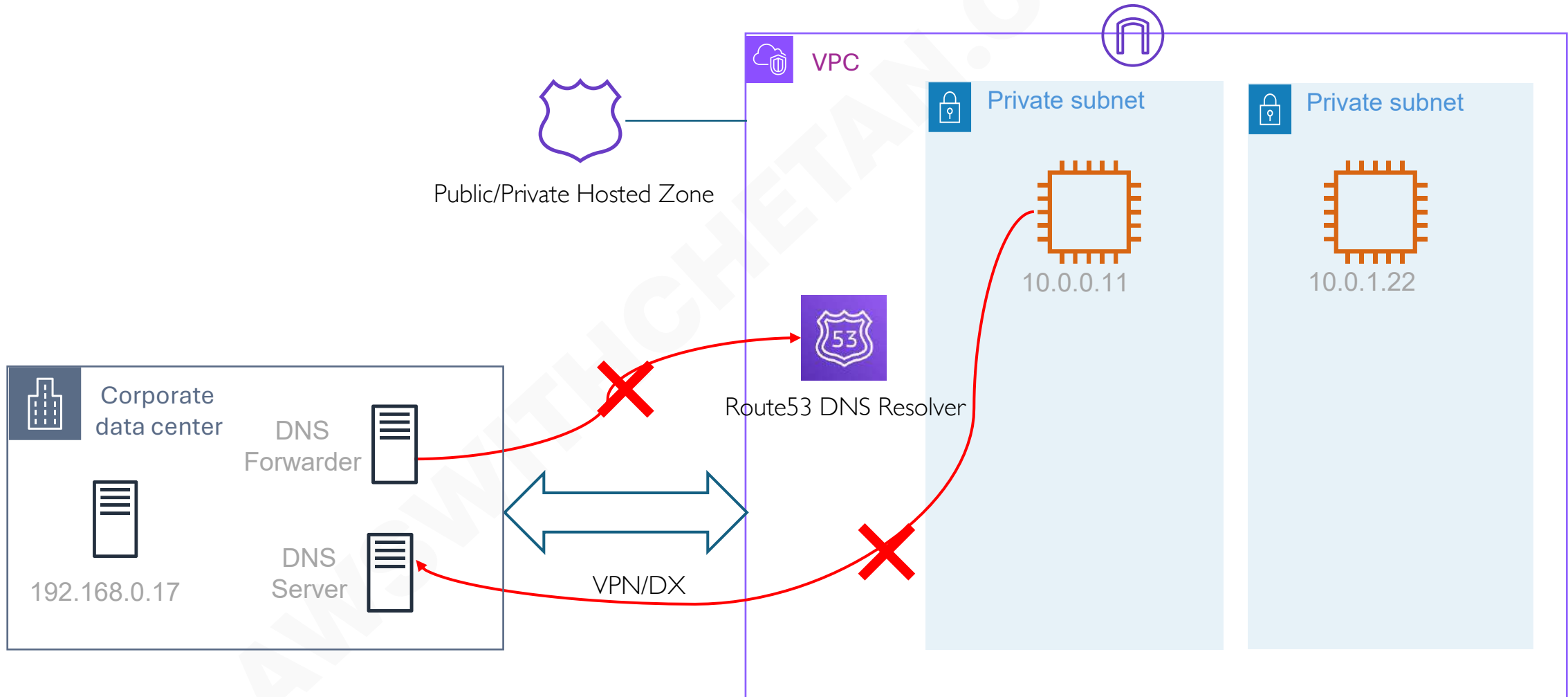
# Route 53 DNS Resolver

- VPC comes with default DNS server also called as Route53 DNS Resolver
- Runs at VPC Base + 2
- Resolves DNS requests from:
  - Route 53 Private Hosted Zone
  - Forwards other requests to Public DNS (including Route 53 Public Hosted Zones)
- Accessible from within the VPC
- Can not be access from outside VPC
- Can not access other DNS (e.g. on-premises)

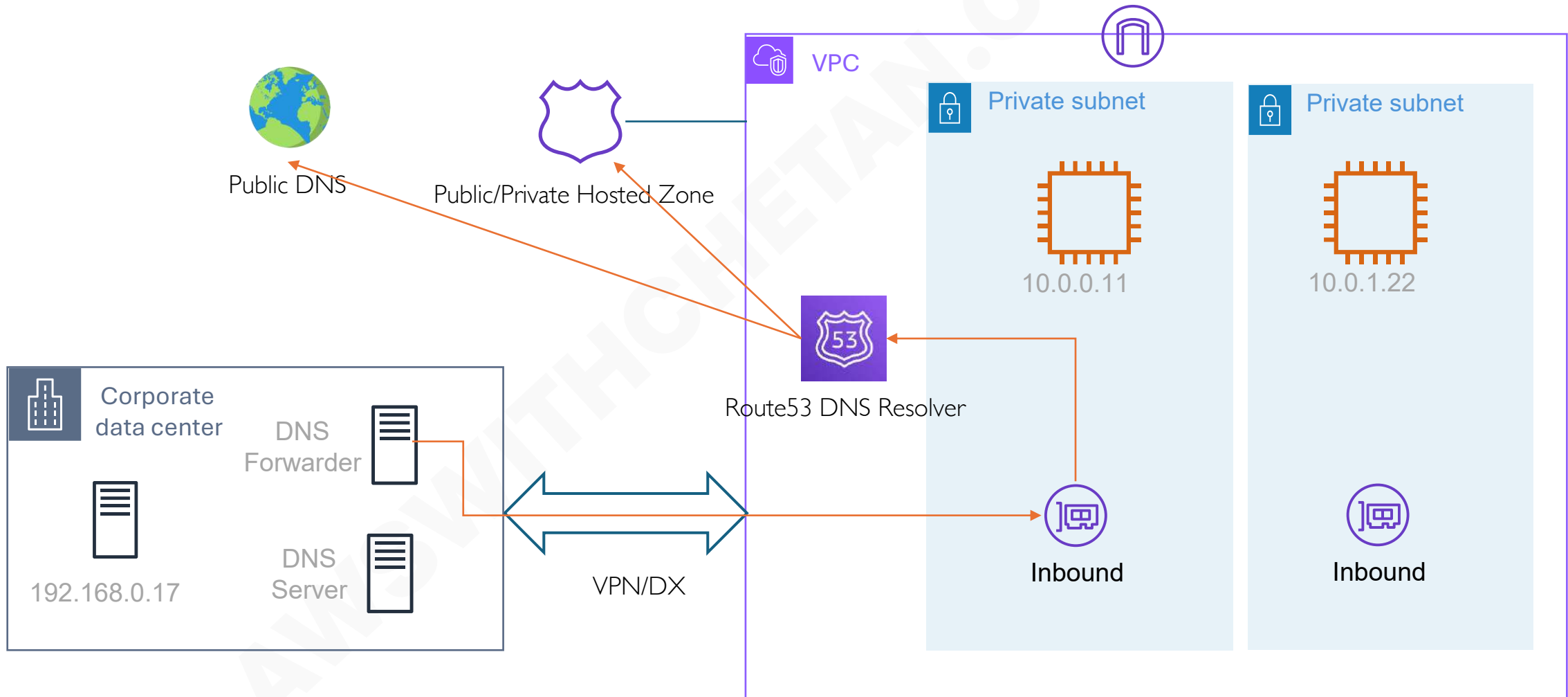




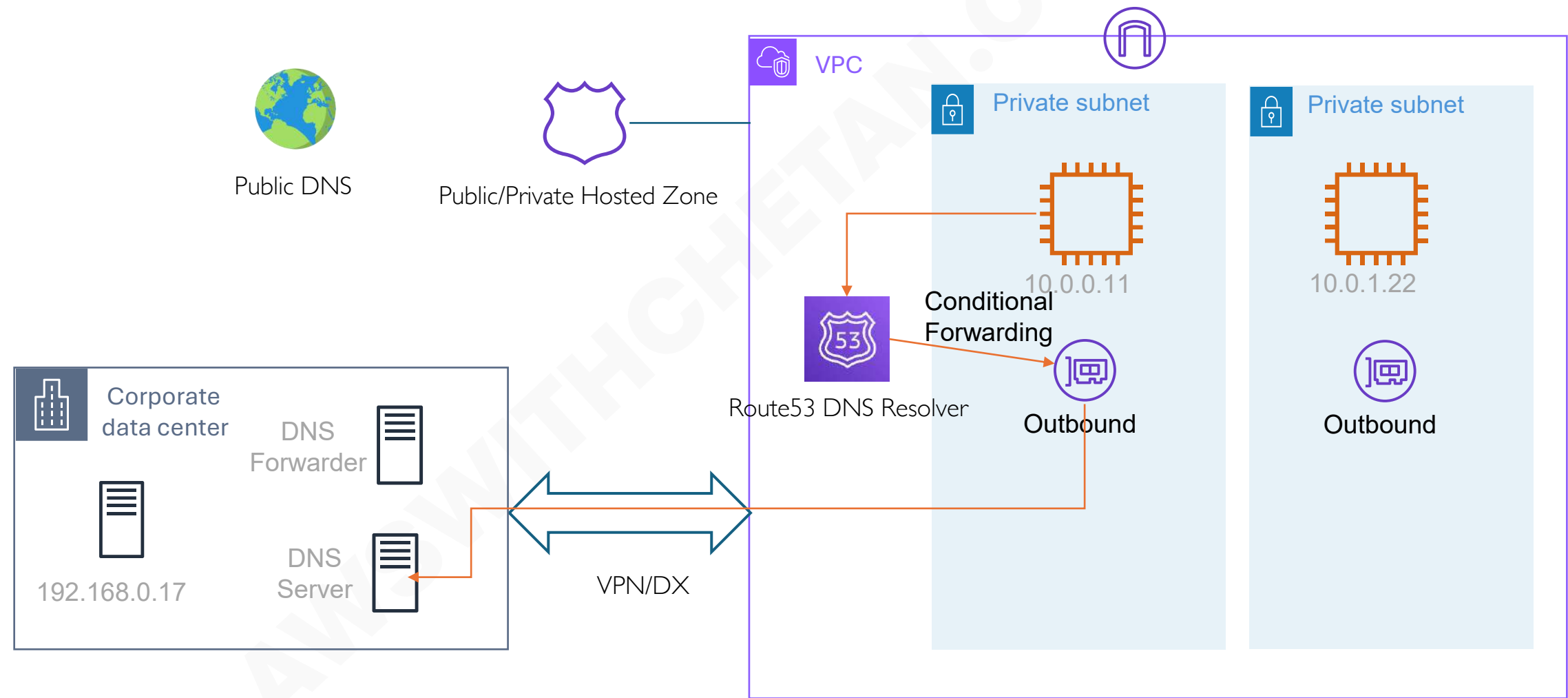
# Hybrid DNS



# Route 53 – Inbound resolver endpoint



# Route 53 – Outbound resolver endpoint





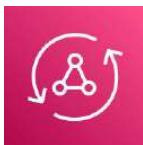
# AWS Serverless

# AWS Serverless Services

## API and Application Integrations



API Gateway



AWS AppSync



SQS



SNS



EventBridge

## Compute



AWS Lambda

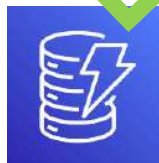


AWS Fargate

## Storage and Databases



S3



DynamoDB



Aurora  
Serverless

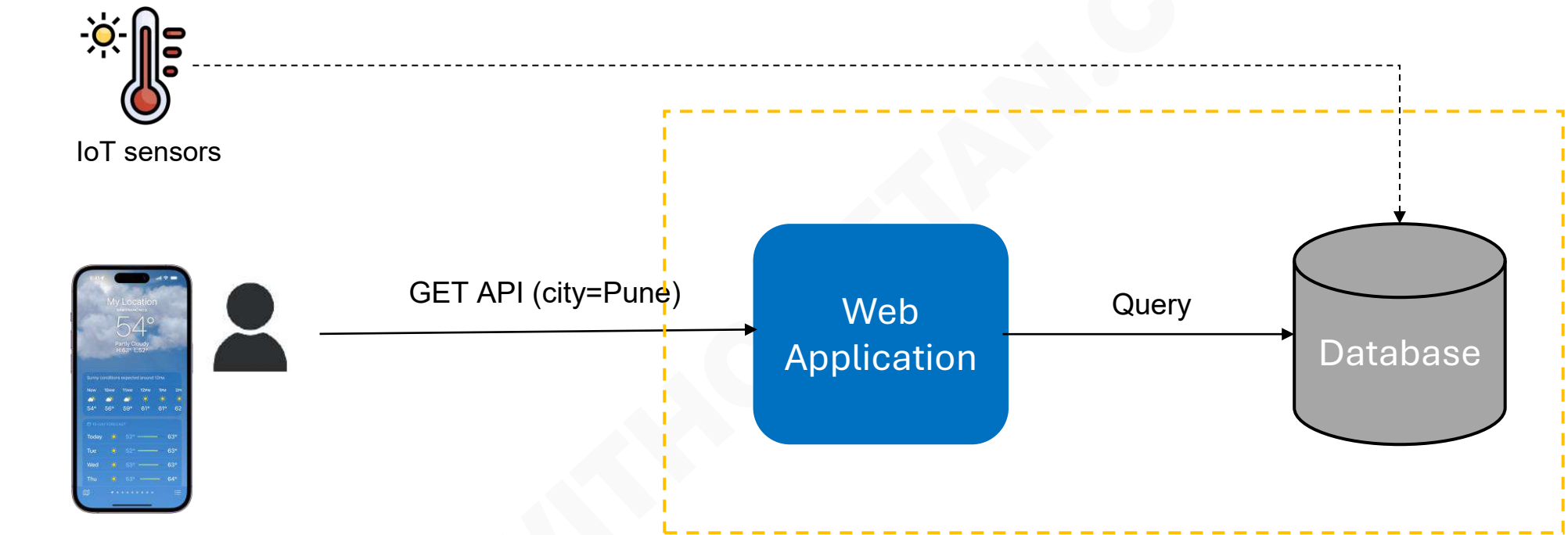


Redshift  
Serverless

# Serverless

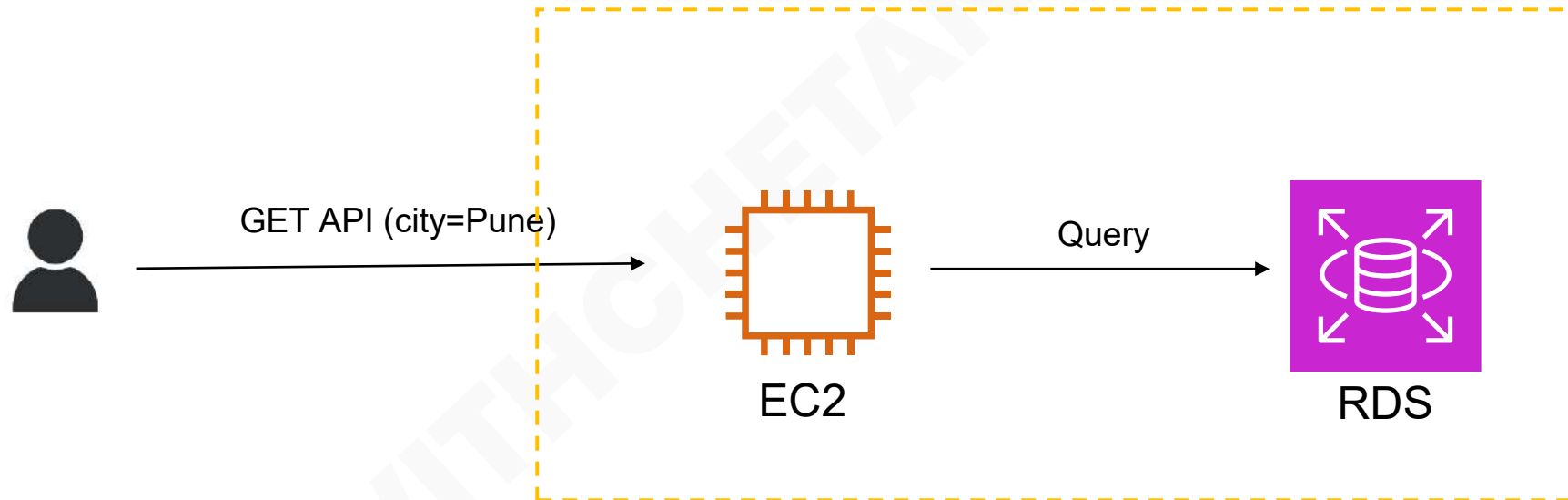
- ✓ No infrastructure management
- ✓ Automatic scaling
- ✓ Pay-per-use pricing
- ✓ Built-in availability and fault tolerance
- ✓ Event-driven execution
- ✓ Short-lived, stateless compute
- ✓ Faster development

# A simple weather app



A simple Weather App

# Hosting weather app on EC2

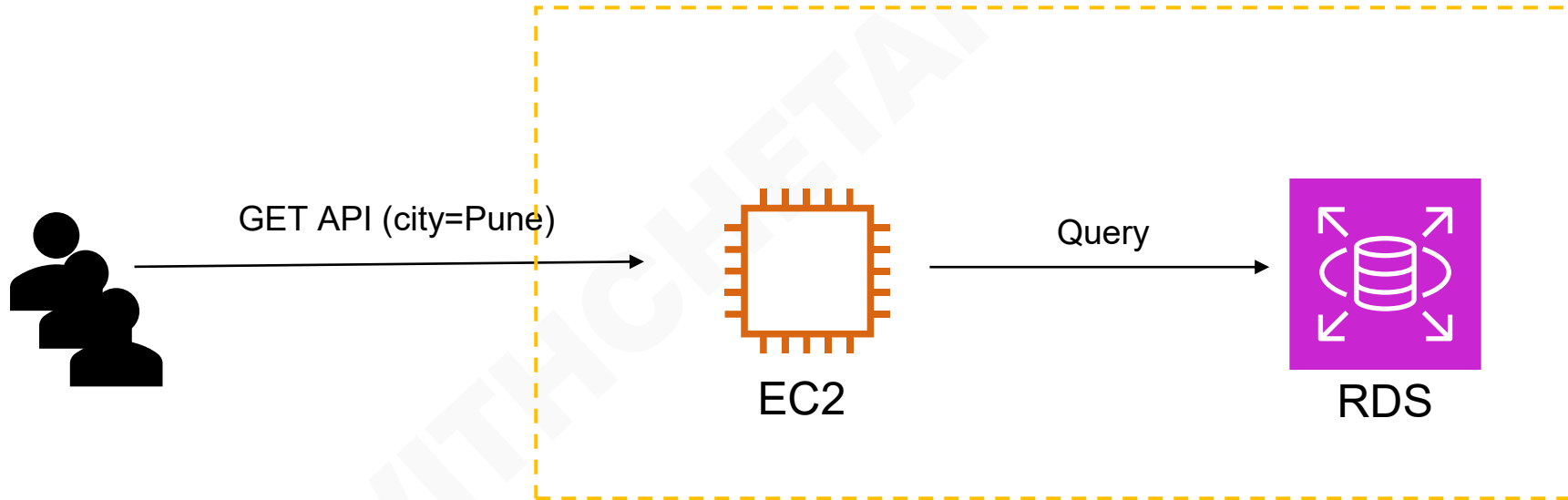


**A simple Weather App**



# Hosting weather app on EC2

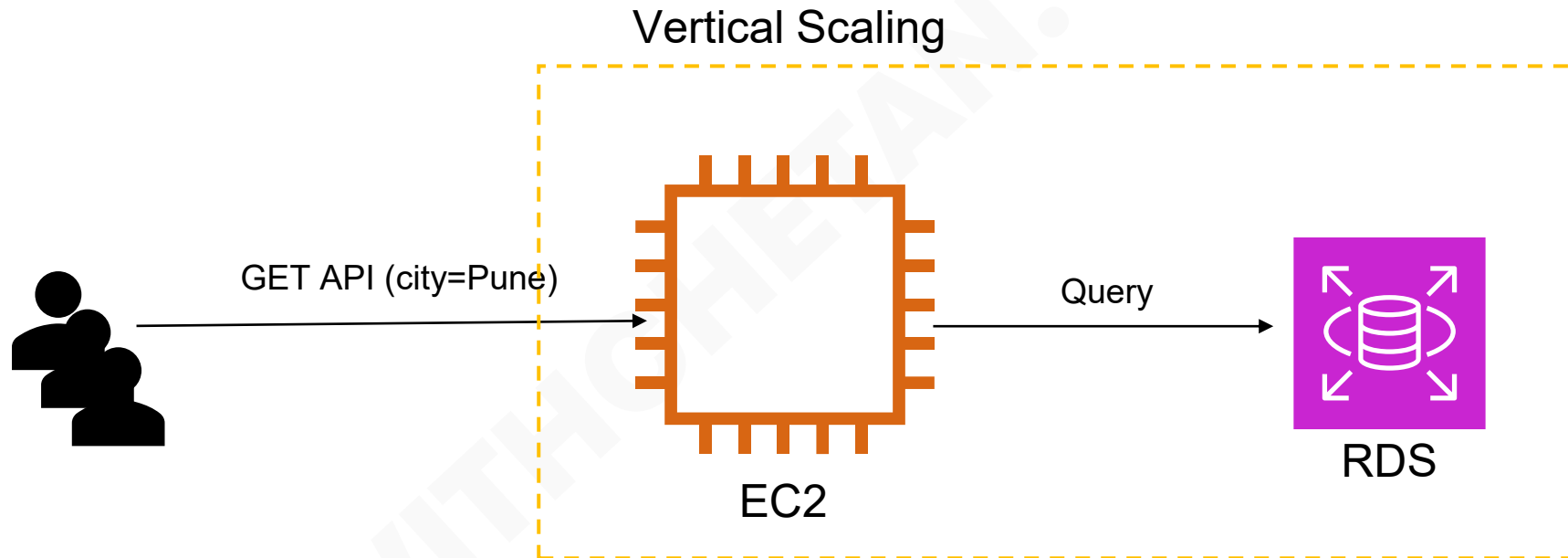
What if there is heavy traffic?



**A simple Weather App**

# Hosting application on EC2

What if there is heavy traffic?

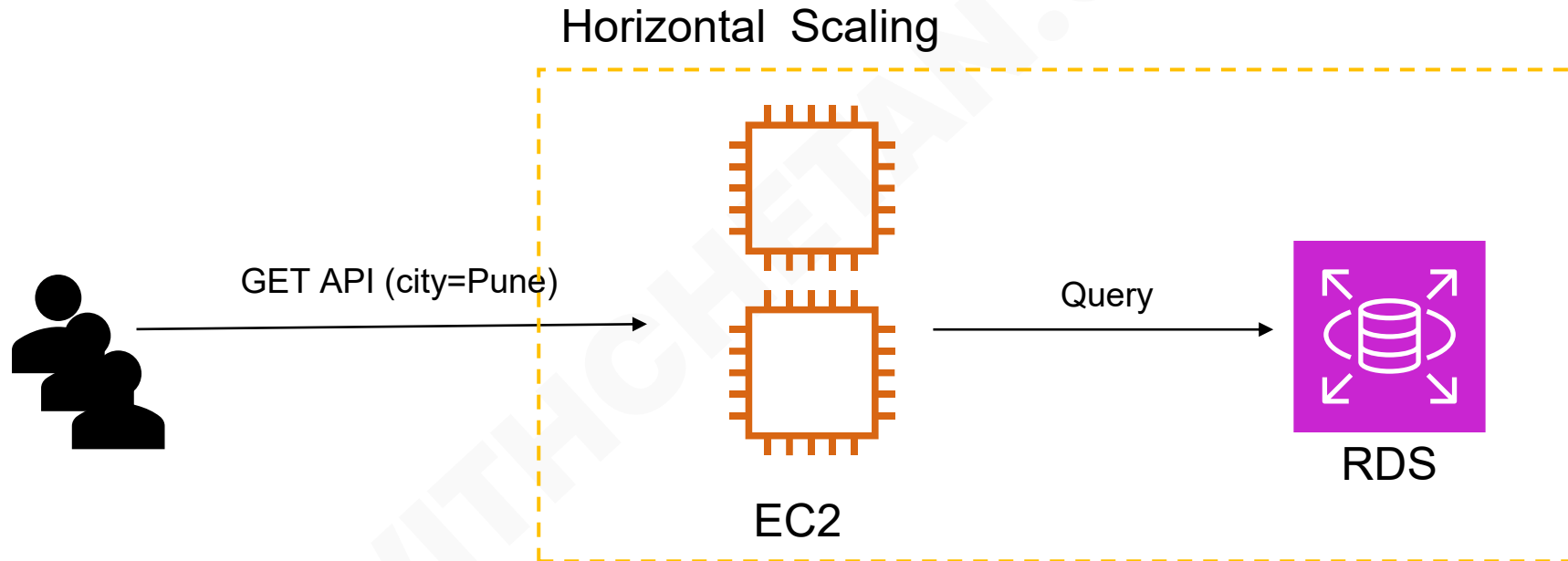


## A simple Weather App

- ✓ Simple architecture
- ✗ Infrastructure Management
- ✗ High cost even when no traffic

# Hosting application on EC2

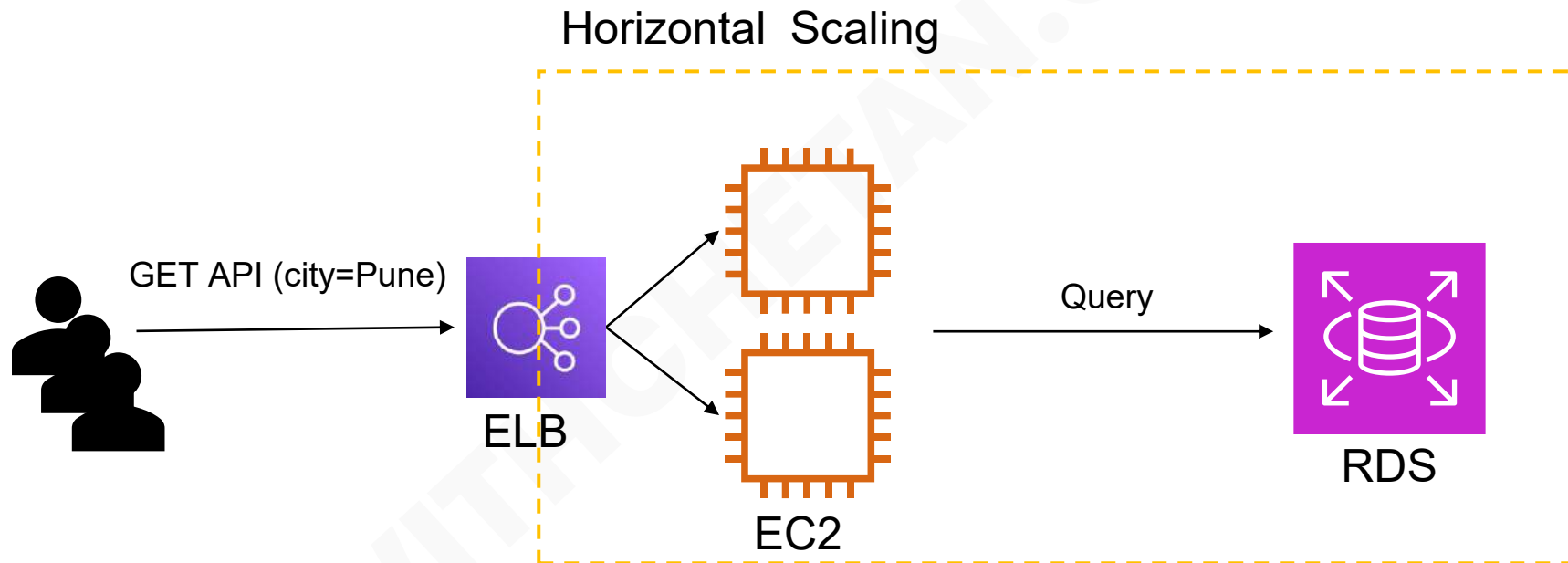
What if there is heavy traffic?



**A simple Weather App**

# Hosting application on EC2

What if there is heavy traffic?

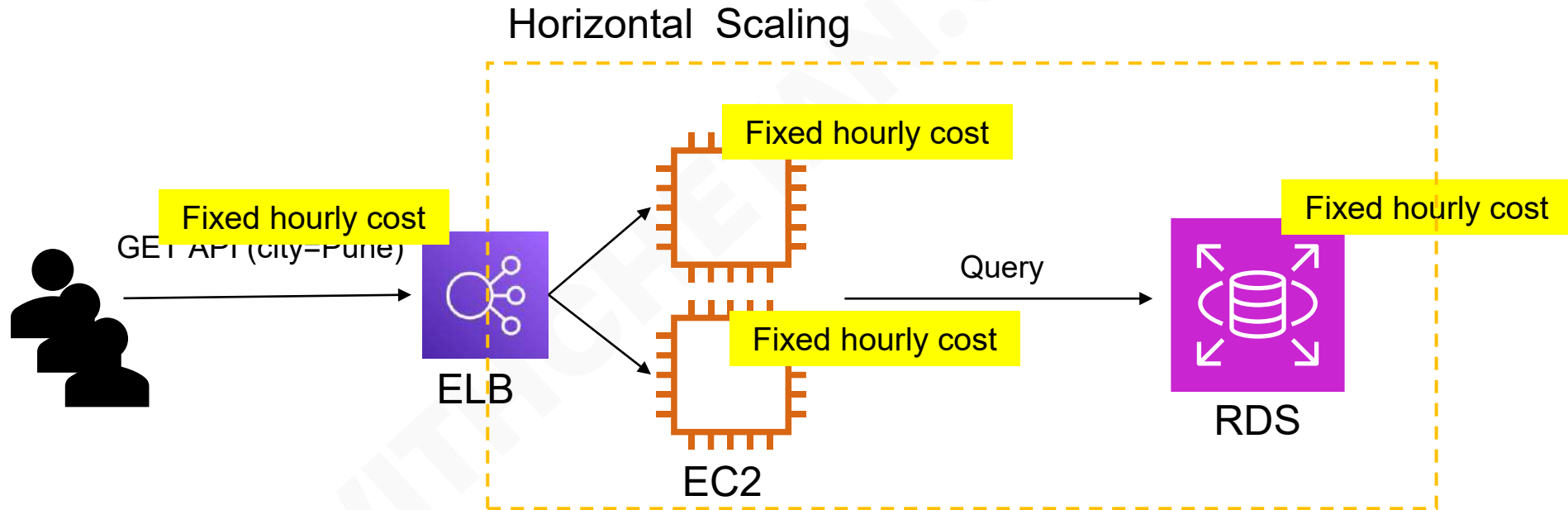


**A simple Weather App**

- ✓ Highly Available
- ✗ Infrastructure Management
- ✗ High cost even when no traffic

# Hosting application on EC2

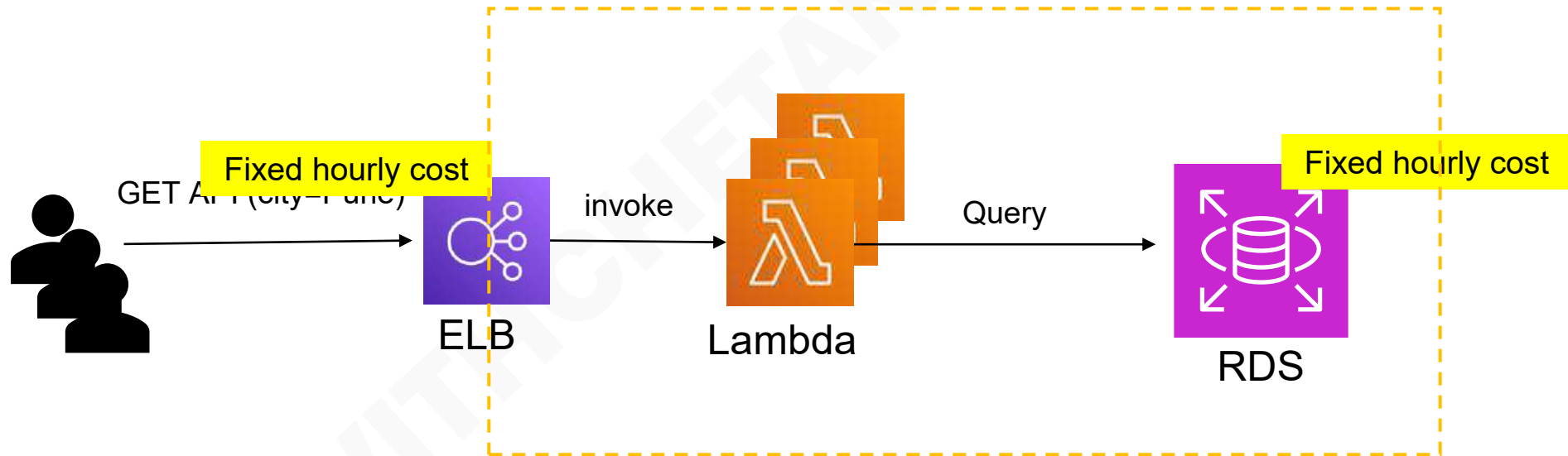
What if there is no traffic?



**A simple Weather App**

- ✓ Highly Available
- ✗ Infrastructure Management
- ✗ High cost even when no traffic

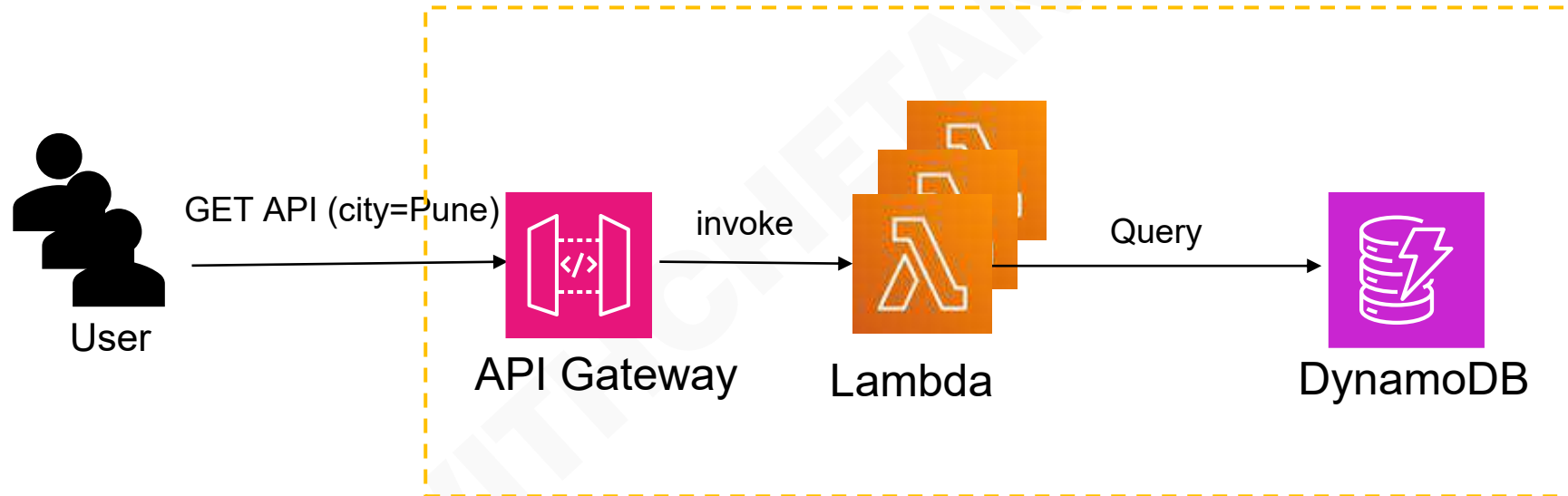
# Hosting weather app using AWS Lambda



**A simple Weather App**

- ✓ Highly Available and Scalable compute
- ✓ No cost for Lambda when no traffic

# A serverless weather app



## A simple Weather App – Fully Serverless

- ✓ Highly Available, Scalable
- ✓ No infrastructure management
- ✓ No infrastructure cost when no traffic
- ✓ Pay per unit

# Cost comparison

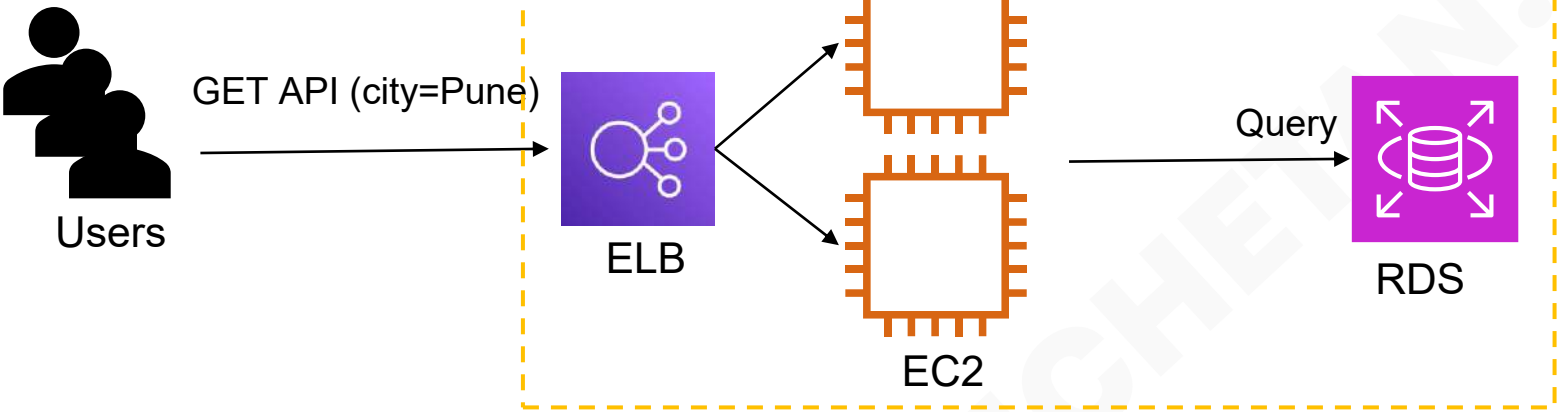
Total monthly requests = 10k x 30 = 300k

| Service                                 | Cost/hr  | Monthly                         |
|-----------------------------------------|----------|---------------------------------|
| EC2<br>m5.large (2 vcpu, 8G Memory) x 2 | \$0.101  | 730 hr x \$0.101 x 2 = \$147.46 |
| ELB                                     | \$0.0239 | 730 hr x \$0.0239 = \$17.44     |
| Total                                   |          | <b>\$164.9</b>                  |

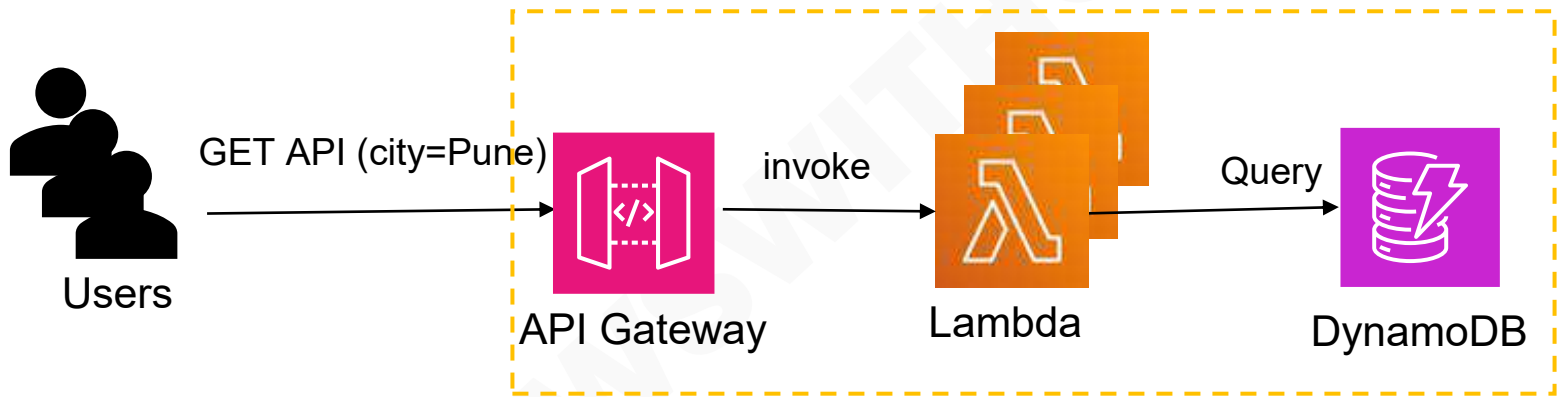
| Service         | Cost/request                       | Monthly       |
|-----------------|------------------------------------|---------------|
| Lambda (128MB)  | \$0.0000166667 for every GB-second | \$1.25        |
| Lambda Requests | \$0.20 per 1M requests             | \$0.08        |
| API Gateway     | \$3.50 per 1M                      | \$1.20        |
| Total           |                                    | <b>\$2.53</b> |

Assumption: 128mb memory, 2 sec execution time

## Option 1



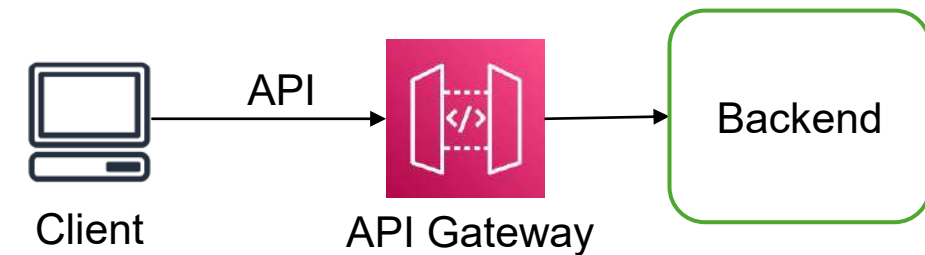
## Option 2





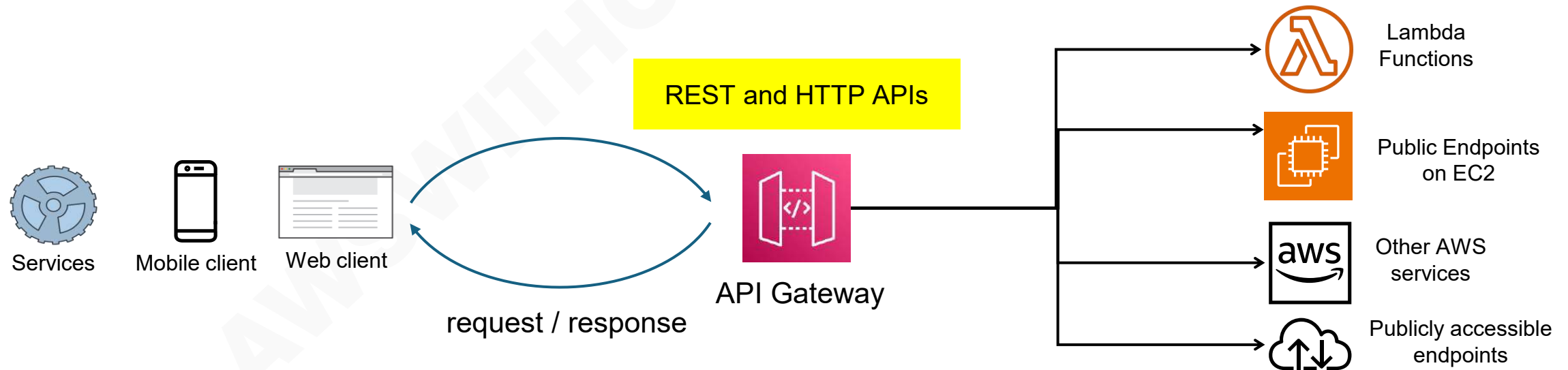
# Amazon API Gateway

- API Gateway is a fully managed API service
- API gateway provides:
  - REST APIs, HTTPS APIs and WebSocket APIs
  - Use AWS Lambda, Amazon ECS, EKS, Elastic Beanstalk etc. for hosting backend services.
  - Out of the box monitoring and logging with Amazon CloudWatch
  - Support User authentication & authorization, API throttling, API keys etc.
  - Response cache, API versioning, importing APIs, AWS WAF integration and many such features.



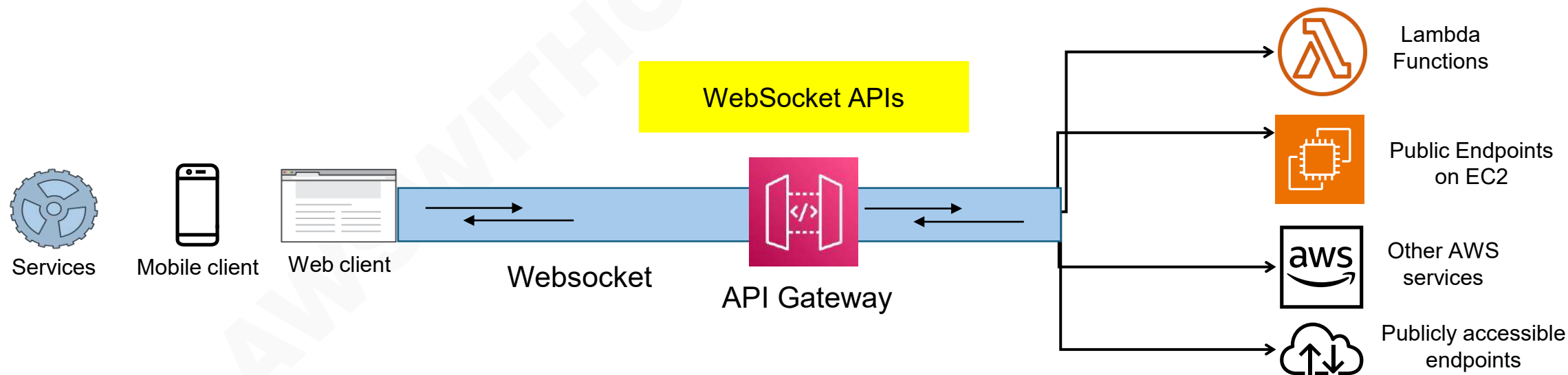
# API Gateway – REST and HTTP APIs

- REST APIs and HTTP APIs are both RESTful API products.
- HTTP APIs are designed with minimal features at a lower price.
- Choose REST APIs if you need features such as API keys, per-client throttling, request validation, AWS WAF integration, or private API endpoints.



# API Gateway – WebSocket APIs

- Uses persistent WebSocket connections instead of request/response like REST.
- Supports real-time, bidirectional communication between clients and the server.
- Ideal for chat apps, notifications, live dashboards, multiplayer games, IoT updates, etc.



# API gateway method integrations

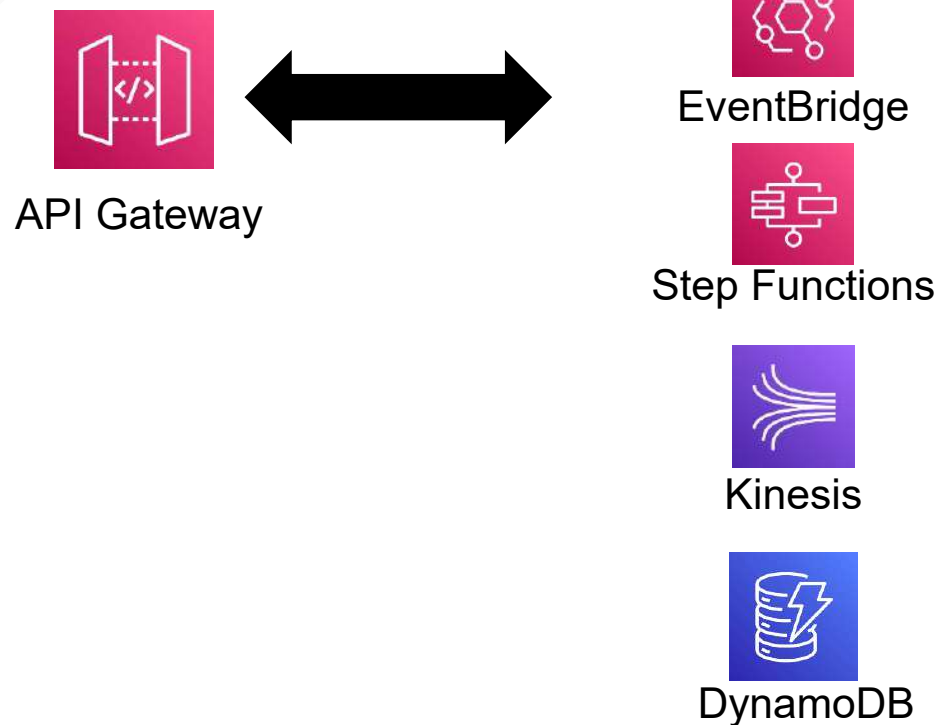
An integration in API Gateway defines how an API method connects to and invokes its backend

- **Lambda Integration** – Triggers AWS Lambda functions; perfect for fully serverless backend logic.
- **HTTP / HTTP Proxy Integration** – Sends requests to public HTTP endpoints or proxies the entire request to your backend as-is.
- **AWS Service Integration** – Directly invokes AWS services without writing backend code. This supports AWS services including SQS, SNS, Step Functions, DynamoDB, Kinesis, EventBridge and many many more.
- **Mock Integration** – Returns a predefined response without calling any backend; great for demos and testing.
- **VPC Link Integration** – Privately connects to services inside your VPC via an ALB/NLB, without exposing them publicly.

# API gateway method integrations

## Commonly used AWS services integration

- SQS – Send messages
- SNS – Publish notifications
- Step Functions – Start workflows
- Kinesis – Put records
- DynamoDB – CRUD operations
- EventBridge – Put events



# API Gateway Endpoints

## Edge-Optimized Endpoint (for REST APIs)

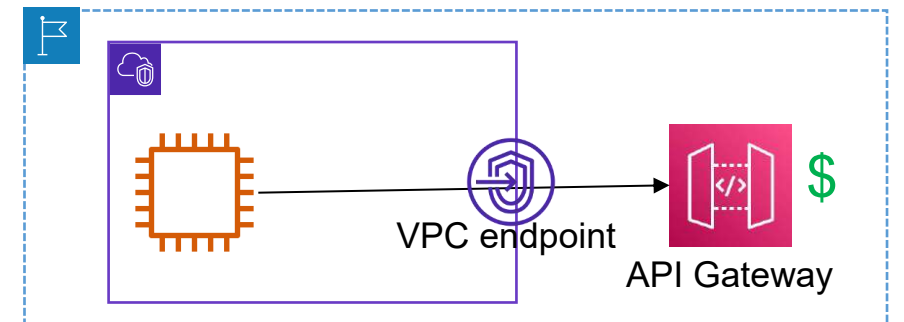
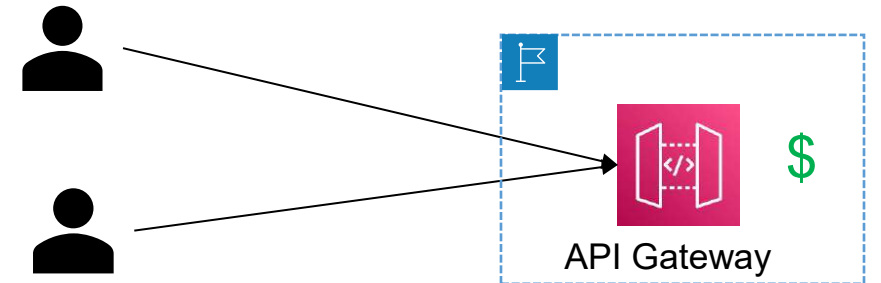
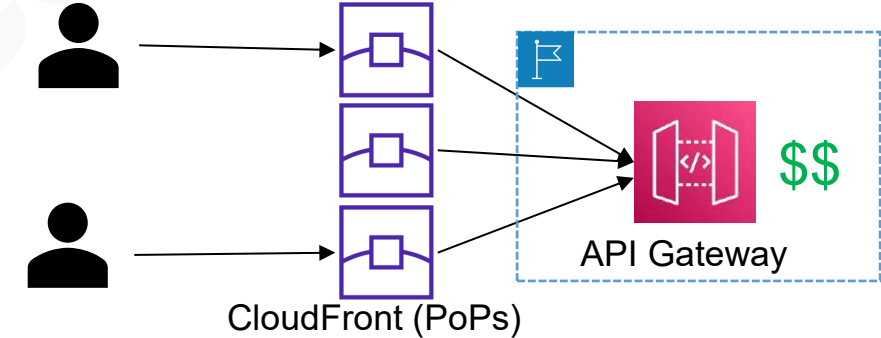
- Uses CloudFront to route requests through AWS edge locations.
- Best for global clients needing low latency.

## Regional Endpoint

- API is deployed in a specific AWS Region.
- Best when clients are mainly in the same region or when using your own custom CDN or private networking.

## Private Endpoint

- Accessible only from your VPC using an Interface VPC Endpoint (PrivateLink).
- Not publicly reachable — ideal for secure, internal, service-to-service communication.

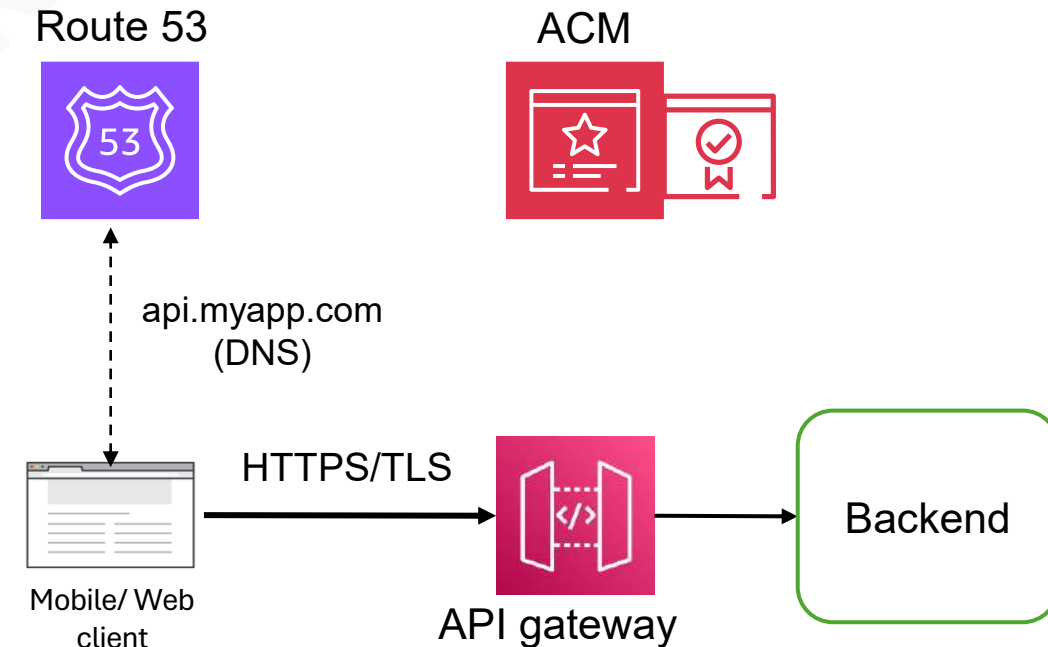


# API Gateway - Custom domain name

- By default, API endpoint receives AWS provided domain name

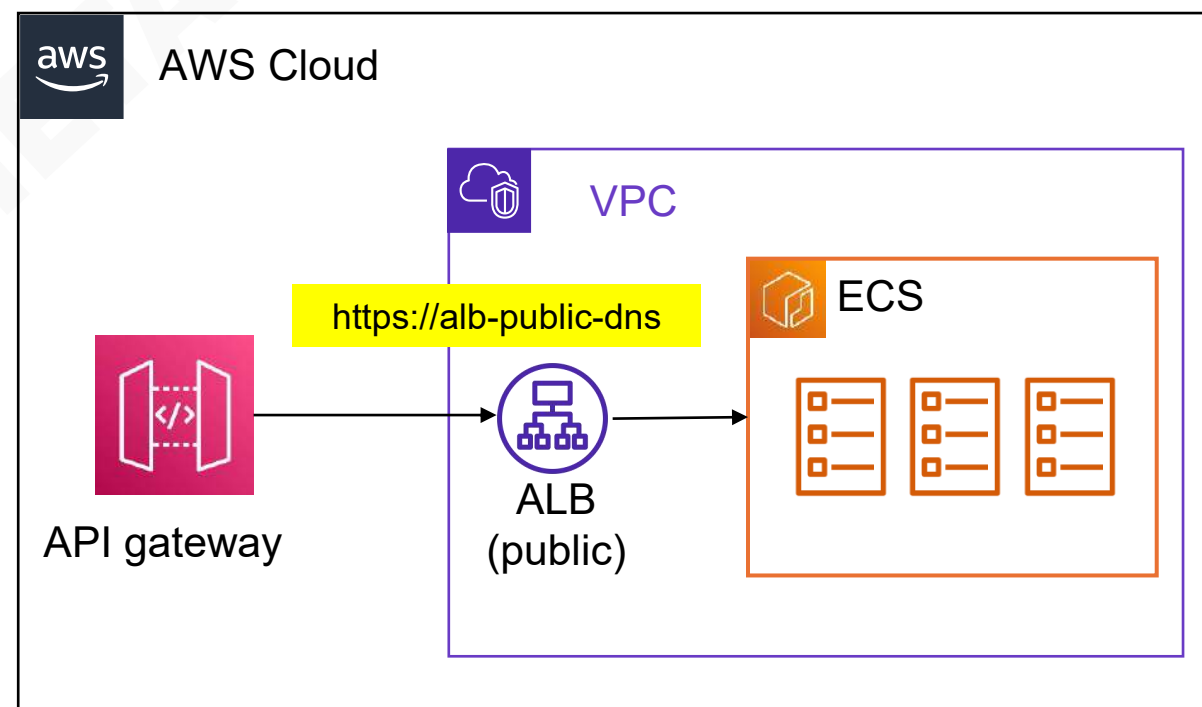
`https://{api-id}.execute-api.{region}.amazonaws.com/{stage}`

- We can use custom domain name e.g. `api.myapp.com`
- You must provide an SSL/TLS certificate from ACM
- For edge-optimized REST APIs, the domain is deployed through CloudFront automatically and hence Certificate must be in **us-east-1** region
- For Regional REST API or HTTP API, Certificate must be in the same region as the API



# API gateway integration with private backend services

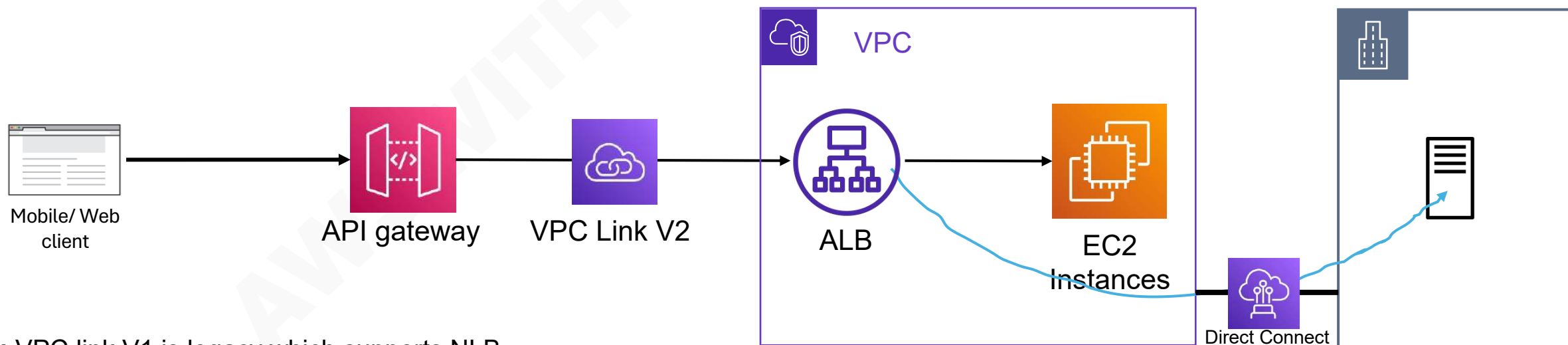
- API gateway is a Public service and sits outside of the VPC
- It accesses backend service endpoints using their Public DNS
- It can not directly access the private resources inside a VPC e.g. EC2 instance Private IP, internal application load balancer etc.





# API gateway integration with private backend services

- To reach private resources (like EC2, ECS, internal ALBs) you can use API Gateway **VPC link V2**.
- Creates a private connection between API Gateway and ALB hosted inside your VPC
- The API backend targets can be EC2 instances, ECS tasks/services or On-premises servers which are accessible through the ALB (hybrid-connectivity)



**Note:** VPC link V1 is legacy which supports NLB

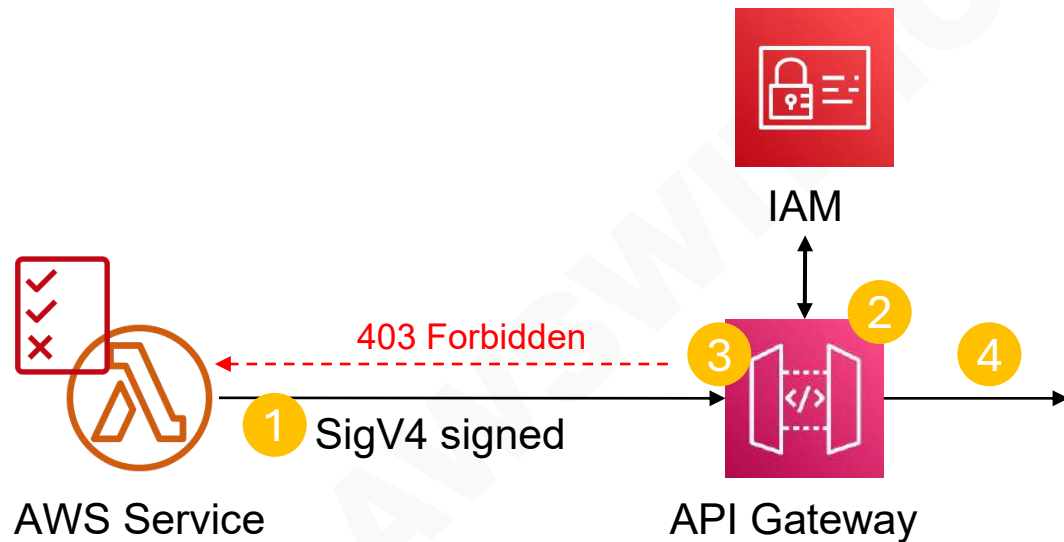
# API Authentication & Authorization

- Using IAM Permissions
- Using Amazon Cognito
- Using Lambda Authorizer (custom authorizer)
- More ways..

AWSWITHCHETAN.COM

# Using IAM

- Useful when APIs are internal or API to be invoked by other AWS services e.g. Lambda, Step Functions, ECS task etc.
- API Request must include method's authorizationType=AWS\_IAM



## Caller sample IAM policy

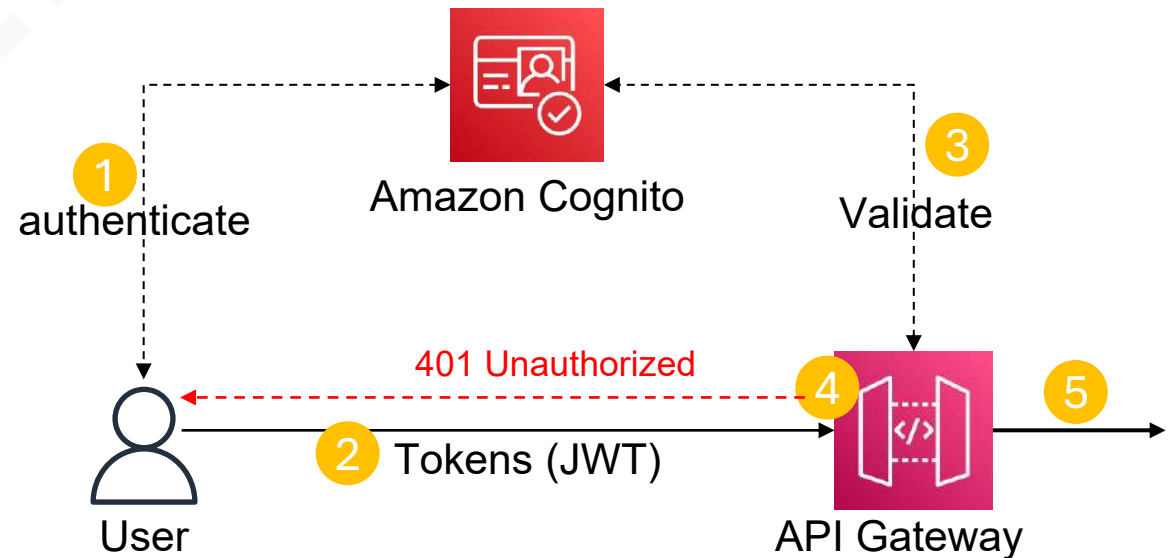
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "execute-api:Invoke"
      ],
      "Resource": [
        "arn:aws:execute-api:us-east-1:account-id:api-id/*/GET/pets"
      ]
    },
    {
      "Effect": "Deny",
      "Action": [
        "execute-api:Invoke"
      ],
      "Resource": [
        "arn:aws:execute-api:us-east-1:account-id:api-id/*/POST/pets"
      ]
    }
  ]
}
```

# Using Amazon Cognito

Amazon Cognito is an AWS service that provides secure user sign-up, sign-in, and access control for web and mobile apps, acting as a managed identity provider (IdP)

## Authentication and Authorization for API gateway:

1. User signs-in to Amazon Cognito User pool and receives JWT tokens
2. Sends JWT tokens to API gateway endpoint
3. API gateway validates JWT tokens
4. If invalid -> 401 Unauthorized
5. If invalid -> Retrieve claims (email, groups, custom claims) and forwards the request to the backend
6. Backend application handles the request and responds back

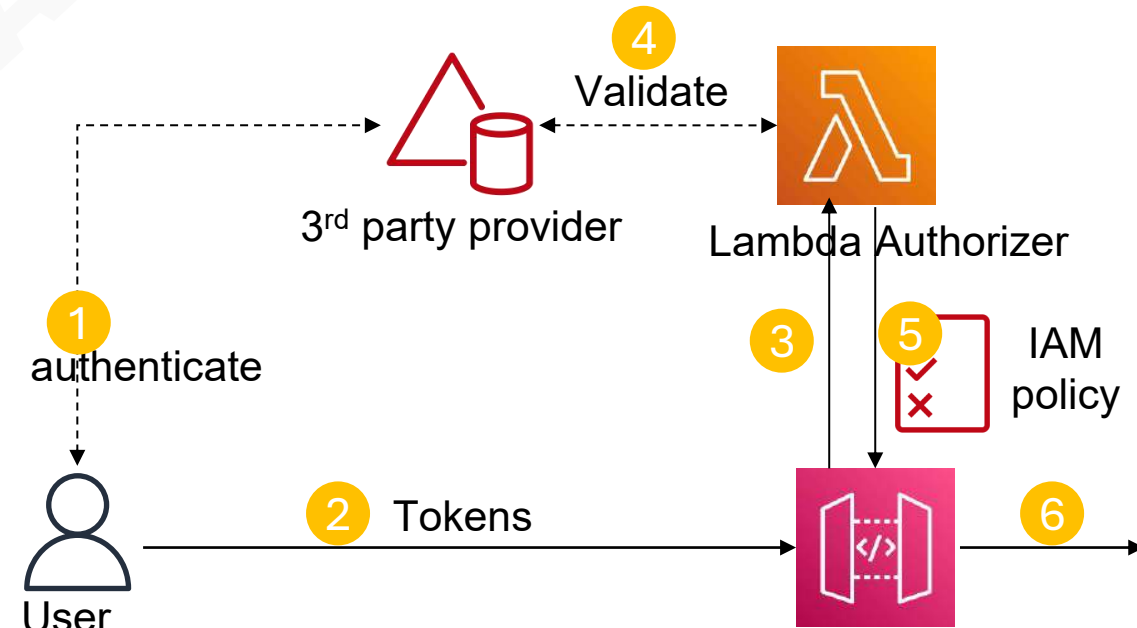


# Using Lambda Authorizer

Lambda authorization should be used when you want full flexibility to implement custom authentication and fine-grained authorization tailored to your application's needs.

## Authentication and Authorization for API gateway:

1. User signs in with a 3rd-party IdP and the client obtains tokens (JWT/opaque token/keys).
2. Client calls the API Gateway endpoint and includes the token in the request header.
3. API Gateway invokes the configured Lambda authorizer
4. Lambda authorizer validates the token
5. If valid, Lambda returns an IAM-style policy Document, principal Id, and optional context to API Gateway.
6. API Gateway evaluates the policy; if allowed it forwards the request to the backend



# API Gateway Resource policy

- Resource Policies control who can access your API at the network and account level, **before** authentication/authorization happens.
- They work similarly to S3 bucket policies - attached directly to the API Gateway resource.
- Useful for restricting public APIs to certain IPs or AWS accounts even before other auth layers.
- Let you allow or deny access based on:
  - AWS accounts
  - IAM principals
  - Source IP ranges
  - VPC endpoints (for Private APIs)
  - AWS Organizations
- Critical for Private APIs, where access must be restricted to specific VPC Interface Endpoints.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "arn:aws:execute-api:us-east-1:111111111111:api-id/*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

# Amazon API Gateway – Good to know

- **API Versioning** – Lets you publish multiple versions of your API (e.g., v1, v2) so clients can upgrade safely without breaking existing integrations.
- **API Stages & Environments** – Provides separate deployments like dev, test, and prod, each with independent settings, logging, throttling, and stage variables.
- **API Keys** – Used to identify and throttle clients through usage plans; not meant for authentication or security by themselves.
- **API Caching** – Improves performance by caching API responses at the stage level, reducing backend load and lowering latency.
- **Request & Response Transformation** - Enables mapping templates to modify headers, payloads, or formats between client and backend (e.g., JSON <> XML).
- **AWS WAF Integration** – You can attach AWS WAF to API Gateway (regional REST & HTTP APIs) to protect against common web exploits like SQL injection, XSS, bot attacks, and abnormal request patterns.

# Amazon API Gateway – Summary

- API Gateway supports REST, HTTP, and WebSocket APIs, enabling traditional request/response, lightweight microservices, and real-time bidirectional communication.
- APIs method integrations – Lambda, HTTP/proxy, AWS services, Mock and VPC Link.
- API Gateway offers edge-optimized, regional, and private endpoints to match global performance, regional access, or VPC-only connectivity needs.
- API Gateway enforces HTTPS for all endpoints, and edge-optimized custom domains require the ACM TLS certificate to be created in the us-east-1 (N. Virginia) region.
- VPC Link v2 enables secure, high-performance private connectivity from API Gateway to services inside your VPC using ALBs without exposing them publicly. VPC Link V1 supports NLB.
- API Gateway supports IAM, Cognito User Pools, and Lambda Authorizers, allowing granular security through signatures, JWT tokens, or custom auth logic.
- You can map your APIs to custom domain names with ACM certificates.
- API Gateway provides versioning, stages/environments, API keys with usage plans, caching, request/response transformations, and WAF integration to improve manageability, performance, and security.



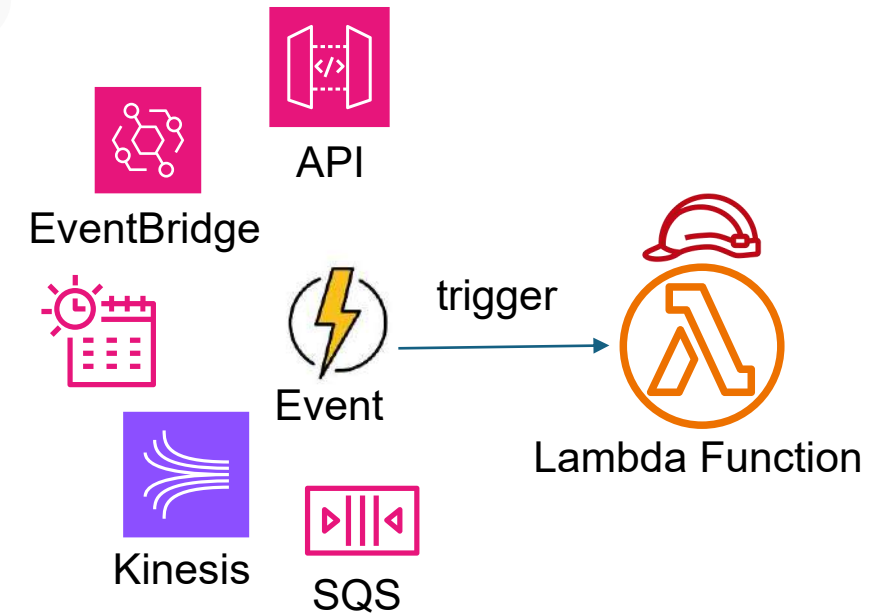




# AWS Lambda

# AWS Lambda

- A serverless, event-driven compute service that lets you run functions without provisioning servers (Function-as-a-Service).
- Ideal for **stateless** execution
- Scales automatically as per #requests, messages etc.
- Just write a function or upload a package with code and dependencies.
- Supports many languages (Python, Go, Node.js, Java, C#, Powershell, Ruby, Go and more)
- Pay as per number of invocations, execution time and assigned memory (Per GB-second).
- Lambda integrates natively with CloudWatch for logs, metrics, and performance monitoring.
- Uses IAM role to access other AWS services e.g. S3, DynamoDB



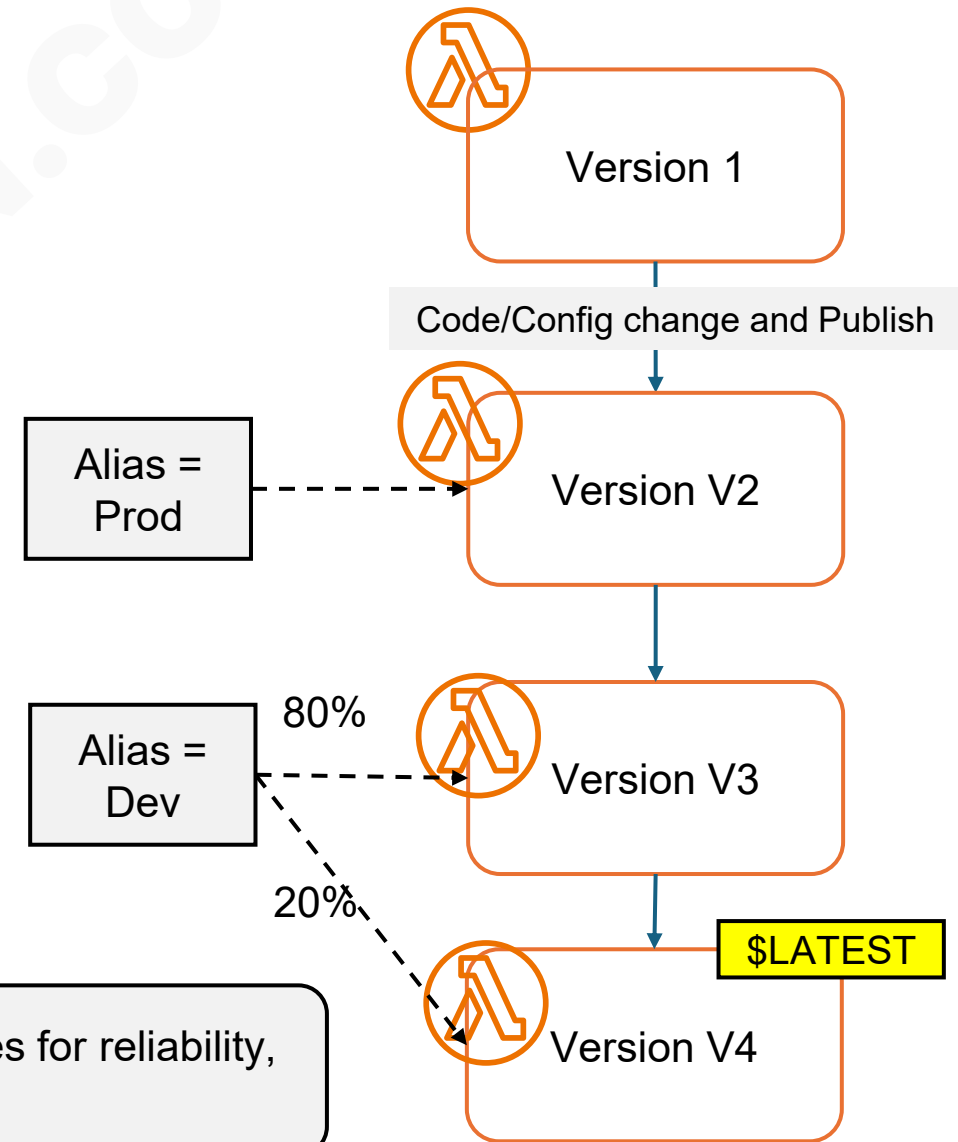
50+ event sources

- Memory: 128mb to 10gb
- Max execution time: Up to 15 mins
- IAM role
- More..

**New ! Lambda Durable functions**

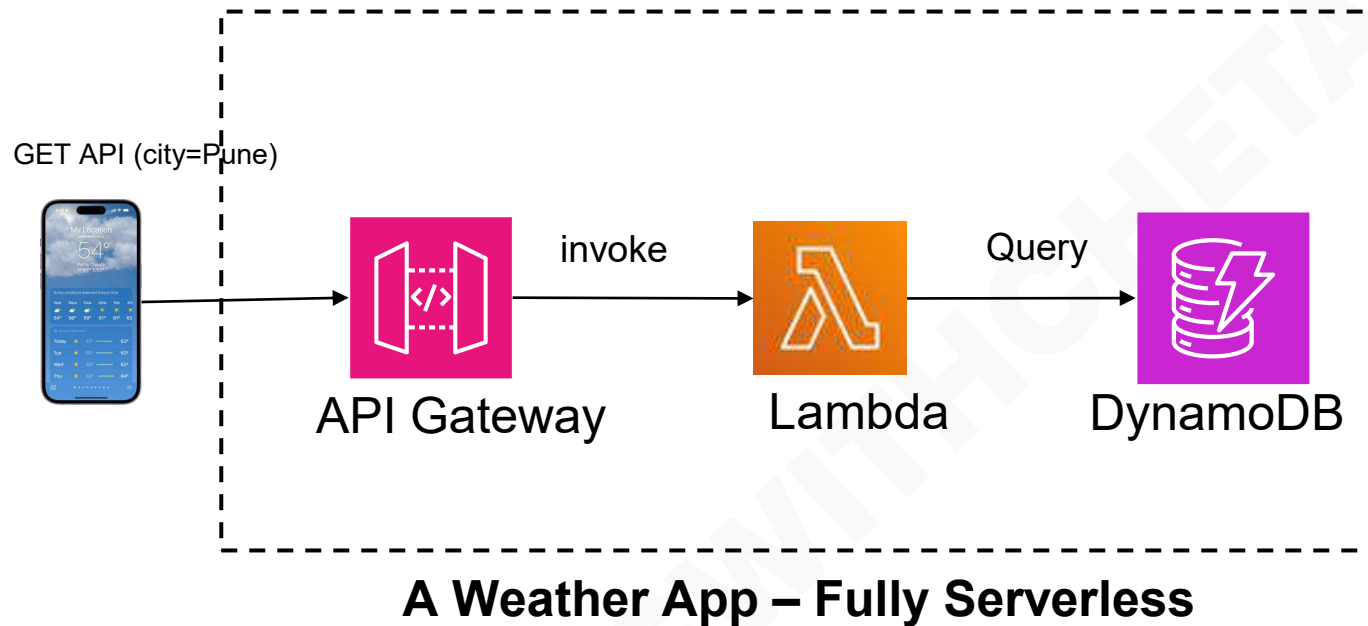
# Lambda function Versions and Aliases

- Versions are immutable snapshots of your Lambda code + configuration (timeout, memory, env vars).
- `$LATEST` represents the most recent deployed code.
- Aliases are named pointers (e.g., dev, test, prod) that map to a specific version.
- Aliases allow safe deployments by switching traffic from one version to another without any change at client side.
- Aliases can use traffic shifting (weighted routing), example 80% to v3, 20% to v4 for canary releases.
- IAM policies, triggers (API Gateway, EventBridge), and permissions can be attached to aliases, enabling environment isolation.



Tip: Never use `$LATEST` in production. Use specific versions + aliases for reliability, rollback, and controlled releases.

# Exercise – A simple weather app with AWS Lambda



*Reference code files are provided with this lecture.*

- 1 Create DynamoDB table and add sample items for cities and temperature e.g. Pune (String), 28 (Number)
- 2 Create Lambda function (Python 3.11) and add code provided with this lecture. Update dynamodb table name.
- 3 Create IAM role for Lambda to have Read-only permissions for DynamoDB. Configure Lambda function to use this role and change execution time limit to 1 min.
- 4 Create API Gateway REST API with Any method and invoke Lambda function with proxy integration. Deploy API (use any stage name)
- 5 Create sample HTML web page using the code provided and replace API endpoint with your endpoint.
- 6 Open HTML page and access the simple weather app.

# Lambda – Important to know

- Lambda Concurrency and Throttling
- Lambda Reserved Concurrency
- Lambda Cold start and Provisioned Concurrency
- Lambda Synchronous and Asynchronous execution
- Lambda SnapStart
- Lambda inside VPC

AWSWITHCHETAN.COM

# Lambda Concurrency

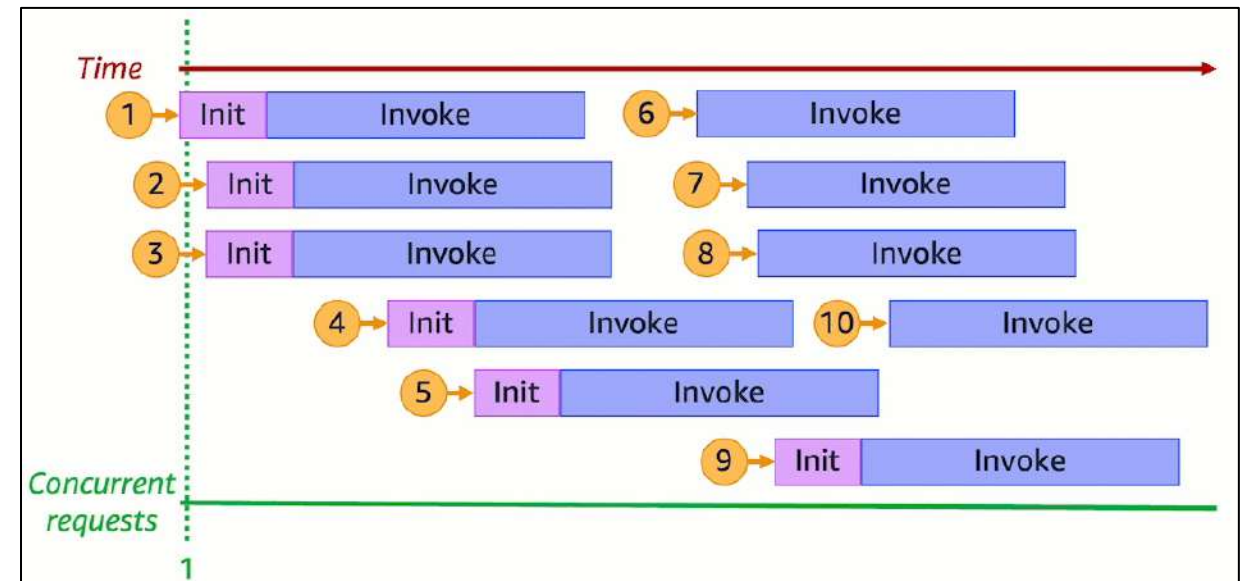
- Lambda invokes your function in a secure and isolated execution environment.
- To handle a request, Lambda must first initialize an execution environment (the Init phase), before using it to invoke your function (the Invoke phase)
- By default, your account has a concurrency limit of 1,000 concurrent executions across all functions in a Region.



## Example:

For 10 requests your lambda function concurrency may look like this

**Total concurrency = 6**



# Lambda Concurrency

Formula to calculate desired concurrency that you need:

$$\text{Concurrency} = (\text{average requests per second}) * (\text{average request duration in seconds})$$

## Example:

If average number of requests per second is 100 and average duration for lambda function to execute each request is 0.5 seconds, then desired concurrency =  $100 \times 0.5 = 50$

# Reserved Concurrency

- By default, your account has a concurrency limit of 1,000 concurrent executions across all functions in a Region.
- Your functions share this pool of 1,000 concurrency on an on-demand basis.
- Your functions experiences throttling (they start to drop requests) if you run out of available concurrency.
- For critical functions in your account, you can set the minimum and maximum concurrency for the function by setting Reserved Concurrency
- When a function has reserved concurrency, no other function can use that concurrency.
- There is no cost for setting Reserved concurrency

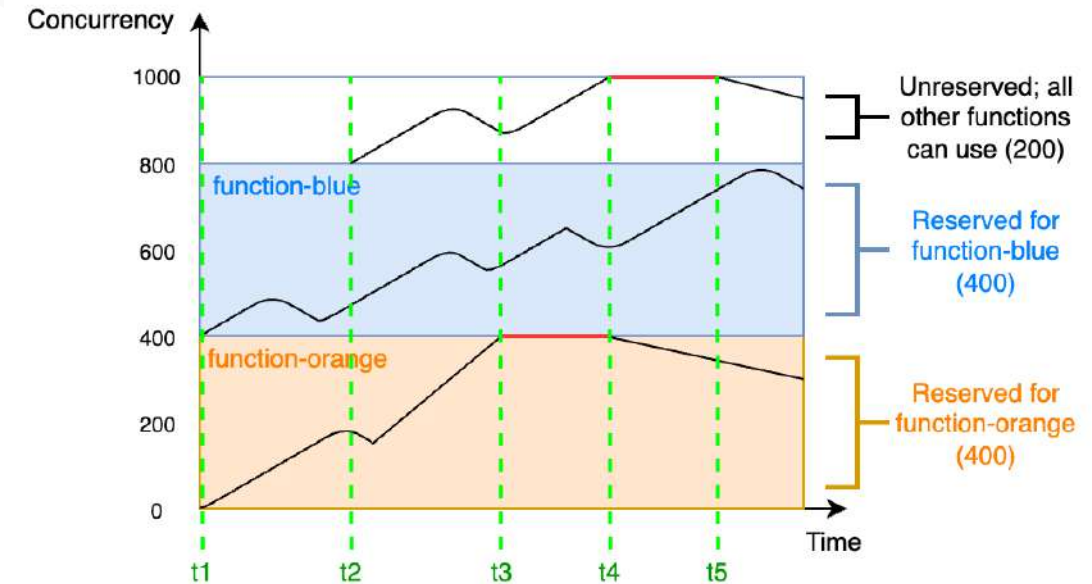
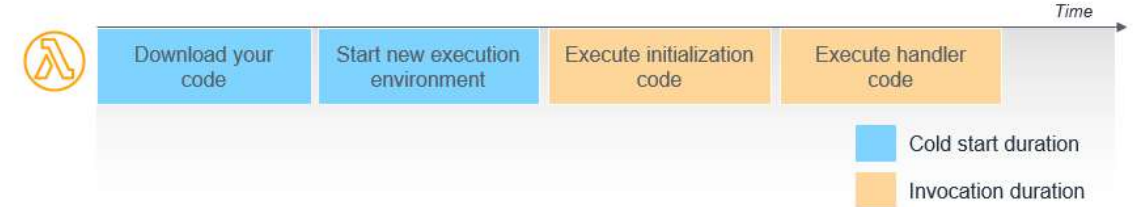


Image source: <https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>

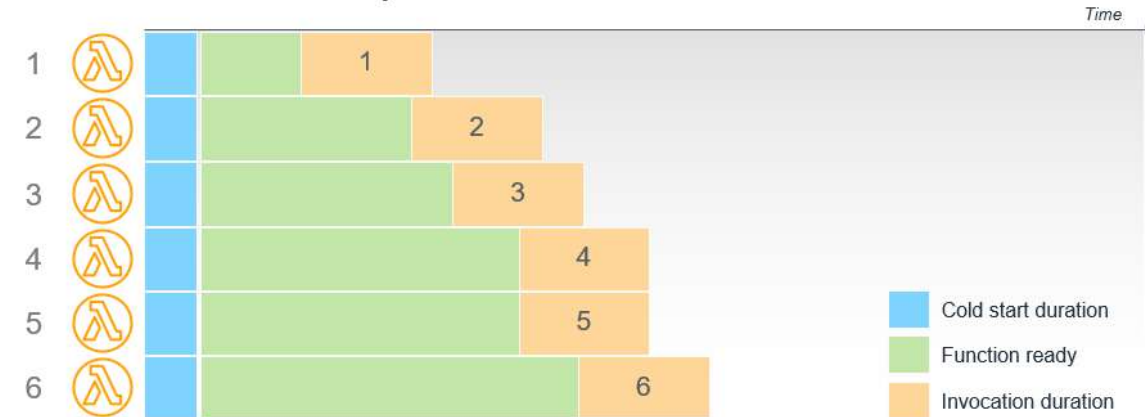


# Cold start and Provisioned Concurrency

- Before execution starts, Lambda has to download the code and initialize the environment
- This contributes to Cold start time for the function
- For latency sensitive executions, this might add considerable delay, and this cold start can be mitigated with Lambda **Provisioned Concurrency**.
- Provisioned Concurrency keeps the pre-initialized execution environments for the lambda function
- You pay for the Provisioned Concurrency even if there are no lambda functions currently consuming the capacity
- You can enable/disable Provisioned concurrency depending on expected load



Lambda Cold Start



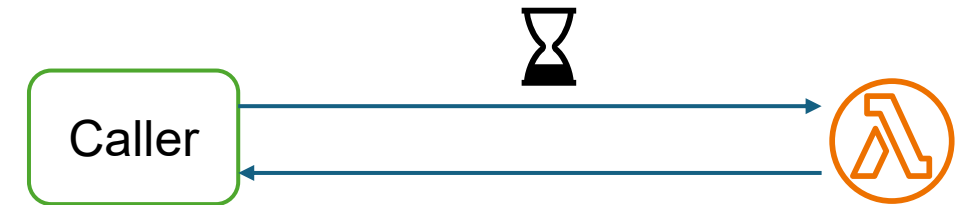
Example: Provisioned Concurrency of 6

**Remember:** Reserved Concurrency solves throttling issue, and Provisioned Concurrency solves Cold start issue

# Lambda Synchronous and Asynchronous invocation

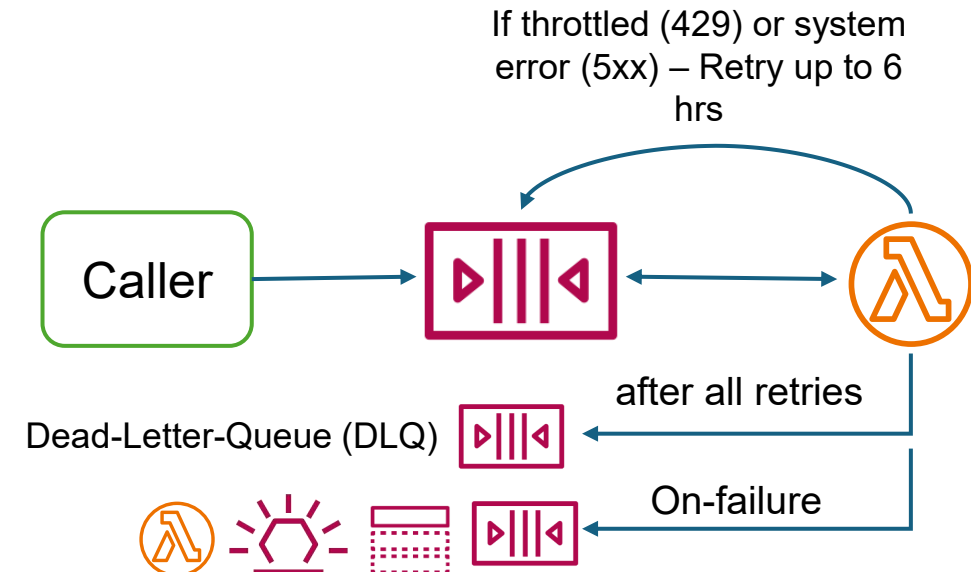
## Synchronous invocation

- Caller waits for the function to finish and receives the response immediately.
- On Lambda throttling, caller receives TooManyRequestsException (429) error
- API gateway, ALB makes synchronous lambda invocation
- **Use cases:** APIs, Real-time processing (auth, validation, lookups)

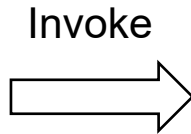


## Asynchronous invocation

- Lambda queues the event internally and execution happens later.
- On error, lambda retries 2 more times and then send to DLQ (SQS, SNS). You can also set failure targets.
- If error due to throttling (429) or system errors (5xx) then sends event back to the queue and retries for up to 6 hrs in the interval from 1 sec to 5 mins
- S3 Object events, SNS/EventBridge, DynamoDB streams, Kinesis streams etc. makes asynchronous lambda invocation
- **Use cases:** Event processing, data processing, order processing

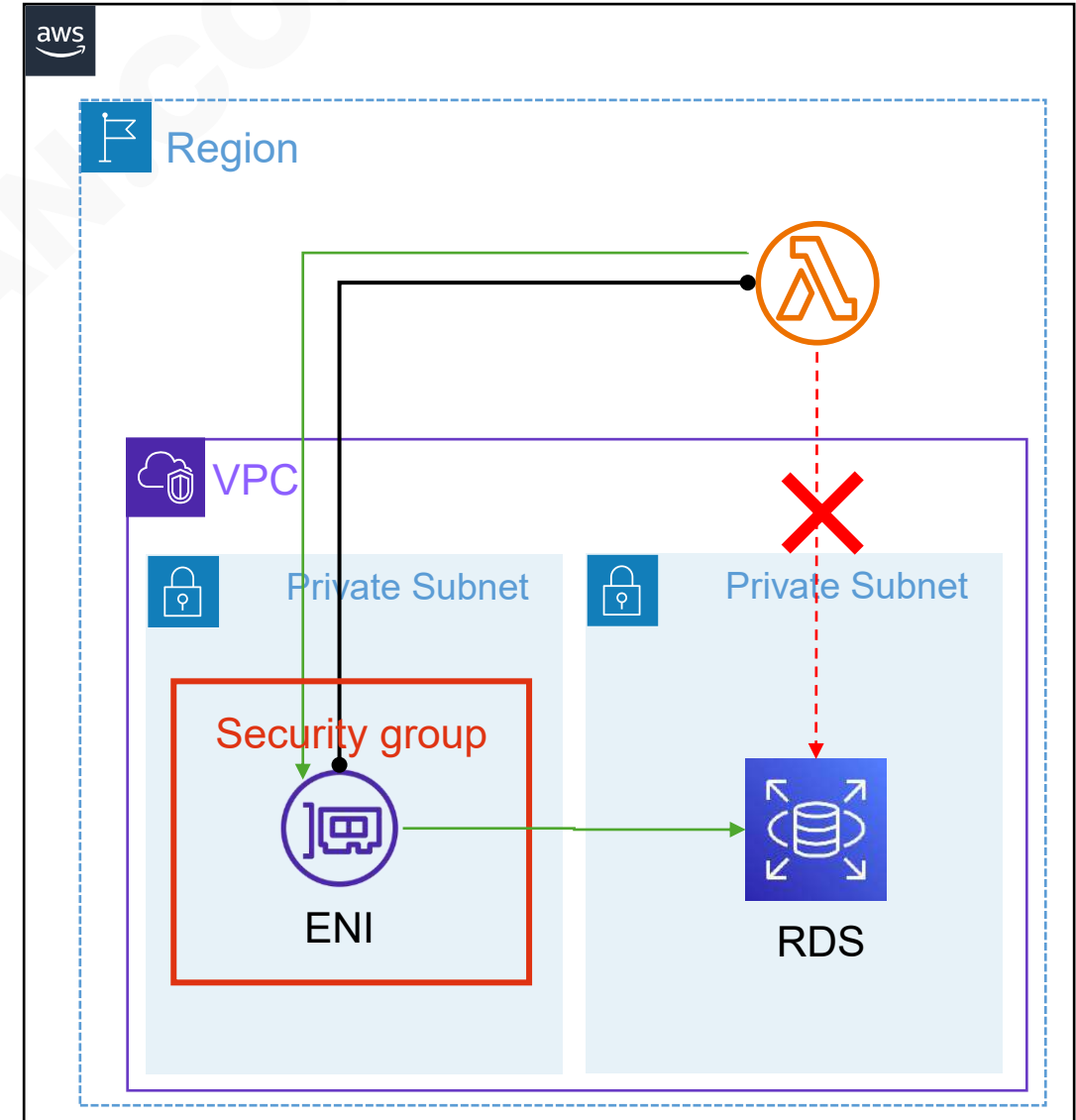


- ```
graph LR; A[Execution environment] --> B[Download Code]; B --> C[Start Runtime]; C --> D[Initialize Function code]; D --> E[Create MircoVM Snapshot]; style E fill:#90EE90
```



# Lambda in VPC

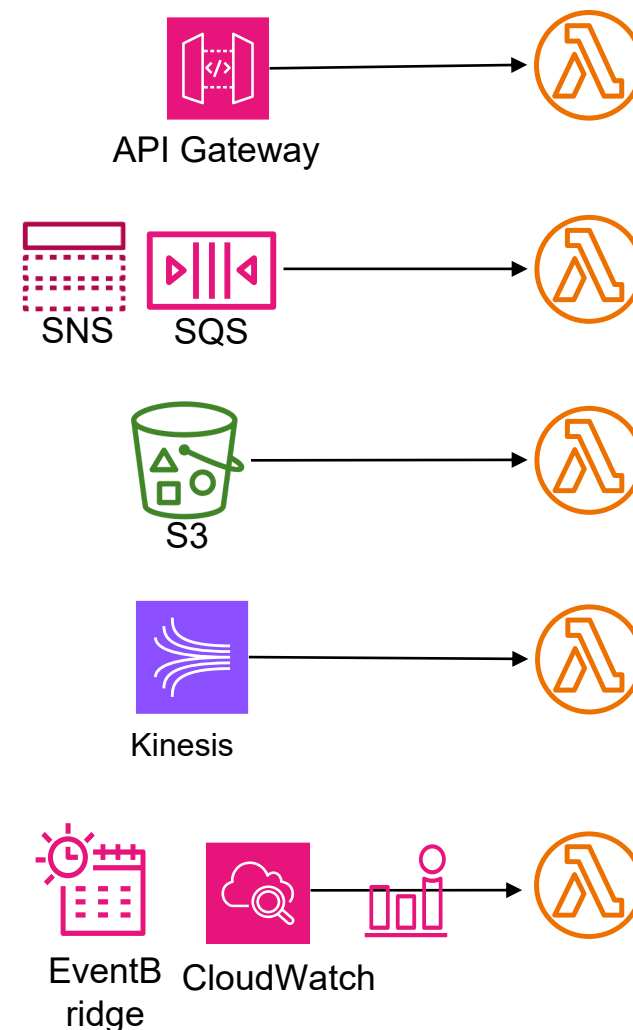
- Lambda function (by default) is provisioned outside of VPC
- It can not access Private resources inside a VPC e.g. Databases, EC2 Private IPs, Internal ALB etc.
- You can create Lambda function inside VPC
- This creates an ENI in the VPC subnet through which it can access other resources inside VPC
- All other VPC controls apply to ENI e.g. Security Group, subnet route table etc.
- To allow Lambda to access internet, you must provision NAT gateway and configure route for Lambda subnet
- **Use case:** Lambda need to access RDS database, ElastiCache or OpenSearch service inside VPC



# Lambda patterns and Use Cases

# Lambda common patterns

1. Execute on each API request - Backend for API gateway or Elastic load balancer
2. Message processing – Read SQS messages as they arrive in the queue
3. S3 event processing – Trigger lambda when Object is uploaded to S3 bucket
4. Real-time data processing – Read data from Kinesis data stream and process in real-time
5. Event processing – EventBridge rule or schedule, CloudWatch alarm etc.



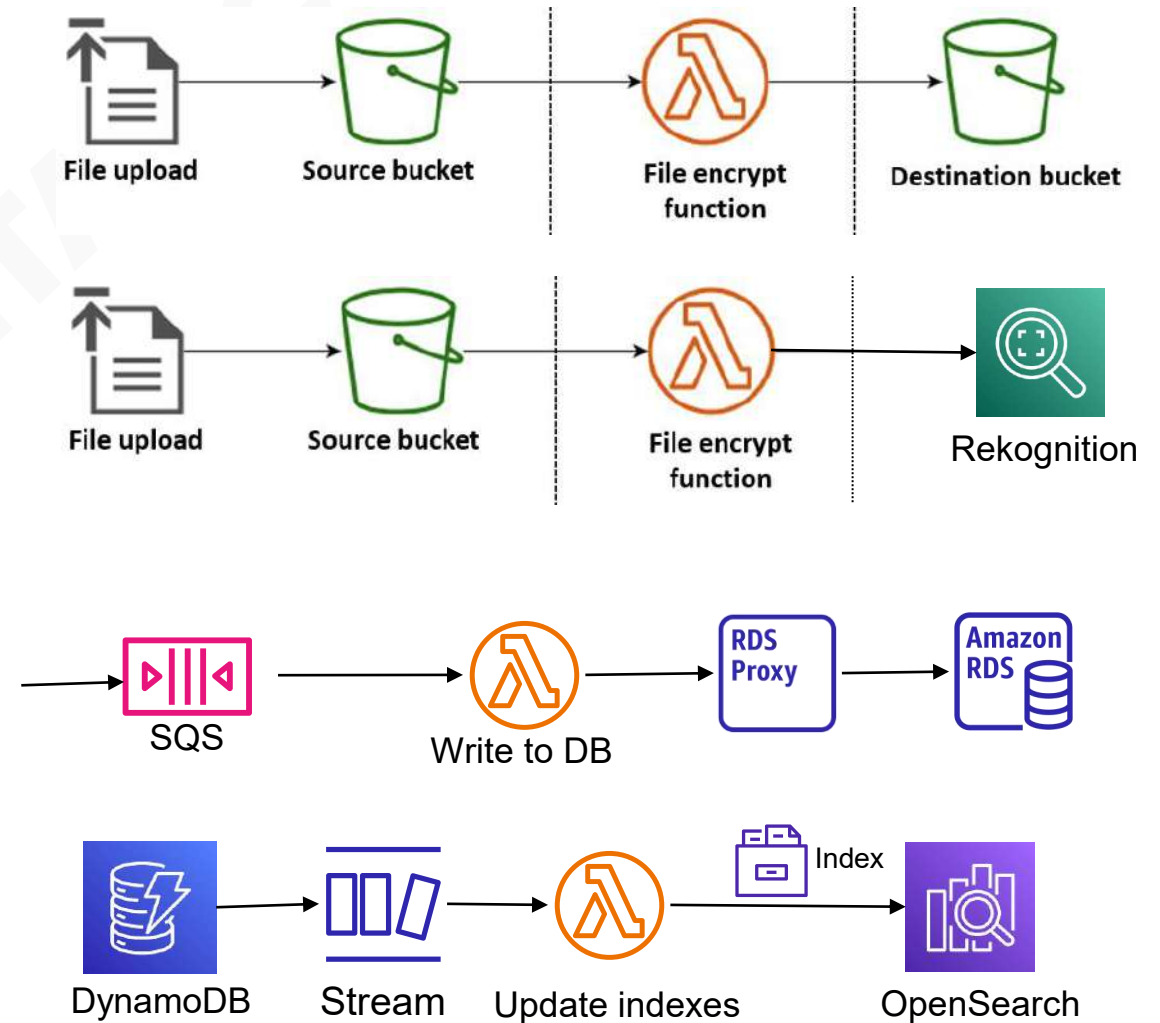
# Lambda common Use Cases

- **File Processing Apps**

- PDF encryption
- Image analysis

- **Database Integration**

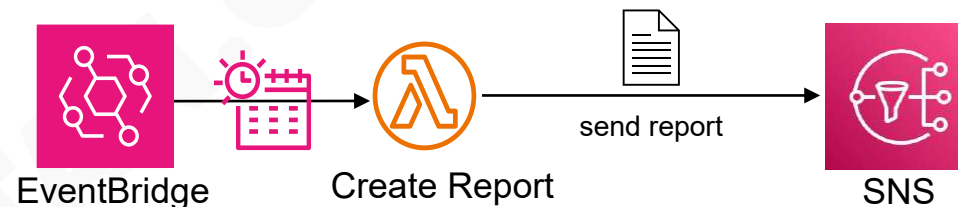
- Asynchronous writes to Database
- Database Event handler



# Lambda common Use Cases

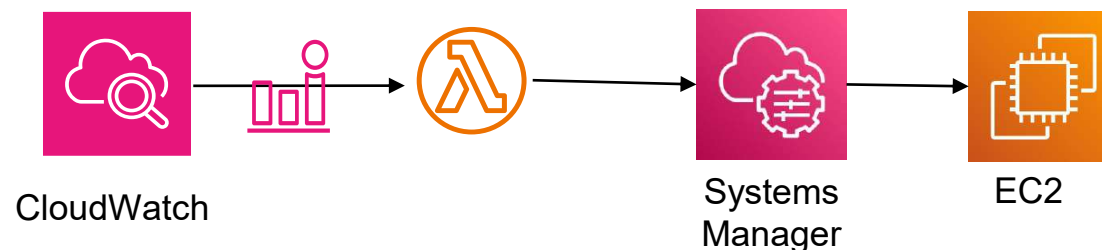
- **Scheduled Tasks**

- Creating and sending reports
- Trigger a maintenance activity



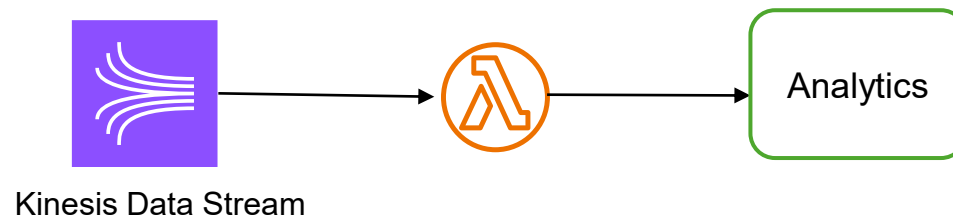
- **Incidence response**

- Trigger action based on CloudWatch Alarms



- **Real-time stream processing**

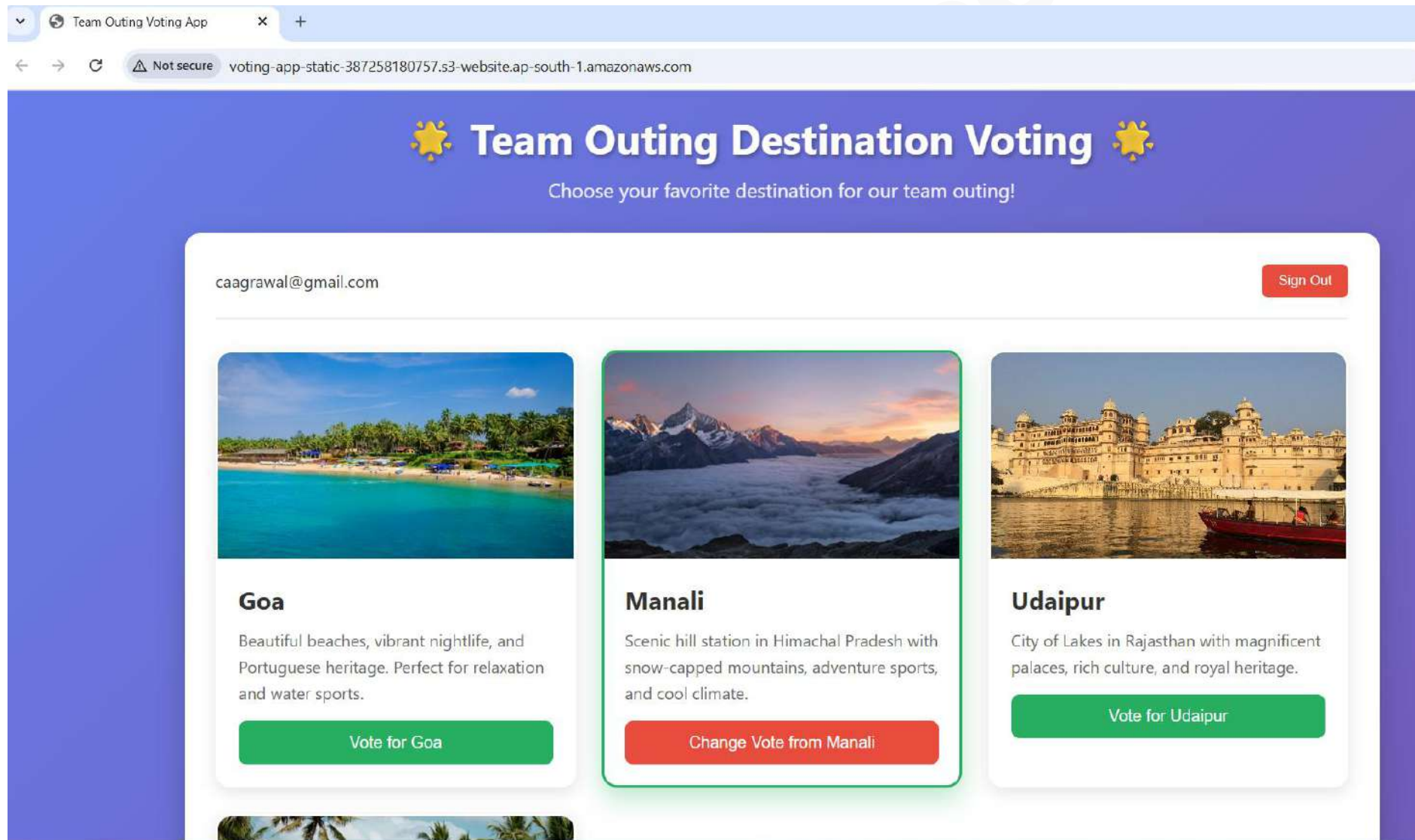
- Click-stream Analytics
- IoT Sensor data processing





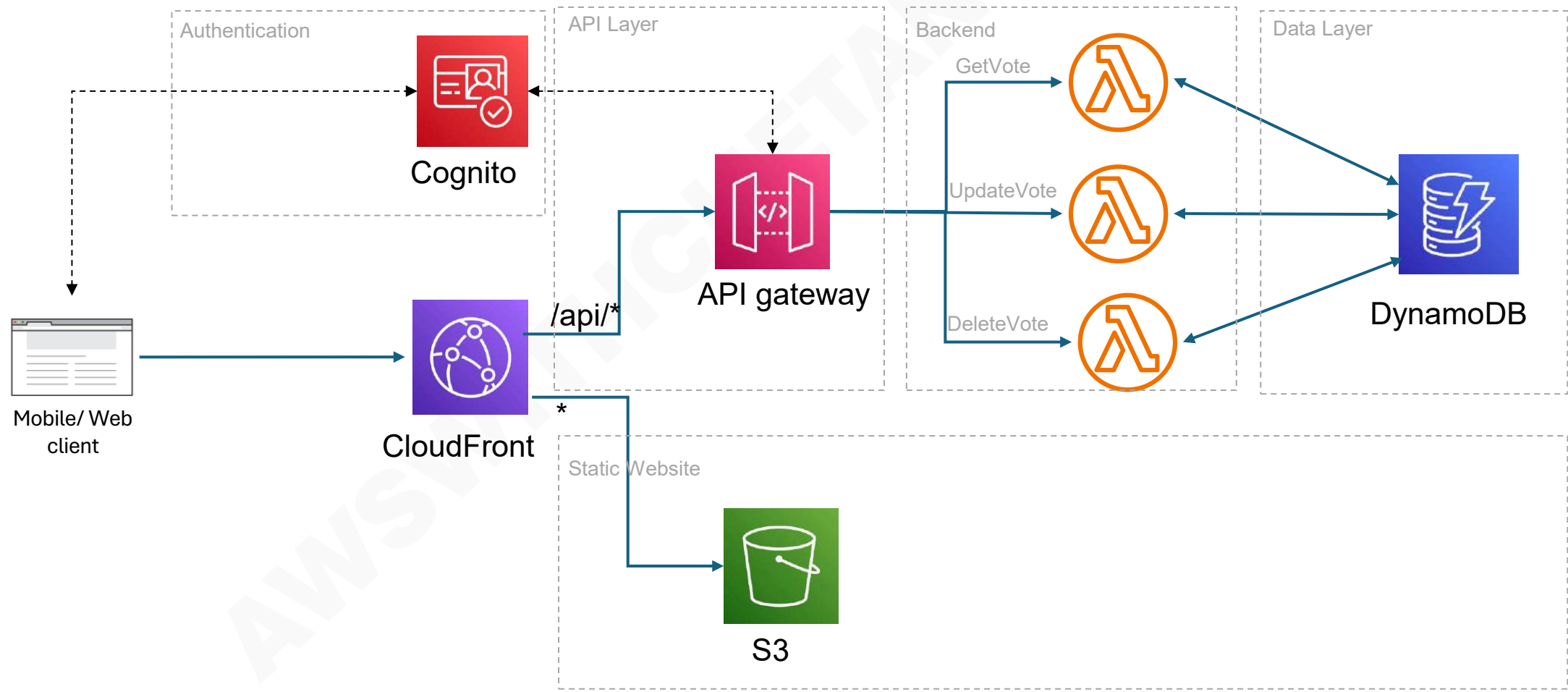
Let's talk architecture

# Serverless voting app



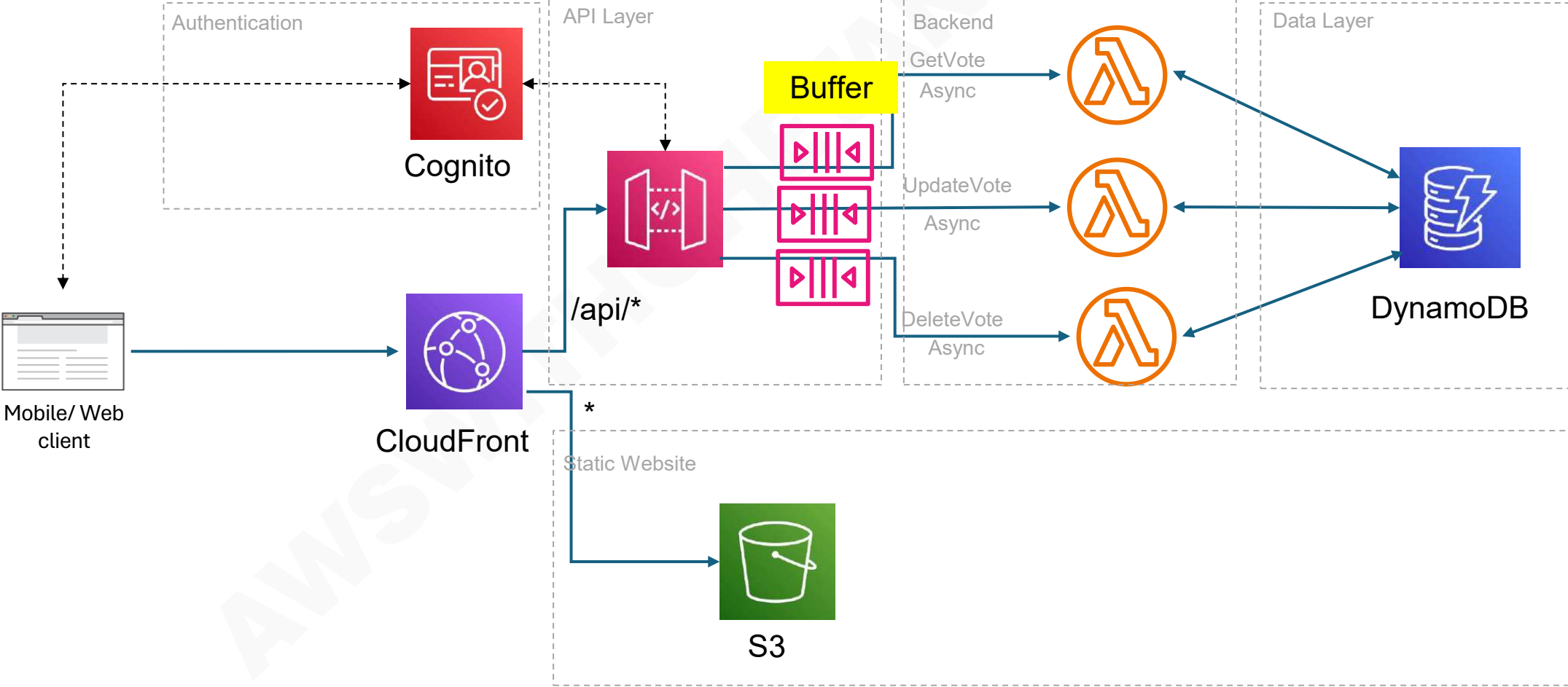
# Architecture

Real-time (synchronous)



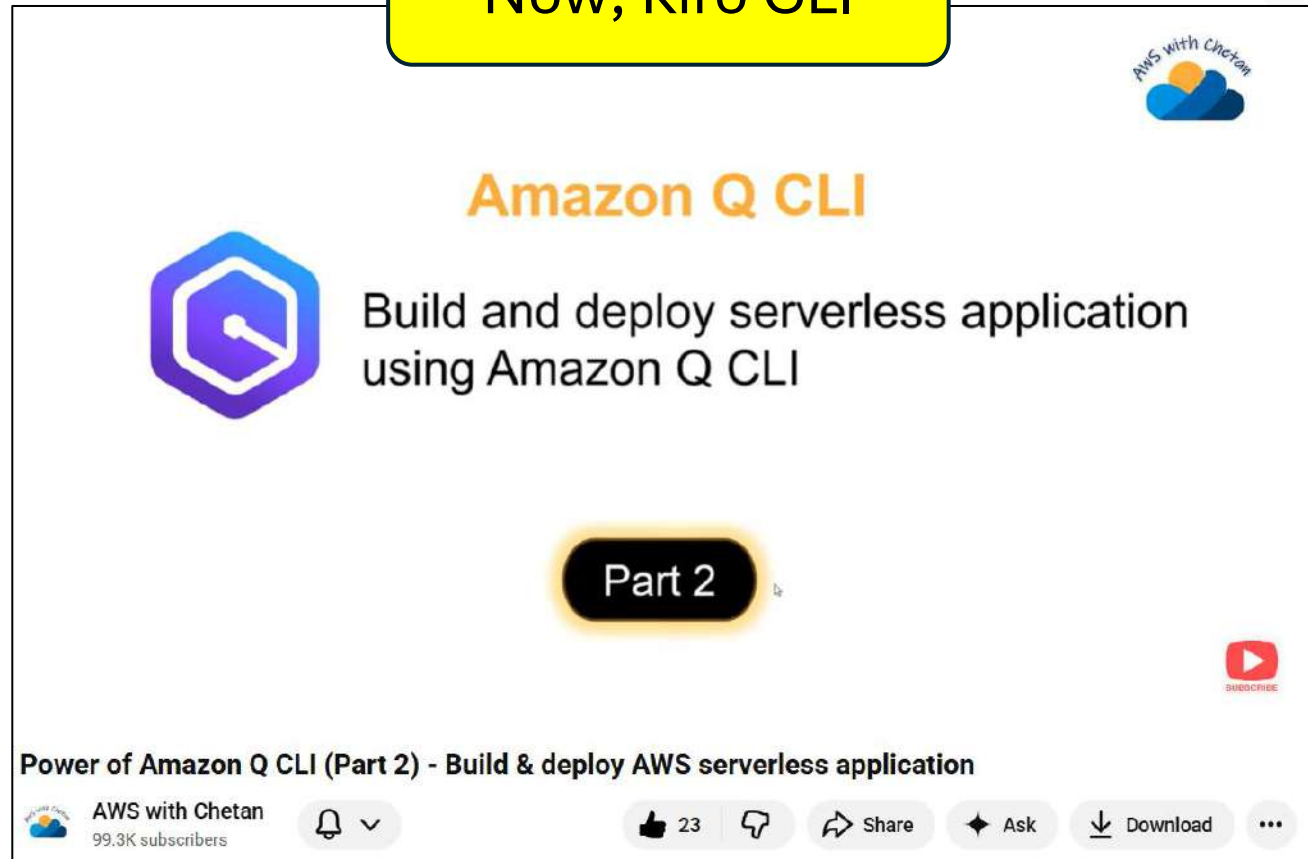
# Architecture

Event based (Asynchronous)



# Watch video..

Now, Kiro CLI



[https://youtu.be/oy\\_yHDCuabl](https://youtu.be/oy_yHDCuabl)

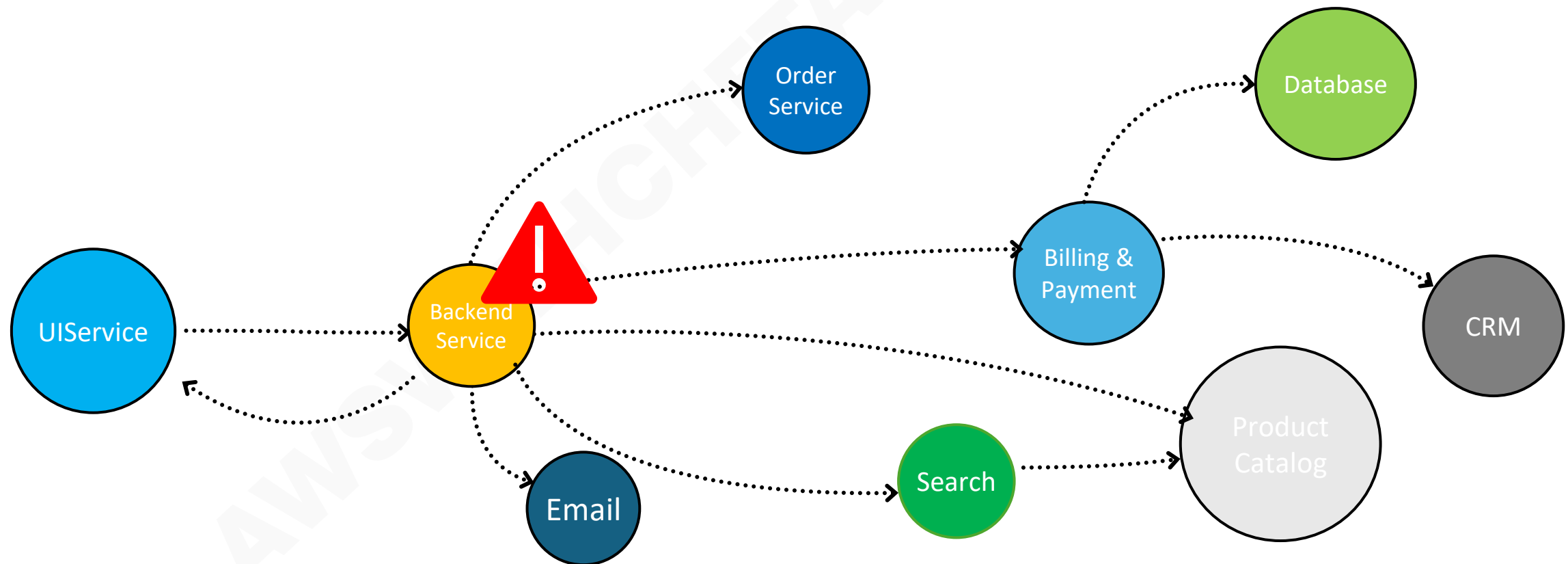


# AWS Integration Services

Amazon SQS, SNS and EventBridge

# Why do we need Integration services?

- Modern applications rely on many services working together and how these services communicate determines how reliable and scalable the system is.



# Why do we need Integration services?

- Modern applications rely on many services working together and how these services communicate determines how reliable and scalable the system is.

## **Solution: Loosely coupled systems**

- Enable asynchronous communication
- Allow individual services to scale independently
- Absorb sudden traffic spikes
- Prevent failures in one service from impacting others



# Synchronous vs Asynchronous communication

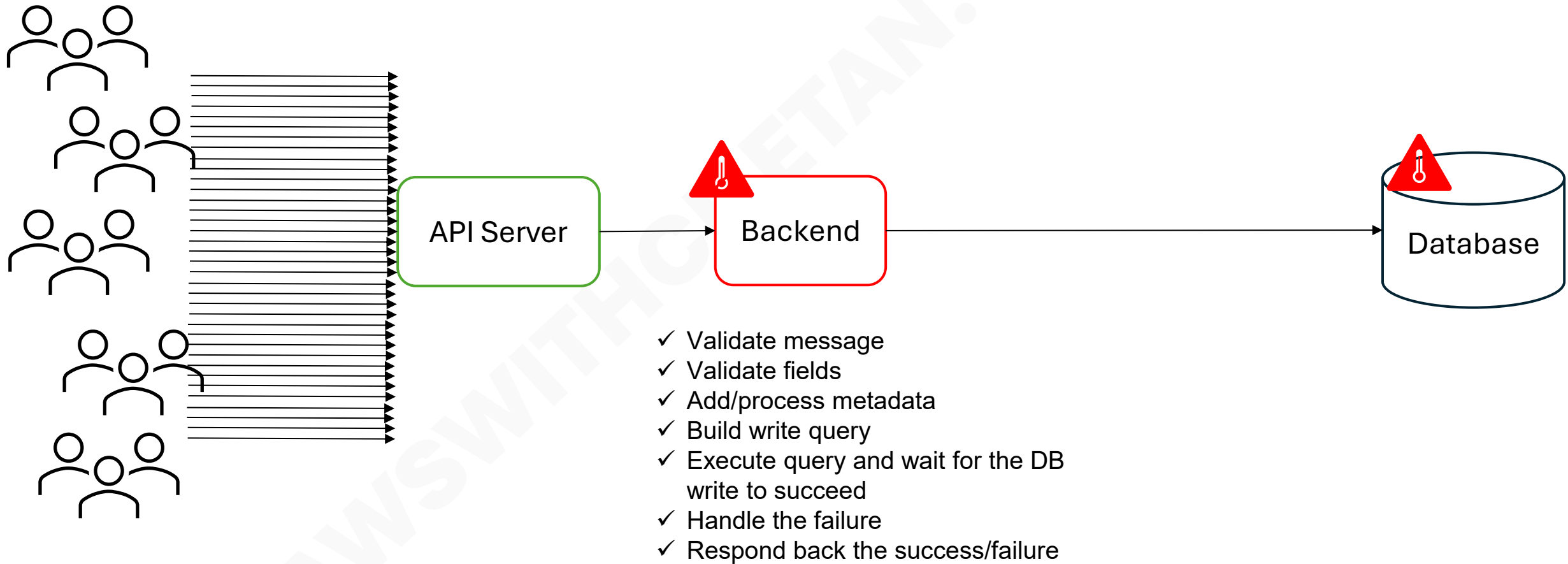
## Synchronous communication



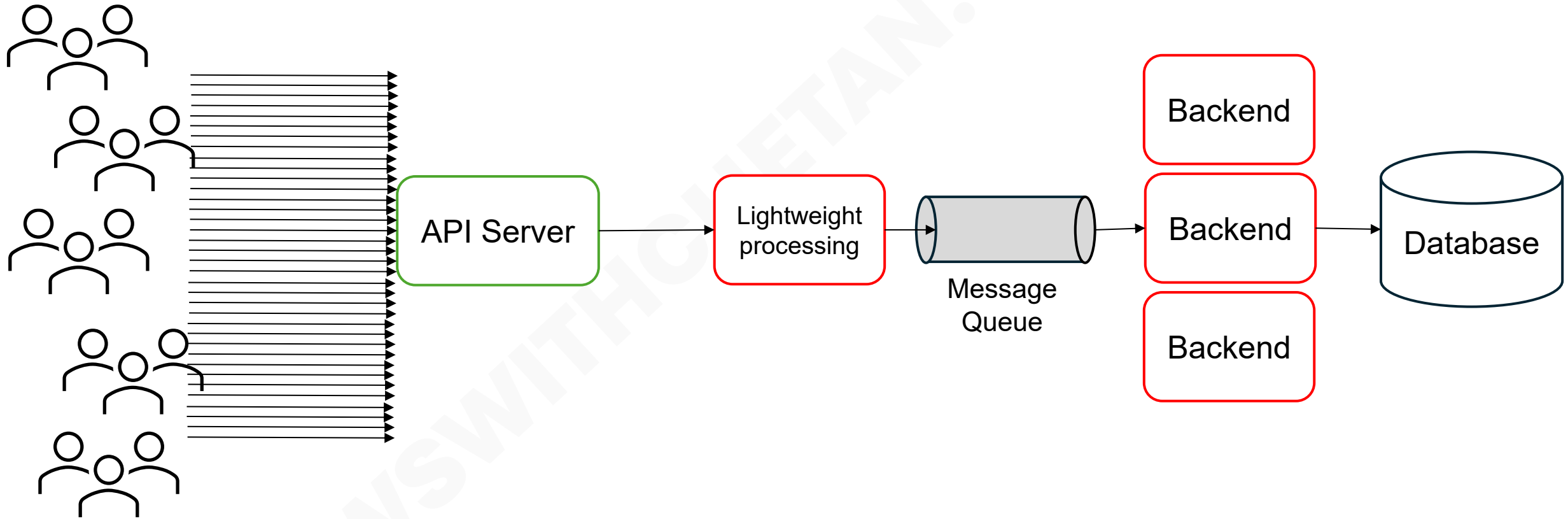
## Asynchronous communication



# A voting app..



# A voting app..



# Application Integration services



## Simple Queue Service (SQS)

Managed message queue service



## Simple Notification Service (SNS)

Notification service which delivers SMS, Email, Mobile Push notifications



## Amazon EventBridge

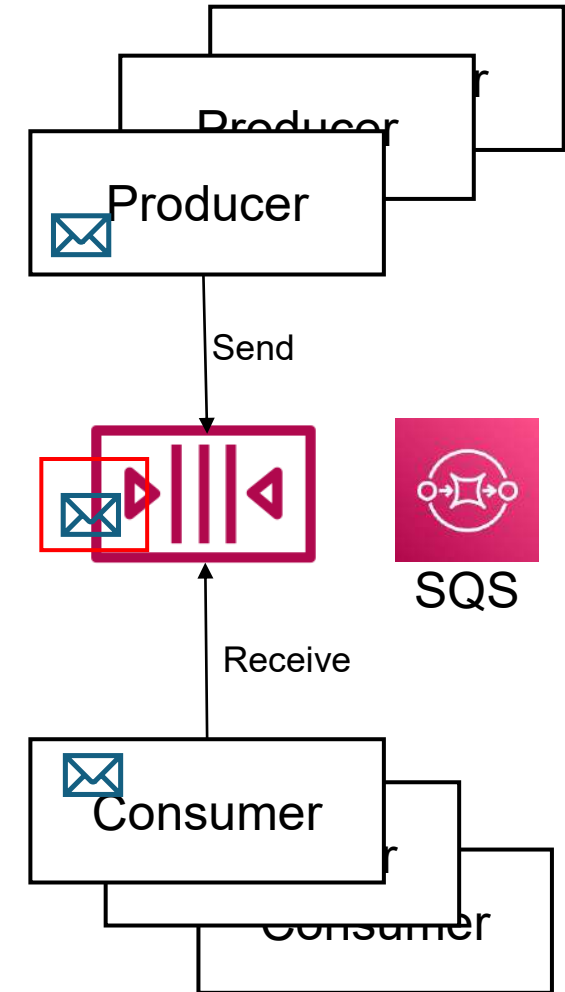
Event bus enabling integration between AWS services, SaaS and your applications

# Amazon SQS - Simple Queue Service

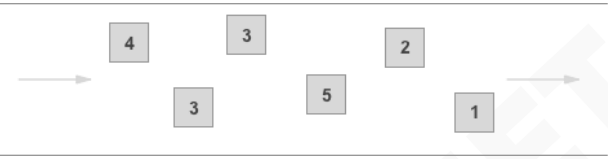
- Amazon SQS is a Highly available distributed message queue system.
- SQS supports two type of queues: **Standard** and **FIFO**
- SQS supports multiple producers (writers) and consumers (readers) for the same queue.

## How it works?

- Producers send message to the queue.
- Message is stored in the queue for the duration defined by retention period.
- Consumers request message from the queue.
- Request for message locks the messages during the processing by the consumer application (Visibility timeout).
- After processing consumer deletes the message (DeleteMessage api).
- If not deleted, messages will stay in the queue until the retention period.



# Standard Queue vs FIFO Queue

|                  | Standard Queue                                                                      | FIFO Queue                                                                          |
|------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Structure        |   |  |
| Throughput       | Nearly unlimited API requests per second.                                           | Up to ~300 TPS without batching (up to ~3,000 with batching).                       |
| Delivery         | At-least-once — messages might be delivered more than once.                         | Exactly-once processing — no duplicates.                                            |
| Message Ordering | Best-effort ordering (not guaranteed).                                              | Strict FIFO (first-in, first-out) within groups.                                    |
| Cost             | Lower                                                                               | Higher                                                                              |
| When to use?     | For high throughput where occasional duplicates or slight reordering is acceptable. | Critical workflows needing exact order and deduplication.                           |

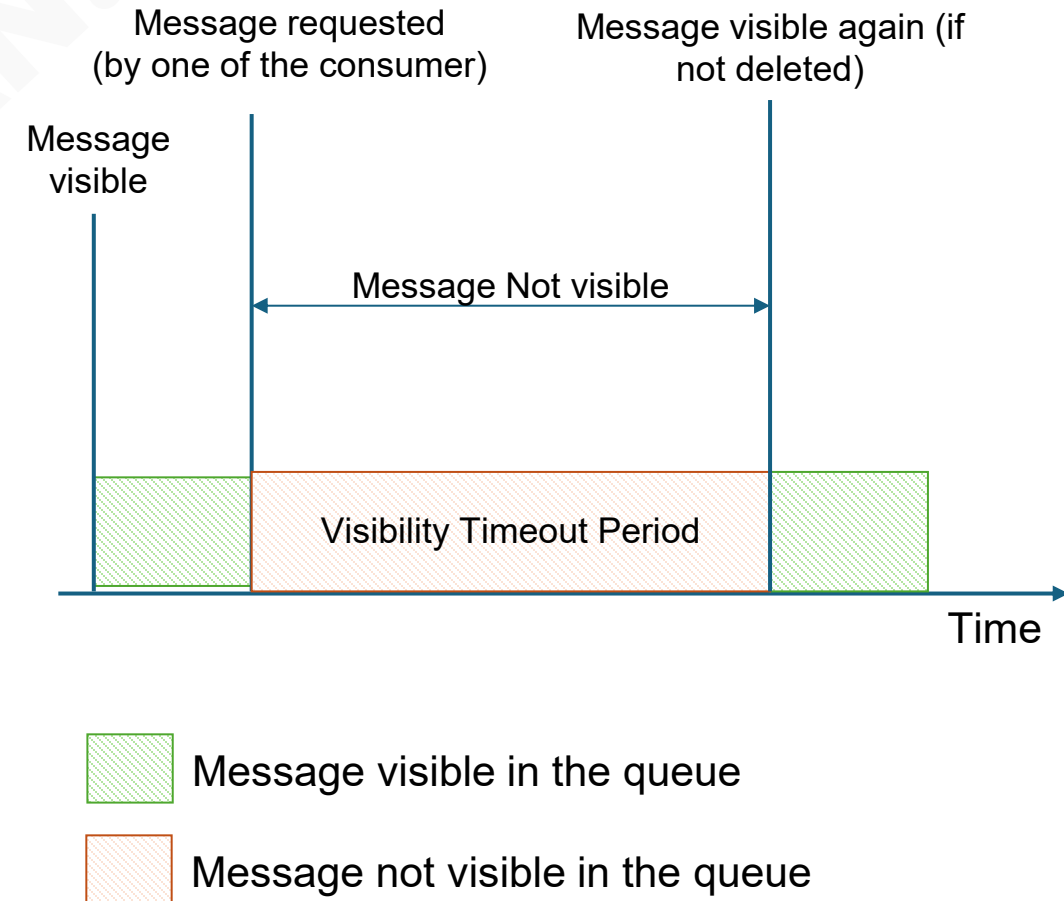
# Amazon SQS - Features

- Message Visibility Timeout
- Message Retention and Size
- Batch Operations
- Long polling
- Delay Queue
- Dead Letter Queue (DLQ)

# Amazon SQS – Features

## Message Visibility Timeout

- When a consumer requests a message from SQS, a message becomes temporarily invisible to other consumers
- Default visibility timeout = 30 seconds, can be set 0 sec-12 hrs.
- Ideally message should be deleted after processed by the consumer.
- If message is not processed within Visibility timeout period, it becomes visible again and can be picked up by other consumers.
- A consumer can extend the visibility timeout time by calling the **ChangeMessageVisibility** API.
- If the visibility timeout is set too high and the consumer crashes, message re-processing will be delayed.
- If the visibility timeout is set too low, messages may be delivered multiple times, increasing the chance of duplicates.

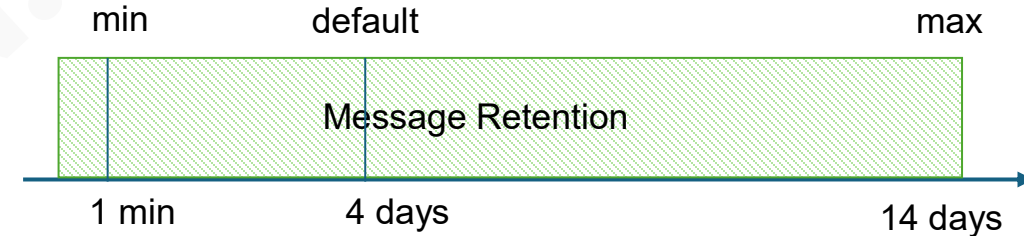




# Amazon SQS – Features

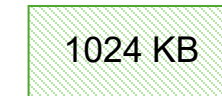
## Message Retention

- Messages can be retained in SQS queue for 1 min to 14 days (default 4 days)



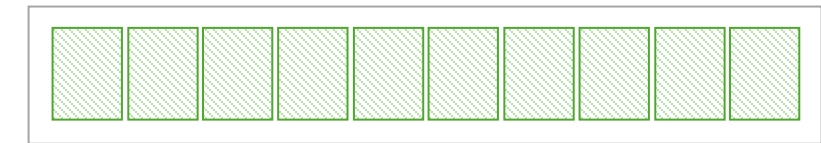
## Message Size

- Up to 1024 KB



## Batch Operations

- Amazon SQS batch actions allow you to send, receive, or delete up to 10 messages in a single API call.
- Reduces SQS API costs and improves application performance.
- Must use AWS APIs or SDK to use batch operations (SendMessageBatch, DeleteMessageBatch API)

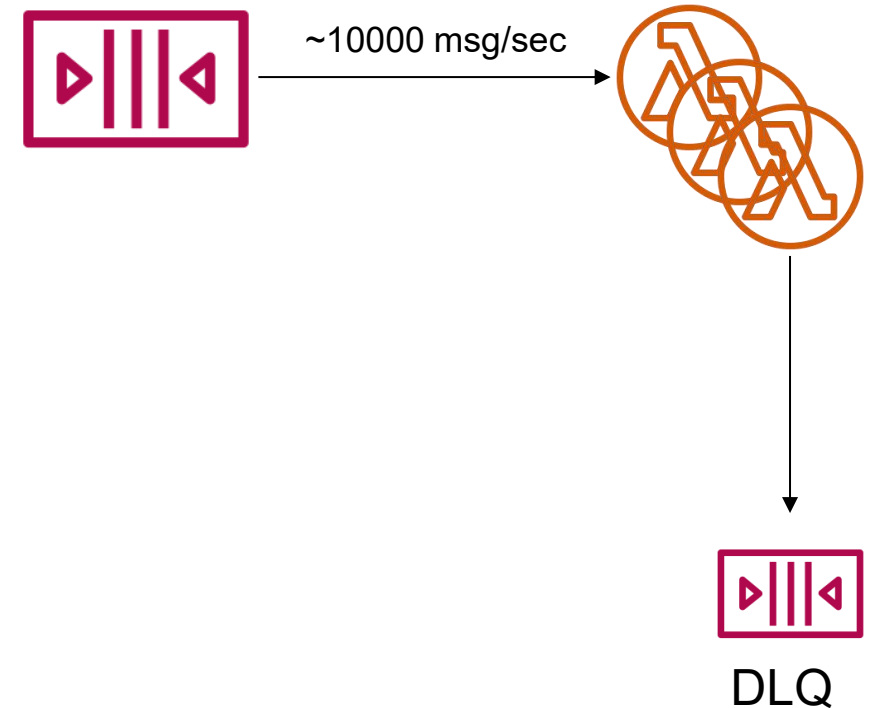


Up to 10 messages in single API call

# Amazon SQS – Features

## Lambda with SQS

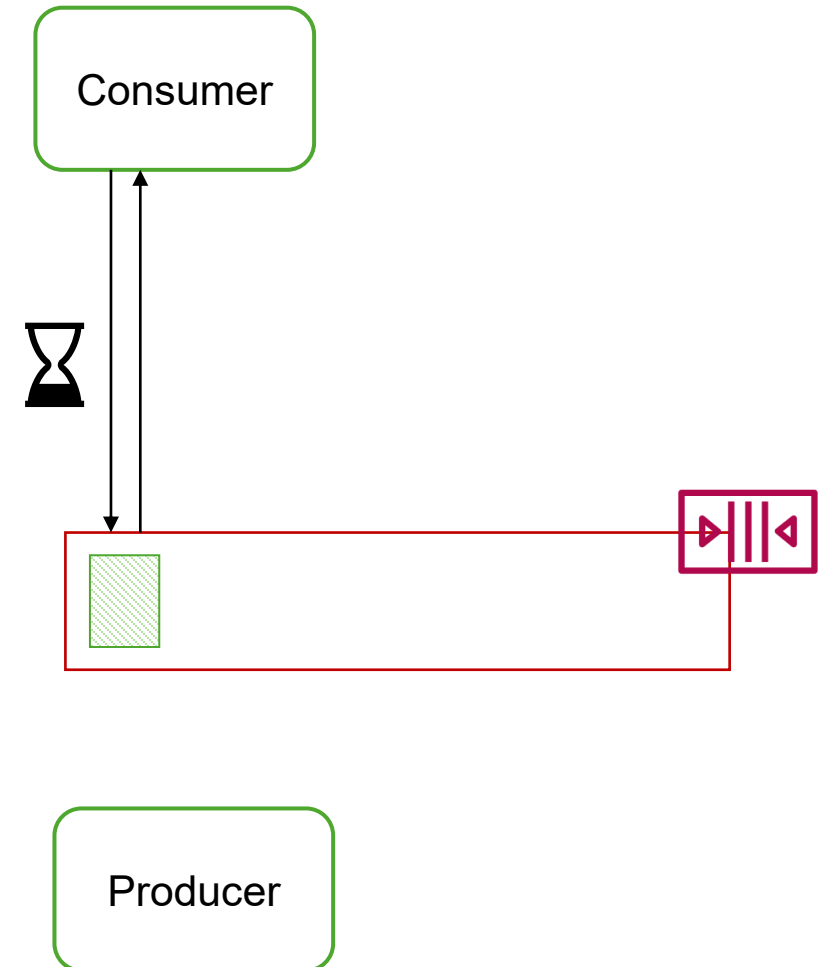
- Lambda uses Event source mapping to poll the messages from the SQS queue.
- No need to write polling code/logic.
- Event source mapping defines how many messages Lambda receives per invocation (batch size 1–10).
- Lambda scales concurrency based on queue depth
- Lambda deletes messages from the queue only after successful processing
- Supports partial batch response, allowing only failed messages to be retried.
- Works with both Standard and FIFO queues.



# Amazon SQS – Features

## Long Polling

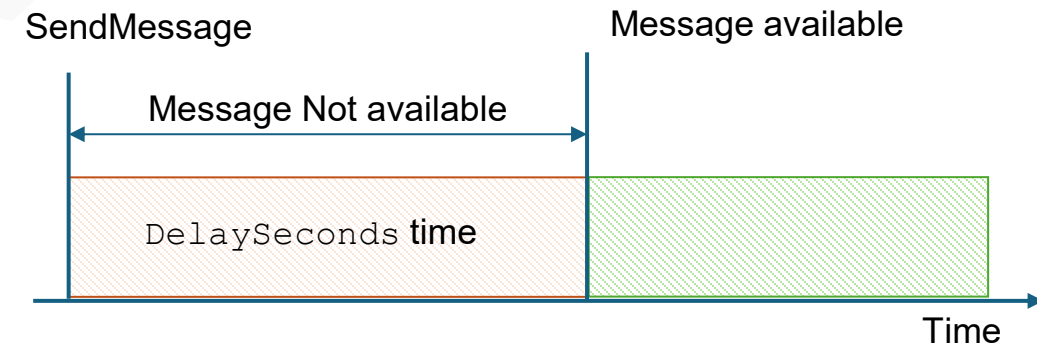
- SQS waits **up to 20 seconds** for a message instead of returning immediately with “no messages”.
- Reduces the number of SQS API calls, thereby lowers API cost
- Reduces application latency by returning the message as soon as it's available in the queue.
- Can be configured at Standard or FIFO queue level or API level (using WaitTimeSeconds parameter)



# Amazon SQS – Features

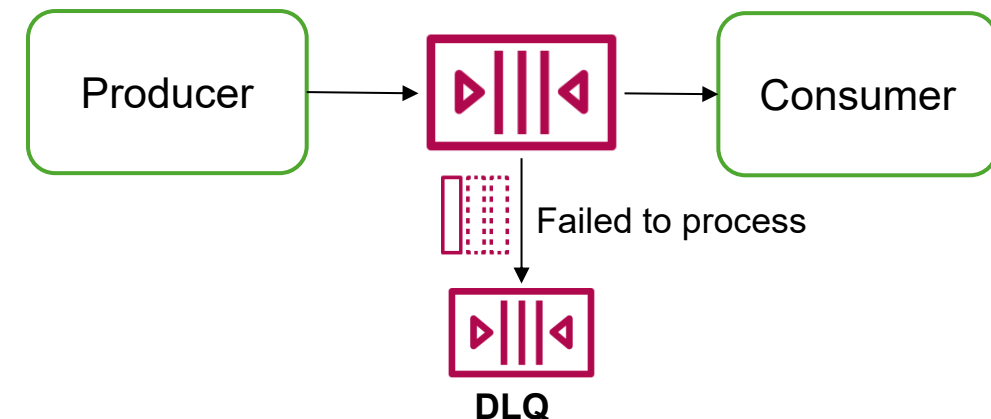
## Delay queues

- A delay queue postpones message delivery for a specified time.
- Delay can be set from 0 to 15 minutes.
- Producers send messages normally, but consumers cannot see them until the delay expires.
- Delay can be applied per queue or per message (using `DelaySeconds` parameter).
- **Use case:** Undo option after you click the Send button while sending an email



## Dead Letter Queue (DLQ)

- A DLQ stores messages that repeatedly fail to be processed.
- You configure a Maximum Receives (e.g., 5 attempts).
- After exceeding this count, the message is moved to the DLQ.
- Prevents “poison messages” from retrying forever.
- Useful to troubleshoot problematic payloads.



# Amazon SQS – Security

## Encryption in Transit

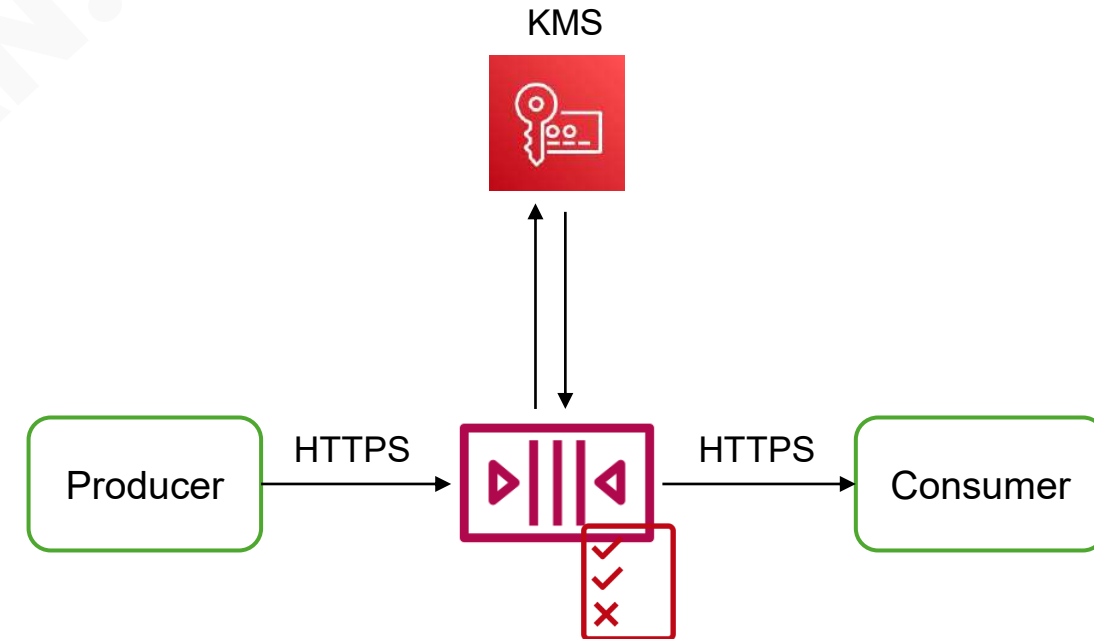
- All Amazon SQS API calls are secured using HTTPS, protecting data while it moves between producers, SQS, and consumers.

## Encryption at Rest

- SSE-SQS: Uses an AWS-managed SQS key with automatic management and rotation.
- SSE-KMS: Uses a customer-managed AWS KMS key for more control, auditing, and custom key policies.

## SQS Queue IAM Policy

- Controls who can access the queue and what actions they can perform (`SendMessage`, `ReceiveMessage`, `DeleteMessage` etc.).
- Used to allow cross-account access as well as access by other AWS services e.g. Simple Notification Service (SNS)



# Sample queue policies

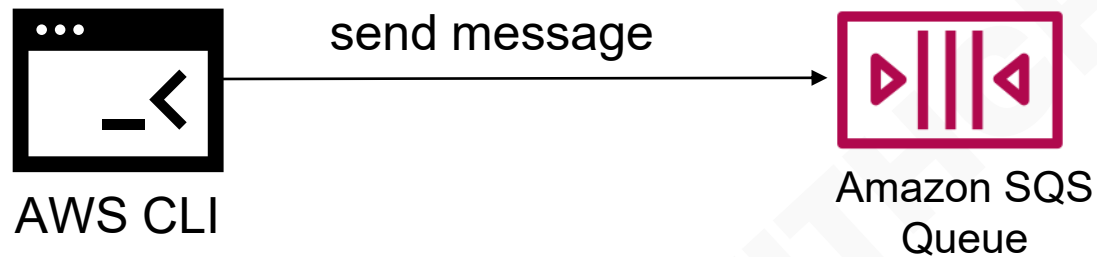
## Allow cross-account access

```
{
  "Version": "2012-10-17",
  "Id": "CrossAccountRoleAccess",
  "Statement": [
    {
      "Sid": "AllowRoleFromAnotherAccount",
      "Effect": "Allow",
      "Principal": {
        "AWS":
"arn:aws:iam::222222222222:role/ExternalAppRole"
      },
      "Action": [
        "sqs:SendMessage",
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage"
      ],
      "Resource": "arn:aws:sqs:us-east-1:111111111111:MyQueue"
    }
  ]
}
```

## Allow SNS to publish message

```
{
  "Version": "2012-10-17",
  "Id": "CrossAccountSNSPublish",
  "Statement": [
    {
      "Sid": "AllowCrossAccountSNSToPublish",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:us-east-1:111111111111:MyQueue",
      "Condition": {
        "ArnEquals": {
          "aws:SourceArn": "arn:aws:sns:us-east-1:111111111111:MySNSTopic"
        }
      }
    }
  ]
}
```

# Exercise – Send message to SQS queue

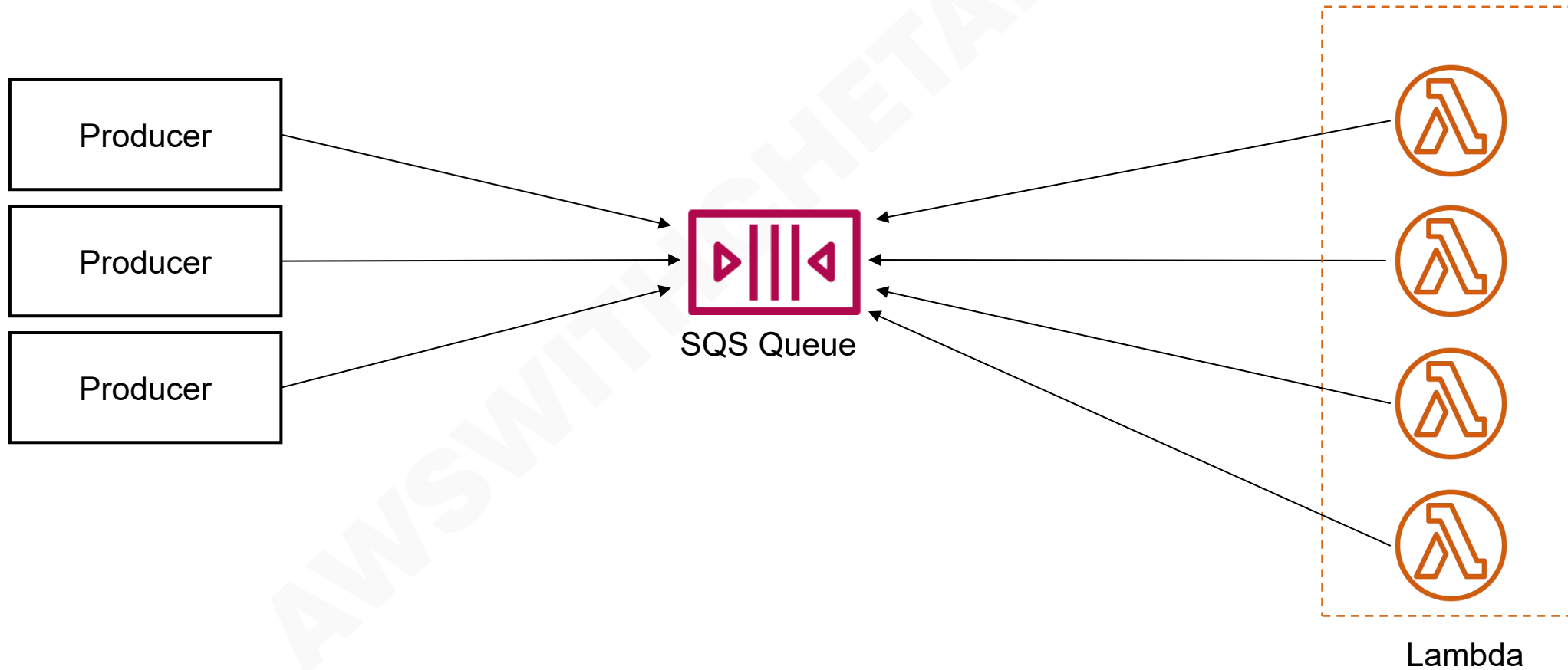


- 1 Create SQS queue (standard queue)
- 2 Use AWS CLI to Send message to the queue
- 3 From SQS console, verify if you see the messages

```
$aws sqs send-message --queue-url <QUEUE_URL> --message-body "Hello, this is test message 1"
```

# SQS architecture patterns

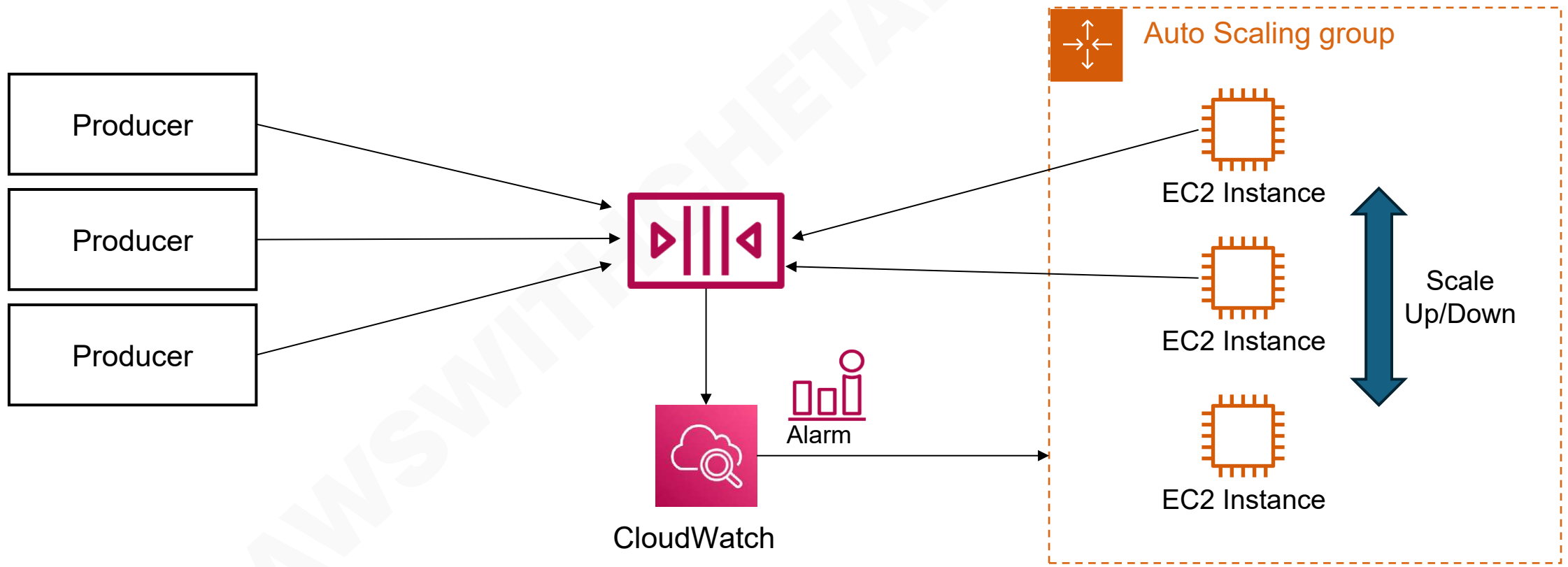
- Automatic scaling of Lambda functions based on SQS queue depth (# messages)





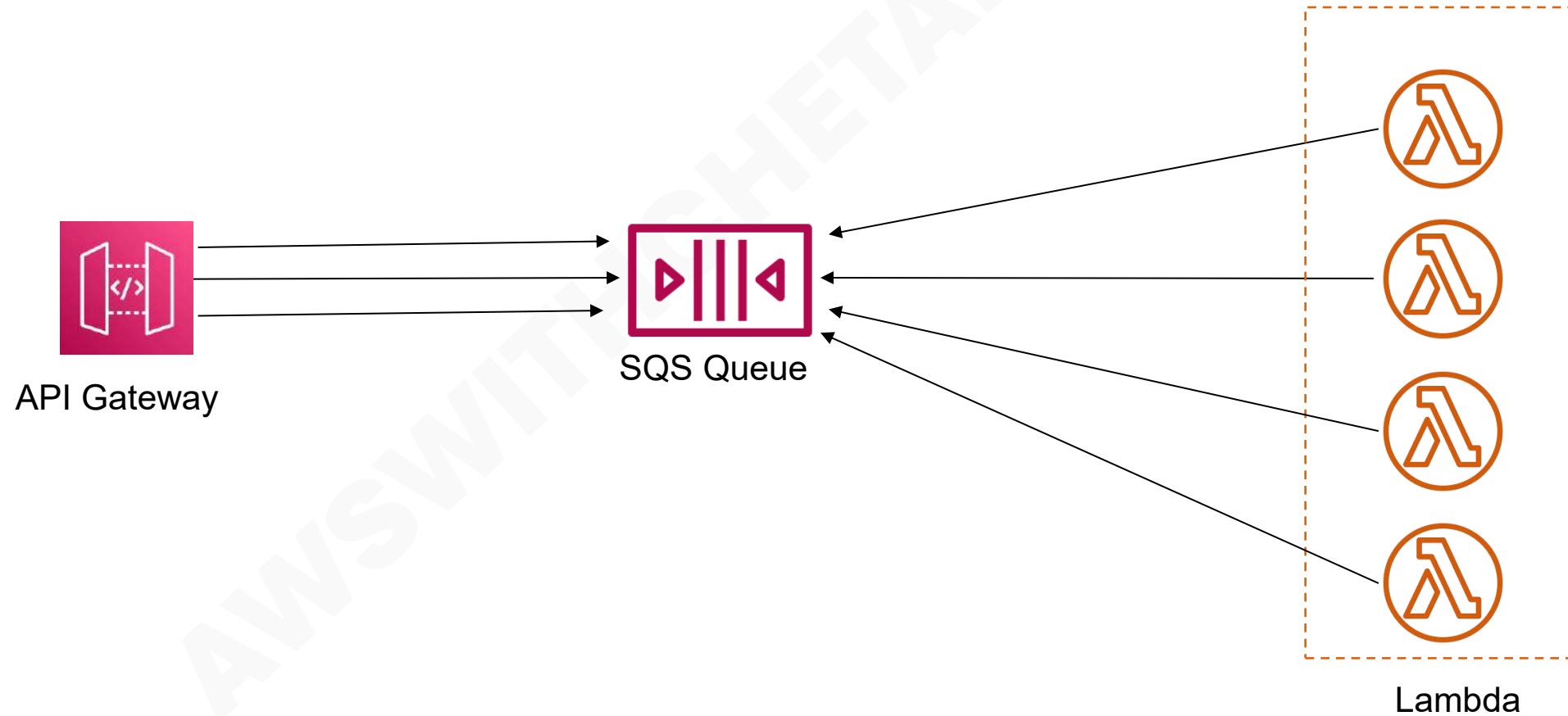
# SQS architecture patterns

- Scaling EC2 instances based on SQS Queue depth (#messages)



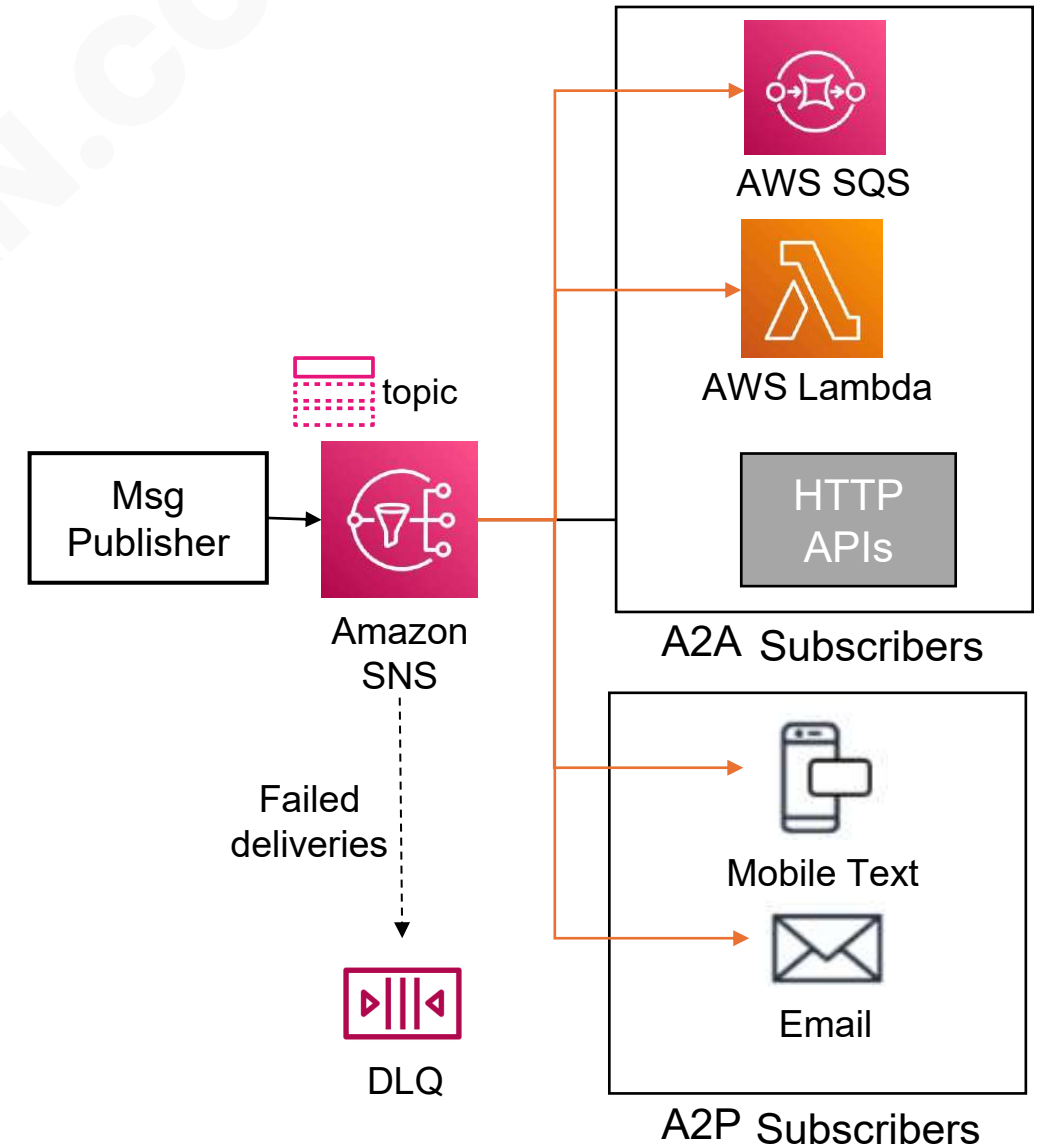
# SQS architecture patterns

- SQS as a buffer to process spiky requests received through API Gateway



# Amazon SNS - Simple Notification Service

- Amazon SNS is a Pub/Sub notification service
- SNS enables application-to-application (A2A) and Application-to-Person (A2P) messaging.
- Decouples publishers and subscribers.
- Ideal for microservices, distributed, and serverless apps.
- Supports dead-letter queue (DLQ) for troubleshooting or reprocessing of the failed deliveries
- SNS supports Standard Topic and FIFO Topic



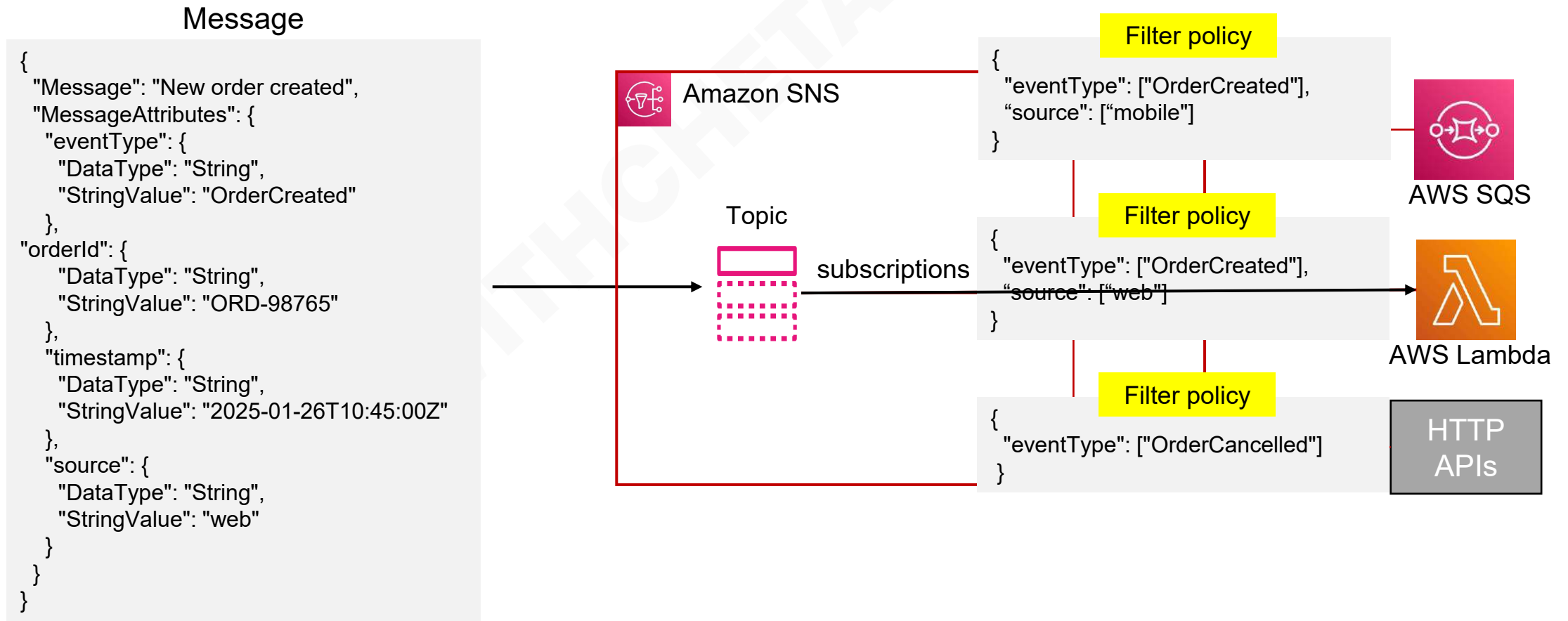
# Amazon SNS – Topic types

You can create Standard Topic or FIFO Topic (like SQS)

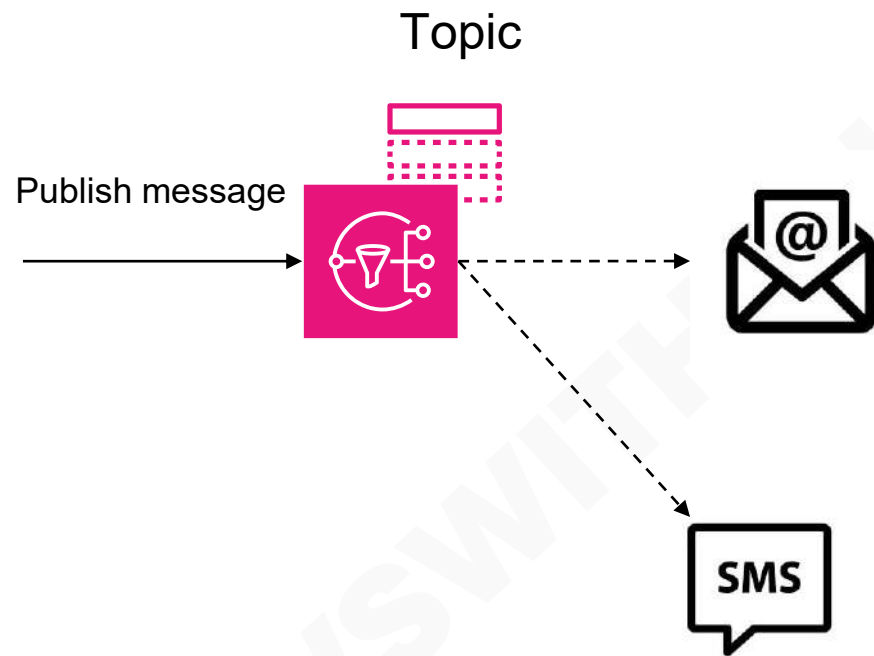
|                         | Standard Topic                                                     | FIFO Topic                                                |
|-------------------------|--------------------------------------------------------------------|-----------------------------------------------------------|
| Message ordering        | Best efforts basis (not guaranteed)                                | Strictly first-in-first-out                               |
| Throughput              | Nearly unlimited                                                   | ~300 messages/sec or 10MB/sec                             |
| Delivery                | At-least-once (messages might be delivered more than once.)        | Exactly-once (No duplicates).                             |
| Supported subscriptions | SQS, Kinesis Data Firehose, Lambda, HTTPS, SMS, Email, Mobile push | Only FIFO SQS queue                                       |
| Cost                    | Lower                                                              | Higher                                                    |
| When to use?            | Fan-out where ordering and duplicates doesn't create a problem.    | Critical workflows needing exact order and deduplication. |

# Amazon SNS – Message filtering

- By default, an Amazon SNS topic subscriber receives every message that's published to the topic.
- To receive only a subset of the messages, a subscriber must assign a **filter policy** to the topic subscription.



# Exercise – Send email/SMS using SNS



- 1 Create a SNS topic
- 2 Add email subscription for the topic. You will receive confirmation email. Click the link in the email to confirm.
- 3 Add SMS subscription by adding your phone number. You will receive text message with code. Enter code to confirm your subscription.
- 4 Using SNS console, publish a message onto the topic
- 5 Verify if you receive the message over an email

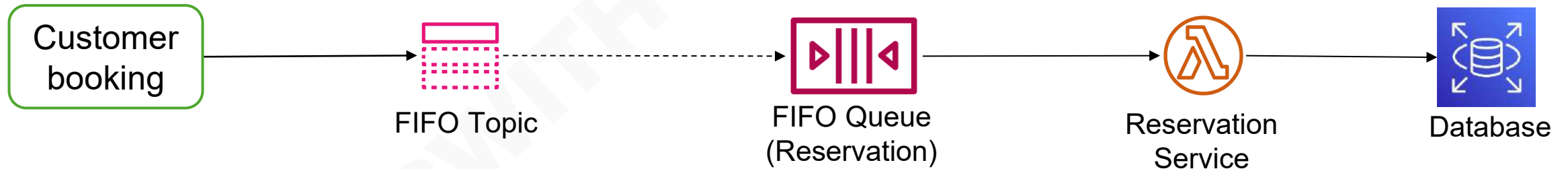
Note: In some countries there are restrictions on sending SMS like SMS are delivered only during specific time of the day or promotional messages are blocked etc.

# Common architecture patterns

# SNS FIFO Topic

## Booking/Reservation system

- Ordering is preserved
- No Duplicates

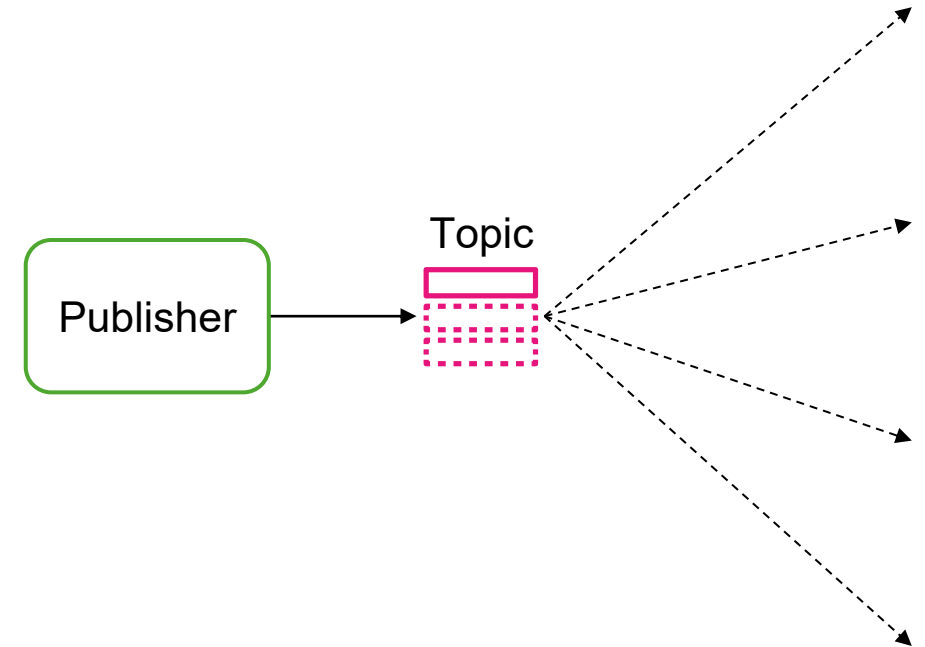




# SNS Fan-out

Fan-out means "Send once, deliver to many" for use cases where same message needs to be processed by multiple systems.

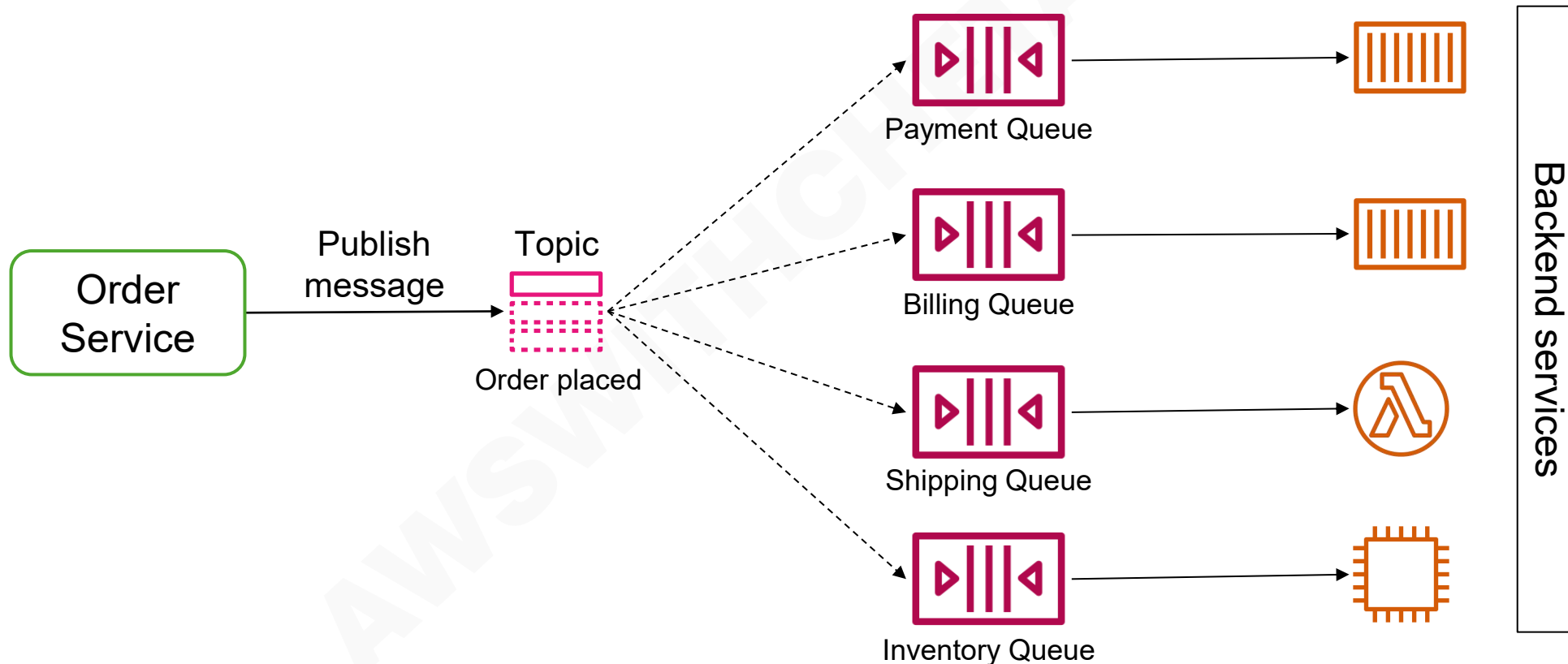
- Avoid sending the same message to many subscribers manually thereby simplifying application logic/errors/retries
- Prevent message loss if the producer crashes mid-delivery.
- Easy to add new consumers later without changing the producer.



# SNS Fan-out patterns

## SNS + SQS

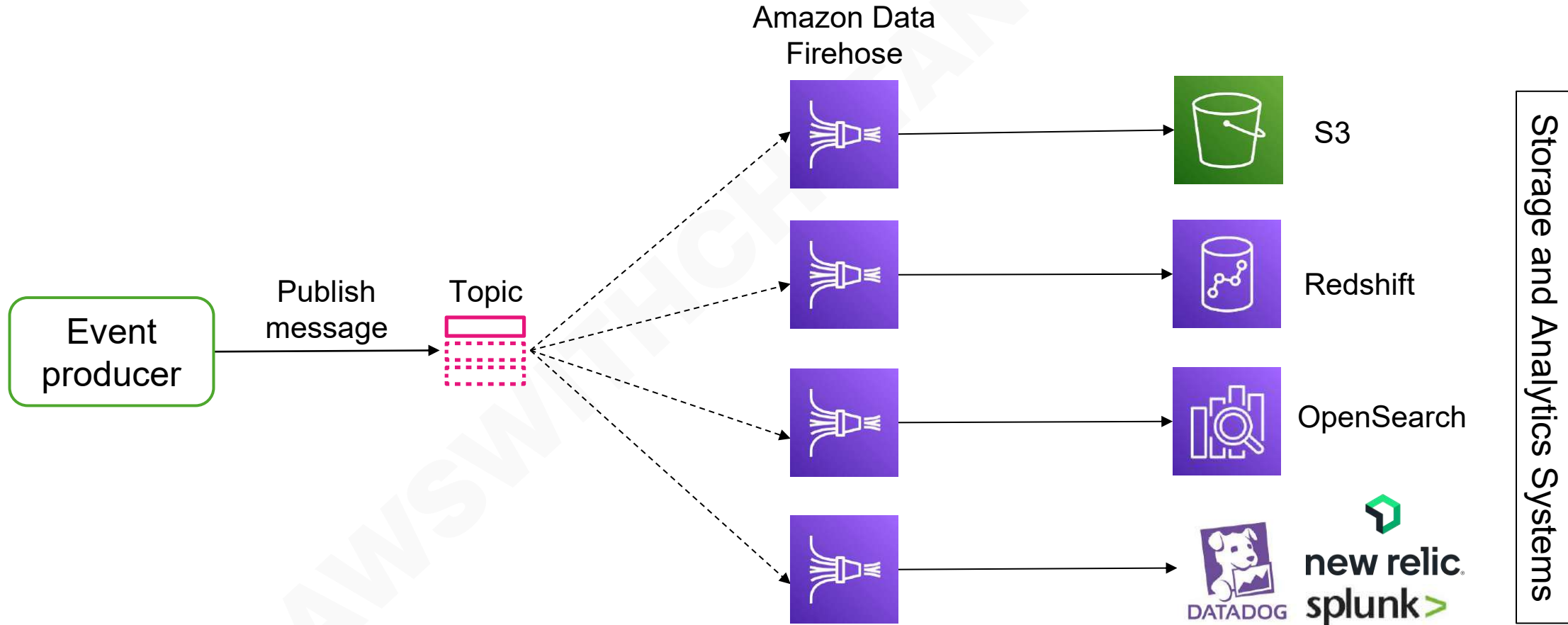
SNS and SQS queues are often used for Fan-out pattern when same message needs to be processed by different backend systems.



# SNS Fan-out patterns

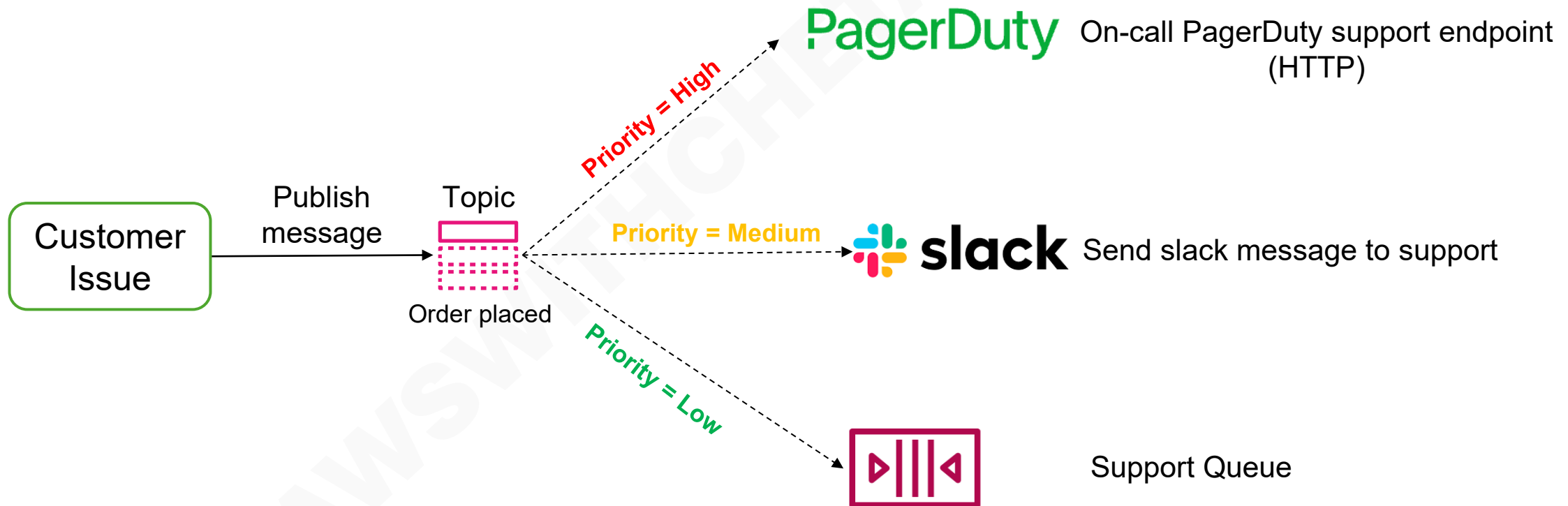
## SNS + Firehose

Fan-out to storage and analytics services (subscribe up to 5 Firehose)



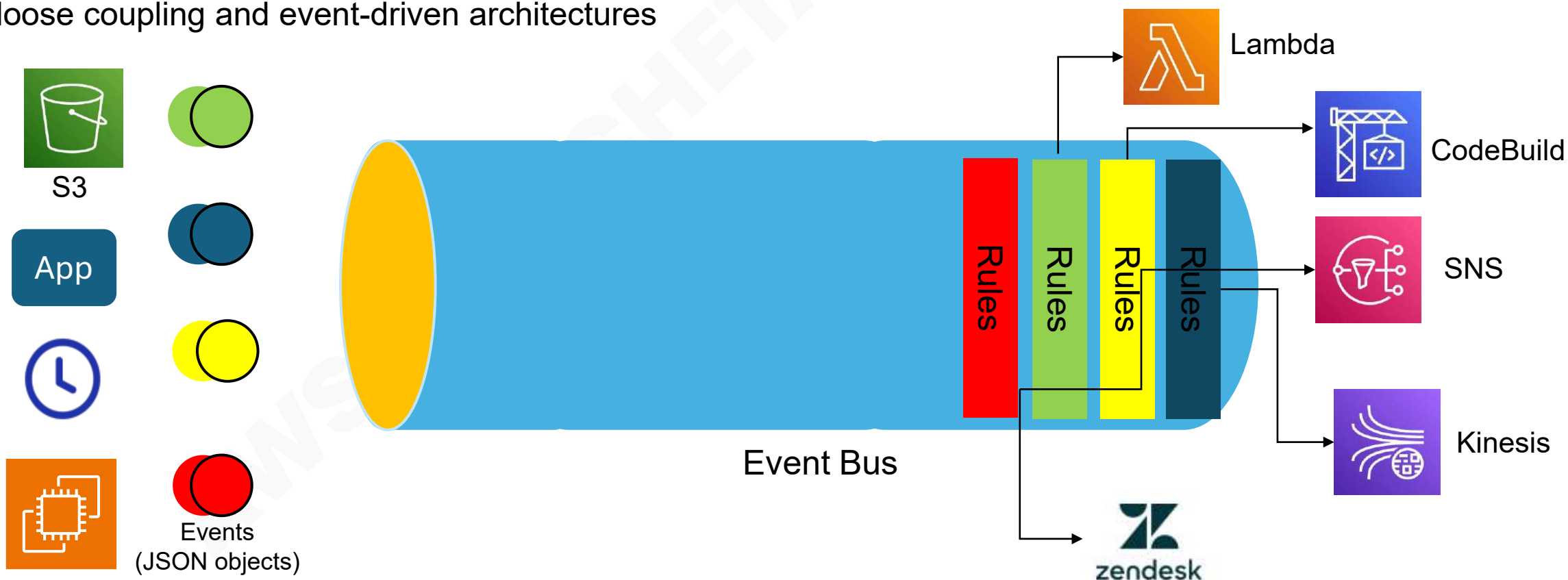
# SNS message filtering pattern

**Message Filtering** - Message filtering ensures that only relevant messages are delivered to relevant destination (subscriber), reducing unnecessary processing and improving efficiency.



# Amazon EventBridge

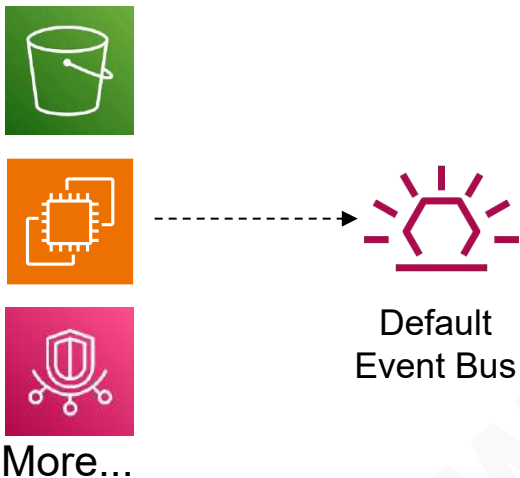
- Enables event-based communication between applications and services without requiring point-to-point integrations
- Events generated by a source are routed to one or more destinations based on defined rules, enabling loose coupling and event-driven architectures



# Amazon EventBridge – Types of event buses

## Default Event Bus

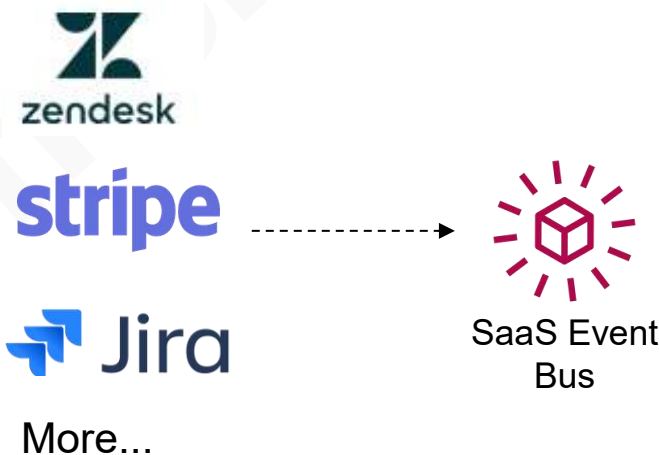
- Automatically created per AWS account; receives events from all supported AWS services



**AWS Services**

## Partner Event Bus

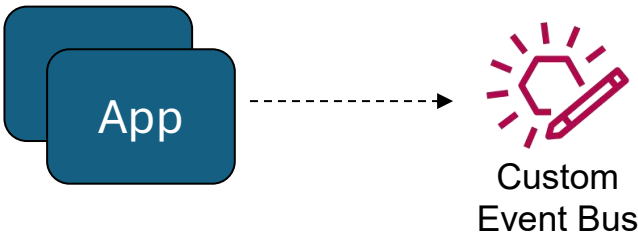
- Used to receive events directly from integrated SaaS partner applications.



**AWS SaaS Partners**

## Custom Event Bus

- Created for specific use cases to receive events from custom applications or services, including cross-account scenarios.



# Amazon EventBridge - Features

- Integrates with 130+ event sources and 40+ targets such as SNS, SQS, and Kinesis.
- Integration without writing custom code, integrates with many SaaS platforms without requiring web-hooks.
- Handles retries with exponential backoff for up to 24 hours for the failed deliveries.
- Rules for event filtering and event transformation before delivery
- Supports schema registries to discover, versioning, and reuse event schemas across applications.
- Allows event archiving and replays which is useful for reprocessing, disaster recovery and debugging.



Amazon EventBridge

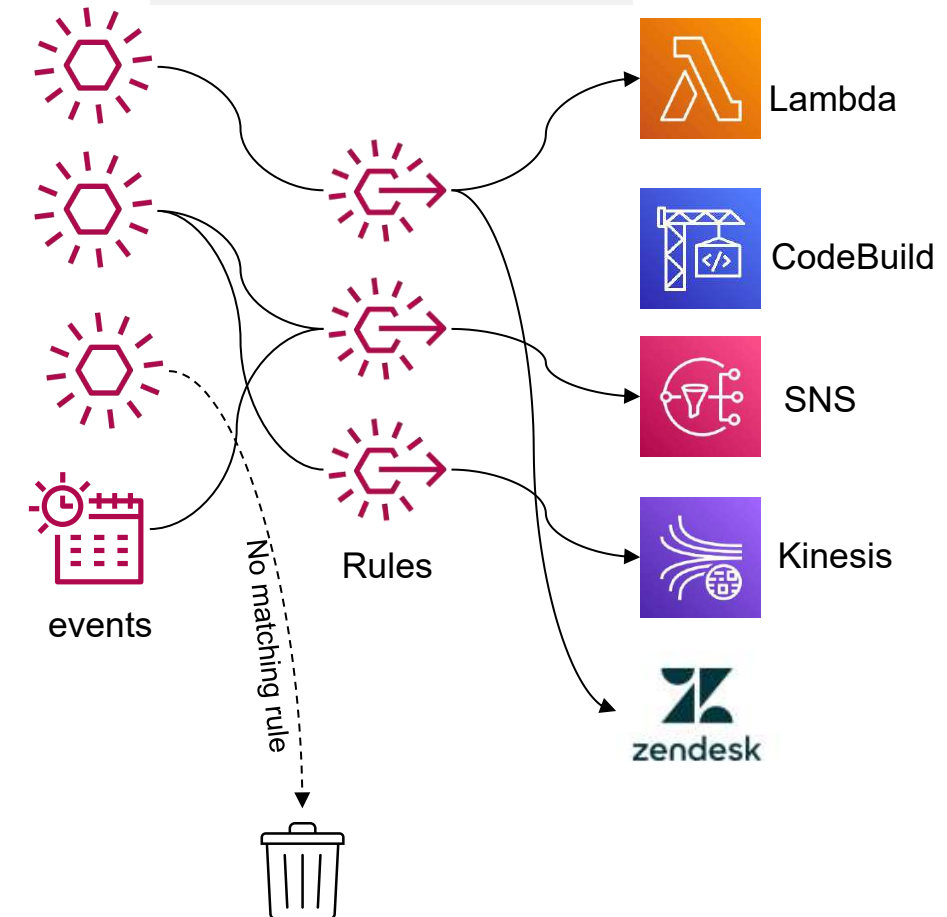
# Amazon EventBridge Rules

- Rules defined in EventBridge determine when and how events are routed to targets.
- A rule matches incoming events (from AWS services, custom apps, or SaaS) based on patterns you define.
- Uses event pattern filtering (JSON pattern matching) to select only relevant events.
- A single rule can target multiple destinations (e.g., Lambda, SQS, SNS, Step Functions, API Gateway, EventBridge buses).
- Multiple rules can match the same event enabling fan-out.
- You can have catch-all type rule to send events to default target
- Supports input transformations (e.g. OrderNumber -> OrderId)

## EventBridge Rule types:

- Schedule-Based Rules: Trigger actions based on time schedules
- Event Pattern-Based Rules: Trigger when an event matches a defined event pattern

```
{
  "detail-type": ["OrderEvent"],
  "detail": {
    "status": ["Confirmed"]
  },
  ..
  ..
}
```





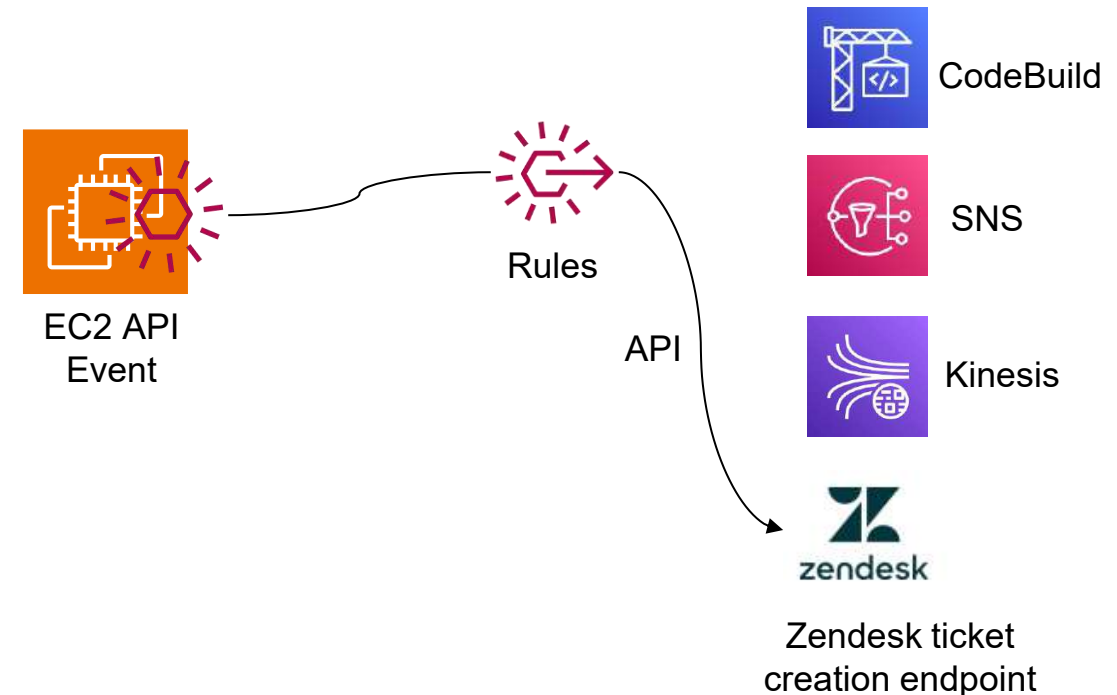
# Sample EC2 API event

```
{
  "version": "0",
  "id": "abcd1234-5678-9012-efgh-345678901234",
  "detail-type": "EC2 Instance State-change Notification",
  "source": "aws.ec2",
  "account": "123456789012",
  "time": "2023-11-14T10:20:30Z",
  "region": "us-east-1",
  "resources": [
    "arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234ef567890"
  ],
  "detail": {
    "instance-id": "i-0abcd1234ef567890",
    "state": "stopped"
  }
}
```

## EC2 API Event

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": {
    "state": ["stopped"]
  }
}
```

## Rule



# Rules with advanced matching/filters

- Prefix matching

```
{ "detail": { "username": [{ "prefix": "admin" }] } }
```

- Anything-but matching

```
{ "detail": { "status": [{ "anything-but": ["failed", "cancelled"] }] } }
```

- Numeric matching

```
{ "detail": { "amount": [{ "numeric": [ ">", 100 ] }] } }
```

- IP address matching

```
{ "detail": { "sourceIp": [{ "cidr": "10.0.0.0/16" }] } }
```

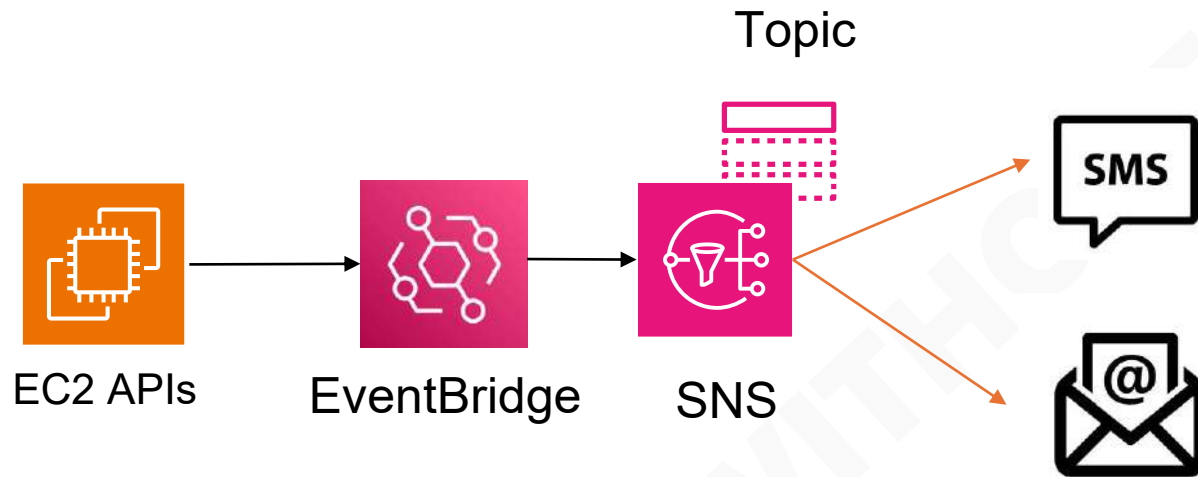
- Exists/Not Exists matching

```
{ "detail": { "errorCode": [{ "exists": true }] } }
```

- AND/OR logic

```
{ "detail": { "status": ["created", "cancelled"] } }
```

# Exercise - Send notification when EC2 instance is launched



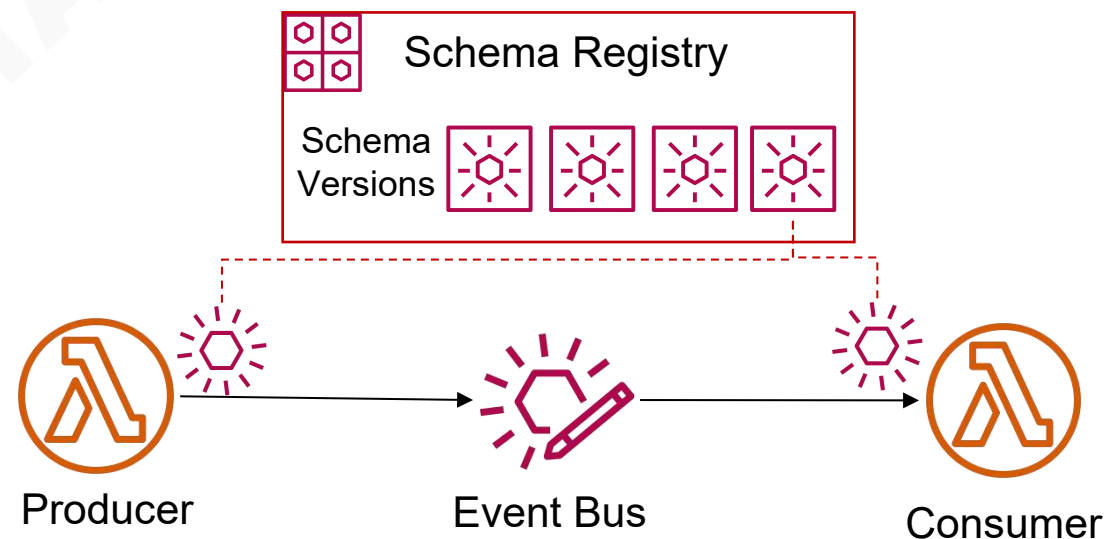
- 1 Create EventBridge Rule for EC2-state-change event using default event bus
- 2 Target should be the SNS topic that you created earlier
- 3 Launch a test EC2 instance. Verify if you receive the message over an email or SMS.

Note: In some countries there are restrictions on sending SMS like SMS are delivered only during specific time of the day or promotional messages are blocked etc.

# EventBridge Schema Registry

Schema registry stores event structure to enable consistent communication between the event producers and event consumers thereby reducing integration errors.

- Helps developers understand event formats without manually inspecting payloads.
- Supports multiple schema types: OpenAPI, JSON, Apache Avro etc.
- Can generate code bindings (Java, Python, TypeScript, etc.)
- Schemas include versioning, allowing evolution of event formats over time.



# Event Archiving and Replay

EventBridge can store (archive) a copy of all events that pass through an event bus so you can view, replay, and debug them later.

- You can archive all events or only events matching specific rules.
- Retention period up to 7 years.
- Replayed events maintain the original event structure and timestamp.
- Useful for debugging, auditing, or recovering from downstream failures.

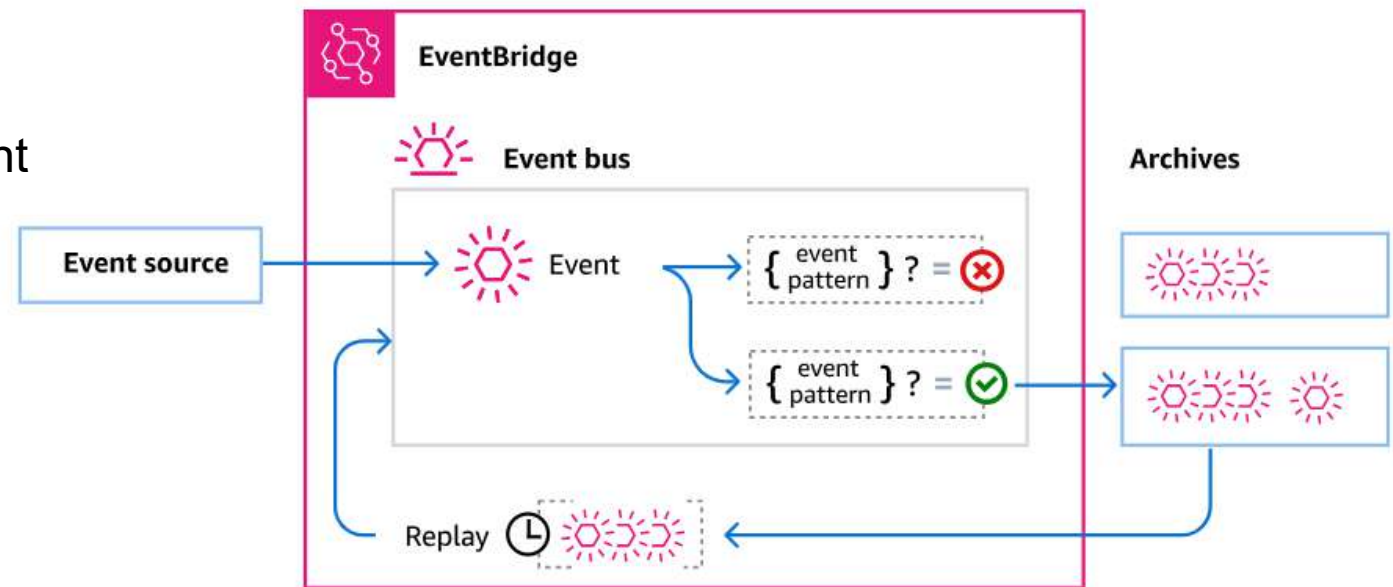


Image source: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-archive.html>

# Amazon EventBridge - Key points

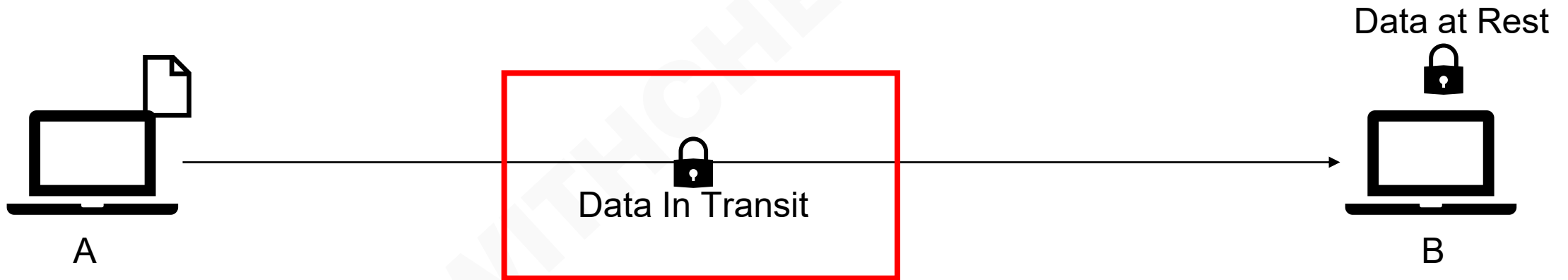
- Amazon EventBridge is a serverless event bus that routes events from AWS services, SaaS applications, and custom apps to targets using rules.
- It supports schedule-based events and event pattern-based routing, allowing you to deliver events only to the services that need them.
- Rules provide powerful content-based filtering, including prefix matching, numeric filters, anything-but matching, exists checks, and IP/CIDR matching.
- Targets can include Lambda, SQS, SNS, Step Functions, API destinations, Event buses, and more.
- EventBridge offers a Schema Registry to automatically discover event structures, version them, and generate code bindings for strongly typed development.
- It supports event archiving, allowing you to retain events for up to 7 years and replay them later for debugging, recovery, or onboarding new consumers.

Overall, EventBridge helps build loosely coupled, event-driven architectures by routing the right events to the right services with minimal integration effort.

# AWS Data Security

Securing data in transit, at rest and secrets

# Data Security





# HTTPS and SSL/TLS

Certificate  
Authority (CA)



HTTP**S** Traffic

https://s

k.com



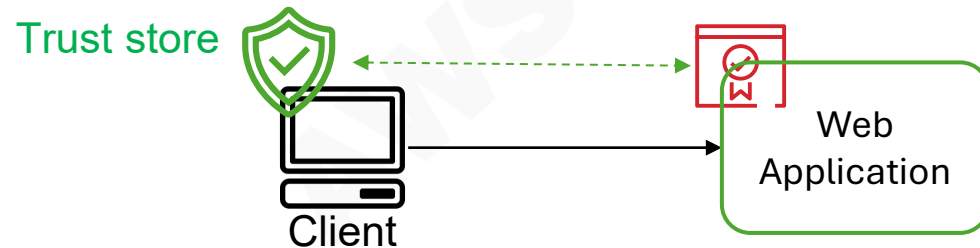
<https://youtu.be/cLYv4uSFJA8>

e Key

# Public vs Private SSL/TLS certificate

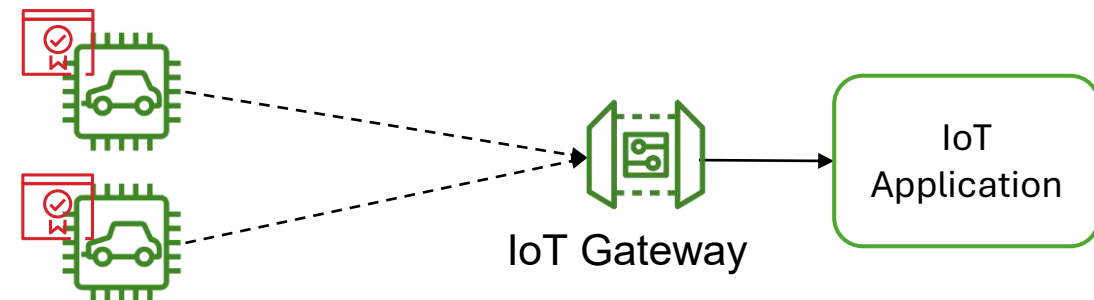
## Public Certificate

- Issued by globally trusted Certificate Authorities (CAs) like DigiCert, Let's Encrypt, Symantec, Amazon ATS.
- Trusted by all major browsers and operating systems automatically.
- Used for internet-facing websites and APIs.
- Require domain ownership validation (DNS, HTTP, or email).
- Usually have longer validity (months/years)



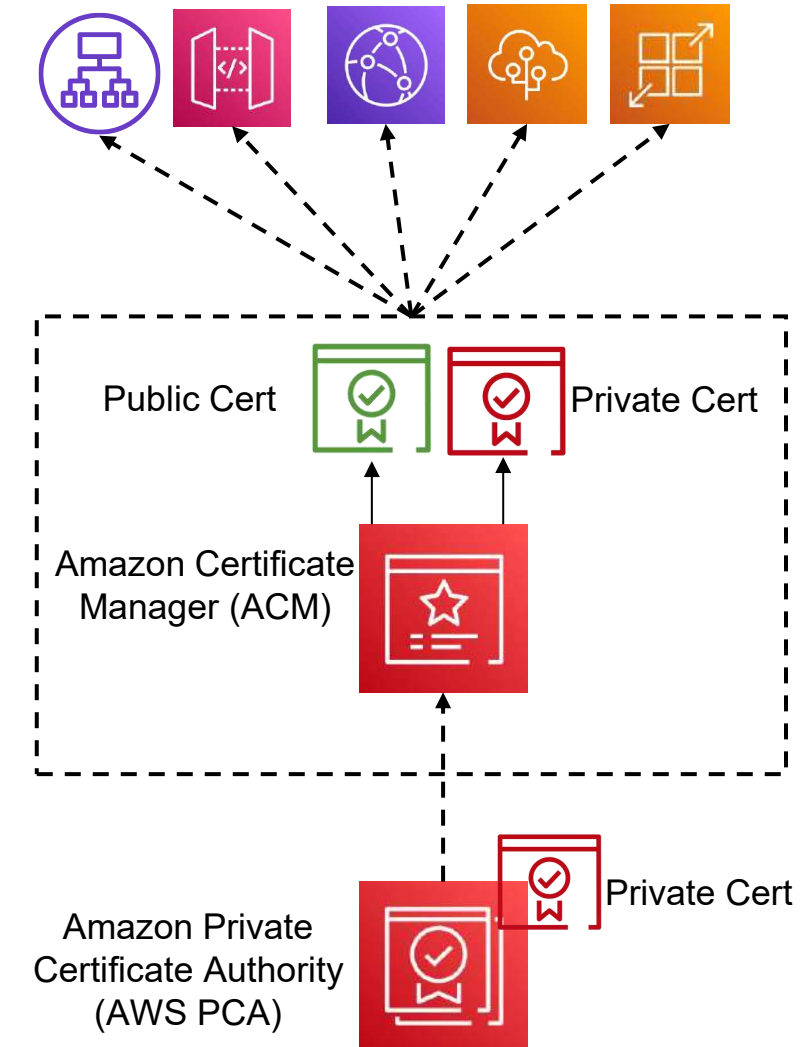
## Private Certificate

- Issued by an internal private CA.
- Not trusted by browsers unless the private CA's root certificate is installed manually.
- Used for communication between internal systems, microservices, intranets, VPNs or private IoT devices e.g. Connected cars
- Can issue both server and client certificates (mTLS).
- Customizable certificate policies, fields, and lifetimes.
- Useful for short-lived certificates in zero-trust setups.



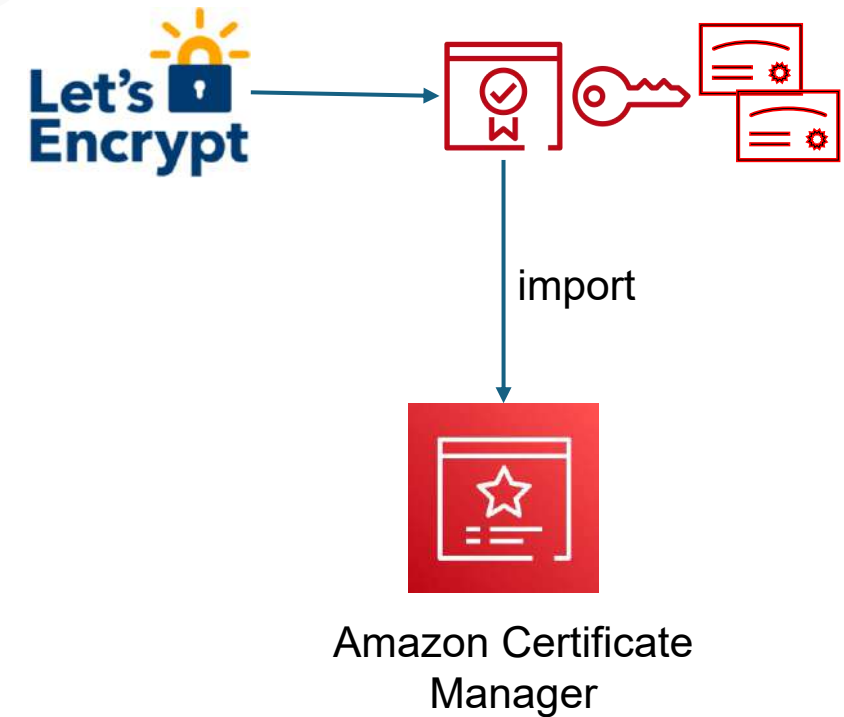
# Amazon Certificate Manager (ACM)

- Provisions, stores, and renews **Public** and **Private** SSL/TLS certificates (X.509) used for securing websites and applications.
- ACM can issue wildcard certificates (e.g., \*.example.com) for flexible subdomain coverage.
- Public certificates are free for use with AWS services like CloudFront, ALB, API Gateway, App Runner, EKS, BeanStalk etc.
- ACM allows exporting the Public certificates to use with your own application or EC2 instances (*additional cost per certificate*).
- ACM auto-renews certificates before expiry. No manual steps required as long as validation (DNS/email) remains intact.
- To issue and manage Private Certificates, ACM natively integrates with Amazon Private Certificate Authority (Private CA)
- ACM allows importing the 3<sup>rd</sup> party certificates



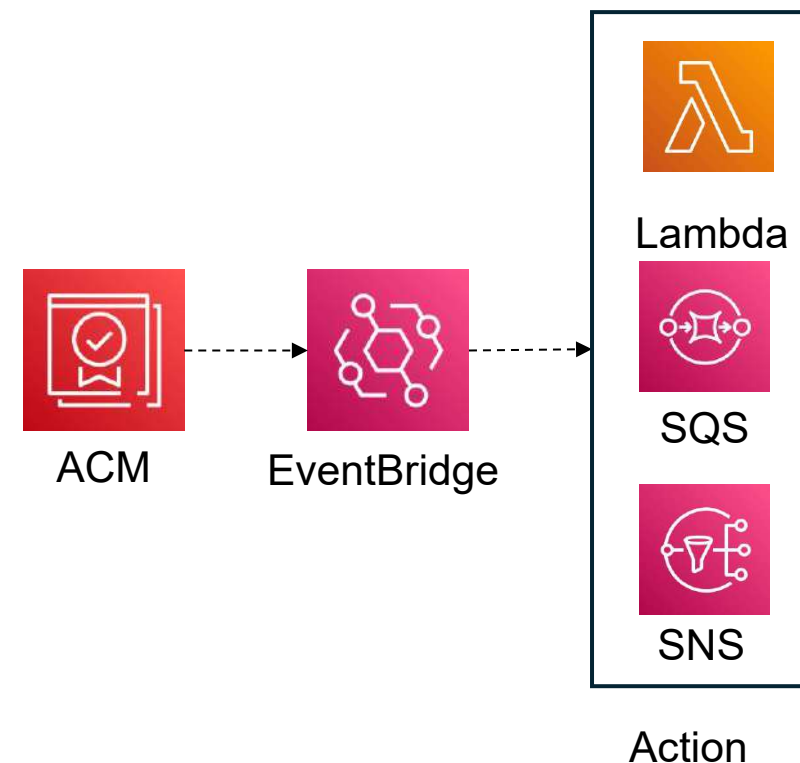
# Importing Public Certificate into ACM

- You can import certificates created outside AWS (e.g., Let's Encrypt, DigiCert, OpenSSL).
- You must provide three components:
  - Certificate
  - Private key
  - Certificate chain (intermediate CA certificates)
- You can attach imported certificates to AWS services just like ACM-issued certs.
- Imported certificates do not auto-renew. You must re-import the certificate before expiry.
- ACM sends expiration notification starting # days before expiry (default: 45 days)



# ACM Certificate Lifecycle events

- ACM sends daily expiration events for all active certificates (public, private and imported) starting 45 days prior to expiration.
- These events are automatically published to Amazon EventBridge (no setup required).
- Events include status changes such as:
  - Cert. Expiration approaching event
  - Cert. Expired event
  - Cert. Renewal action required event
  - more..
- You can create EventBridge rules to trigger actions when these events occur (Lambda, SNS, Slack alerts, ticketing systems).

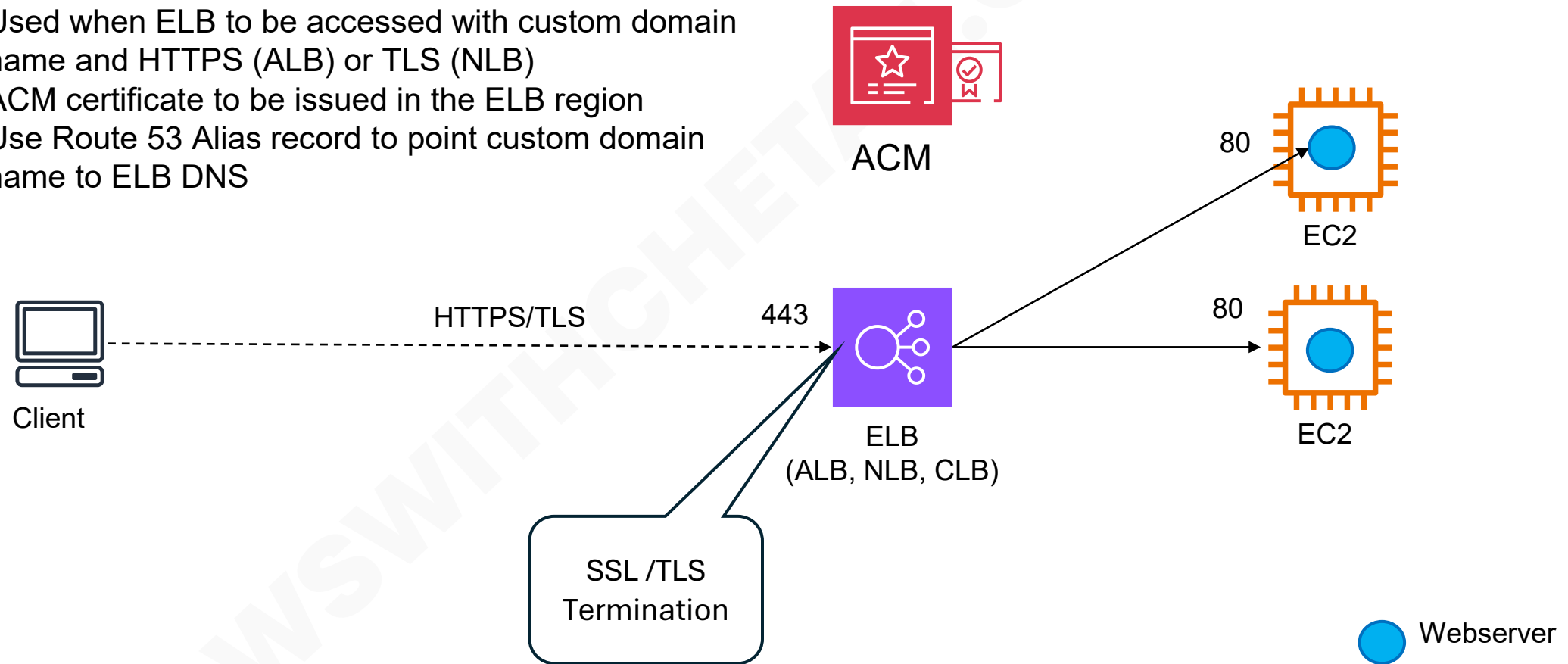


Helps avoid downtime caused by expired or validation-failed certificates.

# ACM Certificates – Common usage

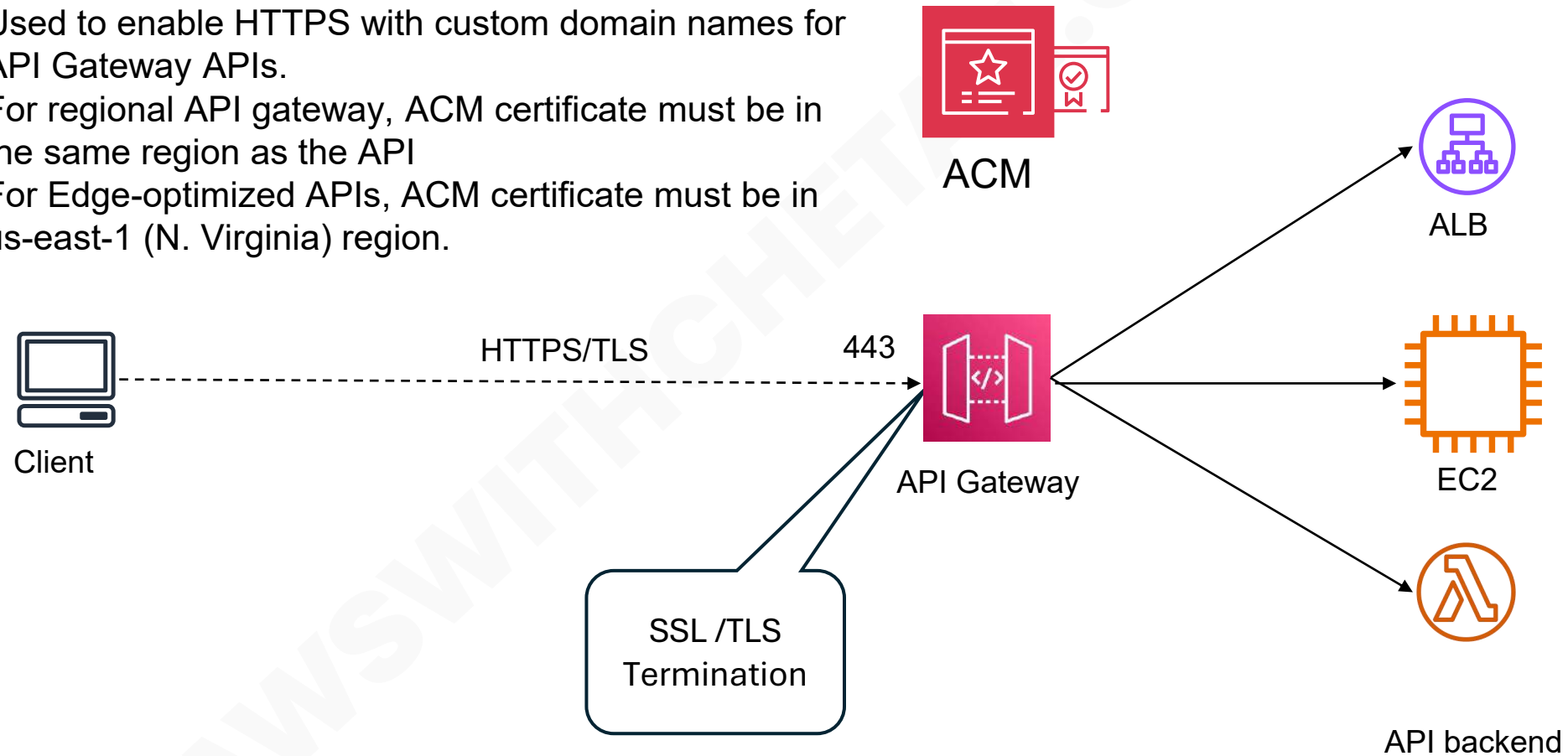
# ACM with ELB

- Used when ELB to be accessed with custom domain name and HTTPS (ALB) or TLS (NLB)
- ACM certificate to be issued in the ELB region
- Use Route 53 Alias record to point custom domain name to ELB DNS



# ACM with API Gateway

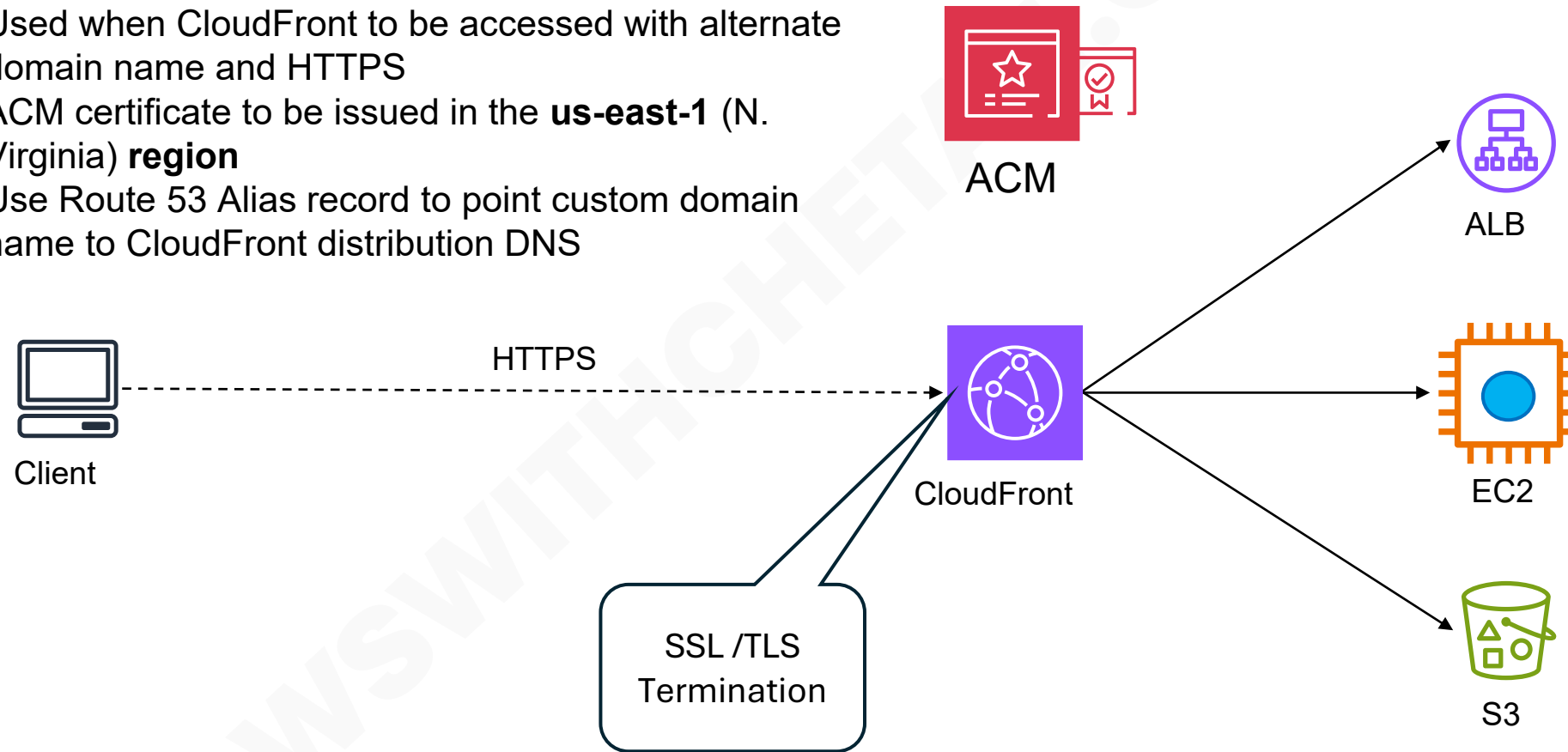
- Used to enable HTTPS with custom domain names for API Gateway APIs.
- For regional API gateway, ACM certificate must be in the same region as the API
- For Edge-optimized APIs, ACM certificate must be in us-east-1 (N. Virginia) region.





# ACM with CloudFront

- Used when CloudFront to be accessed with alternate domain name and HTTPS
- ACM certificate to be issued in the **us-east-1** (N. Virginia) **region**
- Use Route 53 Alias record to point custom domain name to CloudFront distribution DNS



# Amazon Private CA (PCA)

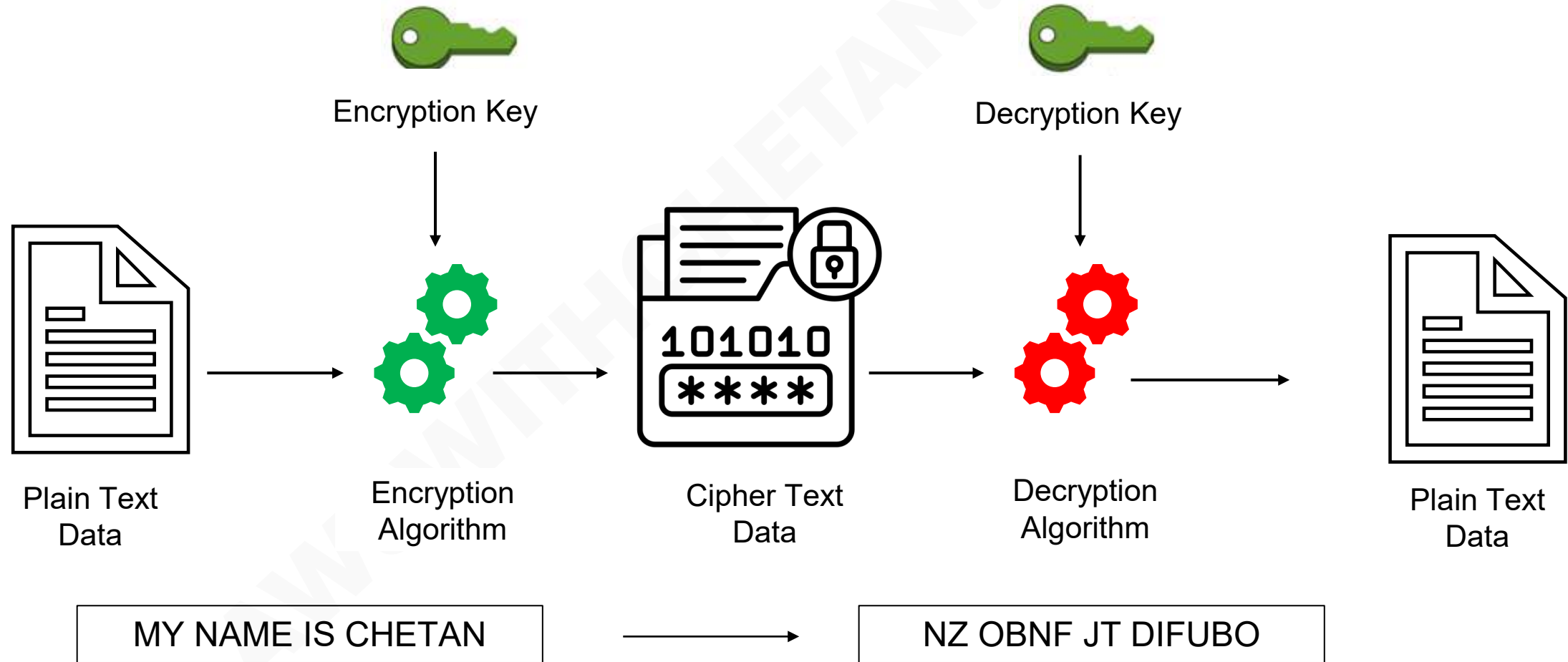
- Acts as a fully managed Certificate Authority, so you don't manage HSMs, hardware, or CA servers.
- Creates private, internal-use certificates used for intranet, internal microservices, on-prem systems, IoT devices, VPN users.
- You pay a monthly fee per CA plus per-certificate issuance. Unlike ACM public certs, these are not free.
- Supports hierarchical CA structure - Root and subordinate CAs to match enterprise PKI architecture.
- Enables mTLS (mutual TLS)
- You can enforce policies, extensions, key types, and constraints through templates.



Amazon Private  
Certificate Authority

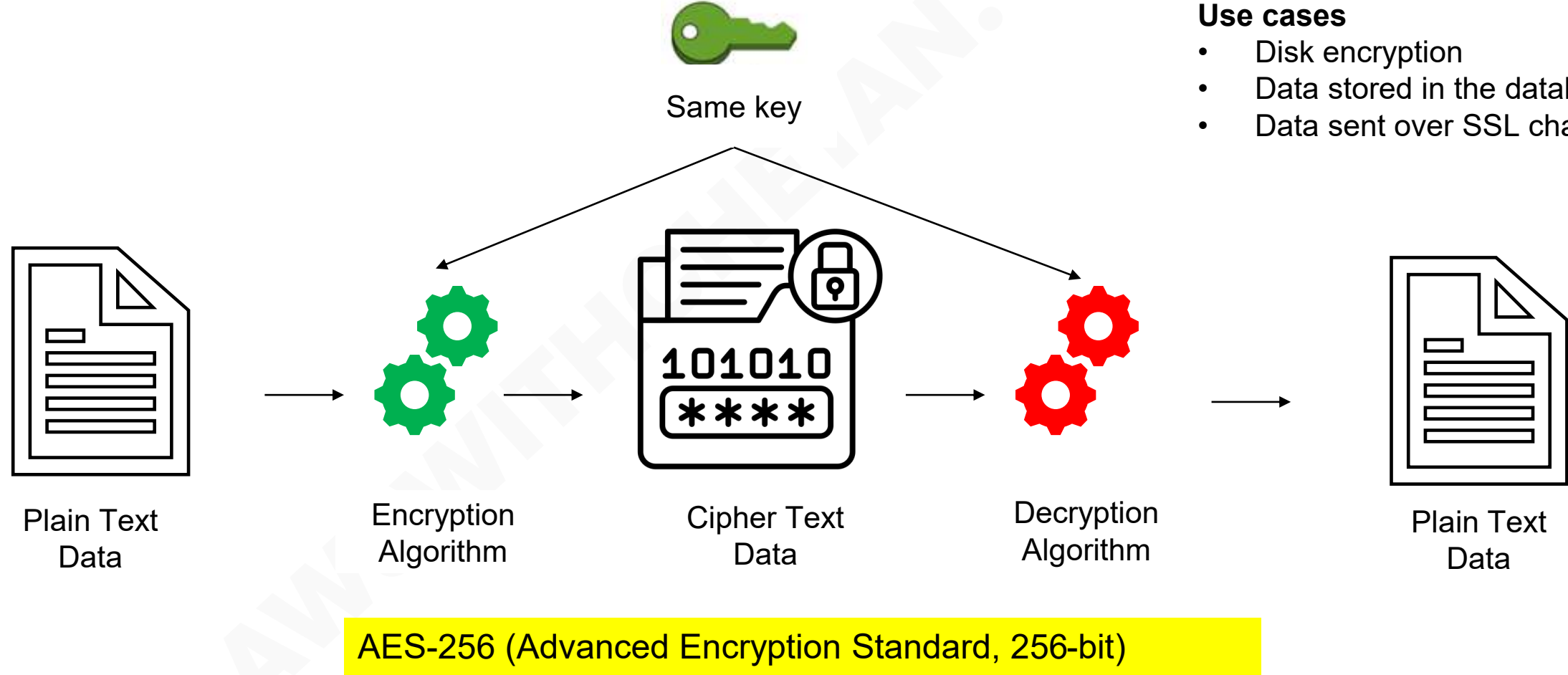
# Encrypting data at rest

# Data at rest – Encryption and Decryption



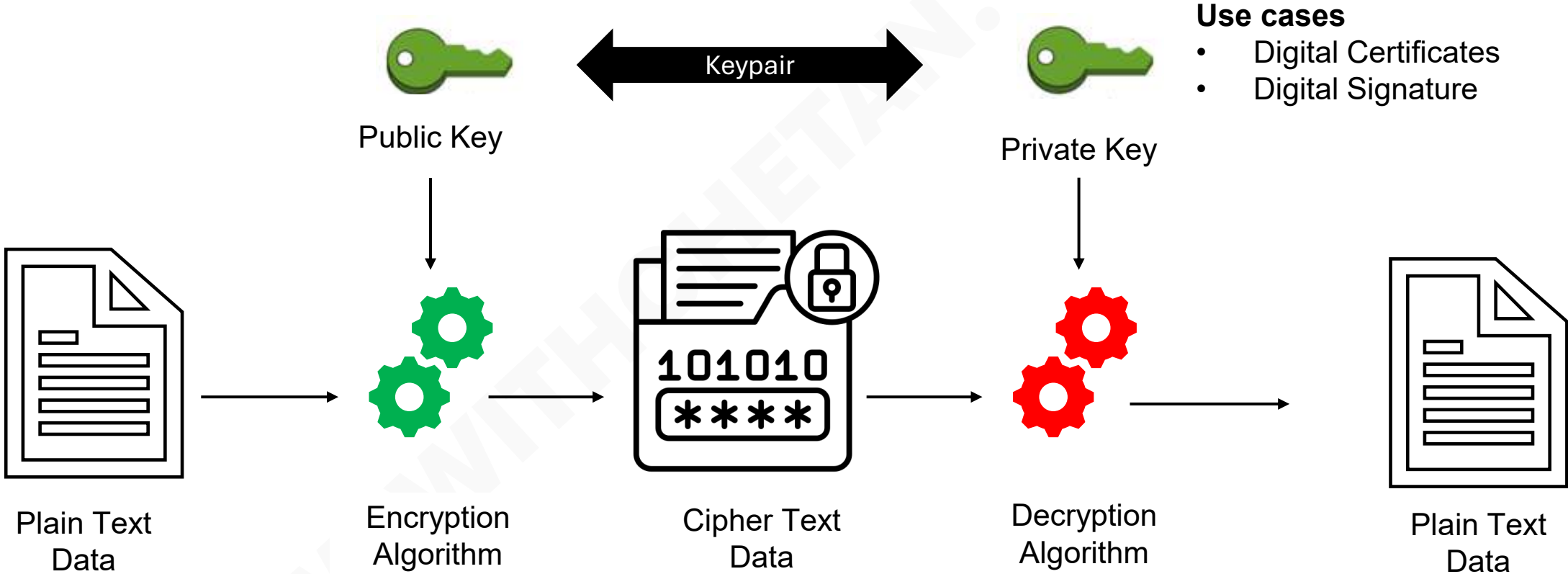
# Data at rest – Symmetric encryption

- Use cases**
- Disk encryption
  - Data stored in the databases
  - Data sent over SSL channel



AES-256 (Advanced Encryption Standard, 256-bit)

# Data at rest – Asymmetric Encryption



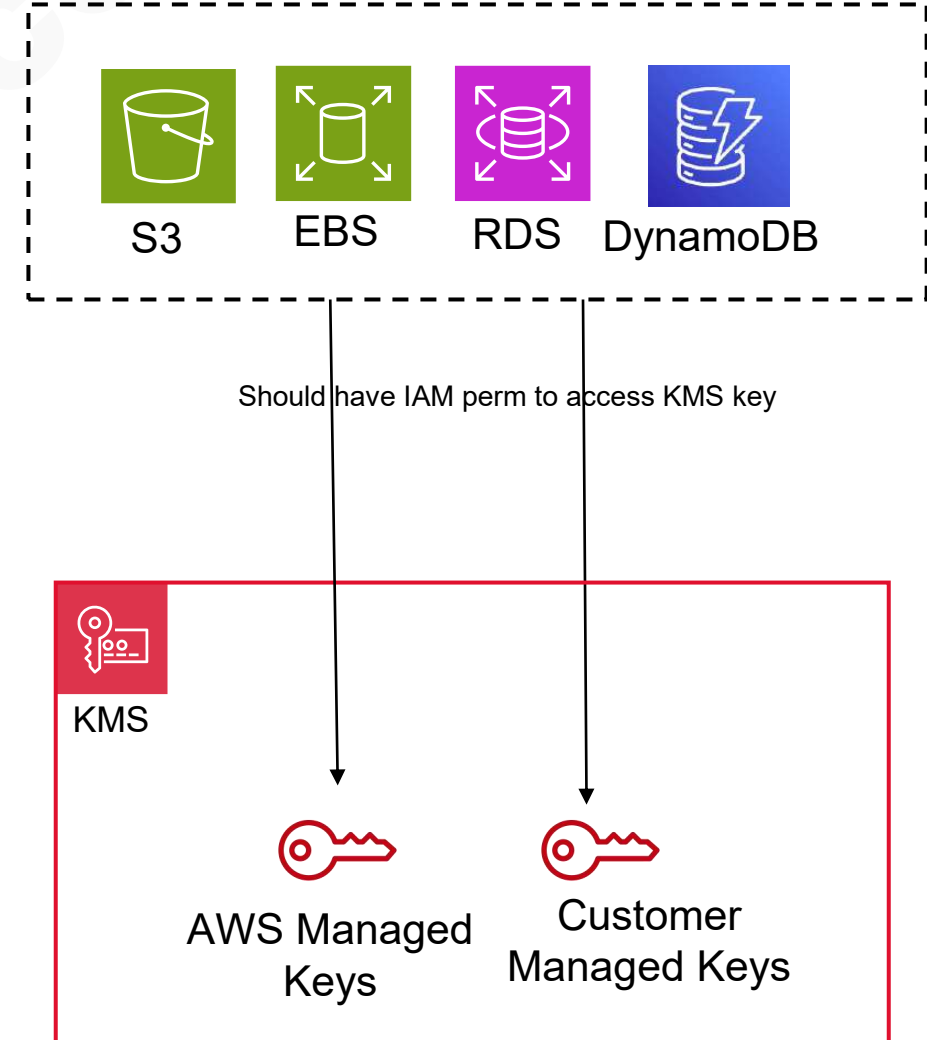
RSA (Rivest–Shamir–Adleman) or ECC (Elliptic Curve Cryptography)



# AWS KMS

# AWS Key Management Service (KMS)

- AWS KMS is a data encryption and key management service
- Supports both **Symmetric** and **Asymmetric** keys
- Supports various encryption algorithms (e.g. AES, ECC etc.) and keys (RSA-2048, ECC NIST P-256 etc.)
- Keys never leave KMS - Operations happen inside AWS-managed HSMs (FIPS 140-2 validated).
- User or AWS service needs IAM permissions to use KMS service and encryption keys (called Key policy).
- KMS keys can be shared with other AWS accounts.
- KMS keys are regional, but you can also create Multi-region KMS keys to replicate keys across regions for DR, backup etc.





Okay, got it... but

**Does KMS perform the actual Data encryption?**

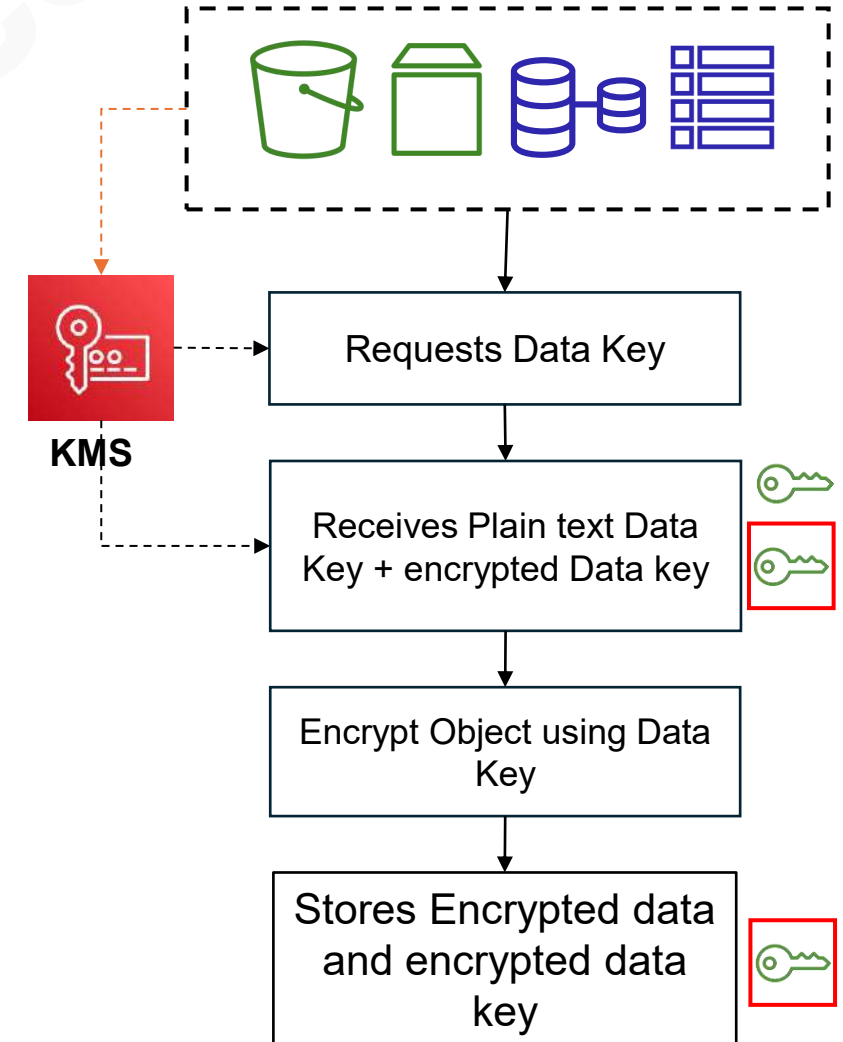
For example, encrypting each S3 object or EBS volume or snapshots or DynamoDB table data?

No

**AWS Service** performs the encryption using **Envelope Encryption** method.

# Envelope encryption

- Most of the AWS Services (e.g. S3, EBS, RDS, DynamoDB etc.) use Envelope encryption
- Using the KMS Key (CMK), KMS generates a Data Key and returns:
  - Plaintext key → for encryption
  - Encrypted key → safe to store
- AWS Service encrypts your data locally
- Uses the plaintext data key inside their own infrastructure to encrypt the data at huge scale and high speed. This happens on the storage nodes, not inside KMS.
- Plaintext key is immediately discarded
- Only the encrypted (KMS-protected) key is stored next to the encrypted data.



# KMS Key policy

KMS keys comes with the key policy which controls who can use or manage a KMS key (like S3 bucket policies.)

You can grant different level of access to the key such as:

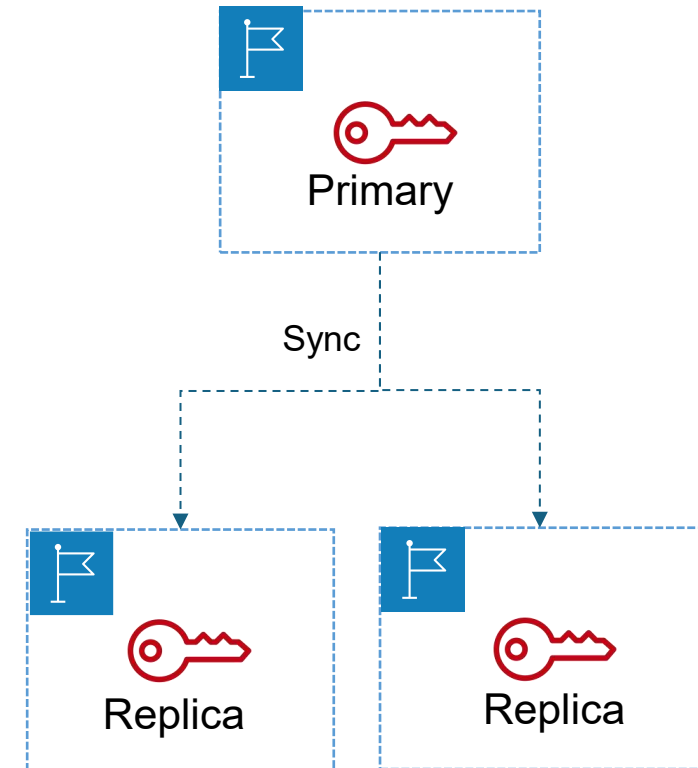
- Administrator – Manage keys (Rotate, manage permissions)
- Key Users – Encrypt/decrypt and generate Data Keys
- Cross-account access

When sharing the resources with other AWS account e.g. AMIs or encrypted snapshots, KMS key policy should be updated to allow KMS key access to the target AWS account

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowKeyAdminAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS":
          "arn:aws:iam::123456789012:role/AdminRole"
      },
      "Action": [
        "kms:DescribeKey",
        "kms:Create*",
        "kms:Enable*",
        "kms:Disable*",
        "kms:ScheduleKeyDeletion",
        "kms:CancelKeyDeletion",
        "kms:Put*",
        "kms:Update*",
        "kms:List*",
        "kms:Revoke*"
      ],
      "Resource": "*"
    }
  ]
}
```

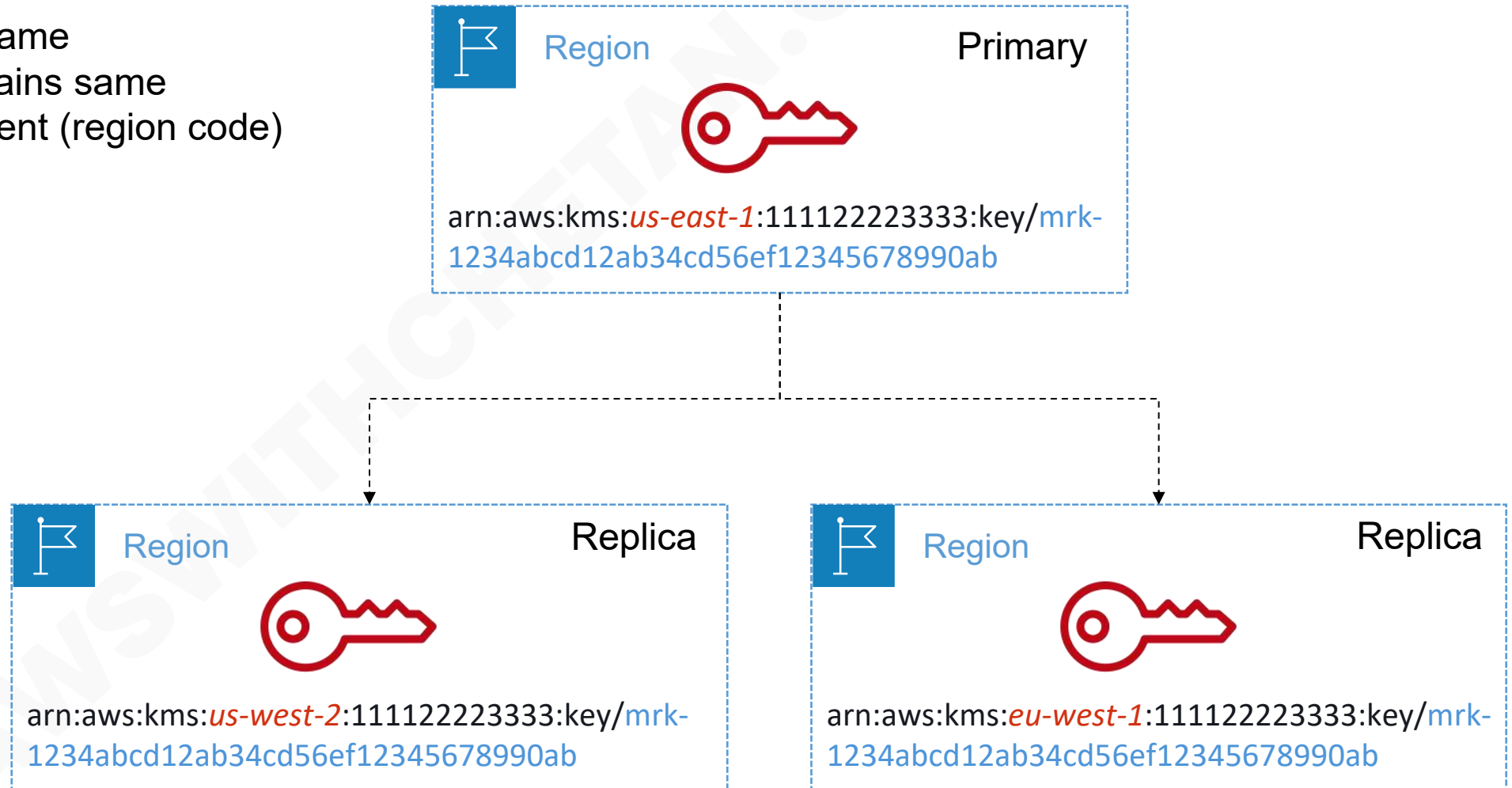
# KMS Multi-region Keys

- Designed for cross-region workloads
- Consists of Primary key and Replica keys
- Replicas share identical key material and key ID but different ARNs.
- Automatic synchronization of key metadata, policy, tags, and rotation state from primary to replicas.
- No re-encryption required. Encryption/decryption happens in the same region as the data.



# KMS Multi-region Keys

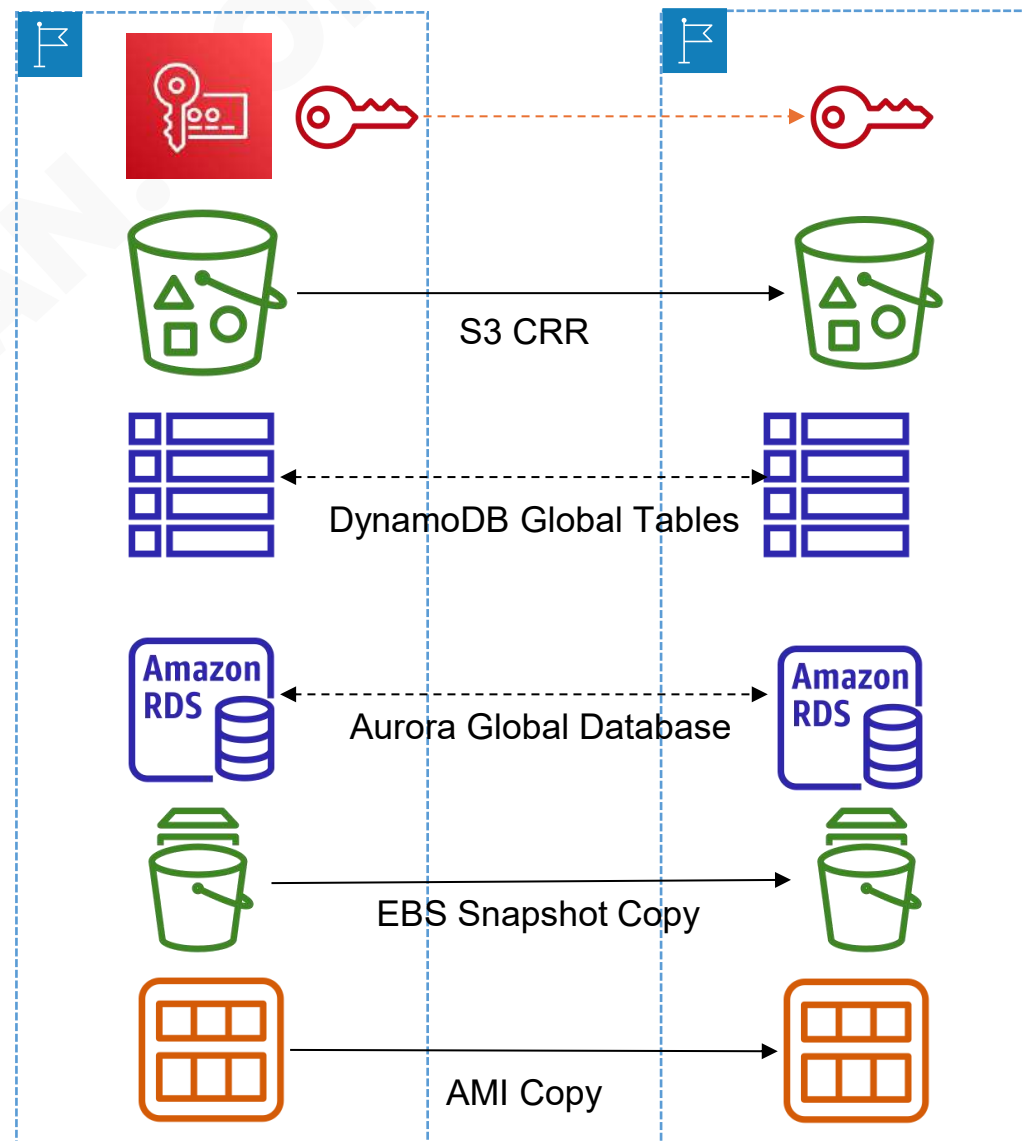
- Key ID remains same
- Key Material remains same
- Key ARN is different (region code)



# Multi-region key scenarios

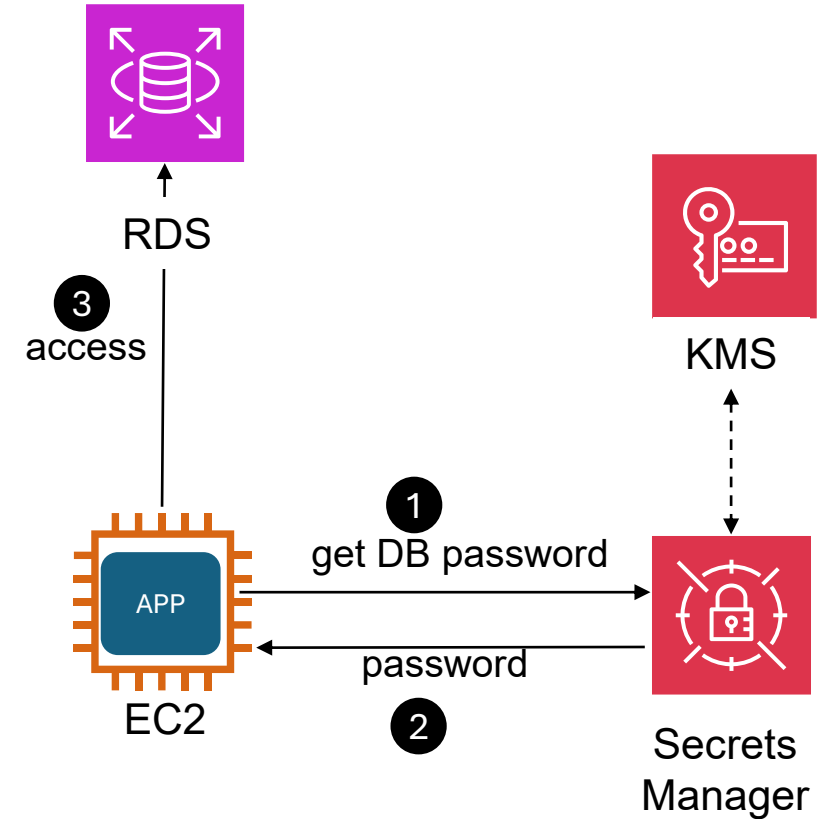
- S3 Bucket Replication across Regions (CRR)
- DynamoDB Global Tables
- Aurora Global Database
- EBS Snapshot copy across regions
- AMI copy across regions

When sharing the resources with other AWS account  
e.g. AMIs or encrypted snapshots, KMS key policy  
should be updated to allow KMS key access to the  
target AWS account



# AWS Secrets Manager

- Helps you to securely store, retrieve and rotate credentials for your databases and other services.
- You can store passwords, API keys, Tokens or any other login credentials.
- No need to hardcode credentials in the application code.
- Rotate credentials automatically for integrated services (e.g. RDS - MySQL, PostgreSQL, Aurora etc. ) or invoke Lambda function to rotate the credentials.
- Secrets are encrypted using KMS
- Access to secrets is controlled using IAM permissions

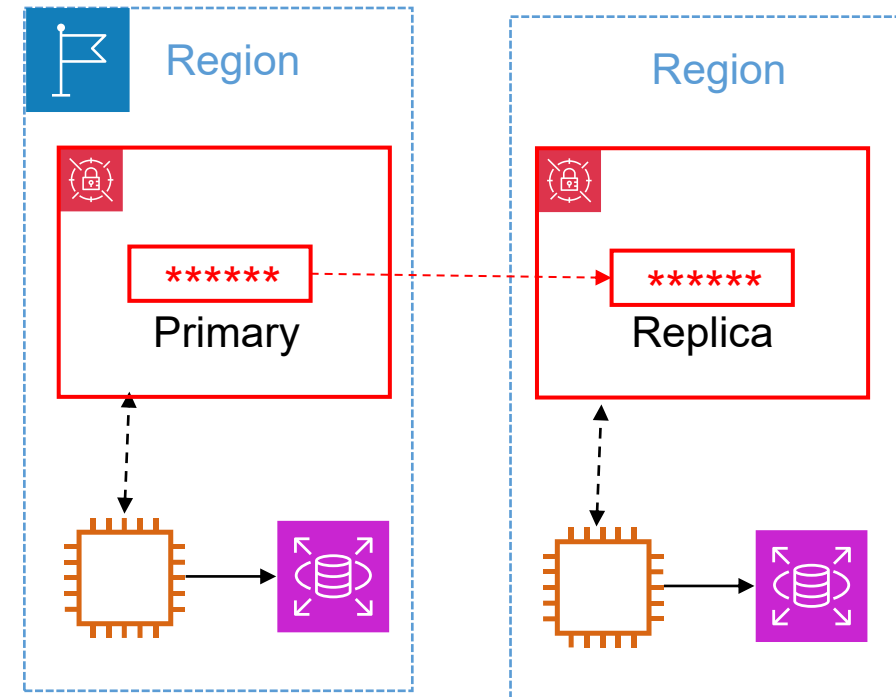


# Multi-region secrets

- You can replicate your secrets in multiple AWS Regions to support applications spread across those Regions to meet Regional access and low latency requirements.
- You can promote a replica to a primary during failover.
- Secret rotation happens in the primary region only; replicas automatically receive updated values.

## Use cases:

- EKS/ECS clusters in multiple regions
- Multi-region Lambda/API workloads





# Secrets Manager vs SSM Parameter Store

✓ Secrets Manager is designed for secrets whereas Parameter Store is designed for configuration.

|                      | Secrets Manager                                                                                                                                                        | SSM Parameter Store                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is it?          | Allows storing and rotating secrets (passwords, keys, tokens)                                                                                                          | Allows storing configuration parameters (strings, JSON, paths)                                                                                           |
| Rotation             | Built-in secrets rotation                                                                                                                                              | Not supported                                                                                                                                            |
| Multi-region support | Multi-region with secret replicas                                                                                                                                      | Not supported                                                                                                                                            |
| Encryption           | Secrets are encrypted by default                                                                                                                                       | Optional encryption                                                                                                                                      |
| Used For             | <ul style="list-style-type: none"> <li>• Database passwords</li> <li>• OAuth tokens</li> <li>• Third-party API keys</li> <li>• Certificates or private keys</li> </ul> | <ul style="list-style-type: none"> <li>• App settings</li> <li>• Feature flags</li> <li>• App versions</li> <li>• Non-sensitive configuration</li> </ul> |

# Infrastructure as Code

AWS CloudFormation and CDK

# Infrastructure and application deployment..

## Infrastructure as Code

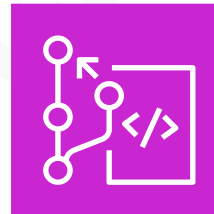


AWS CloudFormation

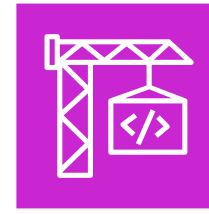


AWS CDK

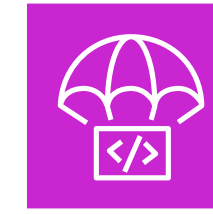
## DevOps – CI/CD



CodeCommit



CodeBuild



CodeDeploy



CodePipeline

## Application Deployment



ElasticBeanstalk



Amazon Lightsail



Amplify

# Why Infrastructure as Code?

# Using AWS CLI

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications ResourceType=vpc,Tags=[{Key=Name,Value=MyVPC}]
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --vpc-id vpc-xxxxxxxxx --internet-gateway-id igw-xxxxxxxxx
aws ec2 create-subnet --vpc-id vpc-xxxxxxxxx --cidr-block 10.0.0.0/24 --availability-zone ap-south-1a
aws ec2 create-subnet --vpc-id vpc-xxxxxxxxx --cidr-block 10.0.1.0/24 --availability-zone ap-south-1b
aws ec2 create-subnet --vpc-id vpc-xxxxxxxxx --cidr-block 10.0.11.0/24 --availability-zone ap-south-1a
aws ec2 create-subnet --vpc-id vpc-xxxxxxxxx --cidr-block 10.0.12.0/24 --availability-zone ap-south-1b
aws ec2 create-route-table --vpc-id vpc-xxxxxxxxx
aws ec2 create-route --route-table-id rtb-xxxxxxxxx --destination-cidr-block 0.0.0.0/0 --gateway-id igw-xxxxxxxxx
aws ec2 associate-route-table --route-table-id rtb-xxxxxxxxx --subnet-id subnet-xxxxxxxxx
aws ec2 create-route-table --vpc-id vpc-xxxxxxxxx
aws ec2 associate-route-table --route-table-id rtb-xxxxxxxxx --subnet-id subnet-xxxxxxxxx
aws ec2 allocate-address --domain vpc
aws ec2 create-nat-gateway --subnet-id subnet-xxxxxxxxx --allocation-id eipalloc-xxxxxxxxx
aws ec2 create-route --route-table-id rtb-xxxxxxxxx --destination-cidr-block 0.0.0.0/0 --gateway-id nat-xxxxxxxxx
...
```

# Using AWS SDK (python – boto3)

vpc.py

```
import boto3
ec2 = boto3.client("ec2", region_name="ap-south-1")

# VPC
vpc = ec2.create_vpc(CidrBlock="10.0.0.0/16")["Vpc"]["VpcId"]
ec2.create_tags(Resources=[vpc], Tags=[{"Key": "Name", "Value": "MyVPC"}])

# Internet Gateway
igw = ec2.create_internet_gateway()["InternetGateway"]["InternetGatewayId"]
ec2.attach_internet_gateway(VpcId=vpc, InternetGatewayId=igw)

# Subnets
subnets = []
for cidr, az in [("10.0.0.0/24", "ap-south-1a"), ("10.0.1.0/24", "ap-south-1b"),
                ("10.0.11.0/24", "ap-south-1a"), ("10.0.12.0/24", "ap-south-1b")]:
    subnets.append(ec2.create_subnet(VpcId=vpc, CidrBlock=cidr,
    AvailabilityZone=az)["Subnet"]["SubnetId"])

# Public route table
rtb_public = ec2.create_route_table(VpcId=vpc)["RouteTable"]["RouteTableId"]
ec2.create_route(RouteTableId=rtb_public, DestinationCidrBlock="0.0.0.0/0", GatewayId=igw)
ec2.associate_route_table(RouteTableId=rtb_public, SubnetId=subnets[0])

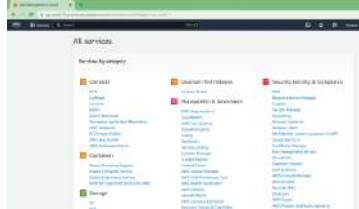
# NAT Gateway
eip = ec2.allocate_address(Domain="vpc")["AllocationId"]
nat = ec2.create_nat_gateway(SubnetId=subnets[0], AllocationId=eip)["NatGateway"]["NatGatewayId"]
...
```



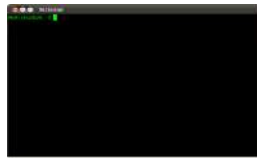
\$python vpc.py



# Ways to deploy AWS infrastructure



AWS Console



AWS CLI



AWS SDK

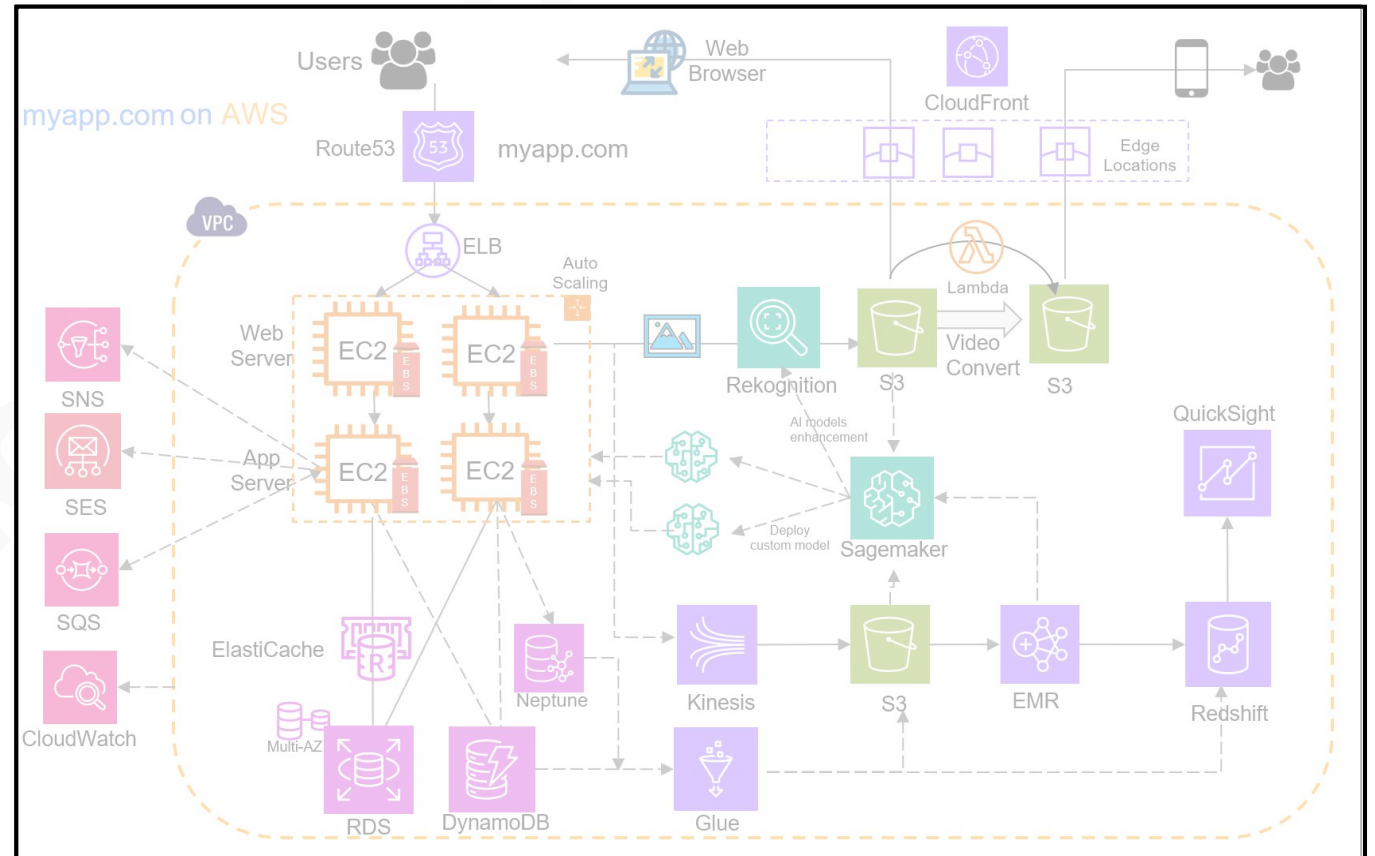
C++, go, java, Java script,  
Kotlin, .Net, Node.js, PHP,  
python, ruby, rust, swift



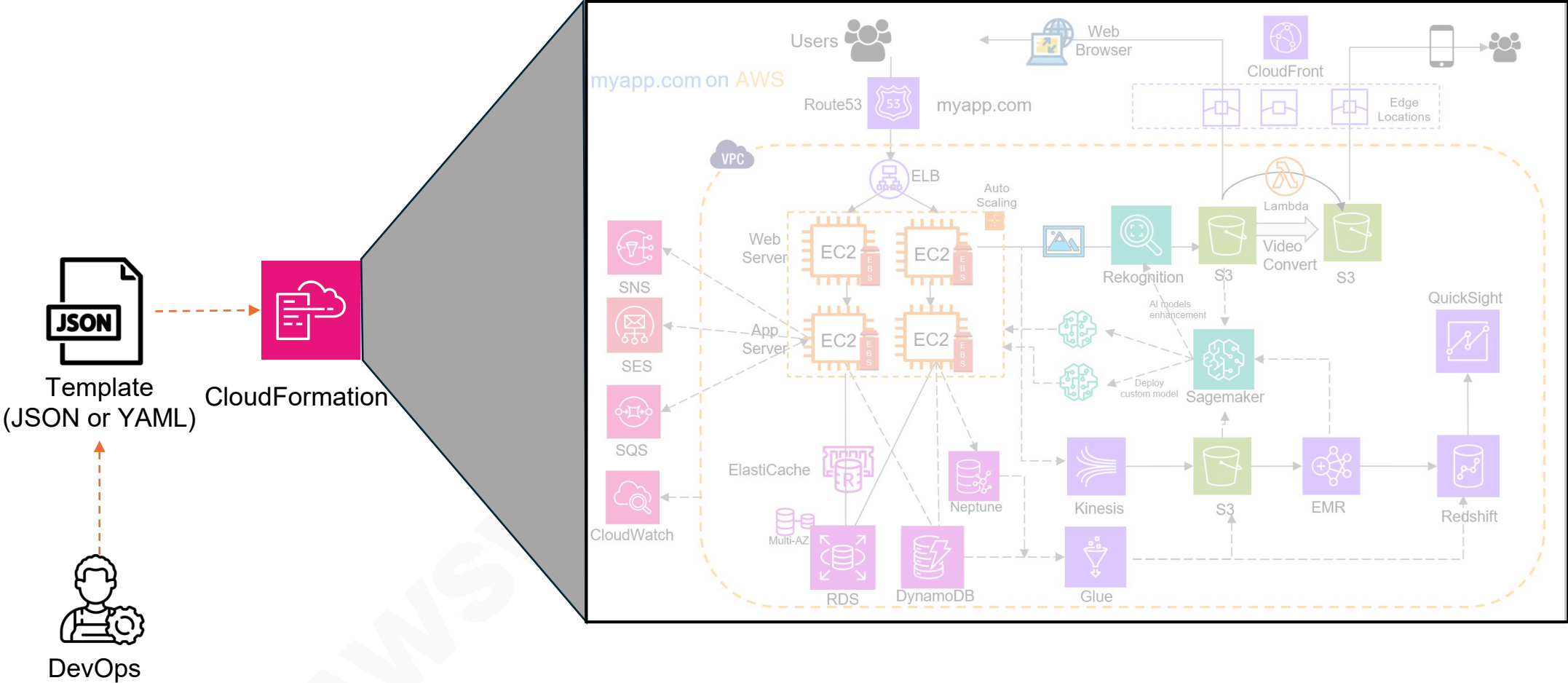
AWS CloudFormation



CDK

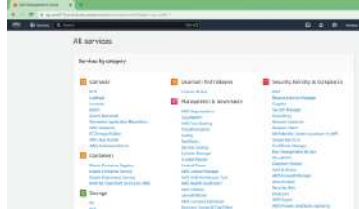


# Using AWS CloudFormation

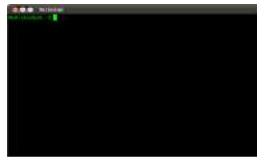




# Ways to deploy AWS infrastructure



AWS Console



AWS CLI



AWS SDK

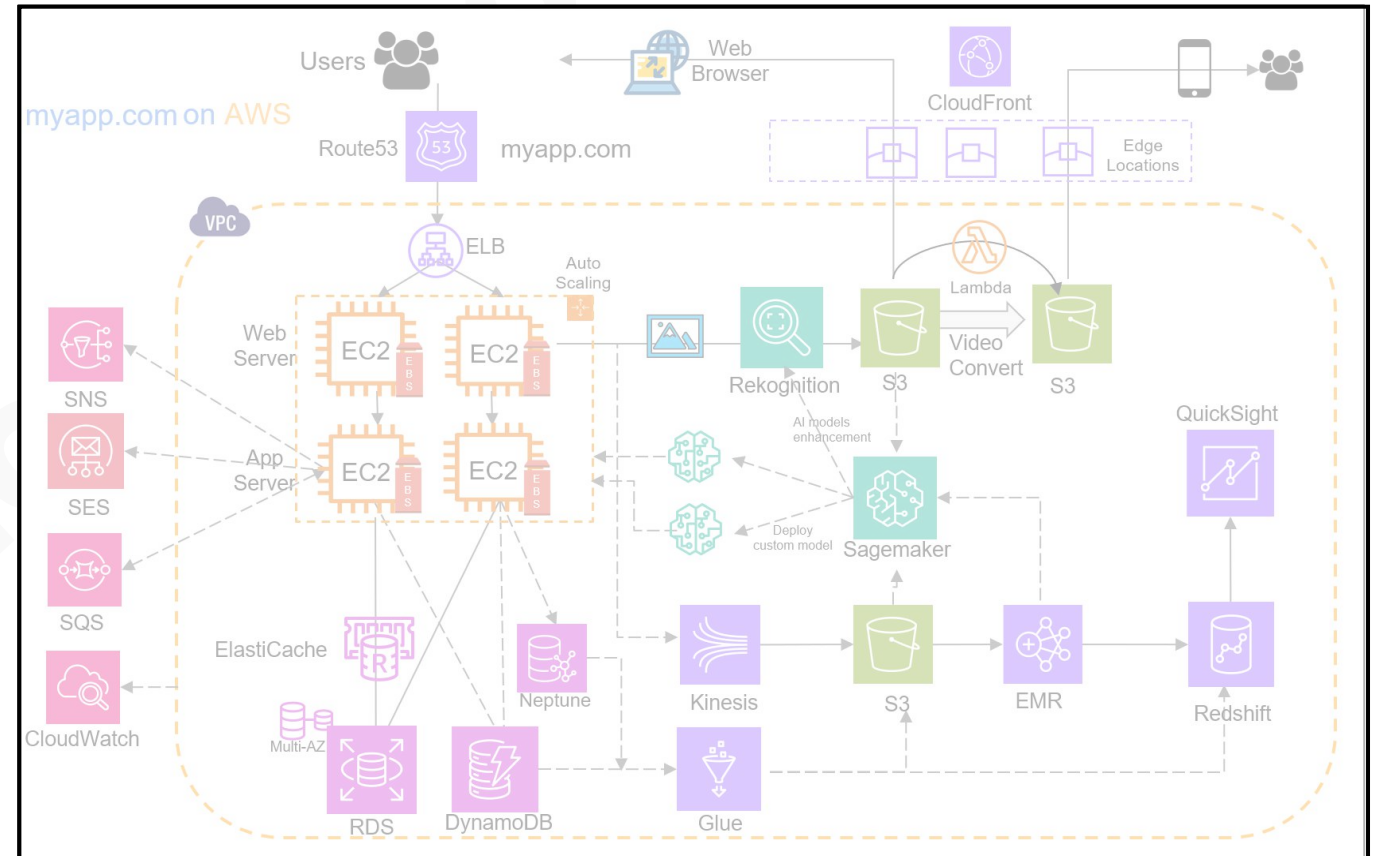
C++, go, java, Java script,  
Kotlin, .Net, Node.js, PHP,  
python, ruby, rust, swift



AWS CloudFormation  
(YAML or JSON)

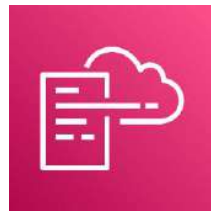


CDK



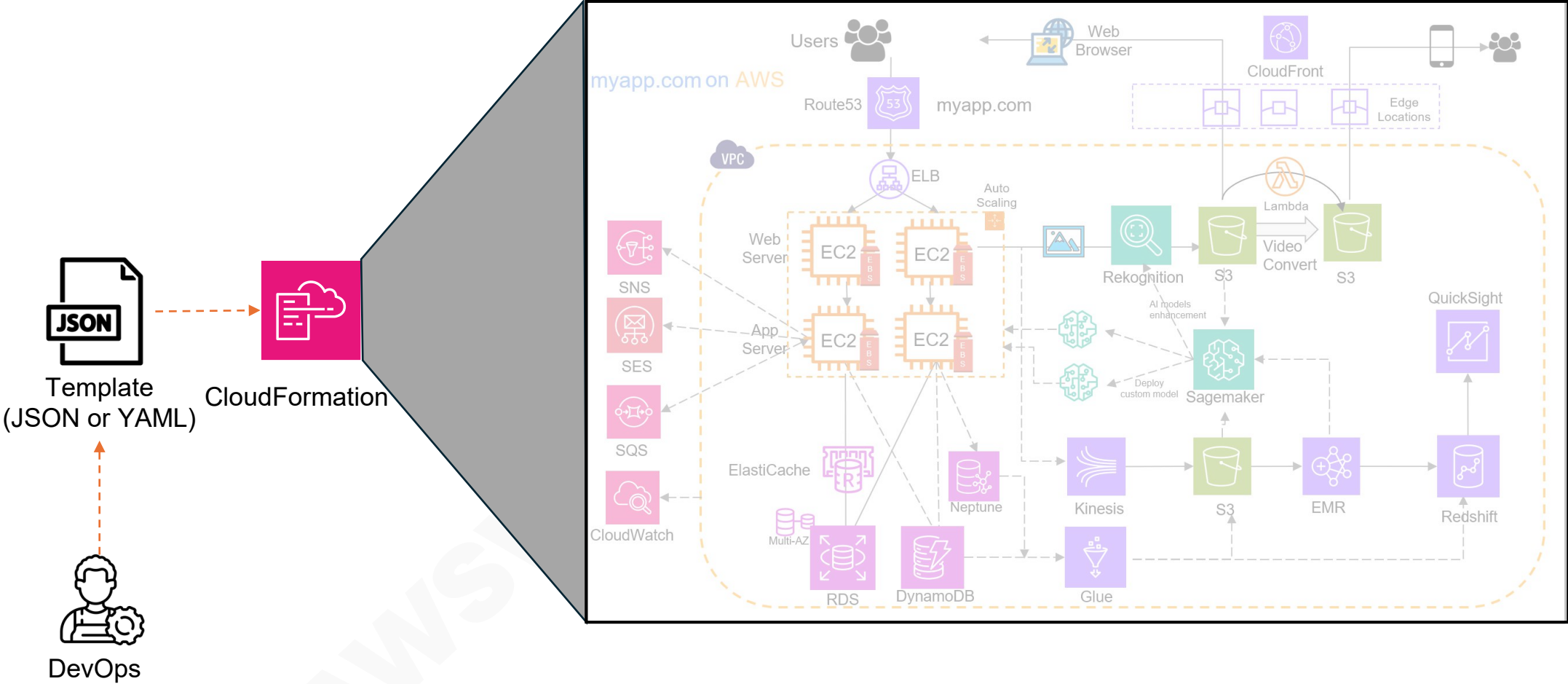
3<sup>rd</sup> Party/  
Opensource





# AWS CloudFormation

# Using AWS CloudFormation

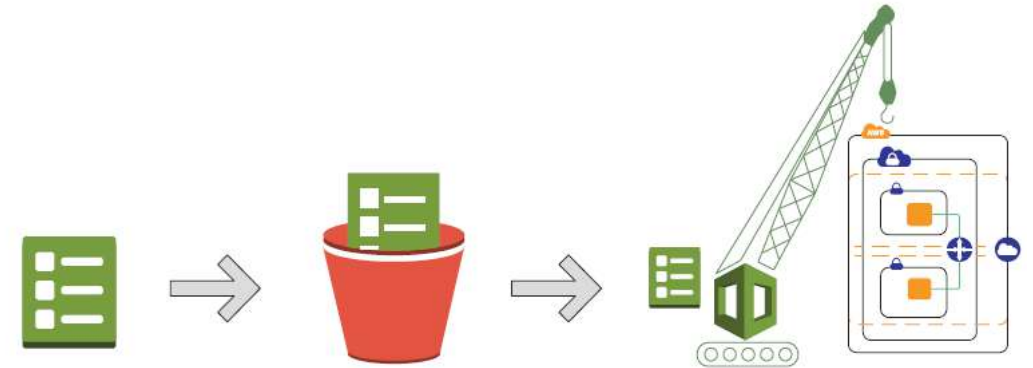


# AWS CloudFormation

AWS CloudFormation is a declarative way of outlining your AWS Infrastructure for any resources.

For example, within a CloudFormation template, you say:

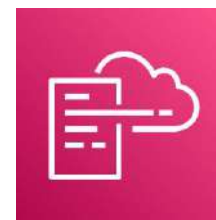
- ✓ I want a VPC and Subnets
- ✓ I want an internet gateway and attach it to the VPC
- ✓ I want a security group
- ✓ I want two EC2 machines using this security group in the subnet just created



Then CloudFormation creates those for you, in the **right order**, possibly **in parallel** and with the **exact configuration** that you specify.

# Benefits of using AWS CloudFormation

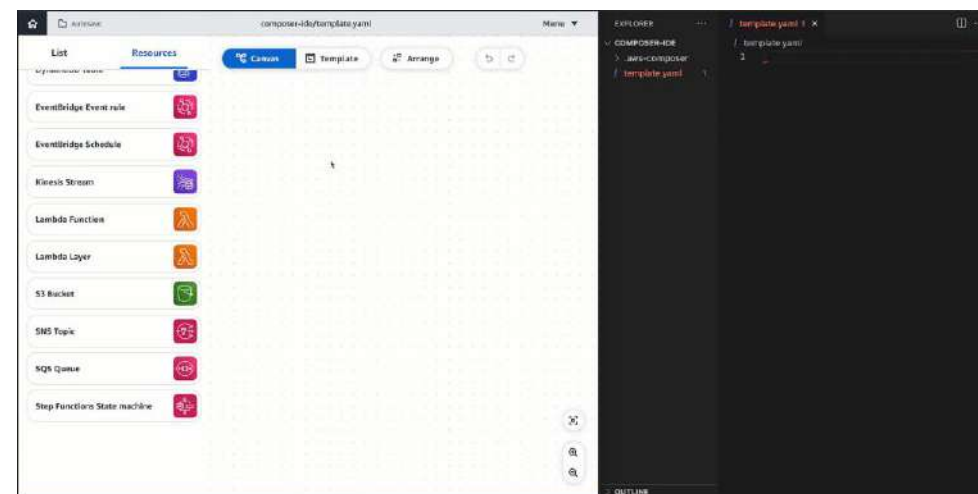
- AWS resources are created & deleted automatically, hence no manual errors
- Resources are created and deleted as a group (called Stack), hence no ghost resources on stack deletion
- AWS resources which are not dependent, are created in the parallel which speeds up infrastructure deployment significantly.
- Same CloudFormation template can be deployed in different AWS regions or accounts with minimal or no changes. Templates can be easily shared among teams.
- The template can be **version controlled** (using git, svn etc.) so it's easy to go back to previous deployment
- Supports creating CloudFormation template visually (drag/drop) using Infrastructure composer



AWS  
CloudFormation



Infrastructure  
Composer



# AWS CloudFormation – Important to know

- CloudFormation Template structure
- Nested Stacks
- StackSets
- Change Sets and Drift Detection
- Automatic Rollback and Deletion behavior

AWSWITHCHETAN.COM

# CloudFormation Template structure

- AWSTemplateFormatVersion: 2010-09-09
- **Description:** Text that describes the template
- **Metadata**
  - ✓ Additional information to improve usability e.g. parameter grouping for UI
- **Parameters**
  - ✓ Input values that you can pass when creating stacks
  - ✓ Can have default values and allowed values (with patterns)
- **Mappings**
  - ✓ Fixed variables (key-value pairs) defined for lookup
  - ✓ Common use: AMI IDs per region, instance sizes per env
- **Conditions**
  - ✓ Add flexibility to the template by evaluating conditions
  - ✓ Use condition Intrinsic functions: Fn::And, Fn::Or, Fn::If etc.
  - ✓ Example: If env=prod create EBS volume of size 1TB or 100GB

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Sample CloudFormation template
for SAA exam preparation

Metadata:
  Author: Chetan
  Purpose: Demonstrate CFN Template sections

Parameters:
  Env:
    Type: String
    Default: dev
    AllowedValues: [dev, prod]
    Description: Deployment environment

Mappings:
  InstanceConfig:
    dev: { InstanceType: t2.micro }
    prod: { InstanceType: t3.small }

Conditions:
  IsProd: !Equals [ !Ref Env, prod ]
...
...
```

# CloudFormation Template structure

- **Resources** (**Mandatory**)
  - ✓ Declares AWS resources to include in the stack
  - ✓ Example: EC2 instances, S3 buckets, RDS databases)
- **Outputs**
  - ✓ Values returned after stack creation
  - ✓ Can be exported for cross-stack references

```
Resources:
  EC2Instance:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-0abcdef1234567890
      InstanceType: !FindInMap [ InstanceConfig,
!Ref Env, InstanceType ]
      Tags:
        - Key: Name
          Value: !Sub "${Env}-ec2"
        - Key: Environment
          Value: !Ref Env
        - !If
          - IsProd
          - { Key: Critical, Value: "true" }
          - !Ref AWS::NoValue
  ElasticIP:
    Type: AWS::EC2::EIP
    Properties:
      Domain: vpc
  EIPAssociation:
    Type: AWS::EC2::EIPAssociation
    Properties:
      InstanceId: !Ref EC2Instance
      EIP: !Ref ElasticIP
Outputs:
  ElasticIP:
    Description: Elastic IP address
    Value: !Ref ElasticIP
```



# CloudFormation template examples

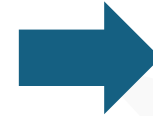
```
AWSTemplateFormatVersion: '2010-09-09'
Description: Launch an EC2 instance with SSH access

Resources:
  EC2Instance:
    Type: 'AWS::EC2::Instance'
    Properties:
      InstanceType: t2.micro
      KeyName: !Ref KeyName
      ImageId: ami-0c55b159cbfafa1f0 # Replace with your AMI ID
      SecurityGroups:
        - !Ref InstanceSecurityGroup

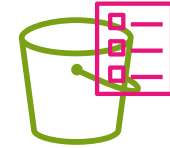
  InstanceSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: Allow SSH
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 22
          ToPort: 22
          CidrIp: 0.0.0.0/0

Parameters:
  KeyName:
    Description: Name of an existing EC2 KeyPair
    Type: String

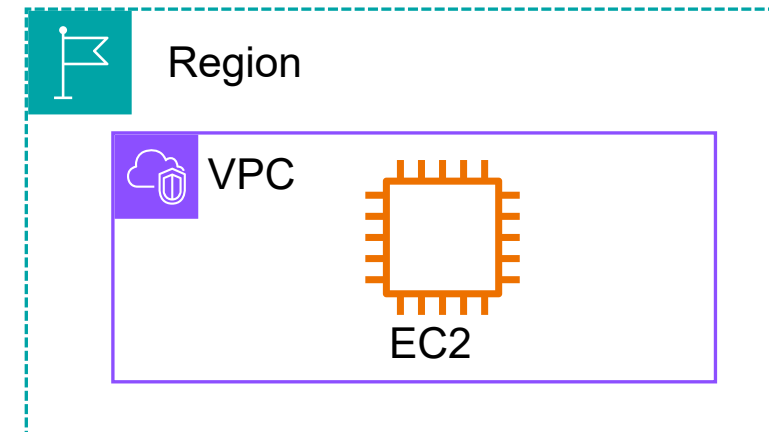
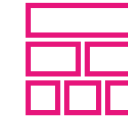
Outputs:
  InstanceId:
    Value: !Ref EC2Instance
  PublicIP:
    Value: !GetAtt EC2Instance.PublicIp
```



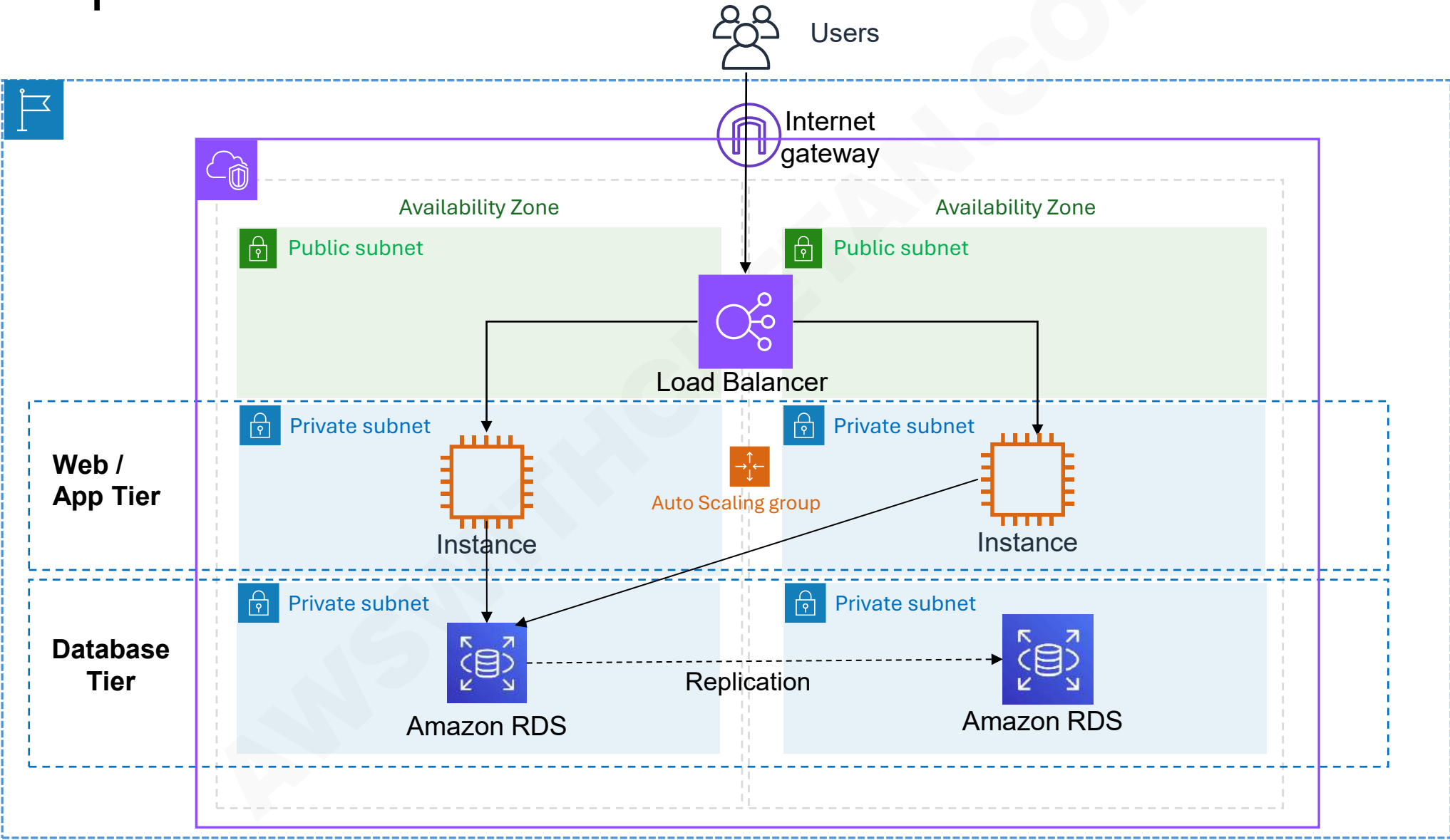
Upload to S3 bucket



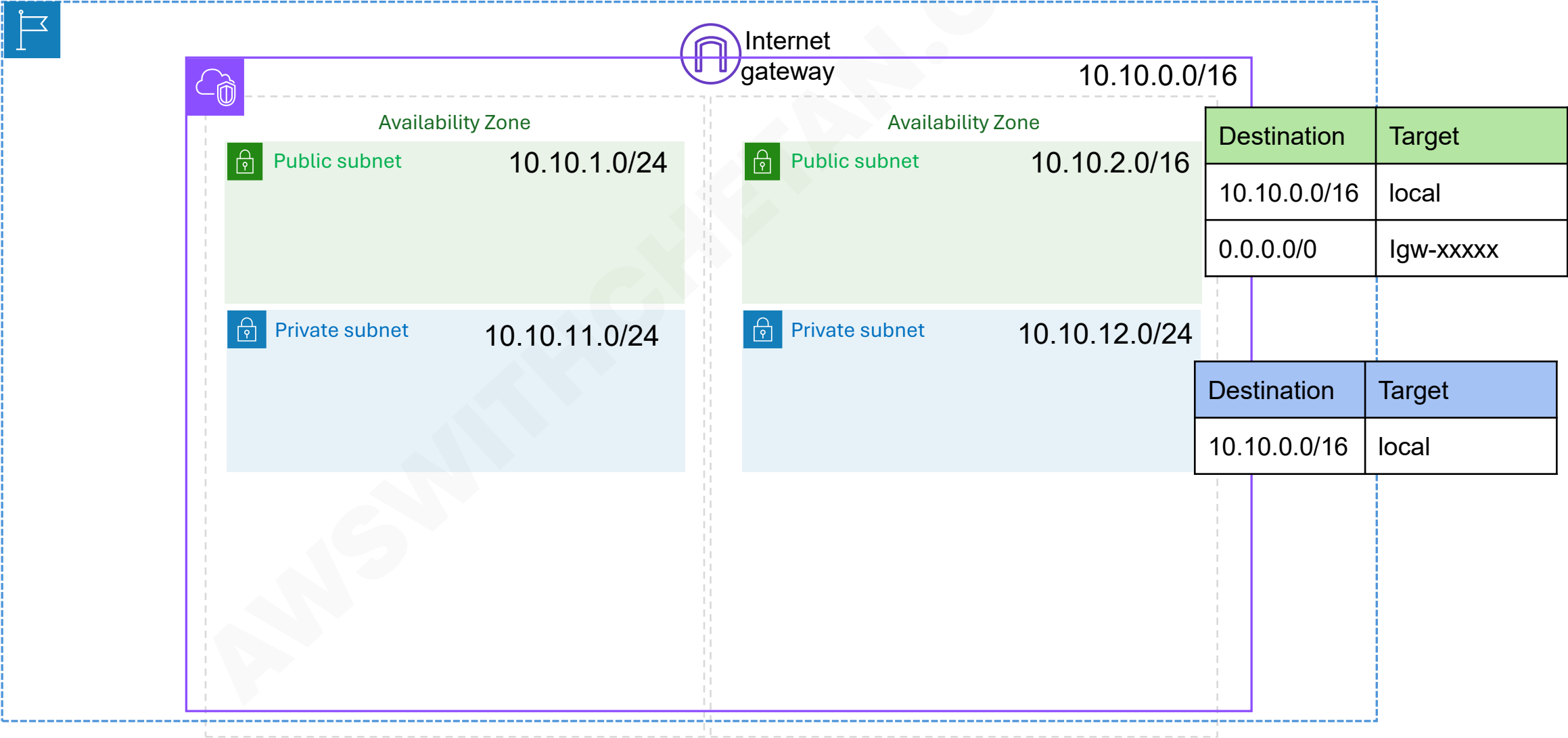
Create Stack



# Sample architecture..

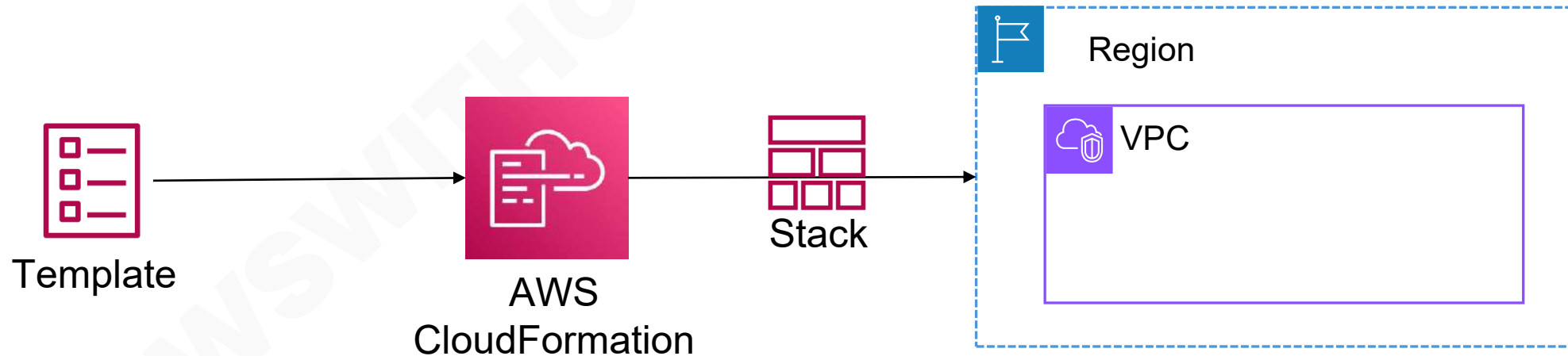


# Let's create VPC and related resources..



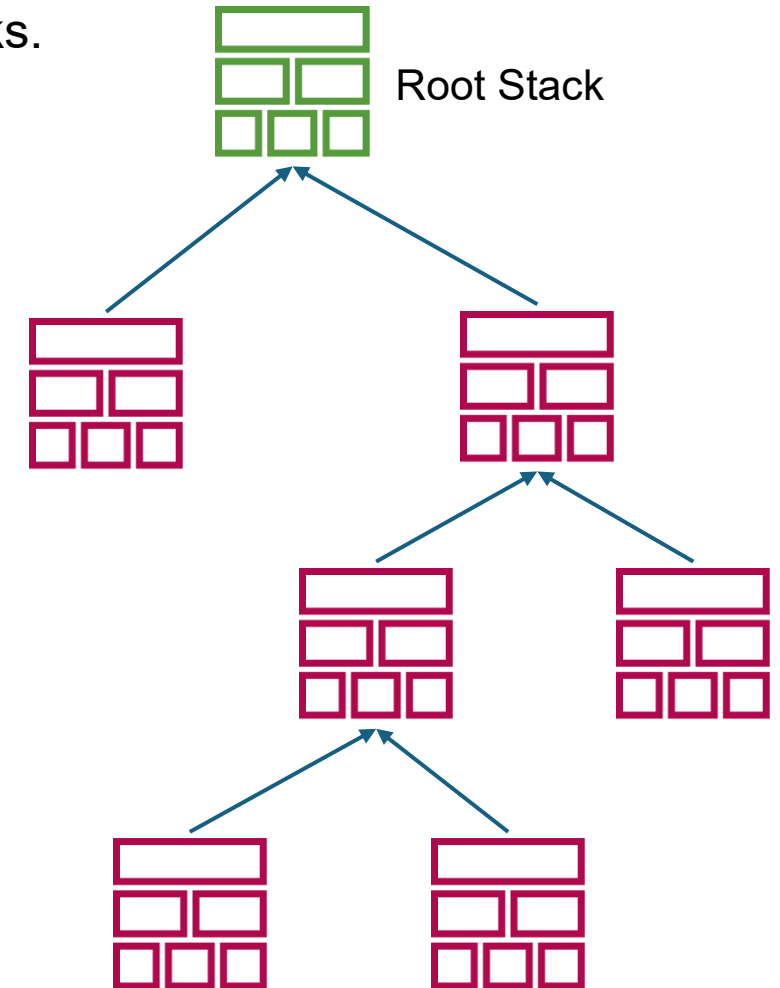
# Exercise – Create VPC using CloudFormation

1. Download a CloudFormation template: <https://github.com/awswithchetan/aws-solutions-architect-associate/blob/main/vpc.yaml>
2. Use this template to create a CloudFormation stack



# Nested Stack

- A parent stack creates and manages one or more child (nested) stacks.
- Useful for modularity, reuse, and managing template size limits.
- Common exam use cases:
  - Reusing VPC, IAM, or security baseline templates
  - Separating networking, compute, and database layers



# Example: Spitting the template using nested stack

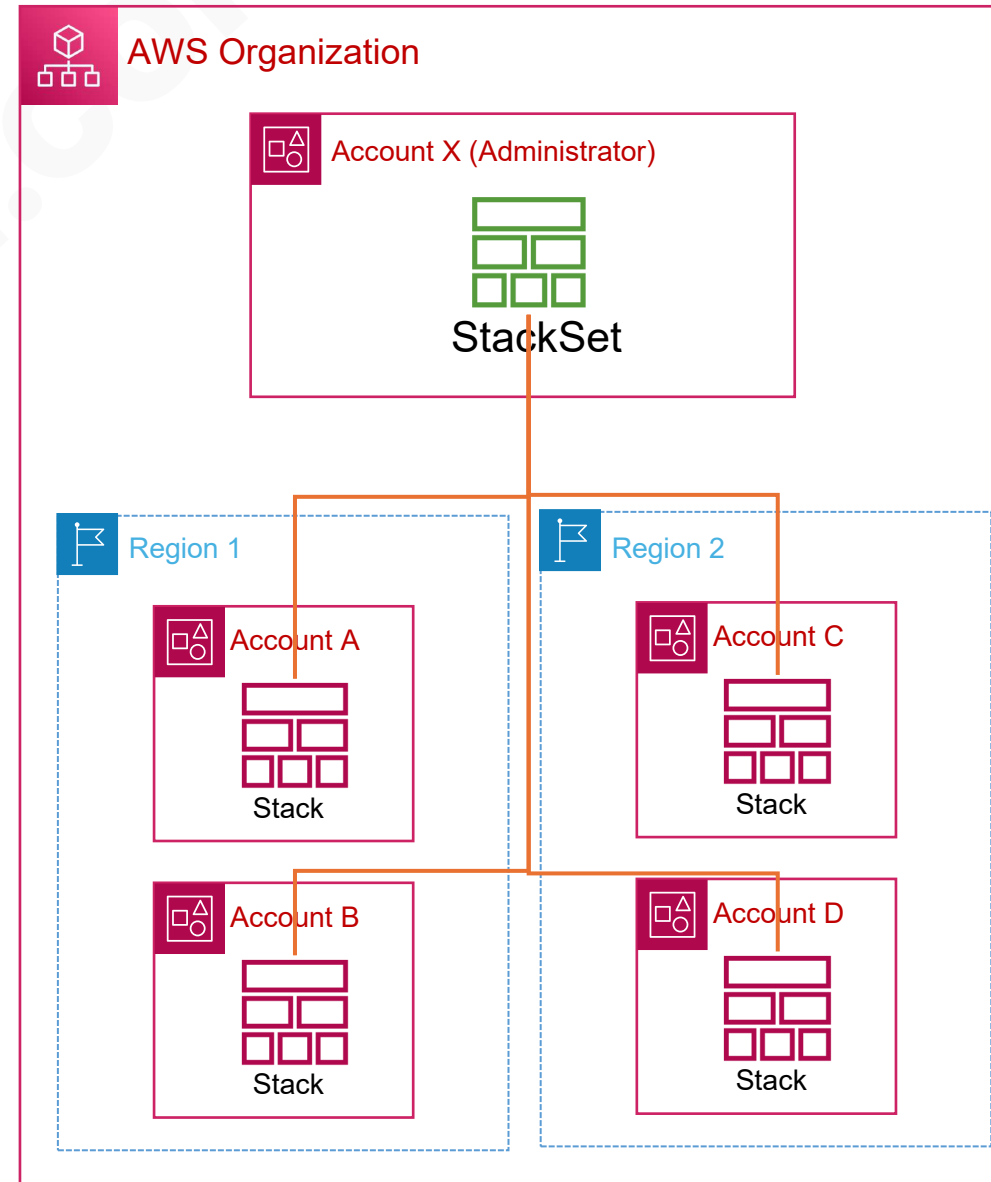
| Before nesting stacks                                                                                                                                                                                                                                                                                                                                                                                                                                          | After nesting stacks                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> AWSTemplateFormatVersion: 2010-09-09 Parameters:   InstanceType:     Type: String     Default: t2.micro     Description: The EC2 instance type    Environment:     Type: String     Default: Production     Description: The deployment environment  Resources:   MyEC2Instance:     Type: AWS::EC2::Instance     Properties:       ImageId: ami-1234567890abcdef0       InstanceType: !Ref InstanceType    MyS3Bucket:     Type: AWS::S3::Bucket </pre> | <pre> AWSTemplateFormatVersion: 2010-09-09 Resources:   MyFirstNestedStack:     Type: AWS::CloudFormation::Stack     Properties:       TemplateURL: https://s3.amazonaws.com/amzn-s3-demo-bucket/first-nested-stack.yaml       Parameters:         # Pass parameters to the nested stack if needed         InstanceType: t3.micro    MySecondNestedStack:     Type: AWS::CloudFormation::Stack     Properties:       TemplateURL: https://s3.amazonaws.com/amzn-s3-demo-bucket/second-nested-stack.yaml       Parameters:         # Pass parameters to the nested stack if needed         Environment: Testing     DependsOn: MyFirstNestedStack </pre> |

Source: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-nested-stacks.html>

# StackSets

- StackSets let you deploy same CloudFormation stacks across multiple AWS accounts and multiple Regions.
- Commonly used with AWS Organizations for centralized management.
- Creates stack instances in target accounts and Regions.
- Updates to a StackSet can be rolled out automatically to all stack instances.
- Supports drift detection across accounts and Regions.
- Provides failure tolerance and deployment order controls.
- Used for organization-wide resources like IAM roles, Config rules, and security baselines.

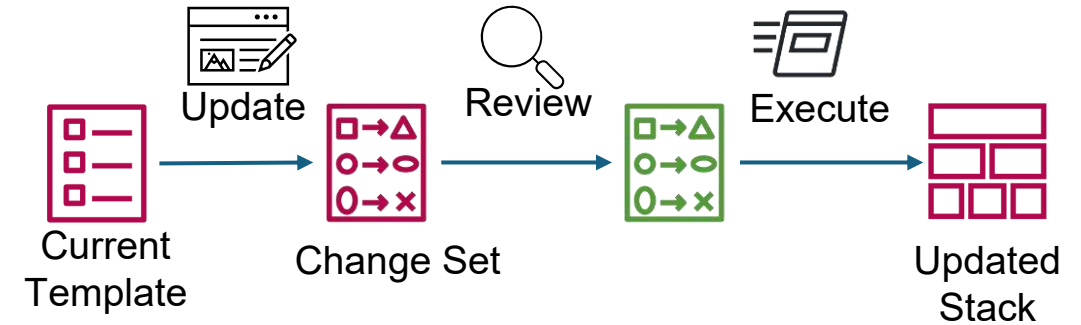
- ✓ Nested stacks = modular templates in one account/Region
- ✓ StackSets = same template across many accounts/Regions



# Change Sets and Drift Detection

## CloudFormation Change Sets

- Change sets let you preview changes before updating a stack.
- Show which resources will be added, modified, or deleted.
- Help avoid unintended resource replacement or deletion.
- Does not make changes until explicitly executed.





# Change Sets and Drift Detection

## Drift Detection

- Drift detection identifies manual changes made outside CloudFormation.
- Compares the actual resource configuration with the template.
- Marks stacks/resources as IN\_SYNC or MODIFIED.
- Helps detect configuration drift and enforce IaC discipline.

- ✓ Change Sets = Review impact before applying stack updates
- ✓ Drift Detection = Identify the resources modified outside of CloudFormation

| Differences (4)                     |                                          |           |                                    |                                                  |
|-------------------------------------|------------------------------------------|-----------|------------------------------------|--------------------------------------------------|
| <input checked="" type="checkbox"/> | Property                                 | Change    | Expected value                     | Current value                                    |
| <input checked="" type="checkbox"/> | AttributeDefinitions.1                   | ADD       | -                                  | {"AttributeName": "stuff", "AttributeType": "S"} |
| <input checked="" type="checkbox"/> | ProvisionedThroughput.ReadCapacityUnits  | NOT_EQUAL | 5                                  | 6                                                |
| <input checked="" type="checkbox"/> | ProvisionedThroughput.WriteCapacityUnits | NOT_EQUAL | 5                                  | 4                                                |
| <input checked="" type="checkbox"/> | Tags                                     | REMOVE    | [{"Key": "test", "Value": "test"}] | -                                                |

| Expected                                                                                                                                                                                                                                                                                                                                                                        | Actual                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{   "AttributeDefinitions": [     {       "AttributeName": "Name",       "AttributeType": "S"     }   ],   "KeySchema": [     {       "AttributeName": "Name",       "KeyType": "HASH"     }   ],   "ProvisionedThroughput": {     "ReadCapacityUnits": 5,     "WriteCapacityUnits": 5   },   "Tags": [     {       "Key": "test",       "Value": "test"     }   ] }</pre> | <pre>{   "AttributeDefinitions": [     {       "AttributeName": "Name",       "AttributeType": "S"     },     {       "AttributeName": "stuff",       "AttributeType": "S"     }   ],   "KeySchema": [     {       "AttributeName": "Name",       "KeyType": "HASH"     }   ],   "ProvisionedThroughput": {     "ReadCapacityUnits": 6,     "WriteCapacityUnits": 4   } }</pre> |

Source: <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/detect-drift-stack.html>

# Automatic Rollback and Deletion behavior

## Rollback:

- Rollback occurs automatically if stack creation or update fails.
- During rollback, CloudFormation attempts to return the stack to the last known stable state.
- For stack creation failures, all successfully created resources are deleted by default.
- You can disable Rollback during stack creation (useful for troubleshooting).

## Resource Deletion Policy:

- Retain – Resource is kept after stack deletion. Prevent accidental deletion of data
- Snapshot – Creates snapshot (for supported resources like RDS).
- Delete – default behavior.

Resources:

MyDB:

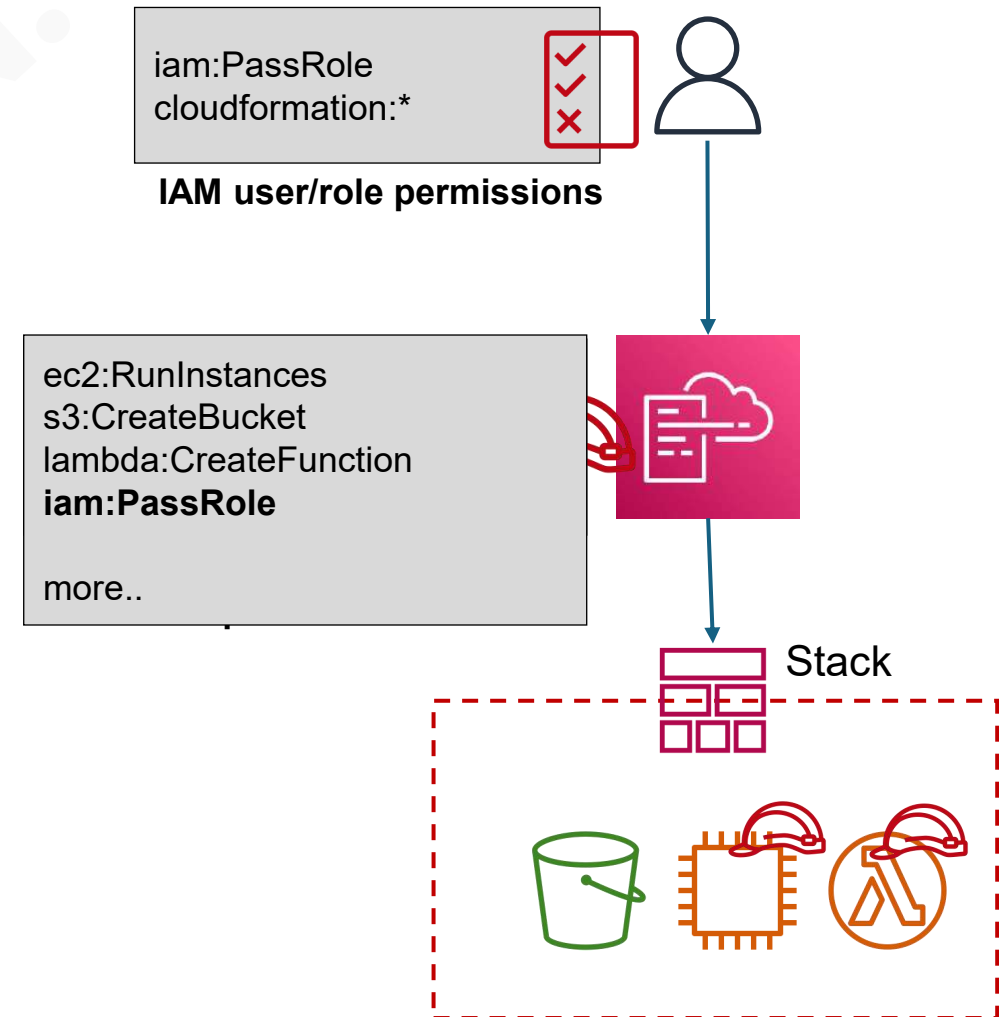
Type: AWS::RDS::DBInstance

DeletionPolicy: Retain

...

# CloudFormation IAM Service Role

- CloudFormation does not have permissions by default to create resources.
- It uses the permissions of the IAM user or role that launches the stack OR uses **CloudFormation service role**.
- **iam:PassRole** allows an IAM principal to pass a Service Role to the CloudFormation service.
- If CloudFormation service needs to further create AWS resources (e.g. EC2 instance with IAM role) then Service role should also include **iam:PassRole** permissions.



# AWS CloudFormation - Summary

- AWS-native Infrastructure as Code service to provision and manage AWS resources consistently.
- Uses JSON/YAML template to define the desired state of infrastructure
- Template contains different sections like Resources (mandatory), Parameters, Mappings, Conditions, Outputs etc.
- Nested stacks allow large templates to be split into smaller, reusable components managed by parent template.
- StackSets enable deploying the same CloudFormation template across multiple AWS accounts and Regions
- StackSets are commonly used with AWS Organizations for centralized governance.
- Change sets let you preview which resources will be added, modified, or deleted before executing the changes.
- Drift detection identifies changes made outside CloudFormation by comparing actual resources with the template.
- CloudFormation automatically rolls back failed stack creations or updates to the last stable state.
- DeletionPolicy controls resource behavior during rollback or stack deletion. Supports Retain, Snapshot & Delete actions.
- CloudFormation can use a IAM Service Role to create/update/delete resources.
- IAM user/role should have iam:PassRole permissions to pass Service Role to CloudFormation.

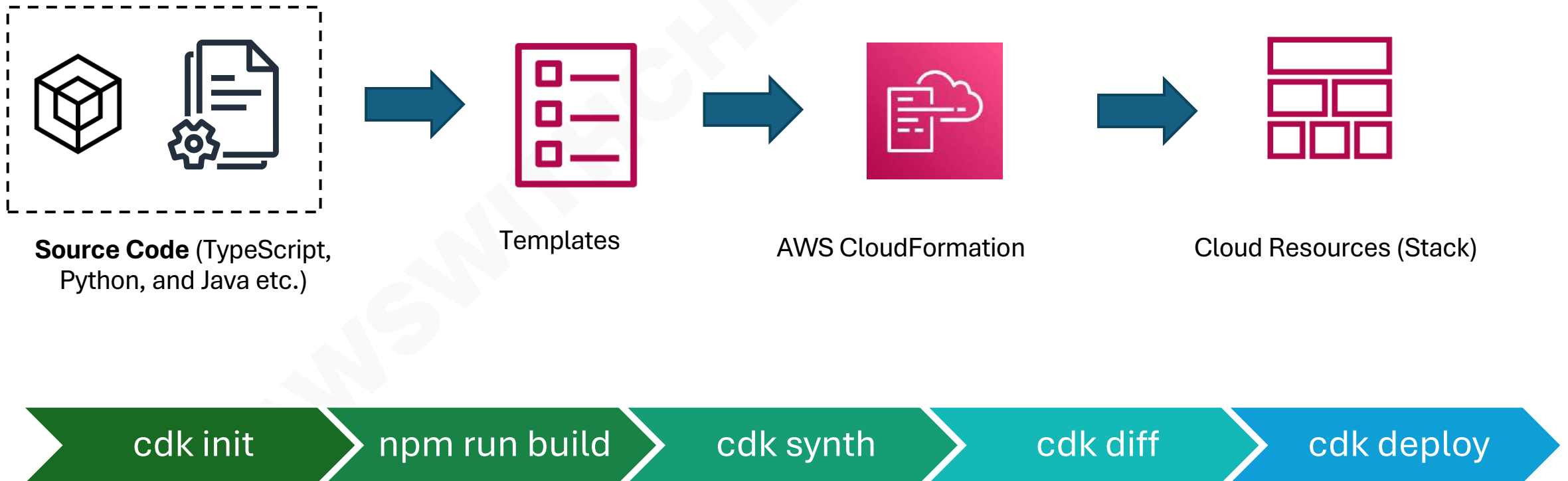




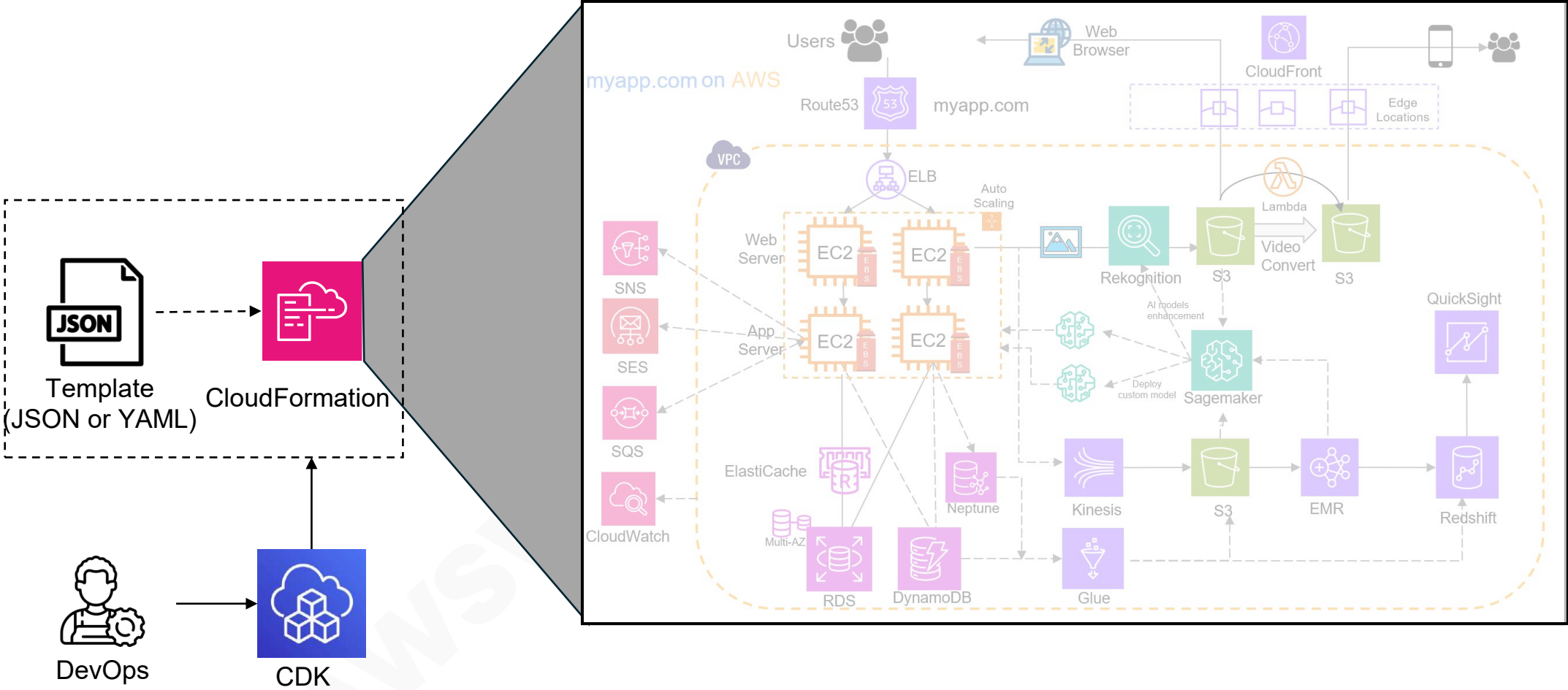
# AWS CDK

# AWS Cloud Development Kit (CDK)

An open-source software development framework to define your cloud application resources using familiar programming languages.



# Using AWS CDK







 Write code in language of choice

# CDK code example

```
from aws_cdk import (
    aws_ec2 as ec2,
    core
)
class EC2InstanceStack(core.Stack):
    def __init__(self, scope: core.Construct, id: str, **kwargs) -> None:
        super().__init__(scope, id, **kwargs)

        # VPC where the instance will be launched
        vpc = ec2.Vpc.from_lookup(self, "VPC", is_default=True)

        # Security Group to allow SSH access
        security_group = ec2.SecurityGroup(self, "InstanceSG",
            vpc=vpc,
            description="Allow SSH access",
            allow_all_outbound=True
        )
        security_group.add_ingress_rule(
            peer=ec2.Peer.any_ipv4(),
            connection=ec2.Port.tcp(22),
            description="Allow SSH access from anywhere"
        )
        instance = ec2.Instance(self, "EC2Instance",
            instance_type=ec2.InstanceType("t2.micro"),
            machine_image=ec2.MachineImage.latest_amazon_linux(),
            vpc=vpc,
            security_group=security_group,
            key_name="your-key-pair-name" # Replace with your key pair name
        )

        # Output the instance ID and public IP
        core.CfnOutput(self, "InstanceID", value=instance.instance_id)
        core.CfnOutput(self, "InstancePublicIP", value=instance.instance_public_ip)

app = core.App()
EC2InstanceStack(app, "EC2InstanceStack")
app.synth()
```

*ec2-instance-cdk.py*



```
$npm install -g aws-cdk
```

```
$mkdir ec2-instance-cdk
```

```
$cd ec2-instance-cdk
```

```
$cdk init app --language python
```

```
$pip install aws-cdk.aws-ec2
```

```
$cdk deploy
```



# Application Deployment Services

AWS Elastic Beanstalk and Amazon Lightsail

# In this section..

## Infrastructure as Code

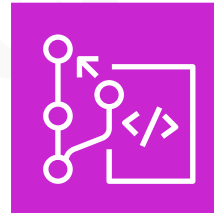


AWS CloudFormation

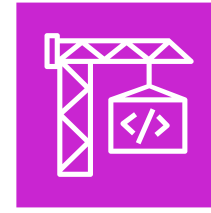


AWS CDK

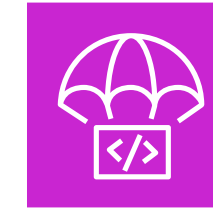
## DevOps – CI/CD



CodeCommit



CodeBuild



CodeDeploy



CodePipeline

## Application Deployment



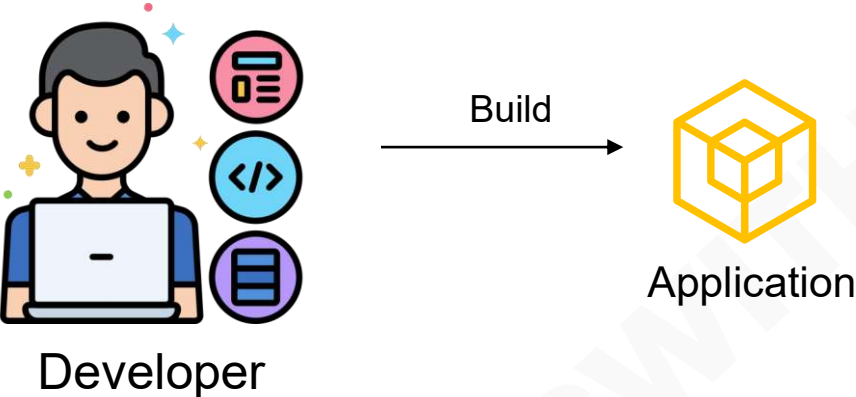
Elastic Beanstalk



Amazon Lightsail

# Application Deployment

# What developers want?



# What developers want?



Developer

Build and Run



Application

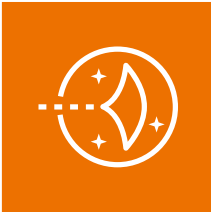


Managed  
Application  
Platform

Virtual Server to run small  
apps and websites



AWS Elastic  
Beanstalk



Amazon Lightsail

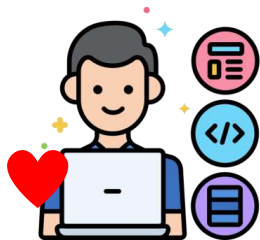
# AWS Elastic Beanstalk

- AWS Elastic Beanstalk is a Platform as a Service (PaaS)
- Developers can deploy applications quickly without managing infrastructure details.
- It abstracts underlying infrastructure such as EC2, load balancers, Auto Scaling groups, and scaling policies.
- Just upload application code, and Elastic Beanstalk automatically provisions, manages, and scales the environment.
- Supports multiple application platforms including Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker.
- Runs applications on common web servers like Apache, Nginx, Passenger, and IIS.

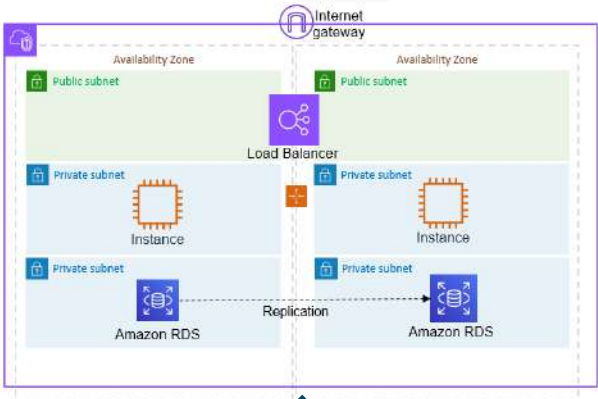


AWS Elastic Beanstalk

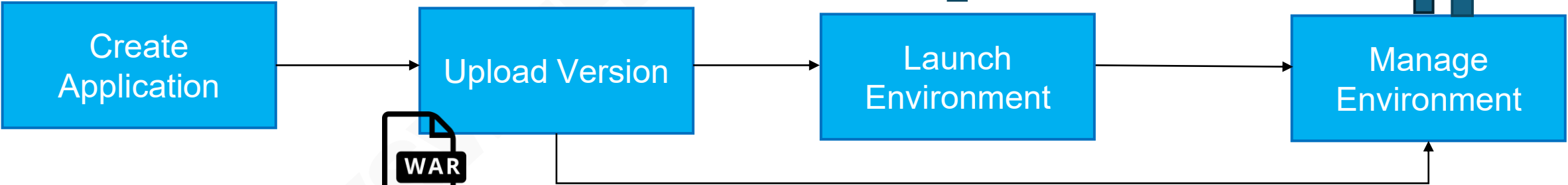
# AWS Elastic Beanstalk



AWS Elastic Beanstalk



Health monitoring



Deploy New Version

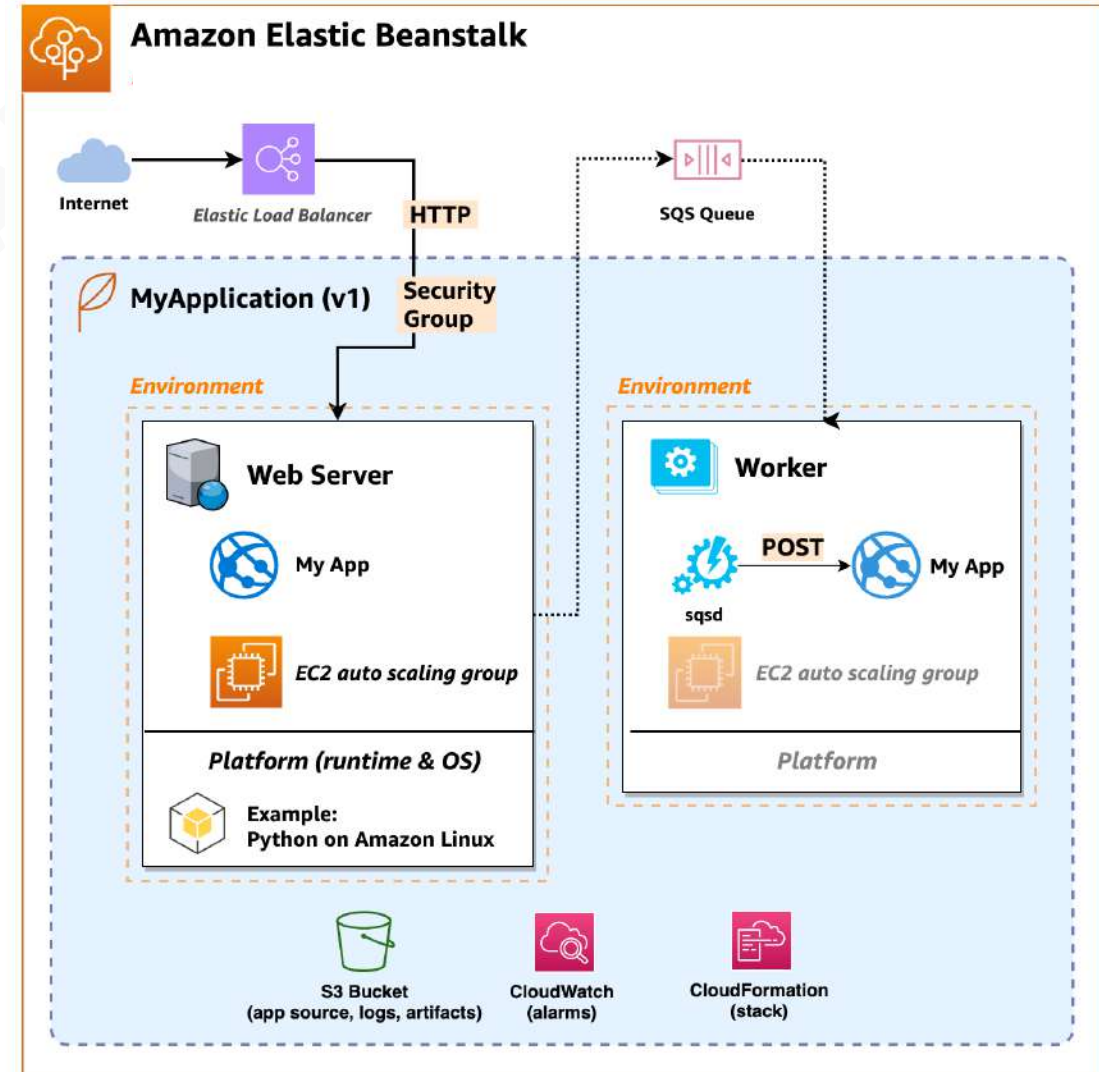
# AWS Elastic Beanstalk - Platform Support

| Application frameworks                                                                                                                                                                                               | Container platforms                                                       | AWS services                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>■ Go</li> <li>■ Java SE</li> <li>■ Java with Tomcat</li> <li>■ .NET on Windows Server with IIS</li> <li>■ Node.js</li> <li>■ PHP</li> <li>■ Python</li> <li>■ Ruby</li> </ul> | <ul style="list-style-type: none"> <li>■ Docker</li> <li>■ ECS</li> </ul> | <ul style="list-style-type: none"> <li>■ EC2</li> <li>■ Elastic Load Balancer</li> <li>■ Auto-scaling Group</li> <li>■ CloudWatch</li> <li>■ RDS</li> <li>■ Route 53</li> <li>■ CloudFront</li> <li>■ More..</li> </ul> |



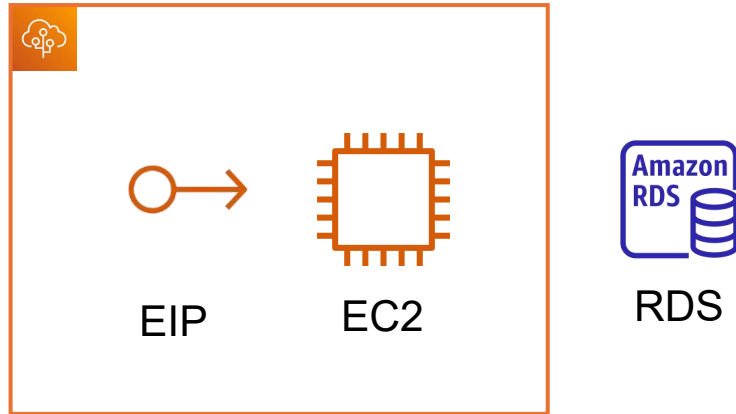
# Elastic Beanstalk Environments

- **Web Server environment**
  - Used for HTTP/HTTPS request–response web applications.
  - Supports capacity modes:
    - Single-Instance web environment
    - Load Balanced web environment
- **Worker environment**
  - Used for background or asynchronous processing.
  - Always process messages from an Amazon SQS queue
  - Do not receive direct HTTP traffic from users
  - Are always load-balanced internally using Auto Scaling



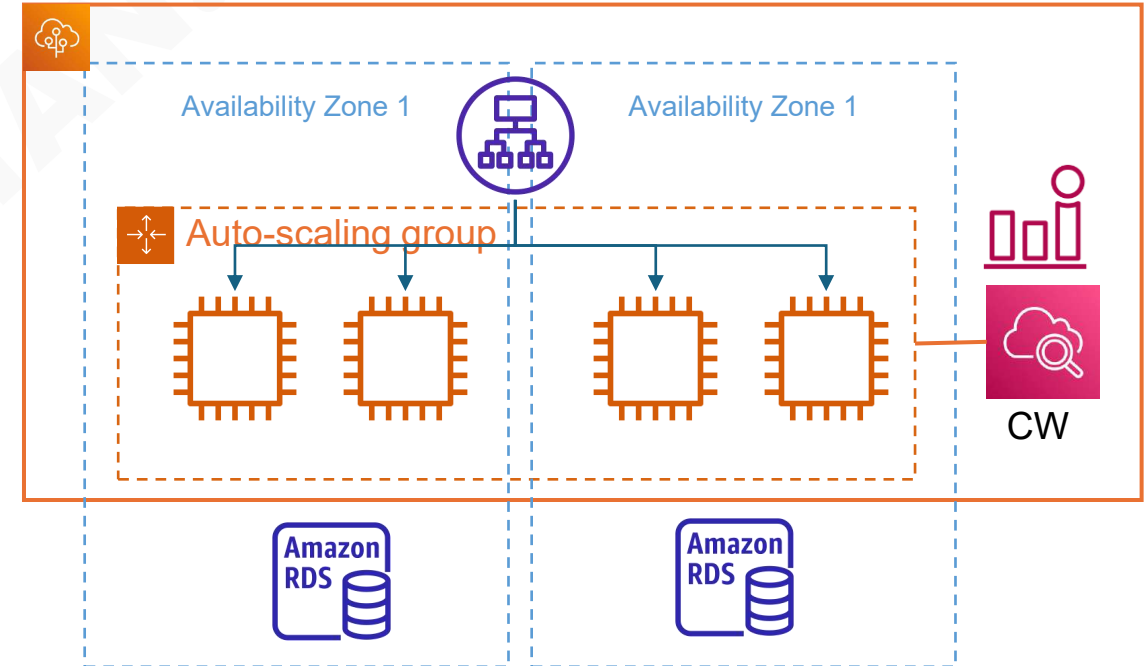
# Web server environment

## Single-Instance



- One EC2 instance
- No load balancer
- Lowest cost
- Suitable for dev, test, demos, or very low traffic apps

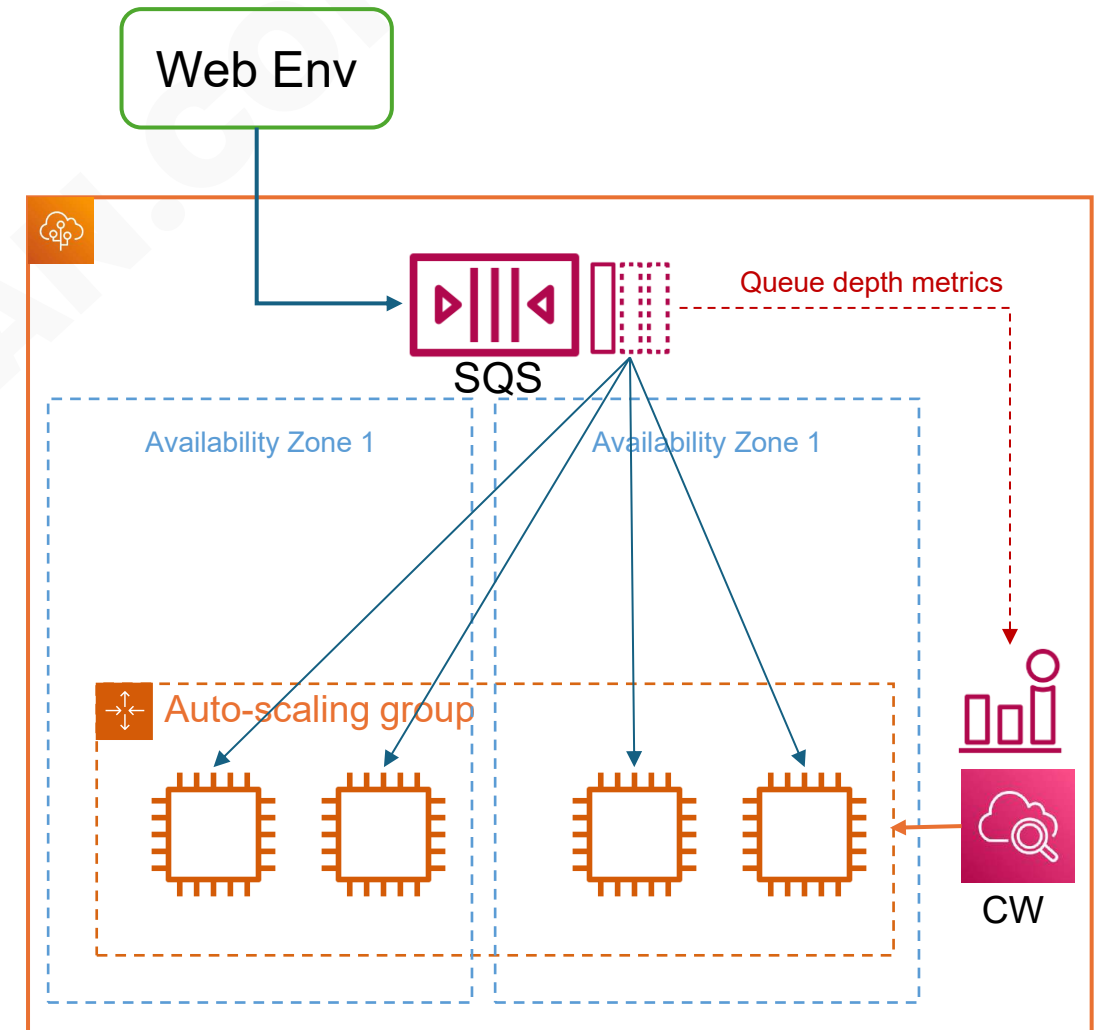
## Load-Balanced



- Highly available and Scalable
- Use ALB and Autoscaling group
- Suitable for high traffic production apps

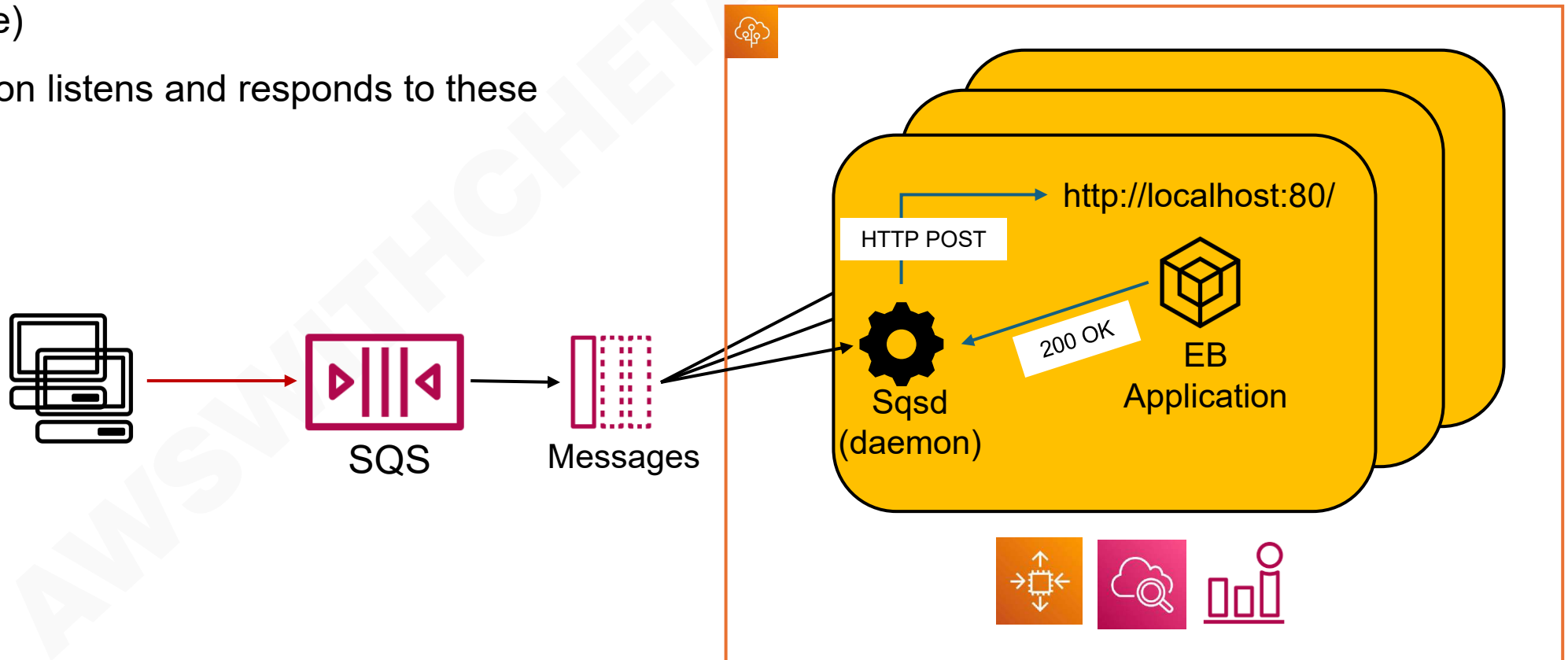
# Worker environment

- Used for background / asynchronous processing
- Optionally, automatically provisions an Amazon SQS queue to receive work messages.
- EC2 instances poll the SQS queue and process messages.
- Uses Auto Scaling based on SQS queue depth (messages in queue).
- Supports cron.yaml for scheduled background jobs.
- If processing fails, messages can be retried or sent to a dead-letter queue (DLQ).
- Typically paired with a web environment that pushes jobs to SQS.
- Best for tasks like image processing, email sending, data transformation, and batch jobs.



# Worker environment – How it works?

- Elastic Beanstalk runs the SQS daemon process on the worker instances
- Daemon pulls the message from SQS queue and makes HTTP POST request on localhost:80 (configurable)
- EB Application listens and responds to these requests



# Elastic Beanstalk deployment methods

- **All at once** – Deploys the new version to all instances simultaneously, causing small downtime during deployment.
- **Rolling** – Updates instances in batches, reducing downtime but temporarily lowering capacity.
- **Rolling with additional batch** – Launches an extra batch of instances before updating, maintaining capacity during deployment.
- **Immutable** – Creates a new set of instances with the new version and swaps them after validation, offering the safest deployment.
- **Traffic splitting** – Gradually shifts a percentage of traffic to the new version while both versions run in parallel.
- **Blue/Green** - Runs two separate environments (current and new), then switches all traffic to the new environment at once after validation, enabling fast rollback.

# Elastic Beanstalk deployment methods

| Deployment Method             | Code deployed to           | Deployment Time                                                                                                                                                                                                                                                                                                                                        | Downtime | DNS Change | Rollback process                            |
|-------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------|---------------------------------------------|
| All at Once                   | Existing instances         |                                                                                                                                                                                                                                                                      | Yes      | No         | Manual redeploy                             |
| Rolling                       | Existing instances         |                                                                                                                                                                                   | No       | No         | Manual redeploy                             |
| Rolling with additional batch | New and existing instances |                                                                                                                                                                                   | No       | No         | Manual redeploy                             |
| Immutable                     | New instances              |             | No       | No         | Terminate new instances                     |
| Traffic Splitting             | New instances              |         | No       | No         | Reroute traffic and terminate new instances |
| Blue/Green                    | New instances              |     | No       | Yes        | Swap URL                                    |

# Amazon Lightsail

- For everything you need to jumpstart your project on AWS.
- Quickly deploy popular applications
  - ✓ WordPress, LAMP, Nginx, MEAN, Node.js etc.
- Create preconfigured Virtual Private Server (VPS) or Database or Container service
- Fixed price as per selected bundle
- **Use Cases:**
  - ✓ Simple web applications
  - ✓ Websites (templates for WordPress, Magento, Plesk, Joomla)
  - ✓ Dev / Test environment



Amazon  
Lightsail

## Select a platform

☒ Linux/Unix  
26 blueprints

☐ Microsoft Windows  
6 blueprints

## Select a blueprint

Apps + OS

Operating System (OS) only

☒ WordPress  
6.6.1-0

☐ WordPress  
Multisite  
6.6.1-0

☐ LAMP (PHP 8)  
8.3.9-2

☐ Node.js  
18.20.4-0

☐ Joomla  
5.1.2-1

☐ Magento  
2.4.7-4

☐ MEAN  
7.0.12-3

☐ Drupal  
10.3.1-2

☐ GitLab CE  
16.11.5-ce.0-0

☐ Redmine  
5.1.3-1

☐ Nginx  
1.25.5-2

☐ Ghost  
5.88.3-0

☐ Django  
4.2.14-2

☐ PrestaShop  
8.1.7-1

☐ Plesk Hosting  
Stack on Ubuntu  
(BYOL)  
18.0.62

☐ cPanel & WHM  
for AlmaLinux  
RELEASE Tier

# AWS Monitoring and Logging

Amazon CloudWatch, AWS X-ray, AWS Health



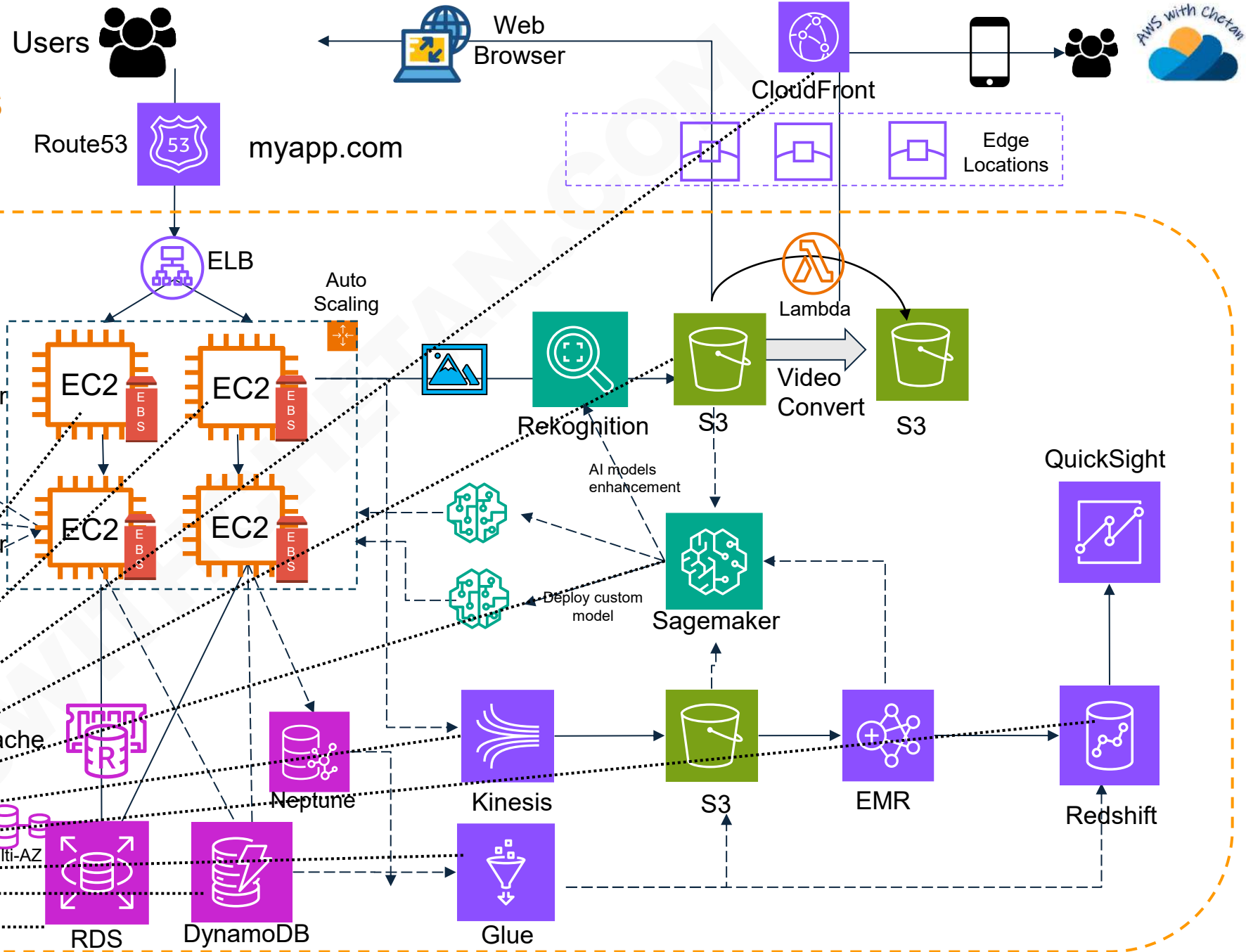
# AWS Monitoring and logging services

- **Amazon CloudWatch**
- AWS X-Ray
- AWS Health

AWSWITHCHETAN.COM



myapp.com on AWS





# Amazon CloudWatch

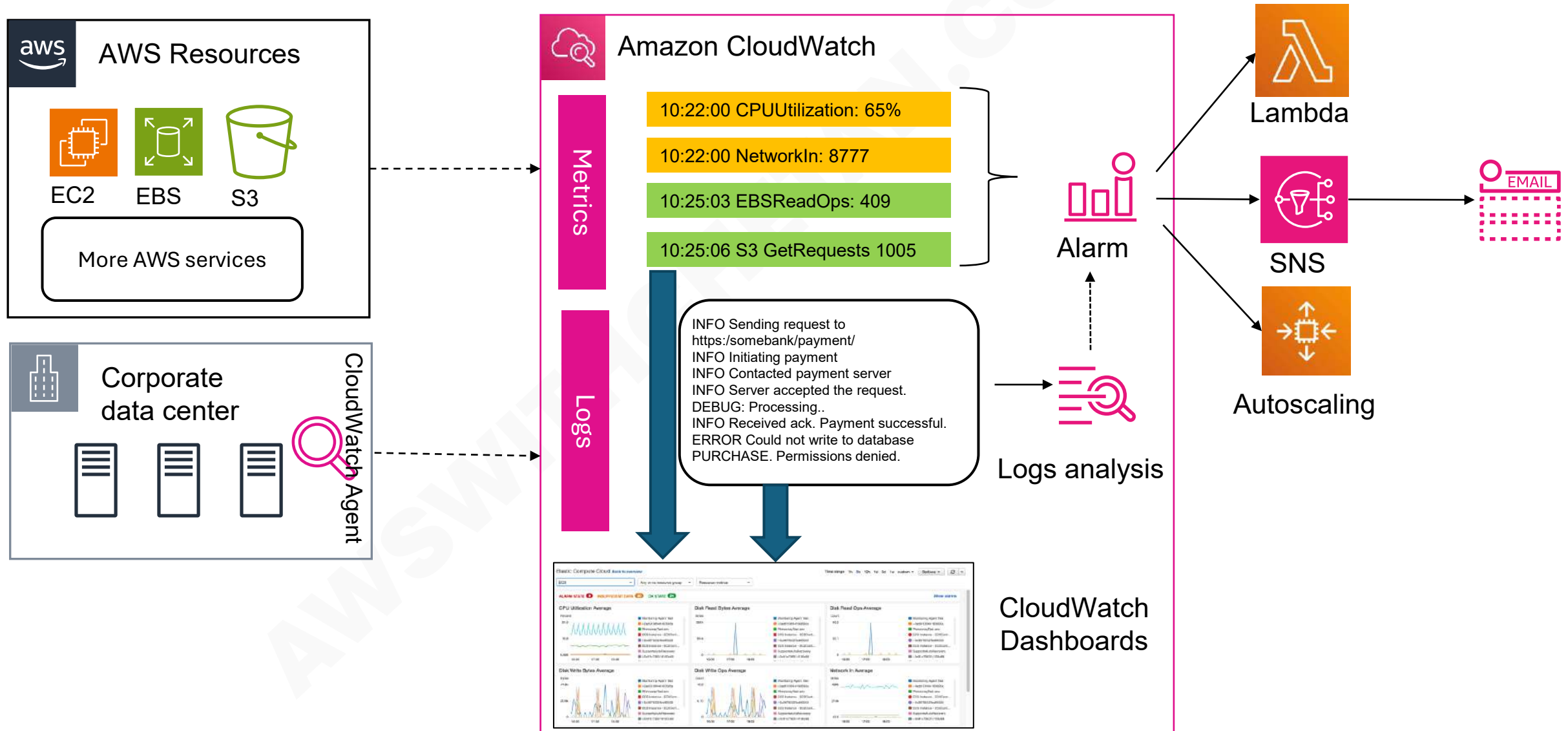
# Amazon CloudWatch

- Collect and tracks metrics, logs, and events for AWS resources and applications.
- Create custom dashboards for a visual representation of metrics and logs, aiding in monitoring and decision-making.
- Provides real-time and historical data
- Set alarms to automatically trigger actions (e.g., scaling, notifications) based on defined thresholds.
- Seamlessly integrates with majority of the AWS services (e.g., EC2, RDS, Lambda) and also supports third-party apps via API.



Amazon  
CloudWatch

# How CloudWatch works?

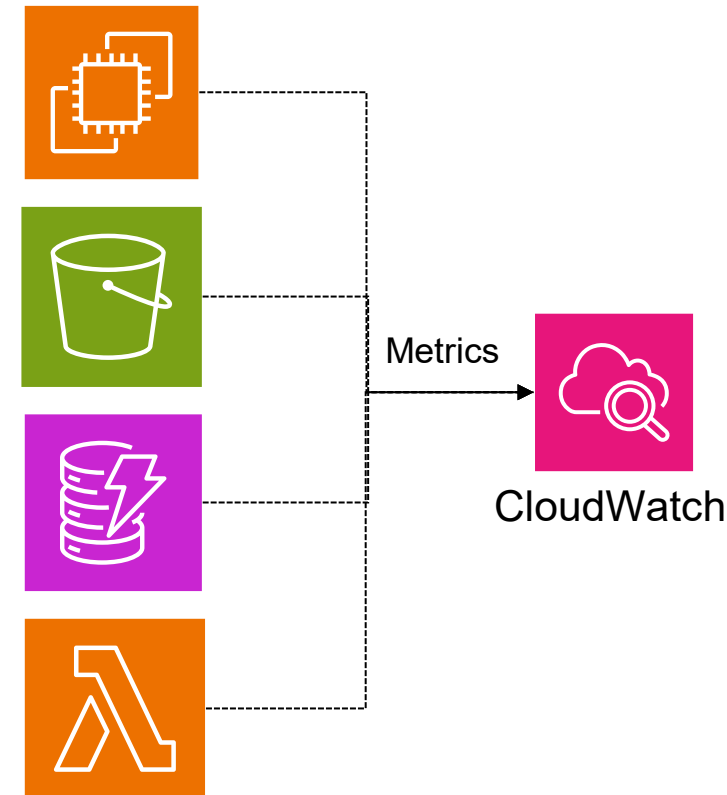


# Amazon CloudWatch Features

- **CloudWatch Metrics**
- **CloudWatch Metric Streams**
- **CloudWatch Alarms**
- **CloudWatch Logs**
- **CloudWatch Logs insights**
- **CloudWatch Dashboards**
- CloudWatch Events -> Now Amazon EventBridge
- CloudWatch Synthetics
- CloudWatch ServiceLens
- CloudWatch Anomaly detection
- CloudWatch RUM
- More..

# CloudWatch Metrics - Basics

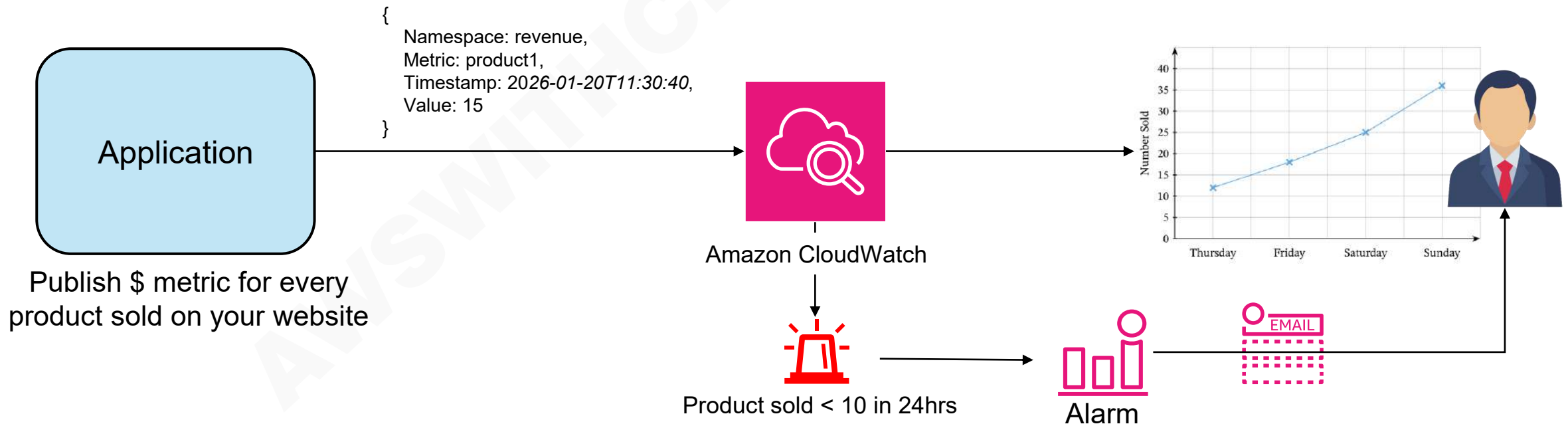
- CloudWatch Metric is the unit of measurement for system performance at a specific time, such as:
  - EC2: CPUUtilization
  - Network: NetworkIn and NetworkOUT Bytes
  - EBS: DiskReads and DiskWrites
  - S3: NumberOfRequests
- Metrics are organized into namespaces based on the AWS service. e.g. AWS/EC2, AWS/S3, AWS/EBS, AWS/ELB etc.
- A dimension is a key-value attribute that identifies a metric (e.g. For AWS/EC2 namespace, InstanceId is one of the dimension)
- Metrics data is stored as a timeseries to be able to analyse and act based on metrics values (e.g. raise an alarm when CPUUtilization > 90% for 5 mins)
- CloudWatch Supports:
  - **Basic monitoring** – Metrics are published every 5 minutes (default)
  - **Detailed monitoring** – Metrics are published every 1 minute (enabled manually, additional cost).



# CloudWatch custom metrics

- CloudWatch supports publishing custom metrics which can be pushed to existing or custom namespaces.
- Custom metrics are commonly used for application-level and business metrics.

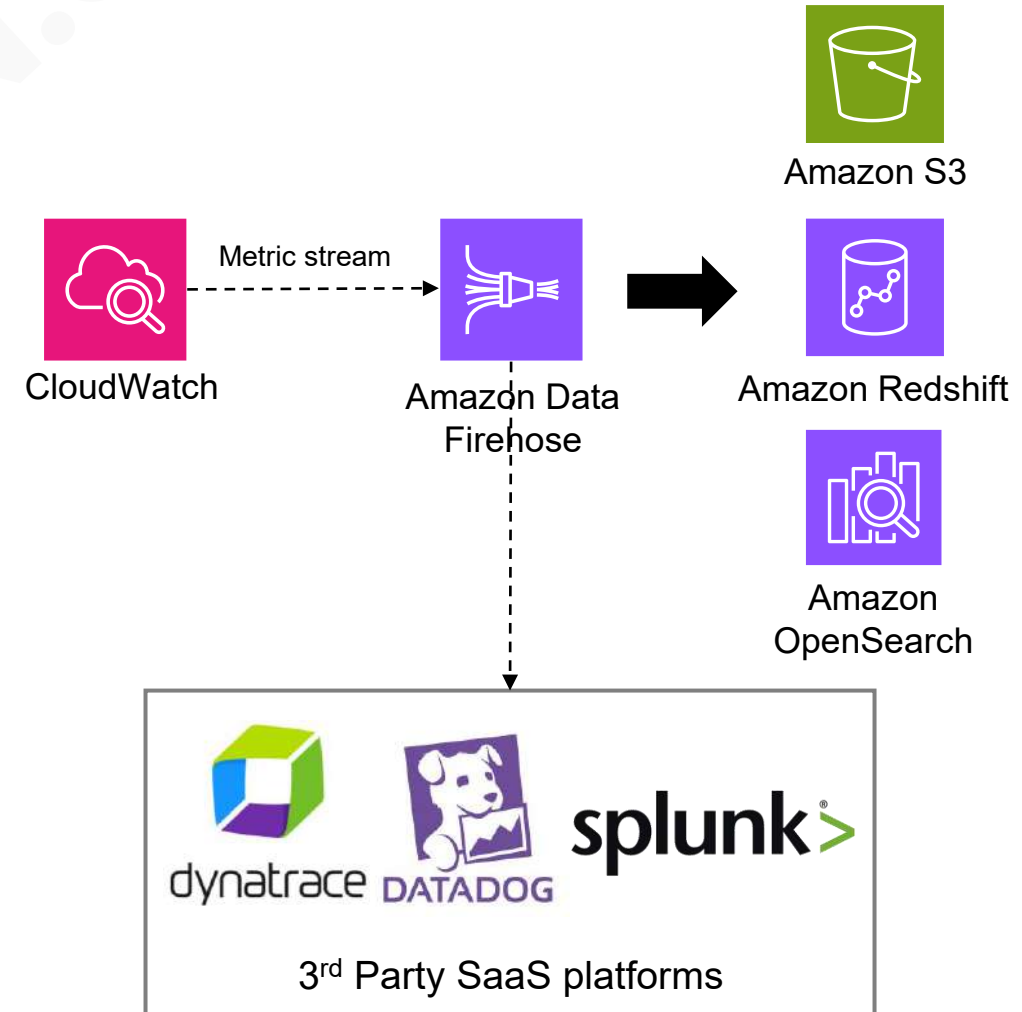
```
$aws cloudwatch put-metric-data --metric-name product1 --namespace revenue --value 15 --timestamp 2026-01-20T11:30:40.000Z
```





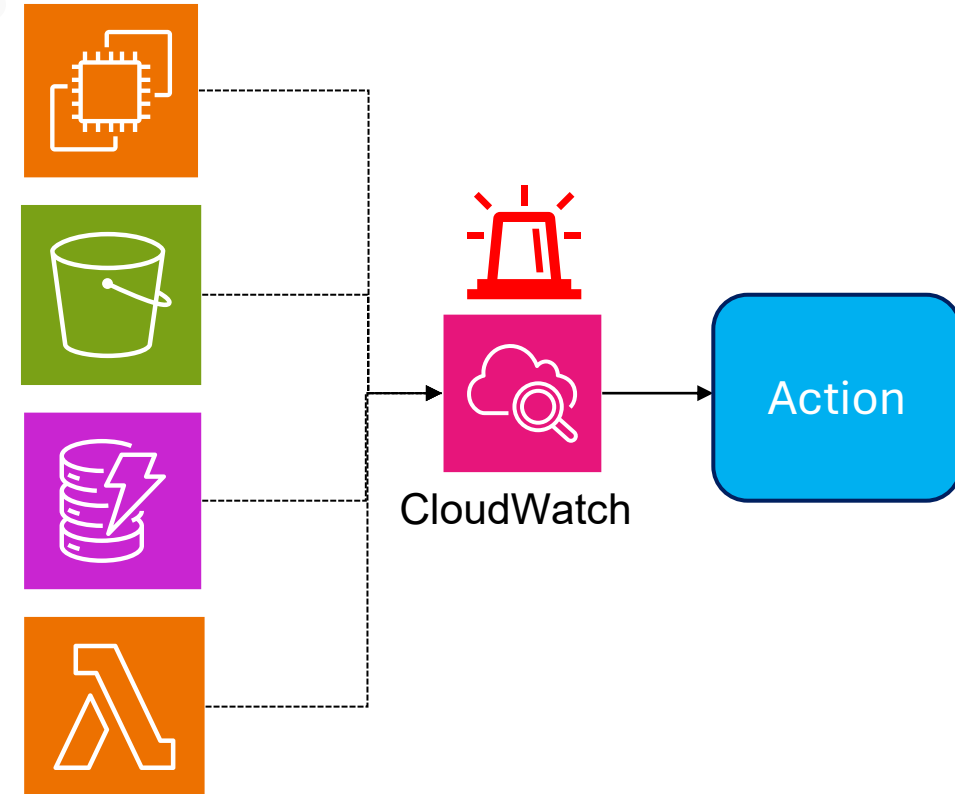
# CloudWatch Metric Streams

- CloudWatch Metric Streams allow you to continuously stream metrics from CloudWatch to different destinations.
- Metrics are delivered in near real-time with low latency, making them suitable for real-time monitoring and analytics.
- Metrics can be streamed to Amazon Data Firehose as the delivery service.
- CloudWatch Metric Streams also support third-party monitoring tools such as Datadog / Dynatrace / New Relic / Splunk / Sumo Logic
- You can filter metrics by namespaces to stream only a selected subset, reducing cost and noise.



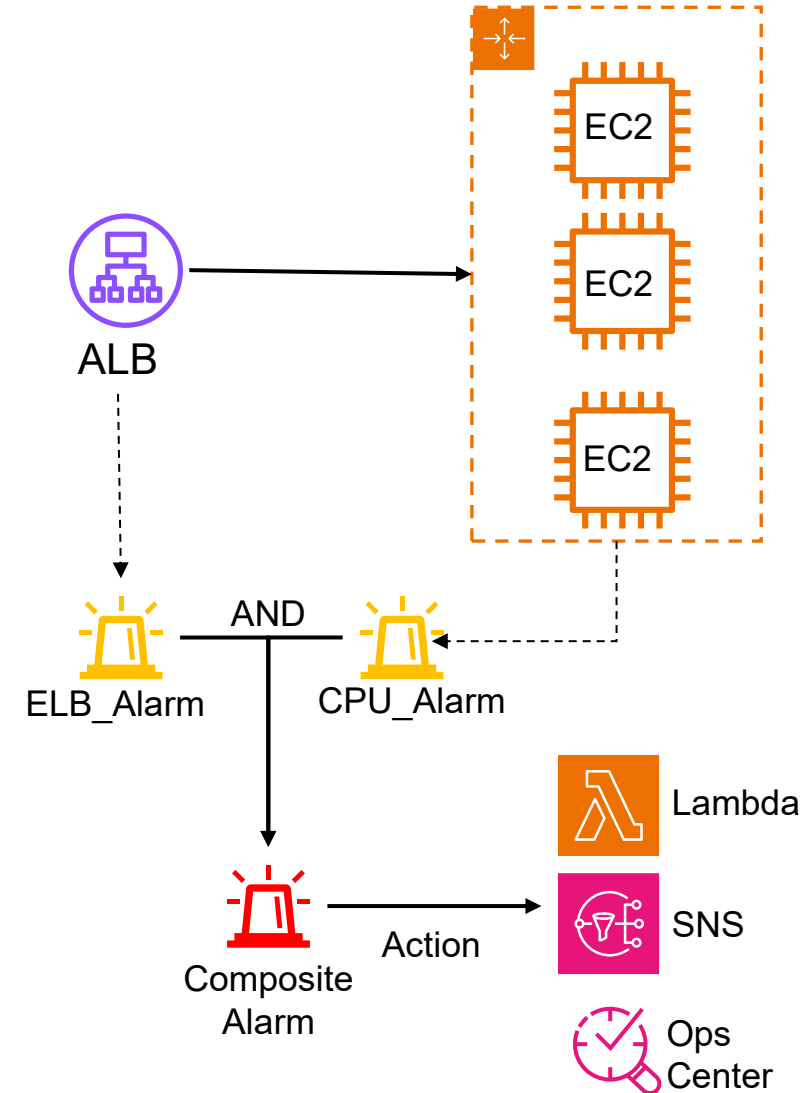
# Amazon CloudWatch Alarms

- CloudWatch Alarms allows you to take action when the metrics fall outside of the levels (high or low thresholds)
- A CloudWatch Alarm is always in one of three states: OK, ALARM, INSUFFICIENT\_DATA
- Alarms actions
  - **Auto Scaling:** Increase or decrease EC2 instances “desired” count.
  - **EC2 Actions:** Stop, terminate, reboot or recover an EC2 instance.
  - **SNS notifications:** Send a notification into an SNS topic.
  - More..
- Alarms can evaluate metrics using different statistics such as **Average, Sum, Minimum, Maximum, and Percentiles.**
- You can also combine multiple alarms to create a Composite Alarm

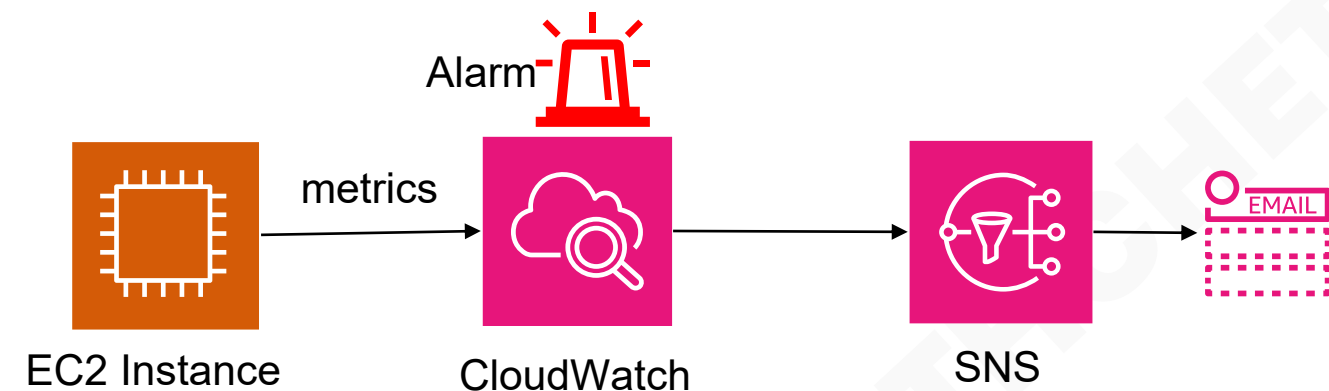


# Composite Alarm

- Composite alarms combine multiple CloudWatch alarms using logical rules (AND / OR / NOT).
- They don't monitor metrics directly; they evaluate the state of other alarms.
- Help reduce alert noise by triggering only when meaningful conditions occur together.
- Actions (SNS, Lambda, OpsCenter, etc.) are triggered only when the composite condition is met.
- Example:
  - CPU\_Alarm: Avg. EC2 CPU Utilization > 80%
  - ELB\_Alarm: Application Load Balancer 5XX errors > 50
  - Composite Alarm: ALARM(CPU\_Alarm) AND ALARM(ELB\_Alarm)



# Exercise - CloudWatch Metrics and Alarm



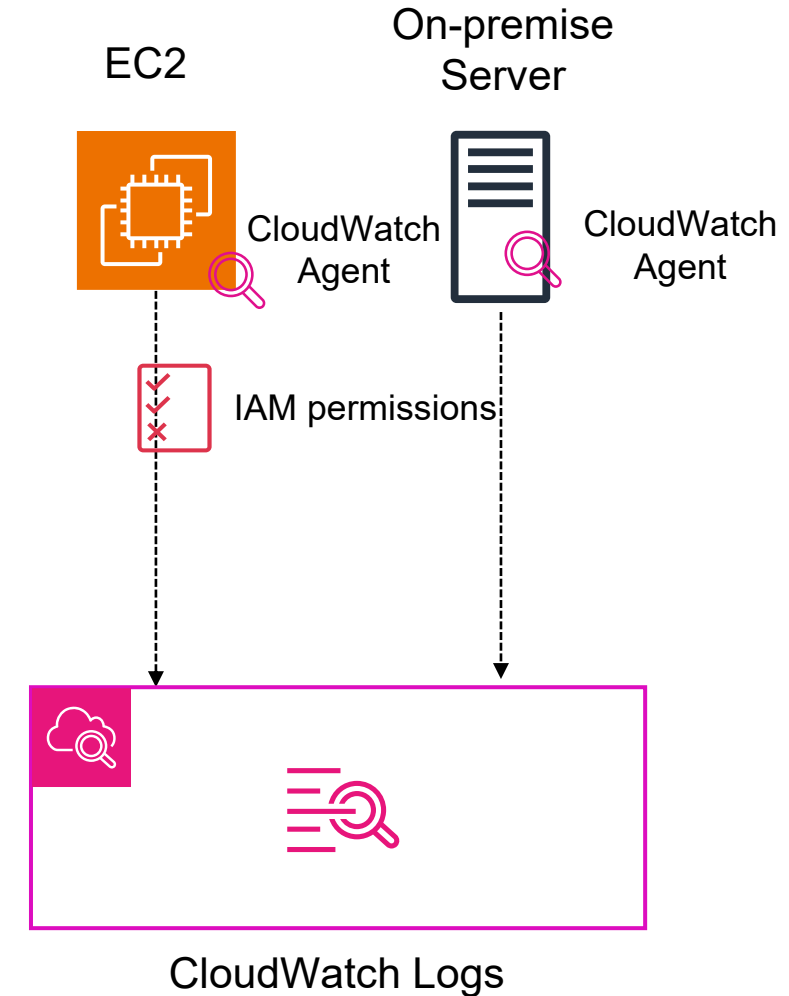
- 1 Create a SNS notification topic and create an email subscription. Confirm subscription by clicking the link received in the email.
- 2 Launch an EC2 instance
- 3 Go to AWS CloudWatch -> Alarms -> Create new alarms for your EC2 instance when CPUUtilization > 50 percent
- 4 Login into EC2 instance over SSH and load the CPU by using *stress* command
- 5 Wait up to 5 mins to see EC2 CPU utilization going high. This should trigger an Alarm.
- 6 Leave the EC2 instance running if you will be continuing with CloudWatch Logs lecture and corresponding exercise. Otherwise terminate the instance.

Commands to install and run stress to load the EC2 CPU:

```
sudo yum install -y stress
stress --cpu 1 --timeout 600
```

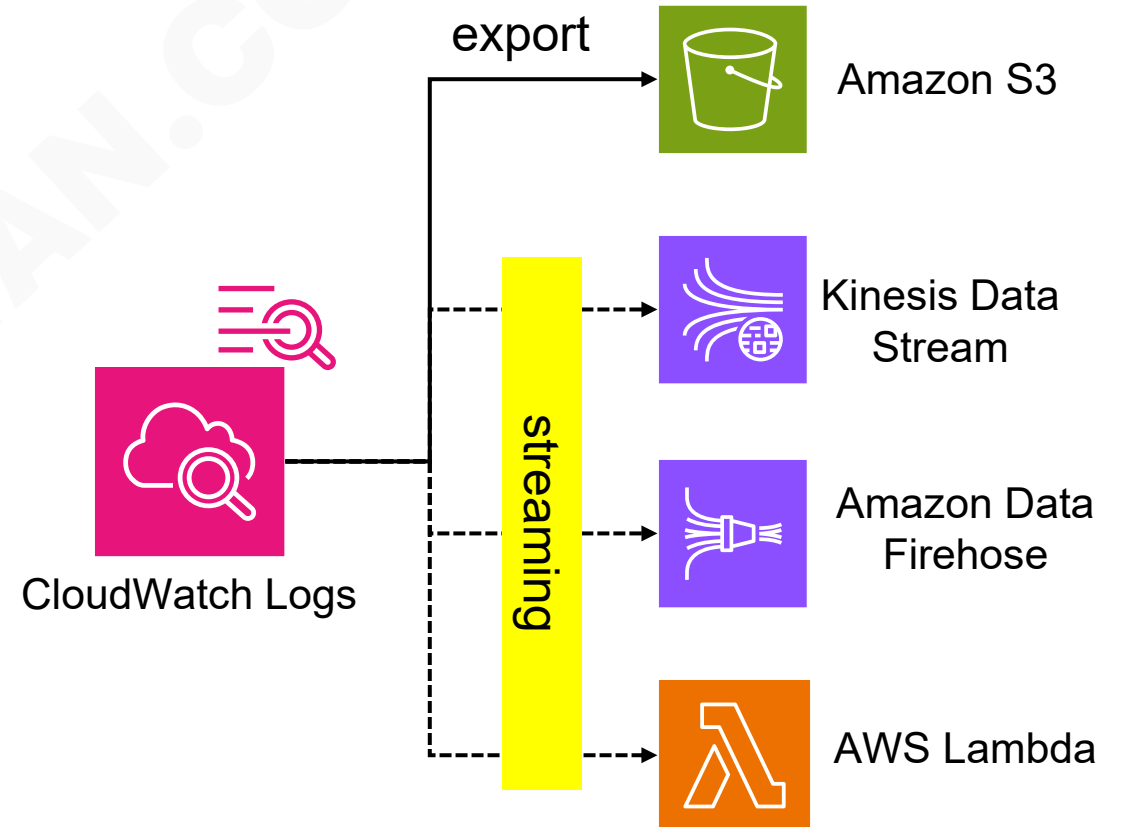
# Amazon CloudWatch Logs

- Collect, store, monitor, and analyze logs from AWS services and applications.
- Sources: EC2, ECS, Elastic Beanstalk, Lambda, API Gateway, CloudTrail, Route 53, VPC Flow Logs and more AWS services + any custom application
- For collecting logs from EC2 or on-premises server, install **CloudWatch Agent**.
- Log Structure
  - **Log Group**: A logical container for logs, usually represents one application or service. (e.g. /aws/ec2/application/)
  - **Log Stream**: A sequence of log events, typically represents an EC2 instance, container, Lambda execution, or log file. (e.g. instance\_id )
- Log Retention & Storage
  - By default, logs are retained forever. You can configure log retention policies (from 1 day up to 10 years).
  - Storage - Standard log storage, Infrequent Access (Logs-IA) for lower cost
- **Logs are encrypted by default.** Optionally use Customer managed KMS key for logs encryption (for compliance and audit purpose).



# Amazon CloudWatch Logs - Destination

- Logs can be further sent to different AWS services for processing, storage and analytics.
- CloudWatch has built-in integration with
  - Amazon S3 (log exports and long-term storage)
  - Kinesis Data Streams
  - Amazon Data Firehose
  - AWS Lambda

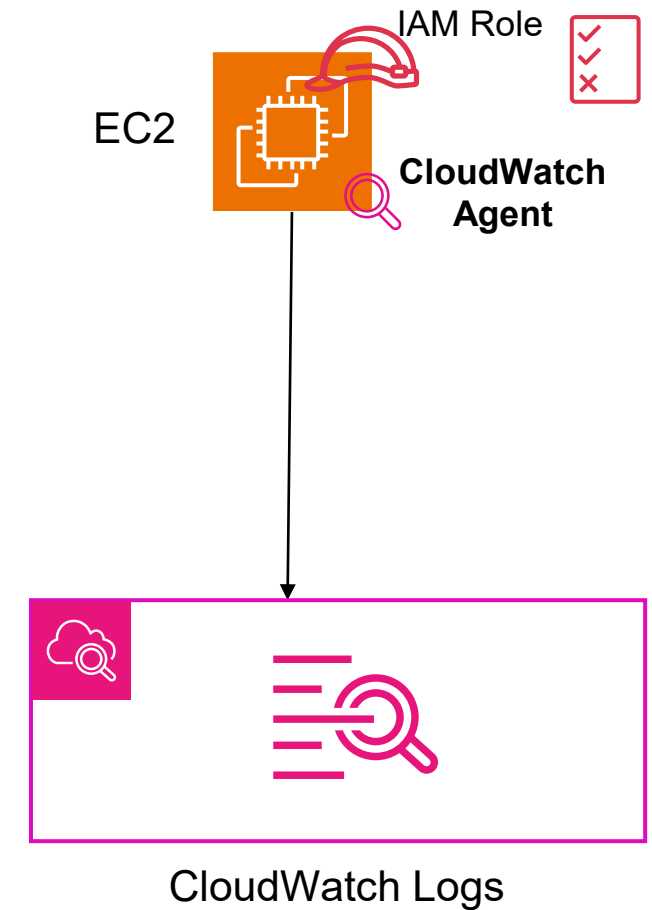


# EC2 Logs

- By default, EC2 logs are NOT sent to CloudWatch Logs.
- We must install the CloudWatch Agent on the EC2 instance or on-premise server.
- Additionally, attach an IAM role with permissions to push logs to CloudWatch.

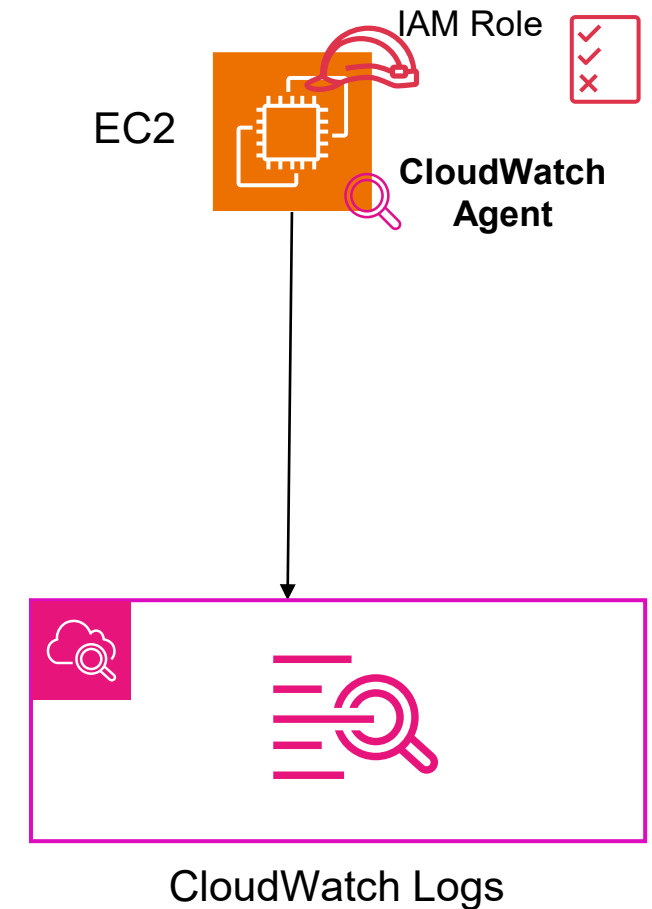
```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": [  
        "logs:CreateLogGroup",  
        "logs:CreateLogStream",  
        "logs:PutLogEvents",  
        "logs:DescribeLogStreams"  
      ],  
      "Resource": "*"   
    }  
  ]  
}
```

IAM Policy



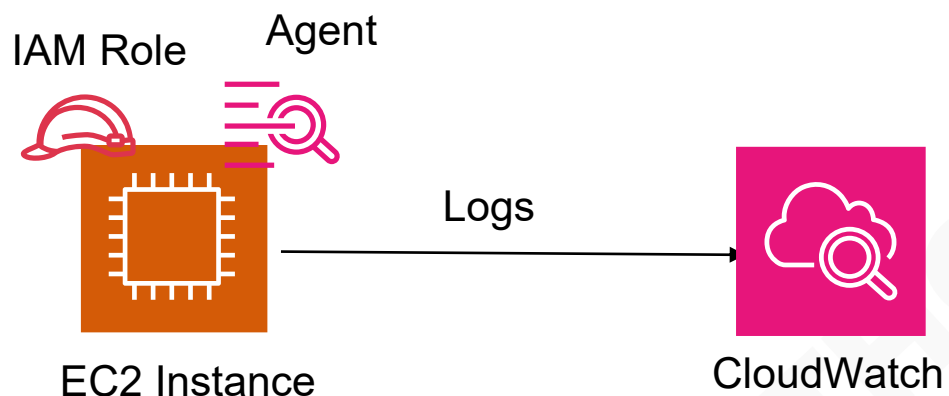
# EC2 Logs

- By default, EC2 logs are NOT sent to CloudWatch Logs.
- We must install the CloudWatch Agent on the EC2 instance or on-premise server.
- Additionally, attach an IAM role with permissions to push logs to CloudWatch.
- CloudWatch Agents allows collecting extra metrics beyond default EC2 monitoring
  - CPU (granular): active, idle, user, system, guest, steal
  - Disk: space (free/used/total), I/O (reads, writes, bytes, IOPS)
  - Memory (RAM ): free, used, total, cached, inactive (EC2 default has NO RAM)
  - Network: TCP/UDP connections, packets, bytes
  - Processes: total, running, sleeping, blocked, dead, idle
  - Swap: free, used, used %





# Exercise - CloudWatch Logs



- 1 Create an IAM role for EC2 with IAM policy CloudWatchAgentServerPolicy
- 2 Attach this role to an EC2 instance
- 3 Login to EC2 over SSH and install AWS CloudWatch agent
- 4 Create CloudWatch agent configuration file for setting up logs that you want to send to CloudWatch service
- 5 Start CloudWatch agent service
- 6 Create a dummy log file in the directory as configured in the agent config file
- 7 Go to CloudWatch Logs. You should see a new CloudWatch Logs group and Log stream. Check if you see the log entries.
- 8 (Optional) Go to CloudWatch Logs insights and run few queries to filter the logs by some keywords e.g. ERROR
- 9 After the exercise, terminate EC2 instance and delete CloudWatch Logs group

# Installing CloudWatch agent



[Get started](#)
[Service guides](#)
[Developer tools](#)
[AI resources](#)

▼ Installing the CloudWatch agent

[Install and Configure Amazon CloudWatch Agent with Workload Detection in the CloudWatch console](#)
[Manual installation on Amazon EC2](#)
[Install the CloudWatch agent using AWS Systems Manager](#)
[Install the CloudWatch agent on on-premises servers](#)
[Install the CloudWatch agent on new instances using CloudFormation](#)
[Install the CloudWatch agent with the Amazon CloudWatch Observability EKS add-on or the Helm chart](#)

► Create the CloudWatch agent configuration file

[Documentation](#) > [Amazon CloudWatch](#) > [User Guide](#)

## Installing the CloudWatch agent

[↓ PDF](#)
[↓ RSS](#)

☐ Focus mode

You can install the CloudWatch agent on your Amazon EC2 instances, on-premises servers, and in containerized environments to collect metrics, logs, and traces. The agent supports Linux, Windows, and macOS operating systems. There are several ways to install the agent, including using Systems Manager, from the CloudWatch console, installing from the command line, or installing using a configuration file. Before installing, make sure you have the required IAM permissions and network access configured.

### Topics

- [Install and Configure Amazon CloudWatch Agent with Workload Detection in the CloudWatch console](#)
- [Manual installation on Amazon EC2](#)
- [Install the CloudWatch agent using AWS Systems Manager](#)
- [Install the CloudWatch agent on on-premises servers](#)
- [Install the CloudWatch agent on new instances using CloudFormation](#)
- [Install the CloudWatch agent with the Amazon CloudWatch Observability EKS add-on or the Helm chart](#)

# Exercise - Useful commands

1. Perform following action using root user. Run this command to change user to root: `sudo su`
2. Install AWS CloudWatch agent: `yum install amazon-cloudwatch-agent`
3. Create CloudWatch agent configuration file using vi or vim editor. Sample file config.json provided for download.
4. Start CloudWatch agent: `/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -c file:<path to the /config file that you created> -s`
5. Check the status of CloudWatch agent: `/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -m ec2 -a status`
6. Now create dummy log file in the `/var/log/` directory. Sample application.log provided for download.

# Exercise - Sample files

## 1. config.json

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/application.log",
            "log_group_name": "/myapplication",
            "log_stream_name": "{instance_id}/application_logs",
            "timestamp_format": "%b %d %H:%M:%S"
          }
        ]
      }
    }
  }
}
```

## 2. application.log

```
INFO MainApp - Application started successfully.
DEBUG ServiceA - Initializing Service A with config: {timeout: 5000ms, retries: 3}
INFO ServiceB - Fetching data from external API...
ERROR ServiceB - Failed to fetch data from API: TimeoutException
DEBUG DatabaseConnection - Attempting to connect to database at 10.0.0.1:5432
INFO DatabaseConnection - Connection to database established successfully.
ERROR UserService - User not found for ID: 12345
INFO Authentication - User login successful for username: johndoe
DEBUG ServiceA - Processing user input: {userId: 12345, action: 'login'}
INFO ServiceA - User session initialized for userID: 12345
ERROR PaymentService - Payment failed: Insufficient funds for userID: 12345
DEBUG PaymentService - Payment request payload: {amount: 100.00, currency: 'USD'}
INFO NotificationService - Sending email notification to userID: 12345
ERROR NotificationService - Failed to send email to userID: 12345: SMTPServerException
DEBUG CacheManager - Cache miss for key: userDetails_12345
INFO ServiceC - Scheduled job 'DataSync' completed successfully.
DEBUG ServiceC - DataSync job processed 150 records.
ERROR ServiceC - DataSync job failed: NullPointerException encountered in processRecords()
INFO MainApp - Shutting down application...
DEBUG MainApp - Application shutdown completed. Cleanup performed.
```

# CloudWatch additional Features – good to know

- S3 Export
- CloudWatch Logs Subscription
- CloudWatch Logs Aggregation
- CloudWatch Logs Insights
- Additional Logs Insights:
  - Container Insights
  - Lambda Insights
  - Database Insights
  - Contributor Insights
  - Application Insights
- CloudWatch Dashboards and sharing



# CloudWatch Logs – S3 Export

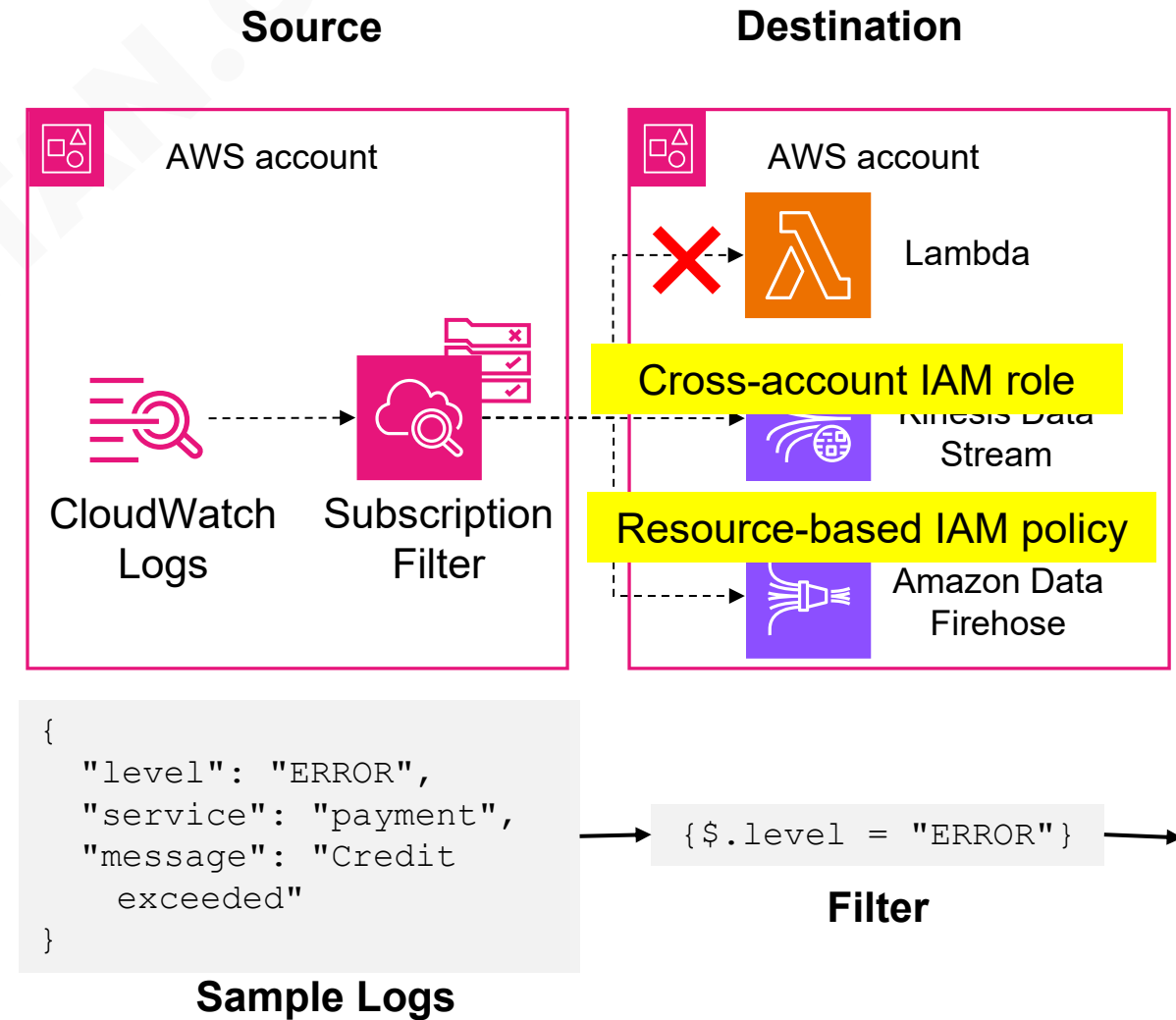
- Export CloudWatch Logs to Amazon S3 buckets
- Logs encryption using SSE-KMS and SSE-CMK
- Log can take up to 12 hours to be exported into S3 bucket (not real-time)
- Can use with S3 features like S3 Object Lock and configure retention period to keep the logs for long duration and to meet regulatory or compliance requirements.
- Use cases:
  - Long term archival (cost benefits)
  - Logs query and analysis using AWS services like Athena



Tip: It's cheaper to keep logs in S3 than CloudWatch

# CloudWatch Logs Subscription

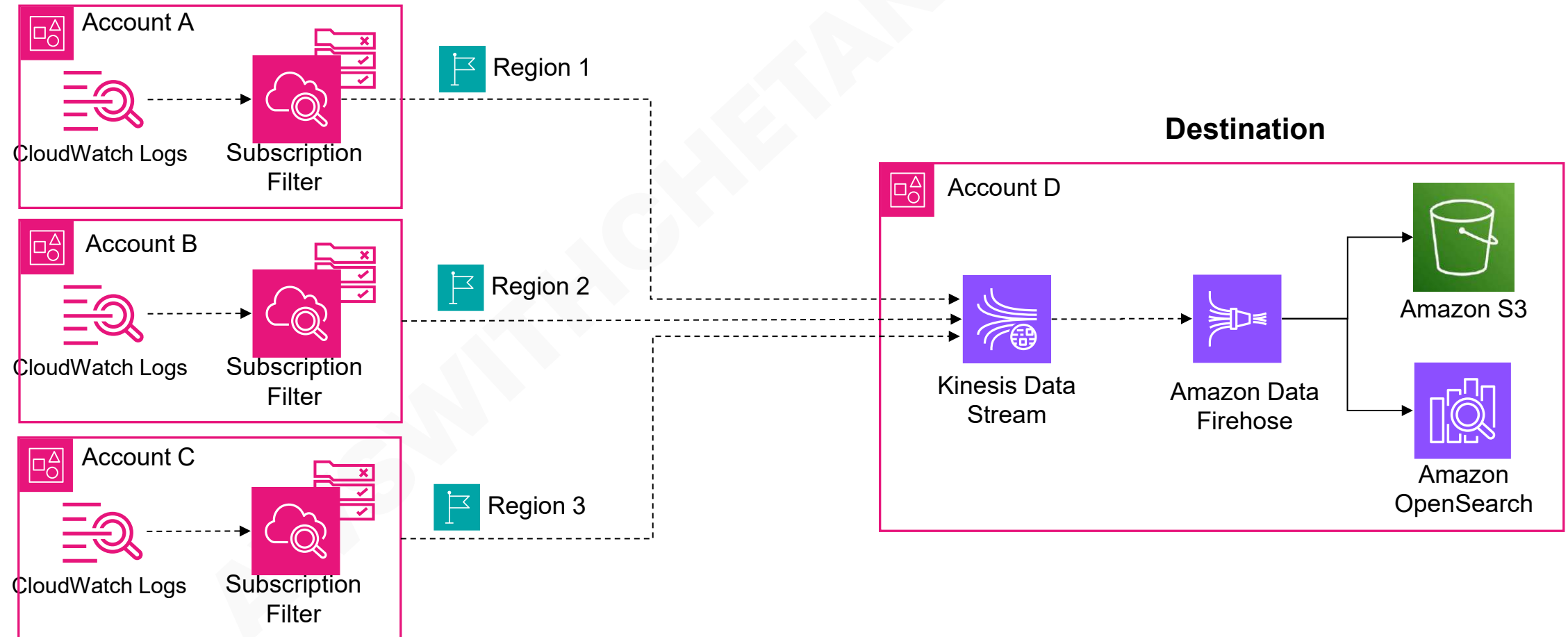
- Streams log events in real-time from CloudWatch Logs to other AWS services for processing and analysis.
- Uses Subscription Filter to configure:
  - Which log events to capture (using filter patterns)
  - Where to send them (destination service)
- Filters are applied at Log group level e.g. /aws/ecs/webapp/ (Max 2 subscriptions per group)
- Supports cross-account subscription and delivery
  - For Amazon Data Firehose target, configure IAM resource policy for Data Firehose to allow access to Account A CloudWatch service (logs.amazonaws.com)
  - For Kinesis Data Stream target, create Cross-account IAM role in Account B to allow access to Account A CloudWatch service (logs.amazonaws.com)
  - Lambda target is not supported for cross-account





# CloudWatch Logs Aggregation

- Supports aggregating logs across Multiple AWS account and multiple AWS regions

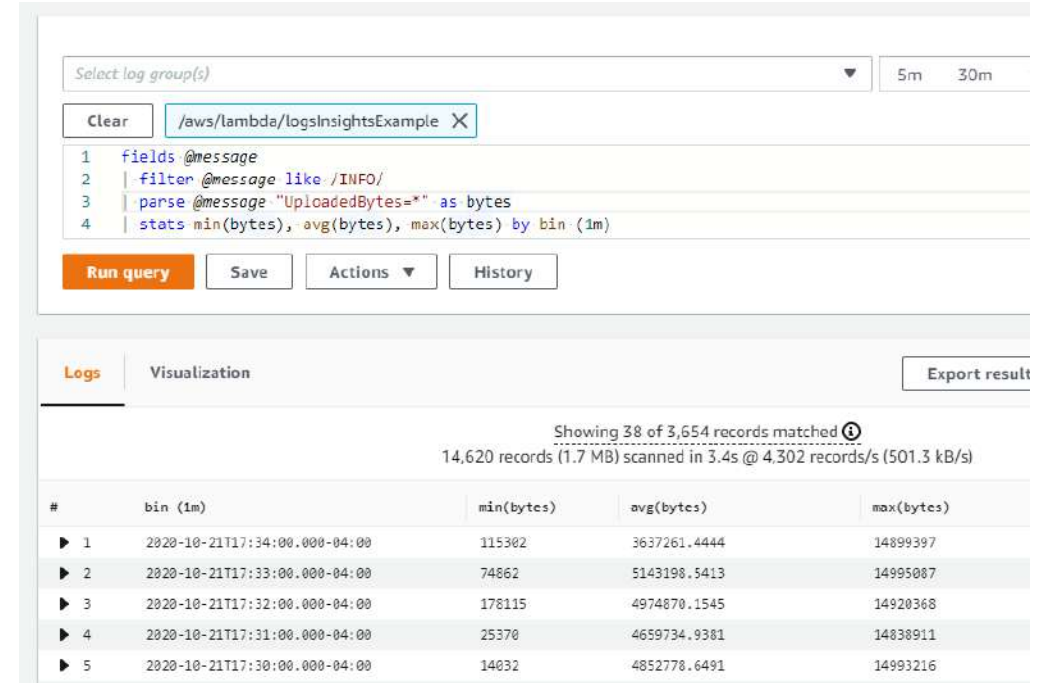




# CloudWatch Logs Insights

- Allows users to run SQL-like queries on CloudWatch Logs.
- Logs are analyzed in place without exporting or moving data.
- Queries can be executed across multiple log groups simultaneously.
- The service supports filtering, parsing, sorting, and aggregation operations.
- Results can be visualized using charts or tables.
- Logs Insights is read-only and does not alter log data
- **Use case:** For ad-hoc investigation and debugging

```
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 20
```



The screenshot shows the AWS CloudWatch Logs Insights interface. At the top, there's a dropdown to 'Select log group(s)' and a time range selector set to '5m'. Below this is a 'Clear' button and a text input containing '/aws/lambda/logsInsightsExample'. The main area contains a SQL-like query:

```
1 fields @message
2 | filter @message like /INFO/
3 | parse @message "UploadedBytes=" as bytes
4 | stats min(bytes), avg(bytes), max(bytes) by bin (1m)
```

Below the query are buttons for 'Run query', 'Save', 'Actions', and 'History'. The results are displayed in a table with tabs for 'Logs' and 'Visualization'. The 'Logs' tab is active, showing a table with 5 rows of data. Above the table, it says 'Showing 38 of 3,654 records matched' and '14,620 records (1.7 MB) scanned in 3.4s @ 4,302 records/s (501.3 kB/s)'. The table has columns for '#', 'bin (1m)', 'min(bytes)', 'avg(bytes)', and 'max(bytes)'.

| # | bin (1m)                      | min(bytes) | avg(bytes)   | max(bytes) |
|---|-------------------------------|------------|--------------|------------|
| 1 | 2020-10-21T17:34:00.000-04:00 | 115302     | 3637261.4444 | 14899397   |
| 2 | 2020-10-21T17:33:00.000-04:00 | 74862      | 5143198.5413 | 14995087   |
| 3 | 2020-10-21T17:32:00.000-04:00 | 178115     | 4974870.1545 | 14920368   |
| 4 | 2020-10-21T17:31:00.000-04:00 | 25370      | 4659734.9381 | 14838911   |
| 5 | 2020-10-21T17:30:00.000-04:00 | 14032      | 4852778.6491 | 14993216   |

# CloudWatch Insights – good to know

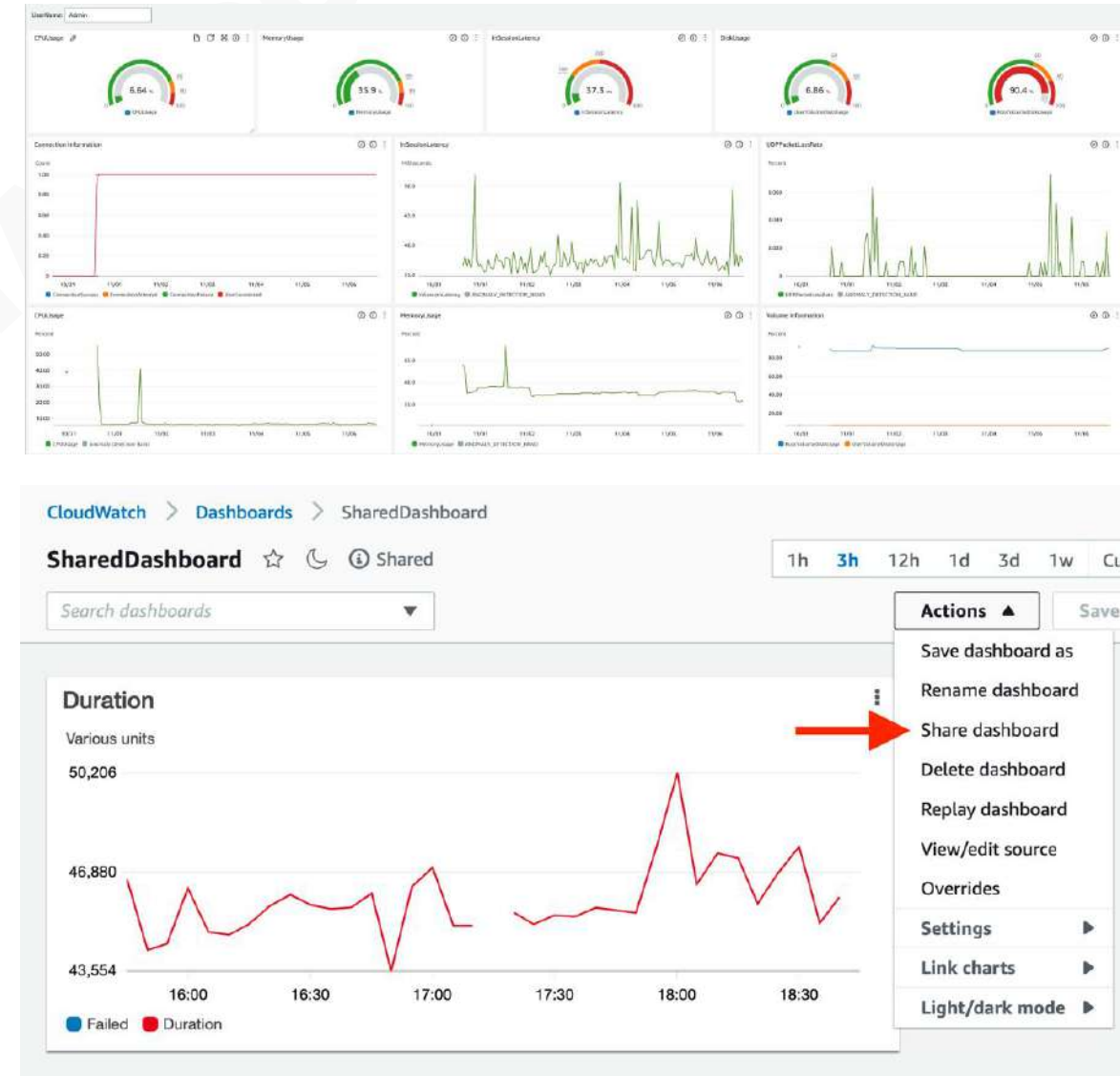
CloudWatch Insights use logs and metrics together to provide service-specific visibility into application, compute, container, and database workloads.

- **Container Insights:** Monitors ECS, EKS, Kubernetes for container CPU, memory, network, pod & node metrics.
- **Lambda Insights:** Deep visibility into Lambda performance like memory usage, cold starts, duration, concurrency.
- **Application Insights:** Automatically detects application issues (latency, errors) using logs & metrics. Best for EC2 + EBS + ELB based apps.
- **Database Insights:** Performance monitoring for RDS & Aurora like load, waits, SQL performance.
- **Contributor Insights:** Identifies top contributors in logs/metrics. Example: Which IP/user/API causes most errors?



# CloudWatch Dashboards and sharing

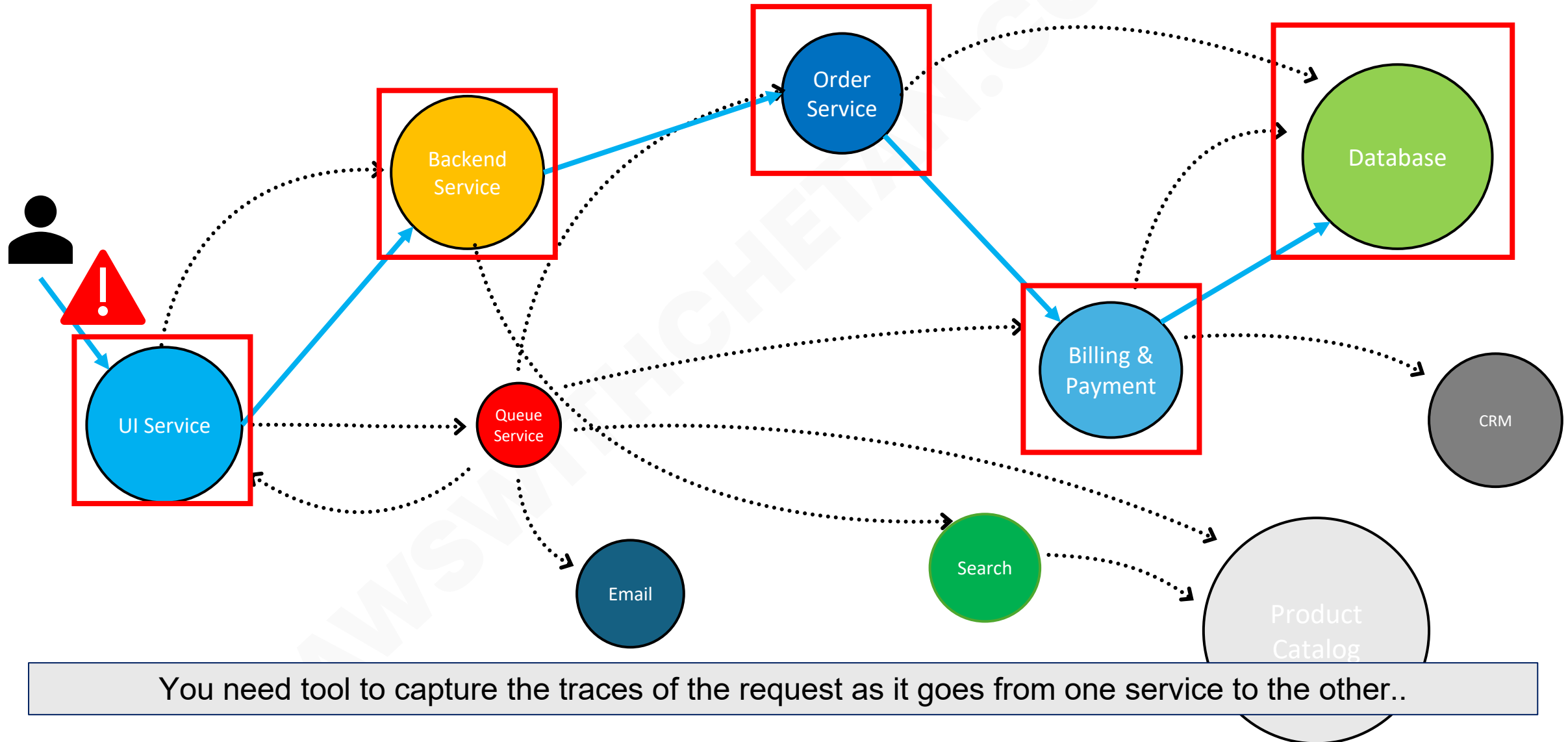
- Amazon CloudWatch offers automatic pre-built dashboards
- You can create your own custom dashboards to monitor your resources in a single view.
- Resources can be across different Regions and accounts.
- CloudWatch Dashboards can be shared with specific users via email, where users log in using a generated username and password.
- Dashboards can also be shared **publicly using a URL**, allowing anyone with the link to view them without AWS access.
- All shared dashboards are read-only and sharing can be revoked at any time.





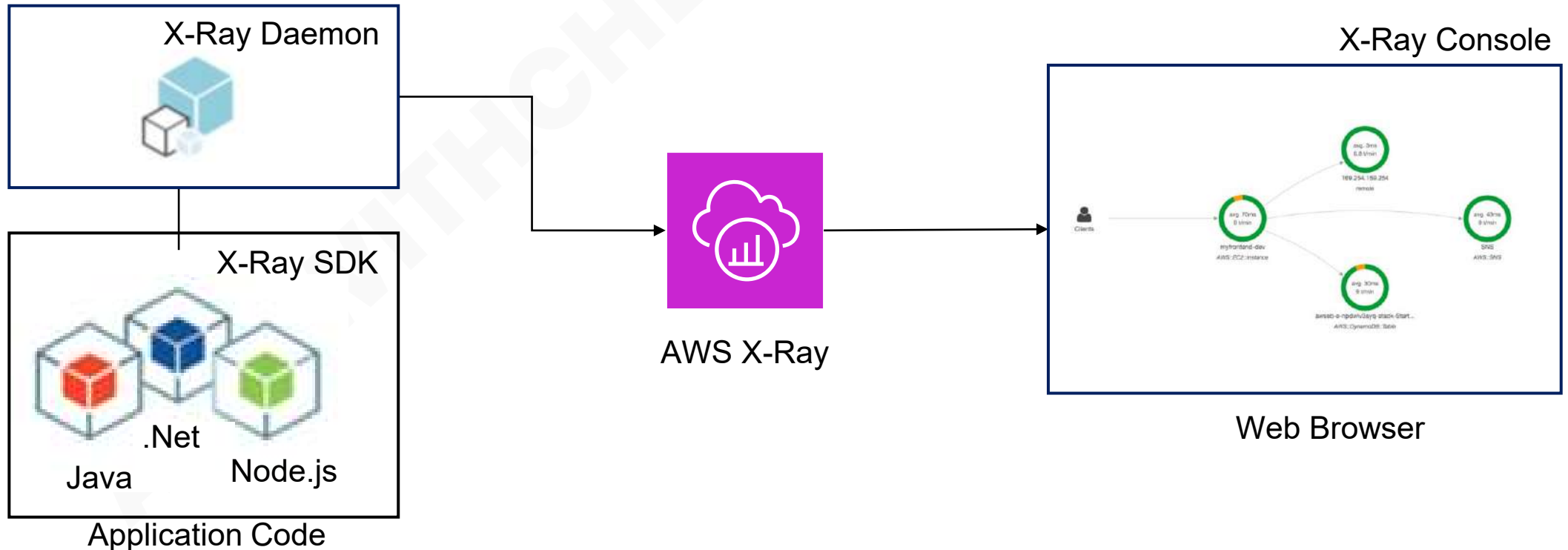
# AWS X-Ray

# Microservices application in real world

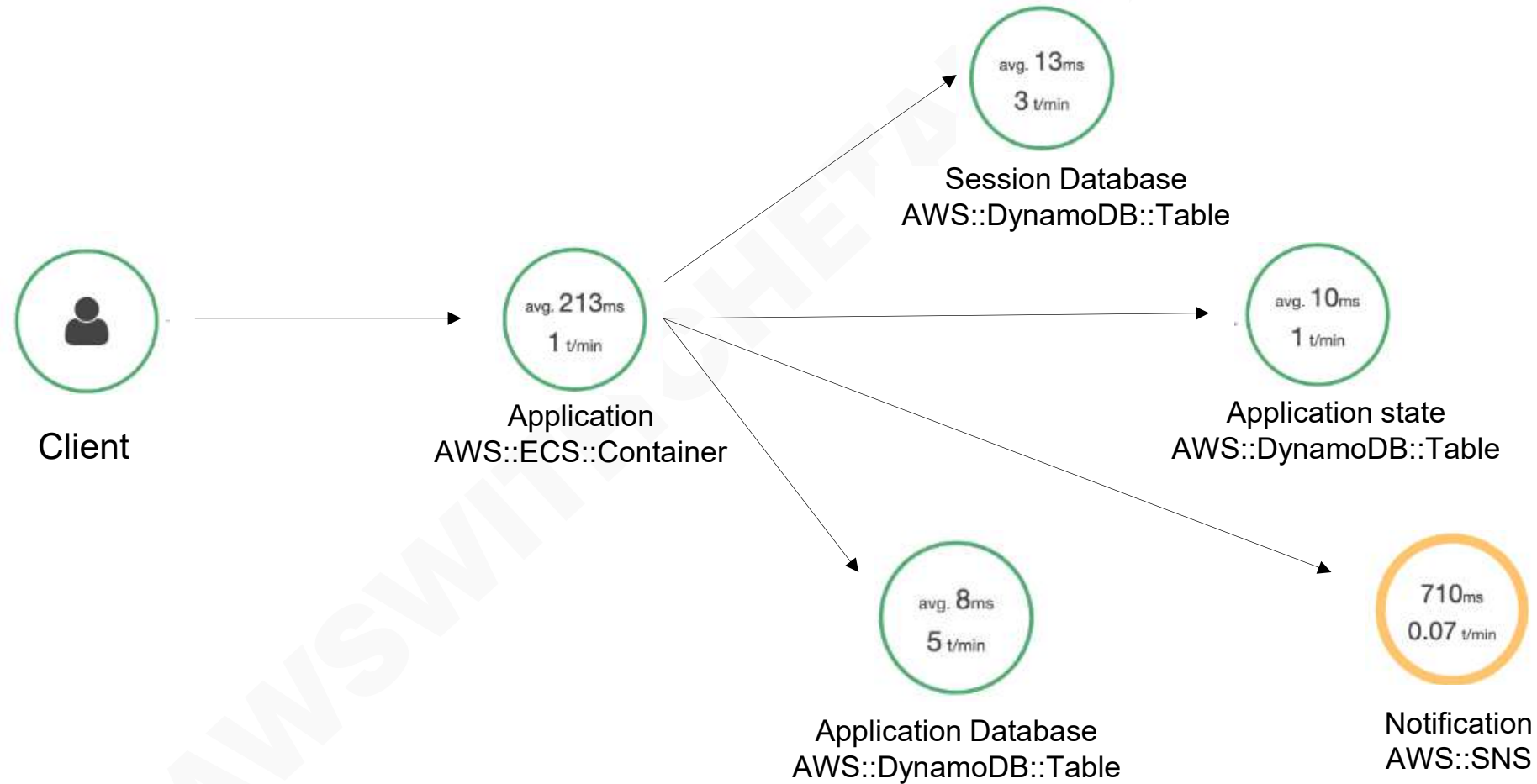


# AWS X-Ray

- Service that collects data for requests that your application serves.
- Provides tools that you can use to view, filter, and gain insights into that data to identify issues and opportunities for optimization.



# AWS X-Ray : Visual Analysis of Applications



# Use AWS X-Ray for..

- Debugging slow responses to the user requests
- Troubleshooting increased error rates
- Tracing down the user request path -> Traces
- Are requests getting completed in SLA time?
- Which service is the bottleneck in the application?
- Which users are impacted by the specific service disruption?
- More..



AWS X-Ray

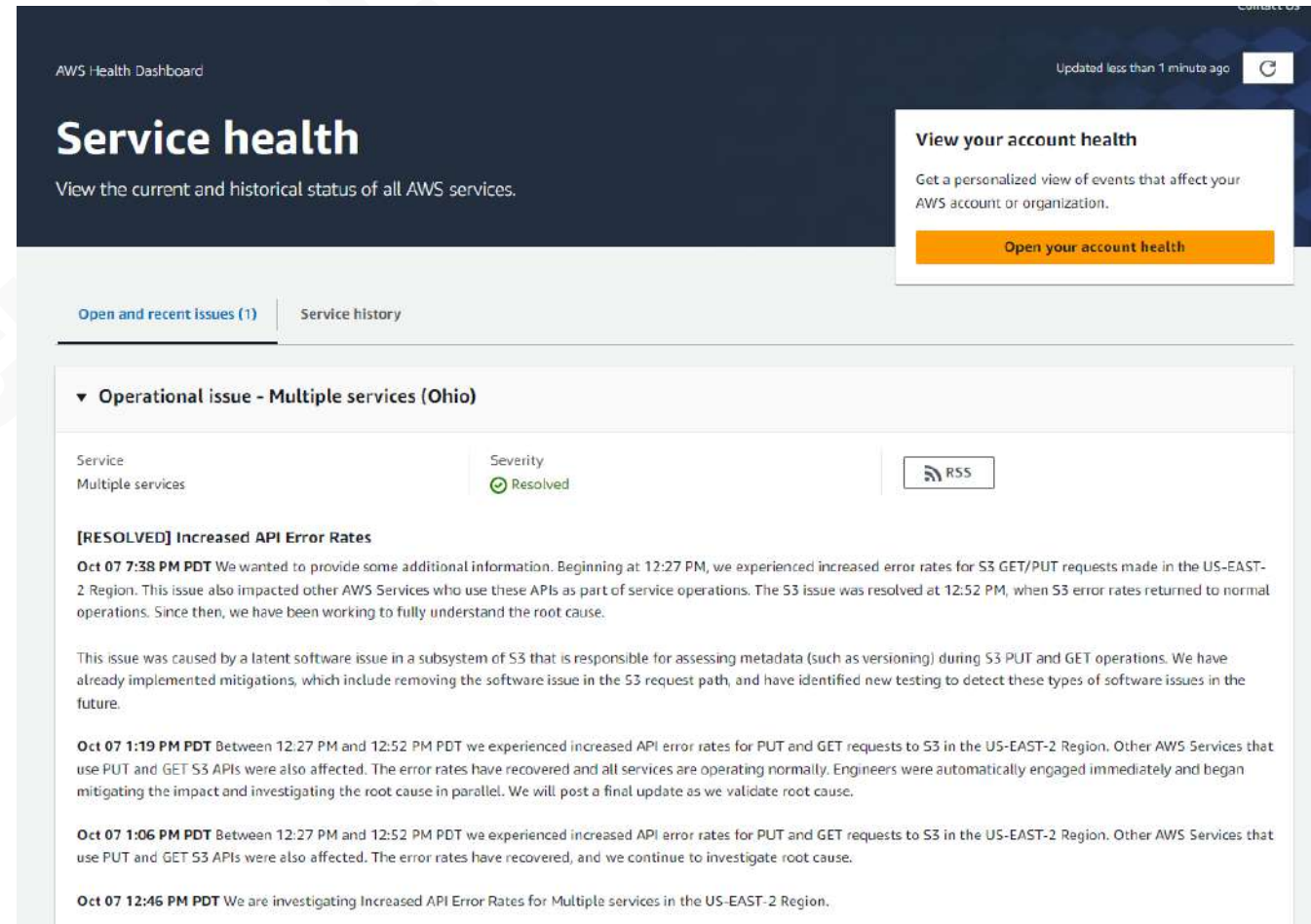




# AWS Health

# AWS Health Dashboard - Service health

- Shows AWS services disruption events



The screenshot displays the AWS Health Dashboard interface. At the top, it says "AWS Health Dashboard" and "Updated less than 1 minute ago". The main heading is "Service health" with a subtext "View the current and historical status of all AWS services." On the right, there's a box titled "View your account health" with a button "Open your account health". Below this, there are tabs for "Open and recent issues (1)" and "Service history". The main content area shows an "Operational issue - Multiple services (Ohio)" with a severity of "Resolved". The issue title is "[RESOLVED] Increased API Error Rates". The description states: "Oct 07 7:38 PM PDT We wanted to provide some additional information. Beginning at 12:27 PM, we experienced increased error rates for S3 GET/PUT requests made in the US-EAST-2 Region. This issue also impacted other AWS Services who use these APIs as part of service operations. The S3 issue was resolved at 12:52 PM, when S3 error rates returned to normal operations. Since then, we have been working to fully understand the root cause. This issue was caused by a latent software issue in a subsystem of S3 that is responsible for assessing metadata (such as versioning) during S3 PUT and GET operations. We have already implemented mitigations, which include removing the software issue in the S3 request path, and have identified new testing to detect these types of software issues in the future." Below this, there are two more entries: "Oct 07 1:19 PM PDT Between 12:27 PM and 12:52 PM PDT we experienced increased API error rates for PUT and GET requests to S3 in the US-EAST-2 Region. Other AWS Services that use PUT and GET S3 APIs were also affected. The error rates have recovered and all services are operating normally. Engineers were automatically engaged immediately and began mitigating the impact and investigating the root cause in parallel. We will post a final update as we validate root cause." and "Oct 07 1:06 PM PDT Between 12:27 PM and 12:52 PM PDT we experienced increased API error rates for PUT and GET requests to S3 in the US-EAST-2 Region. Other AWS Services that use PUT and GET S3 APIs were also affected. The error rates have recovered, and we continue to investigate root cause." The final entry is "Oct 07 12:46 PM PDT We are investigating Increased API Error Rates for Multiple services in the US-EAST-2 Region."

# AWS Health Dashboard - Service history

- Shows AWS services disruption events
- Shows current and historical status of the AWS services health

Service history

List of services

The following table is a running log of AWS service interruptions for the past 12 months. Choose a status icon to see status updates for that service. All dates and times are reported (PDT). To update your time zone, see [Time zone settings](#).

North America

South America

Europe

Africa

Asia Pacific

Middle East

All locales

<

1

2

3

4

5

| Service                             | RSS | ⏮ | Today | 26 Aug | 25 Aug | 24 Aug | 23 Aug | 22 Aug |
|-------------------------------------|-----|---|-------|--------|--------|--------|--------|--------|
| Amazon API Gateway (Calgary)        | 📡   |   | ✅     | ✅      | ✅      | ✅      | ✅      | ✅      |
| Amazon API Gateway (Canada-Central) | 📡   |   | ✅     | ✅      | ✅      | ✅      | ✅      | ✅      |
| Amazon API Gateway (N. California)  | 📡   |   | ✅     | ✅      | ✅      | ✅      | ✅      | ✅      |
| Amazon API Gateway (N. Virginia)    | 📡   |   | ✅     | ✅      | ✅      | ✅      | ✅      | ✅      |
| Amazon API Gateway (Ohio)           | 📡   |   | ✅     | ✅      | ✅      | ✅      | ✅      | ✅      |
| Amazon API Gateway (Oregon)         | 📡   |   | ✅     | ✅      | ✅      | ✅      | ✅      | ✅      |

# AWS Health Dashboard – Your account

- Provides alerts and remediation guidance when AWS is experiencing events that may impact you.
- Proactive notification to help you plan for scheduled activities.

## Your account health

Stay informed of important events affecting your AWS resources.

[Configure EventBridge](#)

Get notifications for events that might affect your services and resources.

[Go to EventBridge](#)

[Open and recent issues \(16\)](#)
[Scheduled changes \(0\)](#)
[Notifications \(3\)](#)
[Event log](#)

### Open and recent issues (16)

View events that might affect your AWS infrastructure. 35 issues were resolved in the past 24 hours.

Service: Elastic Compute Cloud
Clear filter

Event summary

**Operational issue - EC2 (Ohio)**

Last update: February 20, 2022 at 11:16:34 PM UTC-8  
us-east-2

**Operational issue - EC2 (Ohio)**

Last update: February 17, 2022 at 11:56:09 PM UTC-8  
us-east-2

**Operational issue - EC2 (N. Virginia)**

Last update: February 16, 2022 at 1:36:29 AM UTC-8  
us-east-1

### Operational issue - EC2 (Ohio)

[Back to list view](#)

[Details](#)
[Affected resources](#)

#### Event data

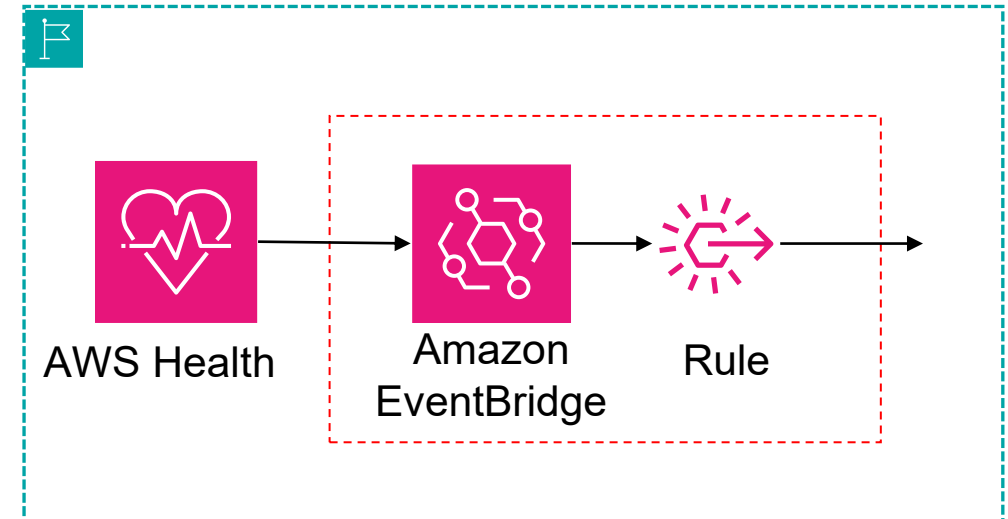
|                            |           |                    |                                        |
|----------------------------|-----------|--------------------|----------------------------------------|
| Service                    | EC2       | Start time         | February 20, 2022 at 11:16:24 PM UTC-8 |
| Status                     | Open      | End time           | -                                      |
| Region / Availability Zone | us-east-1 | Category           | Issue                                  |
| Account specific           | No        | Affected resources | 1                                      |

#### Description

[04:35 AM PST] We are investigating increased EC2 launch failures and networking connectivity issues for some instances in a single Availability Zone (USE1-AZ4) in the US-EAST-1 Region. Other Availability Zones within the US-EAST-1 Region are not affected by this issue.

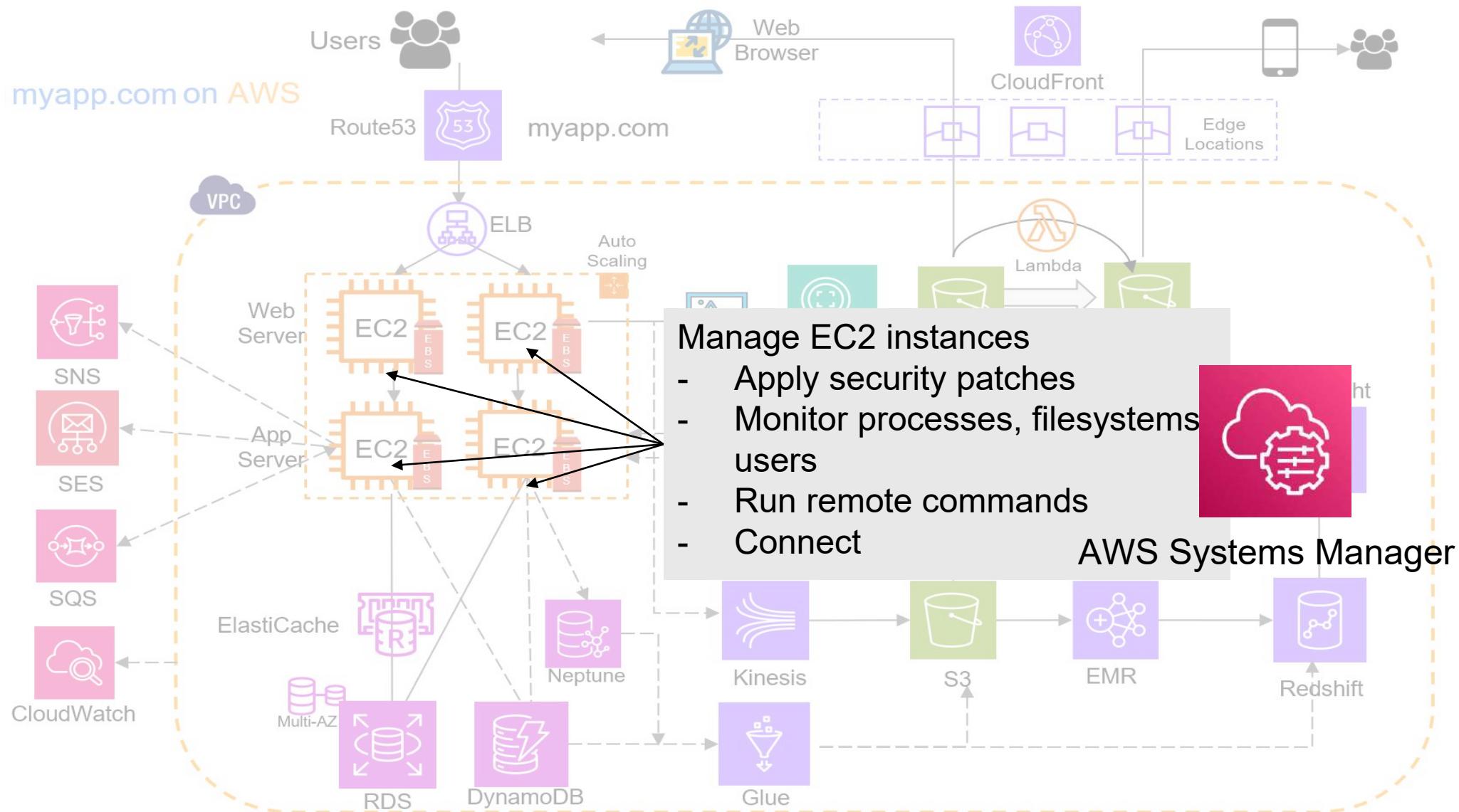
# AWS Health integration with EventBridge

- AWS Health automatically publishes service health events to Amazon EventBridge.
- EventBridge rules can filter events based on service, region, or event type.
- Matching events can trigger automated actions such as:
  - Send SNS notifications
  - Trigger Lambda functions
  - Run SSM Automation runbooks
  - Send message to SQS queue, Kinesis data stream
  - Or create tickets in 3<sup>rd</sup> party systems like JIRA, ServiceNow
- This enables proactive response to AWS outages or maintenance affecting your account.



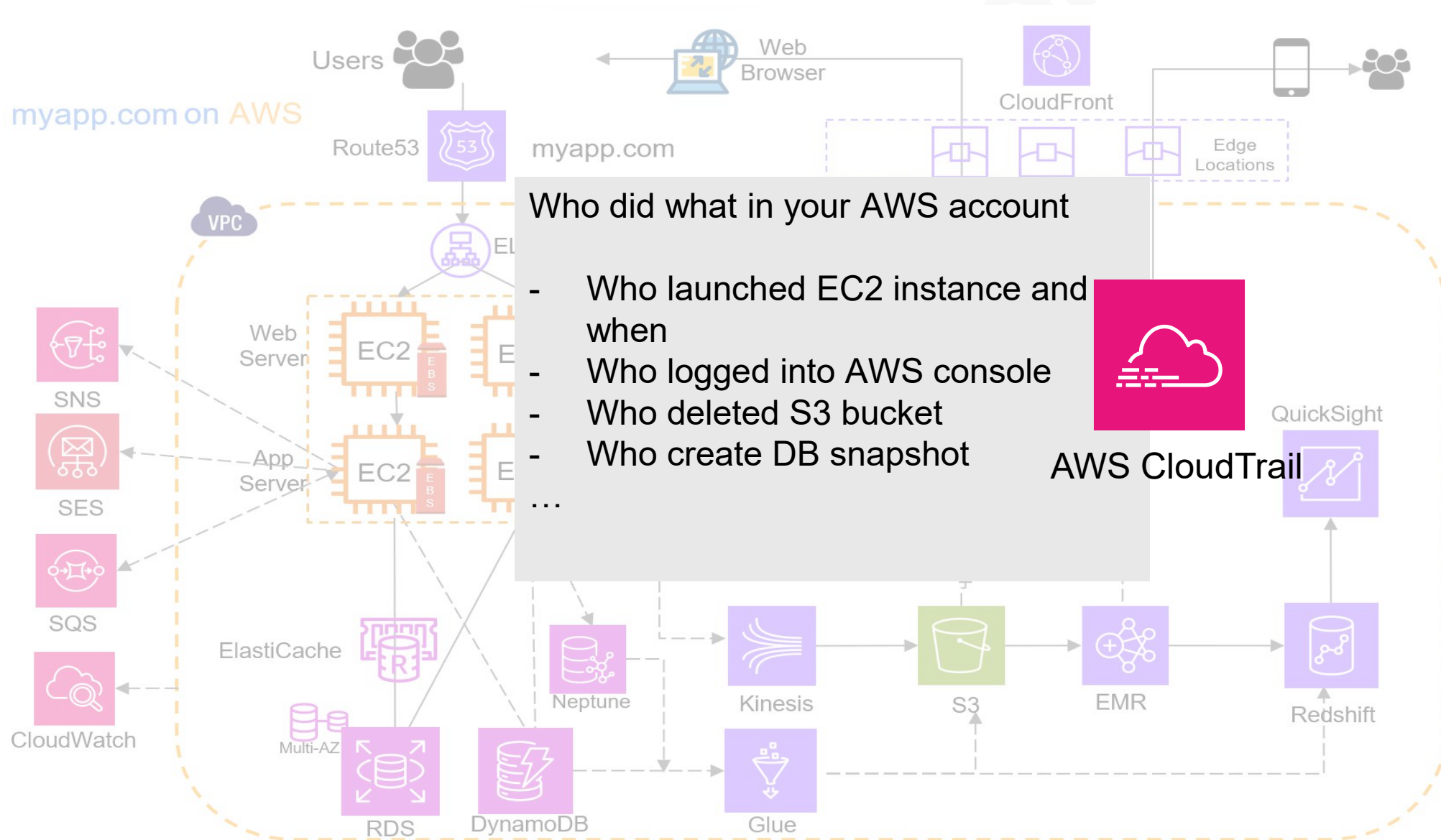
AWS Systems Manager (SSM)  
AWS CloudTrail  
AWS Config

# Managing AWS resources



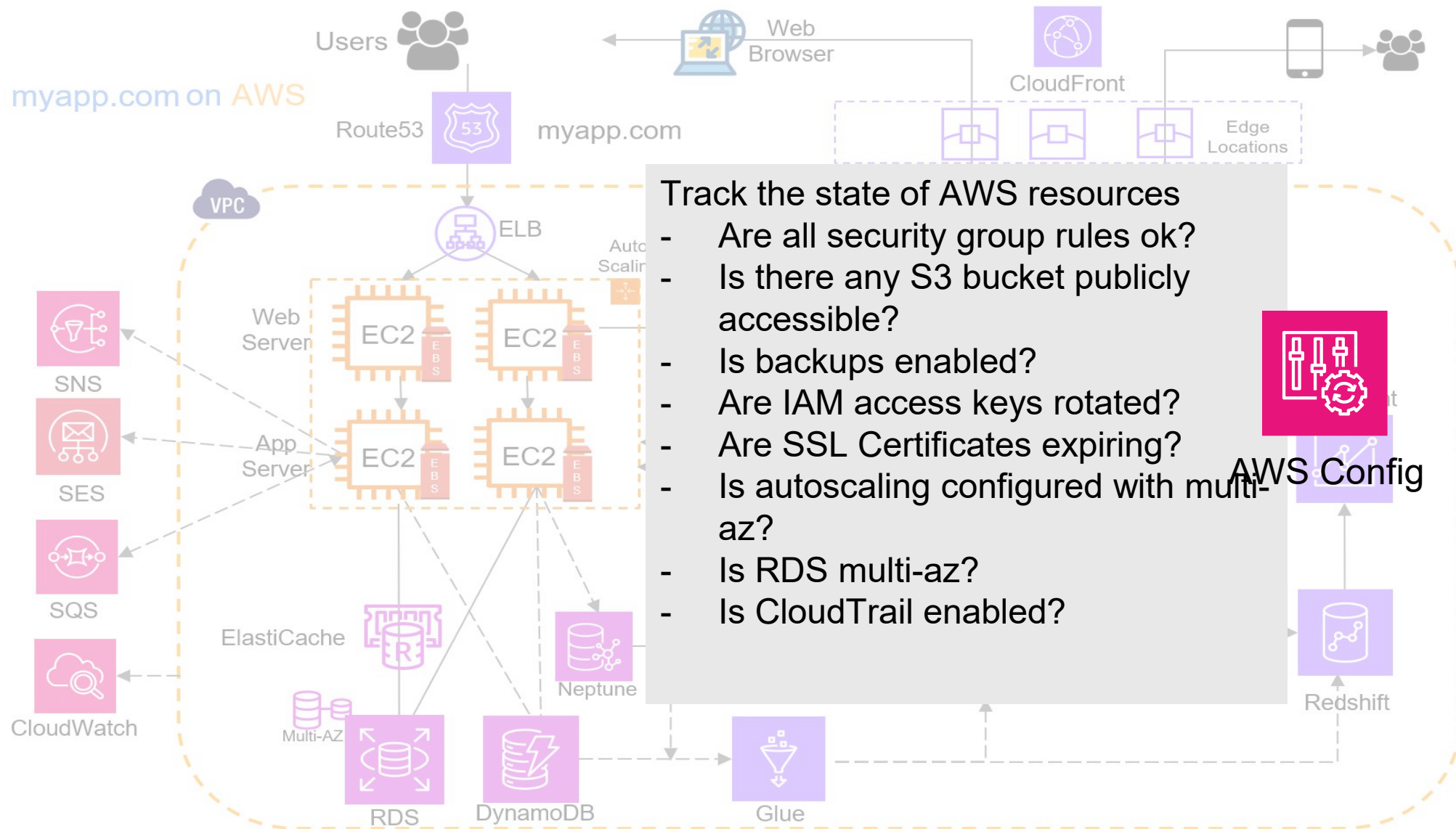


# AWS Account Activity - Tracking





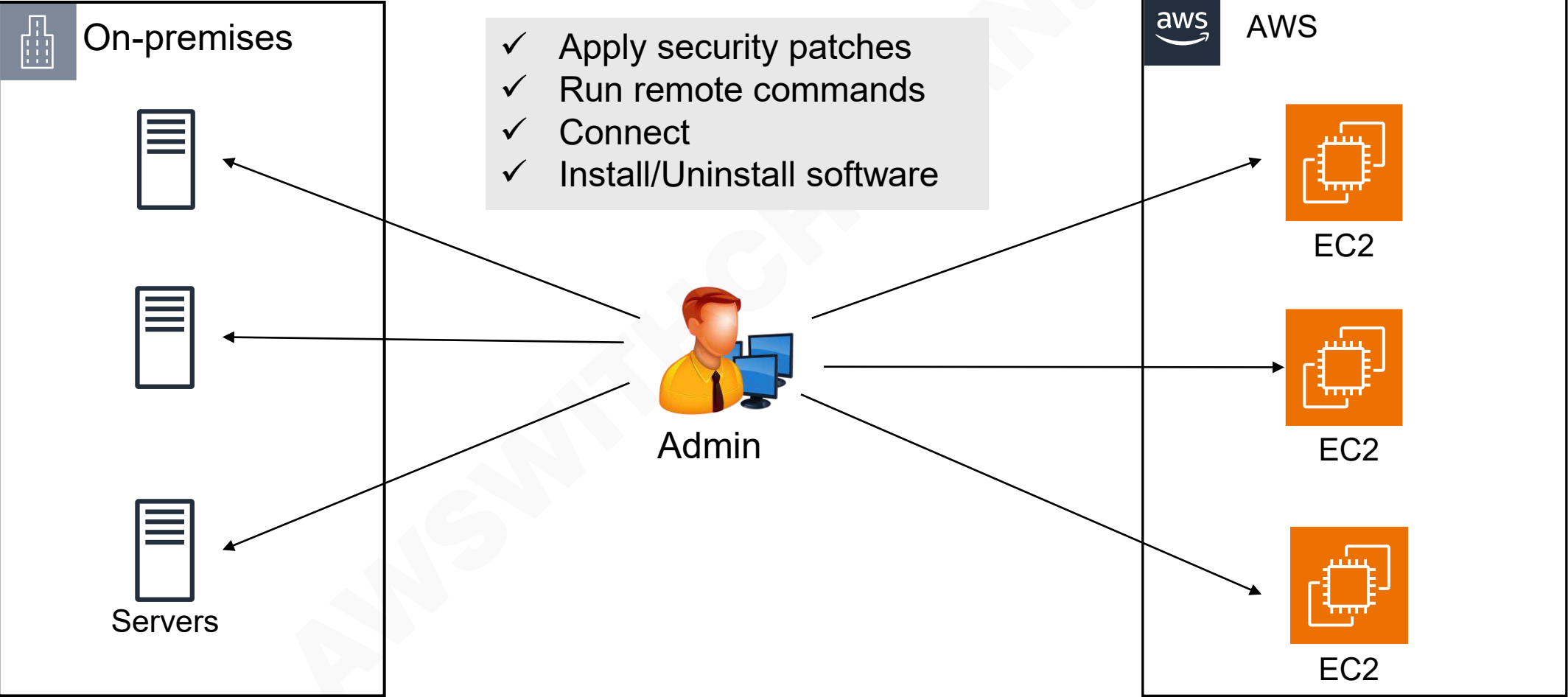
# AWS Resource configuration and state - Tracking





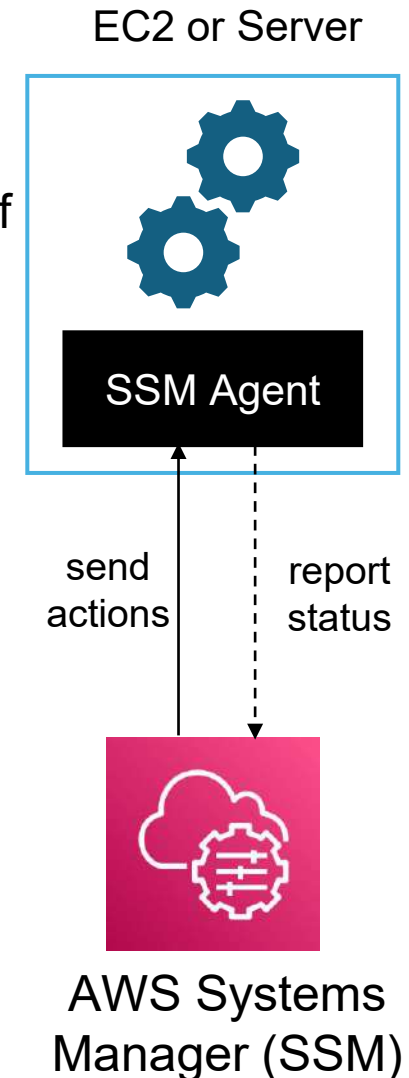
# AWS Systems Manager

# Managing servers



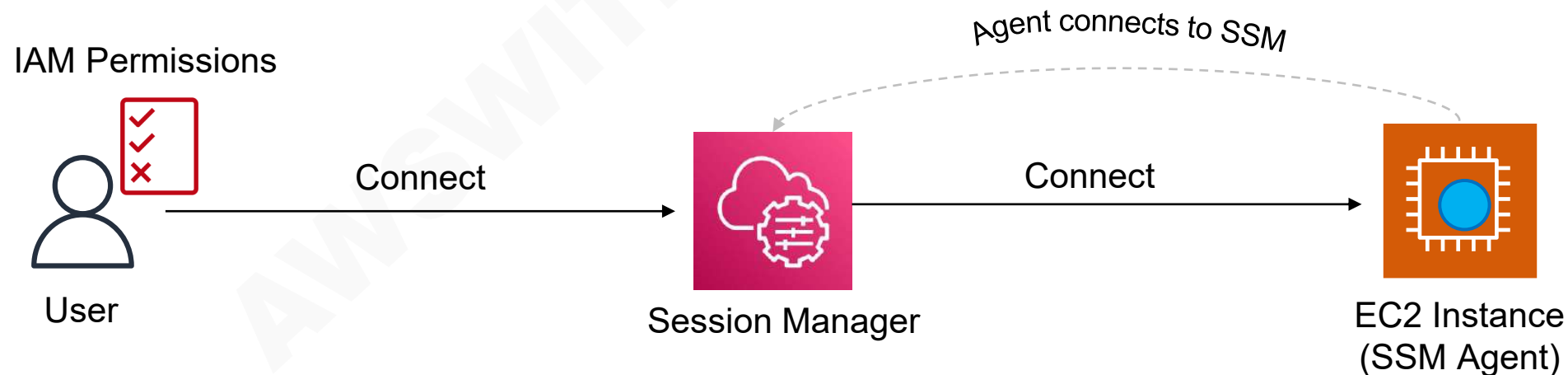
# AWS Systems Manager (SSM)

- Automates common and repetitive IT operations and management tasks
- Works for Linux, Windows, MacOS, and Raspberry Pi OS
- **Run Command** - Author and execute runbooks (SSM documents) across fleet of EC2 or on-premises servers
- **Session Manager** – Provides secure access over HTTPS to connect to instances and servers managed by Systems Manager
- **Fleet Manager** - Get operational insights about the state of your infrastructure
- **Patch Manager** - Select and install OS and software security patches
- **State Manager** – Define the desired configuration (e.g. agents installed) for EC2 instances/servers and apply/re-apply the state by running SSM documents
- **Parameter Store** - Store and manage configuration data and credentials centrally
- More features..



# Systems Manager - Session Manager

- Allows you to start a secure remote session to your EC2 instances or any virtual machine having SSM agent installed.
- Supports Linux, macOS, and Windows.
- **No bastion hosts or SSH keys or SSH port (22) needed.**
- Access is granted through AWS IAM role/policies.

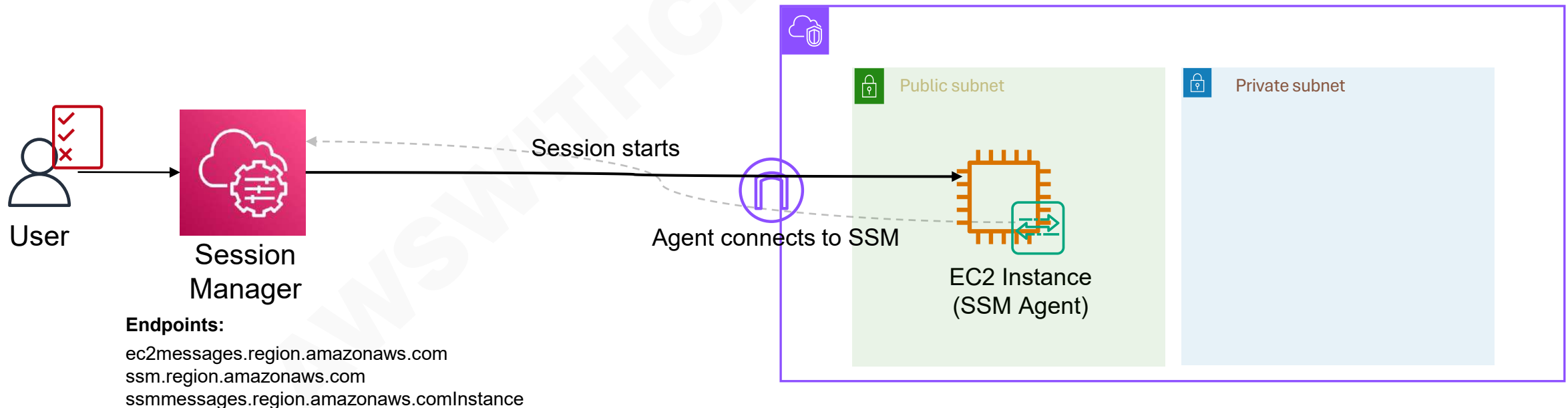


# How it works?

The managed nodes you connect to must also allow HTTPS (port 443) outbound traffic to the following endpoints:

- ✓ [ec2messages.region.amazonaws.com](https://ec2messages.region.amazonaws.com)
- ✓ [ssm.region.amazonaws.com](https://ssm.region.amazonaws.com)
- ✓ [ssmmessages.region.amazonaws.com/Instance](https://ssmmessages.region.amazonaws.com/Instance)

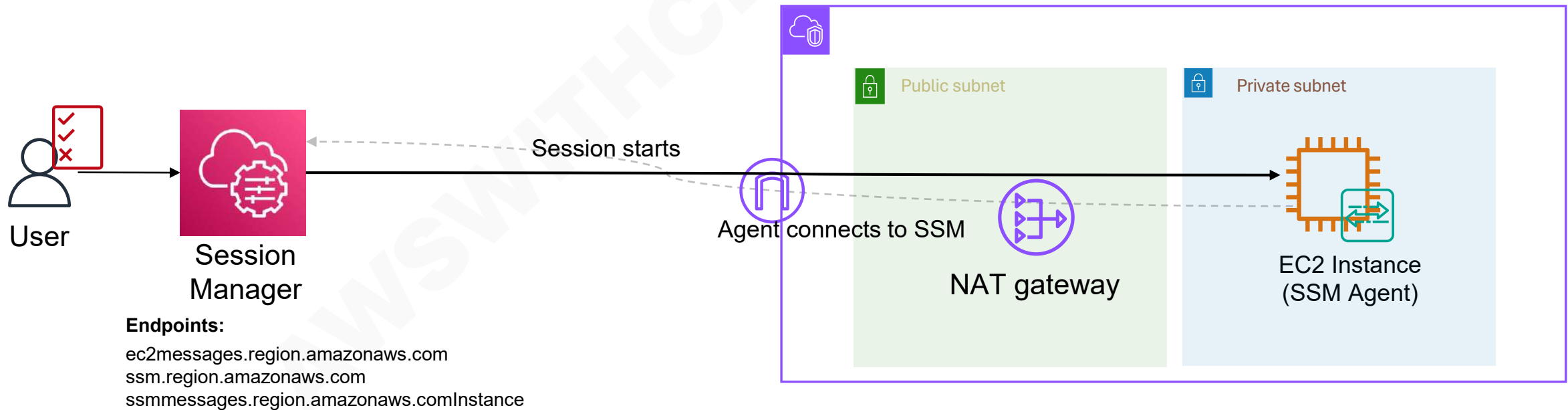
EC2 in Public subnet



# How it works?

The managed nodes you connect to must also allow HTTPS (port 443) outbound traffic to the following endpoints:

- ✓ [ec2messages.region.amazonaws.com](https://ec2messages.region.amazonaws.com)
- ✓ [ssm.region.amazonaws.com](https://ssm.region.amazonaws.com)
- ✓ [ssmmessages.region.amazonaws.com/Instance](https://ssmmessages.region.amazonaws.com/Instance)



# How it works?

The managed nodes you connect to must also allow HTTPS (port 443) outbound traffic to the following endpoints:

- ✓ [ec2messages.region.amazonaws.com](https://ec2messages.region.amazonaws.com)
- ✓ [ssm.region.amazonaws.com](https://ssm.region.amazonaws.com)
- ✓ [ssmmessages.region.amazonaws.com/Instance](https://ssmmessages.region.amazonaws.com/Instance)

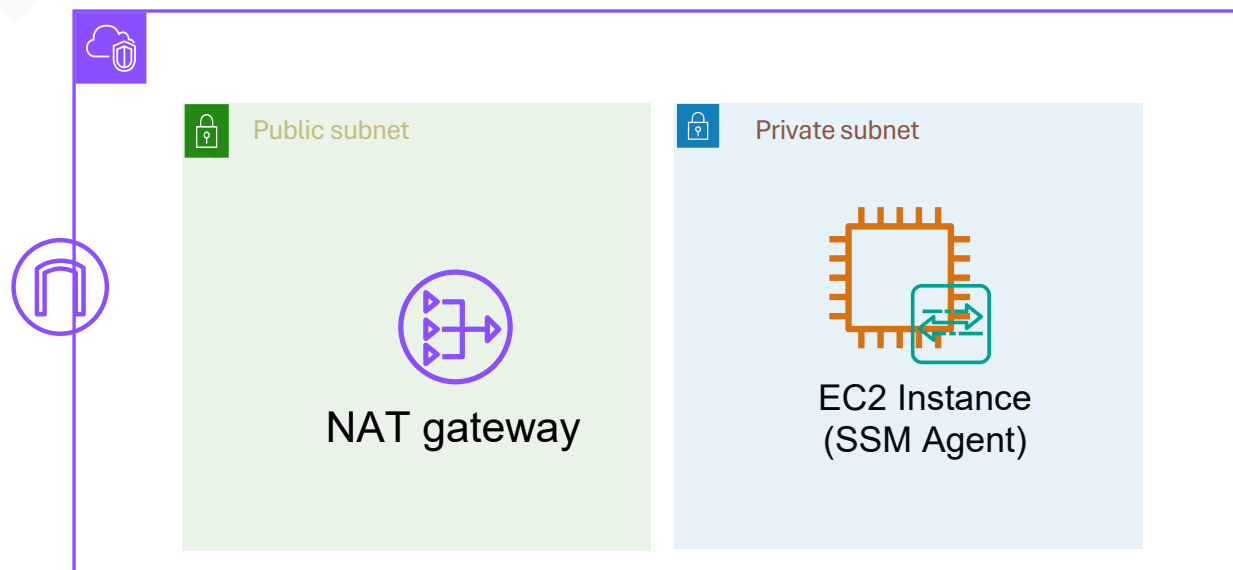
EC2 in Private subnet without outbound internet access



Session  
Manager

**Endpoints:**

ec2messages.region.amazonaws.com  
ssm.region.amazonaws.com  
ssmmessages.region.amazonaws.com/Instance



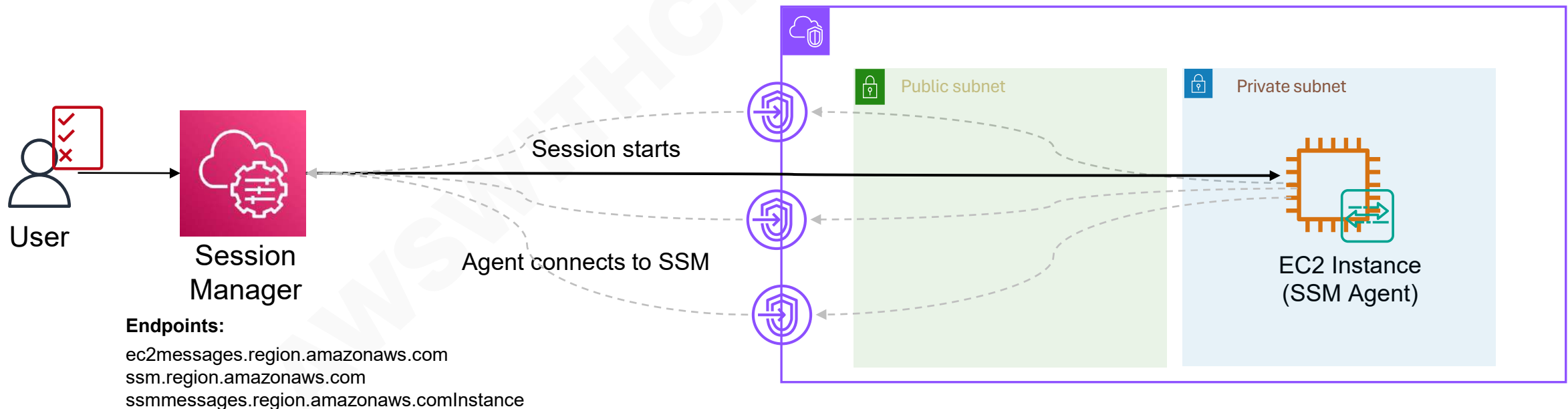


# How it works?

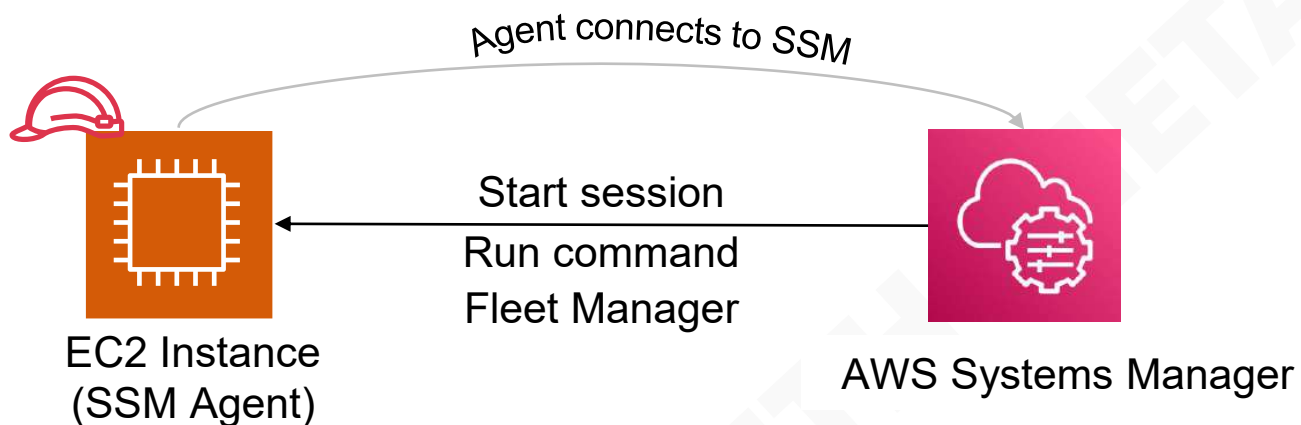
The managed nodes you connect to must also allow HTTPS (port 443) outbound traffic to the following endpoints:

- ✓ [ec2messages.region.amazonaws.com](https://ec2messages.region.amazonaws.com)
- ✓ [ssm.region.amazonaws.com](https://ssm.region.amazonaws.com)
- ✓ [ssmmessages.region.amazonaws.com/Instance](https://ssmmessages.region.amazonaws.com/Instance)

EC2 in Private subnet without outbound internet access



# Exercise – Systems manager for connecting and managing EC2 instances



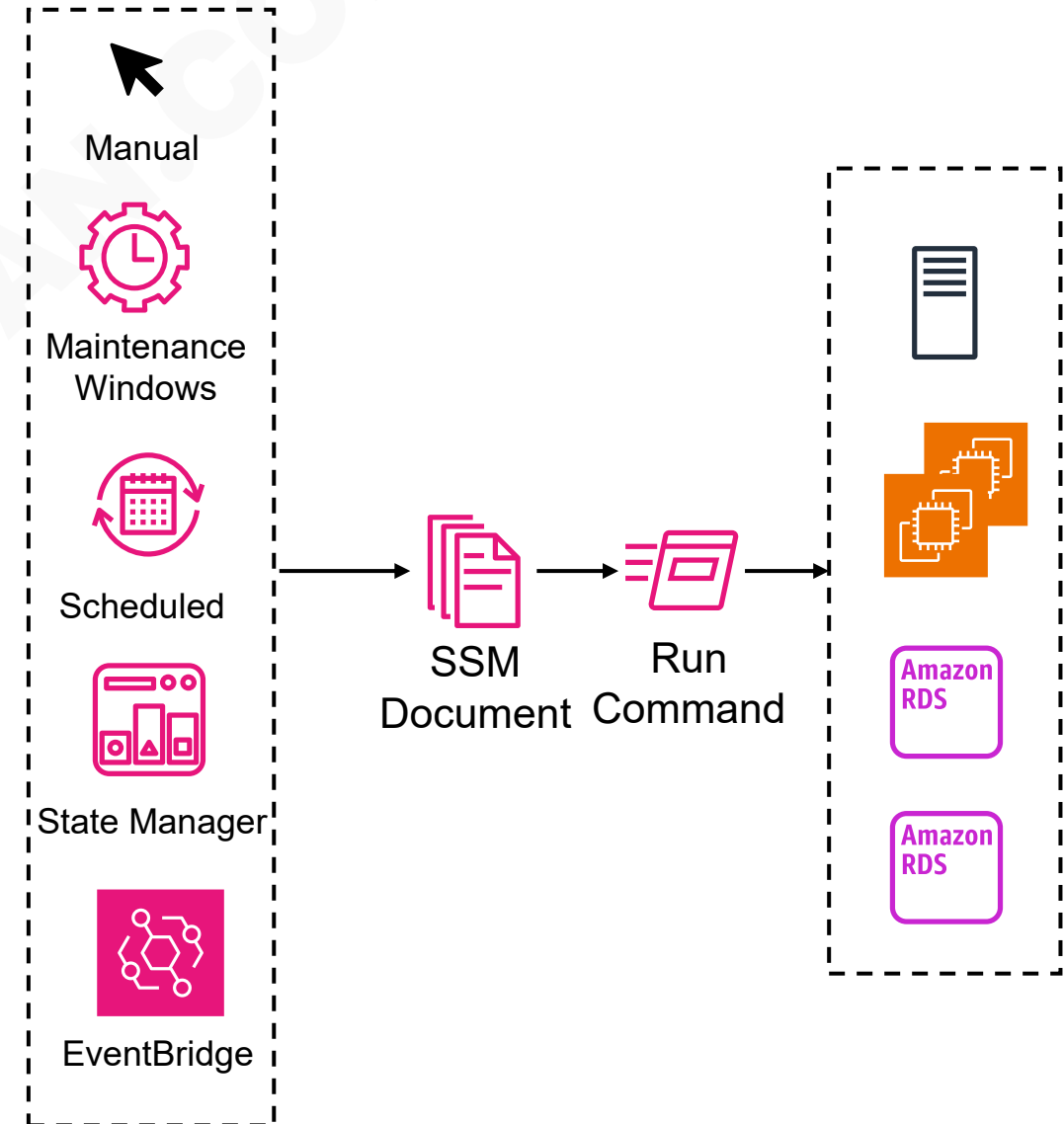
- 1 Create IAM Role for EC2 with permissions **AmazonEC2RoleforSSM**
- 2 Launch EC2 instance (Amazon Linux) and associate this IAM role while launching.
  - Amazon Linux comes with SSM agent pre-installed
  - For security group - No need to have any inbound rule.
- 3 Go to AWS Systems Manager and Session Manager -> Start Session with EC2 instance
- 4 Go to Systems Manager -> Fleet Manager and explore options to manage EC2 instance
- 5 Go to Systems Manager -> Run command and execute simple Shell command on EC2
- 6 Go to EC2 console -> Select Instance -> Connect -> Session Manager
- 7 After the exercise, terminate EC2 instance

# Troubleshooting – If instance does not appear in Systems Manager

- 1 Check if Systems Manager agent is running on EC2 instance
  - a) Open SSH port and connect to instance from your local workstation
  - b) Check if systems manager agent is running by running the command:  
`$sudo systemctl status amazon-ssm-agent`
- 2 Check if correct IAM role is associated with EC2 instance
  - a) For EC2 instance check the IAM role name associated with the instance
  - b) Go to IAM role and verify the IAM policy attached to the role. It should be: AmazonEC2RoleforSSM
- 3 Check if EC2 security group allows Outbound traffic
  - a) For EC2 instance to connect to Systems Manager endpoint, it needs outbound HTTPS connection
  - b) Check if EC2 security group allows outbound traffic (by default all outbound traffic is allowed)
- 4 Restart SSM agent on EC2 instance
  - a) If you modified the IAM role after EC2 instance is launched, you may have to restart the SSM agent.
  - b) Connect to EC2 instance over SSH from your workstation and run the command:  
`$sudo systemctl restart amazon-ssm-agent`

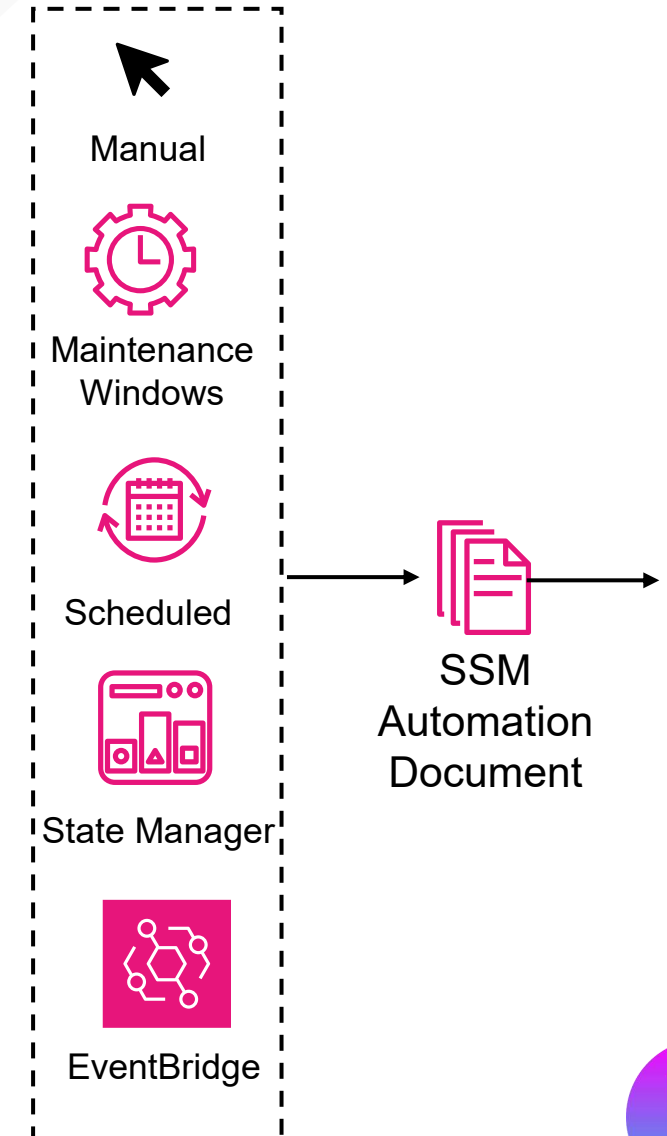
# SSM Documents

- SSM documents define what actions AWS Systems Manager should perform on managed resources.
- SSM documents are written in JSON or YAML and are versioned.
- They can be AWS-managed or customer-created.
- SSM documents are used by Run Command, Automation, State Manager, Patch Manager, and Session Manager.
- Sample documents:
  - `AWS-RunShellScript`
  - `AWS-RunPowerShellScript`
  - `AWS-UpdateSSMAgent`
  - `AWS-ConfigureAWSPackage`
  - `AWSEC2-PatchLoadBalancerInstance`



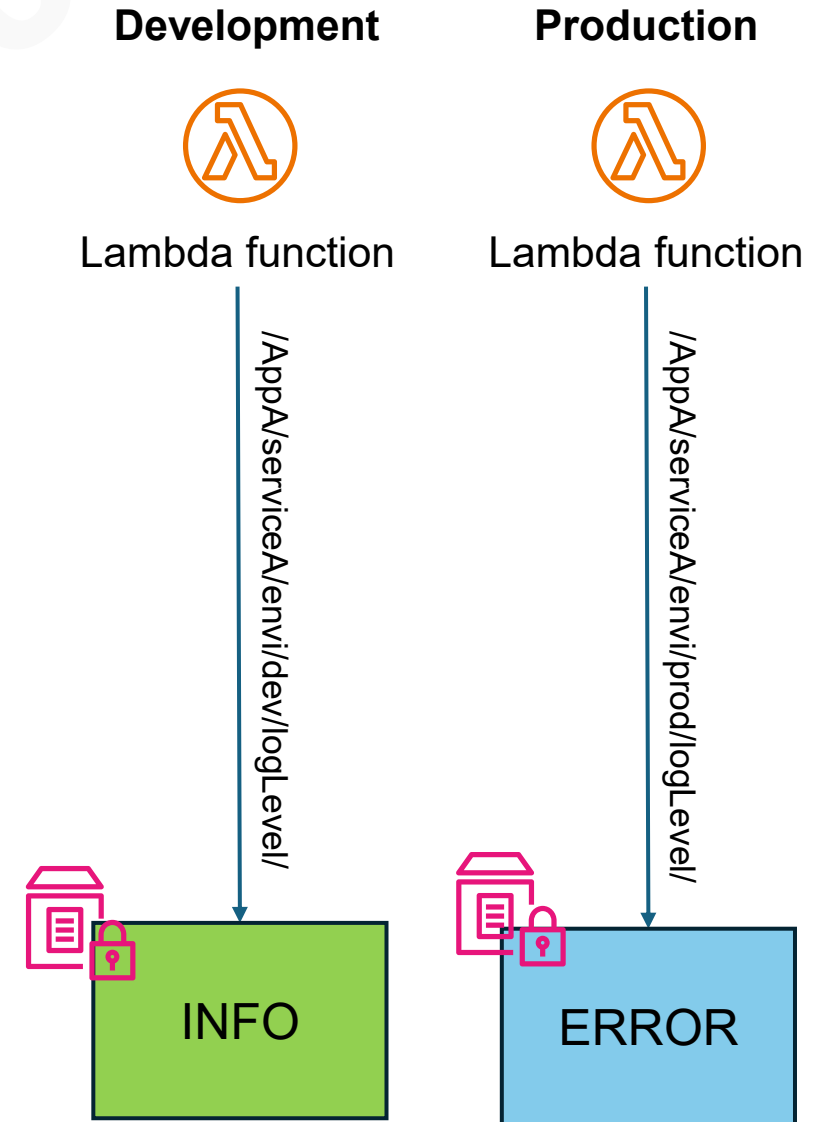
# SSM Automation document

- Automation documents define step-by-step workflows for operational tasks.
- They support branching, waiting, approvals, and rollback.
- Automations can be triggered manually, on a schedule, or by maintenance window or via State Manager or EventBridge.
- Common uses include patching, remediation, maintenance, and deployments.
- Automation documents run with an assumed IAM role.
- **Example:** `AWSEC2-PatchLoadBalancerInstance`
  - Deregisters an EC2 instance from the load balancer
  - Applies OS patches to the instance
  - Reboots the instance if required
  - Re-registers the instance back to the load balancer



# SSM Parameter Store

- It stores key-value parameters such as configuration values, URLs, and credentials.
- Parameters can be plain text or encrypted using AWS KMS.
- Access to parameters is controlled using IAM policies.
- Applications and services can retrieve parameters securely at runtime.
- Supports versioning and parameter history.
- Stores parameters in hierarchical format
  - /AppA/serviceA/env/dev/logLevel = INFO
  - /AppA/serviceA/env/dev/apiBaseUrl = api-dev.example.com
  - /AppA/serviceA/env/dev/requestTimeout = 1000ms
  - /AppA/serviceA/env/prod/logLevel = ERROR
  - /AppA/serviceA/env/prod/apiBaseUrl = api.example.com
  - /AppA/serviceA/env/prod/requestTimeout = 100ms



# Secrets Manager vs SSM Parameter Store

- ✓ Secrets Manager is designed for secrets whereas Parameter Store is designed for configuration.

|                      | Secrets Manager                                                                                                                                                        | SSM Parameter Store                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is it?          | Allows storing and rotating secrets (passwords, keys, tokens)                                                                                                          | Allows storing configuration parameters (strings, JSON, paths)                                                                                           |
| Rotation             | Built-in secrets rotation                                                                                                                                              | Not supported                                                                                                                                            |
| Multi-region support | Multi-region with secret replicas                                                                                                                                      | Not supported                                                                                                                                            |
| Encryption           | Secrets are encrypted by default                                                                                                                                       | Optional encryption                                                                                                                                      |
| Used For             | <ul style="list-style-type: none"> <li>• Database passwords</li> <li>• OAuth tokens</li> <li>• Third-party API keys</li> <li>• Certificates or private keys</li> </ul> | <ul style="list-style-type: none"> <li>• App settings</li> <li>• Feature flags</li> <li>• App versions</li> <li>• Non-sensitive configuration</li> </ul> |

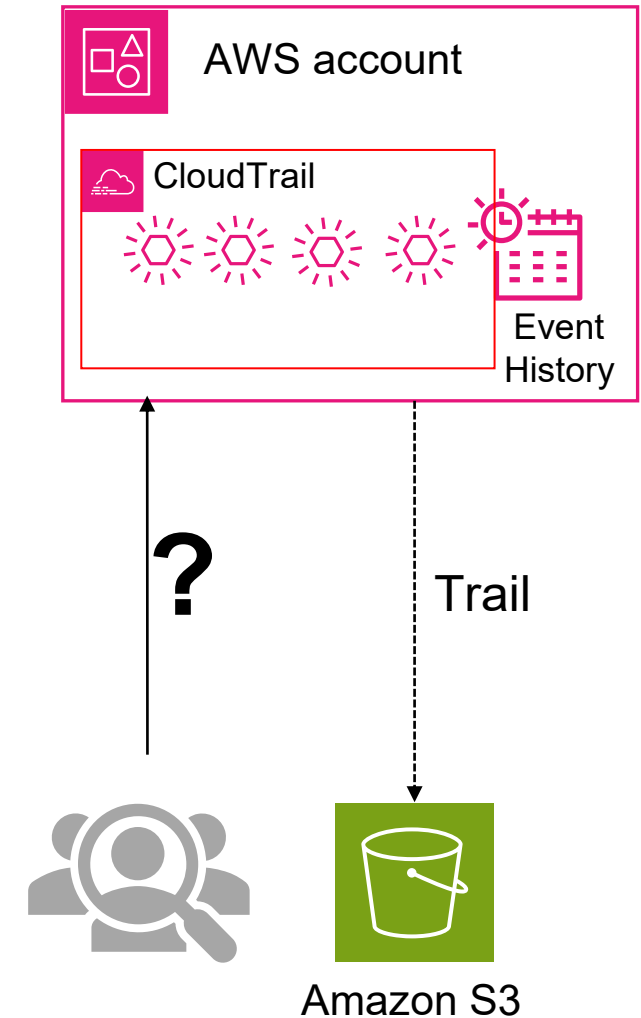


# AWS CloudTrail

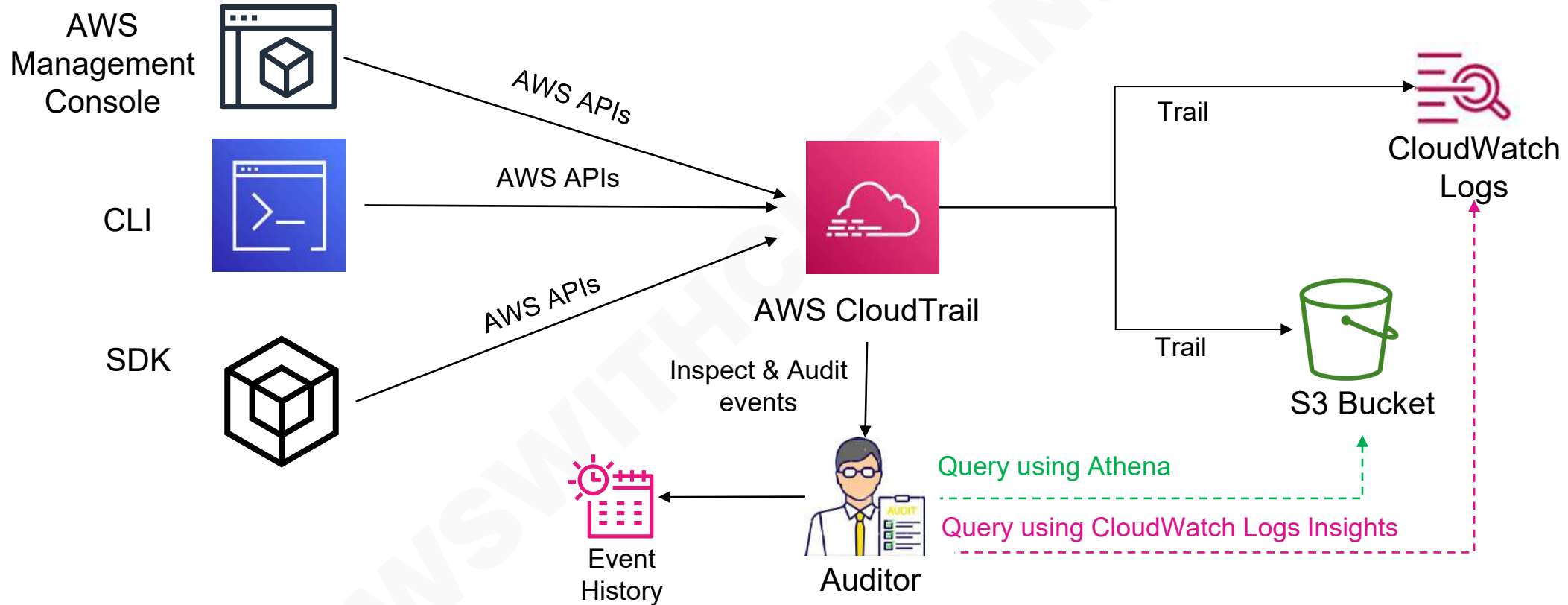


# AWS CloudTrail

- AWS CloudTrail is a service that records everything that happens in your AWS account.
- You can see who did what, when, from where, and using which service.
- Records event history of API calls and user actions made through:
  - AWS Management Console / SDK / CLI / Other AWS services
- By default, CloudTrail shows last 90 days of management events in Event History.
- CloudTrail logs can be delivered centrally using **Trail**. Destination can be:
  - Amazon S3 (for long-term storage)
  - CloudWatch Logs (for monitoring and alerts)
  - Amazon EventBridge
- Trail can be enabled for All regions (Multi-region) or a single region.

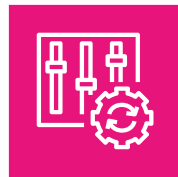


# CloudTrail - Ways to store and access events



# AWS CloudTrail – good to know

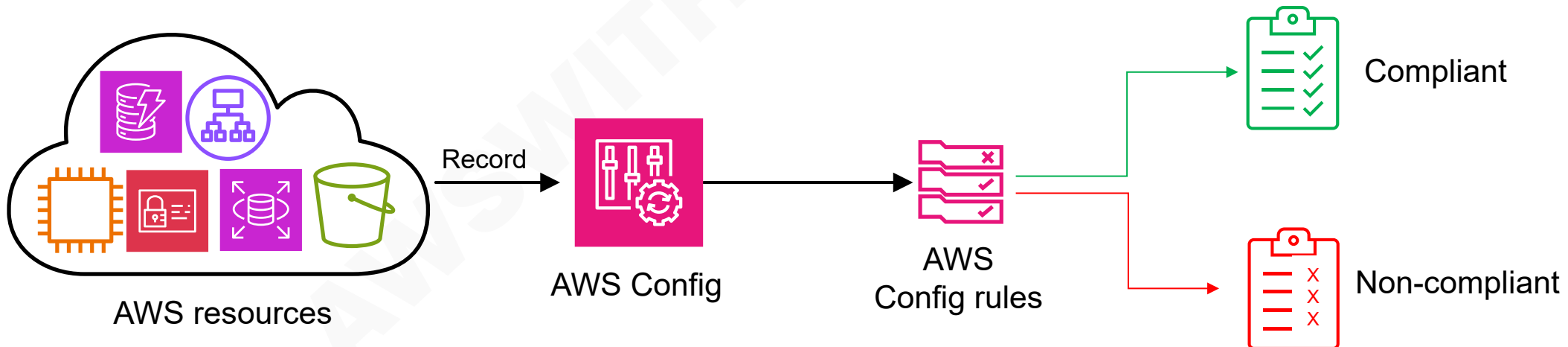
- CloudTrail Event types
  - Management Events: Records control-plane actions like creating, modifying, or deleting AWS resources. Example: `RunInstances`, `CreateBucket` etc.
  - Data Events: Records data-plane actions on resources such as S3 objects or Lambda functions. Example: `GetObject`, `PutObject` (not enabled by default)
- CloudTrail Insights
  - CloudTrail Insights detects unusual API activity using ML.
  - Helps identify spikes, anomalies, or suspicious behavior.
  - Works on management events only.
  - Example: Sudden increase in `TerminateInstances` API calls.
- CloudTrail Lake
  - CloudTrail Lake provides a fully managed immutable event store and allows SQL-based queries
  - Events are aggregated into event data stores, which are immutable collections of events based on criteria you define e.g. Management events, Data Events, Insights events etc.
  - Converts JSON format events to Apache ORC format for easy and efficient query
  - Supports long-term retention (7 years and 10 years retention options)



# AWS Config

# AWS Config

- AWS Config is a service that tracks and records the configuration state of AWS resources over time.
- It helps answer what resources exist, how they are configured, and how they have changed.
- Configuration changes can be sent as a notification and be stored in S3 bucket as a snapshot.
- Config Rules define desired configuration conditions and evaluate resources as compliant or non-compliant.
- Actions include notifications via SNS and automatic or manual remediation using Systems Manager Automation documents.



# AWS Config

## Questions that can be answered by AWS Config:

- Is there a security group which has port 22 (SSH) open for the world (0.0.0.0/0)?
- Are there expired ACM certificates?
- Are there any resources which are not tagged?
- Are there any S3 buckets in my account which are Public (can be read by anyone)?
- Is there any DynamoDB table having provisioned capacity above 100 WCU?
- Is there a RDS database which is not multi-AZ?

[AWS Config](#) > [Advanced queries](#)

### Advanced queries

Use one of the sample queries or write your own query by referring to the configuration schema of the AWS resource.

**Advanced queries (58)**

|                       | Name                                                                             | Description                                                                                        | Creator |
|-----------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|---------|
| <input type="radio"/> | <a href="#">Count EC2 Instances</a>                                              | Count EC2 instances, group by instance type                                                        | AWS     |
| <input type="radio"/> | <a href="#">Count Non-compliant</a>                                              | Count number of non-compliant resources, group by resource type, sort by count in descending order | AWS     |
| <input type="radio"/> | <a href="#">Count by compliant</a>                                               | Count number of resources, group by Config rule compliance status                                  | AWS     |
| <input type="radio"/> | <a href="#">Count of compliant and non-compliant rules of a conformance pack</a> | Count of compliant and non-compliant rules for conformance pack "conformance-pack-12345"           | AWS     |
| <input type="radio"/> | <a href="#">Active DynamoDB tables</a>                                           | List all active DynamoDB tables                                                                    | AWS     |
| <input type="radio"/> | <a href="#">Availability zones for Load Balancer</a>                             | List all availability zones of a Load Balancer "arn:aws:elasticloadbalancing:12345"                | AWS     |
| <input type="radio"/> | <a href="#">DynamoDB tables with disabled SSE</a>                                | List all DynamoDB tables where server-side encryption is disabled                                  | AWS     |
| <input type="radio"/> | <a href="#">Unused EBS</a>                                                       | List all EBS volumes that are not in use                                                           | AWS     |

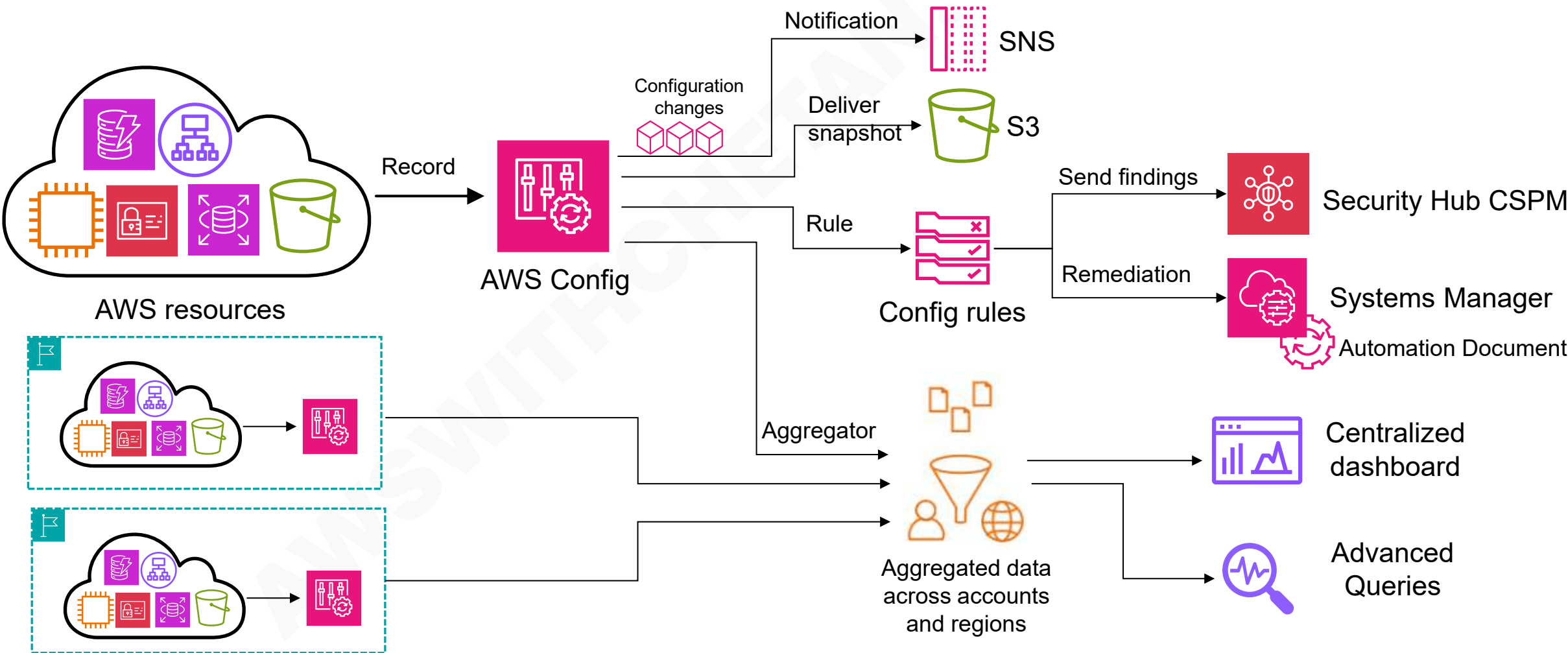
# AWS Config – Features

- **Aggregators:** Configuration Aggregators collect and provide a centralized view of resources across accounts and regions.
- **Advanced Queries:** You can use AWS Config to query the current configuration state of AWS resources based on configuration properties for a single account and Region or across multiple accounts and Regions.

```
SELECT * WHERE resourceType = 'AWS::EC2::SecurityGroup'  
AND configuration.ipPermissions.ipRanges BETWEEN '10.0.0.0' AND '10.0.0.255'  
AND NOT configuration.ipPermissions.ipRanges < '10.0.0.0'  
AND NOT configuration.ipPermissions.ipRanges > '10.0.0.255'
```

- **Remediation:** AWS Config allows you to remediate noncompliant resources using AWS Systems Manager Automation documents
- **Conformance pack:** A conformance pack is a collection of AWS Config rules and remediation actions that can be easily deployed as a single entity in an account and a Region or across an organization in AWS Organizations.

# AWS Config – Features

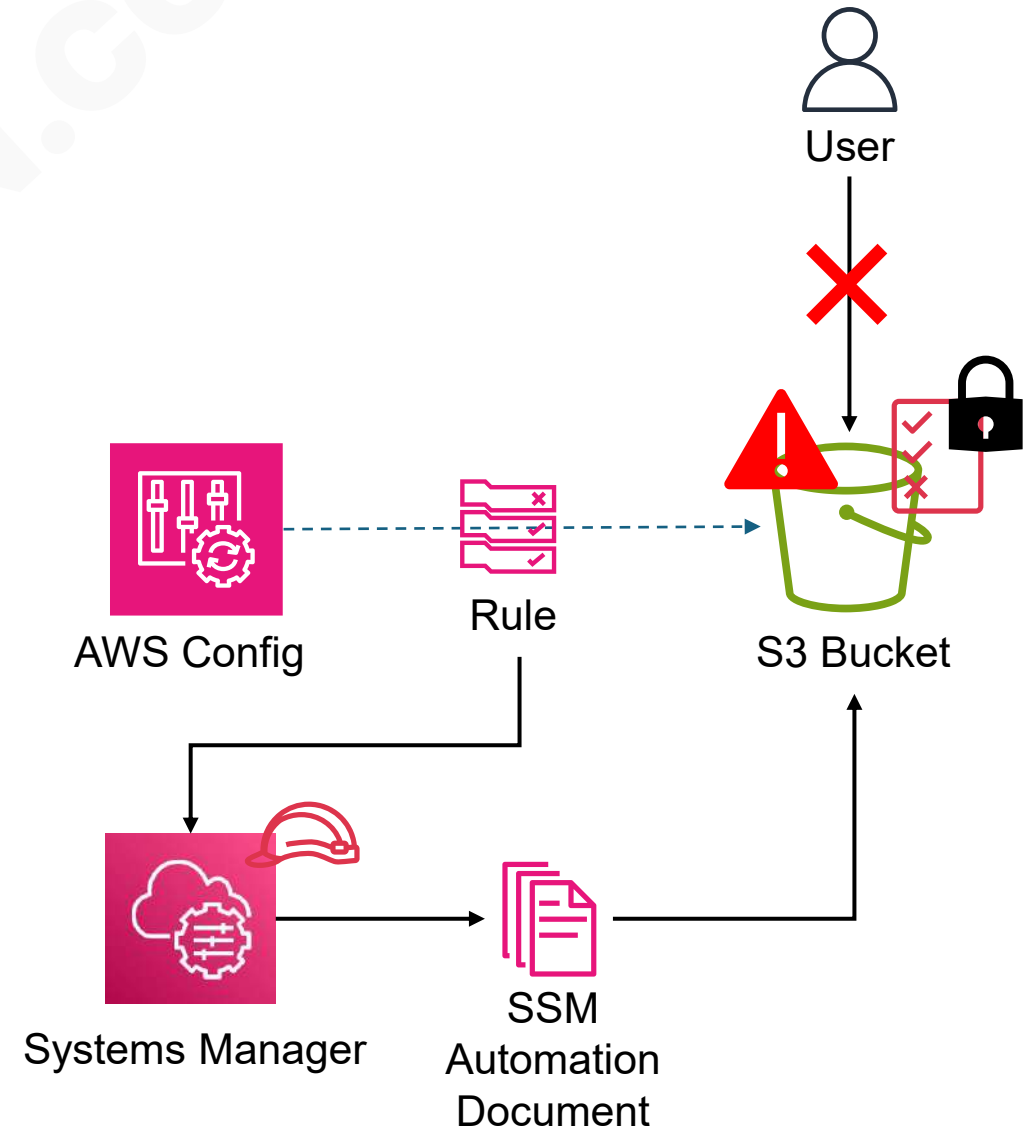




# AWS Config - Features

## Remediation

- AWS Config allows you to remediate noncompliant resources that are evaluated by AWS Config Rules.
- Remediation actions are executed using AWS Systems Manager Automation documents.
- Remediation can be configured to retry automatically if it fails and you can specify the maximum retry attempts and retry interval.
- Systems Manager execution role needs to have permissions to perform the remediation actions.



# AWS Security services

# AWS Security services



## Identity and access



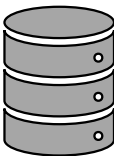
AWS IAM



AWS IAM Identity Center



Amazon Cognito



## Data protection



AWS Key Management Service (KMS)



AWS Certificate Manager



AWS Secrets Manager



Amazon Macie



## Network and Infrastructure



VPC Security Groups and NACL

AWS WAF

AWS Shield

AWS Firewall Manager

AWS Network Firewall



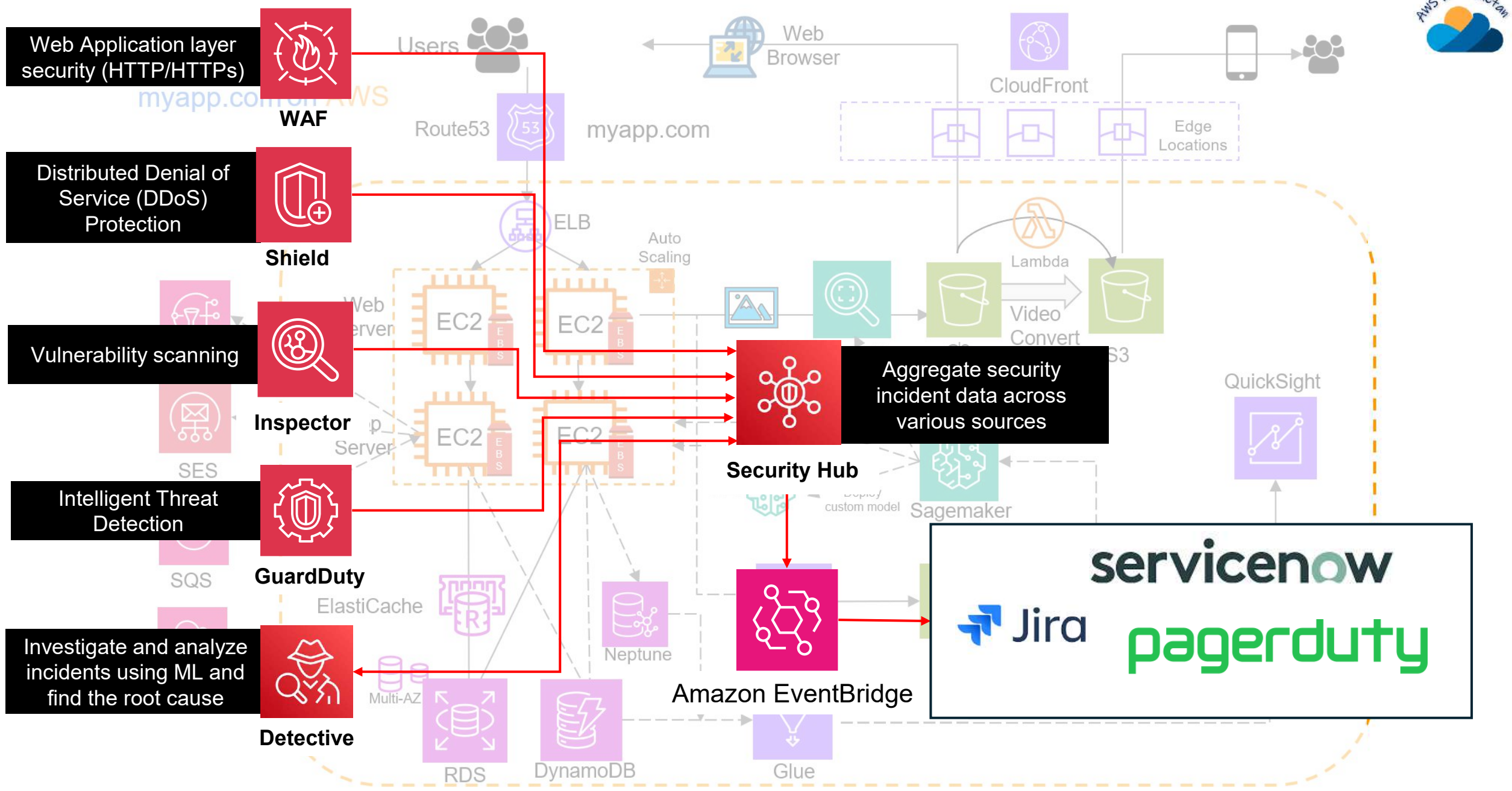
## Threat Detection and incident management

Amazon GuardDuty

Amazon Inspector

Amazon Detective

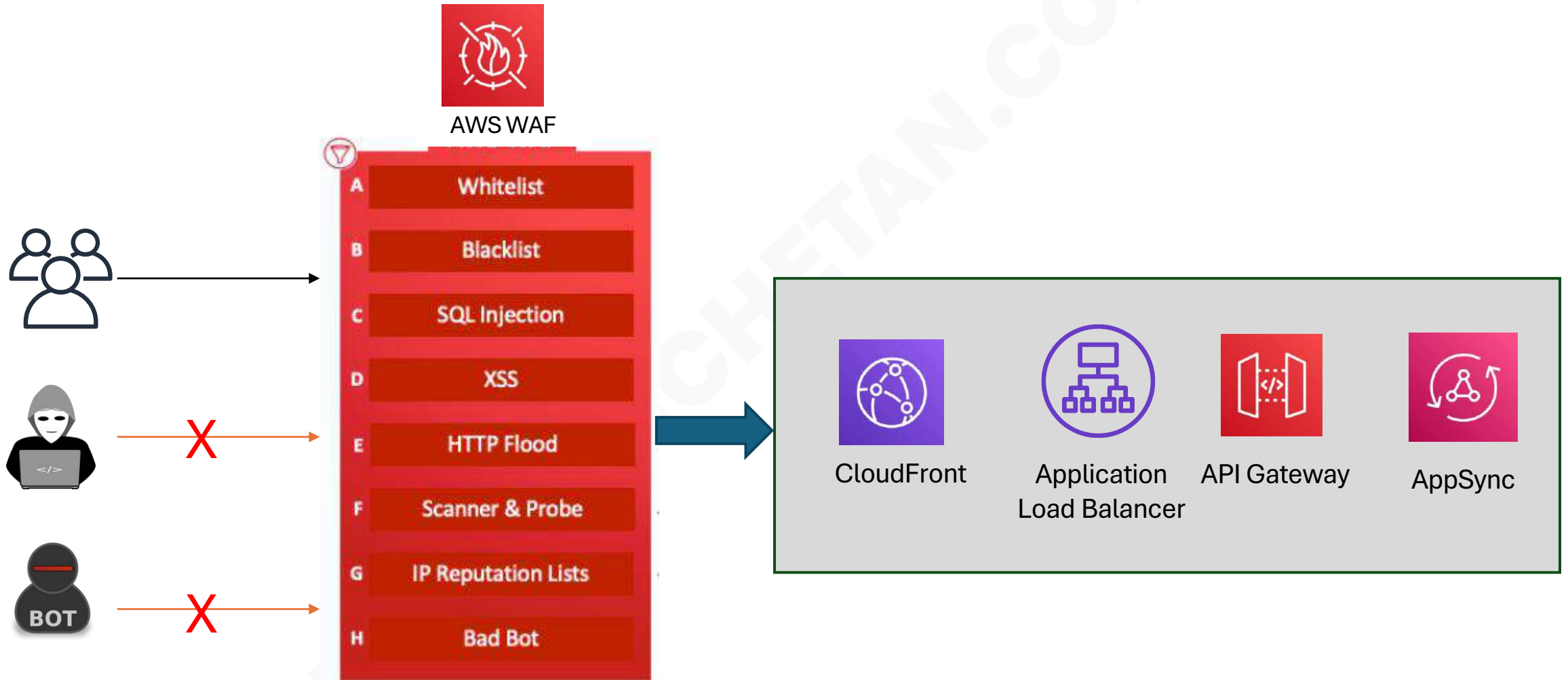
AWS Security Hub





# AWS WAF

# AWS Web Application Firewall (WAF)



# AWS WAF – Web Application Firewall

- Protects web applications from common web exploits (**Layer 7**) – **OWASP Top 10**
- Deploy on **Application Load Balancer**, **API Gateway** or **AppSync** GraphQL APIs and **CloudFront**
- Define Web ACL (Web Access Control List):
  - Block IP addresses
  - Filter traffic based on HTTP headers, HTTP body, or URI strings
  - Detect and block common attacks - SQL injection and Cross-Site Scripting (XSS)
  - Geo-match – allow or block countries or regions
  - Rate-based rules (to count occurrences of events) – for DDoS protection
- Can present CAPTCHA or challenge to prevent bot attacks
- On blocking the malicious traffic WAF returns HTTP 403 status code (Forbidden)

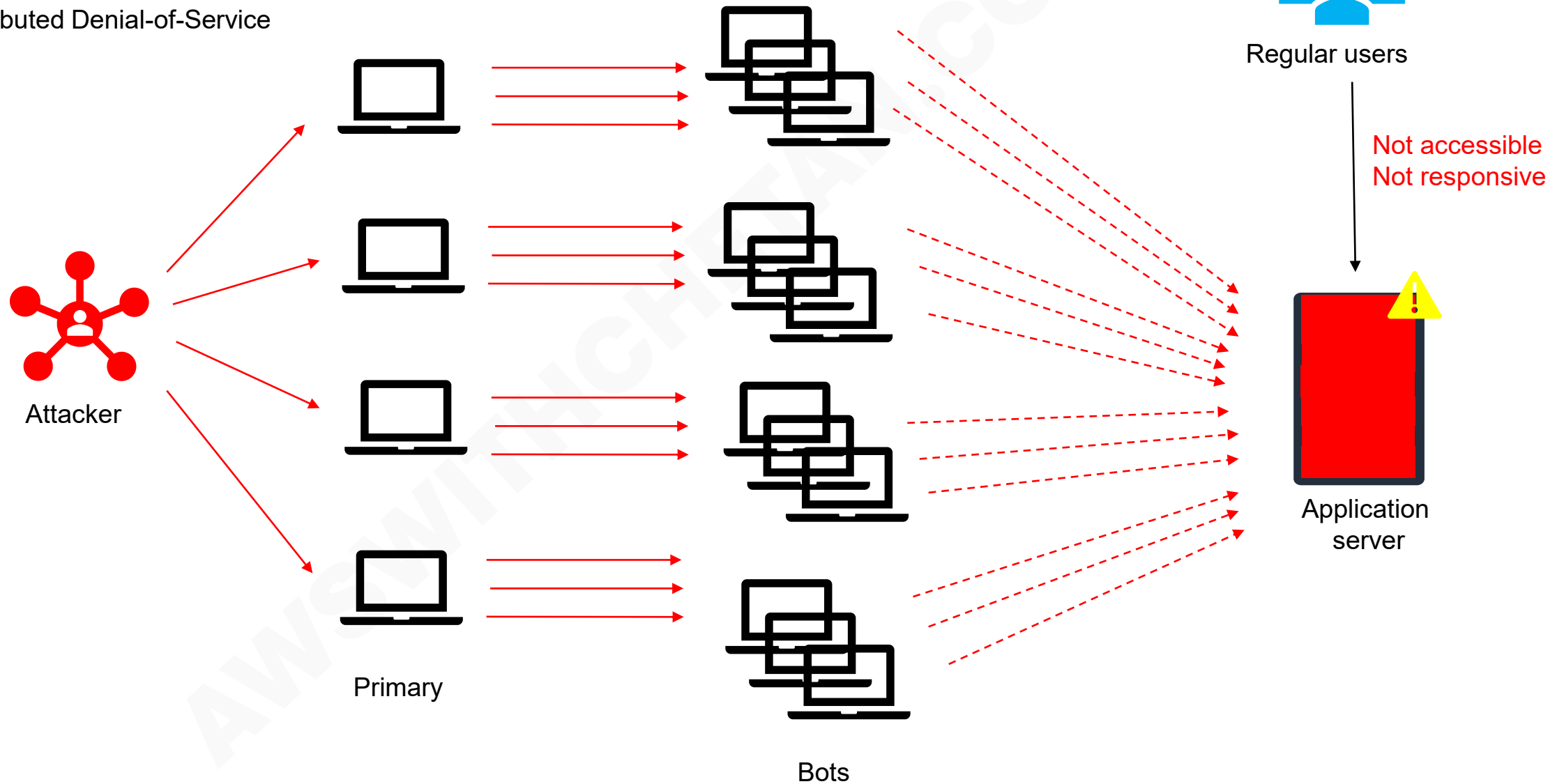


# AWS Shield



# What's a DDOS attack?

Distributed Denial-of-Service

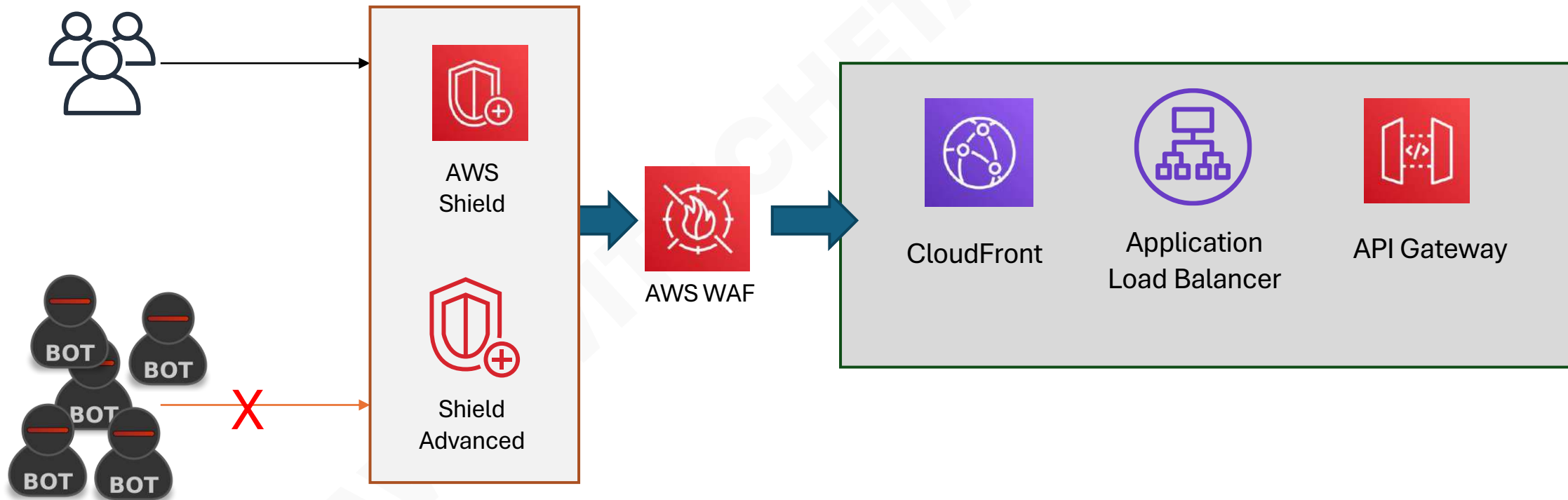


# Common DDoS attacks

- SYN Flood attack: Too many half open TCP connections
- UDP Flood attack: Too many UDP requests
- UDP Reflection attack: Spoof the victim server IP as a source for UDP packet. Victim server receives the unexpected responses.
- DNS Flood attack: Overwhelm the DNS so legitimate users can't find the site
- Slow Loris attack (Layer7): A lot of HTTP connections are opened and maintained
- Cache Busting attack: Request un-cached data from CDN

# AWS Shield

Provides protection against DDoS attack at Network Layer (layer3) and Transport Layer (layer4)



# AWS Shield

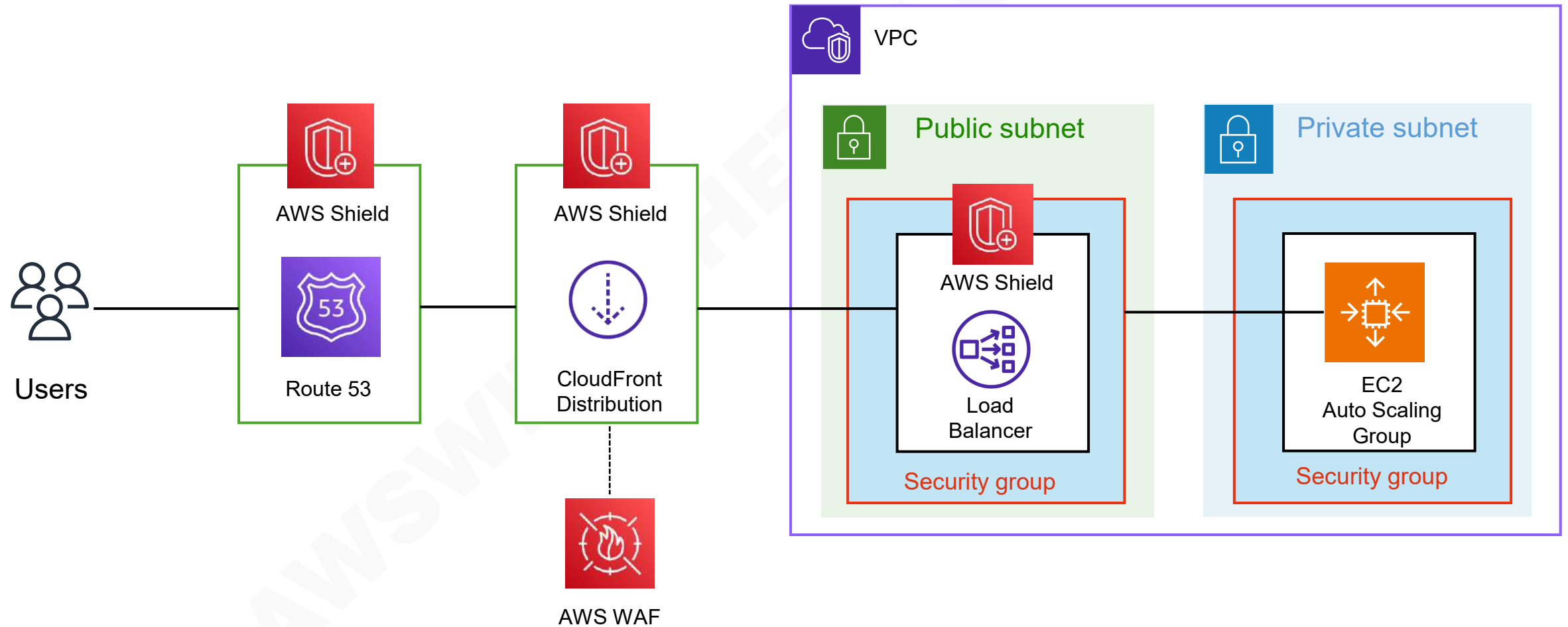
## AWS Shield Standard:

- **Free service** that is activated for every AWS customer

## AWS Shield Advanced:

- Optional DDoS mitigation service (\$3,000 per month per organization)
- Protect against more sophisticated attack on Amazon EC2, Elastic Load Balancing (ELB), Amazon CloudFront, AWS Global Accelerator, and Route 53
- **24/7 access to AWS Shield Response Team (SRT)**
- Protect against higher fees during usage spikes due to DDoS

# Reference Architecture for DDoS Protection

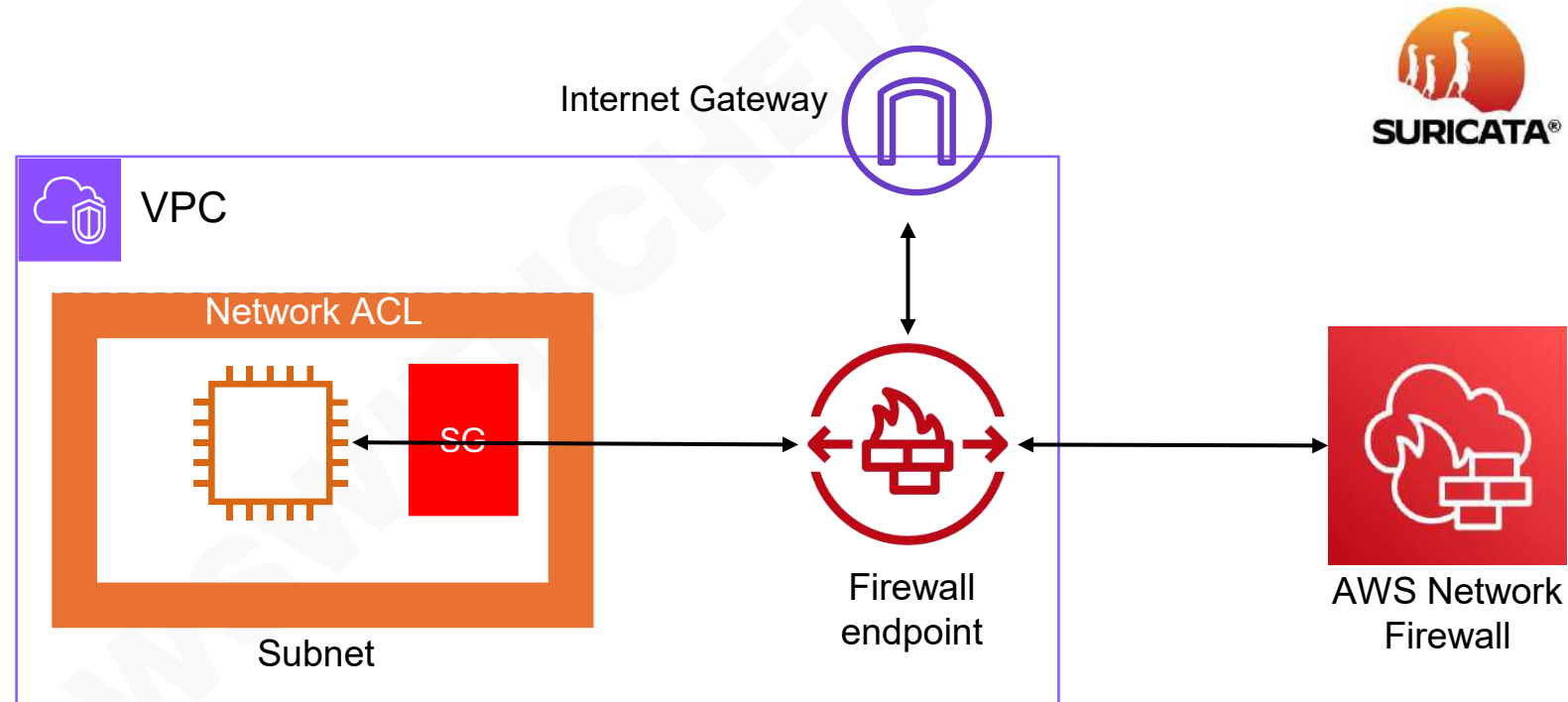




# AWS Network Firewall

# AWS Network Firewall

A stateful network firewall and **intrusion detection and prevention** service for VPC





# AWS Firewall Manager

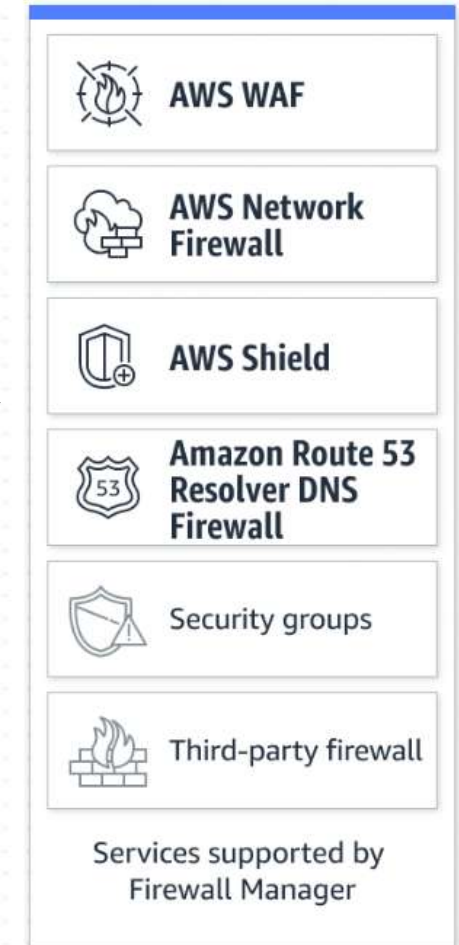


# AWS Firewall Manager

- Centrally configure and manager Firewall rules across AWS accounts
- Manages the rules for AWS WAF, AWS Shield Advanced, Amazon VPC security groups, AWS Network Firewall, and Amazon Route 53 Resolver DNS Firewall
- Integrates with AWS Organization:
  - Provides centralized monitoring of DDoS attacks across AWS organization
  - New accounts added under the AWS Organizations are automatically protected



AWS Firewall  
Manager

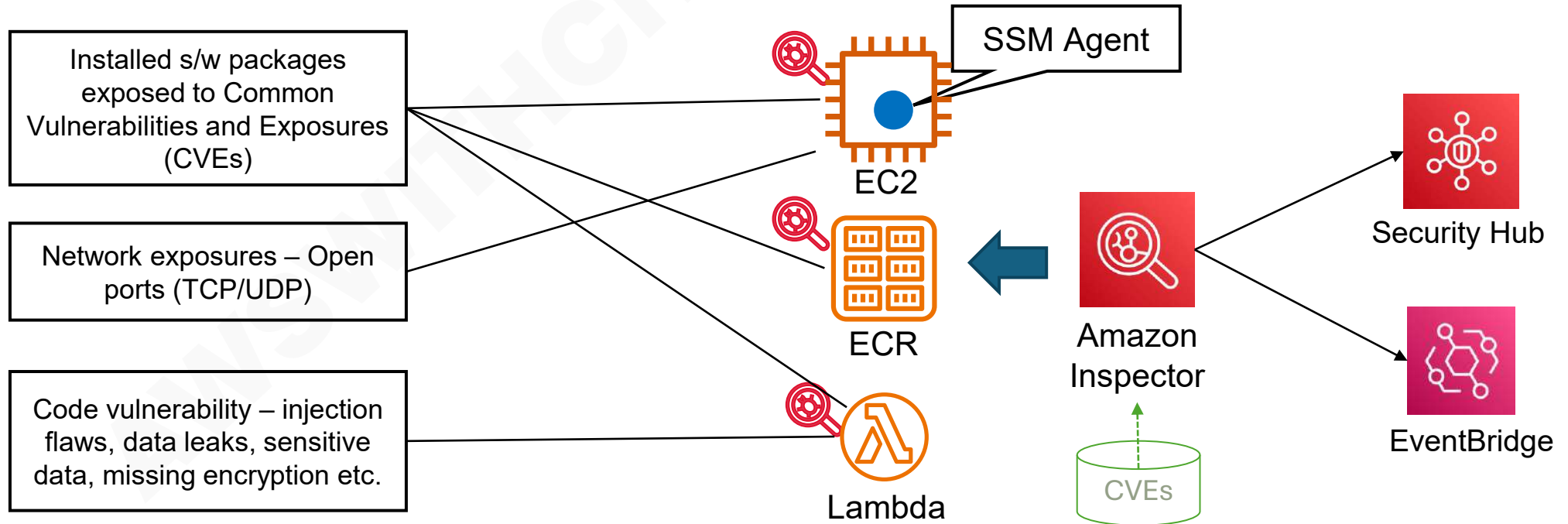




# Amazon Inspector

# Amazon Inspector

- Amazon Inspector is a vulnerability management service that continuously monitors your AWS workloads for software vulnerabilities and unintended network exposure.
- Scans EC2 instances, Container Images (ECR) & Lambda functions for software vulnerabilities (CVEs)
- Publishes findings to Amazon EventBridge and AWS Security Hub for reporting and action
- Inspector can be centrally managed across multiple accounts using **AWS Organizations**.



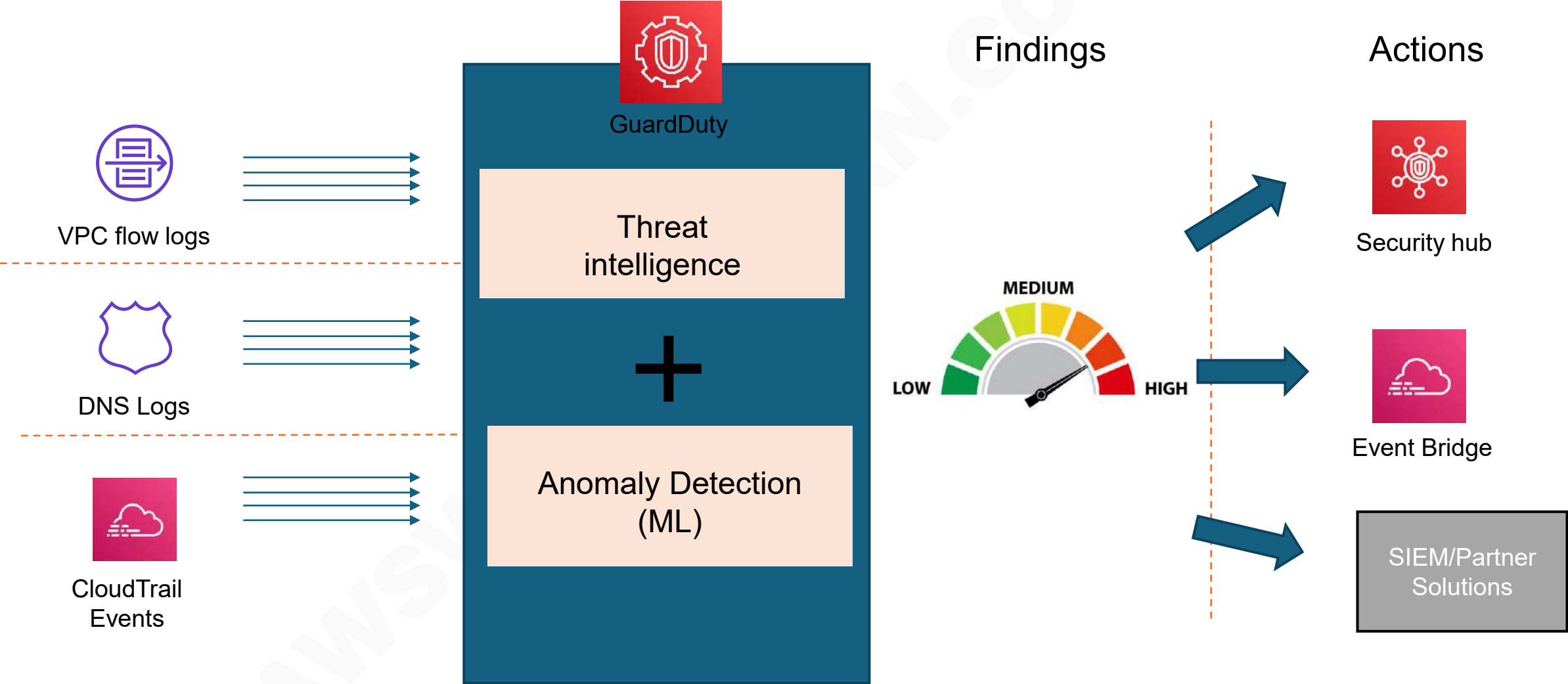
# Amazon GuardDuty

- Intelligent threat detection service - Enable with just “One click”
- Uses Machine learning to detect threats more accurately
- Can detect sophisticated attacks e.g. EC2 instance being used for crypto currency or Bitcoin mining, data exfiltration, unusual access to malicious IP etc.
- Analyses tens of billions of events across multiple AWS data sources such as:
  - **AWS CloudTrail logs:** Unusual API calls, unauthorized deployments.
  - **VPC Flow Logs:** Unusual internal traffic, unusual IP address.
  - **DNS query logs:** Compromised EC2 instances sending encoded data within DNS queries.
- Notifies the security findings:
  - Amazon EventBridge
  - AWS Security Hub

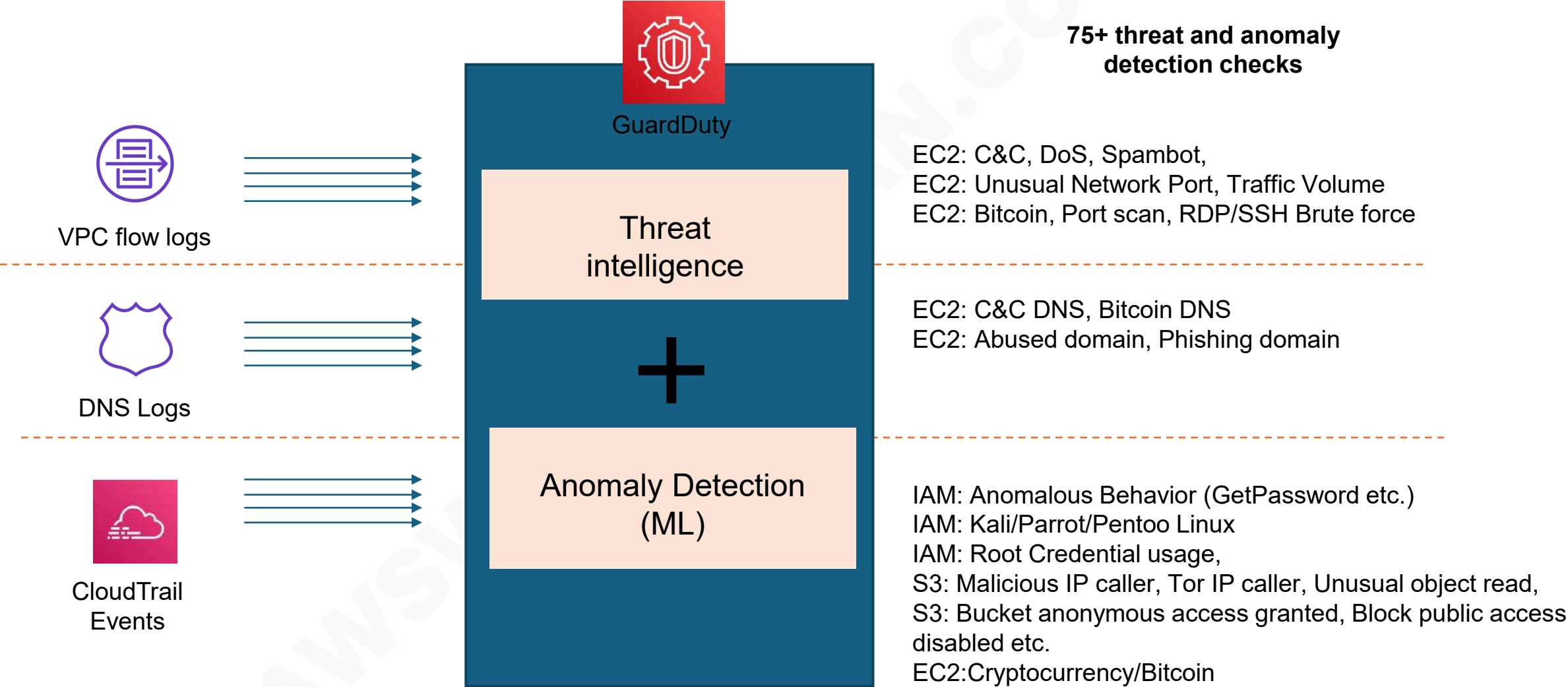


Amazon GuardDuty

# Amazon GuardDuty



# Amazon GuardDuty





# AWS Security Hub

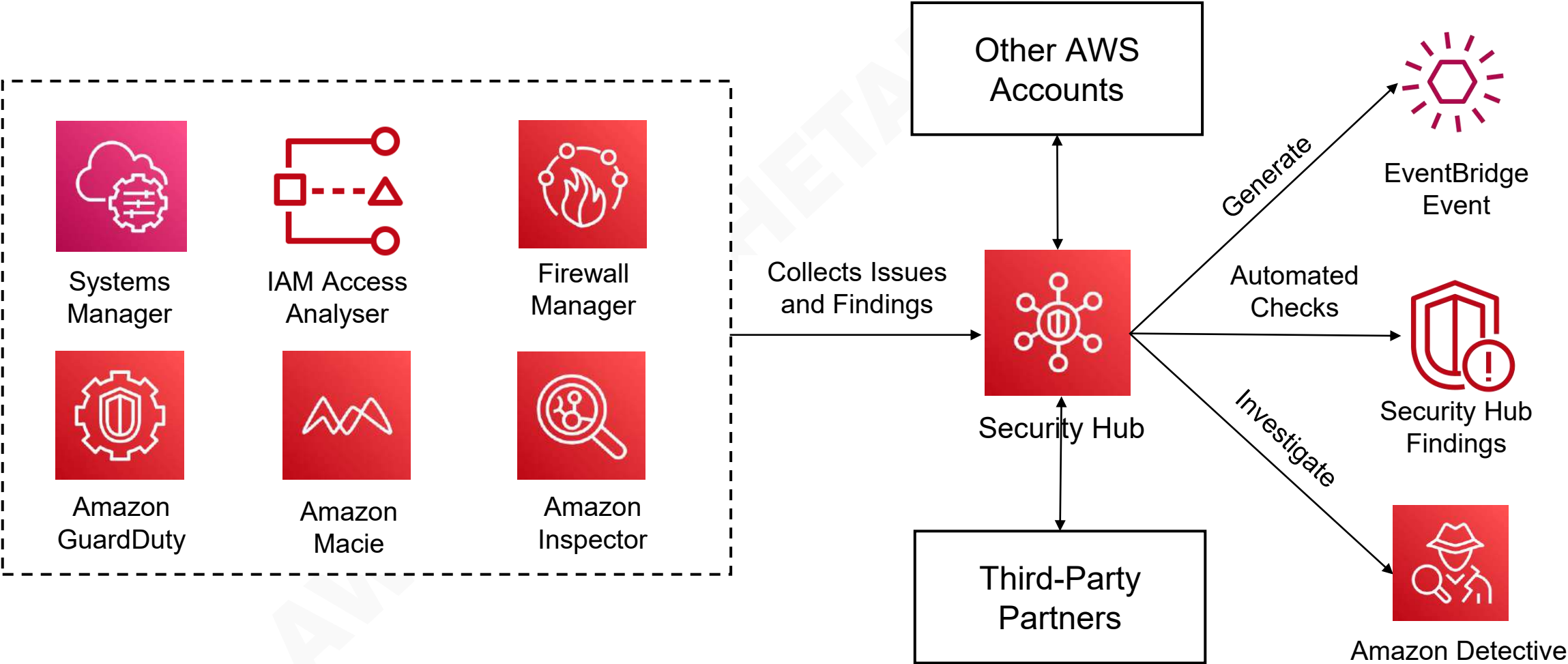
# AWS Security Hub

- Centrally collects security data from across AWS accounts and services.
- Helps analyze security trends to identify and prioritize the security issues across your AWS landscape.
- Automatically aggregates alerts from various AWS services and AWS partner tools such as:
  - Amazon GuardDuty
  - Amazon Inspector
  - Amazon Macie
  - IAM Access Analyzer
  - AWS Systems Manager
  - AWS Firewall Manager
  - AWS Partner Network Solutions

Must enable AWS Config service so that AWS resource state can be captured and analysed

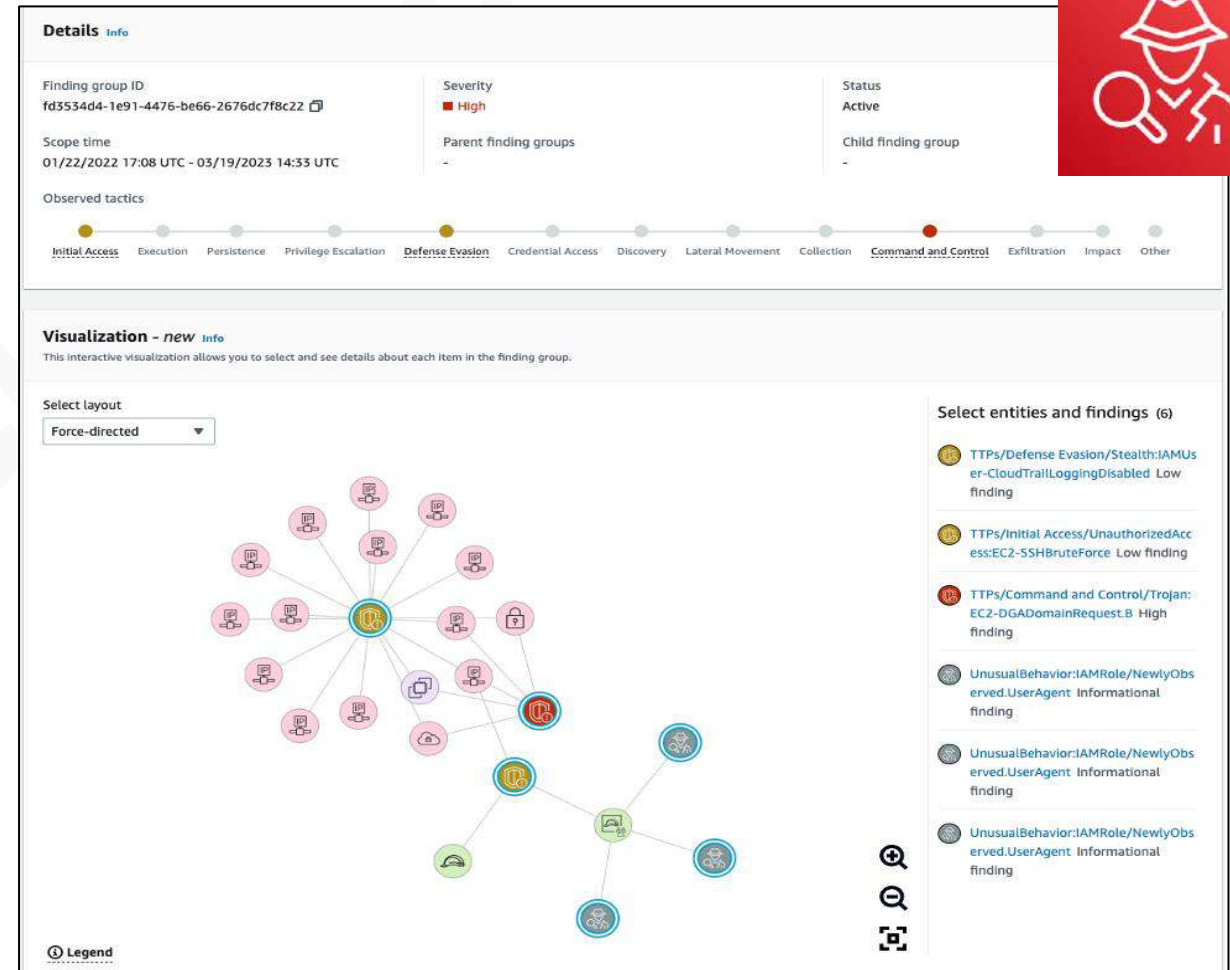


# AWS Security Hub



# Amazon Detective

- Uses Machine learning and Graph technology to analyze, investigate, and identify the **root cause** of security issues
- Uses network traffic data, AWS account activity events and security findings from Amazon Macie, Amazon GuardDuty, Security Hub etc.



# Other Storage services, Hybrid Storage and Data Migration services

# AWS Storage services

✓

Block



Amazon EBS      EC2 instance Storage

File



Amazon EFS      Amazon FSx for NetApp ONTAP      Amazon FSx for Windows File Server      Amazon FSx for Lustre      Amazon FSx for OpenZFS

✓

Object



Amazon S3



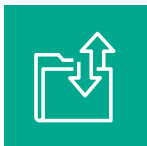
Hybrid Storage and Data Transfer

Hybrid Storage



AWS Storage Gateway

Data Transfer



AWS DataSync      AWS Transfer Family      AWS Data Transfer Terminal

Backup

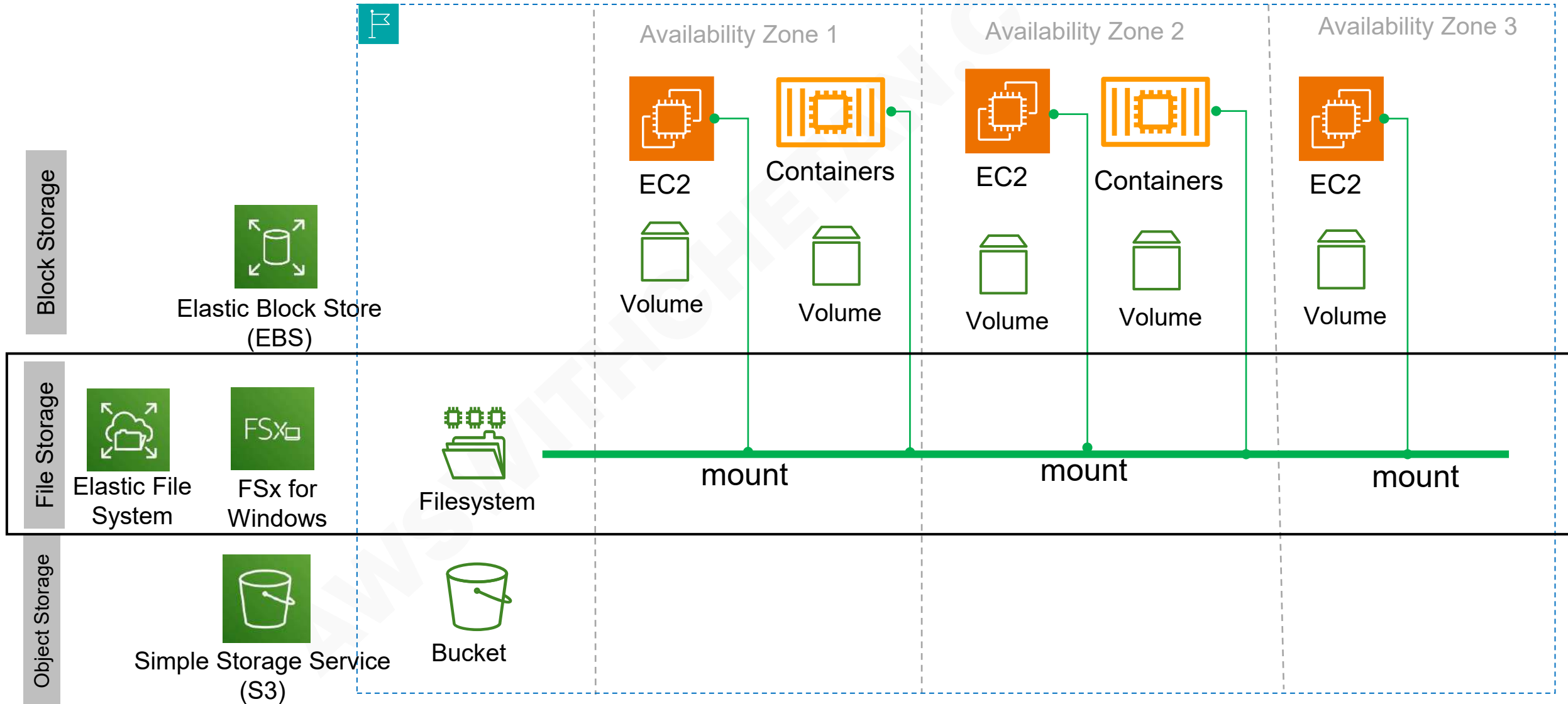


AWS Backup



# Elastic File System (EFS)

# AWS Storage services



# Amazon Elastic File System (EFS)

- A managed **Network File System** (NFS 4)
- Can be shared by hundreds of EC2 Linux instances
- Works with most of the AWS compute services
- Serverless - no need to manage infrastructure
- Elastic – no need to pre-provision the capacity, pay as per the storage used
- Use cases:
  - Containerized and serverless applications
  - Machine learning training
  - Web serving and content management
  - User home directories



EC2



ECS



EKS

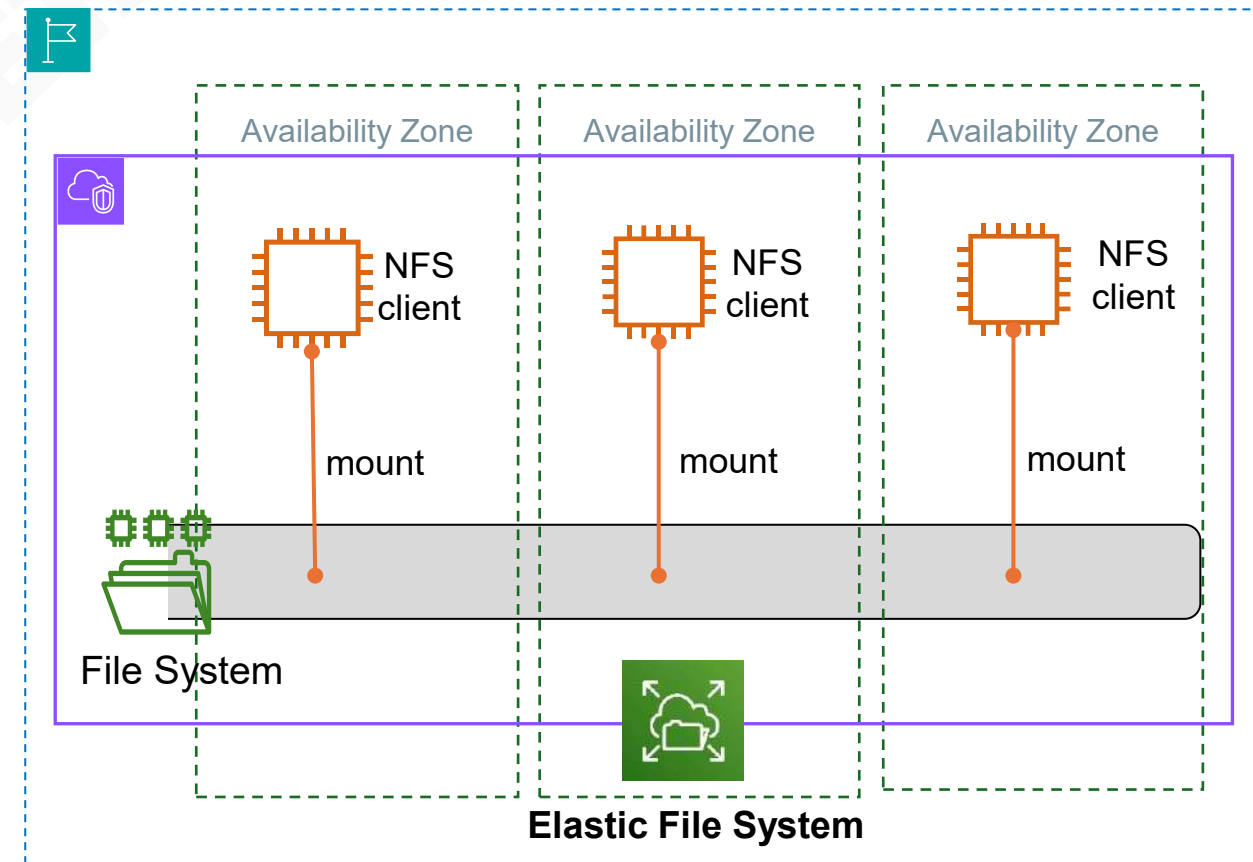


Fargate



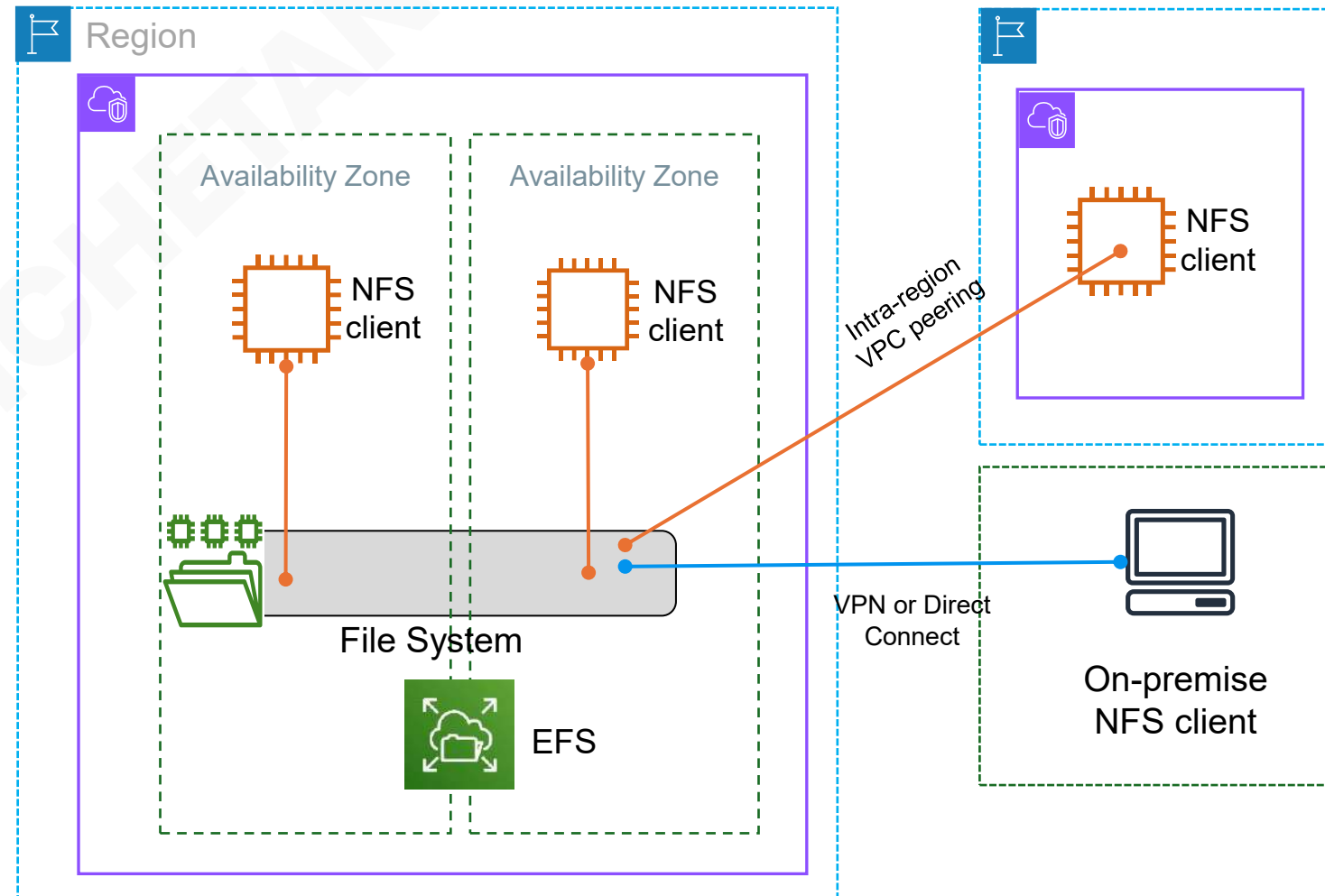
Lambda

SageMaker



# Amazon Elastic File System (EFS)

- EFS filesystem can be accessed by instances in a different VPC within or across AWS regions over VPC peering connection and from On-premises host if it's connected to the VPC over VPN or Direct connect

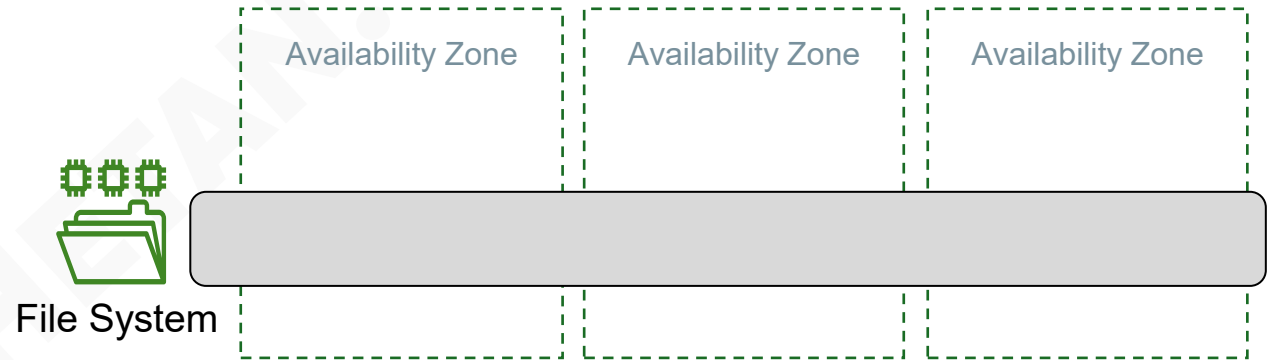




# EFS Filesystem storage options – Regional vs OneZone

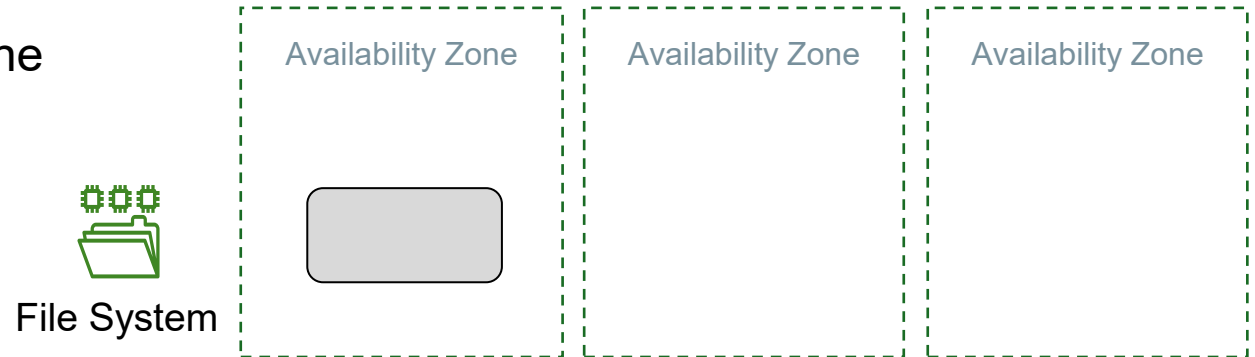
## Standard Storage (Regional)

- Stores data redundantly across multiple Availability Zones in the region
- Provides 11 9's of durability



## OneZone Storage

- Stores data within in a single Availability Zone
- Ideal for non-critical / reproducible data
- 50% cheaper than standard regional store filesystem



# EFS Storage Classes – Based on access pattern

## EFS Standard

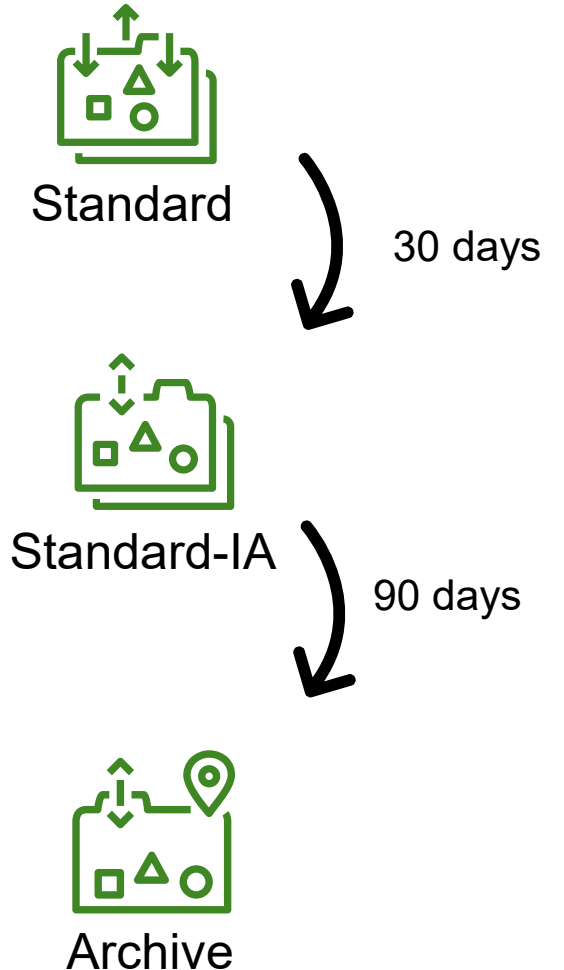
- SSD storage designed to deliver sub-millisecond latency for active data

## EFS Standard–Infrequent Access (EFS–IA)

- Cost-optimized for data accessed only a few times a quarter which doesn't need the sub-millisecond latencies of EFS Standard.
- EFS Lifecycle policy “**Transition into IA**” automatically moves the files from Standard storage to Standard-IA if files are not accessed for 30 days

## EFS Archive

- Cost-optimized for long-lived data accessed a few times a year or less and offering similar performance to EFS IA
- EFS Lifecycle policy “**TransitionToArchive**” automatically moves the files to EFS archive storage if files are not accessed for 90 days
- EFS Lifecycle policy “**Transition to Standard**” is set to None by default.

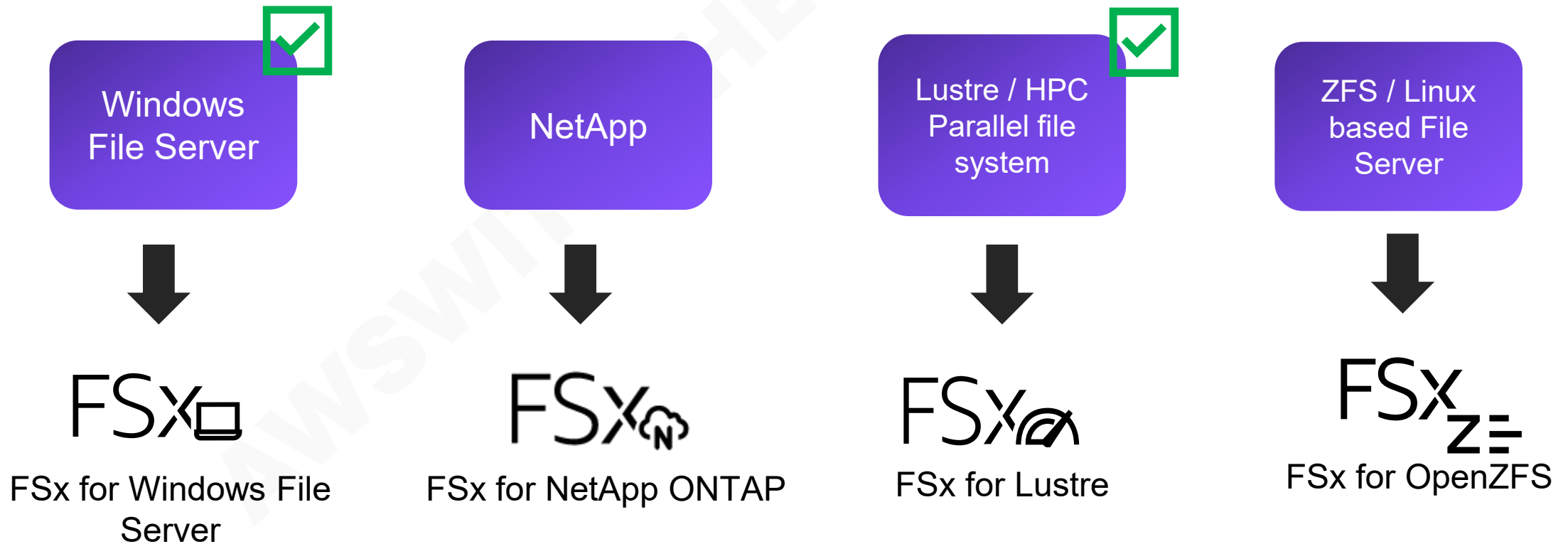




# Amazon FSx

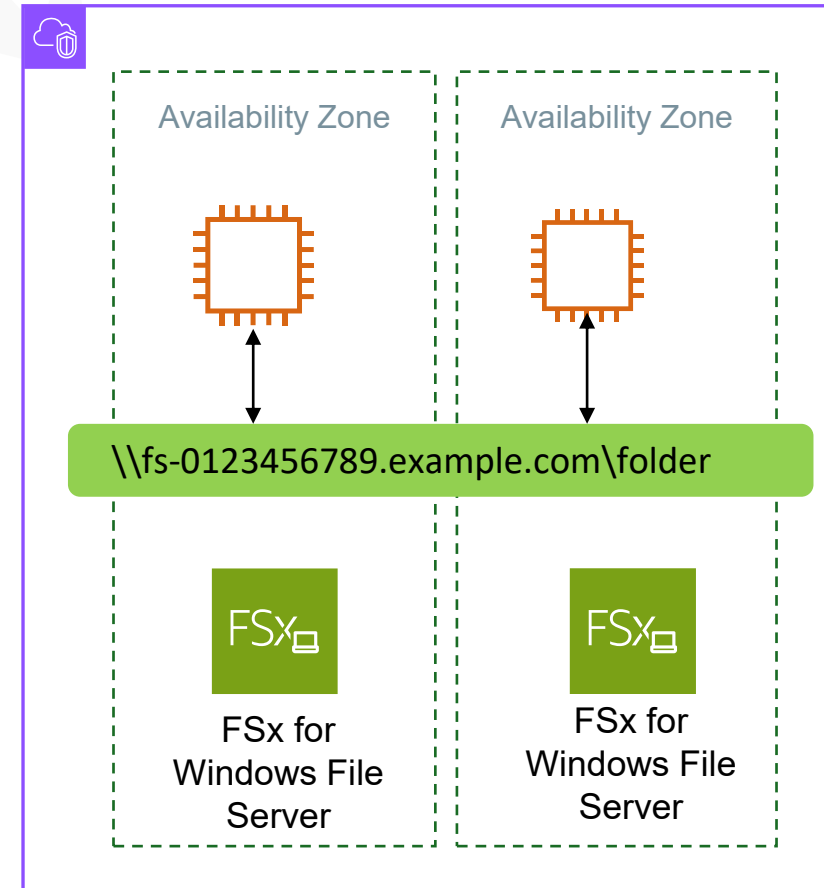
# Amazon FSx

- Like-to-Like 3<sup>rd</sup> party fully managed file systems on AWS



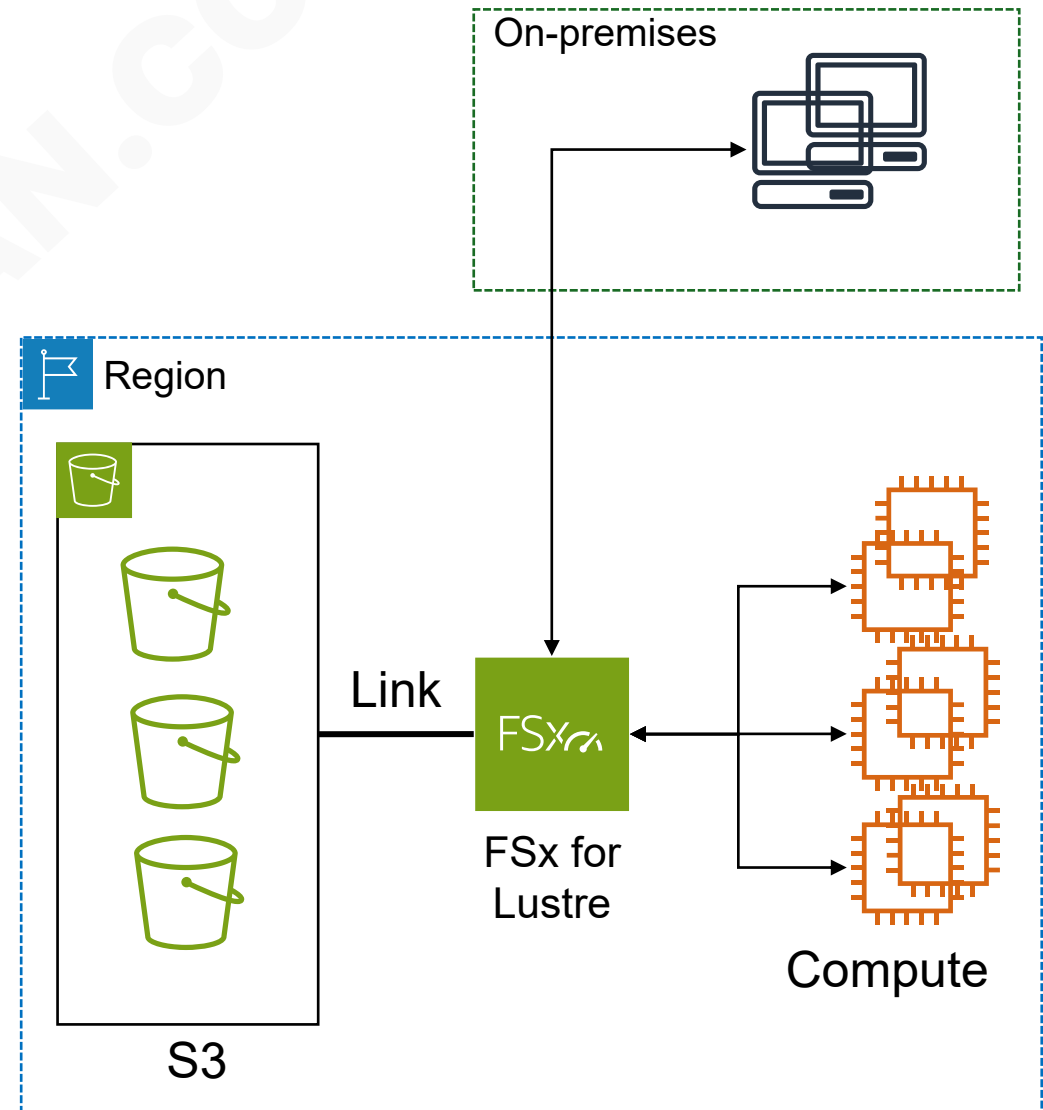
# FSx for Windows File Server

- A fully managed, reliable and scalable **Windows shared file system**
- Uses Windows File Server
- Supports Windows **NTFS** and **SMB** protocols
- Accessible across VPCs, across AWS regions and accounts
- Accessible from on-premises servers
- Integration with Active Directory



# FSx for Lustre

- High performance File System for HPC workloads shared by thousands of compute machines
- Data stored in Amazon S3 is loaded to FSx for processing
- Output of data processing is sent to Amazon S3 for retention
- **Use cases:**
  - Access ML training data in S3
  - Computational Fluid Dynamics (CFD) simulations
  - VFX, Rendering, Transcoding



# AWS Storage services comparison - Cost

## S3



## EBS



## EFS

- Pay as you go.
- No need to pre-provision capacity.
- Per GB per month (e.g. \$0.023 per GB) + \$0.005 per 1000 requests for S3 standard

- Pay for provisioned capacity
- Pay even if not used entire space
- Per GB per month (e.g. \$0.1 per GB)

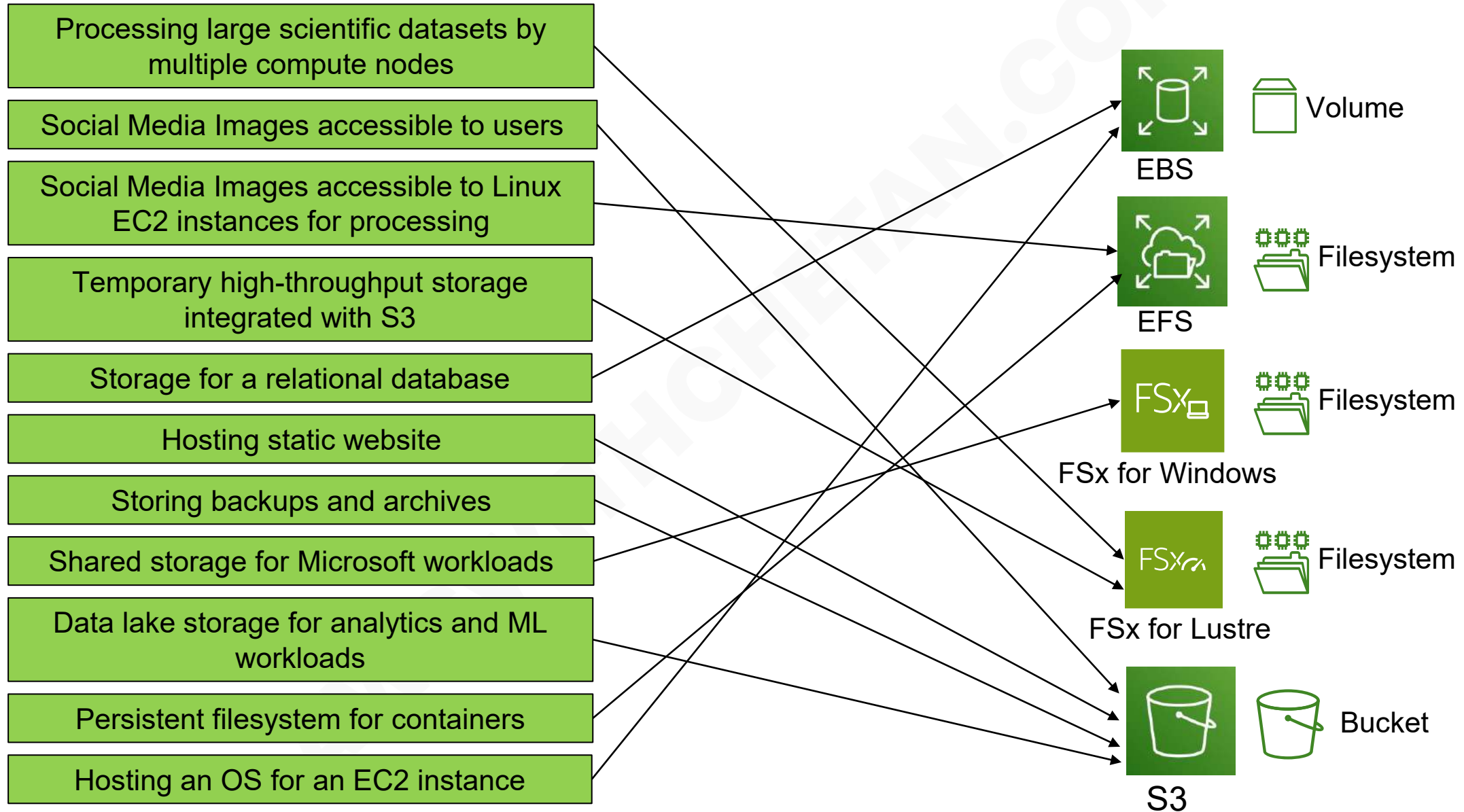
- Pay as you go.
- No need to pre-provision storage
- Per GB month (e.g. \$0.3 per GB)



Filesystem

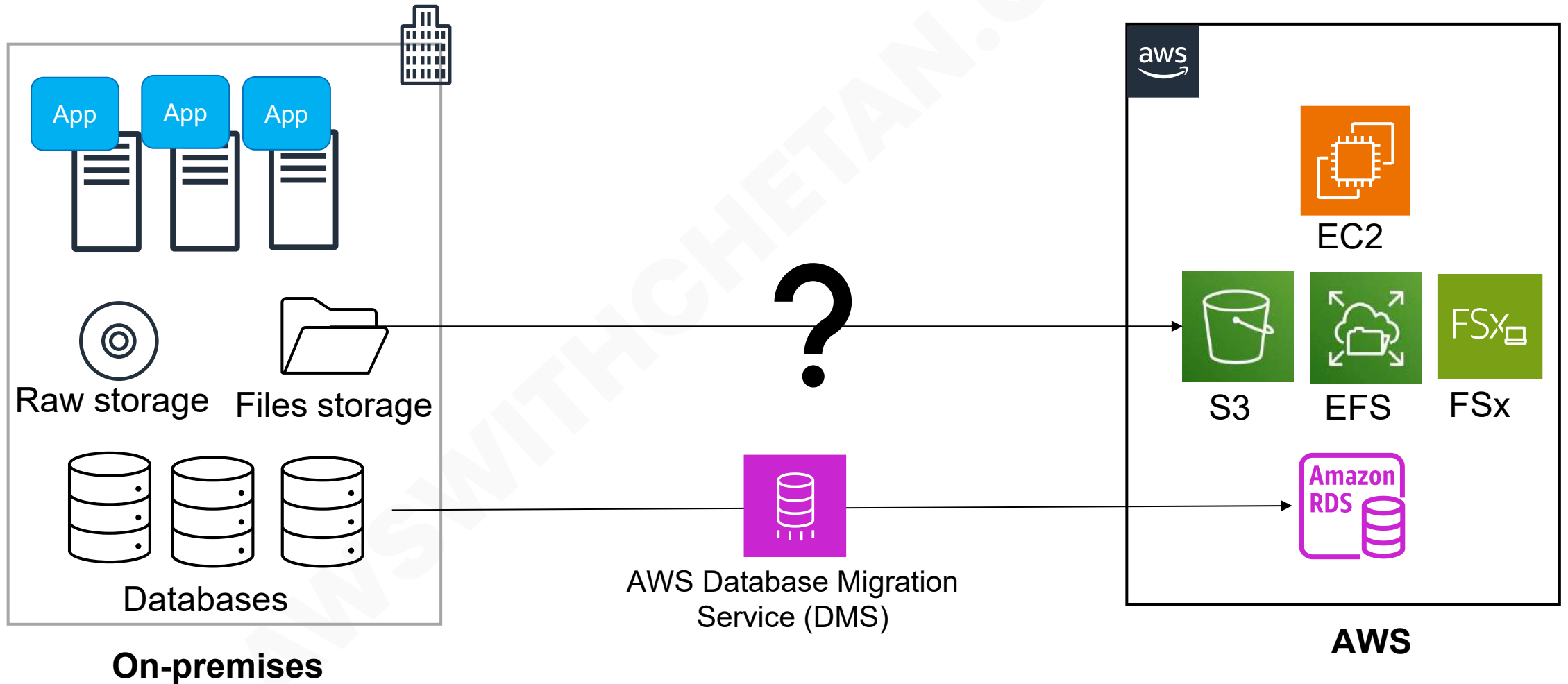
\*Prices in N. Virginia region at the time of recording this video

# AWS Storage services – match the pairs !





# Migrating data to AWS



# Migrating data to AWS



# How much time does it take to move data online?

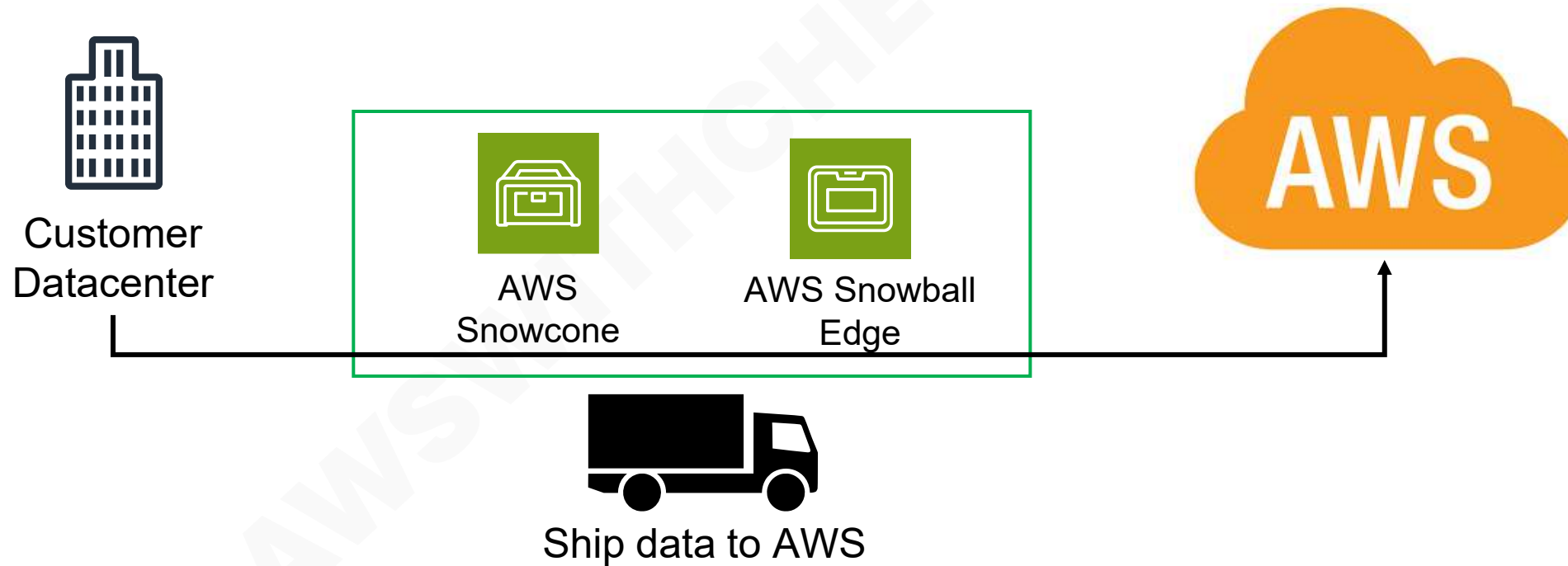
Bandwidth ->

Data size ->

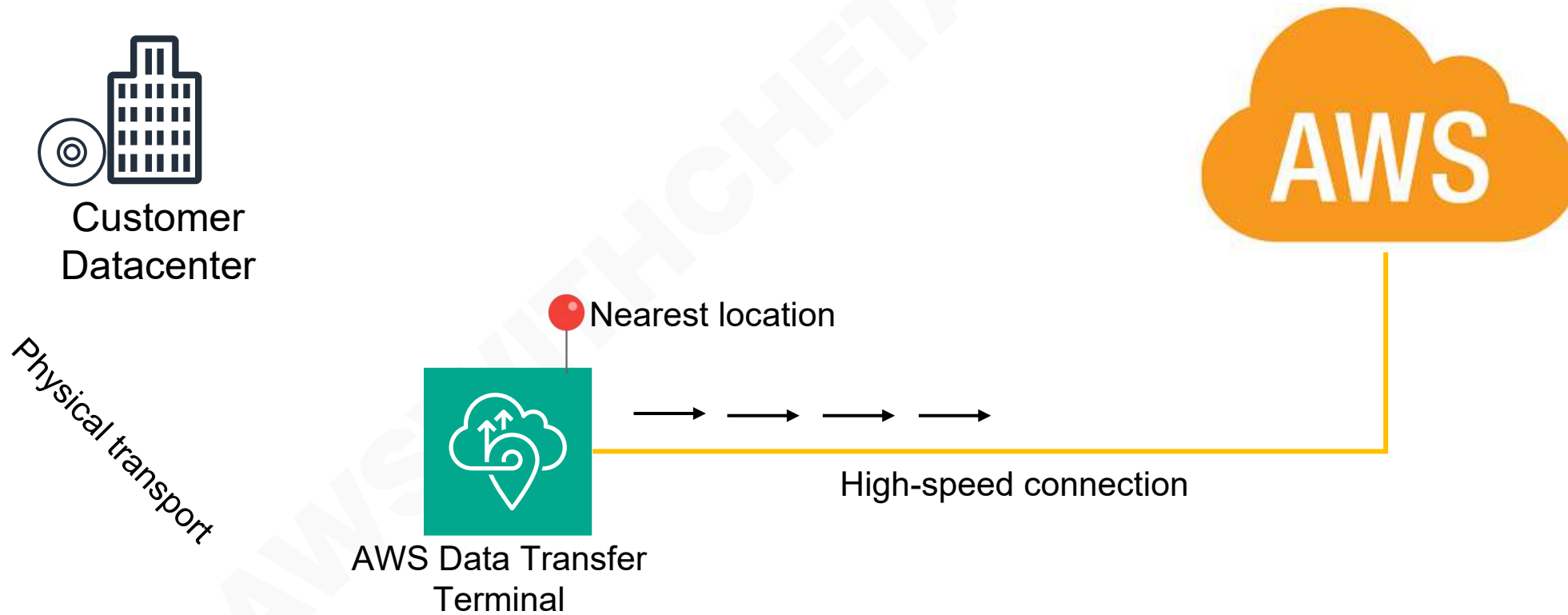
|        | 1 Gbps  | 2 Gbps  | 5 Gbps   | 10 Gbps  |
|--------|---------|---------|----------|----------|
| 500 TB | 58 days | 29 days | 12 days  | 6 days   |
| 5 PB   | 2 years | 1 year  | 116 days | 58 days  |
| 10 PB  | 4 years | 2 years | 232 days | 116 days |

# Offline data transfer with AWS Snow devices

Service Discontinued



# Offline data transfer with AWS Data Transfer Terminal



# AWS Snowcone

Small, portable, rugged, and secure edge computing and data transfer device

Service Discontinued

- Encryption, tamper-evident
- 4.5 pounds (2.1 kg)
- Portable
- Withstands harsh environments
- 8 TB HDD or 14 TB SSD storage
- 2 CPUs, 4 GB of memory
- Wi-Fi or Ethernet data copy
- AWS DataSync agent pre-installed for online data transfer



## Use cases

Industrial IoT – sensor or machine data in a factory, Content distribution and aggregation, data migration

# AWS Snowball Edge

Snowball device with on-board storage and compute power

Service Discontinued

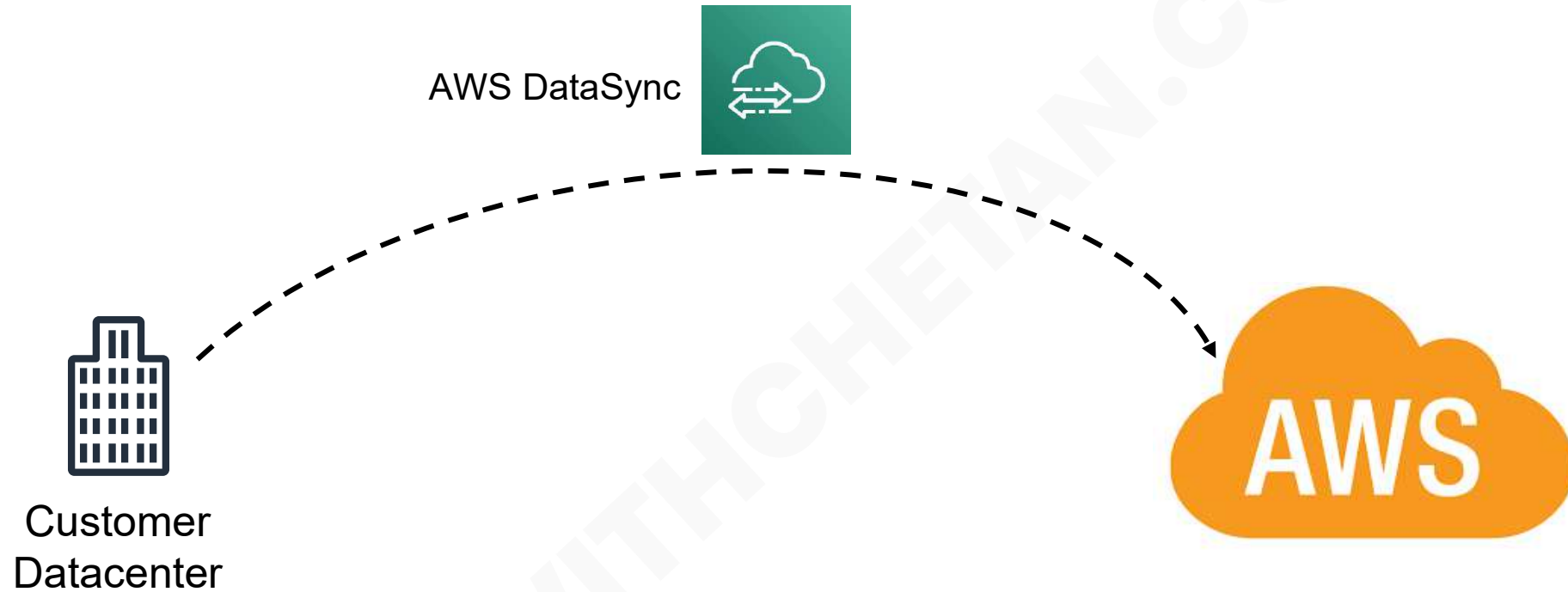
- 49.7 pounds (22.54 Kg)
- 2 x 10 Gbit, 1 x 40 Gbit, 1 x 100 G-bit Network Interfaces
- Device Options - Storage optimized / Compute optimized
- Up to 104 vCPU, 416GB RAM
- Up to 80TB HDD and 210TB SSD

## Use cases

- Storage optimized – Large scale data migration
- Compute optimized – Edge (on-premises) data processing, machine learning, full motion video analytics,

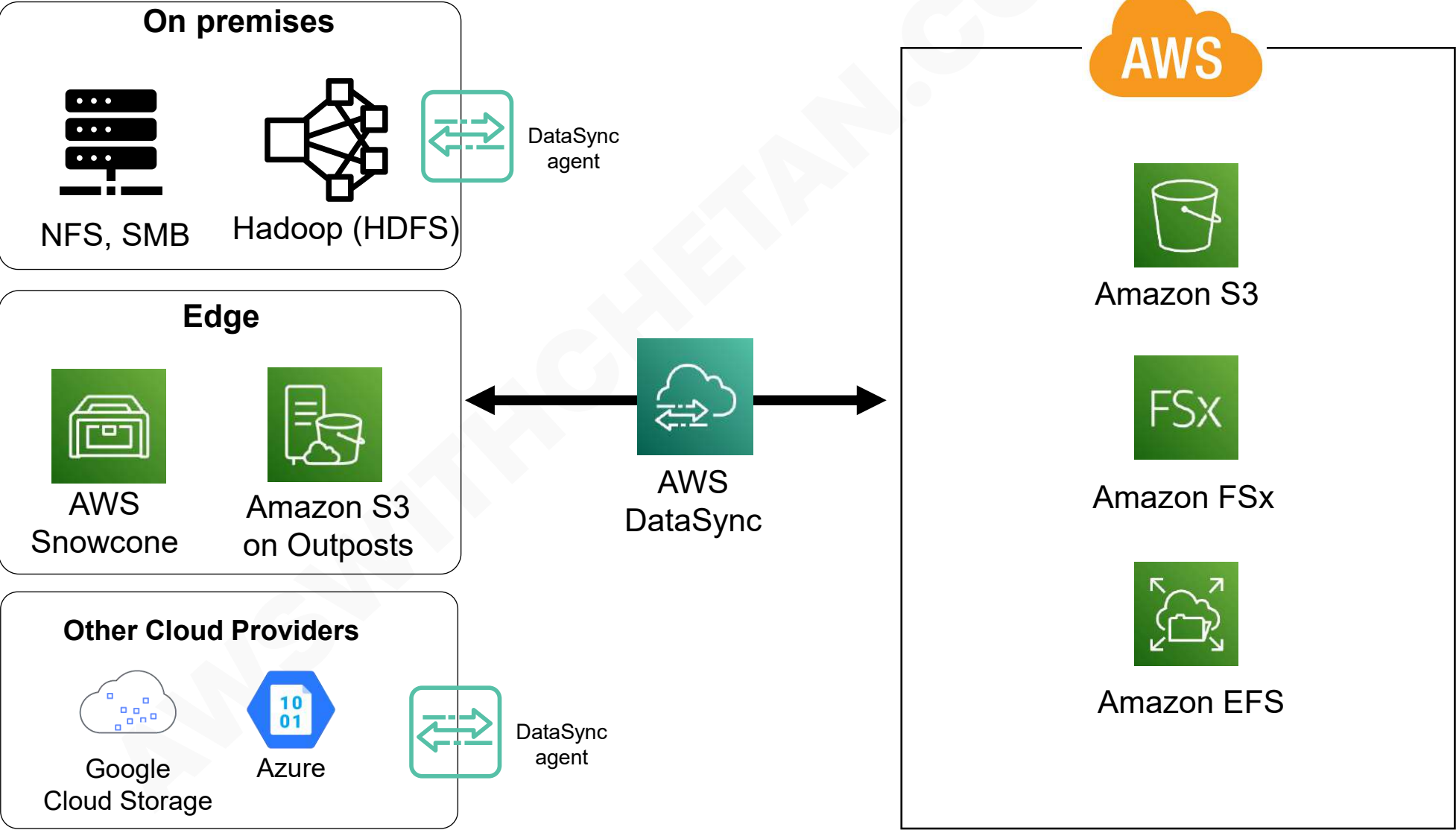


# Online data transfer with AWS DataSync





# AWS DataSync



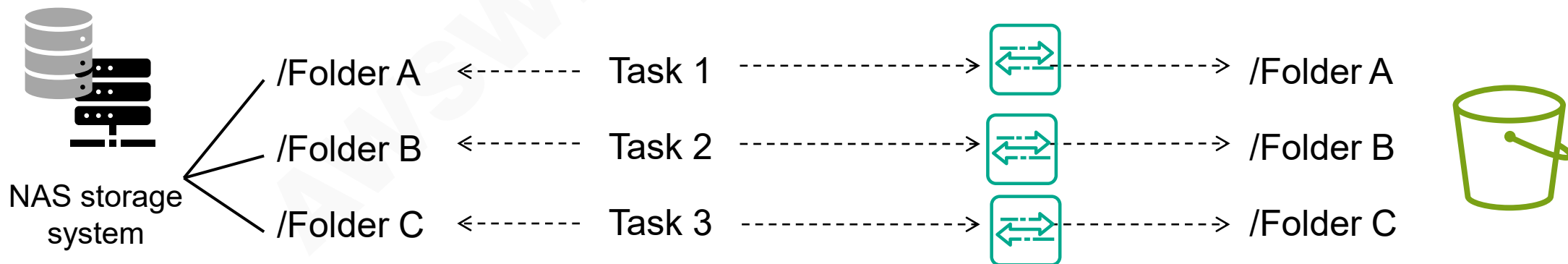
# AWS DataSync

- Moves data between on premises and AWS Storage services
- Supports NFS, SMB, HDFS storage and AWS storage services such as S3, EFS and FSx.
- Data transfer tasks can be scheduled – Hourly, daily, weekly etc.
- You can set limit on the bandwidth usage by DataSync. Single task can utilize up to 10 Gbps.
- Data is copied incrementally and it handles failure during the transmission.
- Can run multiple tasks in parallel



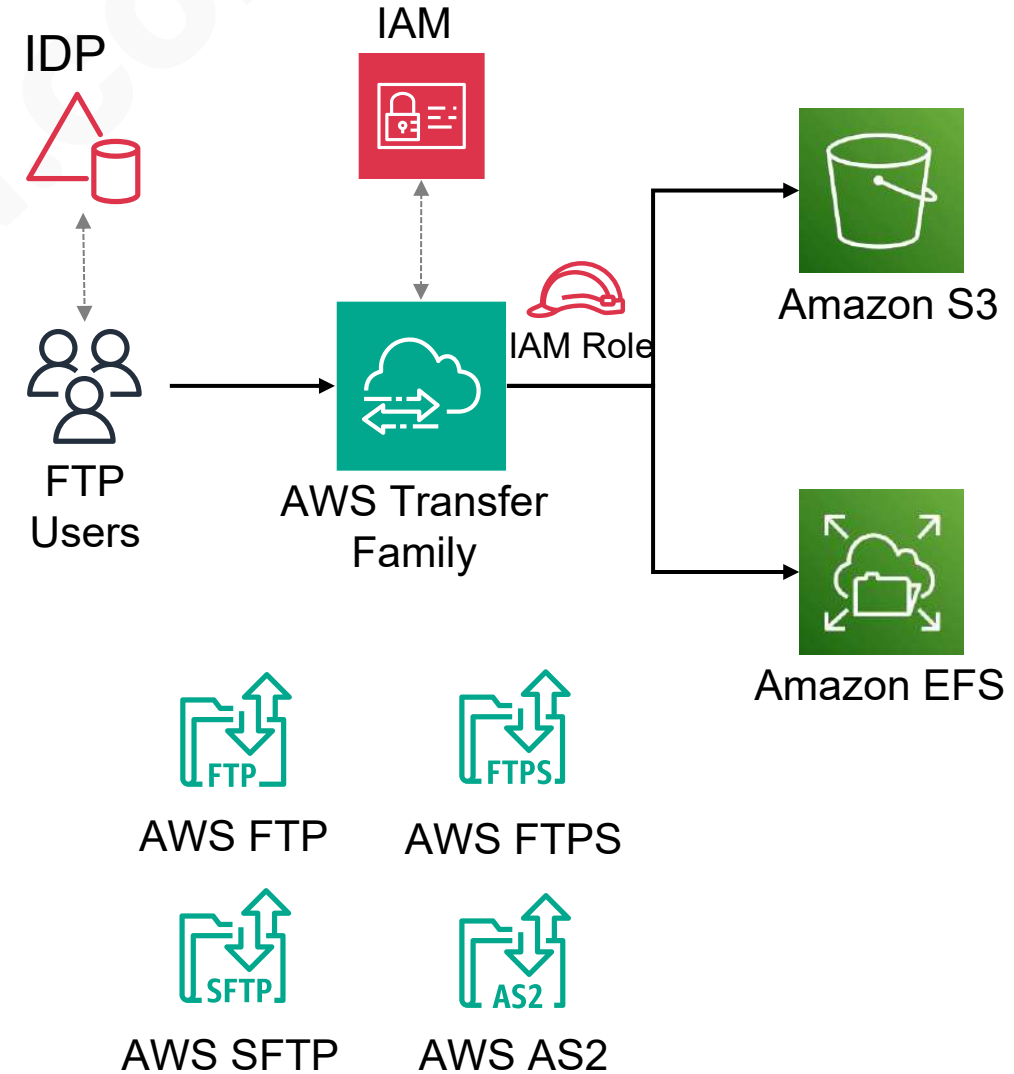
AWS DataSync

## Supports TLS encryption



# AWS Transfer Family

- A fully managed service for **file transfer** into and out of AWS over **SFTP, FTPS, and FTP**.
- Customers can retain existing file-transfer workflows by integrating their application with AWS Transfer family
- Integrates with Amazon S3 and Amazon EFS for storage
- Supported protocols for the file transfer:
  - File Transfer Protocol (FTP)
  - File Transfer Protocol over SSL (FTPS)
  - Secure File Transfer Protocol (SFTP)
  - Application Statement 2 (AS2)
- Supports user authentication with IAM, Active Directory, LDAP and other 3<sup>rd</sup>-party identity providers.
- Use cases:
  - Secure B2B file exchange with partners
  - EDI documents transfer
  - Replacing on-premises FTP/SFTP servers & integrating legacy apps
  - Allowing vendors to upload/download files from/to S3 or EFS



# AWS DataSync vs AWS Transfer Family

## AWS DataSync



- Used for bulk data transfer, data synchronization.
- Common for on-premises to AWS or AWS to AWS transfers for large-scale migrations.
- Supports NFS and SMB protocols
- Support Amazon S3, EFS, and FSx as storage.
- Typically, one-time or scheduled, not for user-driven applications and uploads.
- Not used by end users, hence user access is not a concern

## AWS Transfer Family



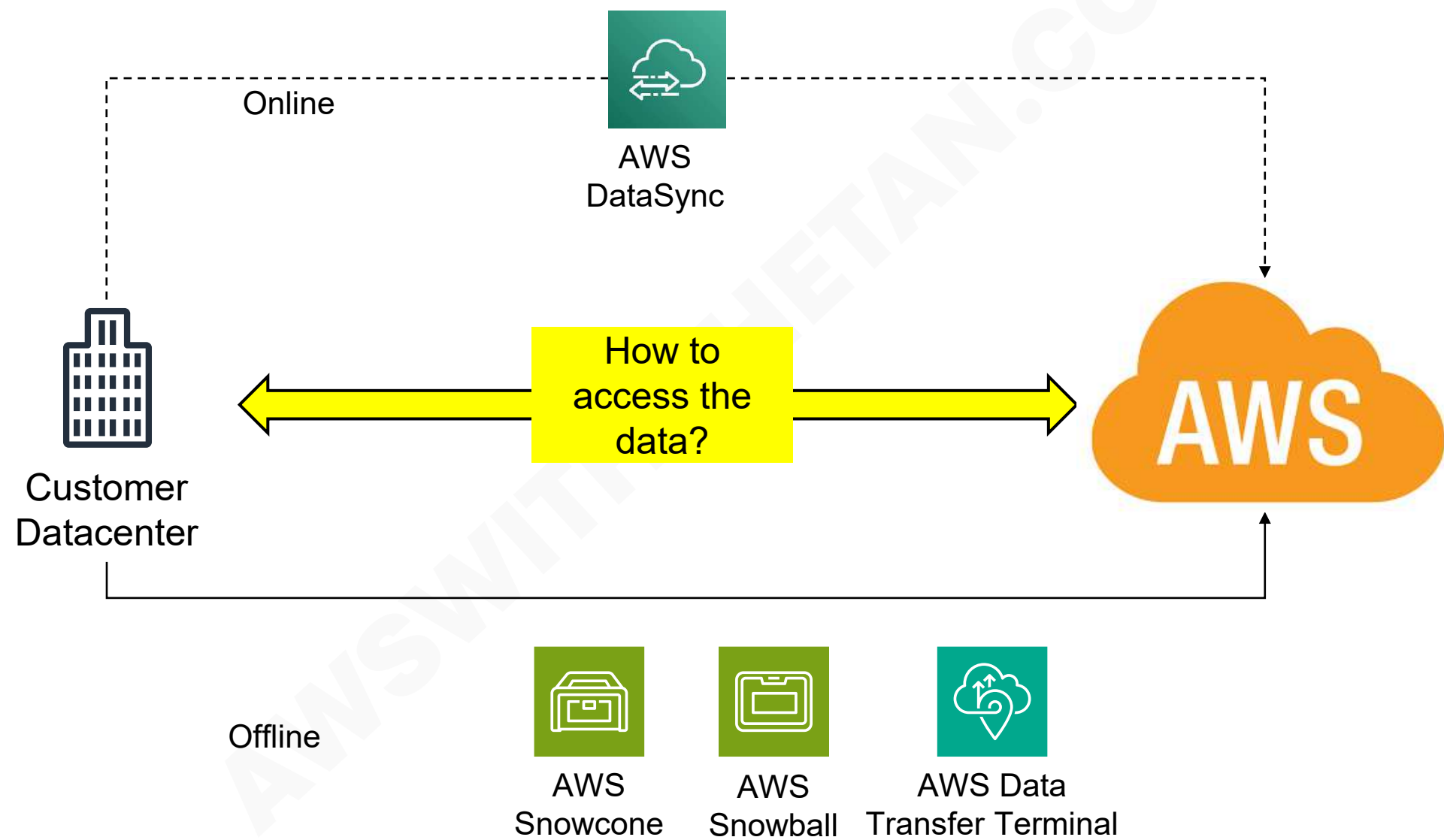
- Used for interactive file transfers and designed for end users, partners, and legacy systems.
- Replaces traditional FTP/SFTP servers.
- Supports file transfer protocols SFTP, FTPS, FTP, or AS2.
- Provides managed endpoints backed by S3 or EFS.
- User authentication and access control are key features.



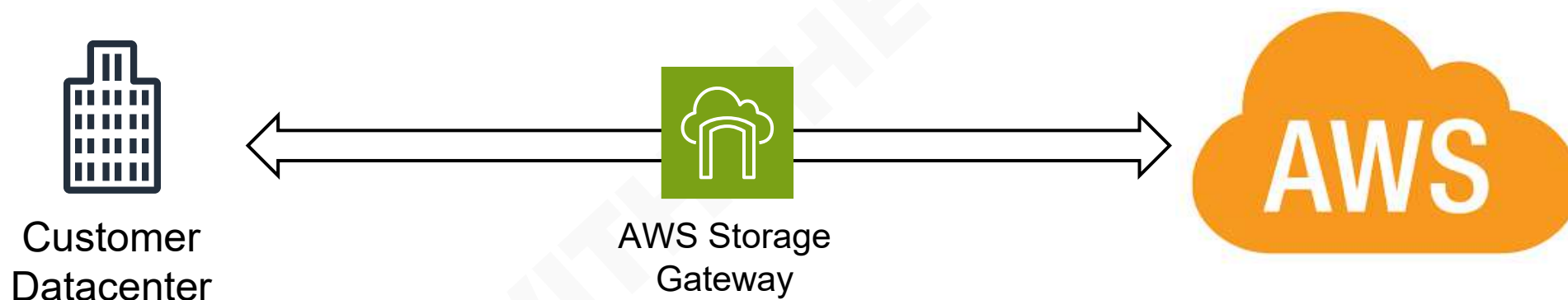
# AWS Storage Gateway

Hybrid storage

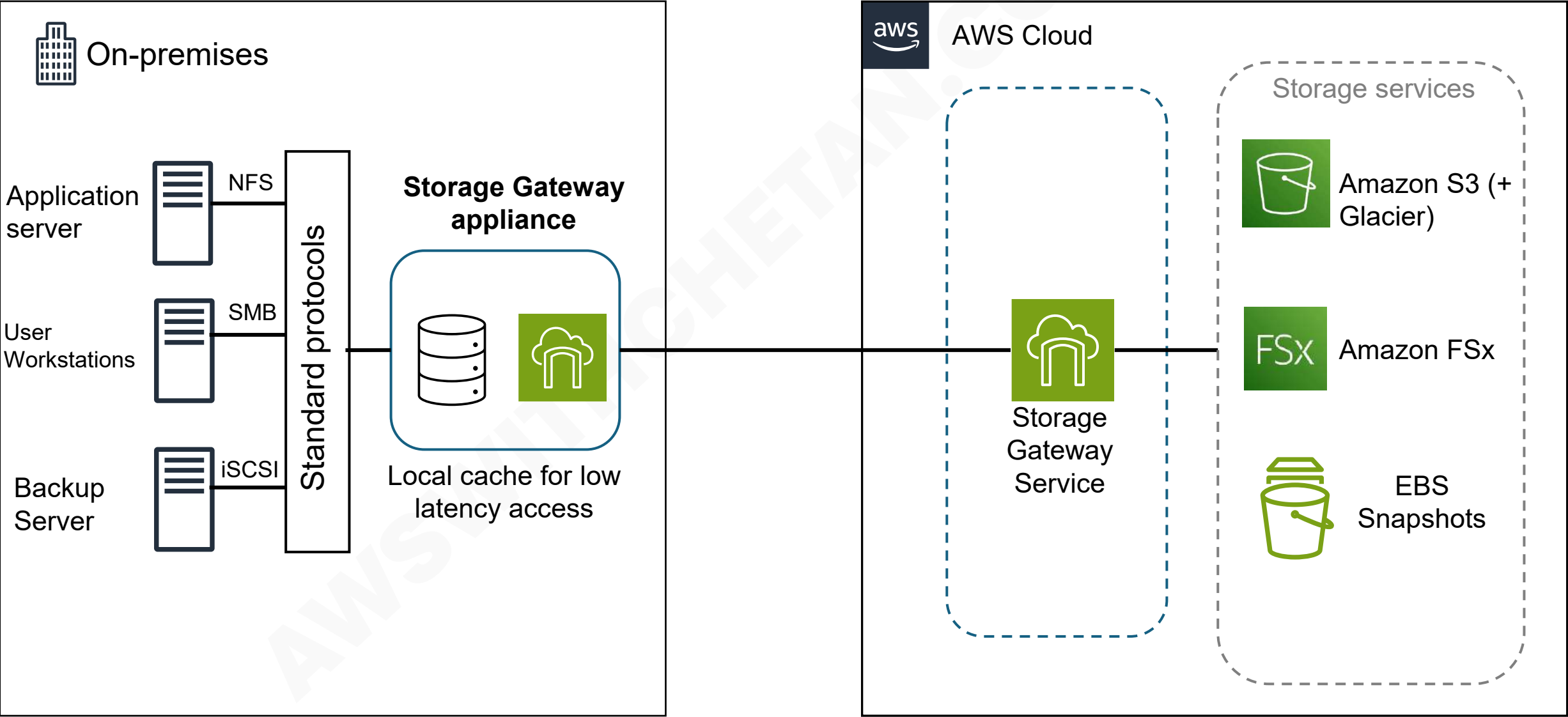
# Access data in AWS from on-premises



# Access data in AWS from on-premises



# Access AWS storage from on-premises





# AWS Storage Gateway

- AWS Storage Gateway connects on-premises applications to AWS cloud storage services
- AWS Storage gateway appliance can be deployed on local Virtual Machines or on EC2 instance in AWS
- Supports storage protocols like NFS, SMB, iSCSI
- There are following different storage gateway types:
  - Amazon S3 File Gateway - Access data in S3 (e.g. logs, images, videos etc.)
  - Amazon FSx File Gateway – Access data in FSx for Windows Server (File access)
  - Volume Gateway – Provides cloud-backed storage volumes as iSCSI devices
    - Cached Volumes – Caches frequently accessed data locally and store rest of the data in S3
    - Stored Volumes – Low latency access to full data and asynchronous backup to S3
  - Tape Gateway - Replace physical tapes with virtual tapes in AWS S3/Glacier (Tape backup)



S3 File Gateway



FSx File Gateway



Volume Gateway



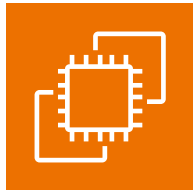
Tape Gateway

# AWS Backup

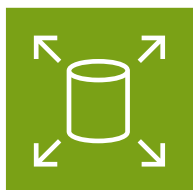
- Centralizes and automates backups for data across AWS services and on-premises.
- Uses **Backup plans** to configure Frequency of the backup, backup time window, AWS resources to backup, storage tier for backup etc.
- Supports lifecycle management policies to move backups into low cost (cost) storage
- Supports Cross-region backups where backups are stored in another region to recover from region level failure
- Supports Cross-account backups
- Backups are incremental (only first backup is a fully copy) for the supported resources
- Integrates with most of the AWS services and external applications such as:



AWS Backup  
Service



EC2



EBS



S3



DynamoDB



RDS



EFS



FSx



Storage Gateway



On EC2

and more..

Let's talk architecture

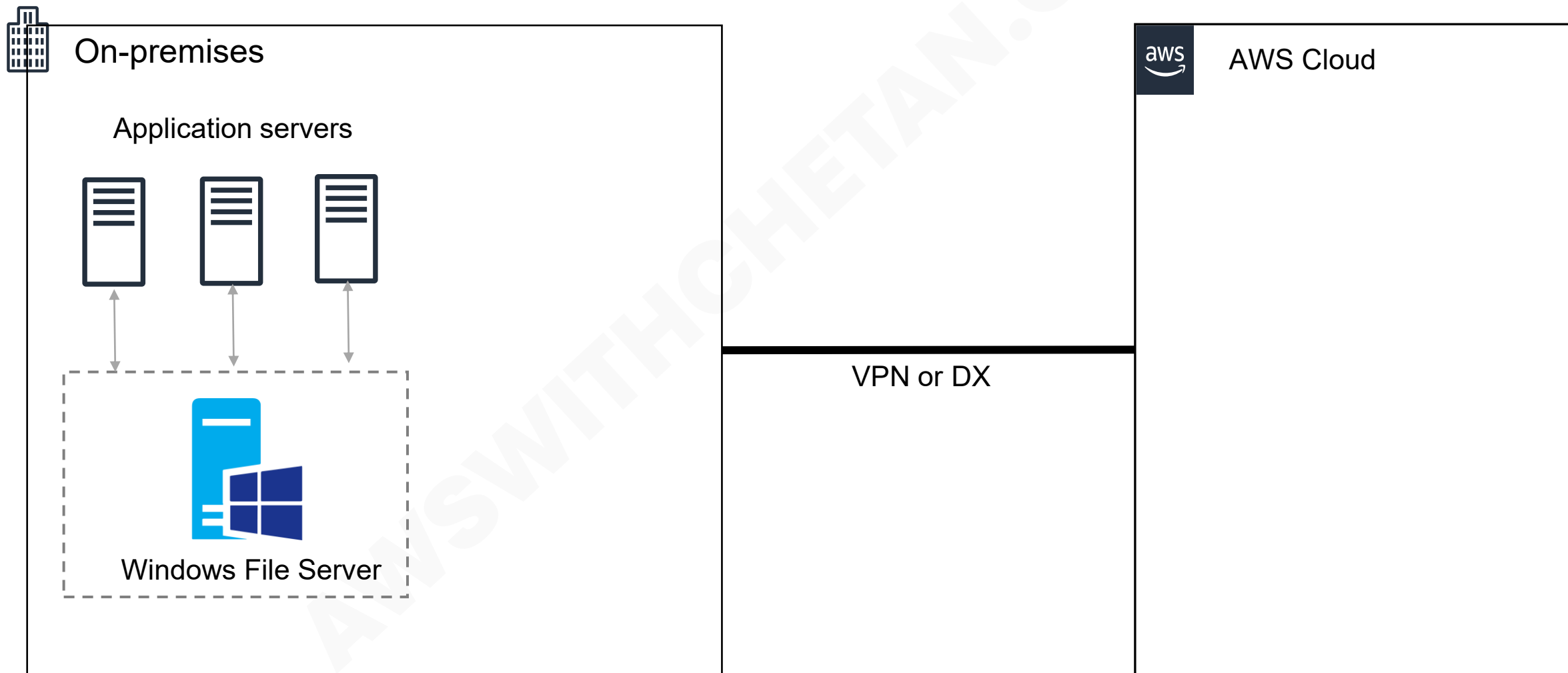
# Scenario and customer requirements

1. A company has TBs of file data hosted on Windows file servers running on premises. This data is actively used every day by users and applications.
2. The company is in the process of migrating its Windows workloads to AWS.
3. Migration will happen gradually.
4. During this transition, users and applications must be able to access both on-premises and AWS-based file storage with minimum latency.
5. The company has private network connectivity to AWS.

## **The solution must:**

- Provide low-latency file access during the migration phase
- Preserve existing file access patterns so users do not need to change how they work
- Minimize operational overhead, avoiding self-managed synchronization or complex tooling

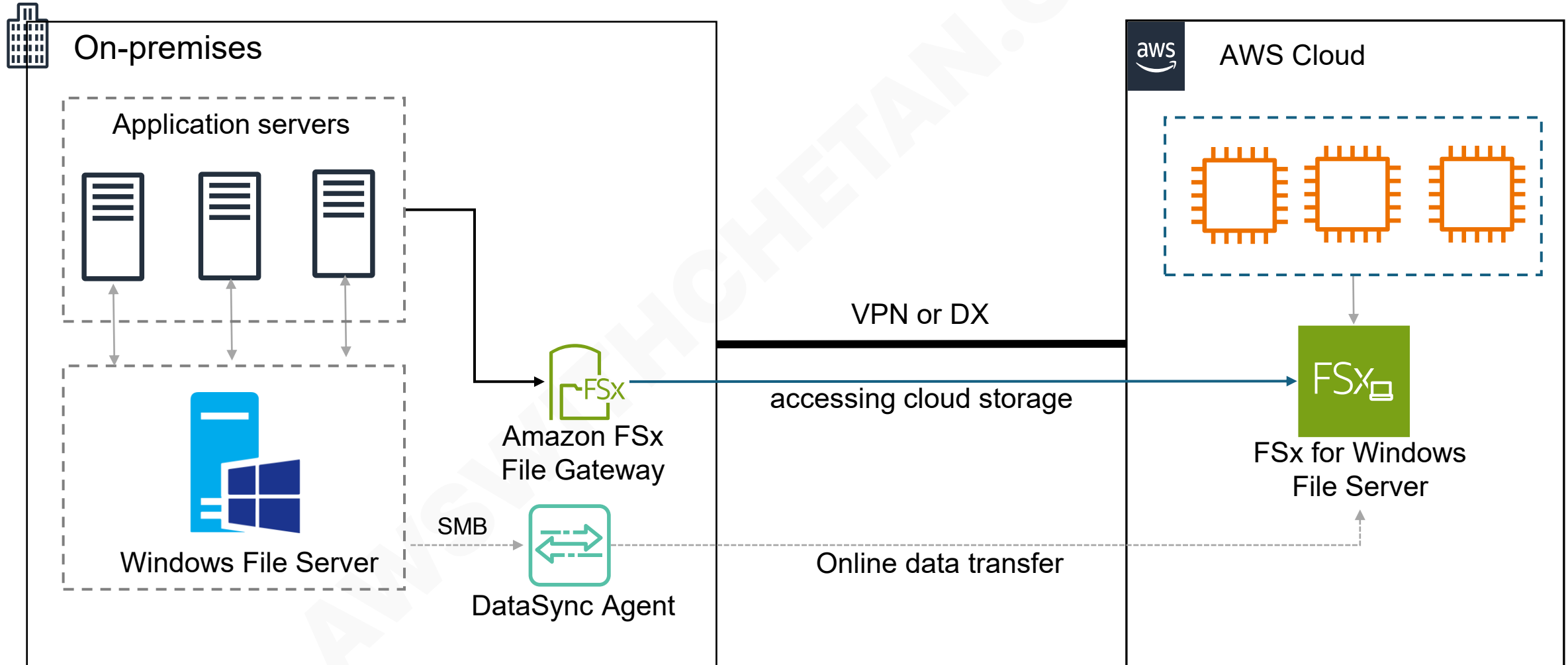
# Current architecture



# Solution Design

1. Deploy Amazon FSx for Windows File Server in AWS to provide a highly available, durable, and managed file system for cloud workloads.
2. Deploy Amazon FSx File Gateway in the on-premises environment to expose the same file system locally with low-latency access.
3. Start moving the existing on-premises file data to the FSx File Gateway so it is stored centrally in FSx while remaining accessible on premises. Use AWS DataSync service for moving the data.
4. Configure AWS-based workloads (EC2) to directly access FSx for Windows File Server in AWS.

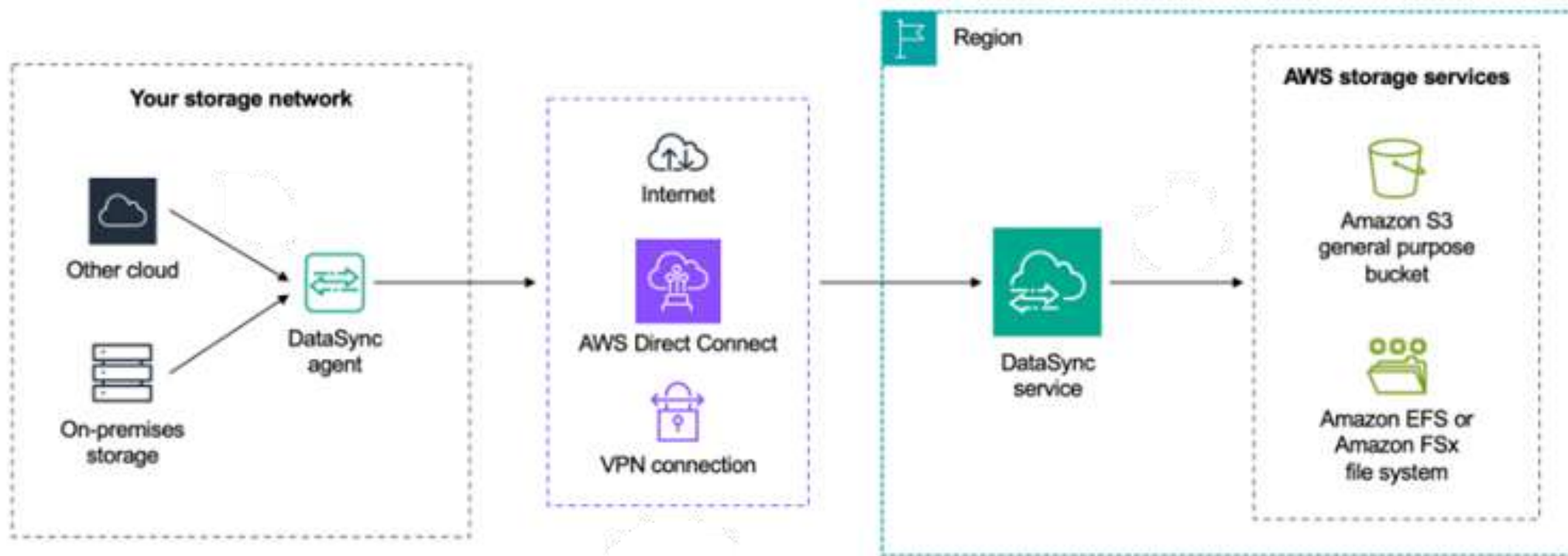
# Solution architecture



Let's talk architecture



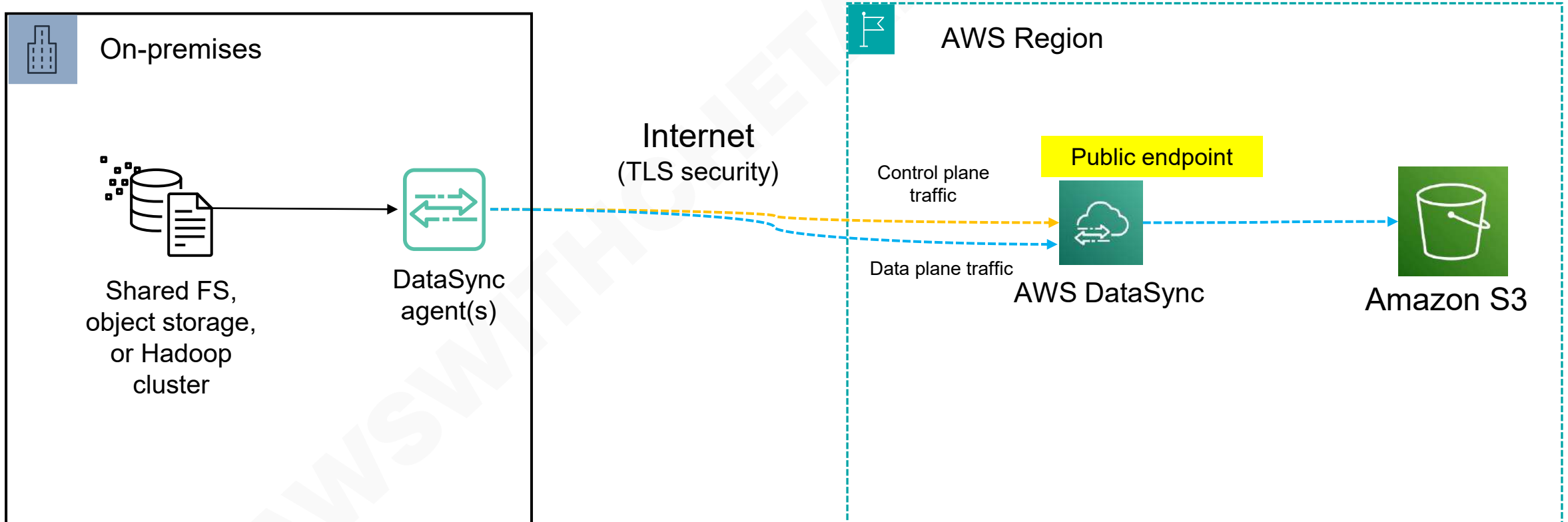
# AWS DataSync – Network connectivity options



Source: <https://docs.aws.amazon.com/datasync/latest/userguide/networking-datasync.html>

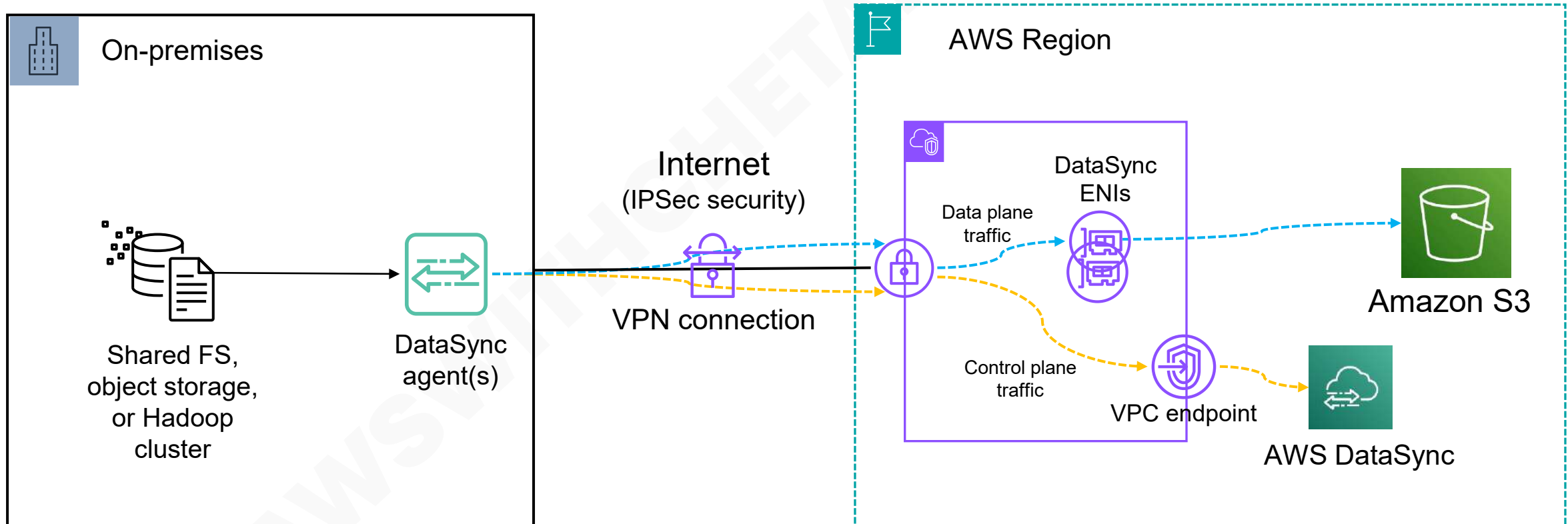
# AWS DataSync – Network connectivity options

## Over the internet



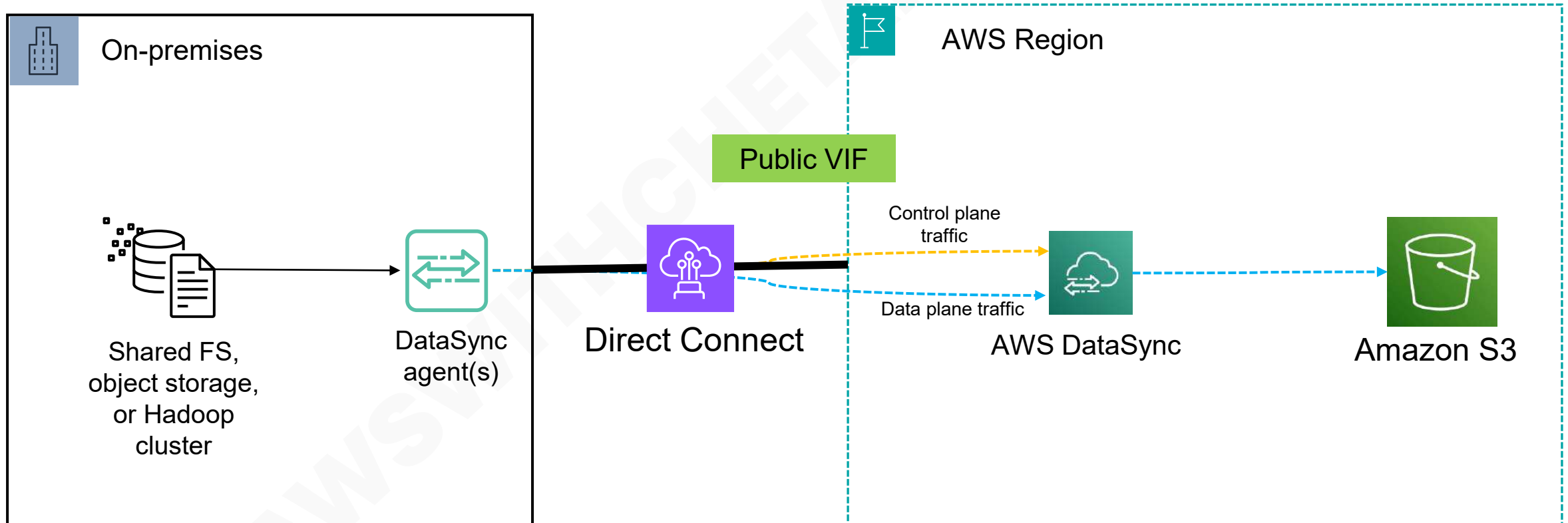
# AWS DataSync – Network connectivity options

## Over the VPN connection



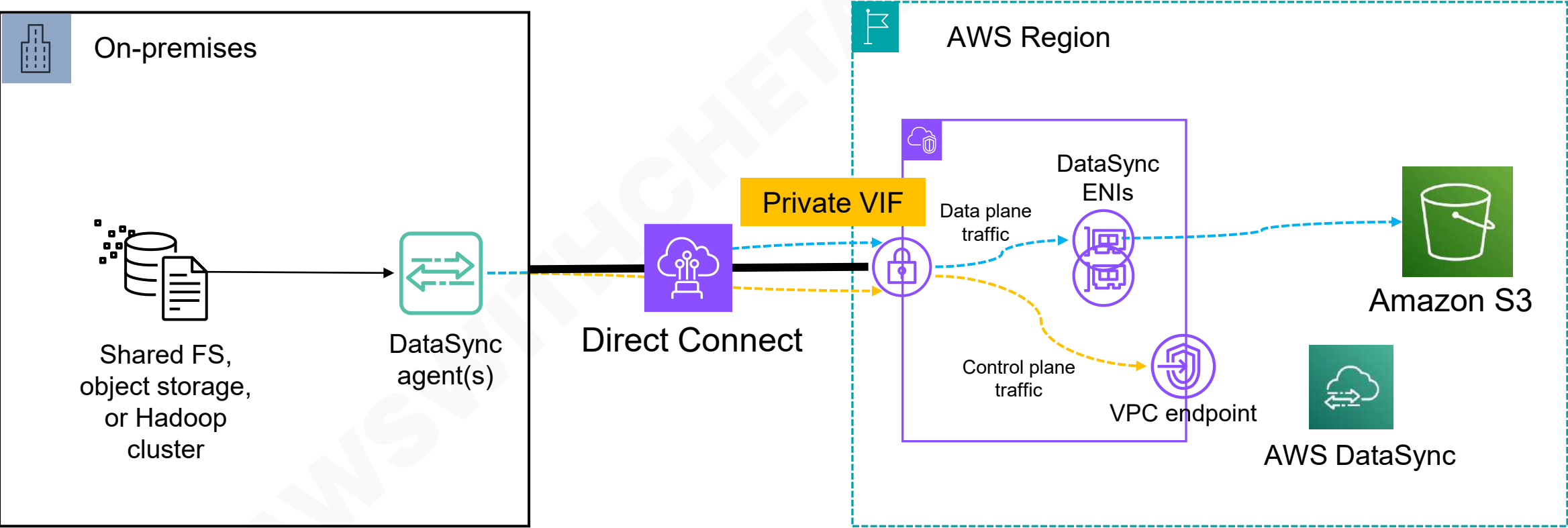
# AWS DataSync – Network connectivity options

## Over the Direct Connect – Public VIF

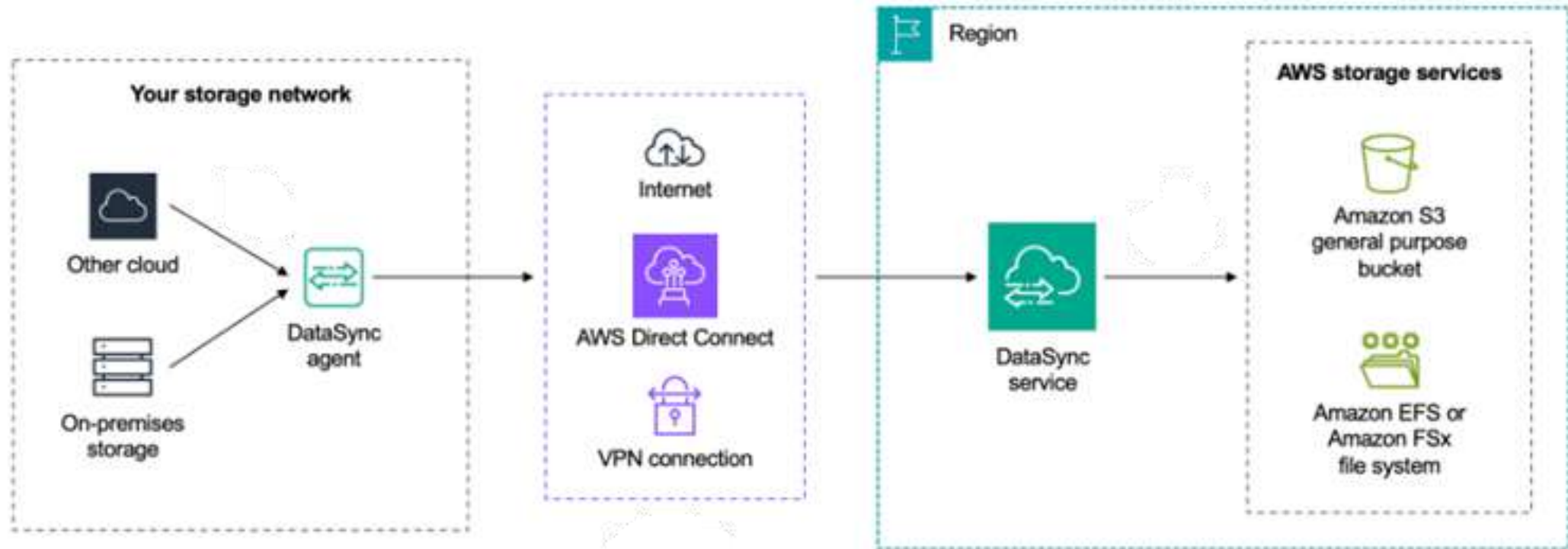


# AWS DataSync – Network connectivity options

Over the Direct Connect – Private VIF or DX gateway



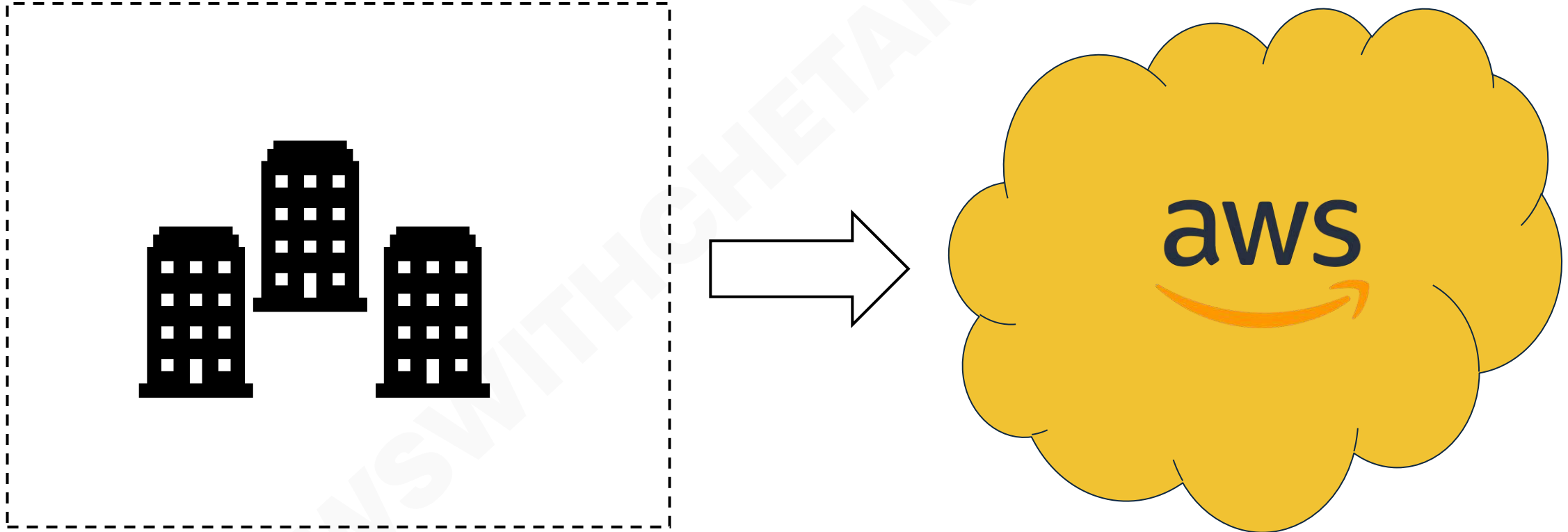
# AWS DataSync – Network connectivity options



Source: <https://docs.aws.amazon.com/datasync/latest/userguide/networking-datasync.html>

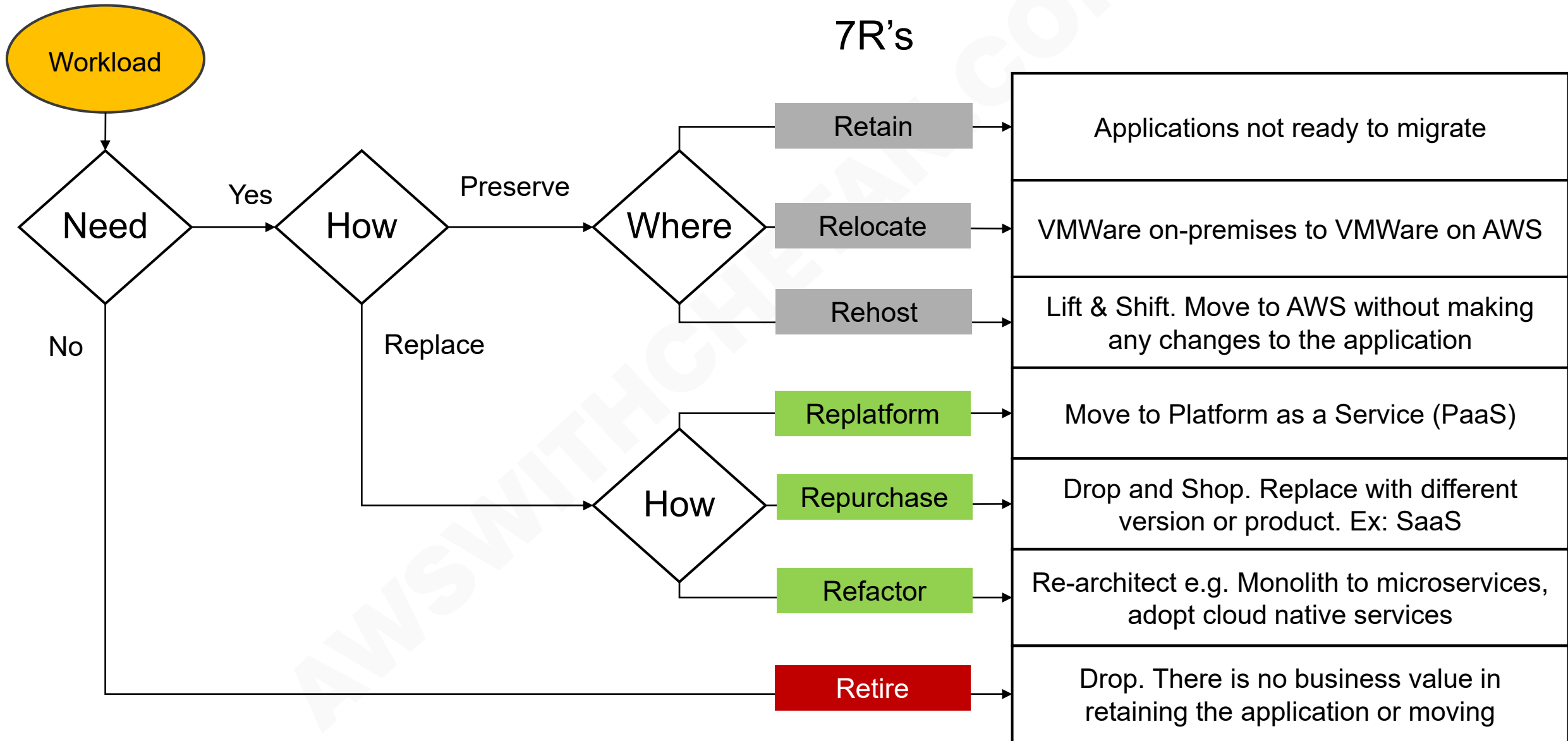
# Migration and Disaster Recovery

# Migration from on-premises to AWS

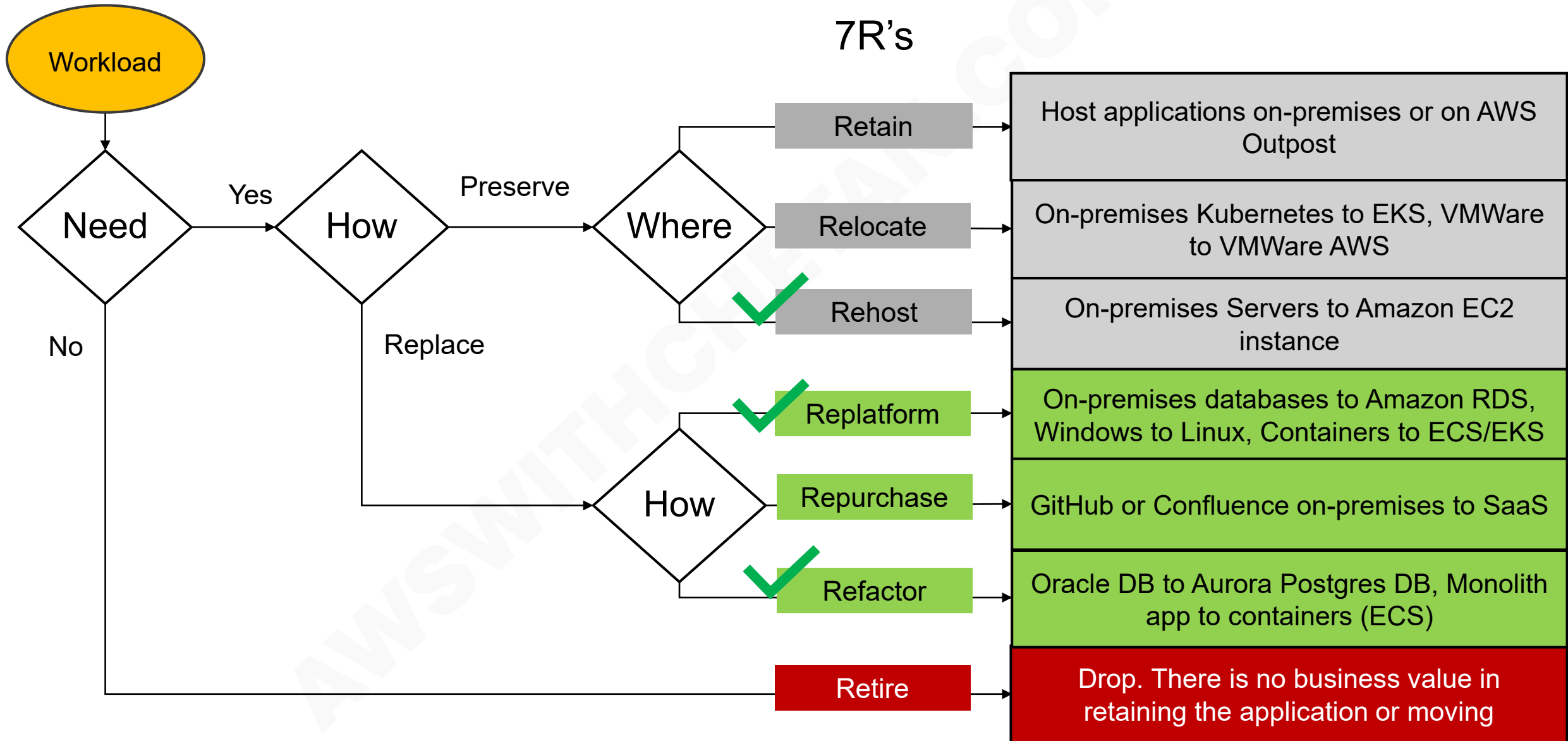




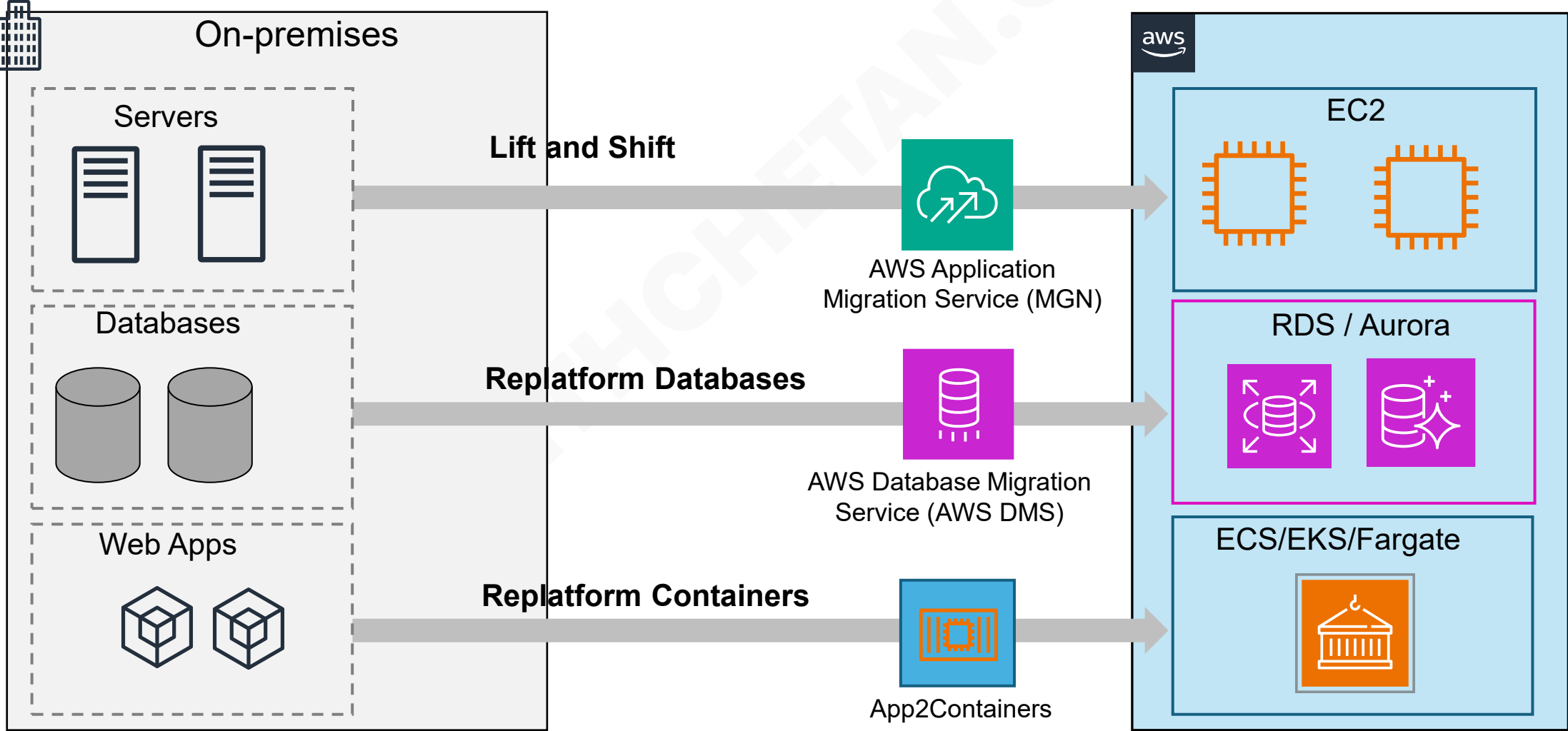
# Migration 7R strategy



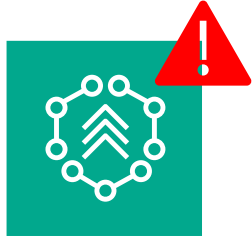
# Migration 7R strategy



# AWS Migration Services



# AWS Migration Services – Updates (Nov 25)



AWS Migration Hub



AWS Application  
Discovery Service (ADS)



App2Containers



AWS Transform

- **Agentic AI-driven modernization**
- Multi-workload support (mainframe, Windows/.NET, VMware, custom)
- Automated code refactoring
- Discovery & assessment with dependency analysis
- Integrated testing and validation
- Generation of modern deployment artifacts
- Pre-built and custom transformation agents

# AWS Migration Services



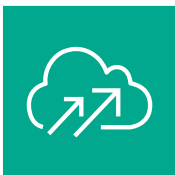
AWS Application  
Discovery Service (ADS)

- Capture system and applications inventory and dependencies
- Agent based and Agentless tools
- Connect and collect details from on-premises IT systems, CMDB etc.



AWS Migration Hub

- Consolidate discovery data across regions
- Visualize discovery data in QuickSight
- Plan for right sized EC2 instances
- Track the entire migration centrally



AWS Application  
Migration Service (MGN)



AWS Database Migration Service  
(AWS DMS)



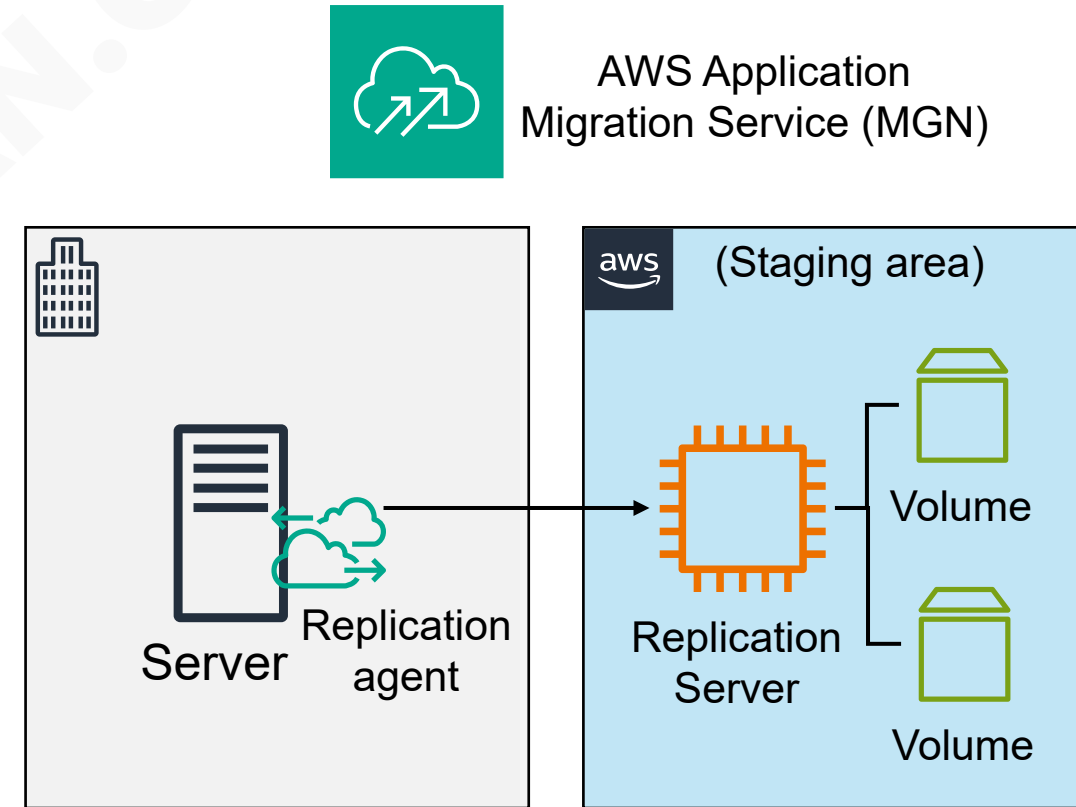
Agentless Collector



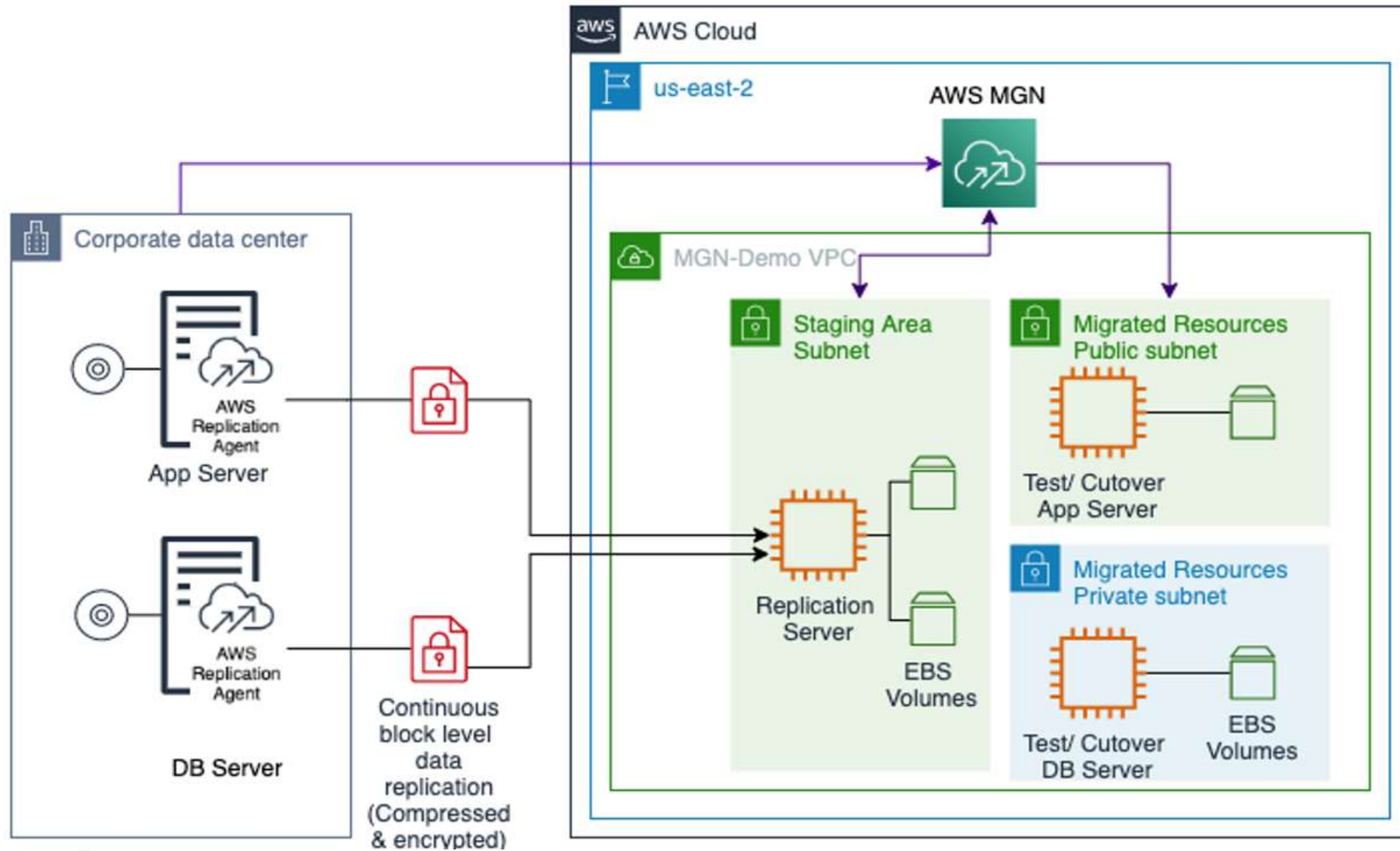
Discovery Agent

# AWS Application Migration Service (MGN)

- Lift-and-shift (rehost) migration service for servers to AWS
- Replication Agent must be installed on each source VM/physical server (on-premises or other clouds)
- Performs continuous block-level replication of server disks
- Replicated data is compressed and encrypted in transit
- Uses an AWS-managed staging area with temporary EC2 + EBS volumes
- Supports Test migrations and Cutover migration launches.
- Maintains near-zero data loss (low RPO) and No downtime during replication phase

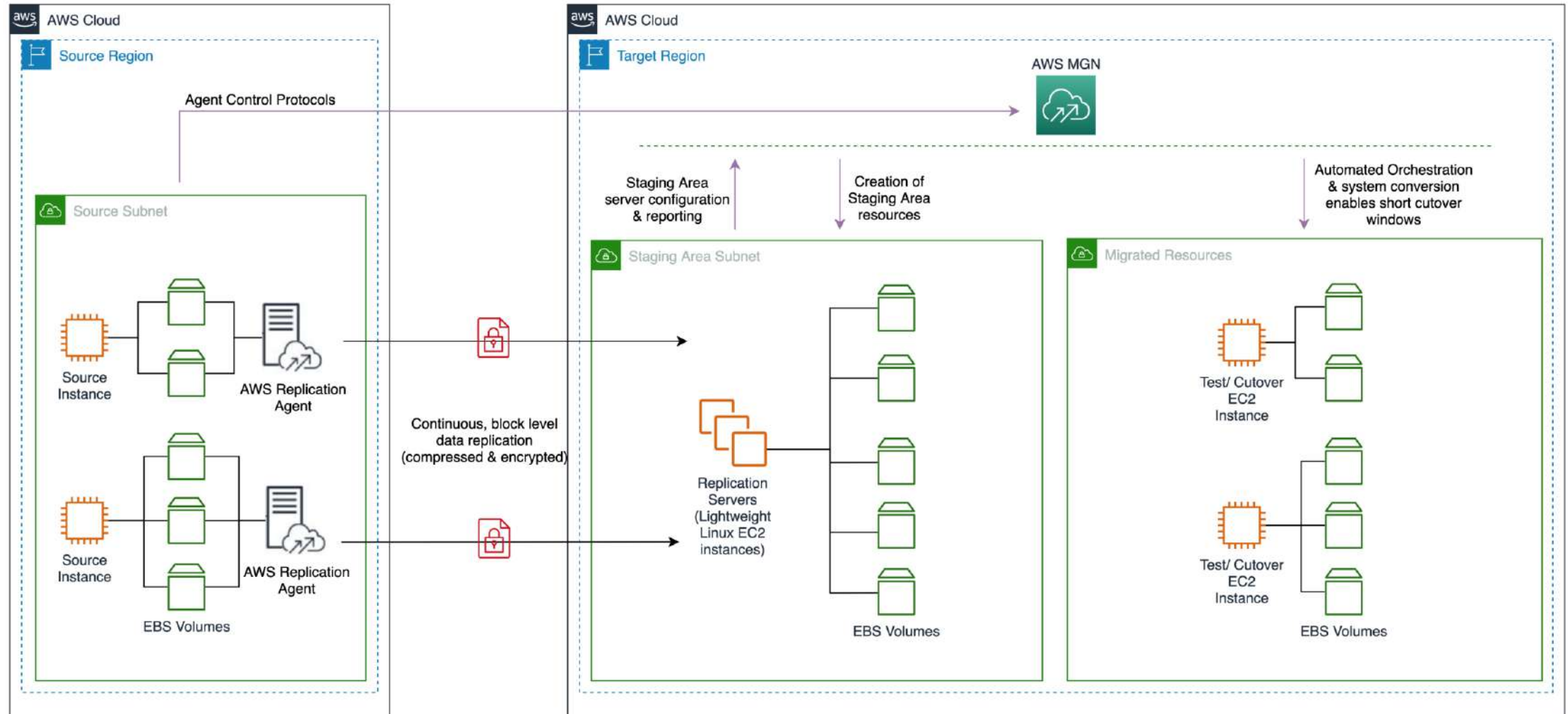


# MGN for On-premises to AWS

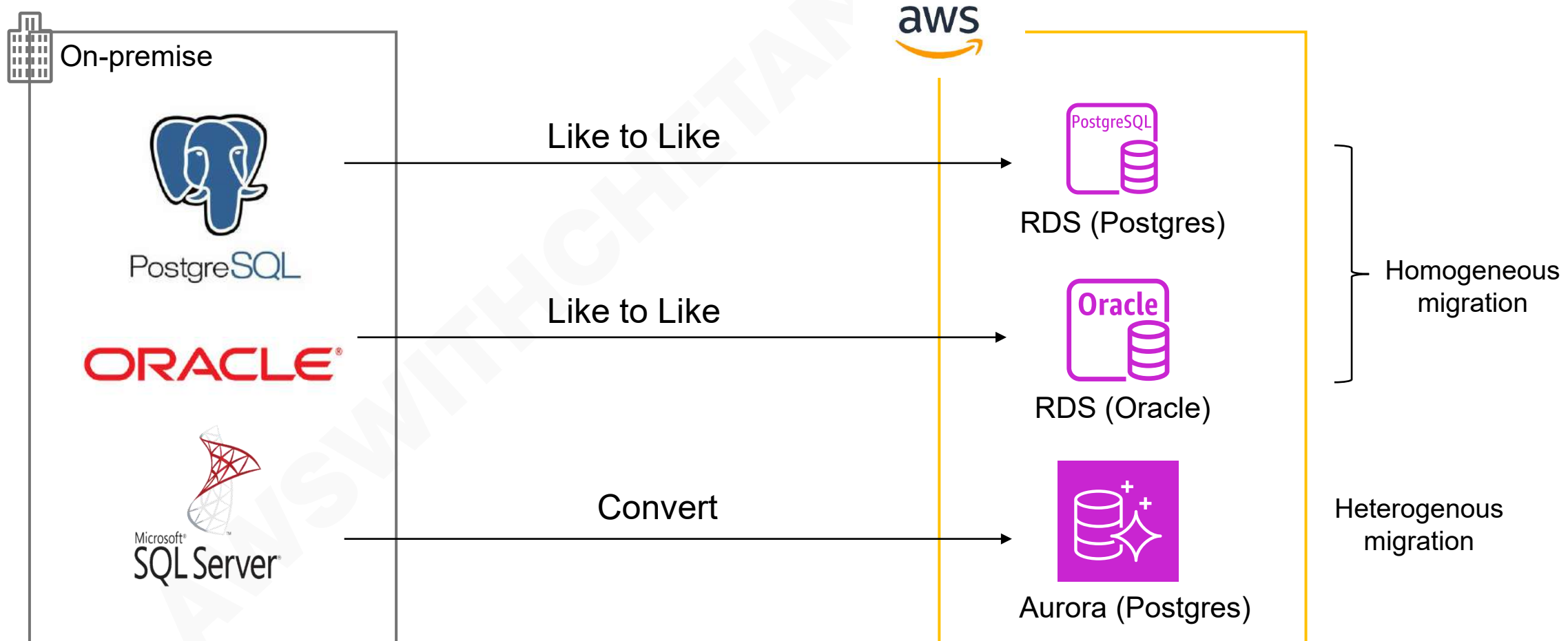




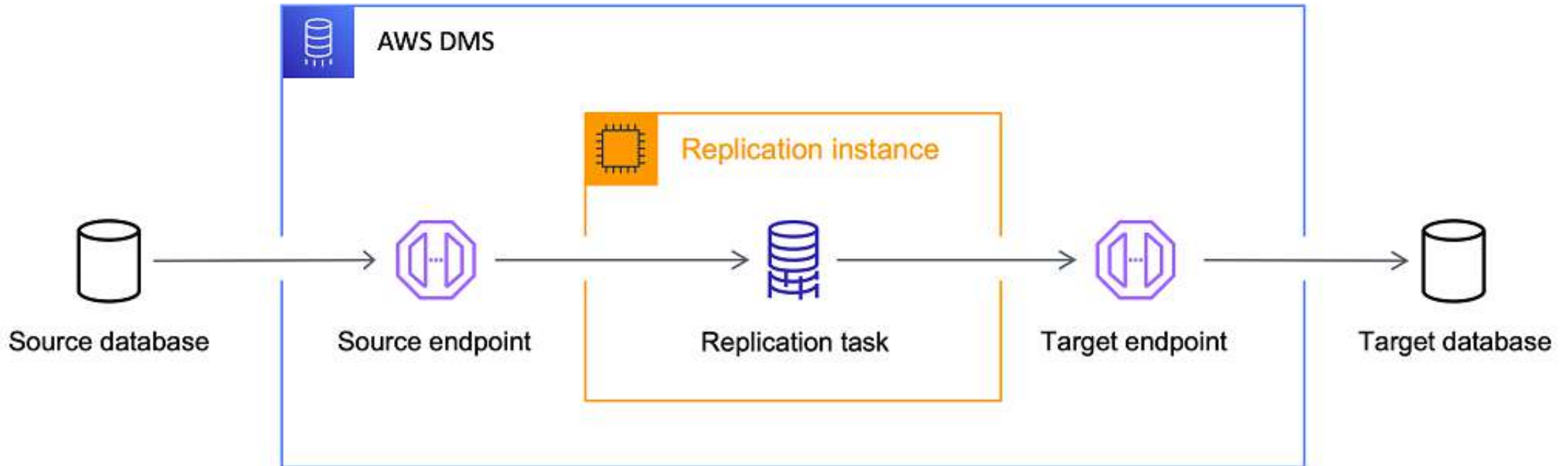
# MGN for AWS to AWS across regions



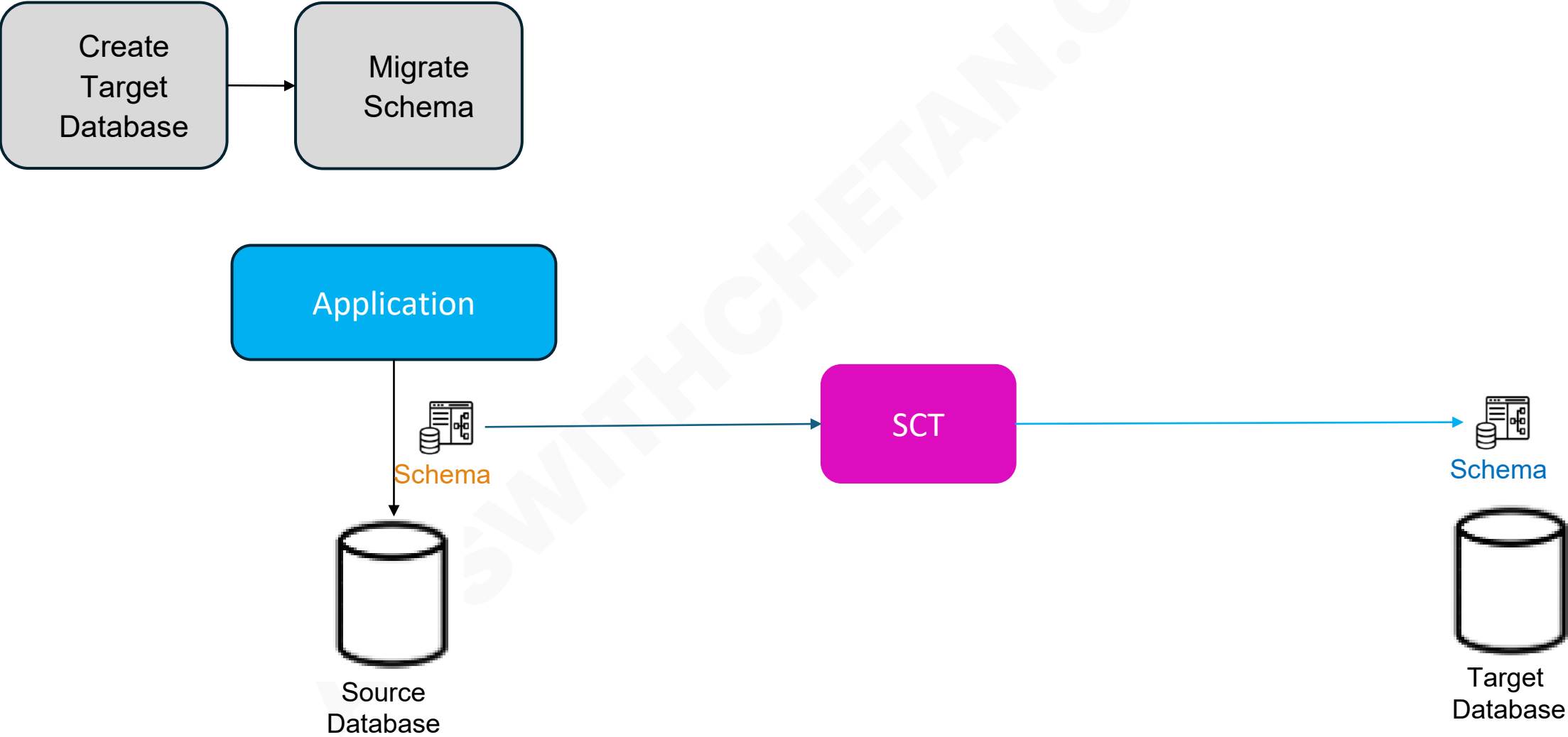
# How to migrate existing database to AWS?



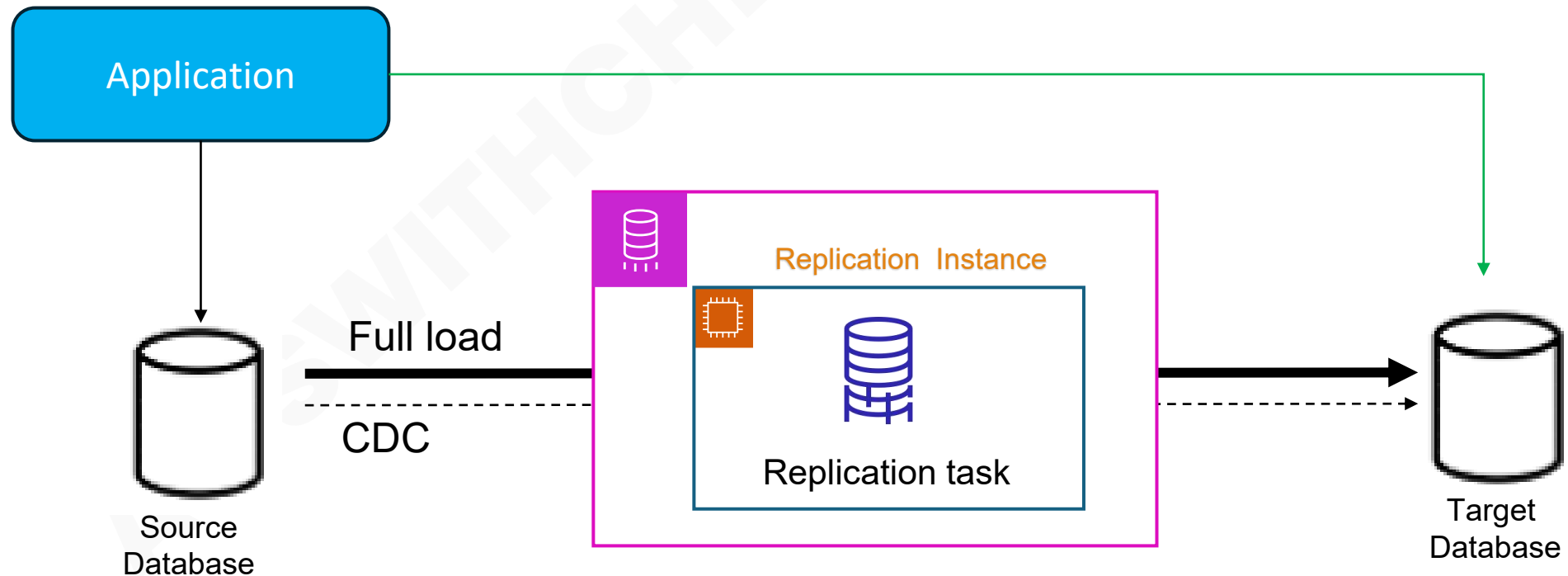
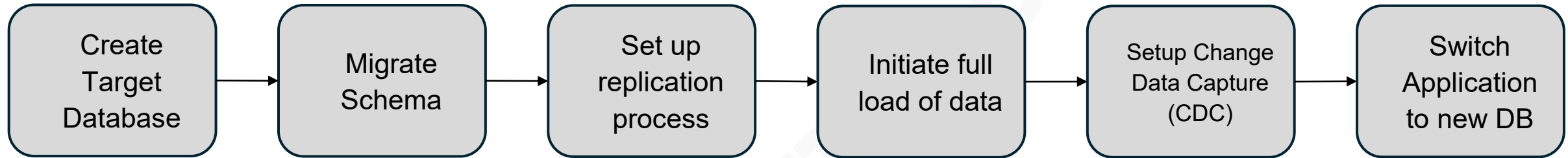
# Amazon Database Migration Service - DMS



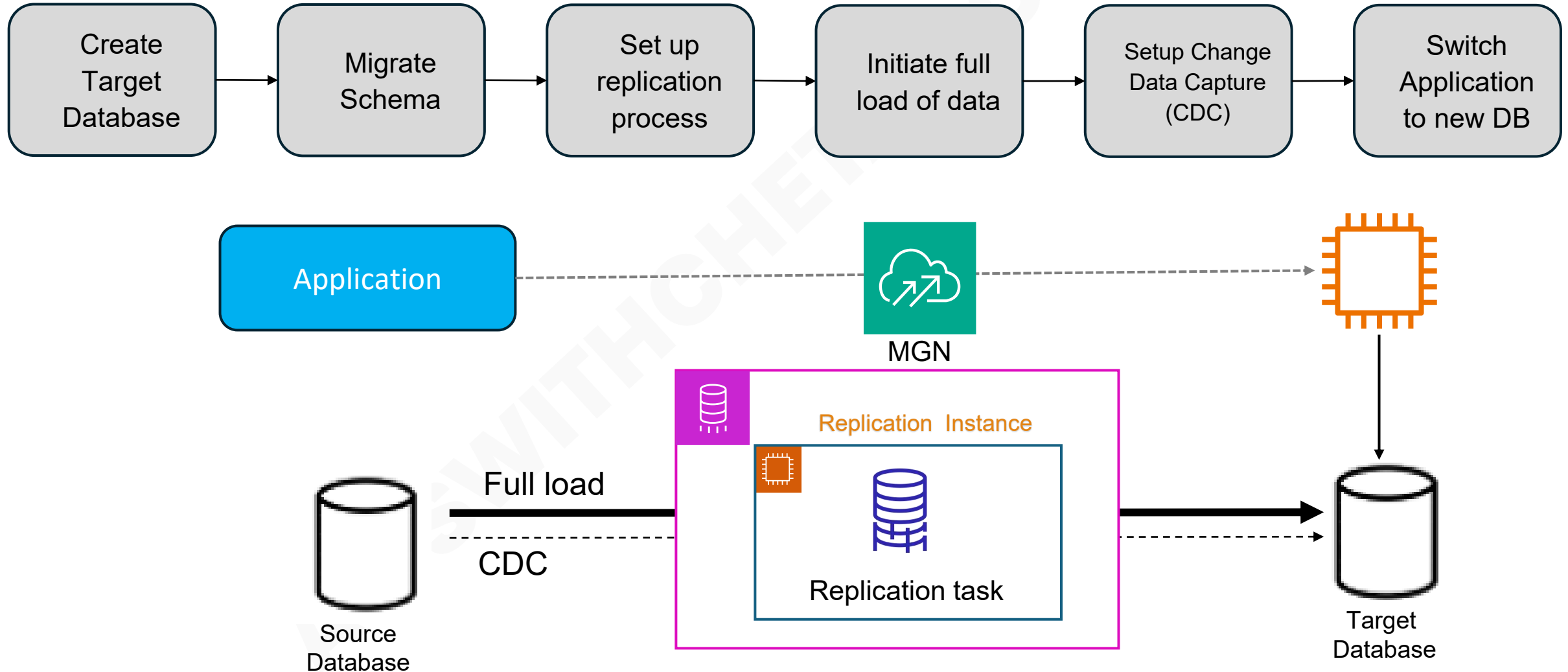
# Schema conversion tool



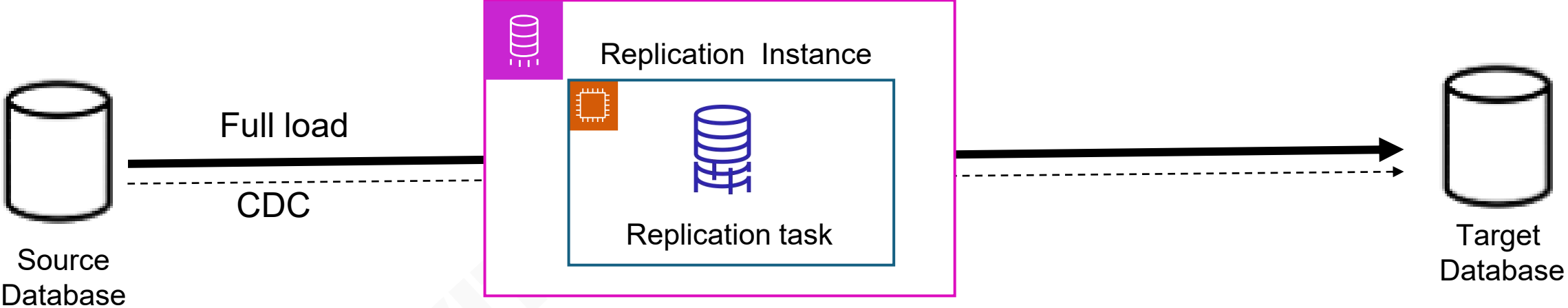
# Database migration steps



# Database migration steps



# DMS replication instance

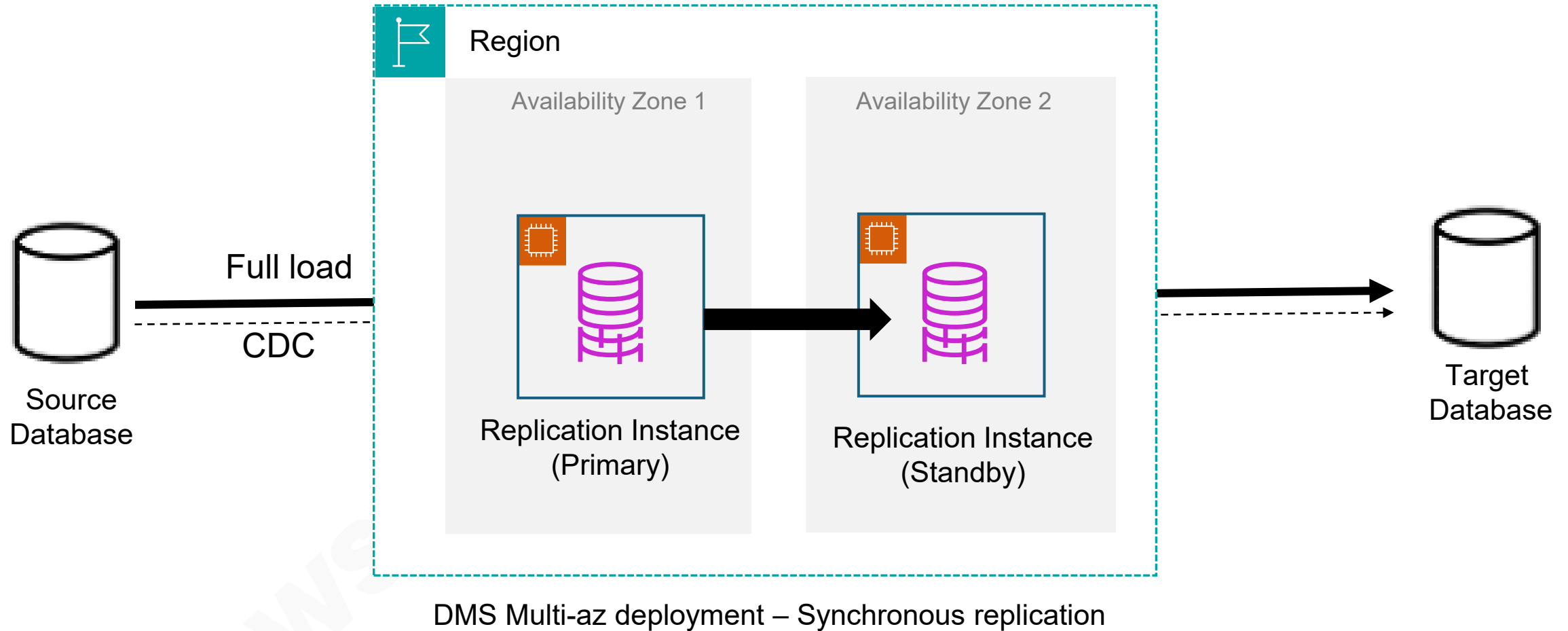


# DMS Replication instance

- The DMS replication instance is an EC2-based server that reads from the source database and writes to the target database.
- AWS DMS supports multiple instance families for the replication instance:
  - General purpose (T3) – Balanced CPU, memory, network; good for small workloads or testing
  - Compute optimized (C5, C6i, etc.) – Higher CPU performance for compute-heavy tasks
  - Memory optimized (R5, R6i, etc.) – More RAM per vCPU for memory-intensive migrations
- How to decide instance family?
  - Memory-optimized instances (R5/R6i etc.) are better for migrations with high throughput (read/writes), high CDC and transactions, or large amounts of in-memory data.
  - Compute optimized instances (C5/C6i etc.) are suited for compute-intensive tasks, such as handling transformations or processing many parallel tasks.



# DMS replication instances - Multi-AZ



# MGN vs DMS



AWS Application  
Migration Service (MGN)

- Replicate application and databases to **Amazon EC2**
- It's a server based like to like migration

Rehost (Lift-and-Shift)

VS



AWS Database Migration Service  
(AWS DMS)

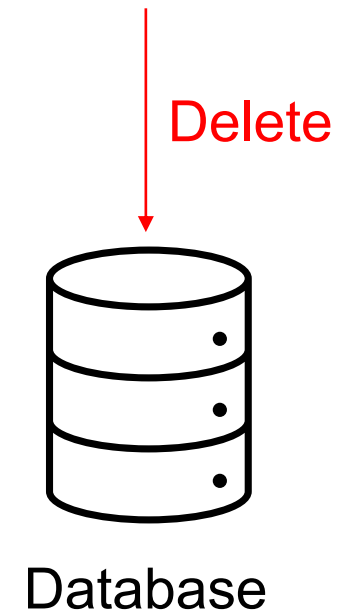
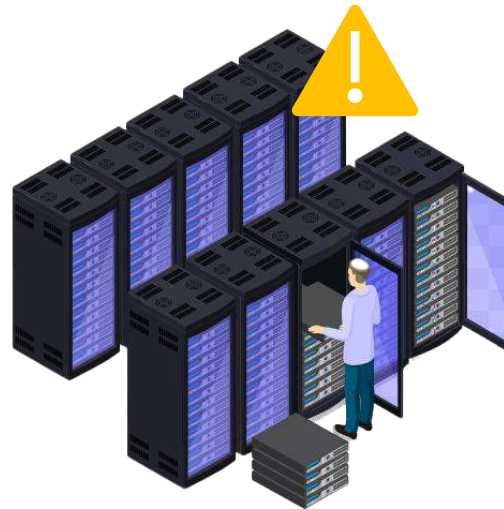
- Migrates on-premises databases to Amazon **RDS**, **Aurora**, Amazon Redshift or DynamoDB databases
- Supports both homogeneous database migration (like to like) and heterogeneous migration (e.g. SQL server to Aurora Postgres) using SCT (Schema Conversion Tool)

Replatform

# Disaster Recovery

# What is Disaster?

- Natural disasters, such as earthquakes or floods
- Technical failures, such as power failure or network connectivity
- Human actions, such as inadvertent misconfiguration or unauthorized/outside party access or modification

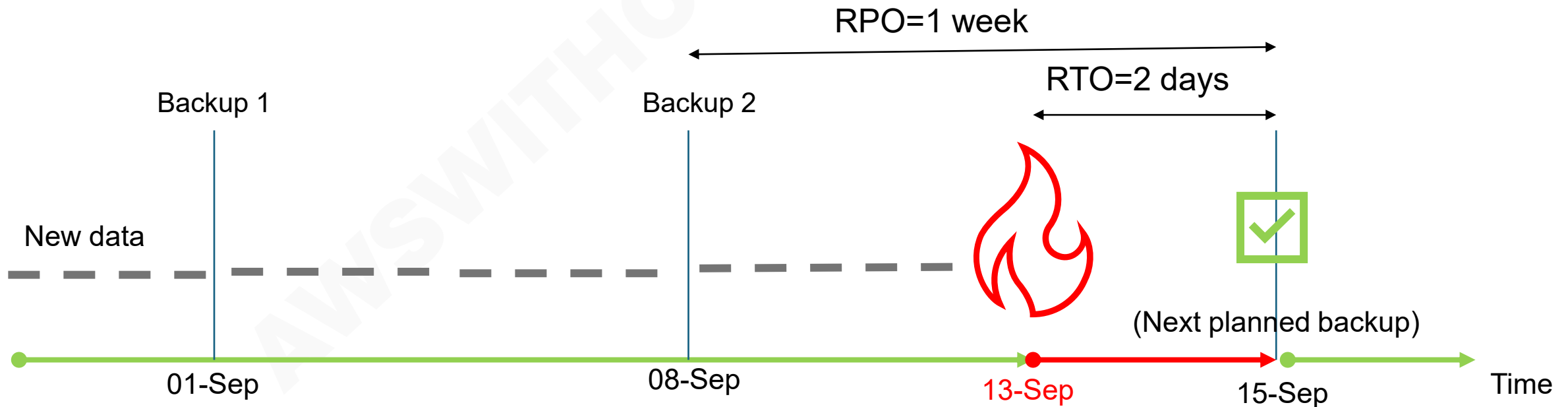


# Disaster Recovery (DR)

The process of preparing for and recovering from a disaster

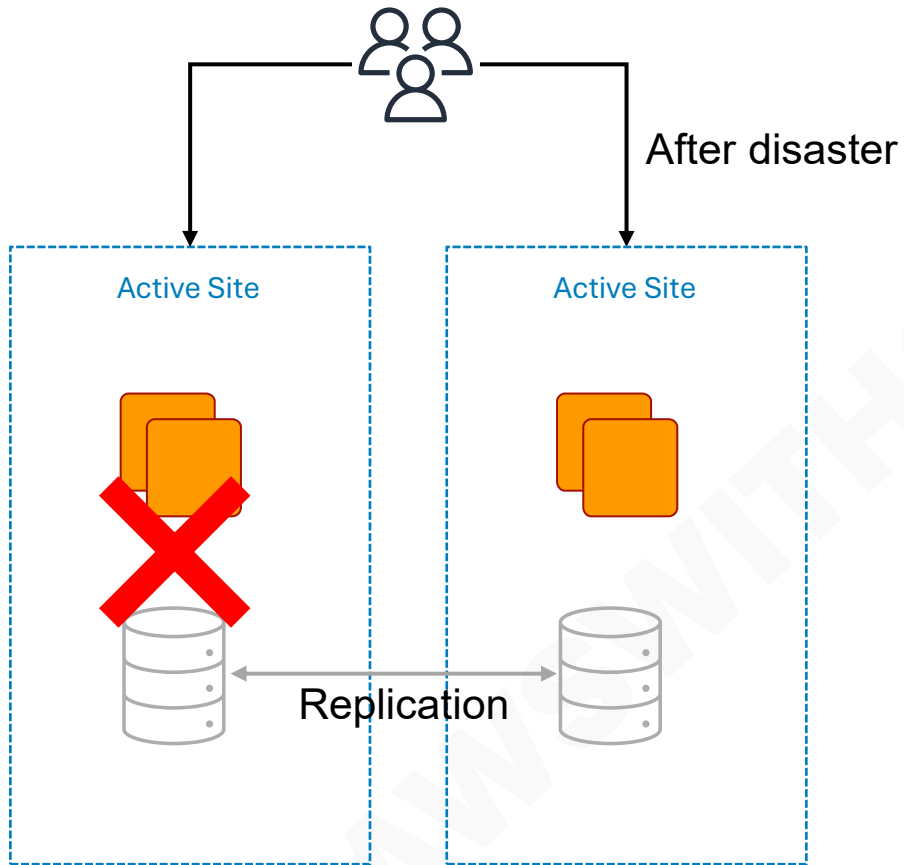
## DR Objectives:

- Recovery point objective (RPO): The maximum acceptable amount of time since the last data recovery point.
- Recovery time objective (RTO): The maximum acceptable delay between the interruption of service and restoration of service.

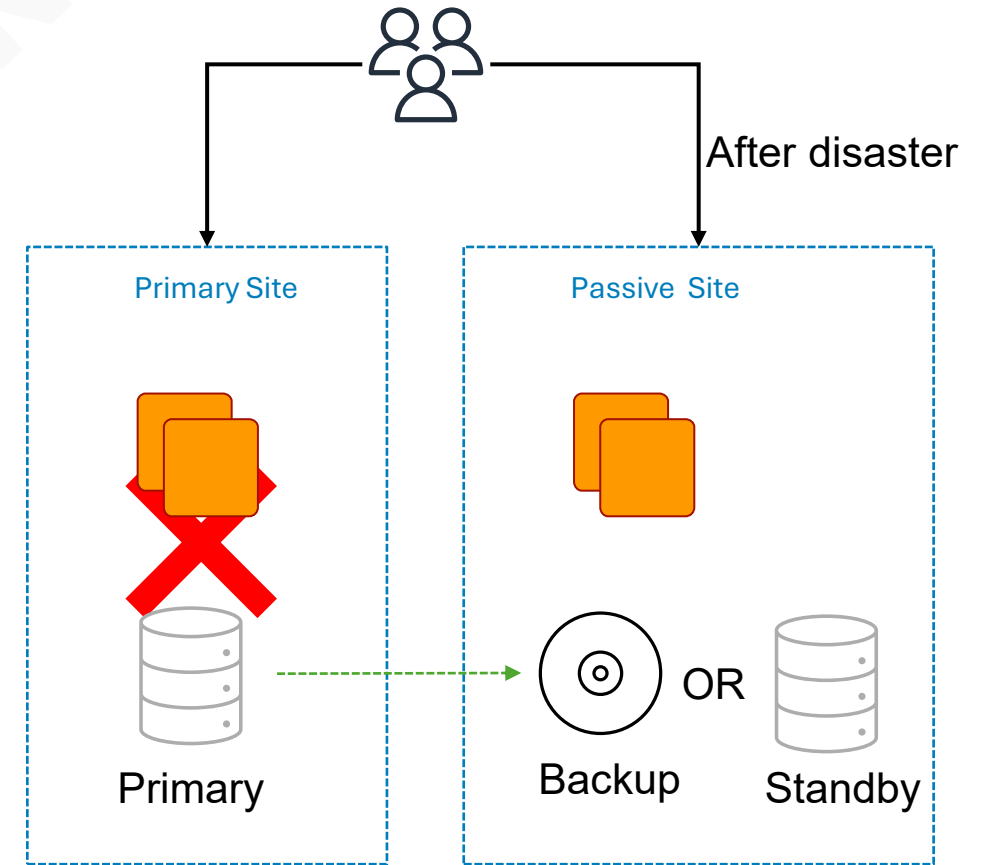


# DR Strategies

## Active/Active



## Active/Passive



Source: [AWS blog/](#)

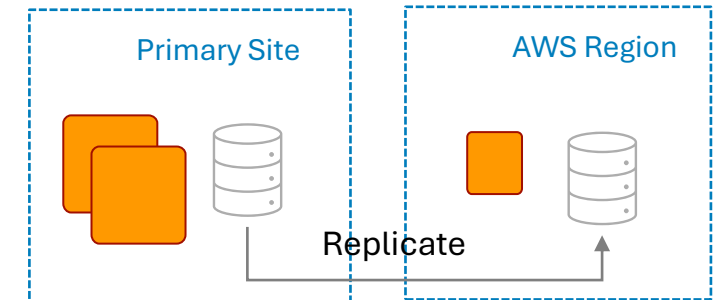
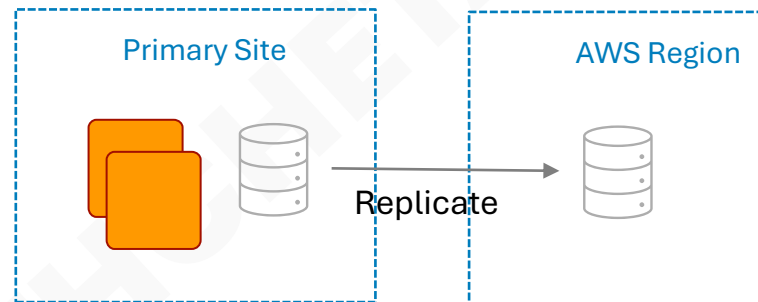
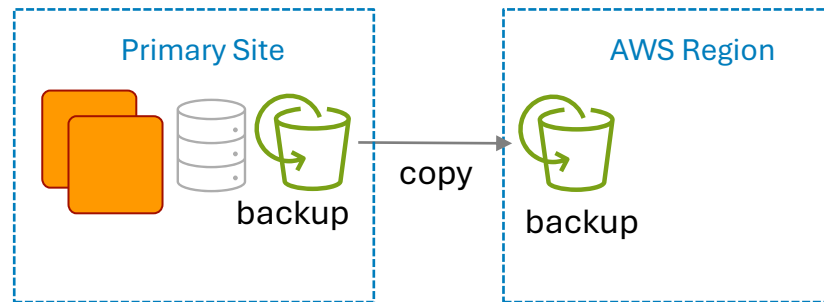
# DR Strategies

Active/Passive

## Backup and restore

## Pilot Light

## Warm Standby



### Before the event

- Take regular backups

### After the event

- Provision infrastructure resources
- Restore from backups

### Before the event

- Replicate data
- Keep Infrastructure elements ready but not On

### After the event

- Bring-up compute

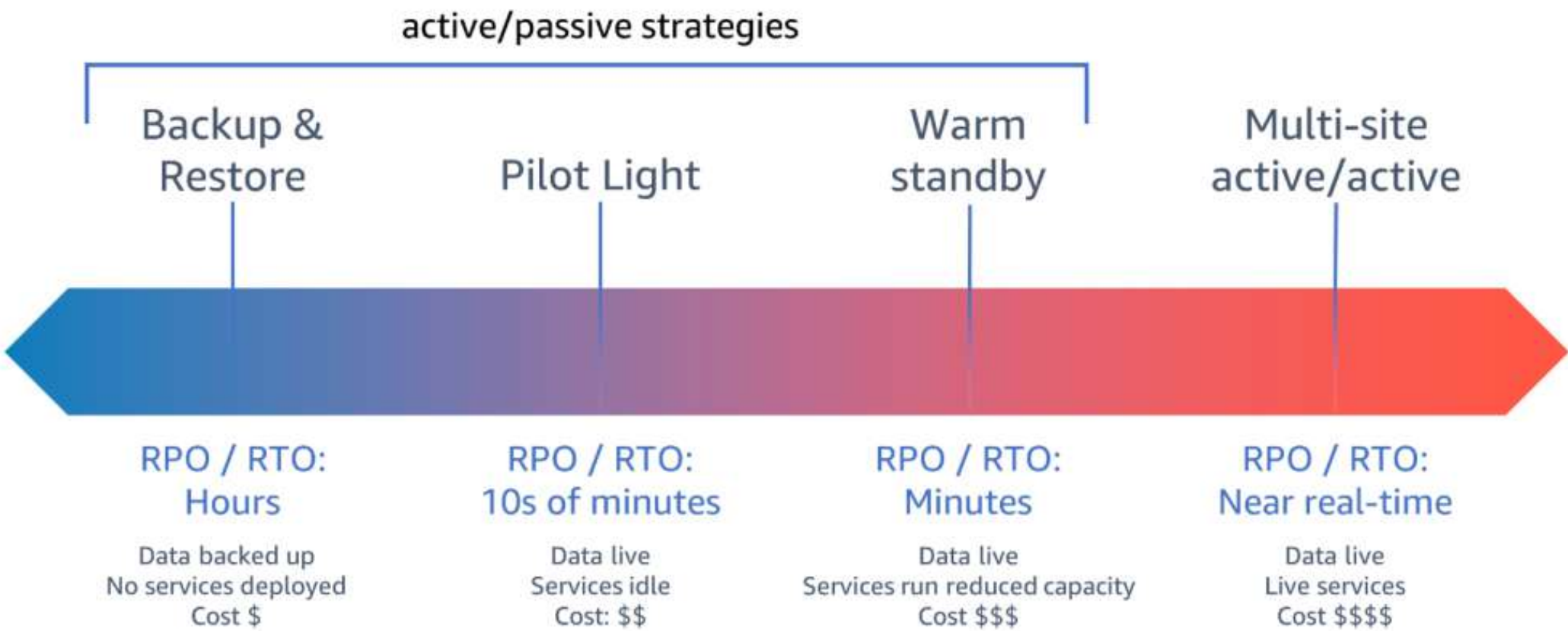
### Before the event

- Replicate data
- Keep infrastructure running at lower capacity

### After the event

- Scale the capacity

# DR Strategies vs RPO/RTO



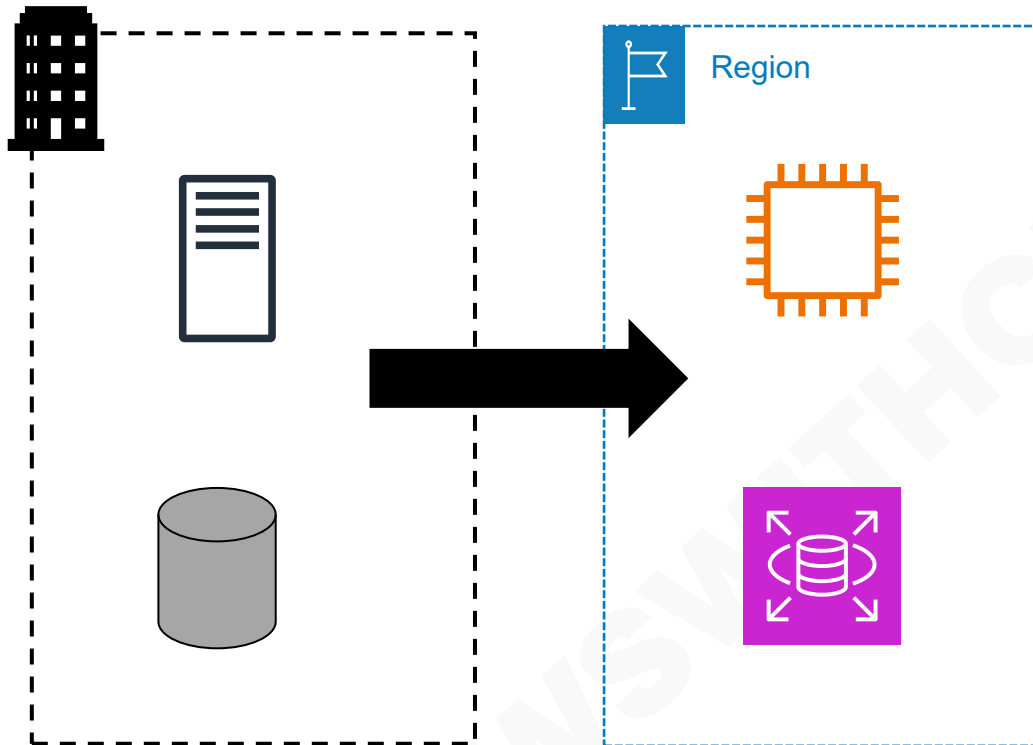
Source: [AWS Blog](#)



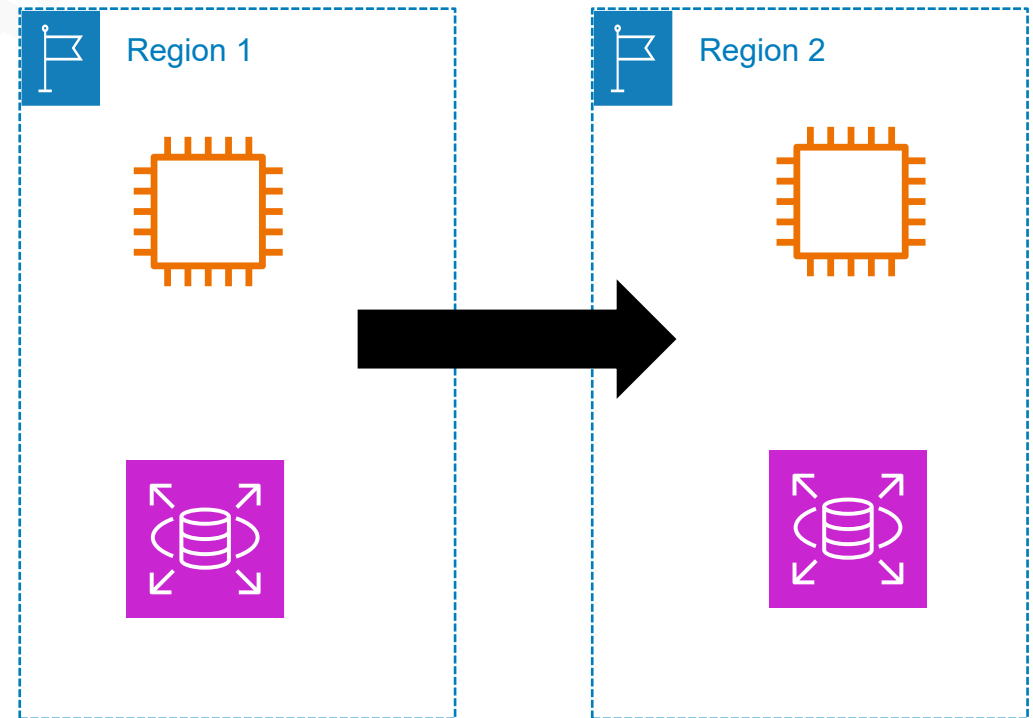
Let's talk architecture

# DR Strategies

## On-premises to AWS

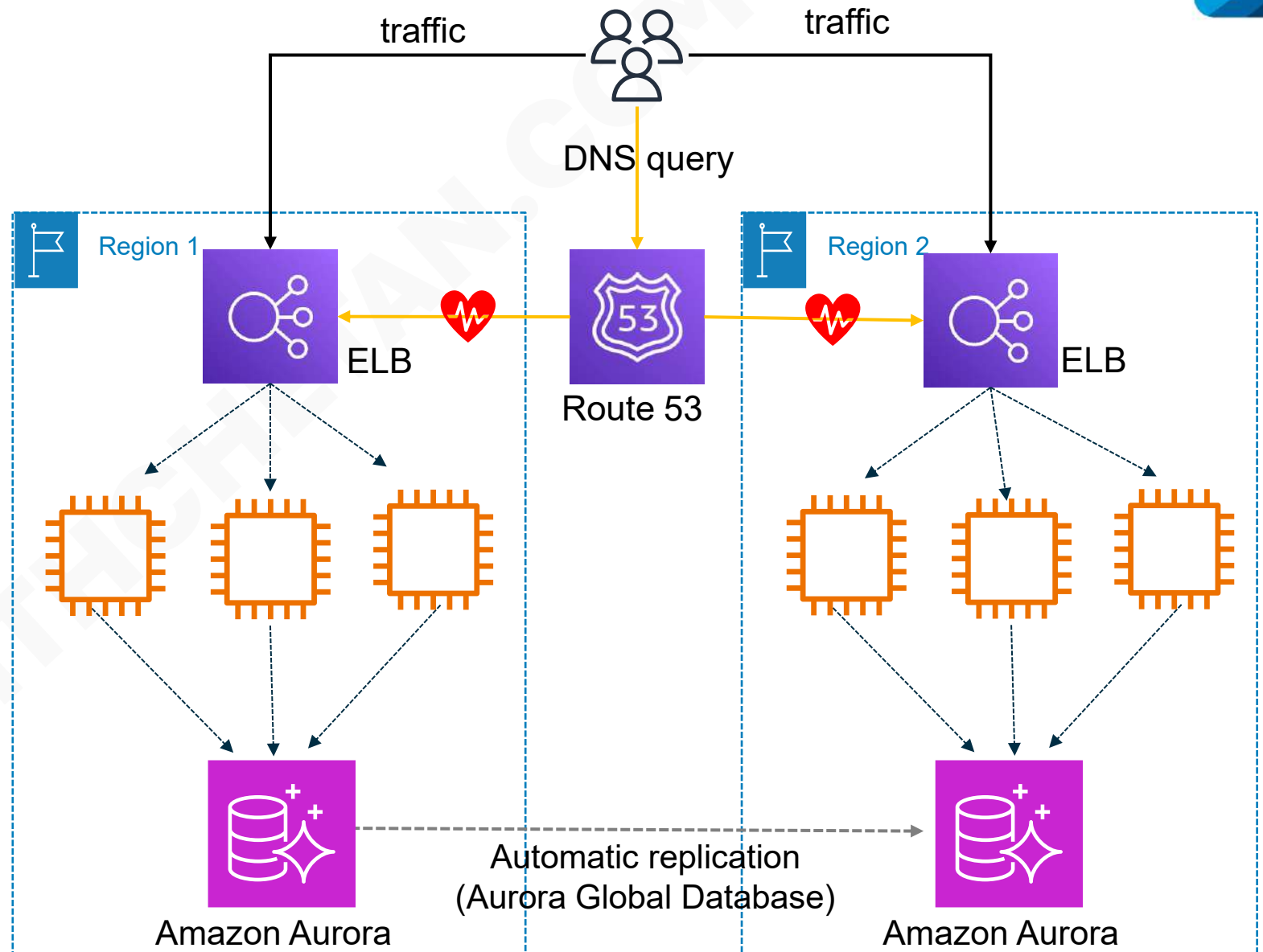


## AWS to AWS



# Active/Active

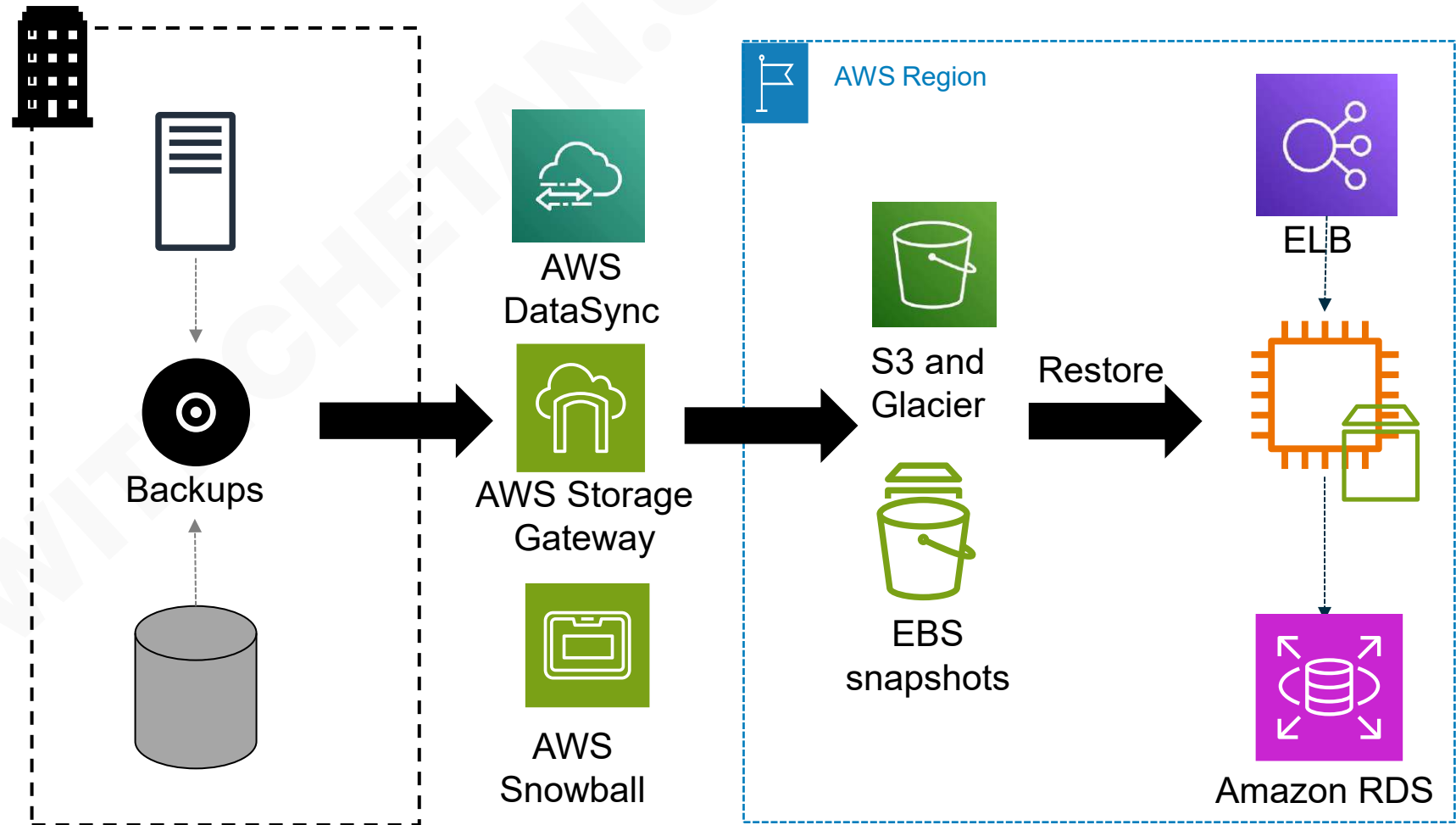
- Possible if both sites are in AWS (in different regions)
- Re-routing the requests to healthy region
- Data is replicated in real-time
- RPO is near zero
- RTO in seconds
- Cost is very high due to multiple sites

**RTO****RPO****COST**

# Backup & Restore

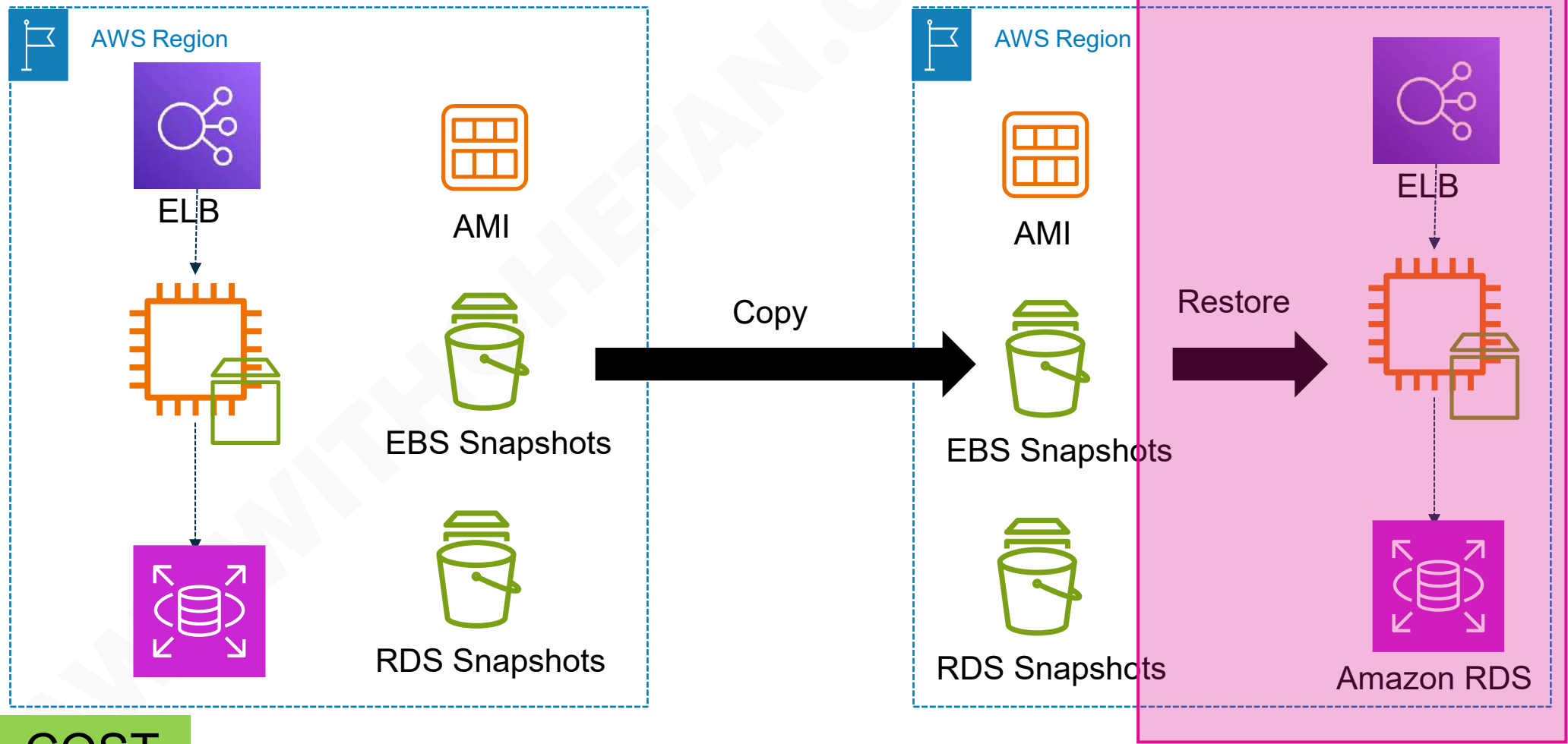
## On-premises to AWS

- Regularly backup the Application Servers data, File servers and Databases
- Move backups to AWS S3 (using Storage gateway, Snowball etc.)
- In case of disaster, launch EC2/RDS and restore data / databases

**RTO****RPO****COST**

# Backup & Restore

## AWS to AWS

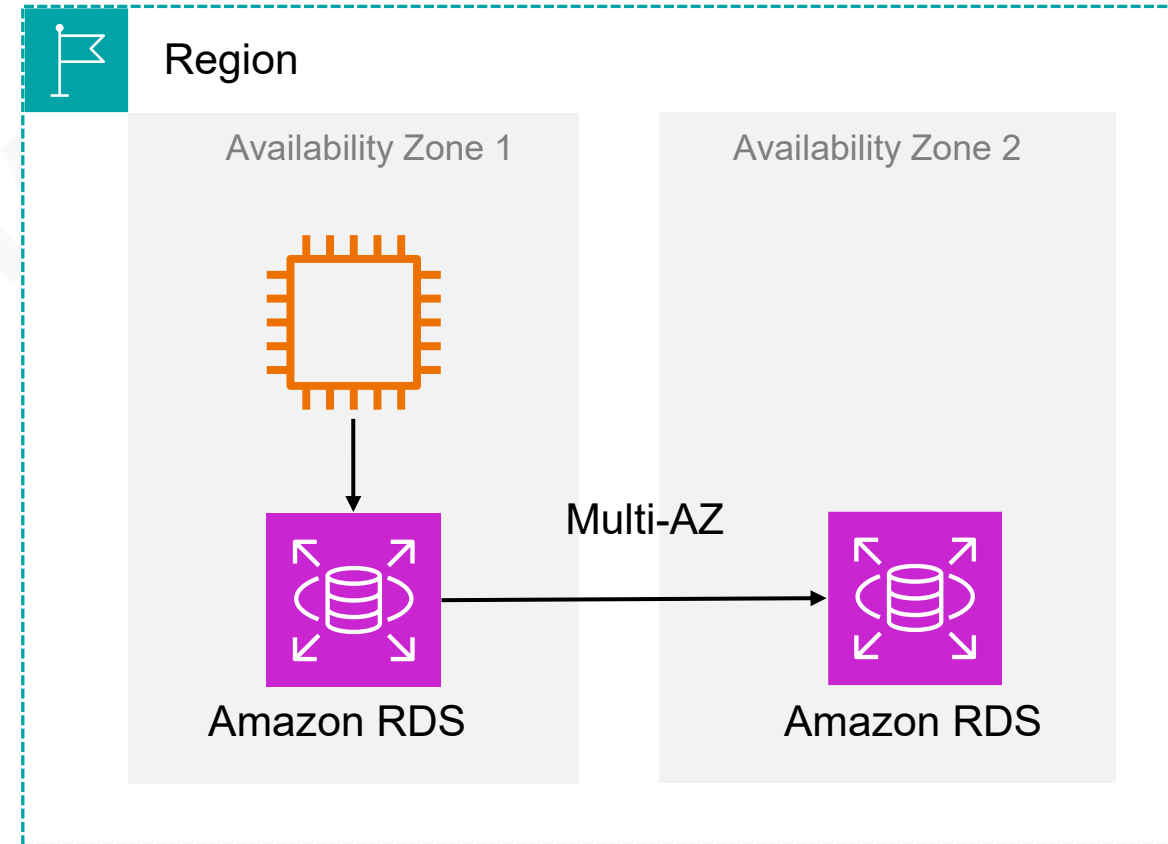


RTO RPO COST

# Amazon RDS Multi-AZ: HA or DR?

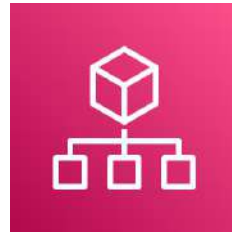
Answer: Highly Available (HA)

If by mistake someone drops important table, then it will be dropped from the secondary DB as well. In this case application will be available but won't be functional causing a disaster.



# AWS Multi-account Management

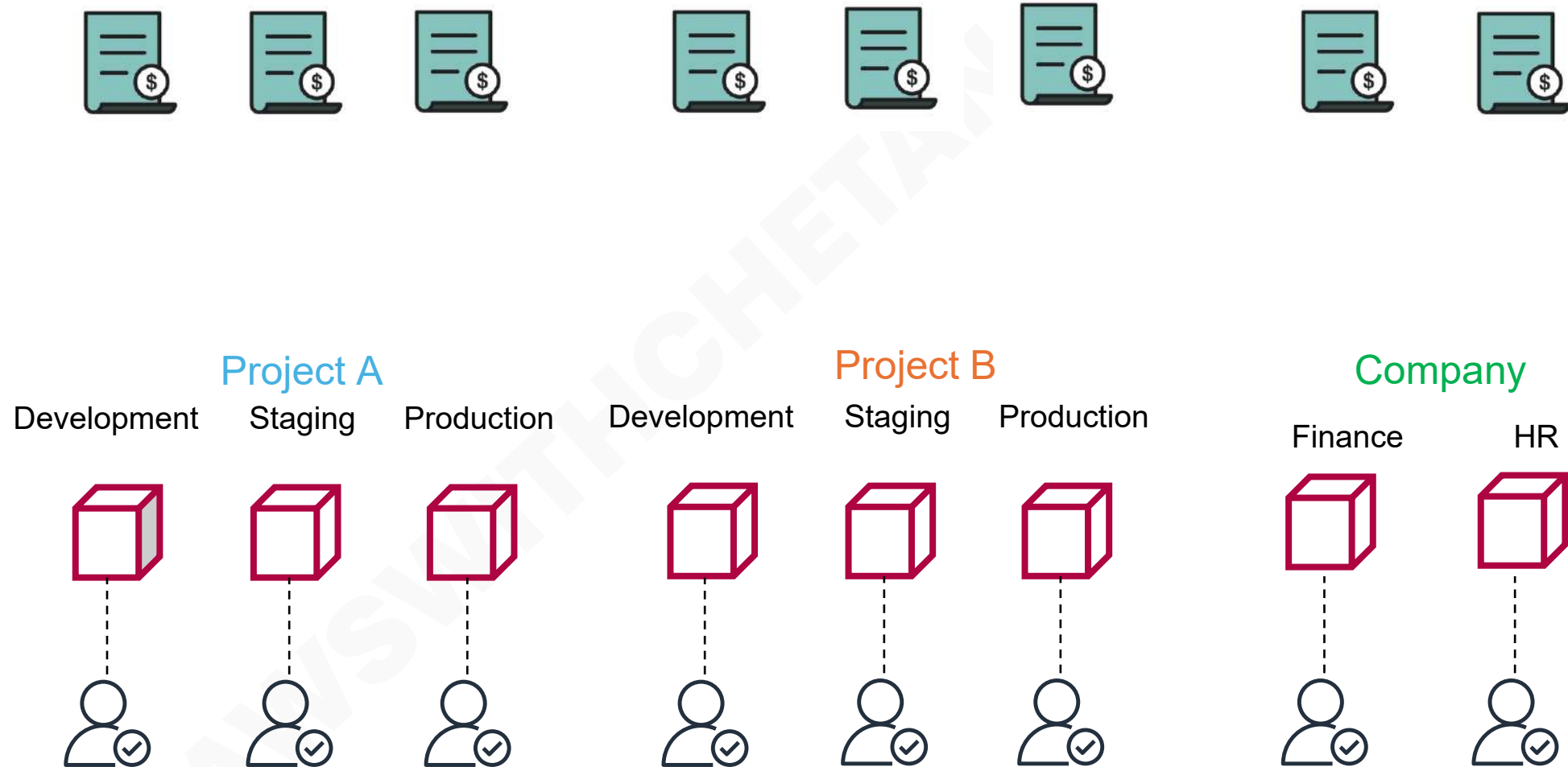
AWS Organizations, AWS Control Tower, IAM Identity Center and more..



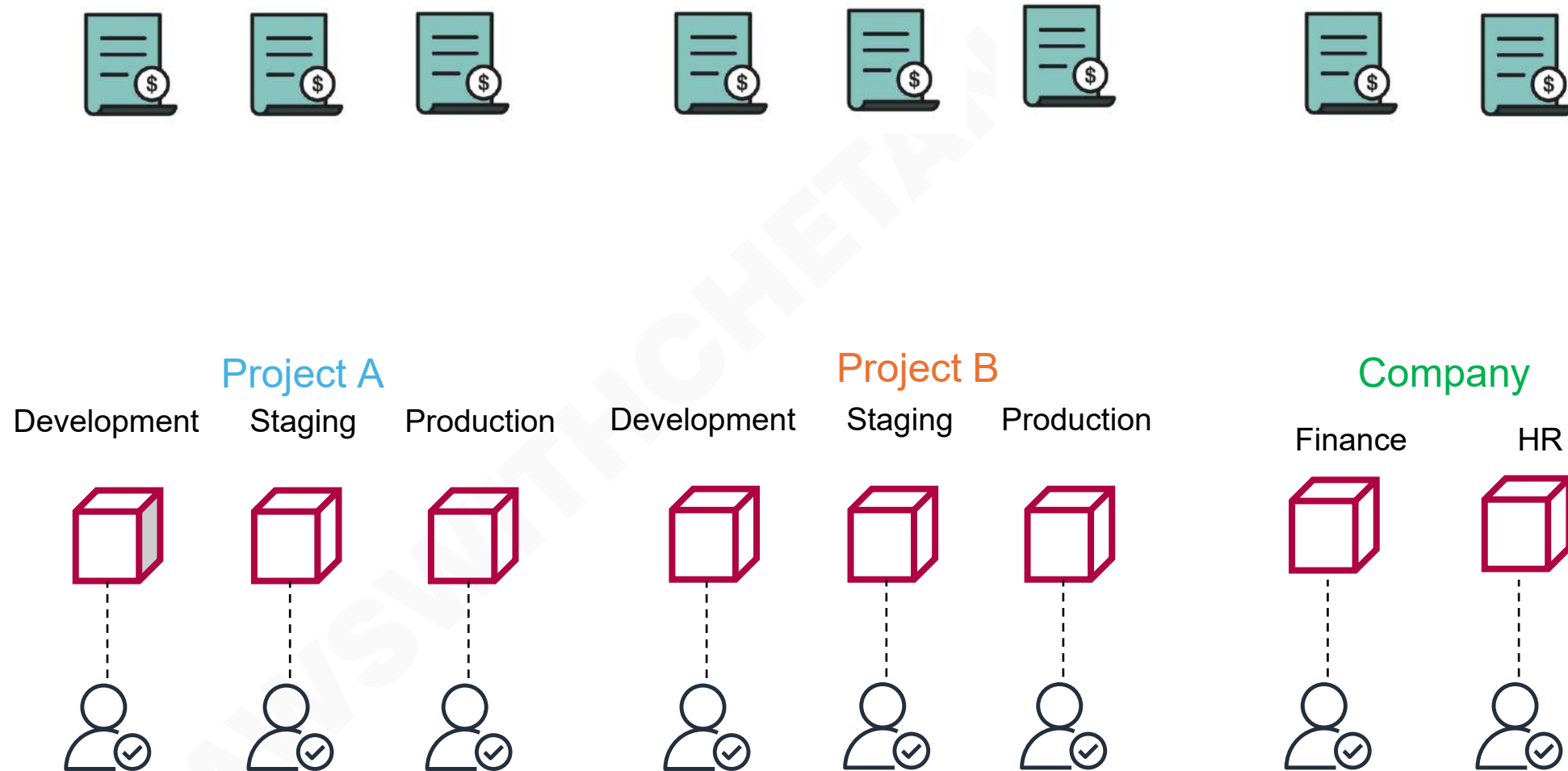
# AWS Organizations



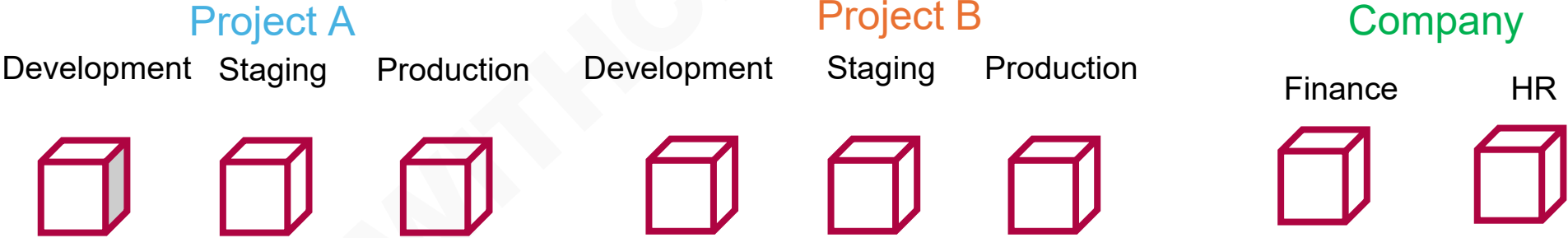
# AWS accounts in an enterprise



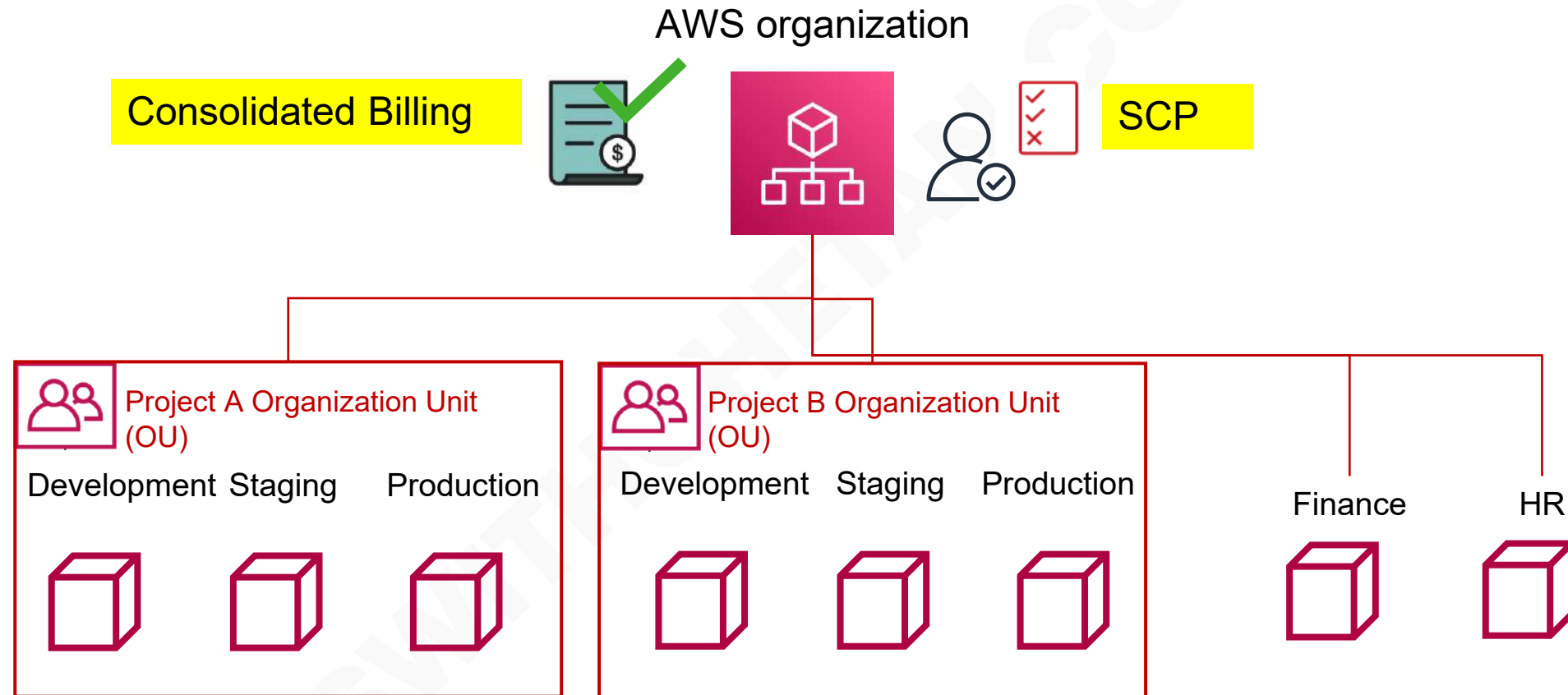
# AWS accounts in an enterprise



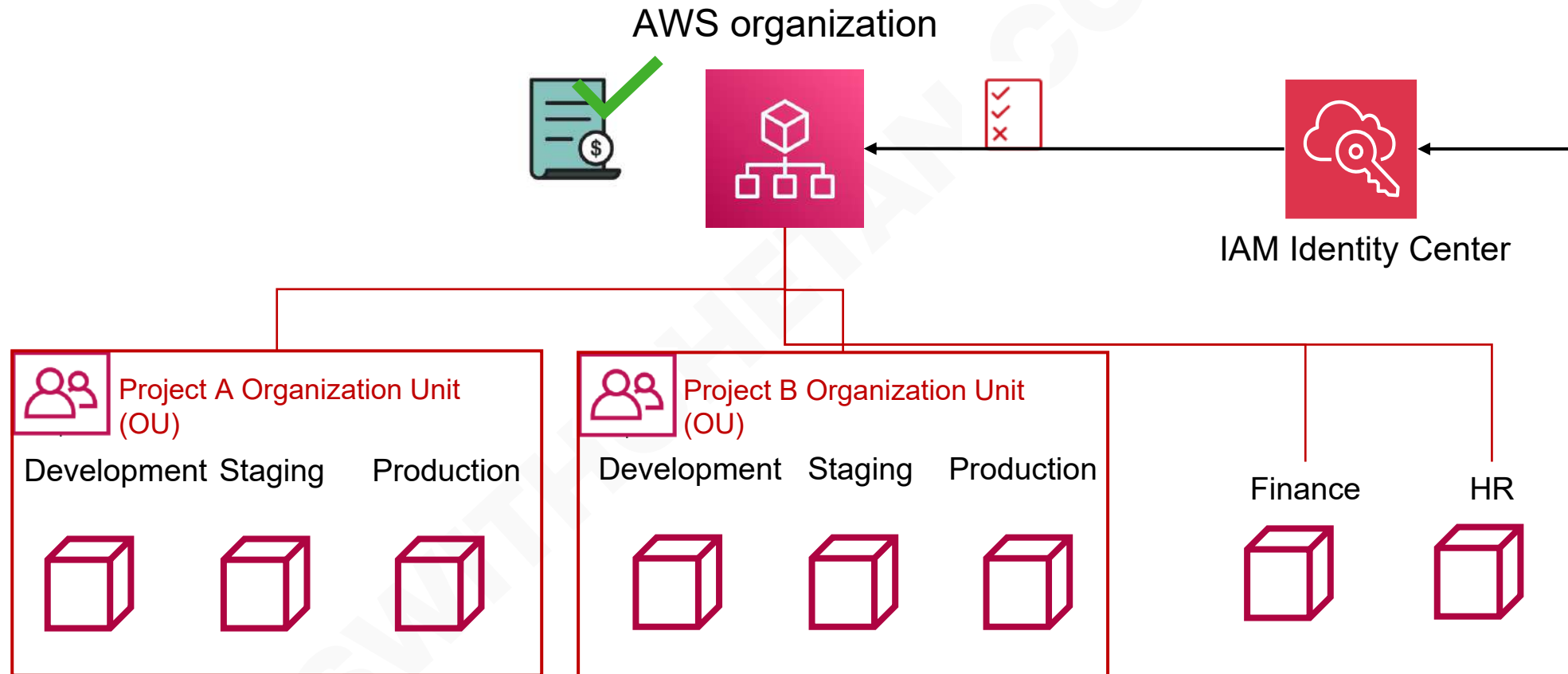
# AWS Organizations



# AWS Organizations

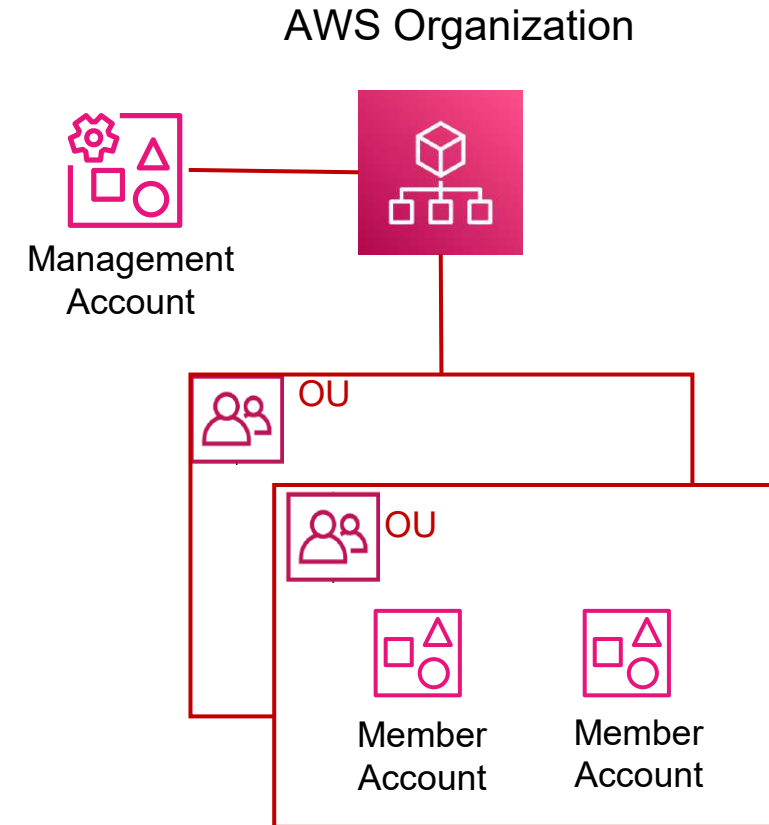


# AWS Organizations



# AWS Organizations Overview

- AWS Organizations is a global service used to centrally manage multiple AWS accounts.
- It consists of one management account and multiple member accounts.
- Each member account can belong to only one organization.
- Member accounts can be grouped into Organizational Units (OUs) for easier management.
- OUs support up to five levels of hierarchical nesting to meet security, compliance, and budgeting needs.
- Each organization has a unique **Organization ID**
- AWS Organizations supports either “Consolidated billing only” or “All features”



# AWS Organizations - Enabling all features

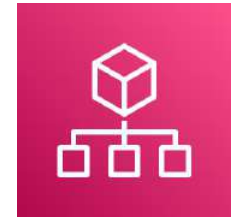
- When all features are enabled, AWS Organizations can:
  - Apply Service Control Policies (SCPs)
  - Enforce Tag Policies
  - Manage Backup policies
  - Use AI services opt-out policies
  - Control access and governance across accounts
  - Enforce guardrails at:
    - Root
    - OU
    - Account level
- This mode turns Organizations into a governance and control plane, not just Consolidated billing.
- Once enabled, can not be rolled back



AWS Organization

# AWS Organizations - Important features

- Service Control Policies (SCP)
- Consolidated Billing
- Tag Policies
- Using **aws:PrincipalOrgID** in IAM resource-based policies

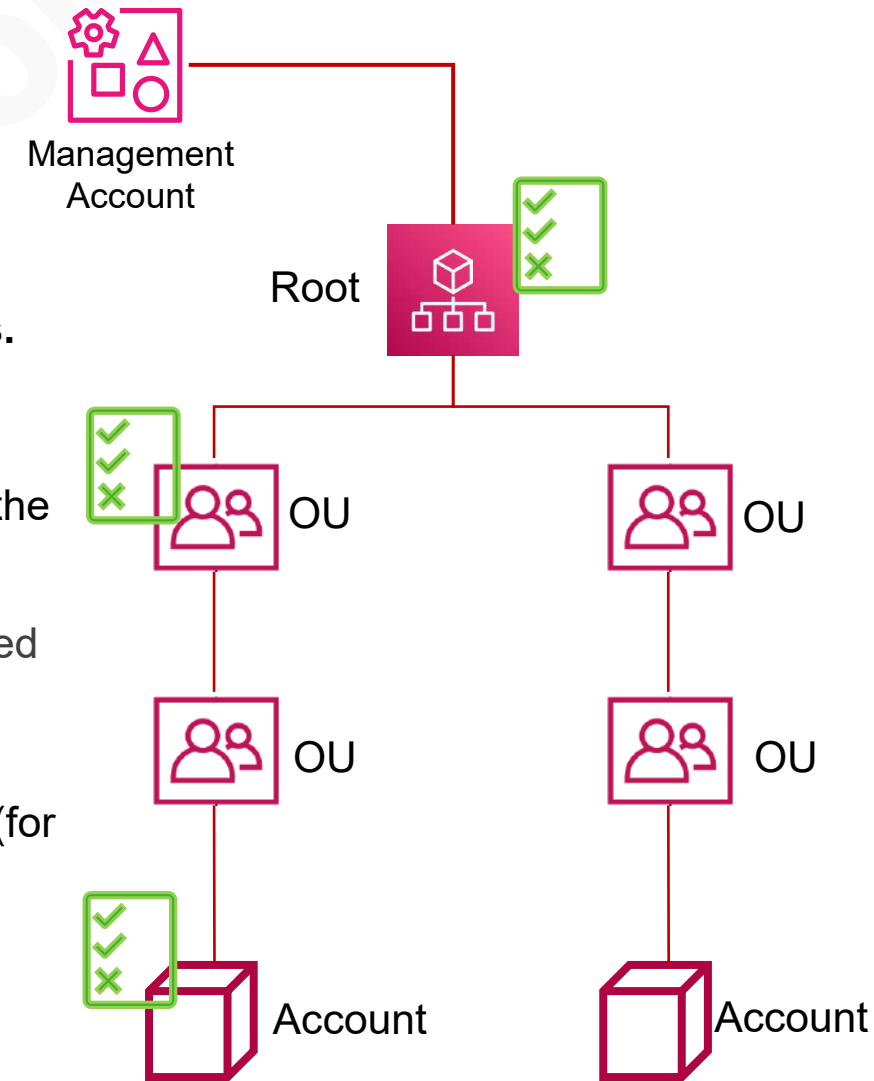


AWS Organization



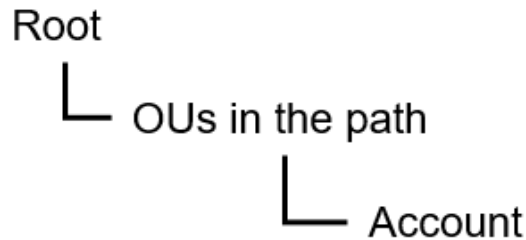
# Service Control Policies (SCP)

- Manage IAM permissions across AWS Accounts in AWS Organization.
- SCP governs the maximum available permissions for the IAM users and roles
- **SCPs do not grant additional permissions to the IAM users and IAM roles.**
- Applied at the Root level or OU level or Account level.
- SCPs don't affect users or roles in the management account. They affect only the member accounts in your organization.
- By default, **FullAWSAccess** is attached at root to allow all actions unless denied
- **Common use cases:**
  - Restrict access to certain services which are banned in your organization (for example: AI services)
  - Restrict access to certain AWS regions where you do not have any workloads
  - Restrict access to certain instance types and size in the lower environments



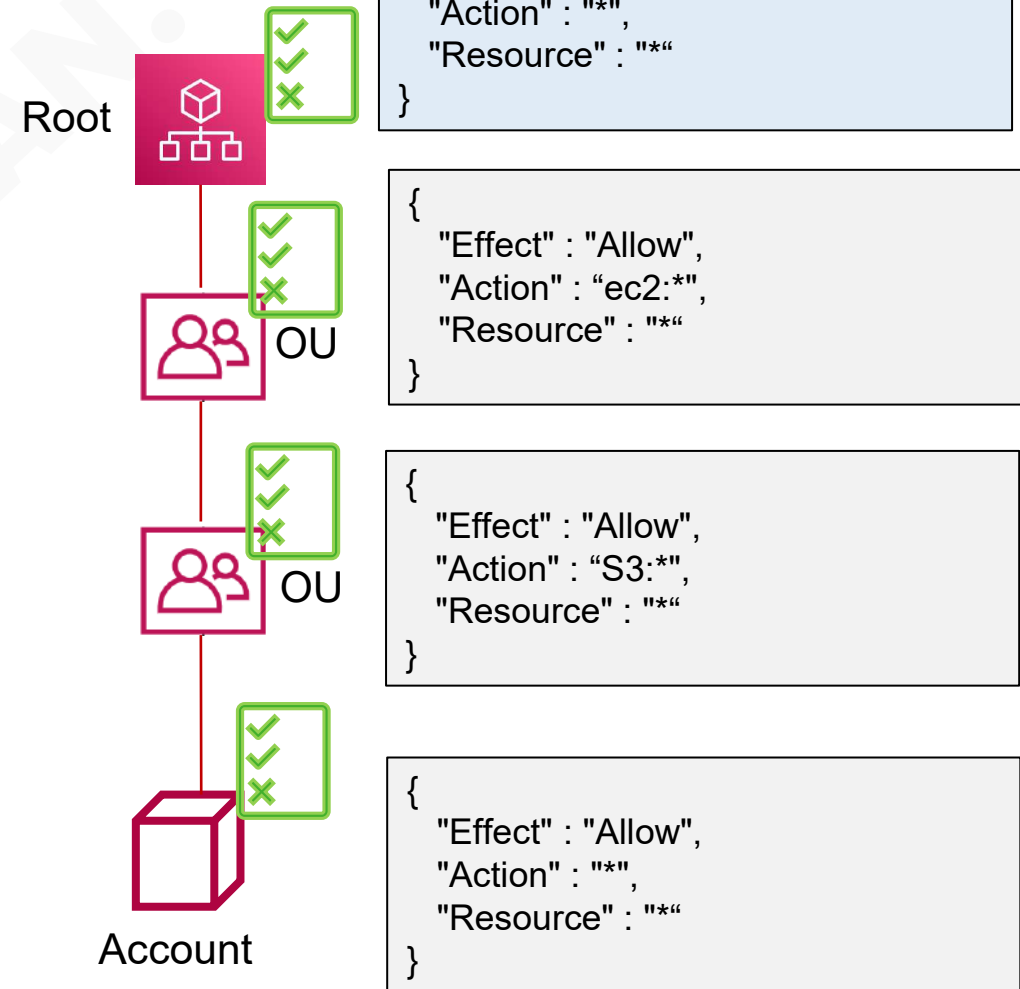
# SCP policy evaluation and effect

- SCP effective permissions are the intersection of all SCPs attached at:



- Explicit Deny in any SCP always overrides everything else
- If an SCP contains Allow statements, any action not explicitly allowed is implicitly denied.
- If no SCP is attached at a level, permissions are inherited from the parent
- SCPs affect all IAM principals in the account: IAM users, IAM Roles and **Root user**

What will be resulting permissions at Account level?



# Sample SCP policies

Apply at Root level or OU level

Restrict all users to launch only t2.micro instance type

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequireMicroInstanceType",
      "Effect": "Deny",
      "Action": "ec2:RunInstances",
      "Resource": [
        "arn:aws:ec2:*:*:instance/*"
      ],
      "Condition": {
        "StringNotEquals": {
          "ec2:InstanceType": "t2.micro"
        }
      }
    }
  ]
}
```

[https://docs.aws.amazon.com/organizations/latest/userguide/orgs\\_manage\\_policies\\_scps\\_examples.html](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_examples.html)

# Sample SCP policies

Apply at Root level or OU level

Restricts all users from uploading unencrypted objects to S3 buckets.

```
{
  "Effect": "Deny",
  "Action": "s3:PutObject",
  "Resource": "*",
  "Condition": {
    "Null": {
      "s3:x-amz-server-side-encryption": "true"
    }
  }
}
```

[https://docs.aws.amazon.com/organizations/latest/userguide/orgs\\_manage\\_policies\\_scps\\_examples.html](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_examples.html)

# Sample SCP policies

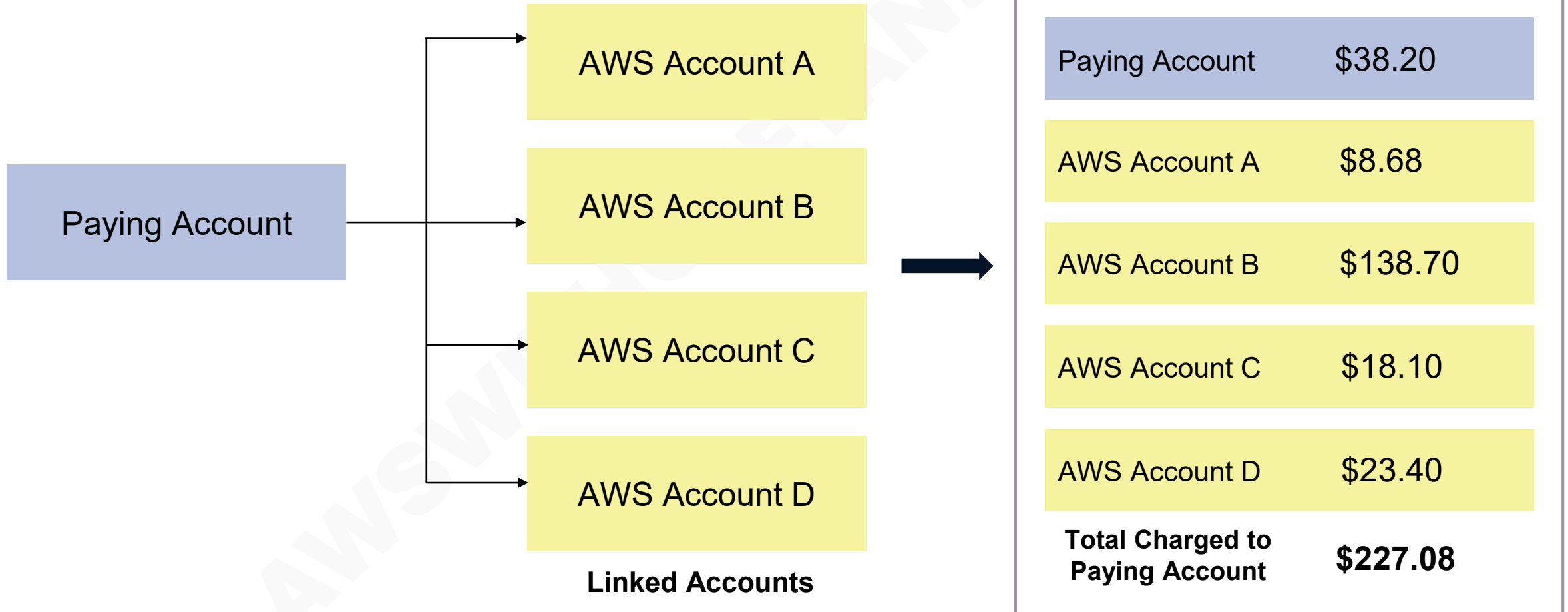
Apply at Root level or OU level

Restricts all users to use only Mumbai region to launch their AWS resources

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": "ap-south-1"
        }
      }
    }
  ]
}
```

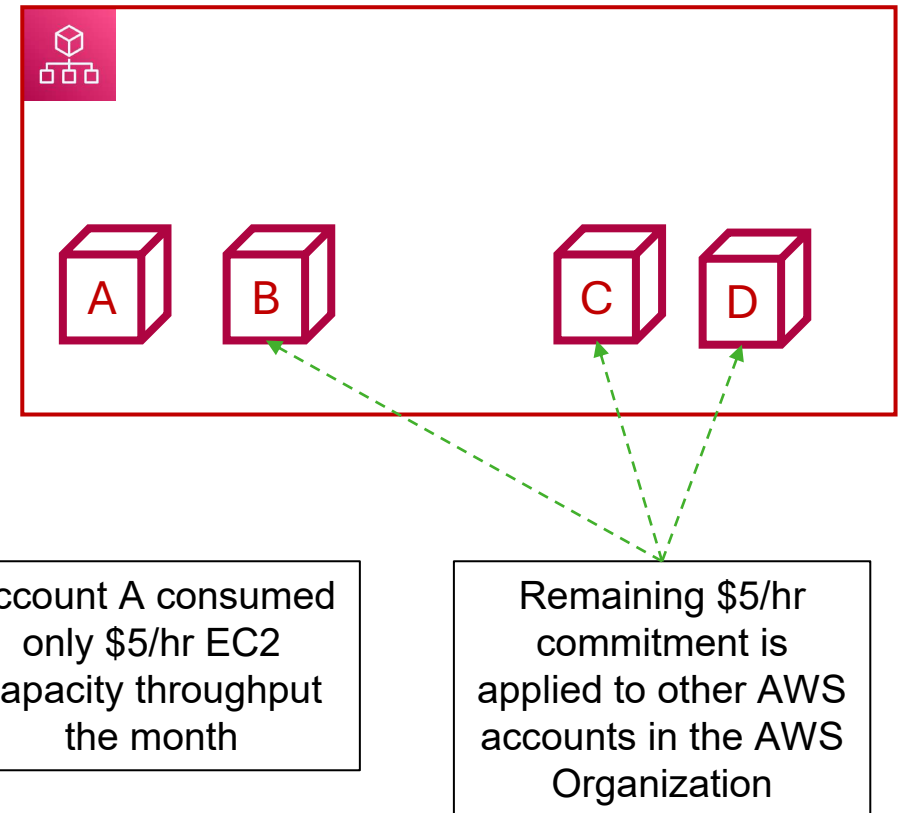
[https://docs.aws.amazon.com/organizations/latest/userguide/orgs\\_manage\\_policies\\_scps\\_examples.html](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_examples.html)

# AWS Organizations – Consolidated Billing



# AWS Organizations – Consolidated Billing

- Management account to pay bill for entire AWS organization (including all member accounts)
- Benefits:
  - One Bill - Get one bill for multiple accounts.
  - Combined Usage - Combine the usage across all accounts in the organization to share the volume pricing discounts, Reserved Instance discounts, and Savings Plans. This can result in a lower charge for your company.
  - No extra Fee - Consolidated billing is offered at no additional cost.



# Exercise : Create new member account and apply SCP

- 1 Login into AWS Management account and go to AWS Organizations
- 2 Under Root OU -> Create a new OU (say Development)
- 3 Under Development OU -> Create a new AWS account (provide email id)
- 4 After account created successfully, login to new account in new browser private window with email id. You would have to choose forgot password option for first time login.
- 5 In the management account, go to SCP and create a new SCP policy which restricts launching only EC2 t2.micro instances. Refer to the policy document in the next slide.
- 6 Attach this policy to the Development OU that you created earlier. It should be automatically applied to the new AWS account that you created.
- 7 In the new AWS account, try to launch t2.large EC2 instance. Access should be denied. But now try launching t2.micro EC2 instance, it should be launched successfully.
- 8 If you do not need a new account that you created, then Close the account from management account screen.



Management account steps



Member account steps

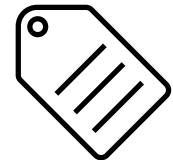


# Sample SCP policy:

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Deny",
    "Action": "ec2:RunInstances",
    "Resource": "arn:aws:ec2:*:*:instance/*",
    "Condition": {
      "StringNotEquals": {
        "ec2:InstanceType": "t2.micro"
      }
    }
  }
}
```

# AWS Organizations – Tag policies

- Used to standardize resource tagging across all AWS accounts in an organization
- Define rules for tag keys and values, including:
  - Required capitalization and case (for example, CostCenter vs costcenter)
  - Allowed values for specific tag keys
- Enforcement modes:
  - Enforced → Blocks noncompliant tagging on specified resource types
  - Monitoring only → Allows tagging but flags noncompliance
- Compliance can be viewed or fixed using:
  - AWS Resource Groups
  - Tag Editor
  - Tagging API
- Tag policies require AWS Organizations all features enabled
- Tag policies do not grant or deny service permissions (they only govern tagging)

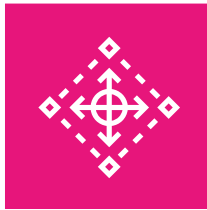


Key = Value

# IAM Resource Policies with aws:PrincipalOrgID

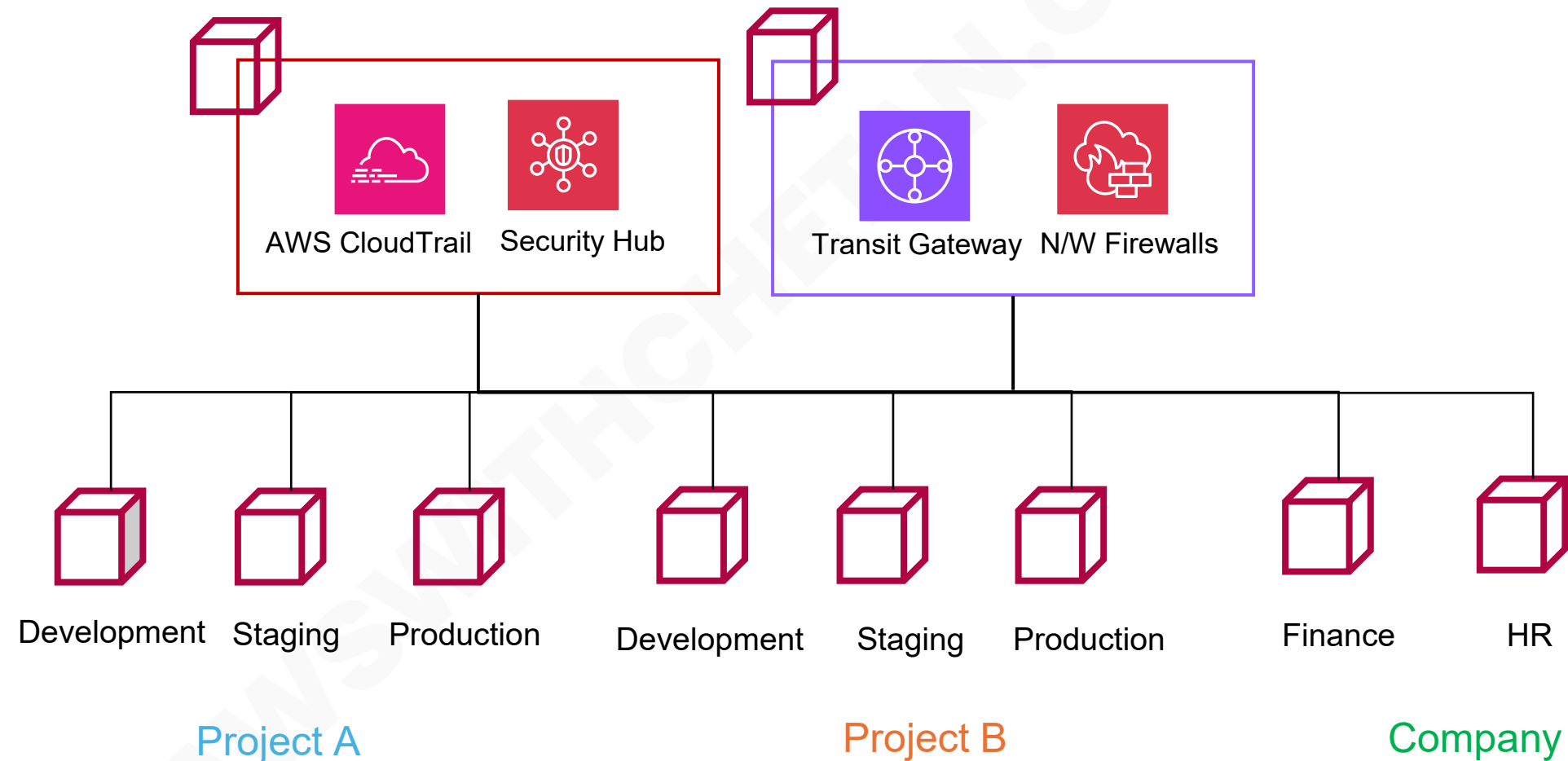
- AWS Organization has unique Organization ID
- This organization ID can be used in IAM resource-based policies
- It ensures that only principals belonging to a specific AWS Organization can access resources like S3 buckets, VPC endpoints, or KMS keys etc.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowGetObject",
      "Effect": "Allow",
      "Principal": {
        "AWS": [ "*" ]
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::finops-data/*",
      "Condition": {
        "StringEquals": {
          "aws:PrincipalOrgID": ["o-ea3x9t19cz"]
        }
      }
    }
  ]
}
```



# AWS Control Tower

# Multi-account guardrails and best practices



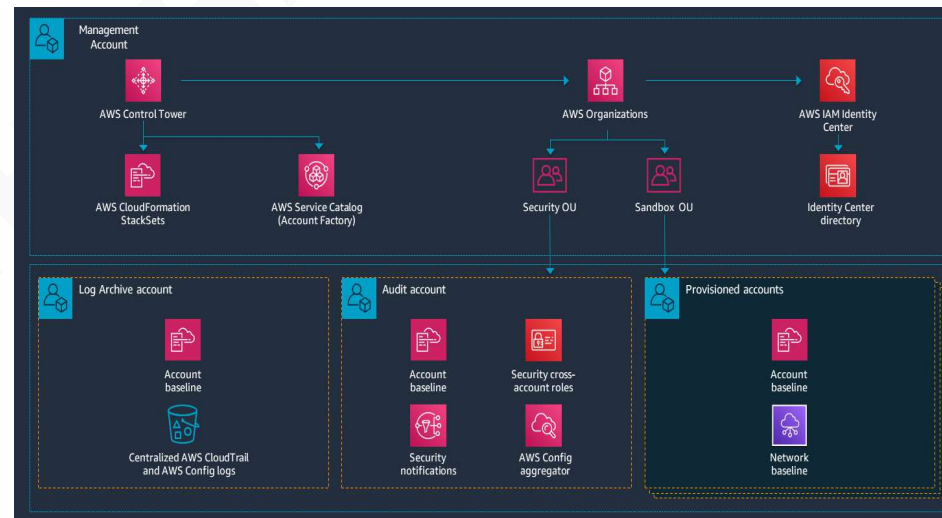
# Multi-account guardrails and best practices

- How to deploy this kind of Multi-account setup from scratch which provides systematic, structured way to create, manage and deploy AWS accounts, SCP policies, IAM users, create cloudtrail trails, config rules etc.?
- How to ensure that every new account provisioned is setup with these security best practices and guardrails?

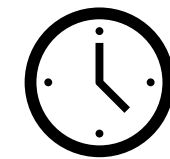
Enter..



AWS Control Tower



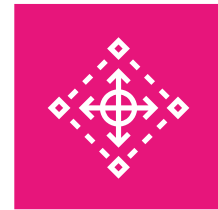
AWS Landing Zone



In less than an hour..

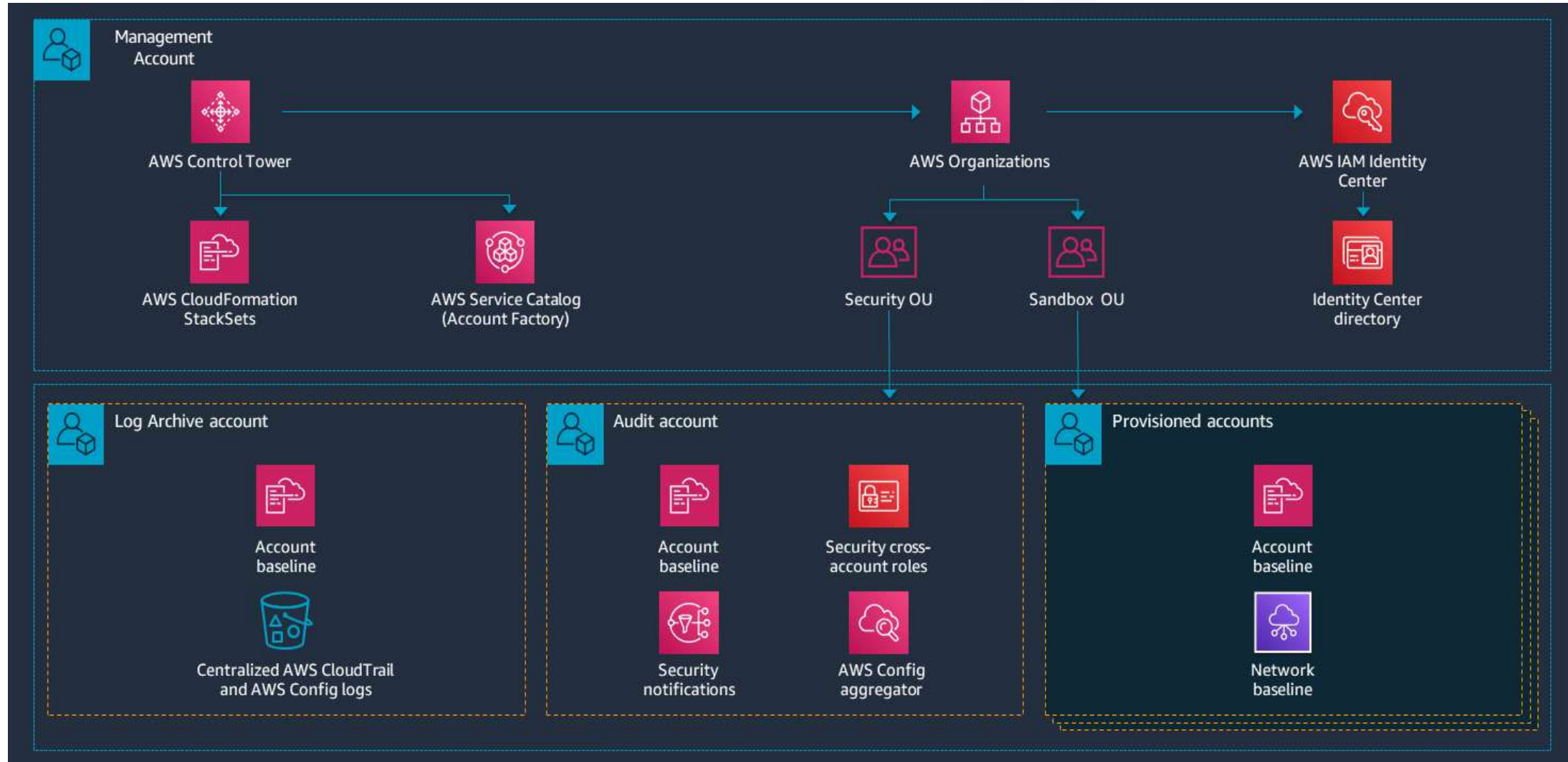
# AWS Control Tower

- **Landing Zone:** A well-architected, multi account-environment that's based on security and compliance best practices.
  - **Mandatory guardrails:** Disallow root access, disallow changes to encryption, enforce MFA, disallow public access to S3 buckets etc.
  - **Elective guardrails:** Disallow SSH/RDP from internet, Enforce tagging, Disallow RDS snapshot public sharing etc.
- Works with AWS Organizations, AWS IAM Identity Center, AWS CloudFormation & Service Catalog.
- **Account Factory:** A template for creating new accounts with pre-configured resources and controls e.g. Security groups, VPC, CloudTrail trail etc.
- **Dashboard:** Centralized view of entire Landing zone, compliance policies, non-compliant resources organized by accounts and Ous.



AWS Control Tower

# AWS Landing zone provisioned by AWS Control Tower



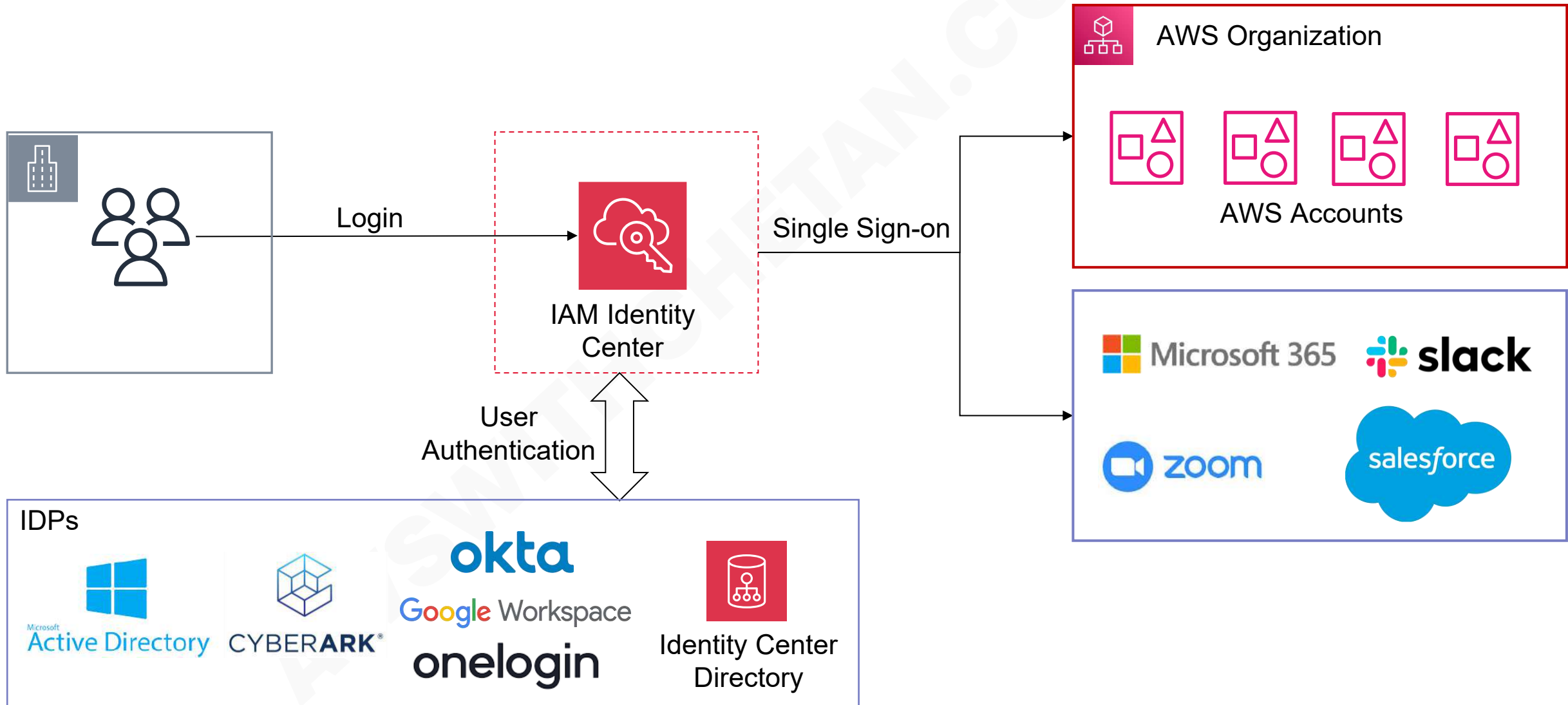
<https://docs.aws.amazon.com/prescriptive-guidance/latest/migration-aws-environment/building-landing-zones.html>





# IAM Identity Center

# AWS IAM Identity Center



# IAM Identity Center – Key Features

## User logins

- Centralized (SSO) login for all accounts in AWS Organization
- Centralized (SSO) login for all business applications

## Identity Provider (IdP)

- Built-in Identity Center Directory
- Integration with AWS Managed Microsoft Active Directory
- Integration with on-premises Microsoft Active Directory
- External IdPs via SAML2.0



AWS IAM Identity Center

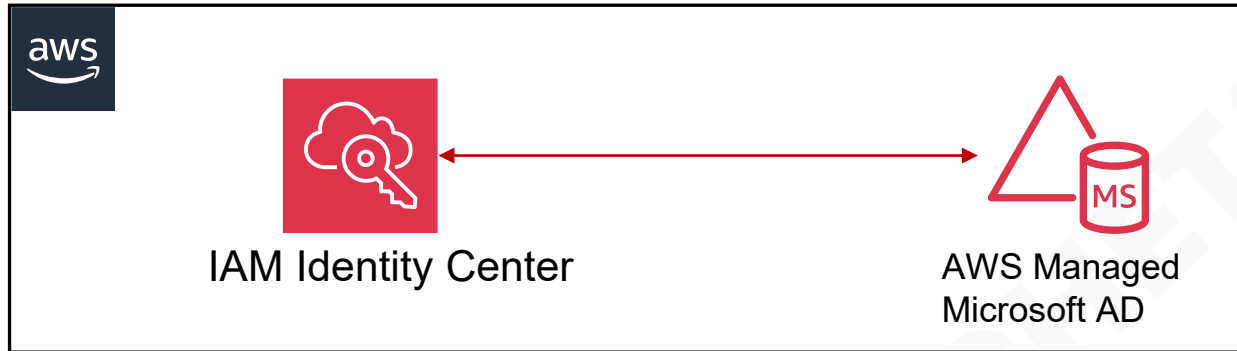
## Permission Sets

- Permission sets define what users can do in AWS accounts.
- Internally translated into IAM roles in each target account.
- Assigned to Users or Groups in Identity center.

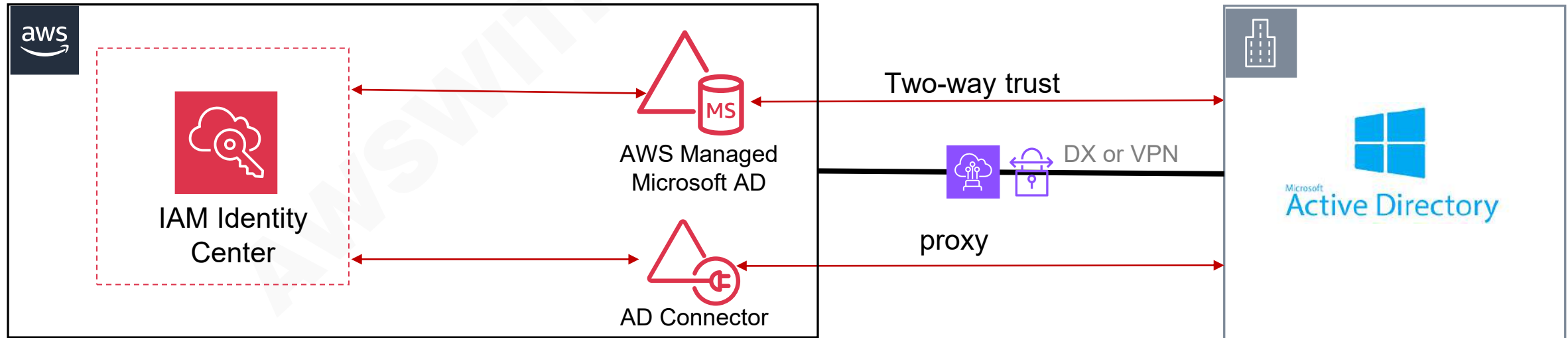
*In 2022, AWS Single Sign-On was renamed to AWS IAM Identity Center.*

# IAM Identity Center integration with Active Directory

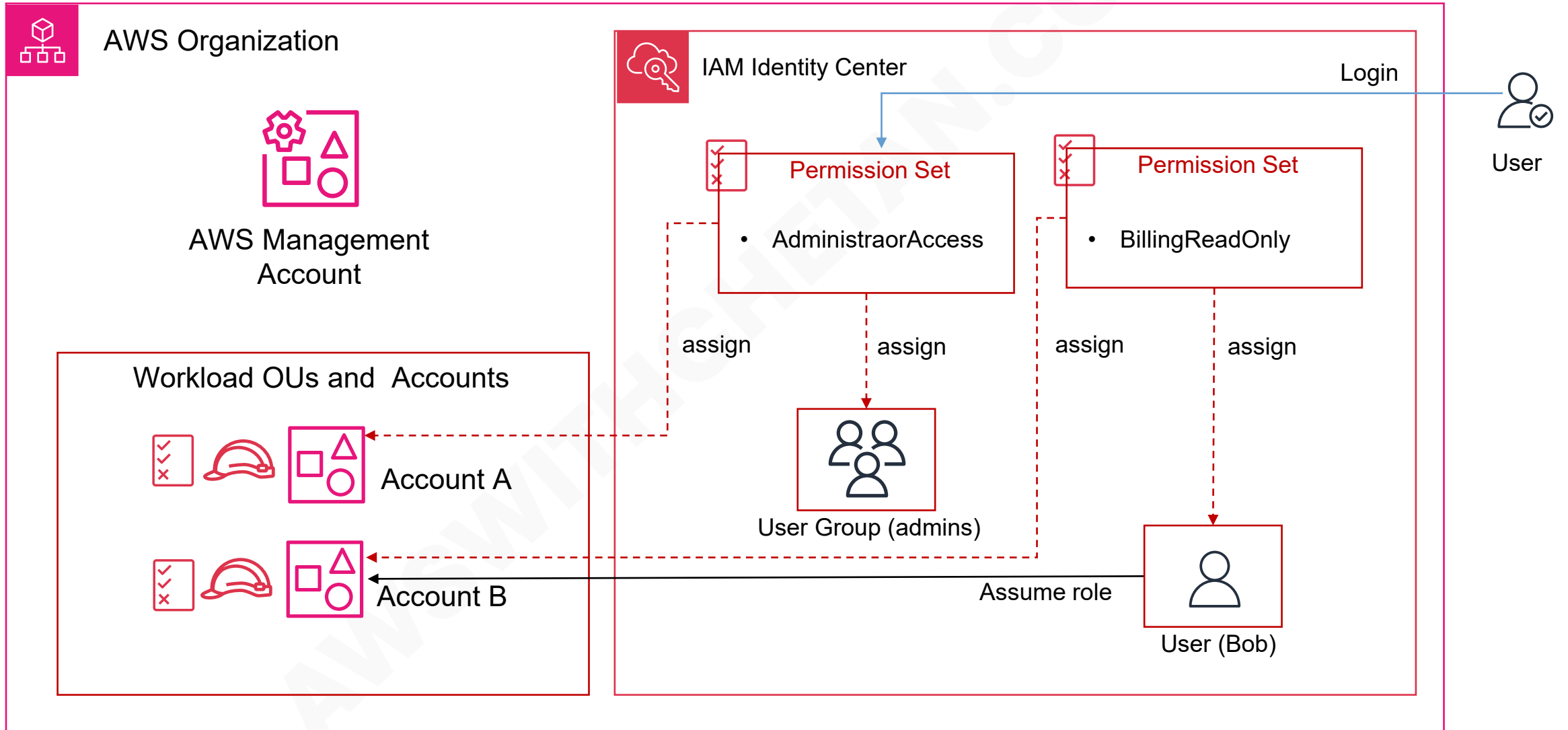
- Use AWS Managed Microsoft Active Directory (AD)



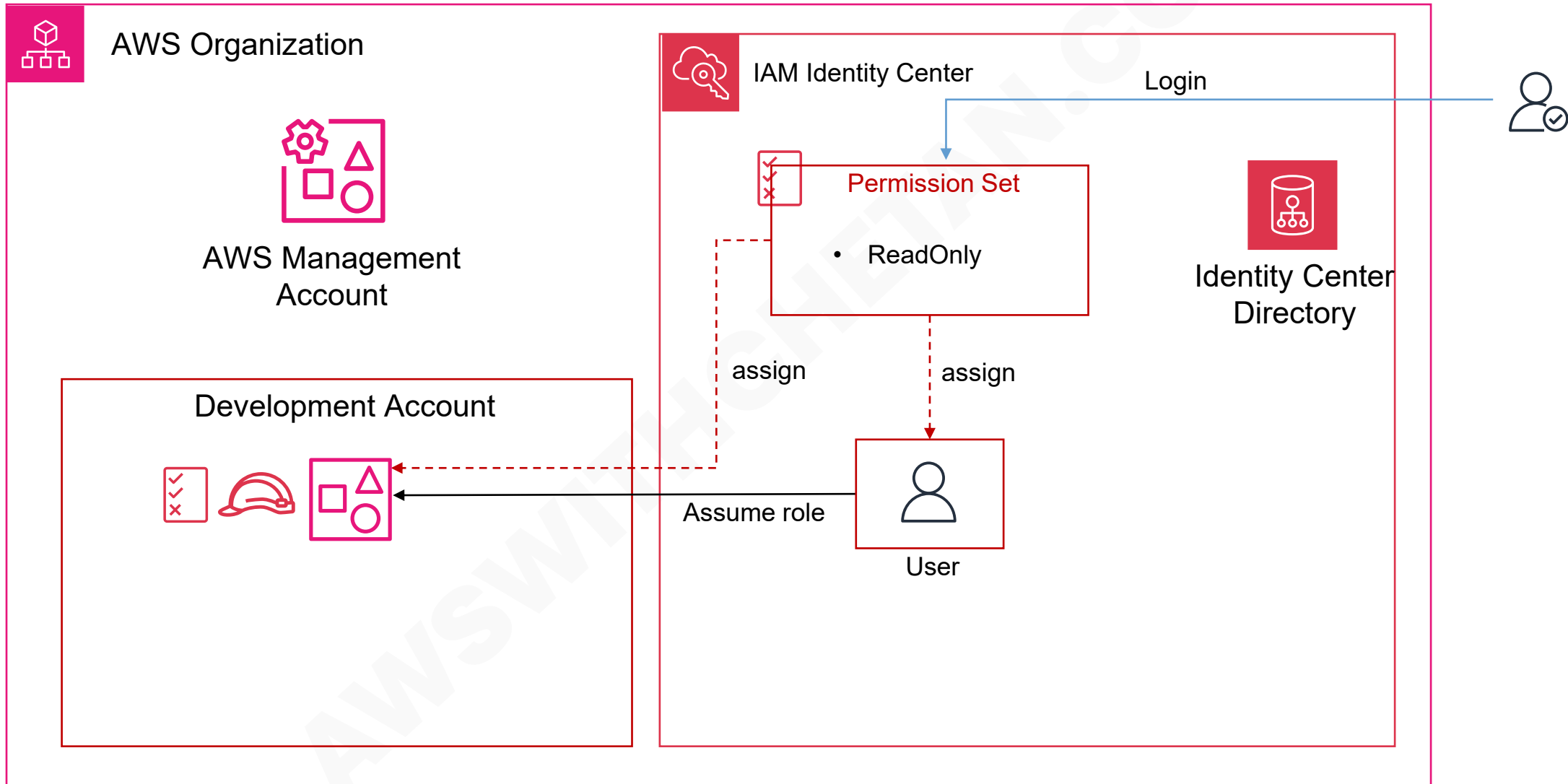
- Use Self-managed Microsoft Active Directory (AD)



# How Permission sets work?



# Exercise: Login to AWS account using IAM Identity Center

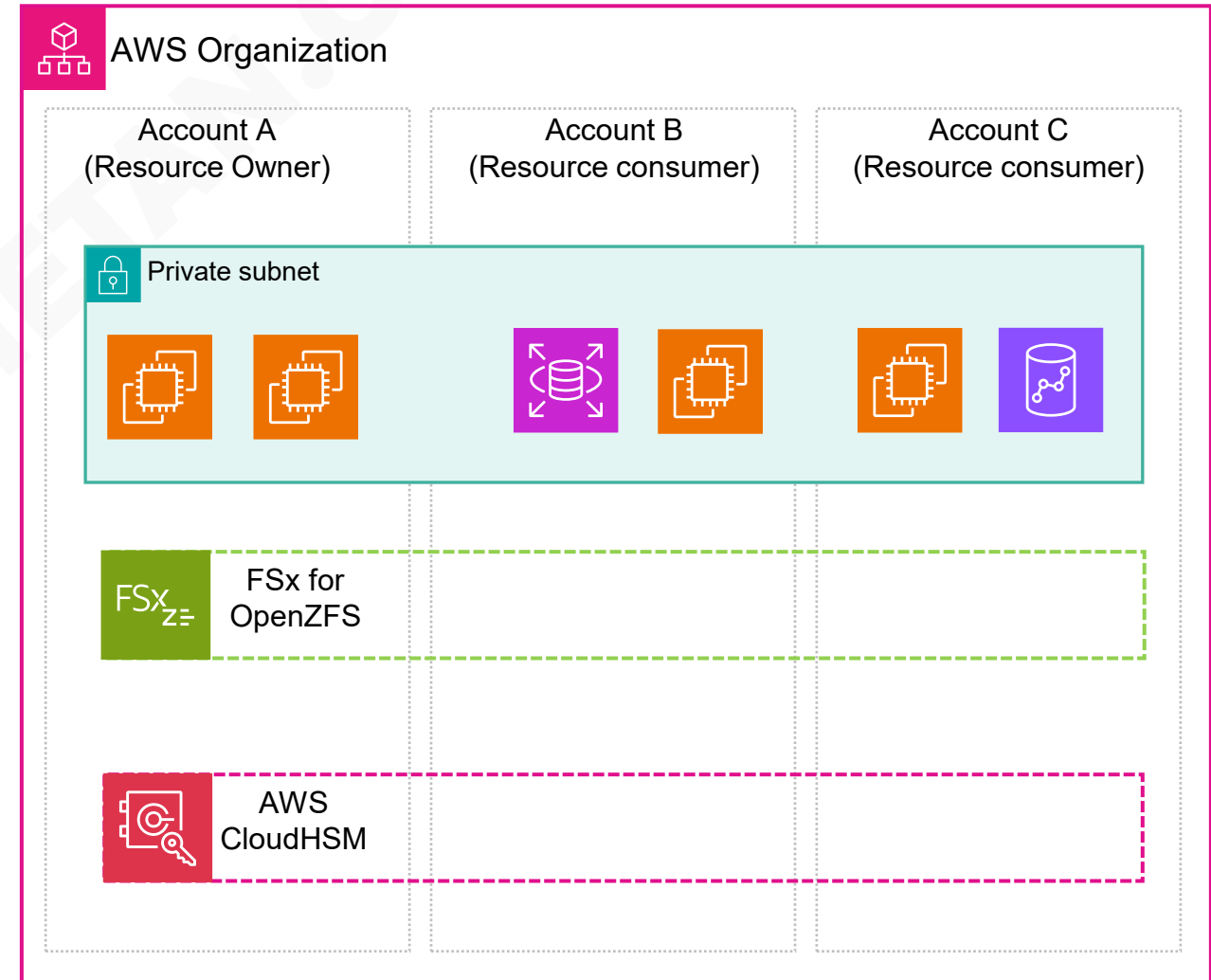




# AWS Resource Access Manager

# AWS Resource Access Manager (RAM)

- Share AWS resources with other AWS Accounts
- In AWS organization, resource can be shared with a single account or OU or all accounts
- Resource can also be shared with AWS account outside of AWS organization
- Supports AWS services and resources like VPC subnets, Transit gateway, Route53, EC2 dedicated host, CloudHSM and more..





# IAM Advanced

# IAM Roles and AWS STS

- **IAM roles do not have long-term credentials.**
- When a role is assumed, AWS STS (Security Token Service) issues temporary security credentials for that role.
- These credentials include:
  - Access key ID
  - Secret access key
  - **Session token**
- The credentials are time-limited and automatically expire (typically 15 min – 12 hours).
- AWS STS provides APIs such as AssumeRole, AssumeRoleWithSAML, and AssumeRoleWithWebIdentity which provides temporary credentials.
- STS APIs to get temporary credentials:
  - [AssumeRole](#) - For AWS-to-AWS (EC2/Lambda accessing S3 etc.)
  - [AssumeRoleWithSAML](#) - For Enterprise SAML 2.0 (Login with AD FS, Okta, Azure AD etc.)
  - [AssumeRoleWithWebIdentity](#) - For Web/mobile identity (Login with Google, Facebook, Cognito etc.)

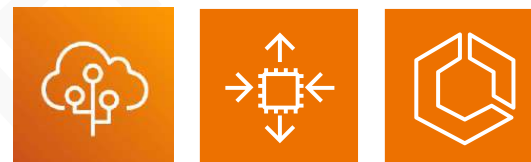


# IAM Role types

- Service IAM Role



- AWS Service-Linked IAM Role



- Cross-account IAM Role



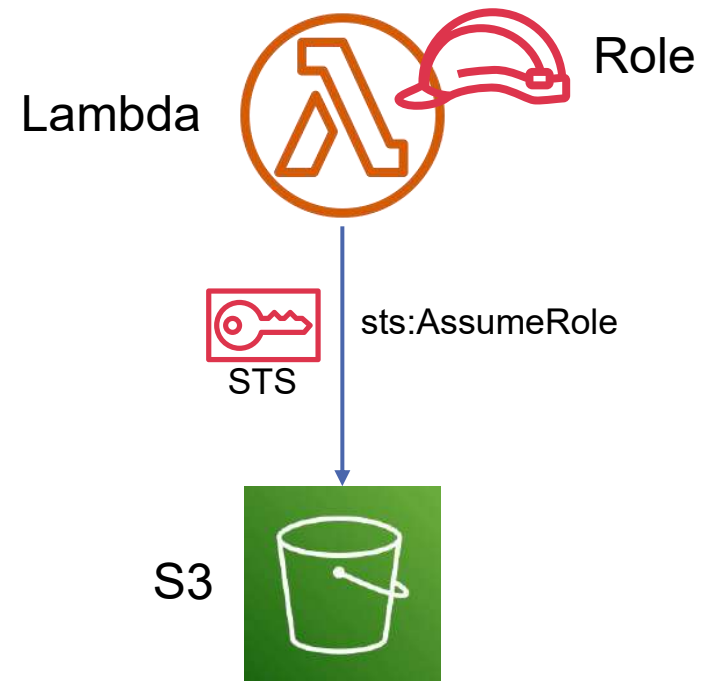
- Federated IAM Role



# IAM Role types

- **AWS Service Role**

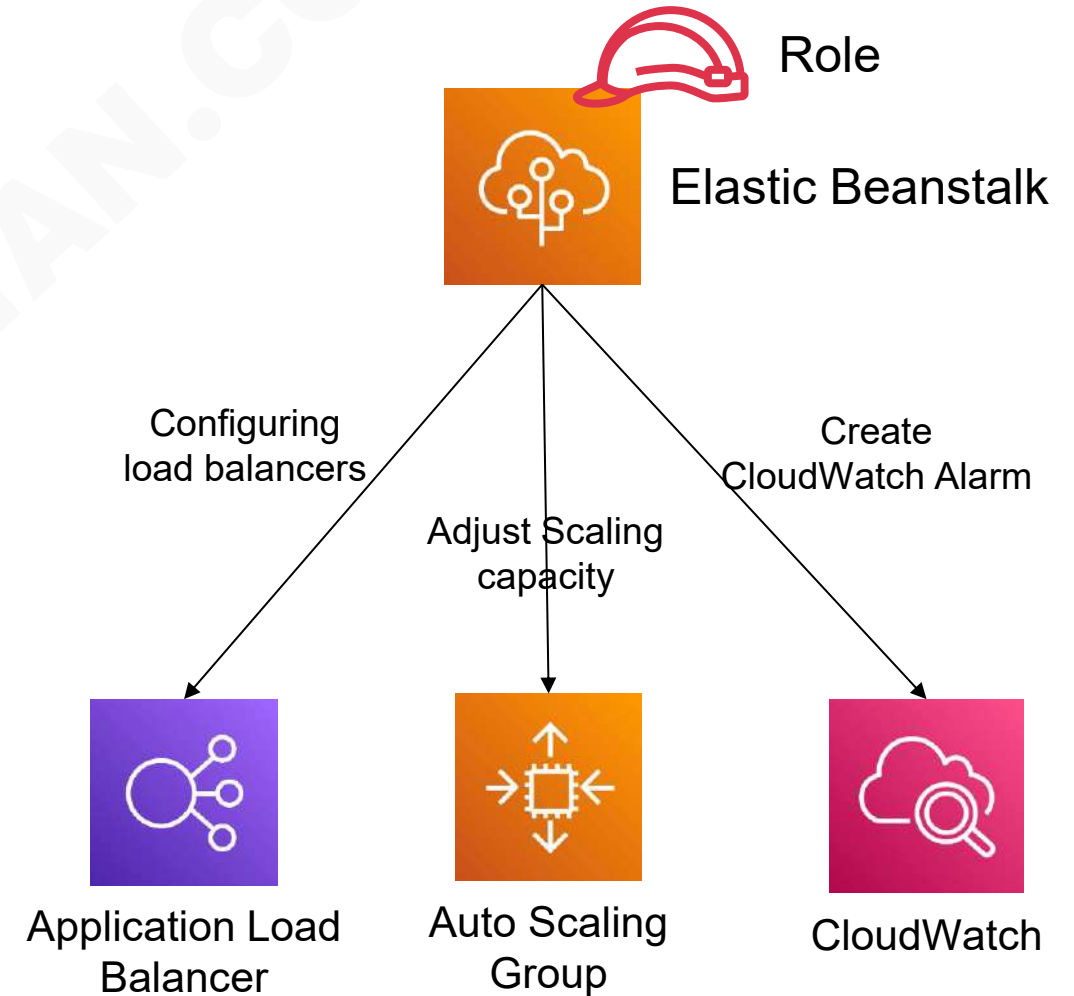
- A role that an AWS service assumes to perform actions on your behalf.
- Used when an AWS service (like EC2, Lambda, or ECS) needs permissions to access other AWS resources.
- You define what the service can do by attaching policies to the role.
- Examples:
  - EC2 instance reading/writing to S3
  - Lambda writing to S3, CloudWatch Logs
  - Lambda reading/writing to DynamoDB



# IAM Role types

- **Service-Linked Role**

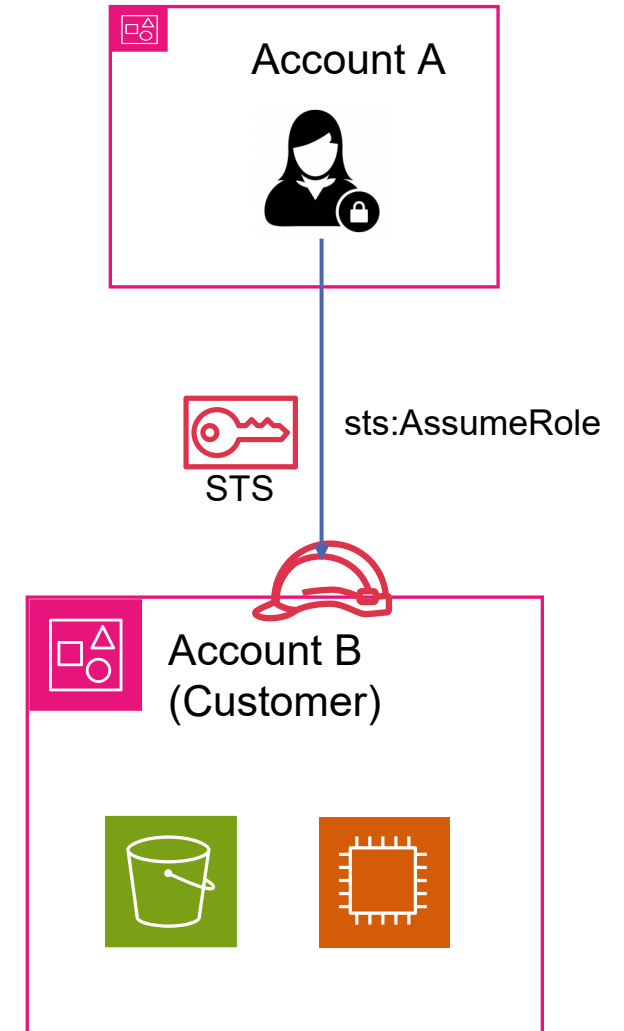
- A special type of service role that is created and **managed by AWS** for specific services.
- Automatically includes predefined permissions needed by that service.
- You can view or delete it but cannot modify its policy.
- Example services that create these:
  - AWS Config
  - Amazon RDS
  - Elastic Beanstalk



# IAM Role types

- **Cross-account Role**

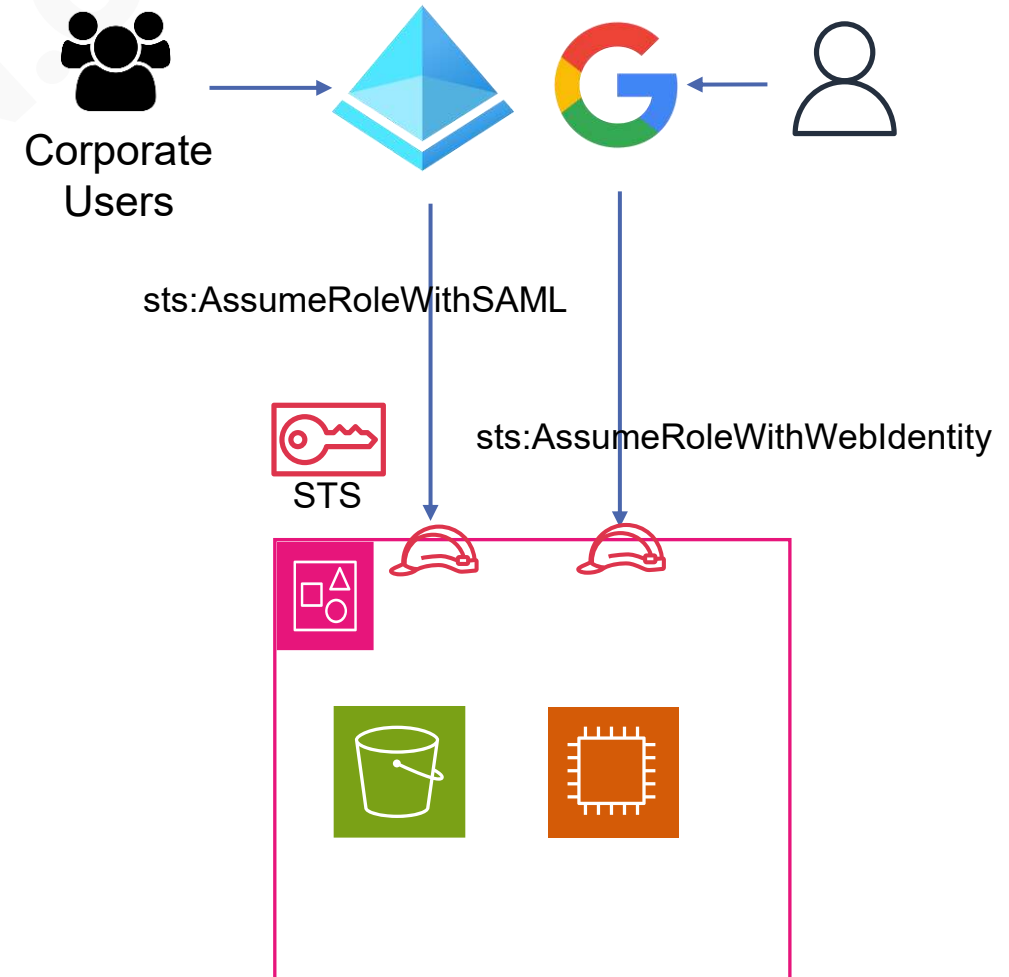
- Allows entities in another AWS account to assume the role and access resources in your account.
- Requires a trust policy specifying the external account or principal.
- Example:
  - Account A's user Ben assumes a role in Account B to read from an S3 bucket.



# IAM Role types

- **Federated Role**

- Used when external identities (e.g., Active Directory, Google Workspace, Okta, etc.) need temporary access to AWS.
- Users authenticate with an identity provider (IdP) using SAML 2.0 or OIDC, then assume the role through STS.
  - AssumeRoleWithSAML
  - AssumeRoleWithWebIdentity
- Enables SSO (Single Sign-On) without creating IAM users.



# IAM Federated Role vs IAM Identity Center

## IAM Federated vs IAM Identity Center

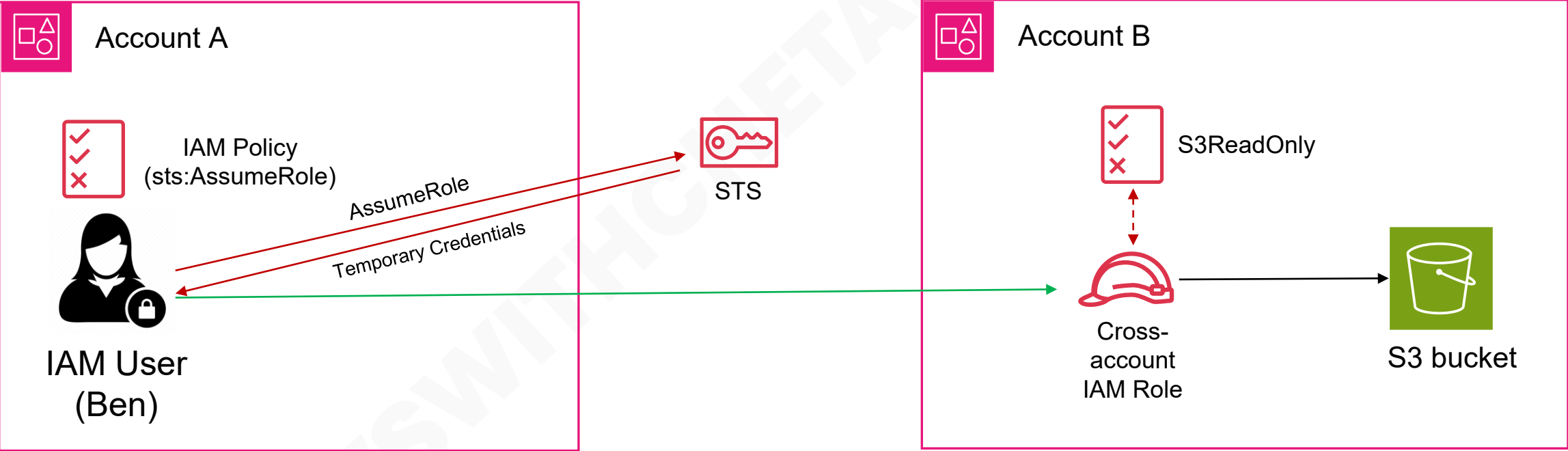
- For using IAM Federated role, you use AWS STS that issues temporary credentials as per role for a given AWS account.
- IAM Identity Center is a managed SSO service that uses STS internally to provide centralized, organization-wide access across multiple AWS accounts.
- With STS, you manage IAM roles and federation manually; with IAM Identity Center, AWS manages role creation and assignment using permission sets.



IAM Identity Center



# Exercise: IAM cross-account role



# Exercise: Cross-account access

## Pre-requisites:

1. There should be two AWS Accounts. You may work with your friend for Account B if don't have two accounts.
2. In Account B, there should be existing S3 buckets. We will just verify the list bucket permissions.
- 1 In Account B, login as an admin user and create cross-account IAM Role (say CrossAccountRoleForAccountA) and associate **AmazonS3ReadOnlyAccess** policy. Provide the Account A ID while creating this cross-account role.
- 2 In Account A, login as an admin user and create an IAM Policy to grant user permission to AssumeRole in Account B and attach this policy to any of the IAM user in Account A (Ben)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::<AccountB>:role/CrossAccountRoleForAccountA"
    }
  ]
}
```

IAM Policy

# Exercise: Verify cross-account access through AWS console

- 1 Login in Account A as a normal IAM user (Ben) -> Go to Account ID (right corner) -> Add Session -> Switch Role
- 2 Provide the details of Account B ID and cross-account Role name (CrossAccountRoleForAccountA) and select the color. If successful, you will see Ben user logged into Account B. Verify the user details – It should show federated user.
- 3 Go to S3 service and check if you are able to see the existing buckets in Account B.

# Exercise: Verify cross-account access through AWS CLI

- 1 From your command line (for user Ben in Account A), run the following command to AssumeRole in Account B and get the temporary IAM credentials.

```
aws sts assume-role --role-arn arn:aws:iam::<AccountB>:role/CrossAccountRoleForAccountA -  
-role-session-name AccountBSession
```

The AWS STS will return the Temporary security credentials including Access Key ID, Secret Access Key and Session Token

- 2 Set the environment variables for temporary credentials

## Linux or mac:

```
export AWS_ACCESS_KEY_ID=<access key id>  
export AWS_SECRET_ACCESS_KEY=<secret access key>  
export AWS_SESSION_TOKEN=<session token>
```

**Windows CMD:** `$env:` for windows powershell and `set` of windows cmd

```
set AWS_ACCESS_KEY_ID=<access key id>  
set AWS_SECRET_ACCESS_KEY=<secret access key>  
set AWS_SESSION_TOKEN=<session token>
```

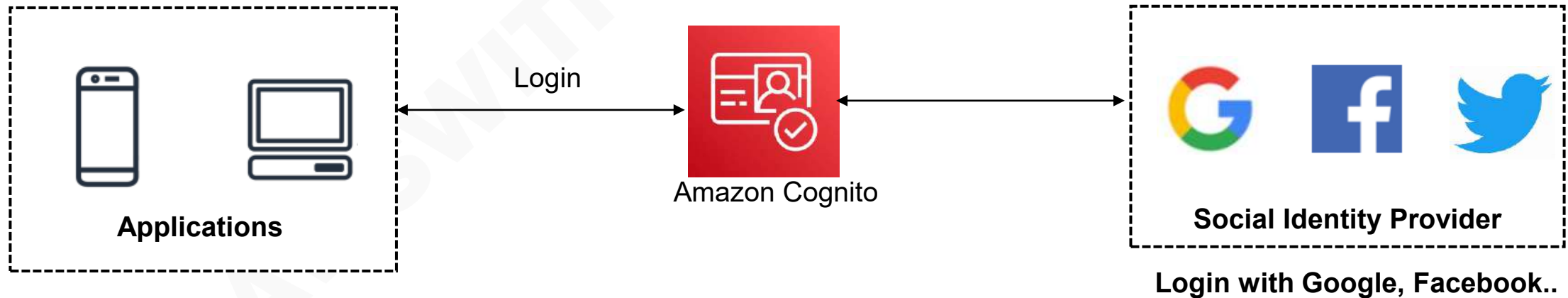
# Exercise: Verify cross-account access through AWS CLI

- 3 Try to run the S3 real-only commands. These will be executed against Account B resources

```
aws s3 ls //should list the buckets in Account B
```

# Amazon Cognito

- An identity platform for web and mobile apps users.
- Instead of creating them an IAM user, you create a user in Cognito.
- Create a **User pool** when you want to authenticate and authorize users to your app or API.
- Create an **Identity pool** when you want to authorize authenticated or anonymous users to access your AWS resources.



# AWS Billing and Cost management

# How to estimate, view and analyze AWS cost?

- How to estimate cost of AWS services for your architecture?

AWS Pricing Calculator

<https://calculator.aws/>



- View overall AWS monthly Bills

AWS Billing Dashboard

Billing and Cost Management console



- Analyze the cost per service, region etc. and also see forecasted usage

Cost Explorer

Billing and Cost Management console



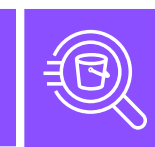
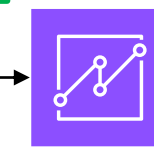
- Granular level report of cost breakdowns. Use your BI tools to analyze cost.

Data Export  
Legacy: Cost and Usage Report 2.0

Download Legacy CUR from S3 Bucket



Visualize in QuickSight



Query

QuickSight Athena Redshift

- How to track the free tier usage?

Free tier usage dashboard

Billing and Cost Management console





# AWS Pricing calculator

←

→

↺

calculator.aws/#/estimate

🔍

☆

🏠

👤

⋮

aws pricing calculator

Feedback

Language: English ▼

Contact Sales

Create an AWS Account

AWS Pricing Calculator

>

My Estimate

My Estimate

Edit

Export ▼

Share

Estimate summary

Info

Upfront cost

0.00 USD

Monthly cost

361.98 USD

Total 12 months cost

4,343.76 USD

Includes upfront cost

Getting Started with AWS

Get started for free

Contact Sales

My Estimate

Duplicate

Delete

Move to

Create group

Add support

Add service

🔍

Find resources

<

1

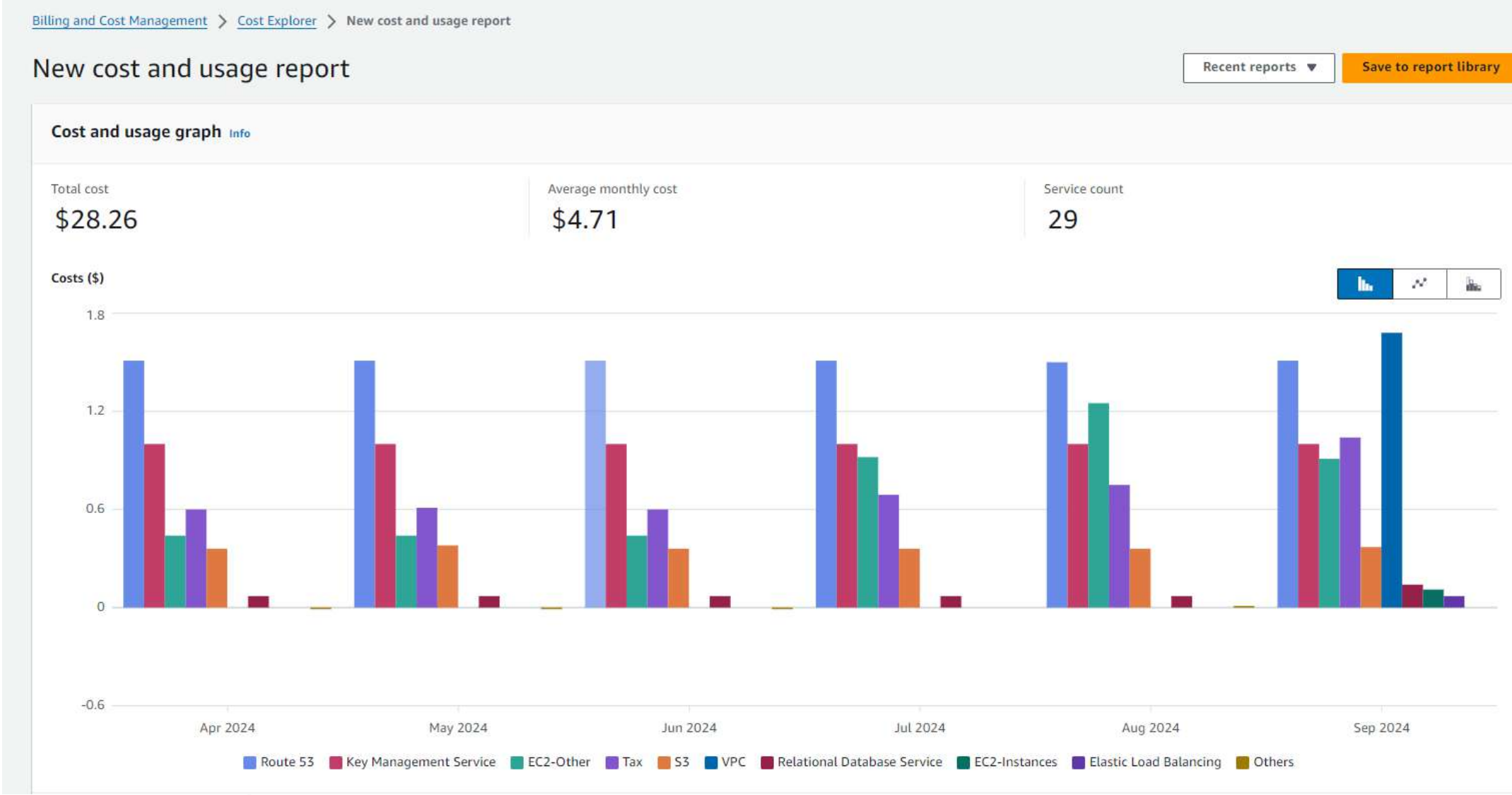
>

⚙️

| <input type="checkbox"/> | Service Name ▼         | Status ▼ | Upfront cost ▼ | Monthly cost ▼ | Description ▼ | Region         |
|--------------------------|------------------------|----------|----------------|----------------|---------------|----------------|
| <input type="checkbox"/> | Amazon EC2             | -        | 0.00 USD       | 163.18 USD     | -             | US East (Ohio) |
| <input type="checkbox"/> | Amazon Simple Stora... | -        | 0.00 USD       | 32.54 USD      | -             | US East (Ohio) |
| <input type="checkbox"/> | Amazon RDS for Post... | -        | 0.00 USD       | 166.26 USD     | -             | US East (Ohio) |



# AWS Cost explorer



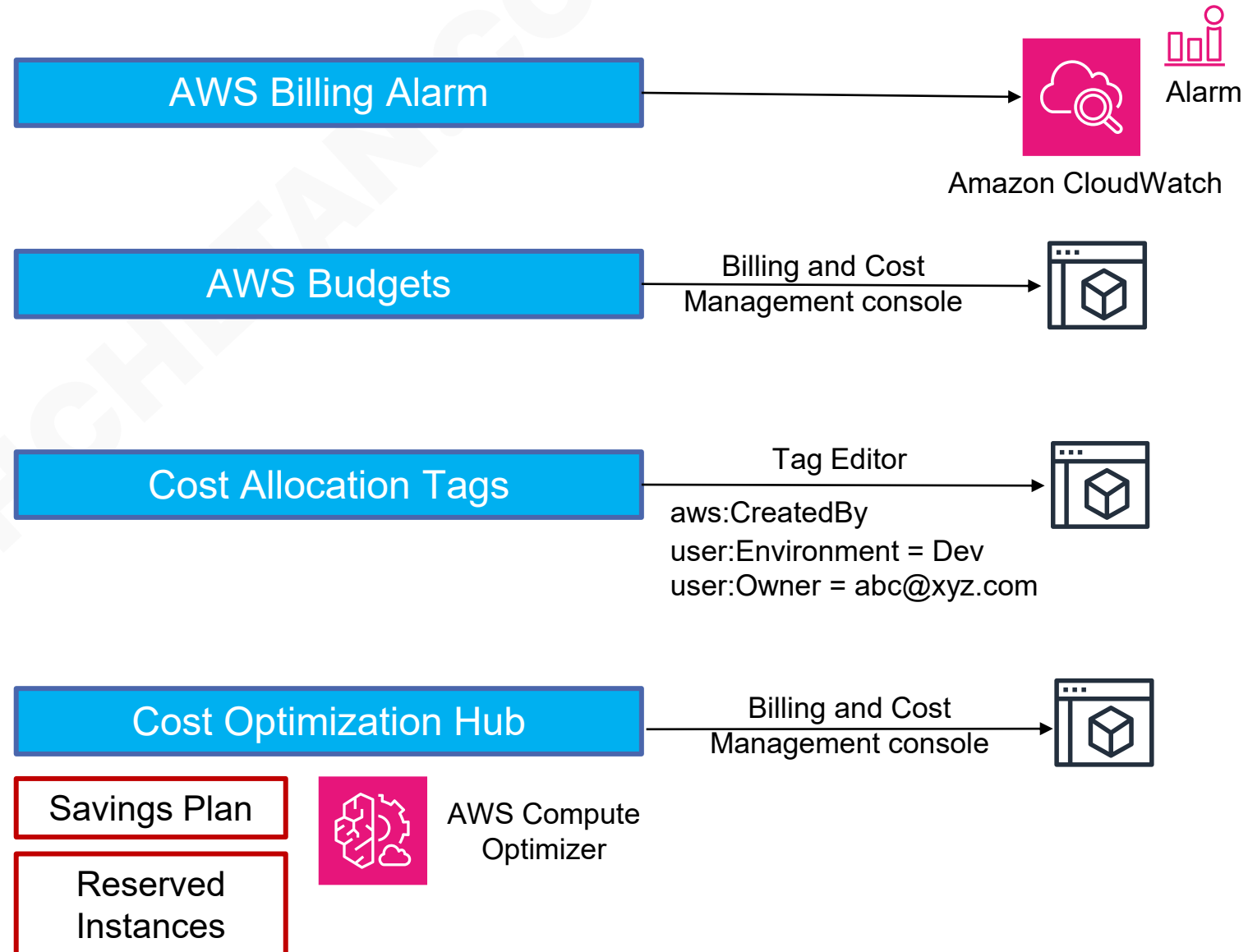
# Free tier usage dashboard

- [AWS Billing and Cost Management console](#) -> [Cost Analysis](#) -> [Free tier](#)

| Free Tier offers in use (39)                   |                                                                                                                     |                              |                                                                                                 |                               |                        |  |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------|------------------------|--|
| <input type="text" value="Find service name"/> |                                                                                                                     |                              | <span>&lt;</span> <span>1</span> <span>2</span> <span>3</span> <span>4</span> <span>&gt;</span> |                               |                        |  |
| Service                                        | AWS Free Tier usage limit                                                                                           | Current usage                | Forecasted usage                                                                                | MTD actual usage %            | MTD forecasted         |  |
| Amazon Redshift                                | 750.0 Hrs for free per month during a short-term trial as part of AWS Free Usage Tier (Global-Node:dc2.large)       | 664 Hrs                      | 709 Hrs                                                                                         | <div><div></div></div> 88.47% | <div><div></div></div> |  |
| Amazon DynamoDB                                | 18600.0 ReadCapacityUnit-Hrs are always free per month as part of AWS Free Usage Tier (APS3-ReadCapacityUnit-Hrs)   | 13,300 ReadCapacityUnit-Hrs  | 14,217 ReadCapacityUnit-Hrs                                                                     | <div><div></div></div> 71.51% | <div><div></div></div> |  |
| Amazon DynamoDB                                | 18600.0 WriteCapacityUnit-Hrs are always free per month as part of AWS Free Usage Tier (APS3-WriteCapacityUnit-Hrs) | 10,640 WriteCapacityUnit-Hrs | 11,374 WriteCapacityUnit-Hrs                                                                    | <div><div></div></div> 57.20% | <div><div></div></div> |  |
| Amazon DynamoDB                                | 18600.0 ReadCapacityUnit-Hrs are always free per month as part of AWS Free Usage Tier (USW2-ReadCapacityUnit-Hrs)   | 9,975 ReadCapacityUnit-Hrs   | 10,663 ReadCapacityUnit-Hrs                                                                     | <div><div></div></div> 53.63% | <div><div></div></div> |  |
| Amazon DynamoDB                                | 18600.0 WriteCapacityUnit-Hrs are always free per month as part of AWS Free Usage Tier (USW2-                       | 9,975 WriteCapacityUnit-Hrs  | 10,663 WriteCapacityUnit-Hrs                                                                    | <div><div></div></div> 53.63% | <div><div></div></div> |  |

# How to track and control cost?

- Notify when total usage exceeds certain threshold e.g. > \$100 for current month
- Set cost budgets for overall usage or AWS Services e.g. \$1000 for EC2, \$500 for S3 and get notified when exceeded
- Want to track cost by environments, projects, services, users or based on different dimensions?
- Want to get recommendations for purchasing Savings plan, RIs, right sizing and configuration of AWS resource (e.g. EC2, EBS etc.) ?

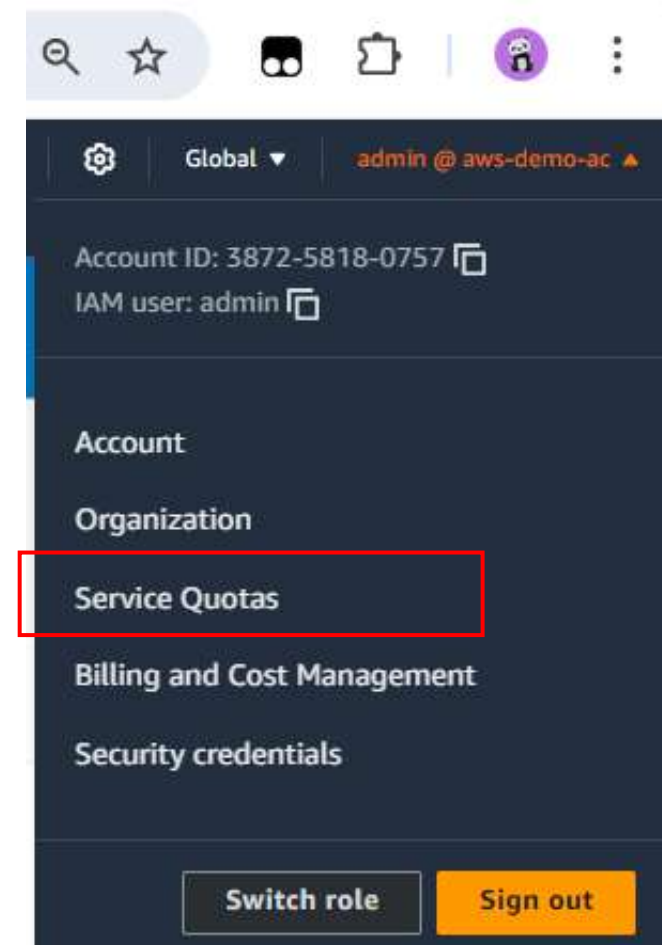


# Exercise: AWS Billing Alarm

DO THIS IN **N.VIRGINIA** (us-east-1) REGION

Set a Billing alert so that you get notified when your AWS usage bill exceeds some threshold (say > \$5/month)

1. Billing and Cost Management -> Billing Preferences -> Alert preferences -> Edit -> Receive CloudWatch Billing Alerts -> Save
2. In Amazon SNS -> Create a new topic and subscribe with your email id
3. In Amazon CloudWatch -> Go to -> Alarms -> Billing -> Create alarm
  - Metric Name: EstimatedCharges
  - Currency: USD
  - Statistic: Maximum
  - Period: 6 hours
  - Threshold Type: Static
  - Whenever EstimatedCharges is: **Greater**, than -> **5**
4. Alarm State trigger: In alarm -> Send a notification to the following SNS topic -> Select an existing SNS topic -> Select the topic you created earlier -> next -> Alarm name: BillingAlarm5USD -> next -> Create alarm





# AWS Budgets

AWS Budgets

Download CSV

Create budget

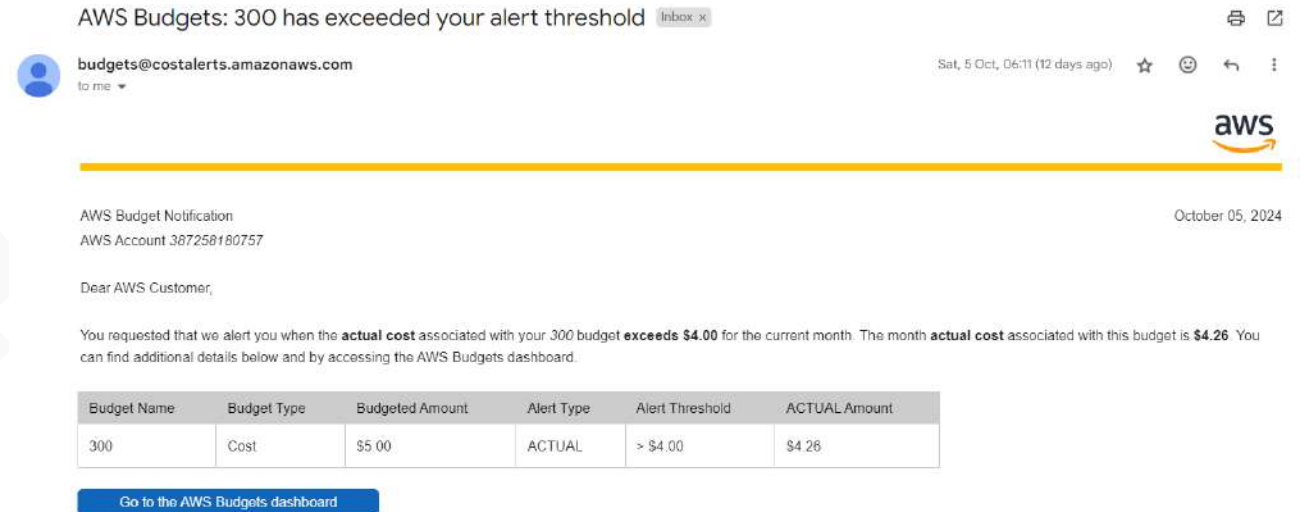
| All budgets (8)                                   | Cost budgets (6) | Usage budgets (2) | Reservation budgets (0) |                   |                               |                               |     |  |
|---------------------------------------------------|------------------|-------------------|-------------------------|-------------------|-------------------------------|-------------------------------|-----|--|
| Budget name                                       | Budget type      | Current           | Budgeted                | Forecasted        | Current vs. budgeted          | Forecasted vs. budgeted       |     |  |
| <a href="#">Project Nemo Cost Budget</a>          | Cost             | \$42.48           | \$40.00                 | \$42.48           | <div><div></div></div> 106.2% | <div><div></div></div> 106.2% | ... |  |
| <a href="#">Eastern US Regional Budget</a>        | Cost             | \$82.95           | \$100.00                | \$93.32           | <div><div></div></div> 82.95% | <div><div></div></div> 93.32% | ... |  |
| <a href="#">Total Monthly Cost Budget</a>         | Cost             | \$138.80          | \$200.00                | \$163.76          | <div><div></div></div> 69.4%  | <div><div></div></div> 81.88% | ... |  |
| <a href="#">Total EC2 Cost Budget</a>             | Cost             | \$135.34          | \$200.00                | \$159.05          | <div><div></div></div> 67.67% | <div><div></div></div> 79.53% | ... |  |
| <a href="#">Loft</a>                              | Cost             | \$40.29           | \$65.00                 | \$50.53           | <div><div></div></div> 61.99% | <div><div></div></div> 77.74% | ... |  |
| <a href="#">Monthly DataTransfer Usage Budget</a> | Usage            | 2.19 GB           | 4 GB                    | 2.75 GB           | <div><div></div></div> 54.65% | <div><div></div></div> 68.85% | ... |  |
| <a href="#">S3 Usage Budget</a>                   | Usage            | 2,608 Requests    | 5,500 Requests          | 3,309.22 Requests | <div><div></div></div> 47.42% | <div><div></div></div> 60.17% | ... |  |
| <a href="#">Quarterly Budget</a>                  | Cost             | \$131.41          | \$550.00                | \$473.95          | <div><div></div></div> 23.89% | <div><div></div></div> 86.17% | ... |  |

<https://aws.amazon.com/blogs/aws-cloud-financial-management/beginners-guide-to-aws-cost-management/>

# Exercise: AWS Budget

Go to Billing and Cost Management -> Budgets

- Use a template -> Monthly cost budget
- Provide Budget name and budget value in USD (e.g. 5)
- Provide the email id to which you should receive an email when usage exceeds the budgeted value
- Create budget



Email sent by AWS



# AWS Cost Optimization Hub – Savings plan

[Billing and Cost Management](#) > [Savings Plans](#) > [Recommendations](#)

[Home](#)  
[Cost Explorer](#)  
[Saved reports](#)  
[Budgets](#)  
[Recommendations](#)  
[Savings Plans](#)  
[Overview](#)  
[Inventory](#)  
[Recommendations](#)  
[Purchase Savings Plans](#)  
[Utilization Report](#)  
[Coverage Report](#)  
[Reservations](#)  
[Overview](#)  
[Recommendations](#)  
[Utilization Report](#)  
[Coverage Report](#)  
[My Billing Dashboard](#)

Recommendation options

Savings Plans type

☒ Compute
 ☐ EC2 Instance

Savings Plans term

☒ 1-year
 ☐ 3-year

Payment option

☒ All upfront
 ☐ Partial upfront
 ☐ No upfront

Based on the past

☐ 7 days
 ☒ 30 days
 ☐ 60 days

Recommendation: Purchase a Compute Savings Plan at a commitment of \$0.30/hour

You could save an estimated \$48 monthly by purchasing the recommended Compute Savings Plan.

Based on your past 30 days of usage, we recommend purchasing a Savings Plan with a commitment of **\$0.30/hour** for a **1-year term**. With this commitment, we project that you could save an average of **\$0.07/hour** - representing a **14%** savings compared to On-Demand. To account for variable usage patterns, this recommendation maximizes your savings by leaving an average **\$0.09/hour** of On-Demand spend.

| Before recommended purchase                                                                                                                                                                         | After recommended purchase (based on your past 30 days of usage)                                                                                                                        |                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>Monthly On-Demand spend ⓘ</div> <div><b>\$329</b> (\$0.45/hour)</div> <div>Your estimated On-Demand spend based on your usage over the past 30 days (including all active Savings Plans)</div> | <div>Estimated monthly spend ⓘ</div> <div><b>\$281</b> (\$0.39/hour)</div> <div>Your recommended \$0.30/hour Savings Plans commitment + an average \$0.09/hour of On-Demand spend</div> | <div>Estimated monthly savings ⓘ</div> <div><b>\$48</b> (\$0.07/hour)</div> <div>14% monthly savings over On-Demand<br/>\$329 - \$281 = \$48</div> |

This recommendation examines your usage over the past 30 days (including your existing Savings Plans and EC2 Reserved Instances) and calculates what your costs would have been had you purchased the recommended Savings Plans. See applicable rates for Savings Plans [here](#). To generate this recommendation, AWS simulates your bill for different commitment amounts and recommends the commitment amount that provides the greatest estimated savings. [Learn more](#)

Recommended Compute Savings Plans

[Download CSV](#)
[Add selected Savings Plan\(s\) to cart](#)

| ✕ Term                                     | Payment option | Recommended commitment | Estimated hourly savings ⬆ |
|--------------------------------------------|----------------|------------------------|----------------------------|
| <input checked="" type="checkbox"/> 1-year | All upfront    | \$0.30/hour            | \$0.07 (14%)               |

Save with Savings Plans

Savings Plans offer discounts in exchange for an hourly commitment to compute usage. Savings Plans discounts

<https://aws.amazon.com/blogs/aws-cloud-financial-management/beginners-guide-to-aws-cost-management/>

# AWS Cost Optimization Hub - Reservation

[Billing and Cost Management](#) > [Reservations](#) > Recommendations

[Settings](#)

\$3,020

Estimated Annual Savings\*

28%

Savings vs. On-Demand

4

Purchase Recommendations

Based on your past 60 days of EC2 usage, we have identified **4 one-year, partial-upfront, standard RI purchase recommendations** to save an estimated **\$3,020 annually**, representing a savings of **28% versus on-demand costs**. You can take action on these recommendations in the [EC2 Reservation Purchase Console](#).

Generate recommendations based on:

All accounts

Individual accounts

Sort by:

Monthly Estimated Savings

Download CSV

| Purchase Recommendations (4)                                                                                                                                                                                                                                                                                                                                                      | Details                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Buy 1 g3s.xlarge reserved instance</b></p> <p>US West (Oregon)   Windows (Amazon VPC)   Shared</p> <p><i>Based on your past 60 days of on-demand usage, we recommend purchasing 1 g3s.xlarge reserved instance.</i></p> <p><a href="#">View Associated EC2 Usage</a></p>                                                                                                    | <p><b>\$163.92 monthly savings</b></p> <p>Upfront Cost: \$3,105.00</p> <p>Recurring Monthly Cost: \$259.15</p> <p>Expected RI Utilization: 100%</p> |
| <p><b>Buy 3 m5.large reserved instances</b> <span>Size flexible**</span></p> <p>US East (N. Virginia)   Linux/UNIX   Shared</p> <p><i>Based on your past 60 days of on-demand usage, we recommend purchasing 3 m5.large reserved instances to cover 12 normalized units per hour of m5 family usage to maximize savings.</i></p> <p><a href="#">View Associated EC2 Usage</a></p> | <p><b>\$82.73 monthly savings</b></p> <p>Upfront Cost: \$768.00</p> <p>Recurring Monthly Cost: \$63.51</p> <p>Expected RI Utilization: 100%</p>     |
| <p><b>Buy 2 t2.nano reserved instances</b> <span>Size flexible**</span></p> <p>US East (N. Virginia)   Linux/UNIX   Shared</p> <p><i>Based on your past 60 days of on-demand usage, we recommend purchasing 2 t2.nano reserved instances to cover 0.5 normalized units per hour of t2 family usage to maximize savings.</i></p> <p><a href="#">View Associated EC2 Usage</a></p>  | <p><b>\$3.49 monthly savings</b></p> <p>Upfront Cost: \$30.00</p> <p>Recurring Monthly Cost: \$2.48</p> <p>Expected RI Utilization: 100%</p>        |

Select a service

Elastic Compute Cloud (EC2)

RI Recommendation Parameters ⓘ

RI term

☒ 1 year
 ☐ 3 years

Offering Class

☒ Standard
 ☐ Convertible

Payment option

☐ All upfront
 ☒ Partial upfront
 ☐ No upfront

Based on the past

☐ 7 days
 ☐ 30 days
 ☒ 60 days

Additional Filters

Linked Account

Include all

<https://aws.amazon.com/blogs/aws-cloud-financial-management/beginners-guide-to-aws-cost-management/>

All rights reserved © www.awswithchetan.com

Chetan Agrawal

Before you take the exam

# Before you take the exam..

- How to approach exam questions - Tips
- Getting 30 mins extra time for the exam (as applicable)
- Scheduling the exam
- Getting 50% discount on your next AWS certification exam

# Know your exam

- Exam: **AWS Certified Solutions Architect Associate**
- Exam code: **SAA-C03**
- Total 65 questions
- Time: 130 Minutes
- 50 questions are scored and 15 are unscored
- There are two types of questions in the exam:
  - Multiple choice: Has one correct response and three incorrect responses
  - Multiple response: Has two or more correct responses out of five or more response options
- Minimum passing score: 720/1000



## Category

Associate

## Exam duration

130 minutes

## Exam format

65 questions; either multiple choice or multiple response

## Cost

150 USD. Visit [Exam pricing](#) for additional cost information, including foreign exchange rates

## Testing options

Pearson VUE testing center or online proctored exam

## Languages offered

English, French (France), Italian, Japanese, Korean, Portuguese (Brazil), Spanish (Latin America), Spanish (Spain), Simplified Chinese, and Traditional Chinese

<https://aws.amazon.com/certification/certified-solutions-architect-associate/>

# How to approach exam questions

## Multiple choice

A company wants to save costs by purchasing EC2 instances for predictable workloads that run continuously for one year. Which purchasing option is best for this use case?

- ☐ On-Demand
- ☒ Reserved Instances
- ☐ Spot Instances
- ☐ Dedicated Hosts

## Multiple responses

How does AWS charge for AWS Lambda usage once the free tier has been exceeded? (Select TWO.)

- ☒ By the time it takes for the lambda function to run
- ☐ By number of the versions of the lambda function
- ☒ By the number of the requests made for a given lambda function
- ☐ By the programming language
- ☐ By the total number of Lambda functions in the account

# How to approach exam questions

- ✓ Don't spend more than a minute and half for a question.
- ✓ If in doubt, select the answer which came first to your mind and Mark that question for review
- ✓ Even if you do not have any idea about the question, just answer it and Mark the question for review
- ✓ Try to finish the first pass over all 65 questions in around ~100 mins
- ✓ In last 15-30 mins, go over all the *Marked for review* questions and double check your answers.

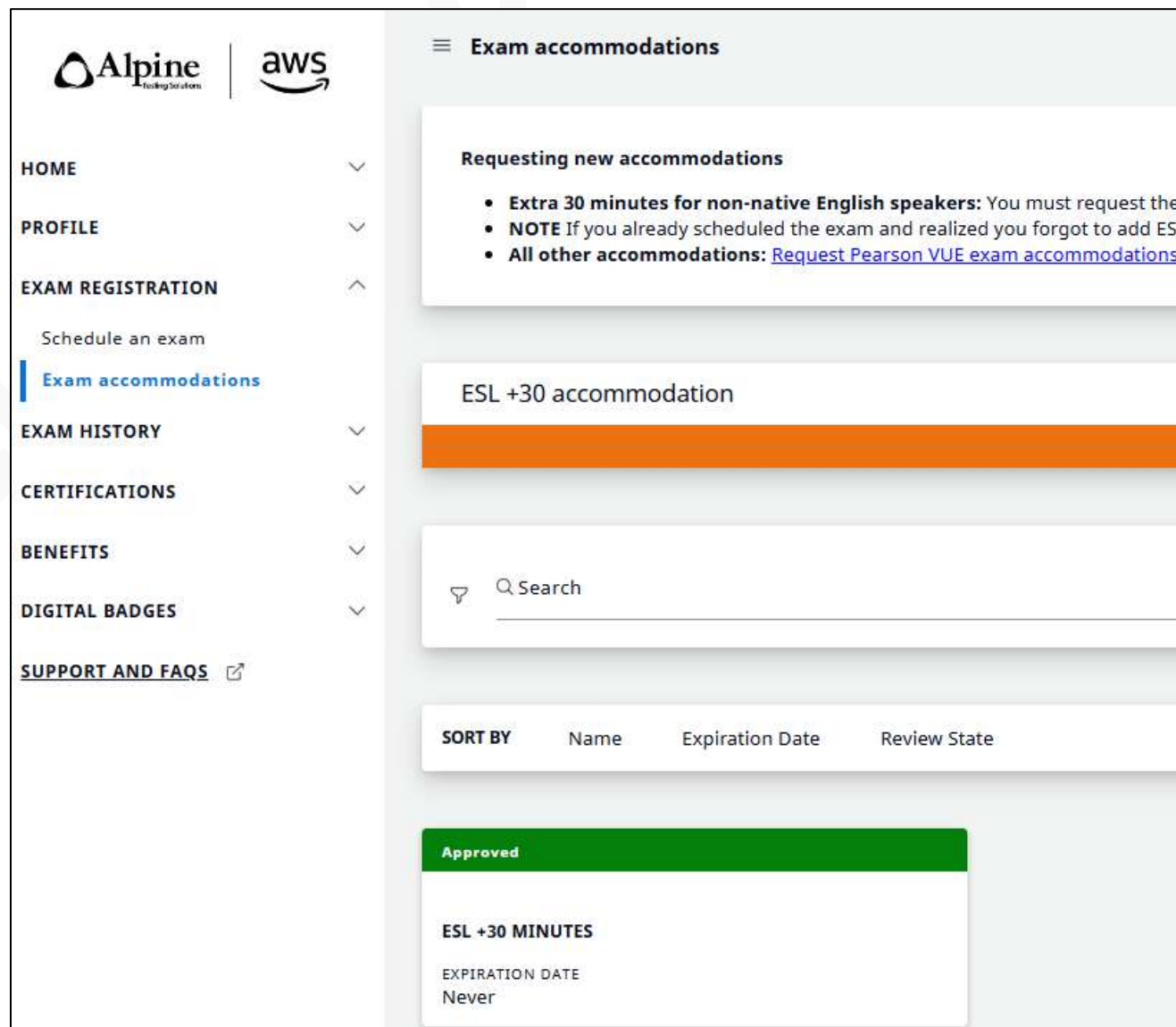


Mark for review



# Getting 30 mins extra time for the exam

- For non-native English speakers AWS provides 30 mins extra time
- You can request it through AWS certification portal -> Exam accommodations -> ELS + 30 accommodation
- Once opted, it will be there for all your future AWS certification exams



The screenshot shows the AWS Exam Accommodations page. The left sidebar contains navigation links: HOME, PROFILE, EXAM REGISTRATION (with sub-link Schedule an exam), Exam accommodations (highlighted), EXAM HISTORY, CERTIFICATIONS, BENEFITS, DIGITAL BADGES, and SUPPORT AND FAQS. The main content area is titled 'Exam accommodations' and includes a section for 'Requesting new accommodations' with bullet points: 'Extra 30 minutes for non-native English speakers', a 'NOTE' about scheduling, and a link to 'Request Pearson VUE exam accommodations'. Below this is a section for 'ESL +30 accommodation' with an orange bar. A search bar is present. At the bottom, a table shows the status 'Approved' for 'ESL +30 MINUTES' with an expiration date of 'Never'.

| Sort By | Name            | Expiration Date | Review State |
|---------|-----------------|-----------------|--------------|
|         | ESL +30 MINUTES | Never           | Approved     |



# Let's connect



Please subscribe to YouTube channel



<https://www.youtube.com/@AWSwithChetan>



Connect with me on LinkedIn



<https://www.linkedin.com/in/chetan-agrawal-30107310/>



Thank you and All the best!